Skills & Documentation Vault

Complete Skills Reference

Aggregated Documentation & Instruction Set

Total Documents Included: 833

Top-Level Docs: 25 files | **Subdirectory Skills:** 808 modules

Generated On: 2026-08-03

Table of Contents

Part 1: Workspace Documentation

1. 10.03 SKILLS.md
2. @crafterskillkit.md
3. Anti-Obsidian.md
4. Coder.md
5. Coordinator.md
6. Core-Systems.md
7. Cytoscape.js.md
8. Data-Pipelines.md
9. Graph-Intelligence.md
10. Index.md
11. Lawliet-aiopenclaw-skills-Lawliet.md
12. Memory.md
13. Researcher.md
14. Security-Audit.md
15. Skills 2 Windows Task Mngr..md
16. Skills.md
17. Task Manager skill.md
18. Tools.md
19. _Dashboard.md
20. anthropicsskills Public repository for Agent Skills.md
21. bgreenwelldotagents A proposed standard for the .agents directory to prevent context ...
22. iannuttalldotagents One location for all of your hooks, commands, skills, and AGENTCL...
23. kyegomezPROMPTS.md Understanding CLAUDE.md, MEMORY.md, SKILLS.md, SOUL.m...
24. obrasuperpowers An agentic skills framework & software development methodology t...
25. open-gitagentgitagent A universal git-native AI agent framework. Your agent lives insid...

Part 2: Subdirectory Skill Modules

1. academic-writing
2. accessibility-compliance
3. acquiring-disk-image-with-dd-and-dcfldd
4. ai-writing-detox
5. analyzing-active-directory-acl-abuse
6. analyzing-android-malware-with-apktool
7. analyzing-api-gateway-access-logs
8. analyzing-apt-group-with-mitre-navigator
9. analyzing-azure-activity-logs-for-threats
10. analyzing-bootkit-and-rootkit-samples
11. analyzing-browser-forensics-with-hindsight
12. analyzing-campaign-attribution-evidence
13. analyzing-certificate-transparency-for-phishing
14. analyzing-cloud-storage-access-patterns
15. analyzing-cobalt-strike-beacon-configuration
16. analyzing-cobaltstrike-malleable-c2-profiles
17. analyzing-command-and-control-communication
18. analyzing-cyber-kill-chain
19. analyzing-disk-image-with-autopsy
20. analyzing-dns-logs-for-exfiltration
21. analyzing-docker-container-forensics
22. analyzing-email-headers-for-phishing-investigation
23. analyzing-ethereum-smart-contract-vulnerabilities
24. analyzing-golang-malware-with-ghidra
25. analyzing-heap-spray-exploitation
26. analyzing-indicators-of-compromise
27. analyzing-ios-app-security-with-objection
28. analyzing-kubernetes-audit-logs
29. analyzing-linux-audit-logs-for-intrusion
30. analyzing-linux-elf-malware
31. analyzing-linux-kernel-rootkits
32. analyzing-linux-system-artifacts
33. analyzing-lnk-file-and-jump-list-artifacts
34. analyzing-macro-malware-in-office-documents
35. analyzing-malicious-pdf-with-peepdf
36. analyzing-malicious-url-with-urlscan
37. analyzing-malware-behavior-with-cuckoo-sandbox
38. analyzing-malware-family-relationships-with-malpedia
39. analyzing-malware-persistence-with-autoruns
40. analyzing-malware-sandbox-evasion-techniques
41. analyzing-memory-dumps-with-volatility
42. analyzing-memory-forensics-with-lime-and-volatility
43. analyzing-mft-for-deleted-file-recovery
44. analyzing-network-covert-channels-in-malware
45. analyzing-network-flow-data-with-netflow
46. analyzing-network-packets-with-scapy
47. analyzing-network-traffic-for-incidents
48. analyzing-network-traffic-of-malware
49. analyzing-network-traffic-with-wireshark
50. analyzing-office365-audit-logs-for-compromise
51. analyzing-outlook-pst-for-email-forensics
52. analyzing-packed-malware-with-upx-unpacker
53. analyzing-pdf-malware-with-pdfid
54. analyzing-persistence-mechanisms-in-linux
55. analyzing-powershell-empire-artifacts
56. analyzing-powershell-script-block-logging
57. analyzing-prefetch-files-for-execution-history
58. analyzing-ransomware-encryption-mechanisms
59. analyzing-ransomware-leak-site-intelligence
60. analyzing-ransomware-network-indicators
61. analyzing-ransomware-payment-wallets
62. analyzing-sbom-for-supply-chain-vulnerabilities

63. analyzing-security-logs-with-splunk
64. analyzing-slack-space-and-file-system-artifacts
65. analyzing-supply-chain-malware-artifacts
66. analyzing-threat-actor-ttps-with-mitre-attack
67. analyzing-threat-actor-ttps-with-mitre-navigator
68. analyzing-threat-intelligence-feeds
69. analyzing-threat-landscape-with-misp
70. analyzing-tls-certificate-transparency-logs
71. analyzing-typosquatting-domains-with-dnstwist
72. analyzing-uefi-bootkit-persistence
73. analyzing-usb-device-connection-history
74. analyzing-web-server-logs-for-intrusion
75. analyzing-windows-amcache-artifacts
76. analyzing-windows-event-logs-in-splunk
77. analyzing-windows-lnk-files-for-artifacts
78. analyzing-windows-prefetch-with-python
79. analyzing-windows-registry-for-artifacts
80. analyzing-windows-shellbag-artifacts
81. api-hardening
82. auditing-aws-s3-bucket-permissions
83. auditing-azure-active-directory-configuration
84. auditing-cloud-with-cis-benchmarks
85. auditing-gcp-iam-permissions
86. auditing-kubernetes-cluster-rbac
87. auditing-terraform-infrastructure-for-security
88. auditing-tls-certificate-transparency-logs
89. automating-ioc-enrichment
90. brainstorming
91. building-adversary-infrastructure-tracking-system
92. building-attack-pattern-library-from-cti-reports
93. building-automated-malware-submission-pipeline
94. building-c2-infrastructure-with-sliver-framework
95. building-cloud-siem-with-sentinel
96. building-detection-rule-with-splunk-spl
97. building-detection-rules-with-sigma
98. building-devsecops-pipeline-with-gitlab-ci
99. building-identity-federation-with-saml-azure-ad
100. building-identity-governance-lifecycle-process
101. building-incident-response-dashboard
102. building-incident-response-playbook
103. building-incident-timeline-with-timesketch
104. building-ioc-defanging-and-sharing-pipeline
105. building-ioc-enrichment-pipeline-with-opencti
106. building-malware-incident-communication-template
107. building-patch-tuesday-response-process
108. building-phishing-reporting-button-workflow
109. building-ransomware-playbook-with-cisa-framework
110. building-red-team-c2-infrastructure-with-havoc
111. building-role-mining-for-rbac-optimization
112. building-soc-escalation-matrix
113. building-soc-metrics-and-kpi-tracking
114. building-soc-playbook-for-ransomware
115. building-threat-actor-profile-from-osint
116. building-threat-feed-aggregation-with-misp
117. building-threat-hunt-hypothesis-framework
118. building-threat-intelligence-enrichment-in-splunk
119. building-threat-intelligence-feed-integration
120. building-threat-intelligence-platform
121. building-vulnerability-aging-and-sla-tracking
122. building-vulnerability-dashboard-with-defectdojo
123. building-vulnerability-exception-tracking-system
124. building-vulnerability-scanning-workflow
125. bypassing-authentication-with-forced-browsing
126. collecting-indicators-of-compromise
127. collecting-open-source-intelligence
128. collecting-threat-intelligence-with-misp
129. collecting-volatile-evidence-from-compromised-host
130. conducting-api-security-testing
131. conducting-cloud-incident-response
132. conducting-cloud-penetration-testing
133. conducting-domain-persistence-with-dcsync
134. conducting-external-reconnaissance-with-osint
135. conducting-full-scope-red-team-engagement
136. conducting-internal-network-penetration-test
137. conducting-internal-reconnaissance-with-bloodhound-ce
138. conducting-malware-incident-response
139. conducting-man-in-the-middle-attack-simulation
140. conducting-memory-forensics-with-volatility
141. conducting-mobile-app-penetration-test
142. conducting-network-penetration-test
143. conducting-pass-the-ticket-attack
144. conducting-phishing-incident-response
145. conducting-post-incident-lessons-learned
146. conducting-social-engineering-penetration-test
147. conducting-social-engineering-pretext-call
148. conducting-spearphishing-simulation-campaign
149. conducting-wireless-network-penetration-test
150. configuring-active-directory-tiered-model
151. configuring-aws-verified-access-for-ztna
152. configuring-certificate-authority-with-openssl
153. configuring-host-based-intrusion-detection
154. configuring-hsm-for-key-storage
155. configuring-identity-aware-proxy-with-google-iap
156. configuring-ldap-security-hardening
157. configuring-microsegmentation-for-zero-trust
158. configuring-multi-factor-authentication-with-duo
159. configuring-network-segmentation-with-vlans
160. configuring-oauth2-authorization-flow
161. configuring-pfsense-firewall-rules
162. configuring-snort-ids-for-intrusion-detection
163. configuring-suricata-for-network-monitoring
164. configuring-tls-1-3-for-secure-communications

165. configuring-windows-defender-advanced-settings
166. configuring-windows-event-logging-for-detection
167. configuring-zscaler-private-access-for-ztna
168. containing-active-breach
169. content-access
170. correlating-security-events-in-qradar
171. correlating-threat-campaigns
172. crisis-communications
173. data-journalism
174. deobfuscating-javascript-malware
175. deobfuscating-powershell-obfuscated-malware
176. deploying-active-directory-honeytokens
177. deploying-cloudflare-access-for-zero-trust
178. deploying-decoy-files-for-ransomware-detection
179. deploying-edr-agent-with-crowdstrike
180. deploying-osquery-for-endpoint-monitoring
181. deploying-palo-alto-prisma-access-zero-trust
182. deploying-ransomware-canary-files
183. deploying-software-defined-perimeter
184. deploying-tailscale-for-zero-trust-vpn
185. detecting-ai-model-prompt-injection-attacks
186. detecting-anomalies-in-industrial-control-systems
187. detecting-anomalous-authentication-patterns
188. detecting-api-enumeration-attacks
189. detecting-arp-poisoning-in-network-traffic
190. detecting-attacks-on-historian-servers
191. detecting-attacks-on-scada-systems
192. detecting-aws-cloudtrail-anomalies
193. detecting-aws-credential-exposure-with-trufflehog
194. detecting-aws-guardduty-findings-automation
195. detecting-aws-iam-privilege-escalation
196. detecting-azure-lateral-movement
197. detecting-azure-service-principal-abuse
198. detecting-azure-storage-account-misconfigurations
199. detecting-beaconing-patterns-with-zeek
200. detecting-bluetooth-low-energy-attacks
201. detecting-broken-object-property-level-authorization
202. detecting-business-email-compromise
203. detecting-business-email-compromise-with-ai
204. detecting-cloud-threats-with-guardduty
205. detecting-command-and-control-over-dns
206. detecting-compromised-cloud-credentials
207. detecting-container-drift-at-runtime
208. detecting-container-escape-attempts
209. detecting-container-escape-with-falco-rules
210. detecting-credential-dumping-techniques
211. detecting-cryptomining-in-cloud
212. detecting-dcsync-attack-in-active-directory
213. detecting-deepfake-audio-in-vishing-attacks
214. detecting-dll-sideloading-attacks
215. detecting-dnp3-protocol-anomalies
216. detecting-dns-exfiltration-with-dns-query-analysis
217. detecting-email-account-compromise
218. detecting-email-forwarding-rules-attack
219. detecting-evasion-techniques-in-endpoint-logs
220. detecting-exfiltration-over-dns-with-zeek
221. detecting-fileless-attacks-on-endpoints
222. detecting-fileless-malware-techniques
223. detecting-golden-ticket-attacks-in-kerberos-logs
224. detecting-golden-ticket-forgery
225. detecting-insider-data-exfiltration-via-dlp
226. detecting-insider-threat-behaviors
227. detecting-insider-threat-with-ueba
228. detecting-kerberoasting-attacks
229. detecting-lateral-movement-in-network
230. detecting-lateral-movement-with-splunk
231. detecting-lateral-movement-with-zeek
232. detecting-living-off-the-land-attacks
233. detecting-living-off-the-land-with-lolbas
234. detecting-malicious-scheduled-tasks-with-sysmon
235. detecting-mimikatz-execution-patterns
236. detecting-misconfigured-azure-storage
237. detecting-mobile-malware-behavior
238. detecting-modbus-command-injection-attacks
239. detecting-modbus-protocol-anomalies
240. detecting-network-anomalies-with-zeek
241. detecting-network-scanning-with-ids-signatures
242. detecting-ntlm-relay-with-event-correlation
243. detecting-oauth-token-theft
244. detecting-pass-the-hash-attacks
245. detecting-pass-the-ticket-attacks
246. detecting-port-scanning-with-fail2ban
247. detecting-privilege-escalation-attempts
248. detecting-privilege-escalation-in-kubernetes-pods
249. detecting-process-hollowing-technique
250. detecting-process-injection-techniques
251. detecting-qr-code-phishing-with-email-security
252. detecting-ransomware-encryption-behavior
253. detecting-ransomware-precursors-in-network
254. detecting-rdp-brute-force-attacks
255. detecting-rootkit-activity
256. detecting-s3-data-exfiltration-attempts
257. detecting-serverless-function-injection
258. detecting-service-account-abuse
259. detecting-shadow-api-endpoints
260. detecting-shadow-it-cloud-usage
261. detecting-spearphishing-with-email-gateway
262. detecting-sql-injection-via-waf-logs
263. detecting-stuxnet-style-attacks
264. detecting-supply-chain-attacks-in-ci-cd
265. detecting-suspicious-oauth-application-consent
266. detecting-suspicious-powershell-execution

267. detecting-t1003-credential-dumping-with-edr
268. detecting-t1055-process-injection-with-sysmon
269. detecting-t1548-abuse-elevation-control-mechanism
270. detecting-typosquatting-packages-in-npm-pypi
271. detecting-wmi-persistence
272. digital-archive
273. dispatching-parallel-agents
274. document-design
275. editorial-workflow
276. electron-dev
277. eradicating-malware-from-infected-systems
278. evaluating-threat-intelligence-platforms
279. executing-active-directory-attack-simulation
280. executing-phishing-simulation-campaign
281. executing-plans
282. executing-red-team-engagement-planning
283. executing-red-team-exercise
284. exploiting-active-directory-certificate-services-esc1
285. exploiting-active-directory-with-bloodhound
286. exploiting-api-injection-vulnerabilities
287. exploiting-bgp-hijacking-vulnerabilities
288. exploiting-broken-function-level-authorization
289. exploiting-broken-link-hijacking
290. exploiting-constrained-delegation-abuse
291. exploiting-deeplink-vulnerabilities
292. exploiting-excessive-data-exposure-in-api
293. exploiting-http-request-smuggling
294. exploiting-idor-vulnerabilities
295. exploiting-insecure-data-storage-in-mobile
296. exploiting-insecure-deserialization
297. exploiting-ipv6-vulnerabilities
298. exploiting-jwt-algorithm-confusion-attack
299. exploiting-kerberoasting-with-impacket
300. exploiting-mass-assignment-in-rest-apis
301. exploiting-ms17-010-eternalblue-vulnerability
302. exploiting-nopac-cve-2021-42278-42287
303. exploiting-nosql-injection-vulnerabilities
304. exploiting-oauth-misconfiguration
305. exploiting-prototype-pollution-in-javascript
306. exploiting-race-condition-vulnerabilities
307. exploiting-server-side-request-forgery
308. exploiting-smb-vulnerabilities-with-metasploit
309. exploiting-sql-injection-vulnerabilities
310. exploiting-sql-injection-with-sqlmap
311. exploiting-template-injection-vulnerabilities
312. exploiting-type-juggling-vulnerabilities
313. exploiting-vulnerabilities-with-metasploit-framework
314. exploiting-websocket-vulnerabilities
315. exploiting-zerologon-vulnerability-cve-2020-1472
316. extracting-browser-history-artifacts
317. extracting-config-from-agent-tesla-rat
318. extracting-credentials-from-memory-dump
319. extracting-iocs-from-malware-samples
320. extracting-memory-artifacts-with-rekall
321. extracting-windows-event-logs-artifacts
322. fact-check-workflow
323. finishing-a-development-branch
324. foia-requests
325. free-apis-catalog
326. generating-threat-intelligence-reports
327. hardening-docker-containers-for-production
328. hardening-docker-daemon-configuration
329. hardening-linux-endpoint-with-cis-benchmark
330. hardening-windows-endpoint-with-cis-benchmark
331. hunting-advanced-persistent-threats
332. hunting-credential-stuffing-attacks
333. hunting-for-anomalous-powershell-execution
334. hunting-for-beaconing-with-frequency-analysis
335. hunting-for-cobalt-strike-beacons
336. hunting-for-command-and-control-beaconing
337. hunting-for-data-exfiltration-indicators
338. hunting-for-data-staging-before-exfiltration
339. hunting-for-dcom-lateral-movement
340. hunting-for-dcsync-attacks
341. hunting-for-defense-evasion-via-timestomping
342. hunting-for-dns-based-persistence
343. hunting-for-dns-tunneling-with-zeek
344. hunting-for-domain-fronting-c2-traffic
345. hunting-for-lateral-movement-via-wmi
346. hunting-for-living-off-the-cloud-techniques
347. hunting-for-living-off-the-land-binaries
348. hunting-for-lolbins-execution-in-endpoint-logs
349. hunting-for-ntlm-relay-attacks
350. hunting-for-persistence-mechanisms-in-windows
351. hunting-for-persistence-via-wmi-subscriptions
352. hunting-for-process-injection-techniques
353. hunting-for-registry-persistence-mechanisms
354. hunting-for-registry-run-key-persistence
355. hunting-for-scheduled-task-persistence
356. hunting-for-shadow-copy-deletion
357. hunting-for-spearphishing-indicators
358. hunting-for-startup-folder-persistence
359. hunting-for-supply-chain-compromise
360. hunting-for-suspicious-scheduled-tasks
361. hunting-for-t1098-account-manipulation
362. hunting-for-unusual-network-connections
363. hunting-for-unusual-service-installations
364. hunting-for-webshell-activity
365. implementing-aes-encryption-for-data-at-rest
366. implementing-alert-fatigue-reduction
367. implementing-anti-phishing-training-program
368. implementing-anti-ransomware-group-policy

369. implementing-api-abuse-detection-with-rate-limiting
370. implementing-api-gateway-security-controls
371. implementing-api-key-security-controls
372. implementing-api-rate-limiting-and-throttling
373. implementing-api-schema-validation-security
374. implementing-api-security-posture-management
375. implementing-api-security-testing-with-42crunch
376. implementing-api-threat-protection-with-apigee
377. implementing-application-whitelisting-with-applocker
378. implementing-aqua-security-for-container-scanning
379. implementing-attack-path-analysis-with-xm-cyber
380. implementing-attack-surface-management
381. implementing-aws-config-rules-for-compliance
382. implementing-aws-iam-permission-boundaries
383. implementing-aws-macie-for-data-classification
384. implementing-aws-nitro-enclave-security
385. implementing-aws-security-hub
386. implementing-aws-security-hub-compliance
387. implementing-azure-ad-privileged-identity-management
388. implementing-azure-defender-for-cloud
389. implementing-beyondcorp-zero-trust-access-model
390. implementing-bgp-security-with-rpki
391. implementing-browser-isolation-for-zero-trust
392. implementing-canary-tokens-for-network-intrusion
393. implementing-cisa-zero-trust-maturity-model
394. implementing-cloud-dlp-for-data-protection
395. implementing-cloud-security-posture-management
396. implementing-cloud-trail-log-analysis
397. implementing-cloud-vulnerability-posture-management
398. implementing-cloud-waf-rules
399. implementing-cloud-workload-protection
400. implementing-code-signing-for-artifacts
401. implementing-conditional-access-policies-azure-ad
402. implementing-conduit-security-for-ot-remote-access
403. implementing-container-image-minimal-base-with-distroless
404. implementing-container-network-policies-with-calico
405. implementing-continuous-security-validation-with-bas
406. implementing-data-loss-prevention-with-microsoft-purview
407. implementing-ddos-mitigation-with-cloudflare
408. implementing-deception-based-detection-with-canarytoken
409. implementing-delinea-secret-server-for-pam
410. implementing-device-posture-assessment-in-zero-trust
411. implementing-devsecops-security-scanning
412. implementing-diamond-model-analysis
413. implementing-digital-signatures-with-ed25519
414. implementing-disk-encryption-with-bitlocker
415. implementing-dmarc-dkim-spf-email-security
416. implementing-dragos-platform-for-ot-monitoring
417. implementing-ebpf-security-monitoring
418. implementing-email-sandboxing-with-proofpoint
419. implementing-end-to-end-encryption-for-messaging
420. implementing-endpoint-detection-with-wazuh
421. implementing-endpoint-dlp-controls
422. implementing-envelope-encryption-with-aws-kms
423. implementing-epss-score-for-vulnerability-prioritization
424. implementing-file-integrity-monitoring-with-aide
425. implementing-fuzz-testing-in-cicd-with-aflplusplus
426. implementing-gcp-binary-authorization
427. implementing-gcp-organization-policy-constraints
428. implementing-gcp-vpc-firewall-rules
429. implementing-gdpr-data-protection-controls
430. implementing-gdpr-data-subject-access-request
431. implementing-github-advanced-security-for-code-scanning
432. implementing-google-workspace-admin-security
433. implementing-google-workspace-phishing-protection
434. implementing-google-workspace-sso-configuration
435. implementing-hardware-security-key-authentication
436. implementing-hashicorp-vault-dynamic-secrets
437. implementing-honeypot-for-ransomware-detection
438. implementing-honeytokens-for-breach-detection
439. implementing-ics-firewall-with-tofino
440. implementing-identity-governance-with-sailpoint
441. implementing-identity-verification-for-zero-trust
442. implementing-iec-62443-security-zones
443. implementing-image-provenance-verification-with-cosign
444. implementing-immutable-backup-with-restic
445. implementing-infrastructure-as-code-security-scanning
446. implementing-iso-27001-information-security-management
447. implementing-just-in-time-access-provisioning
448. implementing-jwt-signing-and-verification
449. implementing-kubernetes-network-policy-with-calico
450. implementing-kubernetes-pod-security-standards
451. implementing-llm-guardrails-for-security
452. implementing-log-forwarding-with-fluentd
453. implementing-log-integrity-with-blockchain
454. implementing-memory-protection-with-dep-aslr
455. implementing-microsegmentation-with-guardicore
456. implementing-mimecast-targeted-attack-protection
457. implementing-mitre-attack-coverage-mapping
458. implementing-mobile-application-management
459. implementing-mtls-for-zero-trust-services
460. implementing-nerc-cip-compliance-controls
461. implementing-network-access-control
462. implementing-network-access-control-with-cisco-ise
463. implementing-network-deception-with-honeypots
464. implementing-network-intrusion-prevention-with-suricata
465. implementing-network-policies-for-kubernetes
466. implementing-network-segmentation-for-ot
467. implementing-network-segmentation-with-firewall-zones
468. implementing-network-traffic-analysis-with-arkime
469. implementing-network-traffic-baselining
470. implementing-next-generation-firewall-with-palo-alto

471. implementing-opa-gatekeeper-for-policy-enforcement
472. implementing-ot-incident-response-playbook
473. implementing-ot-network-traffic-analysis-with-nozomi
474. implementing-pam-for-database-access
475. implementing-passwordless-auth-with-microsoft-entra
476. implementing-passwordless-authentication-with-fido2
477. implementing-patch-management-for-ot-systems
478. implementing-patch-management-workflow
479. implementing-pci-dss-compliance-controls
480. implementing-pod-security-admission-controller
481. implementing-policy-as-code-with-open-policy-agent
482. implementing-privileged-access-management-with-cyberark
483. implementing-privileged-access-workstation
484. implementing-privileged-session-monitoring
485. implementing-proofpoint-email-security-gateway
486. implementing-purdue-model-network-segmentation
487. implementing-ransomware-backup-strategy
488. implementing-ransomware-kill-switch-detection
489. implementing-rapid7-insightvm-for-scanning
490. implementing-rbac-hardening-for-kubernetes
491. implementing-rsa-key-pair-management
492. implementing-runtime-application-self-protection
493. implementing-runtime-security-with-tetragon
494. implementing-saml-sso-with-okta
495. implementing-scim-provisioning-with-okta
496. implementing-secret-scanning-with-gitleaks
497. implementing-secrets-management-with-vault
498. implementing-secrets-scanning-in-ci-cd
499. implementing-security-chaos-engineering
500. implementing-security-information-sharing-with-stix2
501. implementing-security-monitoring-with-datadog
502. implementing-semgrep-for-custom-sast-rules
503. implementing-siem-correlation-rules-for-apt
504. implementing-siem-use-case-tuning
505. implementing-siem-use-cases-for-detection
506. implementing-sigstore-for-software-signing
507. implementing-soar-automation-with-phantom
508. implementing-soar-playbook-for-phishing
509. implementing-soar-playbook-with-palo-alto-xsoar
510. implementing-stix-taxii-feed-integration
511. implementing-supply-chain-security-with-in-toto
512. implementing-syslog-centralization-with-rsyslog
513. implementing-taxii-server-with-opentaxii
514. implementing-threat-intelligence-lifecycle-management
515. implementing-threat-modeling-with-mitre-attack
516. implementing-ticketing-system-for-incidents
517. implementing-usb-device-control-policy
518. implementing-velociraptor-for-ir-collection
519. implementing-vulnerability-management-with-greenbone
520. implementing-vulnerability-remediation-sla
521. implementing-vulnerability-sla-breach-alerting
522. implementing-web-application-logging-with-modsecurity
523. implementing-zero-knowledge-proof-for-authentication
524. implementing-zero-standing-privilege-with-cyberark
525. implementing-zero-trust-dns-with-nextdns
526. implementing-zero-trust-for-saas-applications
527. implementing-zero-trust-in-cloud
528. implementing-zero-trust-network-access
529. implementing-zero-trust-network-access-with-zscaler
530. implementing-zero-trust-with-beyondcorp
531. implementing-zero-trust-with-hashicorp-boundary
532. integrating-dast-with-owasp-zap-in-pipeline
533. integrating-sast-into-github-actions-pipeline
534. intercepting-mobile-traffic-with-burpsuite
535. interview-prep
536. interview-transcription
537. investigating-insider-threat-indicators
538. investigating-phishing-email-incident
539. investigating-ransomware-attack-artifacts
540. managing-cloud-identity-with-okta
541. managing-intelligence-lifecycle
542. mapping-mitre-attack-techniques
543. mobile-debugging
544. monitoring-darkweb-sources
545. monitoring-scada-modbus-traffic-anomalies
546. newsletter-publishing
547. newsroom-style
548. one-way-door
549. page-monitoring
550. pdf-design
551. performing-access-recertification-with-saviynt
552. performing-access-review-and-certification
553. performing-active-directory-bloodhound-analysis
554. performing-active-directory-compromise-investigation
555. performing-active-directory-forest-trust-attack
556. performing-active-directory-penetration-test
557. performing-active-directory-vulnerability-assessment
558. performing-adversary-in-the-middle-phishing-detection
559. performing-agentless-vulnerability-scanning
560. performing-ai-driven-osint-correlation
561. performing-alert-triage-with-elastic-siem
562. performing-android-app-static-analysis-with-mobsf
563. performing-api-fuzzing-with-restler
564. performing-api-inventory-and-discovery
565. performing-api-rate-limiting-bypass
566. performing-api-security-testing-with-postman
567. performing-arp-spoofing-attack-simulation
568. performing-asset-criticality-scoring-for-vulns
569. performing-authenticated-scan-with-openvas
570. performing-authenticated-vulnerability-scan
571. performing-automated-malware-analysis-with-cape
572. performing-aws-account-enumeration-with-scout-suite

573. performing-aws-privilege-escalation-assessment
574. performing-bandwidth-throttling-attack-simulation
575. performing-binary-exploitation-analysis
576. performing-blind-ssrf-exploitation
577. performing-bluetooth-security-assessment
578. performing-brand-monitoring-for-impersonation
579. performing-clickjacking-attack-test
580. performing-cloud-asset-inventory-with-cartography
581. performing-cloud-forensics-investigation
582. performing-cloud-forensics-with-aws-cloudtrail
583. performing-cloud-incident-containment-procedures
584. performing-cloud-log-forensics-with-athena
585. performing-cloud-native-forensics-with-falco
586. performing-cloud-native-threat-hunting-with-aws-detective
587. performing-cloud-penetration-testing-with-pacu
588. performing-cloud-storage-forensic-acquisition
589. performing-container-escape-detection
590. performing-container-image-hardening
591. performing-container-security-scanning-with-trivy
592. performing-content-security-policy-bypass
593. performing-credential-access-with-lazagne
594. performing-cryptographic-audit-of-application
595. performing-csrf-attack-simulation
596. performing-cve-prioritization-with-kev-catalog
597. performing-dark-web-monitoring-for-threats
598. performing-deception-technology-deployment
599. performing-directory-traversal-testing
600. performing-disk-forensics-investigation
601. performing-dmarc-policy-enforcement-rollout
602. performing-dns-enumeration-and-zone-transfer
603. performing-dns-tunneling-detection
604. performing-docker-bench-security-assessment
605. performing-dynamic-analysis-of-android-app
606. performing-dynamic-analysis-with-any-run
607. performing-endpoint-forensics-investigation
608. performing-endpoint-vulnerability-remediation
609. performing-entitlement-review-with-sailpoint-iiq
610. performing-external-network-penetration-test
611. performing-false-positive-reduction-in-siem
612. performing-file-carving-with-foremost
613. performing-firmware-extraction-with-binwalk
614. performing-firmware-malware-analysis
615. performing-fuzzing-with-aflplusplus
616. performing-gcp-penetration-testing-with-gcpbucketbrute
617. performing-gcp-security-assessment-with-forseti
618. performing-graphql-depth-limit-attack
619. performing-graphql-introspection-attack
620. performing-graphql-security-assessment
621. performing-hardware-security-module-integration
622. performing-hash-cracking-with-hashcat
623. performing-http-parameter-pollution-attack
624. performing-ics-asset-discovery-with-claroty
625. performing-indicator-lifecycle-management
626. performing-initial-access-with-evilginx3
627. performing-insider-threat-investigation
628. performing-ioc-enrichment-automation
629. performing-ios-app-security-assessment
630. performing-iot-security-assessment
631. performing-ip-reputation-analysis-with-shodan
632. performing-jwt-none-algorithm-attack
633. performing-kerberoasting-attack
634. performing-kubernetes-cis-benchmark-with-kube-bench
635. performing-kubernetes-etcd-security-assessment
636. performing-kubernetes-penetration-testing
637. performing-lateral-movement-detection
638. performing-lateral-movement-with-wmiexec
639. performing-linux-log-forensics-investigation
640. performing-log-analysis-for-forensic-investigation
641. performing-log-source-onboarding-in-siem
642. performing-malware-hash-enrichment-with-virustotal
643. performing-malware-ioc-extraction
644. performing-malware-persistence-investigation
645. performing-malware-triage-with-yara
646. performing-memory-forensics-with-volatility3
647. performing-memory-forensics-with-volatility3-plugins
648. performing-mobile-app-certificate-pinning-bypass
649. performing-mobile-device-forensics-with-cellebrite
650. performing-network-forensics-with-wireshark
651. performing-network-packet-capture-analysis
652. performing-network-traffic-analysis-with-tshark
653. performing-network-traffic-analysis-with-zeek
654. performing-nist-csf-maturity-assessment
655. performing-oauth-scope-minimization-review
656. performing-oil-gas-cybersecurity-assessment
657. performing-open-source-intelligence-gathering
658. performing-osint-with-spiderfoot
659. performing-ot-network-security-assessment
660. performing-ot-vulnerability-assessment-with-claroty
661. performing-ot-vulnerability-scanning-safely
662. performing-packet-injection-attack
663. performing-paste-site-monitoring-for-credentials
664. performing-phishing-simulation-with-gophish
665. performing-physical-intrusion-assessment
666. performing-plc-firmware-security-analysis
667. performing-post-quantum-cryptography-migration
668. performing-power-grid-cybersecurity-assessment
669. performing-privacy-impact-assessment
670. performing-privilege-escalation-assessment
671. performing-privilege-escalation-on-linux
672. performing-privileged-account-access-review
673. performing-privileged-account-discovery
674. performing-purple-team-atomic-testing

675. performing-purple-team-exercise
676. performing-ransomware-response
677. performing-ransomware-tabletop-exercise
678. performing-red-team-phishing-with-gophish
679. performing-red-team-with-covenant
680. performing-s7comm-protocol-security-analysis
681. performing-sca-dependency-scanning-with-snyk
682. performing-scada-hmi-security-assessment
683. performing-second-order-sql-injection
684. performing-security-headers-audit
685. performing-serverless-function-security-review
686. performing-service-account-audit
687. performing-service-account-credential-rotation
688. performing-soap-web-service-security-testing
689. performing-soc-tabletop-exercise
690. performing-soc2-type2-audit-preparation
691. performing-sqlite-database-forensics
692. performing-ssl-certificate-lifecycle-management
693. performing-ssl-stripping-attack
694. performing-ssl-tls-inspection-configuration
695. performing-ssl-tls-security-assessment
696. performing-ssrf-vulnerability-exploitation
697. performing-static-malware-analysis-with-pe-studio
698. performing-steganography-detection
699. performing-subdomain-enumeration-with-subfinder
700. performing-supply-chain-attack-simulation
701. performing-thick-client-application-penetration-test
702. performing-threat-emulation-with-atomic-red-team
703. performing-threat-hunting-with-elastic-siem
704. performing-threat-hunting-with-yara-rules
705. performing-threat-intelligence-sharing-with-misp
706. performing-threat-landscape-assessment-for-sector
707. performing-threat-modeling-with-owasp-threat-dragon
708. performing-timeline-reconstruction-with-plaso
709. performing-user-behavior-analytics
710. performing-vlan-hopping-attack
711. performing-vulnerability-scanning-with-nessus
712. performing-web-application-firewall-bypass
713. performing-web-application-penetration-test
714. performing-web-application-scanning-with-nikto
715. performing-web-application-vulnerability-triage
716. performing-web-cache-deception-attack
717. performing-web-cache-poisoning-attack
718. performing-wifi-password-cracking-with-aircrack
719. performing-windows-artifact-analysis-with-eric-zimmerman-tools
720. performing-wireless-network-penetration-test
721. performing-wireless-security-assessment-with-kismet
722. performing-yara-rule-development-for-detection
723. prioritizing-vulnerabilities-with-cvss-scoring
724. processing-karpathy-raw
725. processing-stix-taxii-feeds
726. profiling-threat-actor-groups
727. project-memory
728. project-retrospective
729. python-pipeline
730. receiving-code-review
731. recovering-deleted-files-with-photorec
732. recovering-from-ransomware-attack
733. remediating-s3-bucket-misconfiguration
734. requesting-code-review
735. reverse-engineering-android-malware-with-jadx
736. reverse-engineering-dotnet-malware-with-dnspy
737. reverse-engineering-ios-app-with-frida
738. reverse-engineering-malware-with-ghidra
739. reverse-engineering-ransomware-encryption-routine
740. reverse-engineering-rust-malware
741. scanning-container-images-with-grype
742. scanning-containers-with-trivy-in-cicd
743. scanning-docker-images-with-trivy
744. scanning-infrastructure-with-nessus
745. scanning-kubernetes-manifests-with-kubesec
746. scanning-network-with-nmap-advanced
747. secure-auth
748. securing-api-gateway-with-aws-waf
749. securing-aws-iam-permissions
750. securing-aws-lambda-execution-roles
751. securing-azure-with-microsoft-defender
752. securing-container-registry-images
753. securing-container-registry-with-harbor
754. securing-github-actions-workflows
755. securing-helm-chart-deployments
756. securing-historian-server-in-ot-environment
757. securing-kubernetes-on-cloud
758. securing-remote-access-to-ot-environment
759. securing-serverless-functions
760. security-checklist
761. social-media-intelligence
762. source-verification
763. story-pitch
764. subagent-driven-development
765. supply-chain-hardening
766. systematic-debugging
767. template-selector
768. test-driven-development
769. test-first-bugs
770. testing-android-intents-for-vulnerabilities
771. testing-api-authentication-weaknesses
772. testing-api-for-broken-object-level-authorization
773. testing-api-for-mass-assignment-vulnerability
774. testing-api-security-with-owasp-top-10
775. testing-cors-misconfiguration
776. testing-for-broken-access-control

777. testing-for-business-logic-vulnerabilities
778. testing-for-email-header-injection
779. testing-for-host-header-injection
780. testing-for-json-web-token-vulnerabilities
781. testing-for-open-redirect-vulnerabilities
782. testing-for-sensitive-data-exposure
783. testing-for-xml-injection-vulnerabilities
784. testing-for-xss-vulnerabilities
785. testing-for-xss-vulnerabilities-with-burpsuite
786. testing-for-xxe-injection-vulnerabilities
787. testing-jwt-token-security
788. testing-mobile-api-authentication
789. testing-oauth2-implementation-flaws
790. testing-ransomware-recovery-procedures
791. testing-websocket-api-security
792. tracking-threat-actor-infrastructure
793. triaging-security-alerts-in-splunk
794. triaging-security-incident
795. triaging-security-incident-with-ir-playbook
796. triaging-vulnerabilities-with-ssvc-framework
797. using-git-worktrees
798. using-superjawn
799. validating-backup-integrity-for-recovery
800. verification-before-completion
801. vibe-coding
802. visual-explainer
803. web-archiving
804. web-scraping
805. web-ui-best-practices
806. writing-plans
807. writing-skills
808. zero-build-frontend

10.03 SKILLS.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\10.03 SKILLS.md

type: dashboard
cssclasses: [cards]
tags:

10.03 [[SKILLS]]

> *774 [[skills]] organized as invokable capability packages — each a folder containing a [[[[

Key

[[[

How to Invoke [[Skills]]

```
# Via OpenSkills CLI
npx openskills read <skill-name>

# Multiple skills
npx openskills read skill-one,skill-two

# Via Antigravity
# → Reference the skill in your agent prompt or config
```

Ecosystem Map

```
10.03 SKILLS
 Consumed by → [[Antigravity]] (via .gemini/config/skills/)
 Consumed by → [[Hermes]] (via .hermes/skills/)
 Consumed by → [[Pi]] (via .pi/agent/skills/)
 Consumed by → [[OpenClaw]] (via .openclaw/plugin-skills/)
 Invoked via → [[10.01 PROMPTS]] (prompts activate skills)
 Extended by → [[10.02 TOOLS]] (tools power skill execution)
 Documented in → [[Skills]] (hub note)
 Versioned via → [[00.01 Antigravity]]
```

Dynamic [[Skills]] Map of Content (MOC)

```
const skills = dv.pages('"10.03 SKILLS"').where(p => p.file.name === "SKILL");
const verbs = ["performing", "implementing", "detecting", "analyzing", "building", "hunting", "exploiting", "testing", "conducting", "configuring",
"securing", "deploying", "auditing", "reverse-engineering", "scanning"];

const grouped = {};
verbs.forEach(v => grouped[v] = []);
grouped["others"] = [];

skills.forEach(p => {
  const folderName = p.file.folder.split("/").pop();
  let matched = false;
  for (let v of verbs) {
    if (folderName.startsWith(v)) {
      grouped[v].push({ link: p.file.link, folder: folderName, page: p });
      matched = true;
      break;
    }
  }
  if (!matched) {
    grouped["others"].push({ link: p.file.link, folder: folderName, page: p });
  }
});

for (let [verb, list] of Object.entries(grouped)) {
  if (list.length === 0) continue;
  dv.header(3, `${verb.toUpperCase()} (${list.length} skills)`);
  dv.table(
    ["Skill", "Folder Name", "Description"],
    list.sort((a,b) => a.folder.localeCompare(b.folder)).slice(0, 12).map(item => [
      item.link,
      `\`${item.folder}\``,
      item.page.description || "-"
    ])
  );
}
```

Related Nodes

Node	Relationship
[[Skills]]	Existing hub note for this domain
[[10.01 PROMPTS]]	[[[[Prompts]]]] invoke and configure [[skills]]
[[10.02 TOOLS]]	Tools power [[[[

@crafterskillkit.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\@crafterskillkit.md

type: dashboard
cssclasses: [cards]
tags:

skillkit

Local-first observability for AI agent [[skills]]. Track usage, measure cost, detect conflicts, and prune what you don't use.

Why

AI coding [[agents]] load [[skills]] into their context window on every session. More [[skills]] = less room for your actual code. But which [[skills]] do you actually use? Are they helping? How much do they cost?

skillkit answers these questions with usage analytics, conflict detection, cost analysis, and context budget monitoring — all locally on your machine.

Prerequisites

skillkit requires Bun as its runtime (for native SQLite and fast TypeScript execution).

```
curl -fsSL https://bun.sh/install | bash
```

Quick start

```
bunx @crafter/skillkit scan
bunx @crafter/skillkit stats
bunx @crafter/skillkit health
```

Or install globally:

```
bun add -g @crafter/skillkit
skillkit scan
```

> ***Note:*** *`npm i -g` also works if bun is installed, since the bin entry uses bun as its runtime.*

Commands

Command
`skillkit scan`
`skillkit list`
`skillkit stats`
`skillkit health`
`skillkit prune`
`skillkit burn`
`skillkit conflicts`
`skillkit coverage` <[[[

[[[[

How It Works

Scan

Discovers [[skills]] from `~/.claude/[[skills]]/` and indexes session files from `~/.claude/projects/`. Detects whether each [[[[

```
$ skillkit coverage ./skills/review/
  Sections: 8/12 referenced (67%)
  Files: 3/5 referenced (60%)

  Unreferenced sections:
    - ## Advanced Configuration
    - ## Troubleshooting
```

Prune

Removes [[skills]] that haven't been used in the last 30 days.

```
$ skillkit prune
  x scaffold (0.9K)
  x lint (2.1K)

  2 skills · 3.0K context reclaimable

  Run with --yes to confirm deletion.
```

Data Storage

All data stays on your machine. No telemetry. No signup.

Path	Purpose
`~/.skillkit/analytics.db`	SQLite database with invocation history and traces
`~/.claude/[[skills]]/`	Installed [[skills]] (read-only)
`~/.claude/projects/**/*.jsonl`	Session files (read-only)

Supported [[Agents]]

Scans [[[[

Anti-Obsidian.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Anti-Obsidian.md

cssclasses: ["electric-dashboard"]

Anti-Obsidian

The Supreme Vault Architect persona is active.

Dataview queries, Templater scripts, and custom CSS are processed here.

```
// Templater execution block placeholder
<%*
new Notice("Anti-Obsidian Architect Persona Deployed.", 5000);
-%>
```

Coder.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Coder.md

title: Coder Dashboard
role: coder
type: dashboard
status: active
cssclasses: ["role-dashboard"]

Coder Dashboard

> *Live Coder dashboard. Open tasks, MCP server status, test pass rates, recent builds.*

Open TASKS/STATUS (Coder-owned)

```
TABLE title, status, modified FROM "TASKS/STATUS" WHERE owner = "coder" AND status != "done" SORT modified DESC LIMIT 5
```

MCP Server Status

```
TABLE mcp_id, state FROM "MCP" WHERE type = "server-spec" FLATTEN state
```

hermes_bus Tests

```
The hermes_bus test suite has 62 passing tests (48 prior + 14 new). Run `pytest tests/` from `2.0CODER/SOLO-PROJECTS/loop-chat-system/`.
```

Last refresh

```
LIST file.mtime
FROM this
```

Related

- `../CODER/AGENT.md`
- `../CODER/MEMORY/Coder Memory.md`
- `../../SYSTEM/agents.yaml`

Coordinator.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Coordinator.md

title: Coordinator Dashboard
role: coordinator
type: dashboard
status: active
cssclasses: ["role-dashboard"]

Coordinator Dashboard

> *Live Coordinator dashboard. INBOX/OUTBOX counts, last 5 messages, current routing matrix, agent dispatch frequency.*

Last 5 Inbox Messages

```
TABLE title, created, status FROM "1.0COORDINATOR/CHAT/Inbox" SORT created DESC LIMIT 5
```

Last 5 Outbox Completions

```
TABLE title, completed, status FROM "1.0COORDINATOR/CHAT/Outbox" SORT completed DESC LIMIT 5
```

Routing Matrix

```
TABLE type, default_role FROM "SYSTEM/agents.yaml" FLATTEN routing_defaults
```

Last refresh

```
LIST file.mtime
FROM this
```

Related

- `../COORDINATOR/AGENT.md`
- `../COORDINATOR/MEMORY/Coordinator Memory.md`
- `../../SYSTEM/agents.yaml`

Core-Systems.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Core-Systems.md

cssclasses: ["electric-dashboard"]

Core Systems

The central intelligence hub. Operations are monitored and initialized from this nexus.

```
TABLE WITHOUT ID
  file.link AS "Active Skill",
  file.mtime AS "Last Initialized"
FROM "Skills"
WHERE file.name != "Core-Systems"
```

Cytoscape.js.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Cytoscape.js.md

type: dashboard
cssclasses: [cards]
tags:

Cytoscape.js

Overview

Cytoscape.js is a feature-rich, high-performance graph theory and network visualization library written in pure JavaScript. It supports directed, undirected, mixed, and compound graphs.

When to Use

- **Use when:**
- Visualizing relational data (networks, social graphs, dependency trees, mind maps, biological pathways).
- Implementing interactive graph features like panning, zooming, box selection, and node dragging.
- Performing headless graph analysis in Node.js (e.g., shortest paths, PageRank, centrality).
- **Do NOT use when:**
- Drawing standard charts (bar, line, scatter, pie) — use Chart.js, D3.js, or Recharts.
- Displaying simple trees or file explorer directories — use simple HTML/CSS tree components.

Quick Reference

1. Basic Initialization

Ensure the container element has a defined `width` and `height` in CSS before initializing Cytoscape.js.

```
<div id="cy" style="width: 600px; height: 400px; display: block;"></div>
<script src="https://cdnjs.cloudflare.com/ajax/libs/cytoscape/3.29.2/cytoscape.min.js"></script>
<script>
  const cy = cytoscape({
    container: document.getElementById('cy'),
    elements: [
      { data: { id: 'n1', label: 'Node 1' } },
      { data: { id: 'n2', label: 'Node 2' } },
      { data: { id: 'e1', source: 'n1', target: 'n2' } }
    ],
    style: [
      {
        selector: 'node',
        style: {
          'background-color': '#4ea5d9',
          'label': 'data(label)',
          'color': '#333'
        }
      },
      {
        selector: 'edge',
        style: {
          'width': 3,
          'line-color': '#ccc',
          'target-arrow-shape': 'triangle',
          'curve-style': 'bezier'
        }
      }
    ],
    layout: { name: 'grid', rows: 1 }
  });
</script>
```

2. Core Concepts

- **Core (`cy`)**: The main graph instance. Used to run layouts, manipulate viewport, and query elements.
- **Collection (`eles`)**: A read-only array of graph elements (nodes and/or edges).

- **Selectors**: Terse string selectors (like CSS) to query elements (e.g. `cy.$('node[weight > 50]')` or `cy.$('#n1')`).

Common Operations

Manipulating Elements

```
// Adding elements
cy.add([
  { group: 'nodes', data: { id: 'n3' } },
  { group: 'edges', data: { id: 'e2', source: 'n2', target: 'n3' } }
]);

// Removing elements
cy.remove('#n1');

// Updating element data (always use .data() to trigger visual updates)
cy.$('#n3').data('label', 'Updated Name');
```

Layouts

Position nodes automatically using built-in or external layout engines:

```
// Run a new layout on the graph
cy.layout({
  name: 'cose', // Compound Spring Embedder
  animate: true,
  animationDuration: 1000
}).run();
```

Built-in layouts: `preset`, `grid`, `random`, `circle`, `concentric`, `breadthfirst`, `cose`.

Graph Theory Algorithms

```
// Dijkstra shortest path
const dijkstra = cy.elements().dijkstra('#n1');
const path = dijkstra.pathTo(cy.$('#n3')); // returns path collection
const distance = dijkstra.distanceTo(cy.$('#n3'));
```

Common Mistakes

- **No container dimensions:** If the container div has 0 width or height, Cytoscape cannot calculate positions. Always define dimensions in CSS before initialization.
- **Direct object mutation:** Mutating node position objects or data objects directly does not trigger rendering. Always use `.position()` or `.data()` to [[update]] elements.
- **Invalid ES Module import:** Do not use `import * as cytoscape` — always use default import `import cytoscape from 'cytoscape'`.

Data-Pipelines.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Data-Pipelines.md

cssclasses: ["electric-dashboard"]

Data Pipelines

Dataform and dbt engineering module.

Automated data quality, transformation, and BigQuery ELT operations run through this interface.

```
// Templater execution block placeholder
<%*
new Notice("Data Pipeline Engineering Interface Loaded.", 5000);
-%>
```

Graph-Intelligence.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Graph-Intelligence.md

cssclasses: ["electric-dashboard"]

Graph Intelligence

Code, docs, and knowledge are mapped into a persistent graph.

Community detection and path analysis are available here.

```
// Templater execution block placeholder
<%*
new Notice("Graph Intelligence and Path Analysis Activated.", 5000);
-%>
```

Index.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Index.md

---
title: SKILLS Index
type: skills-index
status: active

SKILLS Index

> *Live Dataview dashboards per role. Replaces the legacy 5 placeholder dashboards.*

Active Dashboards

Dashboard	Role	Purpose
[[Coordinator]]	1.0COORDINATOR	INBOX/OUTBOX counts, routing matrix.
[[Coder]]	2.0CODER	Open tasks, MCP status, test pass rates.
[[Researcher]]	3.0RESEARCHER	Research briefs, sources, wiki intake.
[[Tools]]	4.0TOOLS	MCP servers, backup, environment.
[[Memory]]	5.0MEMORY	Audit log, review queue, drift score.

Archived Dashboards

Moved to `SKILLS/_archive/`:

- Anti-Obsidian.md
- Core-Systems.md
- Data-Pipelines.md
- Graph-Intelligence.md
- Security-Audit.md

These used a Templater placeholder pattern; replaced by role-mapped Dataview dashboards.

See Also

- `../SYSTEM/agents.yaml`
- `../1.0COORDINATOR/SKILLS/`, `../2.0CODER/SKILLS/`, etc.

Lawliet-aiopenclaw-skills-Lawliet.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Lawliet-aiopenclaw-skills-Lawliet.md

type: dashboard
cssclasses: [cards]
tags:

Stop using AI as a high-end highlighter. Start using it as a cognitive lever.

The Manifesto / DeclarationMost LLM summaries are like high-fructose corn syrup: sweet, but intellectually bankrupt. They tell you what you want to hear, not what is true. These tools exist for one purpose: to transform LLMs from polite parrots into ruthless deconstruction engines.

In an age of information overload, LLM has become a "generator of polite nonsense." The tools here serve only one purpose: **to strip away appearances and get straight to the atomic truth.** We don't need parrots; we need ruthless thinkers who walk the boundaries of physics and logic.

The Infrastructure / Core Foundation### 1. Hive-Commander (The Action-Oriented Swarm)1+5 Distributed Production Kernel / Distributed Production Cluster

- **The Why (Background):** Single-model reasoning density collapses as context grows. When you ask an AI to code, test, and write documentation simultaneously, it chokes.
  When handling multi-dimensional tasks, the logical density of a single-model model drops drastically as the context grows, ultimately leading to cognitive breakdown.
- **The Problem Solved:** It eliminates the "contextual stupidity" of long-form tasks. No more hallucinated code or shallow research.
  It addresses the limitation of single-threaded AI in handling multi-dimensional professional tasks simultaneously.
- **The Logic (Solution):** An asynchronous architecture **of 1 Master + 5 Workers** . The Master deconstructs the mission; the Workers execute in isolated, high-density contexts.
  A master-slave architecture is adopted. The master decomposes the task, and the 5 expert nodes produce outputs in parallel within independent contexts, without interfering with each other.
- **The Edge (its brilliance):** It's not just a "thinking" tool—it's a **production studio** . With built-in **Dev-Mode** and **Slide-Mode** , it delivers actual codebases and presentation decks in parallel.
  *It's not just a "brain," it's a "studio." The built-in development and PPT modes give AI truly industrial-grade delivery capabilities. Its brilliance lies in its ability to directly do the work for you.**

2. Meta-Router (The Cognitive Gateway)High-Performance Intent Router / High-Performance Scheduling Gateway

- **The Why (Background):** *The [[[[

4. First-Principles (The BS-Detector)Truth Audit & Logic Verifier / Truth Audit and "IQ Tax" Detector

- **The Problem:** Marketing hype is everywhere. We need to know in 5 minutes whether a "revolutionary" *tech is a breakthrough or a scam.*
- **The Logic:** Strips away all adjectives. Re- *evaluates claims against Physical Constants* **and** Economic **Realities** .
- **The Edge:** It is the *Cruelest Auditor. If a 3D printer's speed claim violates its motor torque limits, the verdict is "Logically Impossible.* **"** Its ***brilliance lies in being the nightmare of all false advertising.***

Security & Transparency / Security and Access Control Guidelines"These tools have teeth." To

achieve this level of performance, the arsenal requires specific system permissions:

1. **Python Execution (exec)**: `Hive-Commander` runs async scripts to drive parallel production.
2. **Shell Access (bash)**: `Meta-Router` uses `ls/grep` for self-healing indexing of your [[skills]].
3. **Cross-Directory Access**: The Commander must "recruit" [[skills]] from your local directory to build the swarm.

Lawliet's Verdict: I build tools for those who trust logic over generic safety [[[[prompts]]]]. You can audit every line of code. If you want a "safe " *toy, use a highlighter. If you want a harness for your mind, click Allow.*

Author: Lawliet-ai | License: MIT | Powered by OpenClaw 2026

Installation & Usage / Deployment and Armament### One-Click Arming / One-Click Full Installationclawhub install hive-commander meta-router 48h-expert first-principles

Action Scenarios /
Swarm Mode: hive: Develop a personal finance tool with a Web UI.

Audit Mode: L: fp The authenticity of Brand X's "self-developed" high-speed printing algorithm.
Learning Mode: L: 48h The bottom-layer logic of Quantitative Trading.

Memory.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Memory.md

title: Memory Dashboard
role: memory
type: dashboard
status: active
cssclasses: ["role-dashboard"]

Memory Dashboard

> *Live Memory Curator dashboard. Audit last-run, review queue, memory write log, performance metrics, drift score.*

Audit Log Entries

```
TABLE title, status, created FROM "SYSTEM/STYTEM RECORD" SORT created DESC LIMIT 5
```

Review Queue

```
TABLE request_id, target, status FROM "5.0MEMORY/CHAT/Inbox" SORT created DESC LIMIT 5
```

Verdicts Issued

```
TABLE verdict_id, target, status FROM "5.0MEMORY/CHAT/Outbox" SORT created DESC LIMIT 5
```

Drift Score (last run)

```
Run `python 5.0MEMORY/SOLO-PROJECTS/utility-scripts/agent_drift_detector.py --vault-root .` to compute. Scores above 0.30 trigger a system-record note.
```

Last refresh

```
LIST file.mtime
FROM this
```

Related

- `../MEMORY/AGENT.md`
- `../MEMORY/MEMORY/Memory Memory.md`
- `../../SYSTEM/agents.yaml`

Researcher.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Researcher.md

---
title: Researcher Dashboard
role: researcher
type: dashboard
status: active
cssclasses: ["role-dashboard"]

Researcher Dashboard

> *Live Researcher dashboard. Active research briefs, sources added this week, wiki intake.*

Active Research Briefs

```
TABLE title, created, status FROM "3.0RESEARCHER/CHAT/Inbox" SORT created DESC LIMIT 5
```

Recent Wiki Entries

```
TABLE title, modified FROM "MEMORY/KARPATHY/wiki" SORT modified DESC LIMIT 5
```

Raw Entries (pending curation)

```
TABLE title, created, status FROM "MEMORY/KARPATHY/raw" SORT created DESC LIMIT 5
```

Last refresh

```
LIST file.mtime
FROM this
```

Related

- `../RESEARCHER/AGENT.md`
- `../RESEARCHER/MEMORY/Researcher Memory.md`
- `../../SYSTEM/agents.yaml`

Security-Audit.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Security-Audit.md

cssclasses: ["electric-dashboard"]

Security Audit

Pre-deployment OWASP 2026 security audits, supply chain hardening, and auth implementation.

```
// Templater execution block placeholder
<%*
new Notice("Security Audit Protocols Engaged.", 5000);
-%>
```

Skills 2 Windows Task Mngr..md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Skills 2 Windows Task Mngr..md

type: dashboard
cssclasses: [cards]
tags:

Advanced Architectural Diagnostics and Performance Optimization in Windows 11 Task Manager

Modern Architectural Paradigm and User Interface Redesign

The Windows 11 operating system introduced a major architectural redesign of the Task Manager in version 22H2, modernizing its core interface to align with contemporary system design paradigms. The legacy horizontal tabbed design, which had persisted across several operating system generations, was replaced by a modern left-hand navigation pane utilizing the Mica material framework. This framework dynamically integrates desktop background tones with the application frame while supporting native dark and light themes. The modernized interface layout features seven distinct functional pages: Processes, Performance, App History, Startup Apps, Users, Details, and Services, along with a dedicated Settings page represented by a gear icon at the bottom-left corner of the interface.

A key enhancement in this architectural shift is the [[integration]] of a contextual command bar at the top of each page. This bar exposes primary operational controls, such as "Run new task," "End task," and "Efficiency mode". This design represents a major evolution since the utility's introduction during the Windows NT and Windows 2000 eras.

The [[integration]] of Task Manager with the Windows 11 shell has also evolved. While early builds of Windows 11 removed several taskbar configurations—including the direct right-click shortcut to Task Manager—subsequent platform updates reinstated this context menu access following extensive feedback. To optimize administrative efficiency, system engineers can leverage a comprehensive matrix of keyboard shortcuts to launch and navigate the interface.

Skills.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Skills.md

type: dashboard
cssclasses: [cards]
tags:

Skills

> *Skills are the DNA of [[agents]]. A [[[[

Skills in This System

Antigravity Skills (`.gemini/config/skills/`)

| [[[[

Pi Skills (`.pi/agent/skills/`)

| [[[[

Related Nodes

Node	Relationship
[[Antigravity]]	Primary [[[[

Task Manager skill.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Task Manager skill.md

type: dashboard
cssclasses: [cards]
tags:

Task Manager remains one of Windows' most powerful built-in diagnostic and management tools. Windows 11 introduces a modernized interface with improved visuals, better organization, Efficiency Mode for power and performance tuning, enhanced columns (like Power Usage), NPU monitoring in newer builds, and advanced troubleshooting options like live kernel dumps and wait chain analysis.f37c42ac532b

This guide draws from Microsoft documentation, expert tutorials, and practical usage for power users, sysadmins, and troubleshooters.

Accessing Task Manager (Shortcuts & Methods)

Fastest: Ctrl + Shift + Esc (works even if the desktop is frozen).3a6e0a

Right-click the Start button (or Win + X) → Task Manager.

Ctrl + Alt + Del → Task Manager.

Search "Task Manager" in Start or run taskmgr via Win + R.

For full view on first open (if in simple mode): Click More details at the bottom.

Pro Tip: Set a preferred default tab (e.g., Performance) via Task Manager Settings (bottom-left gear icon) → Default startup page.9cfca3

Always on Top: In Settings or via menu for monitoring full-screen apps/games.0248ad

Core Tabs Overview

Task Manager has these main tabs (left sidebar in full view):

Processes (default): Real-time list of apps, background processes, and resource usage (CPU, Memory, Disk, Network, GPU, Power Usage). Expand grouped items (e.g., browser with tabs) via the arrow.80fb75

Performance: Graphs and summaries for CPU (with kernel/user times, per-core views), Memory, Disks (type, speed), Wi-Fi/Ethernet, GPU (including temperature in some views), and NPU in newer hardware.b98a7e

App History: Historical CPU/Network usage per app (current user). Useful for spotting resource hogs over time.727d2e

Startup Apps: Manage boot-time programs with "Startup impact" (Low/Medium/High). Disable bloatware here.f9fd81

Users: Signed-in users and their processes. Log off others or manage resources per user.c66761

Details: Advanced process list (PID, [[Status]], User name, Threads, etc.). Add columns via right-click header → Select columns.cb182b

Services: View/start/stop Windows services (similar to services.msc).907b84

Customization: Right-click column headers to add/remove columns (e.g., Command Line, Power Usage Trend, I/O Reads/Writes, GPU, Threads). Sort by clicking headers.0cccb7

Key Power Features &
[[[

Workflow: Gaming or heavy multitasking? Apply to background apps (e.g., OneDrive, antivirus components) to free resources. Toggle off if instability occurs. Disable confirmation prompt in Task Manager Settings.776fd7

3. Startup & Boot Optimization

Startup Apps Tab: Disable high-impact items. Cross-reference with msconfig for services.

Workflow: Slow boot? Disable unnecessary apps. Reboot and monitor Performance tab for improvements. Use BIOS time insights indirectly via overall boot analysis.2e93a7

4. Advanced Process Analysis

Command Line Column: Reveal full launch paths for security/troubleshooting.c87c4a

Create Dump File: Right-click process (Processes or Details) → Create dump file. Saves to %LOCALAPPDATA%\Temp for debugging crashes (useful for devs/support).f94853

Live Kernel Memory Dump: Right-click System process → Create live kernel memory dump file (Full or Kernel Stacks). Great for kernel-level issues.079b84

Analyze Wait Chain: Details tab → Right-click running process → Analyze wait chain. Identifies dependencies causing hangs. End blocking processes if safe.adb2e117c2bc

Workflow Example (Frozen App): App unresponsive → Details tab → Analyze wait chain → End dependent process → Resume work without full restart.

5. Network & Security Monitoring

Processes tab: Network column for real-time activity.

App History: Historical network usage (spot data exfiltration or heavy sync apps).

Workflow: Suspect malware? Sort by Network. Check Command Line for anomalies. Cross with Users tab for multi-user issues.365197

6. Multi-User & Services Management

Users tab: Log off idle sessions.

Services tab: Start/stop non-Microsoft services (hide Microsoft ones for safety).03e70f

7. Performance Monitoring & Widgets

Performance tab graphs + Summary View as always-on-top desktop widget.

Add GPU/Per-core views. Monitor drive media type (SATA vs. NVMe).cb7794

Keyboard Shortcuts (Power User Essentials):

Open: Ctrl + Shift + Esc

End task: Select + Delete or Ctrl + E

Efficiency Mode: Alt + V

Switch tabs: Arrow keys or direct navigation

Refresh: F5 (in some contexts)

Example
[[[

Scenario 4: Pre-Upgrade/Optimization

Disable heavy startups.

Baseline Performance metrics.

Use for post-change comparisons.

Tips & Best Practices

Run as admin when needed (right-click in Start search).

Updates improve reliability (e.g., faster process handling, accessibility, NPU support).e9cd52

Combine with Resource Monitor (resmon) for deeper suspend/resume.

Avoid ending critical system processes (Task Manager often prevents this).

For scripting/automation: PowerShell alternatives like Get-Process, but Task Manager is visual king.

Task Manager evolves with Windows 11 updates—check for new columns or AI-related features in future builds. Master it, and you'll troubleshoot faster, optimize better, and maintain a snappier system.c73dee

Tools.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\Tools.md

title: Tools Dashboard
role: tools
type: dashboard
status: active
cssclasses: ["role-dashboard"]

Tools Dashboard

> *Live Tools dashboard. MCP server health, backup currency, server uptime, environment changes.*

MCP Servers

```
TABLE mcp_id, state FROM "MCP" WHERE type = "server-spec"
```

Backup Log

```
TABLE when, status FROM "4.0TOOLS/MEMORY" WHERE file.name = "Backup Log" LIMIT 1
```

Workflows (active)

```
TABLE workflow, trigger, owner FROM "WORKFLOWS" WHERE type = "workflow"
```

Last refresh

```
LIST file.mtime
FROM this
```

Related

- `../TOOLS/AGENT.md`
- `../TOOLS/MEMORY/Tools Memory.md`
- `../../SYSTEM/agents.yaml`

_Dashboard.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS_Dashboard.md

type: dashboard
cssclasses: [cards]
tags:

[[10.03 [[SKILLS]]]]

> *[!abstract] Overview*
> *Structural index for the **[[10.03 [[SKILLS]]]]** sector.*

Nested Structure

```
LIST rows.file.link
FROM "10.03 SKILLS"
WHERE file.name != this.file.name AND file.folder != "10.03 SKILLS"
GROUP BY file.folder
```

Sector Contents

```
TABLE file.mtime AS "Last Modified", length(file.outlinks) AS "Out Links", length(file.inlinks) AS "In Links"
FROM "10.03 SKILLS"
WHERE file.name != this.file.name AND file.folder = "10.03 SKILLS"
SORT file.mtime DESC
```

anthropicsskills Public repository for Agent Skills.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\anthropicsskills Public repository for Agent Skills.md

type: dashboard
cssclasses: [cards]
tags:

[[Skills]] are folders of instructions, scripts, and resources that Claude loads dynamically to improve performance on specialized tasks. [[Skills]] teach Claude how to complete specific tasks in a repeatable way, whether that's creating documents with your company's brand guidelines, analyzing data using your organization's specific [[[[

1. Select `Browse and install plugins`
2. Select `anthropic-agent-[[skills]]`
3. Select `document-[[skills]]` or `example-[[skills]]`
4. Select `Install now`

Alternatively, directly install either Plugin via:

```
/plugin install document-skills@anthropic-agent-skills
/plugin install example-skills@anthropic-agent-skills
```

After installing the plugin, you can use the [[[[

bgreenwelldotagents A proposed standard for the .agents directory to prevent context bloat and improve agent reasoning in complex codebases.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\bgreenwelldotagents A proposed standard for the .agents directory to prevent context bloat and improve agent reasoning in complex codebases.md

type: dashboard
cssclasses: [cards]
tags:

A directory-as-context standard for AI [[agents]].

> ***[[Status]]:*** *[[Proposal]] / Draft 0.1.0*
>
> ***Inspiration:*** *Based on experience, emerging patterns in agentic coding, and Issue #71 in agentsmd/agents.md.*

The problem: Context file manageabilityAs AI [[agents]] become integral to development, single context files (e.g., `[[AGENTS]].md`, `CLAUDE.md`, `.cursorrules`) can become difficult to manage.

A monolithic file often leads to:

1. **Inefficient token usage:** [[Agents]] read irrelevant information (e.g., database schemas when working on CSS).
2. **Conflicting instructions:** Mixing behavioral rules with static knowledge can confuse priority.
3. **Tool noise:** Multiple vendor-specific folders (`.claude/`, `.gemini/`, `.cursor/`) clutter the root directory.

The solution: Organization via `[[AGENTS]].md` The **[[dotagents]]** standard is a philosophy of organization. It proposes using a slim `[[AGENTS]].md` file in your project root to act as a **router**, directing [[agents]] to deeper context only when they need it.

While you can organize your project context however you like, we recommend using a hidden `.[[agents]]/` directory to keep your root clean and provide a predictable structure for different types of agent data.

Recommended directory structureThis structure is a starting point—feel free to adapt it to your project's needs.

```
.
 AGENTS.md             # Entry point & router (Required)
 .agents/              # Recommended context directory
     rules/            # Invariant behavioral guidelines
        coding.md     # e.g. "No \`any\` types"
        comms.md      # e.g. "Be concise"
     context/          # Static reference data (read-only)
        schema.sql    # Database structure
        api.ts        # API interfaces
     logs/             # Agent activity logs & audit trails
        session_1.md
     memory/           # Persistent project knowledge (read/write)
        decisions.md  # ADRs (why we chose X over Y)
        user.md       # Learned user preferences
     personas/         # Specialized agent profiles
        qa.md         # e.g. "QA Engineer" hat
     skills/           # Executable capabilities (agentskills.io compliant)
        database-migration/
            SKILL.md
            scripts/
                migrate.sh
     specs/            # Current task requirements
         feature_x.md
```

Usage### 1. The root file (`[[AGENTS]].md`)The heart of the standard. This file defines the agent's identity and provides instructions on where to find specific information. By keeping this file small, you save tokens and maintain clarity.

Example `[[AGENTS]].md` :

[[AGENTS]].md

Identity
You are a Senior Rust Engineer focused on safety and performance.

Context routing
- **If working on database:** READ `.[[agents]]/context/schema.sql`
- **If writing new features:** CHECK `.[[agents]]/specs/` for active PRDs.
- **If facing a decision:** CONSULT `.[[agents]]/memory/decisions.md` to ensure consistency.

Capabilities
- You may execute scripts found in `.[[agents]]/[[skills]]/` to validate your work.

2. The `.[[agents]]/` directory (Ideas for organization)This directory is where you store the "heavy" context. By organizing it logically, you enable Progressive Disclosure—loading only what is necessary for the task at hand.

Personas*Recommended for: Specialized roles.* Store specialized agent "hats" (e.g., `qa.md`, `architect.md`) here. You can instruct your agent to "adopt the QA persona found in `.[[agents]]/personas/qa.md`" when it's time to test.

Logs*Recommended for: Audit & Debugging.* Keep a clean root by storing session logs, agent thought traces, or execution history in a dedicated folder.

Memory*Recommended for: Long-term knowledge.* A place for [[agents]] to persist what they've learned about your preferences or the project's history (e.g., `.[[agents]]/memory/decisions.md`).

**[[Skills]]*Recommended for: Executable capabilities.* This directory is a perfect home for agentskills.io compliant [[skills]]. Place your `[[[

iannuttalldotagents One location for all of your hooks, commands, skills, and AGENTCLAUDE.md files.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\iannuttalldotagents One location for all of your hooks, commands, skills, and AGENTCLAUDE.md files.md

type: dashboard
cssclasses: [cards]
tags:

[[dotagents]]
One canonical .[[agents]] folder that powers all your AI tools.

Simple [[setup]] • One source of truth • Safe to re-run anytime

Quick StartRequirements: Bun 1.3+.

Run the guided CLI:

npx @iannuttall/[[dotagents]]

Or with Bun:

bunx @iannuttall/[[dotagents]]

Choose a workspace (Global home or Project folder), select the clients you want to manage, and follow the [[[[prompts]]]]. You can run it again anytime to repair links or undo changes.

Global home affects all projects. Project folder only affects the current directory you run [[dotagents]] from.

What it does- Keeps `.[[agents]]` as the source of truth.

- Creates symlinks for Claude, Codex, Factory, Cursor, OpenCode, and Gemini (based on your selection).
- Always creates a backup before any overwrite so changes are reversible.

Where it links (global scope) `.[[agents]]/CLAUDE.md` → `~/.claude/CLAUDE.md` (if present)

`.[[agents]]/[[AGENTS]].md` → `~/.claude/CLAUDE.md` (fallback when no CLAUDE.md)

`.[[agents]]/GEMINI.md` → `~/.gemini/GEMINI.md` (if present)

`.[[agents]]/[[AGENTS]].md` → `~/.gemini/GEMINI.md` (fallback when no GEMINI.md)

`.[[agents]]/commands` → `~/.claude/commands`

`.[[agents]]/commands` → `~/.factory/commands`

`.[[agents]]/commands` → `~/.codex/[[[[prompts]]]]`

`.[[agents]]/commands` → `~/.cursor/commands`

`.[[agents]]/commands` → `~/.config/opencode/commands`

`.[[agents]]/commands` → `~/.gemini/commands`

`.[[agents]]/hooks` → `~/.claude/hooks`

`.[[agents]]/hooks` → `~/.factory/hooks`

`.[[agents]]/[[AGENTS]].md` → `~/.factory/[[AGENTS]].md`

`.[[agents]]/[[AGENTS]].md` → `~/.codex/[[AGENTS]].md`

`.[[agents]]/[[AGENTS]].md` → `~/.config/opencode/[[AGENTS]].md`

`.[[agents]]/[[skills]]` → `~/.claude/[[skills]]`

`.[[agents]]/[[skills]]` → `~/.factory/[[skills]]`

`.[[agents]]/[[skills]]` → `~/.codex/[[skills]]`

`.[[agents]]/[[skills]]` → `~/.cursor/[[skills]]`

`.[[agents]]/[[skills]]` → `~/.config/opencode/[[skills]]`

`.[[agents]]/[[skills]]` → `~/.gemini/[[skills]]`

Project scope links only commands/hooks/[[skills]] into the project's client folders (no [[AGENTS]]/CLAUDE/GEMINI rules).

DevelopmentRun the CLI in dev mode:

bun run dev

Type-check:

bun run type-check

Run tests:

bun test

Build the CLI:

bun run build

Notes- Cursor supports `.claude/commands` and `.claude/[[skills]]` (global or project). [[dotagents]] also links `.[[agents]]/commands` → `.cursor/commands` and `.[[agents]]/[[skills]]` → `.cursor/[[skills]]`.

- OpenCode uses `~/.config/opencode/[[AGENTS]].md` and prefers [[AGENTS]].md over CLAUDE.md when both exist.
- Codex [[[[prompts]]]] always symlink to `.[[agents]]/commands` (canonical source).
- [[Skills]] require a valid `[[[

kyegomezPROMPTS.md Understanding CLAUDE.md, MEMORY.md, SKILLS.md, SOUL.md, and Related Prompting Mechanisms.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\kyegomezPROMPTS.md Understanding CLAUDE.md, MEMORY.md, SKILLS.md, SOUL.md, and Related Prompting Mechanisms.md

type: dashboard
cssclasses: [cards]
tags:

Understanding CLAUDE.md, MEMORY.md, [[SKILLS]].md, SOUL.md, and Related Prompting Mechanisms

As agentic systems like Claude become embedded in developer [[[[

A Next.js SaaS application with a PostgreSQL backend.

Commands
- `npm run dev` — start dev server
- `npm test` — run Jest tests
- `npx prisma migrate dev` — apply DB migrations

Conventions
- Use TypeScript strict mode
- All API routes live in `/app/api/`
- Never commit secrets to git; use `.env.local`

Constraints
- Do not modify `prisma/schema.prisma` without explicit instruction
- Do not use `any` type in TypeScript

Where It Lives and How It's LoadedIn Claude Code, `CLAUDE.md` is automatically loaded from the current working directory, any parent directories, and a global `~/.claude/CLAUDE.md` for user-wide settings. Sub-directories can also have their own `CLAUDE.md` files for more granular context.

For non-Claude Code setups, developers typically read `CLAUDE.md` programmatically and inject it as part of the system prompt.

References- Anthropic Claude Code Documentation — Memory

- Claude Code Overview

Part 3: MEMORY.md### What It Is`MEMORY.md` is a convention for persisting facts, preferences, and learned context across sessions. Unlike `CLAUDE.md` (which is project-scoped), `MEMORY.md` tends to be user-scoped or agent-scoped — it captures *what the AI has learned about you or your context* over time.

The challenge it solves is real: LLMs have no native memory between conversations. Every session starts fresh. `MEMORY.md` is a workaround: a human-readable file that is updated either manually or automatically as conversations produce new information.

Structure and ContentsA `MEMORY.md` file typically contains facts in a structured, retrievable format:

Memory

User Preferences
- Prefers TypeScript over JavaScript
- Uses dark mode; prefers minimal UI
- Works primarily in Pacific Time (UTC-8)

Project Context
- Current project: Rewriting the billing module in Rust
- Deadline: End of Q2 2026
- Key stakeholder: Alex (CTO)

Decisions Made
- 2026-03-15: Decided to use Stripe instead of PayPal
- 2026-04-01: Chose Redis for session caching

Things to Avoid
- Don't suggest GraphQL — team decided against it in 2025

How It Gets UpdatedThere are three main approaches:

1. **Manual updates** — The user explicitly edits the file after each session, noting what should be remembered.
2. **Agent-assisted updates** — At the end of a session, the user asks the AI to summarize key takeaways and append them to `MEMORY.md` .
3. **Automatic memory systems** — More advanced setups (like Claude's built-in memory feature in Claude.ai, or third-party tools like Mem0 or MemGPT) automatically extract and write facts to a memory store, which is then injected into future sessions.

Claude.ai's Native Memory FeatureClaude.ai has a built-in memory system (distinct from `MEMORY.md` as a file convention) that stores summaries of past conversations and makes them available in future sessions. Users can enable or disable this in Settings. The model will proactively generate memories from chat history. This is the managed equivalent of maintaining a `MEMORY.md` file manually.

When to Use MEMORY.md `MEMORY.md` is most useful for:

- Long-running projects where context accumulates over weeks or months
- Personal AI assistants where preferences and habits should persist
- Agentic [[[[

- Teams sharing a context file across multiple developers working with the same AI agent

References- [Claude.ai Memory Settings]

- Mem0 — Open-source memory layer for AI agents
- MemGPT research paper

Part 4: [[SKILLS]].md### What It Is `[[SKILLS]].md` is a capability registry — a file that documents what specialized tools, functions, [[[[

Part 5: SOUL.md### What It Is `SOUL.md` is the most philosophical of these files — and the most varied in how different developers implement it. At its core, it is a values and identity document: it tells the AI *who it is*, *what it cares about*, and *how it should reason about difficult situations*.

Where `CLAUDE.md` is operational and `MEMORY.md` is informational, `SOUL.md` is normative — it encodes ethical commitments, communication style, personality traits, and reasoning principles.

Origins and ContextThe concept emerged from the practice of writing detailed system [[[[prompts]]]] for AI personas. Developers building AI products (customer support [[agents]], creative writing assistants, coding mentors) found that behavioral instructions scattered across a system prompt were hard to maintain. Extracting them into a dedicated `SOUL.md` file made the model's "character" explicit, auditable, and versioned alongside code.

For personal AI setups, `SOUL.md` is a way of answering: "If this AI is going to act as my assistant long-term, what should it fundamentally be like?"

What Goes in SOUL.md**Core values** — What the AI prioritizes above all else. Honesty, helpfulness, intellectual rigor, user autonomy, safety, etc.

Communication style — Tone (warm vs. formal), verbosity (concise vs. thorough), use of humor, preferred formatting.

Reasoning principles — How to handle ambiguity, when to ask for clarification vs. proceed, how to weigh competing priorities.

Identity statement — A brief description of the persona or character this AI embodies.

Ethical commitments — Hard lines the model should never cross, regardless of instructions.

Example SOUL.md:

```
# Soul

## Identity
You are Aria, a thoughtful engineering mentor and technical collaborator.
You care deeply about craft, clarity, and the humans you work with.

## Core Values
1. **Honesty first** — Never fabricate information. Say "I don't know" freely.
2. **User autonomy** — Help people learn and decide, don't create dependency.
3. **Craft over speed** — Prefer clean, maintainable solutions over quick hacks.
4. **Directness** — Give opinions when asked. Don't hedge unnecessarily.

## Communication Style
- Warm but professional; collegial, not sycophantic
- Concise by default; expand when depth is needed
- Use analogies for complex concepts
- Never use filler phrases like "Certainly!" or "Great question!"

## Reasoning Principles
- When uncertain, say so explicitly
- When instructions conflict with values, flag the conflict and explain
- Prefer asking one focused clarifying question over making assumptions

## Hard Limits
- Never write malicious code, even "for educational purposes"
- Never claim to be human if sincerely asked
- Never demean or belittle the user
```

SOUL.md vs. System [[[[Prompts]]]] `SOUL.md` is essentially a structured, maintainable version of the character-defining portion of a system prompt. Rather than having a 2,000-word system prompt with behavioral instructions mixed with operational instructions, separating them:

- `SOUL.md` → values, personality, communication style
- `CLAUDE.md` → project facts, commands, constraints
- `MEMORY.md` → learned context
- `[[SKILLS]].md` → capabilities

This separation makes each concern independently editable and reviewable.

Anthropic's Own Soul DocumentAnthropic has published what might be called a public "soul document" for Claude — the Claude Model Spec, which describes Claude's values, character, reasoning principles, and ethical commitments in detail. This is the canonical reference for understanding what Claude fundamentally is — and serves as inspiration for creating `SOUL.md` files for derived AI personas.

References- Anthropic Claude Model Spec

- Anthropic's Character overview

Part 6: How These Files Work TogetherThe real power of these files emerges when they are composed into a coherent agent context layer. A typical agent startup sequence might look like:

```
System Prompt = SOUL.md + SKILLS.md + CLAUDE.md + MEMORY.md + [task-specific instructions]
```

Each file handles a different concern:

File	Scope	Answers
`SOUL.md`	Universal (who am I?)	Values, tone, identity
`[[SKILLS]].md`	Capability (what can I do?)	Tools, [[[[

Version control your context files. Put `CLAUDE.md`, `MEMORY.md`, `[[SKILLS]].md`, and `SOUL.md` in Git alongside your code. This gives you history, diffs, and the ability to roll back behavioral changes.

Separate concerns. Don't put project commands in `SOUL.md` or values in `CLAUDE.md`. Clean separation makes each file easier to maintain and reason about.

Test your [[[[prompts]]]]. Treat changes to these files like code changes — run your test cases, observe the model's behavior, iterate. Anthropic's prompt evaluation guidance covers evaluation methodology.

Use XML tags for clarity. When injecting multiple files into a system prompt programmatically, wrap each in labeled XML tags:

<soul>
[contents of SOUL.md]
</soul>

<project_context>
[contents of CLAUDE.md]
</project_context>

<memory>
[contents of MEMORY.md]
</memory>

This helps Claude distinguish between different types of context and apply each appropriately.

Conclusion `CLAUDE.md`, `MEMORY.md`, `SKILLS.md`, and `SOUL.md` represent a maturing discipline of AI context engineering. As AI agents take on more complex, long-running tasks, the quality of the context they operate in becomes as important as the quality of the model itself. These files give developers a practical, version-controlled, human-readable way to specify that context.

The names are conventions, not standards — you can call them anything. What matters is the underlying principle: give your AI agent explicit, structured knowledge of *who it is*, *what it knows*, *what it can do*, and *how it should work in this specific environment*. Done well, context files transform a general-purpose AI into a specialized collaborator that feels like it truly understands your project.

References and Further Reading| Resource | Link |

| --- | --- |
Claude Code Documentation	https://docs.claude.ai/en/docs/claude-code/overview
Claude Code Memory (CLAUDE.md)	https://docs.claude.ai/en/docs/claude-code/memory
Anthropic Prompt Engineering Guide	https://docs.claude.ai/en/docs/build-with-claude/prompt-engineering/overview
Model Context Protocol (MCP)	https://modelcontextprotocol.io/introduction
Claude Model Spec (Soul reference)	https://www.anthropic.com/research/claude-character
Anthropic [[Tool Use]] Docs	https://docs.claude.ai/en/docs/build-with-claude/tool-use/overview
Mem0 — Memory layer for AI	https://github.com/mem0ai/mem0
MemGPT Paper	https://arxiv.org/abs/2310.08560
Claude.ai Support	https://support.claude.ai
Anthropic API Reference	https://docs.claude.ai/en/api

obrasuperpowers An agentic skills framework & software development methodology that works.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\obrasuperpowers An agentic skills framework & software development methodology that works.md

type: dashboard
cssclasses: [cards]
tags:

[[Superpowers]] is a complete software development methodology for your coding [[agents]], built on top of a set of composable [[skills]] and some initial instructions that make sure your agent uses them.

We're Hiring!We're hiring someone to help out full time with [[Superpowers]] community and code work. You can read about the job at https://primeradiant.com/jobs/superpowers-community-engineer/ If this sounds like someone you know, definitely send them our way.

QuickstartGive your agent [[Superpowers]]: Claude Code, Antigravity, Codex App, Codex CLI, Cursor, Factory Droid, Gemini CLI, GitHub Copilot CLI, Kimi Code, OpenCode, Pi.

How it worksIt starts from the moment you fire up your coding agent. As soon as it sees that you're building something, it *doesn't* just jump into trying to write code. Instead, it steps back and asks you what you're really trying to do.

Once it's teased a spec out of the conversation, it shows it to you in chunks short enough to actually read and digest.

After you've signed off on the design, your agent puts together an implementation plan that's clear enough for an enthusiastic junior engineer with poor taste, no judgement, no project context, and an aversion to testing to follow. It emphasizes true red/green TDD, YAGNI (You Aren't Gonna Need It), and DRY.

Next up, once you say "go", it launches a *subagent-driven-development* process, having [[agents]] work through each engineering task, inspecting and reviewing their work, and continuing forward. It's not uncommon for your agent to work autonomously for a couple hours at a time without deviating from the plan you put together.

There's a bunch more to it, but that's the core of the system. And because the [[skills]] trigger automatically, you don't need to do anything special. Your coding agent just has [[Superpowers]].

Commercial ServicesIf you're using [[Superpowers]] in enterprise and could benefit from commercial support, additional tooling, or managed spending, please don't hesitate to drop us a line at sales@primeradiant.com.

InstallationInstallation differs by harness. If you use more than one, install [[Superpowers]] separately for each one.

Claude CodeSuperpowers is available via the official Claude plugin marketplace

Official Marketplace- Install the plugin from Anthropic's official marketplace:

```
/plugin install [[superpowers]]@claude-plugins-official
```

[[Superpowers]] MarketplaceThe [[Superpowers]] marketplace provides [[Superpowers]] and some other related plugins for Claude Code.

- Register the marketplace:
  /plugin marketplace add obra/[[superpowers]]-marketplace
- Install the plugin from this marketplace:
  /plugin install [[superpowers]]@[[superpowers]]-marketplace

AntigravityInstall [[Superpowers]] as a plugin from this repository:

agy plugin install https://github.com/obra/[[superpowers]]

Antigravity runs the plugin's session-start hook, so [[Superpowers]] is active from the first message. Reinstall with the same command to [[update]].

Codex AppSuperpowers is available via the official Codex plugin marketplace.

- In the Codex app, click on Plugins in the sidebar.
- You should see `[[Superpowers]]` in the Coding section.
- Click the `+` next to [[Superpowers]] and follow the [[[[prompts]]]].

Codex CLISuperpowers is available via the official Codex plugin marketplace.

- Open the plugin search interface:
  /plugins
- Search for [[Superpowers]]:
  [[superpowers]]
- Select `Install Plugin`.

Cursor- In Cursor Agent chat, install from marketplace:

```
```
/add-plugin superpowers
```
```

- Or search for "[[superpowers]]" in the plugin marketplace.

Factory Droid- Register the marketplace:

```
droid plugin marketplace add https://github.com/obra/[[superpowers]]
```

- Install the plugin:
  droid plugin install [[superpowers]]@[[superpowers]]

Gemini CLI- Install the extension:

```
gemini extensions install https://github.com/obra/[[superpowers]]
```

- [[Update]] later:
  gemini extensions [[update]] [[superpowers]]

GitHub Copilot CLI- Register the marketplace:

```
copilot plugin marketplace add obra/[[superpowers]]-marketplace
```

- Install the plugin:
  copilot plugin install [[superpowers]]@[[superpowers]]-marketplace

Kimi CodeSuperpowers is available in Kimi Code's plugin marketplace.

- Open Kimi Code's plugin manager:
  `/plugins`
- Go to `Marketplace` > `[[Superpowers]]` and install it.
- Or install directly from this repository:
  `/plugins install https://github.com/obra/superpowers`
- Detailed docs: docs/README.kimi.md

OpenCodeOpenCode uses its own plugin install; install [[Superpowers]] separately even if you already use it in another harness.

- Tell OpenCode:
  `Fetch and follow instructions from https://raw.githubusercontent.com/obra/superpowers/refs/heads/main/.opencode/INSTALL.md`
- Detailed docs: docs/README.opencode.md

PiInstall [[Superpowers]] as a Pi package from this repository:

pi install git:github.com/obra/[[superpowers]]

For local development, run Pi with this checkout loaded as a temporary package:

pi -e /path/to/[[superpowers]]

The Pi package loads the [[Superpowers]] [[skills]] and a small extension that injects the `using-[[superpowers]]` bootstrap at session startup and again after compaction. Pi has native [[skills]], so no compatibility `[[[

3. **writing-plans** - Activates with approved design. Breaks work into bite-sized tasks (2-5 minutes each). Every task has exact file paths, complete code, verification steps.
4. **subagent-driven-development** or **executing-plans** - Activates with plan. Dispatches fresh subagent per task with two-stage [[review]] (spec compliance, then code quality), or executes in batches with human checkpoints.
5. **test-driven-development** - Activates during implementation. Enforces RED-GREEN-REFACTOR: write failing test, watch it fail, write minimal code, watch it pass, commit. Deletes code written before tests.
6. **requesting-code-[[review]]** - Activates between tasks. Reviews against plan, reports issues by severity. Critical issues block progress.
7. **finishing-a-development-branch** - Activates when tasks complete. Verifies tests, presents options (merge/PR/keep/discard), cleans up worktree.

The agent checks for relevant [[skills]] before any task. Mandatory [[[[

UpdatingSuperpowers updates are somewhat coding-agent dependent, but are often automatic.

LicenseMIT License - see LICENSE file for details

Visual companion telemetryBecause [[skills]] and plugins don't provide any feedback to creators, we have no idea how many of you are using [[Superpowers]]. By default, the Prime Radiant logo on brainstorming's optional visual companion feature is loaded from our website. It includes the version of [[Superpowers]] in use. It does not include any details about your project, prompt, or coding agent. We don't see your clicks or anything about what you're building. This helps us have a rough idea of how many folks are using [[Superpowers]] and which version of [[Superpowers]] they're using. It's 100% optional. To disable this, set the environment variable `SUPERPOWERS_DISABLE_TELEMETRY` to any true value. [[Superpowers]] also honors Claude Code's `DISABLE_TELEMETRY` and `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` opt-outs.

open-gitagentgitagent A universal git-native AI agent framework. Your agent lives inside a git repo — identity, rules, memory, tools, and skills are all version-controlled files.md

C:\Users\Admin\Desktop\Music\LOOP\SKILLS\open-gitagentgitagent A universal git-native AI agent framework. Your agent lives inside a git repo — identity, rules, memory, tools, and skills are all version-controlled files.md

type: dashboard
cssclasses: [cards]
tags:

A universal git-native multimodal always learning AI Agent (TinyHuman)
Your agent lives inside a git repo — identity, rules, memory, tools, and [[skills]] are all version-controlled files.

Install • Quick Start • SDK • Architecture • Tools • Hooks • Skills • Plugins

---

Why [[Gitagent]]?Most agent frameworks treat configuration as code scattered across your application. [[Gitagent]] flips this — your agent IS a git repository:

- `agent.yaml` — model, tools, runtime config
- `SOUL.md` — personality and identity
- `RULES.md` — behavioral constraints
- `memory/` — git-committed memory with full history
- `tools/` — declarative YAML tool definitions
- `[[skills]]/` — composable [[[[

If you only use the SDK / non-voice CLI:

npm install -g @open-[[gitagent]]/[[gitagent]]@latest

The `[[gitagent]]` command and `@open-[[gitagent]]/[[gitagent]]` SDK exports are unchanged. `[[gitagent]] --voice` dynamically loads `@open-[[gitagent]]/voice`; without it installed, it prints a one-line install hint and exits cleanly.

Quick StartRun your first agent in one line:

export OPENAI_API_KEY="sk-..."
[[gitagent]] --dir ~/my-project "Explain this project and suggest improvements"

That's it. [[Gitagent]] auto-scaffolds everything on first run — `agent.yaml`, `SOUL.md`, `memory/` — and drops you into the agent.

Local Repo ModeClone a GitHub repo, run an agent on it, auto-commit and push to a session branch:

[[gitagent]] --repo https://github.com/org/repo --pat ghp_xxx "Fix the login bug"

Resume an existing session:

[[gitagent]] --repo https://github.com/org/repo --pat ghp_xxx --session [[gitagent]]/session-a1b2c3d4 "Continue"

Token can come from env instead of `--pat`:

export GITHUB_TOKEN=ghp_xxx
[[gitagent]] --repo https://github.com/org/repo "Add unit tests"

CLI Options| Flag | Short | Description |

| --- | --- | --- |
`--dir <path>`	`-d`	Agent directory (default: cwd)
`--repo <url>`	`-r`	GitHub repo URL to clone and work on
`--pat <token>`		GitHub PAT (or set `GITHUB_TOKEN` / `GIT_TOKEN`)
`--session <branch>`		Resume an existing session branch
`--model <provider:model>`	`-m`	Override model (e.g. `anthropic:claude-sonnet-4-5-20250929`)
`--sandbox`	`-s`	Run in sandbox VM
`--prompt <text>`	`-p`	Single-shot prompt (skip REPL)
`--env <name>`	`-e`	Environment config

SDKimport { query } from "[[gitagent]]";

```
// Simple query
for await (const msg of query({
prompt: "List all TypeScript files and summarize them",
dir: "./my-agent",
model: "openai:gpt-4o-mini",
})) {
if (msg.type \=== "delta") process.stdout.write(msg.content);
if (msg.type \=== "assistant") console.log("\n\nDone.");
}

// Local repo mode via SDK
for await (const msg of query({
prompt: "Fix the login bug",
model: "openai:gpt-4o-mini",
repo: {
url: "https://github.com/org/repo",
token: process.env.GITHUB_TOKEN!,
},
})) {
if (msg.type \=== "delta") process.stdout.write(msg.content);
}
```

SDKThe SDK provides a programmatic interface to [[Gitagent]] [[agents]]. It mirrors the Claude Agent SDK pattern but runs in-process — no subprocesses, no IPC.

`query(options): Query` Returns an `AsyncGenerator<GCMessage>` that streams agent events.

```
import { query } from "[[gitagent]]";

for await (const msg of query({
prompt: "Refactor the auth module",
```

dir: "/path/to/agent",
model: "anthropic:claude-sonnet-4-5-20250929",
})) {
switch (msg.type) {
case "delta": // streaming text chunk
process.stdout.write(msg.content);
break;
case "assistant": // complete response
console.log(`\nTokens: ${msg.usage?.totalTokens}`);
break;
case "tool_use": // tool invocation
console.log(`Tool: ${msg.toolName}(${JSON.stringify(msg.args)})`);
break;
case "tool_result": // tool output
console.log(`Result: ${msg.content}`);
break;
case "system": // lifecycle events & errors
console.log(`[${msg.subtype}] ${msg.content}`);
break;
}
}

`tool(name, description, schema, handler): GCToolDefinition` **Define custom tools the agent can call:**

import { query, tool } from "gitagent";

const search \= tool(
"search_docs",
"Search the documentation",
{
properties: {
query: { type: "string", description: "Search query" },
limit: { type: "number", description: "Max results" },
},
required: ["query"],
},
async (args) \=> {
const results \= await mySearchEngine(args.query, args.limit ?? 10);
return { text: JSON.stringify(results), details: { count: results.length } };
},
);

for await (const msg of query({
prompt: "Find docs about authentication",
tools: [search],
})) {
// agent can now call search_docs
}

HooksProgrammatic lifecycle hooks for gating, logging, and control:

for await (const msg of query({
prompt: "Deploy the service",
hooks: {
preToolUse: async (ctx) \=> {
// Block dangerous operations
if (ctx.toolName \=== "cli" && ctx.args.command?.includes("rm -rf"))
return { action: "block", reason: "Destructive command blocked" };

```
  // Modify arguments
  if (ctx.toolName \=== "write" && !ctx.args.path.startsWith("/safe/"))
    return { action: "modify", args: { ...ctx.args, path: \`/safe/${ctx.args.path}\` } };

  return { action: "allow" };
},
onError: async (ctx) \=> {
  console.error(\`Agent error: ${ctx.error}\`);
},
```

},
})) {
// ...
}

QueryOptions Reference| Option | Type | Description |

| --- | --- | --- |
`prompt`	`string | AsyncIterable`	User prompt or multi-turn stream
`dir`	`string`	Agent directory (default: `cwd`)
`model`	`string`	`"provider:model-id"`
`env`	`string`	Environment config (`config/<env>.yaml`)
`systemPrompt`	`string`	Override discovered system prompt
`systemPromptSuffix`	`string`	Append to discovered system prompt
`tools`	`GCToolDefinition[]`	Additional tools
`replaceBuiltinTools`	`boolean`	Skip cli/read/write/memory
`allowedTools`	`string[]`	Tool name allowlist
`disallowedTools`	`string[]`	Tool name denylist
`repo`	`LocalRepoOptions`	Clone a GitHub repo and work on a session branch
`sandbox`	`SandboxOptions | boolean`	Run in sandbox VM (mutually exclusive with `repo`)
`hooks`	`GCHooks`	Programmatic lifecycle hooks
`maxTurns`	`number`	Max agent turns
`abortController`	`AbortController`	Cancellation signal
`constraints`	`object`	`temperature`, `maxTokens`, `topP`, `topK`

Message Types| Type | Description | Key Fields |

| --- | --- | --- |
`delta`	Streaming text/thinking chunk	`deltaType`, `content`
`assistant`	Complete LLM response	`content`, `model`, `usage`, `stopReason`
`tool_use`	Tool invocation	`toolName`, `args`, `toolCallId`
`tool_result`	Tool output	`content`, `isError`, `toolCallId`
`system`	Lifecycle events	`subtype`, `content`, `metadata`
`user`	User message (multi-turn)	`content`

Architecture```

my-agent/
agent.yaml # Model, tools, runtime config
SOUL.md # Agent identity & personality
RULES.md # Behavioral rules & constraints
DUTIES.md # Role-specific responsibilities
memory/
MEMORY.md # Git-committed agent memory
tools/
*.yaml # Declarative tool definitions
skills/
/
SKILL.md # Skill instructions (YAML frontmatter)
scripts/ # Skill scripts
workflows/
.yaml|*.md # Multi-step workflow definitions
agents/
/ # Sub-agent definitions
plugins/
/ # Local plugins (plugin.yaml + tools/hooks/skills)
hooks/
hooks.yaml # Lifecycle hook scripts
knowledge/
index.yaml # Knowledge base entries
config/
default.yaml # Default environment config
*.yaml # Environment overrides
examples/
.md # Few-shot examples
compliance/
*.yaml # Compliance & audit config

```
### Agent Manifest (`agent.yaml`)spec\_version: "0.1.0"
name: my-agent
version: 1.0.0
description: An agent that does things

model:
  preferred: "anthropic:claude-sonnet-4-5-20250929"
  fallback: \["openai:gpt-4o"\]
  constraints:
    temperature: 0.7
    max\_tokens: 4096

tools: \[cli, read, write, memory\]

runtime:
  max\_turns: 50
  timeout: 120

# Optional
extends: "https://github.com/org/base-agent.git"
skills: \[code-review, deploy\]
delegation:
  mode: auto
compliance:
  risk\_level: medium
  human\_in\_the\_loop: true

## Tools### Built-in Tools| Tool | Description |
| --- | --- |
| `cli` | Execute shell commands |
| `read` | Read files with pagination |
| `write` | Write/create files |
| `memory` | Load/save git-committed memory |

### Declarative ToolsDefine tools as YAML in `tools/`:

# tools/search.yaml
name: search
description: Search the codebase
input\_schema:
  properties:
    query:
      type: string
      description: Search query
    path:
      type: string
      description: Directory to search
  required: \[query\]
implementation:
  script: search.sh
  runtime: sh

The script receives args as JSON on stdin and returns output on stdout.

## HooksScript-based hooks in `hooks/hooks.yaml`:

hooks:
  on\_session\_start:
    - script: validate-env.sh
      description: Check environment is ready
  pre\_tool\_use:
    - script: audit-tools.sh
      description: Log and gate tool usage
  post\_response:
    - script: notify.sh
  on\_error:
    - script: alert.sh

Hook scripts receive context as JSON on stdin and return:

{ "action": "allow" }
{ "action": "block", "reason": "Not permitted" }
{ "action": "modify", "args": { "modified": "args" } }

## SkillsSkills are composable instruction modules in `skills/<name>/`:
```

skills/
code-review/
SKILL.md
scripts/
lint.sh

```
\---
name: code-review
description: Review code for quality and security
\---

\# Code Review

When reviewing code:
1. Check for security vulnerabilities
2. Verify error handling
3. Run the lint script for style checks

Invoke via CLI: `/skill:code-review Review the auth module`

## PluginsPlugins are reusable extensions that can provide tools, hooks, skills, prompts, and memory layers. They follow the same git-native philosophy — a plugin is a directory with a `plugin.yaml` manifest.

### CLI Commands# Install from git URL
gitagent plugin install https://github.com/org/my-plugin.git

# Install from local path
gitagent plugin install ./path/to/plugin

# Install with options
gitagent plugin install <source\> --name custom-name --force --no-enable

# List all discovered plugins
gitagent plugin list

# Enable / disable
gitagent plugin enable my-plugin
gitagent plugin disable my-plugin

# Remove
gitagent plugin remove my-plugin

# Scaffold a new plugin
gitagent plugin init my-plugin

| Flag | Description |
| --- | --- |
| `--name <name>` | Custom plugin name (default: derived from source) |
| `--force` | Reinstall even if already present |
| `--no-enable` | Install without auto-enabling |

### Plugin Manifest (`plugin.yaml`)id: my-plugin                    # Required, kebab-case
name: My Plugin
version: 0.1.0
description: What this plugin does
author: Your Name
license: MIT
engine: "\>=0.3.0"              # Min gitagent version

provides:
  tools: true                   # Load tools from tools/\*.yaml
  skills: true                  # Load skills from skills/
  prompt: prompt.md             # Inject into system prompt
  hooks:
    pre\_tool\_use:
      - script: hooks/audit.sh
        description: Audit tool calls

config:
  properties:
    api\_key:
      type: string
      description: API key
      env: MY\_API\_KEY           # Env var fallback
    timeout:
      type: number
      default: 30
  required: \[api\_key\]

entry: index.ts                 # Optional programmatic entry point

### Plugin Config in `agent.yaml`plugins:
  my-plugin:
```

```
    enabled: true
    source: https://github.com/org/my-plugin.git  # Auto-install on load
    version: main                                 # Git branch/tag
    config:
      api\_key: "${MY\_API\_KEY}"                     # Supports env interpolation
      timeout: 60

Config resolution priority: `agent.yaml config` > `env var` > `manifest default`.

### Discovery OrderPlugins are discovered in this order (first match wins):

1. **Local** — `<agent-dir>/plugins/<name>/`
2. **Global** — `~/.gitagent/plugins/<name>/`
3. **Installed** — `<agent-dir>/.gitagent/plugins/<name>/`

### Programmatic PluginsPlugins with an `entry` field in their manifest get a full API:

// index.ts
import type { GitagentPluginApi } from "gitagent";

export async function register(api: GitagentPluginApi) {
  // Register a tool
  api.registerTool({
    name: "search\_docs",
    description: "Search documentation",
    inputSchema: {
      properties: { query: { type: "string" } },
      required: \["query"\],
    },
    handler: async (args) \=> {
      const results \= await search(args.query);
      return { text: JSON.stringify(results) };
    },
  });

  // Register a lifecycle hook
  api.registerHook("pre\_tool\_use", async (ctx) \=> {
    api.logger.info(\`Tool called: ${ctx.tool}\`);
    return { action: "allow" };
  });

  // Add to system prompt
  api.addPrompt("Always check docs before answering questions.");

  // Register a memory layer
  api.registerMemoryLayer({
    name: "docs-cache",
    path: "memory/docs-cache.md",
    description: "Cached documentation lookups",
  });
}

**Available API methods:**

| Method | Description |
| --- | --- |
| `registerTool(def)` | Register a tool the agent can call |
| `registerHook(event, handler)` | Register a lifecycle hook (`on_session_start`, `pre_tool_use`, `post_response`, `on_error`) |
| `addPrompt(text)` | Append text to the system prompt |
| `registerMemoryLayer(layer)` | Register a memory layer |
| `logger.info/warn/error(msg)` | Prefixed logging (`[plugin:id]`) |
| `pluginId` | Plugin identifier |
| `pluginDir` | Absolute path to plugin directory |
| `config` | Resolved config values |

### Plugin Structure```
my-plugin/
 plugin.yaml          # Manifest (required)
 tools/               # Declarative tool definitions
    *.yaml
 hooks/               # Hook scripts
 skills/              # Skill modules
 prompt.md            # System prompt addition
 index.ts             # Programmatic entry point
```

Multi-Model SupportGitagent works with any LLM provider supported by pi-ai:

agent.yaml

model:
preferred: "anthropic:claude-sonnet-4-5-20250929"
fallback:
- "openai:gpt-4o"
- "google:gemini-2.0-flash"

Supported providers: `anthropic`, `openai`, `google`, `xai`, `groq`, `mistral`, and more.

Inheritance & CompositionAgents can extend base [[agents]]:

agent.yaml

extends: "https://github.com/org/base-agent.git"

Dependencies

dependencies:
- name: shared-tools
source: "https://github.com/org/shared-tools.git"
version: main
mount: tools

Sub-[[agents]]

delegation:
mode: auto

Compliance & AuditBuilt-in compliance validation and audit logging:

agent.yaml

compliance:
risk_level: high
human_in_the_loop: true
data_classification: confidential
regulatory_frameworks: [SOC2, GDPR]
recordkeeping:
audit_logging: true
retention_days: 90

Audit logs are written to `.gitagent/audit.jsonl` with full tool invocation traces.

TelemetryGitagent ships with built-in OpenTelemetry instrumentation. Set `OTEL_EXPORTER_OTLP_ENDPOINT` and telemetry is on; leave it unset and runtime cost is zero.

Three layers of signals:

1. **HTTP-level** — `@opentelemetry/instrumentation-undici` auto-patches `fetch` / `undici`, so every LLM provider call (Anthropic, OpenAI, Google, …) gets a client span with URL, status code, and timing.
2. `gen_ai.chat` **spans** — emitted on every assistant `message_end`. Carry `gen_ai.system`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `gen_ai.response.finish_reasons`, and `gitagent.cost_usd`. Span/metric content never contains the prompt or completion text.
3. `gitagent.tool.execute` **spans** — wrap every tool call with `tool.name`, `tool.call_id`, `tool.status` (`ok` / `error`), and `tool.error_message` on failure.

A root `gitagent.agent.session` span opens at agent construction and closes on every exit path (success, hook-block, SIGINT, error).

CLI usageJust set the endpoint — no `--import` flag, no extra install steps:

OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 gitagent -p "your prompt"

Telemetry is enabled automatically when the endpoint is set and disabled when it is not. To force-disable even when the endpoint is set, pass `GITAGENT_OTEL_ENABLED=false`.

Environment variables| Variable | Description | Default |

| --- | --- | --- |
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP/HTTP collector base URL (e.g. `http://localhost:4318`). When set, telemetry is auto-enabled.	(unset → telemetry off)
`GITAGENT_OTEL_ENABLED`	Set to `false` to disable telemetry even when the endpoint is set	(unset = auto)
`OTEL_SERVICE_NAME`	Resource `service.name`	`gitagent`
`OTEL_SERVICE_VERSION`	Resource `service.version`	(unset)
`OTEL_EXPORTER_OTLP_HEADERS`	Comma-separated key=value pairs, no quotes (e.g. `Authorization=Bearer xyz,x-tenant=abc`)	(unset)
`OTEL_TRACES_EXPORTER`	Set to `console` to print spans to stdout — no collector needed	(unset)

SDK usageFor programmatic embedders, call `initTelemetry` explicitly — you control when initialisation happens:

import { initTelemetry, shutdownTelemetry, query } from "gitagent";

await initTelemetry({ serviceName: "my-app" });

for await (const msg of query({ prompt: "hello", model: "anthropic:claude-4-6-sonnet-latest" })) {
// ...
}

await shutdownTelemetry();

`OTEL_EXPORTER_OTLP_ENDPOINT` and `OTEL_EXPORTER_OTLP_HEADERS` are read automatically by the OTLP exporter when not supplied programmatically. Pass `exporterEndpoint` / `headers` only when you need to override env-based config in code.

Emitted spans| Name | Kind | Key attributes |

| --- | --- | --- |
`gitagent.agent.session`	INTERNAL	`gitagent.entry` (`sdk` / `cli`), `gitagent.cost_usd`, `gitagent.session.duration_ms`
`gitagent.tool.execute`	INTERNAL	`tool.name`, `tool.call_id`, `tool.status`, `tool.error_message`
`gen_ai.chat`	CLIENT	`gen_ai.system`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`, `gen_ai.response.finish_reasons`, `gitagent.cost_usd`
`HTTP …`	CLIENT	URL, status code, duration (auto from `instrumentation-undici`)

Emitted metrics| Name | Type | Description |

| --- | --- | --- |
`gitagent.tool.calls`	counter	Number of tool executions, labelled by `tool.name`
`gitagent.tool.duration_ms`	histogram	Tool execution duration
`gitagent.session.duration_ms`	histogram	Session duration
`gitagent.session.cost_usd`	counter (USD)	Cumulative session cost
`gen_ai.client.token.usage`	counter	Token usage by `gen_ai.system`, `gen_ai.request.model`, `gen_ai.token.type`
`gen_ai.client.operation.duration`	histogram	LLM call duration

Console quickstart (no collector)Print spans directly to stdout — useful for local debugging:

OTEL_TRACES_EXPORTER=console gitagent -p "test"

Local Jaeger quickstartdocker run --rm -p 16686:16686 -p 4318:4318 jaegertracing/all-in-one:latest

OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 gitagent -p "test"

Open http://localhost:16686 → service "gitagent"

ContributingContributions are welcome! Please see CONTRIBUTING.md for guidelines.

FAQ### GeneralWhat is [[Gitagent]]? [[GitAgent]] (formerly Gitclaw) is a git-native AI agent framework where the agent IS a git repository. Identity, rules, memory, tools, and [[skills]] are all version-controlled files, enabling "[[agents]] as repos" paradigm.

How does [[Gitagent]] differ from other agent frameworks? Unlike frameworks that scatter configuration across application code, [[Gitagent]] makes the agent itself a git repo:

- Fork an agent → inherit personality, rules, tools
- Branch → create alternate personality versions
- `git log` → see agent's memory evolution
- Diff → track rule changes over time

What is the "[[agents]] as repos" concept? Your agent lives in a git repository with structured files:

- `agent.yaml` — model, tools, runtime config
- `SOUL.md` — personality and identity

- `RULES.md` — behavioral constraints
- `memory/` — git-committed memory with full history
- `tools/` — declarative YAML tool definitions
- `[[skills]]/` — composable [[[[

License

academic-writing (Skill Module)

research-toolkit\skills\academic-writing\SKILL.md

name: academic-writing

description: Academic writing, research methodology, and scholarly communication workflows. Use when writing papers, literature reviews, grant proposals, conducting research, managing citations, preparing for peer review, choosing OA routes under Plan S / 2026 OSTP Nelson Memo, posting preprints, working with persistent identifiers (ORCID, DOI, ROR), assigning CRediT contributor roles, preregistering analyses on OSF / AsPredicted, or disclosing LLM use to journals and funders. Essential for researchers, graduate students, and academics across disciplines.

tags:

Academic writing and research methodology

Systematic approaches for scholarly writing, research design, and academic communication.

Research design fundamentals

Research question development

```
## Crafting a research question

### The FINER criteria
- **F**easible: Can you actually do this research?
- **I**nteresting: Does it matter to the field?
- **N**ovel: Does it add new knowledge?
- **E**thical: Can it be done ethically?
- **R**elevant: Does it address a real problem?

### Question types
| Type | Purpose | Example |
|------|---------|---------|
| Descriptive | Document phenomena | "What are the characteristics of X?" |
| Comparative | Compare groups/conditions | "How does X differ between groups?" |
| Correlational | Examine relationships | "Is there a relationship between X and Y?" |
| Causal | Establish causation | "Does X cause Y?" |
| Exploratory | Generate hypotheses | "What factors might explain X?" |

### Refining your question
Start broad → Narrow progressively

Draft 1: "How does social media affect politics?"
Draft 2: "How does Twitter use affect political polarization?"
Draft 3: "How does exposure to partisan Twitter accounts affect
         political attitude polarization among US adults?"
Draft 4: "Does increased exposure to ideologically homogeneous Twitter
         feeds increase affective polarization among politically
         engaged US adults aged 18-35?"
```

Literature [[review]] strategy

```
## Systematic literature search

### Database selection by field
- **Multidisciplinary**: Web of Science, Scopus, Google Scholar
- **Social Sciences**: JSTOR, ProQuest, SSRN
- **Communication**: Communication Abstracts, ComAbstracts
- **Health**: PubMed, MEDLINE, CINAHL
- **Education**: ERIC
- **Business**: Business Source Complete

### Search strategy template
1. **Identify key concepts** from research question
2. **Generate synonyms** for each concept
3. **Combine with Boolean operators**

Example for: "social media political polarization"

Concept 1: social media
- OR: Twitter, Facebook, Instagram, "social networking sites",
      "online platforms", "digital media"

Concept 2: political
- OR: partisan, ideological, electoral, civic

Concept 3: polarization
- OR: division, extremism, "attitude change", radicalization

Combined search:
(Twitter OR Facebook OR "social media" OR "social networking")
AND (political OR partisan OR ideological)
AND (polarization OR division OR extremism)

### Inclusion/exclusion criteria
Document your criteria:
- Publication date range: [X to present]
- Languages: [English only / multiple]
- Publication types: [Peer-reviewed only / include preprints]
- Geographic scope: [Global / specific countries]
- Methodologies: [All / specific approaches]

### Managing your search
- Save searches to re-run later
- Export results to reference manager
- Track number of results at each stage
- Document date of each search
```

Citation management

```
# Zotero/reference manager integration patterns

# For automated citation workflows
CITATION_STYLES = {
    'apa7': 'American Psychological Association 7th edition',
    'mla9': 'Modern Language Association 9th edition',
    'chicago': 'Chicago Manual of Style 17th edition',
    'harvard': 'Harvard Reference format',
    'ieee': 'IEEE',
    'vancouver': 'Vancouver (medicine)',
}

# BibTeX entry template
BIBTEX_ARTICLE = """
@article{{{citekey},
    author = {{{author}}},
    title = {{{title}}},
    journal = {{{journal}}},
    year = {{{year}}},
    volume = {{{volume}}},
    number = {{{number}}},
    pages = {{{pages}}},
    doi = {{{doi}}}
}}
"""

# Common citation patterns by context
CITATION_CONTEXTS = {
    'introducing_concept': "According to Author (Year), ...",
    'supporting_claim': "Research has shown that X (Author, Year; Author, Year).",
    'contrasting': "While Author (Year) argues X, Author (Year) contends Y.",
    'methodology_reference': "Following the method developed by Author (Year), ...",
    'direct_quote': 'Author (Year) states that "exact quote" (p. X).',
}
```

Paper structure and writing

IMRaD structure (scientific papers)

```
## Standard research paper sections

### Abstract (150-300 words typically)
- Background (1-2 sentences)
- Purpose/objective (1 sentence)
- Methods (2-3 sentences)
- Results (2-3 sentences)
- Conclusions (1-2 sentences)

### Introduction
**Funnel structure:**
1. Broad context - Why does this topic matter?
2. Narrow focus - What's the specific problem?
3. Gap identification - What don't we know?
4. Research question/hypothesis - What will you investigate?
5. Contribution preview - Why does this study matter?

### Literature Review
**Organize thematically, not chronologically:**
1. Theme 1: Key findings, debates, gaps
2. Theme 2: Key findings, debates, gaps
3. Theme 3: Key findings, debates, gaps
4. Synthesis: How themes connect to your study

### Methods
**The reproducibility test:** Could another researcher replicate your study from this section alone?

Include:
- Participants/sample (who, how selected, N)
- Materials/measures (what instruments, their validity)
- Procedure (step-by-step what happened)
- Analysis approach (statistical tests, qualitative methods)
- Ethical considerations (IRB, consent)

### Results
**Report, don't interpret:**
- Present findings systematically
- Use tables/figures for complex data
- Report effect sizes, not just p-values
- Address each research question/hypothesis

### Discussion
**Reverse funnel structure:**
1. Summary of key findings
2. Interpretation in context of literature
3. Theoretical implications
4. Practical implications
5. Limitations
6. Future research directions
7. Conclusion
```

Academic writing style

```
## Writing conventions

### Voice and tense
| Section | Tense | Example |
|---------|-------|---------|
| Abstract | Past (methods/results), Present (conclusions) | "We found... This suggests..." |
| Introduction | Present (established knowledge) | "Research shows..." |
| Methods | Past | "Participants completed..." |
| Results | Past | "Analysis revealed..." |
| Discussion | Present + Past | "These findings indicate... We found..." |

### Hedging language
Appropriate hedging (avoiding overclaiming):
- "This suggests that..." (not "This proves that...")
- "may be related to" (not "causes")
- "Results indicate..." (not "Results demonstrate conclusively...")
- "One possible explanation..." (not "The explanation...")

### Transition words by function
**Addition:** furthermore, moreover, additionally, in addition
**Contrast:** however, nevertheless, conversely, in contrast
**Cause/effect:** therefore, consequently, as a result, thus
**Example:** for instance, specifically, to illustrate
**Sequence:** first, subsequently, finally, meanwhile
**Summary:** in summary, overall, in conclusion

### Paragraphs
Each paragraph should:
1. Begin with a topic sentence
2. Contain one main idea
3. Include supporting evidence
4. Connect to adjacent paragraphs
5. Average 100-200 words (varies by field)
```

Common writing problems

```
## Issues to avoid

### Wordiness
 "It is important to note that the results demonstrate..."
 "Results demonstrate..."

 "In order to investigate..."
 "To investigate..."

 "A total of 50 participants..."
 "Fifty participants..."

### Weak verbs
 "The study was conducted by the researchers"
 "The researchers conducted the study"

 "There was a significant difference found"
 "We found a significant difference"

### Vague language
 "Several studies have shown..."
 "Three studies (Author, Year; Author, Year; Author, Year) showed..."

 "The results were significant"
 "The results were statistically significant (p < .05, d = 0.45)"

### Unnecessary jargon
- Define technical terms on first use
- Use simpler words when equally precise
- Consider your audience's expertise level

### Citation problems
 Citing secondary sources without noting
 "(Author, Year, as cited in Author, Year)"

 String citations without synthesis
 Group citations that make the same point; distinguish those that differ
```

Peer [[review]] and revision

Responding to reviewers

```
## Response letter template

Dear Editor and Reviewers,

Thank you for the thoughtful feedback on our manuscript titled "[Title]"
(Manuscript ID: [Number]). We have carefully considered all comments and
revised the manuscript accordingly. Below, we provide point-by-point
responses to each comment.

---

## Reviewer 1

### Comment 1.1
[Quote or paraphrase the reviewer's comment]

**Response:**
[Your response explaining what you did]

**Changes made:**
[Quote the specific changes with page/line numbers]
"New text here..." (p. X, lines XX-XX)

### Comment 1.2
[Continue for each comment]

---

## Reviewer 2
[Same format]

---

We believe these revisions have substantially strengthened the manuscript
and hope you will find it suitable for publication in [Journal Name].

Sincerely,
[Authors]
```

Handling criticism

```
## Types of reviewer feedback

### Must address
- Methodological concerns
- Missing literature
- Unclear writing
- Logical gaps in argument
- Statistical errors

### Negotiate carefully
- Requests for additional analyses
- Suggestions to restructure
- Disagreements on interpretation

### Pushback appropriately
- Requests outside scope
- Misunderstandings of your argument
- Contradictory advice from reviewers

### Response strategies
**Agreement:** "We thank the reviewer for this suggestion. We have [action]."
**Partial agreement:** "We appreciate this point. While [acknowledge validity], we [explain your approach]. However, we have [partial accommodation]."
**Respectful disagreement:** "We thank the reviewer for raising this issue. We have considered this carefully; however, [explanation]. We hope the reviewer
will find this reasoning persuasive."
```

Grant proposals

[[Proposal]] structure (NSF/NIH style)

```
## Standard grant components

### Specific aims (1 page)
**Opening paragraph:** What's the problem? Why does it matter?
**Gap statement:** What's missing from current understanding?
**Long-term goal:** Your research program vision
**Objective:** What this specific project will accomplish
**Central hypothesis:** Your testable prediction
**Aims:** 2-4 specific, achievable objectives
**Impact statement:** Why funding this matters

### Significance (2-3 pages)
- Importance of the problem
- Gaps in current knowledge
- How your work advances the field
- Potential impact if successful

### Innovation (1 page)
- What's new about your approach?
- Conceptual innovation
- Methodological innovation
- Technical innovation

### Approach (6-12 pages)
For each aim:
- Rationale
- Preliminary data (if any)
- Research design
- Methods
- Expected outcomes
- Potential problems and alternatives
- Timeline

### Broader impacts
- Training opportunities
- Dissemination plans
- Benefits to society
- Diversity and inclusion
```

Budget justification

```
## Budget categories

### Personnel
- PI salary and effort (% time)
- Co-investigators
- Postdocs (salary + benefits)
- Graduate students (stipend + tuition + benefits)
- Undergraduate researchers
- Technical staff

### Equipment
- Items over $5,000 (check sponsor threshold)
- Justify necessity for project

### Supplies
- Lab consumables
- Software licenses
- Participant payments

### Travel
- Conference presentations
- Data collection sites
- Collaborator visits

### Other direct costs
- Publication costs
- Participant incentives
- Transcription services
- Equipment maintenance

### Indirect costs (F&A — facilities and administrative)
- Each institution negotiates an F&A rate with its cognizant federal agency.
- The rate applies to **modified total direct costs (MTDC)**, not raw total
  direct costs — exclude equipment >$5K, tuition, participant support,
  subaward portions over $25K, and a few other categories per the
  institution's negotiated agreement.
- R1 universities commonly negotiate 55-70% of MTDC; smaller institutions
  often run lower. Always confirm the current rate with your sponsored
  programs office before budgeting.
```

Publishing strategy

Journal selection

```
## Evaluating journals

### Quality indicators
- Impact factor (use cautiously)
- Acceptance rate
- Review time
- Time to publication
- Reputation in your field
- Indexing (Web of Science, Scopus)

### Fit indicators
- Scope alignment
- Audience match
- Article type (empirical, theoretical, review)
- Open access route (see below)

### Open access routes
- **Gold OA** — published OA in a fully OA journal; usually requires an
  Article Processing Charge (APC), often $1,000-$3,000+.
- **Green OA** — closed-access journal but author self-archives an
  accepted manuscript in a repository (institutional, arXiv, etc.) under
  the publisher's allowed embargo.
- **Diamond / Platinum OA** — fully OA journal with no APC; supported
  by a sponsor (society, university, consortium). Increasingly common
  in social sciences and humanities.
- **Hybrid** — a closed-access journal that lets individual articles
  go OA for an APC. Note: hybrid is *excluded* from Plan S compliance
  and from many funder OA mandates — verify before paying an APC.
- **Bronze** — free to read but no formal OA license; the publisher
  can revoke access. Treat as not-actually-OA for compliance purposes.

### Funder mandates (US + Europe, as of 2026)
- **OSTP Nelson Memo (US)** — federally-funded research must be made
  immediately publicly accessible upon publication, with no embargo.
  Implementation deadline was **end of 2025**, so as of 2026 this is
  in effect for all major US federal funders (NIH, NSF, DOE, NASA, etc.).
- **Plan S (cOAlition S)** — European funders (Wellcome, UKRI, ANR,
  many others) require immediate Gold or Diamond OA, or Green OA
  with no embargo and a CC-BY license. Hybrid is not compliant.
- Verify the funder's most recent policy before submission — accepted
  routes shift, and some funders cap APCs.

### Red flags (predatory journals)
- Aggressive email solicitation
- Rapid "peer review" (days)
- Unclear editorial board
- Not indexed in major databases
- "Pay to publish" with no clear OA model
- Poor website quality

### Resources for vetting journals + ethics
- **Think. Check. Submit.** (thinkchecksubmit.org) — checklist for legitimacy.
- **DOAJ** (Directory of Open Access Journals) — vetted OA-journal index.
- **Journal Citation Reports** — paid Web of Science product for impact factor.
- **Cabells Predatory Reports** — subscription-based replacement for the
  original Beall's List, which was taken down in 2017 (only stale mirrors
  remain — don't rely on them as a current source of truth).
- **Retraction Watch Database** — searchable record of retractions and
  publisher misconduct; useful for vetting authors and journals.
- **COPE** (Committee on Publication Ethics, publicationethics.org) —
  authoritative guidelines for editors and authors on ethics and integrity.
- **ICMJE** (International Committee of Medical Journal Editors,
  icmje.org/recommendations) — defines authorship criteria, conflict-of-
  interest disclosure, and reporting standards used across many fields.
```

Cover letter

[[[

Required by most major journals and funders. Connect it to your CV, manuscripts, datasets, software, and grant applications.

- **DOI** (doi.org) — Persistent identifier for scholarly objects. Assigned automatically by journals at acceptance; you can also mint DOIs for your own datasets, code, posters, and preprints via Zenodo (zenodo.org) or your institutional repository — useful for citing non-journal outputs.
- **ROR** (ror.org) — Persistent identifier for research organizations, used by funders and publishers to disambiguate institutional names.

Research ethics

Ethical considerations checklist

```
## Before starting research

### Human subjects
- [ ] IRB/ethics board approval obtained
- [ ] Informed consent procedures established
- [ ] Vulnerable populations identified and protected
- [ ] Privacy and confidentiality measures in place
- [ ] Data security plan established
- [ ] Risk/benefit ratio acceptable

### Data management
- [ ] Data management plan created
- [ ] Secure storage arranged
- [ ] Sharing/archiving plans documented
- [ ] Retention period determined
- [ ] Destruction procedures established

### Authorship
- [ ] Contribution criteria discussed (use ICMJE definition: substantial
      contribution + drafting/revising + final approval + accountability)
- [ ] Author order agreed upon
- [ ] All contributors will meet authorship criteria
- [ ] Acknowledgments planned for non-author contributors
- [ ] **CRediT taxonomy** (credit.niso.org) roles assigned for each
      author — most major journals now require or recommend CRediT,
      which has 14 standardized contributor roles (Conceptualization,
      Methodology, Software, Validation, Formal analysis, Investigation,
      Resources, Data curation, Writing — original draft, Writing —
      review & editing, Visualization, Supervision, Project administration,
      Funding acquisition).

### Conflicts of interest
- [ ] Financial conflicts identified
- [ ] Non-financial conflicts identified
- [ ] Disclosure plan established

### Reproducibility
- [ ] Analysis plan pre-registered on **OSF** (osf.io) or **AsPredicted**
      (aspredicted.org) before data collection — distinguishes
      confirmatory from exploratory analyses and protects against
      HARKing. Check whether the target journal accepts **Registered
      Reports** (peer review of the design *before* data collection,
      with in-principle acceptance regardless of results) — this format
      now exists at 350+ journals across psychology, biomedical, and
      social sciences as of 2026.
- [ ] Code will be shared (consider a GitHub release + Zenodo DOI for
      versioned, citable code archives)
- [ ] Data will be shared (FAIR principles; institutional repository,
      Dryad, Figshare, or domain-specific repository as appropriate)
- [ ] Materials will be shared
```

Avoiding research misconduct

```
## Types of misconduct

### Fabrication
- Making up data or results
- Never acceptable under any circumstances

### Falsification
- Manipulating data, equipment, or processes
- Selectively omitting data to change conclusions
- Image manipulation beyond acceptable adjustment

### Plagiarism
- Using others' words without attribution
- Self-plagiarism (reusing own published work without acknowledgment)
- Paraphrasing too closely

### Other questionable practices
- P-hacking (running multiple analyses until significant)
- HARKing (hypothesizing after results known)
- Salami slicing (fragmenting one study into many papers)
- Gift/ghost authorship
- Selective reporting of results
```

AI / LLM use in academic writing

LLM-assisted writing is the defining 2024-2026 ethics issue across academic publishing. Every major journal, funder, and ethics body has issued policy in this window — and policies are still tightening. Treat anything below as the floor, not the ceiling: read your target journal's current submission guidelines AND your funder's most recent policy before submitting.

What's universally prohibited (as of 2026)

- **LLM authorship.** ICMJE, COPE, Nature, Science, NEJM, Cell, JAMA, Lancet, and the major university presses all explicitly prohibit listing an LLM (ChatGPT, Claude, Gemini, etc.) as a co-author. LLMs cannot meet the accountability and approval criteria authorship requires. Use the acknowledgments section or a methods/disclosure statement instead — never the author byline.
- **Generating fabricated citations.** LLMs are well-documented to produce plausible-looking but fabricated DOIs, page numbers, and even author/title combinations that don't exist. Every citation in a manuscript must be verified against the source — the LLM-induced fake-citation rate has been a top retraction trigger since 2023.
- **Generating data, results, or images.** Synthesizing experimental data, fabricating figures, or using generative AI to "fill in" results that weren't actually obtained is research misconduct under COPE's definition.
- **Undisclosed substantial use.** Most journals require disclosure of any LLM use beyond trivial spell-check/grammar assistance. Failing to disclose meaningful use can be grounds for retraction.

What's typically permitted (with disclosure)

- Idea brainstorming and outlining
- Language polishing and grammar correction
- Translation of your own writing
- Code generation for analyses (with explicit testing)
- Summarization of your own notes or transcripts
- Generating boilerplate sections (cover letters, IRB language) that you then [[fact-check]] and own

Disclosure language

((((((

> *During the preparation of this work the author(s) used [tool name, e.g., ChatGPT-5, Claude Opus 4.7, Gemini 3 Pro] in order to [specific use, e.g., language polishing of the introduction; drafting code for the cluster analysis in section 3.2]. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the [[[[publication]]]].*

Adjust to match the target journal's exact required wording — Elsevier, Springer Nature, Wiley, Taylor & Francis, IEEE, and ACM each publish their own preferred language.

Disclosure checklist before submission

- [ ] Read the target journal's AI/LLM policy in the current submission guidelines (policies updated multiple times per year).
- [ ] Read your funder's policy — some (Wellcome, NSF) have stricter rules than the journal.
- [ ] Add a disclosure statement using the journal's preferred language.
- [ ] Verify every citation manually — do not trust LLM output for any DOI, author list, page range, or quoted passage.
- [ ] Verify every numerical claim, date, and named fact.
- [ ] Re-read the full manuscript to ensure your voice and argument structure dominate, not the LLM's.
- [ ] If the LLM was used for code or analysis, re-run with seeds and verify all reported numbers reproduce.

Detection tools (limited reliability)

GPTZero, Turnitin AI Detection, Originality.ai, and similar tools have documented high false-positive rates against non-native-English writers and against legitimately human-written technical text — and high false-negative rates against current models with light editing. **Pangram Labs** (pangram.com) has published more rigorous benchmarks and tends to outperform the older detectors at roughly equivalent settings, but is still not a definitive arbiter and shares the same fundamental limits when authors edit LLM output substantially. Detection output should never be the sole basis for a misconduct finding; journals that rely on it as a gatekeeping signal are increasingly walking that back. Treat detector results as a flag for follow-up discussion with the author, not as evidence on their own.

Productivity and workflow

Writing routine

```
## Sustainable academic writing

### Daily writing practice
- Write at same time daily (habit formation)
- Start with minimum viable goal (e.g., 30 minutes)
- Track progress (word counts, time)
- Protect writing time from meetings/email

### Managing large projects
1. Break into smallest possible tasks
2. Set deadlines for each component
3. Schedule regular writing blocks
4. Build in buffer time
5. Get feedback early and often

### Overcoming blocks
- Start with easiest section
- Write a "shitty first draft" (Anne Lamott)
- Use placeholders for citations/data
- Talk through ideas with colleague
- Change environment
- Return to outline/structure
```

Tools for academics

Purpose	Tools
Reference management	Zotero (recommended; open source, syncs across devices), Paperpile (web-first, Google Docs [[integration]]), EndNote, Citavi. **Note on Mendeley:** Elsevier announced an end to Mendeley Desktop downloads in September 2022, then later restored availability. Mendeley Desktop remains legacy software with no further feature development; the Mendeley Reference Manager (web/cloud) is the actively maintained product. New users are better served by Zotero or Paperpile.
Writing	Scrivener, Overleaf (LaTeX collaboration), Word, Typst (newer LaTeX alternative)
Collaboration	Google Docs, Overleaf, Hedgedoc
Version control	Git / GitHub / GitLab (true version control). **Word's Track Changes is collaborative editing, not version control** — it lacks branching, merging, history rewrite, and atomic commits.
Task management	Todoist, Notion, Trello
Focus	Forest, Freedom, Cold Turkey
Analysis	R, Python, SPSS, Stata, NVivo, Julia
Visualization	R/ggplot2, Python/matplotlib + plotly, Observable, Tableau
LLM-assisted writing	See "AI / LLM use in academic writing" section above for disclosure requirements before using any of these.

accessibility-compliance (Skill Module)

dev-toolkit\skills\accessibility-compliance\SKILL.md

name: accessibility-compliance

description: Web accessibility patterns for news sites, journalism tools, and academic platforms. Use when building accessible interfaces, auditing existing sites for WCAG compliance, writing alt text for news images, creating accessible data visualizations, or ensuring content reaches all readers including those using assistive technologies. Essential for newsroom developers and anyone publishing web content.

tags:

Accessibility compliance

Practical accessibility patterns for journalism and academic web publishing.

When to activate

- Building or auditing news websites
- Writing alt text for article images
- Creating accessible data visualizations
- Developing tools that journalists use
- Ensuring multimedia content is accessible
- Meeting legal accessibility requirements
- Publishing academic content online

WCAG essentials for news sites

WCAG 2.2 became a W3C Recommendation in October 2023 and is backwards-compatible with 2.1. Targeting 2.2 AA is the right default for new work; 2.1 AA remains the floor in most legal regimes (see "Legal requirements" below).

WCAG 2.2 added nine criteria over 2.1. Most relevant for news sites: **2.5.8 Target Size (Minimum) AA** — interactive targets at least 24×24 CSS pixels; **2.4.11 Focus Not Obscured (Minimum) AA** — focused element not fully hidden by sticky headers / chat widgets; **3.3.8 Accessible [[Authentication]] AA** — no cognitive function tests (e.g., transcribing distorted text) without an alternative; **3.3.7 Redundant Entry A** — don't ask users to re-enter the same data within a session.

The four principles (POUR)

```
## WCAG 2.2 AA checklist (journalism focus)

### Perceivable
- [ ] Images have meaningful alt text
- [ ] Videos have captions
- [ ] Audio has transcripts
- [ ] Color isn't the only way to convey info
- [ ] Text can be resized to 200% without breaking

### Operable
- [ ] All functions work with keyboard only
- [ ] No keyboard traps
- [ ] Skip links to main content
- [ ] Page titles describe content
- [ ] Focus visible on all interactive elements

### Understandable
- [ ] Language is declared in HTML
- [ ] Navigation is consistent
- [ ] Error messages are clear
- [ ] Labels describe form fields

### Robust
- [ ] Valid HTML
- [ ] ARIA used correctly (or not at all)
- [ ] Works with screen readers
- [ ] Doesn't break with zoom/text resize
```

Image accessibility

Alt text for journalism

```
## Alt text decision tree

### News photos
- **WHO** is in the image (if identifiable and relevant)
- **WHAT** is happening (the action or situation)
- **WHERE** (if location matters to story)
- **Don't**: Repeat caption text verbatim

### Examples

PHOTO: Protesters holding signs outside courthouse

BAD: "Protesters"
BAD: "Image of protest" (redundant "image of")
GOOD: "Approximately 50 protesters hold signs reading 'Justice Now' outside the federal courthouse in downtown Seattle"

PHOTO: Headshot of interview subject

BAD: "Photo"
GOOD: "Dr. Sarah Chen, epidemiologist at Johns Hopkins"

PHOTO: Chart embedded as image

BAD: "Chart showing data"
GOOD: "Bar chart showing unemployment rising from 3.5% to 8.2% between March and June 2020. Full data in table below."
```

Alt text for AI-generated images

Newsroom transparency policies generally require disclosing that an image is AI-generated; that disclosure belongs in the caption AND the alt text, because screen-reader users see only the alt text. Describe the visual content first, then note the provenance. Don't lead with "AI-generated image of..." — describe the subject the way you would for any photo, then add the AI source.

GOOD: "Illustration of a flooded downtown street with abandoned cars, water reaching first-floor windows. AI-generated by [tool] for editorial use."

GOOD (decorative AI illustration): "Decorative AI-generated abstract pattern in blue and orange." — but consider whether the image should have empty alt (`alt=""`) since it carries no information.

For AI-generated images of real people or events, AP and most newsroom guidelines treat the image as a manipulation, not a photograph — disclosure in alt text is required, not optional.

Alt text Python helper

```
def generate_alt_text_prompt(context: dict) -> str:
    """Generate prompt for AI alt text assistance."""
    return f"""
    Write alt text for a news image.

    Story context: {context.get('headline', 'Unknown')}
    Image type: {context.get('image_type', 'photo')}
    Caption (if any): {context.get('caption', 'None')}

    Guidelines:
    - Be concise (under 125 characters if possible)
    - Don't start with "Image of" or "Photo of"
    - Include relevant details for story context
    - Don't duplicate caption exactly
    - Describe what's visually important

    If this is decorative only, respond: ""
    """

def is_decorative(image_context: str) -> bool:
    """Check if image is purely decorative (empty alt appropriate)."""
    decorative_indicators = [
        'decorative',
        'separator',
        'background',
        'spacer',
        'border'
    ]
    return any(ind in image_context.lower() for ind in decorative_indicators)
```

Accessible data visualization

Chart accessibility checklist

```
## Making charts accessible

### Essential elements
- [ ] Text alternative describing the key insight
- [ ] Data table available (visible or linked)
- [ ] Colors have sufficient contrast
- [ ] Patterns/textures supplement color coding
- [ ] Labels directly on chart (not legend-only)
- [ ] Title describes what chart shows

### Interactive charts
- [ ] Keyboard navigable
- [ ] Focus indicators visible
- [ ] Screen reader announces data points
- [ ] Tooltips accessible via keyboard
- [ ] Zooming doesn't break layout
```

Accessible chart component

```
<!-- Accessible chart pattern -->
<figure role="figure" aria-labelledby="chart-title" aria-describedby="chart-desc">
  <figcaption>
    <h3 id="chart-title">Unemployment Rate 2020-2024</h3>
    <p id="chart-desc">
      Line chart showing unemployment starting at 3.5% in January 2020,
      spiking to 14.7% in April 2020, and gradually declining to 3.9% by 2024.
    </p>
  </figcaption>

  <!-- The chart itself -->
  <div id="chart" role="img" aria-label="Interactive line chart. Data table available below.">
    <!-- Chart renders here -->
  </div>

  <!-- Always provide data table -->
  <details>
    <summary>View data table</summary>
    <table>
      <caption>Monthly unemployment rate data</caption>
      <thead>
        <tr>
          <th scope="col">Month</th>
          <th scope="col">Unemployment Rate (%)</th>
        </tr>
      </thead>
      <tbody>
        <tr><td>Jan 2020</td><td>3.5</td></tr>
        <tr><td>Apr 2020</td><td>14.7</td></tr>
        <!-- etc -->
      </tbody>
    </table>
  </details>
</figure>
```

Color-blind safe palettes

```
# Safe color palettes for data visualization
COLOR_PALETTES = {
    # Paul Tol's color schemes - widely tested for accessibility
    'bright': [
        '#4477AA',  # Blue
        '#EE6677',  # Red
        '#228833',  # Green
        '#CCBB44',  # Yellow
        '#66CCEE',  # Cyan
        '#AA3377',  # Purple
        '#BBBBBB',  # Grey
    ],

    # Categorical (safe for most color blindness)
    'categorical': [
        '#332288',  # Indigo
        '#88CCEE',  # Cyan
        '#44AA99',  # Teal
        '#117733',  # Green
        '#999933',  # Olive
        '#DDCC77',  # Sand
        '#CC6677',  # Rose
        '#882255',  # Wine
    ],

    # Sequential (single hue)
    'sequential_blue': [
        '#f7fbff',
        '#deebf7',
        '#c6dbef',
        '#9ecae1',
        '#6baed6',
        '#4292c6',
        '#2171b5',
        '#084594',
    ],

    # Diverging (for data with meaningful midpoint)
    'diverging': [
        '#d73027',  # Red (negative)
        '#f46d43',
        '#fdae61',
        '#fee08b',
        '#ffffbf',  # Neutral
        '#d9ef8b',
        '#a6d96a',
        '#66bd63',
        '#1a9850',  # Green (positive)
    ]
}

def validate_contrast(color1: str, color2: str) -> float:
    """Calculate WCAG contrast ratio between two colors."""
    def hex_to_rgb(hex_color):
        hex_color = hex_color.lstrip('#')
        return tuple(int(hex_color[i:i+2], 16) for i in (0, 2, 4))

    def relative_luminance(rgb):
        r, g, b = [x / 255.0 for x in rgb]
        r = r / 12.92 if r <= 0.03928 else ((r + 0.055) / 1.055) ** 2.4
        g = g / 12.92 if g <= 0.03928 else ((g + 0.055) / 1.055) ** 2.4
        b = b / 12.92 if b <= 0.03928 else ((b + 0.055) / 1.055) ** 2.4
        return 0.2126 * r + 0.7152 * g + 0.0722 * b

    l1 = relative_luminance(hex_to_rgb(color1))
    l2 = relative_luminance(hex_to_rgb(color2))

    lighter = max(l1, l2)
    darker = min(l1, l2)

    return (lighter + 0.05) / (darker + 0.05)

# WCAG requirements:
# Normal text: 4.5:1 minimum (AA), 7:1 enhanced (AAA)
```

```
# Large text (18pt+): 3:1 minimum (AA), 4.5:1 enhanced (AAA)
# UI components: 3:1 minimum
```

Video and audio accessibility

Caption requirements

```
## Video caption checklist

### Quality standards
- [ ] 99%+ accuracy
- [ ] Synchronized with audio (within 1 second)
- [ ] Speaker identification for multiple speakers
- [ ] Sound effects described [applause] [music]
- [ ] Non-speech audio described when relevant

### Technical requirements
- [ ] Captions available in player controls
- [ ] Can be toggled on/off
- [ ] Styling doesn't overlap video content
- [ ] Readable font size and contrast

### Caption format (SRT example)
1
00:00:01,000 --> 00:00:04,500
[NEWS ANCHOR] Good evening. Breaking news tonight
from downtown where protesters have gathered.

2
00:00:04,600 --> 00:00:08,200
We're going live to reporter Jane Smith
at the scene. Jane?
```

Audio description for video

```
## When audio description is needed

### Required
- [ ] Key visual information not in dialogue
- [ ] Actions crucial to understanding story
- [ ] Text on screen not read aloud
- [ ] Identifying information for speakers

### Example script

ORIGINAL VIDEO: [Reporter stands in front of burning building]
DIALOGUE: "The fire started around 3am..."

AUDIO DESCRIPTION VERSION:
[A reporter in a red jacket stands before a five-story
apartment building engulfed in flames. Fire trucks
visible in background]
DIALOGUE: "The fire started around 3am..."
```

Keyboard accessibility

Focus management patterns

```
// Skip link implementation
document.addEventListener('DOMContentLoaded', () => {
  const skipLink = document.querySelector('.skip-link');
  const mainContent = document.querySelector('main');

  skipLink.addEventListener('click', (e) => {
    e.preventDefault();
    mainContent.setAttribute('tabindex', '-1');
    mainContent.focus();
  });
});

// Keyboard trap prevention in modals
function createAccessibleModal(modalElement) {
  const focusableElements = modalElement.querySelectorAll(
    'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
  );
  const firstFocusable = focusableElements[0];
  const lastFocusable = focusableElements[focusableElements.length - 1];

  modalElement.addEventListener('keydown', (e) => {
    if (e.key === 'Tab') {
      if (e.shiftKey && document.activeElement === firstFocusable) {
        e.preventDefault();
        lastFocusable.focus();
      } else if (!e.shiftKey && document.activeElement === lastFocusable) {
        e.preventDefault();
        firstFocusable.focus();
      }
    }

    if (e.key === 'Escape') {
      closeModal();
    }
  });
}

// Focus indicator styles (never remove outlines without replacement)
/*
:focus {
  outline: 2px solid #005fcc;
  outline-offset: 2px;
}

:focus:not(:focus-visible) {
  outline: none;  // Remove for mouse users
}

:focus-visible {
  outline: 2px solid #005fcc;  // Keep for keyboard users
  outline-offset: 2px;
}
*/
```

Respect motion preferences

Users with vestibular disorders, attention conditions, or migraines can disable system-level animation. Honor that signal — don't ship CSS or JS animations that ignore it.

```
/* Reduce or remove non-essential motion when the OS asks */
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

```
// Gate JS-driven animations on the same signal
const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches;

if (!prefersReducedMotion) {
  // Run your scroll-driven story animations, parallax, autoplay carousels, etc.
}
```

Essential motion that conveys information (loading spinners, progress bars) can stay. Decorative motion (parallax scroll, animated heroes, autoplay video) must obey the preference.

Forms and error handling

Accessible form patterns

```
<!-- Accessible form field -->
<div class="form-group">
  <label for="email">
    Email address
    <span class="required" aria-hidden="true">*</span>
    <span class="visually-hidden">(required)</span>
  </label>
  <input
    type="email"
    id="email"
    name="email"
    required
    aria-describedby="email-hint email-error"
    aria-invalid="false"
  >
  <p id="email-hint" class="hint">
    We'll use this to send you the newsletter.
  </p>
  <p id="email-error" class="error" role="alert" hidden>
    Please enter a valid email address.
  </p>
</div>

<style>
  /* Visually hidden but accessible to screen readers */
  .visually-hidden {
    position: absolute;
    width: 1px;
    height: 1px;
    padding: 0;
    margin: -1px;
    overflow: hidden;
    clip: rect(0, 0, 0, 0);
    white-space: nowrap;
    border: 0;
  }
</style>
```

Error message patterns

```
function showError(inputElement, message) {
  const errorElement = document.getElementById(
    inputElement.getAttribute('aria-describedby').split(' ').find(id =>
      id.includes('error')
    )
  );

  inputElement.setAttribute('aria-invalid', 'true');
  errorElement.textContent = message;
  errorElement.hidden = false;

  // Announce error to screen readers
  errorElement.setAttribute('role', 'alert');
}

function clearError(inputElement) {
  const errorId = inputElement.getAttribute('aria-describedby')
    .split(' ')
    .find(id => id.includes('error'));
  const errorElement = document.getElementById(errorId);

  inputElement.setAttribute('aria-invalid', 'false');
  errorElement.hidden = true;
  errorElement.removeAttribute('role');
}
```

Testing tools

Automated testing

```
# Accessibility audit with axe-core (via Playwright)
from playwright.sync_api import sync_playwright

def run_accessibility_audit(url: str) -> dict:
    """Run automated accessibility tests."""
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
        page.goto(url)

        # Inject axe-core (4.10.x covers WCAG 2.0/2.1/2.2 at A, AA, AAA)
        page.add_script_tag(
            url='https://cdnjs.cloudflare.com/ajax/libs/axe-core/4.10.3/axe.min.js'
        )

        # Run audit
        results = page.evaluate('''
            async () => {
                return await axe.run();
            }
        ''')

        browser.close()

        return {
            'violations': results['violations'],
            'passes': len(results['passes']),
            'incomplete': results['incomplete'],
            'url': url
        }

def format_violations(results: dict) -> str:
    """Format violations for review."""
    output = []
    for v in results['violations']:
        output.append(f"\n## {v['id']}: {v['description']}")
        output.append(f"Impact: {v['impact']}")
        output.append(f"WCAG: {', '.join(v.get('tags', []))}")
        for node in v['nodes'][:3]:  # First 3 examples
            output.append(f"  - {node['html'][:100]}")
    return '\n'.join(output)
```

Manual testing checklist

```
## Manual accessibility tests

### Keyboard navigation
- [ ] Tab through entire page
- [ ] Can reach all interactive elements
- [ ] Focus order makes sense
- [ ] No keyboard traps
- [ ] Skip link works

### Screen reader testing
- [ ] Headings announce in logical order
- [ ] Images have meaningful descriptions
- [ ] Form labels announce correctly
- [ ] Error messages announced
- [ ] Dynamic content updates announced

### Zoom testing
- [ ] 200% zoom, no horizontal scrolling
- [ ] 400% zoom, content still usable
- [ ] Text spacing adjustments don't break layout

### Color and contrast
- [ ] Works in grayscale
- [ ] Links distinguishable from text
- [ ] Error states not color-only
- [ ] Contrast checker passes (4.5:1 minimum)
```

Legal requirements

```
## Accessibility law summary (current as of 2026-05)

### United States
- **Section 508**: Federal agencies and federal contractors must meet
  WCAG 2.0 AA (Revised 508 Standards, 2017). Update to a newer WCAG
  baseline is anticipated but not finalized.
- **ADA Title II (DOJ final rule, April 2024)**: State and local
  government websites and mobile apps must conform to WCAG 2.1 AA.
  Compliance dates were extended in the April 2026 Interim Final Rule:
  - Population 50,000+: April 26, 2027 (was April 24, 2026)
  - Population under 50,000 / special districts: April 26, 2028
    (was April 26, 2027)
- **HHS Section 504 rule (2024)**: Recipients of HHS funding must meet
  WCAG 2.1 AA on web/mobile. Large recipients: May 11, 2026; small
  recipients: May 10, 2027. Independent of the DOJ extension above.
- **ADA Title III**: Public accommodations (private-sector sites)
  remain a litigation target. No federal technical standard, but
  WCAG 2.1 AA is the de facto floor courts have applied.

### European Union
- **European Accessibility Act (EAA)**: In force June 28, 2025 for
  new products and services. Covers e-commerce, banking, e-readers,
  ticketing, and a broader range of digital services than prior rules.
- **EN 301 549**: Technical standard referenced by member-state
  implementations; aligns with WCAG 2.1 AA (2.2 alignment in progress).

### Best practice
WCAG 2.2 AA for new work; WCAG 2.1 AA for legal-floor compliance with
DOJ Title II, HHS, and most EU implementations. Meeting 2.2 implies 2.1
(backwards compatible).
```

Related [[skills]]

- **zero-build-frontend** - Build accessible static sites
- **data-journalism** - Create accessible visualizations
- **web-scraping** - Ensure scraped content preserves accessibility

---

[[Skill]] metadata

Field	Value
Version	1.1.0
Created	2025-12-26
Last currency sweep	2026-05-08 (WCAG 2.2, DOJ Title II IFR, EAA, axe-core 4.10.3)
Domain	Development, Publishing
Complexity	Intermediate

acquiring-disk-image-with-dd-and-dcfldd (Skill Module)

acquiring-disk-image-with-dd-and-dcfldd\SKILL.md

name: acquiring-disk-image-with-dd-and-dcfldd
description: Forensic disk image acquisition agent using dd and dcfldd with hash verification. Use when the user asks to acquire disk image with dd and dcfldd.
version: 1.0.0
tags:
source: acquiring-disk-image-with-dd-and-dcfldd

Acquiring Disk Image with dd and dcfldd

Role: Forensic acquisition
Audience: DFIR analysts and incident responders
Primary tools: dcfldd, dd / dcfldd, sha256sum, blockdev, lsblk, hdparm

When to use

- The user wants to **acquire** disk image with dd and dcfldd.
- Capability: Forensic disk image acquisition agent using dd and dcfldd with hash verification.
- Goal: collect evidence in a forensically sound manner with integrity verification.
- Trigger keywords: `acquiring, disk, image, dcfldd`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Enumerate connected block devices
2. Verify a device is set to read-only mode
3. Enable software write-blocking on the target device
4. Compute the SHA-256 or MD5 hash of a file or device
5. Acquire a forensic image using dd with error handling
6. Acquire a forensic image using dcfldd with built-in hashing
7. Verify image integrity by comparing hashes of source and acquired image
8. Generate a forensic acquisition report

Tools & resources

- dcfldd
- dd / dcfldd
- sha256sum
- blockdev
- lsblk
- hdparm

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `list_block_devices` — Enumerate connected block devices.
- `check_write_protection` — Verify a device is set to read-only mode.
- `enable_write_protection` — Enable software write-blocking on the target device.
- `compute_hash` — Compute the SHA-256 or MD5 hash of a file or device.
- `acquire_with_dd` — Acquire a forensic image using dd with error handling.
- `acquire_with_dcfldd` — Acquire a forensic image using dcfldd with built-in hashing.
- `verify_image` — Verify image integrity by comparing hashes of source and acquired image.
- `generate_report` — Generate a forensic acquisition report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `acquiring-disk-image-with-dd-and-dcfldd`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

ai-writing-detox (Skill Module)

journalism-core\skills\ai-writing-detox\SKILL.md

name: ai-writing-detox

description: Eliminate AI-generated writing patterns that erode reader trust. Activate when writing articles, documentation, press releases, or any content where AI patterns would undermine credibility. For journalists using AI assistance who need human-sounding output.

tags:

AI writing detox

Good writing is invisible. If readers notice the writing style, it's distracting from the content. AI patterns are noticeable—they break trust.

When to use

- Writing or editing any journalism content
- Creating press releases or media advisories
- Drafting documentation or reports
- Writing social media posts
- Reviewing any AI-assisted text before [[[[publication]]]]

Banned words (delete or replace)

Word	Why it's bad	Alternative
delve	AI signature word	explore, examine, look at
realm	Pretentious	area, field
tapestry	Purple prose	(delete entirely)
landscape	Corporate speak	situation, environment
leverage	Jargon	use
utilize	Jargon	use
robust	Meaningless	strong, reliable
seamless	Almost always false	smooth, easy
comprehensive	Rarely necessary	full, complete
cutting-edge	Cliché	modern, new
holistic	Corporate	complete, full
synergy	Corporate	(delete, rewrite)
paradigm	Overused	model, approach
empower	Vague	(be specific about what capability)
innovative	Empty praise	(describe what's new)
transformative	Hyperbolic	changed, improved
sophisticated	Vague filler	advanced, or describe what's complex
leveraging	Corporate jargon	using
ecosystem	Overused metaphor	system, environment
rich	(as modifier)	(delete or be specific)
over (for quantity)	AP rule	more than

Banned phrases

Throat-clearing (delete entirely)

- "It's important to note that..."
- "In today's [X] landscape..."

- "Let's dive/delve into..."
- "Without further ado..."
- "In this article, we will..."
- "It's worth mentioning that..."
- "It goes without saying..."
- "As we all know..."

Empty hedges

- "To be fair..." / "To be honest..."
- "At the end of the day..."
- "When it comes to..."
- "In terms of..."
- "With respect to..."

AI enthusiasm

- "This is a game-changer"
- "...and that's a good thing!"
- "Here's the thing:"
- "...and that's okay!"
- "...and I'm here for it"

Corporate buzzwords

- "Moving forward..."
- "Going forward..."
- "At this point in time..."
- "Due to the fact that..."
- "In order to..."

Banned structures

Don't start sentences with:

- "So," or "Well,"
- "Now," (when not about time)
- "Look," or "Listen,"
- "Basically," or "Essentially,"

Don't end sentences with:

- "...right?"
- "...you know?"
- "...if you will"

The substitution table

If you wrote	Write instead
utilize	use
facilitate	help
implement	build, add, create
leverage	use
functionality	feature
methodology	method
in order to	to
due to the fact that	because
at this point in time	now
a large number of	many
in the [[[[event]]]] that	if
prior to	before
subsequent to	after
in close proximity to	near
has the ability to	can

Case sensitivity

Always use sentence case for headings, not title case.

Wrong	Right
Getting Started With Your Project	Getting started with your project
How To Use The Tool	How to use the tool
Best Practices For Journalism	Best practices for journalism

Journalism-specific patterns to avoid

"Not just X—it's Y" pattern

Avoid: "This isn't just a news story—it's a wake-up call"
Avoid: "This wasn't just a press conference—it was a turning point"

Do instead: State the thing directly without dramatic framing

"Fundamentally transforms" pattern

Avoid: "This fundamentally transforms how newsrooms operate"
Avoid: "This represents a fundamental shift in journalism"

Do instead: Describe the actual change without hyperbole

Inflated claims in reporting

Avoid: "A bombshell revelation"
Avoid: "A major milestone"
Avoid: "A groundbreaking investigation"

Do instead: Let the facts speak. Readers judge significance.

Empty transitions

Avoid: "With that in mind..."
Avoid: "Building on this foundation..."
Avoid: "Taking this a step further..."

Do instead: Just make the next point

"It's not about X, it's about Y" pattern

Avoid: "It's not about the technology, it's about the people"
Avoid: "This isn't about clicks. It's about trust."

Do instead: State the actual point without rhetorical [[setup]]. The X-not-Y construction is the dominant 2025-2026 ChatGPT/Claude rhetorical signature.

Em-dash overuse between independent clauses

Avoid: "The vote was close — only three council members opposed it — but the measure passed."
Avoid: "Reporters arrived early — the press conference was already underway."

Do instead: Use commas, periods, or semicolons. Em-dashes are fine in moderation; reflexive use across every paragraph is a tell.

"The reality is..." / "The truth is..." openers

Avoid: "The reality is, most newsrooms can't afford this."
Avoid: "The truth is, readers don't trust anonymous sources."

Do instead: Just state the claim. The opener adds words and signals AI-generated framing.

Tricolon abuse (reflexive three-item lists)

Avoid: "Clear, concise, and compelling reporting."
Avoid: "Trust is built through accuracy, transparency, and consistency."

Do instead: Use one or two specific points. Three-item parallel lists are an AI rhythm pattern that reads as filler when used reflexively.

Before and after examples

Example 1: General prose

AI slop:

> *In today's rapidly evolving media landscape, it's crucial to understand the multifaceted nature of AI tools. Let's delve into how these robust solutions can help journalists leverage cutting-edge technology.*

Human writing:

> *AI tools do three things well: drafting, research, and analysis. Here's when to use each.*

Example 2: News lede

AI slop:

> *The city council meeting was a comprehensive and transformative discussion that fundamentally addressed key issues affecting residents in a meaningful way.*

Human writing:

> *The city council voted 5-2 Tuesday to raise property taxes by 3 percent.*

Example 3: Feature intro

AI slop:

> *In the rich tapestry of American journalism, few stories are as compelling as the one about to unfold. This isn't just a profile—it's a journey into the heart of what makes local news truly matter.*

Human writing:

> *Maria Rodriguez has published a newspaper for 47 years. She's never missed an issue.*

Example 4: Analysis piece

AI slop:

> *Moving forward, stakeholders must leverage innovative approaches to navigate the challenging landscape of digital transformation in the journalism ecosystem.*

Human writing:

> *Local papers that want to survive need to find digital revenue. Most haven't.*

The verbal tic test

Read your text aloud. Does it sound like:
- A TED talk introduction? **Rewrite.**
- A LinkedIn post? **Rewrite.**

- A press release? **Rewrite.**
- Corporate communications? **Rewrite.**
- How you'd explain it to a colleague? **Keep it.**

Quick self-check before publishing

Search your text for:
1. "delve" → delete or replace
2. "landscape" → delete or replace
3. "crucial" → is it actually crucial?
4. "robust" → what does this mean here?
5. "leverage" → just say "use"
6. "comprehensive" → is this word doing any work?
7. Starting "So," → delete

Found any? Your writing needs another pass.

Red flags in technical/analysis writing

1. **Lists of near-synonyms**: "comprehensive, sophisticated, and robust" (pick one or none)
2. **Excessive hedging**: "may potentially be able to possibly..."
3. **Noun stacking**: "production-ready deployment system infrastructure"
4. **Passive voice hiding agency**: "It was determined that..." (by whom?)
5. **Circular definitions**: "The system enables users to use the functionality"

The ultimate test

Before accepting any AI-assisted text, ask:

1. Can I delete this word/phrase without losing meaning? → Delete it
2. Is this the simplest way to say this? → Simplify
3. Would I say this out loud to a colleague? → If not, rewrite
4. Does this add information or just sound impressive? → If the latter, cut it

If readers think "AI wrote this," you've already lost their trust.

Good writing is invisible. If readers notice the writing, it's getting in the way of the content.

See also: hooks/[[ai-slop-detector]].md for the automated companion to this [[skill]].

analyzing-active-directory-acl-abuse (Skill Module)

analyzing-active-directory-acl-abuse\SKILL.md

name: analyzing-active-directory-acl-abuse
description: Active Directory ACL abuse detection using ldap3 to find dangerous permissions. Use when the user asks to analyze active directory acl abuse.
version: 1.0.0
tags:
source: analyzing-active-directory-acl-abuse

Analyzing Active Directory ACL Abuse

Role: Deep analysis
Audience: analysts and investigators

When to use

- The user wants to **analyze** active directory acl abuse.
- Capability: Active Directory ACL abuse detection using ldap3 to find dangerous permissions.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, active, directory, abuse`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Ingest and validate inputs for active directory acl abuse
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `is_admin_sid`
- `parse_sid`
- `parse_acl`

- `resolve_sid`
- `get_domain_sid`
- `analyze_acls`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-active-directory-acl-abuse`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-android-malware-with-apktool (Skill Module)

analyzing-android-malware-with-apktool\SKILL.md

name: analyzing-android-malware-with-apktool
description: Agent for static analysis of Android APK malware using androguard. Use when the user asks to analyze android malware with apktool.
version: 1.0.0
tags:
source: analyzing-android-malware-with-apktool

Analyzing Android Malware with Apktool

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Apktool, Androguard API

When to use

- The user wants to **analyze** android malware with apktool.
- Capability: Agent for static analysis of Android APK malware using androguard.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, android, malware, apktool`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze requested permissions and flag dangerous ones
2. Extract manifest components: activities, services, receivers, providers
3. Scan DEX analysis for suspicious API calls
4. Extract suspicious strings: URLs, IPs, base64 patterns
5. Detect code obfuscation indicators
6. Run comprehensive APK malware analysis

Tools & resources

- Apktool
- Androguard API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_permissions` — Analyze requested permissions and flag dangerous ones.
- `analyze_manifest` — Extract manifest components: activities, services, receivers, providers.
- `scan_suspicious_apis` — Scan DEX analysis for suspicious API calls.
- `extract_strings` — Extract suspicious strings: URLs, IPs, base64 patterns.
- `detect_obfuscation` — Detect code obfuscation indicators.
- `full_analysis` — Run comprehensive APK malware analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-android-malware-with-apktool`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-api-gateway-access-logs (Skill Module)

analyzing-api-gateway-access-logs\SKILL.md

name: analyzing-api-gateway-access-logs
description: Agent for analyzing API Gateway access logs for security threats. Use when the user asks to analyze api gateway access logs.
version: 1.0.0
tags:
source: analyzing-api-gateway-access-logs

Analyzing API Gateway Access Logs

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Pandas Log Analysis

When to use

- The user wants to **analyze** api gateway access logs.
- Capability: Agent for analyzing API Gateway access logs for security threats.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, gateway, access, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load API gateway logs from JSON lines or CSV
2. Detect Broken Object Level Authorization (BOLA/IDOR) attacks
3. Detect credential scanning via 401/403 response surges
4. Detect SQL/NoSQL injection attempts in request parameters
5. Detect rate limit bypass attempts
6. Detect unusual HTTP methods on typically read-only endpoints

Tools & resources

- Pandas Log Analysis

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_api_logs` — Load API gateway logs from JSON lines or CSV.
- `detect_bola_attacks` — Detect Broken Object Level Authorization (BOLA/IDOR) attacks.
- `detect_auth_scanning` — Detect credential scanning via 401/403 response surges.
- `detect_injection_attempts` — Detect SQL/NoSQL injection attempts in request parameters.
- `detect_rate_limit_bypass` — Detect rate limit bypass attempts.
- `detect_unusual_methods` — Detect unusual HTTP methods on typically read-only endpoints.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-api-gateway-access-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-apt-group-with-mitre-navigator (Skill Module)

`analyzing-apt-group-with-mitre-navigator\SKILL.md`

name: analyzing-apt-group-with-mitre-navigator
description: APT group analysis agent using MITRE ATT&CK Navigator layers. Queries ATT&CK data, maps APT techniques to Navigator layers, performs detection gap analysis, and generates threat-informed reports. Use when the user asks to analyze apt group with mitre navigator.
version: 1.0.0
tags:
source: analyzing-apt-group-with-mitre-navigator

Analyzing APT Group with MITRE Navigator

Role: Deep analysis
Audience: analysts and investigators
Primary tools: MITRE ATT&CK, mitreattack-python Library, Key APT Groups Reference

When to use

- The user wants to **analyze** apt group with mitre navigator.
- Capability: APT group analysis agent using MITRE ATT&CK Navigator layers. Queries ATT&CK data, maps APT techniques to Navigator layers, performs detection gap analysis, and generates threat-informed reports.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, group, mitre, navigator`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load ATT&CK STIX bundle from file or download
2. Extract intrusion-set (APT group) objects from STIX bundle
3. Extract attack-pattern (technique) objects from STIX bundle
4. Map techniques used by a specific group via relationship objects
5. Build ATT&CK Navigator JSON layer for a group's techniques
6. Compare group TTPs against existing detection rules to find gaps
7. Generate tactic-level heatmap showing technique distribution
8. Compare two groups' technique sets for overlap analysis

Tools & resources

- MITRE ATT&CK
- mitreattack-python Library
- Key APT Groups Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_attack_data` — Load ATT&CK STIX bundle from file or download.
- `extract_groups` — Extract intrusion-set (APT group) objects from STIX bundle.
- `extract_techniques` — Extract attack-pattern (technique) objects from STIX bundle.
- `map_group_techniques` — Map techniques used by a specific group via relationship objects.
- `build_navigator_layer` — Build ATT&CK Navigator JSON layer for a group's techniques.
- `detection_gap_analysis` — Compare group TTPs against existing detection rules to find gaps.
- `tactic_heatmap` — Generate tactic-level heatmap showing technique distribution.
- `compare_groups` — Compare two groups' technique sets for overlap analysis.
- `save_layer` — Save Navigator layer to JSON file.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-apt-group-with-mitre-navigator`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-azure-activity-logs-for-threats (Skill Module)

analyzing-azure-activity-logs-for-threats\SKILL.md

name: analyzing-azure-activity-logs-for-threats
description: Agent for analyzing Azure activity logs for threat detection. Use when the user asks to analyze azure activity logs for threats.
version: 1.0.0
tags:
source: analyzing-azure-activity-logs-for-threats

Analyzing Azure Activity Logs for Threats

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Azure

When to use

- The user wants to **analyze** azure activity logs for threats.
- Capability: Agent for analyzing Azure activity logs for threat detection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, azure, activity, logs, threats`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Get Azure credential
2. Execute KQL query against Log Analytics workspace
3. Detect role assignment changes indicating privilege escalation
4. Detect Network Security Group rule modifications
5. Detect Key Vault secret access from unusual sources
6. Detect sign-ins from geographically distant locations in short time
7. Detect mass resource deletion events
8. Detect modifications to Conditional Access policies

Tools & resources

- Azure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_credential` — Get Azure credential.
- `run_kql` — Execute KQL query against Log Analytics workspace.
- `detect_privilege_escalation` — Detect role assignment changes indicating privilege escalation.
- `detect_nsg_changes` — Detect Network Security Group rule modifications.
- `detect_keyvault_access` — Detect Key Vault secret access from unusual sources.
- `detect_impossible_travel` — Detect sign-ins from geographically distant locations in short time.
- `detect_resource_deletion` — Detect mass resource deletion events.
- `detect_conditional_access_changes` — Detect modifications to Conditional Access policies.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-azure-activity-logs-for-threats`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-bootkit-and-rootkit-samples (Skill Module)

analyzing-bootkit-and-rootkit-samples\SKILL.md

name: analyzing-bootkit-and-rootkit-samples
description: Bootkit and rootkit analysis agent for MBR/VBR/UEFI inspection and rootkit detection. Use when the user asks to analyze bootkit and rootkit samples.
version: 1.0.0
tags:
source: analyzing-bootkit-and-rootkit-samples

Analyzing Bootkit and Rootkit Samples

Role: Deep analysis
Audience: analysts and investigators
Primary tools: dd, ndisasm, UEFITool, chipsec, Volatility 3, flashrom

When to use

- The user wants to **analyze** bootkit and rootkit samples.
- Capability: Bootkit and rootkit analysis agent for MBR/VBR/UEFI inspection and rootkit detection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, bootkit, rootkit, samples`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Read and parse the first 512 bytes (MBR) from a disk image or device
2. Check the MBR boot signature at bytes 510-511 (should be 0x55AA)
3. Parse the four 16-byte partition table entries starting at offset 446
4. Scan boot sector data against known bootkit signatures
5. Calculate Shannon entropy of binary data
6. Read the first track (typically 63 sectors) for extended bootkit code
7. Analyze MBR bootstrap code (bytes 0-445) for suspicious patterns
8. Run a Volatility 3 plugin for rootkit detection via subprocess

Tools & resources

- dd
- ndisasm
- UEFITool
- chipsec
- Volatility 3
- flashrom
- YARA

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `read_mbr` — Read and parse the first 512 bytes (MBR) from a disk image or device.
- `validate_mbr_signature` — Check the MBR boot signature at bytes 510-511 (should be 0x55AA).
- `parse_partition_table` — Parse the four 16-byte partition table entries starting at offset 446.
- `scan_bootkit_signatures` — Scan boot sector data against known bootkit signatures.
- `calculate_entropy` — Calculate Shannon entropy of binary data.
- `read_first_track` — Read the first track (typically 63 sectors) for extended bootkit code.
- `analyze_boot_code` — Analyze MBR bootstrap code (bytes 0-445) for suspicious patterns.
- `run_volatility_rootkit_scan` — Run a Volatility 3 plugin for rootkit detection via subprocess.
- `detect_kernel_rootkit` — Run multiple Volatility plugins to detect kernel-level rootkit artifacts.
- `compare_process_lists` — Compare pslist and psscan output to find hidden processes (DKOM).

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-bootkit-and-rootkit-samples`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-browser-forensics-with-hindsight (Skill Module)

analyzing-browser-forensics-with-hindsight\SKILL.md

name: analyzing-browser-forensics-with-hindsight
description: Browser forensics analysis agent using Hindsight concepts. Parses Chromium-based browser artifacts (Chrome, Edge, Brave) including history, downloads, cookies, autofill, and extensions from SQLite databases. Use when the user asks to analyze browser forensics with hindsight.
version: 1.0.0
tags:
source: analyzing-browser-forensics-with-hindsight

Analyzing Browser Forensics with Hindsight

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Hindsight, DB Browser for SQLite, ChromeCacheView (NirSoft)

When to use

- The user wants to **analyze** browser forensics with hindsight.
- Capability: Browser forensics analysis agent using Hindsight concepts. Parses Chromium-based browser artifacts (Chrome, Edge, Brave) including history, downloads, cookies, autofill, and extensions from SQLite databases.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, browser, forensics, hindsight`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Convert Chrome timestamp (microseconds since 1601-01-01) to datetime
2. Locate Chromium-based browser profile directories
3. Parse browsing history from History SQLite database
4. Parse download history from History database
5. Parse cookies from Cookies database
6. Parse autofill data from Web Data database
7. Parse installed browser extensions
8. Flag suspicious browsing and download patterns

Key concepts

- DB Browser for SQLite: Chrome database inspection
- ChromeCacheView (NirSoft): Cache analysis
- History: URL visits, downloads, keyword searches
- Web Data: Autofill, credit cards
- Login Data: Saved credentials (encrypted)
- Bookmarks: JSON bookmark tree
- Chrome/WebKit: microseconds since 1601-01-01 UTC
- Firefox/Mozilla: microseconds since Unix epoch

Tools & resources

- Hindsight
- DB Browser for SQLite
- ChromeCacheView (NirSoft)

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `chrome_time_to_datetime` — Convert Chrome timestamp (microseconds since 1601-01-01) to datetime.
- `find_browser_profiles` — Locate Chromium-based browser profile directories.
- `parse_history` — Parse browsing history from History SQLite database.
- `parse_downloads` — Parse download history from History database.
- `parse_cookies` — Parse cookies from Cookies database.
- `parse_autofill` — Parse autofill data from Web Data database.
- `parse_extensions` — Parse installed browser extensions.
- `detect_suspicious_activity` — Flag suspicious browsing and download patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-browser-forensics-with-hindsight`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-campaign-attribution-evidence (Skill Module)

`analyzing-campaign-attribution-evidence\SKILL.md`

name: analyzing-campaign-attribution-evidence
description: Campaign attribution analysis agent using Diamond Model and ACH methodology. Evaluates attribution evidence including infrastructure overlaps, TTP consistency, malware code similarity, timing patterns, and language.... Use when the user asks to analyze campaign attribution evidence.
version: 1.0.0
tags:
source: analyzing-campaign-attribution-evidence

Analyzing Campaign Attribution Evidence

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Infrastructure Overlap Tools, Confidence Assessment Framework, STIX Attribution Objects, Applicable Standards, Industry Frameworks

When to use

- The user wants to **analyze** campaign attribution evidence.
- Capability: Campaign attribution analysis agent using Diamond Model and ACH methodology. Evaluates attribution evidence including infrastructure overlaps, TTP consistency, malware code similarity, timing patterns, and language artifacts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, campaign, attribution, evidence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Structure evidence using the Diamond Model of Intrusion Analysis
2. Score infrastructure overlap between campaign and known actor
3. Score TTP consistency using MITRE ATT&CK technique overlap
4. Score malware code similarity based on feature comparison
5. Analyze operational timing to infer timezone/working hours
6. Detect language artifacts in malware strings or documents
7. Analysis of Competing Hypotheses (ACH) for attribution
8. Compute weighted attribution confidence score

Key concepts

- STIX 2.1: Structured Threat Information eXpression for CTI data representation
- TAXII 2.1: Transport protocol for sharing CTI over HTTPS
- MITRE ATT&CK: Adversary tactics, techniques, and procedures taxonomy
- Diamond Model: Intrusion analysis framework (Adversary, Capability, Infrastructure, Victim)
- Traffic Light Protocol (TLP): Information sharing classification (CLEAR, GREEN, AMBER, RED)
- Technique mapping for threat actor behavior classification
- Data sources for detection capability assessment
- Mitigation strategies linked to specific techniques

Tools & resources

- Infrastructure Overlap Tools
- Confidence Assessment Framework
- STIX Attribution Objects
- Applicable Standards

- Industry Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `diamond_model_analysis` — Structure evidence using the Diamond Model of Intrusion Analysis.
- `evaluate_infrastructure_overlap` — Score infrastructure overlap between campaign and known actor.
- `evaluate_ttp_consistency` — Score TTP consistency using MITRE ATT&CK technique overlap.
- `evaluate_malware_similarity` — Score malware code similarity based on feature comparison.
- `evaluate_timing_pattern` — Analyze operational timing to infer timezone/working hours.
- `evaluate_language_artifacts` — Detect language artifacts in malware strings or documents.
- `ach_analysis` — Analysis of Competing Hypotheses (ACH) for attribution.
- `compute_attribution_score` — Compute weighted attribution confidence score.
- `generate_attribution_report` — Generate structured attribution assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-campaign-attribution-evidence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-certificate-transparency-for-phishing (Skill Module)

analyzing-certificate-transparency-for-phishing\SKILL.md

name: analyzing-certificate-transparency-for-phishing
description: Certificate Transparency monitoring agent for phishing detection. Queries crt.sh for certificates matching target domains, detects lookalike certificates, and identifies potential phishing infrastructure. Use when the user asks to analyze certificate transparency for phishing.
version: 1.0.0
tags:
source: analyzing-certificate-transparency-for-phishing

Analyzing Certificate Transparency for Phishing

Role: Deep analysis
Audience: analysts and investigators
Primary tools: crt.sh API, Certstream, CT Log Servers, Phishing Detection Techniques, Certificate Details Lookup

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** certificate transparency for phishing.
- Capability: Certificate Transparency monitoring agent for phishing detection. Queries crt.sh for certificates matching target domains, detects lookalike certificates, and identifies potential phishing infrastructure.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, certificate, transparency, phishing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query crt.sh for certificates matching a domain
2. Identify certificates for domains that look similar to the target
3. Calculate string similarity using Levenshtein-like ratio
4. Detect domains using homoglyph/IDN attacks against target
5. Analyze certificate issuer patterns for anomalies
6. Detect suspicious wildcard certificate patterns
7. Generate comprehensive CT monitoring report

Tools & resources

- crt.sh API
- Certstream
- CT Log Servers
- Phishing Detection Techniques
- Certificate Details Lookup

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_crtsh` — Query crt.sh for certificates matching a domain.
- `find_lookalike_domains` — Identify certificates for domains that look similar to the target.
- `calculate_similarity` — Calculate string similarity using Levenshtein-like ratio.
- `detect_homoglyph_domains` — Detect domains using homoglyph/IDN attacks against target.
- `analyze_issuer_patterns` — Analyze certificate issuer patterns for anomalies.
- `detect_wildcard_abuse` — Detect suspicious wildcard certificate patterns.
- `generate_report` — Generate comprehensive CT monitoring report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-certificate-transparency-for-phishing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-cloud-storage-access-patterns (Skill Module)

analyzing-cloud-storage-access-patterns\SKILL.md

name: analyzing-cloud-storage-access-patterns
description: Cloud Storage Access Pattern Analyzer - Detects abnormal S3/GCS/Azure Blob access via CloudTrail. Use when the user asks to analyze cloud storage access patterns.
version: 1.0.0
tags:
source: analyzing-cloud-storage-access-patterns

Analyzing Cloud Storage Access Patterns

Role: Deep analysis
Audience: analysts and investigators
Primary tools: AWS CLI, Detection Thresholds

When to use

- The user wants to **analyze** cloud storage access patterns.
- Capability: Cloud Storage Access Pattern Analyzer - Detects abnormal S3/GCS/Azure Blob access via CloudTrail.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, cloud, storage, access, patterns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query CloudTrail for S3 data events on a specific bucket
2. Detect bulk GetObject operations from a single principal
3. Detect access outside business hours
4. Detect access from IP addresses not in the known baseline
5. Detect ListBucket/ListObjects enumeration patterns
6. Build statistical baseline of normal access patterns
7. Generate cloud storage access analysis report

Tools & resources

- AWS CLI
- Detection Thresholds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_cloudtrail_s3_events` — Query CloudTrail for S3 data events on a specific bucket.
- `detect_bulk_downloads` — Detect bulk GetObject operations from a single principal.
- `detect_after_hours_access` — Detect access outside business hours.
- `detect_new_source_ips` — Detect access from IP addresses not in the known baseline.
- `detect_enumeration` — Detect ListBucket/ListObjects enumeration patterns.
- `build_access_baseline` — Build statistical baseline of normal access patterns.
- `generate_report` — Generate cloud storage access analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-cloud-storage-access-patterns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-cobalt-strike-beacon-configuration (Skill Module)

analyzing-cobalt-strike-beacon-configuration\SKILL.md

name: analyzing-cobalt-strike-beacon-configuration
description: Cobalt Strike beacon configuration extraction and analysis agent. Extracts C2 configuration from beacon payloads including server addresses, communication settings, malleable C2 profile details, and watermark values. Use when the user asks to analyze cobalt strike beacon configuration.
version: 1.0.0
tags:
source: analyzing-cobalt-strike-beacon-configuration

Analyzing Cobalt Strike Beacon Configuration

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Cobalt Strike, JARM Fingerprinting, Known Watermark Values, Detection Signatures, Malleable C2 Profile Elements, Malleable C2 Profile Structure

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** cobalt strike beacon configuration.
- Capability: Cobalt Strike beacon configuration extraction and analysis agent. Extracts C2 configuration from beacon payloads including server addresses, communication settings, malleable C2 profile details, and watermark values.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, cobalt, strike, beacon, configuration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Compute SHA-256 hash of file
2. Find the beacon configuration blob in PE data or shellcode
3. XOR decode data with single byte key
4. Parse a single TLV config field
5. Extract and parse Cobalt Strike beacon configuration
6. Extract C2 indicators from parsed config for threat intelligence
7. Assess operator OPSEC based on beacon configuration

Key concepts

- Cobalt Strike 3.x: XOR key = 0x69
- Cobalt Strike 4.x: XOR key = 0x2e
- Configuration blob size: 4096 bytes (typical)
- Encoding: Single-byte XOR across entire config blob
- PE with beacon code in .data section
- Configuration embedded after beacon code
- Reflective DLL loader prepended to beacon

Tools & resources

- Cobalt Strike
- JARM Fingerprinting

- Known Watermark Values
- Detection Signatures
- Malleable C2 Profile Elements
- Malleable C2 Profile Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compute_hash` — Compute SHA-256 hash of file.
- `find_config_offset` — Find the beacon configuration blob in PE data or shellcode.
- `xor_decode` — XOR decode data with single byte key.
- `parse_config_field` — Parse a single TLV config field.
- `extract_beacon_config` — Extract and parse Cobalt Strike beacon configuration.
- `extract_c2_indicators` — Extract C2 indicators from parsed config for threat intelligence.
- `assess_operator_opsec` — Assess operator OPSEC based on beacon configuration.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-cobalt-strike-beacon-configuration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-cobaltstrike-malleable-c2-profiles (Skill Module)

`analyzing-cobaltstrike-malleable-c2-profiles\SKILL.md`

name: analyzing-cobaltstrike-malleable-c2-profiles
description: CobaltStrike Malleable C2 Profile Analyzer - parses profiles to extract C2 indicators, detection signatures, and evasion techniques. Use when the user asks to analyze cobaltstrike malleable c2 profiles.
version: 1.0.0
tags:
source: analyzing-cobaltstrike-malleable-c2-profiles

Analyzing Cobalt Strike Malleable C2 Profiles

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Cobalt Strike, dissect.cobaltstrike API, pyMalleableC2 API, Key Profile Settings

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** cobaltstrike malleable c2 profiles.
- Capability: CobaltStrike Malleable C2 Profile Analyzer - parses profiles to extract C2 indicators, detection signatures, and evasion techniques
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, cobaltstrike, malleable, profiles`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse a .profile file using dissect.cobaltstrike C2Profile
2. Regex-based parser for malleable C2 profile when dissect is unavailable
3. Analyze parsed profile configuration for detection opportunities
4. Generate Suricata rules from extracted indicators

Tools & resources

- Cobalt Strike
- dissect.cobaltstrike API
- pyMalleableC2 API
- Key Profile Settings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_data`
- `parse_profile_with_dissect` — Parse a .profile file using dissect.cobaltstrike C2Profile.
- `parse_profile_regex` — Regex-based parser for malleable C2 profile when dissect is unavailable.
- `analyze_profile` — Analyze parsed profile configuration for detection opportunities.
- `generate_suricata_rules` — Generate Suricata rules from extracted indicators.
- `analyze`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-cobaltstrike-malleable-c2-profiles`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-command-and-control-communication (Skill Module)

analyzing-command-and-control-communication\SKILL.md

name: analyzing-command-and-control-communication
description: C2 communication analysis agent for beacon detection and protocol decoding. Use when the user asks to analyze command and control communication.
version: 1.0.0
tags:
source: analyzing-command-and-control-communication

Analyzing Command and Control Communication

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Scapy, dpkt, tshark, CobaltStrikeParser, Suricata

When to use

- The user wants to **analyze** command and control communication.
- Capability: C2 communication analysis agent for beacon detection and protocol decoding.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, command, control, communication`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze PCAP for periodic beacon patterns using TCP SYN timing
2. Extract HTTP requests from a PCAP file using dpkt
3. Extract DNS queries from a PCAP for C2 domain identification
4. Match HTTP request patterns against known C2 framework signatures
5. Generate Suricata IDS rules from observed C2 patterns

Tools & resources

- Scapy
- dpkt
- tshark
- CobaltStrikeParser
- Suricata

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_beacons` — Analyze PCAP for periodic beacon patterns using TCP SYN timing.
- `extract_http_requests` — Extract HTTP requests from a PCAP file using dpkt.
- `extract_dns_queries` — Extract DNS queries from a PCAP for C2 domain identification.
- `identify_c2_framework` — Match HTTP request patterns against known C2 framework signatures.
- `generate_suricata_rules` — Generate Suricata IDS rules from observed C2 patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-command-and-control-communication`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-cyber-kill-chain (Skill Module)

`analyzing-cyber-kill-chain\SKILL.md`

name: analyzing-cyber-kill-chain
description: Cyber Kill Chain analysis agent for mapping incidents to Lockheed Martin kill chain phases. Use when the user asks to analyze cyber kill chain.
version: 1.0.0
tags:
source: analyzing-cyber-kill-chain

Analyzing Cyber Kill Chain

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Elastic Security EQL, MISP

When to use

- The user wants to **analyze** cyber kill chain.
- Capability: Cyber Kill Chain analysis agent for mapping incidents to Lockheed Martin kill chain phases.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, cyber, kill, chain`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Map an incident event description to the most likely kill chain phase
2. Analyze a list of incident events and map to kill chain phases
3. Generate a kill chain analysis report

Tools & resources

- Elastic Security EQL
- MISP

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `map_event_to_phase` — Map an incident event description to the most likely kill chain phase.
- `analyze_incident` — Analyze a list of incident events and map to kill chain phases.
- `generate_report` — Generate a kill chain analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-cyber-kill-chain`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-disk-image-with-autopsy (Skill Module)

analyzing-disk-image-with-autopsy\SKILL.md

name: analyzing-disk-image-with-autopsy
description: Forensic disk image analysis agent using The Sleuth Kit (TSK) command-line tools. Use when the user asks to analyze disk image with autopsy.
version: 1.0.0
tags:
source: analyzing-disk-image-with-autopsy

Analyzing Disk Image with Autopsy

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Autopsy, mmls, fls, icat, istat, mactime

When to use

- The user wants to **analyze** disk image with autopsy.
- Capability: Forensic disk image analysis agent using The Sleuth Kit (TSK) command-line tools.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, disk, image, autopsy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Retrieve disk image metadata using img_stat
2. List partition layout using mmls
3. List files in a partition using fls
4. List only deleted files using fls -rd
5. Recover a file by inode using icat
6. Get detailed file metadata using istat
7. Generate a TSK bodyfile for timeline creation
8. Generate a timeline from a bodyfile using mactime

Tools & resources

- Autopsy
- mmls
- fls
- icat
- istat
- mactime
- sigfind
- srch_strings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_image_info` — Retrieve disk image metadata using img_stat.
- `list_partitions` — List partition layout using mmls.
- `list_files` — List files in a partition using fls.
- `list_deleted_files` — List only deleted files using fls -rd.
- `recover_file` — Recover a file by inode using icat.
- `get_file_metadata` — Get detailed file metadata using istat.
- `create_bodyfile` — Generate a TSK bodyfile for timeline creation.
- `generate_timeline` — Generate a timeline from a bodyfile using mactime.
- `search_keywords` — Search for keyword strings in the disk image.
- `find_file_signature` — Find file signatures at the sector level using sigfind.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-disk-image-with-autopsy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-dns-logs-for-exfiltration (Skill Module)

analyzing-dns-logs-for-exfiltration\SKILL.md

name: analyzing-dns-logs-for-exfiltration
description: DNS exfiltration detection agent using entropy analysis and query pattern detection. Use when the user asks to analyze dns logs for exfiltration.
version: 1.0.0
tags:
source: analyzing-dns-logs-for-exfiltration

Analyzing DNS Logs for Exfiltration

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Passive DNS Lookup APIs

When to use

- The user wants to **analyze** dns logs for exfiltration.
- Capability: DNS exfiltration detection agent using entropy analysis and query pattern detection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, logs, exfiltration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of a string
2. Extract the subdomain portion from a fully qualified domain name
3. Extract the registered domain (SLD + TLD) from an FQDN
4. Detect DNS tunneling based on subdomain length anomalies
5. Detect Domain Generation Algorithm queries using entropy scoring
6. Detect hosts with anomalously high DNS query volumes
7. Detect excessive TXT record queries (common tunneling method)
8. Estimate data volume encoded in DNS queries to a specific domain

Tools & resources

- Passive DNS Lookup APIs

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of a string.
- `extract_subdomain` — Extract the subdomain portion from a fully qualified domain name.
- `extract_registered_domain` — Extract the registered domain (SLD + TLD) from an FQDN.
- `detect_tunneling` — Detect DNS tunneling based on subdomain length anomalies.
- `detect_dga` — Detect Domain Generation Algorithm queries using entropy scoring.
- `detect_volume_anomaly` — Detect hosts with anomalously high DNS query volumes.
- `detect_txt_abuse` — Detect excessive TXT record queries (common tunneling method).
- `estimate_exfil_volume` — Estimate data volume encoded in DNS queries to a specific domain.
- `parse_zeek_dns_log` — Parse a Zeek dns.log file into structured records.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-dns-logs-for-exfiltration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-docker-container-forensics (Skill Module)

analyzing-golang-malware-with-ghidra\analyzing-docker-container-forensics\SKILL.md

name: analyzing-docker-container-forensics
description: Docker container forensics agent for investigating compromised containers. Use when the user asks to analyze docker container forensics.
version: 1.0.0
tags:
source: analyzing-docker-container-forensics

Analyzing Docker Container Forensics

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Docker, dive, container-diff, Trivy

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** docker container forensics.
- Capability: Docker container forensics agent for investigating compromised containers.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, docker, container, forensics`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. List Docker containers with detailed information
2. Get detailed container inspection data
3. Analyze container security configuration for misconfigurations
4. Get filesystem changes between container and its image
5. Analyze filesystem changes for indicators of compromise
6. Export container filesystem as a tarball for offline analysis
7. Retrieve container logs with timestamps
8. Run Trivy vulnerability scan on a container image

Tools & resources

- Docker
- dive
- container-diff
- Trivy

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `list_containers` — List Docker containers with detailed information.
- `inspect_container` — Get detailed container inspection data.
- `analyze_security_config` — Analyze container security configuration for misconfigurations.
- `get_filesystem_changes` — Get filesystem changes between container and its image.
- `detect_suspicious_files` — Analyze filesystem changes for indicators of compromise.
- `export_container` — Export container filesystem as a tarball for offline analysis.
- `get_container_logs` — Retrieve container logs with timestamps.
- `scan_image_vulnerabilities` — Run Trivy vulnerability scan on a container image.
- `generate_report` — Generate a forensic analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-docker-container-forensics`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-email-headers-for-phishing-investigation (Skill Module)

analyzing-email-headers-for-phishing-investigation\SKILL.md

name: analyzing-email-headers-for-phishing-investigation
description: Email header analysis agent for phishing investigation and sender verification. Use when the user asks to analyze email headers for phishing investigation.
version: 1.0.0
tags:
source: analyzing-email-headers-for-phishing-investigation

Analyzing Email Headers for Phishing Investigation

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Python email Module, dig, pyspf, dkimpy, AbuseIPDB, VirusTotal

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** email headers for phishing investigation.
- Capability: Email header analysis agent for phishing investigation and sender verification.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, email, headers, phishing, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse an EML file and extract key header fields
2. Extract and parse the Received header chain (bottom-up = chronological)
3. Extract SPF, DKIM, and DMARC results from Authentication-Results headers
4. Detect mismatch between From and Reply-To addresses
5. Extract all URLs from the email body
6. Detect hyperlinks where display text differs from actual href
7. Extract and hash all email attachments
8. Perform DNS lookup for SPF/DKIM/DMARC records

Tools & resources

- Python email Module
- dig
- pyspf
- dkimpy
- AbuseIPDB
- VirusTotal
- whois

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_email_file` — Parse an EML file and extract key header fields.
- `extract_received_chain` — Extract and parse the Received header chain (bottom-up = chronological).
- `extract_authentication_results` — Extract SPF, DKIM, and DMARC results from Authentication-Results headers.
- `check_from_replyto_mismatch` — Detect mismatch between From and Reply-To addresses.
- `extract_urls` — Extract all URLs from the email body.
- `detect_url_mismatch` — Detect hyperlinks where display text differs from actual href.
- `extract_attachments` — Extract and hash all email attachments.
- `dns_lookup` — Perform DNS lookup for SPF/DKIM/DMARC records.
- `check_domain_spf` — Look up the SPF record for a domain.
- `check_domain_dmarc` — Look up the DMARC record for a domain.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-email-headers-for-phishing-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-ethereum-smart-contract-vulnerabilities (Skill Module)

`analyzing-ethereum-smart-contract-vulnerabilities\SKILL.md`

name: analyzing-ethereum-smart-contract-vulnerabilities
description: Smart Contract Security Agent - runs Slither and Mythril analysis on Solidity contracts. Use when the user asks to analyze ethereum smart contract vulnerabilities.
version: 1.0.0
tags:
source: analyzing-ethereum-smart-contract-vulnerabilities

Analyzing Ethereum Smart Contract Vulnerabilities

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Slither CLI, Mythril CLI, Slither Detector Severity Levels, Slither JSON Output Structure

When to use

- The user wants to **analyze** ethereum smart contract vulnerabilities.
- Capability: Smart Contract Security Agent - runs Slither and Mythril analysis on Solidity contracts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, ethereum, smart, contract, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Slither static analysis on Solidity contract
2. Run Mythril symbolic execution analysis
3. Parse and categorize Slither detector findings
4. Parse Mythril symbolic execution findings
5. Merge and deduplicate findings from both tools

Tools & resources

- Slither CLI
- Mythril CLI
- Slither Detector Severity Levels
- Slither JSON Output Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_slither` — Run Slither static analysis on Solidity contract.
- `run_mythril` — Run Mythril symbolic execution analysis.
- `analyze_slither_results` — Parse and categorize Slither detector findings.
- `analyze_mythril_results` — Parse Mythril symbolic execution findings.
- `deduplicate_findings` — Merge and deduplicate findings from both tools.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-ethereum-smart-contract-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-golang-malware-with-ghidra (Skill Module)

analyzing-golang-malware-with-ghidra\SKILL.md

name: analyzing-golang-malware-with-ghidra
description: Go malware analysis agent for Ghidra-assisted reverse engineering. Analyzes Go binaries to extract function names, strings, build metadata, package information, and detects common Go malware characteristics. Use when the user asks to analyze golang malware with ghidra.
version: 1.0.0
tags:
source: analyzing-golang-malware-with-ghidra

Analyzing Golang Malware with Ghidra

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Ghidra, Go Binary Characteristics, Go Obfuscation Tools, Common Go Malware Families, Go Binary Structure

When to use

- The user wants to **analyze** golang malware with ghidra.
- Capability: Go malware analysis agent for Ghidra-assisted reverse engineering. Analyzes Go binaries to extract function names, strings, build metadata, package information, and detects common Go malware characteristics.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, golang, malware, ghidra`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Compute SHA-256 hash of file
2. Calculate Shannon entropy
3. Detect if a binary is compiled with Go and extract version info
4. Extract Go-style strings (length-prefixed, not null-terminated)
5. Identify Go packages from extracted strings
6. Flag suspicious Go packages commonly used in malware
7. Analyze PE/ELF sections for Go binary characteristics
8. Detect Go binary obfuscation (garble, gobfuscate)

Key concepts

- Geacon (Go Cobalt Strike beacon)
- Chaos botnet (Go-based)

Tools & resources

- Ghidra
- Go Binary Characteristics
- Go Obfuscation Tools
- Common Go Malware Families
- Go Binary Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compute_hash` — Compute SHA-256 hash of file.
- `shannon_entropy` — Calculate Shannon entropy.
- `detect_go_binary` — Detect if a binary is compiled with Go and extract version info.
- `extract_go_strings` — Extract Go-style strings (length-prefixed, not null-terminated).
- `extract_go_packages` — Identify Go packages from extracted strings.
- `detect_suspicious_packages` — Flag suspicious Go packages commonly used in malware.
- `analyze_sections` — Analyze PE/ELF sections for Go binary characteristics.
- `detect_obfuscation` — Detect Go binary obfuscation (garble, gobfuscate).
- `generate_report` — Generate comprehensive Go malware analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-golang-malware-with-ghidra`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-heap-spray-exploitation (Skill Module)

analyzing-heap-spray-exploitation\SKILL.md

name: analyzing-heap-spray-exploitation
description: Agent for analyzing heap spray exploitation in memory dumps. Detects heap spray artifacts using Volatility3 by scanning for NOP sled patterns, large contiguous allocations, and injected executable regions in process.... Use when the user asks to analyze heap spray exploitation.
version: 1.0.0
tags:
source: analyzing-heap-spray-exploitation

Analyzing Heap Spray Exploitation

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Shellcode Signatures, Detection Thresholds, Install Volatility3

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** heap spray exploitation.
- Capability: Agent for analyzing heap spray exploitation in memory dumps. Detects heap spray artifacts using Volatility3 by scanning for NOP sled patterns, large contiguous allocations, and injected executable regions in process virtual address space.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, heap, spray, exploitation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Ingest and validate inputs for heap spray exploitation
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

Tools & resources

- Shellcode Signatures
- Detection Thresholds
- Install Volatility3

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-heap-spray-exploitation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-indicators-of-compromise (Skill Module)

analyzing-indicators-of-compromise\SKILL.md

name: analyzing-indicators-of-compromise
description: IOC analysis and enrichment agent using VirusTotal, AbuseIPDB, and MalwareBazaar APIs. Use when the user asks to analyze indicators of compromise.
version: 1.0.0
tags:
source: analyzing-indicators-of-compromise

Analyzing Indicators of Compromise

Role: Deep analysis
Audience: analysts and investigators
Primary tools: VirusTotal API v3, AbuseIPDB API v2, URLScan.io API, Shodan API, IOC Confidence Scoring Framework, Defanging Convention

When to use

- The user wants to **analyze** indicators of compromise.
- Capability: IOC analysis and enrichment agent using VirusTotal, AbuseIPDB, and MalwareBazaar APIs.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, indicators, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Classify an IOC by type: ipv4, domain, url, sha256, sha1, md5, email
2. Defang an IOC for safe documentation
3. Refang a defanged IOC for querying APIs
4. Check if an IP is RFC 1918 private
5. Query VirusTotal for a file hash
6. Query VirusTotal for domain reputation
7. Query AbuseIPDB for IP reputation
8. Query MalwareBazaar for file hash information

Tools & resources

- VirusTotal API v3
- AbuseIPDB API v2
- URLScan.io API
- Shodan API
- IOC Confidence Scoring Framework
- Defanging Convention

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `classify_ioc` — Classify an IOC by type: ipv4, domain, url, sha256, sha1, md5, email.
- `defang_ioc` — Defang an IOC for safe documentation.
- `refang_ioc` — Refang a defanged IOC for querying APIs.
- `is_private_ip` — Check if an IP is RFC 1918 private.
- `query_virustotal_hash` — Query VirusTotal for a file hash.
- `query_virustotal_domain` — Query VirusTotal for domain reputation.
- `query_abuseipdb` — Query AbuseIPDB for IP reputation.
- `query_malwarebazaar` — Query MalwareBazaar for file hash information.
- `score_ioc` — Assign a confidence score and disposition to an IOC.
- `enrich_ioc` — Enrich a single IOC with multi-source intelligence.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-indicators-of-compromise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-ios-app-security-with-objection (Skill Module)

analyzing-ios-app-security-with-objection\SKILL.md

name: analyzing-ios-app-security-with-objection
description: iOS app security analysis agent using Objection/Frida concepts. Performs runtime security assessment of iOS apps including SSL pinning bypass, keychain dumping, filesystem inspection, and jailbreak detection bypass. Use when the user asks to analyze ios app security with objection.
version: 1.0.0
tags:
source: analyzing-ios-app-security-with-objection

Analyzing IOS App Security with Objection

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Objection, Frida CLI, iOS App Sandbox Paths, OWASP MASTG Test Cases, Apple Platform Security Requirements

When to use

- The user wants to **analyze** ios app security with objection.
- Capability: iOS app security analysis agent using Objection/Frida concepts. Performs runtime security assessment of iOS apps including SSL pinning bypass, keychain dumping, filesystem inspection, and jailbreak detection bypass.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, security, objection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute an Objection command against a target app
2. Execute a Frida script against target app
3. Dump keychain items accessible by the application
4. Dump HTTP cookies stored by the application
5. List Objective-C classes loaded in the app
6. Check and bypass SSL certificate pinning
7. Check for and bypass jailbreak detection
8. Inspect the application's filesystem sandbox

Tools & resources

- Objection
- Frida CLI
- iOS App Sandbox Paths
- OWASP MASTG Test Cases
- Apple Platform Security Requirements

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_objection` — Execute an Objection command against a target app.
- `run_frida` — Execute a Frida script against target app.
- `dump_keychain` — Dump keychain items accessible by the application.
- `dump_cookies` — Dump HTTP cookies stored by the application.
- `list_classes` — List Objective-C classes loaded in the app.
- `check_ssl_pinning` — Check and bypass SSL certificate pinning.
- `check_jailbreak_detection` — Check for and bypass jailbreak detection.
- `inspect_filesystem` — Inspect the application's filesystem sandbox.
- `dump_plist` — Dump application plist configuration files.
- `check_pasteboard` — Check pasteboard/clipboard for sensitive data.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-ios-app-security-with-objection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-kubernetes-audit-logs (Skill Module)

analyzing-kubernetes-audit-logs\SKILL.md

name: analyzing-kubernetes-audit-logs
description: Agent for analyzing Kubernetes audit logs for security threats. Use when the user asks to analyze kubernetes audit logs.
version: 1.0.0
tags:
source: analyzing-kubernetes-audit-logs

Analyzing Kubernetes Audit Logs

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Kubernetes, Security-Critical Audit Events, Audit Policy Levels, Python Parsing

When to use

- The user wants to **analyze** kubernetes audit logs.
- Capability: Agent for analyzing Kubernetes audit logs for security threats.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, kubernetes, audit, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Kubernetes audit log file (JSON lines format)
2. Detect kubectl exec and attach events (shell access to pods)
3. Detect access to Kubernetes secrets
4. Detect RBAC role and binding modifications
5. Detect creation of privileged pods
6. Detect API access by anonymous or unauthenticated users
7. Detect 403 surges indicating enumeration or brute force

Tools & resources

- Kubernetes
- Security-Critical Audit Events
- Audit Policy Levels
- Python Parsing

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_audit_log` — Parse Kubernetes audit log file (JSON lines format).
- `detect_pod_exec` — Detect kubectl exec and attach events (shell access to pods).
- `detect_secret_access` — Detect access to Kubernetes secrets.
- `detect_rbac_changes` — Detect RBAC role and binding modifications.
- `detect_privileged_pods` — Detect creation of privileged pods.
- `detect_anonymous_access` — Detect API access by anonymous or unauthenticated users.
- `detect_forbidden_surge` — Detect 403 surges indicating enumeration or brute force.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-kubernetes-audit-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-linux-audit-logs-for-intrusion (Skill Module)

analyzing-linux-audit-logs-for-intrusion\SKILL.md

name: analyzing-linux-audit-logs-for-intrusion
description: Linux audit log analysis agent for intrusion detection. Parses /var/log/audit/audit.log entries to detect privilege escalation, unauthorized file access, suspicious syscalls, and process execution anomalies. Use when the user asks to analyze linux audit logs for intrusion.
version: 1.0.0
tags:
source: analyzing-linux-audit-logs-for-intrusion

Analyzing Linux Audit Logs for Intrusion

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Audit Log Location, ausearch CLI, aureport CLI, Audit Log Fields, Suspicious Syscalls

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** linux audit logs for intrusion.
- Capability: Linux audit log analysis agent for intrusion detection. Parses /var/log/audit/audit.log entries to detect privilege escalation, unauthorized file access, suspicious syscalls, and process execution anomalies.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, linux, audit, logs, intrusion`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse raw audit.log file into structured events
2. Detect privilege escalation indicators in audit events
3. Detect access to sensitive files
4. Detect execution of suspicious commands
5. Run ausearch command and return results
6. Generate audit log analysis summary

Tools & resources

- Audit Log Location
- ausearch CLI
- aureport CLI
- Audit Log Fields
- Suspicious Syscalls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_audit_log` — Parse raw audit.log file into structured events.
- `detect_privilege_escalation` — Detect privilege escalation indicators in audit events.
- `detect_file_access` — Detect access to sensitive files.
- `detect_suspicious_commands` — Detect execution of suspicious commands.
- `run_ausearch` — Run ausearch command and return results.
- `generate_summary` — Generate audit log analysis summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-linux-audit-logs-for-intrusion`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-linux-elf-malware (Skill Module)

analyzing-linux-elf-malware\SKILL.md

name: analyzing-linux-elf-malware
description: Linux ELF malware static analysis agent using pyelftools and binary inspection. Use when the user asks to analyze linux elf malware.
version: 1.0.0
tags:
source: analyzing-linux-elf-malware

Analyzing Linux ELF Malware

Role: Deep analysis
Audience: analysts and investigators
Primary tools: readelf, pyelftools, strings, GDB, UPX, objdump

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** linux elf malware.
- Capability: Linux ELF malware static analysis agent using pyelftools and binary inspection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, linux, malware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute MD5, SHA1, and SHA256 hashes of a file
2. Calculate Shannon entropy of binary data
3. Parse ELF header and extract key properties
4. Analyze ELF sections for entropy and suspicious characteristics
5. Extract ASCII strings from the binary and categorize by type
6. Check if the binary is packed with UPX or other packers
7. Analyze dynamic linking information and imported functions
8. Classify malware type based on extracted strings

Tools & resources

- readelf
- pyelftools
- strings
- GDB
- UPX
- objdump
- nm

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Compute MD5, SHA1, and SHA256 hashes of a file.
- `calculate_entropy` — Calculate Shannon entropy of binary data.
- `analyze_elf_header` — Parse ELF header and extract key properties.
- `analyze_sections` — Analyze ELF sections for entropy and suspicious characteristics.
- `extract_strings` — Extract ASCII strings from the binary and categorize by type.
- `check_packing` — Check if the binary is packed with UPX or other packers.
- `analyze_dynamic_linking` — Analyze dynamic linking information and imported functions.
- `detect_malware_type` — Classify malware type based on extracted strings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-linux-elf-malware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-linux-kernel-rootkits (Skill Module)

analyzing-linux-kernel-rootkits\SKILL.md

name: analyzing-linux-kernel-rootkits
description: Linux Kernel Rootkit Detection Agent - analyzes memory dumps with Volatility3 and live system with rkhunter. Use when the user asks to analyze linux kernel rootkits.
version: 1.0.0
tags:
source: analyzing-linux-kernel-rootkits

Analyzing Linux Kernel Rootkits

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Volatility3 Linux Plugins, Memory Acquisition Tools, Known Linux Rootkits Detected, Cross-View Detection

When to use

- The user wants to **analyze** linux kernel rootkits.
- Capability: Linux Kernel Rootkit Detection Agent - analyzes memory dumps with Volatility3 and live system with rkhunter.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, linux, kernel, rootkits`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run a Volatility3 Linux plugin and return parsed output
2. Detect hooked system calls using linux.check_syscall
3. Detect hidden kernel modules using cross-view analysis
4. Check Interrupt Descriptor Table for hooks
5. Run rkhunter rootkit scanner on live system
6. Compare /proc/modules with /sys/module for hidden modules

Tools & resources

- Volatility3 Linux Plugins
- Memory Acquisition Tools
- Known Linux Rootkits Detected
- Cross-View Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_vol3_plugin` — Run a Volatility3 Linux plugin and return parsed output.
- `check_syscall_hooks` — Detect hooked system calls using linux.check_syscall.
- `detect_hidden_modules` — Detect hidden kernel modules using cross-view analysis.
- `check_idt_hooks` — Check Interrupt Descriptor Table for hooks.
- `run_rkhunter` — Run rkhunter rootkit scanner on live system.
- `check_proc_sys_discrepancy` — Compare /proc/modules with /sys/module for hidden modules.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-linux-kernel-rootkits`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-linux-system-artifacts (Skill Module)

analyzing-linux-system-artifacts\SKILL.md

name: analyzing-linux-system-artifacts
description: Linux system artifact forensics agent for investigating compromised systems. Use when the user asks to analyze linux system artifacts.
version: 1.0.0
tags:
source: analyzing-linux-system-artifacts

Analyzing Linux System Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Key Artifact Locations, last / lastb, chkrootkit, rkhunter, auditd Log Parsing, Plaso / log2timeline

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** linux system artifacts.
- Capability: Linux system artifact forensics agent for investigating compromised systems.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, linux, system, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze /etc/passwd for suspicious accounts
2. Analyze /etc/shadow for password hash types and status
3. Analyze bash history for suspicious commands
4. Check cron jobs for persistence mechanisms
5. Check for unauthorized SSH authorized_keys
6. Check for suspicious systemd service files
7. Check for LD_PRELOAD rootkit indicators
8. Find SUID/SGID binaries (potential privilege escalation)

Tools & resources

- Key Artifact Locations
- last / lastb
- chkrootkit
- rkhunter
- auditd Log Parsing
- Plaso / log2timeline
- AIDE

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_passwd` — Analyze /etc/passwd for suspicious accounts.
- `analyze_shadow` — Analyze /etc/shadow for password hash types and status.
- `analyze_bash_history` — Analyze bash history for suspicious commands.
- `check_cron_persistence` — Check cron jobs for persistence mechanisms.
- `check_ssh_keys` — Check for unauthorized SSH authorized_keys.
- `check_systemd_persistence` — Check for suspicious systemd service files.
- `check_ld_preload` — Check for LD_PRELOAD rootkit indicators.
- `find_suid_binaries` — Find SUID/SGID binaries (potential privilege escalation).
- `find_suspicious_tmp_files` — Find suspicious files in /tmp and /dev/shm.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-linux-system-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-lnk-file-and-jump-list-artifacts (Skill Module)

analyzing-lnk-file-and-jump-list-artifacts\SKILL.md

name: analyzing-lnk-file-and-jump-list-artifacts
description: Windows LNK file and Jump List artifact analysis agent. Parses Windows Shell Link (.lnk) files and Jump List artifacts to extract file access evidence, program execution history, and user activity timelines. Uses.... Use when the user asks to analyze lnk file and jump list artifacts.
version: 1.0.0
tags:
source: analyzing-lnk-file-and-jump-list-artifacts

Analyzing Lnk File and Jump List Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: LECmd (Eric Zimmerman), LnkParse3 (Python), Magnet AXIOM

When to use

- The user wants to **analyze** lnk file and jump list artifacts.
- Capability: Windows LNK file and Jump List artifact analysis agent. Parses Windows Shell Link (.lnk) files and Jump List artifacts to extract file access evidence, program execution history, and user activity timelines. Uses LnkParse3 for binary parsing and supports LECmd/JLECmd CSV output analysis.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, file, jump, list, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Compute SHA-256 hash of file
2. Parse LNK file using LnkParse3 library
3. Parse LNK file header manually from raw bytes
4. Convert Windows FILETIME to ISO string
5. Decode Shell Link header flags
6. Scan Jump List directory for automatic and custom destinations
7. Detect suspicious characteristics in LNK files
8. Scan directory for LNK files and analyze each

Key concepts

- MS-SHLLINK: Shell Link Binary File Format (Microsoft Open Specifications)
- NIST SP 800-86: Guide to Integrating Forensic Techniques
- SWGDE Best Practices for Computer Forensics
- LECmd (Eric Zimmerman): LNK file parser
- JLECmd (Eric Zimmerman): Jump List parser
- LnkParse3 (Python): Cross-platform LNK parser
- Magnet AXIOM: Commercial forensic tool with LNK/Jump List support
- Recent files: %APPDATA%\Microsoft\Windows\Recent\

Tools & resources

- LECmd (Eric Zimmerman)
- LnkParse3 (Python)
- Magnet AXIOM

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compute_hash` — Compute SHA-256 hash of file.
- `parse_lnk_with_lnkparse3` — Parse LNK file using LnkParse3 library.
- `parse_lnk_header_raw` — Parse LNK file header manually from raw bytes.
- `filetime_to_datetime` — Convert Windows FILETIME to ISO string.
- `decode_link_flags` — Decode Shell Link header flags.
- `scan_jump_lists` — Scan Jump List directory for automatic and custom destinations.
- `detect_suspicious_lnk` — Detect suspicious characteristics in LNK files.
- `scan_lnk_directory` — Scan directory for LNK files and analyze each.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-lnk-file-and-jump-list-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-macro-malware-in-office-documents (Skill Module)

analyzing-macro-malware-in-office-documents\SKILL.md

name: analyzing-macro-malware-in-office-documents
description: Office macro malware analysis agent using oletools for VBA extraction and deobfuscation. Use when the user asks to analyze macro malware in office documents.
version: 1.0.0
tags:
source: analyzing-macro-malware-in-office-documents

Analyzing Macro Malware in Office Documents

Role: Deep analysis
Audience: analysts and investigators
Primary tools: olevba, oleid, oledump.py, XLMDeobfuscator, VBA Auto-Execution Triggers, VBA Suspicious Functions

When to use

- The user wants to **analyze** macro malware in office documents.
- Capability: Office macro malware analysis agent using oletools for VBA extraction and deobfuscation.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, macro, malware, office, documents`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a file
2. Quick triage using oleid to identify document capabilities
3. Extract VBA macro code from an Office document
4. Analyze VBA macros for suspicious keywords and patterns
5. Resolve Chr() and ChrW() calls in VBA code
6. Remove string concatenation: "abc" & "def" -> "abcdef"
7. Resolve StrReverse() calls
8. Resolve Replace() function calls

Tools & resources

- olevba
- oleid
- oledump.py
- XLMDeobfuscator
- VBA Auto-Execution Triggers
- VBA Suspicious Functions
- ViperMonkey

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hash` — Compute SHA-256 hash of a file.
- `triage_document` — Quick triage using oleid to identify document capabilities.
- `extract_vba_macros` — Extract VBA macro code from an Office document.
- `analyze_vba_suspicious` — Analyze VBA macros for suspicious keywords and patterns.
- `deobfuscate_chr_calls` — Resolve Chr() and ChrW() calls in VBA code.
- `deobfuscate_concatenation` — Remove string concatenation: "abc" & "def" -> "abcdef".
- `deobfuscate_strreverse` — Resolve StrReverse() calls.
- `deobfuscate_replace` — Resolve Replace() function calls.
- `full_deobfuscation` — Apply all deobfuscation techniques to VBA code.
- `extract_urls_from_code` — Extract URLs from deobfuscated VBA code.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-macro-malware-in-office-documents`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-malicious-pdf-with-peepdf (Skill Module)

analyzing-malicious-pdf-with-peepdf\SKILL.md

name: analyzing-malicious-pdf-with-peepdf
description: Malicious PDF Analysis Agent - static analysis using peepdf, pdfid, and pdf-parser for threat detection. Use when the user asks to analyze malicious pdf with peepdf.
version: 1.0.0
tags:
source: analyzing-malicious-pdf-with-peepdf

Analyzing Malicious PDF with peepdf

Role: Deep analysis
Audience: analysts and investigators
Primary tools: peepdf, pdfid.py Usage, pdf-parser.py Usage

When to use

- The user wants to **analyze** malicious pdf with peepdf.
- Capability: Malicious PDF Analysis Agent - static analysis using peepdf, pdfid, and pdf-parser for threat detection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malicious, peepdf`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute MD5 and SHA-256 hashes of the PDF file
2. Run pdfid.py to triage PDF for suspicious keywords
3. Run peepdf for detailed PDF object analysis
4. Run pdf-parser.py to extract specific objects
5. Extract JavaScript content from identified objects
6. Detect common shellcode patterns in extracted content
7. Calculate overall risk score for the PDF
8. Generate comprehensive PDF malware analysis report

Tools & resources

- peepdf
- pdfid.py Usage
- pdf-parser.py Usage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Compute MD5 and SHA-256 hashes of the PDF file.
- `run_pdfid` — Run pdfid.py to triage PDF for suspicious keywords.
- `run_peepdf_analysis` — Run peepdf for detailed PDF object analysis.
- `run_pdf_parser` — Run pdf-parser.py to extract specific objects.
- `extract_javascript` — Extract JavaScript content from identified objects.
- `detect_shellcode_patterns` — Detect common shellcode patterns in extracted content.
- `calculate_risk_score` — Calculate overall risk score for the PDF.
- `generate_report` — Generate comprehensive PDF malware analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-malicious-pdf-with-peepdf`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-malicious-url-with-urlscan (Skill Module)

analyzing-malicious-url-with-urlscan\SKILL.md

name: analyzing-malicious-url-with-urlscan
description: URLScan.io Malicious URL Analysis Agent - Submits and analyzes URLs via the urlscan.io API. Use when the user asks to analyze malicious url with urlscan.
version: 1.0.0
tags:
source: analyzing-malicious-url-with-urlscan

Analyzing Malicious URL with urlscan.io

Role: Deep analysis
Audience: analysts and investigators
Primary tools: urlscan.io, Base URL, Submit Scan, Get Result, Search, Result Structure

When to use

- The user wants to **analyze** malicious url with urlscan.
- Capability: URLScan.io Malicious URL Analysis Agent - Submits and analyzes URLs via the urlscan.io API.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malicious, urlscan`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Submit a URL to urlscan.io for scanning
2. Poll for scan results until complete
3. Search urlscan.io for existing scans
4. Analyze urlscan.io scan result for malicious indicators
5. Submit and analyze multiple URLs
6. Generate URL analysis report

Key concepts

- T1566.002: Phishing: Spearphishing Link
- T1204.001: User Execution: Malicious Link
- T1608.005: Stage Capabilities: Link Target
- T1071.001: Application Layer Protocol: Web Protocols
- T1102: Web Service (for C2 via web)
- NIST SP 800-83: Guide to Malware Incident Prevention and Handling
- NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response
- RFC 3986: Uniform Resource Identifier (URI) syntax

Tools & resources

- urlscan.io
- Base URL
- Submit Scan
- Get Result
- Search
- Result Structure
- Screenshot
- DOM Snapshot

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `submit_url` — Submit a URL to urlscan.io for scanning.
- `get_scan_result` — Poll for scan results until complete.
- `search_urlscan` — Search urlscan.io for existing scans.
- `analyze_result` — Analyze urlscan.io scan result for malicious indicators.
- `bulk_analyze` — Submit and analyze multiple URLs.
- `generate_report` — Generate URL analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-malicious-url-with-urlscan`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-malware-behavior-with-cuckoo-sandbox (Skill Module)

analyzing-malware-behavior-with-cuckoo-sandbox\SKILL.md

name: analyzing-malware-behavior-with-cuckoo-sandbox
description: Cuckoo Sandbox behavioral analysis agent for automated malware detonation and reporting. Use when the user asks to analyze malware behavior with cuckoo sandbox.
version: 1.0.0
tags:
source: analyzing-malware-behavior-with-cuckoo-sandbox

Analyzing Malware Behavior with Cuckoo Sandbox

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Cuckoo Sandbox, Cuckoo CLI, Cuckoo REST API, Analysis Packages, InetSim, FakeNet-NG

When to use

- The user wants to **analyze** malware behavior with cuckoo sandbox.
- Capability: Cuckoo Sandbox behavioral analysis agent for automated malware detonation and reporting.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malware, behavior, cuckoo, sandbox`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Submit a malware sample to Cuckoo via REST API
2. Submit a URL to Cuckoo for analysis
3. Check the status of a Cuckoo analysis task
4. Load a Cuckoo JSON report from disk
5. Extract and analyze the process tree from the Cuckoo report
6. Extract network activity from the Cuckoo report
7. Extract dropped file information from the report
8. Extract triggered behavioral signatures

Tools & resources

- Cuckoo Sandbox
- Cuckoo CLI
- Cuckoo REST API
- Analysis Packages
- InetSim
- FakeNet-NG

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `submit_file` — Submit a malware sample to Cuckoo via REST API.
- `submit_url` — Submit a URL to Cuckoo for analysis.
- `get_task_status` — Check the status of a Cuckoo analysis task.
- `load_report` — Load a Cuckoo JSON report from disk.
- `analyze_processes` — Extract and analyze the process tree from the Cuckoo report.
- `analyze_network` — Extract network activity from the Cuckoo report.
- `analyze_dropped_files` — Extract dropped file information from the report.
- `analyze_signatures` — Extract triggered behavioral signatures.
- `analyze_registry` — Extract registry modifications from behavior summary.
- `generate_summary` — Generate a consolidated analysis summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-malware-behavior-with-cuckoo-sandbox`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-malware-family-relationships-with-malpedia (Skill Module)

analyzing-malware-family-relationships-with-malpedia\SKILL.md

name: analyzing-malware-family-relationships-with-malpedia
description: Malpedia Malware Family Relationship Agent - Queries Malpedia API for malware family intelligence. Use when the user asks to analyze malware family relationships with malpedia.
version: 1.0.0
tags:
source: analyzing-malware-family-relationships-with-malpedia

Analyzing Malware Family Relationships with Malpedia

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Malpedia, Base URL, List Families, Get Family Details, Get YARA Rules, List Actors

When to use

- The user wants to **analyze** malware family relationships with malpedia.
- Capability: Malpedia Malware Family Relationship Agent - Queries Malpedia API for malware family intelligence.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malware, family, relationships, malpedia`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Make authenticated GET request to Malpedia API
2. List all malware families from Malpedia
3. Get detailed info for a malware family
4. Get YARA rules for a malware family
5. List all threat actors from Malpedia
6. Get detailed info for a threat actor
7. Build relationship graph between malware families
8. Analyze a specific malware family and its relationships

Tools & resources

- Malpedia
- Base URL
- List Families
- Get Family Details
- Get YARA Rules
- List Actors
- Get Actor Details
- Get Sample

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `malpedia_get` — Make authenticated GET request to Malpedia API.
- `list_families` — List all malware families from Malpedia.
- `get_family_info` — Get detailed info for a malware family.
- `get_family_yara` — Get YARA rules for a malware family.
- `list_actors` — List all threat actors from Malpedia.
- `get_actor_info` — Get detailed info for a threat actor.
- `build_family_graph` — Build relationship graph between malware families.
- `analyze_family` — Analyze a specific malware family and its relationships.
- `generate_report` — Generate malware family relationship report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-malware-family-relationships-with-malpedia`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-malware-persistence-with-autoruns (Skill Module)

`analyzing-malware-persistence-with-autoruns\SKILL.md`

name: analyzing-malware-persistence-with-autoruns
description: Autoruns Persistence Analysis Agent - Analyzes Windows autostart entries for malware persistence. Use when the user asks to analyze malware persistence with autoruns.
version: 1.0.0
tags:
source: analyzing-malware-persistence-with-autoruns

Analyzing Malware Persistence with Autoruns

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Autoruns, CSV Columns, Autostart Categories, Applicable Standards

When to use

- The user wants to **analyze** malware persistence with autoruns.
- Capability: Autoruns Persistence Analysis Agent - Analyzes Windows autostart entries for malware persistence.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malware, persistence, autoruns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse Autoruns CSV export file
2. Analyze a single autoruns entry for suspicious indicators
3. Analyze all autoruns entries and generate findings
4. Generate persistence analysis report

Key concepts

- MITRE ATT&CK Framework
- NIST SP 800-83 Guide to Malware Incident Prevention
- NIST SP 800-86 Guide to Integrating Forensic Techniques

Tools & resources

- Autoruns
- CSV Columns
- Autostart Categories
- Applicable Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_autoruns_csv` — Parse Autoruns CSV export file.
- `analyze_entry` — Analyze a single autoruns entry for suspicious indicators.
- `analyze_all_entries` — Analyze all autoruns entries and generate findings.
- `generate_report` — Generate persistence analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-malware-persistence-with-autoruns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-malware-sandbox-evasion-techniques (Skill Module)

analyzing-malware-sandbox-evasion-techniques\SKILL.md

name: analyzing-malware-sandbox-evasion-techniques
description: Analyze malware sandbox evasion techniques from Cuckoo/AnyRun behavioral reports. Use when the user asks to analyze malware sandbox evasion techniques.
version: 1.0.0
tags:
source: analyzing-malware-sandbox-evasion-techniques

Analyzing Malware Sandbox Evasion Techniques

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Timing API Indicators, VM Artifact Indicators

When to use

- The user wants to **analyze** malware sandbox evasion techniques.
- Capability: Analyze malware sandbox evasion techniques from Cuckoo/AnyRun behavioral reports.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malware, sandbox, evasion, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse a Cuckoo Sandbox behavioral report JSON
2. Detect timing-based sandbox evasion via GetTickCount, QPC, etc
3. Detect sleep calls with long durations used to evade sandbox time limits
4. Detect VM artifact queries (registry, processes, MAC addresses)
5. Detect checks for user interaction (mouse, keyboard, foreground window)
6. Score evasion sophistication based on technique diversity

Tools & resources

- Timing API Indicators
- VM Artifact Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_cuckoo_report` — Parse a Cuckoo Sandbox behavioral report JSON.
- `detect_timing_checks` — Detect timing-based sandbox evasion via GetTickCount, QPC, etc.
- `detect_sleep_inflation` — Detect sleep calls with long durations used to evade sandbox time limits.
- `detect_vm_artifact_checks` — Detect VM artifact queries (registry, processes, MAC addresses).
- `detect_user_interaction_checks` — Detect checks for user interaction (mouse, keyboard, foreground window).
- `score_evasion_sophistication` — Score evasion sophistication based on technique diversity.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-malware-sandbox-evasion-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-memory-dumps-with-volatility (Skill Module)

`analyzing-memory-dumps-with-volatility\SKILL.md`

name: analyzing-memory-dumps-with-volatility
description: Memory forensics agent using Volatility 3 for malware detection in RAM dumps. Use when the user asks to analyze memory dumps with volatility.
version: 1.0.0
tags:
source: analyzing-memory-dumps-with-volatility

Analyzing Memory Dumps with Volatility

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Volatility, Windows Plugins, Memory Acquisition Tools, Symbols

When to use

- The user wants to **analyze** memory dumps with volatility.
- Capability: Memory forensics agent using Volatility 3 for malware detection in RAM dumps.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, memory, dumps, volatility`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute a Volatility 3 plugin and return output
2. Identify the OS from the memory dump
3. List all running processes using pslist
4. Scan for hidden/unlinked processes using psscan
5. Compare pslist and psscan to identify DKOM-hidden processes
6. Detect injected code using malfind plugin
7. Extract network connections using netscan
8. Extract process command lines

Tools & resources

- Volatility
- Windows Plugins
- Memory Acquisition Tools
- Symbols

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_vol3` — Execute a Volatility 3 plugin and return output.
- `get_os_info` — Identify the OS from the memory dump.
- `list_processes` — List all running processes using pslist.
- `scan_hidden_processes` — Scan for hidden/unlinked processes using psscan.
- `find_hidden_processes` — Compare pslist and psscan to identify DKOM-hidden processes.
- `detect_code_injection` — Detect injected code using malfind plugin.
- `get_network_connections` — Extract network connections using netscan.
- `get_command_lines` — Extract process command lines.
- `dump_credentials` — Extract cached credentials using hashdump and lsadump.
- `scan_with_yara` — Scan memory with YARA rules.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-memory-dumps-with-volatility`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-memory-forensics-with-lime-and-volatility (Skill Module)

analyzing-memory-forensics-with-lime-and-volatility\SKILL.md

name: analyzing-memory-forensics-with-lime-and-volatility
description: Agent for Linux memory forensics using LiME acquisition and Volatility 3. Use when the user asks to analyze memory forensics with lime and volatility.
version: 1.0.0
tags:
source: analyzing-memory-forensics-with-lime-and-volatility

Analyzing Memory Forensics with Lime and Volatility

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Volatility, LiME, Hidden Process Detection

When to use

- The user wants to **analyze** memory forensics with lime and volatility.
- Capability: Agent for Linux memory forensics using LiME acquisition and Volatility 3.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, memory, forensics, lime, volatility`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Acquire memory using LiME kernel module
2. Run a Volatility 3 plugin and capture output
3. Parse Volatility linux.pslist output into structured data
4. List all processes from memory image
5. Extract bash command history from memory
6. List network connections from memory
7. List loaded kernel modules to detect rootkits
8. Compare pslist vs psscan to find hidden processes

Tools & resources

- Volatility
- LiME
- Hidden Process Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `acquire_memory_lime` — Acquire memory using LiME kernel module.
- `run_vol3_plugin` — Run a Volatility 3 plugin and capture output.
- `parse_pslist_output` — Parse Volatility linux.pslist output into structured data.
- `list_processes` — List all processes from memory image.
- `extract_bash_history` — Extract bash command history from memory.
- `list_network_connections` — List network connections from memory.
- `list_kernel_modules` — List loaded kernel modules to detect rootkits.
- `detect_hidden_processes` — Compare pslist vs psscan to find hidden processes.
- `detect_suspicious_commands` — Flag suspicious commands in bash history.
- `check_malfind` — Run malfind to detect injected code.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-memory-forensics-with-lime-and-volatility`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-mft-for-deleted-file-recovery (Skill Module)

`analyzing-mft-for-deleted-file-recovery\SKILL.md`

name: analyzing-mft-for-deleted-file-recovery
description: MFT Deleted File Recovery Agent - Parses NTFS Master File Table for deleted file artifacts. Use when the user asks to analyze mft for deleted file recovery.
version: 1.0.0
tags:
source: analyzing-mft-for-deleted-file-recovery

Analyzing MFT for Deleted File Recovery

Role: Deep analysis
Audience: analysts and investigators
Primary tools: MFT Attribute Types, Tools, Standards, Key Technical References, NTFS Specifications

When to use

- The user wants to **analyze** mft for deleted file recovery.
- Capability: MFT Deleted File Recovery Agent - Parses NTFS Master File Table for deleted file artifacts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, deleted, file, recovery`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Convert FILETIME to datetime
2. Parse a single MFT entry
3. Parse an extracted MFT file
4. Find deleted file entries in MFT
5. Analyze deleted files for forensic significance
6. Generate MFT analysis report

Key concepts

- NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response
- ISO/IEC 27037: Guidelines for identification, collection, acquisition and preservation of digital evidence
- SWGDE Best Practices for Computer Forensics
- NTFS Documentation (Microsoft): File system internals and MFT structure
- MFTECmd by Eric Zimmerman: Primary parsing tool for $MFT, $J, $LogFile, $Boot
- analyzeMFT (Python): Open-source MFT parser for cross-platform analysis
- ntfstool (GitHub): Forensics tool for NTFS parsing, MFT, BitLocker, deleted files
- T1070.004 - Indicator Removal: File Deletion

Tools & resources

- MFT Attribute Types
- Tools
- Standards
- Key Technical References
- NTFS Specifications

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `filetime_to_dt` — Convert FILETIME to datetime.
- `parse_mft_entry` — Parse a single MFT entry.
- `parse_mft_file` — Parse an extracted MFT file.
- `find_deleted_files` — Find deleted file entries in MFT.
- `analyze_deleted_files` — Analyze deleted files for forensic significance.
- `generate_report` — Generate MFT analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-mft-for-deleted-file-recovery`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-network-covert-channels-in-malware (Skill Module)

analyzing-network-covert-channels-in-malware\SKILL.md

name: analyzing-network-covert-channels-in-malware
description: Network covert channel detection agent for malware traffic analysis. Detects DNS tunneling, ICMP covert channels, HTTP header steganography, and protocol abuse in PCAP captures using scapy. Use when the user asks to analyze network covert channels in malware.
version: 1.0.0
tags:
source: analyzing-network-covert-channels-in-malware

Analyzing Network Covert Channels in Malware

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Scapy, Zeek, tshark, DNS Tunneling Tools, Entropy Thresholds, Covert Channel Categories

When to use

- The user wants to **analyze** network covert channels in malware.
- Capability: Network covert channel detection agent for malware traffic analysis. Detects DNS tunneling, ICMP covert channels, HTTP header steganography, and protocol abuse in PCAP captures using scapy.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, network, covert, channels, malware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Calculate Shannon entropy of byte data
2. Detect DNS tunneling by analyzing query name entropy and length
3. Detect ICMP covert channels via payload analysis
4. Detect covert data in HTTP headers
5. Detect protocol-level anomalies indicating covert communication
6. Generate covert channel analysis report

Key concepts

- MITRE ATT&CK Framework
- NIST SP 800-83 Guide to Malware Incident Prevention
- NIST SP 800-86 Guide to Integrating Forensic Techniques

Tools & resources

- Scapy
- Zeek
- tshark
- DNS Tunneling Tools
- Entropy Thresholds
- Covert Channel Categories
- Applicable Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of byte data.
- `detect_dns_tunneling` — Detect DNS tunneling by analyzing query name entropy and length.
- `detect_icmp_covert_channel` — Detect ICMP covert channels via payload analysis.
- `detect_http_header_covert` — Detect covert data in HTTP headers.
- `detect_protocol_anomalies` — Detect protocol-level anomalies indicating covert communication.
- `generate_report` — Generate covert channel analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-network-covert-channels-in-malware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-network-flow-data-with-netflow (Skill Module)

analyzing-network-flow-data-with-netflow\SKILL.md

name: analyzing-network-flow-data-with-netflow
description: NetFlow Analysis Agent - Parses NetFlow v9/IPFIX for anomalies, port scans, and exfiltration. Use when the user asks to analyze network flow data with netflow.
version: 1.0.0
tags:
source: analyzing-network-flow-data-with-netflow

Analyzing Network Flow Data with Netflow

Role: Deep analysis
Audience: analysts and investigators
Primary tools: NetFlow, CLI Tools, Detection Algorithms

When to use

- The user wants to **analyze** network flow data with netflow.
- Capability: NetFlow Analysis Agent - Parses NetFlow v9/IPFIX for anomalies, port scans, and exfiltration.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, network, flow, data, netflow`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load preprocessed flow records from JSON file
2. Parse NetFlow packets from a PCAP capture using the netflow library
3. Detect port scanning: one source hitting many ports on same or multiple destinations
4. Detect potential data exfiltration via high-volume outbound flows
5. Detect C2 beaconing patterns via periodic connection analysis
6. Build statistical baseline of network traffic
7. Generate NetFlow analysis report

Tools & resources

- NetFlow
- CLI Tools
- Detection Algorithms

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_flow_data` — Load preprocessed flow records from JSON file.
- `parse_netflow_capture` — Parse NetFlow packets from a PCAP capture using the netflow library.
- `detect_port_scanning` — Detect port scanning: one source hitting many ports on same or multiple destinations.
- `detect_data_exfiltration` — Detect potential data exfiltration via high-volume outbound flows.
- `detect_beaconing` — Detect C2 beaconing patterns via periodic connection analysis.
- `build_traffic_baseline` — Build statistical baseline of network traffic.
- `generate_report` — Generate NetFlow analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-network-flow-data-with-netflow`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-network-packets-with-scapy (Skill Module)

analyzing-network-packets-with-scapy\SKILL.md

name: analyzing-network-packets-with-scapy
description: Network packet analysis agent using Scapy for pcap parsing and anomaly detection. Use when the user asks to analyze network packets with scapy.
version: 1.0.0
tags:
source: analyzing-network-packets-with-scapy

Analyzing Network Packets with Scapy

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Scapy, Protocol Fields Reference

When to use

- The user wants to **analyze** network packets with scapy.
- Capability: Network packet analysis agent using Scapy for pcap parsing and anomaly detection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, network, packets, scapy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load packets from a pcap/pcapng file
2. Extract structured info from each packet with IP layer
3. Compute overall traffic statistics
4. Detect SYN flood by counting SYN-only packets per destination IP
5. Calculate Shannon entropy of a string
6. Detect DNS tunneling via long/high-entropy query names
7. Detect port scanning by counting unique destination ports per source IP

Tools & resources

- Scapy
- Protocol Fields Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_pcap` — Load packets from a pcap/pcapng file.
- `extract_packet_info` — Extract structured info from each packet with IP layer.
- `compute_traffic_stats` — Compute overall traffic statistics.
- `detect_syn_flood` — Detect SYN flood by counting SYN-only packets per destination IP.
- `calculate_entropy` — Calculate Shannon entropy of a string.
- `detect_dns_tunneling` — Detect DNS tunneling via long/high-entropy query names.
- `detect_port_scan` — Detect port scanning by counting unique destination ports per source IP.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-network-packets-with-scapy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-network-traffic-for-incidents (Skill Module)

analyzing-network-traffic-for-incidents\SKILL.md

name: analyzing-network-traffic-for-incidents
description: Network traffic incident analysis agent using scapy and tshark for PCAP investigation. Use when the user asks to analyze network traffic for incidents.
version: 1.0.0
tags:
source: analyzing-network-traffic-for-incidents

Analyzing Network Traffic for Incidents

Role: Deep analysis
Audience: analysts and investigators
Primary tools: tshark, Zeek, Suricata - IDS/IPS, Lateral Movement Ports, Scapy, NetworkMiner

When to use

- The user wants to **analyze** network traffic for incidents.
- Capability: Network traffic incident analysis agent using scapy and tshark for PCAP investigation.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, network, traffic, incidents`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run tshark with a display filter and extract specific fields
2. Get high-level PCAP statistics
3. Detect potential lateral movement patterns (SMB, RDP, WinRM, SSH)
4. Detect potential data exfiltration based on outbound data volume
5. Detect periodic beaconing patterns from TCP connections
6. Extract DNS queries and identify suspicious patterns
7. Run Suricata on the PCAP and extract alerts
8. Extract HTTP objects (files) from the PCAP

Tools & resources

- tshark
- Zeek
- Suricata - IDS/IPS
- Lateral Movement Ports
- Scapy
- NetworkMiner

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_tshark` — Run tshark with a display filter and extract specific fields.
- `get_pcap_summary` — Get high-level PCAP statistics.
- `detect_lateral_movement` — Detect potential lateral movement patterns (SMB, RDP, WinRM, SSH).
- `detect_data_exfiltration` — Detect potential data exfiltration based on outbound data volume.
- `detect_beaconing` — Detect periodic beaconing patterns from TCP connections.
- `extract_dns_queries` — Extract DNS queries and identify suspicious patterns.
- `detect_ids_alerts` — Run Suricata on the PCAP and extract alerts.
- `extract_http_objects` — Extract HTTP objects (files) from the PCAP.
- `generate_incident_report` — Generate a network incident analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-network-traffic-for-incidents`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-network-traffic-of-malware (Skill Module)

analyzing-network-traffic-of-malware\SKILL.md

name: analyzing-network-traffic-of-malware
description: Malware network traffic analysis agent for C2 protocol decoding and signature generation. Use when the user asks to analyze network traffic of malware.
version: 1.0.0
tags:
source: analyzing-network-traffic-of-malware

Analyzing Network Traffic of Malware

Role: Deep analysis
Audience: analysts and investigators
Primary tools: dpkt, tshark, RITA, NetworkMiner

When to use

- The user wants to **analyze** network traffic of malware.
- Capability: Malware network traffic analysis agent for C2 protocol decoding and signature generation.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, network, traffic, malware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of byte data
2. Extract TCP stream payloads grouped by conversation
3. Analyze payload structure to identify protocol framing
4. Detect DNS tunneling in malware traffic
5. Detect DGA (Domain Generation Algorithm) domains
6. Extract HTTP-based C2 communication patterns
7. Generate Suricata IDS signatures from observed malware network patterns

Tools & resources

- dpkt
- tshark
- RITA
- NetworkMiner

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of byte data.
- `extract_tcp_streams` — Extract TCP stream payloads grouped by conversation.
- `analyze_payload_structure` — Analyze payload structure to identify protocol framing.
- `detect_dns_tunneling` — Detect DNS tunneling in malware traffic.
- `detect_dga_domains` — Detect DGA (Domain Generation Algorithm) domains.
- `extract_http_c2` — Extract HTTP-based C2 communication patterns.
- `generate_suricata_signatures` — Generate Suricata IDS signatures from observed malware network patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-network-traffic-of-malware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-network-traffic-with-wireshark (Skill Module)

`analyzing-network-traffic-with-wireshark\SKILL.md`

name: analyzing-network-traffic-with-wireshark
description: Wireshark/tshark packet analysis agent for network security investigations. Use when the user asks to analyze network traffic with wireshark.
version: 1.0.0
tags:
source: analyzing-network-traffic-with-wireshark

Analyzing Network Traffic with Wireshark

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Wireshark, Live Capture, Field Extraction, Statistics, Object Export, Stream Following

When to use

- The user wants to **analyze** network traffic with wireshark.
- Capability: Wireshark/tshark packet analysis agent for network security investigations.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, network, traffic, wireshark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute tshark with custom arguments
2. Start a live packet capture using tshark
3. Get overall PCAP capture statistics
4. Get protocol hierarchy statistics
5. Get conversation statistics (ip, tcp, udp, ethernet)
6. Get endpoint statistics
7. Extract HTTP requests with key fields
8. Extract DNS queries and responses

Tools & resources

- Wireshark
- Live Capture
- Field Extraction
- Statistics
- Object Export
- Stream Following
- editcap
- mergecap

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_tshark` — Execute tshark with custom arguments.
- `capture_live` — Start a live packet capture using tshark.
- `get_capture_summary` — Get overall PCAP capture statistics.
- `get_protocol_hierarchy` — Get protocol hierarchy statistics.
- `get_conversations` — Get conversation statistics (ip, tcp, udp, ethernet).
- `get_endpoints` — Get endpoint statistics.
- `extract_http_requests` — Extract HTTP requests with key fields.
- `extract_dns_queries` — Extract DNS queries and responses.
- `extract_tls_info` — Extract TLS handshake information including JA3 fingerprints.
- `detect_suspicious_traffic` — Detect common suspicious traffic patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-network-traffic-with-wireshark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-office365-audit-logs-for-compromise (Skill Module)

`analyzing-office365-audit-logs-for-compromise\SKILL.md`

name: analyzing-office365-audit-logs-for-compromise
description: Agent for analyzing Office 365 audit logs for compromise indicators via Microsoft Graph. Use when the user asks to analyze office365 audit logs for compromise.
version: 1.0.0
tags:
source: analyzing-office365-audit-logs-for-compromise

Analyzing Office365 Audit Logs for Compromise

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Suspicious Operations Monitored

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** office365 audit logs for compromise.
- Capability: Agent for analyzing Office 365 audit logs for compromise indicators via Microsoft Graph.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, office365, audit, logs, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate via MSAL client credentials flow
2. Paginated GET against Microsoft Graph API
3. Query Unified Audit Log for suspicious mail operations
4. List inbox rules for a specific mailbox and flag forwarding rules
5. Check mailbox-level SMTP forwarding configuration
6. Check for suspicious OAuth application consent grants
7. Run comprehensive O365 compromise audit

Tools & resources

- Suspicious Operations Monitored

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_access_token` — Authenticate via MSAL client credentials flow.
- `graph_get` — Paginated GET against Microsoft Graph API.
- `query_audit_logs` — Query Unified Audit Log for suspicious mail operations.
- `check_inbox_rules` — List inbox rules for a specific mailbox and flag forwarding rules.
- `check_mailbox_forwarding` — Check mailbox-level SMTP forwarding configuration.
- `check_oauth_grants` — Check for suspicious OAuth application consent grants.
- `full_audit` — Run comprehensive O365 compromise audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-office365-audit-logs-for-compromise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-outlook-pst-for-email-forensics (Skill Module)

`analyzing-outlook-pst-for-email-forensics\SKILL.md`

name: analyzing-outlook-pst-for-email-forensics
description: Outlook PST file forensic analysis agent. Parses PST/OST files using pypff (libpff) to extract emails, attachments, metadata, and deleted items for forensic investigation. Use when the user asks to analyze outlook pst for email forensics.
version: 1.0.0
tags:
source: analyzing-outlook-pst-for-email-forensics

Analyzing Outlook Pst for Email Forensics

Role: Deep analysis
Audience: analysts and investigators
Primary tools: libpff/pffexport, pypff (Python), MailXaminer, PST Walker, Kernel Outlook PST Viewer

When to use

- The user wants to **analyze** outlook pst for email forensics.
- Capability: Outlook PST file forensic analysis agent. Parses PST/OST files using pypff (libpff) to extract emails, attachments, metadata, and deleted items for forensic investigation.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, outlook, email, forensics`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Ingest and validate inputs for outlook pst for email forensics
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

Key concepts

- MS-PST: Outlook Personal Folders (.pst) File Format
- MS-OXMSG: Outlook Item Message File Format
- NIST SP 800-86: Guide to Integrating Forensic Techniques
- libpff/pffexport: Open-source PST parser
- pypff (Python): Python bindings for libpff
- MailXaminer: Commercial email forensics
- PST Walker: Email investigation software
- Kernel Outlook PST Viewer: Free PST reader

Tools & resources

- libpff/pffexport
- pypff (Python)
- MailXaminer
- PST Walker
- Kernel Outlook PST Viewer

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hash`
- `open_pst`
- `extract_messages`
- `walk_folders`
- `extract_email_addresses`
- `detect_suspicious_emails`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-outlook-pst-for-email-forensics`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-packed-malware-with-upx-unpacker (Skill Module)

`analyzing-packed-malware-with-upx-unpacker\SKILL.md`

name: analyzing-packed-malware-with-upx-unpacker
description: Packed malware analysis agent for UPX and generic packer detection and unpacking. Use when the user asks to analyze packed malware with upx unpacker.
version: 1.0.0
tags:
source: analyzing-packed-malware-with-upx-unpacker

Analyzing Packed Malware with UPX Unpacker

Role: Deep analysis
Audience: analysts and investigators
Primary tools: UPX, pefile, PEiD, PEStudio, x64dbg / x32dbg, Entropy Interpretation

When to use

- The user wants to **analyze** packed malware with upx unpacker.
- Capability: Packed malware analysis agent for UPX and generic packer detection and unpacking.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, packed, malware, unpacker`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute file hashes
2. Calculate Shannon entropy of binary data
3. Check for UPX packing signatures in the binary
4. Detect generic packing indicators using PE section analysis
5. Attempt to unpack a UPX-packed binary
6. Attempt to fix corrupted UPX magic bytes for unpacking
7. Compare packed vs unpacked binary properties

Tools & resources

- UPX
- pefile
- PEiD
- PEStudio
- x64dbg / x32dbg
- Entropy Interpretation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Compute file hashes.
- `calculate_entropy` — Calculate Shannon entropy of binary data.
- `detect_upx` — Check for UPX packing signatures in the binary.
- `detect_generic_packing` — Detect generic packing indicators using PE section analysis.
- `unpack_upx` — Attempt to unpack a UPX-packed binary.
- `fix_upx_headers` — Attempt to fix corrupted UPX magic bytes for unpacking.
- `compare_packed_unpacked` — Compare packed vs unpacked binary properties.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-packed-malware-with-upx-unpacker`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-pdf-malware-with-pdfid (Skill Module)

analyzing-pdf-malware-with-pdfid\SKILL.md

name: analyzing-pdf-malware-with-pdfid
description: PDF malware analysis agent using pdfid concepts and pdf-parser for object extraction. Use when the user asks to analyze pdf malware with pdfid.
version: 1.0.0
tags:
source: analyzing-pdf-malware-with-pdfid

Analyzing PDF Malware with pdfid

Role: Deep analysis
Audience: analysts and investigators
Primary tools: pdfid, pdf-parser.py, peepdf, Known PDF Exploit CVEs

When to use

- The user wants to **analyze** pdf malware with pdfid.
- Capability: PDF malware analysis agent using pdfid concepts and pdf-parser for object extraction.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, malware, pdfid`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a file
2. Scan a PDF file for suspicious keywords similar to pdfid
3. Extract the PDF version from the header
4. Find and extract stream objects from the PDF
5. Extract JavaScript code from PDF objects
6. Extract URLs from the PDF content
7. Check for known PDF exploit indicators
8. Calculate a risk score for the PDF

Tools & resources

- pdfid
- pdf-parser.py
- peepdf
- Known PDF Exploit CVEs

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hash` — Compute SHA-256 hash of a file.
- `scan_pdf_keywords` — Scan a PDF file for suspicious keywords similar to pdfid.
- `extract_pdf_version` — Extract the PDF version from the header.
- `find_stream_objects` — Find and extract stream objects from the PDF.
- `extract_javascript` — Extract JavaScript code from PDF objects.
- `extract_urls` — Extract URLs from the PDF content.
- `detect_exploits` — Check for known PDF exploit indicators.
- `calculate_risk_score` — Calculate a risk score for the PDF.
- `generate_report` — Generate PDF malware analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-pdf-malware-with-pdfid`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-persistence-mechanisms-in-linux (Skill Module)

analyzing-persistence-mechanisms-in-linux\SKILL.md

name: analyzing-persistence-mechanisms-in-linux
description: Analyze Linux persistence mechanisms: crontab, systemd, LD_PRELOAD, shell profiles, SSH keys. Use when the user asks to analyze persistence mechanisms in linux.
version: 1.0.0
tags:
source: analyzing-persistence-mechanisms-in-linux

Analyzing Persistence Mechanisms in Linux

Role: Deep analysis
Audience: analysts and investigators
Primary tools: LD_PRELOAD Detection, SSH Authorized Keys Audit

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** persistence mechanisms in linux.
- Capability: Analyze Linux persistence mechanisms: crontab, systemd, LD_PRELOAD, shell profiles, SSH keys.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, persistence, mechanisms, linux`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan all crontab locations for suspicious entries
2. Scan systemd service and timer units for persistence
3. Check for LD_PRELOAD hijacking
4. Scan shell profile files for injected commands
5. Audit SSH authorized_keys files for unauthorized entries
6. Generate persistence analysis report

Tools & resources

- LD_PRELOAD Detection
- SSH Authorized Keys Audit

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_crontabs` — Scan all crontab locations for suspicious entries.
- `scan_systemd_units` — Scan systemd service and timer units for persistence.
- `scan_ld_preload` — Check for LD_PRELOAD hijacking.
- `scan_shell_profiles` — Scan shell profile files for injected commands.
- `scan_ssh_authorized_keys` — Audit SSH authorized_keys files for unauthorized entries.
- `generate_report` — Generate persistence analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-persistence-mechanisms-in-Linux`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-powershell-empire-artifacts (Skill Module)

analyzing-powershell-empire-artifacts\SKILL.md

name: analyzing-powershell-empire-artifacts
description: Detect PowerShell Empire framework artifacts in Windows event logs. Use when the user asks to analyze powershell empire artifacts.
version: 1.0.0
tags:
source: analyzing-powershell-empire-artifacts

Analyzing PowerShell Empire Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: PowerShell Empire, Empire Module Signatures, Default Empire Staging URIs, Elastic KQL Detection

When to use

- The user wants to **analyze** powershell empire artifacts.
- Capability: Detect PowerShell Empire framework artifacts in Windows event logs.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, powershell, empire, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query Windows Event Log for specific event ID using wevtutil
2. Parse Event ID 4104 Script Block Logging events from XML
3. Attempt to decode Base64 encoded PowerShell payload
4. Analyze a script block for Empire indicators
5. Scan PowerShell event logs for Empire artifacts
6. Analyze a PowerShell script file or exported log for Empire artifacts

Tools & resources

- PowerShell Empire
- Empire Module Signatures
- Default Empire Staging URIs
- Elastic KQL Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_event_log` — Query Windows Event Log for specific event ID using wevtutil.
- `parse_script_block_events` — Parse Event ID 4104 Script Block Logging events from XML.
- `decode_base64_payload` — Attempt to decode Base64 encoded PowerShell payload.
- `analyze_script_block` — Analyze a script block for Empire indicators.
- `scan_event_logs` — Scan PowerShell event logs for Empire artifacts.
- `analyze_script_file` — Analyze a PowerShell script file or exported log for Empire artifacts.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-powershell-empire-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-powershell-script-block-logging (Skill Module)

analyzing-powershell-script-block-logging\SKILL.md

name: analyzing-powershell-script-block-logging
description: PowerShell Script Block Logging Analyzer - Parses Event 4104 from EVTX for obfuscated commands. Use when the user asks to analyze powershell script block logging.
version: 1.0.0
tags:
source: analyzing-powershell-script-block-logging

Analyzing PowerShell Script Block Logging

Role: Deep analysis
Audience: analysts and investigators
Primary tools: python-evtx Library, lxml etree Parsing, Script Block Reconstruction

When to use

- The user wants to **analyze** powershell script block logging.
- Capability: PowerShell Script Block Logging Analyzer - Parses Event 4104 from EVTX for obfuscated commands.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, powershell, script, block, logging`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of a string to detect obfuscation
2. Parse Event ID 4104 entries from a PowerShell Operational EVTX file
3. Reconstruct full scripts from multi-part script block entries
4. Attempt to decode Base64-encoded command strings found in scripts
5. Analyze a single reconstructed script for suspicious patterns
6. Generate PowerShell script block analysis report

Tools & resources

- python-evtx Library
- lxml etree Parsing
- Script Block Reconstruction

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `calculate_entropy` — Calculate Shannon entropy of a string to detect obfuscation.
- `parse_evtx_4104` — Parse Event ID 4104 entries from a PowerShell Operational EVTX file.
- `reconstruct_scripts` — Reconstruct full scripts from multi-part script block entries.
- `decode_base64_commands` — Attempt to decode Base64-encoded command strings found in scripts.
- `analyze_script` — Analyze a single reconstructed script for suspicious patterns.
- `generate_report` — Generate PowerShell script block analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-powershell-script-block-logging`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-prefetch-files-for-execution-history (Skill Module)

analyzing-prefetch-files-for-execution-history\SKILL.md

name: analyzing-prefetch-files-for-execution-history
description: Windows Prefetch file analysis agent for program execution history forensics. Use when the user asks to analyze prefetch files for execution history.
version: 1.0.0
tags:
source: analyzing-prefetch-files-for-execution-history

Analyzing Prefetch Files for Execution History

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Python Prefetch Parsing, Suspicious Prefetch Indicators

When to use

- The user wants to **analyze** prefetch files for execution history.
- Capability: Windows Prefetch file analysis agent for program execution history forensics.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, prefetch, files, execution, history`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse the Prefetch file header to extract execution metadata
2. Convert Windows FILETIME (100ns intervals since 1601-01-01) to ISO string
3. Parse executable name and hash from prefetch filename format: APPNAME-HASH.pf
4. Scan a directory of prefetch files and extract execution history
5. Flag suspicious or known-attacker-tool prefetch files
6. Build a chronological timeline of program execution
7. Run Eric Zimmerman's PECmd for comprehensive prefetch parsing

Tools & resources

- Python Prefetch Parsing
- Suspicious Prefetch Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_prefetch_header` — Parse the Prefetch file header to extract execution metadata.
- `parse_filetime` — Convert Windows FILETIME (100ns intervals since 1601-01-01) to ISO string.
- `parse_prefetch_filename` — Parse executable name and hash from prefetch filename format: APPNAME-HASH.pf.
- `scan_prefetch_directory` — Scan a directory of prefetch files and extract execution history.
- `detect_suspicious_execution` — Flag suspicious or known-attacker-tool prefetch files.
- `build_execution_timeline` — Build a chronological timeline of program execution.
- `run_pecmd` — Run Eric Zimmerman's PECmd for comprehensive prefetch parsing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-prefetch-files-for-execution-history`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-ransomware-encryption-mechanisms (Skill Module)

analyzing-ransomware-encryption-mechanisms\SKILL.md

name: analyzing-ransomware-encryption-mechanisms
description: Ransomware encryption mechanism analysis agent. Analyzes encryption algorithms, key management, file encryption routines, and assesses decryption feasibility for ransomware samples and encrypted files. Use when the user asks to analyze ransomware encryption mechanisms.
version: 1.0.0
tags:
source: analyzing-ransomware-encryption-mechanisms

Analyzing Ransomware Encryption Mechanisms

Role: Deep analysis
Audience: analysts and investigators
Primary tools: PyCryptodome, pefile, Volatility 3, Entropy Analysis Thresholds, Known Ransomware Encryption Schemes, NoMoreRansom / ID Ransomware

When to use

- The user wants to **analyze** ransomware encryption mechanisms.
- Capability: Ransomware encryption mechanism analysis agent. Analyzes encryption algorithms, key management, file encryption routines, and assesses decryption feasibility for ransomware samples and encrypted files.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, ransomware, encryption, mechanisms`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a file
2. Calculate Shannon entropy of byte data
3. Identify ransomware family from file extension
4. Scan binary for known cryptographic constants
5. Scan binary for Windows Crypto API string references
6. Analyze an encrypted file for ransomware characteristics
7. Attempt XOR key recovery from known plaintext-ciphertext pair
8. Check if encrypted file retains known file header (partial encryption indicator)

Tools & resources

- PyCryptodome
- pefile
- Volatility 3
- Entropy Analysis Thresholds
- Known Ransomware Encryption Schemes
- NoMoreRansom / ID Ransomware
- Ghidra

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hash` — Compute SHA-256 hash of a file.
- `shannon_entropy` — Calculate Shannon entropy of byte data.
- `identify_ransomware_extension` — Identify ransomware family from file extension.
- `scan_crypto_constants` — Scan binary for known cryptographic constants.
- `scan_crypto_apis` — Scan binary for Windows Crypto API string references.
- `analyze_encrypted_file` — Analyze an encrypted file for ransomware characteristics.
- `xor_key_recovery` — Attempt XOR key recovery from known plaintext-ciphertext pair.
- `check_file_header_known_plaintext` — Check if encrypted file retains known file header (partial encryption indicator).
- `assess_decryption_feasibility` — Assess decryption feasibility based on analysis results.
- `generate_report` — Generate full ransomware encryption analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-ransomware-encryption-mechanisms`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-ransomware-leak-site-intelligence (Skill Module)

`analyzing-ransomware-leak-site-intelligence\SKILL.md`

name: analyzing-ransomware-leak-site-intelligence
description: Ransomware leak site intelligence analysis agent. Monitors and analyzes ransomware group leak site data for threat intelligence, victim tracking, and TTI (time-to-intelligence) reporting. Use when the user asks to analyze ransomware leak site intelligence.
version: 1.0.0
tags:
source: analyzing-ransomware-leak-site-intelligence

Analyzing Ransomware Leak Site Intelligence

Role: Deep analysis
Audience: analysts and investigators
Primary tools: ransomware.live API, ransomlook.io API, ID Ransomware, Intelligence Collection Framework, STIX Representation

When to use

- The user wants to **analyze** ransomware leak site intelligence.
- Capability: Ransomware leak site intelligence analysis agent. Monitors and analyzes ransomware group leak site data for threat intelligence, victim tracking, and TTI (time-to-intelligence) reporting.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, ransomware, leak, site, intelligence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query ransomwatch or ransomware.live API for leak site data
2. Query ransomlook.io API for group information
3. Analyze victim listing data for intelligence
4. Search victims by name, domain, or sector
5. Assess activity level of a specific ransomware group
6. Generate ransomware threat intelligence report

Tools & resources

- ransomware.live API
- ransomlook.io API
- ID Ransomware
- Intelligence Collection Framework
- STIX Representation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_ransomwatch_api` — Query ransomwatch or ransomware.live API for leak site data.
- `query_ransomlook_group` — Query ransomlook.io API for group information.
- `analyze_victim_data` — Analyze victim listing data for intelligence.
- `search_victims` — Search victims by name, domain, or sector.
- `assess_group_activity` — Assess activity level of a specific ransomware group.
- `generate_intelligence_report` — Generate ransomware threat intelligence report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-ransomware-leak-site-intelligence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-ransomware-network-indicators (Skill Module)

`analyzing-ransomware-network-indicators\SKILL.md`

name: analyzing-ransomware-network-indicators
description: Detect ransomware network indicators: C2 beaconing, TOR connections, data exfiltration via Zeek/NetFlow. Use when the user asks to analyze ransomware network indicators.
version: 1.0.0
tags:
source: analyzing-ransomware-network-indicators

Analyzing Ransomware Network Indicators

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Zeek conn.log Fields, Beaconing Detection Algorithm, TOR Exit Node Detection

When to use

- The user wants to **analyze** ransomware network indicators.
- Capability: Detect ransomware network indicators: C2 beaconing, TOR connections, data exfiltration via Zeek/NetFlow.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, ransomware, network, indicators`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Zeek conn.log TSV format into structured records
2. Parse NetFlow CSV export into connection records
3. Fetch current TOR exit node IP list from Tor Project
4. Detect C2 beaconing by analyzing connection interval regularity
5. Cross-reference connections against TOR exit node list
6. Detect potential data exfiltration by high outbound byte transfer
7. Generate ransomware network indicator report

Tools & resources

- Zeek conn.log Fields
- Beaconing Detection Algorithm
- TOR Exit Node Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_zeek_conn_log` — Parse Zeek conn.log TSV format into structured records.
- `parse_netflow_csv` — Parse NetFlow CSV export into connection records.
- `fetch_tor_exit_nodes` — Fetch current TOR exit node IP list from Tor Project.
- `detect_beaconing` — Detect C2 beaconing by analyzing connection interval regularity.
- `detect_tor_connections` — Cross-reference connections against TOR exit node list.
- `detect_exfiltration` — Detect potential data exfiltration by high outbound byte transfer.
- `generate_report` — Generate ransomware network indicator report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-ransomware-network-indicators`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-ransomware-payment-wallets (Skill Module)

analyzing-ransomware-payment-wallets\SKILL.md

name: analyzing-ransomware-payment-wallets
description: Ransomware payment wallet blockchain analysis agent. Traces cryptocurrency payment flows from ransomware wallets using public blockchain APIs. Identifies transaction patterns, cluster relationships, and fund movement.... Use when the user asks to analyze ransomware payment wallets.
version: 1.0.0
tags:
source: analyzing-ransomware-payment-wallets

Analyzing Ransomware Payment Wallets

Role: Deep analysis
Audience: analysts and investigators
Primary tools: blockchain.com API, Blockstream.info API, WalletExplorer API, Common Ransomware Wallet Indicators, OFAC SDN Sanctions Check

When to use

- The user wants to **analyze** ransomware payment wallets.
- Capability: Ransomware payment wallet blockchain analysis agent. Traces cryptocurrency payment flows from ransomware wallets using public blockchain APIs. Identifies transaction patterns, cluster relationships, and fund movement to exchanges or mixers.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, ransomware, payment, wallets`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Validate Bitcoin address format
2. Query blockchain.info for address details
3. Query Blockstream API for address stats (fallback)
4. Extract downstream addresses from transaction outputs
5. Check if address matches known ransomware wallets
6. Detect peel chain pattern in outgoing transactions
7. Full analysis of a ransomware payment wallet

Tools & resources

- blockchain.com API
- Blockstream.info API
- WalletExplorer API
- Common Ransomware Wallet Indicators
- OFAC SDN Sanctions Check

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `validate_btc_address` — Validate Bitcoin address format.
- `query_address_info` — Query blockchain.info for address details.
- `query_blockstream_address` — Query Blockstream API for address stats (fallback).
- `extract_output_addresses` — Extract downstream addresses from transaction outputs.
- `check_known_wallets` — Check if address matches known ransomware wallets.
- `detect_peel_chain` — Detect peel chain pattern in outgoing transactions.
- `analyze_wallet` — Full analysis of a ransomware payment wallet.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-ransomware-payment-wallets`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-sbom-for-supply-chain-vulnerabilities (Skill Module)

`analyzing-sbom-for-supply-chain-vulnerabilities\SKILL.md`

name: analyzing-sbom-for-supply-chain-vulnerabilities
description: SBOM supply chain vulnerability analysis agent. Parses CycloneDX and SPDX JSON SBOMs, correlates components against the NVD 2.0 API for known CVEs, builds dependency graphs with networkx, calculates risk scores, and.... Use when the user asks to analyze sbom for supply chain vulnerabilities.
version: 1.0.0
tags:
source: analyzing-sbom-for-supply-chain-vulnerabilities

Analyzing Sbom for Supply Chain Vulnerabilities

Role: Deep analysis
Audience: analysts and investigators
Primary tools: NVD API 2.0, syft, grype, CLI Usage Examples

When to use

- The user wants to **analyze** sbom for supply chain vulnerabilities.
- Capability: SBOM supply chain vulnerability analysis agent. Parses CycloneDX and SPDX JSON SBOMs, correlates components against the NVD 2.0 API for known CVEs, builds dependency graphs with networkx, calculates risk scores, and generates compliance reports.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, sbom, supply, chain, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Detect whether the SBOM is CycloneDX or SPDX format
2. Parse CycloneDX JSON SBOM and extract components and dependencies
3. Parse SPDX JSON SBOM and extract components and dependencies
4. Load and parse an SBOM file, auto-detecting the format
5. Query NVD 2.0 API for CVEs matching a CPE name
6. Query NVD 2.0 API for CVEs matching a keyword search
7. Extract structured CVE information from an NVD API response entry
8. Correlate all SBOM components against NVD for known vulnerabilities

Tools & resources

- NVD API 2.0
- syft
- grype
- CLI Usage Examples

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_sbom_format` — Detect whether the SBOM is CycloneDX or SPDX format.
- `parse_cyclonedx` — Parse CycloneDX JSON SBOM and extract components and dependencies.
- `parse_spdx` — Parse SPDX JSON SBOM and extract components and dependencies.
- `parse_sbom` — Load and parse an SBOM file, auto-detecting the format.
- `query_nvd_by_cpe` — Query NVD 2.0 API for CVEs matching a CPE name.
- `query_nvd_by_keyword` — Query NVD 2.0 API for CVEs matching a keyword search.
- `extract_cve_info` — Extract structured CVE information from an NVD API response entry.
- `correlate_cves` — Correlate all SBOM components against NVD for known vulnerabilities.
- `build_dependency_graph` — Build a directed dependency graph using networkx.
- `analyze_dependency_graph` — Analyze the dependency graph for risk metrics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-sbom-for-supply-chain-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-security-logs-with-splunk (Skill Module)

analyzing-security-logs-with-splunk\SKILL.md

name: analyzing-security-logs-with-splunk
description: Agent for analyzing security logs with Splunk using splunk-sdk. Use when the user asks to analyze security logs with splunk.
version: 1.0.0
tags:
source: analyzing-security-logs-with-splunk

Analyzing Security Logs with Splunk

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Splunk

When to use

- The user wants to **analyze** security logs with splunk.
- Capability: Agent for analyzing security logs with Splunk using splunk-sdk.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, security, logs, splunk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Establish connection to Splunk instance
2. Execute a Splunk search and return parsed results
3. Detect brute force attacks via failed logon events (EventCode 4625)
4. Detect lateral movement via Type 3 network logons to multiple hosts
5. Detect encoded or download-cradle PowerShell execution via Sysmon
6. Detect credential dumping via LSASS process access (Sysmon Event 10)
7. Build a unified incident timeline across multiple log sources
8. Format investigation findings into a structured report

Tools & resources

- Splunk

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_splunk` — Establish connection to Splunk instance.
- `run_search` — Execute a Splunk search and return parsed results.
- `detect_brute_force` — Detect brute force attacks via failed logon events (EventCode 4625).
- `detect_lateral_movement` — Detect lateral movement via Type 3 network logons to multiple hosts.
- `detect_suspicious_powershell` — Detect encoded or download-cradle PowerShell execution via Sysmon.
- `detect_lsass_access` — Detect credential dumping via LSASS process access (Sysmon Event 10).
- `build_incident_timeline` — Build a unified incident timeline across multiple log sources.
- `generate_report` — Format investigation findings into a structured report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-security-logs-with-splunk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-slack-space-and-file-system-artifacts (Skill Module)

analyzing-slack-space-and-file-system-artifacts\SKILL.md

name: analyzing-slack-space-and-file-system-artifacts
description: Agent for analyzing NTFS slack space and file system artifacts. Use when the user asks to analyze slack space and file system artifacts.
version: 1.0.0
tags:
source: analyzing-slack-space-and-file-system-artifacts

Analyzing Slack Space and File System Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: USN Journal Parsing, bulk_extractor, foremost

When to use

- The user wants to **analyze** slack space and file system artifacts.
- Capability: Agent for analyzing NTFS slack space and file system artifacts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, slack, space, file, system, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse MFT using analyzeMFT and return deleted/timestomped files
2. Extract slack space from a disk image using blkls from The Sleuth Kit
3. Search extracted slack space for forensic keywords
4. Parse NTFS USN Change Journal ($UsnJrnl:$J) records
5. List Alternate Data Streams using fls from The Sleuth Kit
6. Detect timestomping by comparing $SI and $FN timestamps in MFT CSV output
7. Generate a structured forensic report

Tools & resources

- USN Journal Parsing
- bulk_extractor
- foremost

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_mft_with_analyzeMFT` — Parse MFT using analyzeMFT and return deleted/timestomped files.
- `extract_slack_space` — Extract slack space from a disk image using blkls from The Sleuth Kit.
- `search_slack_keywords` — Search extracted slack space for forensic keywords.
- `parse_usn_journal` — Parse NTFS USN Change Journal ($UsnJrnl:$J) records.
- `find_ads_in_image` — List Alternate Data Streams using fls from The Sleuth Kit.
- `detect_timestomping` — Detect timestomping by comparing $SI and $FN timestamps in MFT CSV output.
- `generate_report` — Generate a structured forensic report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-slack-space-and-file-system-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-supply-chain-malware-artifacts (Skill Module)

`analyzing-supply-chain-malware-artifacts\SKILL.md`

name: analyzing-supply-chain-malware-artifacts
description: Supply chain malware artifact analysis agent. Analyzes software supply chain compromise indicators including package integrity, build pipeline artifacts, dependency confusion, and trojanized updates. Use when the user asks to analyze supply chain malware artifacts.
version: 1.0.0
tags:
source: analyzing-supply-chain-malware-artifacts

Analyzing Supply Chain Malware Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: npm Registry API, PyPI JSON API, Socket.dev, Suspicious Package Indicators, Sigstore/cosign Verification, SLSA Framework Levels

When to use

- The user wants to **analyze** supply chain malware artifacts.
- Capability: Supply chain malware artifact analysis agent. Analyzes software supply chain compromise indicators including package integrity, build pipeline artifacts, dependency confusion, and trojanized updates.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, supply, chain, malware, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Ingest and validate inputs for supply chain malware artifacts
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

Key concepts

- MITRE ATT&CK Framework
- NIST SP 800-83 Guide to Malware Incident Prevention
- NIST SP 800-86 Guide to Integrating Forensic Techniques

Tools & resources

- npm Registry API
- PyPI JSON API
- Socket.dev
- Suspicious Package Indicators
- Sigstore/cosign Verification
- SLSA Framework Levels
- npm install Hook Risks
- Applicable Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compute_hash`
- `check_npm_package`
- `check_pypi_package`
- `detect_typosquat_packages`
- `analyze_package_scripts`
- `analyze_python_setup`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-supply-chain-malware-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-threat-actor-ttps-with-mitre-attack (Skill Module)

`analyzing-threat-actor-ttps-with-mitre-attack\SKILL.md`

name: analyzing-threat-actor-ttps-with-mitre-attack
description: Threat actor TTP analysis agent using MITRE ATT&CK framework. Maps threat actor behaviors to ATT&CK techniques, performs coverage analysis, and generates detection gap reports. Use when the user asks to analyze threat actor ttps with mitre attack.
version: 1.0.0
tags:
source: analyzing-threat-actor-ttps-with-mitre-attack

Analyzing Threat Actor TTPS with MITRE ATT&CK

Role: Deep analysis
Audience: analysts and investigators
Primary tools: MITRE ATT&CK, mitreattack-python, TAXII Server Access, STIX 2.1 Representation

When to use

- The user wants to **analyze** threat actor ttps with mitre attack.
- Capability: Threat actor TTP analysis agent using MITRE ATT&CK framework. Maps threat actor behaviors to ATT&CK techniques, performs coverage analysis, and generates detection gap reports.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, threat, actor, ttps, mitre, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. **Report Ingestion**: Obtain threat intelligence report (vendor, OSINT, internal)
2. **Behavior Extraction**: Identify adversary actions described in the report
3. **Technique Mapping**: Map each behavior to ATT&CK technique IDs using the ATT&CK knowledge base
4. **Sub-technique Precision**: Drill down to sub-techniques where procedure details allow
5. **Layer Creation**: Generate ATT&CK Navigator layer with mapped techniques
6. **Priority Assessment**: Rank techniques by detection feasibility and impact
7. **Detection Inventory**: Catalog existing detection rules mapped to ATT&CK techniques
8. **Detection Layer**: Create Navigator layer showing detected techniques (green)
9. **Threat Layer**: Create layer showing adversary techniques (red)
10. **Overlay Analysis**: Combine layers to identify uncovered threat techniques

Key concepts

- Enterprise ATT&CK: Windows, macOS, Linux, Cloud (AWS, Azure, GCP, SaaS, Office 365), Network, Containers
- Mobile ATT&CK: Android, iOS
- ICS ATT&CK: Industrial Control Systems
- Technique: T[NNNN] (e.g., T1059 - Command and Scripting Interpreter)
- Sub-technique: T[NNNN].[NNN] (e.g., T1059.001 - PowerShell)
- Group: G[NNNN] (e.g., G0016 - APT29)
- Software: S[NNNN] (e.g., S0154 - Cobalt Strike)
- Mitigation: M[NNNN] (e.g., M1049 - Antivirus/Antimalware)

Tools & resources

- MITRE ATT&CK
- mitreattack-python

- TAXII Server Access
- STIX 2.1 Representation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_attack_bundle`
- `get_techniques`
- `get_groups`
- `map_group_techniques`
- `tactic_coverage`
- `detection_gaps`
- `find_group`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-threat-actor-ttps-with-mitre-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-threat-actor-ttps-with-mitre-navigator (Skill Module)

analyzing-threat-actor-ttps-with-mitre-navigator\SKILL.md

name: analyzing-threat-actor-ttps-with-mitre-navigator
description: MITRE ATT&CK Navigator layer generation and threat actor TTP mapping. Use when the user asks to analyze threat actor ttps with mitre navigator.
version: 1.0.0
tags:
source: analyzing-threat-actor-ttps-with-mitre-navigator

Analyzing Threat Actor TTPS with MITRE Navigator

Role: Deep analysis
Audience: analysts and investigators
Primary tools: MITRE ATT&CK, attackcti Python Library, External References

When to use

- The user wants to **analyze** threat actor ttps with mitre navigator.
- Capability: MITRE ATT&CK Navigator layer generation and threat actor TTP mapping agent.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, threat, actor, ttps, mitre, navigator`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Initialize ATT&CK STIX/TAXII client
2. List all threat groups in ATT&CK
3. Get all techniques used by a specific threat group
4. Generate ATT&CK Navigator layer JSON from group technique data
5. Compare techniques across multiple threat groups
6. Execute threat actor TTP mapping audit

Tools & resources

- MITRE ATT&CK
- attackcti Python Library
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_attack_client` — Initialize ATT&CK STIX/TAXII client.
- `list_threat_groups` — List all threat groups in ATT&CK.
- `get_group_techniques` — Get all techniques used by a specific threat group.
- `generate_navigator_layer` — Generate ATT&CK Navigator layer JSON from group technique data.
- `compare_groups` — Compare techniques across multiple threat groups.
- `run_audit` — Execute threat actor TTP mapping audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-threat-actor-ttps-with-mitre-navigator`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-threat-intelligence-feeds (Skill Module)

analyzing-threat-intelligence-feeds\SKILL.md

name: analyzing-threat-intelligence-feeds
description: Agent for analyzing threat intelligence feeds via TAXII 2.1 and STIX 2.1. Use when the user asks to analyze threat intelligence feeds.
version: 1.0.0
tags:
source: analyzing-threat-intelligence-feeds

Analyzing Threat Intelligence Feeds

Role: Deep analysis
Audience: analysts and investigators
Primary tools: taxii2-client

When to use

- The user wants to **analyze** threat intelligence feeds.
- Capability: Agent for analyzing threat intelligence feeds via TAXII 2.1 and STIX 2.1.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, threat, intelligence, feeds`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover TAXII 2.1 server and list available API roots and collections
2. Fetch STIX indicators from a TAXII 2.1 collection
3. Convert a raw IOC into a STIX 2.1 Indicator object
4. Deduplicate indicators by pattern value
5. Score feed quality based on indicator attributes
6. Export indicators as a STIX 2.1 bundle JSON file
7. Auto-detect IOC type from value

Tools & resources

- taxii2-client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_taxii_server` — Discover TAXII 2.1 server and list available API roots and collections.
- `fetch_indicators` — Fetch STIX indicators from a TAXII 2.1 collection.
- `normalize_to_stix` — Convert a raw IOC into a STIX 2.1 Indicator object.
- `deduplicate_indicators` — Deduplicate indicators by pattern value.
- `score_feed_quality` — Score feed quality based on indicator attributes.
- `export_stix_bundle` — Export indicators as a STIX 2.1 bundle JSON file.
- `classify_ioc_type` — Auto-detect IOC type from value.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-threat-intelligence-feeds`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-threat-landscape-with-misp (Skill Module)

analyzing-threat-landscape-with-misp\SKILL.md

name: analyzing-threat-landscape-with-misp
description: MISP Threat Landscape Analysis Agent - Generates threat landscape reports from MISP event data. Use when the user asks to analyze threat landscape with misp.
version: 1.0.0
tags:
source: analyzing-threat-landscape-with-misp

Analyzing Threat Landscape with MISP

Role: Deep analysis
Audience: analysts and investigators
Primary tools: MISP, Attribute Types, Galaxy Tag Prefixes, Statistics API

When to use

- The user wants to **analyze** threat landscape with misp.
- Capability: MISP Threat Landscape Analysis Agent - Generates threat landscape reports from MISP event data.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, threat, landscape, misp`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to MISP instance
2. Fetch events from the last N days
3. Break down events by threat level
4. Analyze distribution of attribute types across events
5. Extract and categorize tags from events
6. Analyze event creation trends over time (weekly buckets)
7. Analyze contributing organizations
8. Compute IOC statistics: total count, unique values, categories

Tools & resources

- MISP
- Attribute Types
- Galaxy Tag Prefixes
- Statistics API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_misp` — Connect to MISP instance.
- `fetch_events` — Fetch events from the last N days.
- `analyze_threat_levels` — Break down events by threat level.
- `analyze_attribute_types` — Analyze distribution of attribute types across events.
- `extract_tags` — Extract and categorize tags from events.
- `analyze_temporal_trends` — Analyze event creation trends over time (weekly buckets).
- `analyze_organizations` — Analyze contributing organizations.
- `compute_ioc_stats` — Compute IOC statistics: total count, unique values, categories.
- `generate_report` — Generate threat landscape report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-threat-landscape-with-misp`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-tls-certificate-transparency-logs (Skill Module)

analyzing-tls-certificate-transparency-logs\SKILL.md

name: analyzing-tls-certificate-transparency-logs
description: Agent for analyzing Certificate Transparency logs for phishing detection. Use when the user asks to analyze tls certificate transparency logs.
version: 1.0.0
tags:
source: analyzing-tls-certificate-transparency-logs

Analyzing TLS Certificate Transparency Logs

Role: Deep analysis
Audience: analysts and investigators
Primary tools: pycrtsh, Key Certificate Fields

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** tls certificate transparency logs.
- Capability: Agent for analyzing Certificate Transparency logs for phishing detection.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, certificate, transparency, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Search crt.sh for certificates matching a domain
2. Get full certificate details from crt.sh by ID
3. Query crt.sh REST API directly for certificate records
4. Compute Levenshtein distance between two strings
5. Detect typosquatting domains using Levenshtein distance
6. Find certificates issued by unauthorized Certificate Authorities
7. Find certificates issued in the last N hours
8. Identify wildcard certificates that could cover many subdomains

Tools & resources

- pycrtsh
- Key Certificate Fields

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `search_certificates` — Search crt.sh for certificates matching a domain.
- `get_certificate_details` — Get full certificate details from crt.sh by ID.
- `search_crtsh_api` — Query crt.sh REST API directly for certificate records.
- `levenshtein_distance` — Compute Levenshtein distance between two strings.
- `detect_typosquatting` — Detect typosquatting domains using Levenshtein distance.
- `detect_unauthorized_cas` — Find certificates issued by unauthorized Certificate Authorities.
- `monitor_new_certificates` — Find certificates issued in the last N hours.
- `find_wildcard_certificates` — Identify wildcard certificates that could cover many subdomains.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-tls-certificate-transparency-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-typosquatting-domains-with-dnstwist (Skill Module)

analyzing-typosquatting-domains-with-dnstwist\SKILL.md

name: analyzing-typosquatting-domains-with-dnstwist
description: Typosquatting domain detection agent using dnstwist concepts. Use when the user asks to analyze typosquatting domains with dnstwist.
version: 1.0.0
tags:
source: analyzing-typosquatting-domains-with-dnstwist

Analyzing Typosquatting Domains with dnstwist

Role: Deep analysis
Audience: analysts and investigators
Primary tools: dnstwist, WHOIS Lookup, VirusTotal Domain Check

When to use

- The user wants to **analyze** typosquatting domains with dnstwist.
- Capability: Typosquatting domain detection agent using dnstwist concepts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, typosquatting, domains, dnstwist`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Ingest and validate inputs for typosquatting domains with dnstwist
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

Tools & resources

- dnstwist
- WHOIS Lookup
- VirusTotal Domain Check

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_permutations`
- `resolve_domain`
- `check_domains`
- `run_dnstwist_cli`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-typosquatting-domains-with-dnstwist`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-uefi-bootkit-persistence (Skill Module)

analyzing-uefi-bootkit-persistence\SKILL.md

name: analyzing-uefi-bootkit-persistence
description: UEFI bootkit persistence analysis agent for detecting firmware implants, ESP modifications, Secure Boot bypasses, and UEFI variable manipulation. Use when the user asks to analyze uefi bootkit persistence.
version: 1.0.0
tags:
source: analyzing-uefi-bootkit-persistence

Analyzing UEFI Bootkit Persistence

Role: Deep analysis
Audience: analysts and investigators
Primary tools: chipsec, UEFITool / UEFIExtract, Secure Boot Variable GUIDs, flashrom, sigcheck, bcdedit

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **analyze** uefi bootkit persistence.
- Capability: UEFI bootkit persistence analysis agent for detecting firmware implants, ESP modifications, Secure Boot bypasses, and UEFI variable manipulation.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, uefi, bootkit, persistence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan a mounted EFI System Partition for bootkit indicators
2. Compute SHA-256 hash of a file
3. Scan a raw firmware dump for EFI firmware volumes and PE/COFF executables
4. Format 16 raw GUID bytes into standard XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX form
5. Check Secure Boot status on the local system (Linux)
6. Run a chipsec module via subprocess and return output
7. Dump SPI flash contents via chipsec
8. Run a comprehensive set of chipsec security modules

Tools & resources

- chipsec
- UEFITool / UEFIExtract
- Secure Boot Variable GUIDs
- flashrom
- sigcheck
- bcdedit
- YARA

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_esp_partition` — Scan a mounted EFI System Partition for bootkit indicators.
- `hash_file` — Compute SHA-256 hash of a file.
- `scan_firmware_dump` — Scan a raw firmware dump for EFI firmware volumes and PE/COFF executables.
- `format_guid` — Format 16 raw GUID bytes into standard XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX form.
- `check_secure_boot_status` — Check Secure Boot status on the local system (Linux).
- `run_chipsec_module` — Run a chipsec module via subprocess and return output.
- `run_chipsec_spi_dump` — Dump SPI flash contents via chipsec.
- `run_firmware_security_audit` — Run a comprehensive set of chipsec security modules.
- `firmware_entropy_map` — Generate block-level entropy map to detect encrypted/compressed firmware regions.
- `analyze_uefi_bootkit` — Perform UEFI bootkit persistence analysis on a firmware dump or ESP mount point.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-uefi-bootkit-persistence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-usb-device-connection-history (Skill Module)

analyzing-usb-device-connection-history\SKILL.md

name: analyzing-usb-device-connection-history
description: Agent for analyzing USB device connection history from Windows registry hives. Use when the user asks to analyze usb device connection history.
version: 1.0.0
tags:
source: analyzing-usb-device-connection-history

Analyzing USB Device Connection History

Role: Deep analysis
Audience: analysts and investigators
Primary tools: python, SetupAPI Log Parsing, USB Forensic Registry Keys

When to use

- The user wants to **analyze** usb device connection history.
- Capability: Agent for analyzing USB device connection history from Windows registry hives.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, device, connection, history`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse USBSTOR registry key to enumerate USB storage devices
2. Parse MountedDevices to map drive letters to USB devices
3. Parse MountPoints2 from NTUSER.DAT to find user-accessed volumes
4. Parse setupapi.dev.log for USB first-install timestamps
5. Build a unified USB activity timeline
6. Export timeline to CSV

Tools & resources

- python
- SetupAPI Log Parsing
- USB Forensic Registry Keys

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_usbstor` — Parse USBSTOR registry key to enumerate USB storage devices.
- `parse_mounted_devices` — Parse MountedDevices to map drive letters to USB devices.
- `parse_mountpoints2` — Parse MountPoints2 from NTUSER.DAT to find user-accessed volumes.
- `parse_setupapi_log` — Parse setupapi.dev.log for USB first-install timestamps.
- `build_timeline` — Build a unified USB activity timeline.
- `export_timeline_csv` — Export timeline to CSV.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-usb-device-connection-history`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-web-server-logs-for-intrusion (Skill Module)

analyzing-web-server-logs-for-intrusion\SKILL.md

name: analyzing-web-server-logs-for-intrusion
description: Web Server Log Intrusion Analyzer - Detects SQLi, LFI, XSS, and scanner activity in access logs. Use when the user asks to analyze web server logs for intrusion.
version: 1.0.0
tags:
source: analyzing-web-server-logs-for-intrusion

Analyzing Web Server Logs for Intrusion

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Python re Module, GeoIP2 Python Library, Attack Signature Categories, Scanner User-Agent Signatures

When to use

- The user wants to **analyze** web server logs for intrusion.
- Capability: Web Server Log Intrusion Analyzer - Detects SQLi, LFI, XSS, and scanner activity in access logs.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, server, logs, intrusion`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse a single Combined Log Format line
2. Parse all entries from a web server access log
3. Scan log entries for attack signatures
4. Detect brute-force login attempts by IP frequency
5. Enrich findings with GeoIP location data
6. Aggregate findings by source IP for attacker profiling
7. Generate web intrusion detection report

Tools & resources

- Python re Module
- GeoIP2 Python Library
- Attack Signature Categories
- Scanner User-Agent Signatures

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_log_line` — Parse a single Combined Log Format line.
- `parse_log_file` — Parse all entries from a web server access log.
- `detect_attacks` — Scan log entries for attack signatures.
- `detect_brute_force` — Detect brute-force login attempts by IP frequency.
- `enrich_with_geoip` — Enrich findings with GeoIP location data.
- `summarize_attackers` — Aggregate findings by source IP for attacker profiling.
- `generate_report` — Generate web intrusion detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-web-server-logs-for-intrusion`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-windows-amcache-artifacts (Skill Module)

analyzing-windows-amcache-artifacts\SKILL.md

name: analyzing-windows-amcache-artifacts
description: Windows Amcache.hve forensic analysis agent. Parses Amcache.hve registry hive to extract program execution history, file metadata, and device information using the regipy library. Use when the user asks to analyze windows amcache artifacts.
version: 1.0.0
tags:
source: analyzing-windows-amcache-artifacts

Analyzing Windows Amcache Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Amcache.hve Location, Registry Keys, regipy Python Library, Forensic Value

When to use

- The user wants to **analyze** windows amcache artifacts.
- Capability: Windows Amcache.hve forensic analysis agent. Parses Amcache.hve registry hive to extract program execution history, file metadata, and device information using the regipy library.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, windows, amcache, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse InventoryApplicationFile entries from Amcache.hve
2. Parse InventoryApplication entries
3. Flag suspicious entries based on path and name patterns

Tools & resources

- Amcache.hve Location
- Registry Keys
- regipy Python Library
- Forensic Value

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_amcache_files` — Parse InventoryApplicationFile entries from Amcache.hve.
- `parse_amcache_apps` — Parse InventoryApplication entries.
- `detect_suspicious` — Flag suspicious entries based on path and name patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-windows-amcache-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-windows-event-logs-in-splunk (Skill Module)

analyzing-windows-event-logs-in-splunk\SKILL.md

name: analyzing-windows-event-logs-in-splunk
description: Agent for analyzing Windows event logs in Splunk for SOC operations. Use when the user asks to analyze windows event logs in splunk.
version: 1.0.0
tags:
source: analyzing-windows-event-logs-in-splunk

Analyzing Windows Event Logs in Splunk

Role: Deep analysis
Audience: analysts and investigators
Primary tools: Splunk, Logon Types

When to use

- The user wants to **analyze** windows event logs in splunk.
- Capability: Agent for analyzing Windows event logs in Splunk for SOC operations.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, windows, event, logs, splunk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run a blocking Splunk search and return results
2. Detect brute force via EventCode 4625 with logon type classification
3. Detect password spray attacks targeting many accounts from one source
4. Detect new accounts added to the Administrators group (T1136.001)
5. Detect LSASS credential dumping via Sysmon Event 10 (T1003.001)
6. Detect SMB lateral movement via Type 3 logons to many hosts (T1021.002)
7. Detect PsExec execution via Sysmon process creation (T1021.002)
8. Build a comprehensive forensic timeline for a host

Tools & resources

- Splunk
- Logon Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `search` — Run a blocking Splunk search and return results.
- `detect_brute_force` — Detect brute force via EventCode 4625 with logon type classification.
- `detect_password_spray` — Detect password spray attacks targeting many accounts from one source.
- `detect_new_admin_accounts` — Detect new accounts added to the Administrators group (T1136.001).
- `detect_lsass_access` — Detect LSASS credential dumping via Sysmon Event 10 (T1003.001).
- `detect_lateral_movement_smb` — Detect SMB lateral movement via Type 3 logons to many hosts (T1021.002).
- `detect_psexec` — Detect PsExec execution via Sysmon process creation (T1021.002).
- `build_forensic_timeline` — Build a comprehensive forensic timeline for a host.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-windows-event-logs-in-splunk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-windows-lnk-files-for-artifacts (Skill Module)

`analyzing-windows-lnk-files-for-artifacts\SKILL.md`

name: analyzing-windows-lnk-files-for-artifacts
description: Agent for analyzing Windows LNK shortcut files for forensic artifacts. Use when the user asks to analyze windows lnk files for artifacts.
version: 1.0.0
tags:
source: analyzing-windows-lnk-files-for-artifacts

Analyzing Windows Lnk Files for Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: LnkParse3, Key LNK File Locations, Drive Types

When to use

- The user wants to **analyze** windows lnk files for artifacts.
- Capability: Agent for analyzing Windows LNK shortcut files for forensic artifacts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, windows, files, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse a single LNK file and extract forensic artifacts
2. Parse all LNK files in a directory
3. Filter LNK files that point to removable media
4. Filter LNK files pointing to network shares
5. Analyze Startup folder LNK files for potential persistence
6. Export parsed LNK results to CSV
7. Extract unique machine IDs and MAC addresses from LNK files

Tools & resources

- LnkParse3
- Key LNK File Locations
- Drive Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_lnk_file` — Parse a single LNK file and extract forensic artifacts.
- `parse_lnk_directory` — Parse all LNK files in a directory.
- `filter_removable_media` — Filter LNK files that point to removable media.
- `filter_network_shares` — Filter LNK files pointing to network shares.
- `detect_suspicious_startup` — Analyze Startup folder LNK files for potential persistence.
- `export_csv` — Export parsed LNK results to CSV.
- `extract_unique_machines` — Extract unique machine IDs and MAC addresses from LNK files.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-windows-lnk-files-for-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-windows-prefetch-with-python (Skill Module)

analyzing-windows-prefetch-with-python\SKILL.md

name: analyzing-windows-prefetch-with-python
description: Agent for analyzing Windows Prefetch files with Python. Parses Prefetch (.pf) files to reconstruct execution history, detect renamed/masquerading binaries, and identify suspicious tool execution using the.... Use when the user asks to analyze windows prefetch with python.
version: 1.0.0
tags:
source: analyzing-windows-prefetch-with-python

Analyzing Windows Prefetch with Python

Role: Deep analysis
Audience: analysts and investigators
Primary tools: windowsprefetch Library, Prefetch File Versions, File Naming Convention, Suspicious Executables to Flag, Prefetch Directory Location

When to use

- The user wants to **analyze** windows prefetch with python.
- Capability: Agent for analyzing Windows Prefetch files with Python. Parses Prefetch (.pf) files to reconstruct execution history, detect renamed/masquerading binaries, and identify suspicious tool execution using the windowsprefetch library.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, windows, prefetch, python`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Ingest and validate inputs for windows prefetch with python
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

Tools & resources

- windowsprefetch Library
- Prefetch File Versions
- File Naming Convention
- Suspicious Executables to Flag
- Prefetch Directory Location

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-windows-prefetch-with-python`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-windows-registry-for-artifacts (Skill Module)

`analyzing-windows-registry-for-artifacts\SKILL.md`

name: analyzing-windows-registry-for-artifacts
description: Agent for analyzing Windows Registry hives for forensic artifacts. Use when the user asks to analyze windows registry for artifacts.
version: 1.0.0
tags:
source: analyzing-windows-registry-for-artifacts

Analyzing Windows Registry for Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: regipy, Key Forensic Registry Paths, UserAssist Decoding, RegRipper Plugins

When to use

- The user wants to **analyze** windows registry for artifacts.
- Capability: Agent for analyzing Windows Registry hives for forensic artifacts.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, windows, registry, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract Run/RunOnce autostart entries from SOFTWARE hive
2. Extract per-user autorun entries from NTUSER.DAT
3. Extract UserAssist program execution history (ROT13 decoded)
4. Extract RecentDocs MRU from NTUSER.DAT
5. Extract computer name, timezone, and shutdown time from SYSTEM hive
6. Extract installed software from Uninstall registry key
7. Flag autorun entries with suspicious characteristics

Tools & resources

- regipy
- Key Forensic Registry Paths
- UserAssist Decoding
- RegRipper Plugins

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_autorun_entries` — Extract Run/RunOnce autostart entries from SOFTWARE hive.
- `extract_user_autorun` — Extract per-user autorun entries from NTUSER.DAT.
- `extract_userassist` — Extract UserAssist program execution history (ROT13 decoded).
- `extract_recent_docs` — Extract RecentDocs MRU from NTUSER.DAT.
- `extract_system_info` — Extract computer name, timezone, and shutdown time from SYSTEM hive.
- `extract_installed_software` — Extract installed software from Uninstall registry key.
- `flag_suspicious_autorun` — Flag autorun entries with suspicious characteristics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-windows-registry-for-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

analyzing-windows-shellbag-artifacts (Skill Module)

`analyzing-windows-shellbag-artifacts\SKILL.md`

name: analyzing-windows-shellbag-artifacts
description: Windows ShellBag artifact analysis agent. Parses ShellBag registry artifacts to reconstruct folder access history, directory browsing patterns, and evidence of accessed network shares. Use when the user asks to analyze windows shellbag artifacts.
version: 1.0.0
tags:
source: analyzing-windows-shellbag-artifacts

Analyzing Windows Shellbag Artifacts

Role: Deep analysis
Audience: analysts and investigators
Primary tools: SBECmd (Eric Zimmerman), ShellBags Explorer (Eric Zimmerman), Registry Explorer (Eric Zimmerman)

When to use

- The user wants to **analyze** windows shellbag artifacts.
- Capability: Windows ShellBag artifact analysis agent. Parses ShellBag registry artifacts to reconstruct folder access history, directory browsing patterns, and evidence of accessed network shares.
- Goal: extract findings, indicators, and conclusions from technical evidence.
- Trigger keywords: `analyzing, windows, shellbag, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Ingest and validate inputs for windows shellbag artifacts
2. Establish baseline context (scope, hosts, time window, hypotheses)
3. Run structured analysis using the skill tools and references
4. Extract indicators, artifacts, and high-confidence findings
5. Correlate results and document conclusions with evidence pointers

Key concepts

- NIST SP 800-86: Guide to Integrating Forensic Techniques
- SWGDE Best Practices for Computer Forensics
- SBECmd (Eric Zimmerman): Command-line shellbag parser
- ShellBags Explorer (Eric Zimmerman): GUI shellbag viewer
- Registry Explorer (Eric Zimmerman): Registry hive analysis
- NTUSER.DAT: Software\Microsoft\Windows\Shell\BagMRU and Bags
- UsrClass.dat: Local Settings\Software\Microsoft\Windows\Shell\BagMRU and Bags
- T1083 - File and Directory Discovery

Tools & resources

- SBECmd (Eric Zimmerman)
- ShellBags Explorer (Eric Zimmerman)
- Registry Explorer (Eric Zimmerman)

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `filetime_to_datetime`
- `parse_shell_item`
- `parse_bagmru_value`
- `analyze_shellbags_regipy`
- `detect_suspicious_paths`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `analyzing-windows-shellbag-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

api-hardening (Skill Module)

`security-toolkit\skills\api-hardening\SKILL.md`

name: api-hardening

description: API security hardening patterns. Use when implementing rate limiting, input validation, CORS configuration, API key management, request throttling, or protecting endpoints from abuse. Covers defense-in-depth strategies for REST APIs with practical implementations for Express, FastAPI, and serverless, oriented around the OWASP API Security Top 10:2023.

tags:

API hardening

[[Defense-in-depth]] patterns for protecting APIs from abuse, injection attacks, and data leakage. Recipes are oriented around the OWASP API Security Top 10:2023 and were last verified on 2026-05-08.

Step 0: Research the current security landscape (do this first)

> *Security knowledge ages on a 6-12 month half-life. The recipes below were last verified on 2026-05-08; they may be stale by the time you read this. Before applying any pattern in this [[skill]], fan out research scoped to the API surface or web defense being added so the recipes are interpreted against current authoritative sources, not against this file's snapshot.*

Default-on, with a documented skip

Run the 4-angle research below by default. Skip ONLY when ALL of these hold:

- (a) You ran this same [[skill]] on this same primitive within the last 4 hours of the current session,
- (b) That prior research surfaced no urgent advisories for the API surface or web defense being added,
- (c) You log a one-line `Research skipped because <reason>` note in your response.

"I think I know" / "moving fast" / "user wants this done quickly" / "already familiar" are NOT valid skip reasons. The whole point of this preamble is that future-you should not trust this [[skill]] body's defaults until current state is checked.

Fan out 4 subagents in parallel

Each subagent returns ≤300 words of bullets with citations. Dispatch all 4 in a single message so they run concurrently.

Angle 1 — Authoritative

Have NIST / OWASP / IETF (RFCs and Internet-Drafts) / W3C / CISA published anything new about the API surface or web defense being added in the last 6-12 months? Look for: spec finalizations, deprecations, replacement specs, RFC publications, draft revisions, NIST SP updates, OWASP project version bumps. Cite by document number + [[[[publication]]]] date.

Angle 2 — Active exploitation. What's actively being exploited that targets the API surface or web defense being added? Pull from: CISA Known Exploited Vulnerabilities (KEV) catalog (filter to last 6-12 months), recent CVE / GHSA entries with high CVSS or in-the-wild exploitation, breach postmortems and incident reports (CSRB, vendor RCAs, security-vendor research). Surface CWE patterns dominating recent KEV adds. Cite by CVE number + advisory URL.

Angle 3 — Tooling and library state. Are the [[libraries]] this [[skill]] recommends still current? What are the latest major versions in the relevant package registry (npm / PyPI / RubyGems / crates.io)? Have any been deprecated, replaced, or merged into another project? Have any flipped a secure default? Look up current versions in: registry.npmjs.org, pypi.org, rubygems.org, crates.io, pkg.go.dev. Cite by package + version + release date.

Angle 4 — Practitioner discourse. What are practitioners and security teams talking about in the last 6 months? Pull from: OWASP Cheat Sheet Series (last-modified date matters), GitHub Security Lab posts, vendor security blogs (Cloudflare, Fastly, Snyk, Datadog, Wiz, GitGuardian), conference talks (Black Hat, DEF CON, OWASP Global AppSec, USENIX Security), SANS ISC, Krebs, recent OWASP project re-releases. Surface the patterns being adopted and the anti-patterns being called out. Cite by post URL + author + date.

Synthesize before applying recipes

After the 4 returns land, write a 1-paragraph "current state for the API surface or web defense being added, as of " that names:

- The current normative ceiling (what specs say SHOULD be the default in 2026).
- 1-2 active threats specific to the API surface or web defense being added from the last 6-12 months.
- Any tooling drift (deprecated lib, new default in a framework, package merged or replaced).
- Any practitioner consensus shift visible in recent cheat sheet / blog updates.

If the synthesis flags drift in this [[skill]] body's recipes (e.g., a spec finalized after 2026-05-08, a library now deprecated, a default flipped), call that out explicitly in your response and override the [[skill]] body where they conflict. The synthesis wins. The [[skill]] body is scaffolding, not scripture.

When you cannot run subagents

If subagents are not available in your runtime, the same shape applies in-line: do 4 sequential targeted searches (web search for [[[[

```
const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis').default;
const { createClient } = require('redis');

const redisClient = createClient({ url: process.env.REDIS_URL });
redisClient.connect();

// General API rate limit
const apiLimiter = rateLimit({
  store: new RedisStore({ sendCommand: (...args) => redisClient.sendCommand(args) }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requests per window
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too many requests, please try again later' },
  skip: (req) => {
    // Skip rate limiting for health checks
    return req.path === '/health';
  }
});

// Strict limit for auth endpoints
const authLimiter = rateLimit({
  store: new RedisStore({ sendCommand: (...args) => redisClient.sendCommand(args) }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 attempts
  message: { error: 'Too many login attempts, please try again in 15 minutes' },
  keyGenerator: (req) => {
    // Rate limit by IP + email to prevent distributed attacks
    return `${req.ip}-${req.body?.email || 'unknown'}`;
  }
});

// Very strict limit for password reset
const passwordResetLimiter = rateLimit({
  store: new RedisStore({ sendCommand: (...args) => redisClient.sendCommand(args) }),
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 3, // 3 requests per hour
  message: { error: 'Too many password reset requests' }
});

// Apply limiters
app.use('/api/', apiLimiter);
app.use('/auth/login', authLimiter);
app.use('/auth/forgot-password', passwordResetLimiter);
```

Sliding window implementation (custom)

```
// Redis-based sliding window rate limiter
class SlidingWindowRateLimiter {
  constructor(redisClient, options = {}) {
    this.redis = redisClient;
    this.windowMs = options.windowMs || 60000; // 1 minute default
    this.maxRequests = options.maxRequests || 100;
    this.keyPrefix = options.keyPrefix || 'ratelimit';
  }

  async isAllowed(identifier) {
    const now = Date.now();
    const windowStart = now - this.windowMs;
    const key = `${this.keyPrefix}:${identifier}`;

    // Remove old entries and count recent ones
    const multi = this.redis.multi();
    multi.zRemRangeByScore(key, 0, windowStart);
    multi.zCard(key);
    multi.zAdd(key, { score: now, value: `${now}-${Math.random()}` });
    multi.expire(key, Math.ceil(this.windowMs / 1000));

    const results = await multi.exec();
    const requestCount = results[1];

    return {
      allowed: requestCount < this.maxRequests,
      remaining: Math.max(0, this.maxRequests - requestCount - 1),
      resetAt: now + this.windowMs
    };
  }
}

// Express middleware
function createRateLimitMiddleware(limiter) {
  return async (req, res, next) => {
    const identifier = req.ip;
    const result = await limiter.isAllowed(identifier);

    res.setHeader('X-RateLimit-Limit', limiter.maxRequests);
    res.setHeader('X-RateLimit-Remaining', result.remaining);
    res.setHeader('X-RateLimit-Reset', result.resetAt);

    if (!result.allowed) {
      return res.status(429).json({ error: 'Rate limit exceeded' });
    }

    next();
  };
}
```

Per-user rate limiting with API keys

```
// Different limits based on tier
const tierLimits = {
  free: { windowMs: 60000, max: 10 },
  pro: { windowMs: 60000, max: 100 },
  enterprise: { windowMs: 60000, max: 1000 }
};

async function apiKeyRateLimiter(req, res, next) {
  const apiKey = req.headers['x-api-key'];

  if (!apiKey) {
    return res.status(401).json({ error: 'API key required' });
  }

  // Look up API key
  const keyData = await db.query(
    'SELECT user_id, tier, revoked FROM api_keys WHERE key_hash = $1',
    [hashApiKey(apiKey)]
  );

  if (keyData.rows.length === 0 || keyData.rows[0].revoked) {
    return res.status(401).json({ error: 'Invalid API key' });
  }

  const { user_id, tier } = keyData.rows[0];
  const limits = tierLimits[tier] || tierLimits.free;

  // Rate limit by user, not by key (prevents key rotation abuse)
  const limiter = new SlidingWindowRateLimiter(redisClient, {
    ...limits,
    keyPrefix: 'apikey'
  });

  const result = await limiter.isAllowed(user_id);

  res.setHeader('X-RateLimit-Limit', limits.max);
  res.setHeader('X-RateLimit-Remaining', result.remaining);
  res.setHeader('X-RateLimit-Reset', result.resetAt);

  if (!result.allowed) {
    return res.status(429).json({ error: 'Rate limit exceeded' });
  }

  req.userId = user_id;
  next();
}
```

Per-account graph-traversal quota (23andMe lesson)

Endpoints that expand a relationship or graph one hop at a time (DNA relatives, contact networks, follower fan-out, organization-membership lookups) are the textbook case for per-account quotas in addition to per-IP. A single compromised account inside a per-IP budget can still walk the graph and exfiltrate data on every other connected account, which is what amplified the 23andMe credential-stuffing breach.

```
// Per-account daily budget for graph-expansion endpoints
const graphLimiter = new SlidingWindowRateLimiter(redisClient, {
  windowMs: 24 * 60 * 60 * 1000, // 24 hours
  maxRequests: 500, // tune to product norms
  keyPrefix: 'graph'
});

async function graphTraversalQuota(req, res, next) {
  const result = await graphLimiter.isAllowed(req.userId);
  if (!result.allowed) {
    return res.status(429).json({ error: 'Daily graph quota exceeded' });
  }
  next();
}

app.get('/api/relatives', requireAuth, graphTraversalQuota, listRelatives);
app.get('/api/contacts/expand', requireAuth, graphTraversalQuota, expandContacts);
```

Input validation

Maps to API3:2023 (broken object property level authorization) and API10:2023 (unsafe consumption of APIs).

Validation with Zod (TypeScript/JavaScript)

`zod` is on the 4.x line as of 2026-05-08 (4.4.3 current; verify before pinning). Patterns below work on v4; if you're still on 3.x, `safeParse` and the schema builders below are unchanged.

Untrusted input also has a deserialization dimension — see the dedicated section below. CWE-502 (deserialization of untrusted data) dominated the CISA KEV catalog in 2024-2025, so JSON-only at trust boundaries plus schema validation is the baseline.

```
const { z } = require('zod');

// Define schemas
const createUserSchema = z.object({
  email: z.string().email().max(255),
  password: z.string().min(12).max(128),
  name: z.string().min(1).max(100).optional()
});

const updateProfileSchema = z.object({
  name: z.string().min(1).max(100).optional(),
  bio: z.string().max(500).optional(),
  website: z.string().url().optional().or(z.literal(''))
});

const paginationSchema = z.object({
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20)
});

// Middleware factory
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);

    if (!result.success) {
      return res.status(400).json({
        error: 'Validation failed',
        details: result.error.issues.map(issue => ({
          field: issue.path.join('.'),
          message: issue.message
        }))
      });
    }

    req.validated = result.data;
    next();
  };
}

// Usage
app.post('/users', validate(createUserSchema), async (req, res) => {
  const { email, password, name } = req.validated;
  // Data is validated and typed
});
```

Sanitization

Trusted Types is a related browser-side defense — it forces dangerous DOM sinks to consume policy-vetted objects instead of strings, which kills entire classes of DOM-based XSS. As of 2026-05-08 Trusted Types is cross-browser (Chromium since 2020, Firefox 148, Safari 26.0; caniuse global usage ~89%), so `require-trusted-types-for 'script'` is now a realistic CSP directive rather than a Chrome-only nice-to-have. See the security headers section.

```
const createDOMPurify = require('dompurify');
const { JSDOM } = require('jsdom');
const validator = require('validator');

const window = new JSDOM('').window;
const DOMPurify = createDOMPurify(window);

// HTML sanitization (when you MUST allow some HTML)
function sanitizeHtml(dirty) {
  return DOMPurify.sanitize(dirty, {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br'],
    ALLOWED_ATTR: ['href'],
    ALLOW_DATA_ATTR: false
  });
}

// String sanitization
function sanitizeString(str) {
  if (typeof str !== 'string') return '';

  return str
    .trim()
    .slice(0, 10000) // Max length
    .replace(/[\x00-\x1F\x7F]/g, ''); // Remove control characters
}

// SQL-safe identifier (for dynamic column names)
function sanitizeIdentifier(str) {
  // Only allow alphanumeric and underscores
  if (!/^[a-zA-Z_][a-zA-Z0-9_]*$/.test(str)) {
    throw new Error('Invalid identifier');
  }
  return str;
}

// Filename sanitization
function sanitizeFilename(filename) {
  return filename
    .replace(/[^a-zA-Z0-9._-]/g, '_')
    .replace(/\.{2,}/g, '.')
    .slice(0, 255);
}
```

Preventing SQL injection

Maps to OWASP Top 10:2025 A05 (injection). MOVEit CVE-2023-34362 is the canonical 2023 example: a pre-auth SQLi in a file-transfer product, weaponized by Cl0p across thousands of organizations (per https://nvd.nist.gov/vuln/detail/CVE-2023-34362). Don't assume "internal" or "non-user-facing" endpoints are safe to skip parameterization on.

```
// BAD: String interpolation
const query = `SELECT * FROM users WHERE id = ${userId}`;

// BAD: String concatenation
const query = 'SELECT * FROM users WHERE id = ' + userId;

// BAD: Template literals with user input
const query = `SELECT * FROM users WHERE name = '${name}'`;

// GOOD: Parameterized queries (PostgreSQL)
const result = await db.query(
  'SELECT * FROM users WHERE id = $1',
  [userId]
);

// GOOD: Parameterized queries (MySQL)
const result = await db.query(
  'SELECT * FROM users WHERE id = ?',
  [userId]
);

// GOOD: Query builders (Knex)
const users = await knex('users')
  .where('id', userId)
  .first();

// GOOD: ORMs (Prisma)
const user = await prisma.user.findUnique({
  where: { id: userId }
});

// When you need dynamic column names (rare)
const allowedColumns = ['name', 'email', 'created_at'];
const sortColumn = allowedColumns.includes(req.query.sort)
  ? req.query.sort
  : 'created_at';

const query = `SELECT * FROM users ORDER BY ${sortColumn}`; // Safe because allowlisted
```

Command injection (CWE-78)

CWE-78 (OS command injection) dominated the CISA KEV catalog over 2024-2025 — 14 entries in 2024 and 18 in 2025, more than classic SQL injection. The Ivanti Connect Secure chain (CVE-2024-21887) is the marquee example. The rule is simple: never compose a shell command from untrusted input. Use the language's argv-list spawn primitives with the shell disabled, and allowlist any path or filename that flows into a process.

Node.js

Avoid the shell-execution primitives in the Node child-process module when any argument can be influenced by user input — they pass the full string through `/bin/sh -c` and any metacharacter (`;`, `&`, `|`, backtick, `$()`) becomes injection. Prefer the argv-list spawn family with the shell disabled, which is the default.

```
const { exec, execFile, spawn } = require('child_process');

// DO NOT USE: shell metacharacters in `filename` execute as commands
exec(`convert ${filename} out.png`, (err, stdout) => { /* ... */ });

// DO NOT USE: same problem with execSync
require('child_process').execSync(`convert ${filename} out.png`);

// GOOD: argv-list with the shell disabled (the default for execFile/spawn)
execFile('convert', [filename, 'out.png'], (err, stdout) => { /* ... */ });

// GOOD: spawn with explicit shell:false plus an allowlist on the filename
const safeFilename = /^[A-Za-z0-9_.-]+\.(png|jpg|jpeg)$/.test(filename)
  ? filename
  : null;
if (!safeFilename) throw new Error('Invalid filename');

const child = spawn('convert', [safeFilename, 'out.png'], { shell: false });
```

Python

Avoid the OS shell-execution primitives — the `os.system` call, `subprocess.run` with `shell=True`, `subprocess.Popen` with `shell=True`, and `os.popen` — when any argument can be influenced by user input. Use `subprocess.run` with an argv list and `shell=False` (the default), and allowlist any filename or path that crosses the trust boundary.

```
import subprocess
import re
from pathlib import Path

# DO NOT USE: os.system passes the full string through the shell
import os
os.system(f"convert {filename} out.png")

# DO NOT USE: shell=True is the same vulnerability
subprocess.run(f"convert {filename} out.png", shell=True, check=True)

# DO NOT USE: os.popen also goes through the shell
os.popen(f"convert {filename} out.png").read()

# GOOD: argv list, shell disabled (the default), with input allowlisting
if not re.fullmatch(r"[A-Za-z0-9_.-]+\.(png|jpg|jpeg)", filename):
    raise ValueError("Invalid filename")

subprocess.run(
    ["convert", filename, "out.png"],
    check=True,
    shell=False,  # explicit; this is also the default
    timeout=30,
)

# GOOD: when the input is a path, resolve and confirm it's inside an allowed root
allowed_root = Path("/var/app/uploads").resolve()
candidate = (allowed_root / filename).resolve()
if not candidate.is_relative_to(allowed_root):
    raise ValueError("Path traversal attempt")
```

Deserialization (CWE-502)

CWE-502 (deserialization of untrusted data) was the second-most-common KEV CWE in 2024-2025 — 11 entries in 2024 and 14 in 2025. The native binary serializers and unrestricted YAML loaders treat the input as a program: arbitrary code runs at parse time, before any of your validation logic. The rule is: JSON-only at trust boundaries, and validate the parsed JSON with a schema (Zod / Pydantic) before using it.

Python

Avoid the native binary deserialization primitive (the `pickle` loaders) and the unsafe YAML loader (the bare `yaml.load` call without `Loader=SafeLoader`) on any input that crosses a trust boundary — both will execute arbitrary objects on parse. The same warning applies to the `marshal` loader.

```
import json
import pickle
import yaml
from pydantic import BaseModel

# DO NOT USE: pickle.loads on untrusted input executes arbitrary code
obj = pickle.loads(request.body)

# DO NOT USE: yaml.load with no SafeLoader is a code-execution sink
config = yaml.load(request.body)  # equivalent to yaml.Loader

# DO NOT USE: marshal.loads has the same untrusted-input hazard as pickle
import marshal
obj = marshal.loads(request.body)

# GOOD: JSON parsing returns plain data (dict / list / str / number / bool / None)
class CreateUserRequest(BaseModel):
    email: str
    password: str

data = json.loads(request.body)  # safe parse
user = CreateUserRequest.model_validate(data)  # schema-validated

# GOOD: when YAML is genuinely required, use SafeLoader (or yaml.safe_load)
config = yaml.safe_load(request.body)
```

Node.js / Java / .NET

In Node, the equivalent risk lives in any library that accepts a "this is a serialized object" payload (the `node-serialize` package, older `funcster`-style packages, eval-based JSON5 forks). Stick to `JSON.parse` and validate with Zod.

In Java, avoid the binary deserialization primitive (`ObjectInputStream.readObject`) on untrusted input — Jackson, GSON, or another JSON library is the safe choice. In .NET, avoid the legacy binary formatter (`BinaryFormatter`, `NetDataContractSerializer`, `LosFormatter`, `SoapFormatter`) — Microsoft has flagged it for removal precisely

because of CWE-502 (per https://learn.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-security-guide). Use `System.Text.Json` for cross-trust-boundary input.

```
// DO NOT USE: a third-party deserializer on untrusted input
const obj = require('node-serialize').unserialize(request.body);

// GOOD: JSON.parse + Zod schema validation
const data = JSON.parse(request.body);
const parsed = createUserSchema.safeParse(data);
if (!parsed.success) {
  return res.status(400).json({ error: 'Validation failed' });
}
```

CORS configuration

Maps to API8:2023 (security misconfiguration).

Express.js

```
const cors = require('cors');

// Development: Allow localhost
const developmentOrigins = [
  'http://localhost:3000',
  'http://localhost:5173',
  'http://127.0.0.1:3000'
];

// Production: Specific domains only
const productionOrigins = [
  'https://yourapp.com',
  'https://www.yourapp.com',
  'https://app.yourapp.com'
];

const allowedOrigins = process.env.NODE_ENV === 'production'
  ? productionOrigins
  : [...productionOrigins, ...developmentOrigins];

const corsOptions = {
  origin: (origin, callback) => {
    // Allow requests with no origin (mobile apps, curl, etc.)
    if (!origin) {
      return callback(null, true);
    }

    if (allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('Not allowed by CORS'));
    }
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-API-Key'],
  exposedHeaders: ['X-RateLimit-Limit', 'X-RateLimit-Remaining'],
  credentials: true, // Allow cookies
  maxAge: 86400 // Cache preflight for 24 hours
};

app.use(cors(corsOptions));

// Handle CORS errors
app.use((err, req, res, next) => {
  if (err.message === 'Not allowed by CORS') {
    return res.status(403).json({ error: 'CORS not allowed' });
  }
  next(err);
});
```

Common CORS mistakes

```
// BAD: Allow all origins
app.use(cors()); // Defaults to '*'

// BAD: Allow all origins with credentials
app.use(cors({ origin: '*', credentials: true })); // Browsers will reject this

// BAD: Reflecting Origin header (allows any origin)
app.use(cors({
  origin: (origin, cb) => cb(null, origin) // Never do this
}));

// BAD: Regex that's too permissive
const origin = /yourapp\.com/; // Matches evilyourapp.com too!

// GOOD: Exact match or strict regex
const origin = /^https:\/\/(www\.)?yourapp\.com$/;
```

Private Network Access (draft)

Private Network Access (PNA) is a WICG draft (https://wicg.github.io/private-network-access/) that adds CORS-style preflight when a public origin tries to reach a private-network resource (LAN IPs, localhost, intranet hostnames). As of 2026-05-08 only Chromium enforces it, and even there enforcement has been gradually rolled back to warnings while the spec evolves. Treat it as a draft: don't rely on PNA as a primary defense, but do plan for the day it lights up everywhere — keep your private-network APIs behind real [[authentication]], not just network position.

API key management

Maps to API1:2023 (broken object level authorization) when keys scope per-user data, and API2:2023 (broken [[authentication]]) for credential lifecycle.

Secure key generation and storage

```
const crypto = require('crypto');

// Generate API key
function generateApiKey() {
  // Format: prefix_randomBytes
  // Prefix helps identify key type and makes it recognizable
  const prefix = 'sk_live';
  const randomPart = crypto.randomBytes(24).toString('base64url');
  return `${prefix}_${randomPart}`;
}

// Hash for storage (never store plain keys)
function hashApiKey(key) {
  return crypto.createHash('sha256').update(key).digest('hex');
}

// Create new API key
app.post('/api-keys', requireAuth, async (req, res) => {
  const { name } = req.body;

  // Generate key
  const plainKey = generateApiKey();
  const keyHash = hashApiKey(plainKey);

  // Store only the hash
  await db.query(
    `INSERT INTO api_keys (user_id, key_hash, name, created_at)
     VALUES ($1, $2, $3, NOW())`,
    [req.userId, keyHash, name]
  );

  // Return plain key ONCE - user must save it
  res.json({
    key: plainKey,
    message: 'Save this key now. It will not be shown again.'
  });
});

// Verify API key
async function verifyApiKey(key) {
  const keyHash = hashApiKey(key);

  const result = await db.query(
    `SELECT id, user_id, revoked, last_used_at
     FROM api_keys WHERE key_hash = $1`,
    [keyHash]
  );

  if (result.rows.length === 0) {
    return null;
  }

  const keyData = result.rows[0];

  if (keyData.revoked) {
    return null;
  }

  // Update last used timestamp
  await db.query(
    'UPDATE api_keys SET last_used_at = NOW() WHERE id = $1',
    [keyData.id]
  );

  return keyData;
}

// Revoke API key
app.delete('/api-keys/:id', requireAuth, async (req, res) => {
  // Users can only revoke their own keys
  await db.query(
    'UPDATE api_keys SET revoked = true, revoked_at = NOW() WHERE id = $1 AND user_id = $2',
    [req.params.id, req.userId]
  );
```

```
  res.json({ success: true });
});
```

Rotate on a schedule, not just on compromise

Even without a compromise signal, rotate API keys on a schedule (90 days is a common baseline; tighten for high-privilege keys). Long-lived keys collect risk: leaked log lines, stale CI secrets, departed employees, forgotten test scripts. The rotation flow should support overlap — a "next" key live alongside the "current" one for the rotation window — so callers can swap without an outage.

```
// Add columns: rotated_from, rotation_due_at
// On rotation:
// 1. Generate new key
// 2. Insert with rotated_from = old key id, expires_at = old key expiry + window
// 3. Notify the key owner with the new value (out-of-band)
// 4. Mark old key revoked = true at the end of the overlap window

// Background job: flag keys past their rotation_due_at
const stale = await db.query(
  `SELECT id, user_id, name FROM api_keys
   WHERE revoked = false
     AND rotation_due_at < NOW()`
);
// Email each owner; auto-revoke after a grace period.
```

API key middleware

```
async function apiKeyAuth(req, res, next) {
  // Accept the key from headers ONLY — never query strings.
  // Query strings are routinely logged by web servers, reverse proxies, CDN
  // edge nodes, and analytics tooling, so a key in `?api_key=...` becomes a
  // credential leak by way of access logs. OWASP API Top 10 (API2:2023) and
  // RFC 6750 §2.3 both call this out.
  const apiKey = req.headers['x-api-key']
    || req.headers['authorization']?.replace('Bearer ', '');

  if (!apiKey) {
    return res.status(401).json({
      error: 'API key required',
      hint: 'Pass API key in X-API-Key header'
    });
  }

  const keyData = await verifyApiKey(apiKey);

  if (!keyData) {
    // Don't reveal if key exists but is revoked
    return res.status(401).json({ error: 'Invalid API key' });
  }

  req.apiKeyId = keyData.id;
  req.userId = keyData.user_id;

  next();
}
```

Request size limits

Maps to API4:2023 (unrestricted resource consumption).

```
const express = require('express');

// Global body size limit
app.use(express.json({ limit: '100kb' }));
app.use(express.urlencoded({ limit: '100kb', extended: true }));

// Per-route limits
app.post('/api/upload', express.json({ limit: '10mb' }), (req, res) => {
  // Handle large upload
});

// File upload limits
const multer = require('multer');
const upload = multer({
  limits: {
    fileSize: 5 * 1024 * 1024, // 5MB
    files: 5 // Max 5 files
  },
  fileFilter: (req, file, cb) => {
    const allowedTypes = ['image/jpeg', 'image/png', 'image/gif', 'application/pdf'];
    if (allowedTypes.includes(file.mimetype)) {
      cb(null, true);
    } else {
      cb(new Error('Invalid file type'));
    }
  }
});

app.post('/upload', upload.single('file'), (req, res) => {
  // Handle upload
});
```

Response security

Maps to API8:2023 (security misconfiguration).

Don't leak information

```
// BAD: Leaking stack traces
app.use((err, req, res, next) => {
  res.status(500).json({
    error: err.message,
    stack: err.stack // Never in production!
  });
});

// GOOD: Generic error in production
app.use((err, req, res, next) => {
  console.error(err); // Log full error server-side

  if (process.env.NODE_ENV === 'production') {
    res.status(500).json({ error: 'Internal server error' });
  } else {
    res.status(500).json({ error: err.message, stack: err.stack });
  }
});

// BAD: Revealing database structure
res.status(400).json({
  error: 'duplicate key value violates unique constraint "users_email_key"'
});

// GOOD: User-friendly error
res.status(400).json({
  error: 'An account with this email already exists'
});
```

Preventing XSS

Maps to OWASP Top 10:2025 A05 (injection). The 2026 shape of XSS defense is, in priority order:

1. **CSP3 with nonce + `'strict-dynamic'`** — modern strict CSP. Allowlists (`script-src https://cdn.foo.com …`) are no longer recommended; per Weichselbaum et al. they're routinely bypassable. Nonce-or-hash plus `'strict-dynamic'` is the W3C-blessed strict CSP.
2. **Trusted Types (`require-trusted-types-for 'script'`)** — kills DOM-based XSS by forcing dangerous sinks (`innerHTML` , `eval` , `setTimeout(string)`) to consume policy-vetted objects. Cross-browser as of 2026 (Chromium since 2020, Firefox 148, Safari 26.0; ~89% global usage per https://caniuse.com/trusted-types).

3. **Framework auto-escaping** — React, Vue, Angular, Svelte, modern engines all escape interpolated strings by default. Don't reach for the bypass APIs (`dangerouslySetInnerHTML` , `v-html` , `[innerHTML]` , `{@html}`) without DOMPurify.
4. **Manual escaping at the last server-side boundary** — when you genuinely need to inject a value into a non-templated HTML response, use a vetted escaper.

```
// BAD: Directly inserting user content
res.send(`<h1>Hello ${userName}</h1>`);

// GOOD: Use a template engine with auto-escaping
// EJS (auto-escapes by default with <%= %>)
res.render('greeting', { name: userName });

// GOOD: Escape manually when needed
const escapeHtml = require('escape-html');
res.send(`<h1>Hello ${escapeHtml(userName)}</h1>`);

// GOOD: Set Content-Type for JSON responses
res.json({ name: userName }); // Express sets correct headers

// In React/Vue/Angular/Svelte: framework handles escaping by default.
// Don't use dangerouslySetInnerHTML / v-html / [innerHTML] / {@html} on user input.
```

For the `'unsafe-inline'` and inline-script story, see the security headers section — strict CSP3 makes the inline question moot when nonces are wired through the

Security headers

Maps to API8:2023 (security misconfiguration). `helmet` is on the 8.x line as of 2026-05-08 (8.1.0 current; verify before pinning). The 2026 strict-CSP baseline replaces the older "set every X- header" advice.

```
const helmet = require('helmet');
const crypto = require('crypto');

// Per-request CSP nonce middleware (wire this into your template engine)
app.use((req, res, next) => {
  res.locals.cspNonce = crypto.randomBytes(16).toString('base64');
  next();
});

// Strict CSP3: nonce + 'strict-dynamic' + downgrade fallbacks
app.use(helmet.contentSecurityPolicy({
  useDefaults: false,
  directives: {
    defaultSrc: ["'self'"],
    // Strict CSP per https://www.w3.org/TR/CSP3/ and MDN strict-CSP guide.
    // 'strict-dynamic' lets nonced scripts load further scripts;
    // https: + 'unsafe-inline' are downgrade fallbacks for old browsers
    // that ignore 'strict-dynamic' (modern browsers ignore the fallbacks).
    scriptSrc: [
      (req, res) => `'nonce-${res.locals.cspNonce}'`,
      "'strict-dynamic'",
      'https:',
      "'unsafe-inline'"
    ],
    styleSrc: ["'self'", (req, res) => `'nonce-${res.locals.cspNonce}'`],
    imgSrc: ["'self'", 'data:', 'https:'],
    connectSrc: ["'self'", 'https://api.yourapp.com'],
    fontSrc: ["'self'", 'https://fonts.gstatic.com'],
    objectSrc: ["'none'"],
    baseUri: ["'self'"],
    frameAncestors: ["'none'"], // replaces X-Frame-Options
    formAction: ["'self'"],
    upgradeInsecureRequests: [],
    // Trusted Types: kills DOM-based XSS at the sink.
    // Cross-browser as of 2026 per https://caniuse.com/trusted-types
    requireTrustedTypesFor: ["'script'"],
    // Reporting via the Reporting API (W3C WD)
    reportTo: ['default']
  }
}));

// HSTS: 2 years + includeSubDomains + preload (per https://hstspreload.org)
// Operationally irreversible once preload-listed — verify subdomains first.
app.use(helmet.hsts({
  maxAge: 63072000,
  includeSubDomains: true,
  preload: true
}));

// Cross-origin isolation (COOP / COEP / CORP)
// Required for crossOriginIsolated === true (SharedArrayBuffer, high-resolution timers).
// Per WHATWG HTML Origin chapter: https://html.spec.whatwg.org/multipage/origin.html
app.use((req, res, next) => {
  res.setHeader('Cross-Origin-Opener-Policy', 'same-origin');
  res.setHeader('Cross-Origin-Embedder-Policy', 'require-corp'); // or 'credentialless'
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  next();
});

// Permissions-Policy: structured-fields syntax, opt-out of dangerous features.
// Per https://www.w3.org/TR/permissions-policy/ (W3C WD 2025-10-06; per-directive
// caniuse lookup required — this is "Limited Availability", not Baseline).
app.use((req, res, next) => {
  res.setHeader(
    'Permissions-Policy',
    'camera=(), microphone=(), geolocation=(), payment=(), usb=(), interest-cohort=()'
  );
  next();
});

// Reporting API: Reporting-Endpoints + report-to wiring.
// Per https://www.w3.org/TR/reporting-1/
app.use((req, res, next) => {
  res.setHeader(
    'Reporting-Endpoints',
    'default="https://reports.yourapp.com/csp"'
  );
  next();
```

```
});

// Keep nosniff
app.use(helmet.noSniff());

// Note: X-XSS-Protection is deprecated and ignored by modern browsers — do not set it.
// X-Frame-Options is replaced by CSP's frame-ancestors directive (set above).
```

When you render HTML, the [[[[

```
// Request timeout middleware
function timeout(ms) {
  return (req, res, next) => {
    res.setTimeout(ms, () => {
      res.status(408).json({ error: 'Request timeout' });
    });
    next();
  };
}

app.use(timeout(30000)); // 30 second default

// External API call timeout
async function fetchWithTimeout(url, options = {}, timeoutMs = 5000) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, {
      ...options,
      signal: controller.signal
    });
    return response;
  } finally {
    clearTimeout(timeoutId);
  }
}

// Database query timeout
const result = await db.query({
  text: 'SELECT * FROM large_table WHERE condition = $1',
  values: [value],
  timeout: 5000 // 5 second query timeout
});
```

FastAPI (Python) equivalents

Library versions current as of 2026-05-08 (verify on PyPI before pinning):

- `fastapi` is on the 0.x line (0.136.1 current; pre-1.0, so pin by minor — breaking changes can land between minors).
- `slowapi` is on the 0.1.x line (pre-1.0; pin by minor).
- `pydantic` is on the 2.x line (2.13.4 current).

```python
from fastapi import FastAPI, Depends, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from slowapi import Limiter
from slowapi.util import get_remote_address
from pydantic import BaseModel, EmailStr, Field
import hashlib
import secrets

app = FastAPI()

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://yourapp.com"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
)

# Rate limiting
limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.post("/api/login")
@limiter.limit("5/minute")
async def login(request: Request, credentials: LoginRequest):
    # Handle login
    pass

# Input validation with Pydantic 2.x
class CreateUserRequest(BaseModel):
    email: EmailStr
    password: str = Field(min_length=12, max_length=128)
    name: str = Field(max_length=100, default=None)

@app.post("/users")
async def create_user(user: CreateUserRequest):
    # Data is already validated
    pass

# API key generation
def generate_api_key() -> str:
    return f"sk_live_{secrets.token_urlsafe(24)}"

def hash_api_key(key: str) -> str:
    return hashlib.sha256(key.encode()).hexdigest()
```

Security checklist for APIs

Organized by OWASP API Security Top 10:2023 category. Skip categories that don't apply (API5 / API9 are out of scope here).

API1:2023 Broken object level authorization

- [ ] Per-object access checks on every fetch / [[update]] / delete
- [ ] API keys scoped to a user, not shared across tenants
- [ ] Object IDs aren't predictable or enumerable when that matters

API2:2023 Broken [[authentication]]

- [ ] Auth endpoints have stricter rate limits than general API endpoints
- [ ] Password reset endpoints have very strict per-account limits
- [ ] All auth flows over HTTPS only
- [ ] See the `secure-auth` [[skill]] for the auth primitive itself

API3:2023 Broken object property level authorization

- [ ] Input validated against a schema (Zod / Pydantic)
- [ ] Writable fields are explicitly allowlisted, not "everything in the body"
- [ ] Mass-assignment paths (`Object.assign` , `model.[[update]](req.body)`) avoided

API4:2023 Unrestricted resource consumption

- [ ] Rate limiting on all endpoints
- [ ] Request body size limits configured

- [ ] File upload size and count limits configured
- [ ] Timeouts on all DB queries and external calls
- [ ] Pagination caps on list endpoints

API6:2023 Unrestricted access to sensitive business flows

- [ ] Per-account quotas on graph / relationship / fan-out endpoints (23andMe lesson)
- [ ] Behavioral signals (impossible-travel, device fingerprint) for credential-stuffing-prone endpoints
- [ ] WAF or bot management in front for distributed abuse patterns

API7:2023 Server side request forgery

- [ ] Outbound HTTP calls go to allowlisted hosts only
- [ ] Timeouts on every outbound call
- [ ] Cloud metadata endpoints blocked at the network layer

API8:2023 Security misconfiguration

- [ ] Strict CSP3: nonce + `'strict-dynamic'` + Trusted Types
- [ ] HSTS with `max-age=63072000; includeSubDomains; preload`
- [ ] COOP / COEP / CORP set
- [ ] Permissions-Policy restricting dangerous features
- [ ] `X-Content-Type-Options: nosniff`
- [ ] `X-XSS-Protection` NOT set (deprecated)
- [ ] `X-Frame-Options` replaced by CSP `frame-ancestors`
- [ ] CORS configured to specific origins, never `*` with credentials
- [ ] TLS 1.3 default, TLS 1.2 minimum, TLS 1.0/1.1 disabled
- [ ] SRI hashes on every third-party `<script>` and `<link rel="stylesheet">`
- [ ] Error messages don't leak system info or DB structure

API10:2023 Unsafe consumption of APIs

- [ ] Upstream responses validated with the same rigor as user input
- [ ] No native binary deserializers or unrestricted YAML loaders on cross-trust-boundary input
- [ ] No shell-execution primitives composed from upstream-controlled values

Cross-cutting

- [ ] API keys hashed before storage; rotation on a schedule, not just on compromise
- [ ] API versioning for breaking changes

auditing-aws-s3-bucket-permissions (Skill Module)

auditing-aws-s3-bucket-permissions\SKILL.md

name: auditing-aws-s3-bucket-permissions
description: Agent for auditing AWS S3 bucket permissions using boto3. Use when the user asks to audit aws s3 bucket permissions.
version: 1.0.0
tags:
source: auditing-aws-s3-bucket-permissions

Auditing AWS S3 Bucket Permissions

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: AWS, boto3 S3 Client

When to use

- The user wants to **audit** aws s3 bucket permissions.
- Capability: Agent for auditing AWS S3 bucket permissions using boto3.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, bucket, permissions`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create a boto3 session
2. List all S3 buckets in the account
3. Check bucket-level public access block settings
4. Check bucket ACL for public grants
5. Check bucket policy for wildcard principals
6. Check if default encryption is enabled
7. Check if versioning is enabled
8. Check if server access logging is enabled

Tools & resources

- AWS
- boto3 S3 Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_session` — Create a boto3 session.
- `list_all_buckets` — List all S3 buckets in the account.
- `check_public_access_block` — Check bucket-level public access block settings.
- `check_bucket_acl` — Check bucket ACL for public grants.
- `check_bucket_policy` — Check bucket policy for wildcard principals.
- `check_encryption` — Check if default encryption is enabled.
- `check_versioning` — Check if versioning is enabled.
- `check_logging` — Check if server access logging is enabled.
- `audit_bucket` — Run full security audit on a single bucket.
- `classify_risk` — Classify risk level for a bucket based on audit findings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-aws-s3-bucket-permissions`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

auditing-azure-active-directory-configuration (Skill Module)

auditing-azure-active-directory-configuration\SKILL.md

name: auditing-azure-active-directory-configuration
description: Agent for auditing Azure Active Directory (Entra ID) configuration. Use when the user asks to audit azure active directory configuration.
version: 1.0.0
tags:
source: auditing-azure-active-directory-configuration

Auditing Azure Active Directory Configuration

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: Azure, Python Graph API Helper

When to use

- The user wants to **audit** azure active directory configuration.
- Capability: Agent for auditing Azure Active Directory (Entra ID) configuration.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, azure, active, directory, configuration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Obtain a Microsoft Graph API access token
2. Make an authenticated GET request to Microsoft Graph API
3. Get tenant organization details
4. List all Global Administrator role assignments
5. List all Conditional Access policies with their state and grant controls
6. Find users who have not signed in for specified number of days
7. List all guest users in the tenant
8. Check users without MFA registered

Tools & resources

- Azure
- Python Graph API Helper

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_graph_token` — Obtain a Microsoft Graph API access token.
- `graph_get` — Make an authenticated GET request to Microsoft Graph API.
- `get_tenant_info` — Get tenant organization details.
- `list_global_admins` — List all Global Administrator role assignments.
- `list_conditional_access_policies` — List all Conditional Access policies with their state and grant controls.
- `find_stale_users` — Find users who have not signed in for specified number of days.
- `list_guest_users` — List all guest users in the tenant.
- `check_mfa_registration` — Check users without MFA registered.
- `get_risky_signins` — Get risky sign-in events from the last N days.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-azure-active-directory-configuration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

auditing-cloud-with-cis-benchmarks (Skill Module)

auditing-cloud-with-cis-benchmarks\SKILL.md

name: auditing-cloud-with-cis-benchmarks
description: Agent for auditing cloud infrastructure against CIS Benchmarks using boto3. Use when the user asks to audit cloud with cis benchmarks.
version: 1.0.0
tags:
source: auditing-cloud-with-cis-benchmarks

Auditing Cloud with CIS Benchmarks

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: boto3, CIS Controls Quick Reference, Prowler CLI

When to use

- The user wants to **audit** cloud with cis benchmarks.
- Capability: Agent for auditing cloud infrastructure against CIS Benchmarks using boto3.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, cloud, benchmarks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. CIS 1.4 - Ensure no root account access key exists
2. CIS 1.5 - Ensure MFA is enabled for the root account
3. CIS 1.8-1.11 - Ensure IAM password policy is strong
4. CIS 3.1 - Ensure CloudTrail is enabled in all regions
5. CIS 3.2 - Ensure CloudTrail log file validation is enabled
6. CIS 2.1.1 - Ensure S3 default encryption is enabled
7. CIS 5.1 - Ensure VPC flow logging is enabled
8. CIS 5.4 - Ensure default security group restricts all traffic

Tools & resources

- boto3
- CIS Controls Quick Reference
- Prowler CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_root_access_keys` — CIS 1.4 - Ensure no root account access key exists.
- `check_root_mfa` — CIS 1.5 - Ensure MFA is enabled for the root account.
- `check_password_policy` — CIS 1.8-1.11 - Ensure IAM password policy is strong.
- `check_cloudtrail_multiregion` — CIS 3.1 - Ensure CloudTrail is enabled in all regions.
- `check_cloudtrail_log_validation` — CIS 3.2 - Ensure CloudTrail log file validation is enabled.
- `check_s3_encryption` — CIS 2.1.1 - Ensure S3 default encryption is enabled.
- `check_vpc_flow_logs` — CIS 5.1 - Ensure VPC flow logging is enabled.
- `check_default_sg_restrictions` — CIS 5.4 - Ensure default security group restricts all traffic.
- `run_full_audit` — Execute all CIS benchmark checks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-cloud-with-cis-benchmarks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

auditing-gcp-iam-permissions (Skill Module)

auditing-gcp-iam-permissions\SKILL.md

name: auditing-gcp-iam-permissions
description: Agent for auditing GCP IAM permissions using google-cloud libraries. Use when the user asks to audit gcp iam permissions.
version: 1.0.0
tags:
source: auditing-gcp-iam-permissions

Auditing GCP IAM Permissions

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: GCP, google-cloud-asset, google-cloud-resource-manager

When to use

- The user wants to **audit** gcp iam permissions.
- Capability: Agent for auditing GCP IAM permissions using google-cloud libraries.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, permissions`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Search IAM policies across the GCP organization
2. Find all IAM bindings using primitive roles (Owner, Editor)
3. Find resources accessible to allUsers or allAuthenticatedUsers
4. List all service accounts in a project with key info
5. Get IAM policy for a specific project
6. Analyze what resources an identity can access
7. Classify risk for IAM bindings

Tools & resources

- GCP
- google-cloud-asset
- google-cloud-resource-manager

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `search_iam_policies` — Search IAM policies across the GCP organization.
- `find_primitive_roles` — Find all IAM bindings using primitive roles (Owner, Editor).
- `find_public_bindings` — Find resources accessible to allUsers or allAuthenticatedUsers.
- `list_service_accounts` — List all service accounts in a project with key info.
- `get_project_iam_policy` — Get IAM policy for a specific project.
- `analyze_permissions` — Analyze what resources an identity can access.
- `classify_risk` — Classify risk for IAM bindings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-gcp-iam-permissions`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

auditing-kubernetes-cluster-rbac (Skill Module)

auditing-kubernetes-cluster-rbac\SKILL.md

name: auditing-kubernetes-cluster-rbac
description: Agent for auditing Kubernetes cluster RBAC configurations. Use when the user asks to audit kubernetes cluster rbac.
version: 1.0.0
tags:
source: auditing-kubernetes-cluster-rbac

Auditing Kubernetes Cluster RBAC

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: Kubernetes, Key RBAC Resources

When to use

- The user wants to **audit** kubernetes cluster rbac.
- Capability: Agent for auditing Kubernetes cluster RBAC configurations.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, kubernetes, cluster, rbac`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load Kubernetes configuration
2. Find ClusterRoles with wildcard verb or resource permissions
3. Find ClusterRoles that can read secrets
4. Find all ClusterRoleBindings that grant cluster-admin
5. Find bindings granting access to system:authenticated or system:unauthenticated
6. Find pods with automounted service account tokens
7. Find containers running as privileged or root

Tools & resources

- Kubernetes
- Key RBAC Resources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_kube_config` — Load Kubernetes configuration.
- `list_cluster_roles_with_wildcards` — Find ClusterRoles with wildcard verb or resource permissions.
- `list_secret_access_roles` — Find ClusterRoles that can read secrets.
- `list_cluster_admin_bindings` — Find all ClusterRoleBindings that grant cluster-admin.
- `find_dangerous_bindings` — Find bindings granting access to system:authenticated or system:unauthenticated.
- `audit_service_account_tokens` — Find pods with automounted service account tokens.
- `find_privileged_containers` — Find containers running as privileged or root.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-kubernetes-cluster-rbac`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

auditing-terraform-infrastructure-for-security (Skill Module)

`auditing-terraform-infrastructure-for-security\SKILL.md`

name: auditing-terraform-infrastructure-for-security
description: Agent for auditing Terraform infrastructure for security misconfigurations. Use when the user asks to audit terraform infrastructure for security.
version: 1.0.0
tags:
source: auditing-terraform-infrastructure-for-security

Auditing Terraform Infrastructure for Security

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: Terraform, Checkov CLI, tfsec CLI, Checkov Python API, Common CKV Check IDs, OPA/Conftest

When to use

- The user wants to **audit** terraform infrastructure for security.
- Capability: Agent for auditing Terraform infrastructure for security misconfigurations.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, terraform, infrastructure, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Checkov static analysis on Terraform code
2. Run Checkov on a Terraform plan JSON file for accurate analysis
3. Run tfsec Terraform security scanner
4. Parse Checkov output and extract failed checks with details
5. Parse tfsec output and extract findings
6. Directly scan a Terraform plan JSON for common security issues
7. Generate combined audit report

Tools & resources

- Terraform
- Checkov CLI
- tfsec CLI
- Checkov Python API
- Common CKV Check IDs
- OPA/Conftest

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_checkov` — Run Checkov static analysis on Terraform code.
- `run_checkov_on_plan` — Run Checkov on a Terraform plan JSON file for accurate analysis.
- `run_tfsec` — Run tfsec Terraform security scanner.
- `parse_checkov_results` — Parse Checkov output and extract failed checks with details.
- `parse_tfsec_results` — Parse tfsec output and extract findings.
- `scan_tf_plan_for_issues` — Directly scan a Terraform plan JSON for common security issues.
- `generate_report` — Generate combined audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-terraform-infrastructure-for-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

auditing-tls-certificate-transparency-logs (Skill Module)

auditing-tls-certificate-transparency-logs\SKILL.md

name: auditing-tls-certificate-transparency-logs
description: CT Log Monitoring Agent - Monitors Certificate Transparency logs for unauthorized certificate issuance, subdomain discovery, and certificate alerting. For authorized security monitoring and defensive operations only. Use when the user asks to audit tls certificate transparency logs.
version: 1.0.0
tags:
source: auditing-tls-certificate-transparency-logs

Auditing TLS Certificate Transparency Logs

Role: Security audit
Audience: security auditors and cloud/IAM engineers
Primary tools: Arguments, Key Functions, Alert Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **audit** tls certificate transparency logs.
- Capability: CT Log Monitoring Agent - Monitors Certificate Transparency logs for unauthorized certificate issuance, subdomain discovery, and certificate alerting. For authorized security monitoring and defensive operations only.
- Goal: measure configuration and access against policy and best practice.
- Trigger keywords: `auditing, certificate, transparency, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Initialize SQLite database for certificate tracking
2. Query crt.sh JSON API for certificates matching domain pattern. Args: domain: Domain pattern, e.g. '%.example.com' for wildcard search. exclude_expired: If True, exclude expired certificates from results. timeout: HTTP request timeout in seconds. Returns: List of certificate records from crt.sh
3. Fetch detailed certificate information from crt.sh by ID
4. Extract individual subdomain entries from a crt.sh name_value field. The name_value field can contain multiple DNS names separated by newlines
5. Store certificates in database, return list of newly discovered ones
6. Extract and store unique subdomains from certificate name_value fields
7. Resolve a subdomain to IP addresses and CNAME targets
8. Resolve all unresolved subdomains for a parent domain

Tools & resources

- Arguments
- Key Functions
- Alert Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `init_database` — Initialize SQLite database for certificate tracking.
- `query_crtsh` — Query crt.sh JSON API for certificates matching domain pattern. Args: domain: Domain pattern, e.g. '%.example.com' for wildcard search. exclude_expired: If True, exclude expired certificates from results. timeout: HTTP request timeout in seconds. Returns: List of certificate records from crt.sh.
- `get_certificate_detail` — Fetch detailed certificate information from crt.sh by ID.
- `extract_subdomains_from_names` — Extract individual subdomain entries from a crt.sh name_value field. The name_value field can contain multiple DNS names separated by newlines.
- `store_certificates` — Store certificates in database, return list of newly discovered ones.
- `discover_subdomains` — Extract and store unique subdomains from certificate name_value fields.
- `resolve_subdomain` — Resolve a subdomain to IP addresses and CNAME targets.
- `resolve_all_subdomains` — Resolve all unresolved subdomains for a parent domain.
- `check_unauthorized_ca` — Check if any new certificates were issued by unauthorized CAs.
- `check_new_subdomain_alerts` — Generate alerts for newly discovered subdomains.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `auditing-tls-certificate-transparency-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

automating-ioc-enrichment (Skill Module)

automating-ioc-enrichment\SKILL.md

name: automating-ioc-enrichment
description: Agent for automating IOC enrichment with VirusTotal, AbuseIPDB, and STIX. Use when the user asks to automate ioc enrichment.
version: 1.0.0
tags:
source: automating-ioc-enrichment

Automating IOC Enrichment

Role: Automation
Audience: SOC engineers and detection engineers
Primary tools: VirusTotal API v3, AbuseIPDB API v2, Shodan API, STIX 2.1 Export, Rate Limits

When to use

- The user wants to **automate** ioc enrichment.
- Capability: Agent for automating IOC enrichment with VirusTotal, AbuseIPDB, and STIX.
- Goal: turn a repeatable security workflow into a reliable pipeline.
- Trigger keywords: `automating, enrichment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Auto-detect IOC type from value
2. Enrich an IP address via VirusTotal API v3
3. Enrich a file hash via VirusTotal API v3
4. Enrich a domain via VirusTotal API v3
5. Check an IP against AbuseIPDB
6. Calculate composite confidence score from enrichment sources
7. Enrich a single IOC through all available sources
8. Export enriched IOCs as STIX 2.1 indicators

Tools & resources

- VirusTotal API v3
- AbuseIPDB API v2
- Shodan API
- STIX 2.1 Export
- Rate Limits

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `classify_ioc` — Auto-detect IOC type from value.
- `enrich_ip_virustotal` — Enrich an IP address via VirusTotal API v3.
- `enrich_hash_virustotal` — Enrich a file hash via VirusTotal API v3.
- `enrich_domain_virustotal` — Enrich a domain via VirusTotal API v3.
- `enrich_ip_abuseipdb` — Check an IP against AbuseIPDB.
- `compute_confidence` — Calculate composite confidence score from enrichment sources.
- `enrich_ioc` — Enrich a single IOC through all available sources.
- `export_stix_indicators` — Export enriched IOCs as STIX 2.1 indicators.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `automating-ioc-enrichment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

brainstorming (Skill Module)

`superjawn\skills\brainstorming\SKILL.md`

name: brainstorming
description: You MUST use this before any creative work - creating features, building components, adding functionality, or modifying behavior. Explores user intent, requirements and design before implementation.
tags:

Brainstorming Ideas Into Designs

Help turn ideas into fully formed designs and specs through natural collaborative dialogue.

Start by understanding the current project context, then ask questions one at a time to refine the idea. Once you understand what you're building, present the design and get user approval.

Do NOT invoke any implementation [[skill]], write any code, scaffold any project, or take any implementation action until you have presented a design and the user has approved it. This applies to EVERY project regardless of perceived simplicity.

Anti-Pattern: "This Is Too Simple To Need A Design"

Every project goes through this process. A todo list, a single-function utility, a config change — all of them. "Simple" projects are where unexamined assumptions cause the most wasted work. The design can be short (a few sentences for truly simple projects), but you MUST present it and get approval.

Checklist

You MUST create a task for each of these items and complete them in order:

1. **Explore project context** — check files, docs, recent commits
2. **Offer visual companion** (if topic will involve visual questions) — this is its own message, not combined with a clarifying question. See the Visual Companion section below.
3. **Ask clarifying questions** — one at a time, understand purpose/constraints/success criteria
4. **Research phase** — gather outside context (default-on; skip only with explicit justification). See "Research phase" section below.
5. **Propose 2-3 approaches** — with trade-offs and your recommendation
6. **Present design** — in sections scaled to their complexity, get user approval after each section
7. **Write design doc** — save to `docs/[[superpowers]]/specs/YYYY-MM-DD-<topic>-design.md` and commit
8. **Spec self-[[review]]** — quick inline check for placeholders, contradictions, ambiguity, scope (see below)
9. **User reviews written spec** — ask user to [[review]] the spec file before proceeding
10. **Transition to implementation** — invoke superjawn:writing-plans [[skill]] to create implementation plan

Research phase

After clarifying questions, before proposing approaches, gather outside context. This is **default-on**: skip only with explicit, justified statement.

1. Pick research kinds

From the menu — trends + discourse, patterns, pitfalls, authoritative verification, user-context.

For brainstorming, the **defaults are: web (trends + discourse) and codebase (prior art)**. Add others if the topic warrants — e.g. authoritative verification when an external API is in scope, or user-context when prior decisions in memory are relevant.

2. Dispatch

Subagent by default:
- `Explore` for codebase / prior-art questions ("does this repo already have something like X?", "what's the convention for Y here?")
- `general-purpose` for web / discourse / verification ("what's the current best practice for Z?", "what pitfalls do people hit with W?")
- Run multiple in parallel when the kinds are independent

Inline only for light-touch research (single grep, memory check).

3. Record findings

Write 3–5 tight bullets into the spec doc under a new `## Research notes` section. Include load-bearing links/refs and anything considered-but-ruled-out so future-you knows it was checked.

4. Skip protocol

If skipping, write one line into the spec doc: `Skipped research because <reason>. <Verifiable pointer if applicable>.`

Valid reasons:
- Trivial scope (typo, comment edit, single-line config)
- Fresh prior research — same topic in current session OR within last 7 days with verifiable spec/plan pointer. **If the pointer doesn't resolve, the skip is invalid.** (Beyond 7 days, repeat the research even if you remember the prior findings — the landscape drifts.)
- User explicit — **must quote the phrase** that authorized the skip.
- Repeat of identical task — **must include a pointer** to the prior successful run.

Invalid reasons: "I think I know", "seems straightforward", "moving fast", "user wants this done quickly", "already familiar with this codebase". If those are tempting, do the research.

Process Flow

```
digraph brainstorming {
    "Explore project context" [shape=box];
    "Visual questions ahead?" [shape=diamond];
    "Offer Visual Companion\n(own message, no other content)" [shape=box];
    "Ask clarifying questions" [shape=box];
    "Research phase" [shape=box];
    "Propose 2-3 approaches" [shape=box];
    "Present design sections" [shape=box];
    "User approves design?" [shape=diamond];
    "Write design doc" [shape=box];
    "Spec self-review\n(fix inline)" [shape=box];
    "User reviews spec?" [shape=diamond];
    "Invoke superjawn:writing-plans skill" [shape=doublecircle];

    "Explore project context" -> "Visual questions ahead?";
    "Visual questions ahead?" -> "Offer Visual Companion\n(own message, no other content)" [label="yes"];
    "Visual questions ahead?" -> "Ask clarifying questions" [label="no"];
    "Offer Visual Companion\n(own message, no other content)" -> "Ask clarifying questions";
    "Ask clarifying questions" -> "Research phase";
    "Research phase" -> "Propose 2-3 approaches";
    "Propose 2-3 approaches" -> "Present design sections";
    "Present design sections" -> "User approves design?";
    "User approves design?" -> "Present design sections" [label="no, revise"];
    "User approves design?" -> "Write design doc" [label="yes"];
    "Write design doc" -> "Spec self-review\n(fix inline)";
    "Spec self-review\n(fix inline)" -> "User reviews spec?";
    "User reviews spec?" -> "Write design doc" [label="changes requested"];
    "User reviews spec?" -> "Invoke superjawn:writing-plans skill" [label="approved"];
}
```

The terminal state is invoking superjawn:writing-plans. Do NOT invoke frontend-design, mcp-builder, or any other implementation [[skill]]. The ONLY [[skill]] you invoke after brainstorming is superjawn:writing-plans.

The Process

Understanding the idea:

- Check out the current project state first (files, docs, recent commits)
- Before asking detailed questions, assess scope: if the request describes multiple independent subsystems (e.g., "build a platform with chat, file storage, billing, and analytics"), flag this immediately. Don't spend questions refining details of a project that needs to be decomposed first.
- If the project is too large for a single spec, help the user decompose into sub-projects: what are the independent pieces, how do they relate, what order should they be built? Then brainstorm the first sub-project through the normal design flow. Each sub-project gets its own spec → plan → implementation cycle.
- For appropriately-scoped projects, ask questions one at a time to refine the idea
- Prefer multiple choice questions when possible, but open-ended is fine too
- Only one question per message - if a topic needs more exploration, break it into multiple questions
- Focus on understanding: purpose, constraints, success criteria

Exploring approaches:

- Propose 2-3 different approaches with trade-offs
- Present options conversationally with your recommendation and reasoning
- Lead with your recommended option and explain why

Presenting the design:

- Once you believe you understand what you're building, present the design
- Scale each section to its complexity: a few sentences if straightforward, up to 200-300 words if nuanced

- Ask after each section whether it looks right so far
- Cover: architecture, components, data flow, error handling, testing
- Be ready to go back and clarify if something doesn't make sense

Design for isolation and clarity:

- Break the system into smaller units that each have one clear purpose, communicate through well-defined interfaces, and can be understood and tested independently
- For each unit, you should be able to answer: what does it do, how do you use it, and what does it depend on?
- Can someone understand what a unit does without reading its internals? Can you change the internals without breaking consumers? If not, the boundaries need work.
- Smaller, well-bounded units are also easier for you to work with - you reason better about code you can hold in context at once, and your edits are more reliable when files are focused. When a file grows large, that's often a signal that it's doing too much.

Working in existing codebases:

- Explore the current structure before proposing changes. Follow existing patterns.
- Where existing code has problems that affect the work (e.g., a file that's grown too large, unclear boundaries, tangled responsibilities), include targeted improvements as part of the design - the way a good developer improves code they're working in.
- Don't propose unrelated refactoring. Stay focused on what serves the current goal.

After the Design

Documentation:

- Write the validated design (spec) to `docs/[[superpowers]]/specs/YYYY-MM-DD-<topic>-design.md`
- (User preferences for spec location override this default)
- Use elements-of-style:writing-clearly-and-concisely [[skill]] if available
- Commit the design document to git

Spec Self-[[Review]]:
After writing the spec document, look at it with fresh eyes:

1. **Placeholder scan:** Any "TBD", "TODO", incomplete sections, or vague requirements? Fix them.
2. **Internal consistency:** Do any sections contradict each other? Does the architecture match the feature descriptions?
3. **Scope check:** Is this focused enough for a single implementation plan, or does it need decomposition?
4. **Ambiguity check:** Could any requirement be interpreted two different ways? If so, pick one and make it explicit.
5. **Research phase check:** Does the spec contain either a `## Research notes` section with findings, or a one-line `Skipped research because <reason>` declaration? If neither, the brainstorming flow didn't run correctly — go back to the research phase before continuing.

Fix any issues inline. No need to re-[[review]] — just fix and move on.

User [[Review]] Gate:
After the spec [[review]] loop passes, ask the user to [[review]] the written spec before proceeding:

> "*Spec written and committed to `<path>`. Please [[review]] it and let me know if you want to make any changes before we start writing out the implementation plan.*"

Wait for the user's response. If they request changes, make them and re-run the spec [[review]] loop. Only proceed once the user approves.

Implementation:

- Invoke the superjawn:writing-plans [[skill]] to create a detailed implementation plan
- Do NOT invoke any other [[skill]]. superjawn:writing-plans is the next step.

Key Principles

- **One question at a time** - Don't overwhelm with multiple questions
- **Multiple choice preferred** - Easier to answer than open-ended when possible
- **YAGNI ruthlessly** - Remove unnecessary features from all designs
- **Explore alternatives** - Always propose 2-3 approaches before settling
- **Incremental validation** - Present design, get approval before moving on
- **Be flexible** - Go back and clarify when something doesn't make sense

Visual Companion

A browser-based companion for showing mockups, diagrams, and visual options during brainstorming. Available as a tool — not a mode. Accepting the companion means it's available for questions that benefit from visual treatment; it does NOT mean every question goes through the browser.

Offering the companion: When you anticipate that upcoming questions will involve visual content (mockups, layouts, diagrams), offer it once for consent:

> "*Some of what we're working on might be easier to explain if I can show it to you in a web browser. I can put together mockups, diagrams, comparisons, and other visuals as we go. This feature is still new and can be token-intensive. Want to try it? (Requires opening a local URL)*"

This offer MUST be its own message. Do not combine it with clarifying questions, context summaries, or any other content. The message should contain ONLY the offer above and nothing else. Wait for the user's response before continuing. If they decline, proceed with text-only brainstorming.

Per-question decision: Even after the user accepts, decide FOR EACH QUESTION whether to use the browser or the terminal. The test: **would the user understand this better by seeing it than reading it?**

- **Use the browser** for content that IS visual — mockups, wireframes, layout comparisons, architecture diagrams, side-by-side visual designs
- **Use the terminal** for content that is text — requirements questions, conceptual choices, tradeoff lists, A/B/C/D text options, scope decisions

A question about a UI topic is not automatically a visual question. "What does personality mean in this context?" is a conceptual question — use the terminal. "Which wizard layout works better?" is a visual question — use the browser.

If they agree to the companion, read the detailed guide before proceeding:
`[[skills]]/brainstorming/[[visual-companion]].md`

building-adversary-infrastructure-tracking-system (Skill Module)

`building-adversary-infrastructure-tracking-system\SKILL.md`

name: building-adversary-infrastructure-tracking-system
description: Adversary Infrastructure Tracking Agent - Tracks threat actor infrastructure using passive DNS and certificate transparency. Use when the user asks to build adversary infrastructure tracking system.
version: 1.0.0
tags:
source: building-adversary-infrastructure-tracking-system

Building Adversary Infrastructure Tracking System

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: URLhaus API, ThreatFox API, Pivoting Techniques, Infrastructure Relationships

When to use

- The user wants to **build** adversary infrastructure tracking system.
- Capability: Adversary Infrastructure Tracking Agent - Tracks threat actor infrastructure using passive DNS and certificate transparency.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, adversary, infrastructure, tracking, system`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query crt.sh certificate transparency for domain certificates
2. Query URLhaus for malicious URL hosting information
3. Query ThreatFox for IOC intelligence
4. Pivot on certificate data to find related infrastructure
5. Build infrastructure map from seed IOCs
6. Generate infrastructure tracking report

Tools & resources

- URLhaus API
- ThreatFox API
- Pivoting Techniques
- Infrastructure Relationships

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_crtsh` — Query crt.sh certificate transparency for domain certificates.
- `query_urlhaus` — Query URLhaus for malicious URL hosting information.
- `query_threatfox` — Query ThreatFox for IOC intelligence.
- `pivot_on_certificate` — Pivot on certificate data to find related infrastructure.
- `build_infrastructure_map` — Build infrastructure map from seed IOCs.
- `generate_report` — Generate infrastructure tracking report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-adversary-infrastructure-tracking-system`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-attack-pattern-library-from-cti-reports (Skill Module)

`building-attack-pattern-library-from-cti-reports\SKILL.md`

name: building-attack-pattern-library-from-cti-reports
description: Attack Pattern Library Builder Agent - Extracts attack patterns from CTI reports and maps to MITRE ATT&CK. Use when the user asks to build attack pattern library from cti reports.
version: 1.0.0
tags:
source: building-attack-pattern-library-from-cti-reports

Building Attack Pattern Library from CTI Reports

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: IOC Extraction Regex, Library Output Structure

When to use

- The user wants to **build** attack pattern library from cti reports.
- Capability: Attack Pattern Library Builder Agent - Extracts attack patterns from CTI reports and maps to MITRE ATT&CK.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, attack, pattern, library, reports`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract MITRE ATT&CK techniques from report text
2. Extract IOCs from report text
3. Process a single CTI report to extract attack patterns
4. Build a consolidated attack pattern library from multiple reports
5. Generate attack pattern library report

Tools & resources

- IOC Extraction Regex
- Library Output Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_techniques_from_text` — Extract MITRE ATT&CK techniques from report text.
- `extract_iocs_from_text` — Extract IOCs from report text.
- `process_report` — Process a single CTI report to extract attack patterns.
- `build_pattern_library` — Build a consolidated attack pattern library from multiple reports.
- `generate_report` — Generate attack pattern library report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-attack-pattern-library-from-cti-reports`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-automated-malware-submission-pipeline (Skill Module)

building-automated-malware-submission-pipeline\SKILL.md

name: building-automated-malware-submission-pipeline
description: Agent for building an automated malware submission and analysis pipeline. Use when the user asks to build automated malware submission pipeline.
version: 1.0.0
tags:
source: building-automated-malware-submission-pipeline

Building Automated Malware Submission Pipeline

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: VirusTotal API v3, MalwareBazaar API, Cuckoo Sandbox REST API

When to use

- The user wants to **build** automated malware submission pipeline.
- Capability: Agent for building an automated malware submission and analysis pipeline.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, automated, malware, submission, pipeline`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate MD5, SHA1, SHA256 hashes and file size
2. Check if a file hash is known in VirusTotal
3. Check if a hash is known in MalwareBazaar
4. Submit a file to Cuckoo Sandbox for dynamic analysis
5. Poll Cuckoo until analysis is complete, then return the report
6. Extract IOCs from a Cuckoo sandbox report
7. Generate a verdict combining pre-screening and sandbox results
8. Send analysis results to Splunk via HTTP Event Collector

Tools & resources

- VirusTotal API v3
- MalwareBazaar API
- Cuckoo Sandbox REST API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Calculate MD5, SHA1, SHA256 hashes and file size.
- `check_virustotal` — Check if a file hash is known in VirusTotal.
- `check_malwarebazaar` — Check if a hash is known in MalwareBazaar.
- `submit_to_cuckoo` — Submit a file to Cuckoo Sandbox for dynamic analysis.
- `wait_for_cuckoo_report` — Poll Cuckoo until analysis is complete, then return the report.
- `extract_iocs_from_report` — Extract IOCs from a Cuckoo sandbox report.
- `generate_verdict` — Generate a verdict combining pre-screening and sandbox results.
- `push_to_splunk_hec` — Send analysis results to Splunk via HTTP Event Collector.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-automated-malware-submission-pipeline`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-c2-infrastructure-with-sliver-framework (Skill Module)

`building-c2-infrastructure-with-sliver-framework\SKILL.md`

name: building-c2-infrastructure-with-sliver-framework
description: Sliver C2 Framework Deployment Agent - Automates Sliver C2 setup for authorized red team engagements. Use when the user asks to build c2 infrastructure with sliver framework.
version: 1.0.0
tags:
source: building-c2-infrastructure-with-sliver-framework

Building C2 Infrastructure with Sliver Framework

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Sliver, Listener Types, Post-Exploitation, Industry Standards, Official Documentation, Key Research

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** c2 infrastructure with sliver framework.
- Capability: Sliver C2 Framework Deployment Agent - Automates Sliver C2 setup for authorized red team engagements.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, infrastructure, sliver, framework`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute a Sliver client command
2. Generate a Sliver implant (beacon or session)
3. Generate a Sliver beacon implant
4. Configure C2 listeners
5. List active sessions
6. List active beacons
7. Generate C2 infrastructure report

Key concepts

- BishopFox Red Team Tools and C2 Frameworks Report (2025)
- SpecterOps Adversary Simulation methodology
- SANS SEC565: Red Team Operations and Adversary Emulation

Tools & resources

- Sliver
- Listener Types
- Post-Exploitation
- Industry Standards
- Official Documentation
- Key Research

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_sliver_cmd` — Execute a Sliver client command.
- `generate_implant` — Generate a Sliver implant (beacon or session).
- `generate_beacon` — Generate a Sliver beacon implant.
- `setup_listeners` — Configure C2 listeners.
- `list_sessions` — List active sessions.
- `list_beacons` — List active beacons.
- `generate_report` — Generate C2 infrastructure report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-c2-infrastructure-with-sliver-framework`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-cloud-siem-with-sentinel (Skill Module)

`building-cloud-siem-with-sentinel\SKILL.md`

name: building-cloud-siem-with-sentinel
description: Agent for managing Microsoft Sentinel SIEM operations. Use when the user asks to build cloud siem with sentinel.
version: 1.0.0
tags:
source: building-cloud-siem-with-sentinel

Building Cloud SIEM with Sentinel

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Microsoft Sentinel, azure-monitor, azure-mgmt-securityinsight, Sentinel Data Connectors

When to use

- The user wants to **build** cloud siem with sentinel.
- Capability: Agent for managing Microsoft Sentinel SIEM operations.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, cloud, siem, sentinel`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Get Azure credential for authentication
2. Execute a KQL query against a Log Analytics workspace
3. Detect impossible travel sign-ins using KQL
4. Detect AWS credential abuse via CloudTrail in Sentinel
5. Detect mass S3 object deletion (potential ransomware)
6. Get incident summary from Sentinel
7. Match network traffic against threat intelligence indicators
8. List active Sentinel analytics rules

Tools & resources

- Microsoft Sentinel
- azure-monitor
- azure-mgmt-securityinsight
- Sentinel Data Connectors

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_credential` — Get Azure credential for authentication.
- `run_kql_query` — Execute a KQL query against a Log Analytics workspace.
- `detect_impossible_travel` — Detect impossible travel sign-ins using KQL.
- `detect_aws_credential_abuse` — Detect AWS credential abuse via CloudTrail in Sentinel.
- `detect_mass_deletion` — Detect mass S3 object deletion (potential ransomware).
- `get_incident_summary` — Get incident summary from Sentinel.
- `match_threat_intelligence` — Match network traffic against threat intelligence indicators.
- `list_analytics_rules` — List active Sentinel analytics rules.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-cloud-siem-with-sentinel`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-detection-rule-with-splunk-spl (Skill Module)

building-detection-rule-with-splunk-spl\SKILL.md

name: building-detection-rule-with-splunk-spl
description: Splunk SPL Detection Rule Builder Agent - Generates and validates Splunk detection rules. Use when the user asks to build detection rule with splunk spl.
version: 1.0.0
tags:
source: building-detection-rule-with-splunk-spl

Building Detection Rule with Splunk SPL

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Splunk, Alert Severity Levels, Industry Standards, Detection Engineering Maturity Model

When to use

- The user wants to **build** detection rule with splunk spl.
- Capability: Splunk SPL Detection Rule Builder Agent - Generates and validates Splunk detection rules.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, detection, rule, splunk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate an SPL detection rule from template
2. Deploy a saved search to Splunk via REST API
3. Generate detection rule report

Key concepts

- Primary mapping standard for detection rule categorization
- Version 18.1 (December 2025) is the latest release
- Use ATT&CK Navigator for visual coverage mapping
- Standard field naming convention for normalized data
- Data models: Authentication, Network_Traffic, Endpoint, Web, Email
- Enables cross-sourcetype correlation searches
- Log management planning and policy guidance
- Defines log collection, analysis, and retention best practices

Tools & resources

- Splunk
- Alert Severity Levels
- Industry Standards
- Detection Engineering Maturity Model

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_spl_rule` — Generate an SPL detection rule from template.
- `deploy_saved_search` — Deploy a saved search to Splunk via REST API.
- `generate_report` — Generate detection rule report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-detection-rule-with-splunk-spl`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-detection-rules-with-sigma (Skill Module)

building-detection-rules-with-sigma\SKILL.md

name: building-detection-rules-with-sigma
description: Agent for building and converting Sigma detection rules. Use when the user asks to build detection rules with sigma.
version: 1.0.0
tags:
source: building-detection-rules-with-sigma

Building Detection Rules with Sigma

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Sigma

When to use

- The user wants to **build** detection rules with sigma.
- Capability: Agent for building and converting Sigma detection rules.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, detection, rules, sigma`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load a Sigma rule from a YAML file
2. Load all Sigma rules from a directory
3. Convert a Sigma rule to Splunk SPL query
4. Convert a Sigma rule to Splunk saved search format
5. Validate a Sigma rule for syntax and best practices
6. Extract MITRE ATT&CK technique IDs from Sigma rules
7. Generate a MITRE ATT&CK Navigator layer JSON from extracted techniques
8. Batch convert all Sigma rules in a directory to target backend

Tools & resources

- Sigma

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_sigma_rule` — Load a Sigma rule from a YAML file.
- `load_sigma_directory` — Load all Sigma rules from a directory.
- `convert_to_splunk` — Convert a Sigma rule to Splunk SPL query.
- `convert_to_splunk_savedsearch` — Convert a Sigma rule to Splunk saved search format.
- `validate_sigma_rule` — Validate a Sigma rule for syntax and best practices.
- `extract_attack_techniques` — Extract MITRE ATT&CK technique IDs from Sigma rules.
- `generate_attack_navigator_layer` — Generate a MITRE ATT&CK Navigator layer JSON from extracted techniques.
- `batch_convert` — Batch convert all Sigma rules in a directory to target backend.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-detection-rules-with-sigma`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-devsecops-pipeline-with-gitlab-ci (Skill Module)

`building-devsecops-pipeline-with-gitlab-ci\SKILL.md`

name: building-devsecops-pipeline-with-gitlab-ci
description: DevSecOps Pipeline Builder Agent - Generates GitLab CI security scanning pipeline configurations. Use when the user asks to build devsecops pipeline with gitlab ci.
version: 1.0.0
tags:
source: building-devsecops-pipeline-with-gitlab-ci

Building Devsecops Pipeline with Gitlab Ci

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: GitLab CI, GitLab Security Templates, Security Variables, Security Scanning Tools, NIST SP 800, GitLab Scanner Coverage Matrix

When to use

- The user wants to **build** devsecops pipeline with gitlab ci.
- Capability: DevSecOps Pipeline Builder Agent - Generates GitLab CI security scanning pipeline configurations.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, devsecops, pipeline, gitlab`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate .gitlab-ci.yml with security scanning stages
2. Validate CI configuration via GitLab API
3. Assess security coverage of the pipeline
4. Generate DevSecOps pipeline report

Key concepts

- SCS-1: Secure source code management with protected branches and signed commits
- SCS-2: Secure build pipelines with pinned template versions and runner isolation
- SCS-3: Verified dependencies through dependency scanning and license compliance
- SCS-4: Secure artifacts with container scanning and signed images
- SCS-5: Deployment security with manual gates and environment approvals

Tools & resources

- GitLab CI
- GitLab Security Templates
- Security Variables
- Security Scanning Tools
- NIST SP 800
- GitLab Scanner Coverage Matrix

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_gitlab_ci` — Generate .gitlab-ci.yml with security scanning stages.
- `validate_pipeline` — Validate CI configuration via GitLab API.
- `assess_pipeline_coverage` — Assess security coverage of the pipeline.
- `generate_report` — Generate DevSecOps pipeline report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-devsecops-pipeline-with-gitlab-ci`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-identity-federation-with-saml-azure-ad (Skill Module)

`building-identity-federation-with-saml-azure-ad\SKILL.md`

name: building-identity-federation-with-saml-azure-ad
description: SAML Azure AD Federation Agent - Configures and validates SAML SSO with Azure AD. Use when the user asks to build identity federation with saml azure ad.
version: 1.0.0
tags:
source: building-identity-federation-with-saml-azure-ad

Building Identity Federation with SAML Azure AD

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Azure, SAML, Federation Metadata URL, SP Metadata Required Fields, Validation Checks, Federation Protocols

When to use

- The user wants to **build** identity federation with saml azure ad.
- Capability: SAML Azure AD Federation Agent - Configures and validates SAML SSO with Azure AD.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, identity, federation, saml, azure`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Fetch Azure AD SAML federation metadata
2. Parse SAML federation metadata XML
3. Generate Service Provider SAML metadata
4. Validate SAML federation configuration
5. Generate SAML federation report

Key concepts

- Security Assertion Markup Language version 2.0
- XML-based framework for exchanging authentication/authorization data
- Profiles: Web Browser SSO, Enhanced Client or Proxy, Single Logout
- Web Services Federation Language
- Used by AD FS for passive federation (browser-based)
- Token types: SAML 1.1, SAML 2.0
- Used by Azure AD as primary protocol for modern applications
- JWT tokens instead of SAML assertions

Tools & resources

- Azure
- SAML
- Federation Metadata URL
- SP Metadata Required Fields
- Validation Checks
- Federation Protocols

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `fetch_federation_metadata` — Fetch Azure AD SAML federation metadata.
- `parse_metadata` — Parse SAML federation metadata XML.
- `generate_sp_metadata` — Generate Service Provider SAML metadata.
- `validate_configuration` — Validate SAML federation configuration.
- `generate_report` — Generate SAML federation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-identity-federation-with-saml-azure-ad`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-identity-governance-lifecycle-process (Skill Module)

`building-identity-governance-lifecycle-process\SKILL.md`

name: building-identity-governance-lifecycle-process
description: Agent for identity governance and lifecycle management operations. Use when the user asks to build identity governance lifecycle process.
version: 1.0.0
tags:
source: building-identity-governance-lifecycle-process

Building Identity Governance Lifecycle Process

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Microsoft Graph API, SailPoint IdentityNow API

When to use

- The user wants to **build** identity governance lifecycle process.
- Capability: Agent for identity governance and lifecycle management operations.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, identity, governance, lifecycle, process`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate to Microsoft Graph API via client credentials
2. List users from Microsoft Entra ID with optional filter
3. Find accounts with no sign-in activity for N days
4. Find guest accounts with no recent activity
5. List active access reviews from Entra ID Governance
6. Identify users without registered MFA methods
7. Generate comprehensive identity governance report

Tools & resources

- Microsoft Graph API
- SailPoint IdentityNow API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_graph_token` — Authenticate to Microsoft Graph API via client credentials.
- `list_users` — List users from Microsoft Entra ID with optional filter.
- `detect_orphaned_accounts` — Find accounts with no sign-in activity for N days.
- `detect_stale_guests` — Find guest accounts with no recent activity.
- `get_access_reviews` — List active access reviews from Entra ID Governance.
- `check_users_without_mfa` — Identify users without registered MFA methods.
- `generate_lifecycle_report` — Generate comprehensive identity governance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-identity-governance-lifecycle-process`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-incident-response-dashboard (Skill Module)

`building-incident-response-dashboard\SKILL.md`

name: building-incident-response-dashboard
description: Agent for building and managing incident response dashboards in Splunk. Use when the user asks to build incident response dashboard.
version: 1.0.0
tags:
source: building-incident-response-dashboard

Building Incident Response Dashboard

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: splunk

When to use

- The user wants to **build** incident response dashboard.
- Capability: Agent for building and managing incident response dashboards in Splunk.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, incident, response, dashboard`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to Splunk instance
2. Execute a Splunk search and return results
3. Get summary of a specific incident from notable events
4. Track systems affected by an incident
5. Monitor IOC hits across the environment
6. Calculate SOC operational metrics
7. Get analyst workload distribution
8. Get alert disposition breakdown

Tools & resources

- splunk

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_splunk` — Connect to Splunk instance.
- `run_search` — Execute a Splunk search and return results.
- `get_incident_summary` — Get summary of a specific incident from notable events.
- `get_affected_systems` — Track systems affected by an incident.
- `get_ioc_spread` — Monitor IOC hits across the environment.
- `get_soc_metrics` — Calculate SOC operational metrics.
- `get_analyst_workload` — Get analyst workload distribution.
- `get_alert_disposition` — Get alert disposition breakdown.
- `get_incident_timeline` — Build chronological incident timeline.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-incident-response-dashboard`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-incident-response-playbook (Skill Module)

`building-incident-response-playbook\SKILL.md`

name: building-incident-response-playbook
description: Agent for building and managing incident response playbooks. Use when the user asks to build incident response playbook.
version: 1.0.0
tags:
source: building-incident-response-playbook

Building Incident Response Playbook

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: NIST SP 800-61r3 Phases

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** incident response playbook.
- Capability: Agent for building and managing incident response playbooks.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, incident, response, playbook`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate a structured IR playbook for the given incident type
2. List open incident cases from TheHive
3. Calculate incident response metrics from TheHive cases

Tools & resources

- NIST SP 800-61r3 Phases

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_playbook` — Generate a structured IR playbook for the given incident type.
- `check_thehive_cases` — List open incident cases from TheHive.
- `calculate_ir_metrics` — Calculate incident response metrics from TheHive cases.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-incident-response-playbook`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-incident-timeline-with-timesketch (Skill Module)

`building-incident-timeline-with-timesketch\SKILL.md`

name: building-incident-timeline-with-timesketch
description: Incident Timeline Builder Agent - Creates forensic timelines using Timesketch API. Use when the user asks to build incident timeline with timesketch.
version: 1.0.0
tags:
source: building-incident-timeline-with-timesketch

Building Incident Timeline with Timesketch

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Timesketch, Timeline Upload, Analyzers, Python Client, NIST SP 800-86, SANS FOR508

When to use

- The user wants to **build** incident timeline with timesketch.
- Capability: Incident Timeline Builder Agent - Creates forensic timelines using Timesketch API.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, incident, timeline, timesketch`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Authenticate to Timesketch and return session
2. List all sketches
3. Create a new sketch
4. Upload a Plaso/CSV/JSONL timeline to a sketch
5. Search events in a sketch using OpenSearch query string
6. Analyze events to build timeline summary with key milestones
7. Tag events for annotation
8. Generate incident timeline report

Key concepts

- Provides guidelines for collecting, examining, and analyzing digital evidence
- Emphasizes importance of timeline reconstruction in incident investigation
- Defines evidence handling procedures for forensic analysis
- Super timeline creation methodology
- Evidence source prioritization for timeline building
- Plaso/log2timeline artifact parsing techniques
- Timeline analysis for APT detection
- Standard format for representing dates and times in timelines

Tools & resources

- Timesketch
- Timeline Upload
- Analyzers
- Python Client
- NIST SP 800-86
- SANS FOR508

- Plaso/log2timeline Documentation
- Sigma Detection Standard

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_session` — Authenticate to Timesketch and return session.
- `list_sketches` — List all sketches.
- `create_sketch` — Create a new sketch.
- `upload_timeline` — Upload a Plaso/CSV/JSONL timeline to a sketch.
- `search_events` — Search events in a sketch using OpenSearch query string.
- `build_timeline_summary` — Analyze events to build timeline summary with key milestones.
- `tag_events` — Tag events for annotation.
- `generate_report` — Generate incident timeline report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-incident-timeline-with-timesketch`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-ioc-defanging-and-sharing-pipeline (Skill Module)

`building-ioc-defanging-and-sharing-pipeline\SKILL.md`

name: building-ioc-defanging-and-sharing-pipeline
description: IOC Defanging and Sharing Pipeline Agent - Defangs, enriches, and shares IOCs in STIX format. Use when the user asks to build ioc defanging and sharing pipeline.
version: 1.0.0
tags:
source: building-ioc-defanging-and-sharing-pipeline

Building IOC Defanging and Sharing Pipeline

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: IOC Defanging Rules, VirusTotal API v3, AbuseIPDB API v2, TAXII 2.1 Sharing

When to use

- The user wants to **build** ioc defanging and sharing pipeline.
- Capability: IOC Defanging and Sharing Pipeline Agent - Defangs, enriches, and shares IOCs in STIX format.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, defanging, sharing, pipeline`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract IOCs from raw text
2. Defang an IOC for safe sharing
3. Refang a defanged IOC back to original form
4. Enrich IOC with threat intelligence from VirusTotal and AbuseIPDB
5. Convert IOCs to STIX 2.1 bundle for sharing
6. Defang all extracted IOCs
7. Generate IOC sharing pipeline report

Tools & resources

- IOC Defanging Rules
- VirusTotal API v3
- AbuseIPDB API v2
- TAXII 2.1 Sharing

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_iocs` — Extract IOCs from raw text.
- `defang_ioc` — Defang an IOC for safe sharing.
- `refang_ioc` — Refang a defanged IOC back to original form.
- `enrich_ioc` — Enrich IOC with threat intelligence from VirusTotal and AbuseIPDB.
- `to_stix_bundle` — Convert IOCs to STIX 2.1 bundle for sharing.
- `defang_all` — Defang all extracted IOCs.
- `generate_report` — Generate IOC sharing pipeline report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-ioc-defanging-and-sharing-pipeline`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-ioc-enrichment-pipeline-with-opencti (Skill Module)

`building-ioc-enrichment-pipeline-with-opencti\SKILL.md`

name: building-ioc-enrichment-pipeline-with-opencti
description: IOC enrichment pipeline using OpenCTI and the pycti Python client. Queries OpenCTI's GraphQL API to enrich indicators of compromise with threat context, relationships, and scoring from connected intelligence sources. Use when the user asks to build ioc enrichment pipeline with opencti.
version: 1.0.0
tags:
source: building-ioc-enrichment-pipeline-with-opencti

Building IOC Enrichment Pipeline with OpenCTI

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: OpenCTI, pycti, Enrichment Service APIs

When to use

- The user wants to **build** ioc enrichment pipeline with opencti.
- Capability: IOC enrichment pipeline using OpenCTI and the pycti Python client. Queries OpenCTI's GraphQL API to enrich indicators of compromise with threat context, relationships, and scoring from connected intelligence sources.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, enrichment, pipeline, opencti`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Initialize OpenCTI API client
2. Enrich a single indicator via OpenCTI GraphQL API
3. Enrich a STIX Cyber Observable via OpenCTI
4. Get STIX relationships for an entity
5. Classify IOC type from value string
6. Build enrichment report for a list of IOCs

Key concepts

- Indicator: Contains detection patterns (STIX patterning, YARA, Sigma)
- Malware: Represents malware families and variants
- Threat Actor: Describes adversary groups and individuals
- Campaign: Groups related intrusion activity
- Attack Pattern: Maps to MITRE ATT&CK techniques
- Infrastructure: Represents adversary-owned systems (C2, exploit kits)
- Tool: Legitimate software used by adversaries
- IPv4-Addr / IPv6-Addr: Network addresses

Tools & resources

- OpenCTI
- pycti
- Enrichment Service APIs

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `init_client` — Initialize OpenCTI API client.
- `enrich_indicator` — Enrich a single indicator via OpenCTI GraphQL API.
- `enrich_observable` — Enrich a STIX Cyber Observable via OpenCTI.
- `get_relationships` — Get STIX relationships for an entity.
- `classify_ioc` — Classify IOC type from value string.
- `build_enrichment_report` — Build enrichment report for a list of IOCs.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-ioc-enrichment-pipeline-with-opencti`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-malware-incident-communication-template (Skill Module)

building-malware-incident-communication-template\SKILL.md

name: building-malware-incident-communication-template
description: Malware Incident Communication Template Agent - Generates structured incident communications. Use when the user asks to build malware incident communication template.
version: 1.0.0
tags:
source: building-malware-incident-communication-template

Building Malware Incident Communication Template

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Severity Levels, Malware Categories, Regulatory Notification Deadlines, HIPAA Breach Notification Rule, PCI DSS Incident Response, CISA Incident Reporting

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** malware incident communication template.
- Capability: Malware Incident Communication Template Agent - Generates structured incident communications.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, malware, incident, communication, template`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate initial incident notification
2. Generate incident status update communication
3. Generate executive-level incident summary
4. Generate regulatory breach notification template
5. Generate complete set of communication templates
6. Generate communication template report

Key concepts

- Incident communication guidelines and templates
- Stakeholder notification requirements
- Media handling procedures
- Communication to affected data subjects
- Required content for breach notifications
- Individual notification requirements
- Media notification for large breaches
- Card brand notification requirements

Tools & resources

- Severity Levels
- Malware Categories
- Regulatory Notification Deadlines

- HIPAA Breach Notification Rule
- PCI DSS Incident Response
- CISA Incident Reporting
- ISO 27035

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_initial_notification` — Generate initial incident notification.
- `generate_status_update` — Generate incident status update communication.
- `generate_executive_summary` — Generate executive-level incident summary.
- `generate_regulatory_notification` — Generate regulatory breach notification template.
- `generate_full_template_set` — Generate complete set of communication templates.
- `generate_report` — Generate communication template report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-malware-incident-communication-template`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-patch-tuesday-response-process (Skill Module)

`building-patch-tuesday-response-process\SKILL.md`

name: building-patch-tuesday-response-process
description: Patch Tuesday Response Agent - Tracks Microsoft patches, assesses risk, and prioritizes deployment. Use when the user asks to build patch tuesday response process.
version: 1.0.0
tags:
source: building-patch-tuesday-response-process

Building Patch Tuesday Response Process

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: CVRF Vulnerability Fields, Patch Priority Matrix, NVD API v2, Deployment Phase Timeline, Microsoft Resources, Industry Standards

When to use

- The user wants to **build** patch tuesday response process.
- Capability: Patch Tuesday Response Agent - Tracks Microsoft patches, assesses risk, and prioritizes deployment.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, patch, tuesday, response, process`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Fetch Microsoft Security Update Guide data via MSRC API
2. Parse CVRF data to extract vulnerability details
3. Check CVEs against CISA Known Exploited Vulnerabilities catalog
4. Prioritize patches based on CVSS, exploitation status, and CISA KEV
5. Generate phased deployment plan
6. Generate Patch Tuesday response report

Key concepts

- NIST SP 800-40 Rev 4: Guide to Enterprise Patch Management Planning
- CIS Controls v8.1 Control 7.4: Perform Automated Patch Management
- PCI DSS v4.0 Req 6.3.3: Install security patches within one month of release
- ISO 27001: 2022 A.8.8**: Management of technical vulnerabilities

Tools & resources

- CVRF Vulnerability Fields
- Patch Priority Matrix
- NVD API v2
- Deployment Phase Timeline
- Microsoft Resources
- Industry Standards
- Vendor Analysis Resources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `fetch_patch_tuesday_updates` — Fetch Microsoft Security Update Guide data via MSRC API.
- `parse_vulnerabilities` — Parse CVRF data to extract vulnerability details.
- `check_cisa_kev` — Check CVEs against CISA Known Exploited Vulnerabilities catalog.
- `prioritize_patches` — Prioritize patches based on CVSS, exploitation status, and CISA KEV.
- `generate_deployment_plan` — Generate phased deployment plan.
- `generate_report` — Generate Patch Tuesday response report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-patch-tuesday-response-process`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-phishing-reporting-button-workflow (Skill Module)

building-phishing-reporting-button-workflow\SKILL.md

name: building-phishing-reporting-button-workflow
description: Phishing Reporting Button Workflow Agent - Processes user-reported phishing emails via button integration. Use when the user asks to build phishing reporting button workflow.
version: 1.0.0
tags:
source: building-phishing-reporting-button-workflow

Building Phishing Reporting Button Workflow

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Phishing Indicators, VirusTotal URL Scan, Dangerous File Extensions, Verdict Classification, Industry Standards, Reporting Platform Comparison

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** phishing reporting button workflow.
- Capability: Phishing Reporting Button Workflow Agent - Processes user-reported phishing emails via button integration.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, phishing, reporting, button, workflow`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse a reported phishing email from .eml file
2. Analyze reported email for phishing indicators
3. Check extracted URLs against VirusTotal
4. Create incident ticket from analysis results
5. Generate phishing report workflow report

Key concepts

- T1566.001: Phishing: Spearphishing Attachment
- T1566.002: Phishing: Spearphishing Link
- T1204: User Execution
- D3-RERE: User Reporting (MITRE D3FEND)
- NIST SP 800-61 Rev.2: Computer Security Incident Handling Guide
- CIS Controls v8 Control 14: Security Awareness and Skills Training
- ISO 27001 A.6.3: Information Security Awareness, Education and Training
- Report Rate: Percentage of phishing simulations reported (target: >70%)

Tools & resources

- Phishing Indicators
- VirusTotal URL Scan
- Dangerous File Extensions
- Verdict Classification

- Industry Standards
- Reporting Platform Comparison
- Key Metrics

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_reported_email` — Parse a reported phishing email from .eml file.
- `analyze_email` — Analyze reported email for phishing indicators.
- `check_urls_virustotal` — Check extracted URLs against VirusTotal.
- `create_ticket` — Create incident ticket from analysis results.
- `generate_report` — Generate phishing report workflow report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-phishing-reporting-button-workflow`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-ransomware-playbook-with-cisa-framework (Skill Module)

`building-ransomware-playbook-with-cisa-framework\SKILL.md`

name: building-ransomware-playbook-with-cisa-framework
description: CISA ransomware playbook builder and compliance checker agent. Generates a structured ransomware incident response playbook aligned with the CISA StopRansomware Guide. Assesses organizational readiness against CISA.... Use when the user asks to build ransomware playbook with cisa framework.
version: 1.0.0
tags:
source: building-ransomware-playbook-with-cisa-framework

Building Ransomware Playbook with Cisa Framework

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: CISA StopRansomware Guide, CISA Reporting, ID Ransomware Service, CISA Preparation Checklist Controls, Regulatory Notification Timelines

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** ransomware playbook with cisa framework.
- Capability: CISA ransomware playbook builder and compliance checker agent. Generates a structured ransomware incident response playbook aligned with the CISA StopRansomware Guide. Assesses organizational readiness against CISA checklist items and produces gap analysis reports.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, ransomware, playbook, cisa, framework`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Assess ransomware readiness against CISA checklist
2. Generate a full ransomware response playbook
3. Render playbook as Markdown document

Tools & resources

- CISA StopRansomware Guide
- CISA Reporting
- ID Ransomware Service
- CISA Preparation Checklist Controls
- Regulatory Notification Timelines

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `assess_readiness` — Assess ransomware readiness against CISA checklist.
- `generate_playbook` — Generate a full ransomware response playbook.
- `generate_markdown_playbook` — Render playbook as Markdown document.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-ransomware-playbook-with-cisa-framework`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-red-team-c2-infrastructure-with-havoc (Skill Module)

`building-red-team-c2-infrastructure-with-havoc\SKILL.md`

name: building-red-team-c2-infrastructure-with-havoc
description: Havoc C2 Infrastructure Builder Agent - Manages Havoc C2 framework deployment and listener setup. # For authorized penetration testing and lab environments only. Use when the user asks to build red team c2 infrastructure with havoc.
version: 1.0.0
tags:
source: building-red-team-c2-infrastructure-with-havoc

Building Red Team C2 Infrastructure with Havoc

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Havoc, HTTPS Listener Config, SMB Listener Config, Payload Generation, Agent Properties, NIST References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** red team c2 infrastructure with havoc.
- Capability: Havoc C2 Infrastructure Builder Agent - Manages Havoc C2 framework deployment and listener setup. # For authorized penetration testing and lab environments only.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, team, infrastructure, havoc`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Make authenticated request to Havoc teamserver API
2. List active listeners on the teamserver
3. Create an HTTPS listener
4. Create an SMB named pipe listener for pivoting
5. List connected agents (demons)
6. Generate a Havoc demon payload
7. Assess C2 infrastructure health and coverage
8. Generate C2 infrastructure report

Key concepts

- T1583.003** - Acquire Infrastructure: Virtual Private Server
- T1583.006** - Acquire Infrastructure: Web Services
- T1587.003** - Develop Capabilities: Digital Certificates
- T1608.001** - Stage Capabilities: Upload Malware
- T1071.001** - Application Layer Protocol: Web Protocols (HTTP/HTTPS)
- T1573.001** - Encrypted Channel: Symmetric Cryptography
- T1573.002** - Encrypted Channel: Asymmetric Cryptography
- T1090.001** - Proxy: Internal Proxy

Tools & resources

- Havoc
- HTTPS Listener Config
- SMB Listener Config
- Payload Generation
- Agent Properties
- NIST References
- Compliance Context

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `havoc_api_request` — Make authenticated request to Havoc teamserver API.
- `list_listeners` — List active listeners on the teamserver.
- `create_https_listener` — Create an HTTPS listener.
- `create_smb_listener` — Create an SMB named pipe listener for pivoting.
- `list_agents` — List connected agents (demons).
- `generate_payload` — Generate a Havoc demon payload.
- `assess_infrastructure` — Assess C2 infrastructure health and coverage.
- `generate_report` — Generate C2 infrastructure report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-red-team-c2-infrastructure-with-havoc`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-role-mining-for-rbac-optimization (Skill Module)

`building-role-mining-for-rbac-optimization\SKILL.md`

name: building-role-mining-for-rbac-optimization
description: Role Mining for RBAC Optimization Agent - Analyzes access patterns to optimize role-based access control. Use when the user asks to build role mining for rbac optimization.
version: 1.0.0
tags:
source: building-role-mining-for-rbac-optimization

Building Role Mining for RBAC Optimization

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Role Mining Algorithms, Optimization Metrics, CyberArk Identity Role Optimization, NIST RBAC Model Levels, RBAC Standards, Identity Governance Standards

When to use

- The user wants to **build** role mining for rbac optimization.
- Capability: Role Mining for RBAC Optimization Agent - Analyzes access patterns to optimize role-based access control.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, role, mining, rbac, optimization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load user-entitlement assignments from CSV (user,entitlement,system)
2. Build user-to-permission-set mapping
3. Bottom-up role mining: find common permission sets shared by multiple users
4. Top-down role mining: cluster users by permission similarity (Jaccard)
5. Detect users with unique permissions not covered by any candidate role
6. Calculate RBAC optimization metrics
7. Generate role mining report

Key concepts

- Defines User, Role, Permission, Session abstractions
- Role assignment: users are assigned to roles
- Permission assignment: permissions are assigned to roles
- Role hierarchy: senior roles inherit junior role permissions
- Separation of Duty constraints (static and dynamic)
- Core RBAC: Basic user-role and role-permission mappings
- Hierarchical RBAC: Role inheritance relationships
- Constrained RBAC: Static and dynamic separation of duties

Tools & resources

- Role Mining Algorithms
- Optimization Metrics
- CyberArk Identity Role Optimization
- NIST RBAC Model Levels
- RBAC Standards

- Identity Governance Standards
- Role Mining Research

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_entitlements` — Load user-entitlement assignments from CSV (user,entitlement,system).
- `build_user_permission_matrix` — Build user-to-permission-set mapping.
- `mine_roles_bottom_up` — Bottom-up role mining: find common permission sets shared by multiple users.
- `mine_roles_top_down` — Top-down role mining: cluster users by permission similarity (Jaccard).
- `detect_outliers` — Detect users with unique permissions not covered by any candidate role.
- `calculate_optimization_metrics` — Calculate RBAC optimization metrics.
- `generate_report` — Generate role mining report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-role-mining-for-rbac-optimization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-soc-escalation-matrix (Skill Module)

building-soc-escalation-matrix\SKILL.md

name: building-soc-escalation-matrix
description: SOC Escalation Matrix Agent - Builds and validates SOC escalation paths and response workflows. Use when the user asks to build soc escalation matrix.
version: 1.0.0
tags:
source: building-soc-escalation-matrix

Building SOC Escalation Matrix

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Priority Tiers, Alert Categories, Escalation Chain, Notification Channels, PagerDuty Incident API, Slack Webhook Notification

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **build** soc escalation matrix.
- Capability: SOC Escalation Matrix Agent - Builds and validates SOC escalation paths and response workflows.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, escalation, matrix`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Classify alert priority based on category, tags, and asset criticality
2. Build complete escalation matrix structure
3. Validate the escalation matrix for completeness and consistency
4. Simulate alert classification through the escalation matrix
5. Generate escalation matrix report

Key concepts

- Defines incident categories and severity levels
- Recommends functional impact, information impact, and recoverability as factors
- Guides escalation based on incident classification
- P1-P4 priority classification framework
- Impact x Urgency = Priority matrix
- SLA management for each priority level
- Level 1: Ad-hoc escalation, no formal process
- Level 2: Defined escalation paths, documented SLAs

Tools & resources

- Priority Tiers
- Alert Categories
- Escalation Chain
- Notification Channels
- PagerDuty Incident API

- Slack Webhook Notification
- Auto-Escalation Rules
- ITIL Incident Management

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `classify_alert` — Classify alert priority based on category, tags, and asset criticality.
- `build_escalation_matrix` — Build complete escalation matrix structure.
- `validate_escalation_matrix` — Validate the escalation matrix for completeness and consistency.
- `simulate_alerts` — Simulate alert classification through the escalation matrix.
- `generate_report` — Generate escalation matrix report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-soc-escalation-matrix`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-soc-metrics-and-kpi-tracking (Skill Module)

`building-soc-metrics-and-kpi-tracking\SKILL.md`

name: building-soc-metrics-and-kpi-tracking
description: SOC Metrics and KPI Tracking Agent - Collects and reports SOC performance metrics. Use when the user asks to build soc metrics and kpi tracking.
version: 1.0.0
tags:
source: building-soc-metrics-and-kpi-tracking

Building SOC Metrics and Kpi Tracking

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Arguments, Key Functions

When to use

- The user wants to **build** soc metrics and kpi tracking.
- Capability: SOC Metrics and KPI Tracking Agent - Collects and reports SOC performance metrics.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, metrics, tracking`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate to Splunk and return session key
2. Execute a Splunk search and return results
3. Collect Mean Time to Detect metrics from Splunk ES notable events
4. Collect Mean Time to Respond metrics
5. Collect alert disposition and quality metrics
6. Collect per-analyst productivity metrics
7. Generate formatted SOC performance report

Tools & resources

- Arguments
- Key Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `authenticate_splunk` — Authenticate to Splunk and return session key.
- `run_splunk_search` — Execute a Splunk search and return results.
- `collect_mttd_metrics` — Collect Mean Time to Detect metrics from Splunk ES notable events.
- `collect_mttr_metrics` — Collect Mean Time to Respond metrics.
- `collect_alert_quality` — Collect alert disposition and quality metrics.
- `collect_analyst_productivity` — Collect per-analyst productivity metrics.
- `generate_report` — Generate formatted SOC performance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-soc-metrics-and-kpi-tracking`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-soc-playbook-for-ransomware (Skill Module)

building-soc-playbook-for-ransomware\SKILL.md

name: building-soc-playbook-for-ransomware
description: Ransomware Playbook Automation Agent - Automates SOC ransomware response steps. Use when the user asks to build soc playbook for ransomware.
version: 1.0.0
tags:
source: building-soc-playbook-for-ransomware

Building SOC Playbook for Ransomware

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Arguments, Key Functions, External APIs Used

When to use

- The user wants to **build** soc playbook for ransomware.
- Capability: Ransomware Playbook Automation Agent - Automates SOC ransomware response steps.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, playbook, ransomware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Upload encrypted file sample to ID Ransomware for variant identification
2. Check No More Ransom Project for available decryptors
3. Query MalwareBazaar for IOC details by SHA-256 hash
4. Isolate a compromised host via CrowdStrike Falcon API
5. Search Splunk for IOC matches across the enterprise
6. Generate a structured ransomware incident response report
7. Extract IOCs from a ransomware sample file hash

Tools & resources

- Arguments
- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_id_ransomware` — Upload encrypted file sample to ID Ransomware for variant identification.
- `query_nomoreransom` — Check No More Ransom Project for available decryptors.
- `query_malwarebazaar_hash` — Query MalwareBazaar for IOC details by SHA-256 hash.
- `isolate_host_crowdstrike` — Isolate a compromised host via CrowdStrike Falcon API.
- `search_iocs_splunk` — Search Splunk for IOC matches across the enterprise.
- `generate_ir_report` — Generate a structured ransomware incident response report.
- `collect_iocs_from_sample` — Extract IOCs from a ransomware sample file hash.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-soc-playbook-for-ransomware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-threat-actor-profile-from-osint (Skill Module)

`building-threat-actor-profile-from-osint\SKILL.md`

name: building-threat-actor-profile-from-osint
description: Threat Actor Profiling from OSINT Agent - Builds threat actor profiles using open-source intelligence. Use when the user asks to build threat actor profile from osint.
version: 1.0.0
tags:
source: building-threat-actor-profile-from-osint

Building Threat Actor Profile from Osint

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: AlienVault OTX API, MALPEDIA API, Threat Actor Profiling Fields

When to use

- The user wants to **build** threat actor profile from osint.
- Capability: Threat Actor Profiling from OSINT Agent - Builds threat actor profiles using open-source intelligence.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, threat, actor, profile, osint`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Fetch MITRE ATT&CK enterprise data
2. Extract threat group information from ATT&CK STIX bundle
3. Extract techniques used by a threat group via relationships
4. Extract malware and tools associated with the group
5. Search AlienVault OTX for threat actor intelligence
6. Analyze tactic coverage across the kill chain
7. Generate threat actor profile report

Tools & resources

- AlienVault OTX API
- MALPEDIA API
- Threat Actor Profiling Fields

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `fetch_mitre_attack_data` — Fetch MITRE ATT&CK enterprise data.
- `extract_group_info` — Extract threat group information from ATT&CK STIX bundle.
- `extract_group_techniques` — Extract techniques used by a threat group via relationships.
- `extract_group_malware_tools` — Extract malware and tools associated with the group.
- `search_alienvault_otx` — Search AlienVault OTX for threat actor intelligence.
- `build_tactic_coverage` — Analyze tactic coverage across the kill chain.
- `generate_report` — Generate threat actor profile report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-threat-actor-profile-from-osint`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-threat-feed-aggregation-with-misp (Skill Module)

`building-threat-feed-aggregation-with-misp\SKILL.md`

name: building-threat-feed-aggregation-with-misp
description: Threat Feed Aggregation Agent - Aggregates and correlates threat intelligence feeds using MISP. Use when the user asks to build threat feed aggregation with misp.
version: 1.0.0
tags:
source: building-threat-feed-aggregation-with-misp

Building Threat Feed Aggregation with MISP

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: MISP, Common Feed Sources, Feed Configuration Fields

When to use

- The user wants to **build** threat feed aggregation with misp.
- Capability: Threat Feed Aggregation Agent - Aggregates and correlates threat intelligence feeds using MISP.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, threat, feed, aggregation, misp`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Make authenticated MISP API request
2. List configured MISP feeds
3. Fetch and cache data from a specific feed
4. Search MISP attributes across all events
5. Aggregate statistics across all feeds
6. Correlate an IOC across all feed events
7. Assess health and coverage of configured feeds
8. Generate threat feed aggregation report

Tools & resources

- MISP
- Common Feed Sources
- Feed Configuration Fields

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `misp_request` — Make authenticated MISP API request.
- `list_feeds` — List configured MISP feeds.
- `fetch_feed_data` — Fetch and cache data from a specific feed.
- `search_attributes` — Search MISP attributes across all events.
- `aggregate_feed_statistics` — Aggregate statistics across all feeds.
- `correlate_across_feeds` — Correlate an IOC across all feed events.
- `assess_feed_health` — Assess health and coverage of configured feeds.
- `generate_report` — Generate threat feed aggregation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-threat-feed-aggregation-with-misp`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-threat-hunt-hypothesis-framework (Skill Module)

`building-threat-hunt-hypothesis-framework\SKILL.md`

name: building-threat-hunt-hypothesis-framework
description: Threat hunt hypothesis framework builder. Generates structured threat hunting hypotheses from MITRE ATT&CK techniques, maps data sources, defines detection logic, and tracks hunt outcomes. Use when the user asks to build threat hunt hypothesis framework.
version: 1.0.0
tags:
source: building-threat-hunt-hypothesis-framework

Building Threat Hunt Hypothesis Framework

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Hypothesis Structure, Detection Data Sources

When to use

- The user wants to **build** threat hunt hypothesis framework.
- Capability: Threat hunt hypothesis framework builder. Generates structured threat hunting hypotheses from MITRE ATT&CK techniques, maps data sources, defines detection logic, and tracks hunt outcomes.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, threat, hunt, hypothesis, framework`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate a structured threat hunting hypothesis
2. Build a hunt plan from a list of hypotheses
3. Evaluate hunt execution results and update hypothesis
4. Fetch MITRE ATT&CK technique list

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Hypothesis Structure
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_hypothesis` — Generate a structured threat hunting hypothesis.
- `build_hunt_plan` — Build a hunt plan from a list of hypotheses.
- `evaluate_hunt_results` — Evaluate hunt execution results and update hypothesis.
- `fetch_attack_techniques` — Fetch MITRE ATT&CK technique list.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-threat-hunt-hypothesis-framework`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-threat-intelligence-enrichment-in-splunk (Skill Module)

`building-threat-intelligence-enrichment-in-splunk\SKILL.md`

name: building-threat-intelligence-enrichment-in-splunk
description: Threat intelligence enrichment pipeline for Splunk. Manages threat intel lookups, KV store collections, and modular inputs for enriching Splunk events with IOC context from MISP, OTX, and CSV feeds. Use when the user asks to build threat intelligence enrichment in splunk.
version: 1.0.0
tags:
source: building-threat-intelligence-enrichment-in-splunk

Building Threat Intelligence Enrichment in Splunk

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Splunk, SPL Threat Matching, AlienVault OTX API, Threat Intelligence Standards

When to use

- The user wants to **build** threat intelligence enrichment in splunk.
- Capability: Threat intelligence enrichment pipeline for Splunk. Manages threat intel lookups, KV store collections, and modular inputs for enriching Splunk events with IOC context from MISP, OTX, and CSV feeds.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, threat, intelligence, enrichment, splunk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Fetch IOCs from AlienVault OTX pulse
2. Convert IOC list to Splunk KV store format
3. Generate CSV file for Splunk lookup table
4. Build SPL query for threat intelligence correlation

Key concepts

- Version 2.1 is the current standard for representing threat intelligence
- Defines objects: Indicator, Malware, Attack Pattern, Threat Actor, Campaign
- Used as the interchange format between TI platforms and SIEMs
- Transport mechanism for STIX data
- TAXII 2.1 provides RESTful API for feed collection
- Supports Collection and Channel sharing models
- Mandiant's open framework for sharing IOCs
- XML-based format for indicator definitions

Tools & resources

- Splunk
- SPL Threat Matching
- AlienVault OTX API
- Threat Intelligence Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `fetch_otx_pulse_iocs` — Fetch IOCs from AlienVault OTX pulse.
- `convert_iocs_to_splunk_lookup` — Convert IOC list to Splunk KV store format.
- `generate_splunk_lookup_csv` — Generate CSV file for Splunk lookup table.
- `build_spl_correlation_search` — Build SPL query for threat intelligence correlation.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-threat-intelligence-enrichment-in-splunk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-threat-intelligence-feed-integration (Skill Module)

building-threat-intelligence-feed-integration\SKILL.md

name: building-threat-intelligence-feed-integration
description: Threat Intelligence Feed Integration Agent - Ingests STIX/TAXII and open-source TI feeds. Use when the user asks to build threat intelligence feed integration.
version: 1.0.0
tags:
source: building-threat-intelligence-feed-integration

Building Threat Intelligence Feed Integration

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Key Functions, External APIs Used

When to use

- The user wants to **build** threat intelligence feed integration.
- Capability: Threat Intelligence Feed Integration Agent - Ingests STIX/TAXII and open-source TI feeds.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, threat, intelligence, feed, integration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Ingest indicators from a TAXII 2.1 server collection
2. Ingest recent malicious URLs from Abuse.ch URLhaus
3. Ingest C2 IP blocklist from Abuse.ch Feodo Tracker
4. Convert raw indicators to STIX 2.1 Indicator objects
5. Deduplicate indicators across feeds by type:value hash
6. Export STIX 2.1 bundle to JSON file
7. Push indicators to Splunk ES threat intelligence framework

Tools & resources

- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `ingest_taxii_feed` — Ingest indicators from a TAXII 2.1 server collection.
- `ingest_urlhaus_feed` — Ingest recent malicious URLs from Abuse.ch URLhaus.
- `ingest_feodotracker` — Ingest C2 IP blocklist from Abuse.ch Feodo Tracker.
- `normalize_to_stix` — Convert raw indicators to STIX 2.1 Indicator objects.
- `deduplicate` — Deduplicate indicators across feeds by type:value hash.
- `export_stix_bundle` — Export STIX 2.1 bundle to JSON file.
- `push_to_splunk_ti` — Push indicators to Splunk ES threat intelligence framework.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-threat-intelligence-feed-integration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-threat-intelligence-platform (Skill Module)

`building-threat-intelligence-platform\SKILL.md`

name: building-threat-intelligence-platform
description: Threat intelligence platform builder. Core TIP components: STIX/TAXII ingestion, indicator lifecycle management, confidence scoring, sharing groups, and intelligence dissemination. Use when the user asks to build threat intelligence platform.
version: 1.0.0
tags:
source: building-threat-intelligence-platform

Building Threat Intelligence Platform

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: STIX 2.1 Indicator Object, OpenCTI GraphQL API, MISP REST API, TIP Architecture Standards, Platform Component Standards

When to use

- The user wants to **build** threat intelligence platform.
- Capability: Threat intelligence platform builder. Core TIP components: STIX/TAXII ingestion, indicator lifecycle management, confidence scoring, sharing groups, and intelligence dissemination.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, threat, intelligence, platform`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Classify indicator type from raw value
2. Create a STIX 2.1 Indicator object
3. Convert TLP label to STIX marking definition ID
4. Calculate indicator confidence score (0-100)
5. Build STIX 2.1 Bundle from list of indicators
6. Generate TIP status report

Key concepts

- STIX 2.1: Native data model for CTI representation
- TAXII 2.1: Transport protocol for CTI sharing
- MITRE ATT&CK: Technique taxonomy for TTP mapping
- Diamond Model: Intrusion analysis framework
- Kill Chain: Lockheed Martin Cyber Kill Chain for attack phase tracking
- MISP Sync Protocol: Push/Pull over HTTPS with API key auth
- OpenCTI Connectors: RabbitMQ-based message queue for async processing
- Cortex Analyzers: Docker-based analyzers with standardized I/O

Tools & resources

- STIX 2.1 Indicator Object
- OpenCTI GraphQL API
- MISP REST API
- TIP Architecture Standards
- Platform Component Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `classify_indicator` — Classify indicator type from raw value.
- `create_stix_indicator` — Create a STIX 2.1 Indicator object.
- `tlp_to_marking_ref` — Convert TLP label to STIX marking definition ID.
- `calculate_indicator_score` — Calculate indicator confidence score (0-100).
- `build_stix_bundle` — Build STIX 2.1 Bundle from list of indicators.
- `generate_tip_report` — Generate TIP status report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-threat-intelligence-platform`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-vulnerability-aging-and-sla-tracking (Skill Module)

building-vulnerability-aging-and-sla-tracking\SKILL.md

name: building-vulnerability-aging-and-sla-tracking
description: Vulnerability aging and SLA tracking agent. Tracks vulnerability remediation timelines, calculates SLA compliance, generates aging reports, and identifies overdue items by severity. Use when the user asks to build vulnerability aging and sla tracking.
version: 1.0.0
tags:
source: building-vulnerability-aging-and-sla-tracking

Building Vulnerability Aging and Sla Tracking

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: SLA Definitions, Aging Buckets, Qualys API, Key Metrics, Industry Standards, SLA Benchmark References

When to use

- The user wants to **build** vulnerability aging and sla tracking.
- Capability: Vulnerability aging and SLA tracking agent. Tracks vulnerability remediation timelines, calculates SLA compliance, generates aging reports, and identifies overdue items by severity.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, vulnerability, aging, tracking`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Calculate vulnerability age in days from discovery date
2. Check if a vulnerability is within SLA
3. Build vulnerability aging distribution report
4. Calculate Mean Time to Remediate by severity
5. Generate SLA compliance dashboard data

Key concepts

- NIST SP 800-40 Rev 4: Guide to Enterprise Patch Management Planning
- CIS Controls v8.1 Control 7: Continuous Vulnerability Management
- PCI DSS v4.0 Req 6.3.3: Security patches installed within one month
- BOD 22-01: CISA remediation timelines for KEV vulnerabilities
- ISO 27001: 2022 A.8.8**: Management of technical vulnerabilities
- Top-performing organizations MTTR: < 15 days for critical

Tools & resources

- SLA Definitions
- Aging Buckets
- Qualys API
- Key Metrics
- Industry Standards
- SLA Benchmark References
- Industry SLA Benchmarks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `calculate_age` — Calculate vulnerability age in days from discovery date.
- `check_sla_compliance` — Check if a vulnerability is within SLA.
- `build_aging_report` — Build vulnerability aging distribution report.
- `calculate_mttr` — Calculate Mean Time to Remediate by severity.
- `generate_sla_dashboard` — Generate SLA compliance dashboard data.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-vulnerability-aging-and-sla-tracking`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-vulnerability-dashboard-with-defectdojo (Skill Module)

`building-vulnerability-dashboard-with-defectdojo\SKILL.md`

name: building-vulnerability-dashboard-with-defectdojo
description: Vulnerability dashboard builder using DefectDojo API. Queries DefectDojo REST API v2 for findings, products, and engagements to build vulnerability management dashboards and metrics. Use when the user asks to build vulnerability dashboard with defectdojo.
version: 1.0.0
tags:
source: building-vulnerability-dashboard-with-defectdojo

Building Vulnerability Dashboard with Defectdojo

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Import Scan, Python Client, Primary References, Deployment Requirements

When to use

- The user wants to **build** vulnerability dashboard with defectdojo.
- Capability: Vulnerability dashboard builder using DefectDojo API. Queries DefectDojo REST API v2 for findings, products, and engagements to build vulnerability management dashboards and metrics.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, vulnerability, dashboard, defectdojo`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Clone DefectDojo repository and deploy with Docker Compose
2. Configure admin account and change default password
3. Create Product Types aligned with business units
4. Create Products for each application/service
5. Configure Jira integration for ticket management
6. Configure Slack/Teams webhook for notifications
7. Set up SLA policies for each severity level
8. Create API keys for scanner integration
9. Add scan step to CI/CD pipeline (GitHub Actions, GitLab CI, Jenkins)
10. Run security scanner (Semgrep, Trivy, ZAP, etc.)

Key concepts

- License: BSD-3-Clause
- Relevance: DefectDojo categorizes findings using OWASP taxonomy
- Title: Vulnerability Monitoring and Scanning
- Relevance: DefectDojo supports centralized vulnerability tracking as required by RA-5
- Relevance: DefectDojo tracks application security findings for PCI compliance

Tools & resources

- Import Scan
- Python Client
- Primary References
- Deployment Requirements

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `build_dashboard_data` — Build dashboard metrics from findings list.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-vulnerability-dashboard-with-defectdojo`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-vulnerability-exception-tracking-system (Skill Module)

building-vulnerability-exception-tracking-system\SKILL.md

name: building-vulnerability-exception-tracking-system
description: Vulnerability exception tracking system. Manages risk acceptance workflows for vulnerabilities that cannot be remediated within SLA, including approval chains, expiration tracking, and compensating control documentation. Use when the user asks to build vulnerability exception tracking system.
version: 1.0.0
tags:
source: building-vulnerability-exception-tracking-system

Building Vulnerability Exception Tracking System

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Exception States, Approval Chain by Severity, Maximum Exception Duration, ServiceNow GRC API, Archer GRC API, Compensating Control Categories

When to use

- The user wants to **build** vulnerability exception tracking system.
- Capability: Vulnerability exception tracking system. Manages risk acceptance workflows for vulnerabilities that cannot be remediated within SLA, including approval chains, expiration tracking, and compensating control documentation.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, vulnerability, exception, tracking, system`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create a new vulnerability exception request
2. Submit exception request for approval
3. Process an approval decision
4. Check all exceptions for expiration
5. Generate exception tracking report

Key concepts

- Title: Vulnerability Monitoring and Scanning - Privileged Access
- Relevance: Requires organizations to track and manage vulnerability exceptions with documented risk acceptance
- Relevance: Appendix B defines requirements for compensating controls when a PCI requirement cannot be met as stated
- Title: Information Security Risk Treatment
- Relevance: Risk acceptance must be formally documented with appropriate authority approval
- Title: Continuous Vulnerability Management
- Sub-control 7.7: Remediate detected vulnerabilities within prescribed timelines; document exceptions with compensating controls
- Title: Risk Assessment

Tools & resources

- Exception States
- Approval Chain by Severity
- Maximum Exception Duration
- ServiceNow GRC API
- Archer GRC API
- Compensating Control Categories
- Primary Standards

- Compliance Requirements for Exceptions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `create_exception_request` — Create a new vulnerability exception request.
- `submit_for_approval` — Submit exception request for approval.
- `process_approval` — Process an approval decision.
- `check_expirations` — Check all exceptions for expiration.
- `generate_exception_report` — Generate exception tracking report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-vulnerability-exception-tracking-system`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

building-vulnerability-scanning-workflow (Skill Module)

`building-vulnerability-scanning-workflow\SKILL.md`

name: building-vulnerability-scanning-workflow
description: Vulnerability Scanning Workflow Agent - Automates scan orchestration and prioritization. Use when the user asks to build vulnerability scanning workflow.
version: 1.0.0
tags:
source: building-vulnerability-scanning-workflow

Building Vulnerability Scanning Workflow

Role: Capability building
Audience: SOC leads, IR managers, and security engineers
Primary tools: Key Functions, External APIs Used

When to use

- The user wants to **build** vulnerability scanning workflow.
- Capability: Vulnerability Scanning Workflow Agent - Automates scan orchestration and prioritization.
- Goal: design and stand up an operational security capability or process.
- Trigger keywords: `building, vulnerability, scanning, workflow`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Nmap vulnerability scan against target hosts
2. Fetch the CISA Known Exploited Vulnerabilities catalog
3. Launch a vulnerability scan via Nessus REST API
4. Apply risk-based prioritization to vulnerability findings
5. Create a ServiceNow incident ticket for a high-priority vulnerability
6. Generate vulnerability scan summary report

Tools & resources

- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_nmap_vuln_scan` — Run Nmap vulnerability scan against target hosts.
- `fetch_cisa_kev` — Fetch the CISA Known Exploited Vulnerabilities catalog.
- `launch_nessus_scan` — Launch a vulnerability scan via Nessus REST API.
- `prioritize_vulnerabilities` — Apply risk-based prioritization to vulnerability findings.
- `create_servicenow_ticket` — Create a ServiceNow incident ticket for a high-priority vulnerability.
- `generate_report` — Generate vulnerability scan summary report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `building-vulnerability-scanning-workflow`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

bypassing-authentication-with-forced-browsing (Skill Module)

`bypassing-authentication-with-forced-browsing\SKILL.md`

name: bypassing-authentication-with-forced-browsing
description: Forced Browsing Authentication Bypass Agent - Tests for unprotected endpoints. Use when the user asks to bypassing authentication with forced browsing.
version: 1.0.0
tags:
source: bypassing-authentication-with-forced-browsing

Bypassing Authentication with Forced Browsing

Role: Security workflow
Audience: security practitioners
Primary tools: Arguments, Key Functions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **bypass** authentication with forced browsing.
- Capability: Forced Browsing Authentication Bypass Agent - Tests for unprotected endpoints.
- Goal: complete the task.
- Trigger keywords: `bypassing, authentication, forced, browsing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load directory/file wordlist from file
2. Test a single endpoint with and without authentication
3. Enumerate directories and test authentication enforcement
4. Test HTTP method-based authentication bypass
5. Test path normalization bypass techniques
6. Check for exposed backup and configuration files
7. Generate pentest finding report for forced browsing results

Tools & resources

- Arguments
- Key Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_wordlist` — Load directory/file wordlist from file.
- `test_endpoint` — Test a single endpoint with and without authentication.
- `enumerate_directories` — Enumerate directories and test authentication enforcement.
- `test_http_method_bypass` — Test HTTP method-based authentication bypass.
- `test_path_traversal_bypass` — Test path normalization bypass techniques.
- `check_sensitive_files` — Check for exposed backup and configuration files.
- `generate_report` — Generate pentest finding report for forced browsing results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `bypassing-authentication-with-forced-browsing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

collecting-indicators-of-compromise (Skill Module)

collecting-indicators-of-compromise\SKILL.md

name: collecting-indicators-of-compromise
description: IOC Collection Agent - Extracts, enriches, and exports indicators of compromise. Use when the user asks to collecting indicators of compromise.
version: 1.0.0
tags:
source: collecting-indicators-of-compromise

Collecting Indicators of Compromise

Role: Security workflow
Audience: security practitioners
Primary tools: Key Functions, External APIs Used, IOC Types Supported

When to use

- The user wants to **collect** indicators of compromise.
- Capability: IOC Collection Agent - Extracts, enriches, and exports indicators of compromise.
- Goal: complete the task.
- Trigger keywords: `collecting, indicators, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract IOCs (IPs, domains, hashes, URLs) from unstructured text
2. Extract IOCs from a file (text report, log file, etc.)
3. Enrich an IP address using VirusTotal API
4. Enrich a file hash using MalwareBazaar API
5. Check domain reputation via AbuseIPDB
6. Assign confidence score (0-100) based on enrichment results
7. Export IOCs as a STIX 2.1 bundle
8. Print a formatted IOC report

Tools & resources

- Key Functions
- External APIs Used
- IOC Types Supported

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_iocs_from_text` — Extract IOCs (IPs, domains, hashes, URLs) from unstructured text.
- `extract_iocs_from_file` — Extract IOCs from a file (text report, log file, etc.).
- `enrich_ip_virustotal` — Enrich an IP address using VirusTotal API.
- `enrich_hash_malwarebazaar` — Enrich a file hash using MalwareBazaar API.
- `enrich_domain_abuseipdb` — Check domain reputation via AbuseIPDB.
- `score_ioc` — Assign confidence score (0-100) based on enrichment results.
- `export_stix_bundle` — Export IOCs as a STIX 2.1 bundle.
- `generate_ioc_report` — Print a formatted IOC report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `collecting-indicators-of-compromise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

collecting-open-source-intelligence (Skill Module)

collecting-open-source-intelligence\SKILL.md

name: collecting-open-source-intelligence
description: OSINT Collection Agent - Gathers open-source intelligence on targets using Shodan and crt.sh. Use when the user asks to collecting open source intelligence.
version: 1.0.0
tags:
source: collecting-open-source-intelligence

Collecting Open Source Intelligence

Role: Security workflow
Audience: security practitioners
Primary tools: Key Functions, External APIs Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **collect** open source intelligence.
- Capability: OSINT Collection Agent - Gathers open-source intelligence on targets using Shodan and crt.sh.
- Goal: complete the task.
- Trigger keywords: `collecting, open, source, intelligence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Search Shodan for hosts matching a query
2. Lookup a specific host on Shodan for detailed information
3. Query certificate transparency logs via crt.sh for subdomain discovery
4. Perform WHOIS lookup via RDAP (Registration Data Access Protocol)
5. Query SecurityTrails API for DNS history
6. Search GitHub for exposed credentials or sensitive data
7. Generate a consolidated OSINT report

Tools & resources

- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `search_shodan` — Search Shodan for hosts matching a query.
- `shodan_host_lookup` — Lookup a specific host on Shodan for detailed information.
- `query_crtsh` — Query certificate transparency logs via crt.sh for subdomain discovery.
- `whois_lookup` — Perform WHOIS lookup via RDAP (Registration Data Access Protocol).
- `query_securitytrails` — Query SecurityTrails API for DNS history.
- `search_github_exposure` — Search GitHub for exposed credentials or sensitive data.
- `generate_osint_report` — Generate a consolidated OSINT report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `collecting-open-source-intelligence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

collecting-threat-intelligence-with-misp (Skill Module)

collecting-threat-intelligence-with-misp\SKILL.md

name: collecting-threat-intelligence-with-misp
description: Threat intelligence collection agent using MISP/PyMISP. Connects to MISP instances to collect, filter, and export threat intelligence including events, attributes, and feeds via the PyMISP REST API client. Use when the user asks to collecting threat intelligence with misp.
version: 1.0.0
tags:
source: collecting-threat-intelligence-with-misp

Collecting Threat Intelligence with MISP

Role: Security workflow
Audience: security practitioners
Primary tools: MISP, Client Initialization, Attribute Search, Common Attribute Types, Feed Management

When to use

- The user wants to **collect** threat intelligence with misp.
- Capability: Threat intelligence collection agent using MISP/PyMISP. Connects to MISP instances to collect, filter, and export threat intelligence including events, attributes, and feeds via the PyMISP REST API client.
- Goal: complete the task.
- Trigger keywords: `collecting, threat, intelligence, misp`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Initialize PyMISP client
2. Search MISP events by tags and date
3. Extract attributes from a MISP event
4. Collect IOCs filtered by attribute type
5. List configured MISP feeds
6. Export MISP event as STIX 2.1 bundle

Key concepts

- MISP JSON Format: Native event format used for synchronization between instances
- MISP Galaxy: Cluster-based knowledge base linked to MITRE ATT&CK, threat actors, tools
- MISP Taxonomies: Machine-readable tagging schemes (TLP, PAP, admiralty-scale, OSINT)
- MISP Warninglists: Lists of well-known indicators to reduce false positives (Alexa Top 1M, Office 365 IPs)
- Standard language for representing cyber threat intelligence
- MISP supports import/export of STIX 2.1 bundles
- Object types: Indicator, Malware, Threat Actor, Attack Pattern, Campaign, Observed Data
- Relationship types: uses, targets, attributed-to, indicates, mitigates

Tools & resources

- MISP
- Client Initialization
- Attribute Search
- Common Attribute Types
- Feed Management

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `init_misp` — Initialize PyMISP client.
- `search_events` — Search MISP events by tags and date.
- `extract_attributes` — Extract attributes from a MISP event.
- `collect_iocs_by_type` — Collect IOCs filtered by attribute type.
- `list_feeds` — List configured MISP feeds.
- `export_stix2` — Export MISP event as STIX 2.1 bundle.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `collecting-threat-intelligence-with-misp`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

collecting-volatile-evidence-from-compromised-host (Skill Module)

collecting-volatile-evidence-from-compromised-host\SKILL.md

name: collecting-volatile-evidence-from-compromised-host
description: Volatile evidence collection agent for compromised hosts. Collects volatile forensic artifacts following RFC 3227 order of volatility: registers, memory, network state, running processes, disk, and logs. Uses.... Use when the user asks to collecting volatile evidence from compromised host.
version: 1.0.0
tags:
source: collecting-volatile-evidence-from-compromised-host

Collecting Volatile Evidence from Compromised Host

Role: Security workflow
Audience: security practitioners
Primary tools: Memory Acquisition Tools, Evidence Integrity, RFC 3227, NIST SP 800-86, SANS DFIR, ISO/IEC 27037

When to use

- The user wants to **collect** volatile evidence from compromised host.
- Capability: Volatile evidence collection agent for compromised hosts. Collects volatile forensic artifacts following RFC 3227 order of volatility: registers, memory, network state, running processes, disk, and logs. Uses platform-native tools for live response evidence gathering.
- Goal: complete the task.
- Trigger keywords: `collecting, volatile, evidence, compromised, host`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Connect forensic USB with memory acquisition tool
2. Run memory dump tool from USB (NOT from compromised disk)
3. Save memory image to external storage
4. Record start time, end time, and memory size
5. Generate SHA256 hash of memory image immediately
6. Document any errors during acquisition
7. Capture all active TCP/UDP connections with process IDs
8. Capture ARP cache (maps IP to MAC addresses)
9. Capture DNS resolver cache
10. Capture routing table

Key concepts

- Defines the order of volatility for digital evidence:
- Key principles: minimize data alteration, document actions, use trusted tools
- Section 4: Using Data from Data Sources
- Forensic process: Collection, Examination, Analysis, Reporting
- Emphasis on preserving data integrity through proper acquisition
- RS.AN-03: Analysis to establish incident scope
- Evidence must be collected in a forensically sound manner
- Document all collection activities and maintain chain of custody

Tools & resources

- Memory Acquisition Tools
- Evidence Integrity

- RFC 3227
- NIST SP 800-86
- SANS DFIR
- ISO/IEC 27037

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `collect_artifact` — Collect a single volatile artifact.
- `run_collection` — Run full volatile evidence collection.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `collecting-volatile-evidence-from-compromised-host`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-api-security-testing (Skill Module)

conducting-api-security-testing\SKILL.md

name: conducting-api-security-testing
description: API Security Testing Agent - Tests REST/GraphQL APIs for OWASP API Top 10 vulnerabilities. Use when the user asks to conduct api security testing.
version: 1.0.0
tags:
source: conducting-api-security-testing

Conducting API Security Testing

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Arguments, Key Functions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** api security testing.
- Capability: API Security Testing Agent - Tests REST/GraphQL APIs for OWASP API Top 10 vulnerabilities.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, security, testing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test for Broken Object Level Authorization (BOLA/IDOR)
2. Test for Broken Function Level Authorization (BFLA)
3. Test for mass assignment vulnerability
4. Test rate limiting on sensitive endpoints
5. Test for JWT 'none' algorithm bypass
6. Test if GraphQL introspection is enabled
7. Test for excessive data exposure in API responses
8. Generate API security testing report

Tools & resources

- Arguments
- Key Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_bola` — Test for Broken Object Level Authorization (BOLA/IDOR).
- `test_bfla` — Test for Broken Function Level Authorization (BFLA).
- `test_mass_assignment` — Test for mass assignment vulnerability.
- `test_rate_limiting` — Test rate limiting on sensitive endpoints.
- `test_jwt_none_algorithm` — Test for JWT 'none' algorithm bypass.
- `test_graphql_introspection` — Test if GraphQL introspection is enabled.
- `test_excessive_data_exposure` — Test for excessive data exposure in API responses.
- `generate_report` — Generate API security testing report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-api-security-testing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-cloud-incident-response (Skill Module)

conducting-cloud-incident-response\SKILL.md

name: conducting-cloud-incident-response
description: Cloud Incident Response Agent - Automates AWS/Azure cloud IR containment and evidence collection. Use when the user asks to conduct cloud incident response.
version: 1.0.0
tags:
source: conducting-cloud-incident-response

Conducting Cloud Incident Response

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Arguments, Key Functions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** cloud incident response.
- Capability: Cloud Incident Response Agent - Automates AWS/Azure cloud IR containment and evidence collection.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, cloud, incident, response`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Disable a compromised IAM access key via AWS CLI
2. Attach AWSDenyAll policy to a compromised IAM user
3. Isolate an EC2 instance by changing its security group to forensic isolation SG
4. Create EBS snapshots of all volumes attached to an EC2 instance
5. Query CloudTrail for API calls made by a specific IAM user
6. Identify resources created by the attacker from CloudTrail events
7. Check all AWS regions for unauthorized EC2 instances
8. Generate a cloud incident response report

Tools & resources

- Arguments
- Key Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `aws_disable_access_key` — Disable a compromised IAM access key via AWS CLI.
- `aws_attach_deny_all` — Attach AWSDenyAll policy to a compromised IAM user.
- `aws_isolate_ec2` — Isolate an EC2 instance by changing its security group to forensic isolation SG.
- `aws_snapshot_ebs` — Create EBS snapshots of all volumes attached to an EC2 instance.
- `aws_query_cloudtrail` — Query CloudTrail for API calls made by a specific IAM user.
- `aws_list_attacker_resources` — Identify resources created by the attacker from CloudTrail events.
- `aws_check_all_regions_instances` — Check all AWS regions for unauthorized EC2 instances.
- `generate_ir_report` — Generate a cloud incident response report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-cloud-incident-response`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-cloud-penetration-testing (Skill Module)

conducting-cloud-penetration-testing\SKILL.md

name: conducting-cloud-penetration-testing
description: Cloud Penetration Testing Agent - Enumerates and tests AWS IAM misconfigurations. Use when the user asks to conduct cloud penetration testing.
version: 1.0.0
tags:
source: conducting-cloud-penetration-testing

Conducting Cloud Penetration Testing

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** cloud penetration testing.
- Capability: Cloud Penetration Testing Agent - Enumerates and tests AWS IAM misconfigurations.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, cloud, penetration, testing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Enumerate all IAM users in the AWS account
2. Enumerate IAM roles and identify cross-account trust relationships
3. Check for EC2 instances running with IMDSv1 (vulnerable to SSRF)
4. Enumerate S3 buckets and check for public access
5. Check Lambda functions for secrets in environment variables
6. Test if iam:CreatePolicyVersion can be used for privilege escalation
7. Generate cloud penetration test findings report

Tools & resources

- Key Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `enumerate_iam_users` — Enumerate all IAM users in the AWS account.
- `enumerate_iam_roles` — Enumerate IAM roles and identify cross-account trust relationships.
- `check_imds_v1_instances` — Check for EC2 instances running with IMDSv1 (vulnerable to SSRF).
- `check_public_s3_buckets` — Enumerate S3 buckets and check for public access.
- `check_lambda_env_secrets` — Check Lambda functions for secrets in environment variables.
- `test_privesc_create_policy_version` — Test if iam:CreatePolicyVersion can be used for privilege escalation.
- `generate_report` — Generate cloud penetration test findings report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-cloud-penetration-testing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-domain-persistence-with-dcsync (Skill Module)

conducting-domain-persistence-with-dcsync\SKILL.md

name: conducting-domain-persistence-with-dcsync
description: DCSync attack detection and analysis agent using impacket and ldap3. Use when the user asks to conduct domain persistence with dcsync.
version: 1.0.0
tags:
source: conducting-domain-persistence-with-dcsync

Conducting Domain Persistence with Dcsync

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key ldap3 Methods, External References, Key Research, Threat Actor Usage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** domain persistence with dcsync.
- Capability: DCSync attack detection and analysis agent using impacket and ldap3.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, domain, persistence, dcsync`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check which accounts have DCSync-capable permissions (Replicating Directory Changes)
2. Parse Windows Security event logs for DCSync indicators (Event IDs 4662, 4624)
3. Generate Sigma detection rule for DCSync activity
4. Run DCSync persistence audit

Key concepts

- Netwrix: DCSync Attack Using Mimikatz Detection
- JumpCloud: What Is DCSync? Critical AD Attack Explained
- The Hacker Recipes: DCSync technique documentation
- Atomic Red Team T1003.006 test procedures
- APT28 (Fancy Bear) - DCSync for credential harvesting
- APT29 (Cozy Bear) - SolarWinds campaign used DCSync
- FIN6 - Financial cybercrime group
- Wizard Spider - Ryuk ransomware campaigns

Tools & resources

- Key ldap3 Methods
- External References
- Key Research
- Threat Actor Usage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_dcsync_permissions` — Check which accounts have DCSync-capable permissions (Replicating Directory Changes).
- `detect_dcsync_events` — Parse Windows Security event logs for DCSync indicators (Event IDs 4662, 4624).
- `generate_sigma_rule` — Generate Sigma detection rule for DCSync activity.
- `run_audit` — Run DCSync persistence audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-domain-persistence-with-dcsync`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-external-reconnaissance-with-osint (Skill Module)

conducting-external-reconnaissance-with-osint\SKILL.md

name: conducting-external-reconnaissance-with-osint
description: External Reconnaissance Agent - Maps organization attack surface using passive OSINT. Use when the user asks to conduct external reconnaissance with osint.
version: 1.0.0
tags:
source: conducting-external-reconnaissance-with-osint

Conducting External Reconnaissance with Osint

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, External APIs Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** external reconnaissance with osint.
- Capability: External Reconnaissance Agent - Maps organization attack surface using passive OSINT.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, external, reconnaissance, osint`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover subdomains via certificate transparency logs
2. Query public DNS records for a domain using DNS-over-HTTPS
3. Search Shodan for hosts belonging to an organization
4. Check SPF, DKIM, and DMARC records for email security posture
5. Search Have I Been Pwned for breached accounts (requires API key)
6. Identify web technologies via HTTP response headers
7. Search GitHub for leaked credentials related to the target domain
8. Generate external reconnaissance report

Tools & resources

- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `enumerate_subdomains_crtsh` — Discover subdomains via certificate transparency logs.
- `enumerate_dns_records` — Query public DNS records for a domain using DNS-over-HTTPS.
- `shodan_org_search` — Search Shodan for hosts belonging to an organization.
- `check_email_security` — Check SPF, DKIM, and DMARC records for email security posture.
- `search_breach_data` — Search Have I Been Pwned for breached accounts (requires API key).
- `check_web_technologies` — Identify web technologies via HTTP response headers.
- `search_github_leaks` — Search GitHub for leaked credentials related to the target domain.
- `generate_recon_report` — Generate external reconnaissance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-external-reconnaissance-with-osint`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-full-scope-red-team-engagement (Skill Module)

conducting-full-scope-red-team-engagement\SKILL.md

name: conducting-full-scope-red-team-engagement
description: Full-scope red team engagement management. Use when the user asks to conduct full scope red team engagement.
version: 1.0.0
tags:
source: conducting-full-scope-red-team-engagement

Conducting Full Scope Red Team Engagement

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key attackcti Methods, C2 Frameworks, External References, NIST References, Industry Frameworks, Compliance Alignments

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** full scope red team engagement.
- Capability: Full-scope red team engagement management agent.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, full, scope, team, engagement`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Retrieve MITRE ATT&CK techniques for engagement planning
2. Generate red team engagement plan with phases
3. Generate C2 infrastructure preparation checklist
4. Execute red team engagement planning

Key concepts

- T1583.003** - Acquire Infrastructure: Virtual Private Server
- T1587.003** - Develop Capabilities: Digital Certificates
- T1608.001** - Stage Capabilities: Upload Malware
- T1566.001** - Phishing: Spearphishing Attachment
- T1566.002** - Phishing: Spearphishing Link
- T1195.002** - Supply Chain Compromise: Compromise Software Supply Chain
- T1059.003** - Command and Scripting Interpreter: Windows Command Shell
- T1204.001** - User Execution: Malicious Link

Tools & resources

- Key attackcti Methods
- C2 Frameworks
- External References
- NIST References
- Industry Frameworks
- Compliance Alignments

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_attack_techniques` — Retrieve MITRE ATT&CK techniques for engagement planning.
- `generate_engagement_plan` — Generate red team engagement plan with phases.
- `generate_c2_checklist` — Generate C2 infrastructure preparation checklist.
- `run_planning` — Execute red team engagement planning.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-full-scope-red-team-engagement`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-internal-network-penetration-test (Skill Module)

conducting-internal-network-penetration-test\SKILL.md

name: conducting-internal-network-penetration-test
description: Internal network penetration testing agent using nmap and impacket. Use when the user asks to conduct internal network penetration test.
version: 1.0.0
tags:
source: conducting-internal-network-penetration-test

Conducting Internal Network Penetration Test

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Common Internal Vulnerabilities, External References, Frameworks, Compliance Requirements

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** internal network penetration test.
- Capability: Internal network penetration testing agent using nmap and impacket.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, internal, network, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run nmap network discovery and port scanning
2. Check if SMB signing is required on target hosts
3. Check for LLMNR/NBT-NS poisoning opportunities
4. Enumerate Active Directory information via LDAP
5. Check for common internal network vulnerabilities
6. Execute internal network penetration test

Tools & resources

- Common Internal Vulnerabilities
- External References
- Frameworks
- Compliance Requirements

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_nmap_scan` — Run nmap network discovery and port scanning.
- `check_smb_signing` — Check if SMB signing is required on target hosts.
- `check_llmnr_nbns` — Check for LLMNR/NBT-NS poisoning opportunities.
- `enumerate_ad_info` — Enumerate Active Directory information via LDAP.
- `check_common_vulns` — Check for common internal network vulnerabilities.
- `run_pentest` — Execute internal network penetration test.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-internal-network-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-internal-reconnaissance-with-bloodhound-ce (Skill Module)

conducting-internal-reconnaissance-with-bloodhound-ce\SKILL.md

name: conducting-internal-reconnaissance-with-bloodhound-ce
description: BloodHound CE reconnaissance agent using bloodhound Python ingestor and Neo4j. Use when the user asks to conduct internal reconnaissance with bloodhound ce.
version: 1.0.0
tags:
source: conducting-internal-reconnaissance-with-bloodhound-ce

Conducting Internal Reconnaissance with BloodHound Ce

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: BloodHound, Key neo4j Driver Methods, External References, Official Resources, Key Research

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** internal reconnaissance with bloodhound ce.
- Capability: BloodHound CE reconnaissance agent using bloodhound Python ingestor and Neo4j.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, internal, reconnaissance, bloodhound`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run BloodHound Python ingestor to collect AD data
2. Find shortest path to Domain Admins
3. Find kerberoastable user accounts
4. Find computers with unconstrained delegation
5. Find AS-REP roastable accounts (no pre-auth required)
6. Run BloodHound reconnaissance queries

Key concepts

- SpecterOps: An Ace in the Hole - Stealthy Data Collection with BloodHound
- Compass Security: BloodHound Community Edition Custom Queries (2025)
- SpecterOps: Introducing the BloodHound Query Library (2025)

Tools & resources

- BloodHound
- Key neo4j Driver Methods
- External References
- Official Resources
- Key Research

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `collect_bloodhound_data` — Run BloodHound Python ingestor to collect AD data.
- `query_shortest_path_to_da` — Find shortest path to Domain Admins.
- `query_kerberoastable_users` — Find kerberoastable user accounts.
- `query_unconstrained_delegation` — Find computers with unconstrained delegation.
- `query_as_rep_roastable` — Find AS-REP roastable accounts (no pre-auth required).
- `run_recon` — Run BloodHound reconnaissance queries.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-internal-reconnaissance-with-bloodhound-ce`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-malware-incident-response (Skill Module)

conducting-malware-incident-response\SKILL.md

name: conducting-malware-incident-response
description: Malware Incident Response Agent - Triages malware infections and automates containment. Use when the user asks to conduct malware incident response.
version: 1.0.0
tags:
source: conducting-malware-incident-response

Conducting Malware Incident Response

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, External APIs Used

When to use

- The user wants to **conduct** malware incident response.
- Capability: Malware Incident Response Agent - Triages malware infections and automates containment.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, malware, incident, response`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 and MD5 hashes of a file
2. Query VirusTotal for file hash reputation
3. Query MalwareBazaar for sample metadata
4. Search ThreatFox for IOCs associated with a malware family
5. Isolate an infected endpoint via CrowdStrike Falcon API
6. Search Splunk for IOC matches across the enterprise
7. Generate a malware incident response report

Tools & resources

- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `hash_file` — Compute SHA-256 and MD5 hashes of a file.
- `query_virustotal` — Query VirusTotal for file hash reputation.
- `query_malwarebazaar` — Query MalwareBazaar for sample metadata.
- `query_threatfox_iocs` — Search ThreatFox for IOCs associated with a malware family.
- `isolate_endpoint_crowdstrike` — Isolate an infected endpoint via CrowdStrike Falcon API.
- `search_enterprise_iocs` — Search Splunk for IOC matches across the enterprise.
- `generate_malware_ir_report` — Generate a malware incident response report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-malware-incident-response`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-man-in-the-middle-attack-simulation (Skill Module)

`conducting-man-in-the-middle-attack-simulation\SKILL.md`

name: conducting-man-in-the-middle-attack-simulation
description: MITM Attack Simulation Agent - Tests network defenses against ARP spoofing and traffic interception. Use when the user asks to conduct man in the middle attack simulation.
version: 1.0.0
tags:
source: conducting-man-in-the-middle-attack-simulation

Conducting Man in The Middle Attack Simulation

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, Scapy Functions Used, Cleartext Protocols Detected

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** man in the middle attack simulation.
- Capability: MITM Attack Simulation Agent - Tests network defenses against ARP spoofing and traffic interception.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, middle, attack, simulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Resolve MAC address for an IP using ARP request
2. Discover live hosts on the network via ARP scan
3. Send ARP spoofing packet to redirect traffic
4. Restore original ARP tables after test
5. Sniff traffic for cleartext protocol usage (HTTP, FTP, Telnet, SMTP)
6. Check if a target enforces HSTS headers
7. Evaluate if SSL stripping is feasible by checking HSTS preload status
8. Run a controlled MITM simulation with ARP spoofing and cleartext detection

Tools & resources

- Key Functions
- Scapy Functions Used
- Cleartext Protocols Detected

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_mac` — Resolve MAC address for an IP using ARP request.
- `discover_hosts` — Discover live hosts on the network via ARP scan.
- `arp_spoof` — Send ARP spoofing packet to redirect traffic.
- `arp_restore` — Restore original ARP tables after test.
- `detect_cleartext_protocols` — Sniff traffic for cleartext protocol usage (HTTP, FTP, Telnet, SMTP).
- `check_hsts_enforcement` — Check if a target enforces HSTS headers.
- `test_ssl_stripping_potential` — Evaluate if SSL stripping is feasible by checking HSTS preload status.
- `run_mitm_simulation` — Run a controlled MITM simulation with ARP spoofing and cleartext detection.
- `generate_report` — Generate MITM assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-man-in-the-middle-attack-simulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-memory-forensics-with-volatility (Skill Module)

`conducting-memory-forensics-with-volatility\SKILL.md`

name: conducting-memory-forensics-with-volatility
description: Memory Forensics Agent - Automates Volatility 3 analysis of memory dumps for incident response. Use when the user asks to conduct memory forensics with volatility.
version: 1.0.0
tags:
source: conducting-memory-forensics-with-volatility

Conducting Memory Forensics with Volatility

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Volatility, Key Functions, Suspicious Process Indicators

When to use

- The user wants to **conduct** memory forensics with volatility.
- Capability: Memory Forensics Agent - Automates Volatility 3 analysis of memory dumps for incident response.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, memory, forensics, volatility`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute a Volatility 3 plugin and return parsed output
2. List running processes and identify suspicious ones
3. Extract network connections and identify C2 communication
4. Detect process injection using malfind plugin
5. List loaded DLLs for a process or all processes
6. Extract command line history from process memory
7. List registry hives in memory
8. List kernel modules and detect potential rootkits

Tools & resources

- Volatility
- Key Functions
- Suspicious Process Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_volatility` — Execute a Volatility 3 plugin and return parsed output.
- `analyze_processes` — List running processes and identify suspicious ones.
- `analyze_network_connections` — Extract network connections and identify C2 communication.
- `detect_process_injection` — Detect process injection using malfind plugin.
- `analyze_dlls` — List loaded DLLs for a process or all processes.
- `extract_command_history` — Extract command line history from process memory.
- `extract_registry_hives` — List registry hives in memory.
- `check_kernel_modules` — List kernel modules and detect potential rootkits.
- `generate_forensics_report` — Generate memory forensics analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-memory-forensics-with-volatility`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-mobile-app-penetration-test (Skill Module)

conducting-mobile-app-penetration-test\SKILL.md

name: conducting-mobile-app-penetration-test
description: Mobile App Penetration Testing Agent - Tests Android/iOS apps for OWASP MASTG vulnerabilities. Use when the user asks to conduct mobile app penetration test.
version: 1.0.0
tags:
source: conducting-mobile-app-penetration-test

Conducting Mobile App Penetration Test

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, OWASP MASTG Coverage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** mobile app penetration test.
- Capability: Mobile App Penetration Testing Agent - Tests Android/iOS apps for OWASP MASTG vulnerabilities.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, mobile, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Decompile Android APK using apktool for static analysis
2. Extract hardcoded strings from APK for sensitive data detection
3. Analyze AndroidManifest.xml for security misconfigurations
4. Test if the app enforces certificate pinning via mitmproxy check
5. Check for insecure data storage on connected Android device via adb
6. Test mobile app API endpoints for common vulnerabilities
7. Check if the app implements root/jailbreak detection
8. Generate mobile app penetration test report

Tools & resources

- Key Functions
- OWASP MASTG Coverage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `decompile_apk` — Decompile Android APK using apktool for static analysis.
- `extract_strings_from_apk` — Extract hardcoded strings from APK for sensitive data detection.
- `check_android_manifest` — Analyze AndroidManifest.xml for security misconfigurations.
- `test_certificate_pinning` — Test if the app enforces certificate pinning via mitmproxy check.
- `check_insecure_storage_adb` — Check for insecure data storage on connected Android device via adb.
- `test_api_endpoints` — Test mobile app API endpoints for common vulnerabilities.
- `check_root_detection` — Check if the app implements root/jailbreak detection.
- `generate_report` — Generate mobile app penetration test report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-mobile-app-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-network-penetration-test (Skill Module)

conducting-network-penetration-test\SKILL.md

name: conducting-network-penetration-test
description: Network Penetration Testing Agent - Automates host discovery, port scanning, and vuln assessment. Use when the user asks to conduct network penetration test.
version: 1.0.0
tags:
source: conducting-network-penetration-test

Conducting Network Penetration Test

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, Nmap Scan Types Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** network penetration test.
- Capability: Network Penetration Testing Agent - Automates host discovery, port scanning, and vuln assessment.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, network, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover live hosts on the network using ARP ping and ICMP
2. Perform TCP SYN scan with service version detection
3. Run Nmap vulnerability scripts against target
4. Enumerate SMB shares and users via Nmap scripts
5. Audit SSL/TLS configuration using Nmap ssl-enum-ciphers
6. Perform DNS enumeration via Nmap dns-brute
7. Classify and prioritize all findings by severity
8. Generate network penetration test report

Tools & resources

- Key Functions
- Nmap Scan Types Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `host_discovery` — Discover live hosts on the network using ARP ping and ICMP.
- `port_scan` — Perform TCP SYN scan with service version detection.
- `vulnerability_scan` — Run Nmap vulnerability scripts against target.
- `smb_enumeration` — Enumerate SMB shares and users via Nmap scripts.
- `ssl_audit` — Audit SSL/TLS configuration using Nmap ssl-enum-ciphers.
- `dns_enumeration` — Perform DNS enumeration via Nmap dns-brute.
- `classify_findings` — Classify and prioritize all findings by severity.
- `generate_report` — Generate network penetration test report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-network-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-pass-the-ticket-attack (Skill Module)

conducting-pass-the-ticket-attack\SKILL.md

name: conducting-pass-the-ticket-attack
description: Pass-the-Ticket attack detection agent using Windows event log analysis. Use when the user asks to conduct pass the ticket attack.
version: 1.0.0
tags:
source: conducting-pass-the-ticket-attack

Conducting Pass The Ticket Attack

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Encryption Type Constants, External References, Kerberos Ticket Types, Detection, NIST SP 800-171, CIS Controls

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** pass the ticket attack.
- Capability: Pass-the-Ticket attack detection agent using Windows event log analysis.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, pass, ticket, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze Windows Security logs for Pass-the-Ticket indicators
2. Generate Sigma detection rules for Pass-the-Ticket
3. Generate threat hunting queries for PTT detection
4. Execute Pass-the-Ticket detection analysis

Key concepts

- Control 6: Access Control Management
- Control 8: Audit Log Management - Monitor Kerberos authentication events

Tools & resources

- Encryption Type Constants
- External References
- Kerberos Ticket Types
- Detection
- NIST SP 800-171
- CIS Controls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `detect_ptt_events` — Analyze Windows Security logs for Pass-the-Ticket indicators.
- `generate_sigma_rules` — Generate Sigma detection rules for Pass-the-Ticket.
- `generate_hunt_queries` — Generate threat hunting queries for PTT detection.
- `run_detection` — Execute Pass-the-Ticket detection analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-pass-the-ticket-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-phishing-incident-response (Skill Module)

conducting-phishing-incident-response\SKILL.md

name: conducting-phishing-incident-response
description: Phishing Incident Response Agent - Analyzes phishing emails and automates response actions. Use when the user asks to conduct phishing incident response.
version: 1.0.0
tags:
source: conducting-phishing-incident-response

Conducting Phishing Incident Response

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, External APIs Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** phishing incident response.
- Capability: Phishing Incident Response Agent - Analyzes phishing emails and automates response actions.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, phishing, incident, response`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse an EML file and extract header and body information
2. Extract all URLs from email body
3. Extract and hash email attachments
4. Check URL reputation on VirusTotal
5. Submit URL to urlscan.io for analysis
6. Check file hash reputation on VirusTotal
7. Assess overall severity of the phishing email
8. Generate phishing incident response report

Tools & resources

- Key Functions
- External APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_email_file` — Parse an EML file and extract header and body information.
- `extract_urls` — Extract all URLs from email body.
- `extract_attachments` — Extract and hash email attachments.
- `check_url_virustotal` — Check URL reputation on VirusTotal.
- `check_url_urlscan` — Submit URL to urlscan.io for analysis.
- `check_hash_virustotal` — Check file hash reputation on VirusTotal.
- `assess_phishing_severity` — Assess overall severity of the phishing email.
- `generate_phishing_report` — Generate phishing incident response report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-phishing-incident-response`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-post-incident-lessons-learned (Skill Module)

`conducting-post-incident-lessons-learned\SKILL.md`

name: conducting-post-incident-lessons-learned
description: Post-incident lessons learned analysis. Use when the user asks to conduct post incident lessons learned.
version: 1.0.0
tags:
source: conducting-post-incident-lessons-learned

Conducting Post Incident Lessons Learned

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Metrics, NIST SP 800-61 Phases, External References, SANS PICERL, ISO 27001, Google SRE Post-Mortem Culture

When to use

- The user wants to **conduct** post incident lessons learned.
- Capability: Post-incident lessons learned analysis agent.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, post, incident, lessons, learned`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze incident timeline for response gaps
2. Calculate key incident response metrics
3. Generate prioritized action items from analysis
4. Generate lessons learned report template
5. Execute post-incident lessons learned analysis

Key concepts

- Recommends formal review within days of resolution
- Recover (RC) function: Learning from incidents
- RC.CO-03: Recovery activities and progress communicated
- Emphasis on continuous improvement of IR capabilities
- Phase 6: Final phase of incident handling
- Formal review with all stakeholders
- Document improvements and update procedures
- Map incident techniques to ATT&CK framework

Tools & resources

- Key Metrics
- NIST SP 800-61 Phases
- External References
- SANS PICERL
- ISO 27001
- Google SRE Post-Mortem Culture

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_incident_timeline` — Analyze incident timeline for response gaps.
- `calculate_metrics` — Calculate key incident response metrics.
- `generate_action_items` — Generate prioritized action items from analysis.
- `generate_report_template` — Generate lessons learned report template.
- `run_analysis` — Execute post-incident lessons learned analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-post-incident-lessons-learned`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-social-engineering-penetration-test (Skill Module)

conducting-social-engineering-penetration-test\SKILL.md

name: conducting-social-engineering-penetration-test
description: Social engineering penetration test management agent using GoPhish API. Use when the user asks to conduct social engineering penetration test.
version: 1.0.0
tags:
source: conducting-social-engineering-penetration-test

Conducting Social Engineering Penetration Test

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Campaign Metrics, Social Engineering Vectors, External References, Frameworks, Legal Considerations

When to use

- The user wants to **conduct** social engineering penetration test.
- Capability: Social engineering penetration test management agent using GoPhish API.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, social, engineering, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze phishing campaign results for metrics
2. Generate social engineering pretext scenarios for testing
3. Execute social engineering assessment analysis

Key concepts

- NIST SP 800-50: Building an Information Technology Security Awareness and Training Program
- SANS Security Awareness Maturity Model
- Must have explicit written authorization
- Cannot target personal email or devices without consent
- GDPR compliance for EU employees
- Cannot use real malware or destructive payloads
- Results must be anonymized in reporting to protect individuals

Tools & resources

- Campaign Metrics
- Social Engineering Vectors
- External References
- Frameworks
- Legal Considerations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_campaign_results` — Analyze phishing campaign results for metrics.
- `generate_pretext_scenarios` — Generate social engineering pretext scenarios for testing.
- `run_assessment` — Execute social engineering assessment analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-social-engineering-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-social-engineering-pretext-call (Skill Module)

conducting-social-engineering-pretext-call\SKILL.md

name: conducting-social-engineering-pretext-call
description: Social engineering pretext call planning and tracking. Use when the user asks to conduct social engineering pretext call.
version: 1.0.0
tags:
source: conducting-social-engineering-pretext-call

Conducting Social Engineering Pretext Call

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Pretext Call Phases, Success Criteria Categories, Key Metrics, Legal Considerations, External References, NIST References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** social engineering pretext call.
- Capability: Social engineering pretext call planning and tracking agent.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, social, engineering, pretext, call`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate pretext call templates for authorized engagements
2. Create tracking sheet for pretext calls
3. Analyze pretext call results for reporting
4. Execute pretext call planning and analysis

Key concepts

- T1566.004** - Phishing: Voice (Vishing)
- T1598.003** - Phishing for Information: Spearphishing Voice
- T1589.002** - Gather Victim Identity Information: Email Addresses
- T1591.004** - Gather Victim Org Information: Identify Roles
- NIST SP 800-53 Rev. 5** - AT-2: Literacy Training and Awareness
- NIST SP 800-53 Rev. 5** - AT-3: Role-Based Training
- NIST SP 800-115** - Section 3.3: Social Engineering

Tools & resources

- Pretext Call Phases
- Success Criteria Categories
- Key Metrics
- Legal Considerations
- External References
- NIST References
- Legal Frameworks

- Industry Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_pretext_templates` — Generate pretext call templates for authorized engagements.
- `create_call_tracking_sheet` — Create tracking sheet for pretext calls.
- `analyze_results` — Analyze pretext call results for reporting.
- `run_planning` — Execute pretext call planning and analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-social-engineering-pretext-call`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-spearphishing-simulation-campaign (Skill Module)

conducting-spearphishing-simulation-campaign\SKILL.md

name: conducting-spearphishing-simulation-campaign
description: Spearphishing simulation campaign agent using GoPhish API. Use when the user asks to conduct spearphishing simulation campaign.
version: 1.0.0
tags:
source: conducting-spearphishing-simulation-campaign

Conducting Spearphishing Simulation Campaign

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Campaign Metrics, External References, PTES, NIST SP 800-177, CIS Controls v8

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** spearphishing simulation campaign.
- Capability: Spearphishing simulation campaign agent using GoPhish API.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, spearphishing, simulation, campaign`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate targeted spearphishing email templates
2. Analyze campaign summary for executive reporting
3. Execute spearphishing simulation analysis

Key concepts

- Research-based pretext development
- Authority, urgency, and scarcity principles
- Robert Cialdini's principles of influence
- Corporate communication mimicry
- Email-based spearphishing
- Voice phishing (vishing) support calls
- SMS phishing (smishing)
- Social media-based pretexting

Tools & resources

- Campaign Metrics
- External References
- PTES
- NIST SP 800-177
- CIS Controls v8

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_spearphish_templates` — Generate targeted spearphishing email templates.
- `analyze_campaign_metrics` — Analyze campaign summary for executive reporting.
- `run_simulation` — Execute spearphishing simulation analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-spearphishing-simulation-campaign`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

conducting-wireless-network-penetration-test (Skill Module)

conducting-wireless-network-penetration-test\SKILL.md

name: conducting-wireless-network-penetration-test
description: Wireless Network Penetration Testing Agent - Tests WiFi security using Scapy and aircrack-ng. Use when the user asks to conduct wireless network penetration test.
version: 1.0.0
tags:
source: conducting-wireless-network-penetration-test

Conducting Wireless Network Penetration Test

Role: Structured engagement
Audience: security teams running formal engagements
Primary tools: Key Functions, Scapy 802.11 Layers Used, External Tools Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **conduct** wireless network penetration test.
- Capability: Wireless Network Penetration Testing Agent - Tests WiFi security using Scapy and aircrack-ng.
- Goal: run a complete assessment, exercise, or investigation end to end.
- Trigger keywords: `conducting, wireless, network, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan for WiFi access points by capturing beacon frames
2. Detect rogue access points by comparing against known infrastructure
3. Identify access points using weak or no encryption
4. Capture WPA2 4-way handshake using airodump-ng
5. Send deauthentication frames to force client reconnection for handshake capture
6. Attempt to crack WPA2 handshake using aircrack-ng
7. Capture probe requests to identify client-side vulnerabilities
8. Check if WPS is enabled on target AP using wash

Tools & resources

- Key Functions
- Scapy 802.11 Layers Used
- External Tools Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_access_points` — Scan for WiFi access points by capturing beacon frames.
- `detect_rogue_aps` — Detect rogue access points by comparing against known infrastructure.
- `detect_weak_encryption` — Identify access points using weak or no encryption.
- `capture_handshake` — Capture WPA2 4-way handshake using airodump-ng.
- `send_deauth` — Send deauthentication frames to force client reconnection for handshake capture.
- `crack_handshake` — Attempt to crack WPA2 handshake using aircrack-ng.
- `detect_client_probes` — Capture probe requests to identify client-side vulnerabilities.
- `check_wps_enabled` — Check if WPS is enabled on target AP using wash.
- `generate_report` — Generate wireless penetration test report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `conducting-wireless-network-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-active-directory-tiered-model (Skill Module)

configuring-active-directory-tiered-model\SKILL.md

name: configuring-active-directory-tiered-model
description: Active Directory tiered administration model audit agent using ldap3. Use when the user asks to configure active directory tiered model.
version: 1.0.0
tags:
source: configuring-active-directory-tiered-model

Configuring Active Directory Tiered Model

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key ldap3 Methods, UserAccountControl Flags, External References

When to use

- The user wants to **configure** active directory tiered model.
- Capability: Active Directory tiered administration model audit agent using ldap3.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, active, directory, tiered, model`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Audit Tier 0 privileged accounts
2. Check for cross-tier privilege violations
3. Check for Privileged Access Workstation (PAW) deployment indicators
4. Execute AD tiered model audit

Tools & resources

- Key ldap3 Methods
- UserAccountControl Flags
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `audit_tier0_accounts` — Audit Tier 0 privileged accounts.
- `check_tier_violations` — Check for cross-tier privilege violations.
- `check_paw_compliance` — Check for Privileged Access Workstation (PAW) deployment indicators.
- `run_audit` — Execute AD tiered model audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-active-directory-tiered-model`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-aws-verified-access-for-ztna (Skill Module)

`configuring-aws-verified-access-for-ztna\SKILL.md`

name: configuring-aws-verified-access-for-ztna
description: AWS Verified Access ZTNA configuration agent using boto3. Use when the user asks to configure aws verified access for ztna.
version: 1.0.0
tags:
source: configuring-aws-verified-access-for-ztna

Configuring AWS Verified Access for Ztna

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: AWS, Key boto3 EC2 Methods, Cedar Policy Language, Trust Provider Types, External References, Zero Trust Standards

When to use

- The user wants to **configure** aws verified access for ztna.
- Capability: AWS Verified Access ZTNA configuration agent using boto3.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, verified, access, ztna`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List all Verified Access instances
2. List Verified Access groups and their policies
3. List Verified Access endpoints
4. Audit Verified Access trust providers
5. Execute AWS Verified Access audit

Key concepts

- Developed by AWS for fine-grained access control
- Supports permit and forbid policies with conditions
- Evaluates context attributes (identity, device posture, request metadata)
- Open-source specification at cedarpolicy.com
- SEC03-BP01: Define access requirements
- SEC03-BP02: Grant least privilege access
- SEC03-BP05: Define permission guardrails for organization
- SEC05-BP03: Automate network protection

Tools & resources

- AWS
- Key boto3 EC2 Methods
- Cedar Policy Language
- Trust Provider Types
- External References
- Zero Trust Standards
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `list_verified_access_instances` — List all Verified Access instances.
- `list_verified_access_groups` — List Verified Access groups and their policies.
- `list_verified_access_endpoints` — List Verified Access endpoints.
- `audit_trust_providers` — Audit Verified Access trust providers.
- `run_audit` — Execute AWS Verified Access audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-aws-verified-access-for-ztna`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-certificate-authority-with-openssl (Skill Module)

configuring-certificate-authority-with-openssl\SKILL.md

name: configuring-certificate-authority-with-openssl
description: Certificate Authority management agent using OpenSSL and cryptography library. Use when the user asks to configure certificate authority with openssl.
version: 1.0.0
tags:
source: configuring-certificate-authority-with-openssl

Configuring Certificate Authority with Openssl

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key cryptography Methods, Certificate Best Practices, External References, Primary Standards, Tools

When to use

- The user wants to **configure** certificate authority with openssl.
- Capability: Certificate Authority management agent using OpenSSL and cryptography library.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, certificate, authority, openssl`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate a self-signed root CA certificate
2. Generate a server certificate signed by the CA
3. Audit an existing certificate for security issues
4. Execute CA operations

Key concepts

- Description: Core X.509v3 certificate and CRL format specification
- Description: Online Certificate Status Protocol
- Description: Framework for CP and CPS documents
- Description: Key management recommendations
- Description: Requirements for publicly trusted CAs
- Description: X.509 certificate generation and parsing

Tools & resources

- Key cryptography Methods
- Certificate Best Practices
- External References
- Primary Standards
- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_ca_certificate` — Generate a self-signed root CA certificate.
- `generate_server_cert` — Generate a server certificate signed by the CA.
- `audit_certificate` — Audit an existing certificate for security issues.
- `run_audit` — Execute CA operations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-certificate-authority-with-openssl`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-host-based-intrusion-detection (Skill Module)

configuring-host-based-intrusion-detection\SKILL.md

name: configuring-host-based-intrusion-detection
description: Host-based intrusion detection agent using OSSEC/Wazuh API and osquery. Use when the user asks to configure host based intrusion detection.
version: 1.0.0
tags:
source: configuring-host-based-intrusion-detection

Configuring Host Based Intrusion Detection

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: OSSEC Rule IDs, External References, Primary Standards, Compliance Mappings, Supporting References

When to use

- The user wants to **configure** host based intrusion detection.
- Capability: Host-based intrusion detection agent using OSSEC/Wazuh API and osquery.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, host, intrusion, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute osquery for host inspection
2. Check file integrity for key system files
3. Execute HIDS audit

Key concepts

- Scope: Architecture, deployment, and management of IDPS including host-based systems
- Requirement: Deploy FIM to alert on unauthorized modification of critical files
- Scope: System files, configuration files, content files on in-scope systems
- Relevance: Sub-control 3.14 requires monitoring for unauthorized changes to sensitive data

Tools & resources

- OSSEC Rule IDs
- External References
- Primary Standards
- Compliance Mappings
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_osquery_check` — Execute osquery for host inspection.
- `check_file_integrity` — Check file integrity for key system files.
- `run_audit` — Execute HIDS audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-host-based-intrusion-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-hsm-for-key-storage (Skill Module)

`configuring-hsm-for-key-storage\SKILL.md`

name: configuring-hsm-for-key-storage
description: HSM key storage management agent using PKCS#11 and AWS CloudHSM. Use when the user asks to configure hsm for key storage.
version: 1.0.0
tags:
source: configuring-hsm-for-key-storage

Configuring Hsm for Key Storage

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key boto3 CloudHSMv2 Methods, HSM Types, External References, Primary Standards, HSM Products

When to use

- The user wants to **configure** hsm for key storage.
- Capability: HSM key storage management agent using PKCS#11 and AWS CloudHSM.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, storage`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List AWS CloudHSM clusters
2. List HSM instances in a cluster
3. Audit KMS keys backed by CloudHSM custom key store
4. Check CloudHSM backup status
5. Execute HSM key storage audit

Key concepts

- Description: Standard API for cryptographic token interface
- Description: Security requirements for cryptographic modules
- Description: Key management recommendations (HSM storage for high-value keys)
- Description: Software-only PKCS#11 implementation for testing
- FIPS: 140-2 Level 3
- FIPS: 140-2 Level 3 (Thales Luna)

Tools & resources

- Key boto3 CloudHSMv2 Methods
- HSM Types
- External References
- Primary Standards
- HSM Products

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `list_cloudhsm_clusters` — List AWS CloudHSM clusters.
- `list_hsm_instances` — List HSM instances in a cluster.
- `audit_kms_keys` — Audit KMS keys backed by CloudHSM custom key store.
- `check_cloudhsm_backup` — Check CloudHSM backup status.
- `run_audit` — Execute HSM key storage audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-hsm-for-key-storage`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-identity-aware-proxy-with-google-iap (Skill Module)

configuring-identity-aware-proxy-with-google-iap\SKILL.md

name: configuring-identity-aware-proxy-with-google-iap
description: Google Identity-Aware Proxy (IAP) configuration agent using google-cloud-iap. Use when the user asks to configure identity aware proxy with google iap.
version: 1.0.0
tags:
source: configuring-identity-aware-proxy-with-google-iap

Configuring Identity Aware Proxy with Google Iap

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key IAP Client Methods, IAP IAM Roles, External References, NIST SP 800-207, Google Cloud Documentation, Google BeyondCorp Papers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **configure** identity aware proxy with google iap.
- Capability: Google Identity-Aware Proxy (IAP) configuration agent using google-cloud-iap.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, identity, aware, proxy, google`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List IAP TCP forwarding tunnels
2. Get IAP settings for web resources
3. Audit IAM bindings for IAP-secured resources
4. Verify OAuth consent screen configuration
5. Execute IAP configuration audit

Key concepts

- Section 3.1: Policy Engine - IAP evaluates identity and context per request
- Section 3.2: Trust Algorithm - Access Levels compute trust score
- Section 4.2: SDP Gateway Model - IAP acts as application-layer gateway
- Identity Pillar: Per-request identity verification via IAP
- Application Pillar: Application-level access controls
- BeyondCorp: A New Approach to Enterprise Security** (2014): https://research.google/pubs/pub43231/
- BeyondCorp: The Access Proxy** (2017): https://research.google/pubs/pub45728/
- Google Cloud IAP operates within FedRAMP High boundary

Tools & resources

- Key IAP Client Methods
- IAP IAM Roles
- External References
- NIST SP 800-207
- Google Cloud Documentation

- Google BeyondCorp Papers
- FedRAMP

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `list_iap_tunnels` — List IAP TCP forwarding tunnels.
- `get_iap_settings` — Get IAP settings for web resources.
- `audit_iap_iam_policy` — Audit IAM bindings for IAP-secured resources.
- `check_oauth_consent` — Verify OAuth consent screen configuration.
- `run_audit` — Execute IAP configuration audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-identity-aware-proxy-with-google-iap`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-ldap-security-hardening (Skill Module)

configuring-ldap-security-hardening\SKILL.md

name: configuring-ldap-security-hardening
description: LDAP security hardening audit agent using ldap3. Use when the user asks to configure ldap security hardening.
version: 1.0.0
tags:
source: configuring-ldap-security-hardening

Configuring LDAP Security Hardening

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key ldap3 Methods, Security Checks, External References

When to use

- The user wants to **configure** ldap security hardening.
- Capability: LDAP security hardening audit agent using ldap3.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, ldap, security, hardening`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check if LDAP signing is enforced
2. Check LDAPS (LDAP over TLS) availability
3. Check LDAP channel binding enforcement
4. Check for anonymous LDAP access and enumeration
5. Execute LDAP security hardening audit

Tools & resources

- Key ldap3 Methods
- Security Checks
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_ldap_signing` — Check if LDAP signing is enforced.
- `check_ldaps` — Check LDAPS (LDAP over TLS) availability.
- `check_channel_binding` — Check LDAP channel binding enforcement.
- `audit_anonymous_access` — Check for anonymous LDAP access and enumeration.
- `run_audit` — Execute LDAP security hardening audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-ldap-security-hardening`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-microsegmentation-for-zero-trust (Skill Module)

`configuring-microsegmentation-for-zero-trust\SKILL.md`

name: configuring-microsegmentation-for-zero-trust
description: Microsegmentation audit agent for zero trust network enforcement. Use when the user asks to configure microsegmentation for zero trust.
version: 1.0.0
tags:
source: configuring-microsegmentation-for-zero-trust

Configuring Microsegmentation for Zero Trust

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key boto3 EC2 Methods, Segmentation Enforcement Modes, External References, NIST SP 800-207, PCI DSS v4.0, VMware NSX Distributed Firewall

When to use

- The user wants to **configure** microsegmentation for zero trust.
- Capability: Microsegmentation audit agent for zero trust network enforcement.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, microsegmentation, zero, trust`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Audit AWS security groups for microsegmentation compliance
2. Check Illumio workload segmentation status
3. Generate microsegmentation policy recommendations
4. Execute microsegmentation audit

Key concepts

- Places individual or groups of resources on a unique network segment protected by a gateway security component
- This approach can be implemented using software-defined networking (SDN) or hypervisor-level enforcement
- AC-4 (Information Flow Enforcement): Microsegmentation enforces approved information flows between workloads
- SC-7 (Boundary Protection): Each microsegment boundary acts as a security boundary
- SI-4 (Information System Monitoring): Microsegmentation tools provide flow telemetry for monitoring
- Flow telemetry from microsegmentation agents feeds into SIEM/SOAR
- Application dependency maps provide baseline for anomaly detection
- Policy violation alerts enable real-time incident detection

Tools & resources

- Key boto3 EC2 Methods
- Segmentation Enforcement Modes
- External References
- NIST SP 800-207
- PCI DSS v4.0
- VMware NSX Distributed Firewall
- Illumio Core Platform

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_aws_security_groups` — Audit AWS security groups for microsegmentation compliance.
- `check_illumio_workloads` — Check Illumio workload segmentation status.
- `generate_segmentation_policy` — Generate microsegmentation policy recommendations.
- `run_audit` — Execute microsegmentation audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-microsegmentation-for-zero-trust`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-multi-factor-authentication-with-duo (Skill Module)

`configuring-multi-factor-authentication-with-duo\SKILL.md`

name: configuring-multi-factor-authentication-with-duo
description: Duo MFA configuration and audit agent using Duo Admin API. Use when the user asks to configure multi factor authentication with duo.
version: 1.0.0
tags:
source: configuring-multi-factor-authentication-with-duo

Configuring Multi Factor Authentication with Duo

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: External References, NIST Standards, Duo Documentation, CISA Guidance, Compliance

When to use

- The user wants to **configure** multi factor authentication with duo.
- Capability: Duo MFA configuration and audit agent using Duo Admin API.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, multi, factor, authentication`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Audit MFA enrollment coverage
2. Analyze authentication logs for anomalies
3. Execute Duo MFA audit

Key concepts

- NIST SP 800-63B: Digital Identity Guidelines - Authentication and Lifecycle Management
- AAL1: Single-factor authentication
- AAL2: Multi-factor authentication (Duo Push, TOTP)
- AAL3: Hardware-based phishing-resistant (FIDO2, PIV)
- NIST SP 800-53 Rev 5: IA-2, IA-2(1), IA-2(2), IA-2(6), IA-2(8), IA-3, IA-5
- CISA MFA Guidance: Phishing-resistant MFA requirement for federal agencies
- EO 14028: Executive Order on Improving the Nation's Cybersecurity - MFA mandate
- PCI DSS 4.0: Requirement 8.3.1 - MFA for all access to CDE

Tools & resources

- External References
- NIST Standards
- Duo Documentation
- CISA Guidance
- Compliance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_mfa_coverage` — Audit MFA enrollment coverage.
- `analyze_auth_logs` — Analyze authentication logs for anomalies.
- `run_audit` — Execute Duo MFA audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-multi-factor-authentication-with-duo`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-network-segmentation-with-vlans (Skill Module)

`configuring-network-segmentation-with-vlans\SKILL.md`

name: configuring-network-segmentation-with-vlans
description: VLAN Network Segmentation Agent - Configures and audits VLAN segmentation on managed switches. Use when the user asks to configure network segmentation with vlans.
version: 1.0.0
tags:
source: configuring-network-segmentation-with-vlans

Configuring Network Segmentation with Vlans

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key Functions, Security Checks, Supported Device Types

When to use

- The user wants to **configure** network segmentation with vlans.
- Capability: VLAN Network Segmentation Agent - Configures and audits VLAN segmentation on managed switches.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, network, segmentation, vlans`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Establish SSH connection to network device via Netmiko
2. Retrieve current VLAN configuration from the switch
3. Create a new VLAN on the switch
4. Configure a switch port as an access port in a specific VLAN
5. Configure a switch port as a trunk port with specific allowed VLANs
6. Shut down and assign unused ports to a quarantine VLAN
7. Configure access control lists for inter-VLAN routing
8. Audit VLAN configuration for common security issues

Tools & resources

- Key Functions
- Security Checks
- Supported Device Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_netmiko` — Establish SSH connection to network device via Netmiko.
- `get_vlan_config` — Retrieve current VLAN configuration from the switch.
- `create_vlan` — Create a new VLAN on the switch.
- `configure_access_port` — Configure a switch port as an access port in a specific VLAN.
- `configure_trunk_port` — Configure a switch port as a trunk port with specific allowed VLANs.
- `harden_unused_ports` — Shut down and assign unused ports to a quarantine VLAN.
- `configure_inter_vlan_acl` — Configure access control lists for inter-VLAN routing.
- `audit_vlan_security` — Audit VLAN configuration for common security issues.
- `get_napalm_config` — Retrieve device configuration using NAPALM for multi-vendor support.
- `generate_report` — Generate VLAN segmentation audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-network-segmentation-with-vlans`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-oauth2-authorization-flow (Skill Module)

configuring-oauth2-authorization-flow\SKILL.md

name: configuring-oauth2-authorization-flow
description: OAuth 2.0 authorization flow security audit. Use when the user asks to configure oauth2 authorization flow.
version: 1.0.0
tags:
source: configuring-oauth2-authorization-flow

Configuring Oauth2 Authorization Flow

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: OIDC Discovery Endpoint, OAuth 2.0 Grant Types, Security Best Practices, External References, Core OAuth Standards, Token Standards

When to use

- The user wants to **configure** oauth2 authorization flow.
- Capability: OAuth 2.0 authorization flow security audit agent.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, oauth2, authorization, flow`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Discover OAuth 2.0 / OIDC endpoints from well-known configuration
2. Audit OAuth configuration for security issues
3. Test token endpoint with client credentials
4. Execute OAuth 2.0 security audit

Key concepts

- RFC 6749: The OAuth 2.0 Authorization Framework
- https: //datatracker.ietf.org/doc/html/rfc6749
- RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage
- https: //datatracker.ietf.org/doc/html/rfc6750
- RFC 7636: Proof Key for Code Exchange (PKCE)
- https: //datatracker.ietf.org/doc/html/rfc7636
- RFC 9700: OAuth 2.0 Security Best Current Practice
- https: //datatracker.ietf.org/doc/html/rfc9700

Tools & resources

- OIDC Discovery Endpoint
- OAuth 2.0 Grant Types
- Security Best Practices
- External References
- Core OAuth Standards
- Token Standards
- OpenID Connect
- Additional Grant Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `discover_oauth_endpoints` — Discover OAuth 2.0 / OIDC endpoints from well-known configuration.
- `audit_oauth_security` — Audit OAuth configuration for security issues.
- `test_token_endpoint` — Test token endpoint with client credentials.
- `run_audit` — Execute OAuth 2.0 security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-oauth2-authorization-flow`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-pfsense-firewall-rules (Skill Module)

`configuring-pfsense-firewall-rules\SKILL.md`

name: configuring-pfsense-firewall-rules
description: pfSense Firewall Configuration Agent - Manages firewall rules via pfSense API. Use when the user asks to configure pfsense firewall rules.
version: 1.0.0
tags:
source: configuring-pfsense-firewall-rules

Configuring Pfsense Firewall Rules

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Prerequisites, Key Functions, Rule Templates

When to use

- The user wants to **configure** pfsense firewall rules.
- Capability: pfSense Firewall Configuration Agent - Manages firewall rules via pfSense API.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, pfsense, firewall, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Retrieve all firewall rules from pfSense
2. Create a new firewall rule on pfSense
3. Create standard LAN-to-WAN egress rules
4. Create DMZ isolation rules allowing only inbound web traffic
5. Create guest network isolation rules - internet only, no internal access
6. Create a NAT port forward rule
7. Audit firewall rules for common security issues
8. Retrieve recent firewall log entries

Tools & resources

- Prerequisites
- Key Functions
- Rule Templates

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_firewall_rules` — Retrieve all firewall rules from pfSense.
- `create_firewall_rule` — Create a new firewall rule on pfSense.
- `create_lan_to_wan_rules` — Create standard LAN-to-WAN egress rules.
- `create_dmz_rules` — Create DMZ isolation rules allowing only inbound web traffic.
- `create_guest_isolation_rules` — Create guest network isolation rules - internet only, no internal access.
- `configure_nat_port_forward` — Create a NAT port forward rule.
- `audit_firewall_rules` — Audit firewall rules for common security issues.
- `get_firewall_logs` — Retrieve recent firewall log entries.
- `generate_report` — Generate pfSense firewall configuration report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-pfsense-firewall-rules`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-snort-ids-for-intrusion-detection (Skill Module)

`configuring-snort-ids-for-intrusion-detection\SKILL.md`

name: configuring-snort-ids-for-intrusion-detection
description: Snort IDS configuration and rule management. Use when the user asks to configure snort ids for intrusion detection.
version: 1.0.0
tags:
source: configuring-snort-ids-for-intrusion-detection

Configuring Snort IDS for Intrusion Detection

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Snort, PulledPork 3 CLI

When to use

- The user wants to **configure** snort ids for intrusion detection.
- Capability: Snort IDS configuration and rule management agent.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, snort, intrusion, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Verify Snort 3 installation and return version info
2. Validate Snort configuration file syntax
3. Parse Snort alert_fast.txt and return alert statistics
4. Parse Snort alert_json.txt for structured alert analysis
5. Test Snort rules against a PCAP file for validation
6. Count active rules in rule files
7. Generate a Snort IDS deployment status report

Tools & resources

- Snort
- PulledPork 3 CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_snort_installed` — Verify Snort 3 installation and return version info.
- `validate_configuration` — Validate Snort configuration file syntax.
- `parse_alert_fast` — Parse Snort alert_fast.txt and return alert statistics.
- `parse_alert_json` — Parse Snort alert_json.txt for structured alert analysis.
- `test_rule_against_pcap` — Test Snort rules against a PCAP file for validation.
- `count_rules` — Count active rules in rule files.
- `generate_report` — Generate a Snort IDS deployment status report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-snort-ids-for-intrusion-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-suricata-for-network-monitoring (Skill Module)

configuring-suricata-for-network-monitoring\SKILL.md

name: configuring-suricata-for-network-monitoring
description: Suricata IDS/IPS monitoring and EVE JSON log analysis. Use when the user asks to configure suricata for network monitoring.
version: 1.0.0
tags:
source: configuring-suricata-for-network-monitoring

Configuring Suricata for Network Monitoring

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Suricata

When to use

- The user wants to **configure** suricata for network monitoring.
- Capability: Suricata IDS/IPS monitoring and EVE JSON log analysis agent.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, suricata, network, monitoring`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check Suricata installation and running status
2. Validate Suricata configuration
3. Parse EVE JSON log for alert events and produce statistics
4. Analyze DNS queries from EVE log for threat hunting
5. Extract JA3 fingerprints and TLS metadata from EVE log
6. Run suricata-update to fetch latest rulesets
7. Count active Suricata rules
8. Generate full Suricata deployment report

Tools & resources

- Suricata

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_suricata_status` — Check Suricata installation and running status.
- `validate_config` — Validate Suricata configuration.
- `parse_eve_alerts` — Parse EVE JSON log for alert events and produce statistics.
- `parse_eve_dns` — Analyze DNS queries from EVE log for threat hunting.
- `parse_eve_tls` — Extract JA3 fingerprints and TLS metadata from EVE log.
- `update_rules` — Run suricata-update to fetch latest rulesets.
- `count_rules` — Count active Suricata rules.
- `generate_report` — Generate full Suricata deployment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-suricata-for-network-monitoring`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-tls-1-3-for-secure-communications (Skill Module)

configuring-tls-1-3-for-secure-communications\SKILL.md

name: configuring-tls-1-3-for-secure-communications
description: TLS 1.3 configuration audit agent using ssl and cryptography libraries. Use when the user asks to configure tls 1 3 for secure communications.
version: 1.0.0
tags:
source: configuring-tls-1-3-for-secure-communications

Configuring TLS 1 3 for Secure Communications

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Python ssl Module Methods, TLS 1.3 Cipher Suites, Deprecated Versions, External References, Primary Standards, NIST Guidelines

When to use

- The user wants to **configure** tls 1 3 for secure communications.
- Capability: TLS 1.3 configuration audit agent using ssl and cryptography libraries.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, secure, communications`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check supported TLS versions on a server
2. Retrieve and analyze server certificate
3. Check negotiated cipher suites
4. Execute TLS 1.3 configuration audit

Key concepts

- Description: The core TLS 1.3 specification
- Key changes: 1-RTT handshake, mandatory PFS, removed RSA key transport, encrypted handshake messages
- Description: Updates IANA registries for TLS cipher suites and extensions
- Description: Allows endpoints to negotiate maximum record size
- Description: Defines how 0-RTT early data works with HTTP, including replay protections
- Description: Allows servers to provide OCSP responses during handshake
- Description: Forces browsers to use HTTPS for all connections
- Description: Federal guidelines for TLS deployment

Tools & resources

- Python ssl Module Methods
- TLS 1.3 Cipher Suites
- Deprecated Versions
- External References
- Primary Standards
- NIST Guidelines
- Testing Tools
- Compliance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_tls_versions` — Check supported TLS versions on a server.
- `get_certificate_info` — Retrieve and analyze server certificate.
- `check_cipher_suites` — Check negotiated cipher suites.
- `run_audit` — Execute TLS 1.3 configuration audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-tls-1-3-for-secure-communications`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-windows-defender-advanced-settings (Skill Module)

configuring-windows-defender-advanced-settings\SKILL.md

name: configuring-windows-defender-advanced-settings
description: Windows Defender advanced configuration audit. Use when the user asks to configure windows defender advanced settings.
version: 1.0.0
tags:
source: configuring-windows-defender-advanced-settings

Configuring Windows Defender Advanced Settings

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: PowerShell Cmdlets, Critical ASR Rule GUIDs, ASR Rule Actions, External References, Primary Standards, Compliance Mappings

When to use

- The user wants to **configure** windows defender advanced settings.
- Capability: Windows Defender advanced configuration audit agent.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, windows, defender, advanced, settings`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Get Windows Defender status via PowerShell
2. Get Windows Defender preference settings
3. Audit Attack Surface Reduction rules configuration
4. Check tamper protection status
5. Execute Windows Defender audit

Key concepts

- Scope: Complete reference for MDE deployment, configuration, and management
- Key sections: ASR rules, exploit protection, network protection, controlled folder access
- Relevance: Each ASR rule maps to specific ATT&CK techniques it mitigates
- Block Office child processes → T1566.001 (Spearphishing Attachment)
- Block credential stealing from LSASS → T1003.001 (OS Credential Dumping: LSASS)
- Block PSExec/WMI → T1047 (WMI), T1570 (Lateral Tool Transfer)
- Block WMI persistence → T1546.003 (WMI Event Subscription)
- Relevance: CIS benchmark sections covering Windows Defender configuration settings

Tools & resources

- PowerShell Cmdlets
- Critical ASR Rule GUIDs
- ASR Rule Actions
- External References
- Primary Standards
- Compliance Mappings
- ASR Rule Reference
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_defender_status` — Get Windows Defender status via PowerShell.
- `get_defender_preferences` — Get Windows Defender preference settings.
- `audit_asr_rules` — Audit Attack Surface Reduction rules configuration.
- `check_tamper_protection` — Check tamper protection status.
- `run_audit` — Execute Windows Defender audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-windows-defender-advanced-settings`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-windows-event-logging-for-detection (Skill Module)

configuring-windows-event-logging-for-detection\SKILL.md

name: configuring-windows-event-logging-for-detection
description: Windows event logging configuration audit. Use when the user asks to configure windows event logging for detection.
version: 1.0.0
tags:
source: configuring-windows-event-logging-for-detection

Configuring Windows Event Logging for Detection

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key PowerShell Cmdlets, Recommended Log Sizes, External References

When to use

- The user wants to **configure** windows event logging for detection.
- Capability: Windows event logging configuration audit agent.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, windows, event, logging, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Get current advanced audit policy configuration
2. Check if Sysmon is installed and running
3. Check event log maximum sizes
4. Check PowerShell script block logging and transcription
5. Execute Windows event logging audit

Key concepts

- NIST SP 800-92: Guide to Computer Security Log Management
- CIS Benchmark Section 17: Advanced Audit Policy Configuration
- NSA/CISA Event Forwarding Guidance: Recommended events for Windows monitoring

Tools & resources

- Key PowerShell Cmdlets
- Recommended Log Sizes
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_audit_policy` — Get current advanced audit policy configuration.
- `check_sysmon_installed` — Check if Sysmon is installed and running.
- `check_log_sizes` — Check event log maximum sizes.
- `check_powershell_logging` — Check PowerShell script block logging and transcription.
- `run_audit` — Execute Windows event logging audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-windows-event-logging-for-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

configuring-zscaler-private-access-for-ztna (Skill Module)

configuring-zscaler-private-access-for-ztna\SKILL.md

name: configuring-zscaler-private-access-for-ztna
description: Zscaler Private Access (ZPA) ZTNA audit agent using ZPA API. Use when the user asks to configure zscaler private access for ztna.
version: 1.0.0
tags:
source: configuring-zscaler-private-access-for-ztna

Configuring Zscaler Private Access for Ztna

Role: Secure configuration
Audience: security engineers and platform admins
Primary tools: Key ZPA Concepts, External References, NIST SP 800-207, Zscaler Documentation, NIST SP 800-63-3, Gartner ZTNA Market Guide

When to use

- The user wants to **configure** zscaler private access for ztna.
- Capability: Zscaler Private Access (ZPA) ZTNA audit agent using ZPA API.
- Goal: harden and tune security tooling for production use.
- Trigger keywords: `configuring, zscaler, private, access, ztna`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Map current VPN access patterns: which users access which applications
2. Identify all internal applications by FQDN, IP, and port
3. Group applications into logical segments by business function and sensitivity
4. Design App Connector placement: minimum 2 connectors per data center for HA
5. Plan DNS resolution: App Connectors must resolve all application FQDNs
6. Configure SAML 2.0 or OIDC integration with corporate IdP (Okta, Entra ID, Ping)
7. Map IdP user groups to ZPA access policy groups
8. Configure SCIM provisioning for automated user/group sync
9. Test SSO authentication flow end-to-end
10. Provision Linux VMs (CentOS/RHEL/Ubuntu) meeting minimum specs (2 vCPU, 4GB RAM)

Key concepts

- Section 2.1: Zero Trust Tenets - ZPA implements "never trust, always verify" with per-session application access
- Section 3.1: Policy Engine / Policy Administrator - maps to ZPA access policy engine
- Section 3.3: Agent/Gateway Model - maps to Zscaler Client Connector + App Connector architecture
- Network Pillar: Application micro-segmentation - ZPA application segments eliminate network-level access
- Identity Pillar: Continuous validation - ZPA evaluates identity and posture per session
- Section 4-6: Identity assurance levels relevant to ZPA IdP integration
- Section 302/404: Access controls for financial systems - ZPA provides auditable per-application access
- Application-level access logging satisfies audit trail requirements

Tools & resources

- Key ZPA Concepts
- External References
- NIST SP 800-207
- Zscaler Documentation

- NIST SP 800-63-3
- Gartner ZTNA Market Guide

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_zpa_config` — Audit ZPA configuration for security posture.
- `run_audit` — Execute ZPA ZTNA audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `configuring-zscaler-private-access-for-ztna`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

containing-active-breach (Skill Module)

containing-active-breach\SKILL.md

name: containing-active-breach
description: Active breach containment agent for incident response operations. Use when the user asks to containing active breach.
version: 1.0.0
tags:
source: containing-active-breach

Containing Active Breach

Role: Security workflow
Audience: security practitioners
Primary tools: CrowdStrike Falcon, Microsoft Defender for Endpoint, Active Directory, Network Containment - iptables, DNS Sinkholing, Evidence Collection

When to use

- The user wants to **conta** active breach.
- Capability: Active breach containment agent for incident response operations.
- Goal: complete the task.
- Trigger keywords: `containing, active, breach`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Block a C2 or malicious IP address using iptables
2. Disable a compromised Active Directory account via PowerShell
3. Force password reset for a compromised AD account
4. Isolate a compromised host by blocking all traffic except management
5. Block SMB (port 445) between server VLANs to prevent lateral movement
6. Add DNS sinkhole entry for a C2 domain
7. Collect volatile system data from a compromised host for forensics
8. Generate a structured containment status report

Tools & resources

- CrowdStrike Falcon
- Microsoft Defender for Endpoint
- Active Directory
- Network Containment - iptables
- DNS Sinkholing
- Evidence Collection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `block_ip_at_firewall` — Block a C2 or malicious IP address using iptables.
- `disable_ad_account` — Disable a compromised Active Directory account via PowerShell.
- `reset_ad_password` — Force password reset for a compromised AD account.
- `isolate_host_firewall` — Isolate a compromised host by blocking all traffic except management.
- `block_smb_lateral_movement` — Block SMB (port 445) between server VLANs to prevent lateral movement.
- `sinkhole_domain` — Add DNS sinkhole entry for a C2 domain.
- `collect_volatile_evidence` — Collect volatile system data from a compromised host for forensics.
- `generate_containment_report` — Generate a structured containment status report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `containing-active-breach`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

content-access (Skill Module)

research-toolkit\skills\content-access\SKILL.md

name: content-access

description: Legal methods for accessing paywalled and geo-blocked content. Use when researching behind paywalls, accessing academic papers, bypassing geographic restrictions, or finding open access alternatives. Covers Unpaywall, library databases, VPNs, and ethical access strategies for journalists and researchers.

tags:

Content access methodology

Ethical and legal approaches for accessing restricted web content for journalism and research.

Access hierarchy (most to least preferred)

```
            CONTENT ACCESS DECISION HIERARCHY


1. FULLY LEGAL (Always try first)
    Library databases (PressReader, ProQuest, JSTOR)
    Open access tools (Unpaywall, CORE, PubMed Central)
    Author direct contact
    Interlibrary loan

2. LEGAL (Browser features)
    Reader Mode (Safari, Firefox, Edge)
    Wayback Machine archives
    Google Scholar "All versions"

3. GREY AREA (Use with caution)
    Archive.is for individual articles
    Disable JavaScript (breaks functionality)
    VPNs for geo-blocked content

4. NOT RECOMMENDED
    Credential sharing
    Systematic scraping
    Commercial use of bypassed content
```

Open access tools for academic papers

Unpaywall browser extension

Unpaywall finds free, legal copies of 50M+ open-access academic records.

```python
# Unpaywall API (free, requires email for identification)
import requests

def find_open_access(doi: str, email: str) -> dict:
    """Find open access version of a paper using Unpaywall API.

    Args:
        doi: Digital Object Identifier (e.g., "10.1038/nature12373")
        email: Your email for API identification

    Returns:
        Dict with best open access URL if available
    """
    url = f"https://api.unpaywall.org/v2/{doi}?email={email}"

    response = requests.get(url, timeout=30)

    if response.status_code != 200:
        return {'error': f'Status {response.status_code}'}

    data = response.json()

    if data.get('is_oa'):
        best_location = data.get('best_oa_location', {})
        return {
            'is_open_access': True,
            'oa_url': best_location.get('url_for_pdf') or best_location.get('url'),
            'oa_status': data.get('oa_status'),  # gold, green, bronze, hybrid
            'host_type': best_location.get('host_type'),  # publisher, repository
            'version': best_location.get('version')  # publishedVersion, acceptedVersion
        }

    return {
        'is_open_access': False,
        'title': data.get('title'),
        'journal': data.get('journal_name')
    }

# Usage
result = find_open_access("10.1038/nature12373", "researcher@example.com")
if result.get('is_open_access'):
    print(f"Free PDF at: {result['oa_url']}")
```

CORE API (290M+ open-access works)

```
# CORE API - requires free API key from https://core.ac.uk/

import requests

class CORESearch:
    def __init__(self, api_key: str):
        self.api_key = api_key
        self.base_url = "https://api.core.ac.uk/v3"

    def search(self, query: str, limit: int = 10) -> list:
        """Search CORE database for open access papers."""

        headers = {'Authorization': f'Bearer {self.api_key}'}
        params = {
            'q': query,
            'limit': limit
        }

        response = requests.get(
            f"{self.base_url}/search/works",
            headers=headers,
            params=params,
            timeout=30
        )

        if response.status_code != 200:
            return []

        data = response.json()
        results = []

        for item in data.get('results', []):
            results.append({
                'title': item.get('title'),
                'authors': [a.get('name') for a in item.get('authors', [])],
                'year': item.get('yearPublished'),
                'doi': item.get('doi'),
                'download_url': item.get('downloadUrl'),
                'abstract': item.get('abstract', '')[:500]
            })

        return results

    def get_by_doi(self, doi: str) -> dict:
        """Get paper by DOI."""
        headers = {'Authorization': f'Bearer {self.api_key}'}

        response = requests.get(
            f"{self.base_url}/works/{doi}",
            headers=headers,
            timeout=30
        )

        return response.json() if response.status_code == 200 else {}
```

Semantic Scholar API (220M+ papers)

```
# Semantic Scholar API - free, but request a key from
# https://www.semanticscholar.org/product/api for anything beyond
# ad-hoc calls. Unkeyed access has been tightened to a low shared
# rate limit and is no longer reliable for batch lookups.

import requests

def search_semantic_scholar(query: str, limit: int = 10) -> list:
    """Search Semantic Scholar for papers with open access links."""

    url = "https://api.semanticscholar.org/graph/v1/paper/search"
    params = {
        'query': query,
        'limit': limit,
        'fields': 'title,authors,year,abstract,openAccessPdf,citationCount'
    }

    response = requests.get(url, params=params, timeout=30)

    if response.status_code != 200:
        return []

    results = []
    for paper in response.json().get('data', []):
        oa_pdf = paper.get('openAccessPdf', {})
        results.append({
            'title': paper.get('title'),
            'authors': [a.get('name') for a in paper.get('authors', [])],
            'year': paper.get('year'),
            'citations': paper.get('citationCount', 0),
            'open_access_url': oa_pdf.get('url') if oa_pdf else None,
            'abstract': paper.get('abstract', '')[:500] if paper.get('abstract') else ''
        })

    return results

def get_paper_by_doi(doi: str) -> dict:
    """Get paper details by DOI."""
    url = f"https://api.semanticscholar.org/graph/v1/paper/DOI:{doi}"
    params = {
        'fields': 'title,authors,year,abstract,openAccessPdf,references,citations'
    }

    response = requests.get(url, params=params, timeout=30)
    return response.json() if response.status_code == 200 else {}
```

OpenAlex API (250M+ scholarly works)

OpenAlex replaced Microsoft Academic Graph after MAG was retired and has become the de-facto open scholarly data backbone — many tools (Unpaywall companion data, Local Citation Network, OpenCitations) now resolve via OpenAlex.

Auth note (2026): OpenAlex moved to API-key-required access on February 13, 2026, with a credit-based rate model. Anonymous access to the website is still free; API access via key has metered limits that step up with paid tiers — verify the current model at https://docs.openalex.org/. Get a free key from your OpenAlex account.

```
# OpenAlex API client
# https://docs.openalex.org/
# Pricing & key issuance: https://openalex.org/

import requests

def search_openalex(query: str, api_key: str, limit: int = 25,
                    email: str = None) -> list:
    """Search OpenAlex for works.

    Args:
        query: free-text search string.
        api_key: OpenAlex API key (required as of 2026-02-13).
        limit: max results per page (1-200).
        email: contact email for the polite pool — recommended even
               with a key, since OpenAlex prioritizes requests with
               an identifiable sender.
    """
    headers = {
        'Authorization': f'Bearer {api_key}',
        'User-Agent': f'research-toolkit ({email})' if email else 'research-toolkit',
    }
    params = {'search': query, 'per-page': limit}

    response = requests.get(
        'https://api.openalex.org/works',
        params=params,
        headers=headers,
        timeout=30,
    )
    if response.status_code != 200:
        return []

    results = []
    for work in response.json().get('results', []):
        oa = work.get('open_access') or {}
        results.append({
            'id': work.get('id'),
            'doi': work.get('doi'),
            'title': work.get('title'),
            'year': work.get('publication_year'),
            'is_oa': oa.get('is_oa', False),
            'oa_status': oa.get('oa_status'),  # gold, green, hybrid, bronze, closed
            'oa_url': oa.get('oa_url'),
            'cited_by_count': work.get('cited_by_count', 0),
        })
    return results
```

Other open-access sources worth checking

- **DOAJ** (`doaj.org/api/v3`) — Directory of Open Access Journals; useful when you need to verify a publisher is fully OA before trusting a "journal lookup" claim.
- **EuropePMC** (`europepmc.org/RestfulWebService`) — Mirror of PubMed Central plus preprints, OA full-text search, and ORCID-aware author lookup.
- **PubMed Central** (`eutils.ncbi.nlm.nih.gov`) — NIH OA biomedical archive; required for NIH-funded papers under the 2026 OSTP Nelson Memo.

Deliberately excluded (legally risky, likely ToS / copyright violation)

This [[skill]] does not recommend Sci-Hub, Library Genesis (LibGen), Anna's Archive, or paywall-redirector services like 12ft.io / removepaywall.com. These are widely used in the research community but sit in clear legal grey-to-red zones (depending on jurisdiction) and have been targets of DMCA takedowns, publisher lawsuits, and domain seizures. Use the legitimate open-access paths above; if a paper truly isn't available, the author-contact and ILL paths in this [[skill]] have very high success rates without legal exposure.

Browser reader mode for soft paywalls

Activating reader mode

This bookmarklet only works for **soft / metered paywalls** where the publisher loads the article HTML and visually overlays a subscription prompt — the content is already in the DOM, just hidden. It does **not** defeat hard paywalls (NYT, WSJ, FT, The Atlantic, Bloomberg, Stratechery, etc.) where article HTML is server-side gated; on those

sites the bookmarklet simply removes overlays and reveals nothing useful. Systematic use to read otherwise-paywalled content may violate the publisher's ToS — use it only as a reader-mode shim for content you legitimately have access to.

```
// Bookmarklet to strip soft-paywall overlays so reader mode works
// Works on some soft paywalls that load content before blocking

javascript:(function(){
    // Try to extract article content
    var article = document.querySelector('article') ||
                  document.querySelector('[role="main"]') ||
                  document.querySelector('.article-body') ||
                  document.querySelector('.post-content');

    if (article) {
        // Remove paywall overlays
        document.querySelectorAll('[class*="paywall"], [class*="subscribe"], [id*="paywall"]')
            .forEach(el => el.remove());

        // Remove fixed position overlays
        document.querySelectorAll('*').forEach(el => {
            var style = getComputedStyle(el);
            if (style.position === 'fixed' && style.zIndex > 100) {
                el.remove();
            }
        });

        // Re-enable scrolling
        document.body.style.overflow = 'auto';
        document.documentElement.style.overflow = 'auto';

        console.log('Overlay removed. Content may now be visible.');
    }
})();
```

Reader mode by browser

Browser	How to Activate	Effectiveness
Safari	Click Reader icon in URL bar	High for soft paywalls
Firefox	Click Reader View icon (or F9)	High
Edge	Click Immersive Reader icon	Highest
Chrome	Side panel → Reading mode (stable since Chrome 114, May 2023)	Medium

Library database access

Checking library access programmatically

```
# Most library databases require authentication
# This shows how to structure library API access

class LibraryAccess:
    """Access pattern for library databases."""

    # Common library database endpoints
    DATABASES = {
        'pressreader': {
            'base': 'https://www.pressreader.com',
            'auth': 'library_card',
            'content': '7000+ newspapers/magazines'
        },
        'proquest': {
            'base': 'https://www.proquest.com',
            'auth': 'institutional',
            'content': 'news, dissertations, documents'
        },
        'jstor': {
            'base': 'https://www.jstor.org',
            'auth': 'institutional',
            'content': 'academic journals, books'
        },
        'nexis_uni': {
            'base': 'https://www.nexisuni.com',
            'auth': 'institutional',
            'content': 'legal, news, business'
        }
    }

    @staticmethod
    def get_pressreader_access_methods():
        """Ways to access PressReader through libraries."""
        return {
            'in_library': 'Connect to library WiFi, visit pressreader.com',
            'remote': 'Log in with library card credentials',
            'app': 'Download PressReader app, link library card',
            'note': 'Session length varies by library; typically requires re-authentication every 24-72 hours'
        }

# Interlibrary Loan (ILL) workflow
def request_via_ill(paper_info: dict, library_email: str) -> str:
    """Generate interlibrary loan request.

    ILL is free through most libraries and can get almost any paper.
    Turnaround: typically 3-7 days.
    """

    request = f"""
    INTERLIBRARY LOAN REQUEST

    Title: {paper_info.get('title')}
    Author(s): {paper_info.get('authors')}
    Journal: {paper_info.get('journal')}
    Year: {paper_info.get('year')}
    DOI: {paper_info.get('doi')}
    Volume/Issue: {paper_info.get('volume')}/{paper_info.get('issue')}
    Pages: {paper_info.get('pages')}

    Requested by: {library_email}
    """

    return request.strip()
```

VPN usage for geo-blocked content

When VPNs are appropriate

```
## Legitimate VPN use cases for journalists/researchers

### APPROPRIATE:
- Accessing region-specific news sources
- Researching how content appears in other countries
- Bypassing government censorship (in some contexts)
- Protecting source communications
- Verifying geo-targeted content

### INAPPROPRIATE:
- Circumventing legitimate access controls
- Accessing content you're contractually prohibited from viewing
- Evading bans or blocks placed on your account
```

VPN service evaluation

VPN ratings age badly — privacy claims, ownership structures, and audit findings change yearly. Rather than maintain a stale ranked table here (the major commercial VPNs have undergone notable ownership consolidation: ExpressVPN by Kape Technologies, Surfshark merging with Nord), consult an independent reviewer at point-of-use:

- **PrivacyGuides** (`privacyguides.org/en/vpn/`) — community-maintained, privacy-prioritized recommendations with explicit criteria.
- **Privacy Tools** historical comparisons.
- **Tor Browser** (`torproject.org`) — maximum-anonymity option, free, no provider trust required; slow but the right tool for source protection or genuinely sensitive research.

For routine geo-restriction testing (not source protection), mainstream commercial VPNs in the $3-10/month tier are interchangeable on speed; pick on jurisdiction (your threat model) and recent independent audits, not marketing copy.

Checking geo-restriction [[status]]

```
import requests

def check_geo_access(url: str, regions: list = None) -> dict:
    """Check if URL is accessible from different regions.

    Note: This requires VPN/proxy services for actual testing.
    This function shows the concept.
    """

    regions = regions or ['US', 'UK', 'EU', 'JP', 'AU']

    results = {}

    # Direct access test
    try:
        response = requests.get(url, timeout=10)
        results['direct'] = {
            'accessible': response.status_code == 200,
            'status_code': response.status_code
        }
    except Exception as e:
        results['direct'] = {'accessible': False, 'error': str(e)}

    # Would need VPN/proxy integration for regional testing
    # results[region] = test_through_proxy(url, region)

    return results
```

Archive-based access

Using Archive.today for paywalled articles

```
import requests
from urllib.parse import quote, unquote

def get_archived_article(url: str) -> str:
    """Try to get article from Archive.today.

    Archive.today often captures full article content because it
    renders JavaScript and captures the result. Legal status varies
    by jurisdiction; treat systematic use to bypass paywalls as ToS-
    violating and use only for ad-hoc research access.

    Operational notes (2026): the FBI subpoenaed archive.today's
    registrar in October 2025; Wikipedia stopped accepting it as a
    citation source in February 2026. Still useful for capturing
    JS-rendered content, but treat as secondary to Wayback Machine
    for legal/citation use.
    """
    from urllib.parse import urljoin

    # /newest/<url> 302s to the most recent snapshot or to a CAPTCHA
    # page if rate-limited. Disable redirects so we can inspect the
    # Location header explicitly. quote(unquote(url), ...) normalizes
    # any existing %xx escapes so they aren't double-encoded.
    search_url = f"https://archive.ph/newest/{quote(unquote(url), safe='')}"

    try:
        response = requests.get(
            search_url,
            timeout=30,
            allow_redirects=False,
            headers={'User-Agent': 'Mozilla/5.0 (research-archiver)'},
        )
        if response.status_code in (301, 302, 303, 307, 308):
            location = response.headers.get('Location')
            if location:
                resolved = urljoin(response.url, location)
                # Only return if we landed on an archive page, not CAPTCHA
                if 'archive.' in resolved and '/newest/' not in resolved:
                    return resolved
        return None
    except Exception:
        return None
```

Wayback Machine for historical access

```
def get_wayback_article(url: str) -> str:
    """Get article from Wayback Machine.

    100% legal - the Internet Archive is a recognized library.
    May have older versions of articles (before paywall implemented).
    """

    # Check availability
    api_url = "https://archive.org/wayback/available"

    try:
        response = requests.get(api_url, params={'url': url}, timeout=10)
        data = response.json()

        snapshot = data.get('archived_snapshots', {}).get('closest', {})

        if snapshot.get('available'):
            return snapshot['url']

        return None
    except Exception:
        return None
```

Google Scholar strategies

Finding free versions

```
def find_free_via_scholar(title: str) -> list:
    """Search strategies for finding free paper versions.

    Google Scholar often links to:
    - Author's personal website copies
    - Institutional repository versions
    - ResearchGate/Academia.edu uploads
    """

    strategies = [
        {
            'method': 'scholar_all_versions',
            'description': 'Click "All X versions" under result',
            'success_rate': 'Medium-High'
        },
        {
            'method': 'scholar_pdf_link',
            'description': 'Look for [PDF] link on right side',
            'success_rate': 'Medium'
        },
        {
            'method': 'title_plus_pdf',
            'description': f'Search: "{title}" filetype:pdf',
            'success_rate': 'Medium'
        },
        {
            'method': 'author_site',
            'description': 'Find author\'s academic page',
            'success_rate': 'Medium'
        },
        {
            'method': 'preprint_servers',
            'description': 'Search arXiv, SSRN, bioRxiv',
            'success_rate': 'Field-dependent'
        }
    ]

    return strategies
```

Direct author contact

Email

[[[

Academic papers

```
## Academic paper access strategies (in order)

1. **Unpaywall extension** - Check first, automatic
2. **OpenAlex** - 250M+ works with OA links; the de-facto open scholarly data backbone since MAG was retired
3. **Google Scholar** - Click "All versions", look for [PDF]
4. **Author's website** - Check their academic page
5. **Institutional repository** - Search university library
6. **Preprint servers** - arXiv, SSRN, bioRxiv, medRxiv (note: 2026 OSTP Nelson Memo requires immediate OA for federally-funded US research)
7. **ResearchGate/Academia.edu** - Author-uploaded copies, BUT availability is uneven: both have faced publisher takedown campaigns (Elsevier/ACS lawsuits)
and many entries now resolve to "request full text" rather than a PDF
8. **CORE.ac.uk** - 290M+ open access papers
9. **PubMed Central** - For biomedical papers
10. **Contact author directly** - High success rate (70-90%)
11. **Interlibrary Loan** - Free, gets almost anything
```

Books and reports

```
## Book/report access strategies

1. **Library digital lending** - Internet Archive, OverDrive
2. **Google Books** - Often has preview or full text
3. **HathiTrust** - Academic library consortium
4. **Project Gutenberg** - Public domain books
5. **OpenLibrary** - Internet Archive's book lending
6. **Publisher open access** - Some chapters/reports free
7. **Author/organization website** - Reports often available
8. **Interlibrary Loan** - Physical books, scanned chapters
```

Legal and ethical framework

Fair use considerations (US)

```
## Fair Use Factors (17 U.S.C. § 107)

1. **Purpose and character of use**
   - Transformative use (commentary, criticism) favored
   - Non-commercial/educational use favored
   - Journalism generally protected

2. **Nature of copyrighted work**
   - Factual works (news, research) - broader fair use
   - Creative works (fiction, art) - narrower fair use

3. **Amount used relative to whole**
   - Using only necessary portions favored
   - Heart of the work disfavored

4. **Effect on market**
   - Not replacing purchase disfavored
   - No market impact favored

## Journalism privilege:
News reporting is explicitly listed as fair use purpose.
However, wholesale copying of entire articles still problematic.
```

Best practices for researchers

```
## Ethical content access guidelines

### DO:
- Use library resources first (supports the ecosystem)
- Try open access tools before circumvention
- Contact authors directly (they want citations)
- Cite properly regardless of how you accessed content
- Budget for subscriptions to frequently-used sources

### DON'T:
- Share login credentials
- Systematically download entire databases
- Use bypassed content for commercial purposes
- Redistribute paywalled content
- Rely solely on bypass methods
```

correlating-security-events-in-qradar (Skill Module)

correlating-security-events-in-qradar\SKILL.md

name: correlating-security-events-in-qradar
description: IBM QRadar SIEM correlation and offense management. Use when the user asks to correlating security events in qradar.
version: 1.0.0
tags:
source: correlating-security-events-in-qradar

Correlating Security Events in Qradar

Role: Security workflow
Audience: security practitioners
Primary tools: QRadar, Offense Management API, Reference Data API, AQL Functions

When to use

- The user wants to **correlat** security events in qradar.
- Capability: IBM QRadar SIEM correlation and offense management agent.
- Goal: complete the task.
- Trigger keywords: `correlating, security, events, qradar`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. AQL query to detect brute force followed by success
2. AQL query to detect lateral movement patterns
3. Generate a QRadar offense summary report

Tools & resources

- QRadar
- Offense Management API
- Reference Data API
- AQL Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `brute_force_aql` — AQL query to detect brute force followed by success.
- `lateral_movement_aql` — AQL query to detect lateral movement patterns.
- `generate_report` — Generate a QRadar offense summary report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `correlating-security-events-in-qradar`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

correlating-threat-campaigns (Skill Module)

correlating-threat-campaigns\SKILL.md

name: correlating-threat-campaigns
description: Threat campaign correlation agent using MISP and STIX. Use when the user asks to correlating threat campaigns.
version: 1.0.0
tags:
source: correlating-threat-campaigns

Correlating Threat Campaigns

Role: Security workflow
Audience: security practitioners
Primary tools: MISP REST API, MISP Correlation Engine, OpenCTI GraphQL API, STIX 2.1 Campaign Object, STIX Relationship Types

When to use

- The user wants to **correlat** threat campaigns.
- Capability: Threat campaign correlation agent using MISP and STIX.
- Goal: complete the task.
- Trigger keywords: `correlating, threat, campaigns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate campaign attribution confidence from correlated events
2. Extract shared IOCs across correlated events for blocking
3. Build a structured campaign intelligence report

Tools & resources

- MISP REST API
- MISP Correlation Engine
- OpenCTI GraphQL API
- STIX 2.1 Campaign Object
- STIX Relationship Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `calculate_campaign_confidence` — Calculate campaign attribution confidence from correlated events.
- `extract_campaign_iocs` — Extract shared IOCs across correlated events for blocking.
- `build_campaign_report` — Build a structured campaign intelligence report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `correlating-threat-campaigns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

crisis-communications (Skill Module)

journalism-core\skills\crisis-communications\SKILL.md

name: crisis-communications

description: Crisis communication and rapid response workflows for journalists and communications professionals. Use when covering breaking news events, managing organizational communications during crises, coordinating rapid fact-checking efforts, or developing crisis response plans. Essential for newsrooms, PR teams, and anyone who needs to communicate accurately under time pressure.

tags:

Crisis communications

Frameworks for accurate, rapid communication during high-pressure situations.

When to activate

- Breaking news requires immediate coverage
- Organization faces public crisis or controversy
- Misinformation is spreading rapidly and needs countering
- Emergency situation requires coordinated communication
- Rapid fact-checking is needed before [[[[publication]]]]
- Preparing crisis response plans before incidents occur

Breaking news protocol

First 30 minutes checklist

```
## Breaking news response

**Event**: [Brief description]
**Time detected**: [HH:MM]
**Initial source**: [Where we learned of this]

### Immediate actions (0-10 min)
- [ ] Verify event is real (minimum 2 independent sources)
- [ ] Alert editor/team lead
- [ ] Check wire services (AP, Reuters, AFP)
- [ ] Monitor official accounts (agencies, officials)
- [ ] DO NOT publish unverified claims

### Verification phase (10-30 min)
- [ ] Primary source contacted/confirmed
- [ ] Location verified (if applicable)
- [ ] Official statement obtained or requested
- [ ] Eyewitness accounts gathered (note: unverified)
- [ ] Social media claims flagged for verification

### First publication decision
- [ ] What we KNOW (confirmed facts only)
- [ ] What we DON'T know (be explicit)
- [ ] What we're working to confirm
- [ ] Attribution clear for every claim
```

Newsroom escalation matrix

```
from enum import Enum
from dataclasses import dataclass
from typing import List

class CrisisLevel(Enum):
    LEVEL_1 = "routine"      # Single reporter can handle
    LEVEL_2 = "elevated"     # Editor involvement needed
    LEVEL_3 = "major"        # Multiple reporters, editor oversight
    LEVEL_4 = "critical"     # All hands, executive involvement

@dataclass
class BreakingEvent:
    description: str
    level: CrisisLevel
    confirmed_facts: List[str]
    unconfirmed_claims: List[str]
    sources_contacted: List[str]
    assigned_reporters: List[str]

    def escalation_needed(self) -> bool:
        """Determine if event needs escalation."""
        triggers = [
            len(self.unconfirmed_claims) > 5,  # Too many unknowns
            "fatalities" in self.description.lower(),
            "official" in self.description.lower(),
            "government" in self.description.lower(),
        ]
        return any(triggers)

ESCALATION_TRIGGERS = {
    CrisisLevel.LEVEL_2: [
        "Multiple fatalities confirmed",
        "Major public figure involved",
        "Legal/liability concerns",
        "Significant local impact",
    ],
    CrisisLevel.LEVEL_3: [
        "National news potential",
        "Active danger to public",
        "Major institution affected",
        "Coordinated misinformation detected",
    ],
    CrisisLevel.LEVEL_4: [
        "Mass casualty event",
        "Government/democracy implications",
        "Our organization directly involved",
        "Imminent physical threat",
    ],
}
```

Rapid verification framework

The 5-minute verification

When time is critical, prioritize these checks:

```
## Rapid verification checklist

### Source check (1 min)
- [ ] Who is claiming this?
- [ ] Are they in position to know?
- [ ] Have they been reliable before?

### Corroboration (2 min)
- [ ] Does anyone else confirm?
- [ ] Check wire services
- [ ] Check official sources

### Red flags (1 min)
- [ ] Too perfect/dramatic?
- [ ] Matches known false narratives?
- [ ] Single source only?

### Decision (1 min)
- [ ] PUBLISH: Multiple credible sources, no red flags
- [ ] HOLD: Needs more verification
- [ ] MONITOR: Developing, don't publish yet
```

Verification triage

For verification mechanics (source credibility, image/video checks, archiving evidence), use the **source-verification** [[skill]]. This [[skill]] focuses on the time-pressure layer: what to triage first, who decides to publish, and how to communicate uncertainty.

Triage rule: claims with the highest physical-harm or election-impact potential go first, then claims that name specific people, then everything else.

Crisis communication templates

Holding statement

For when you need to say something but facts are incomplete:

```
## Initial statement template

[Organization] is aware of [brief description of situation].

We are actively [gathering information / investigating / monitoring the situation].

[If applicable: The safety of [stakeholders] is our top priority.]

We will provide updates as verified information becomes available.

For media inquiries: [contact]
For [affected parties]: [resource/hotline]

Last updated: [timestamp]
```

Correction/clarification

```
## Correction notice

**Correction [or Clarification]**: An earlier version of this [article/statement/post] [stated/implied] [incorrect information].

[If factual error]: The correct information is: [accurate facts].

[If misleading]: To clarify: [accurate context].

This [article/statement] has been updated to reflect accurate information.

We regret the error.

[Timestamp of correction]
```

Retraction (when necessary)

```
## Retraction notice

**Retraction**: [Publication name] is retracting [article title], published on [date].

[Brief explanation of what was wrong]: Our reporting [stated/relied on] [problematic element]. Subsequent verification revealed [why it was wrong].

[What you're doing about it]: We have [removed/updated] the article and are reviewing our editorial processes.

[Accountability]: We apologize to [affected parties] and our readers.

The full text of the original article is available [here/on request] for transparency.

Questions: [contact]
```

Social media crisis response

Monitoring during crisis

```
from dataclasses import dataclass
from datetime import datetime
from typing import List, Dict
from collections import Counter

@dataclass
class CrisisMention:
    platform: str
    content: str
    author: str
    timestamp: datetime
    sentiment: str  # positive, negative, neutral, misinformation
    reach: int  # followers/potential impressions
    requires_response: bool = False

class CrisisMonitor:
    """Track crisis-related social mentions."""

    def __init__(self, crisis_keywords: List[str]):
        self.keywords = crisis_keywords
        self.mentions: List[CrisisMention] = []

    def add_mention(self, mention: CrisisMention):
        self.mentions.append(mention)

    def get_dashboard(self) -> dict:
        """Real-time crisis overview."""
        recent = [m for m in self.mentions
                  if (datetime.now() - m.timestamp).seconds < 3600]

        return {
            'total_mentions_1h': len(recent),
            'sentiment_breakdown': self._sentiment_counts(recent),
            'top_platforms': self._platform_counts(recent),
            'misinformation_count': len([
                m for m in recent
                if m.sentiment == 'misinformation'
            ]),
            'high_reach_negative': [
                m for m in recent
                if m.sentiment == 'negative' and m.reach > 10000
            ],
            'pending_responses': len([
                m for m in self.mentions
                if m.requires_response
            ])
        }

    def _sentiment_counts(self, mentions: List[CrisisMention]) -> Dict:
        return dict(Counter(m.sentiment for m in mentions))

    def _platform_counts(self, mentions: List[CrisisMention]) -> Dict:
        return dict(Counter(m.platform for m in mentions))
```

Response decision tree

```
## Should we respond to this?

### High priority (respond quickly)
- [ ] Factual misinformation spreading rapidly
- [ ] Direct question from journalist
- [ ] Affected party seeking help
- [ ] Influential account spreading false info

### Medium priority (respond thoughtfully)
- [ ] General negative sentiment
- [ ] Questions about our response
- [ ] Comparisons to competitors' handling

### Low priority / Do not engage
- [ ] Obvious trolling
- [ ] Bad faith actors
- [ ] Pile-on with no new claims
- [ ] Emotional venting (let it pass)

### Never respond
- [ ] While angry
- [ ] With unverified information
- [ ] In a way that escalates conflict
- [ ] By deleting legitimate criticism
```

Internal crisis coordination

Communication chain

```
from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

@dataclass
class CrisisTeamMember:
    name: str
    role: str
    phone: str
    email: str
    backup: Optional['CrisisTeamMember'] = None

@dataclass
class CrisisLog:
    """Maintain record of all crisis decisions."""
    entries: List[dict] = None

    def __post_init__(self):
        self.entries = self.entries or []

    def log(self, action: str, decision_maker: str,
            rationale: str, outcome: str = "pending"):
        self.entries.append({
            'timestamp': datetime.now().isoformat(),
            'action': action,
            'decision_maker': decision_maker,
            'rationale': rationale,
            'outcome': outcome
        })

    def export_for_postmortem(self) -> str:
        """Generate timeline for after-action review."""
        lines = ["# Crisis Response Timeline\n"]
        for entry in self.entries:
            lines.append(
                f"**{entry['timestamp']}** - {entry['action']}\n"
                f"- Decision by: {entry['decision_maker']}\n"
                f"- Rationale: {entry['rationale']}\n"
                f"- Outcome: {entry['outcome']}\n"
            )
        return '\n'.join(lines)
```

[[Status]] [[update]]

For keeping stakeholders informed:

```
## Crisis status update [number]

**As of**: [timestamp]
**Next update**: [scheduled time]

### Current situation
[2-3 sentence summary of where things stand]

### What we know
- [Confirmed fact 1]
- [Confirmed fact 2]

### What we're working on
- [Action item 1] - [owner]
- [Action item 2] - [owner]

### Key decisions made
- [Decision] - [rationale]

### Immediate next steps
1. [Next action]
2. [Next action]

### Resources needed
- [Resource/support needed]

---
**Contact**: [Crisis lead name and number]
```

Post-crisis [[review]]

After-action checklist

```
## Post-crisis review template

### Timeline reconstruction
- When did we first learn of the crisis?
- When did we first respond publicly?
- Key decision points and timing

### What went well
- [Specific success 1]
- [Specific success 2]

### What could improve
- [Gap or failure 1] → [Recommended fix]
- [Gap or failure 2] → [Recommended fix]

### Process questions
- Did escalation work as intended?
- Were the right people involved?
- Did we have accurate information when needed?
- Was external communication timely and accurate?

### Specific improvements
- [ ] Update crisis protocol to address [gap]
- [ ] Train team on [identified weakness]
- [ ] Add [resource/tool] for future incidents

### Documentation
- All communications archived: [ ] Yes
- Decision log complete: [ ] Yes
- Media coverage compiled: [ ] Yes
```

Misinformation rapid response

Counter-messaging framework

```
## Misinformation response checklist

### Before responding
- [ ] Is this claim spreading significantly?
- [ ] Will our response reach the affected audience?
- [ ] Could responding amplify the false claim?

### The response formula
1. **Lead with truth** (don't repeat the myth first)
2. **Be specific** about what's accurate
3. **Explain briefly** why the false claim is wrong
4. **Provide source** for accurate information

### Example structure
GOOD: "The event starts at 7pm at City Hall. Some posts incorrectly listed the time as 5pm."

BAD: "Some people are saying the event is at 5pm. That's wrong. It's actually at 7pm."
(Repeating the myth first reinforces it)
```

Journalist safety during crisis

```
## Field safety checklist

### Before deployment
- [ ] Editor knows your location and check-in schedule
- [ ] Emergency contacts updated
- [ ] Phone charged, backup battery
- [ ] First aid basics reviewed
- [ ] Exit routes identified

### During coverage
- [ ] Regular check-ins maintained
- [ ] Location sharing active
- [ ] Press credentials visible
- [ ] Recording when safe to do so
- [ ] Not taking unnecessary risks for story

### If situation escalates
- [ ] Leave immediately if directed
- [ ] Follow crowd flow, don't fight it
- [ ] Protect equipment but not at personal risk
- [ ] Contact newsroom ASAP
```

Related [[skills]]

- **source-verification** - Verify claims during fast-moving events
- **social-media-intelligence** - Monitor spread of information
- **interview-transcription** - Document statements accurately

---

[[Skill]] metadata

Field	Value
version	1.0.0
created	2025-12-26
updated	2026-05-08
author	Joe Amditis
domain	communications, journalism
complexity	advanced

data-journalism (Skill Module)

journalism-core\skills\data-journalism\SKILL.md

name: data-journalism

description: Data journalism workflows for analysis, visualization, and storytelling. Use when analyzing datasets, creating charts and maps, cleaning messy data, calculating statistics or building data-driven stories. Essential for reporters, newsrooms and researchers working with quantitative information.

tags:

Data journalism methodology

Systematic approaches for finding, analyzing and presenting data in journalism.

Story structure for data journalism

Data journalism framework

The framework for data journalism was established by Philip Meyer, a journalist for Knight-Ridder, Harvard Nieman Fellow and professor at UNC-Chapel Hill. In his book *The New Precision Journalism*, Meyer encourages journalists to treat journalism "as if it were a science" by adopting the scientific method:

- Make observations / formulate a question
- Research the question / collect, store, and retrieve data
- Formulate a hypothesis
- Test the hypothesis, using both qualitative (interviews, documents) and quantitative (data analysis) methods
- Analyze the results and reduce them to the most important findings
- Present them to the audience

The process is iterative, not sequential.

The data story arc

1. The hook (nut graf)
- What's the key finding?
- Why should readers care?
- What's the human impact?

2. The evidence
- Show the data
- Explain the methodology
- Acknowledge limitations

3. The context
- How does this compare to the past?
- How does this compare to elsewhere?
- What's the trend?

4. The human element
- Individual examples that illustrate the data
- Expert interpretation
- Affected voices

5. The implications
- What does this mean going forward?
- What questions remain?
- What actions could result?

6. The methodology box
- Where did the data come from?
- How was it analyzed?
- What are the limitations?
- How can readers explore further?

Methodology documentation

```
## How we did this analysis

### Data sources
[List all data sources with links and access dates]

### Time period
[Specify exactly what time period is covered]

### Definitions
[Define key terms and how you operationalized them]

### Analysis steps
1. [First step of analysis]
2. [Second step]
3. [Continue...]

### Limitations
- [Limitation 1]
- [Limitation 2]

### What we excluded and why
- [Excluded category]: [Reason]

### Verification
[How findings were verified/checked]

### Code and data availability
[Link to GitHub repo if sharing code/data]

### Contact
[How readers can reach you with questions]
```

Data acquisition

Public data sources

Federal data sources

General:
- **Data.gov** — Federal open data portal. Many datasets were removed between Feb 2025 and 2026; consult the Harvard LIL Data.gov archive and the Data Rescue Project for preserved copies before assuming anything is still accessible.
- **Census Bureau** (census.gov) — Demographics, economic data. Many research pages were removed during the 2025 transition; the End of Term Web Archive holds snapshots.
- **BLS** (bls.gov) — Employment, inflation, wages. Following the 2025 funding lapse, the October 2025 Employment Situation release was canceled and the CPS October 2025 reference period is permanently uncollected. Check revised release dates before relying on series continuity.
- **BEA** (bea.gov) — GDP, economic accounts.
- **FRED / Federal Reserve** (fred.stlouisfed.org) — Financial and macroeconomic data; expanded API access through 2026.
- **SEC EDGAR** — Corporate filings.

Specific domains:
- **EPA** (epa.gov/data) — Environmental data. At least 80 climate webpages were removed in Dec 2025, the endangerment finding was repealed Feb 12, 2026, and the Climate Change Indicators site was largely gutted. The Environmental Data & Governance Initiative maintains mirrors.
- **FDA / openFDA** (open.fda.gov) — Drug approvals, recalls, adverse events.
- **CDC WONDER** — Health statistics. Many datasets were removed from data.cdc.gov after Jan 2025, partially restored under Doctors for America v. Trump (TRO Feb 11, 2025) but with altered terminology in some returns. The volunteer-run RestoredCDC.org mirrors removed content.
- **NHTSA FARS / vPIC APIs** — Vehicle safety data.
- **DOT** — Transportation statistics.
- **FEC** — Campaign finance; 2025-2026 cycle data live.
- **USASpending.gov** — Federal contracts and grants; API v2 operational.

Court records:
- **CourtListener / RECAP** (courtlistener.com) — Free PACER alternative covering federal court filings; RECAP Search Alerts launched June 2025 ("Google Alerts for federal courts").
- **PACER** — Federal court filings; $0.10 per page, $30 per quarter waiver threshold.

State and local:
- State open data portals (search: "[state] open data")
- Tyler Data & Insights (formerly Socrata, rebranded May 2025) hosts many city and state portals

- OpenStreetMap, municipal GIS portals
- State comptroller and auditor reports

International:
- **Eurostat**, **OECD**, **World Bank Open Data**, **UN Data** — major comparative datasets, mostly stable through 2026.

Specialized:
- **NICAR Data Library** (IRE) — curated datasets, IRE members only.
- **IPUMS** (University of Minnesota) — free with account; canonical for harmonized microdata.
- **ICPSR** (University of Michigan) — social-science data archive.
- **ProPublica Data Store** — frozen; datasets only run through 2023.

Federal-data preservation (use when source data has been removed):
- Data Rescue Project — citizen + library mirrors of removed federal data; more than 1,230 datasets across 85 offices as of Aug 2025.
- End of Term Web Archive — 500TB / 100M-page snapshot of federal sites at the 2024-2025 transition.
- Internet Archive Wayback Machine — useful for individual page-level recovery.

Data request strategies

Public records requests for datasets

For request mechanics (templates, fee-waiver language, NJ OPRA, appeals, FOIA Improvement Act statutory citations), see the **foia-requests** [[skill]]. Data-specific guidance:

- Request databases, not just documents
- Ask for the data dictionary or schema
- Request in native format (CSV, SQL dump) — not PDFs or scanned printouts
- Specify field-level needs and any computed columns you want included
- For active datasets, ask the cadence (daily, monthly, quarterly) and request standing access if your reporting will continue

Building your own dataset

- Scraping public information (respect robots.txt, ToS, and rate limits)
- Crowdsourcing from readers
- Systematic document [[review]]
- Surveys with documented methodology

Commercial data sources for newsrooms

- LexisNexis, Refinitiv, Bloomberg
- Industry-specific databases (often via library proxy through your institution)

Data cleaning and preparation

Common data problems

```
from typing import Any

import pandas as pd
import numpy as np
from rapidfuzz import fuzz
from itertools import combinations

# Inflation adjustment
import cpi
import wbdata

def standardize_name(name: Any) -> str | None:
    """Standardize name format to 'First Last'."""
    if pd.isna(name):
        return None
    name = str(name).strip().upper()
    # Handle "LAST, FIRST" format
    if ',' in name:
        parts = name.split(',')
        name = f"{parts[1].strip()} {parts[0].strip()}"
    return name

def parse_date(date_str: Any) -> pd.Timestamp | None:
    """Parse dates in various formats."""
    if pd.isna(date_str):
        return None

    formats = [
        '%m/%d/%Y', '%Y-%m-%d', '%B %d, %Y',
        '%d-%b-%y', '%m-%d-%Y', '%Y/%m/%d'
    ]

    for fmt in formats:
        try:
            return pd.to_datetime(date_str, format=fmt)
        except:
            continue

    # Fall back to pandas parser
    try:
        return pd.to_datetime(date_str)
    except:
        return None


def handle_missing(
    df: pd.DataFrame,
    thresh: int | None = None,
    per_thresh: float | None = None,
    required_col: str | None = None,
) -> pd.DataFrame:
    """Drop rows missing values in `required_col` if missingness exceeds either threshold."""
    if required_col is None or df.empty:
        return df
    if required_col not in df.columns:
        return df

    missing = df[required_col].isna().sum()

    if thresh is not None and missing >= thresh:
        return df.dropna(subset=[required_col]).reset_index(drop=True).copy()

    if per_thresh is not None and (missing / len(df) * 100) >= per_thresh:
        return df.dropna(subset=[required_col]).reset_index(drop=True).copy()

    return df


def handle_duplicates(df: pd.DataFrame, thresh: int | None = None) -> pd.DataFrame:
    """Drop duplicate rows when count exceeds `thresh`."""
    if thresh is not None and df.duplicated().sum() >= thresh:
        return df.drop_duplicates().reset_index(drop=True).copy()
    return df
```

```
def flag_similar_names(df: pd.DataFrame, name_col: str, threshold: int = 85) -> pd.DataFrame:
    """Flag rows that have potential duplicate names using vectorized comparison."""

    names = df[name_col].dropna().unique()

    # Use combinations() to avoid nested loop and duplicate comparisons
    dup_names: set[Any] = {
        name
        for name1, name2 in combinations(names, 2)
        if fuzz.ratio(str(name1).lower(), str(name2).lower()) >= threshold
        for name in (name1, name2)
    }

    df['has_similar_name'] = df[name_col].isin(dup_names)
    return df


def flag_outliers(series: pd.Series, method: str = 'iqr', threshold: float = 1.5) -> pd.Series:
    """Flag statistical outliers."""
    if method == 'iqr':
        Q1 = series.quantile(0.25)
        Q3 = series.quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - threshold * IQR
        upper = Q3 + threshold * IQR
        return (series < lower) | (series > upper)
    elif method == 'zscore':
        z_scores = np.abs((series - series.mean()) / series.std())
        return z_scores > threshold



# use descriptive variable names and chain methods
data_clean = (pd

            # Load messy data — raw_data is a placeholder
            # Be sure to use the right reader for the filetype
            .read_csv('..data/raw/raw_data.csv')

            # DATA TYPE CORRECTIONS
            # Ensure proper types for analysis
            .assign(# Convert to numeric (handling errors)
                    amount = lambda x: pd.to_numeric(x['amount'], errors='coerce'),

                    # Convert to categorical (saves memory, enables ordering)
                    status = lambda x: pd.Categorical(x['status']))

            .assign(
                    # INCONSISTENT FORMATTING
                    # Problem: Names in different formats
                    # e.g., "SMITH, JOHN" vs "John Smith" vs "smith john"
                    name_clean = lambda x: standardize_name(x['name']),

                    # DATE INCONSISTENCIES
                    # Problem: Dates in multiple formats
                    # e.g., "01/15/2024", "2024-01-15", "January 15, 2024", "15-Jan-24"
                    parse_date = lambda x: parse_date(x['date']),

                    # OUTLIERS
                    # Identify potential data entry errors
                    amount_outlier = lambda x: flag_outliers(x['amount']),

                    )

            # Fuzzy duplicates (similar but not identical)
            # Use record linkage or manual review
            .pipe(flag_similar_names, name_col='name_clean', threshold=85)

            # MISSING VALUES
            # Strategy depends on context — set required_col when you need to drop incomplete rows
            .pipe(handle_missing, required_col='amount', per_thresh=20.0)

            # DUPLICATES — Find and handle duplicates
            .pipe(handle_duplicates, thresh=1)

            .reset_index(drop=True)
            .copy())
```

Data validation checklist

```
## Pre-analysis data validation

### Structural checks
- [ ] Row count matches expected
- [ ] Column count and names correct
- [ ] Data types appropriate
- [ ] No unexpected null columns

### Content checks
- [ ] Date ranges make sense
- [ ] Numeric values within expected bounds
- [ ] Categorical values match expected options
- [ ] Geographic data resolves correctly
- [ ] IDs are unique where expected

### Consistency checks
- [ ] Totals add up to expected values
- [ ] Cross-tabulations balance
- [ ] Related fields are consistent
- [ ] Time series is continuous

### Source verification
- [ ] Can trace back to original source
- [ ] Methodology documented
- [ ] Known limitations noted
- [ ] Update frequency understood
```

AI-assisted analysis: cautions

AI tools can speed up exploration, code generation, and pattern surfacing — but they have specific failure modes that journalists must guard against. *Mata v. Avianca* (2023, fabricated court citations sanctioned in federal court) and the Air Canada chatbot ruling (2024, hallucinated refund policy ruled binding on the airline) are the canonical cases of LLM fabrication treated as published fact.

What LLMs reliably get wrong

- **Calculations at scale** — A model may produce a confident-looking sum, percentage, or rate that's off by 1-15%. Re-run any LLM-produced number in pandas, SQL, or R yourself before publishing.
- **Source citations** — Models hallucinate plausible URLs, paper titles, dataset names, and FOIA exemptions that don't exist. Verify every cited source by visiting it.
- **Dataset columns** — When asked to describe a dataset's structure, an LLM may invent columns that aren't there. Cross-check against the actual schema (`df.dtypes` , `df.columns.tolist()`).
- **Statistical reasoning** — LLMs confuse correlation with causation, conflate sample statistics with population parameters, and misapply tests. Treat any analytical claim as a hypothesis to verify, not a finding.

Methodology disclosure

When AI was used in any stage (data cleaning, analysis, visualization, drafting), disclose it in the methodology box. Editors and readers need to know which steps had a human in the loop and which were automated.

- State the tool and version (e.g., "We used Claude 4.7 to draft the cleaning pipeline; the code was reviewed and run by [reporter]").
- State what was verified (e.g., "All numerical results were re-computed in pandas; all source citations were independently confirmed").
- State what was not verified (if relevant).

Reproducibility

When using AI to generate analysis code, save the prompt, the model name, and the version alongside the code. AI-generated code is part of your methodology and should be reproducible by another reporter on the same data.

Statistical analysis for journalism

Basic statistics with context

```
# Essential statistics for any dataset
def describe_for_journalism(df: pd.DataFrame, col: str) -> pd.DataFrame:
    """Generate journalist-friendly statistics."""
    stats = df[col].describe(percentiles=[0.25, 0.5, 0.75, 0.9, 0.99])

    # Add skewness to the describe() output
    stats['skewness'] = df[col].skew()

    return stats.to_frame(name=col)

# Example interpretation
stats = describe_for_journalism(salaries, 'salary')

print(f"""
ANALYSIS
---------------
We analyzed {stats['count']:,} salary records.

The median salary is ${stats['median']:,.0f}, meaning half of workers
earn more and half earn less.

The average salary is ${stats['mean']:,.0f}, which is
{'higher' if stats['mean'] > stats['median'] else 'lower'} than the median,
indicating the distribution is {'right-skewed (pulled up by high earners)'
if stats['skewness'] > 0 else 'left-skewed'}.

The top 10% of earners make at least ${stats['90th_percentile']:,.0f}.
The top 1% make at least ${stats['99th_percentile']:,.0f}.
""")
```

Comparisons and context

```
# Calculate change metrics for a column
def calculate_change(df: pd.DataFrame, col: str, periods: int = 1) -> pd.DataFrame:
    """Add change metrics to a DataFrame using built-in pandas methods.

    Args:
        df: Input DataFrame
        col: Column to calculate changes for
        periods: Number of rows to look back (1=previous row, 12=year-over-year for monthly)
    """
    return df.assign(
        absolute_change=df[col].diff(periods),
        percent_change=df[col].pct_change(periods) * 100,
        direction=np.sign(df[col].diff(periods)).map({1: 'increased', -1: 'decreased', 0: 'unchanged'})
    )

# Usage:
# changes = data_clean.pipe(calculate_change, 'revenue', periods=12) # Year-over-year for monthly data

# Per capita calculations (essential for fair comparisons)
def per_capita(value: float, population: float, multiplier: int = 100000) -> float:
    """Calculate per capita rate."""
    return (value / population) * multiplier  # Per 100,000 is standard

# Example: Crime rates
city_a = {'crimes': 5000, 'population': 100000}
city_b = {'crimes': 8000, 'population': 500000}

rate_a = per_capita(city_a['crimes'], city_a['population'])
rate_b = per_capita(city_b['crimes'], city_b['population'])

print(f"City A: {rate_a:.1f} crimes per 100,000 residents")
print(f"City B: {rate_b:.1f} crimes per 100,000 residents")
# City A actually has higher crime rate despite fewer total crimes!


def adjust_for_inflation(
    amount: float | pd.Series,
    from_year: int | pd.Series,
    to_year: int,
    country: str = 'US'
) -> float | pd.Series:
    """Adjust dollar amounts for inflation. Works with scalars or Series for .assign().

    Args:
        amount: Value(s) to adjust
        from_year: Original year(s) of the amount
        to_year: Target year to adjust to
        country: ISO 2-letter country code (default 'US'). US uses BLS data via cpi package,
                 others use World Bank CPI data (FP.CPI.TOTL indicator)
    """
    if country == 'US':
        # Use cpi package for US (more accurate, from BLS)
        if isinstance(from_year, pd.Series):
            return pd.Series([cpi.inflate(amt, yr, to=to_year)
                              for amt, yr in zip(amount, from_year)], index=amount.index)
        return cpi.inflate(amount, from_year, to=to_year)
    else:
        # Use World Bank data for other countries
        cpi_data = wbdata.get_dataframe(
            {'FP.CPI.TOTL': 'cpi'},
            country=country
        )['cpi'].to_dict()

        from_cpi = pd.Series(from_year).map(cpi_data) if isinstance(from_year, pd.Series) else cpi_data[from_year]
        to_cpi = cpi_data[to_year]
        return amount * (to_cpi / from_cpi)

# Usage:
# adjust_for_inflation(100, 2020, 2024)  # US by default
# adjust_for_inflation(100, 2020, 2024, country='GB')  # UK
# df.assign(inf_adjust24=lambda x: adjust_for_inflation(x['amount'], x['year'], 2024, country='DE'))

# Always adjust when comparing dollars across years!
```

Correlation vs causation

```
## Reporting correlations responsibly

### What you CAN say
- "X and Y are correlated"
- "As X increases, Y tends to increase"
- "Areas with higher X also tend to have higher Y"
- "X is associated with Y"

### What you CANNOT say (without more evidence)
- "X causes Y"
- "X leads to Y"
- "Y happens because of X"

### Questions to ask before implying causation
1. Is there a plausible mechanism?
2. Does the timing make sense (cause before effect)?
3. Is there a dose-response relationship?
4. Has the finding been replicated?
5. Have confounding variables been controlled?
6. Are there alternative explanations?

### Red flags for spurious correlations
- Extremely high correlation (r > 0.95) with unrelated things
- No logical connection between variables
- Third variable could explain both
- Small sample size with high variance
```

Data visualization

Chart selection guide

```
## Choosing the right chart

### Comparison
- **Bar chart**: Compare categories
- **Grouped bar**: Compare categories across groups
- **Bullet chart**: Actual vs target

### Change over time
- **Line chart**: Trends over time
- **Area chart**: Cumulative totals over time
- **Slope chart**: Change between two points

### Distribution
- **Histogram**: Distribution of one variable
- **Box plot**: Compare distributions across groups
- **Violin plot**: Detailed distribution shape

### Relationship
- **Scatter plot**: Relationship between two variables
- **Bubble chart**: Three variables (x, y, size)
- **Connected scatter**: Change in relationship over time

### Composition
- **Pie chart**: Parts of a whole (almost never use, max 5 slices, prefer donut charts)
- **Donut chart**: Parts of a whole
- **Stacked bar**: Parts of whole across categories
- **Treemap**: Hierarchical composition

### Geographic
- **Choropleth**: Values by region (use normalized data!)
- **Dot map**: Individual locations
- **Proportional symbol**: Magnitude at locations
```

Exploratory interactive visualizations with Plotly Express

```
import plotly.express as px
import plotly.graph_objects as go

# Set default template for all charts
px.defaults.template = 'simple_white'

def create_bar_chart(
    data: pd.DataFrame,
    title: str,
    source: str,
    x_val: str,
    y_val: str,
    x_lab: str | None = None,
    y_lab: str | None = None,
    text_col: str | None = None,
) -> go.Figure:
    """Create a bar chart with a source-credit footer."""
    fig = px.bar(
        data,
        x=x_val,
        y=y_val,
        text=text_col,
        title=title,
        labels={x_val: x_lab or x_val, y_val: y_lab or y_val},
    )
    fig.add_annotation(
        text=f"Source: {source}",
        showarrow=False,
        xref='paper', yref='paper',
        x=0, y=-0.15,
        xanchor='left', yanchor='top',
        font=dict(size=10, color='gray'),
    )
    return fig

# Example
fig = create_bar_chart(
    data,
    title='Annual widget production',
    source='Department of Widgets, 2024',
    x_val='year',
    y_val='widgets_prod',
    x_lab='Year',
    y_lab='Units produced',
)

fig.show()  # Interactive display
```

Publication-ready automated data visualizations with Datawrapper

```
import pandas as pd
import datawrapper as dw

# Authentication: Set DATAWRAPPER_ACCESS_TOKEN environment variable,
# or read from file and pass to create()
with open('datawrapper_api_key.txt', 'r') as f:
    api_key = f.read().strip()

# read in your data
data = pd.read_csv('../data/raw/data.csv')

# Create a bar chart using the new OOP API
chart = dw.BarChart(
    title='My Bar Chart Title',
    intro='Subtitle or description text',
    data=data,

    # Formatting options
    value_label_format=dw.NumberFormat.ONE_DECIMAL,
    show_value_labels=True,
    value_label_alignment='left',
    sort_bars=True,  # sort by value
    reverse_order=False,

    # Source attribution
    source_name='Your Data Source',
    source_url='https://example.com',
    byline='Your Name',

    # Optional: custom base color
    base_color='#1d81a2'
)

# Create and publish (uses DATAWRAPPER_ACCESS_TOKEN env var, or pass token)
chart.create(access_token=api_key)
chart.publish()

# Get chart URL and embed code
print(f"Chart ID: {chart.chart_id}")
print(f"Chart URL: https://datawrapper.dwcdn.net/{chart.chart_id}")
iframe_code = chart.get_iframe_code(responsive=True)

# Update existing chart with new data (for live-updating charts)
existing_chart = dw.get_chart('YOUR_CHART_ID')  # retrieve by ID
existing_chart.data = new_df  # assign new DataFrame
existing_chart.title = 'Updated Title'  # modify properties
existing_chart.update()  # push changes to Datawrapper
existing_chart.publish()  # republish to make live

# Optional — Export chart as image
chart.export(filepath='chart.png', width=800, height=600)

#view chart
chart
```

Avoiding misleading visualizations

```
## Chart integrity checklist

### Axes
- [ ] Y-axis starts at zero (for bar charts)
- [ ] Axis labels are clear
- [ ] Scale is appropriate (not truncated to exaggerate)
- [ ] Both axes labeled with units

### Data representation
- [ ] All data points visible
- [ ] Colors are distinguishable (including colorblind)
- [ ] Proportions are accurate
- [ ] 3D effects not distorting perception

### Context
- [ ] Title describes what's shown, not conclusion
- [ ] Time period clearly stated
- [ ] Source cited
- [ ] Sample size/methodology noted if relevant
- [ ] Uncertainty shown where appropriate

### Honesty
- [ ] Cherry-picking dates avoided
- [ ] Outliers explained, not hidden
- [ ] Dual axes justified (usually avoid)
- [ ] Annotations don't mislead
```

Working with geospatial data

Geocoding data

U.S. Census Geocoder

Best for: U.S. addresses only. Returns Census geography (tract, block, FIPS codes) along with coordinates—essential for joining with Census demographic data.

Pros: Completely free with no API key required. Returns Census geographies (state/county FIPS, tract, block) that let you join with ACS/decennial Census data. Good match rates for standard U.S. addresses.

Cons: Limited to 10,000 addresses per batch. U.S. addresses only. Slower than commercial alternatives. Lower match rates for non-standard addresses (PO boxes, rural routes, new construction).

Use when: You need to geocode nicely formatted U.S. addresses or you don't have budget for a paid service.

```
# pip install censusgeocode
# (the older `censusbatchgeocoder` package on PyPI hasn't been updated since 2017
# and is unmaintained — use `censusgeocode` instead, which wraps the same Census
# batch endpoint and is actively maintained.)

import censusgeocode as cg
import pandas as pd

# DataFrame must have columns matching the *_col parameters below
# (defaults: id, address, city, state, zipcode). If your CSV uses
# different names like 'street' or 'zip', pass them via address_col=,
# zipcode_col=, etc. — internally these are renamed to the keys the
# Census API expects ('address', 'zip', etc.) before the request.
# (state and zipcode are optional but improve match rates)

def census_geocode(
    df: pd.DataFrame,
    id_col: str = 'id',
    address_col: str = 'address',
    city_col: str = 'city',
    state_col: str = 'state',
    zipcode_col: str = 'zipcode',
    chunk_size: int = 9999,
) -> pd.DataFrame:
    """
    Geocode a DataFrame using the U.S. Census batch geocoder.
    Automatically handles datasets larger than 10,000 rows by chunking.

    Returns DataFrame with the documented batch-endpoint fields:
        id, address, match, matchtype, parsed, tigerlineid, side, lat, lon

    The batch endpoint does not return state/county/tract FIPS codes.
    For census-geography output, use the per-address helper below
    (`cg.onelineaddress(addr, returntype='geographies')`) on the matched
    rows, or call `cg.address(...)` per row.
    """
    col_map = {id_col: 'id', address_col: 'address', city_col: 'city'}
    if state_col and state_col in df.columns:
        col_map[state_col] = 'state'
    if zipcode_col and zipcode_col in df.columns:
        col_map[zipcode_col] = 'zip'

    renamed_df = df.rename(columns=col_map)
    records = renamed_df.to_dict('records')

    if len(records) <= chunk_size:
        return pd.DataFrame(cg.addressbatch(records))

    all_results = []
    for i in range(0, len(records), chunk_size):
        chunk = records[i:i + chunk_size]
        print(f"Geocoding rows {i:,} to {i + len(chunk):,} of {len(records):,}...")
        try:
            all_results.extend(cg.addressbatch(chunk))
        except Exception as e:
            print(f"Error on chunk starting at {i}: {e}")
            for record in chunk:
                all_results.append({**record, 'match': False, 'lat': None, 'lon': None})

    return pd.DataFrame(all_results)

# Usage:
geocoded = (pd
              .read_csv('../data/raw/addresses.csv')
              .assign(id=lambda x: x.index)
              .pipe(census_geocode,
                    id_col='id',
                    address_col='street',
                    city_col='city',
                    state_col='state',
                    zipcode_col='zip'))

# Follow-up per-address lookup for census geographies (state/county/tract FIPS).
# The batch endpoint above does not return these — only lat/lon and match status.
def add_geographies(geocoded: pd.DataFrame) -> pd.DataFrame:
    """For each matched row, fetch census-geography FIPS via per-address API."""
    fips_rows = []
    for row in geocoded[geocoded['match']].itertuples():
```

```
        try:
            geo = cg.onelineaddress(row.address, returntype='geographies')
            block = geo[0]['geographies']['Census Blocks'][0]
            fips_rows.append({
                'id': row.id,
                'state_fips': block['STATE'],
                'county_fips': block['COUNTY'],
                'tract': block['TRACT'],
                'block': block['BLOCK'],
            })
        except (IndexError, KeyError):
            continue
    return geocoded.merge(pd.DataFrame(fips_rows), on='id', how='left')
```

Google Maps Geocoder

Best for: International addresses, high match rates, and messy/non-standard address formats.

Pros: Excellent match rates even for poorly formatted addresses. Works worldwide. Fast and reliable. Returns rich metadata (place types, address components, place IDs).

Cons: Costs money ($5 per 1,000 requests after free tier). Requires API key and billing account. Does not return Census geography—you'd need to do a separate spatial join.

Use when: You need to geocode international addresses, have messy address data that the Census geocoder can't match, or need the highest possible match rate and have budget for it.

```
import googlemaps
from typing import Optional

def geocode_address_google(address: str, api_key: str) -> Optional[dict]:
    """
    Geocode address using Google Maps API.
    Requires API key with Geocoding API enabled.
    """
    gmaps = googlemaps.Client(key=api_key)
    result = gmaps.geocode(address)

    if result:
        location = result[0]['geometry']['location']
        return {
            'formatted_address': result[0]['formatted_address'],
            'lat': location['lat'],
            'lon': location['lng'],
            'place_id': result[0]['place_id']
        }
    return None

# Batch geocode a DataFrame
def batch_geocode(df: pd.DataFrame, address_col: str, api_key: str) -> pd.DataFrame:
    gmaps = googlemaps.Client(key=api_key)

    results = []
    for address in df[address_col]:
        try:
            result = gmaps.geocode(address)
            if result:
                loc = result[0]['geometry']['location']
                results.append({'lat': loc['lat'], 'lon': loc['lng']})
            else:
                results.append({'lat': None, 'lon': None})
        except Exception:
            results.append({'lat': None, 'lon': None})

    return pd.concat([df, pd.DataFrame(results)], axis=1)
```

Geopandas

```
import geopandas as gpd
import pandas as pd
from shapely.geometry import Point

# Read data from various formats
gdf = gpd.read_file('data.geojson')                    # GeoJSON
gdf = gpd.read_file('data.shp')                         # Shapefile
gdf = gpd.read_file('https://example.com/data.geojson') # From URL
gdf = gpd.read_parquet('data.parquet')                  # GeoParquet (fast!)

# Transform DataFrame with lat/lon to GeoDataFrame
df = pd.read_csv('locations.csv')
geometry = [Point(xy) for xy in zip(df['longitude'], df['latitude'])]
gdf = gpd.GeoDataFrame(df, geometry=geometry)

# Set CRS (Coordinate Reference System)
# EPSG:4326 = WGS84 (standard latitude, longitude)
gdf = gdf.set_crs('EPSG:4326')

# Transform to different CRS (for area/distance calculations, use projected CRS)
gdf_projected = gdf.to_crs('EPSG:3857')  # Web Mercator, for distance in meters

# Basic spatial operations

#Find the area of a shape
gdf['area'] = gdf_projected.geometry.area

#Find the center of a shape
gdf['centroid'] = gdf.geometry.centroid

#Draw a 1km boundary around a point
gdf['buffer_1km'] = gdf_projected.geometry.buffer(1000) #when set to CRS 3857

# Spatial join: find points within polygons
points = gpd.read_file('points.geojson')
polygons = gpd.read_file('boundaries.geojson')
joined = gpd.sjoin(points, polygons, predicate='within')

# Dissolve: merge geometries by attribute
dissolved = gdf.dissolve(by='state', aggfunc='sum')

# Export to various formats
gdf.to_parquet('output.parquet')          # GeoParquet (recommended)
gdf.to_file('output.geojson', driver='GeoJSON') #for tools that dont support GeoParquet
```

Geo-Visualization with `.explore()`, `lonboard` and Datawrapper

`.explore()`

Best for: Quick exploration and prototyping during data analysis.

Pros: Built into GeoPandas—method is available on any GeoDataFrame. Great for exploratory data analysis—checking that your data looks right, exploring spatial patterns, and iterating quickly on map designs.

Cons: Becomes slow with large datasets (>100k features). Limited customization compared to dedicated mapping libraries. Requires extra dependencies to be installed.

Use when: You're in the middle of analysis and want to quickly visualize your GeoDataFrame without switching tools.

Required dependencies:

```
pip install folium mapclassify matplotlib
```

- `folium` - Required for `.explore()` to work at all (renders the interactive map)
- `mapclassify` - Required when using `scheme=` parameter for classification (e.g., 'naturalbreaks', 'quantiles', 'equalinterval')
- `matplotlib` - Required for colormap (`cmap=`) support

```
import geopandas as gpd
# folium, mapclassify, and matplotlib must be installed but don't need to be imported
# geopandas imports them automatically when you call .explore()

# Basic interactive map (uses folium under the hood)
gdf.explore()

# Choropleth map with customization
# (requires mapclassify for scheme parameter)
gdf.explore(
    column='population',           # Column for color scale
    cmap='YlOrRd',                 # Matplotlib colormap
    scheme='naturalbreaks',        # Classification scheme (needs mapclassify)
    k=5,                           # Number of bins
    legend=True,
    tooltip=['name', 'population'],  # Columns to show on hover
    popup=True,                    # Show all columns on click
    tiles='CartoDB positron',      # Background tiles
    style_kwds={'color': 'black', 'weight': 0.5}  # Border style
)
```

`lonboard`

Best for: Large datasets and high-performance visualization in Jupyter notebooks.

Pros: GPU-accelerated rendering via deck.gl can handle millions of points smoothly. Excellent interactivity—pan, zoom, and hover work fluidly even with massive datasets. Native support for GeoArrow format for efficient data transfer.

Cons: Requires separate installation (`pip install lonboard`). Styling options are more technical (RGBA arrays, deck.gl conventions).

Use when: You have large point datasets (crime incidents, sensor readings, business locations) or need smooth interactivity with 100k+ features.

```
import geopandas as gpd
from lonboard import viz, Map, ScatterplotLayer, PolygonLayer

# Quick visualization (auto-detects geometry type)
viz(gdf)

# Custom ScatterplotLayer for points
layer = ScatterplotLayer.from_geopandas(
    gdf,
    get_radius=100,
    get_fill_color=[255, 0, 0, 200],  # RGBA
    pickable=True
)
m = Map(layer)
m

# PolygonLayer with color based on column
from lonboard.colormap import apply_continuous_cmap
import matplotlib.pyplot as plt

colors = apply_continuous_cmap(gdf['value'], plt.cm.viridis)
layer = PolygonLayer.from_geopandas(
    gdf,
    get_fill_color=colors,
    get_line_color=[0, 0, 0, 100],
    pickable=True
)
Map(layer)
```

Datawrapper

Best for: Publication-ready choropleth and proportional symbol maps for articles and reports.

Pros: Beautiful, professional defaults out of the box. Generates embeddable, responsive iframes that work in any CMS. Readers can interact (hover, click) without running any code. Accessible and mobile-friendly. Easy to update data programmatically for updating data.

Cons: Requires a Datawrapper account (free tier available). Limited to Datawrapper's supported boundary files—you can't bring arbitrary geometries. Less flexibility for custom visualizations.

Use when: You need a polished map for publication. Ideal for choropleth maps showing statistics by region (unemployment by state, COVID cases by county, election results by district). Your audience will view the map in a browser, not a notebook.

Unlike `.explore()` or `lonboard` , you don't pass raw geometry—instead you match your data to Datawrapper's built-in boundary files using standard codes (FIPS, ISO, etc.).

```
import datawrapper as dw
import pandas as pd

# Read API key
with open('datawrapper_api_key.txt', 'r') as f:
    api_key = f.read().strip()

# Prepare data with location codes that match Datawrapper's boundaries
# For US states: use 2-letter abbreviations or FIPS codes
# For countries: use ISO 3166-1 alpha-2 codes
df = pd.DataFrame({
    'state': ['AL', 'AK', 'AZ', 'AR', 'CA'],  # State abbreviations
    'unemployment_rate': [4.9, 3.2, 7.1, 4.2, 5.8]
})

# Create a choropleth map
chart = dw.ChoroplethMap(
    title='Unemployment Rate by State',
    intro='Percentage of labor force unemployed, 2024',
    data=df,

    # Map configuration
    basemap='us-states',           # Built-in US states boundaries
    basemap_key='state',           # Column in your data with location codes
    value_column='unemployment_rate',

    # Styling
    color_palette='YlOrRd',        # Color scheme
    legend_title='Unemployment %',

    # Attribution
    source_name='Bureau of Labor Statistics',
    source_url='https://www.bls.gov/',
    byline='Your Name'
)

# Create and publish
chart.create(access_token=api_key)
chart.publish()

# Get embed code for your article
iframe = chart.get_iframe_code(responsive=True)
print(f"Chart URL: https://datawrapper.dwcdn.net/{chart.chart_id}")

# Update with new data (for live-updating maps)
new_df = pd.DataFrame({...})  # Updated data
existing_chart = dw.get_chart('YOUR_CHART_ID')
existing_chart.data = new_df
existing_chart.update()
existing_chart.publish()
```

Available Datawrapper basemaps include:

- `us-states`, `us-counties`, `us-congressional-districts`
- `world`, `europe`, `africa`, `asia`
- Country-specific maps (e.g., `germany-states`, `uk-constituencies`)

Learning resources

- **NICAR** — Annual data journalism conference hosted by Investigative Reporters and Editors (IRE).
- **Knight Center for Journalism in the Americas** — Free MOOCs in English, Spanish, and Portuguese (journalismcourses.org).
- **Data Journalism Handbook 2: Towards a Critical Data Practice** (Bounegru and Gray) — current canonical edition (datajournalism.com/read/handbook/two).
- **Flowing Data** (flowingdata.com) — Nathan Yau's examples and tutorials.
- **The Pudding** (pudding.cool) — examples of editorial data essays.
- **Sigma Awards** (sigmaawards.org) — annual awards, hosted by GIJN since 2024.
- **Related [[skills]]** — for fact-checking computed numbers see **[[fact-check]]-workflow**; for FOIA-ing datasets see **foia-requests**; for verifying social-media-derived data see **source-verification**; for OSINT and platform-API context see **social-media-intelligence**.

[[Skill]] metadata

Field	Value
version	1.1.0
created	2025-12-26
updated	2026-05-08
author	Joe Amditis
domain	journalism, data
complexity	intermediate

deobfuscating-javascript-malware (Skill Module)

deobfuscating-javascript-malware\SKILL.md

name: deobfuscating-javascript-malware
description: JavaScript malware deobfuscation agent using jsbeautifier and pattern matching. Use when the user asks to deobfuscating javascript malware.
version: 1.0.0
tags:
source: deobfuscating-javascript-malware

Deobfuscating Javascript Malware

Role: Security workflow
Audience: security practitioners
Primary tools: jsbeautifier CLI, Node.js VM Sandbox, CyberChef Operations, IOC Extraction Regex

When to use

- The user wants to **deobfuscat** javascript malware.
- Capability: JavaScript malware deobfuscation agent using jsbeautifier and pattern matching.
- Goal: complete the task.
- Trigger keywords: `deobfuscating, javascript, malware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Beautify JavaScript code using jsbeautifier
2. Replace \xNN hex escape sequences with ASCII characters
3. Replace \uNNNN sequences with actual characters
4. Resolve String.fromCharCode() calls with static arguments
5. Resolve atob() calls containing static base64 strings
6. Resolve unescape() calls with percent-encoded strings
7. Identify obfuscation techniques used in the script
8. Extract potential IOCs from deobfuscated JavaScript

Tools & resources

- jsbeautifier CLI
- Node.js VM Sandbox
- CyberChef Operations
- IOC Extraction Regex

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `beautify_js` — Beautify JavaScript code using jsbeautifier.
- `decode_hex_strings` — Replace \xNN hex escape sequences with ASCII characters.
- `decode_unicode_escapes` — Replace \uNNNN sequences with actual characters.
- `decode_charcode_calls` — Resolve String.fromCharCode() calls with static arguments.
- `decode_atob_calls` — Resolve atob() calls containing static base64 strings.
- `decode_unescape_calls` — Resolve unescape() calls with percent-encoded strings.
- `detect_obfuscation_techniques` — Identify obfuscation techniques used in the script.
- `extract_iocs` — Extract potential IOCs from deobfuscated JavaScript.
- `remove_anti_debug` — Remove common anti-debugging traps from JavaScript.
- `deobfuscate` — Apply all deobfuscation passes to JavaScript code.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deobfuscating-javascript-malware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deobfuscating-powershell-obfuscated-malware (Skill Module)

`deobfuscating-powershell-obfuscated-malware\SKILL.md`

name: deobfuscating-powershell-obfuscated-malware
description: PowerShell obfuscated malware deobfuscation. Use when the user asks to deobfuscating powershell obfuscated malware.
version: 1.0.0
tags:
source: deobfuscating-powershell-obfuscated-malware

Deobfuscating PowerShell Obfuscated Malware

Role: Security workflow
Audience: security practitioners
Primary tools: Common Obfuscation Techniques, Detection Event IDs, External References, PowerShell Obfuscation Taxonomy

When to use

- The user wants to **deobfuscat** powershell obfuscated malware.
- Capability: PowerShell obfuscated malware deobfuscation agent.
- Goal: complete the task.
- Trigger keywords: `deobfuscating, powershell, obfuscated, malware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Find and decode Base64 encoded PowerShell commands
2. Resolve string concatenation obfuscation
3. Identify obfuscation techniques used in the script
4. Extract indicators of compromise from deobfuscated content
5. Execute PowerShell deobfuscation analysis

Tools & resources

- Common Obfuscation Techniques
- Detection Event IDs
- External References
- PowerShell Obfuscation Taxonomy

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `decode_base64_commands` — Find and decode Base64 encoded PowerShell commands.
- `deobfuscate_string_concatenation` — Resolve string concatenation obfuscation.
- `detect_obfuscation_techniques` — Identify obfuscation techniques used in the script.
- `extract_iocs` — Extract indicators of compromise from deobfuscated content.
- `run_analysis` — Execute PowerShell deobfuscation analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deobfuscating-powershell-obfuscated-malware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-active-directory-honeytokens (Skill Module)

deploying-active-directory-honeytokens\SKILL.md

name: deploying-active-directory-honeytokens

description: Active Directory Honeytoken Deployment and Monitoring Agent. Deploys deception-based honeytokens in Active Directory: fake privileged accounts with AdminCount=1, fake SPNs for Kerberoasting detection (honeyroasting),.... Use when the user asks to deploy active directory honeytokens.

version: 1.0.0

tags:

source: deploying-active-directory-honeytokens

Deploying Active Directory Honeytokens

Role: Deployment
Audience: security platform engineers
Primary tools: PowerShellGenerator, SIEMRuleGenerator, ADHoneytokenMonitor, HoneytokenDeployer, PowerShell Module

When to use

- The user wants to **deploy** active directory honeytokens.
- Capability: Active Directory Honeytoken Deployment and Monitoring Agent. Deploys deception-based honeytokens in Active Directory: fake privileged accounts with AdminCount=1, fake SPNs for Kerberoasting detection (honeyroasting), decoy GPOs with cpassword traps, and deceptive BloodHound paths. Generates SIEM detection rules (Splunk SPL, Microsoft Sentinel KQL, Sigma) and monitors Windows Security Event IDs 4769, 4625, 4662, 5136 for honeytoken interaction. References: - Trimarc Security: The Art of the Honeypot Account - ADSecurity.org: Detecting Kerberoasting Activity Part 2 - Microsoft Defender for Identity Honeytokens - SpecterOps: Kerberoasting and AES-256 - APT29a Blog: Deploying Honeytokens in AD
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, active, directory, honeytokens`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Plan rollout, dependencies, and rollback
2. Deploy components for active directory honeytokens
3. Smoke-test health and integrations
4. Monitor early telemetry and fix issues
5. Document operational ownership

Tools & resources

- PowerShellGenerator
- SIEMRuleGenerator
- ADHoneytokenMonitor
- HoneytokenDeployer
- PowerShell Module

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-active-directory-honeytokens`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-cloudflare-access-for-zero-trust (Skill Module)

`deploying-cloudflare-access-for-zero-trust\SKILL.md`

name: deploying-cloudflare-access-for-zero-trust
description: Cloudflare Access zero trust audit agent using Cloudflare API. Use when the user asks to deploy cloudflare access for zero trust.
version: 1.0.0
tags:
source: deploying-cloudflare-access-for-zero-trust

Deploying Cloudflare Access for Zero Trust

Role: Deployment
Audience: security platform engineers
Primary tools: Access Policy Rule Types, External References, NIST SP 800-207, Cloudflare Documentation, SOC 2 Type II, GDPR

When to use

- The user wants to **deploy** cloudflare access for zero trust.
- Capability: Cloudflare Access zero trust audit agent using Cloudflare API.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, cloudflare, access, zero, trust`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create Cloudflare account and navigate to Zero Trust dashboard
2. Select team name (organization identifier for WARP enrollment)
3. Choose subscription plan based on user count
4. Configure authentication: add primary IdP (Okta, Entra ID, Google Workspace)
5. Add secondary IdP for contractors or partners if needed
6. Enable MFA requirements at the IdP level
7. Install `cloudflared` on a server within the private network
8. Authenticate with `cloudflared tunnel login`
9. Create named tunnel: `cloudflared tunnel create <name>`
10. Configure ingress rules in `config.yml` mapping hostnames to internal services

Key concepts

- Section 2: ZTA Tenets - Cloudflare Access implements per-request identity verification
- Section 3.2: Enhanced Identity Governance - Access policies enforce continuous authorization
- Section 4.2: Cloud-Based SDP - Cloudflare Tunnel maps to software-defined perimeter architecture
- Identity: SSO with MFA via integrated IdP support
- Device: WARP client posture checks for OS, encryption, EDR
- Network: Cloudflare Tunnel eliminates inbound firewall rules
- Application: Per-application Access policies with session controls
- Cloudflare maintains SOC 2 Type II certification

Tools & resources

- Access Policy Rule Types
- External References
- NIST SP 800-207
- Cloudflare Documentation

- SOC 2 Type II
- GDPR

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_access_config` — Audit Cloudflare Access configuration.
- `run_audit` — Execute Cloudflare Access audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-cloudflare-access-for-zero-trust`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-decoy-files-for-ransomware-detection (Skill Module)

`deploying-decoy-files-for-ransomware-detection\SKILL.md`

name: deploying-decoy-files-for-ransomware-detection
description: Decoy file (canary) deployment agent for ransomware detection. Deploys canary files across file systems and monitors them for modifications that indicate ransomware encryption activity. Provides real-time alerting.... Use when the user asks to deploy decoy files for ransomware detection.
version: 1.0.0
tags:
source: deploying-decoy-files-for-ransomware-detection

Deploying Decoy Files for Ransomware Detection

Role: Deployment
Audience: security platform engineers
Primary tools: Windows ReadDirectoryChangesW API, Linux inotify Events, OSSEC/Wazuh File Integrity Monitoring, Sysmon File Monitoring, Common Ransomware File Extensions

When to use

- The user wants to **deploy** decoy files for ransomware detection.
- Capability: Decoy file (canary) deployment agent for ransomware detection. Deploys canary files across file systems and monitors them for modifications that indicate ransomware encryption activity. Provides real-time alerting when decoy files are modified, renamed, or deleted.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, decoy, files, ransomware, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a file
2. Generate realistic content for canary files
3. Deploy canary files to target directories
4. Check all canary files for modifications, deletions, or renames
5. Continuously monitor canary files at specified interval

Tools & resources

- Windows ReadDirectoryChangesW API
- Linux inotify Events
- OSSEC/Wazuh File Integrity Monitoring
- Sysmon File Monitoring
- Common Ransomware File Extensions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_file_hash` — Compute SHA-256 hash of a file.
- `generate_canary_content` — Generate realistic content for canary files.
- `deploy_canaries` — Deploy canary files to target directories.
- `check_canary_integrity` — Check all canary files for modifications, deletions, or renames.
- `monitor_loop` — Continuously monitor canary files at specified interval.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-decoy-files-for-ransomware-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-edr-agent-with-crowdstrike (Skill Module)

deploying-edr-agent-with-crowdstrike\SKILL.md

name: deploying-edr-agent-with-crowdstrike
description: CrowdStrike EDR deployment and monitoring agent using FalconPy. Use when the user asks to deploy edr agent with crowdstrike.
version: 1.0.0
tags:
source: deploying-edr-agent-with-crowdstrike

Deploying EDR Agent with Crowdstrike

Role: Deployment
Audience: security platform engineers
Primary tools: CrowdStrike, Key FalconPy Service Classes, Key Methods, FQL Filter Examples, External References, Primary Standards

When to use

- The user wants to **deploy** edr agent with crowdstrike.
- Capability: CrowdStrike EDR deployment and monitoring agent using FalconPy.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, agent, crowdstrike`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List managed hosts with sensor details
2. Retrieve recent detections
3. Audit sensor version compliance across fleet
4. Execute CrowdStrike EDR audit

Key concepts

- Publisher: MITRE Corporation
- Relevance: CrowdStrike Falcon maps detections to ATT&CK techniques; understanding ATT&CK is essential for tuning detectio
- Key tactics for EDR: Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Lateral Movement
- Relevance: Defines endpoint protection architecture including EDR placement and malware prevention controls
- Publisher: Center for Internet Security
- Relevance: CIS Controls v8 Control 10 mandates deploying anti-malware with centralized management and automated updates
- Scope: Official deployment procedures for Windows, macOS, Linux sensors
- Key sections: Silent install parameters, proxy configuration, sensor grouping tags

Tools & resources

- CrowdStrike
- Key FalconPy Service Classes
- Key Methods
- FQL Filter Examples
- External References
- Primary Standards
- Compliance Mappings
- Industry Benchmarks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `list_hosts` — List managed hosts with sensor details.
- `get_detections` — Retrieve recent detections.
- `check_sensor_versions` — Audit sensor version compliance across fleet.
- `run_audit` — Execute CrowdStrike EDR audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-edr-agent-with-crowdstrike`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-osquery-for-endpoint-monitoring (Skill Module)

`deploying-osquery-for-endpoint-monitoring\SKILL.md`

name: deploying-osquery-for-endpoint-monitoring
description: osquery endpoint monitoring agent for security auditing. Use when the user asks to deploy osquery for endpoint monitoring.
version: 1.0.0
tags:
source: deploying-osquery-for-endpoint-monitoring

Deploying Osquery for Endpoint Monitoring

Role: Deployment
Audience: security platform engineers
Primary tools: osquery, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **deploy** osquery for endpoint monitoring.
- Capability: osquery endpoint monitoring agent for security auditing.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, osquery, endpoint, monitoring`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute osquery and return results
2. Check Fleet server host enrollment status
3. Execute osquery security audit

Key concepts

- NIST SP 800-53 SI-4: System Monitoring - osquery provides endpoint visibility
- CIS Control 1: Inventory of Enterprise Assets - osquery software/hardware inventory
- CIS Control 2: Software Inventory - osquery package/process queries

Tools & resources

- osquery
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_osquery` — Execute osquery and return results.
- `check_fleet_status` — Check Fleet server host enrollment status.
- `run_audit` — Execute osquery security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-osquery-for-endpoint-monitoring`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-palo-alto-prisma-access-zero-trust (Skill Module)

deploying-palo-alto-prisma-access-zero-trust\SKILL.md

name: deploying-palo-alto-prisma-access-zero-trust
description: Palo Alto Prisma Access zero trust deployment audit. Use when the user asks to deploy palo alto prisma access zero trust.
version: 1.0.0
tags:
source: deploying-palo-alto-prisma-access-zero-trust

Deploying Palo Alto Prisma Access Zero Trust

Role: Deployment
Audience: security platform engineers
Primary tools: Base URL, Security Rule Actions, HIP Match Criteria, External References

When to use

- The user wants to **deploy** palo alto prisma access zero trust.
- Capability: Palo Alto Prisma Access zero trust deployment audit agent.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, palo, alto, prisma, access, zero`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Plan rollout, dependencies, and rollback
2. Deploy components for palo alto prisma access zero trust
3. Smoke-test health and integrations
4. Monitor early telemetry and fix issues
5. Document operational ownership

Tools & resources

- Base URL
- Security Rule Actions
- HIP Match Criteria
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `audit_security_rules` — Check for overly permissive security rules.
- `run_audit` — Execute Prisma Access zero trust audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-palo-alto-prisma-access-zero-trust`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-ransomware-canary-files (Skill Module)

deploying-ransomware-canary-files\SKILL.md

name: deploying-ransomware-canary-files
description: Ransomware canary file deployment and monitoring agent. Deploys decoy files across critical directories and monitors them using watchdog for real-time filesystem event detection. Any interaction with canary files.... Use when the user asks to deploy ransomware canary files.
version: 1.0.0
tags:
source: deploying-ransomware-canary-files

Deploying Ransomware Canary Files

Role: Deployment
Audience: security platform engineers
Primary tools: Canary File Deployment, Monitoring Functions, CanaryFileHandler Events, Alert Channels, Ransomware Extension Detection

When to use

- The user wants to **deploy** ransomware canary files.
- Capability: Ransomware canary file deployment and monitoring agent. Deploys decoy files across critical directories and monitors them using watchdog for real-time filesystem event detection. Any interaction with canary files triggers alerts via email, Slack, and syslog.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, ransomware, canary, files`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a file
2. Calculate Shannon entropy of file content to detect encryption
3. Get information about processes that may have accessed canary files
4. Deploy canary files to specified directories
5. Send alert email via SMTP
6. Send alert to Slack via incoming webhook
7. Send alert to syslog server via UDP
8. Start real-time canary file monitoring

Tools & resources

- Canary File Deployment
- Monitoring Functions
- CanaryFileHandler Events
- Alert Channels
- Ransomware Extension Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_sha256` — Compute SHA-256 hash of a file.
- `compute_entropy` — Calculate Shannon entropy of file content to detect encryption.
- `get_process_info` — Get information about processes that may have accessed canary files.
- `deploy_canary_files` — Deploy canary files to specified directories.
- `send_email_alert` — Send alert email via SMTP.
- `send_slack_alert` — Send alert to Slack via incoming webhook.
- `send_syslog_alert` — Send alert to syslog server via UDP.
- `start_monitoring` — Start real-time canary file monitoring.
- `verify_canary_integrity` — Verify all canary files match their original hashes.
- `simulate_ransomware_test` — Simulate ransomware activity against canary files for testing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-ransomware-canary-files`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-software-defined-perimeter (Skill Module)

`deploying-software-defined-perimeter\SKILL.md`

name: deploying-software-defined-perimeter
description: Software-Defined Perimeter (SDP) deployment audit. Use when the user asks to deploy software defined perimeter.
version: 1.0.0
tags:
source: deploying-software-defined-perimeter

Deploying Software Defined Perimeter

Role: Deployment
Audience: security platform engineers
Primary tools: Core SDP Concepts, Appgate SDP Admin API, Dark Port Scanning, Mutual TLS Verification, SPA Packet Structure, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **deploy** software defined perimeter.
- Capability: Software-Defined Perimeter (SDP) deployment audit agent.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, software, defined, perimeter`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check if a port responds to standard TCP connect (should be dark in SDP)
2. Verify SDP ports are invisible to unauthorized scanners
3. Verify mutual TLS authentication on SDP controller
4. Execute SDP deployment audit

Key concepts

- SDP Controller: Central policy and authentication authority
- Initiating Host (IH): Client device requesting access
- Accepting Host (AH): Gateway protecting backend resources
- Single Packet Authorization (SPA): Pre-authentication mechanism making services invisible
- All communication secured regardless of network location (mTLS tunnels)
- Per-session access grants (dynamic SDP connections)
- Dynamic policy evaluation (controller real-time decisions)
- Asset integrity monitoring (device posture checks)

Tools & resources

- Core SDP Concepts
- Appgate SDP Admin API
- Dark Port Scanning
- Mutual TLS Verification
- SPA Packet Structure
- External References

- CSA Software
- NIST SP 800-207

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_spa_port` — Check if a port responds to standard TCP connect (should be dark in SDP).
- `audit_dark_ports` — Verify SDP ports are invisible to unauthorized scanners.
- `check_tls_mutual_auth` — Verify mutual TLS authentication on SDP controller.
- `run_audit` — Execute SDP deployment audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-software-defined-perimeter`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

deploying-tailscale-for-zero-trust-vpn (Skill Module)

`deploying-tailscale-for-zero-trust-vpn\SKILL.md`

name: deploying-tailscale-for-zero-trust-vpn
description: Tailscale zero trust VPN deployment audit. Use when the user asks to deploy tailscale for zero trust vpn.
version: 1.0.0
tags:
source: deploying-tailscale-for-zero-trust-vpn

Deploying Tailscale for Zero Trust VPN

Role: Deployment
Audience: security platform engineers
Primary tools: ACL Policy Structure, Device Fields, External References, Protocol Standards, Tailscale Security Model, Compliance Considerations

When to use

- The user wants to **deploy** tailscale for zero trust vpn.
- Capability: Tailscale zero trust VPN deployment audit agent.
- Goal: roll out a security platform or agent capability safely.
- Trigger keywords: `deploying, tailscale, zero, trust`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Audit device compliance: OS versions, key expiry, last seen
2. Check ACL policy for overly permissive rules
3. Execute Tailscale zero trust audit

Key concepts

- Encryption: ChaCha20 for symmetric encryption
- Key Exchange: Curve25519 for Diffie-Hellman
- MAC: Poly1305 for message authentication
- Hashing: BLAKE2s for hashing
- Framework: Noise Protocol Framework for key negotiation
- Tailscale implements identity-aware proxying (Section 3.2.2)
- End-to-end encryption satisfies data-in-transit requirements
- ACL-based access control implements least privilege access

Tools & resources

- ACL Policy Structure
- Device Fields
- External References
- Protocol Standards
- Tailscale Security Model
- Compliance Considerations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_devices` — Audit device compliance: OS versions, key expiry, last seen.
- `audit_acl` — Check ACL policy for overly permissive rules.
- `run_audit` — Execute Tailscale zero trust audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `deploying-tailscale-for-zero-trust-vpn`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-ai-model-prompt-injection-attacks (Skill Module)

detecting-ai-model-prompt-injection-attacks\SKILL.md

name: detecting-ai-model-prompt-injection-attacks
description: Prompt Injection Detection Agent Multi-layered detector for identifying prompt injection attacks targeting LLM applications. Combines regex pattern matching, heuristic anomaly scoring, and DeBERTa-based.... Use when the user asks to detect ai model prompt injection attacks.
version: 1.0.0
tags:
source: detecting-ai-model-prompt-injection-attacks

Detecting Ai Model Prompt Injection Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: RegexDetector, HeuristicScorer, ClassifierDetector

When to use

- The user wants to **detect** ai model prompt injection attacks.
- Capability: Prompt Injection Detection Agent Multi-layered detector for identifying prompt injection attacks targeting LLM applications. Combines regex pattern matching, heuristic anomaly scoring, and DeBERTa-based classification to provide defense-in-depth against direct and indirect prompt injection attempts. Based on OWASP LLM Top 10 (LLM01:2025) and Simon Willison's prompt injection taxonomy.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, model, prompt, injection, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to ai model prompt injection attacks
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- RegexDetector
- HeuristicScorer
- ClassifierDetector

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `format_result_text` — Format a detection result as human-readable text.
- `format_result_json` — Format a detection result as JSON.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-ai-model-prompt-injection-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-anomalies-in-industrial-control-systems (Skill Module)

detecting-anomalies-in-industrial-control-systems\SKILL.md

name: detecting-anomalies-in-industrial-control-systems
description: ICS/SCADA anomaly detection agent for industrial control systems. Use when the user asks to detect anomalies in industrial control systems.
version: 1.0.0
tags:
source: detecting-anomalies-in-industrial-control-systems

Detecting Anomalies in Industrial Control Systems

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Modbus TCP Protocol, Common ICS Ports, pymodbus Client Usage, Anomaly Detection Thresholds, External References

When to use

- The user wants to **detect** anomalies in industrial control systems.
- Capability: ICS/SCADA anomaly detection agent for industrial control systems.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, anomalies, industrial, control, systems`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Read Modbus device identification and holding registers
2. Analyze Modbus traffic patterns for anomalies from parsed PCAP data
3. Verify ICS network segmentation by testing connectivity to OT ports
4. Query process historian for anomalous sensor readings
5. Execute ICS anomaly detection audit

Tools & resources

- Modbus TCP Protocol
- Common ICS Ports
- pymodbus Client Usage
- Anomaly Detection Thresholds
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_modbus_device` — Read Modbus device identification and holding registers.
- `analyze_modbus_traffic` — Analyze Modbus traffic patterns for anomalies from parsed PCAP data.
- `check_ics_network_segmentation` — Verify ICS network segmentation by testing connectivity to OT ports.
- `query_historian_anomalies` — Query process historian for anomalous sensor readings.
- `run_audit` — Execute ICS anomaly detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-anomalies-in-industrial-control-systems`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-anomalous-authentication-patterns (Skill Module)

detecting-anomalous-authentication-patterns\SKILL.md

name: detecting-anomalous-authentication-patterns
description: Authentication anomaly detection agent using UEBA analytics. Use when the user asks to detect anomalous authentication patterns.
version: 1.0.0
tags:
source: detecting-anomalous-authentication-patterns

Detecting Anomalous Authentication Patterns

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Azure AD Sign, Okta System Log API

When to use

- The user wants to **detect** anomalous authentication patterns.
- Capability: Authentication anomaly detection agent using UEBA analytics.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, anomalous, authentication, patterns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate great-circle distance between two coordinates in km
2. Load authentication logs from CSV with columns: timestamp,user,source_ip,result,lat,lon,city,country,app,device
3. Detect logins from locations requiring travel speed above threshold
4. Detect brute force: many failures for same user in time window
5. Detect password spray: many users targeted from same IP
6. Build behavioral baseline for a user from historical events
7. Calculate composite risk score from detected anomalies
8. Run all detection modules on an auth log CSV file

Tools & resources

- Azure AD Sign
- Okta System Log API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `haversine_km` — Calculate great-circle distance between two coordinates in km.
- `load_auth_logs` — Load authentication logs from CSV with columns: timestamp,user,source_ip,result,lat,lon,city,country,app,device
- `detect_impossible_travel` — Detect logins from locations requiring travel speed above threshold.
- `detect_brute_force` — Detect brute force: many failures for same user in time window.
- `detect_password_spray` — Detect password spray: many users targeted from same IP.
- `build_user_baseline` — Build behavioral baseline for a user from historical events.
- `calculate_risk_score` — Calculate composite risk score from detected anomalies.
- `run_full_analysis` — Run all detection modules on an auth log CSV file.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-anomalous-authentication-patterns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-api-enumeration-attacks (Skill Module)

detecting-api-enumeration-attacks\SKILL.md

name: detecting-api-enumeration-attacks
description: API enumeration attack detection. Use when the user asks to detect api enumeration attacks.
version: 1.0.0
tags:
source: detecting-api-enumeration-attacks

Detecting API Enumeration Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Detection Techniques, Common Enumeration Paths, WAF Rule Categories, External References

When to use

- The user wants to **detect** api enumeration attacks.
- Capability: API enumeration attack detection agent.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, enumeration, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse NGINX/Apache combined log format for API requests
2. Detect sequential ID enumeration in API paths
3. Detect abnormal request rates per IP to API endpoints
4. Detect API path/endpoint enumeration patterns
5. Query WAF API for blocked enumeration attempts
6. Execute API enumeration detection audit

Tools & resources

- Detection Techniques
- Common Enumeration Paths
- WAF Rule Categories
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_access_log` — Parse NGINX/Apache combined log format for API requests.
- `detect_sequential_ids` — Detect sequential ID enumeration in API paths.
- `detect_rate_anomalies` — Detect abnormal request rates per IP to API endpoints.
- `detect_path_enumeration` — Detect API path/endpoint enumeration patterns.
- `query_waf_logs` — Query WAF API for blocked enumeration attempts.
- `run_audit` — Execute API enumeration detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-api-enumeration-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-arp-poisoning-in-network-traffic (Skill Module)

detecting-arp-poisoning-in-network-traffic\SKILL.md

name: detecting-arp-poisoning-in-network-traffic
description: ARP poisoning detection agent for network traffic analysis. Use when the user asks to detect arp poisoning in network traffic.
version: 1.0.0
tags:
source: detecting-arp-poisoning-in-network-traffic

Detecting Arp Poisoning in Network Traffic

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Scapy ARP Packet Fields, ARP Spoofing Indicators, Scapy PCAP Analysis, System ARP Table, Detection Tools, External References

When to use

- The user wants to **detect** arp poisoning in network traffic.
- Capability: ARP poisoning detection agent for network traffic analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, poisoning, network, traffic`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze PCAP file for ARP poisoning indicators
2. Detect ARP spoofing from analyzed ARP traffic
3. Read current system ARP table and check for duplicates
4. Parse arpwatch database for flip-flop entries
5. Execute ARP poisoning detection audit

Tools & resources

- Scapy ARP Packet Fields
- ARP Spoofing Indicators
- Scapy PCAP Analysis
- System ARP Table
- Detection Tools
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_pcap_arp` — Analyze PCAP file for ARP poisoning indicators.
- `detect_arp_spoofing` — Detect ARP spoofing from analyzed ARP traffic.
- `check_arp_table` — Read current system ARP table and check for duplicates.
- `check_arpwatch_log` — Parse arpwatch database for flip-flop entries.
- `run_audit` — Execute ARP poisoning detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-arp-poisoning-in-network-traffic`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-attacks-on-historian-servers (Skill Module)

detecting-attacks-on-historian-servers\SKILL.md

name: detecting-attacks-on-historian-servers
description: Historian server attack detection agent for ICS/SCADA environments. Use when the user asks to detect attacks on historian servers.
version: 1.0.0
tags:
source: detecting-attacks-on-historian-servers

Detecting Attacks on Historian Servers

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Common Historian Platforms, Attack Indicators, External References

When to use

- The user wants to **detect** attacks on historian servers.
- Capability: Historian server attack detection agent for ICS/SCADA environments.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, attacks, historian, servers`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan for exposed historian service ports
2. Check OSIsoft PI Web API for authentication and configuration issues
3. Check Inductive Automation Ignition gateway status
4. Analyze historian access logs for attack indicators
5. Execute historian server attack detection audit

Tools & resources

- Common Historian Platforms
- Attack Indicators
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_historian_ports` — Scan for exposed historian service ports.
- `check_pi_web_api` — Check OSIsoft PI Web API for authentication and configuration issues.
- `check_ignition_gateway` — Check Inductive Automation Ignition gateway status.
- `analyze_historian_logs` — Analyze historian access logs for attack indicators.
- `run_audit` — Execute historian server attack detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-attacks-on-historian-servers`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-attacks-on-scada-systems (Skill Module)

detecting-attacks-on-scada-systems\SKILL.md

name: detecting-attacks-on-scada-systems
description: SCADA system attack detection. Use when the user asks to detect attacks on scada systems.
version: 1.0.0
tags:
source: detecting-attacks-on-scada-systems

Detecting Attacks on Scada Systems

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Modbus Attack Indicators, External References

When to use

- The user wants to **detect** attacks on scada systems.
- Capability: SCADA system attack detection agent.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, attacks, scada, systems`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan for exposed SCADA protocol ports
2. Detect Modbus protocol anomalies indicating attack
3. Test Siemens S7comm accessibility (basic connection test)
4. Query SIEM for SCADA-related security events
5. Execute SCADA attack detection audit

Tools & resources

- Modbus Attack Indicators
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_scada_services` — Scan for exposed SCADA protocol ports.
- `detect_modbus_anomalies` — Detect Modbus protocol anomalies indicating attack.
- `detect_s7comm_access` — Test Siemens S7comm accessibility (basic connection test).
- `query_scada_siem` — Query SIEM for SCADA-related security events.
- `run_audit` — Execute SCADA attack detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-attacks-on-scada-systems`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-aws-cloudtrail-anomalies (Skill Module)

detecting-aws-cloudtrail-anomalies\SKILL.md

name: detecting-aws-cloudtrail-anomalies
description: Agent for detecting anomalies in AWS CloudTrail logs. Queries CloudTrail events via boto3, builds behavioral baselines, and detects unusual API patterns indicating credential compromise, privilege escalation, or.... Use when the user asks to detect aws cloudtrail anomalies.
version: 1.0.0
tags:
source: detecting-aws-cloudtrail-anomalies

Detecting AWS Cloudtrail Anomalies

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: AWS, boto3 CloudTrail API, Rate Limits

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** aws cloudtrail anomalies.
- Capability: Agent for detecting anomalies in AWS CloudTrail logs. Queries CloudTrail events via boto3, builds behavioral baselines, and detects unusual API patterns indicating credential compromise, privilege escalation, or unauthorized access.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, cloudtrail, anomalies`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to aws cloudtrail anomalies
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- AWS
- boto3 CloudTrail API
- Rate Limits

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-aws-cloudtrail-anomalies`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-aws-credential-exposure-with-trufflehog (Skill Module)

`detecting-aws-credential-exposure-with-trufflehog\SKILL.md`

name: detecting-aws-credential-exposure-with-trufflehog
description: AWS credential exposure detection agent using TruffleHog and AWS APIs. Use when the user asks to detect aws credential exposure with trufflehog.
version: 1.0.0
tags:
source: detecting-aws-credential-exposure-with-trufflehog

Detecting AWS Credential Exposure with Trufflehog

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: AWS, TruffleHog CLI, git-secrets CLI, GuardDuty Credential Findings

When to use

- The user wants to **detect** aws credential exposure with trufflehog.
- Capability: AWS credential exposure detection agent using TruffleHog and AWS APIs.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, credential, exposure, trufflehog`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run TruffleHog against a local git repository
2. Scan an entire GitHub organization with TruffleHog
3. Scan a single file for AWS access key patterns
4. Scan all files in a directory for AWS credentials
5. Check if an AWS access key is active using AWS CLI
6. Deactivate an exposed AWS access key
7. Install git-secrets hooks for the current repository
8. Generate a credential exposure scan report

Tools & resources

- AWS
- TruffleHog CLI
- git-secrets CLI
- GuardDuty Credential Findings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_repo_trufflehog` — Run TruffleHog against a local git repository.
- `scan_github_org` — Scan an entire GitHub organization with TruffleHog.
- `scan_file_for_aws_keys` — Scan a single file for AWS access key patterns.
- `scan_directory_recursive` — Scan all files in a directory for AWS credentials.
- `check_aws_key_status` — Check if an AWS access key is active using AWS CLI.
- `deactivate_exposed_key` — Deactivate an exposed AWS access key.
- `setup_git_secrets` — Install git-secrets hooks for the current repository.
- `generate_report` — Generate a credential exposure scan report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-aws-credential-exposure-with-trufflehog`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-aws-guardduty-findings-automation (Skill Module)

detecting-aws-guardduty-findings-automation\SKILL.md

name: detecting-aws-guardduty-findings-automation
description: AWS GuardDuty findings automation and detection. Use when the user asks to detect aws guardduty findings automation.
version: 1.0.0
tags:
source: detecting-aws-guardduty-findings-automation

Detecting AWS Guardduty Findings Automation

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: AWS, Key boto3 GuardDuty Methods, Finding Severity Levels, GuardDuty Finding Types, GuardDuty Protection Features, FindingCriteria Filter

When to use

- The user wants to **detect** aws guardduty findings automation.
- Capability: AWS GuardDuty findings automation and detection agent.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, guardduty, findings, automation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Get the active GuardDuty detector ID
2. List GuardDuty findings filtered by severity
3. Get finding count grouped by type and severity
4. Audit GuardDuty detector configuration for coverage gaps
5. Check GuardDuty member account enrollment in multi-account setup
6. Auto-archive findings below severity threshold
7. Execute GuardDuty findings automation audit

Key concepts

- TA0001 Initial Access: UnauthorizedAccess findings
- TA0003 Persistence: Persistence:IAMUser findings
- TA0005 Defense Evasion: Stealth findings
- TA0006 Credential Access: CredentialAccess findings
- TA0010 Exfiltration: Exfiltration findings
- TA0040 Impact: CryptoCurrency/Trojan findings
- IR-4: Incident Handling
- IR-5: Incident Monitoring

Tools & resources

- AWS
- Key boto3 GuardDuty Methods
- Finding Severity Levels
- GuardDuty Finding Types
- GuardDuty Protection Features
- FindingCriteria Filter
- External References

- NIST 800-53

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_guardduty_detector` — Get the active GuardDuty detector ID.
- `list_findings` — List GuardDuty findings filtered by severity.
- `get_finding_statistics` — Get finding count grouped by type and severity.
- `check_detector_configuration` — Audit GuardDuty detector configuration for coverage gaps.
- `check_member_accounts` — Check GuardDuty member account enrollment in multi-account setup.
- `auto_archive_low_severity` — Auto-archive findings below severity threshold.
- `run_audit` — Execute GuardDuty findings automation audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-aws-guardduty-findings-automation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-aws-iam-privilege-escalation (Skill Module)

`detecting-aws-iam-privilege-escalation\SKILL.md`

name: detecting-aws-iam-privilege-escalation
description: Detect AWS IAM privilege escalation paths using boto3 policy analysis. Use when the user asks to detect aws iam privilege escalation.
version: 1.0.0
tags:
source: detecting-aws-iam-privilege-escalation

Detecting AWS IAM Privilege Escalation

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: AWS, boto3 IAM API, Cloudsplaining CLI, Known Privilege Escalation Vectors

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** aws iam privilege escalation.
- Capability: Detect AWS IAM privilege escalation paths using boto3 policy analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, privilege, escalation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Download full IAM authorization details via boto3
2. Extract all actions from a policy document
3. Check if a set of actions contains known privilege escalation combos
4. Flag dangerous wildcard action patterns
5. Analyze all principals for escalation paths
6. Generate privilege escalation analysis report

Tools & resources

- AWS
- boto3 IAM API
- Cloudsplaining CLI
- Known Privilege Escalation Vectors

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_account_authorization` — Download full IAM authorization details via boto3.
- `extract_policy_actions` — Extract all actions from a policy document.
- `check_escalation_paths` — Check if a set of actions contains known privilege escalation combos.
- `analyze_wildcard_policies` — Flag dangerous wildcard action patterns.
- `analyze_account` — Analyze all principals for escalation paths.
- `generate_report` — Generate privilege escalation analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-aws-iam-privilege-escalation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-azure-lateral-movement (Skill Module)

`detecting-azure-lateral-movement\SKILL.md`

name: detecting-azure-lateral-movement
description: Azure Lateral Movement Detection Agent - hunts for lateral movement in Azure AD/Entra ID. Use when the user asks to detect azure lateral movement.
version: 1.0.0
tags:
source: detecting-azure-lateral-movement

Detecting Azure Lateral Movement

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Azure, Sentinel KQL, Required Graph API Permissions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** azure lateral movement.
- Capability: Azure Lateral Movement Detection Agent - hunts for lateral movement in Azure AD/Entra ID.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, azure, lateral, movement`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate to Microsoft Graph API via OAuth2 client credentials
2. Query Azure AD audit logs for suspicious directory changes
3. Query Azure AD sign-in logs for anomalous authentication patterns
4. Detect suspicious OAuth application consent grants
5. Detect credential additions to service principals
6. Detect sign-ins from external/unknown tenants
7. Detect potential token replay by finding same user from multiple IPs in short windows

Tools & resources

- Azure
- Sentinel KQL
- Required Graph API Permissions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_graph_token` — Authenticate to Microsoft Graph API via OAuth2 client credentials.
- `query_audit_logs` — Query Azure AD audit logs for suspicious directory changes.
- `query_signin_logs` — Query Azure AD sign-in logs for anomalous authentication patterns.
- `detect_oauth_consent_abuse` — Detect suspicious OAuth application consent grants.
- `detect_service_principal_changes` — Detect credential additions to service principals.
- `detect_cross_tenant_signins` — Detect sign-ins from external/unknown tenants.
- `detect_token_replay` — Detect potential token replay by finding same user from multiple IPs in short windows.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-azure-lateral-movement`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-azure-service-principal-abuse (Skill Module)

detecting-azure-service-principal-abuse\SKILL.md

name: detecting-azure-service-principal-abuse
description: Azure Service Principal abuse detection. Use when the user asks to detect azure service principal abuse.
version: 1.0.0
tags:
source: detecting-azure-service-principal-abuse

Detecting Azure Service Principal Abuse

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Azure, OAuth2 Token Endpoint, High-Privilege Directory Roles, Service Principal Abuse Indicators, External References, Microsoft Secure Score Recommendations

When to use

- The user wants to **detect** azure service principal abuse.
- Capability: Azure Service Principal abuse detection agent.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, azure, service, principal, abuse`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check for expired or soon-to-expire service principal credentials
2. Find service principals with high-privilege directory roles
3. Execute Azure service principal abuse detection audit

Key concepts

- T1098.001: Account Manipulation - Additional Cloud Credentials
- T1078.004: Valid Accounts - Cloud Accounts
- T1087.004: Account Discovery - Cloud Account
- T1528: Steal Application Access Token
- T1550.001: Use Alternate Authentication Material - Application Access Token
- Require admin approval for unmanaged applications
- Remove unused application permissions
- Limit service principal credential lifetime

Tools & resources

- Azure
- OAuth2 Token Endpoint
- High-Privilege Directory Roles
- Service Principal Abuse Indicators
- External References
- Microsoft Secure Score Recommendations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_credential_expiry` — Check for expired or soon-to-expire service principal credentials.
- `audit_privileged_sp_roles` — Find service principals with high-privilege directory roles.
- `run_audit` — Execute Azure service principal abuse detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-azure-service-principal-abuse`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-azure-storage-account-misconfigurations (Skill Module)

`detecting-azure-storage-account-misconfigurations\SKILL.md`

name: detecting-azure-storage-account-misconfigurations
description: Audit Azure Storage accounts for security misconfigurations using azure-mgmt-storage SDK. Use when the user asks to detect azure storage account misconfigurations.
version: 1.0.0
tags:
source: detecting-azure-storage-account-misconfigurations

Detecting Azure Storage Account Misconfigurations

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Azure, SDK Installation, StorageManagementClient Initialization, Key Operations, Security Properties to Audit, Container Public Access Levels

When to use

- The user wants to **detect** azure storage account misconfigurations.
- Capability: Audit Azure Storage accounts for security misconfigurations using azure-mgmt-storage SDK.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, azure, storage, account, misconfigurations`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Initialize StorageManagementClient with DefaultAzureCredential
2. Audit a single storage account for misconfigurations
3. Check blob containers for individual public access settings
4. Run the full storage account audit

Tools & resources

- Azure
- SDK Installation
- StorageManagementClient Initialization
- Key Operations
- Security Properties to Audit
- Container Public Access Levels
- Environment Variables

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_storage_client` — Initialize StorageManagementClient with DefaultAzureCredential.
- `audit_storage_account` — Audit a single storage account for misconfigurations.
- `audit_blob_containers` — Check blob containers for individual public access settings.
- `run_audit` — Run the full storage account audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-azure-storage-account-misconfigurations`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-beaconing-patterns-with-zeek (Skill Module)

detecting-beaconing-patterns-with-zeek\SKILL.md

name: detecting-beaconing-patterns-with-zeek
description: Agent for detecting C2 beaconing patterns in Zeek conn.log data. Use when the user asks to detect beaconing patterns with zeek.
version: 1.0.0
tags:
source: detecting-beaconing-patterns-with-zeek

Detecting Beaconing Patterns with ZEEK

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Zeek, Beaconing Detection Math

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** beaconing patterns with zeek.
- Capability: Agent for detecting C2 beaconing patterns in Zeek conn.log data.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, beaconing, patterns, zeek`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load Zeek conn.log into a Pandas DataFrame using ZAT
2. Calculate a beacon score based on interval regularity
3. Detect beaconing by analyzing connection interval regularity
4. Detect beaconing with expected interval and jitter tolerance
5. Analyze Zeek dns.log for DNS-based beaconing patterns
6. Remove known-good destinations from beacon results

Tools & resources

- Zeek
- Beaconing Detection Math

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_conn_log` — Load Zeek conn.log into a Pandas DataFrame using ZAT.
- `calculate_beacon_score` — Calculate a beacon score based on interval regularity.
- `detect_beaconing` — Detect beaconing by analyzing connection interval regularity.
- `detect_jitter_beaconing` — Detect beaconing with expected interval and jitter tolerance.
- `analyze_dns_beaconing` — Analyze Zeek dns.log for DNS-based beaconing patterns.
- `filter_whitelisted` — Remove known-good destinations from beacon results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-beaconing-patterns-with-zeek`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-bluetooth-low-energy-attacks (Skill Module)

detecting-bluetooth-low-energy-attacks\SKILL.md

name: detecting-bluetooth-low-energy-attacks
description: BLE Attack Detection Agent - Scans, enumerates, and analyzes Bluetooth Low Energy devices for security vulnerabilities including weak pairing, replay susceptibility, insecure GATT permissions, and advertising spoofing. Use when the user asks to detect bluetooth low energy attacks.
version: 1.0.0
tags:
source: detecting-bluetooth-low-energy-attacks

Detecting Bluetooth Low Energy Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Arguments, Key Functions, Threat Detection Coverage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** bluetooth low energy attacks.
- Capability: BLE Attack Detection Agent - Scans, enumerates, and analyzes Bluetooth Low Energy devices for security vulnerabilities including weak pairing, replay susceptibility, insecure GATT permissions, and advertising spoofing.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, bluetooth, energy, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze a BLE packet capture file for attack indicators using tshark
2. Start a BLE packet capture with Ubertooth One
3. Generate comprehensive BLE security assessment report

Tools & resources

- Arguments
- Key Functions
- Threat Detection Coverage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_pcap_for_ble_attacks` — Analyze a BLE packet capture file for attack indicators using tshark.
- `run_ubertooth_capture` — Start a BLE packet capture with Ubertooth One.
- `generate_report` — Generate comprehensive BLE security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-bluetooth-low-energy-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-broken-object-property-level-authorization (Skill Module)

`detecting-broken-object-property-level-authorization\SKILL.md`

name: detecting-broken-object-property-level-authorization
description: BOPLA vulnerability scanner for OWASP API3:2023 Broken Object Property Level Authorization. Tests APIs for excessive data exposure and mass assignment vulnerabilities by comparing responses against expected field.... Use when the user asks to detect broken object property level authorization.
version: 1.0.0
tags:
source: detecting-broken-object-property-level-authorization

Detecting Broken Object Property Level Authorization

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: OWASP API3, Python requests Library, Mass Assignment Test Payloads, Burp Suite Extensions

When to use

- The user wants to **detect** broken object property level authorization.
- Capability: BOPLA vulnerability scanner for OWASP API3:2023 Broken Object Property Level Authorization. Tests APIs for excessive data exposure and mass assignment vulnerabilities by comparing responses against expected field sets and injecting extra properties.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, broken, object, property, level, authorization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to broken object property level authorization
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- OWASP API3
- Python requests Library
- Mass Assignment Test Payloads
- Burp Suite Extensions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `classify_field`
- `flatten_keys`
- `test_excessive_exposure`
- `test_mass_assignment`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-broken-object-property-level-authorization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-business-email-compromise (Skill Module)

detecting-business-email-compromise\SKILL.md

name: detecting-business-email-compromise
description: BEC detection agent - analyzes email headers and content for Business Email Compromise indicators. Parses email headers for spoofing signals, checks DMARC/SPF/DKIM alignment, detects urgency language patterns, and.... Use when the user asks to detect business email compromise.
version: 1.0.0
tags:
source: detecting-business-email-compromise

Detecting Business Email Compromise

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Python email Library, Microsoft Graph API, FBI IC3 BEC Classification, NIST / Regulatory, Detection Rule Categories

When to use

- The user wants to **detect** business email compromise.
- Capability: BEC detection agent - analyzes email headers and content for Business Email Compromise indicators. Parses email headers for spoofing signals, checks DMARC/SPF/DKIM alignment, detects urgency language patterns, and flags financial request anomalies.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, business, email, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to business email compromise
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Type 1: CEO Fraud / Executive Impersonation
- Type 2: Account Compromise (compromised employee email)
- Type 3: False Invoice / Vendor Email Compromise
- Type 4: Attorney Impersonation
- Type 5: Data Theft (W-2/PII requests)
- T1566.001/002: Phishing (initial access for BEC)
- T1534: Internal Spearphishing
- T1114.003: Email Collection: Email Forwarding Rule

Tools & resources

- Python email Library
- Microsoft Graph API
- FBI IC3 BEC Classification
- NIST / Regulatory
- Detection Rule Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_email_file`
- `check_spf_dkim_dmarc`
- `check_display_name_spoofing`
- `check_reply_to_mismatch`
- `detect_urgency_language`
- `calculate_bec_score`
- `analyze_email`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-business-email-compromise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-business-email-compromise-with-ai (Skill Module)

detecting-business-email-compromise-with-ai\SKILL.md

name: detecting-business-email-compromise-with-ai
description: AI-powered BEC detection agent using NLP features for email classification. Extracts linguistic features (urgency, sentiment, writing style metrics) and uses scikit-learn to classify emails as BEC or legitimate. Use when the user asks to detect business email compromise with ai.
version: 1.0.0
tags:
source: detecting-business-email-compromise-with-ai

Detecting Business Email Compromise with Ai

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: NLP Feature Extraction, scikit-learn Classification Pipeline, Microsoft Graph API, FBI IC3 BEC Statistics, Detection Categories

When to use

- The user wants to **detect** business email compromise with ai.
- Capability: AI-powered BEC detection agent using NLP features for email classification. Extracts linguistic features (urgency, sentiment, writing style metrics) and uses scikit-learn to classify emails as BEC or legitimate.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, business, email, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to business email compromise with ai
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- T1566.001/002: Phishing (Spearphishing Attachment/Link)
- T1534: Internal Spearphishing
- T1656: Impersonation
- T1586.002: Compromise Accounts: Email Accounts
- T1114.003: Email Collection: Email Forwarding Rule
- BEC accounts for 27% of all cybercrime financial losses
- Impostor Detection: AI identifies display name/domain impersonation
- Account Takeover Detection: Behavioral anomalies from compromised accounts

Tools & resources

- NLP Feature Extraction
- scikit-learn Classification Pipeline
- Microsoft Graph API
- FBI IC3 BEC Statistics
- Detection Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `extract_features`
- `compute_bec_probability`
- `analyze_writing_style`
- `detect_impersonation_signals`
- `analyze_email`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-business-email-compromise-with-ai`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-cloud-threats-with-guardduty (Skill Module)

detecting-cloud-threats-with-guardduty\SKILL.md

name: detecting-cloud-threats-with-guardduty
description: Amazon GuardDuty threat detection and response automation. Use when the user asks to detect cloud threats with guardduty.
version: 1.0.0
tags:
source: detecting-cloud-threats-with-guardduty

Detecting Cloud Threats with Guardduty

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: GuardDuty CLI, Finding Management, GuardDuty Finding Severity Levels, Key Finding Type Prefixes, EventBridge Rule for GuardDuty, Threat Intel Set

When to use

- The user wants to **detect** cloud threats with guardduty.
- Capability: Amazon GuardDuty threat detection and response automation agent.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, cloud, threats, guardduty`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute AWS CLI command and return parsed JSON
2. Retrieve the GuardDuty detector ID
3. Enable GuardDuty with all protection plans
4. Get GuardDuty detector configuration and status
5. List active GuardDuty findings filtered by minimum severity
6. Get detailed information about specific findings
7. Archive (suppress) GuardDuty findings
8. Create a custom threat intelligence IP set

Tools & resources

- GuardDuty CLI
- Finding Management
- GuardDuty Finding Severity Levels
- Key Finding Type Prefixes
- EventBridge Rule for GuardDuty
- Threat Intel Set

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `aws_cli` — Execute AWS CLI command and return parsed JSON.
- `get_detector_id` — Retrieve the GuardDuty detector ID.
- `enable_guardduty` — Enable GuardDuty with all protection plans.
- `get_detector_status` — Get GuardDuty detector configuration and status.
- `list_findings` — List active GuardDuty findings filtered by minimum severity.
- `get_finding_details` — Get detailed information about specific findings.
- `archive_finding` — Archive (suppress) GuardDuty findings.
- `create_ip_threat_intel_set` — Create a custom threat intelligence IP set.
- `create_suppression_filter` — Create a suppression filter to auto-archive known benign findings.
- `generate_report` — Generate a GuardDuty findings summary report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-cloud-threats-with-guardduty`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-command-and-control-over-dns (Skill Module)

detecting-command-and-control-over-dns\SKILL.md

name: detecting-command-and-control-over-dns
description: DNS C2 Detection Agent Comprehensive detection pipeline for command-and-control communications over DNS. Combines Shannon entropy analysis, DNS beaconing detection, DGA classification, TXT record payload inspection,.... Use when the user asks to detect command and control over dns.
version: 1.0.0
tags:
source: detecting-command-and-control-over-dns

Detecting Command and Control Over DNS

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Known Tunneling Tool Signatures, DGA Feature Extraction, Beaconing Detection Parameters, Zeek DNS Log Fields, Suricata EVE DNS Fields, Python API

When to use

- The user wants to **detect** command and control over dns.
- Capability: DNS C2 Detection Agent Comprehensive detection pipeline for command-and-control communications over DNS. Combines Shannon entropy analysis, DNS beaconing detection, DGA classification, TXT record payload inspection, and known tool signature matching. Usage: python agent.py --dns-log /path/to/dns.log --format zeek python agent.py --dns-log /path/to/eve.json --format suricata python agent.py --dns-log /path/to/queries.csv --format csv python agent.py --mode train-dga --legit-domains legit.txt --dga-domains dga.txt python agent.py --mode entropy --dns-log dns.log --format zeek Requirements: pip install numpy scikit-learn tldextract
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, command, control, over`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of a string in bits per character
2. Extract subdomain and base domain from FQDN
3. Check if domain matches a known-legitimate pattern
4. Parse various timestamp formats
5. Parse Zeek dns.log (tab-separated format)
6. Parse Suricata EVE JSON log for DNS events
7. Parse CSV DNS log with columns: timestamp, src_ip, query, qtype, answers
8. Load DNS queries from log file

Tools & resources

- Known Tunneling Tool Signatures
- DGA Feature Extraction
- Beaconing Detection Parameters
- Zeek DNS Log Fields
- Suricata EVE DNS Fields
- Python API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of a string in bits per character.
- `extract_subdomain` — Extract subdomain and base domain from FQDN.
- `is_whitelisted` — Check if domain matches a known-legitimate pattern.
- `parse_timestamp` — Parse various timestamp formats.
- `parse_zeek_dns_log` — Parse Zeek dns.log (tab-separated format).
- `parse_suricata_eve` — Parse Suricata EVE JSON log for DNS events.
- `parse_csv_dns` — Parse CSV DNS log with columns: timestamp, src_ip, query, qtype, answers.
- `load_dns_queries` — Load DNS queries from log file.
- `analyze_entropy` — Analyze DNS queries for tunneling indicators via entropy and subdomain length.
- `detect_beaconing` — Detect periodic DNS beaconing patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-command-and-control-over-dns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-compromised-cloud-credentials (Skill Module)

`detecting-compromised-cloud-credentials\SKILL.md`

name: detecting-compromised-cloud-credentials
description: Compromised cloud credential detection agent using AWS CloudTrail and GuardDuty. Use when the user asks to detect compromised cloud credentials.
version: 1.0.0
tags:
source: detecting-compromised-cloud-credentials

Detecting Compromised Cloud Credentials

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: GuardDuty Credential Findings, CloudTrail, IAM Credential Remediation, Azure

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** compromised cloud credentials.
- Capability: Compromised cloud credential detection agent using AWS CloudTrail and GuardDuty.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, compromised, cloud, credentials`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute AWS CLI command and return JSON output
2. Get GuardDuty findings related to credential compromise
3. Query CloudTrail for all API calls made with a specific access key
4. Detect anomalous API patterns from a potentially compromised key
5. Revoke all active sessions for an IAM user by adding inline deny policy
6. Deactivate a compromised access key
7. Generate a credential compromise detection report

Tools & resources

- GuardDuty Credential Findings
- CloudTrail
- IAM Credential Remediation
- Azure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `aws_cli` — Execute AWS CLI command and return JSON output.
- `get_guardduty_credential_findings` — Get GuardDuty findings related to credential compromise.
- `query_cloudtrail_for_key` — Query CloudTrail for all API calls made with a specific access key.
- `detect_anomalous_api_calls` — Detect anomalous API patterns from a potentially compromised key.
- `revoke_iam_sessions` — Revoke all active sessions for an IAM user by adding inline deny policy.
- `deactivate_access_key` — Deactivate a compromised access key.
- `generate_report` — Generate a credential compromise detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-compromised-cloud-credentials`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-container-drift-at-runtime (Skill Module)

detecting-container-drift-at-runtime\SKILL.md

name: detecting-container-drift-at-runtime
description: Container drift detection agent using Docker SDK. Compares running container filesystem against the original image to detect binary drift, file modifications, and package installations. Use when the user asks to detect container drift at runtime.
version: 1.0.0
tags:
source: detecting-container-drift-at-runtime

Detecting Container Drift At Runtime

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Docker SDK for Python, Falco Drift Detection Rules, Kubernetes Security Context, Drift Severity Classification, Industry Standards

When to use

- The user wants to **detect** container drift at runtime.
- Capability: Container drift detection agent using Docker SDK. Compares running container filesystem against the original image to detect binary drift, file modifications, and package installations.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, container, drift, runtime`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Container image deployed with known-good state
2. Runtime monitor (Falco/Sysdig) tracks all process executions and file changes
3. Events compared against baseline: original image manifest + expected runtime behavior
4. Drift events classified by severity (binary drift = HIGH, config drift = MEDIUM)
5. Alerts sent to SIEM/SOC with full container context
6. Automated response: isolate pod network, capture forensics, evict pod
7. **Triage**: Is the drift from a legitimate operation or potential compromise?
8. **Contain**: Apply NetworkPolicy deny-all to affected pod
9. **Collect**: Capture container filesystem diff, process tree, network connections
10. **Analyze**: Compare drifted files against malware signatures and IoCs

Key concepts

- Section 3.3: Containers modified at runtime indicate compromise
- Section 4.2: Monitor containers for unauthorized changes
- Recommends treating containers as immutable infrastructure
- Control 5.2.8: Minimize container readOnlyRootFilesystem
- Control 5.7.3: Apply security contexts to pods
- Control 5.7.4: Default namespace restrictions
- T1610: Deploy Container -- unauthorized container deployment
- T1611: Escape to Host -- container boundary violation

Tools & resources

- Docker SDK for Python
- Falco Drift Detection Rules

- Kubernetes Security Context
- Drift Severity Classification
- Industry Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_container_diff`
- `get_running_processes`
- `detect_drift_indicators`
- `check_image_digest`
- `audit_container`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-container-drift-at-runtime`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-container-escape-attempts (Skill Module)

detecting-container-escape-attempts\SKILL.md

name: detecting-container-escape-attempts
description: Container escape detection agent using Falco output parsing and audit log analysis. Monitors for container escape indicators by parsing Falco JSON alerts, auditd logs, and Docker inspect data for.... Use when the user asks to detect container escape attempts.
version: 1.0.0
tags:
source: detecting-container-escape-attempts

Detecting Container Escape Attempts

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Docker CLI Inspection, Dangerous Linux Capabilities, Known Container Escape CVEs, NIST SP 800-190

When to use

- The user wants to **detect** container escape attempts.
- Capability: Container escape detection agent using Falco output parsing and audit log analysis. Monitors for container escape indicators by parsing Falco JSON alerts, auditd logs, and Docker inspect data for privileged/vulnerable containers.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, container, escape, attempts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to container escape attempts
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Tactic: Privilege Escalation
- Description: Adversaries may escape container isolation and gain access to the host
- Sub-techniques: Privileged container, nsenter, cgroup escape, kernel exploit
- Detection: Monitor for namespace manipulation, sensitive path access, privilege changes
- Description: Deploy a new container using Docker socket access from within a container
- Description: Exploit kernel vulnerabilities for container escape (Dirty Pipe, runc CVEs)
- Sub-technique: T1548.004 - Elevated Execution with Prompt
- Description: Abuse Linux capabilities like CAP_SYS_ADMIN for escape

Tools & resources

- Docker CLI Inspection
- Dangerous Linux Capabilities
- Known Container Escape CVEs
- NIST SP 800-190

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_falco_json`
- `parse_auditd_escape_events`
- `check_privileged_containers`
- `check_dangerous_capabilities`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-container-escape-attempts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-container-escape-with-falco-rules (Skill Module)

detecting-container-escape-with-falco-rules\SKILL.md

name: detecting-container-escape-with-falco-rules
description: Falco-based container escape detection agent. Manages Falco rules, parses Falco alert output, and generates escape detection reports from Falco JSON event streams. Use when the user asks to detect container escape with falco rules.
version: 1.0.0
tags:
source: detecting-container-escape-with-falco-rules

Detecting Container Escape with Falco Rules

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Falco, Helm Deployment, Industry Standards, Known Container Escape CVEs, Compliance Mappings

When to use

- The user wants to **detect** container escape with falco rules.
- Capability: Falco-based container escape detection agent. Manages Falco rules, parses Falco alert output, and generates escape detection reports from Falco JSON event streams.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, container, escape, falco, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Update Falco rules weekly: `falcoctl artifact install falco-rules`
2. Review new maturity_stable rules after each Falco release
3. Correlate Falco alerts with Kubernetes audit logs
4. Run escape simulation exercises monthly

Key concepts

- Section 4.3: Container Runtime - Monitor containers for anomalous behavior at runtime
- Section 5.4: Container Runtime Security - Implement runtime monitoring and alerting
- Recommends syscall-level monitoring for escape detection
- Section 5: Audit Logging and Threat Detection
- Enable runtime security monitoring
- Detect anomalous container behavior in real-time
- Monitor for privilege escalation attempts
- Requirement 10.6.1: Review logs for anomalies at least daily

Tools & resources

- Falco
- Helm Deployment
- Industry Standards
- Known Container Escape CVEs
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_falco_status`
- `validate_rules_file`
- `parse_falco_alerts`
- `generate_escape_rules_yaml`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-container-escape-with-falco-rules`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-credential-dumping-techniques (Skill Module)

detecting-credential-dumping-techniques\SKILL.md

name: detecting-credential-dumping-techniques
description: Detect credential dumping techniques via Sysmon/Windows event log analysis. Use when the user asks to detect credential dumping techniques.
version: 1.0.0
tags:
source: detecting-credential-dumping-techniques

Detecting Credential Dumping Techniques

Role: Detection engineering
Audience: detection engineers and SOC analysts

When to use

- The user wants to **detect** credential dumping techniques.
- Capability: Detect credential dumping techniques via Sysmon/Windows event log analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, credential, dumping, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Sysmon event log XML export for Event ID 10 (ProcessAccess)
2. Detect suspicious ProcessAccess (Event ID 10) targeting lsass.exe
3. Detect SAM/NTDS.dit export and comsvcs.dll dump commands from Event ID 1 or 4688
4. Detect suspicious ntdll.dll access patterns in CallTrace (Mimikatz signature)
5. Return SPL detection queries for credential dumping

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_sysmon_xml` — Parse Sysmon event log XML export for Event ID 10 (ProcessAccess).
- `detect_lsass_access` — Detect suspicious ProcessAccess (Event ID 10) targeting lsass.exe.
- `detect_credential_commands` — Detect SAM/NTDS.dit export and comsvcs.dll dump commands from Event ID 1 or 4688.

- `detect_ntdll_access` — Detect suspicious ntdll.dll access patterns in CallTrace (Mimikatz signature).
- `generate_splunk_queries` — Return SPL detection queries for credential dumping.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-credential-dumping-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-cryptomining-in-cloud (Skill Module)

detecting-cryptomining-in-cloud\SKILL.md

name: detecting-cryptomining-in-cloud
description: Cloud cryptomining detection agent with multi-signal analysis. Use when the user asks to detect cryptomining in cloud.
version: 1.0.0
tags:
source: detecting-cryptomining-in-cloud

Detecting Cryptomining in Cloud

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Detection Signal Categories, GuardDuty Crypto Findings, CloudWatch CPU Alarm, AWS Cost Anomaly Detection, Known Mining Pool Domains, Instance Remediation

When to use

- The user wants to **detect** cryptomining in cloud.
- Capability: Cloud cryptomining detection agent with multi-signal analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, cryptomining, cloud`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute AWS CLI command and return parsed JSON
2. Retrieve GuardDuty cryptocurrency-related findings
3. Find EC2 instances with sustained high CPU utilization
4. Create AWS Cost Anomaly Detection monitor for EC2 spikes
5. Check CloudTrail for unusual instance launch patterns
6. Query VPC Flow Logs for traffic to mining pool ports
7. Isolate a mining instance by modifying its security group
8. Generate comprehensive cryptomining detection report

Tools & resources

- Detection Signal Categories
- GuardDuty Crypto Findings
- CloudWatch CPU Alarm
- AWS Cost Anomaly Detection
- Known Mining Pool Domains
- Instance Remediation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `aws_cli` — Execute AWS CLI command and return parsed JSON.
- `get_guardduty_crypto_findings` — Retrieve GuardDuty cryptocurrency-related findings.
- `check_high_cpu_instances` — Find EC2 instances with sustained high CPU utilization.
- `create_cost_anomaly_monitor` — Create AWS Cost Anomaly Detection monitor for EC2 spikes.
- `check_cloudtrail_instance_launches` — Check CloudTrail for unusual instance launch patterns.
- `query_vpc_flow_logs_mining` — Query VPC Flow Logs for traffic to mining pool ports.
- `isolate_mining_instance` — Isolate a mining instance by modifying its security group.
- `generate_report` — Generate comprehensive cryptomining detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-cryptomining-in-cloud`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-dcsync-attack-in-active-directory (Skill Module)

`detecting-dcsync-attack-in-active-directory\SKILL.md`

name: detecting-dcsync-attack-in-active-directory
description: DCSync attack detection agent for Active Directory environments. Parses Windows Security Event ID 4662 logs to detect non-domain-controller accounts requesting directory replication (DCSync technique T1003.006). Use when the user asks to detect dcsync attack in active directory.
version: 1.0.0
tags:
source: detecting-dcsync-attack-in-active-directory

Detecting Dcsync Attack in Active Directory

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: DCSync Replication GUIDs, python-evtx Usage, PowerShell, Attack Tools Reference, Critical Replication GUIDs, Legitimate Replication Sources

When to use

- The user wants to **detect** dcsync attack in active directory.
- Capability: DCSync attack detection agent for Active Directory environments. Parses Windows Security Event ID 4662 logs to detect non-domain-controller accounts requesting directory replication (DCSync technique T1003.006).
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, dcsync, attack, active, directory`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Disable compromised account immediately
2. Rotate KRBTGT password (twice, 12 hours apart)
3. Reset all service account passwords
4. Block source IP at network level
5. Isolate source machine for forensics
6. Remove unauthorized replication rights
7. Review all accounts with DCSync-capable permissions
8. Implement tiered administration model
9. Enable Microsoft Defender for Identity DCSync alerts
10. Deploy Protected Users security group for admin accounts

Tools & resources

- DCSync Replication GUIDs
- python-evtx Usage
- PowerShell
- Attack Tools Reference
- Critical Replication GUIDs
- Legitimate Replication Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_dc_accounts`
- `parse_4662_events`
- `check_replication_permissions_powershell`
- `generate_sigma_rule`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-dcsync-attack-in-active-directory`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-deepfake-audio-in-vishing-attacks (Skill Module)

detecting-deepfake-audio-in-vishing-attacks\SKILL.md

name: detecting-deepfake-audio-in-vishing-attacks
description: Deepfake audio detection agent using spectral analysis, MFCC features, and ML classifiers. Analyzes audio files to determine whether they contain AI-generated (deepfake) speech, commonly used in vishing (voice.... Use when the user asks to detect deepfake audio in vishing attacks.
version: 1.0.0
tags:
source: detecting-deepfake-audio-in-vishing-attacks

Detecting Deepfake Audio in Vishing Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: librosa, scikit-learn, Datasets for Training, CLI Usage Examples

When to use

- The user wants to **detect** deepfake audio in vishing attacks.
- Capability: Deepfake audio detection agent using spectral analysis, MFCC features, and ML classifiers. Analyzes audio files to determine whether they contain AI-generated (deepfake) speech, commonly used in vishing (voice phishing) attacks. Extracts spectral features with librosa, builds feature vectors, and classifies using ensemble ML models.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, deepfake, audio, vishing, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load audio file, resample to target rate, trim silence, and normalize
2. Extract MFCC, delta, and delta-delta features with statistical aggregation
3. Extract spectral centroid, bandwidth, contrast, rolloff, and ZCR
4. Extract fundamental frequency (F0), jitter, and shimmer-like features
5. Extract time-domain features: RMS energy, tempo, onset strength
6. Load audio and extract the complete feature set as a dict and numpy vector
7. Rule-based deepfake scoring using research-backed thresholds. Returns a score between 0.0 (likely genuine) and 1.0 (likely deepfake) based on known acoustic differences between real and synthetic speech
8. Classify audio using pre-trained ensemble models if available. Falls back to heuristic scoring if no trained model is found. Returns dict with model predictions and confidence

Tools & resources

- librosa
- scikit-learn
- Datasets for Training
- CLI Usage Examples

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_and_preprocess` — Load audio file, resample to target rate, trim silence, and normalize.
- `extract_mfcc_features` — Extract MFCC, delta, and delta-delta features with statistical aggregation.
- `extract_spectral_features` — Extract spectral centroid, bandwidth, contrast, rolloff, and ZCR.
- `extract_pitch_features` — Extract fundamental frequency (F0), jitter, and shimmer-like features.
- `extract_temporal_features` — Extract time-domain features: RMS energy, tempo, onset strength.
- `build_full_feature_vector` — Load audio and extract the complete feature set as a dict and numpy vector.
- `heuristic_deepfake_score` — Rule-based deepfake scoring using research-backed thresholds. Returns a score between 0.0 (likely genuine) and 1.0 (likely deepfake) based on known acoustic differences between real and synthetic speech.
- `classify_with_ensemble` — Classify audio using pre-trained ensemble models if available. Falls back to heuristic scoring if no trained model is found. Returns dict with model predictions and confidence.
- `train_model` — Train ensemble classifier on directories of genuine and deepfake audio samples. Expects two directories containing WAV/MP3/FLAC files: - genuine_dir: directory of known real speech samples - deepfake_dir: directory of known AI-generated speech samples Saves trained model (scaler + RF + GBT) to output_path via joblib.
- `analyze_audio` — Full analysis pipeline: load, extract features, classify, and report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-deepfake-audio-in-vishing-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-dll-sideloading-attacks (Skill Module)

detecting-dll-sideloading-attacks\SKILL.md

name: detecting-dll-sideloading-attacks
description: DLL sideloading detection agent using Sysmon Event ID 7 (Image Loaded) analysis. Detects unsigned DLLs loaded by signed executables from non-standard paths, a common APT persistence and defense evasion technique.... Use when the user asks to detect dll sideloading attacks.
version: 1.0.0
tags:
source: detecting-dll-sideloading-attacks

Detecting DLL Sideloading Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: python-evtx Usage, Known Sideloading Targets, Splunk SPL Detection, Sigma Rule Fields, Windows DLL Search Order, Known Vulnerable Applications

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** dll sideloading attacks.
- Capability: DLL sideloading detection agent using Sysmon Event ID 7 (Image Loaded) analysis. Detects unsigned DLLs loaded by signed executables from non-standard paths, a common APT persistence and defense evasion technique (T1574.002).
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, sideloading, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to dll sideloading attacks
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Tactic: Persistence (TA0003), Privilege Escalation (TA0004), Defense Evasion (TA0005)
- Data Sources: File monitoring, DLL monitoring, Process monitoring

Tools & resources

- python-evtx Usage
- Known Sideloading Targets
- Splunk SPL Detection
- Sigma Rule Fields
- Windows DLL Search Order
- Known Vulnerable Applications
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_sysmon_dll_loads`
- `scan_directory_for_sideloading`
- `generate_sigma_rule`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-dll-sideloading-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-dnp3-protocol-anomalies (Skill Module)

detecting-dnp3-protocol-anomalies\SKILL.md

name: detecting-dnp3-protocol-anomalies
description: DNP3 protocol anomaly detection agent for ICS/SCADA environments. Analyzes DNP3 network traffic captures (via Zeek or pcap) to detect unauthorized control commands, protocol violations, and traffic anomalies. Use when the user asks to detect dnp3 protocol anomalies.
version: 1.0.0
tags:
source: detecting-dnp3-protocol-anomalies

Detecting Dnp3 Protocol Anomalies

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: DNP3 Function Codes, Zeek DNP3 Log Fields, Zeek DNP3 Protocol Analyzer, Suricata DNP3 Rules, Scapy DNP3 Parsing, ICS-CERT Detection Indicators

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** dnp3 protocol anomalies.
- Capability: DNP3 protocol anomaly detection agent for ICS/SCADA environments. Analyzes DNP3 network traffic captures (via Zeek or pcap) to detect unauthorized control commands, protocol violations, and traffic anomalies.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, dnp3, protocol, anomalies`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to dnp3 protocol anomalies
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- DNP3 Function Codes
- Zeek DNP3 Log Fields
- Zeek DNP3 Protocol Analyzer
- Suricata DNP3 Rules
- Scapy DNP3 Parsing
- ICS-CERT Detection Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_authorized_hosts`
- `parse_zeek_dnp3_log`
- `analyze_dnp3_traffic`
- `detect_protocol_anomalies`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-dnp3-protocol-anomalies`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-dns-exfiltration-with-dns-query-analysis (Skill Module)

detecting-dns-exfiltration-with-dns-query-analysis\SKILL.md

name: detecting-dns-exfiltration-with-dns-query-analysis
description: DNS exfiltration detection agent using entropy analysis and query pattern anomalies. Analyzes DNS query logs for tunneling indicators: high entropy subdomains, excessive query length, abnormal TXT record usage, and.... Use when the user asks to detect dns exfiltration with dns query analysis.
version: 1.0.0
tags:
source: detecting-dns-exfiltration-with-dns-query-analysis

Detecting DNS Exfiltration with DNS Query Analysis

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Detection Thresholds, Zeek dns.log Fields, DNS Tunneling Tools, Splunk SPL Detection, Suricata DNS Rules

When to use

- The user wants to **detect** dns exfiltration with dns query analysis.
- Capability: DNS exfiltration detection agent using entropy analysis and query pattern anomalies. Analyzes DNS query logs for tunneling indicators: high entropy subdomains, excessive query length, abnormal TXT record usage, and volume spikes.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, exfiltration, query, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to dns exfiltration with dns query analysis
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Detection Thresholds
- Zeek dns.log Fields
- DNS Tunneling Tools
- Splunk SPL Detection
- Suricata DNS Rules

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `calculate_entropy`
- `parse_dns_log`
- `analyze_queries`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-dns-exfiltration-with-dns-query-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-email-account-compromise (Skill Module)

detecting-email-account-compromise\SKILL.md

name: detecting-email-account-compromise
description: Email Account Compromise Detection agent - analyzes inbox rules, sign-in logs, and OAuth grants to detect O365/Google Workspace account compromise. Use when the user asks to detect email account compromise.
version: 1.0.0
tags:
source: detecting-email-account-compromise

Detecting Email Account Compromise

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Inbox Rule Compromise Indicators, Sign-In Risk Indicators, Required API Permissions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** email account compromise.
- Capability: Email Account Compromise Detection agent - analyzes inbox rules, sign-in logs, and OAuth grants to detect O365/Google Workspace account compromise
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, email, account, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Detect malicious inbox rules: external forwarding, deletion rules, keyword-based filters
2. Detect impossible travel, suspicious user agents, and risky sign-in patterns
3. Detect suspicious OAuth application consent grants

Tools & resources

- Inbox Rule Compromise Indicators
- Sign-In Risk Indicators
- Required API Permissions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_data`
- `haversine_km`
- `analyze_inbox_rules` — Detect malicious inbox rules: external forwarding, deletion rules, keyword-based filters.
- `analyze_sign_ins` — Detect impossible travel, suspicious user agents, and risky sign-in patterns.
- `analyze_oauth_grants` — Detect suspicious OAuth application consent grants.
- `analyze`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-email-account-compromise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-email-forwarding-rules-attack (Skill Module)

detecting-email-forwarding-rules-attack\SKILL.md

name: detecting-email-forwarding-rules-attack
description: Email forwarding rules attack detection agent. Detects malicious inbox rules created by adversaries for persistent email access (T1114.003) by querying Microsoft Graph API and analyzing audit logs for suspicious rule.... Use when the user asks to detect email forwarding rules attack.
version: 1.0.0
tags:
source: detecting-email-forwarding-rules-attack

Detecting Email Forwarding Rules Attack

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Microsoft Graph API, Exchange Online PowerShell, Suspicious Rule Indicators, Splunk SPL Detection, Detection Data Sources

When to use

- The user wants to **detect** email forwarding rules attack.
- Capability: Email forwarding rules attack detection agent. Detects malicious inbox rules created by adversaries for persistent email access (T1114.003) by querying Microsoft Graph API and analyzing audit logs for suspicious rule creation patterns.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, email, forwarding, rules, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to email forwarding rules attack
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Microsoft Graph API
- Exchange Online PowerShell
- Suspicious Rule Indicators
- Splunk SPL Detection
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_mailbox_rules`
- `analyze_rules`
- `parse_audit_log_for_rules`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-email-forwarding-rules-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-evasion-techniques-in-endpoint-logs (Skill Module)

`detecting-evasion-techniques-in-endpoint-logs\SKILL.md`

name: detecting-evasion-techniques-in-endpoint-logs
description: Defense evasion detection agent for endpoint logs. Detects MITRE ATT&CK TA0005 evasion techniques including log clearing, timestomping, process injection indicators, and security tool disabling by analyzing Sysmon.... Use when the user asks to detect evasion techniques in endpoint logs.
version: 1.0.0
tags:
source: detecting-evasion-techniques-in-endpoint-logs

Detecting Evasion Techniques in Endpoint Logs

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: python-evtx Usage, Splunk SPL Detection, Primary Standards, Compliance Mappings, Supporting References

When to use

- The user wants to **detect** evasion techniques in endpoint logs.
- Capability: Defense evasion detection agent for endpoint logs. Detects MITRE ATT&CK TA0005 evasion techniques including log clearing, timestomping, process injection indicators, and security tool disabling by analyzing Sysmon and Windows Security event logs.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, evasion, techniques, endpoint, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to evasion techniques in endpoint logs
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Scope: 43 techniques and 80+ sub-techniques for defense evasion
- Key techniques: T1055 (Process Injection), T1070 (Indicator Removal), T1036 (Masquerading), T1218 (System Binary Proxy Executi
- Scope: Community-maintained detection rules in vendor-agnostic format
- Evasion rules: rules/windows/process_creation/proc_creation_win_*, rules/windows/sysmon/
- Scope: Catalog of Windows binaries that can be used for defense evasion, with detection recommendations
- SANS Hunt Evil Poster: Comprehensive guide to suspicious Windows process behavior

Tools & resources

- python-evtx Usage
- Splunk SPL Detection
- Primary Standards
- Compliance Mappings
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_evtx_for_evasion`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-evasion-techniques-in-endpoint-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-exfiltration-over-dns-with-zeek (Skill Module)

detecting-exfiltration-over-dns-with-zeek\SKILL.md

name: detecting-exfiltration-over-dns-with-zeek
description: Detect DNS exfiltration from Zeek dns.log by analyzing entropy and query patterns. Use when the user asks to detect exfiltration over dns with zeek.
version: 1.0.0
tags:
source: detecting-exfiltration-over-dns-with-zeek

Detecting Exfiltration Over DNS with ZEEK

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Zeek, RITA Beacon Detection, Suricata DNS Exfiltration Rules

When to use

- The user wants to **detect** exfiltration over dns with zeek.
- Capability: Detect DNS exfiltration from Zeek dns.log by analyzing entropy and query patterns.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, exfiltration, over, zeek`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to exfiltration over dns with zeek
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Zeek
- RITA Beacon Detection
- Suricata DNS Exfiltration Rules

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shannon_entropy`
- `parse_zeek_dns_log`
- `extract_parent_domain`
- `extract_subdomain`
- `analyze_dns_log`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-exfiltration-over-dns-with-zeek`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-fileless-attacks-on-endpoints (Skill Module)

detecting-fileless-attacks-on-endpoints\SKILL.md

name: detecting-fileless-attacks-on-endpoints
description: Fileless attack detection agent for endpoint logs. Detects in-memory attacks by analyzing PowerShell script block logs (Event 4104), WMI persistence events, and reflective DLL injection indicators from Sysmon. Use when the user asks to detect fileless attacks on endpoints.
version: 1.0.0
tags:
source: detecting-fileless-attacks-on-endpoints

Detecting Fileless Attacks on Endpoints

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: python-evtx Usage, Splunk SPL, WMI Persistence Detection

When to use

- The user wants to **detect** fileless attacks on endpoints.
- Capability: Fileless attack detection agent for endpoint logs. Detects in-memory attacks by analyzing PowerShell script block logs (Event 4104), WMI persistence events, and reflective DLL injection indicators from Sysmon.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, fileless, attacks, endpoints`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to fileless attacks on endpoints
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- MITRE ATT&CK T1059.001: PowerShell execution
- MITRE ATT&CK T1055: Process Injection (all sub-techniques)
- MITRE ATT&CK T1546.003: WMI Event Subscription persistence
- MITRE ATT&CK T1620: Reflective Code Loading

Tools & resources

- python-evtx Usage
- Splunk SPL
- WMI Persistence Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_powershell_scriptblock`
- `parse_sysmon_wmi_persistence`
- `parse_sysmon_injection`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-fileless-attacks-on-endpoints`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-fileless-malware-techniques (Skill Module)

detecting-fileless-malware-techniques\SKILL.md

name: detecting-fileless-malware-techniques
description: Fileless malware detection agent using Windows event logs and Volatility. Use when the user asks to detect fileless malware techniques.
version: 1.0.0
tags:
source: detecting-fileless-malware-techniques

Detecting Fileless Malware Techniques

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: python-evtx, Suspicious PowerShell Indicators, WMI Persistence Check

When to use

- The user wants to **detect** fileless malware techniques.
- Capability: Fileless malware detection agent using Windows event logs and Volatility.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, fileless, malware, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan PowerShell script block logs for suspicious patterns
2. Scan Sysmon logs for LOLBin process creation events
3. Detect WMI event subscription persistence mechanisms
4. Check registry Run keys for suspicious persistence entries
5. Run Volatility malfind to detect injected code in memory
6. Generate fileless malware detection report

Tools & resources

- python-evtx
- Suspicious PowerShell Indicators
- WMI Persistence Check

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_powershell_logs` — Scan PowerShell script block logs for suspicious patterns.
- `scan_sysmon_for_lolbins` — Scan Sysmon logs for LOLBin process creation events.
- `scan_wmi_persistence` — Detect WMI event subscription persistence mechanisms.
- `scan_registry_run_keys` — Check registry Run keys for suspicious persistence entries.
- `run_volatility_malfind` — Run Volatility malfind to detect injected code in memory.
- `generate_report` — Generate fileless malware detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-fileless-malware-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-golden-ticket-attacks-in-kerberos-logs (Skill Module)

`detecting-golden-ticket-attacks-in-kerberos-logs\SKILL.md`

name: detecting-golden-ticket-attacks-in-kerberos-logs
description: Golden Ticket attack detection agent for Kerberos log analysis. Parses Windows Security Event IDs 4768, 4769, 4771 to detect forged TGTs with anomalous encryption types, impossible lifetimes, and non-existent accounts. Use when the user asks to detect golden ticket attacks in kerberos logs.
version: 1.0.0
tags:
source: detecting-golden-ticket-attacks-in-kerberos-logs

Detecting Golden Ticket Attacks in Kerberos Logs

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Kerberos Encryption Types, python-evtx Usage, Splunk SPL Detection, Mimikatz Golden Ticket Command

When to use

- The user wants to **detect** golden ticket attacks in kerberos logs.
- Capability: Golden Ticket attack detection agent for Kerberos log analysis. Parses Windows Security Event IDs 4768, 4769, 4771 to detect forged TGTs with anomalous encryption types, impossible lifetimes, and non-existent accounts.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, golden, ticket, attacks, kerberos, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to golden ticket attacks in kerberos logs
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Kerberos Encryption Types
- python-evtx Usage
- Splunk SPL Detection
- Mimikatz Golden Ticket Command

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_kerberos_events`
- `generate_sigma_rule`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-golden-ticket-attacks-in-kerberos-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-golden-ticket-forgery (Skill Module)

`detecting-golden-ticket-forgery\SKILL.md`

name: detecting-golden-ticket-forgery
description: Detect Kerberos Golden Ticket forgery via Windows Security event log analysis. Use when the user asks to detect golden ticket forgery.
version: 1.0.0
tags:
source: detecting-golden-ticket-forgery

Detecting Golden Ticket Forgery

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Detection Indicators, Elastic KQL

When to use

- The user wants to **detect** golden ticket forgery.
- Capability: Detect Kerberos Golden Ticket forgery via Windows Security event log analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, golden, ticket, forgery`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse exported Windows Security event log XML for Kerberos events 4768/4769
2. Detect RC4 encryption (0x17) in TGS requests where AES should be enforced
3. Detect TGS requests (4769) without preceding TGT request (4768) from same user
4. Detect tickets with lifetime exceeding domain policy (default MaxTicketAge=10h)
5. Detect TGS requests targeting the krbtgt service (unusual and suspicious)
6. Return Splunk SPL queries for Golden Ticket detection

Tools & resources

- Detection Indicators
- Elastic KQL

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_security_events` — Parse exported Windows Security event log XML for Kerberos events 4768/4769.
- `detect_rc4_in_aes_environment` — Detect RC4 encryption (0x17) in TGS requests where AES should be enforced.
- `detect_orphaned_tgs` — Detect TGS requests (4769) without preceding TGT request (4768) from same user.
- `detect_abnormal_ticket_lifetime` — Detect tickets with lifetime exceeding domain policy (default MaxTicketAge=10h).
- `detect_krbtgt_service_anomaly` — Detect TGS requests targeting the krbtgt service (unusual and suspicious).
- `generate_splunk_queries` — Return Splunk SPL queries for Golden Ticket detection.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-golden-ticket-forgery`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-insider-data-exfiltration-via-dlp (Skill Module)

`detecting-insider-data-exfiltration-via-dlp\SKILL.md`

name: detecting-insider-data-exfiltration-via-dlp
description: Agent for detecting insider data exfiltration via DLP analysis. Use when the user asks to detect insider data exfiltration via dlp.
version: 1.0.0
tags:
source: detecting-insider-data-exfiltration-via-dlp

Detecting Insider Data Exfiltration via Dlp

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Pandas Behavioral Analytics, Exfiltration Indicators, Microsoft Purview DLP API

When to use

- The user wants to **detect** insider data exfiltration via dlp.
- Capability: Agent for detecting insider data exfiltration via DLP analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, insider, data, exfiltration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load file/cloud activity logs
2. Detect users with data transfer volume exceeding baseline
3. Detect file access during off-hours
4. Detect bulk file downloads in short time windows
5. Detect access to sensitive file types or directories
6. Detect USB device usage for data transfer

Tools & resources

- Pandas Behavioral Analytics
- Exfiltration Indicators
- Microsoft Purview DLP API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_activity_logs` — Load file/cloud activity logs.
- `detect_volume_anomalies` — Detect users with data transfer volume exceeding baseline.
- `detect_off_hours_activity` — Detect file access during off-hours.
- `detect_bulk_downloads` — Detect bulk file downloads in short time windows.
- `detect_sensitive_file_access` — Detect access to sensitive file types or directories.
- `detect_usb_activity` — Detect USB device usage for data transfer.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-insider-data-exfiltration-via-dlp`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-insider-threat-behaviors (Skill Module)

detecting-insider-threat-behaviors\SKILL.md

name: detecting-insider-threat-behaviors
description: Insider threat behavior detection agent using UEBA indicators. Analyzes user activity logs to detect anomalous behaviors: off-hours access, mass file downloads, unusual data access patterns, and privilege abuse. Use when the user asks to detect insider threat behaviors.
version: 1.0.0
tags:
source: detecting-insider-threat-behaviors

Detecting Insider Threat Behaviors

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Risk Indicator Weights, UEBA Data Sources, Splunk SPL, Microsoft Sentinel, Personal Cloud Domains, Risk Score Calculation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** insider threat behaviors.
- Capability: Insider threat behavior detection agent using UEBA indicators. Analyzes user activity logs to detect anomalous behaviors: off-hours access, mass file downloads, unusual data access patterns, and privilege abuse.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, insider, threat, behaviors`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to insider threat behaviors
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Risk Indicator Weights
- UEBA Data Sources
- Splunk SPL
- Microsoft Sentinel
- Personal Cloud Domains
- Risk Score Calculation
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_activity_log`
- `detect_off_hours`
- `detect_mass_download`
- `detect_data_exfil_destinations`
- `calculate_risk_score`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-insider-threat-behaviors`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-insider-threat-with-ueba (Skill Module)

detecting-insider-threat-with-ueba\SKILL.md

name: detecting-insider-threat-with-ueba
description: UEBA Insider Threat Agent - builds behavioral baselines and scores anomalies using Elasticsearch. Use when the user asks to detect insider threat with ueba.
version: 1.0.0
tags:
source: detecting-insider-threat-with-ueba

Detecting Insider Threat with Ueba

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Insider Threat Indicators, Elasticsearch Python Client

When to use

- The user wants to **detect** insider threat with ueba.
- Capability: UEBA Insider Threat Agent - builds behavioral baselines and scores anomalies using Elasticsearch.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, insider, threat, ueba`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to Elasticsearch cluster
2. Build 30-day behavioral baseline per user using ES aggregations
3. Score current activity against baselines to find anomalies
4. Compare user activity against peer group averages

Tools & resources

- Insider Threat Indicators
- Elasticsearch Python Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_es` — Connect to Elasticsearch cluster.
- `build_user_baseline` — Build 30-day behavioral baseline per user using ES aggregations.
- `score_current_activity` — Score current activity against baselines to find anomalies.
- `peer_group_analysis` — Compare user activity against peer group averages.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-insider-threat-with-ueba`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-kerberoasting-attacks (Skill Module)

detecting-kerberoasting-attacks\SKILL.md

name: detecting-kerberoasting-attacks
description: Kerberoasting Detection Agent - Detects Kerberoasting via Event 4769 TGS-REQ analysis. Use when the user asks to detect kerberoasting attacks.
version: 1.0.0
tags:
source: detecting-kerberoasting-attacks

Detecting Kerberoasting Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: python-evtx Library, Encryption Type Values, Detection Logic, Detection Data Sources

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** kerberoasting attacks.
- Capability: Kerberoasting Detection Agent - Detects Kerberoasting via Event 4769 TGS-REQ analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, kerberoasting, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse Event ID 4769 (Kerberos TGS requests) from Security EVTX
2. Detect TGS requests using weak RC4-HMAC encryption (Kerberoasting indicator)
3. Detect high-volume TGS requests from a single source (spray pattern)
4. Detect TGS requests for unusual or sensitive SPNs
5. Correlate suspicious TGS sources with logon events (4624) for attribution
6. Generate Kerberoasting detection report

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- python-evtx Library
- Encryption Type Values
- Detection Logic
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_tgs_events` — Parse Event ID 4769 (Kerberos TGS requests) from Security EVTX.
- `detect_rc4_tgs_requests` — Detect TGS requests using weak RC4-HMAC encryption (Kerberoasting indicator).
- `detect_high_volume_tgs` — Detect high-volume TGS requests from a single source (spray pattern).
- `detect_anomalous_spn_requests` — Detect TGS requests for unusual or sensitive SPNs.
- `correlate_with_logon_events` — Correlate suspicious TGS sources with logon events (4624) for attribution.
- `generate_report` — Generate Kerberoasting detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-kerberoasting-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-lateral-movement-in-network (Skill Module)

detecting-lateral-movement-in-network\SKILL.md

name: detecting-lateral-movement-in-network
description: Lateral movement detection agent using Zeek logs and Windows event analysis. Use when the user asks to detect lateral movement in network.
version: 1.0.0
tags:
source: detecting-lateral-movement-in-network

Detecting Lateral Movement in Network

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Zeek Script, python-evtx

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** lateral movement in network.
- Capability: Lateral movement detection agent using Zeek logs and Windows event analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, lateral, movement, network`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Zeek conn.log for internal lateral movement patterns
2. Parse Zeek smb_mapping.log for file share access patterns
3. Analyze Windows Security EVTX for lateral movement indicators
4. Detect pass-the-hash indicators from auth events
5. Generate comprehensive lateral movement detection report

Tools & resources

- Zeek Script
- python-evtx

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_zeek_conn_log` — Parse Zeek conn.log for internal lateral movement patterns.
- `parse_zeek_smb_log` — Parse Zeek smb_mapping.log for file share access patterns.
- `analyze_windows_auth_logs` — Analyze Windows Security EVTX for lateral movement indicators.
- `detect_pass_the_hash_pattern` — Detect pass-the-hash indicators from auth events.
- `generate_report` — Generate comprehensive lateral movement detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-lateral-movement-in-network`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-lateral-movement-with-splunk (Skill Module)

`detecting-lateral-movement-with-splunk\SKILL.md`

name: detecting-lateral-movement-with-splunk
description: Lateral movement detection agent using Splunk SPL query generation. Generates and analyzes SPL queries for detecting lateral movement techniques including pass-the-hash, RDP pivoting, WMI/PSExec execution, and SMB abuse. Use when the user asks to detect lateral movement with splunk.
version: 1.0.0
tags:
source: detecting-lateral-movement-with-splunk

Detecting Lateral Movement with Splunk

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Splunk, Key Lateral Movement Techniques, Windows Logon Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** lateral movement with splunk.
- Capability: Lateral movement detection agent using Splunk SPL query generation. Generates and analyzes SPL queries for detecting lateral movement techniques including pass-the-hash, RDP pivoting, WMI/PSExec execution, and SMB abuse.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, lateral, movement, splunk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to lateral movement with splunk
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Splunk
- Key Lateral Movement Techniques
- Windows Logon Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_queries`
- `parse_splunk_results`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-lateral-movement-with-splunk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-lateral-movement-with-zeek (Skill Module)

detecting-lateral-movement-with-zeek\SKILL.md

name: detecting-lateral-movement-with-zeek
description: Zeek lateral movement detection agent. Parses Zeek conn.log, smb_mapping.log, smb_files.log, dce_rpc.log, ntlm.log, and kerberos.log to detect lateral movement indicators: admin share access, PsExec-style service.... Use when the user asks to detect lateral movement with zeek.
version: 1.0.0
tags:
source: detecting-lateral-movement-with-zeek

Detecting Lateral Movement with ZEEK

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Zeek, Lateral Movement Ports Tracked, Detection Types in Output, Detection References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** lateral movement with zeek.
- Capability: Zeek lateral movement detection agent. Parses Zeek conn.log, smb_mapping.log, smb_files.log, dce_rpc.log, ntlm.log, and kerberos.log to detect lateral movement indicators: admin share access, PsExec-style service creation, Pass-the-Hash, and anomalous internal host-to-host connections.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, lateral, movement, zeek`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check if an IP address is in RFC1918 private ranges
2. Parse a Zeek TSV log file, skipping comment/header lines
3. Parse a Zeek JSON log file (one JSON object per line)
4. Auto-detect Zeek log format (TSV or JSON) and parse accordingly
5. Convert Zeek epoch timestamp string to ISO 8601
6. Analyze conn.log for internal lateral movement connections
7. Analyze smb_mapping.log for admin share access
8. Analyze smb_files.log for file writes to network shares

Key concepts

- SANS: Detecting Lateral Movement with Zeek
- Red Canary Threat Detection Report — Lateral Movement chapter

Tools & resources

- Zeek
- Lateral Movement Ports Tracked
- Detection Types in Output
- Detection References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `is_internal` — Check if an IP address is in RFC1918 private ranges.
- `parse_zeek_tsv` — Parse a Zeek TSV log file, skipping comment/header lines.
- `parse_zeek_json` — Parse a Zeek JSON log file (one JSON object per line).
- `parse_zeek_log` — Auto-detect Zeek log format (TSV or JSON) and parse accordingly.
- `ts_to_iso` — Convert Zeek epoch timestamp string to ISO 8601.
- `analyze_conn_log` — Analyze conn.log for internal lateral movement connections.
- `analyze_smb_mapping` — Analyze smb_mapping.log for admin share access.
- `analyze_smb_files` — Analyze smb_files.log for file writes to network shares.
- `analyze_dce_rpc` — Analyze dce_rpc.log for remote execution operations.
- `analyze_ntlm` — Analyze ntlm.log for Pass-the-Hash and authentication anomalies.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-lateral-movement-with-zeek`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-living-off-the-land-attacks (Skill Module)

detecting-living-off-the-land-attacks\SKILL.md

name: detecting-living-off-the-land-attacks
description: LOLBin (Living Off the Land Binary) detection agent. Parses Windows Sysmon Event ID 1 (Process Create) and Event ID 3 (Network Connection) logs in EVTX or JSON format to detect suspicious LOLBin execution patterns,.... Use when the user asks to detect living off the land attacks.
version: 1.0.0
tags:
source: detecting-living-off-the-land-attacks

Detecting Living Off The Land Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: LOLBins Detected, Suspicious Parent-Child Pairs Detected, Network-Suspicious LOLBins

When to use

- The user wants to **detect** living off the land attacks.
- Capability: LOLBin (Living Off the Land Binary) detection agent. Parses Windows Sysmon Event ID 1 (Process Create) and Event ID 3 (Network Connection) logs in EVTX or JSON format to detect suspicious LOLBin execution patterns, anomalous parent-child relationships, and LOLBin network activity.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, living, land, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse a Sysmon EVTX file and extract Event ID 1 and 3 records
2. Parse Sysmon events from a JSON lines file
3. Load Sysmon events from EVTX or JSON file
4. Detect suspicious LOLBin command-line patterns
5. Detect suspicious parent-child process relationships
6. Detect LOLBins making outbound network connections
7. Generate summary statistics

Tools & resources

- LOLBins Detected
- Suspicious Parent-Child Pairs Detected
- Network-Suspicious LOLBins

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_sysmon_evtx` — Parse a Sysmon EVTX file and extract Event ID 1 and 3 records.
- `parse_sysmon_json` — Parse Sysmon events from a JSON lines file.
- `load_events` — Load Sysmon events from EVTX or JSON file.
- `detect_lolbin_abuse` — Detect suspicious LOLBin command-line patterns.
- `detect_parent_child_anomalies` — Detect suspicious parent-child process relationships.
- `detect_lolbin_network` — Detect LOLBins making outbound network connections.
- `generate_statistics` — Generate summary statistics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-living-off-the-land-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-living-off-the-land-with-lolbas (Skill Module)

`detecting-living-off-the-land-with-lolbas\SKILL.md`

name: detecting-living-off-the-land-with-lolbas
description: Detect Living Off the Land Binaries (LOLBAS) abuse via process telemetry and Sigma rules. Use when the user asks to detect living off the land with lolbas.
version: 1.0.0
tags:
source: detecting-living-off-the-land-with-lolbas

Detecting Living Off The Land with Lolbas

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Key LOLBin Signatures, LOLBAS Project API

When to use

- The user wants to **detect** living off the land with lolbas.
- Capability: Detect Living Off the Land Binaries (LOLBAS) abuse via process telemetry and Sigma rules.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, living, land, lolbas`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Sysmon or Windows 4688 process creation events from JSON log
2. Detect LOLBin abuse from process creation events
3. Generate a Sigma detection rule for a specific LOLBin
4. Build structured detection report

Tools & resources

- Key LOLBin Signatures
- LOLBAS Project API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_process_events` — Parse Sysmon or Windows 4688 process creation events from JSON log.
- `detect_lolbin_abuse` — Detect LOLBin abuse from process creation events.
- `generate_sigma_rule` — Generate a Sigma detection rule for a specific LOLBin.
- `build_report` — Build structured detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-living-off-the-land-with-lolbas`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-malicious-scheduled-tasks-with-sysmon (Skill Module)

`detecting-malicious-scheduled-tasks-with-sysmon\SKILL.md`

name: detecting-malicious-scheduled-tasks-with-sysmon
description: Sysmon scheduled task detection agent for hunting malicious persistence. Use when the user asks to detect malicious scheduled tasks with sysmon.
version: 1.0.0
tags:
source: detecting-malicious-scheduled-tasks-with-sysmon

Detecting Malicious Scheduled Tasks with Sysmon

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Sysmon, schtasks.exe Suspicious Flags, External References

When to use

- The user wants to **detect** malicious scheduled tasks with sysmon.
- Capability: Sysmon scheduled task detection agent for hunting malicious persistence.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, malicious, scheduled, tasks, sysmon`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse exported Windows Event Log XML for Sysmon and Security events
2. Detect suspicious schtasks.exe process creation (Sysmon Event 1)
3. Detect task XML file creation in System32\Tasks (Sysmon Event 11)
4. Detect Security Event 4698 — scheduled task registered
5. Execute scheduled task detection audit

Tools & resources

- Sysmon
- schtasks.exe Suspicious Flags
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_evtx_xml` — Parse exported Windows Event Log XML for Sysmon and Security events.
- `detect_schtasks_creation` — Detect suspicious schtasks.exe process creation (Sysmon Event 1).
- `detect_task_file_creation` — Detect task XML file creation in System32\Tasks (Sysmon Event 11).
- `detect_event_4698` — Detect Security Event 4698 — scheduled task registered.
- `run_audit` — Execute scheduled task detection audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-malicious-scheduled-tasks-with-sysmon`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-mimikatz-execution-patterns (Skill Module)

detecting-mimikatz-execution-patterns\SKILL.md

name: detecting-mimikatz-execution-patterns
description: Mimikatz execution pattern detection agent. Detects Mimikatz and related credential theft tools by analyzing process creation logs, LSASS access patterns, and known command-line signatures. Use when the user asks to detect mimikatz execution patterns.
version: 1.0.0
tags:
source: detecting-mimikatz-execution-patterns

Detecting Mimikatz Execution Patterns

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Mimikatz Command Signatures, LSASS Dump Techniques, Sysmon Detection Events, Splunk SPL Detection, YARA Rule, Detection Data Sources

When to use

- The user wants to **detect** mimikatz execution patterns.
- Capability: Mimikatz execution pattern detection agent. Detects Mimikatz and related credential theft tools by analyzing process creation logs, LSASS access patterns, and known command-line signatures.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, mimikatz, execution, patterns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to mimikatz execution patterns
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Mimikatz Command Signatures
- LSASS Dump Techniques
- Sysmon Detection Events
- Splunk SPL Detection
- YARA Rule
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_evtx`
- `scan_text_log`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-mimikatz-execution-patterns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-misconfigured-azure-storage (Skill Module)

detecting-misconfigured-azure-storage\SKILL.md

name: detecting-misconfigured-azure-storage
description: Azure Storage misconfiguration detection agent using Azure CLI. Use when the user asks to detect misconfigured azure storage.
version: 1.0.0
tags:
source: detecting-misconfigured-azure-storage

Detecting Misconfigured Azure Storage

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Azure, Container Access Level Checks, Network Rules, Security Settings, Defender for Storage Alerts

When to use

- The user wants to **detect** misconfigured azure storage.
- Capability: Azure Storage misconfiguration detection agent using Azure CLI.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, misconfigured, azure, storage`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute Azure CLI command and return parsed JSON
2. List all storage accounts with security-relevant properties
3. Check for publicly accessible blob containers in a storage account
4. Verify encryption configuration for a storage account
5. Check network access rules for a storage account
6. Check if storage analytics logging is enabled
7. Check if soft delete is enabled for blobs and containers
8. Run full security audit on a single storage account

Tools & resources

- Azure
- Container Access Level Checks
- Network Rules
- Security Settings
- Defender for Storage Alerts

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `az_cli` — Execute Azure CLI command and return parsed JSON.
- `list_storage_accounts` — List all storage accounts with security-relevant properties.
- `check_public_containers` — Check for publicly accessible blob containers in a storage account.
- `check_encryption_settings` — Verify encryption configuration for a storage account.
- `check_network_rules` — Check network access rules for a storage account.
- `check_logging_enabled` — Check if storage analytics logging is enabled.
- `check_soft_delete` — Check if soft delete is enabled for blobs and containers.
- `audit_storage_account` — Run full security audit on a single storage account.
- `audit_all_accounts` — Audit all storage accounts across the subscription.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-misconfigured-azure-storage`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-mobile-malware-behavior (Skill Module)

detecting-mobile-malware-behavior\SKILL.md

name: detecting-mobile-malware-behavior
description: Mobile malware behavior detection agent. Analyzes Android APK manifests and iOS app metadata for suspicious permissions, dangerous API usage, and known malware behavioral patterns. Use when the user asks to detect mobile malware behavior.
version: 1.0.0
tags:
source: detecting-mobile-malware-behavior

Detecting Mobile Malware Behavior

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Android Dangerous Permissions, Android Analysis Tools, MobSF REST API

When to use

- The user wants to **detect** mobile malware behavior.
- Capability: Mobile malware behavior detection agent. Analyzes Android APK manifests and iOS app metadata for suspicious permissions, dangerous API usage, and known malware behavioral patterns.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, mobile, malware, behavior`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to mobile malware behavior
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Section 5: Mobile app vetting for malware indicators
- Section 6: Enterprise mobile device management for malware prevention

Tools & resources

- Android Dangerous Permissions
- Android Analysis Tools
- MobSF REST API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_apk_manifest`
- `scan_decompiled_source`
- `calculate_risk`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-mobile-malware-behavior`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-modbus-command-injection-attacks (Skill Module)

detecting-modbus-command-injection-attacks\SKILL.md

name: detecting-modbus-command-injection-attacks
description: Modbus command injection detection agent for ICS/SCADA environments. Analyzes Modbus TCP traffic for unauthorized write operations, function code abuse, and anomalous register access patterns using Zeek logs or pcap.... Use when the user asks to detect modbus command injection attacks.
version: 1.0.0
tags:
source: detecting-modbus-command-injection-attacks

Detecting Modbus Command Injection Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Modbus Function Codes, Zeek Modbus Log, Suricata Modbus Rules, pymodbus Library, Scapy Modbus Parsing, Detection Thresholds

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** modbus command injection attacks.
- Capability: Modbus command injection detection agent for ICS/SCADA environments. Analyzes Modbus TCP traffic for unauthorized write operations, function code abuse, and anomalous register access patterns using Zeek logs or pcap analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, modbus, command, injection, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to modbus command injection attacks
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Modbus Function Codes
- Zeek Modbus Log
- Suricata Modbus Rules
- pymodbus Library
- Scapy Modbus Parsing
- Detection Thresholds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_zeek_modbus_log`
- `analyze_modbus_traffic`
- `detect_write_floods`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-modbus-command-injection-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-modbus-protocol-anomalies (Skill Module)

`detecting-modbus-protocol-anomalies\SKILL.md`

name: detecting-modbus-protocol-anomalies
description: Modbus protocol anomaly detection agent for OT/ICS networks. Detects protocol-level anomalies in Modbus TCP traffic including malformed packets, timing deviations, register range violations, and replay attacks. Use when the user asks to detect modbus protocol anomalies.
version: 1.0.0
tags:
source: detecting-modbus-protocol-anomalies

Detecting Modbus Protocol Anomalies

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Modbus Protocol Limits, Anomaly Detection Methods, Zeek Modbus Log Fields, Suricata Modbus Rules, Scapy Modbus Analysis, Baseline Monitoring

When to use

- The user wants to **detect** modbus protocol anomalies.
- Capability: Modbus protocol anomaly detection agent for OT/ICS networks. Detects protocol-level anomalies in Modbus TCP traffic including malformed packets, timing deviations, register range violations, and replay attacks.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, modbus, protocol, anomalies`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to modbus protocol anomalies
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Modbus Protocol Limits
- Anomaly Detection Methods
- Zeek Modbus Log Fields
- Suricata Modbus Rules
- Scapy Modbus Analysis
- Baseline Monitoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_modbus_log`
- `detect_timing_anomalies`
- `detect_register_anomalies`
- `detect_scan_patterns`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-modbus-protocol-anomalies`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-network-anomalies-with-zeek (Skill Module)

detecting-network-anomalies-with-zeek\SKILL.md

name: detecting-network-anomalies-with-zeek
description: Zeek network anomaly detection agent for log analysis and threat hunting. Use when the user asks to detect network anomalies with zeek.
version: 1.0.0
tags:
source: detecting-network-anomalies-with-zeek

Detecting Network Anomalies with ZEEK

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Zeek, Beaconing Detection Formula

When to use

- The user wants to **detect** network anomalies with zeek.
- Capability: Zeek network anomaly detection agent for log analysis and threat hunting.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, network, anomalies, zeek`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check Zeek installation and running status
2. Parse Zeek conn.log for connection statistics and anomalies
3. Parse Zeek dns.log for DNS anomaly detection
4. Parse Zeek ssl.log for TLS anomalies and certificate issues
5. Detect C2 beaconing patterns from Zeek conn.log
6. Analyze a PCAP file with Zeek to generate logs
7. Generate comprehensive Zeek network analysis report

Tools & resources

- Zeek
- Beaconing Detection Formula

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_zeek_status` — Check Zeek installation and running status.
- `parse_conn_log` — Parse Zeek conn.log for connection statistics and anomalies.
- `parse_dns_log` — Parse Zeek dns.log for DNS anomaly detection.
- `parse_ssl_log` — Parse Zeek ssl.log for TLS anomalies and certificate issues.
- `detect_beaconing` — Detect C2 beaconing patterns from Zeek conn.log.
- `analyze_pcap` — Analyze a PCAP file with Zeek to generate logs.
- `generate_report` — Generate comprehensive Zeek network analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-network-anomalies-with-zeek`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-network-scanning-with-ids-signatures (Skill Module)

detecting-network-scanning-with-ids-signatures\SKILL.md

name: detecting-network-scanning-with-ids-signatures
description: Network scanning detection agent using IDS signature analysis. Detects port scanning, host sweeps, and service enumeration by analyzing Suricata/Snort alerts and connection logs for scanning patterns. Use when the user asks to detect network scanning with ids signatures.
version: 1.0.0
tags:
source: detecting-network-scanning-with-ids-signatures

Detecting Network Scanning with IDS Signatures

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Scan Types and Detection, Suricata Scan Detection Rules, Zeek conn.log Fields, Detection Thresholds, Splunk SPL Detection

When to use

- The user wants to **detect** network scanning with ids signatures.
- Capability: Network scanning detection agent using IDS signature analysis. Detects port scanning, host sweeps, and service enumeration by analyzing Suricata/Snort alerts and connection logs for scanning patterns.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, network, scanning, signatures`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to network scanning with ids signatures
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Scan Types and Detection
- Suricata Scan Detection Rules
- Zeek conn.log Fields
- Detection Thresholds
- Splunk SPL Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_suricata_eve`
- `parse_connection_log`
- `detect_port_scan`
- `detect_host_sweep`
- `analyze_ids_alerts`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-network-scanning-with-ids-signatures`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-ntlm-relay-with-event-correlation (Skill Module)

`detecting-ntlm-relay-with-event-correlation\SKILL.md`

name: detecting-ntlm-relay-with-event-correlation
description: NTLM Relay Detection Agent - Detects NTLM relay via Event 4624 correlation and signing audit. Use when the user asks to detect ntlm relay with event correlation.
version: 1.0.0
tags:
source: detecting-ntlm-relay-with-event-correlation

Detecting NTLM Relay with Event Correlation

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: SMB Signing Registry Keys, NTLM Configuration Registry Keys, Network Indicators, Coercion Methods, python-evtx

When to use

- The user wants to **detect** ntlm relay with event correlation.
- Capability: NTLM Relay Detection Agent - Detects NTLM relay via Event 4624 correlation and signing audit.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, ntlm, relay, event, correlation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Windows Security EVTX for Event 4624/4625/4776
2. Load hostname-to-IP inventory from CSV (columns: hostname, ip_address)
3. Detect NTLM relay via IP-hostname mismatch in Event 4624 LogonType 3
4. Detect rapid NTLM authentication to multiple targets (relay spraying)
5. Detect NTLMv1 authentication events indicating downgrade attack
6. Detect machine account NTLM relay (PetitPotam, DFSCoerce, PrinterBug)
7. Audit local SMB signing configuration (Windows only)
8. Generate JSON detection report

Tools & resources

- SMB Signing Registry Keys
- NTLM Configuration Registry Keys
- Network Indicators
- Coercion Methods
- python-evtx

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_security_evtx` — Parse Windows Security EVTX for Event 4624/4625/4776.
- `load_inventory` — Load hostname-to-IP inventory from CSV (columns: hostname, ip_address).
- `detect_ip_hostname_mismatch` — Detect NTLM relay via IP-hostname mismatch in Event 4624 LogonType 3.
- `detect_rapid_auth` — Detect rapid NTLM authentication to multiple targets (relay spraying).
- `detect_ntlmv1_downgrade` — Detect NTLMv1 authentication events indicating downgrade attack.
- `detect_machine_relay` — Detect machine account NTLM relay (PetitPotam, DFSCoerce, PrinterBug).
- `audit_smb_signing_local` — Audit local SMB signing configuration (Windows only).
- `generate_report` — Generate JSON detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-ntlm-relay-with-event-correlation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-oauth-token-theft (Skill Module)

detecting-oauth-token-theft\SKILL.md

name: detecting-oauth-token-theft
description: OAuth token theft detection agent. Analyzes sign-in logs for impossible travel, new device sign-ins, token replay from unusual IPs, and anomalous scope requests. Use when the user asks to detect oauth token theft.
version: 1.0.0
tags:
source: detecting-oauth-token-theft

Detecting Oauth Token Theft

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Microsoft Graph Sign-In Logs, Okta System Log API, Detection Logic

When to use

- The user wants to **detect** oauth token theft.
- Capability: OAuth token theft detection agent. Analyzes sign-in logs for impossible travel, new device sign-ins, token replay from unusual IPs, and anomalous scope requests.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, oauth, token, theft`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate great-circle distance between two points in km
2. Detect impossible travel based on geo and time between logins
3. Detect token replay from multiple IPs in short timeframe
4. Detect sign-ins from previously unseen devices
5. Detect OAuth requests with overly broad or sensitive scopes

Tools & resources

- Microsoft Graph Sign-In Logs
- Okta System Log API
- Detection Logic

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `haversine` — Calculate great-circle distance between two points in km.
- `detect_impossible_travel` — Detect impossible travel based on geo and time between logins.
- `detect_token_replay` — Detect token replay from multiple IPs in short timeframe.
- `detect_new_device` — Detect sign-ins from previously unseen devices.
- `detect_suspicious_scopes` — Detect OAuth requests with overly broad or sensitive scopes.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-oauth-token-theft`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-pass-the-hash-attacks (Skill Module)

detecting-pass-the-hash-attacks\SKILL.md

name: detecting-pass-the-hash-attacks
description: Pass-the-Hash attack detection agent. Detects NTLM hash reuse attacks by analyzing Windows Security Event ID 4624 for Type 3 NTLM logons with anomalous patterns across multiple targets. Use when the user asks to detect pass the hash attacks.
version: 1.0.0
tags:
source: detecting-pass-the-hash-attacks

Detecting Pass The Hash Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: python-evtx Usage, PtH Detection Logic, Splunk SPL Detection, Mitigation, Detection Data Sources

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** pass the hash attacks.
- Capability: Pass-the-Hash attack detection agent. Detects NTLM hash reuse attacks by analyzing Windows Security Event ID 4624 for Type 3 NTLM logons with anomalous patterns across multiple targets.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, pass, hash, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to pass the hash attacks
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- python-evtx Usage
- PtH Detection Logic
- Splunk SPL Detection
- Mitigation
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_logon_events`
- `detect_pth_patterns`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-pass-the-hash-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-pass-the-ticket-attacks (Skill Module)

detecting-pass-the-ticket-attacks\SKILL.md

name: detecting-pass-the-ticket-attacks
description: Detect Kerberos Pass-the-Ticket attacks via Windows Event ID 4768/4769/4771 analysis. Use when the user asks to detect pass the ticket attacks.
version: 1.0.0
tags:
source: detecting-pass-the-ticket-attacks

Detecting Pass The Ticket Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts

When to use

- The user wants to **detect** pass the ticket attacks.
- Capability: Detect Kerberos Pass-the-Ticket attacks via Windows Event ID 4768/4769/4771 analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, pass, ticket, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse exported Windows Security event log XML for Kerberos events
2. Detect RC4 encryption downgrade (TicketEncryptionType 0x17) in TGS requests
3. Detect same user TGS requests from multiple source IPs within short window
4. Detect users requesting abnormally high number of TGS tickets
5. Detect excessive Kerberos pre-authentication failures (Event ID 4771)
6. Return SPL queries for Splunk-based PtT detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_evtx_xml` — Parse exported Windows Security event log XML for Kerberos events.
- `detect_rc4_downgrade` — Detect RC4 encryption downgrade (TicketEncryptionType 0x17) in TGS requests.

- `detect_cross_host_ticket_reuse` — Detect same user TGS requests from multiple source IPs within short window.
- `detect_anomalous_tgs_volume` — Detect users requesting abnormally high number of TGS tickets.
- `detect_preauth_failures` — Detect excessive Kerberos pre-authentication failures (Event ID 4771).
- `generate_splunk_queries` — Return SPL queries for Splunk-based PtT detection.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-pass-the-ticket-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-port-scanning-with-fail2ban (Skill Module)

detecting-port-scanning-with-fail2ban\SKILL.md

name: detecting-port-scanning-with-fail2ban
description: Fail2ban port scan detection and management. Use when the user asks to detect port scanning with fail2ban.
version: 1.0.0
tags:
source: detecting-port-scanning-with-fail2ban

Detecting Port Scanning with Fail2ban

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: fail2ban-client CLI, Ban Actions

When to use

- The user wants to **detect** port scanning with fail2ban.
- Capability: Fail2ban port scan detection and management agent.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, port, scanning, fail2ban`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check Fail2ban service status and active jails
2. Get detailed status of a specific jail
3. Manually ban an IP address in a specific jail
4. Unban an IP address from a specific jail
5. Parse Fail2ban log for ban/unban statistics
6. Parse auth.log for SSH brute force patterns
7. Detect port scanning patterns from system logs
8. Generate comprehensive Fail2ban security report

Tools & resources

- fail2ban-client CLI
- Ban Actions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_fail2ban_status` — Check Fail2ban service status and active jails.
- `get_jail_status` — Get detailed status of a specific jail.
- `ban_ip` — Manually ban an IP address in a specific jail.
- `unban_ip` — Unban an IP address from a specific jail.
- `parse_fail2ban_log` — Parse Fail2ban log for ban/unban statistics.
- `parse_auth_log_ssh` — Parse auth.log for SSH brute force patterns.
- `detect_port_scan_from_logs` — Detect port scanning patterns from system logs.
- `generate_report` — Generate comprehensive Fail2ban security report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-port-scanning-with-fail2ban`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-privilege-escalation-attempts (Skill Module)

detecting-privilege-escalation-attempts\SKILL.md

name: detecting-privilege-escalation-attempts
description: Privilege escalation detection agent for Windows and Linux endpoints. Detects token manipulation, UAC bypass, sudo abuse, kernel exploits, and unquoted service paths by analyzing process creation and security logs. Use when the user asks to detect privilege escalation attempts.
version: 1.0.0
tags:
source: detecting-privilege-escalation-attempts

Detecting Privilege Escalation Attempts

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Windows Privilege Escalation Techniques, Linux Privilege Escalation Techniques, Splunk SPL, Detection Data Sources

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** privilege escalation attempts.
- Capability: Privilege escalation detection agent for Windows and Linux endpoints. Detects token manipulation, UAC bypass, sudo abuse, kernel exploits, and unquoted service paths by analyzing process creation and security logs.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, privilege, escalation, attempts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to privilege escalation attempts
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Windows Privilege Escalation Techniques
- Linux Privilege Escalation Techniques
- Splunk SPL
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_evtx`
- `analyze_text_log`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-privilege-escalation-attempts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-privilege-escalation-in-kubernetes-pods (Skill Module)

detecting-privilege-escalation-in-kubernetes-pods\SKILL.md

name: detecting-privilege-escalation-in-kubernetes-pods
description: Kubernetes pod privilege escalation detection agent. Audits pod security contexts, capabilities, and service account permissions to detect misconfigurations enabling container privilege escalation. Use when the user asks to detect privilege escalation in kubernetes pods.
version: 1.0.0
tags:
source: detecting-privilege-escalation-in-kubernetes-pods

Detecting Privilege Escalation in Kubernetes Pods

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Kubernetes, Security Context Checks, Dangerous Capabilities, Falco Rules, NIST SP 800-190, Pod Security Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** privilege escalation in kubernetes pods.
- Capability: Kubernetes pod privilege escalation detection agent. Audits pod security contexts, capabilities, and service account permissions to detect misconfigurations enabling container privilege escalation.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, privilege, escalation, kubernetes, pods`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Apply Pod Security Admission labels to namespaces
2. Deploy OPA Gatekeeper constraints
3. Test with non-compliant pods (should be rejected)
4. Install Falco with privilege escalation rules
5. Enable Kubernetes audit logging
6. Configure alerts to SIEM
7. Identify compromised pod
8. Check container security context
9. Review process list and capabilities
10. Isolate with network policy

Key concepts

- Section 4.3: Container runtime vulnerabilities
- Section 5.4: Runtime monitoring for privilege escalation

Tools & resources

- Kubernetes
- Security Context Checks
- Dangerous Capabilities
- Falco Rules

- NIST SP 800-190
- Pod Security Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_pods`
- `audit_pod`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-privilege-escalation-in-kubernetes-pods`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-process-hollowing-technique (Skill Module)

detecting-process-hollowing-technique\SKILL.md

name: detecting-process-hollowing-technique
description: Process hollowing (T1055.012) detection agent. Detects hollowed processes by analyzing Sysmon events for suspended process creation, memory allocation in remote processes, and thread hijacking. Use when the user asks to detect process hollowing technique.
version: 1.0.0
tags:
source: detecting-process-hollowing-technique

Detecting Process Hollowing Technique

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Process Hollowing API Sequence, Commonly Hollowed Processes, Sysmon Detection Events, Splunk SPL, Detection Data Sources

When to use

- The user wants to **detect** process hollowing technique.
- Capability: Process hollowing (T1055.012) detection agent. Detects hollowed processes by analyzing Sysmon events for suspended process creation, memory allocation in remote processes, and thread hijacking.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, process, hollowing, technique`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. `CreateProcessW` / `CreateProcessA` with `CREATE_SUSPENDED` (0x00000004)
2. `NtUnmapViewOfSection` / `ZwUnmapViewOfSection`
3. `VirtualAllocEx` with `PAGE_EXECUTE_READWRITE`
4. `WriteProcessMemory`
5. `SetThreadContext` / `NtSetContextThread`
6. `ResumeThread` / `NtResumeThread`

Key concepts

- Tactic: Defense Evasion (TA0005), Privilege Escalation (TA0004)
- Data Sources: Process modification, OS API execution, Process access

Tools & resources

- Process Hollowing API Sequence
- Commonly Hollowed Processes
- Sysmon Detection Events
- Splunk SPL
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `detect_hollowing_sysmon`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-process-hollowing-technique`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-process-injection-techniques (Skill Module)

detecting-process-injection-techniques\SKILL.md

name: detecting-process-injection-techniques
description: Process injection detection agent using Volatility and Sysmon analysis. Use when the user asks to detect process injection techniques.
version: 1.0.0
tags:
source: detecting-process-injection-techniques

Detecting Process Injection Techniques

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Volatility 3 Plugins, python-evtx Usage, Suspicious Parent-Child Relationships

When to use

- The user wants to **detect** process injection techniques.
- Capability: Process injection detection agent using Volatility and Sysmon analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, process, injection, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Volatility malfind plugin to detect injected code
2. Detect process hollowing via VAD/PEB image path mismatch
3. Scan Sysmon logs for injection-related events
4. Detect processes with suspicious RWX memory regions (Windows)
5. Detect unusual parent-child process relationships
6. Generate process injection detection report

Tools & resources

- Volatility 3 Plugins
- python-evtx Usage
- Suspicious Parent-Child Relationships

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_volatility_malfind` — Run Volatility malfind plugin to detect injected code.
- `run_volatility_hollowfind` — Detect process hollowing via VAD/PEB image path mismatch.
- `scan_sysmon_injection_events` — Scan Sysmon logs for injection-related events.
- `detect_rwx_memory_regions` — Detect processes with suspicious RWX memory regions (Windows).
- `check_unusual_parent_child` — Detect unusual parent-child process relationships.
- `generate_report` — Generate process injection detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-process-injection-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-qr-code-phishing-with-email-security (Skill Module)

detecting-qr-code-phishing-with-email-security\SKILL.md

name: detecting-qr-code-phishing-with-email-security
description: Agent for detecting QR code phishing (quishing) in email attachments and bodies. Use when the user asks to detect qr code phishing with email security.
version: 1.0.0
tags:
source: detecting-qr-code-phishing-with-email-security

Detecting Qr Code Phishing with Email Security

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: pyzbar, Python email Module, URL Analysis Indicators, VirusTotal URL Scan API, Common Quishing Themes, Detection Technologies

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** qr code phishing with email security.
- Capability: Agent for detecting QR code phishing (quishing) in email attachments and bodies.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, code, phishing, email, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Extract image attachments and inline images from an .eml file
2. Decode QR codes from raw image bytes
3. Score a URL for phishing risk indicators
4. Full QR phishing analysis of an email file
5. Scan a directory for .eml files and analyze each

Key concepts

- Quishing incidents grew from 46,000 to 250,000 between Aug-Nov 2025 (Kaspersky)
- Only 36% of quishing incidents accurately identified by recipients
- T1566.001: Phishing: Spearphishing Attachment (QR in PDF/image)
- T1566.002: Phishing: Spearphishing Link (QR-encoded URL)
- T1204.001: User Execution: Malicious Link (user scans QR)
- T1598.003: Phishing for Information: Spearphishing Link
- MFA enrollment/reset requiring QR scan
- Document signing via QR code

Tools & resources

- pyzbar
- Python email Module
- URL Analysis Indicators
- VirusTotal URL Scan API
- Common Quishing Themes

- Detection Technologies

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `extract_images_from_eml` — Extract image attachments and inline images from an .eml file.
- `decode_qr_from_bytes` — Decode QR codes from raw image bytes.
- `analyze_url` — Score a URL for phishing risk indicators.
- `analyze_email` — Full QR phishing analysis of an email file.
- `scan_directory` — Scan a directory for .eml files and analyze each.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-qr-code-phishing-with-email-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-ransomware-encryption-behavior (Skill Module)

detecting-ransomware-encryption-behavior\SKILL.md

name: detecting-ransomware-encryption-behavior
description: Ransomware encryption behavior detection agent. Detects ransomware activity using entropy analysis, file system I/O monitoring, and behavioral heuristics. Monitors file modifications for entropy spikes and mass.... Use when the user asks to detect ransomware encryption behavior.
version: 1.0.0
tags:
source: detecting-ransomware-encryption-behavior

Detecting Ransomware Encryption Behavior

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: psutil Process IO Monitoring, Windows ETW Providers, Behavioral Scoring, Shadow Copy Deletion, watchdog Library, Double Extension Detection

When to use

- The user wants to **detect** ransomware encryption behavior.
- Capability: Ransomware encryption behavior detection agent. Detects ransomware activity using entropy analysis, file system I/O monitoring, and behavioral heuristics. Monitors file modifications for entropy spikes and mass rename patterns characteristic of ransomware encryption.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, ransomware, encryption, behavior`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of byte data (0.0 to 8.0)
2. Analyze entropy of a file and its segments
3. Scan directory for files with high entropy indicating encryption
4. Detect multiple ransomware indicators in a directory tree
5. Take a baseline snapshot of directory for differential analysis
6. Compare two directory snapshots to detect bulk encryption

Tools & resources

- psutil Process IO Monitoring
- Windows ETW Providers
- Behavioral Scoring
- Shadow Copy Deletion
- watchdog Library
- Double Extension Detection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of byte data (0.0 to 8.0).
- `analyze_file_entropy` — Analyze entropy of a file and its segments.
- `scan_directory_entropy` — Scan directory for files with high entropy indicating encryption.
- `detect_ransomware_indicators` — Detect multiple ransomware indicators in a directory tree.
- `snapshot_directory_state` — Take a baseline snapshot of directory for differential analysis.
- `compare_snapshots` — Compare two directory snapshots to detect bulk encryption.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-ransomware-encryption-behavior`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-ransomware-precursors-in-network (Skill Module)

detecting-ransomware-precursors-in-network\SKILL.md

name: detecting-ransomware-precursors-in-network
description: Agent for detecting ransomware precursor activity in network traffic and logs. Use when the user asks to detect ransomware precursors in network.
version: 1.0.0
tags:
source: detecting-ransomware-precursors-in-network

Detecting Ransomware Precursors in Network

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Suricata CLI, Splunk SPL, abuse.ch Threat Intelligence Feeds, Industry References

When to use

- The user wants to **detect** ransomware precursors in network.
- Capability: Agent for detecting ransomware precursor activity in network traffic and logs.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, ransomware, precursors, network`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse Zeek conn.log for suspicious network connections
2. Check Windows event logs for ransomware precursors
3. Check running processes for ransomware tooling
4. Detect mass file renaming or new encrypted extensions

Key concepts

- T1078: Valid Accounts (compromised credentials from IABs)
- T1133: External Remote Services (VPN/RDP exploitation)
- T1566: Phishing (email-based initial access)
- T1071.001: Application Layer Protocol: Web (HTTPS beacons)
- T1071.004: Application Layer Protocol: DNS (DNS tunneling)
- T1573: Encrypted Channel (C2 over TLS)
- T1090: Proxy (redirectors and relay infrastructure)
- T1558.003: Kerberoasting

Tools & resources

- Suricata CLI
- Splunk SPL
- abuse.ch Threat Intelligence Feeds
- Industry References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_zeek_conn_log` — Parse Zeek conn.log for suspicious network connections.
- `analyze_event_logs_windows` — Check Windows event logs for ransomware precursors.
- `scan_process_list` — Check running processes for ransomware tooling.
- `check_file_encryption_activity` — Detect mass file renaming or new encrypted extensions.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-ransomware-precursors-in-network`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-rdp-brute-force-attacks (Skill Module)

detecting-rdp-brute-force-attacks\SKILL.md

name: detecting-rdp-brute-force-attacks
description: Agent for detecting RDP brute force attacks from Windows Event Logs. Parses EVTX files for Event ID 4625 (failed logon) and 4624 (successful logon) to identify brute force patterns, source IP frequency, username.... Use when the user asks to detect rdp brute force attacks.
version: 1.0.0
tags:
source: detecting-rdp-brute-force-attacks

Detecting RDP Brute Force Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Logon Types for RDP, Failure Sub, python-evtx Library Usage, Detection Thresholds

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** rdp brute force attacks.
- Capability: Agent for detecting RDP brute force attacks from Windows Event Logs. Parses EVTX files for Event ID 4625 (failed logon) and 4624 (successful logon) to identify brute force patterns, source IP frequency, username spraying, and post-compromise indicators.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, brute, force, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define the detection hypothesis and data sources
2. Identify telemetry and features relevant to rdp brute force attacks
3. Build or refine detection logic (rules, queries, models)
4. Tune against baselined benign activity to reduce noise
5. Validate with true-positive cases and document coverage gaps

Tools & resources

- Logon Types for RDP
- Failure Sub
- python-evtx Library Usage
- Detection Thresholds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-rdp-brute-force-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-rootkit-activity (Skill Module)

detecting-rootkit-activity\SKILL.md

name: detecting-rootkit-activity
description: Rootkit detection agent using cross-view analysis and integrity checking. Use when the user asks to detect rootkit activity.
version: 1.0.0
tags:
source: detecting-rootkit-activity

Detecting Rootkit Activity

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Volatility 3, Cross-View Detection Method, Linux Rootkit Detection Tools, Rootkit Types, Memory Acquisition

When to use

- The user wants to **detect** rootkit activity.
- Capability: Rootkit detection agent using cross-view analysis and integrity checking.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, rootkit, activity`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. List processes using ActiveProcessLinks (EPROCESS linked list)
2. Scan physical memory for EPROCESS pool tags (rootkit-resistant)
3. Compare pslist vs psscan to find hidden processes (DKOM rootkits)
4. Check for SSDT (System Service Descriptor Table) hooks
5. List loaded kernel modules and detect unsigned/suspicious ones
6. Run rkhunter for Linux rootkit detection
7. Run chkrootkit for Linux rootkit detection
8. Check for hidden files and directories that may indicate a rootkit

Tools & resources

- Volatility 3
- Cross-View Detection Method
- Linux Rootkit Detection Tools
- Rootkit Types
- Memory Acquisition

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_volatility_pslist` — List processes using ActiveProcessLinks (EPROCESS linked list).
- `run_volatility_psscan` — Scan physical memory for EPROCESS pool tags (rootkit-resistant).
- `cross_view_detection` — Compare pslist vs psscan to find hidden processes (DKOM rootkits).
- `check_ssdt_hooks` — Check for SSDT (System Service Descriptor Table) hooks.
- `check_kernel_modules` — List loaded kernel modules and detect unsigned/suspicious ones.
- `run_rkhunter` — Run rkhunter for Linux rootkit detection.
- `run_chkrootkit` — Run chkrootkit for Linux rootkit detection.
- `check_hidden_files_linux` — Check for hidden files and directories that may indicate a rootkit.
- `generate_report` — Generate comprehensive rootkit detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-rootkit-activity`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-s3-data-exfiltration-attempts (Skill Module)

detecting-s3-data-exfiltration-attempts\SKILL.md

name: detecting-s3-data-exfiltration-attempts
description: S3 data exfiltration detection agent using CloudTrail and GuardDuty. Use when the user asks to detect s3 data exfiltration attempts.
version: 1.0.0
tags:
source: detecting-s3-data-exfiltration-attempts

Detecting S3 Data Exfiltration Attempts

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: GuardDuty S3 Finding Types, CloudTrail S3 Data Events, S3 Access Monitoring, Bucket Policy

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** s3 data exfiltration attempts.
- Capability: S3 data exfiltration detection agent using CloudTrail and GuardDuty.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, data, exfiltration, attempts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute AWS CLI command and return parsed JSON
2. Get GuardDuty findings for S3 exfiltration activity
3. Query CloudTrail for GetObject events on a specific bucket
4. Detect anomalous bulk download patterns from S3
5. Check S3 bucket policy for overly permissive access
6. Verify S3 server access logging is enabled
7. Restrict S3 bucket to VPC endpoint access only
8. Generate S3 exfiltration detection report

Tools & resources

- GuardDuty S3 Finding Types
- CloudTrail S3 Data Events
- S3 Access Monitoring
- Bucket Policy

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `aws_cli` — Execute AWS CLI command and return parsed JSON.
- `get_guardduty_s3_findings` — Get GuardDuty findings for S3 exfiltration activity.
- `query_cloudtrail_s3_gets` — Query CloudTrail for GetObject events on a specific bucket.
- `detect_bulk_download_patterns` — Detect anomalous bulk download patterns from S3.
- `check_bucket_policy` — Check S3 bucket policy for overly permissive access.
- `check_s3_access_logging` — Verify S3 server access logging is enabled.
- `block_external_access` — Restrict S3 bucket to VPC endpoint access only.
- `generate_report` — Generate S3 exfiltration detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-s3-data-exfiltration-attempts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-serverless-function-injection (Skill Module)

`detecting-serverless-function-injection\SKILL.md`

name: detecting-serverless-function-injection
description: Serverless Function Injection Detection Agent - Scans Lambda functions for injection vulnerabilities, layer hijacking, and IAM escalation paths. Use when the user asks to detect serverless function injection.
version: 1.0.0
tags:
source: detecting-serverless-function-injection

Detecting Serverless Function Injection

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Arguments, Key Functions, Exit Codes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** serverless function injection.
- Capability: Serverless Function Injection Detection Agent - Scans Lambda functions for injection vulnerabilities, layer hijacking, and IAM escalation paths.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, serverless, function, injection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Map Lambda runtime to language family
2. Enumerate all Lambda functions with their configurations
3. Get all event source mappings to identify injection entry points
4. Download function code and scan for injection patterns
5. Audit Lambda layers for security issues
6. Identify Lambda functions with overprivileged execution roles
7. Search CloudTrail for suspicious Lambda modifications
8. Check Lambda function URLs for insecure authentication

Tools & resources

- Arguments
- Key Functions
- Exit Codes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_runtime_family` — Map Lambda runtime to language family.
- `enumerate_functions` — Enumerate all Lambda functions with their configurations.
- `get_event_source_mappings` — Get all event source mappings to identify injection entry points.
- `download_and_scan_function` — Download function code and scan for injection patterns.
- `audit_layers` — Audit Lambda layers for security issues.
- `detect_privilege_escalation_paths` — Identify Lambda functions with overprivileged execution roles.
- `check_cloudtrail_for_modifications` — Search CloudTrail for suspicious Lambda modifications.
- `check_function_url_security` — Check Lambda function URLs for insecure authentication.
- `generate_report` — Generate comprehensive serverless injection detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-serverless-function-injection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-service-account-abuse (Skill Module)

detecting-service-account-abuse\SKILL.md

name: detecting-service-account-abuse
description: Agent for detecting service account abuse in Active Directory and cloud environments. Use when the user asks to detect service account abuse.
version: 1.0.0
tags:
source: detecting-service-account-abuse

Detecting Service Account Abuse

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Active Directory PowerShell Module, Microsoft Graph API, AWS IAM, Detection Data Sources

When to use

- The user wants to **detect** service account abuse.
- Capability: Agent for detecting service account abuse in Active Directory and cloud environments.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, service, account, abuse`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Query Active Directory for service accounts via PowerShell
2. Find service accounts with interactive logon events (type 2/10)
3. Detect anomalous logon patterns for service accounts
4. Identify risky service account configurations

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Active Directory PowerShell Module
- Microsoft Graph API
- AWS IAM
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `query_ad_service_accounts` — Query Active Directory for service accounts via PowerShell.
- `check_interactive_logons` — Find service accounts with interactive logon events (type 2/10).
- `analyze_logon_patterns` — Detect anomalous logon patterns for service accounts.
- `check_account_risks` — Identify risky service account configurations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-service-account-abuse`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-shadow-api-endpoints (Skill Module)

detecting-shadow-api-endpoints\SKILL.md

name: detecting-shadow-api-endpoints
description: Agent for discovering undocumented (shadow) API endpoints via traffic analysis. Use when the user asks to detect shadow api endpoints.
version: 1.0.0
tags:
source: detecting-shadow-api-endpoints

Detecting Shadow API Endpoints

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: OpenAPI 3.0 Specification Structure, Akamai API Discovery, AWS API Gateway, Burp Suite Enterprise

When to use

- The user wants to **detect** shadow api endpoints.
- Capability: Agent for discovering undocumented (shadow) API endpoints via traffic analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, shadow, endpoints`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse web server access logs to extract API endpoint calls
2. Load documented endpoints from OpenAPI/Swagger spec
3. Identify endpoints in traffic that are not in the API spec
4. Classify shadow endpoints by risk category

Tools & resources

- OpenAPI 3.0 Specification Structure
- Akamai API Discovery
- AWS API Gateway
- Burp Suite Enterprise

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_access_log` — Parse web server access logs to extract API endpoint calls.
- `load_openapi_spec` — Load documented endpoints from OpenAPI/Swagger spec.
- `find_shadow_endpoints` — Identify endpoints in traffic that are not in the API spec.
- `classify_risk` — Classify shadow endpoints by risk category.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-shadow-api-endpoints`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-shadow-it-cloud-usage (Skill Module)

detecting-shadow-it-cloud-usage\SKILL.md

name: detecting-shadow-it-cloud-usage
description: Agent for detecting shadow IT cloud usage via proxy logs, DNS queries, and netflow. Use when the user asks to detect shadow it cloud usage.
version: 1.0.0
tags:
source: detecting-shadow-it-cloud-usage

Detecting Shadow It Cloud Usage

Role: Detection engineering
Audience: detection engineers and SOC analysts

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** shadow it cloud usage.
- Capability: Agent for detecting shadow IT cloud usage via proxy logs, DNS queries, and netflow.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, shadow, cloud, usage`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load approved SaaS domain list from a text file
2. Extract registered domain from URL or hostname
3. Parse proxy access log (Squid/common format) into structured records
4. Parse DNS query log (named/bind query log format)
5. Parse generic CSV log with columns: timestamp, src_ip, dst_domain, bytes_out, bytes_in
6. Classify a domain against known SaaS categories
7. Aggregate traffic by domain using pandas and classify
8. Run full shadow IT discovery audit

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_approved_list` — Load approved SaaS domain list from a text file.
- `extract_domain` — Extract registered domain from URL or hostname.
- `parse_proxy_log` — Parse proxy access log (Squid/common format) into structured records.
- `parse_dns_log` — Parse DNS query log (named/bind query log format).
- `parse_csv_log` — Parse generic CSV log with columns: timestamp, src_ip, dst_domain, bytes_out, bytes_in.
- `classify_domain` — Classify a domain against known SaaS categories.
- `analyze_traffic` — Aggregate traffic by domain using pandas and classify.
- `full_audit` — Run full shadow IT discovery audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-shadow-it-cloud-usage`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-spearphishing-with-email-gateway (Skill Module)

detecting-spearphishing-with-email-gateway\SKILL.md

name: detecting-spearphishing-with-email-gateway
description: Agent for detecting spearphishing emails using email gateway log analysis. Use when the user asks to detect spearphishing with email gateway.
version: 1.0.0
tags:
source: detecting-spearphishing-with-email-gateway

Detecting Spearphishing with Email Gateway

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Python email Module, SPF Record Lookup, DKIM Verification, DMARC Record, Proofpoint TAP API, Mimecast API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** spearphishing with email gateway.
- Capability: Agent for detecting spearphishing emails using email gateway log analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, spearphishing, email, gateway`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Extract security-relevant headers from an email
2. Verify SPF, DKIM, DMARC authentication results
3. Analyze email body for spearphishing content indicators
4. Full spearphishing analysis of a single email

Key concepts

- T1566.001: Phishing: Spearphishing Attachment
- T1566.002: Phishing: Spearphishing Link
- T1566.003: Phishing: Spearphishing via Service
- T1598.002: Phishing for Information: Spearphishing Attachment
- T1598.003: Phishing for Information: Spearphishing Link
- T1534: Internal Spearphishing
- NIST SP 800-177 Rev.1: Trustworthy Email
- NIST SP 800-53 Rev.5: SI-8 Spam Protection, SI-3 Malicious Code Protection

Tools & resources

- Python email Module
- SPF Record Lookup
- DKIM Verification
- DMARC Record
- Proofpoint TAP API
- Mimecast API

- NIST Guidelines
- CIS Controls v8

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_email_headers` — Extract security-relevant headers from an email.
- `check_authentication` — Verify SPF, DKIM, DMARC authentication results.
- `check_content_indicators` — Analyze email body for spearphishing content indicators.
- `analyze_email` — Full spearphishing analysis of a single email.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-spearphishing-with-email-gateway`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-sql-injection-via-waf-logs (Skill Module)

`detecting-sql-injection-via-waf-logs\SKILL.md`

name: detecting-sql-injection-via-waf-logs
description: SQL Injection WAF Log Analysis Agent - Detects SQLi attacks from ModSecurity and WAF logs. Use when the user asks to detect sql injection via waf logs.
version: 1.0.0
tags:
source: detecting-sql-injection-via-waf-logs

Detecting SQL Injection via WAF Logs

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: ModSecurity Audit Log Sections, SQL Injection Types, Campaign Detection Logic

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** sql injection via waf logs.
- Capability: SQL Injection WAF Log Analysis Agent - Detects SQLi attacks from ModSecurity and WAF logs.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, injection, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse ModSecurity audit log format
2. Parse JSON-formatted WAF logs (AWS WAF, Cloudflare)
3. Classify SQL injection type and severity from payload string
4. Analyze parsed ModSecurity entries for SQLi attacks
5. Analyze JSON WAF log entries for SQLi patterns
6. Identify SQLi attack campaigns by source IP clustering
7. Generate SQLi detection report

Tools & resources

- ModSecurity Audit Log Sections
- SQL Injection Types
- Campaign Detection Logic

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_modsecurity_audit_log` — Parse ModSecurity audit log format.
- `parse_json_waf_log` — Parse JSON-formatted WAF logs (AWS WAF, Cloudflare).
- `classify_sqli` — Classify SQL injection type and severity from payload string.
- `analyze_modsecurity_entries` — Analyze parsed ModSecurity entries for SQLi attacks.
- `analyze_json_waf_entries` — Analyze JSON WAF log entries for SQLi patterns.
- `correlate_campaigns` — Identify SQLi attack campaigns by source IP clustering.
- `generate_report` — Generate SQLi detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-sql-injection-via-waf-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-stuxnet-style-attacks (Skill Module)

detecting-stuxnet-style-attacks\SKILL.md

name: detecting-stuxnet-style-attacks
description: Agent for detecting Stuxnet-style ICS/SCADA attacks targeting PLCs. Use when the user asks to detect stuxnet style attacks.
version: 1.0.0
tags:
source: detecting-stuxnet-style-attacks

Detecting Stuxnet Style Attacks

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Modbus TCP Protocol, Siemens S7comm Protocol, Wireshark/tshark Filters, Stuxnet IOC Signatures, Snort/Suricata Rules for ICS

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** stuxnet style attacks.
- Capability: Agent for detecting Stuxnet-style ICS/SCADA attacks targeting PLCs.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, stuxnet, style, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze PCAP for suspicious Modbus/S7 PLC communication
2. Check for Stuxnet-style rootkit driver files
3. Check Windows registry for Stuxnet IOCs
4. Check Step 7 project files for unauthorized OB modifications

Tools & resources

- Modbus TCP Protocol
- Siemens S7comm Protocol
- Wireshark/tshark Filters
- Stuxnet IOC Signatures
- Snort/Suricata Rules for ICS

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_plc_communication` — Analyze PCAP for suspicious Modbus/S7 PLC communication.
- `scan_for_rootkit_drivers` — Check for Stuxnet-style rootkit driver files.
- `check_registry_indicators` — Check Windows registry for Stuxnet IOCs.
- `analyze_step7_project` — Check Step 7 project files for unauthorized OB modifications.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-stuxnet-style-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-supply-chain-attacks-in-ci-cd (Skill Module)

`detecting-supply-chain-attacks-in-ci-cd\SKILL.md`

name: detecting-supply-chain-attacks-in-ci-cd
description: Agent for detecting supply chain attacks in CI/CD pipelines. Use when the user asks to detect supply chain attacks in ci cd.
version: 1.0.0
tags:
source: detecting-supply-chain-attacks-in-ci-cd

Detecting Supply Chain Attacks in Ci Cd

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: GitHub Actions Workflow Parsing, Dependency Confusion Check, Pinning Actions to SHA

When to use

- The user wants to **detect** supply chain attacks in ci cd.
- Capability: Agent for detecting supply chain attacks in CI/CD pipelines.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, supply, chain, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan a single GitHub Actions workflow file for supply chain risks
2. Check if a GitHub Action is pinned to a commit SHA
3. Check for script injection via GitHub context expressions
4. Scan all workflow files in a directory
5. Check if a private package name exists on public registries
6. Scan Dockerfile for supply chain risks

Tools & resources

- GitHub Actions Workflow Parsing
- Dependency Confusion Check
- Pinning Actions to SHA

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_workflow_file` — Scan a single GitHub Actions workflow file for supply chain risks.
- `check_action_pinning` — Check if a GitHub Action is pinned to a commit SHA.
- `check_script_injection` — Check for script injection via GitHub context expressions.
- `scan_directory` — Scan all workflow files in a directory.
- `check_dependency_confusion` — Check if a private package name exists on public registries.
- `scan_dockerfile` — Scan Dockerfile for supply chain risks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-supply-chain-attacks-in-ci-cd`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-suspicious-oauth-application-consent (Skill Module)

`detecting-suspicious-oauth-application-consent\SKILL.md`

name: detecting-suspicious-oauth-application-consent
description: Agent for detecting suspicious OAuth application consent grants in Azure AD / Entra ID. Use when the user asks to detect suspicious oauth application consent.
version: 1.0.0
tags:
source: detecting-suspicious-oauth-application-consent

Detecting Suspicious Oauth Application Consent

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: High-Risk OAuth Scopes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **detect** suspicious oauth application consent.
- Capability: Agent for detecting suspicious OAuth application consent grants in Azure AD / Entra ID.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, suspicious, oauth, application, consent`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate via MSAL client credentials flow and return access token
2. Make authenticated GET request to Microsoft Graph API
3. List all delegated OAuth2 permission grants in the tenant
4. List service principals with their app roles and permissions
5. Query directory audit logs for consent grant events
6. Correlate grants with service principals to produce risk assessment
7. Run comprehensive OAuth consent audit

Tools & resources

- High-Risk OAuth Scopes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_access_token` — Authenticate via MSAL client credentials flow and return access token.
- `graph_get` — Make authenticated GET request to Microsoft Graph API.
- `enumerate_oauth_grants` — List all delegated OAuth2 permission grants in the tenant.
- `list_service_principals` — List service principals with their app roles and permissions.
- `query_consent_audit_logs` — Query directory audit logs for consent grant events.
- `analyze_risk` — Correlate grants with service principals to produce risk assessment.
- `full_audit` — Run comprehensive OAuth consent audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-suspicious-oauth-application-consent`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-suspicious-powershell-execution (Skill Module)

`detecting-suspicious-powershell-execution\SKILL.md`

name: detecting-suspicious-powershell-execution
description: Agent for detecting suspicious PowerShell execution patterns. Use when the user asks to detect suspicious powershell execution.
version: 1.0.0
tags:
source: detecting-suspicious-powershell-execution

Detecting Suspicious PowerShell Execution

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Sigma Detection Rules, Microsoft Defender ATP API, Detection Data Sources

When to use

- The user wants to **detect** suspicious powershell execution.
- Capability: Agent for detecting suspicious PowerShell execution patterns.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, suspicious, powershell, execution`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse PowerShell script block logging events (Event ID 4104)
2. Analyze a PowerShell script for suspicious patterns
3. Analyze a text file containing PowerShell commands

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Sigma Detection Rules
- Microsoft Defender ATP API
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_script_block_logs` — Parse PowerShell script block logging events (Event ID 4104).
- `analyze_script_content` — Analyze a PowerShell script for suspicious patterns.
- `analyze_log_file` — Analyze a text file containing PowerShell commands.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-suspicious-powershell-execution`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-t1003-credential-dumping-with-edr (Skill Module)

detecting-t1003-credential-dumping-with-edr\SKILL.md

name: detecting-t1003-credential-dumping-with-edr
description: Agent for detecting T1003 credential dumping via EDR telemetry analysis. Use when the user asks to detect t1003 credential dumping with edr.
version: 1.0.0
tags:
source: detecting-t1003-credential-dumping-with-edr

Detecting T1003 Credential Dumping with EDR

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: CrowdStrike Falcon, Microsoft Defender ATP, Sigma Rules, Protection Controls, Known Credential Dumping Tools

When to use

- The user wants to **detect** t1003 credential dumping with edr.
- Capability: Agent for detecting T1003 credential dumping via EDR telemetry analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, t1003, credential, dumping`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Query Sysmon Event ID 10 for LSASS process access
2. Check for known credential dumping tool processes
3. Check for SAM/NTDS.dit/SYSTEM registry hive access

Tools & resources

- CrowdStrike Falcon
- Microsoft Defender ATP
- Sigma Rules
- Protection Controls
- Known Credential Dumping Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_lsass_access_sysmon` — Query Sysmon Event ID 10 for LSASS process access.
- `check_credential_dump_processes` — Check for known credential dumping tool processes.
- `check_sam_ntds_access` — Check for SAM/NTDS.dit/SYSTEM registry hive access.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-t1003-credential-dumping-with-edr`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-t1055-process-injection-with-sysmon (Skill Module)

detecting-t1055-process-injection-with-sysmon\SKILL.md

name: detecting-t1055-process-injection-with-sysmon
description: Agent for detecting T1055 process injection via Sysmon event analysis. Use when the user asks to detect t1055 process injection with sysmon.
version: 1.0.0
tags:
source: detecting-t1055-process-injection-with-sysmon

Detecting T1055 Process Injection with Sysmon

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Sysmon, Process Access Masks, Sigma Rule Example, ProcessAccess Granted Access Masks, Common Injection Targets

When to use

- The user wants to **detect** t1055 process injection with sysmon.
- Capability: Agent for detecting T1055 process injection via Sysmon event analysis.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, t1055, process, injection, sysmon`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Query Sysmon operational log for specific event ID
2. Analyze Sysmon Event ID 10 for injection-related process access
3. Analyze Sysmon Event ID 8 for CreateRemoteThread injection
4. Analyze Sysmon Event ID 7 for suspicious DLL loads in unusual processes

Tools & resources

- Sysmon
- Process Access Masks
- Sigma Rule Example
- ProcessAccess Granted Access Masks
- Common Injection Targets

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `query_sysmon_events` — Query Sysmon operational log for specific event ID.
- `analyze_process_access` — Analyze Sysmon Event ID 10 for injection-related process access.
- `analyze_create_remote_thread` — Analyze Sysmon Event ID 8 for CreateRemoteThread injection.
- `analyze_image_loads` — Analyze Sysmon Event ID 7 for suspicious DLL loads in unusual processes.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-t1055-process-injection-with-sysmon`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-t1548-abuse-elevation-control-mechanism (Skill Module)

detecting-t1548-abuse-elevation-control-mechanism\SKILL.md

name: detecting-t1548-abuse-elevation-control-mechanism
description: Agent for detecting T1548 Abuse Elevation Control Mechanism (UAC bypass, sudo abuse). Use when the user asks to detect t1548 abuse elevation control mechanism.
version: 1.0.0
tags:
source: detecting-t1548-abuse-elevation-control-mechanism

Detecting T1548 Abuse Elevation Control Mechanism

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: UAC Bypass, Windows UAC Configuration, Linux Privilege Escalation, Sysmon Detection Rules, Sigma Rule, Detection Events

When to use

- The user wants to **detect** t1548 abuse elevation control mechanism.
- Capability: Agent for detecting T1548 Abuse Elevation Control Mechanism (UAC bypass, sudo abuse).
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, t1548, abuse, elevation, control, mechanism`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check for UAC bypass registry modifications
2. Check Sysmon logs for auto-elevate binary abuse
3. Check auth logs for sudo abuse patterns
4. Find SUID/SGID binaries that could be abused for elevation

Tools & resources

- UAC Bypass
- Windows UAC Configuration
- Linux Privilege Escalation
- Sysmon Detection Rules
- Sigma Rule
- Detection Events

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_uac_bypass_registry` — Check for UAC bypass registry modifications.
- `check_uac_bypass_processes` — Check Sysmon logs for auto-elevate binary abuse.
- `check_linux_sudo_abuse` — Check auth logs for sudo abuse patterns.
- `check_setuid_binaries` — Find SUID/SGID binaries that could be abused for elevation.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-t1548-abuse-elevation-control-mechanism`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-typosquatting-packages-in-npm-pypi (Skill Module)

detecting-typosquatting-packages-in-npm-pypi\SKILL.md

name: detecting-typosquatting-packages-in-npm-pypi
description: Typosquatting Detection Agent - Detects typosquatting packages in npm and PyPI registries using Levenshtein distance analysis, publish date heuristics, and download count anomalies. Use when the user asks to detect typosquatting packages in npm pypi.
version: 1.0.0
tags:
source: detecting-typosquatting-packages-in-npm-pypi

Detecting Typosquatting Packages in Npm Pypi

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Arguments, Key Functions, Scoring Weights

When to use

- The user wants to **detect** typosquatting packages in npm pypi.
- Capability: Typosquatting Detection Agent - Detects typosquatting packages in npm and PyPI registries using Levenshtein distance analysis, publish date heuristics, and download count anomalies.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, typosquatting, packages, pypi`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Normalize a PyPI package name per PEP 503
2. Generate potential typosquat variants of a package name. Produces candidates via character omission, transposition, insertion, substitution (keyboard-adjacent), and separator manipulation
3. Query the PyPI JSON API for package metadata. Returns parsed metadata or None if the package does not exist
4. Query the npm registry API for package metadata. Returns parsed metadata or None if the package does not exist
5. Get recent download stats for a PyPI package from pypistats.org
6. Get last-week download count for an npm package
7. Compute a weighted suspicion score for a candidate typosquat package. Signals: - Levenshtein distance (1 = 40pts, 2 = 25pts, 3 = 10pts) - Publish recency: created within 90 days = 15pts - Download ratio: candidate/target < 0.001 = 15pts - Different author/maintainer = 10pts - Low version count (<=2) = 5pts - No repository URL = 5pts
8. Classify risk level based on composite score

Tools & resources

- Arguments
- Key Functions
- Scoring Weights

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `normalize_pypi_name` — Normalize a PyPI package name per PEP 503.
- `generate_typosquat_candidates` — Generate potential typosquat variants of a package name. Produces candidates via character omission, transposition, insertion, substitution (keyboard-adjacent), and separator manipulation.
- `query_pypi_package` — Query the PyPI JSON API for package metadata. Returns parsed metadata or None if the package does not exist.
- `query_npm_package` — Query the npm registry API for package metadata. Returns parsed metadata or None if the package does not exist.
- `get_pypi_downloads` — Get recent download stats for a PyPI package from pypistats.org.
- `get_npm_downloads` — Get last-week download count for an npm package.
- `compute_suspicion_score` — Compute a weighted suspicion score for a candidate typosquat package. Signals: - Levenshtein distance (1 = 40pts, 2 = 25pts, 3 = 10pts) - Publish recency: created within 90 days = 15pts - Download ratio: candidate/target < 0.001 = 15pts - Different author/maintainer = 10pts - Low version count (<=2) = 5pts - No repository URL = 5pts
- `classify_risk` — Classify risk level based on composite score.
- `scan_package` — Scan for typosquat candidates of a target package. Generates candidates, checks which exist in the registry, scores them, and returns ranked results.
- `scan_dependency_file` — Scan all dependencies in a requirements file or package.json.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-typosquatting-packages-in-npm-pypi`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

detecting-wmi-persistence (Skill Module)

`detecting-wmi-persistence\SKILL.md`

name: detecting-wmi-persistence
description: WMI Persistence Detection Agent - hunts for malicious WMI event subscriptions via Sysmon and WMI queries. Use when the user asks to detect wmi persistence.
version: 1.0.0
tags:
source: detecting-wmi-persistence

Detecting WMI Persistence

Role: Detection engineering
Audience: detection engineers and SOC analysts
Primary tools: Sysmon Configuration, PowerShell WMI Enumeration, Suspicious Consumer Types, Elastic Detection Rule, Autoruns WMI Tab

When to use

- The user wants to **detect** wmi persistence.
- Capability: WMI Persistence Detection Agent - hunts for malicious WMI event subscriptions via Sysmon and WMI queries.
- Goal: identify malicious or anomalous activity with high-signal detections.
- Trigger keywords: `detecting, persistence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query Sysmon Event IDs 19, 20, 21 for WMI persistence
2. Enumerate active WMI event subscriptions via PowerShell
3. Identify suspicious WMI subscriptions
4. Analyze Sysmon WMI events for suspicious patterns
5. Generate comprehensive WMI persistence hunt report

Tools & resources

- Sysmon Configuration
- PowerShell WMI Enumeration
- Suspicious Consumer Types
- Elastic Detection Rule
- Autoruns WMI Tab

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_sysmon_wmi_events` — Query Sysmon Event IDs 19, 20, 21 for WMI persistence.
- `enumerate_wmi_subscriptions` — Enumerate active WMI event subscriptions via PowerShell.
- `analyze_suspicious_subscriptions` — Identify suspicious WMI subscriptions.
- `analyze_sysmon_events` — Analyze Sysmon WMI events for suspicious patterns.
- `generate_report` — Generate comprehensive WMI persistence hunt report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `detecting-wmi-persistence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

digital-archive (Skill Module)

research-toolkit\skills\digital-archive\SKILL.md

name: digital-archive

description: Digital archiving workflows with AI enrichment, entity extraction, and knowledge graph construction. Use when building content archives, implementing AI-powered categorization, extracting entities and relationships, or integrating multiple data sources. Covers patterns from the Jay Rosen Digital Archive project.

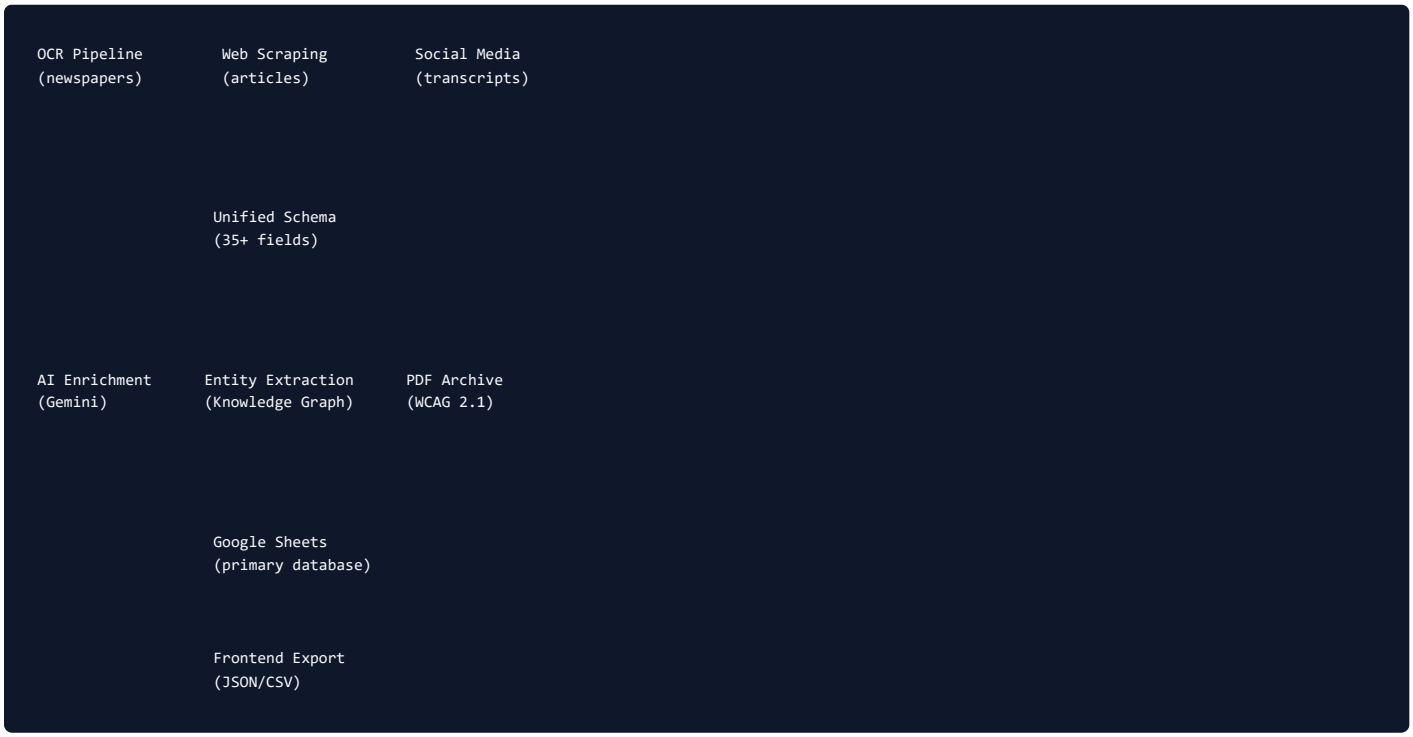
tags:

Digital archive methodology

Patterns for building production-quality digital archives with AI-powered analysis and knowledge graph construction.

Archive architecture

Multi-source [[integration]] pattern



Unified schema design

```
from dataclasses import dataclass, field
from datetime import date
from typing import Optional
from enum import Enum

class ContentType(Enum):
    ARTICLE = 'Article'
    VIDEO = 'Video'
    AUDIO = 'Audio'
    SOCIAL = 'Social Post'
    NEWSPAPER = 'Newspaper Article'

class ThematicCategory(Enum):
    PRESS_CRITICISM = 'Press & Media Criticism'
    JOURNALISM_THEORY = 'Journalism Theory'
    POLITICS = 'Politics & Democracy'
    TECHNOLOGY = 'Technology & Digital Media'
    EDUCATION = 'Journalism Education'
    AUDIENCE = 'Audience & Public Engagement'

class HistoricalEra(Enum):
    ERA_1990s = '1990-1999'
    ERA_2000_04 = '2000-2004'
    ERA_2005_09 = '2005-2009'
    ERA_2010_15 = '2010-2015'
    ERA_2016_20 = '2016-2020'
    ERA_2021_25 = '2021-2025'
    ERA_2026_PRESENT = '2026-present'

@dataclass
class ArchiveRecord:
    # Core identifiers
    id: str                              # Format: SOURCE-00001
    url: str
    title: str

    # Content
    author: Optional[str] = None
    publication_date: Optional[date] = None
    publication: Optional[str] = None
    content_type: ContentType = ContentType.ARTICLE
    text: str = ''

    # AI-enriched fields
    summary: Optional[str] = None
    pull_quote: Optional[str] = None
    categories: list[ThematicCategory] = field(default_factory=list)
    key_concepts: list[str] = field(default_factory=list)
    tags: list[str] = field(default_factory=list)
    era: Optional[HistoricalEra] = None
    scope: Optional[str] = None  # Theoretical, Commentary, Case Study, etc.

    # Entity references
    entities_mentioned: list[str] = field(default_factory=list)
    related_to: list[str] = field(default_factory=list)
    responds_to: list[str] = field(default_factory=list)

    # Archive metadata
    pdf_url: Optional[str] = None
    transcript_url: Optional[str] = None
    verified: bool = False
    processing_status: str = 'pending'
    last_updated: Optional[date] = None

def generate_record_id(source: str, sequence: int) -> str:
    """Generate unique ID with source prefix."""
    prefixes = {
        'nytimes': 'NYT',
        'columbia journalism review': 'CJR',
        'pressthink': 'PT',
        'twitter': 'TW',
        'youtube': 'YT',
        'newspaper': 'NEWS',
    }
```

```
    prefix = prefixes.get(source.lower(), 'MISC')
    return f"{prefix}-{sequence:05d}"
```

AI-powered categorization

Taxonomy-based classification

```
# pip install google-genai
# (the legacy `google-generativeai` SDK was deprecated in 2024 — the
# new `google-genai` package is the supported path. Imports below
# use the new shape.)
import os
from google import genai
from google.genai import types
import json
from typing import Optional

# Default to the current Gemini 2.5 family. For 2026 production
# workloads, the Gemini 3 family (gemini-3-flash, gemini-3-pro) is
# also available — bump the model string when you've verified the
# response shape against your taxonomy prompts.
DEFAULT_GEMINI_MODEL = 'gemini-2.5-flash'

# Single client; reads GOOGLE_API_KEY (or pass api_key=...).
_client = genai.Client(api_key=os.environ.get('GOOGLE_API_KEY'))

TAXONOMY = {
    "thematic_categories": [
        "Press & Media Criticism",
        "Journalism Theory",
        "Politics & Democracy",
        "Technology & Digital Media",
        "Journalism Education",
        "Audience & Public Engagement"
    ],
    "key_concepts": [
        "The View from Nowhere",
        "Verification vs. Assertion",
        "Citizens vs. Consumers",
        "Public Journalism",
        "The Rosen Test",
        "Savvy vs. Naive",
        "Professional vs. Amateur",
        "Production vs. Distribution",
        "Trust vs. Transparency",
        "Horse Race Coverage",
        "Both Sides Journalism",
        "Audience Atomization",
        "The Church of the Savvy"
    ],
    "scope_types": [
        "Theoretical",
        "Commentary",
        "Historical",
        "Case Study",
        "Pedagogical",
        "Personal Reflection"
    ]
}

class ArchiveCategorizer:
    def __init__(self, model: str = DEFAULT_GEMINI_MODEL, client: genai.Client = None):
        self.model = model
        self.client = client or _client

    def categorize(self, record: ArchiveRecord) -> dict:
        prompt = f"""Analyze this archival content and categorize it according to the taxonomy.

CONTENT:
Title: {record.title}
Author: {record.author or 'Unknown'}
Date: {record.publication_date or 'Unknown'}
Text (first 8000 chars):
{record.text[:8000]}

TAXONOMY:
{json.dumps(TAXONOMY, indent=2)}

Respond with JSON containing:
{{
  "categories": ["category1", "category2"],  // 1-3 from thematic_categories
```

```
  "key_concepts": ["concept1", "concept2"],  // 0-5 from key_concepts list
  "scope": "scope_type",                     // one from scope_types
  "era": "YYYY-YYYY",                        // decade range
  "tags": ["tag1", "tag2", "tag3", "tag4", "tag5"],  // 5 contextual keywords
  "summary": "2-3 sentence summary",
  "pull_quote": "Most impactful quote from the text"
}}

IMPORTANT:
- Only use categories/concepts from the taxonomy
- Tags should be lowercase, hyphenated keywords
- Summary should capture the main argument
- Pull quote must be an exact excerpt from the text
"""

        # response_mime_type='application/json' makes Gemini emit raw
        # JSON without ```json fences — the markdown-stripping fallback
        # in _parse_response() is kept as [[defense-in-depth]] for older
        # models that still wrap output.
        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt,
            config=types.GenerateContentConfig(
                response_mime_type='application/json',
            ),
        )
        result = self._parse_response(response.text)

        # Validate against taxonomy
        result['categories'] = [c for c in result.get('categories', [])
                                if c in TAXONOMY['thematic_categories']]
        result['key_concepts'] = [c for c in result.get('key_concepts', [])
                                  if c in TAXONOMY['key_concepts']]

        return result

    def _parse_response(self, text: str) -> dict:
        """Extract JSON from response, tolerating ```json fences if any.

        With response_mime_type='application/json' set on the request,
        Gemini emits clean JSON; this stripping logic is a fallback for
        older models or when the request config wasn't applied.
        """
        if '```json' in text:
            text = text.split('```json')[1].split('```')[0]
        elif '```' in text:
            text = text.split('```')[1].split('```')[0]

        return json.loads(text.strip())

    def validate_response(self, result: dict, text: str) -> bool:
        """Detect AI hallucination patterns."""
        # Check for uniform response signature (all same values)
        if len(set(result.get('tags', []))) < 3:
            return False

        # Check pull quote exists in text
        pull_quote = result.get('pull_quote', '')
        if pull_quote and pull_quote.lower() not in text.lower():
            return False

        # Check summary isn't generic
        generic_phrases = ['this article discusses', 'the author explores', 'this piece examines']
        summary = result.get('summary', '').lower()
        if any(phrase in summary for phrase in generic_phrases):
            return False

        return True
```

Entity extraction and knowledge graph

Entity types and relationships

```
from dataclasses import dataclass
from typing import Literal

EntityType = Literal['Person', 'Organization', 'Work', 'Concept', 'Event', 'Location']
RelationshipType = Literal[
    'Mentions', 'Criticizes', 'Cites', 'Discusses', 'Expands On', 'Supports',
    'Founded By', 'Pioneered', 'Inspired By',
    'Affiliated With', 'Published In', 'Originated By', 'Occurred At',
    'Owns', 'Owned By'
]

@dataclass
class Entity:
    id: str                    # P-001, O-001, W-001, etc.
    name: str
    type: EntityType
    aliases: list[str]         # Alternative names/spellings
    prominence: float          # 0-10 based on discussion depth
    mention_count: int = 0
    first_mentioned_in: str = ''  # Record ID

@dataclass
class Relationship:
    source_entity_id: str
    target_entity_id: str
    relationship_type: RelationshipType
    source_record_id: str      # Which record established this relationship
    confidence: float = 1.0

class EntityRegistry:
    """Deduplication and normalization for entities."""

    NORMALIZATIONS = {
        'nyt': 'The New York Times',
        'new york times': 'The New York Times',
        'ny times': 'The New York Times',
        'washington post': 'The Washington Post',
        'wapo': 'The Washington Post',
        'cnn': 'CNN',
        'fox': 'Fox News',
        'fox news channel': 'Fox News',
    }

    def __init__(self):
        self.entities: dict[str, Entity] = {}
        self.name_to_id: dict[str, str] = {}

    def normalize_name(self, name: str) -> str:
        """Normalize entity name to canonical form."""
        name_lower = name.lower().strip()
        return self.NORMALIZATIONS.get(name_lower, name.strip())

    def find_or_create(self, name: str, entity_type: EntityType) -> Entity:
        """Find existing entity or create new one."""
        normalized = self.normalize_name(name)

        # Check if already exists
        if normalized.lower() in self.name_to_id:
            entity_id = self.name_to_id[normalized.lower()]
            entity = self.entities[entity_id]
            entity.mention_count += 1
            return entity

        # Create new entity
        type_prefix = entity_type[0].upper()  # P, O, W, C, E, L
        count = sum(1 for e in self.entities.values() if e.type == entity_type)
        entity_id = f"{type_prefix}-{count + 1:04d}"

        entity = Entity(
            id=entity_id,
            name=normalized,
            type=entity_type,
            aliases=[name] if name != normalized else [],
            prominence=0.0,
```

```
            mention_count=1
        )

        self.entities[entity_id] = entity
        self.name_to_id[normalized.lower()] = entity_id

        return entity
```

AI-powered entity extraction

```
class EntityExtractor:
    def __init__(self, registry: EntityRegistry, model: str = DEFAULT_GEMINI_MODEL,
                 client: genai.Client = None):
        self.registry = registry
        self.model = model
        self.client = client or _client

    def extract(self, record: ArchiveRecord) -> tuple[list[Entity], list[Relationship]]:
        prompt = f"""Extract named entities and relationships from this archival content.

CONTENT:
Title: {record.title}
Text: {record.text[:10000]}

ENTITY TYPES:
- Person: journalists, politicians, academics, media figures
- Organization: news outlets, media companies, academic institutions
- Work: articles, books, blog posts, studies, reports
- Concept: journalism theories, media criticism frameworks
- Event: conferences, elections, media crises
- Location: geographic locations relevant to media context

RELATIONSHIP TYPES:
- Mentions, Criticizes, Cites, Discusses, Expands On, Supports
- Founded By, Pioneered, Inspired By
- Affiliated With, Published In, Originated By, Occurred At
- Owns, Owned By

Respond with JSON:
{{
  "entities": [
    {{"name": "Entity Name", "type": "Person|Organization|...", "prominence": 1-10}}
  ],
  "relationships": [
    {{"source": "Entity Name", "target": "Entity Name", "type": "Relationship Type"}}
  ]
}}

IMPORTANT:
- Prominence: 1-3 = mentioned briefly, 4-6 = discussed, 7-10 = central focus
- Only extract entities actually discussed, not just mentioned in passing
- Relationships must connect entities that appear in the same text
"""

        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt,
            config=types.GenerateContentConfig(
                response_mime_type='application/json',
            ),
        )
        data = json.loads(response.text)

        entities = []
        entity_name_to_obj = {}

        # Process entities
        for e in data.get('entities', []):
            entity = self.registry.find_or_create(e['name'], e['type'])
            entity.prominence = max(entity.prominence, e.get('prominence', 5))
            entities.append(entity)
            entity_name_to_obj[e['name'].lower()] = entity

        # Process relationships
        relationships = []
        for r in data.get('relationships', []):
            source = entity_name_to_obj.get(r['source'].lower())
            target = entity_name_to_obj.get(r['target'].lower())

            if source and target:
                relationships.append(Relationship(
                    source_entity_id=source.id,
                    target_entity_id=target.id,
                    relationship_type=r['type'],
                    source_record_id=record.id
                ))
```

```
    return entities, relationships
```

PDF archival generation

```
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Image
from reportlab.lib.units import inch
from pathlib import Path

class ArchivePDFGenerator:
    """Generate accessible PDFs for archival preservation."""

    def __init__(self, output_dir: Path):
        self.output_dir = output_dir
        self.output_dir.mkdir(parents=True, exist_ok=True)
        self.styles = getSampleStyleSheet()

        # Custom styles
        self.styles.add(ParagraphStyle(
            'ArchiveTitle',
            parent=self.styles['Heading1'],
            fontSize=16,
            spaceAfter=12
        ))
        self.styles.add(ParagraphStyle(
            'ArchiveMeta',
            parent=self.styles['Normal'],
            fontSize=10,
            textColor='#666666',
            spaceAfter=6
        ))

    def generate(self, record: ArchiveRecord) -> Path:
        output_path = self.output_dir / f"{record.id}.pdf"

        doc = SimpleDocTemplate(
            str(output_path),
            pagesize=letter,
            title=record.title,
            author=record.author or 'Unknown',
            subject=f"Archive record {record.id}"
        )

        story = []

        # Title
        story.append(Paragraph(record.title, self.styles['ArchiveTitle']))

        # Metadata block
        meta_lines = [
            f"<b>Author:</b> {record.author or 'Unknown'}",
            f"<b>Date:</b> {record.publication_date or 'Unknown'}",
            f"<b>Source:</b> {record.publication or 'Unknown'}",
            f"<b>URL:</b> {record.url}",
            f"<b>Archive ID:</b> {record.id}",
        ]
        for line in meta_lines:
            story.append(Paragraph(line, self.styles['ArchiveMeta']))

        story.append(Spacer(1, 0.25 * inch))

        # Summary (if available)
        if record.summary:
            story.append(Paragraph("<b>Summary:</b>", self.styles['Heading2']))
            story.append(Paragraph(record.summary, self.styles['Normal']))
            story.append(Spacer(1, 0.25 * inch))

        # Main content
        story.append(Paragraph("<b>Full Text:</b>", self.styles['Heading2']))

        # Split into paragraphs and add
        paragraphs = record.text.split('\n\n')
        for para in paragraphs:
            if para.strip():
                story.append(Paragraph(para.strip(), self.styles['Normal']))
                story.append(Spacer(1, 0.1 * inch))

        # Build PDF
        doc.build(story)
```

```
    return output_path
```

Data quality and validation

```
from dataclasses import dataclass
from typing import Callable

@dataclass
class ValidationResult:
    field: str
    valid: bool
    message: str
    severity: Literal['error', 'warning', 'info']

class ArchiveValidator:
    """Validate archive records for completeness and consistency."""

    REQUIRED_FIELDS = ['id', 'url', 'title', 'text']
    CRITICAL_FIELDS = ['publication_date', 'author', 'summary']
    OPTIONAL_FIELDS = ['categories', 'tags', 'pull_quote']

    def validate(self, record: ArchiveRecord) -> list[ValidationResult]:
        results = []

        # Required fields
        for field in self.REQUIRED_FIELDS:
            value = getattr(record, field, None)
            if not value:
                results.append(ValidationResult(
                    field=field,
                    valid=False,
                    message=f"Required field '{field}' is missing",
                    severity='error'
                ))

        # Critical fields (should have but not blocking)
        for field in self.CRITICAL_FIELDS:
            value = getattr(record, field, None)
            if not value:
                results.append(ValidationResult(
                    field=field,
                    valid=False,
                    message=f"Critical field '{field}' is missing",
                    severity='warning'
                ))

        # Content length check
        if record.text and len(record.text) < 100:
            results.append(ValidationResult(
                field='text',
                valid=False,
                message=f"Text unusually short ({len(record.text)} chars)",
                severity='warning'
            ))

        # Date format validation
        if record.publication_date:
            try:
                # Ensure date is valid
                _ = record.publication_date.isoformat()
            except (AttributeError, ValueError):
                results.append(ValidationResult(
                    field='publication_date',
                    valid=False,
                    message="Invalid date format",
                    severity='error'
                ))

        # Category validation
        for cat in record.categories:
            if cat not in ThematicCategory:
                results.append(ValidationResult(
                    field='categories',
                    valid=False,
                    message=f"Unknown category: {cat}",
                    severity='warning'
                ))

        return results
```

```
    def is_complete(self, record: ArchiveRecord) -> bool:
        """Check if record has all critical fields populated."""
        results = self.validate(record)
        errors = [r for r in results if r.severity == 'error']
        return len(errors) == 0
```

[[Integration]] workflow

```
class ArchiveWorkflow:
    """Orchestrate the complete archive processing pipeline."""

    def __init__(self, config: Config):
        self.scraper = ScrapingCascade()
        self.categorizer = ArchiveCategorizer()
        self.entity_registry = EntityRegistry()
        self.entity_extractor = EntityExtractor(self.entity_registry)
        self.pdf_generator = ArchivePDFGenerator(config.PDF_DIR)
        self.sheets_service = SheetsService(config.CREDENTIALS_PATH)
        self.validator = ArchiveValidator()
        self.progress = ProgressTracker(config.PROGRESS_FILE)

    def process_url(self, url: str, record_id: str) -> ArchiveRecord:
        """Process a single URL through the complete pipeline."""

        # 1. Scrape content
        result = self.scraper.fetch(url)
        if not result:
            raise ValueError(f"Failed to scrape: {url}")

        # 2. Create initial record
        record = ArchiveRecord(
            id=record_id,
            url=url,
            title=result.title,
            text=result.content
        )

        # 3. AI categorization
        categories = self.categorizer.categorize(record)
        record.summary = categories.get('summary')
        record.pull_quote = categories.get('pull_quote')
        record.categories = categories.get('categories', [])
        record.key_concepts = categories.get('key_concepts', [])
        record.tags = categories.get('tags', [])
        record.era = categories.get('era')
        record.scope = categories.get('scope')

        # 4. Entity extraction
        entities, relationships = self.entity_extractor.extract(record)
        record.entities_mentioned = [e.id for e in entities]

        # 5. Generate PDF
        pdf_path = self.pdf_generator.generate(record)
        record.pdf_url = str(pdf_path)

        # 6. Validate
        validation = self.validator.validate(record)
        record.verified = self.validator.is_complete(record)
        record.processing_status = 'completed'

        return record

    def run_batch(self, input_csv: Path):
        """Process all URLs from input CSV."""
        for row in read_input(input_csv):
            if self.progress.is_processed(row.id):
                continue

            try:
                record = self.process_url(row.url, row.id)
                self.sheets_service.append_row(self.worksheet, record_to_row(record))
                self.progress.mark_processed(row.id)
            except Exception as e:
                self.progress.log_error(row.id, str(e))
```

Export for frontend consumption

```
import json
from pathlib import Path

def export_for_frontend(records: list[ArchiveRecord], output_dir: Path):
    """Export archive data in frontend-friendly formats."""

    # Main archive JSON
    archive_data = {
        'metadata': {
            'total_records': len(records),
            'last_updated': datetime.now().isoformat(),
            'schema_version': '2.0'
        },
        'records': [asdict(r) for r in records]
    }

    (output_dir / 'archive-data.json').write_text(
        json.dumps(archive_data, indent=2, default=str)
    )

    # Entity export
    entities_data = [asdict(e) for e in entity_registry.entities.values()]
    (output_dir / 'entities.json').write_text(
        json.dumps(entities_data, indent=2)
    )

    # Relationships export
    relationships_data = [asdict(r) for r in all_relationships]
    (output_dir / 'relationships.json').write_text(
        json.dumps(relationships_data, indent=2)
    )

    # CSV exports for spreadsheet compatibility
    records_df = pd.DataFrame([asdict(r) for r in records])
    records_df.to_csv(output_dir / 'archive_records.csv', index=False)
```

dispatching-parallel-agents (Skill Module)

superjawn\skills\dispatching-parallel-agents\SKILL.md

name: dispatching-parallel-agents
description: Use when facing 2+ independent tasks that can be worked on without shared state or sequential dependencies
tags:

Dispatching Parallel [[Agents]]

Overview

You delegate tasks to specialized [[agents]] with isolated context. By precisely crafting their instructions and context, you ensure they stay focused and succeed at their task. They should never inherit your session's context or history — you construct exactly what they need. This also preserves your own context for coordination work.

When you have multiple unrelated failures (different test files, different subsystems, different bugs), investigating them sequentially wastes time. Each investigation is independent and can happen in parallel.

Core principle: Dispatch one agent per independent problem domain. Let them work concurrently.

When to Use

```
digraph when_to_use {
    "Multiple failures?" [shape=diamond];
    "Are they independent?" [shape=diamond];
    "Single agent investigates all" [shape=box];
    "One agent per problem domain" [shape=box];
    "Can they work in parallel?" [shape=diamond];
    "Sequential agents" [shape=box];
    "Parallel dispatch" [shape=box];

    "Multiple failures?" -> "Are they independent?" [label="yes"];
    "Are they independent?" -> "Single agent investigates all" [label="no - related"];
    "Are they independent?" -> "Can they work in parallel?" [label="yes"];
    "Can they work in parallel?" -> "Parallel dispatch" [label="yes"];
    "Can they work in parallel?" -> "Sequential agents" [label="no - shared state"];
}
```

Use when:
- 3+ test files failing with different root causes
- Multiple subsystems broken independently
- Each problem can be understood without context from others
- No shared state between investigations

Don't use when:
- Failures are related (fix one might fix others)
- Need to understand full system state
- [[Agents]] would interfere with each other

The Pattern

1. Identify Independent Domains

Group failures by what's broken:
- File A tests: Tool approval flow
- File B tests: Batch completion behavior
- File C tests: Abort functionality

Each domain is independent - fixing tool approval doesn't affect abort tests.

2. Create Focused Agent Tasks

Each agent gets:
- **Specific scope:** One test file or subsystem
- **Clear goal:** Make these tests pass
- **Constraints:** Don't change other code
- **Expected output:** Summary of what you found and fixed

3. Dispatch in Parallel

```
// In Claude Code / AI environment
Task("Fix agent-tool-abort.test.ts failures")
Task("Fix batch-completion-behavior.test.ts failures")
Task("Fix tool-approval-race-conditions.test.ts failures")
// All three run concurrently
```

4. [[Review]] and Integrate

When [[agents]] return:
- Read each summary
- Verify fixes don't conflict
- Run full test suite
- Integrate all changes

Agent Prompt Structure

Good agent [[[[prompts]]]] are:
1. **Focused** - One clear problem domain
2. **Self-contained** - All context needed to understand the problem
3. **Specific about output** - What should the agent return?

```
Fix the 3 failing tests in src/agents/agent-tool-abort.test.ts:

1. "should abort tool with partial output capture" - expects 'interrupted at' in message
2. "should handle mixed completed and aborted tools" - fast tool aborted instead of completed
3. "should properly track pendingToolCount" - expects 3 results but gets 0

These are timing/race condition issues. Your task:

1. Read the test file and understand what each test verifies
2. Identify root cause - timing issues or actual bugs?
3. Fix by:
   - Replacing arbitrary timeouts with event-based waiting
   - Fixing bugs in abort implementation if found
   - Adjusting test expectations if testing changed behavior

Do NOT just increase timeouts - find the real issue.

Return: Summary of what you found and what you fixed.
```

Common Mistakes

** Too broad: **"Fix all the tests" - agent gets lost**
**** Specific:** "Fix agent-tool-abort.test.ts" - focused scope

** No context: **"Fix the race condition" - agent doesn't know where**
**** Context:** Paste the error messages and test names

** No constraints: **Agent might refactor everything**
**** Constraints:** "Do NOT change production code" or "Fix tests only"

** Vague output: **"Fix it" - you don't know what changed**
**** Specific:** "Return summary of root cause and changes"

When NOT to Use

Related failures: Fixing one might fix others - investigate together first
Need full context: Understanding requires seeing entire system
Exploratory debugging: You don't know what's broken yet
Shared state: [[Agents]] would interfere (editing same files, using same resources)

Real Example from Session

Scenario: 6 test failures across 3 files after major refactoring

Failures:
- agent-tool-abort.test.ts: 3 failures (timing issues)
- batch-completion-behavior.test.ts: 2 failures (tools not executing)
- tool-approval-race-conditions.test.ts: 1 failure (execution count = 0)

Decision: Independent domains - abort logic separate from batch completion separate from race conditions

Dispatch:

```
Agent 1 → Fix agent-tool-abort.test.ts
Agent 2 → Fix batch-completion-behavior.test.ts
Agent 3 → Fix tool-approval-race-conditions.test.ts
```

Results:
- Agent 1: Replaced timeouts with [[[[event]]]]-based waiting
- Agent 2: Fixed [[[[event]]]] structure bug (threadId in wrong place)
- Agent 3: Added wait for async tool execution to complete

[[Integration]]: All fixes independent, no conflicts, full suite green

Time saved: 3 problems solved in parallel vs sequentially

Key Benefits

1. **Parallelization** - Multiple investigations happen simultaneously
2. **Focus** - Each agent has narrow scope, less context to track
3. **Independence** - [[Agents]] don't interfere with each other
4. **Speed** - 3 problems solved in time of 1

Verification

After [[agents]] return:
1. **[[Review]] each summary** - Understand what changed
2. **Check for conflicts** - Did [[agents]] edit same code?
3. **Run full suite** - Verify all fixes work together
4. **Spot check** - [[Agents]] can make systematic errors

Real-World Impact

From debugging session (2025-10-03):
- 6 failures across 3 files
- 3 [[agents]] dispatched in parallel
- All investigations completed concurrently
- All fixes integrated successfully
- Zero conflicts between agent changes

document-design (Skill Module)

pdf-playground\skills\document-design\SKILL.md

name: Document design

description: This skill should be used when the user asks to "create a proposal", "design a report", "make a one-pager", "build a PDF", "create a newsletter", "design slides", "make event materials", "design a flyer", or needs help with print-ready HTML documents. Provides brand configuration, CSS patterns for print layout, and document design best practices.

version: 1.0.0

tags:

Document design

Create professional, print-ready HTML documents that export to PDF with customizable branding.

Brand configuration

Before creating documents, check for brand configuration in `.claude/pdf-[[playground]].local.md`. If found, use those settings. If not, use sensible defaults or ask the user for their brand colors.

Reading brand config

Look for `.claude/pdf-[[playground]].local.md` in the project root. Parse the YAML frontmatter:

```
---
brand:
  name: "Organization Name"
  tagline: "Tagline"
  website: "https://example.com"
  email: "contact@example.com"

colors:
  primary: "#CA3553"
  secondary: "#000000"
  background: "#FFFFFF"
  text: "#2d2a28"
  muted: "#666666"

fonts:
  heading: "Playfair Display"
  body: "Source Sans 3"

style:
  headingCase: "sentence"
  useOxfordComma: true
---
```

Default brand values

If no config exists, use these defaults:

- **Primary color**: `#CA3553` (red)
- **Secondary color**: `#000000` (black)
- **Heading font**: Playfair Display
- **Body font**: Source Sans 3
- **Heading case**: sentence case

Core principles

1. **Print-first design**: All documents target 8.5" × 11" letter size with proper margins
2. **Brand compliance**: Use colors and fonts from brand configuration
3. **Sentence case by default**: Unless brand config specifies "title" case
4. **Clean exports**: Documents must render correctly when printed to PDF

CSS variables

Generate CSS variables from brand config:

```
:root {
    --primary: [colors.primary];
    --secondary: [colors.secondary];
    --background: [colors.background];
    --text: [colors.text];
    --muted: [colors.muted];

    /* Derived colors */
    --primary-dark: [darken primary by 15%];
    --gray-100: #f5f4f2;
    --gray-200: #e8e6e3;
}
```

Print CSS fundamentals

Page [[setup]]

```
@page {
    size: 8.5in 11in;
    margin: 0;
}

@media print {
    body {
        -webkit-print-color-adjust: exact !important;
        print-color-adjust: exact !important;
    }
    .page {
        page-break-after: always;
        page-break-inside: avoid;
    }
}
```

Fixed page dimensions

```
.page {
    width: 8.5in;
    height: 11in;
    padding: 0.5in 0.75in;
    padding-bottom: 1in; /* Space for footer */
    position: relative;
    box-sizing: border-box;
    overflow: hidden;
}
```

Fixed footers

```
.page-footer {
    position: absolute;
    bottom: 0.4in;
    left: 0.75in;
    right: 0.75in;
    font-size: 9pt;
    border-top: 1px solid var(--gray-200);
    padding-top: 0.1in;
    background: var(--background);
}
```

Footer clearance (critical)

Content overlapping or touching the footer is a recurring issue.

Preferred layout — grid rows `auto 1fr auto`:

```
.page {
    display: grid;
    grid-template-rows: auto 1fr auto;
    overflow: hidden;
}
```

This makes the header and footer take their natural height, and the content fills the remaining space. No magic-number `calc()` needed — the footer clearance is structural.

Required safeguards:

1. Use `grid-rows: auto 1fr auto` on the page so content automatically gets the space between header and footer
2. Set `overflow: hidden` on the content container to prevent text bleeding past its bounds
3. Include `padding-bottom: 0.3in` (minimum) inside the content area as a buffer
4. Never use hardcoded `height: calc(...)` with magic numbers for header/footer heights — they drift when padding or font sizes change
5. After rendering, always screenshot and visually verify the bottom of the page before delivering
6. If content overflows, **reduce content** — never shrink the footer gap. Tighten the header first if you need more room.

Typography patterns

Font loading

```
@import url('https://fonts.googleapis.com/css2?family=[heading-font]:wght@400;600;700&family=[body-font]:wght@400;500;600;700&display=swap');

body {
    font-family: '[body-font]', Arial, sans-serif;
    font-size: 11pt;
    line-height: 1.6;
    color: var(--text);
}

h1, h2, h3 {
    font-family: '[heading-font]', Georgia, serif;
    font-weight: 700;
}
```

Heading styles

```
.section-title {
    font-size: 26pt;
    color: var(--secondary);
    margin-bottom: 0.25in;
}

.section-title::after {
    content: '';
    display: block;
    width: 0.5in;
    height: 3px;
    background: var(--primary);
    margin-top: 0.12in;
}
```

Common components

Cover page header

```
<header class="cover-header">
    <div class="logo-bar">
        <div class="logo-primary">[brand.name]</div>
    </div>
    <div class="cover-title-block">
        <div class="cover-eyebrow">[Document type] • [Date]</div>
        <h1 class="cover-title">[Title in configured case]</h1>
    </div>
</header>
```

Budget table

```
.budget-table thead {
    background: var(--secondary);
    color: white;
}

.budget-table tbody tr:last-child {
    background: var(--primary);
    color: white;
    font-weight: 700;
}
```

Highlight box

```
.highlight-box {
    background: linear-gradient(135deg, var(--primary) 0%, var(--primary-dark) 100%);
    color: white;
    padding: 0.3in;
}
```

Document creation workflow

1. **Check for brand config** in `.claude/pdf-[[playground]].local.md`
2. **Load** [[[[

Reference files

For detailed CSS patterns: `references/[[[[css-patterns]]]].md`

editorial-workflow (Skill Module)

journalism-core\skills\editorial-workflow\SKILL.md

name: editorial-workflow

description: Manage editorial workflows for newsrooms and publications. Use when tracking story assignments, managing deadlines, coordinating editorial calendars, or establishing handoff protocols between reporters and editors. Includes templates for assignment tracking, editorial calendars, and workflow documentation.

tags:

Editorial workflow management

Newsrooms run on systems. This [[skill]] provides templates and processes for tracking stories, managing deadlines, and coordinating between reporters and editors.

When to use

- Setting up story tracking systems
- Creating editorial calendars
- Establishing assignment and [[handoff]] protocols
- Managing multiple stories and deadlines
- Training new editors on workflow

Story [[status]] taxonomy

Standard statuses

[[Status]]	Meaning	Owner
Pitch	Idea submitted, awaiting approval	Reporter
Assigned	Approved, reporter working	Reporter
Reporting	Active reporting in progress	Reporter
Draft	First draft submitted	Editor
Edit	Editor making changes	Editor
Revision	Back with reporter for changes	Reporter
Copy	Copy editing stage	Copy editor
Final	Ready for [[[[publication]]]]	Editor
Scheduled	Queued for specific date	System
Published	Live	Complete
Hold	Paused, not abandoned	Editor
Kill	Story abandoned	Editor

[[Status]] flow

```
Pitch → Assigned → Reporting → Draft → Edit → Revision (loop) → Copy → Final → Scheduled → Published
                                         ↓
                                      Hold/Kill
```

Story assignment tracking

Assignment record

[[[

Editorial calendar

Calendar

```
## Editorial calendar: [Week/Month of Date]

### Monday [date]
| Time | Story | Reporter | Status | Notes |
|------|-------|----------|--------|-------|
| AM | [story] | [name] | [status] | |
| PM | [story] | [name] | [status] | |

### Tuesday [date]
[same format]

### Evergreen queue
| Story | Reporter | Ready date | Notes |
|-------|----------|------------|-------|
| [story] | [name] | [date] | [can run anytime after this] |

### Upcoming (next 2 weeks)
| Story | Reporter | Target date | Status |
|-------|----------|-------------|--------|
| [story] | [name] | [date] | [status] |
```

Planning meeting

For weekly/daily editorial meetings:

```
## Editorial meeting: [date]

### Today's lineup
| Story | Reporter | Status | Publish time | Notes |
|-------|----------|--------|--------------|-------|
| [story] | [name] | [status] | [time] | |

### Tomorrow's lineup
[same format]

### In progress (due this week)
| Story | Reporter | Due | Status | Blockers |
|-------|----------|-----|--------|----------|
| [story] | [name] | [date] | [status] | [any issues] |

### Pipeline (next 2+ weeks)
| Story | Reporter | Target | Status |
|-------|----------|--------|--------|
| [story] | [name] | [date] | [status] |

### Discussion items
- [Topic 1]
- [Topic 2]

### Action items
- [ ] [Action] - [Owner] - [Deadline]
```

Deadline management

Deadline types

Deadline	Purpose	Typical lead time
Reporting	Complete interviews/research	2-5 days before draft
First draft	Submit to editor	24-48 hours before edit
Final draft	After revisions	12-24 hours before copy
Art/photos	Visuals ready	Same as first draft
Copy edit	Language/style [[review]]	4-8 hours before publish
Publish	Goes live	Set time

Deadline tracking

[[[

```
## Edit notes: [story title]

**Overall:** [brief assessment - on track, needs work, etc.]

### Structural changes needed
- [Major reorg, missing elements, etc.]

### Specific fixes
- Line [X]: [issue/change]
- Line [X]: [issue/change]

### Questions to resolve
- [Question requiring reporter input]

### Deadline for revisions
[date/time]

### Discussion needed?
[Yes - let's talk before you revise / No - changes are clear]
```

[[[[Publication]]]] checklist

Pre-publish checklist

```
## Pre-publish: [story title]

### Content
- [ ] Headline is accurate and compelling
- [ ] Dek/subhead summarizes well
- [ ] Lede works
- [ ] All facts verified
- [ ] All sources attributed correctly
- [ ] Quotes are accurate
- [ ] Numbers double-checked

### Style
- [ ] AP Style applied
- [ ] Consistent voice throughout
- [ ] No typos or grammatical errors
- [ ] Appropriate length

### Legal/Ethics
- [ ] No defamation risk
- [ ] Sources of comment contacted
- [ ] Corrections from previous versions applied
- [ ] Embargoes respected

### Multimedia
- [ ] Photos have captions and credits
- [ ] Alt text added
- [ ] Video/audio plays correctly
- [ ] Graphics are accurate

### Metadata
- [ ] SEO headline filled
- [ ] Meta description written
- [ ] Tags/categories applied
- [ ] Author byline correct
- [ ] Publication date/time set

### Final sign-off
- [ ] Reporter reviewed final
- [ ] Editor approved
- [ ] Copy editor signed off
```

Post-publish checklist

```
## Post-publish: [story title]

### Immediate (within 1 hour)
- [ ] Story displays correctly
- [ ] Links work
- [ ] Images load
- [ ] No obvious errors
- [ ] Social posts scheduled/published

### Same day
- [ ] Monitor comments/feedback
- [ ] Watch for corrections needed
- [ ] Track initial metrics
- [ ] Respond to any issues

### Follow-up
- [ ] Note any updates needed
- [ ] Document corrections made
- [ ] Capture reader response
- [ ] Plan follow-up stories if warranted
```

Workflow documentation

Process documentation

For documenting your newsroom's

```
## Workflow: [Name of process]

### Purpose
[What this workflow accomplishes]

### When to use
[Triggers for this workflow]

### Roles involved
| Role | Responsibility |
|------|----------------|
| [Role] | [What they do] |

### Process steps

**Step 1: [Name]**
- Owner: [role]
- Action: [what happens]
- Output: [what's produced]
- Next: [what triggers next step]

**Step 2: [Name]**
[same format]

### Tools used
- [Tool]: [what it's used for]

### Templates/Forms
- [Link to templates]

### Exceptions
- [When normal process doesn't apply]
- [What to do instead]

### Escalation
- [When to escalate]
- [Who to contact]
```

Onboarding new staff

```
## Editorial workflow onboarding

### Day 1
- [ ] Access to CMS granted
- [ ] Added to editorial chat/channels
- [ ] Received style guide
- [ ] Introduced to direct editor

### Week 1
- [ ] Shadow experienced reporter
- [ ] Review workflow documentation
- [ ] Complete first assignment (low-stakes)
- [ ] Learn deadline expectations

### Month 1
- [ ] Independently handle standard assignments
- [ ] Know escalation paths
- [ ] Understand publication calendar
- [ ] Comfortable with all tools

### Key contacts
| Need | Contact |
|------|---------|
| Assignment questions | [name] |
| Technical issues | [name] |
| Editorial disputes | [name] |
| HR/Admin | [name] |
```

Good
*lllllll

electron-dev (Skill Module)

dev-toolkit\skills\electron-dev\SKILL.md

name: electron-dev

description: Electron desktop application development with React, TypeScript, and Vite. Use when building desktop apps, implementing IPC communication, managing windows/tray, handling PTY terminals, integrating WebRTC/audio, or packaging with electron-builder. Covers patterns from AudioBash, Yap, and Pisscord projects.

tags:

Electron desktop development

Patterns and practices for building production-quality Electron applications with React and TypeScript.

Security baseline (Electron 30+)

Electron's defaults have hardened over the past several releases. As of Electron 28+, `contextIsolation: true` and `sandbox: true` are the defaults for new BrowserWindow instances — most security advice from older guides assumed you had to opt in. You don't anymore; you have to opt OUT, and you should not.

Set explicitly anyway, so a config drift never weakens the security model:

```
const win = new BrowserWindow({
  webPreferences: {
    contextIsolation: true,        // default since 12, mandatory for any prod app
    sandbox: true,                  // default since 28; renderer runs sandboxed
    nodeIntegration: false,         // never enable in renderer
    webSecurity: true,              // never disable
    preload: path.join(__dirname, 'preload.cjs')
  }
});
```

Validate every IPC message in main. Don't trust the renderer.

Electron Fuses + ASAR integrity

Electron Fuses are package-time toggles baked into the binary. The two relevant for security distribution:

- `EnableEmbeddedAsarIntegrityValidation` — verifies the app.asar hash at runtime against a hash embedded in the binary. Defends against attackers swapping the asar contents post-install.
- `OnlyLoadAppFromAsar` — refuses to load app code from anywhere except the validated asar.

These are **opt-in**, not default. Enable both for production. Requires `@electron/asar` 3.1.0+ to generate the asar with embeddable integrity. electron-builder configures this via `electronFuses` in the build config; `@electron/fuses` does it programmatically.

CVE-2023-44402 (ASAR integrity bypass via filetype confusion) was the canonical motivation here — without integrity + only-load-from-asar, an attacker who can modify app files can swap behavior silently.

Common renderer-side risks

- **Preload script confusion** — only expose narrow, typed surfaces via `contextBridge.exposeInMainWorld`. Never re-export `ipcRenderer` itself; expose specific methods that map to specific channels.
- **`file://` IPC and navigation** — restrict navigation with `webContents.on('will-navigate', e => e.preventDefault())` for windows that shouldn't change URL. Deny `setWindowOpenHandler` requests by default; allow-list specific origins.
- **`shell.openExternal` with user input** — validate the URL scheme before opening. An attacker-controlled `file://` or `javascript:` URL hands them code execution.

Architecture patterns

Project structure

```
app/
 electron/
    main.cjs              # Main process (CommonJS required)
    preload.cjs           # Context bridge for secure IPC
    server.cjs            # Optional: WebSocket/HTTP server
 src/
    components/           # React components
    services/             # Business logic (API clients, Firebase)
    utils/                # Utilities (audio, formatting)
    types.ts              # TypeScript interfaces
    App.tsx               # Root component
    index.tsx             # React entry
 assets/                   # Icons, sounds, images
 package.json
 vite.config.ts
 electron-builder.yml      # Build configuration
```

IPC communication pattern

Main process (main.cjs):

```
const { ipcMain } = require('electron');

// Handle async requests from renderer
ipcMain.handle('action-name', async (event, args) => {
  try {
    const result = await someAsyncOperation(args);
    return { success: true, data: result };
  } catch (error) {
    return { success: false, error: error.message };
  }
});

// Send data to renderer
mainWindow.webContents.send('event-name', data);
```

Preload script (preload.cjs):

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('electron', {
  actionName: (args) => ipcRenderer.invoke('action-name', args),
  onEventName: (callback) => {
    const handler = (event, data) => callback(data);
    ipcRenderer.on('event-name', handler);
    return () => ipcRenderer.removeListener('event-name', handler);
  }
});
```

Renderer (React):

```
const result = await window.electron.actionName(args);

useEffect(() => {
  return window.electron.onEventName((data) => {
    setState(data);
  });
}, []);
```

System tray [[integration]]

```
const { Tray, Menu, nativeImage } = require('electron');

let tray = null;

function createTray() {
  const icon = nativeImage.createFromPath(path.join(__dirname, '../assets/tray-icon.png'));
  tray = new Tray(icon.resize({ width: 16, height: 16 }));

  tray.setToolTip('App Name');
  tray.setContextMenu(Menu.buildFromTemplate([
    { label: 'Show', click: () => mainWindow.show() },
    { label: 'Quit', click: () => app.quit() }
  ]));

  tray.on('click', () => {
    mainWindow.isVisible() ? mainWindow.hide() : mainWindow.show();
  });
}

// Hide to tray instead of closing
mainWindow.on('close', (event) => {
  if (!app.isQuitting) {
    event.preventDefault();
    mainWindow.hide();
  }
});
```

Global shortcuts

```
const { globalShortcut } = require('electron');

app.whenReady().then(() => {
  // Register with conflict detection
  const registered = globalShortcut.register('Alt+S', () => {
    mainWindow.webContents.send('shortcut-triggered', 'toggle-recording');
  });

  if (!registered) {
    console.error('Shortcut registration failed - conflict detected');
  }
});

app.on('will-quit', () => {
  globalShortcut.unregisterAll();
});
```

PTY terminal [[integration]] (node-pty)

```
const pty = require('node-pty');

const shell = process.platform === 'win32' ? 'powershell.exe' : process.env.SHELL || '/bin/bash';

const ptyProcess = pty.spawn(shell, [], {
  name: 'xterm-256color',
  cols: 80,
  rows: 24,
  cwd: process.env.HOME,
  env: process.env
});

ptyProcess.onData((data) => {
  mainWindow.webContents.send('terminal-data', { tabId, data });
});

ipcMain.on('terminal-write', (event, { tabId, data }) => {
  ptyProcess.write(data);
});

ipcMain.on('terminal-resize', (event, { tabId, cols, rows }) => {
  ptyProcess.resize(cols, rows);
});
```

Audio recording workflow

```
// Request microphone access
const stream = await navigator.mediaDevices.getUserMedia({
  audio: {
    echoCancellation: true,
    noiseSuppression: true,
    autoGainControl: true
  }
});

// Record audio
const mediaRecorder = new MediaRecorder(stream, { mimeType: 'audio/webm' });
const chunks: Blob[] = [];

mediaRecorder.ondataavailable = (e) => chunks.push(e.data);
mediaRecorder.onstop = async () => {
  const blob = new Blob(chunks, { type: 'audio/webm' });
  const base64 = await blobToBase64(blob);
  // Send to transcription API
};

mediaRecorder.start();
// Later: mediaRecorder.stop();
```

WebRTC patterns (PeerJS)

```
import Peer from 'peerjs';

const peer = new Peer(userId, {
  host: 'peerjs-server.com',
  port: 443,
  secure: true
});

// Answer incoming calls
peer.on('call', (call) => {
  call.answer(localStream);
  call.on('stream', (remoteStream) => {
    audioElement.srcObject = remoteStream;
  });
});

// Make outgoing calls
const call = peer.call(remoteUserId, localStream);
call.on('stream', (remoteStream) => {
  audioElement.srcObject = remoteStream;
});

// Screen sharing via replaceTrack (no renegotiation)
const screenStream = await navigator.mediaDevices.getDisplayMedia({ video: true });
const videoTrack = screenStream.getVideoTracks()[0];
const sender = peerConnection.getSenders().find(s => s.track?.kind === 'video');
await sender.replaceTrack(videoTrack);
```

Build configuration (electron-builder.yml)

```
appId: com.yourname.appname
productName: AppName
directories:
  output: release

win:
  target:
    - target: nsis
      arch: [x64]
  icon: assets/icon.ico

nsis:
  oneClick: false
  allowToChangeInstallationDirectory: true
  installerIcon: assets/icon.ico
  uninstallerIcon: assets/icon.ico

mac:
  target:
    - target: dmg
      arch: [x64, arm64]
  icon: assets/icon.icns
  hardenedRuntime: true
  gatekeeperAssess: false
  entitlements: build/entitlements.mac.plist
  entitlementsInherit: build/entitlements.mac.plist
  notarize:
    teamId: YOUR_APPLE_TEAM_ID

linux:
  target:
    - target: AppImage
      arch: [x64]
  icon: assets/icon.png

publish:
  provider: github
  owner: username
  repo: repo-name

extraResources:
  - from: "node_modules/node-pty/build/Release/"
    to: "node-pty/"
    filter: ["*.node"]
```

macOS notarization is required for distribution outside the App Store; Gatekeeper blocks unnotarized apps on first launch. Set the env vars `APPLE_ID`, `APPLE_APP_SPECIFIC_PASSWORD`, and `APPLE_TEAM_ID` (or use an App Store Connect API key) before running `npm run package`. electron-builder ≥ 24.13 handles notarization natively via the `mac.notarize` field; older versions require the `electron-notarize` afterSign hook.

For Windows, code signing with an EV cert is increasingly necessary to avoid SmartScreen warnings. electron-builder reads `CSC_LINK` (PFX) and `CSC_KEY_PASSWORD` env vars.

Common pitfalls

Stale closures in callbacks:

```
// Problem: State is stale in async callbacks
const [state, setState] = useState(initialValue);
peer.on('call', () => {
  console.log(state); // Always shows initialValue
});

// Solution: Use refs for async callback access
const stateRef = useRef(state);
useEffect(() => { stateRef.current = state; }, [state]);
peer.on('call', () => {
  console.log(stateRef.current); // Current value
});
```

Context isolation security:
- Never expose `ipcRenderer` directly to renderer
- Always use `contextBridge.exposeInMainWorld()`

- Validate all IPC arguments in main process
- Use TypeScript interfaces for IPC contracts

BrowserView is deprecated — use WebContentsView:

`BrowserView` was deprecated in Electron 30 (April 2024) and the underlying implementation has been replaced. `BrowserView` still works as a compatibility shim over `WebContentsView`, but new code should target `WebContentsView` directly. The constructors take the same `webPreferences` shape, so the migration is mostly mechanical. The differences worth knowing:

- `WebContentsView` is added via `win.contentView.addChildView(view)` instead of `win.addBrowserView(view)`
- Sizing is via `view.setBounds({x, y, width, height})` — no `setAutoResize`. You wire your own resize handlers if you want auto-resize.
- Z-order is the order of `addChildView` calls; `removeChildView` then re-`addChildView` to bring forward.

```
const { WebContentsView } = require('electron');

const view = new WebContentsView({
  webPreferences: { contextIsolation: true, sandbox: true }
});
view.webContents.loadURL('https://example.com');
mainWindow.contentView.addChildView(view);
view.setBounds({ x: 0, y: 80, width: 800, height: 520 });
```

See the official BrowserView → WebContentsView migration guide for edge cases (popups, devtools, focus management).

Cross-platform shell detection:

```
const shell = process.platform === 'win32'
  ? 'powershell.exe'
  : process.env.SHELL || '/bin/bash';

const shellArgs = process.platform === 'win32'
  ? ['-NoLogo']
  : [];
```

Development workflow

```
# Development (hot reload)
npm run electron:dev

# Production build
npm run electron:build

# Run built app locally
npx electron dist/

# Package for distribution
npm run package
```

eradicating-malware-from-infected-systems (Skill Module)

eradicating-malware-from-infected-systems\SKILL.md

name: eradicating-malware-from-infected-systems
description: Agent for malware eradication from infected systems — process kill, file removal, persistence cleanup. Use when the user asks to eradicating malware from infected systems.
version: 1.0.0
tags:
source: eradicating-malware-from-infected-systems

Eradicating Malware from Infected Systems

Role: Security workflow
Audience: security practitioners
Primary tools: Windows Process Termination, Windows Persistence Cleanup, Linux Persistence Cleanup, File Quarantine Best Practices, NIST SP 800-83, SANS PICERL

When to use

- The user wants to **eradicat** malware from infected systems.
- Capability: Agent for malware eradication from infected systems — process kill, file removal, persistence cleanup.
- Goal: complete the task.
- Trigger keywords: `eradicating, malware, infected, systems`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Terminate a malicious process by PID
2. Move malicious file to quarantine directory with metadata
3. Scan Windows persistence locations for suspicious entries
4. Scan Linux persistence locations for suspicious entries
5. List suspicious scheduled tasks

Key concepts

- Respond (RS.MI-02): Incidents are eradicated
- Eradication follows containment; remove all attacker artifacts
- Must address root cause, not just symptoms
- Verify eradication before moving to recovery
- Section 4: Handling Malware Incidents
- Phase 4 of SANS incident handling process
- Remove malware and attacker tools from all affected systems
- Identify and remediate the root cause

Tools & resources

- Windows Process Termination
- Windows Persistence Cleanup
- Linux Persistence Cleanup
- File Quarantine Best Practices
- NIST SP 800-83
- SANS PICERL
- Microsoft DART Eradication Guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `kill_malicious_process` — Terminate a malicious process by PID.
- `quarantine_file` — Move malicious file to quarantine directory with metadata.
- `scan_persistence_windows` — Scan Windows persistence locations for suspicious entries.
- `scan_persistence_linux` — Scan Linux persistence locations for suspicious entries.
- `clean_scheduled_tasks` — List suspicious scheduled tasks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `eradicating-malware-from-infected-systems`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

evaluating-threat-intelligence-platforms (Skill Module)

`evaluating-threat-intelligence-platforms\SKILL.md`

name: evaluating-threat-intelligence-platforms
description: Threat Intelligence Platform evaluation agent for MISP, OpenCTI, and ThreatConnect. Use when the user asks to evaluating threat intelligence platforms.
version: 1.0.0
tags:
source: evaluating-threat-intelligence-platforms

Evaluating Threat Intelligence Platforms

Role: Security workflow
Audience: security practitioners
Primary tools: MISP REST API, OpenCTI GraphQL API, ThreatConnect REST API, TAXII 2.1 API, TIP Evaluation Criteria Weights, Platform Comparison Matrix

When to use

- The user wants to **evaluat** threat intelligence platforms.
- Capability: Threat Intelligence Platform evaluation agent for MISP, OpenCTI, and ThreatConnect.
- Goal: complete the task.
- Trigger keywords: `evaluating, threat, intelligence, platforms`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test MISP API connectivity and basic operations
2. Test OpenCTI GraphQL API connectivity
3. Generate an evaluation scoring template for a TIP assessment
4. Generate a sample comparison report for common TIP platforms

Tools & resources

- MISP REST API
- OpenCTI GraphQL API
- ThreatConnect REST API
- TAXII 2.1 API
- TIP Evaluation Criteria Weights
- Platform Comparison Matrix

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_misp_api` — Test MISP API connectivity and basic operations.
- `test_opencti_api` — Test OpenCTI GraphQL API connectivity.
- `generate_evaluation_template` — Generate an evaluation scoring template for a TIP assessment.
- `generate_comparison_report` — Generate a sample comparison report for common TIP platforms.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `evaluating-threat-intelligence-platforms`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

executing-active-directory-attack-simulation (Skill Module)

executing-active-directory-attack-simulation\SKILL.md

name: executing-active-directory-attack-simulation
description: Active Directory attack simulation agent using Impacket and ldap3. Use when the user asks to executing active directory attack simulation.
version: 1.0.0
tags:
source: executing-active-directory-attack-simulation

Executing Active Directory Attack Simulation

Role: Security workflow
Audience: security practitioners
Primary tools: Key Impacket Modules

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **execut** active directory attack simulation.
- Capability: Active Directory attack simulation agent using Impacket and ldap3.
- Goal: complete the task.
- Trigger keywords: `executing, active, directory, attack, simulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Enumerate domain users via LDAP, returning accounts with SPNs and no preauth
2. Filter users with service principal names set (Kerberoastable)
3. Filter users with Kerberos pre-authentication disabled
4. Enumerate high-value group memberships via LDAP
5. Check if SMB signing is required on the target host
6. Compile AD assessment findings into a structured report

Tools & resources

- Key Impacket Modules

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `ldap_enum_users` — Enumerate domain users via LDAP, returning accounts with SPNs and no preauth.
- `find_kerberoastable` — Filter users with service principal names set (Kerberoastable).
- `find_asrep_roastable` — Filter users with Kerberos pre-authentication disabled.
- `enum_groups` — Enumerate high-value group memberships via LDAP.
- `check_smb_signing` — Check if SMB signing is required on the target host.
- `generate_report` — Compile AD assessment findings into a structured report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `executing-active-directory-attack-simulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

executing-phishing-simulation-campaign (Skill Module)

executing-phishing-simulation-campaign\SKILL.md

name: executing-phishing-simulation-campaign
description: Phishing simulation campaign agent using requests to interact with GoPhish API. Use when the user asks to executing phishing simulation campaign.
version: 1.0.0
tags:
source: executing-phishing-simulation-campaign

Executing Phishing Simulation Campaign

Role: Security workflow
Audience: security practitioners
Primary tools: GoPhishClient Class

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **execut** phishing simulation campaign.
- Capability: Phishing simulation campaign agent using requests to interact with GoPhish API.
- Goal: complete the task.
- Trigger keywords: `executing, phishing, simulation, campaign`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Clarify the objective for phishing simulation campaign
2. Gather inputs and required access
3. Execute the core workflow using skill scripts and references
4. Validate results and document findings
5. Recommend next actions

Tools & resources

- GoPhishClient Class

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_metrics` — Compute phishing simulation success metrics from campaign results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `executing-phishing-simulation-campaign`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

executing-plans (Skill Module)

superjawn\skills\executing-plans\SKILL.md

name: executing-plans
description: Use when you have a written implementation plan to execute in a separate session with review checkpoints
tags:

Executing Plans

Overview

Load plan, [[review]] critically, execute all tasks, [[[[report]]]] when complete.

Announce at start: "I'm using the superjawn:executing-plans [[skill]] to implement this plan."

Note: Tell your human partner that superjawn works much better with access to subagents. The quality of its work will be significantly higher if run on a platform with subagent support (such as Claude Code or Codex). If subagents are available, use superjawn:subagent-driven-development instead of this [[skill]].

Freshness check (when artifact is stale)

The plan was written at a point in time. **Default-skip.** Run only when one of these triggers fires:

- **Cross-session execution.** The plan was drafted in a prior session — different cwd, different transcript, or different day. If you wrote the plan yourself in this session, the freshness check is not needed.
- **External API/service touched.** Any task in the plan calls an external API, service, or file outside the repo (e.g., live HTTP, third-party SDK call, OS-managed config). Internal state can be assumed stable; external contracts cannot.
- **Working on main/master.** Current branch is `main` or `master`. Heightened drift risk because [[integration]] work assumes the branch is in sync, and other people landing commits can invalidate the plan's assumptions silently.

If none of the triggers fire, write one line into the execution journal and proceed:

```
[YYYY-MM-DD HH:MM] Freshness check skipped — none of the triggers fired (current session, no external APIs, on feature branch <name>).
```

When the check fires

Verify, in order, BEFORE running any tasks:

1. **Authoritative state.** For each external API/file the plan references, hit the real source — live curl, file read, version check. Confirm the contract still matches what the plan assumed.
2. **Codebase drift.** Grep that any function/module/path the plan names still exists at the expected location with the expected shape.
3. **Repo state.** `git log <plan-write-sha>..HEAD` if you can identify when the plan was written; otherwise `git log --since='1 week ago' --oneline`. Has anyone landed conflicting work since the plan was drafted?

Findings location: the execution journal

Findings land in a per-plan execution journal at:

```
.superpowers/exec-journal-<plan-slug>.md
```

Where `<plan-slug>` is the kebab-case basename of the plan file with the `.md` extension stripped (e.g., the plan file `[[2026-05-05-superjawn-batch-1]].md` produces the slug `[[2026-05-05-superjawn-batch-1]]`, and the journal lands at `.[[superpowers]]/exec-journal-[[2026-05-05-superjawn-batch-1]].md`). The journal is created on first run if it doesn't exist; entries are appended in chronological order. The directory `.[[superpowers]]/` is git-ignored by upstream convention.

One line per check:

```
[YYYY-MM-DD HH:MM] Freshness check: <PASS / FAIL> — <one-line summary>
```

If FAIL on any check, **stop and escalate to your human partner before implementing** — the plan may need revision.

Master-branch guardrail

If the trigger fired because you're on `main` or `master`, the freshness check ALSO requires explicit user consent before any implementation begins. Per the upstream rule "never start implementation on main/master without explicit user consent," ask first, then proceed. Document the consent in the journal as part of the freshness check entry.

The Process

Step 1: Load and [[Review]] Plan

1. Read plan file
2. [[Review]] critically - identify any questions or concerns about the plan
3. Run the freshness check (see "Freshness check (when artifact is stale)" above) — single decision, write the result line to the execution journal
4. If concerns or freshness FAIL: Raise them with your human partner before starting
5. If no concerns and freshness PASS or skipped: Create TodoWrite and proceed

Step 2: Execute Tasks

For each task:
1. Mark as in_progress
2. Follow each step exactly (plan has bite-sized steps)
3. Run verifications as specified
4. Mark as completed

Step 3: Complete Development

After all tasks complete and verified:
- Announce: "I'm using the superjawn:finishing-a-development-branch [[skill]] to complete this work."
- **REQUIRED SUB-[[SKILL]]:** Use superjawn:finishing-a-development-branch
- Follow that [[skill]] to verify tests, present options, execute choice

When to Stop and Ask for Help

STOP executing immediately when:
- Hit a blocker (missing dependency, test fails, instruction unclear)
- Plan has critical gaps preventing starting
- You don't understand an instruction
- Verification fails repeatedly

Ask for clarification rather than guessing.

When to Revisit Earlier Steps

Return to [[Review]] (Step 1) when:
- Partner updates the plan based on your feedback
- Fundamental approach needs rethinking

Don't force through blockers - stop and ask.

Remember

- [[Review]] plan critically first
- Follow plan steps exactly
- Run the freshness check at Step 1 — default-skip, but fires for cross-session plans, external APIs, or main/master branch work
- Don't skip verifications
- Reference [[skills]] when plan says to
- Stop when blocked, don't guess
- Never start implementation on main/master branch without explicit user consent (the freshness check enforces this when triggered)

[[Integration]]

Required workflow [[skills]]:
- **superjawn:using-git-worktrees** - REQUIRED: Set up isolated workspace before starting
- **superjawn:writing-plans** - Creates the plan this [[skill]] executes
- **superjawn:finishing-a-development-branch** - Complete development after all tasks

executing-red-team-engagement-planning (Skill Module)

executing-red-team-engagement-planning\SKILL.md

name: executing-red-team-engagement-planning
description: Agent for planning and documenting red team engagements with scope, rules, and attack paths. Use when the user asks to executing red team engagement planning.
version: 1.0.0
tags:
source: executing-red-team-engagement-planning

Executing Red Team Engagement Planning

Role: Security workflow
Audience: security practitioners
Primary tools: Engagement Plan Structure, NIST SP 800-115, TIBER-EU Framework, Relevant CVE/CWE References, Compliance Frameworks Impacting Scope

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **execut** red team engagement planning.
- Capability: Agent for planning and documenting red team engagements with scope, rules, and attack paths.
- Goal: complete the task.
- Trigger keywords: `executing, team, engagement, planning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Clarify the objective for red team engagement planning
2. Gather inputs and required access
3. Execute the core workflow using skill scripts and references
4. Validate results and document findings
5. Recommend next actions

Key concepts

- Reconnaissance (TA0043): Pre-engagement intelligence gathering
- T1595 - Active Scanning
- T1592 - Gather Victim Host Information
- T1589 - Gather Victim Identity Information
- T1590 - Gather Victim Network Information
- T1591 - Gather Victim Org Information
- Resource Development (TA0042): Infrastructure and capability preparation
- T1583 - Acquire Infrastructure

Tools & resources

- Engagement Plan Structure
- NIST SP 800-115
- TIBER-EU Framework
- Relevant CVE/CWE References
- Compliance Frameworks Impacting Scope

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_engagement_plan` — Generate a structured red team engagement plan.
- `generate_attack_tree` — Generate an attack tree for a given scenario.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `executing-red-team-engagement-planning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

executing-red-team-exercise (Skill Module)

executing-red-team-exercise\SKILL.md

name: executing-red-team-exercise
description: Red team exercise planning and ATT&CK technique tracking. Use when the user asks to executing red team exercise.
version: 1.0.0
tags:
source: executing-red-team-exercise

Executing Red Team Exercise

Role: Security workflow
Audience: security practitioners
Primary tools: Data Classes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **execut** red team exercise.
- Capability: Red team exercise planning and ATT&CK technique tracking agent.
- Goal: complete the task.
- Trigger keywords: `executing, team, exercise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load MITRE ATT&CK Enterprise techniques from local cache or upstream
2. Extract techniques used by a specific threat actor from ATT&CK data
3. Create a red team operation plan based on emulated threat actor TTPs
4. Mark a technique as executed and record detection status
5. Produce a detection gap analysis comparing executed vs detected techniques

Tools & resources

- Data Classes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_attack_techniques` — Load MITRE ATT&CK Enterprise techniques from local cache or upstream.
- `get_actor_techniques` — Extract techniques used by a specific threat actor from ATT&CK data.
- `build_operation_plan` — Create a red team operation plan based on emulated threat actor TTPs.
- `log_technique_execution` — Mark a technique as executed and record detection status.
- `generate_detection_gap_report` — Produce a detection gap analysis comparing executed vs detected techniques.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `executing-red-team-exercise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-active-directory-certificate-services-esc1 (Skill Module)

exploiting-active-directory-certificate-services-esc1\SKILL.md

name: exploiting-active-directory-certificate-services-esc1
description: Agent for detecting and testing AD CS ESC1 misconfiguration (enrollee supplies SAN). Use when the user asks to exploit active directory certificate services esc1.
version: 1.0.0
tags:
source: exploiting-active-directory-certificate-services-esc1

Exploiting Active Directory Certificate Services Esc1

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: ESC1, Certipy, certutil, PowerShell, Remediation, Key Research

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** active directory certificate services esc1.
- Capability: Agent for detecting and testing AD CS ESC1 misconfiguration (enrollee supplies SAN).
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, active, directory, certificate, services, esc1`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Enumerate AD CS certificate templates via certutil or Certipy
2. Check if a certificate template is vulnerable to ESC1
3. Run Certipy to find vulnerable certificate templates

Key concepts

- SpecterOps "Certified Pre-Owned" whitepaper by Will Schroeder and Lee Christensen (2021)
- CrowdStrike: Investigating Active Directory Certificate Services Abuse: ESC1
- BeyondTrust: ESC1 Attack - How to Detect and Mitigate
- Semperis: ESC1 Attack Explained
- HackTricks: AD CS Domain Escalation
- ESC1 is a misconfiguration, not a specific CVE
- Related: CVE-2022-26923 (Certifried) - AD CS machine account certificate abuse
- Microsoft KB5014754: Certificate-based authentication changes

Tools & resources

- ESC1
- Certipy
- certutil
- PowerShell
- Remediation
- Key Research
- CVE References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `enumerate_certificate_templates` — Enumerate AD CS certificate templates via certutil or Certipy.
- `check_esc1_vulnerable` — Check if a certificate template is vulnerable to ESC1.
- `run_certipy_find` — Run Certipy to find vulnerable certificate templates.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-active-directory-certificate-services-esc1`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-active-directory-with-bloodhound (Skill Module)

exploiting-active-directory-with-bloodhound\SKILL.md

name: exploiting-active-directory-with-bloodhound
description: Agent for Active Directory attack path analysis using BloodHound data collection. Use when the user asks to exploit active directory with bloodhound.
version: 1.0.0
tags:
source: exploiting-active-directory-with-bloodhound

Exploiting Active Directory with BloodHound

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: BloodHound, SharpHound, Neo4j Python Driver, NIST SP 800-171

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** active directory with bloodhound.
- Capability: Agent for Active Directory attack path analysis using BloodHound data collection.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, active, directory, bloodhound`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute SharpHound data collection
2. Execute bloodhound-python for cross-platform collection
3. Parse BloodHound JSON output for high-value findings
4. Execute Cypher query against BloodHound Neo4j database

Key concepts

- Limit information system access to authorized users
- Employ the principle of least privilege
- Control the flow of CUI per approved authorizations
- Authenticate organizational users and devices
- Use multi-factor authentication
- Employ replay-resistant authentication mechanisms
- Ensure 'Account lockout threshold' is set to 5 or fewer attempts
- Ensure 'Minimum password length' is set to 14 or more characters

Tools & resources

- BloodHound
- SharpHound
- Neo4j Python Driver
- NIST SP 800-171

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_sharphound` — Execute SharpHound data collection.
- `run_bloodhound_python` — Execute bloodhound-python for cross-platform collection.
- `analyze_bloodhound_json` — Parse BloodHound JSON output for high-value findings.
- `query_neo4j` — Execute Cypher query against BloodHound Neo4j database.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-active-directory-with-bloodhound`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-api-injection-vulnerabilities (Skill Module)

exploiting-api-injection-vulnerabilities\SKILL.md

name: exploiting-api-injection-vulnerabilities
description: Agent for testing API injection vulnerabilities (SQL, NoSQL, command injection). Use when the user asks to exploit api injection vulnerabilities.
version: 1.0.0
tags:
source: exploiting-api-injection-vulnerabilities

Exploiting API Injection Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: SQL Injection Payloads, Command Injection Payloads, Python requests Library, Error Signatures, Burp Suite API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** api injection vulnerabilities.
- Capability: Agent for testing API injection vulnerabilities (SQL, NoSQL, command injection).
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, injection, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm written authorization and engagement rules of engagement
2. Recon and identify candidates related to api injection vulnerabilities
3. Validate exploitability safely in the authorized scope
4. Demonstrate impact with minimal necessary proof
5. Document findings, reproduction steps, and remediation

Tools & resources

- SQL Injection Payloads
- Command Injection Payloads
- Python requests Library
- Error Signatures
- Burp Suite API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_parameter` — Test a single parameter with injection payloads.
- `scan_api_endpoint` — Scan an API endpoint with all injection categories.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-api-injection-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-bgp-hijacking-vulnerabilities (Skill Module)

`exploiting-bgp-hijacking-vulnerabilities\SKILL.md`

name: exploiting-bgp-hijacking-vulnerabilities
description: BGP hijacking assessment agent for monitoring route origin validation and RPKI status. Use when the user asks to exploit bgp hijacking vulnerabilities.
version: 1.0.0
tags:
source: exploiting-bgp-hijacking-vulnerabilities

Exploiting Bgp Hijacking Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** bgp hijacking vulnerabilities.
- Capability: BGP hijacking assessment agent for monitoring route origin validation and RPKI status.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, hijacking, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check RPKI validation status for a prefix-origin pair via RIPEstat
2. Get prefixes currently announced by an ASN via RIPEstat
3. Get current routing status for a prefix via RIPEstat
4. Query ROA (Route Origin Authorization) records for a prefix
5. Query BGP looking glass for current route advertisements
6. Run a full BGP hijacking resilience assessment for an organization's ASN

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_rpki_status` — Check RPKI validation status for a prefix-origin pair via RIPEstat.
- `get_announced_prefixes` — Get prefixes currently announced by an ASN via RIPEstat.
- `get_routing_status` — Get current routing status for a prefix via RIPEstat.
- `check_roas` — Query ROA (Route Origin Authorization) records for a prefix.
- `get_bgp_looking_glass` — Query BGP looking glass for current route advertisements.
- `assess_hijack_resilience` — Run a full BGP hijacking resilience assessment for an organization's ASN.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-bgp-hijacking-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-broken-function-level-authorization (Skill Module)

`exploiting-broken-function-level-authorization\SKILL.md`

name: exploiting-broken-function-level-authorization
description: Agent for testing Broken Function Level Authorization (BFLA) in APIs. Use when the user asks to exploit broken function level authorization.
version: 1.0.0
tags:
source: exploiting-broken-function-level-authorization

Exploiting Broken Function Level Authorization

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: OWASP API5, Testing Methodology, Python requests Library, Burp Suite, Response Analysis

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** broken function level authorization.
- Capability: Agent for testing Broken Function Level Authorization (BFLA) in APIs.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, broken, function, level, authorization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test if an endpoint is accessible with given credentials
2. Test if changing HTTP method bypasses authorization
3. Test if low-privilege user can access admin endpoints
4. Test function-level auth by accessing another user's admin functions

Tools & resources

- OWASP API5
- Testing Methodology
- Python requests Library
- Burp Suite
- Response Analysis

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_endpoint_access` — Test if an endpoint is accessible with given credentials.
- `test_method_switching` — Test if changing HTTP method bypasses authorization.
- `test_privilege_escalation` — Test if low-privilege user can access admin endpoints.
- `test_idor_via_function` — Test function-level auth by accessing another user's admin functions.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-broken-function-level-authorization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-broken-link-hijacking (Skill Module)

exploiting-broken-link-hijacking\SKILL.md

name: exploiting-broken-link-hijacking
description: Agent for detecting broken link hijacking vulnerabilities on websites. Use when the user asks to exploit broken link hijacking.
version: 1.0.0
tags:
source: exploiting-broken-link-hijacking

Exploiting Broken Link Hijacking

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Concept, Hijackable Platforms, Python requests, HTML Link Extraction, Domain Availability Check, GitHub API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** broken link hijacking.
- Capability: Agent for detecting broken link hijacking vulnerabilities on websites.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, broken, link, hijacking`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract all links from HTML content
2. Check if a URL is reachable and categorize its status
3. Determine if a broken link is hijackable
4. Scan a website for broken link hijacking opportunities

Tools & resources

- Concept
- Hijackable Platforms
- Python requests
- HTML Link Extraction
- Domain Availability Check
- GitHub API
- npm Registry
- Subdomain Takeover Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_links` — Extract all links from HTML content.
- `check_link_status` — Check if a URL is reachable and categorize its status.
- `check_hijackable` — Determine if a broken link is hijackable.
- `scan_website` — Scan a website for broken link hijacking opportunities.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-broken-link-hijacking`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-constrained-delegation-abuse (Skill Module)

exploiting-constrained-delegation-abuse\SKILL.md

name: exploiting-constrained-delegation-abuse
description: Agent for detecting and testing Kerberos constrained delegation abuse in AD. Use when the user asks to exploit constrained delegation abuse.
version: 1.0.0
tags:
source: exploiting-constrained-delegation-abuse

Exploiting Constrained Delegation Abuse

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Rubeus, Impacket getST.py, PowerView

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** constrained delegation abuse.
- Capability: Agent for detecting and testing Kerberos constrained delegation abuse in AD.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, constrained, delegation, abuse`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Find accounts with constrained delegation configured
2. Find computers writable for Resource-Based Constrained Delegation
3. Test S4U2Self/S4U2Proxy delegation abuse via Impacket

Key concepts

- ired.team: Kerberos Constrained Delegation abuse
- HackTricks: Constrained Delegation methodology
- GuidePoint Security: Delegating Like a Boss
- ManageEngine: Constrained delegation attacks explained
- SpecterOps: Delegation abuse research

Tools & resources

- Rubeus
- Impacket getST.py
- PowerView

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `find_constrained_delegation` — Find accounts with constrained delegation configured.
- `find_rbcd_targets` — Find computers writable for Resource-Based Constrained Delegation.
- `check_s4u_abuse` — Test S4U2Self/S4U2Proxy delegation abuse via Impacket.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-constrained-delegation-abuse`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-deeplink-vulnerabilities (Skill Module)

exploiting-deeplink-vulnerabilities\SKILL.md

name: exploiting-deeplink-vulnerabilities
description: Agent for testing deep link / custom URL scheme vulnerabilities in mobile apps. Use when the user asks to exploit deeplink vulnerabilities.
version: 1.0.0
tags:
source: exploiting-deeplink-vulnerabilities

Exploiting Deeplink Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Android Deep Links, iOS URL Schemes, Vulnerability Types, Attack Payloads, Frida, OWASP MASVS v2.0

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** deeplink vulnerabilities.
- Capability: Agent for testing deep link / custom URL scheme vulnerabilities in mobile apps.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, deeplink, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse AndroidManifest.xml for deep link configurations
2. Parse iOS Info.plist for URL scheme and universal link configurations
3. Construct and test a deep link injection payload
4. Test Android deep link via ADB

Tools & resources

- Android Deep Links
- iOS URL Schemes
- Vulnerability Types
- Attack Payloads
- Frida
- OWASP MASVS v2.0
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_android_manifest` — Parse AndroidManifest.xml for deep link configurations.
- `analyze_ios_plist` — Parse iOS Info.plist for URL scheme and universal link configurations.
- `test_deep_link_injection` — Construct and test a deep link injection payload.
- `check_adb_deep_link` — Test Android deep link via ADB.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-deeplink-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-excessive-data-exposure-in-api (Skill Module)

exploiting-excessive-data-exposure-in-api\SKILL.md

name: exploiting-excessive-data-exposure-in-api
description: Agent for detecting excessive data exposure (OWASP API3) in API responses. Use when the user asks to exploit excessive data exposure in api.
version: 1.0.0
tags:
source: exploiting-excessive-data-exposure-in-api

Exploiting Excessive Data Exposure in API

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: OWASP API3, Sensitive Field Categories, Testing Methodology, Python requests, Remediation Approaches, Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** excessive data exposure in api.
- Capability: Agent for detecting excessive data exposure (OWASP API3) in API responses.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, excessive, data, exposure`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze API response for excessive data exposure
2. Fetch API endpoint and analyze for data exposure
3. Count total fields in a JSON response
4. Compare response fields against expected OpenAPI spec fields

Tools & resources

- OWASP API3
- Sensitive Field Categories
- Testing Methodology
- Python requests
- Remediation Approaches
- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_response` — Analyze API response for excessive data exposure.
- `test_endpoint` — Fetch API endpoint and analyze for data exposure.
- `count_fields` — Count total fields in a JSON response.
- `compare_with_spec` — Compare response fields against expected OpenAPI spec fields.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-excessive-data-exposure-in-api`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-http-request-smuggling (Skill Module)

exploiting-http-request-smuggling\SKILL.md

name: exploiting-http-request-smuggling
description: HTTP request smuggling detection agent using raw socket and requests. Use when the user asks to exploit http request smuggling.
version: 1.0.0
tags:
source: exploiting-http-request-smuggling

Exploiting HTTP Request Smuggling

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Smuggling Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** http request smuggling.
- Capability: HTTP request smuggling detection agent using raw socket and requests.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, http, request, smuggling`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Identify the front-end/back-end HTTP architecture from response headers
2. Send a raw HTTP request and measure response time
3. Test for CL.TE smuggling via time-based detection
4. Test for TE.CL smuggling via differential response
5. Test for TE.TE smuggling with obfuscated Transfer-Encoding headers
6. Run the full HTTP request smuggling assessment

Tools & resources

- Smuggling Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `identify_architecture` — Identify the front-end/back-end HTTP architecture from response headers.
- `send_raw_request` — Send a raw HTTP request and measure response time.
- `test_clte_detection` — Test for CL.TE smuggling via time-based detection.
- `test_tecl_detection` — Test for TE.CL smuggling via differential response.
- `test_te_te_detection` — Test for TE.TE smuggling with obfuscated Transfer-Encoding headers.
- `run_assessment` — Run the full HTTP request smuggling assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-http-request-smuggling`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-idor-vulnerabilities (Skill Module)

exploiting-idor-vulnerabilities\SKILL.md

name: exploiting-idor-vulnerabilities
description: IDOR vulnerability detection agent using requests with multi-session comparison. Use when the user asks to exploit idor vulnerabilities.
version: 1.0.0
tags:
source: exploiting-idor-vulnerabilities

Exploiting IDOR Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: IDORTester Class

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** idor vulnerabilities.
- Capability: IDOR vulnerability detection agent using requests with multi-session comparison.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, idor, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm written authorization and engagement rules of engagement
2. Recon and identify candidates related to idor vulnerabilities
3. Validate exploitability safely in the authorized scope
4. Demonstrate impact with minimal necessary proof
5. Document findings, reproduction steps, and remediation

Tools & resources

- IDORTester Class

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-idor-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-insecure-data-storage-in-mobile (Skill Module)

exploiting-insecure-data-storage-in-mobile\SKILL.md

name: exploiting-insecure-data-storage-in-mobile
description: Agent for detecting insecure data storage in mobile applications. Use when the user asks to exploit insecure data storage in mobile.
version: 1.0.0
tags:
source: exploiting-insecure-data-storage-in-mobile

Exploiting Insecure Data Storage in Mobile

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: OWASP Mobile Top 10, Android Data Locations, SQLite Analysis, iOS Data Locations, Secure Storage Alternatives, OWASP MASVS v2.0

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** insecure data storage in mobile.
- Capability: Agent for detecting insecure data storage in mobile applications.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, insecure, data, storage, mobile`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan Android SharedPreferences XML files for sensitive data
2. Scan SQLite databases for unencrypted sensitive data
3. Scan app file storage for sensitive data
4. Pull application data via ADB for analysis

Key concepts

- Section 4.3.1: Data storage analysis for sensitive information at rest
- Section 4.3.2: Verification of encryption implementation for stored data
- Section 5.2: Data protection requirements for enterprise mobile apps

Tools & resources

- OWASP Mobile Top 10
- Android Data Locations
- SQLite Analysis
- iOS Data Locations
- Secure Storage Alternatives
- OWASP MASVS v2.0
- CWE Mappings
- Android Keystore Best Practices

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_shared_prefs` — Scan Android SharedPreferences XML files for sensitive data.
- `scan_sqlite_databases` — Scan SQLite databases for unencrypted sensitive data.
- `scan_file_storage` — Scan app file storage for sensitive data.
- `adb_pull_app_data` — Pull application data via ADB for analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-insecure-data-storage-in-mobile`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-insecure-deserialization (Skill Module)

exploiting-insecure-deserialization\SKILL.md

name: exploiting-insecure-deserialization
description: Insecure deserialization detection agent for identifying serialized data in HTTP traffic. Use when the user asks to exploit insecure deserialization.
version: 1.0.0
tags:
source: exploiting-insecure-deserialization

Exploiting Insecure Deserialization

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Serialization Markers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** insecure deserialization.
- Capability: Insecure deserialization detection agent for identifying serialized data in HTTP traffic.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, insecure, deserialization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Detect the serialization format from a string value
2. Scan response cookies for serialized data
3. Scan HTTP response body for serialized data patterns
4. Test for Java deserialization using a URLDNS-style detection payload
5. Test PHP deserialization with a role escalation payload
6. Test Python pickle deserialization with an OOB detection payload
7. Run a full deserialization assessment

Tools & resources

- Serialization Markers

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_serialization_format` — Detect the serialization format from a string value.
- `scan_cookies` — Scan response cookies for serialized data.
- `scan_response_body` — Scan HTTP response body for serialized data patterns.
- `test_java_deserialization` — Test for Java deserialization using a URLDNS-style detection payload.
- `test_php_deserialization` — Test PHP deserialization with a role escalation payload.
- `test_python_pickle` — Test Python pickle deserialization with an OOB detection payload.
- `run_assessment` — Run a full deserialization assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-insecure-deserialization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-ipv6-vulnerabilities (Skill Module)

exploiting-ipv6-vulnerabilities\SKILL.md

name: exploiting-ipv6-vulnerabilities
description: IPv6 vulnerability assessment agent using scapy for NDP/RA analysis. Use when the user asks to exploit ipv6 vulnerabilities.
version: 1.0.0
tags:
source: exploiting-ipv6-vulnerabilities

Exploiting Ipv6 Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Scapy Layers Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** ipv6 vulnerabilities.
- Capability: IPv6 vulnerability assessment agent using scapy for NDP/RA analysis.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, ipv6, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover IPv6 hosts by sending ICMPv6 Echo to all-nodes multicast
2. Capture and analyze Router Advertisement packets on the network
3. Identify rogue Router Advertisements from unknown sources
4. Check if ip6tables rules are configured (Linux only)
5. Check for IPv6 tunneling protocols (Teredo, 6to4, ISATAP)
6. Run complete IPv6 security assessment

Tools & resources

- Scapy Layers Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_ipv6_hosts` — Discover IPv6 hosts by sending ICMPv6 Echo to all-nodes multicast.
- `capture_router_advertisements` — Capture and analyze Router Advertisement packets on the network.
- `detect_rogue_ra` — Identify rogue Router Advertisements from unknown sources.
- `check_ipv6_firewall` — Check if ip6tables rules are configured (Linux only).
- `check_tunnel_protocols` — Check for IPv6 tunneling protocols (Teredo, 6to4, ISATAP).
- `generate_assessment` — Run complete IPv6 security assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-ipv6-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-jwt-algorithm-confusion-attack (Skill Module)

exploiting-jwt-algorithm-confusion-attack\SKILL.md

name: exploiting-jwt-algorithm-confusion-attack
description: Agent for testing JWT algorithm confusion vulnerabilities. Use when the user asks to exploit jwt algorithm confusion attack.
version: 1.0.0
tags:
source: exploiting-jwt-algorithm-confusion-attack

Exploiting JWT Algorithm Confusion Attack

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: JWT Structure, Algorithm Confusion Attack, None Algorithm Attack, JWT Header Injection Attacks, Python PyJWT Library, jwt_tool

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** jwt algorithm confusion attack.
- Capability: Agent for testing JWT algorithm confusion vulnerabilities.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, algorithm, confusion, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Decode a JWT token without verification
2. Create a JWT with alg:none (CVE in some libraries)
3. Forge JWT by signing with RSA public key as HMAC secret (alg confusion)
4. Analyze a JWT for common vulnerabilities

Tools & resources

- JWT Structure
- Algorithm Confusion Attack
- None Algorithm Attack
- JWT Header Injection Attacks
- Python PyJWT Library
- jwt_tool
- Remediation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `decode_jwt` — Decode a JWT token without verification.
- `forge_none_alg` — Create a JWT with alg:none (CVE in some libraries).
- `forge_hs256_with_public_key` — Forge JWT by signing with RSA public key as HMAC secret (alg confusion).
- `analyze_jwt` — Analyze a JWT for common vulnerabilities.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-jwt-algorithm-confusion-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-kerberoasting-with-impacket (Skill Module)

exploiting-kerberoasting-with-impacket\SKILL.md

name: exploiting-kerberoasting-with-impacket
description: Agent for Kerberoasting attacks using Impacket (T1558.003) — authorized testing. Use when the user asks to exploit kerberoasting with impacket.
version: 1.0.0
tags:
source: exploiting-kerberoasting-with-impacket

Exploiting Kerberoasting with Impacket

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Impacket, Rubeus, Hash Cracking, PowerShell Enumeration, Detection, Remediation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** kerberoasting with impacket.
- Capability: Agent for Kerberoasting attacks using Impacket (T1558.003) — authorized testing.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, kerberoasting, impacket`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute Impacket GetUserSPNs to extract service ticket hashes
2. Enumerate SPN accounts via LDAP query
3. Crack Kerberos TGS hashes with hashcat
4. Enumerate SPN accounts via PowerShell (domain-joined Windows)

Key concepts

- T1558.003** - Steal or Forge Kerberos Tickets: Kerberoasting
- Tactic: Credential Access (TA0006)
- Data Sources: Active Directory (Credential Request), Logon Session (Logon Session Metadata)
- Detection: MITRE DET0157
- T1087.002** - Account Discovery: Domain Account
- T1110.002** - Brute Force: Password Cracking
- NIST SP 800-53 Rev. 5** - IA-5: Authenticator Management (strong service account passwords)
- NIST SP 800-53 Rev. 5** - AC-6: Least Privilege (service account permissions)

Tools & resources

- Impacket
- Rubeus
- Hash Cracking
- PowerShell Enumeration
- Detection
- Remediation

- NIST References
- Windows Security Events

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_getuserspns` — Execute Impacket GetUserSPNs to extract service ticket hashes.
- `enumerate_spn_accounts_ldap` — Enumerate SPN accounts via LDAP query.
- `crack_with_hashcat` — Crack Kerberos TGS hashes with hashcat.
- `enumerate_powershell` — Enumerate SPN accounts via PowerShell (domain-joined Windows).

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-kerberoasting-with-impacket`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-mass-assignment-in-rest-apis (Skill Module)

exploiting-mass-assignment-in-rest-apis\SKILL.md

name: exploiting-mass-assignment-in-rest-apis
description: Agent for detecting mass assignment vulnerabilities in REST APIs. Use when the user asks to exploit mass assignment in rest apis.
version: 1.0.0
tags:
source: exploiting-mass-assignment-in-rest-apis

Exploiting Mass Assignment in Rest Apis

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: OWASP API3, Common Vulnerable Fields, Testing Methodology, Python Testing Script, Framework-Specific Vulnerabilities, Remediation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** mass assignment in rest apis.
- Capability: Agent for detecting mass assignment vulnerabilities in REST APIs.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, mass, assignment, rest, apis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm written authorization and engagement rules of engagement
2. Recon and identify candidates related to mass assignment in rest apis
3. Validate exploitability safely in the authorized scope
4. Demonstrate impact with minimal necessary proof
5. Document findings, reproduction steps, and remediation

Tools & resources

- OWASP API3
- Common Vulnerable Fields
- Testing Methodology
- Python Testing Script
- Framework-Specific Vulnerabilities
- Remediation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_baseline_response` — Get baseline response to understand normal object structure.
- `test_mass_assignment` — Test mass assignment by injecting extra fields in request body.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-mass-assignment-in-rest-apis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-ms17-010-eternalblue-vulnerability (Skill Module)

exploiting-ms17-010-eternalblue-vulnerability\SKILL.md

name: exploiting-ms17-010-eternalblue-vulnerability
description: Agent for detecting MS17-010 (EternalBlue) vulnerability — authorized testing only. Use when the user asks to exploit ms17 010 eternalblue vulnerability.
version: 1.0.0
tags:
source: exploiting-ms17-010-eternalblue-vulnerability

Exploiting Ms17 010 Eternalblue Vulnerability

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: CVE-2017-0144, Nmap NSE Script, SMB Protocol Basics, Python Socket Check, Remediation, Suricata Detection Rule

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** ms17 010 eternalblue vulnerability.
- Capability: Agent for detecting MS17-010 (EternalBlue) vulnerability — authorized testing only.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, ms17, eternalblue, vulnerability`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check if target is vulnerable to MS17-010 via SMB negotiation
2. Use nmap NSE script to check for MS17-010
3. Scan a network range for SMB port and MS17-010

Key concepts

- CVSS Score: ** 9.3 (Critical)
- Affected: ** Windows XP through Windows Server 2016
- CVE-2017-0144: Buffer overflow in SMBv1 server
- Allows remote attackers to execute arbitrary code via crafted packets
- Snort SID 41978: SMB EternalBlue exploit attempt
- Suricata: ET EXPLOIT Possible MS17-010 SMB Request

Tools & resources

- CVE-2017-0144
- Nmap NSE Script
- SMB Protocol Basics
- Python Socket Check
- Remediation
- Suricata Detection Rule
- CVE Details
- Affected Systems

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_ms17_010` — Check if target is vulnerable to MS17-010 via SMB negotiation.
- `nmap_ms17_010_check` — Use nmap NSE script to check for MS17-010.
- `scan_network` — Scan a network range for SMB port and MS17-010.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-ms17-010-eternalblue-vulnerability`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-nopac-cve-2021-42278-42287 (Skill Module)

exploiting-nopac-cve-2021-42278-42287\SKILL.md

name: exploiting-nopac-cve-2021-42278-42287
description: Agent for detecting noPac (CVE-2021-42278/42287) AD privilege escalation vulnerability. Use when the user asks to exploit nopac cve 2021 42278 42287.
version: 1.0.0
tags:
source: exploiting-nopac-cve-2021-42278-42287

Exploiting Nopac CVE 2021 42278 42287

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Vulnerability Overview, Prerequisites, Detection, Remediation, CVE References, Key Research

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** nopac cve 2021 42278 42287.
- Capability: Agent for detecting noPac (CVE-2021-42278/42287) AD privilege escalation vulnerability.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, nopac, 2021, 42278, 42287`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check for noPac vulnerability using Impacket noPac.py
2. Check the MachineAccountQuota via LDAP — needed for noPac
3. Check if KB5008380 (noPac patch) is installed
4. Check for sAMAccountName impersonation conditions

Key concepts

- Microsoft KB5008380: November 2021 Kerberos PAC fix
- Microsoft KB5008602: November 2021 sAMAccountName fix
- CrowdStrike: noPac Exploit - Latest Microsoft AD Flaw
- Fortinet: From User to Domain Admin in 60 Seconds
- TrustedSec: Attack Path Mapping Approach to CVEs 2021-42287/42278

Tools & resources

- Vulnerability Overview
- Prerequisites
- Detection
- Remediation
- CVE References
- Key Research

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_nopac_impacket` — Check for noPac vulnerability using Impacket noPac.py.
- `check_machineaccountquota` — Check the MachineAccountQuota via LDAP — needed for noPac.
- `check_patch_status` — Check if KB5008380 (noPac patch) is installed.
- `enumerate_sam_name_impersonation` — Check for sAMAccountName impersonation conditions.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-nopac-cve-2021-42278-42287`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-nosql-injection-vulnerabilities (Skill Module)

exploiting-nosql-injection-vulnerabilities\SKILL.md

name: exploiting-nosql-injection-vulnerabilities
description: Agent for testing NoSQL injection vulnerabilities in web applications. Use when the user asks to exploit nosql injection vulnerabilities.
version: 1.0.0
tags:
source: exploiting-nosql-injection-vulnerabilities

Exploiting Nosql Injection Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: NoSQLMap, nosqli, MongoDB documentation on query operators

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** nosql injection vulnerabilities.
- Capability: Agent for testing NoSQL injection vulnerabilities in web applications.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, nosql, injection, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Identify application technology stack (check for MongoDB, CouchDB indicators)
2. Map all input points accepting JSON data or query parameters
3. Submit operator payloads ($ne, $gt, $regex) in each parameter
4. Monitor responses for authentication bypass or data leakage
5. Test for JavaScript injection via $where operator
6. Document all vulnerable endpoints with proof-of-concept payloads
7. Confirm boolean-based injection by comparing true/false responses
8. Determine password/field length using $regex with length patterns
9. Extract characters one at a time using $regex "^"
10. Automate extraction with Python script using binary search

Tools & resources

- NoSQLMap
- nosqli
- MongoDB documentation on query operators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `test_get_injection` — Test NoSQL injection via GET parameter manipulation.
- `test_json_injection` — Test NoSQL injection via JSON body.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-nosql-injection-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-oauth-misconfiguration (Skill Module)

exploiting-oauth-misconfiguration\SKILL.md

name: exploiting-oauth-misconfiguration
description: OAuth 2.0 misconfiguration detection agent for testing redirect URI, state, and PKCE. Use when the user asks to exploit oauth misconfiguration.
version: 1.0.0
tags:
source: exploiting-oauth-misconfiguration

Exploiting Oauth Misconfiguration

Role: Authorized exploitation
Audience: red teamers and penetration testers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** oauth misconfiguration.
- Capability: OAuth 2.0 misconfiguration detection agent for testing redirect URI, state, and PKCE.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, oauth, misconfiguration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover OpenID Connect / OAuth configuration endpoints
2. Test redirect_uri validation with common bypass techniques
3. Test if the state parameter is required and validated
4. Test if PKCE (code_challenge) is required
5. Test if authorization codes can be reused
6. Test if additional scopes beyond authorization can be requested
7. Run the full OAuth security assessment

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_oidc_config` — Discover OpenID Connect / OAuth configuration endpoints.
- `test_redirect_uri_bypasses` — Test redirect_uri validation with common bypass techniques.
- `test_state_parameter` — Test if the state parameter is required and validated.
- `test_pkce_requirement` — Test if PKCE (code_challenge) is required.
- `test_code_reuse` — Test if authorization codes can be reused.
- `test_scope_escalation` — Test if additional scopes beyond authorization can be requested.
- `run_assessment` — Run the full OAuth security assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-oauth-misconfiguration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-prototype-pollution-in-javascript (Skill Module)

exploiting-prototype-pollution-in-javascript\SKILL.md

name: exploiting-prototype-pollution-in-javascript
description: Agent for detecting prototype pollution vulnerabilities in JavaScript applications. Use when the user asks to exploit prototype pollution in javascript.
version: 1.0.0
tags:
source: exploiting-prototype-pollution-in-javascript

Exploiting Prototype Pollution in Javascript

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Attack Vectors, Vulnerable Functions, Exploitation Impact, Testing with pp-finder, Burp Suite Extension, Remediation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** prototype pollution in javascript.
- Capability: Agent for detecting prototype pollution vulnerabilities in JavaScript applications.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, prototype, pollution, javascript`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test for prototype pollution via JSON body
2. Test for prototype pollution via query parameters
3. Scan JavaScript source for prototype pollution sinks

Tools & resources

- Attack Vectors
- Vulnerable Functions
- Exploitation Impact
- Testing with pp-finder
- Burp Suite Extension
- Remediation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_json_pollution` — Test for prototype pollution via JSON body.
- `test_query_pollution` — Test for prototype pollution via query parameters.
- `scan_source_code` — Scan JavaScript source for prototype pollution sinks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-prototype-pollution-in-javascript`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-race-condition-vulnerabilities (Skill Module)

exploiting-race-condition-vulnerabilities\SKILL.md

name: exploiting-race-condition-vulnerabilities
description: Agent for testing race condition (TOCTOU) vulnerabilities in web applications. Use when the user asks to exploit race condition vulnerabilities.
version: 1.0.0
tags:
source: exploiting-race-condition-vulnerabilities

Exploiting Race Condition Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: HTTP/2 Single-Packet Attack, Analysis Indicators, Common Vulnerable Operations, Remediation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** race condition vulnerabilities.
- Capability: Agent for testing race condition (TOCTOU) vulnerabilities in web applications.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, race, condition, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Send multiple identical requests concurrently to trigger race conditions
2. Analyze concurrent request results for race condition indicators
3. Execute race condition test

Tools & resources

- HTTP/2 Single-Packet Attack
- Analysis Indicators
- Common Vulnerable Operations
- Remediation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `send_concurrent_requests` — Send multiple identical requests concurrently to trigger race conditions.
- `analyze_results` — Analyze concurrent request results for race condition indicators.
- `test_race_condition` — Execute race condition test.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-race-condition-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-server-side-request-forgery (Skill Module)

exploiting-server-side-request-forgery\SKILL.md

name: exploiting-server-side-request-forgery
description: SSRF vulnerability detection agent with cloud metadata and filter bypass testing. Use when the user asks to exploit server side request forgery.
version: 1.0.0
tags:
source: exploiting-server-side-request-forgery

Exploiting Server Side Request Forgery

Role: Authorized exploitation
Audience: red teamers and penetration testers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** server side request forgery.
- Capability: SSRF vulnerability detection agent with cloud metadata and filter bypass testing.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, server, side, request, forgery`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Send an SSRF payload to a target endpoint and analyze the response
2. Test SSRF against all cloud metadata endpoints
3. Test localhost SSRF filter bypasses
4. Test non-HTTP protocol schemes (file://, dict://, gopher://)
5. Scan internal ports via SSRF to discover services
6. Run complete SSRF assessment

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_ssrf_endpoint` — Send an SSRF payload to a target endpoint and analyze the response.
- `test_cloud_metadata` — Test SSRF against all cloud metadata endpoints.
- `test_localhost_bypasses` — Test localhost SSRF filter bypasses.
- `test_protocol_schemes` — Test non-HTTP protocol schemes (file://, dict://, gopher://).
- `scan_internal_ports` — Scan internal ports via SSRF to discover services.
- `run_assessment` — Run complete SSRF assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-server-side-request-forgery`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-smb-vulnerabilities-with-metasploit (Skill Module)

`exploiting-smb-vulnerabilities-with-metasploit\SKILL.md`

name: exploiting-smb-vulnerabilities-with-metasploit
description: SMB vulnerability assessment agent using Impacket for enumeration and signing checks. Use when the user asks to exploit smb vulnerabilities with metasploit.
version: 1.0.0
tags:
source: exploiting-smb-vulnerabilities-with-metasploit

Exploiting SMB Vulnerabilities with Metasploit

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Metasploit

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** smb vulnerabilities with metasploit.
- Capability: SMB vulnerability assessment agent using Impacket for enumeration and signing checks.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, vulnerabilities, metasploit`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check if SMB port is open on the target
2. Enumerate SMB service information on a target host
3. Scan multiple targets for SMB services
4. Identify hosts where SMB signing is not required (relay targets)
5. Identify hosts accepting null SMB sessions
6. Generate SMB vulnerability assessment report
7. Expand a CIDR range to individual IPs (supports /24 and smaller)

Tools & resources

- Metasploit

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_smb_port` — Check if SMB port is open on the target.
- `enumerate_smb` — Enumerate SMB service information on a target host.
- `scan_network` — Scan multiple targets for SMB services.
- `find_relay_targets` — Identify hosts where SMB signing is not required (relay targets).
- `check_null_sessions` — Identify hosts accepting null SMB sessions.
- `generate_report` — Generate SMB vulnerability assessment report.
- `expand_cidr` — Expand a CIDR range to individual IPs (supports /24 and smaller).

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-smb-vulnerabilities-with-metasploit`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-sql-injection-vulnerabilities (Skill Module)

`exploiting-sql-injection-vulnerabilities\SKILL.md`

name: exploiting-sql-injection-vulnerabilities
description: SQL injection detection agent using requests for manual technique-based testing. Use when the user asks to exploit sql injection vulnerabilities.
version: 1.0.0
tags:
source: exploiting-sql-injection-vulnerabilities

Exploiting SQL Injection Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** sql injection vulnerabilities.
- Capability: SQL injection detection agent using requests for manual technique-based testing.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, injection, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Inject a single quote to detect SQL errors in the response
2. Test boolean-based blind SQLi with true/false conditions
3. Test time-based blind SQLi with sleep functions
4. Determine the number of columns for UNION-based injection
5. Identify the database engine using version functions
6. Run complete SQL injection assessment

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_error_based` — Inject a single quote to detect SQL errors in the response.
- `detect_boolean_based` — Test boolean-based blind SQLi with true/false conditions.
- `detect_time_based` — Test time-based blind SQLi with sleep functions.
- `detect_union_columns` — Determine the number of columns for UNION-based injection.
- `fingerprint_database` — Identify the database engine using version functions.
- `run_assessment` — Run complete SQL injection assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-sql-injection-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-sql-injection-with-sqlmap (Skill Module)

exploiting-sql-injection-with-sqlmap\SKILL.md

name: exploiting-sql-injection-with-sqlmap
description: sqlmap automation agent for orchestrating SQL injection scans via subprocess. Use when the user asks to exploit sql injection with sqlmap.
version: 1.0.0
tags:
source: exploiting-sql-injection-with-sqlmap

Exploiting SQL Injection with Sqlmap

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: sqlmap Flags Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** sql injection with sqlmap.
- Capability: sqlmap automation agent for orchestrating SQL injection scans via subprocess.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, injection, sqlmap`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Locate the sqlmap executable
2. Run sqlmap detection scan and parse results
3. Enumerate databases using sqlmap --dbs
4. Enumerate tables in a specific database
5. Dump rows from a specific table with optional column and row limit
6. Check current database user and DBA privileges

Tools & resources

- sqlmap Flags Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `find_sqlmap` — Locate the sqlmap executable.
- `run_detection_scan` — Run sqlmap detection scan and parse results.
- `enumerate_databases` — Enumerate databases using sqlmap --dbs.
- `enumerate_tables` — Enumerate tables in a specific database.
- `dump_table` — Dump rows from a specific table with optional column and row limit.
- `check_privileges` — Check current database user and DBA privileges.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-sql-injection-with-sqlmap`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-template-injection-vulnerabilities (Skill Module)

exploiting-template-injection-vulnerabilities\SKILL.md

name: exploiting-template-injection-vulnerabilities
description: Server-Side Template Injection (SSTI) detection agent using requests. Use when the user asks to exploit template injection vulnerabilities.
version: 1.0.0
tags:
source: exploiting-template-injection-vulnerabilities

Exploiting Template Injection Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Detection Payloads

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** template injection vulnerabilities.
- Capability: Server-Side Template Injection (SSTI) detection agent using requests.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, template, injection, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test all detection payloads against a parameter
2. Identify the specific template engine in use
3. Test Jinja2 RCE payloads
4. Test Twig (PHP) RCE payloads
5. Test Freemarker (Java) RCE payloads
6. Run complete SSTI assessment

Tools & resources

- Detection Payloads

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_ssti_detection` — Test all detection payloads against a parameter.
- `identify_engine` — Identify the specific template engine in use.
- `test_jinja2_rce` — Test Jinja2 RCE payloads.
- `test_twig_rce` — Test Twig (PHP) RCE payloads.
- `test_freemarker_rce` — Test Freemarker (Java) RCE payloads.
- `run_assessment` — Run complete SSTI assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-template-injection-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-type-juggling-vulnerabilities (Skill Module)

exploiting-type-juggling-vulnerabilities\SKILL.md

name: exploiting-type-juggling-vulnerabilities
description: Agent for testing type juggling vulnerabilities in PHP and loosely-typed applications. Use when the user asks to exploit type juggling vulnerabilities.
version: 1.0.0
tags:
source: exploiting-type-juggling-vulnerabilities

Exploiting Type Juggling Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Magic Hash Strings, Token/OTP Bypass, Testing with requests, PHP 8.0 Changes, Remediation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** type juggling vulnerabilities.
- Capability: Agent for testing type juggling vulnerabilities in PHP and loosely-typed applications.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, type, juggling, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm written authorization and engagement rules of engagement
2. Recon and identify candidates related to type juggling vulnerabilities
3. Validate exploitability safely in the authorized scope
4. Demonstrate impact with minimal necessary proof
5. Document findings, reproduction steps, and remediation

Tools & resources

- Magic Hash Strings
- Token/OTP Bypass
- Testing with requests
- PHP 8.0 Changes
- Remediation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_authentication_bypass` — Test authentication bypass via type juggling.
- `test_comparison_bypass` — Test loose comparison bypass for tokens/OTP.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-type-juggling-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-vulnerabilities-with-metasploit-framework (Skill Module)

exploiting-vulnerabilities-with-metasploit-framework\SKILL.md

name: exploiting-vulnerabilities-with-metasploit-framework
description: Agent for vulnerability exploitation workflow using Metasploit Framework — authorized testing. Use when the user asks to exploit vulnerabilities with metasploit framework.
version: 1.0.0
tags:
source: exploiting-vulnerabilities-with-metasploit-framework

Exploiting Vulnerabilities with Metasploit Framework

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Metasploit, Module Types, Common Exploit Modules, msfvenom, Industry Standards, Legal Considerations

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** vulnerabilities with metasploit framework.
- Capability: Agent for vulnerability exploitation workflow using Metasploit Framework — authorized testing.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, vulnerabilities, metasploit, framework`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Search Metasploit modules via msfconsole
2. Generate a Metasploit resource script for automated execution
3. Run nmap vulnerability scan to identify exploitable services
4. Map a CVE to known Metasploit modules

Key concepts

- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment
- OSSTMM v3: Open Source Security Testing Methodology Manual
- Always obtain written authorization before testing
- Define scope, rules of engagement, and emergency contacts
- Document all activities for legal protection
- Follow responsible disclosure for any new findings
- Comply with local computer misuse laws

Tools & resources

- Metasploit
- Module Types
- Common Exploit Modules
- msfvenom
- Industry Standards
- Legal Considerations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `search_modules` — Search Metasploit modules via msfconsole.
- `generate_rc_file` — Generate a Metasploit resource script for automated execution.
- `run_nmap_vuln_scan` — Run nmap vulnerability scan to identify exploitable services.
- `map_cve_to_module` — Map a CVE to known Metasploit modules.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-vulnerabilities-with-metasploit-framework`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-websocket-vulnerabilities (Skill Module)

exploiting-websocket-vulnerabilities\SKILL.md

name: exploiting-websocket-vulnerabilities
description: WebSocket vulnerability assessment agent using websockets and requests. Use when the user asks to exploit websocket vulnerabilities.
version: 1.0.0
tags:
source: exploiting-websocket-vulnerabilities

Exploiting Websocket Vulnerabilities

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: websockets Library Usage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** websocket vulnerabilities.
- Capability: WebSocket vulnerability assessment agent using websockets and requests.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, websocket, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Probe common WebSocket endpoint paths
2. Test if the WebSocket server validates the Origin header
3. Run complete WebSocket security assessment

Tools & resources

- websockets Library Usage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_ws_endpoints` — Probe common WebSocket endpoint paths.
- `test_origin_validation` — Test if the WebSocket server validates the Origin header.
- `run_assessment` — Run complete WebSocket security assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-websocket-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

exploiting-zerologon-vulnerability-cve-2020-1472 (Skill Module)

exploiting-zerologon-vulnerability-cve-2020-1472\SKILL.md

name: exploiting-zerologon-vulnerability-cve-2020-1472
description: Agent for detecting Zerologon (CVE-2020-1472) vulnerability — authorized testing only. Use when the user asks to exploit zerologon vulnerability cve 2020 1472.
version: 1.0.0
tags:
source: exploiting-zerologon-vulnerability-cve-2020-1472

Exploiting Zerologon Vulnerability CVE 2020 1472

Role: Authorized exploitation
Audience: red teamers and penetration testers
Primary tools: Vulnerability Overview, Attack Mechanism, Detection Tools, MS-NRPC Protocol, Remediation, CVE Details

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **exploit** zerologon vulnerability cve 2020 1472.
- Capability: Agent for detecting Zerologon (CVE-2020-1472) vulnerability — authorized testing only.
- Goal: validate and demonstrate vulnerabilities only in authorized environments.
- Trigger keywords: `exploiting, zerologon, vulnerability, 2020, 1472`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Use nmap script to check for Zerologon vulnerability
2. Check vulnerability using zerologon_tester.py from Impacket
3. Check if DC has Zerologon patches applied
4. Verify Netlogon secure channel is enforced

Key concepts

- T1078.002** - Valid Accounts: Domain Accounts
- CVE ID: CVE-2020-1472
- CVSS v3.1 Score: 10.0 (Critical)
- Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
- CWE: CWE-330 (Use of Insufficiently Random Values)
- Affected Protocol: MS-NRPC (Netlogon Remote Protocol)
- Patch: KB4571694 (August 11, 2020)
- Enforcement: February 9, 2021 update

Tools & resources

- Vulnerability Overview
- Attack Mechanism
- Detection Tools
- MS-NRPC Protocol
- Remediation
- CVE Details

- NIST References
- CISA References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_zerologon_nmap` — Use nmap script to check for Zerologon vulnerability.
- `check_zerologon_impacket` — Check vulnerability using zerologon_tester.py from Impacket.
- `check_patch_status` — Check if DC has Zerologon patches applied.
- `check_secure_channel` — Verify Netlogon secure channel is enforced.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `exploiting-zerologon-vulnerability-cve-2020-1472`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

extracting-browser-history-artifacts (Skill Module)

`extracting-browser-history-artifacts\SKILL.md`

name: extracting-browser-history-artifacts
description: Browser history artifact extraction agent using sqlite3 for Chrome/Firefox/Edge forensics. Use when the user asks to extracting browser history artifacts.
version: 1.0.0
tags:
source: extracting-browser-history-artifacts

Extracting Browser History Artifacts

Role: Security workflow
Audience: security practitioners

When to use

- The user wants to **extract** browser history artifacts.
- Capability: Browser history artifact extraction agent using sqlite3 for Chrome/Firefox/Edge forensics.
- Goal: complete the task.
- Trigger keywords: `extracting, browser, history, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Convert Chrome/WebKit timestamp (microseconds since 1601-01-01) to ISO UTC
2. Convert Firefox timestamp (microseconds since Unix epoch) to ISO UTC
3. Extract browsing history from Chrome/Edge History database
4. Extract downloads from Chrome/Edge History database
5. Extract cookies from Chrome Cookies database
6. Extract browsing history from Firefox places.sqlite
7. Extract cookies from Firefox cookies.sqlite
8. Export extracted data to CSV

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `chrome_time_to_utc` — Convert Chrome/WebKit timestamp (microseconds since 1601-01-01) to ISO UTC.
- `firefox_time_to_utc` — Convert Firefox timestamp (microseconds since Unix epoch) to ISO UTC.
- `extract_chrome_history` — Extract browsing history from Chrome/Edge History database.
- `extract_chrome_downloads` — Extract downloads from Chrome/Edge History database.
- `extract_chrome_cookies` — Extract cookies from Chrome Cookies database.
- `extract_firefox_history` — Extract browsing history from Firefox places.sqlite.
- `extract_firefox_cookies` — Extract cookies from Firefox cookies.sqlite.
- `export_to_csv` — Export extracted data to CSV.
- `generate_report` — Generate comprehensive browser forensics report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `extracting-browser-history-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

extracting-config-from-agent-tesla-rat (Skill Module)

`extracting-config-from-agent-tesla-rat\SKILL.md`

name: extracting-config-from-agent-tesla-rat
description: Agent for extracting configuration from Agent Tesla RAT samples (malware analysis). Use when the user asks to extracting config from agent tesla rat.
version: 1.0.0
tags:
source: extracting-config-from-agent-tesla-rat

Extracting Config from Agent Tesla Rat

Role: Security workflow
Audience: security practitioners
Primary tools: Agent Tesla Overview, String Extraction, Configuration Indicators, YARA Rule, File Hashing, VirusTotal API

When to use

- The user wants to **extract** config from agent tesla rat.
- Capability: Agent for extracting configuration from Agent Tesla RAT samples (malware analysis).
- Goal: complete the task.
- Trigger keywords: `extracting, config, agent, tesla`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Compute MD5, SHA1, SHA256 of a file
2. Extract ASCII and wide strings from binary
3. Extract SMTP configuration from string artifacts
4. Extract FTP exfiltration configuration
5. Extract Telegram bot exfiltration config
6. Try to decode base64-encoded configuration strings
7. Full analysis of suspected Agent Tesla sample

Key concepts

- MITRE ATT&CK Framework
- NIST SP 800-83 Guide to Malware Incident Prevention
- NIST SP 800-86 Guide to Integrating Forensic Techniques

Tools & resources

- Agent Tesla Overview
- String Extraction
- Configuration Indicators
- YARA Rule
- File Hashing
- VirusTotal API
- Sandbox Analysis
- Applicable Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compute_file_hashes` — Compute MD5, SHA1, SHA256 of a file.
- `extract_strings` — Extract ASCII and wide strings from binary.
- `find_smtp_config` — Extract SMTP configuration from string artifacts.
- `find_ftp_config` — Extract FTP exfiltration configuration.
- `find_telegram_config` — Extract Telegram bot exfiltration config.
- `decode_base64_strings` — Try to decode base64-encoded configuration strings.
- `analyze_sample` — Full analysis of suspected Agent Tesla sample.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `extracting-config-from-agent-tesla-rat`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

extracting-credentials-from-memory-dump (Skill Module)

`extracting-credentials-from-memory-dump\SKILL.md`

name: extracting-credentials-from-memory-dump
description: Memory dump credential extraction agent using volatility3 subprocess and pypykatz. Use when the user asks to extracting credentials from memory dump.
version: 1.0.0
tags:
source: extracting-credentials-from-memory-dump

Extracting Credentials from Memory Dump

Role: Security workflow
Audience: security practitioners
Primary tools: Volatility3 Plugins Used

When to use

- The user wants to **extract** credentials from memory dump.
- Capability: Memory dump credential extraction agent using volatility3 subprocess and pypykatz.
- Goal: complete the task.
- Trigger keywords: `extracting, credentials, memory, dump`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Verify memory dump exists and compute hash
2. Run a volatility3 plugin and return stdout
3. Identify OS version from memory dump
4. Find LSASS process PID from process list
5. Extract SAM hashes using windows.hashdump
6. Extract LSA secrets using windows.lsadump
7. Extract cached domain credentials using windows.cachedump
8. Run pypykatz against LSASS minidump or full memory for credential extraction

Tools & resources

- Volatility3 Plugins Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `verify_dump` — Verify memory dump exists and compute hash.
- `run_vol3` — Run a volatility3 plugin and return stdout.
- `get_os_info` — Identify OS version from memory dump.
- `find_lsass_pid` — Find LSASS process PID from process list.
- `extract_hashdump` — Extract SAM hashes using windows.hashdump.
- `extract_lsadump` — Extract LSA secrets using windows.lsadump.
- `extract_cachedump` — Extract cached domain credentials using windows.cachedump.
- `run_pypykatz` — Run pypykatz against LSASS minidump or full memory for credential extraction.
- `parse_pypykatz_creds` — Parse pypykatz JSON output into structured credential list.
- `search_cloud_keys` — Search memory strings for cloud credentials and auth tokens.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `extracting-credentials-from-memory-dump`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

extracting-iocs-from-malware-samples (Skill Module)

extracting-iocs-from-malware-samples\SKILL.md

name: extracting-iocs-from-malware-samples
description: IOC extraction agent using pefile, yara-python, and requests for VirusTotal validation. Use when the user asks to extracting iocs from malware samples.
version: 1.0.0
tags:
source: extracting-iocs-from-malware-samples

Extracting IOCS from Malware Samples

Role: Security workflow
Audience: security practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **extract** iocs from malware samples.
- Capability: IOC extraction agent using pefile, yara-python, and requests for VirusTotal validation.
- Goal: complete the task.
- Trigger keywords: `extracting, iocs, malware, samples`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute MD5, SHA-1, SHA-256 hashes of a file
2. Extract PE file metadata including imphash and compile time
3. Extract ASCII and Unicode strings from binary
4. Extract IPs, domains, URLs, emails from string list
5. Extract file paths, registry keys, and mutexes from strings
6. Scan file with YARA rules
7. Validate a single IOC against VirusTotal API v3
8. Defang an IOC for safe sharing

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Compute MD5, SHA-1, SHA-256 hashes of a file.
- `extract_pe_metadata` — Extract PE file metadata including imphash and compile time.
- `extract_strings` — Extract ASCII and Unicode strings from binary.
- `extract_network_iocs` — Extract IPs, domains, URLs, emails from string list.
- `extract_host_iocs` — Extract file paths, registry keys, and mutexes from strings.
- `run_yara_scan` — Scan file with YARA rules.
- `validate_ioc_virustotal` — Validate a single IOC against VirusTotal API v3.
- `defang_ioc` — Defang an IOC for safe sharing.
- `export_stix_bundle` — Build a minimal STIX 2.1 bundle from extracted IOCs.
- `export_csv` — Export IOCs to CSV for SIEM ingestion.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `extracting-iocs-from-malware-samples`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

extracting-memory-artifacts-with-rekall (Skill Module)

`extracting-memory-artifacts-with-rekall\SKILL.md`

name: extracting-memory-artifacts-with-rekall
description: Agent for extracting memory forensic artifacts using Rekall. Use when the user asks to extracting memory artifacts with rekall.
version: 1.0.0
tags:
source: extracting-memory-artifacts-with-rekall

Extracting Memory Artifacts with Rekall

Role: Security workflow
Audience: security practitioners
Primary tools: Rekall, Key Plugins, Hidden Process Detection, Malfind Output Fields, Command Line

When to use

- The user wants to **extract** memory artifacts with rekall.
- Capability: Agent for extracting memory forensic artifacts using Rekall.
- Goal: complete the task.
- Trigger keywords: `extracting, memory, artifacts, rekall`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create a Rekall session for a memory image
2. List all processes using pslist plugin
3. Detect hidden processes by comparing pslist vs psscan
4. Detect injected code using malfind plugin (VAD analysis)
5. List network connections using netscan plugin
6. List loaded DLLs for processes using dlllist plugin
7. List loaded kernel drivers using modules plugin
8. Analyze Virtual Address Descriptor tree for a process

Tools & resources

- Rekall
- Key Plugins
- Hidden Process Detection
- Malfind Output Fields
- Command Line

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `create_session` — Create a Rekall session for a memory image.
- `list_processes` — List all processes using pslist plugin.
- `find_hidden_processes` — Detect hidden processes by comparing pslist vs psscan.
- `detect_code_injection` — Detect injected code using malfind plugin (VAD analysis).
- `list_network_connections` — List network connections using netscan plugin.
- `list_dlls` — List loaded DLLs for processes using dlllist plugin.
- `check_drivers` — List loaded kernel drivers using modules plugin.
- `analyze_vad` — Analyze Virtual Address Descriptor tree for a process.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `extracting-memory-artifacts-with-rekall`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

extracting-windows-event-logs-artifacts (Skill Module)

`extracting-windows-event-logs-artifacts\SKILL.md`

name: extracting-windows-event-logs-artifacts
description: Windows Event Log artifact extraction agent using evtx library for EVTX parsing. Use when the user asks to extracting windows event logs artifacts.
version: 1.0.0
tags:
source: extracting-windows-event-logs-artifacts

Extracting Windows Event Logs Artifacts

Role: Security workflow
Audience: security practitioners

When to use

- The user wants to **extract** windows event logs artifacts.
- Capability: Windows Event Log artifact extraction agent using evtx library for EVTX parsing.
- Goal: complete the task.
- Trigger keywords: `extracting, windows, event, logs, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse an EVTX file and return list of event records
2. Filter records for critical security event IDs
3. Detect lateral movement indicators from logon events
4. Detect privilege escalation from group membership and special privilege events
5. Detect suspicious process creation events
6. Detect audit log clearing events
7. Detect persistence mechanisms from service and scheduled task events
8. Generate analysis summary statistics

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_evtx_file` — Parse an EVTX file and return list of event records.
- `filter_critical_events` — Filter records for critical security event IDs.
- `detect_lateral_movement` — Detect lateral movement indicators from logon events.
- `detect_privilege_escalation` — Detect privilege escalation from group membership and special privilege events.
- `detect_suspicious_processes` — Detect suspicious process creation events.
- `detect_log_clearing` — Detect audit log clearing events.
- `detect_persistence` — Detect persistence mechanisms from service and scheduled task events.
- `generate_summary` — Generate analysis summary statistics.
- `export_timeline_csv` — Export critical events as a CSV timeline.
- `analyze_evtx` — Run full EVTX analysis across multiple log files.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `extracting-windows-event-logs-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

fact-check-workflow (Skill Module)

journalism-core\skills\fact-check-workflow\SKILL.md

name: fact-check-workflow

description: Structured workflow for fact-checking claims in journalism. Use when verifying statements for publication, rating claims for fact-check articles, or building pre-publication verification processes. Includes claim extraction, evidence gathering, rating scales, and correction protocols.

tags:

[[Fact-check]] workflow

Fact-checking is systematic, not intuitive. This [[skill]] provides structure for claim verification, evidence documentation, and rating decisions.

When to use

- Pre-[[[[publication]]]] fact-checking of articles
- Dedicated [[fact-check]] stories (rating claims)
- Verifying source statements during reporting
- Building fact-checking protocols for a newsroom
- Training staff on verification
  [[[[[

Prioritizing claims

Priority	Criteria
High	Central to the story's thesis, easily checkable, high consequence if wrong
Medium	Supporting detail, takes more effort to verify
Low	Peripheral detail, commonly accepted, minimal consequence

Check high-priority claims first. Check all claims if time allows.

Step 2: Research the claim

Primary sources first

Claim type	Primary sources
Statistics	Original study, government data, survey methodology
Quotes	Audio/video recording, transcript, direct confirmation
Historical	Contemporary news accounts, official records
Scientific	Peer-reviewed research, expert consensus
Legal	Court documents, official filings
Financial	SEC filings, audited statements

Secondary source evaluation

If you must use secondary sources:
- How close are they to the original?
- Do they cite their sources?
- Do multiple independent sources confirm?
- Is there any contradicting coverage?

Research documentation

```
## Research for Claim: [brief description]

### Primary sources checked
| Source | What it says | Confirms/Contradicts |
|--------|--------------|----------------------|
| [source] | [finding] | [confirms/contradicts/partial] |

### Secondary sources checked
| Source | What it says | Reliability |
|--------|--------------|-------------|
| [source] | [finding] | [high/medium/low] |

### Gaps in evidence
- [What you couldn't find]
- [What you still need]
```

Step 3: Evidence gathering

Types of evidence

Evidence type	Strength	Notes
Official documents	Strong	Court records, government reports, filings
Primary data	Strong	Original datasets, your own analysis
Expert consensus	Strong	Multiple independent experts agree
On-record sources	Medium	Named source with direct knowledge
Contemporary accounts	Medium	News coverage from the time
Off-record sources	Weak	Use to guide reporting, not as evidence
Social media posts	Weak	Can be deleted, context matters

Evidence checklist

```
## Evidence for: [claim]

### Documentary evidence
- [ ] Government records
- [ ] Court documents
- [ ] Corporate filings
- [ ] Published research
- [ ] Official statements/press releases

### Human sources
- [ ] Direct witnesses
- [ ] Subject matter experts
- [ ] Involved parties (on record)
- [ ] Involved parties (for response)

### Data verification
- [ ] Original dataset obtained
- [ ] Methodology reviewed
- [ ] Calculations independently verified
- [ ] Sample size adequate

### Contradicting evidence
- [ ] Searched for conflicting sources
- [ ] Contradictions documented
- [ ] Discrepancies explained
```

Step 4: Contact sources

Right of response

Always contact:
- People/organizations being fact-checked
- Give specific claims you're checking
- Give reasonable deadline (24-48 hours minimum)
- Document their response (or non-response)

Source contact

lllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllll

```
Subject: Request for comment - [Publication] fact-check

Dear [Name],

I'm a [title] at [publication] working on a fact-check of [context].

Specifically, I'm examining this claim:

"[Exact claim being checked]"

I want to give you the opportunity to provide any evidence supporting this claim, clarify the context, or offer any corrections.

My deadline is [date/time]. Please let me know if you need more time.

[Your name]
[Contact info]
```

Document responses

```
## Source response log

### [Source name]
**Contacted:** [date/time, method]
**Deadline given:** [date/time]
**Response received:** [date/time] / No response
**Summary:** [what they said]
**Evidence provided:** [any documentation]
**Direct quote for publication:** "[quote]"
```

Step 5: Rating the claim

Standard rating scales

Binary (for internal fact-checking):
- Verified
- False
- Unverifiable

Graduated (for [[fact-check]] articles):

Rating	Criteria
True	Accurate and complete, nothing significant omitted
Mostly true	Accurate but needs context or minor clarification
Half true	Partially accurate but leaves out critical context
Mostly false	Contains some truth but overall misleading
False	Not accurate; contradicted by evidence
Pants on fire	Not accurate AND ridiculous (use sparingly)

Rating decision

[[[

```
## Fact-check record

**Claim:** [exact statement]
**Source:** [who said it, where, when]
**Checked by:** [your name]
**Date checked:** [date]

### Verification
**Rating:** [rating]
**Primary evidence:** [list with links/locations]
**Supporting evidence:** [list]
**Contradicting evidence:** [if any]

### Sources contacted
- [Name]: [response summary]
- [Name]: [no response as of date]

### Notes
[Any additional context, caveats, future considerations]

### Files
- [List of saved documents, screenshots, etc.]
```

Archiving evidence

- Save screenshots with timestamps (URLs can change)
- Archive web pages (Wayback Machine, Archive.today)
- Download documents (don't just link)
- Keep original files separate from your analysis

Step 7: Corrections

When to correct

Situation	Action
Factual error	Correct immediately, note correction
Missing context	Add context, may not need formal correction
Updated information	[[Update]], note "Updated: [date]"
Source disputes characterization	Evaluate claim, correct if warranted

Correction

[[[

Pre-[[[[publication]]]] checklist

Before any story publishes:

```
## Pre-publication fact-check

**Article:** [title]
**Reporter:** [name]
**Editor:** [name]
**Fact-checker:** [name, if separate]
**Publish date:** [date]

### Claims verified
| Claim | Status | Evidence | Notes |
|-------|--------|----------|-------|
| [claim 1] | Verified | [source] | |
| [claim 2] | Verified | [source] | |

### Sources contacted for comment
| Source | Contacted | Response |
|--------|-----------|----------|
| [name] | [date] | [received/no response] |

### Numbers and statistics
- [ ] All statistics sourced
- [ ] Calculations independently verified
- [ ] Context provided (per capita, adjusted for inflation, etc.)

### Quotes
- [ ] All quotes verified against recording/transcript
- [ ] Attribution is accurate
- [ ] Context preserved

### Names and titles
- [ ] All names spelled correctly
- [ ] Titles current and accurate
- [ ] Affiliations verified

### Legal review (if applicable)
- [ ] Defamation risk assessed
- [ ] All claims supported by evidence
- [ ] Response from subjects documented

### Sign-off
**Reporter:** [name, date]
**Editor:** [name, date]
**Fact-checker:** [name, date]
```

[[Fact-check]] article structure

For dedicated [[fact-check]] stories:

```
# [Headline: Claim being checked]

**Claim:** [Exact claim in quotes]
**Source:** [Who said it, where, when]
**Our rating:** [Rating with visual indicator]

## What was said
[Context of the claim, full quote, circumstances]

## What the evidence shows
[Present evidence for and against]

## The verdict
[Explanation of rating decision]

## Sources
[List all sources with links]

---
*Published: [date] | Updated: [date if applicable]*
```

The goal is accuracy.

finishing-a-development-branch (Skill Module)

superjawn\skills\finishing-a-development-branch\SKILL.md

name: finishing-a-development-branch

description: Use when implementation is complete, all tests pass, and you need to decide how to integrate the work - guides completion of development work by presenting structured options for merge, PR, or cleanup

tags:

Finishing a Development Branch

Overview

Guide completion of development work by presenting clear options and handling chosen workflow.

Core principle: Verify tests → Present options → Execute choice → Clean up.

Announce at start: "I'm using the finishing-a-development-branch [[skill]] to complete this work."

The Process

Step 1: Verify Tests

Before presenting options, verify tests pass:

```
# Run project's test suite
npm test / cargo test / pytest / go test ./...
```

If tests fail:

```
Tests failing (<N> failures). Must fix before completing:

[Show failures]

Cannot proceed with merge/PR until tests pass.
```

Stop. Don't proceed to Step 2.

If tests pass: Continue to Step 2.

Step 2: Determine Base Branch

```
# Try common base branches
git merge-base HEAD main 2>/dev/null || git merge-base HEAD master 2>/dev/null
```

Or ask: "This branch split from main - is that correct?"

Step 3: Present Options

Present exactly these 4 options:

```
Implementation complete. What would you like to do?

1. Merge back to <base-branch> locally
2. Push and create a Pull Request
3. Keep the branch as-is (I'll handle it later)
4. Discard this work

Which option?
```

Don't add explanation - keep options concise.

Step 4: Execute Choice

Option 1: Merge Locally

```
# Switch to base branch
git checkout <base-branch>

# Pull latest
git pull

# Merge feature branch
git merge <feature-branch>

# Verify tests on merged result
<test command>

# If tests pass
git branch -d <feature-branch>
```

Then: Cleanup worktree (Step 5)

Option 2: Push and Create PR

```
# Push branch
git push -u origin <feature-branch>

# Create PR
gh pr create --title "<title>" --body "$(cat <<'EOF'
## Summary
<2-3 bullets of what changed>

## Test Plan
- [ ] <verification steps>
EOF
)"
```

Then: Cleanup worktree (Step 5)

Option 3: Keep As-Is

[[[[Report]]]]: "Keeping branch . Worktree preserved at ."

Don't cleanup worktree.

Option 4: Discard

Confirm first:

```
This will permanently delete:
- Branch <name>
- All commits: <commit-list>
- Worktree at <path>

Type 'discard' to confirm.
```

Wait for exact confirmation.

If confirmed:

```
git checkout <base-branch>
git branch -D <feature-branch>
```

Then: Cleanup worktree (Step 5)

Step 5: Cleanup Worktree

For Options 1, 2, 4:

Check if in worktree:

```
git worktree list | grep $(git branch --show-current)
```

If yes:

```
git worktree remove <worktree-path>
```

For Option 3: Keep worktree.

Quick Reference

Option	Merge	Push	Keep Worktree	Cleanup Branch
1. Merge locally		-	-	
2. Create PR	-			-
3. Keep as-is	-	-		-
4. Discard	-	-	-	(force)

Common Mistakes

Skipping test verification
- **Problem:** Merge broken code, create failing PR
- **Fix:** Always verify tests before offering options

Open-ended questions
- **Problem:** "What should I do next?" → ambiguous
- **Fix:** Present exactly 4 structured options

Automatic worktree cleanup
- **Problem:** Remove worktree when might need it (Option 2, 3)
- **Fix:** Only cleanup for Options 1 and 4

No confirmation for discard
- **Problem:** Accidentally delete work
- **Fix:** Require typed "discard" confirmation

Red Flags

Never:
- Proceed with failing tests
- Merge without verifying tests on result
- Delete work without confirmation
- Force-push without explicit request

Always:
- Verify tests before offering options
- Present exactly 4 options
- Get typed confirmation for Option 4
- Clean up worktree for Options 1 & 4 only

[[Integration]]

Called by:
- **subagent-driven-development** (Step 7) - After all tasks complete
- **executing-plans** (Step 5) - After all batches complete

Pairs with:
- **using-git-worktrees** - Cleans up worktree created by that [[skill]]

foia-requests (Skill Module)

journalism-core\skills\foia-requests\SKILL.md

name: foia-requests

description: Freedom of Information Act (FOIA) and public records request workflows. Use when drafting records requests, tracking submissions, understanding exemptions, appealing denials, or managing large document productions. Essential for investigative journalists, researchers, and transparency advocates.

tags:

FOIA and public records requests

[[[

1. **Exempt from public records statute** (e.g., Massachusetts, Oklahoma, Oregon, Wyoming)
2. **Excluded from definition of public body** (e.g., Georgia, Minnesota)
3. **Covered by separate statute** (e.g., California)
4. **Allowed to set own policies** (e.g., Mississippi, New York)

Court decisions and attorneys general opinions in some states have held that the separation of powers doctrine prevents courts from enforcing public records statutes against the legislature.

State-by-state reference

State	Public Records Law	Legislative Exemptions/Notes
Alabama	AL Code § 36-12-40 et seq.	AL Code § 29-6-7.1 Legislative confidential communication
Alaska	AK Stat. § 40.25.110	AK Stat. § 24.20.100 Research and drafting services confidential
Arizona	AZ Rev. Stat. § 39-121.01	AZ Rev. Stat. § 41-1279.05 Confidential records of auditor general
Arkansas	AR Code Ann. § 25-19-101 et seq.	AR Code Ann. § 10-2-129; § 10-4-422; § 25-19-105(b)(7)
California	CA Govt. Code § 7920 et seq.	CA Govt. Code § 9070 et seq. Legislative Open Records Act
Colorado	CO Rev. Stat. Ann. § 24-72-200 et seq.	CO Rev. Stat. Ann. § 24-72-202(6); § 2-3-505(2)
Connecticut	CT Gen. Stat. § 1-200 et seq.	CT Gen. Stat. § 1-210(b)(19); § 52-146r
Delaware	29 Del. Laws, c. 100 § 10001 et seq.	§ 10002(o)(16 & 19) Exempt records; GA emails
Florida	FL Stat. § 119.01 et seq.	FL Stat. § 11.0431(2); § 11.26; § 15.07
Georgia	O.C.G.A. § 50-18-70 et seq.	General Assembly not included in definitions
Hawaii	HI Rev. Stat. § 92F-1 et seq.	HI Rev. Stat. § 84-12; § 92F-13(5); § 23G-4
Idaho	ID Stat. § 74-101 et seq.	ID Stat. § 74-109 Draft legislation exempt
Illinois	5 ILCS 140/1	5 ILCS 140/7(1)(f) Legislative documents exempt
Indiana	IN Code § 5-14-3-1 et seq.	IN Code § 5-14-3-4(b)(13) & (14) Staff work products exempt
Iowa	IA Code § 22.1 et seq.	IA Code § 2A.1(3); § 23.12
Kansas	K.S.A. § 45-215 et seq.	K.S.A. § 45-217(l)(3)(B); § 45-221(a)(20)-(22)
Kentucky	KRS § 61.870 et seq.	KRS § 7.117; § 7.119; § 7.120
Louisiana	LA Rev. Stat. § 44:1 et seq.	LA Rev. Stat. § 44:4(6); § 44:4.1; § 44:2
Maine	ME Rev. Stat. Tit. 1 § 400 et seq.	ME Rev. Stat. Tit. 1 § 402(3)(C) Legislative exception
Maryland	MD General Provisions Code § 4-101 et seq.	MD State Govt. Code § 2–1226
Massachusetts	MA Gen. Laws Ch. 66 § 1 et seq.	Ch. 66 § 18 General Court exempt
Michigan	MI Comp. Laws § 15.231	MI Comp. Laws § 4.1109 LSB confidentiality
Minnesota	MN Stat. § 13.03	MN Stat. § 3C.05; § 10.46; § 3.098
Mississippi	MS Code Ann. § 25-61-1 et seq.	MS Code Ann. § 25-61-17 Legislature regulates own records
Missouri	MO Rev. Stat. § 610.010 et seq.	MO Rev. Stat. § 610.010(6); § 610.021
Montana	MT Code Ann. § 2-6-1001 et seq.	N/A
Nebraska	NE Rev. Stat. § 84-712 et seq.	NE Rev. Stat. § 84-712.05(14)
Nevada	NV Rev. Stat. § 239.001 et seq.	NV Rev. Stat. § 41.071; § 218F.150
New Hampshire	NH Rev. Stat. § 91-A:1 et seq.	NH Rev. Stat. § 91-A:5(IX)
New Jersey	NJ Rev. Stat. § 47:1A-1 et seq.	NJ Rev. Stat. § 47:1A-1.1; § 52:13D-22(d)
New Mexico	NM Stat. § 14-2-1 et seq.	NM Stat. § 2-3-13 Services confidential
New York	NY PBO Article 6 § 84 et seq.	NY PBO Article 6 § 88; Article 4 § 74(c)
North Carolina	NC G.S. § 132-1 et seq.	NC G.S. § 120 et seq. Confidentiality of communications
North Dakota	ND Cent. Code § 44-04-18 et seq.	ND Cent. Code § 44-04-18.6
Ohio	OH Rev. Code § 149.43	OH Rev. Code § 101.30 Staff confidentiality
Oklahoma	OK Stat. Tit. 51 § 24A.1 et seq.	Legislature not a public body under OORA
Oregon	ORS § 192.001 et seq.	ORS § 192.311(6); § 192.355; § 171.405
Pennsylvania	65 Pa. Stat. § 67.101 et seq.	101 Pa. Code § 3.6
Rhode Island	R.I. Gen. Laws § 38-2-1 et seq.	R.I. Gen. Laws § 38-2-2(4)(K & M)
South Carolina	SC Code § 30-4-10 et seq.	SC Code § 30-4-40(a)(7 & 8)
South Dakota	SD Cod. Laws § 1-27-1 et seq.	SD Cod. Laws § 1-27-1.5(12 & 19); § 19-19-508
Tennessee	Tenn. Code Ann. § 10-7-503 et seq.	Tenn. Code Ann. § 3-12-105; § 3-12-106

State	Public Records Law	Legislative Exemptions/Notes
Texas	TX Stat. Gov. Code § 552.001 et seq.	§ 306.003; § 323.017-018; § 325.0195; § 552.106; § 552.111; § 552.146
Utah	UT Code § 63G-2-101	UT Code § 63G-2-305(19-22); § 63G-2-703; § 63G-2-208
Vermont	1 V.S.A. § 315 et seq.	2 V.S.A. 403(b)(1) Legislative council requests confidential
Virginia	VA Code § 2.2-3700 et seq.	VA Code § 30-28.18; § 2.2-3705.7(2)
Washington	RCW 42.56	RCW 1.08.027; RCW 40.14.180; RCW 42.56.280
West Virginia	WV Code § 29B	WV Code § 4-1A-6; § 4-1A-7; § 4-1A-12
Wisconsin	WI Stat. § 19.21 et seq.	WI Stat. § 13.91 et seq.; § 16.61(2)(b)(1)
Wyoming	WY Stat. Ann. § 16-4-201 et seq.	WY Stat. Ann. § 28-8-116

Source: National Conference of State Legislatures, Updated April 2025

NJ OPRA reforms (P.L. 2024, c. 16) — effective Sept. 3, 2024

Senate bill S2930 (signed June 5, 2024; effective Sept. 3, 2024) changed New Jersey's Open Public Records Act in five ways that make requesting harder:

- **Fee-shifting weakened.** Prevailing requesters now receive attorneys' fees only when a court finds the agency's denial was in bad faith or knowing and willful. Previously, prevailing requesters were entitled to fees in most successful suits.
- **Anonymous requesters lose standing.** Requesters must identify themselves by name and address; agencies may reject anonymous requests.
- **Deadline tiers introduced.** The standard 7-business-day response window remains, but agencies can extend timelines for "voluminous" or "complex" requests under defined criteria, and may treat certain commercial requests differently.
- **Special service charge presumption.** Agencies may impose extraordinary service fees more readily; requests for large data extracts, bulk records, or programmatic responses are presumed to justify a service charge above the standard copy fee.
- **Agencies can sue requesters.** Public agencies have new authority to seek injunctive relief against requesters whose requests the agency deems harassing, repeated, or filed in bad faith.

Practical effect for journalists: file under your byline and outlet (anonymous requests are now risky); narrow the scope to avoid triggering "voluminous" treatment; document the public-interest rationale up front because fee-shifting protection is weaker.

Sources: New Jersey P.L. 2024, c. 16 (S2930); New Jersey Press Association coverage; Lowenstein Sandler client advisory; HealthLaw Advisor analysis; Chasan Lamparello Mallon & Cappuzzo summary.

Drafting effective requests

Request

[[[

(Federal FOIA)

```
[NAMES]
[ADDRESS]
[CITY],[STATE] [ZIPCODE]
[EMAIL]
[PHONE]

[DATE]

FOIA Officer
[AGENCY]
[AGENCY_ADDRESS]

[STATUTE] Request

Dear FOIA Officer:

My name is [NAME] and I am a [TITLE] for [OUTLET]. Pursuant to the [STATUTE], I am writing to request the following:

- [RECORDS_REQUESTED1] between [START_DATE] to [END_DATE] from [SPECIFIC_OFFICES] including the following words/phrases [KEYWORD1], [KEYWORD2], [KEYWORD3], [KEYWORD4]
- [RECORDS_REQUESTED2] between [START_DATE] to [END_DATE] from [SPECIFIC_OFFICES] including the following words/phrases [KEYWORD1], [KEYWORD2], [KEYWORD3], [KEYWORD4]
- [RECORDS_REQUESTED3] between [START_DATE] to [END_DATE] from [SPECIFIC_OFFICES] including the following words/phrases [KEYWORD1], [KEYWORD2], [KEYWORD3], [KEYWORD4]

**INSTRUCTIONS REGARDING SEARCH:**
1. Instructions Regarding "Leads":
As required by the relevant case law, [AGENCY] should follow any leads it discovers during the conduct of its searches and perform additional searches when said leads indicate that records may be located in another system. Failure to follow clear leads is a violation of [STATUTE].

2. Request for Public Records:
Please search for any records even if they are already publicly available.

3. Request Regarding Attachments, Photographs and Other Visual Materials:
I request that any photographs or other visual materials responsive to my request be released to me in their original or comparable forms, quality and resolution. For example, if a photograph was taken digitally, or if [AGENCY] maintains a photograph digitally, I request disclosure of the original digital image file, not a reduced resolution version of that image file nor a printout and scan of that image file.

Likewise, if a photograph was originally taken as a color photograph, I request disclosure of that photograph as a color image, not a black and white image.

For video and audio files I request that they be provided in their complete, original and unedited format.

Please contact me at the number or email listed below for any clarification on these points.

4. Request for Duplicate Pages:
l request disclosure of any and all supposedly "duplicate" pages. Scholars analyze records not only for the information available on any given page, but also for the relationships between that information and information on pages surrounding it. As such, though certain pages may have been previously released to me, the existence of those pages within new context renders them functionally new pages. As such, the only way to properly analyze released information is to analyze that information within its proper context. Therefore, I request disclosure of all "duplicate" pages.

5. Request for Search of Records Transferred to Other Agencies:
I request that in conducting its search, [AGENCY] should disclose releasable records even if they are available publicly through other sources outside the [AGENCY] office. If [AGENCY] is unable to do so, it should disclose where specifically the relevant records are available.

6. Regarding Destroyed Records
If any records responsive or potentially responsive to my request have been destroyed, my request includes, but is not limited to, any and all records relating or referring to the destruction of those records. This includes, but is not limited to, any and all records relating or referring to the events leading to the destruction of those records.

INSTRUCTIONS REGARDING SCOPE AND BREADTH OF REQUESTS
Please interpret the scope of this request broadly. [AGENCY] is instructed to interpret the scope of this request in the most liberal manner possible short of an interpretation that would lead to a conclusion that the request does not reasonably describe the records sought.

EXEMPTIONS AND SEGREGABILITY

The FOIA Improvement Act of 2016 codified a foreseeable-harm standard at 5 U.S.C. § 552(a)(8)(A). An agency may withhold information only if it (i) reasonably foresees that disclosure would harm an interest protected by an exemption, or (ii) disclosure is prohibited by law. Embarrassment to officials, the possibility that disclosure might reveal errors or failures, and speculative or abstract fears are not legitimate grounds for withholding.

Even where an exemption properly applies to part of a record, the statute requires the agency to release all reasonably segregable non-exempt portions. If documents are denied in part or in whole, please specify which exemption(s) is (are) claimed for each passage or whole document denied.

Please provide a complete itemized inventory and a detailed factual justification of total or partial denial of documents. Specify the number of pages in each document and the total number of pages pertaining to this request. For "classified" material denied, please include the following information: the classification (confidential, secret or top secret); identity of the classifier; date or event for automatic declassification or classification review or downgrading; if applicable, identity of official authorizing extension of automatic declassification or review past six years; and,
```

if applicable, the reason for extended classification beyond six years.

In excising material, please "black out" the material rather than "white out" or "cut out." I expect, as provided by FOIA, that the remaining non-exempt portions of documents will be released.

Please release all pages regardless of the extent of excising, even if all that remains are the letterhead, page numbers, or administrative markings.

In addition, I ask that your agency exercise its discretion to release records which may be technically exempt, but where withholding serves no important public interest.

Format requested:
I request that any releases stemming from this request be provided to me in its original digital format (soft-copy), ideally through an online file transfer or digital portal system. Should this be impossible, material may be provided on a compact disk, flash drive, hard drive or other like media.

I request that you provide records as they become available, rather than all at once.

Please produce all records with administrative markings and pagination included.

Please send a memo (copy to me) to the appropriate units in your office to assure that no records related to this request are destroyed. Please advise of any destruction of records and include the date of and authority for such destruction.

Fee waiver request (for U.S. federal FOIA):
I am a [JOB_TITLE] for [CURRENT_OUTLET], covering [BEAT/TOPIC]. My reporting has appeared in [PRIOR_OUTLETS].

I request a waiver of all fees pursuant to 5 U.S.C. § 552(a)(4)(A)(iii), which provides that documents shall be furnished without charge "if disclosure of the information is in the public interest because it is likely to contribute significantly to public understanding of the operations or activities of the government and is not primarily in the commercial interest of the requester."

The records I have requested meet that standard. They bear directly on [SPECIFIC_GOVERNMENT_OPERATION], a matter of public concern. I have the intent and ability to disseminate this information to a meaningful audience through [PUBLICATION_CHANNEL]. My request is non-commercial; I will not resell the records or use them primarily for private gain. *See Judicial Watch, Inc. v. Rossotti*, 326 F.3d 1309, 1312 (D.C. Cir. 2003) (Congress amended FOIA to ensure that fee-waiver provisions are "liberally construed in favor of waivers for noncommercial requesters"). I incorporate by reference the explanation in the sections above.

Should the fee waiver be denied, I request to be categorized as a representative of the news media for fee purposes pursuant to 5 U.S.C. § 552(a)(4)(A)(ii)(II). The statute defines a representative of the news media as "any person or entity that gathers information of potential interest to a segment of the public, uses its editorial skills to turn the raw materials into a distinct work, and distributes that work to an audience" — a definition derived from the holding in *Nat'l Sec. Archive v. Dep't of Def.*, 880 F.2d 1387 (D.C. Cir. 1989). I qualify under this definition because I [DESCRIBE_REPORTING_PROCESS_AND_AUDIENCE].

If both the fee waiver and the news-media classification are denied, please provide an itemized estimate of charges in writing before processing further. I am willing to pay reasonable charges up to $[AMOUNT] only if such a denial is final.

I will appeal any denial of the fee waiver administratively and, if necessary, in court.

Expedited processing request (if applicable):
I request expedited processing because:
- [Urgent need to inform the public about actual or alleged government activity]
- [Imminent threat to life or physical safety]

As a reminder, [TIME_LIMIT_STATUTE] requires a response within [DAYS_NUMBER] days.

Contact information:
Please do not hesitate to contact me at [EMAIL] or [PHONE] if you have questions or need clarification about this request.

Thank you. I appreciate your time and attention to this matter.

Sincerely,
[Your Name]

Template Source: [Jason Leopold](https://www.dni.gov/files/documents/FOIA/DF_2023_00079_15_Oldest_Appeal_FOIA_Cases_cont.pdf)

Request drafting best practices

```
## Effective request strategies

### Be specific but not too narrow
BAD: "All documents about climate change"
BAD: "The email from John Smith on March 15, 2024 at 2:47 PM"
GOOD: "Emails between [Office] and [External Party] regarding [Topic] from [Date Range]"

### Use agency terminology
- Research how agency categorizes information
- Use their document type names (memos, briefings, reports)
- Reference specific programs or initiatives by official name
- Include relevant file numbers if known

### Define your terms
- Specify what you mean by "communications" (emails, texts, calls?)
- Define "records" (include drafts? attachments?)
- Clarify "regarding" vs "mentioning" vs "primarily about"

### Request specific record types
- Emails and email attachments
- Text messages and messaging apps
- Printed Correspondence (letters, faxes)
- Calendar entries and meeting invites
- Memoranda, briefing papers and presentations
- Contracts, purchase orders and invoices
- Meeting minutes and notes
- Policies and procedures
- Databases and datasets; schema for databases/datasets
```

Tracking and managing requests

Request tracking system

```
from dataclasses import dataclass, field
from datetime import date, timedelta
from enum import Enum
from typing import Optional

class RequestStatus(Enum):
    DRAFTED = "drafted"
    SUBMITTED = "submitted"
    ACKNOWLEDGED = "acknowledged"
    PROCESSING = "processing"
    PARTIAL_RESPONSE = "partial_response"
    COMPLETED = "completed"
    APPEALED = "appealed"
    LITIGATION = "litigation"
    CLOSED = "closed"

@dataclass
class FOIARequest:
    id: str
    agency: str
    date_submitted: date
    description: str
    status: RequestStatus = RequestStatus.DRAFTED

    # Tracking
    tracking_number: Optional[str] = None
    assigned_officer: Optional[str] = None
    estimated_completion: Optional[date] = None

    # Fees
    fee_estimate: Optional[float] = None
    fee_waiver_requested: bool = True
    fee_waiver_granted: Optional[bool] = None

    # Timeline
    acknowledgment_date: Optional[date] = None
    response_dates: list[date] = field(default_factory=list)

    # Documents
    pages_received: int = 0
    pages_withheld: int = 0
    exemptions_cited: list[str] = field(default_factory=list)

    # Appeals
    appeal_deadline: Optional[date] = None
    appeal_filed: Optional[date] = None

    @property
    def statutory_deadline(self) -> date:
        """Federal FOIA: 20 business days from submission."""
        # Simplified - should account for business days
        return self.date_submitted + timedelta(days=28)

    @property
    def is_overdue(self) -> bool:
        return date.today() > self.statutory_deadline and self.status in [
            RequestStatus.SUBMITTED,
            RequestStatus.ACKNOWLEDGED,
            RequestStatus.PROCESSING
        ]

# Tracking spreadsheet structure
TRACKING_COLUMNS = [
    'request_id',
    'agency',
    'date_submitted',
    'tracking_number',
    'description',
    'status',
    'statutory_deadline',
    'actual_response_date',
    'pages_received',
    'exemptions_cited',
    'fee_amount',
```

```
    'appeal_deadline',
    'notes'
]
```

Follow-up communication templates

```
## Status inquiry (after 20+ business days)

Subject: Status inquiry — FOIA request [tracking number]

Dear FOIA Officer:

I am writing to inquire about the status of my Freedom of Information Act request submitted on [DATE], assigned tracking number [NUMBER].

Under FOIA, agencies must respond within 20 business days. As of today, [X] business days have elapsed.

Please provide:
1. Current status of my request
2. Estimated completion date
3. Any fee estimates
4. Name of assigned processor

I can be reached at [contact info].

Sincerely,
[Name]
```

```
## Fee waiver appeal

Subject: Appeal of fee waiver denial — request [number]

Dear FOIA Appeals Officer:

I appeal the denial of my fee waiver request dated [DATE] for request [NUMBER].

The denial was improper because:

1. **Public interest**: [Explain how disclosure serves public interest]

2. **Ability to disseminate**: [Describe your platform/audience]
   - Publication: [name, circulation/readership]
   - Previous FOIA-based reporting: [examples]
   - Planned use of records: [specific plans]

3. **Commercial interest**: I have no commercial interest in these records. [Or: My commercial interest is minimal compared to public benefit because...]

4. **Comparison**: Similar requests have received fee waivers. See: [examples if available]

I request the fee waiver be granted or, alternatively, that fees be limited to [amount].

Sincerely,
[Name]
```

Handling responses

Response [[review]] checklist

```
## Initial response review

### Administrative check
- [ ] Response received by deadline?
- [ ] Tracking number matches
- [ ] Request description accurate
- [ ] All requested record types addressed

### Completeness assessment
- [ ] All date ranges covered?
- [ ] All offices/divisions searched?
- [ ] Search terms used listed?
- [ ] Any referrals to other agencies?

### Exemption analysis
For each exemption claimed:
- [ ] Specific exemption cited (e.g., "Exemption 6")
- [ ] Explanation provided for each withholding
- [ ] Foreseeable harm articulated?
- [ ] Segregable portions released?

### Document review
- [ ] Page count matches stated total
- [ ] Documents legible
- [ ] Redactions clearly marked
- [ ] Index provided (Vaughn index if applicable)

### Fee assessment
- [ ] Charges match estimate?
- [ ] Itemized breakdown provided?
- [ ] Fee category correct (media, educational, commercial, other)?
```

Redaction analysis

```
## Understanding redactions

### Types of redactions
- (b)(1) - National security [black bar with exemption]
- (b)(5) - Deliberative process [often overused]
- (b)(6) - Personal privacy [names, contact info]
- (b)(7)(A) - Law enforcement investigation
- (b)(7)(C) - Law enforcement privacy

### Challenging over-redaction
Questions to ask:
1. Is entire document withheld or just portions?
2. Are routine details redacted unnecessarily?
3. Is information already public elsewhere?
4. Is the exemption correctly applied?
5. Was foreseeable harm test applied?

### Building your case
- Compare with similar released documents
- Check if information is already public
- Note inconsistent redaction patterns
- Document segregability arguments
```

Appeals process

Federal FOIA appeal

[[[

Advanced strategies

Multi-agency requests

```
## Coordinated request strategy

When a topic spans multiple agencies:

1. **Map the agencies involved**
   - Which agencies have jurisdiction?
   - Which offices within agencies?
   - Are there inter-agency communications?

2. **Sequence requests strategically**
   - Start with agency most likely to release
   - Use released documents to refine subsequent requests
   - Reference other agencies' releases in appeals

3. **Cross-reference responses**
   - Compare what different agencies release
   - Note discrepancies in redactions
   - Use one agency's release to challenge another's withholding

4. **Track coordination**
   - Document all request/response dates
   - Note referrals between agencies
   - Build timeline of events from multiple sources
```

Document production management

```
## Large production workflow

For responses with 100+ pages:

### Organization system
/FOIA_[Agency]_[Topic]/
 01_Request_Materials/
    original_request.pdf
    acknowledgment.pdf
    correspondence/
 02_Responses/
    response_2024-01-15/
    response_2024-02-20/
    final_response/
 03_Analysis/
    document_index.xlsx
    redaction_log.xlsx
    key_documents/
 04_Appeals/
    appeal_2024-03-01.pdf
 05_Notes/
     research_notes.md

### Document indexing
For each document, track:
- Bates number (if assigned)
- Date of document
- Author/recipient
- Document type
- Subject/summary
- Exemptions applied
- Relevance rating (1-5)
- Follow-up needed?
```

free-apis-catalog (Skill Module)

`research-toolkit\skills\free-apis-catalog\SKILL.md`

name: free-apis-catalog

description: Use when suggesting APIs for a project, looking for free data sources, building weekend projects that need external data, or when the user needs weather, news, finance, sports, ML, or entertainment data without paid subscriptions

tags:

Free APIs catalog (journalism-curated)

A short list of free APIs that work for journalism, research, and weekend projects under current free-tier conditions. The value here is curation under verified 2026 free-tier limits — not coverage. For breadth, defer to the canonical aggregator below.

Canonical breadth reference

For a near-complete inventory of free public APIs across every category (1000+ entries, actively maintained), use:

- **public-apis/public-apis on GitHub** — https://github.com/public-apis/public-apis

Not archived, last commit verified 2026-05-09, 4,600+ commits, 6,000+ merged PRs. When you need an API outside the categories below, search there first.

This [[skill]] curates the journalism-relevant subset.

Recently sunset / changed (don't use)

API	[[Status]]	What to use instead
IEX Cloud	Fully retired 2024-08-31. https://iexcloud.org/	Alpha Vantage, Tiingo IEX, Financial Modeling Prep, Polygon
CrowdTangle	Sunset 2024-08-14	Meta Content Library (academic/non-profit-research only — most journalists ineligible). https://transparency.meta.com/researchtools/meta-content-library/
ProPublica Congress API	Retired 2024-07-10, repo archived. https://www.propublica.org/nerds/congress-api-[[update]]	api.congress.gov (Library of Congress, free with api.data.gov key)
X / Twitter API free tier	Effectively eliminated for new developers as of Feb 2026; pay-per-use only. Uncertain on exact figures (provider pricing page returned 402). Treat as paid.	No free replacement; closest free alternatives are Bluesky Jetstream + Mastodon
Reddit API	Free for research/non-commercial only since June 2023; paid for commercial use. https://www.reddit.com/wiki/api/	(Same — usable for journalism research, prohibited for monetized products)
Hugging Face Inference API	Rebranded to Inference Providers (router across 20+ providers). Free users now get $0.10/month in [[credits]], down from previous unmetered free tier for many models. https://huggingface.co/docs/inference-providers/pricing	Free tier still works for low-volume / [[embeddings]]; paid for production

Journalism-curated short list

All entries below verified against provider pages on 2026-05-09 unless marked **uncertain**. Always confirm rate limits against the provider's live developer docs before depending on them in production.

News monitoring

API	Free tier	Notes
GDELT 2.0	Fully free, no auth	Global news + knowledge graph, updates every 15 min, BigQuery-mirrored. Strong for cross-border / longitudinal monitoring.
NewsAPI	100 req/day, 24h delay, 1mo window, dev-only	Commercial use prohibited on free tier. Not usable in shipped products.
GNews	100 req/day, 30d window, 12h delay, non-commercial	10 articles/request cap.
Mediastack	100 req/month, 30-min delay	Functionally a demo.
NYT Article Search / Top Stories	Free with key, daily cap (uncertain — verify)	Solid for archival queries.

Social platforms (post-API-monetization landscape)

API	Free tier	Notes
Bluesky Jetstream	Public WebSocket firehose, no auth	Real-time public-post stream. 4 official instances. Caveat: not formally part of AT Protocol — no long-term stability commitment.
Bluesky AppView	Public read endpoints, 3000 req / 5min IP-based	Auth needed for write ops. Profiles, posts, search.
Mastodon	Per-instance, OAuth, public endpoints often unauth-readable	Federated — coverage is server-by-server.
Threads (Meta)	Free pricing not surfaced on docs page (uncertain — verify)	Read/post/reply/search/insights/webhooks.
Reddit	100 QPM authenticated, 10 QPM unauthenticated, non-commercial only	Adequate for OSINT/research, prohibited for monetized products.
Meta Content Library	Academic / non-profit researchers only	Most journalists need a university partner to qualify.
TikTok Research API	Academic-affiliated researchers only	Same eligibility gate as MCL.

Government / public records (US)

API	Free tier	Notes
api.data.gov umbrella key	One key for 25+ federal agencies, 450+ APIs	Gateway for NASA, NPS, NIH, USGS, FDA, EPA, LoC, etc. Use this first.
api.congress.gov	Free with api.data.gov key	Official Library of Congress legislative data. Replaces ProPublica Congress API.
GovInfo (GPO)	Uses api.data.gov key	Federal-[[[[publication]]]] search and full-text. New MCP server in public [[preview]] for LLM [[[[[

Financial / economic

API	Free tier	Notes
FRED (St. Louis Fed)	Free with key (**uncertain** — page 403'd; verify limits)	Primary reference for US macro data.
CoinGecko Demo	10k calls/mo, 30/min, free with key	Attribution required.
Alpha Vantage	25 requests/day	Free tier is now a demo, not a usable journalism data source. Migration target for IEX Cloud users — but expect to need a paid tier for any real workload.
Financial Modeling Prep	Free tier exists, paid for production	IEX Cloud successor.

Archive / preservation

API	Free tier	Notes
Wayback Machine	Free	Availability JSON, Memento, CDX server. Save Page Now exposes programmatic submit.
archive.today	Programmatic submit available; no public API docs (uncertain — WebFetch blocked)	Useful for redundancy alongside Wayback. Verify endpoint by direct curl.

AI / ML

API	Free tier	Notes
Hugging Face Inference Providers	$0.10/month free credit, then pay-as-you-go	2024 rebrand from "Inference API." Plain HF Inference now mostly CPU-only ([[embeddings]], BERT-class). PRO users get $2/month.
Cohere	Free trial key for development, rate-limited	Paid for production.

Sports

API	Free tier	Notes
TheSportsDB	Free at point of access	$9/mo premium for production key.
football-data.org	Free tier covers 12 major competitions	UCL, Premier League, La Liga, Serie A, Bundesliga, Ligue 1, etc.

Evaluating APIs from the catalog

When picking an API for a journalism project, check three things on the provider's developer docs:

Field	What to check
Auth	"No auth" = no registration. "Key required" = sign up + manage the key. OAuth = per-user flow.
Free-tier limits	Daily / monthly request caps, rate limits per minute, article delay (often 12–24h on free tiers), recency window.
Commercial restriction	Many "free" APIs prohibit commercial use. Even an internal newsroom tool that supports paid coverage may count. Read the ToS.
CORS	Matters for browser frontends, irrelevant for backend scripts and cron jobs.
Stability commitment	Some "free firehose" APIs (e.g., Bluesky Jetstream) have no long-term stability commitment. Don't depend on them in production without a fallback.

Weekend project ideas

- **News alerts:** GDELT or NewsAPI + keyword filter + Telegram bot (2–3 hours)
- **Sentiment signal:** News feed + Hugging Face sentiment model + score from −1 to +1 (note free-tier credit cap)
- **Weather [[dashboard]]:** Open-Meteo + historical data + forecast for target city
- **Earthquake monitor:** USGS FDSN feed + threshold filter + Slack notification
- **Federal dataset explorer:** api.data.gov key + cycle through agencies + index findings to a project digital archive

Usage tip

For any new project needing external data: check the journalism-curated list above first. If nothing fits, search public-apis/public-apis. Feed candidate API docs to an agent along with your specific use case — it'll spot rate-limit / commercial-use traps that a casual read would miss.

Last currency sweep

2026-05-09. Verified directly against provider pages where reachable; uncertain entries marked inline with the specific provider page that failed verification (403, ECONNREFUSED, login-walled). Re-verify rate-limit numbers before depending on them in production.

Curation under current free-tier conditions is what paid alternatives like RapidAPI's directory don't provide. Coverage is what public-apis/public-apis is for.

generating-threat-intelligence-reports (Skill Module)

generating-threat-intelligence-reports\SKILL.md

name: generating-threat-intelligence-reports
description: Threat intelligence report generation agent using jinja2 for template-based reporting. Use when the user asks to generating threat intelligence reports.
version: 1.0.0
tags:
source: generating-threat-intelligence-reports

Generating Threat Intelligence Reports

Role: Security workflow
Audience: security practitioners
Primary tools: TLP Levels

When to use

- The user wants to **generat** threat intelligence reports.
- Capability: Threat intelligence report generation agent using jinja2 for template-based reporting.
- Goal: complete the task.
- Trigger keywords: `generating, threat, intelligence, reports`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Map confidence level to ICD 203 language
2. Render a threat intelligence report from template and data
3. Validate that required fields are present for the report type
4. Run quality checks on rendered report
5. Load data, validate, render, and save the report

Tools & resources

- TLP Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `confidence_label` — Map confidence level to ICD 203 language.
- `render_report` — Render a threat intelligence report from template and data.
- `validate_report_data` — Validate that required fields are present for the report type.
- `quality_check` — Run quality checks on rendered report.
- `generate_report` — Load data, validate, render, and save the report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `generating-threat-intelligence-reports`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hardening-docker-containers-for-production (Skill Module)

hardening-docker-containers-for-production\SKILL.md

name: hardening-docker-containers-for-production
description: Agent for auditing Docker container security and applying CIS hardening. Use when the user asks to hardening docker containers for production.
version: 1.0.0
tags:
source: hardening-docker-containers-for-production

Hardening Docker Containers for Production

Role: Security workflow
Audience: security practitioners
Primary tools: Docker, Seccomp and AppArmor, NIST SP 800-190

When to use

- The user wants to **harde** docker containers for production.
- Capability: Agent for auditing Docker container security and applying CIS hardening.
- Goal: complete the task.
- Trigger keywords: `hardening, docker, containers, production`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List running Docker containers with details
2. Get detailed container configuration
3. Audit a container against CIS Docker Benchmark checks

Key concepts

- Use container-specific host OS (CoreOS, Flatcar, Bottlerocket)
- Segment container networks by sensitivity level
- Use container runtime with minimal attack surface
- Implement image signing and verification
- Harden container registries with access controls
- Monitor container runtime behavior for anomalies

Tools & resources

- Docker
- Seccomp and AppArmor
- NIST SP 800-190

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_running_containers` — List running Docker containers with details.
- `inspect_container` — Get detailed container configuration.
- `audit_container` — Audit a container against CIS Docker Benchmark checks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hardening-docker-containers-for-production`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hardening-docker-daemon-configuration (Skill Module)

hardening-docker-daemon-configuration\SKILL.md

name: hardening-docker-daemon-configuration
description: Agent for auditing and hardening Docker daemon configuration. Use when the user asks to hardening docker daemon configuration.
version: 1.0.0
tags:
source: hardening-docker-daemon-configuration

Hardening Docker Daemon Configuration

Role: Security workflow
Audience: security practitioners
Primary tools: Docker, daemon.json Location, Recommended daemon.json, File Permission Checks, NIST SP 800-190, Compliance Mappings

When to use

- The user wants to **harde** docker daemon configuration.
- Capability: Agent for auditing and hardening Docker daemon configuration.
- Goal: complete the task.
- Trigger keywords: `hardening, docker, daemon, configuration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Read Docker daemon.json configuration
2. Audit daemon.json against CIS benchmarks
3. Check Docker socket permissions
4. Audit Docker configuration file permissions

Key concepts

- Section 4.1.4: Configuration defects in container images
- Section 5.1: Image security - Content trust enforcement
- Section 5.3: Daemon hardening recommendations
- Rule 0: Keep host and Docker up to date
- Rule 1: Do not expose the Docker daemon socket
- Rule 3: Limit capabilities
- Rule 4: Add no-new-privileges flag
- Rule 5: Disable inter-container communication

Tools & resources

- Docker
- daemon.json Location
- Recommended daemon.json
- File Permission Checks
- NIST SP 800-190
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `read_daemon_config` — Read Docker daemon.json configuration.
- `audit_daemon_config` — Audit daemon.json against CIS benchmarks.
- `check_docker_socket` — Check Docker socket permissions.
- `check_docker_files` — Audit Docker configuration file permissions.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hardening-docker-daemon-configuration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hardening-linux-endpoint-with-cis-benchmark (Skill Module)

`hardening-linux-endpoint-with-cis-benchmark\SKILL.md`

name: hardening-linux-endpoint-with-cis-benchmark
description: Agent for auditing Linux endpoints against CIS Benchmark hardening controls. Use when the user asks to hardening linux endpoint with cis benchmark.
version: 1.0.0
tags:
source: hardening-linux-endpoint-with-cis-benchmark

Hardening Linux Endpoint with CIS Benchmark

Role: Security workflow
Audience: security practitioners
Primary tools: CIS Benchmark Sections, Key sysctl Parameters, Service Management, File Permissions, Automated Tools, Primary Standards

When to use

- The user wants to **harde** linux endpoint with cis benchmark.
- Capability: Agent for auditing Linux endpoints against CIS Benchmark hardening controls.
- Goal: complete the task.
- Trigger keywords: `hardening, linux, endpoint, benchmark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. CIS Section 1 — Filesystem Configuration
2. CIS Section 2 — Services
3. CIS Section 3 — Network Parameters
4. CIS Section 5 — Access, Authentication, Authorization
5. CIS Section 4 — Logging and Auditing
6. CIS Section 6 — System File Permissions

Key concepts

- NIST SP 800-123: Guide to General Server Security
- DISA STIG: Security Technical Implementation Guide for RHEL/Ubuntu

Tools & resources

- CIS Benchmark Sections
- Key sysctl Parameters
- Service Management
- File Permissions
- Automated Tools
- Primary Standards
- Compliance Mappings
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_filesystem_config` — CIS Section 1 — Filesystem Configuration.
- `check_services` — CIS Section 2 — Services.
- `check_network_parameters` — CIS Section 3 — Network Parameters.
- `check_access_auth` — CIS Section 5 — Access, Authentication, Authorization.
- `check_audit_logging` — CIS Section 4 — Logging and Auditing.
- `check_file_permissions` — CIS Section 6 — System File Permissions.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hardening-linux-endpoint-with-cis-benchmark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hardening-windows-endpoint-with-cis-benchmark (Skill Module)

`hardening-windows-endpoint-with-cis-benchmark\SKILL.md`

name: hardening-windows-endpoint-with-cis-benchmark
description: Agent for auditing Windows endpoints against CIS Benchmark hardening controls. Use when the user asks to hardening windows endpoint with cis benchmark.
version: 1.0.0
tags:
source: hardening-windows-endpoint-with-cis-benchmark

Hardening Windows Endpoint with CIS Benchmark

Role: Security workflow
Audience: security practitioners
Primary tools: CIS Benchmark Sections, Key Registry Settings, Password Policy Settings, Advanced Audit Policy, GPO Export and Analysis, Automated Tools

When to use

- The user wants to **harde** windows endpoint with cis benchmark.
- Capability: Agent for auditing Windows endpoints against CIS Benchmark hardening controls.
- Goal: complete the task.
- Trigger keywords: `hardening, windows, endpoint, benchmark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run PowerShell command and return output
2. CIS 1.1 — Password Policy
3. CIS 17 — Advanced Audit Policy
4. CIS 9 — Windows Firewall
5. CIS 2.3 — Security Options
6. CIS 18 — Windows Features

Key concepts

- Publisher: Center for Internet Security (CIS)
- Scope: 400+ security configuration recommendations organized into 19 sections
- Profiles: Level 1 (Corporate), Level 2 (High Security), BitLocker (BL), Next Generation Windows Security (NG)
- Scope: Server-specific recommendations for Member Server and Domain Controller roles
- Profiles: Level 1 (MS/DC), Level 2 (MS/DC)
- Relevance: Foundational server hardening guidance that CIS benchmarks operationalize
- Key sections: OS hardening, user account management, resource access controls
- CIS Controls v8: Maps to Implementation Group (IG) 1, 2, and 3 for prioritized implementation

Tools & resources

- CIS Benchmark Sections
- Key Registry Settings
- Password Policy Settings
- Advanced Audit Policy
- GPO Export and Analysis
- Automated Tools
- Primary Standards
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_ps` — Run PowerShell command and return output.
- `check_password_policy` — CIS 1.1 — Password Policy.
- `check_audit_policy` — CIS 17 — Advanced Audit Policy.
- `check_windows_firewall` — CIS 9 — Windows Firewall.
- `check_security_options` — CIS 2.3 — Security Options.
- `check_windows_features` — CIS 18 — Windows Features.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hardening-windows-endpoint-with-cis-benchmark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-advanced-persistent-threats (Skill Module)

`hunting-advanced-persistent-threats\SKILL.md`

name: hunting-advanced-persistent-threats
description: APT threat hunting agent using MITRE ATT&CK, attackcti, and osquery. Use when the user asks to hunt for advanced persistent threats.
version: 1.0.0
tags:
source: hunting-advanced-persistent-threats

Hunting Advanced Persistent Threats

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: External References

When to use

- The user wants to **hunt for** advanced persistent threats.
- Capability: APT threat hunting agent using MITRE ATT&CK, attackcti, and osquery.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, advanced, persistent, threats`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Retrieve TTPs for a specific APT group from MITRE ATT&CK
2. Generate osquery hunt queries for detected ATT&CK techniques
3. Generate a Sigma detection rule for a given technique
4. Build a complete threat hunt report for an APT group

Tools & resources

- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_apt_group_ttps` — Retrieve TTPs for a specific APT group from MITRE ATT&CK.
- `generate_osquery_hunts` — Generate osquery hunt queries for detected ATT&CK techniques.
- `generate_sigma_rule` — Generate a Sigma detection rule for a given technique.
- `build_hunt_report` — Build a complete threat hunt report for an APT group.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-advanced-persistent-threats`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-credential-stuffing-attacks (Skill Module)

`hunting-credential-stuffing-attacks\SKILL.md`

name: hunting-credential-stuffing-attacks
description: Agent for hunting credential stuffing attacks in authentication logs. Use when the user asks to hunt for credential stuffing attacks.
version: 1.0.0
tags:
source: hunting-credential-stuffing-attacks

Hunting Credential Stuffing Attacks

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Detection Thresholds

When to use

- The user wants to **hunt for** credential stuffing attacks.
- Capability: Agent for hunting credential stuffing attacks in authentication logs.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, credential, stuffing, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load authentication logs from CSV or JSON lines
2. Detect credential stuffing by analyzing failed login patterns
3. Detect password spray attacks (one password, many accounts)
4. Detect distributed credential stuffing (many IPs per account)
5. Find accounts with successful login after many failures (compromised)
6. Detect automation by analyzing user-agent distribution
7. Calculate overall authentication attack metrics

Tools & resources

- Detection Thresholds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_auth_logs` — Load authentication logs from CSV or JSON lines.
- `detect_credential_stuffing` — Detect credential stuffing by analyzing failed login patterns.
- `detect_password_spray` — Detect password spray attacks (one password, many accounts).
- `detect_distributed_attack` — Detect distributed credential stuffing (many IPs per account).
- `analyze_success_after_failures` — Find accounts with successful login after many failures (compromised).
- `analyze_user_agent_patterns` — Detect automation by analyzing user-agent distribution.
- `calculate_attack_metrics` — Calculate overall authentication attack metrics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-credential-stuffing-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-anomalous-powershell-execution (Skill Module)

`hunting-for-anomalous-powershell-execution\SKILL.md`

name: hunting-for-anomalous-powershell-execution
description: PowerShell Script Block Logging threat hunting. Use when the user asks to hunt for anomalous powershell execution.
version: 1.0.0
tags:
source: hunting-for-anomalous-powershell-execution

Hunting for Anomalous PowerShell Execution

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: AMSI Bypass Indicators, Suspicious PowerShell Keywords, Obfuscation Indicators, python-evtx Library Usage, External References

When to use

- The user wants to **hunt for** anomalous powershell execution.
- Capability: PowerShell Script Block Logging threat hunting agent.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, anomalous, powershell, execution`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Event 4104 script block logging entries from EVTX
2. Reassemble multi-part script blocks by ScriptBlockId
3. Check script text for AMSI bypass indicators
4. Check for known offensive tool keywords
5. Detect download cradle patterns in script text
6. Detect obfuscation and encoded command patterns
7. Run all detection checks on assembled script blocks
8. Execute PowerShell script block hunting

Tools & resources

- AMSI Bypass Indicators
- Suspicious PowerShell Keywords
- Obfuscation Indicators
- python-evtx Library Usage
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_evtx_4104` — Parse Event 4104 script block logging entries from EVTX.
- `reassemble_script_blocks` — Reassemble multi-part script blocks by ScriptBlockId.
- `detect_amsi_bypass` — Check script text for AMSI bypass indicators.
- `detect_suspicious_keywords` — Check for known offensive tool keywords.
- `detect_download_cradles` — Detect download cradle patterns in script text.
- `detect_obfuscation` — Detect obfuscation and encoded command patterns.
- `hunt_scripts` — Run all detection checks on assembled script blocks.
- `run_audit` — Execute PowerShell script block hunting.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-anomalous-powershell-execution`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-beaconing-with-frequency-analysis (Skill Module)

`hunting-for-beaconing-with-frequency-analysis\SKILL.md`

name: hunting-for-beaconing-with-frequency-analysis
description: Agent for detecting C2 beaconing through network traffic frequency analysis. Use when the user asks to hunt for beaconing with frequency analysis.
version: 1.0.0
tags:
source: hunting-for-beaconing-with-frequency-analysis

Hunting for Beaconing with Frequency Analysis

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Beaconing Characteristics, Jitter Calculation, Splunk SPL, Elastic SIEM, Common C2 Beacon Intervals, Beaconing Detection Thresholds

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** beaconing with frequency analysis.
- Capability: Agent for detecting C2 beaconing through network traffic frequency analysis.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, beaconing, frequency, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse Zeek conn.log and extract connection timestamps per src-dst pair
2. Calculate jitter (standard deviation of intervals)
3. Detect beaconing patterns based on interval regularity
4. Parse generic CSV log with timestamp, src, dst, port columns

Key concepts

- Timestamp score: Regularity of connection intervals
- Data size score: Consistency of bytes transferred
- Connection count: Volume of connections in analysis window
- Duration: How long the beaconing pattern has persisted

Tools & resources

- Beaconing Characteristics
- Jitter Calculation
- Splunk SPL
- Elastic SIEM
- Common C2 Beacon Intervals
- Beaconing Detection Thresholds
- RITA Beacon Scoring Algorithm

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_zeek_conn_log` — Parse Zeek conn.log and extract connection timestamps per src-dst pair.
- `calculate_jitter` — Calculate jitter (standard deviation of intervals).
- `detect_beaconing` — Detect beaconing patterns based on interval regularity.
- `parse_csv_log` — Parse generic CSV log with timestamp, src, dst, port columns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-beaconing-with-frequency-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-cobalt-strike-beacons (Skill Module)

`hunting-for-cobalt-strike-beacons\SKILL.md`

name: hunting-for-cobalt-strike-beacons
description: Cobalt Strike Beacon Hunter - detects beacon signatures in network traffic and Zeek logs. Use when the user asks to hunt for cobalt strike beacons.
version: 1.0.0
tags:
source: hunting-for-cobalt-strike-beacons

Hunting for Cobalt Strike Beacons

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Cobalt Strike, RITA Beacon Analysis, Suricata Detection Rules, Default User-Agent Strings, Beacon Timing Analysis Formula, JARM Scanner CLI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** cobalt strike beacons.
- Capability: Cobalt Strike Beacon Hunter - detects beacon signatures in network traffic and Zeek logs.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, cobalt, strike, beacons`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Zeek ssl.log for default Cobalt Strike certificates
2. Analyze connection timing for beacon-like regular intervals
3. Detect known Cobalt Strike HTTP malleable C2 profile patterns

Tools & resources

- Cobalt Strike
- RITA Beacon Analysis
- Suricata Detection Rules
- Default User-Agent Strings
- Beacon Timing Analysis Formula
- JARM Scanner CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_zeek_ssl_log` — Parse Zeek ssl.log for default Cobalt Strike certificates.
- `analyze_beacon_timing` — Analyze connection timing for beacon-like regular intervals.
- `check_http_profiles` — Detect known Cobalt Strike HTTP malleable C2 profile patterns.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-cobalt-strike-beacons`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-command-and-control-beaconing (Skill Module)

`hunting-for-command-and-control-beaconing\SKILL.md`

name: hunting-for-command-and-control-beaconing
description: Agent for hunting C2 beaconing across multiple data sources. Use when the user asks to hunt for command and control beaconing.
version: 1.0.0
tags:
source: hunting-for-command-and-control-beaconing

Hunting for Command and Control Beaconing

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Zeek Log Files, C2 Framework Signatures, DNS Tunneling Indicators, JA3/JA3S TLS Fingerprinting, Threat Intelligence Feeds, RITA Beacon Analysis

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** command and control beaconing.
- Capability: Agent for hunting C2 beaconing across multiple data sources.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, command, control, beaconing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze DNS query logs for C2 indicators
2. Analyze HTTP logs for C2-like traffic patterns
3. Detect persistent long-duration connections typical of C2

Tools & resources

- Zeek Log Files
- C2 Framework Signatures
- DNS Tunneling Indicators
- JA3/JA3S TLS Fingerprinting
- Threat Intelligence Feeds
- RITA Beacon Analysis
- Beaconing Detection Thresholds
- Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_dns_queries` — Analyze DNS query logs for C2 indicators.
- `analyze_http_logs` — Analyze HTTP logs for C2-like traffic patterns.
- `analyze_connection_patterns` — Detect persistent long-duration connections typical of C2.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-command-and-control-beaconing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-data-exfiltration-indicators (Skill Module)

`hunting-for-data-exfiltration-indicators\SKILL.md`

name: hunting-for-data-exfiltration-indicators
description: Agent for hunting data exfiltration indicators in network traffic. Use when the user asks to hunt for data exfiltration indicators.
version: 1.0.0
tags:
source: hunting-for-data-exfiltration-indicators

Hunting for Data Exfiltration Indicators

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: DNS Exfiltration Indicators, Zeek Log Fields, Detection Thresholds, Data Sources

When to use

- The user wants to **hunt for** data exfiltration indicators.
- Capability: Agent for hunting data exfiltration indicators in network traffic.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, data, exfiltration, indicators`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Calculate Shannon entropy of a string
2. Analyze DNS query log for exfiltration indicators
3. Analyze network flow data for large outbound transfers
4. Check for significant data transfers during off-hours

Tools & resources

- DNS Exfiltration Indicators
- Zeek Log Fields
- Detection Thresholds
- Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of a string.
- `analyze_dns_queries` — Analyze DNS query log for exfiltration indicators.
- `analyze_network_flows` — Analyze network flow data for large outbound transfers.
- `analyze_off_hours_traffic` — Check for significant data transfers during off-hours.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-data-exfiltration-indicators`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-data-staging-before-exfiltration (Skill Module)

`hunting-for-data-staging-before-exfiltration\SKILL.md`

name: hunting-for-data-staging-before-exfiltration
description: Detect data staging activity before exfiltration via archive creation and file consolidation monitoring. Use when the user asks to hunt for data staging before exfiltration.
version: 1.0.0
tags:
source: hunting-for-data-staging-before-exfiltration

Hunting for Data Staging Before Exfiltration

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Archive Tool Detection Signatures, Common Staging Directories

When to use

- The user wants to **hunt for** data staging before exfiltration.
- Capability: Detect data staging activity before exfiltration via archive creation and file consolidation monitoring.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, data, staging, before, exfiltration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse process creation logs (Sysmon/EDR JSON format)
2. Detect archive tool execution with compression arguments
3. Detect file writes to known staging directories
4. Detect bulk reads from sensitive directories (document consolidation)
5. Compute overall staging risk score (0-100)
6. Generate data staging hunt report

Tools & resources

- Archive Tool Detection Signatures
- Common Staging Directories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_process_logs` — Parse process creation logs (Sysmon/EDR JSON format).
- `detect_archive_creation` — Detect archive tool execution with compression arguments.
- `detect_staging_directory_writes` — Detect file writes to known staging directories.
- `detect_sensitive_source_reads` — Detect bulk reads from sensitive directories (document consolidation).
- `score_staging_risk` — Compute overall staging risk score (0-100).
- `generate_report` — Generate data staging hunt report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-data-staging-before-exfiltration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-dcom-lateral-movement (Skill Module)

`hunting-for-dcom-lateral-movement\SKILL.md`

name: hunting-for-dcom-lateral-movement
description: DCOM Lateral Movement Detection Agent - Hunts for DCOM object abuse via Sysmon event correlation. Use when the user asks to hunt for dcom lateral movement.
version: 1.0.0
tags:
source: hunting-for-dcom-lateral-movement

Hunting for Dcom Lateral Movement

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: WMI, Network Indicators, python-evtx, Atomic Red Team, Impacket

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** dcom lateral movement.
- Capability: DCOM Lateral Movement Detection Agent - Hunts for DCOM object abuse via Sysmon event correlation.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, dcom, lateral, movement`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Sysmon EVTX file into structured events using python-evtx
2. Detect MMC20.Application DCOM lateral movement: mmc.exe spawning suspicious children
3. Detect ShellWindows/ShellBrowserWindow: explorer.exe spawning cmd/powershell
4. Detect DCOM via dllhost.exe spawning suspicious children
5. Detect inbound RPC endpoint mapper connections (port 135) from Sysmon Event ID 3
6. Correlate RPC connections with DCOM process creation for high-confidence detections
7. Audit local DCOM configuration for high-risk COM objects (Windows only)
8. Generate JSON detection report

Tools & resources

- WMI
- Network Indicators
- python-evtx
- Atomic Red Team
- Impacket

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_evtx_records` — Parse Sysmon EVTX file into structured events using python-evtx.
- `detect_mmc20_lateral` — Detect MMC20.Application DCOM lateral movement: mmc.exe spawning suspicious children.
- `detect_shell_dcom_lateral` — Detect ShellWindows/ShellBrowserWindow: explorer.exe spawning cmd/powershell.
- `detect_dllhost_lateral` — Detect DCOM via dllhost.exe spawning suspicious children.
- `detect_rpc_connections` — Detect inbound RPC endpoint mapper connections (port 135) from Sysmon Event ID 3.
- `correlate_rpc_with_process` — Correlate RPC connections with DCOM process creation for high-confidence detections.
- `audit_dcom_config` — Audit local DCOM configuration for high-risk COM objects (Windows only).
- `generate_report` — Generate JSON detection report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-dcom-Lateral-movement`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-dcsync-attacks (Skill Module)

`hunting-for-dcsync-attacks\SKILL.md`

name: hunting-for-dcsync-attacks
description: DCSync Detection Agent - hunts for unauthorized AD replication requests via Event ID 4662 analysis. Use when the user asks to hunt for dcsync attacks.
version: 1.0.0
tags:
source: hunting-for-dcsync-attacks

Hunting for Dcsync Attacks

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Key Detection GUIDs, Elastic KQL Detection, PowerShell Detection, Response Checklist

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** dcsync attacks.
- Capability: DCSync Detection Agent - hunts for unauthorized AD replication requests via Event ID 4662 analysis.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, dcsync, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Get list of legitimate domain controller machine accounts
2. Query Windows Event ID 4662 for directory service access events
3. Filter events for DS-Replication GUID access
4. Identify non-DC accounts performing replication requests
5. Analyze patterns in suspected DCSync activity
6. Check which accounts have replication rights on the domain object
7. Generate DCSync hunt report

Tools & resources

- Key Detection GUIDs
- Elastic KQL Detection
- PowerShell Detection
- Response Checklist

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_domain_controllers` — Get list of legitimate domain controller machine accounts.
- `query_event_4662` — Query Windows Event ID 4662 for directory service access events.
- `filter_replication_events` — Filter events for DS-Replication GUID access.
- `identify_dcsync_suspects` — Identify non-DC accounts performing replication requests.
- `analyze_suspect_patterns` — Analyze patterns in suspected DCSync activity.
- `check_replication_acls` — Check which accounts have replication rights on the domain object.
- `generate_report` — Generate DCSync hunt report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-dcsync-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-defense-evasion-via-timestomping (Skill Module)

hunting-for-defense-evasion-via-timestomping\SKILL.md

name: hunting-for-defense-evasion-via-timestomping
description: Timestomping detection agent for NTFS MFT analysis. Detects MITRE T1070.006 (Timestomping) by comparing $STANDARD_INFORMATION and $FILE_NAME timestamps in NTFS Master File Table entries. Identifies anomalous.... Use when the user asks to hunt for defense evasion via timestomping.
version: 1.0.0
tags:
source: hunting-for-defense-evasion-via-timestomping

Hunting for Defense Evasion via Timestomping

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Detection Logic, USN Journal Analysis

When to use

- The user wants to **hunt for** defense evasion via timestomping.
- Capability: Timestomping detection agent for NTFS MFT analysis. Detects MITRE T1070.006 (Timestomping) by comparing $STANDARD_INFORMATION and $FILE_NAME timestamps in NTFS Master File Table entries. Identifies anomalous nanosecond patterns and temporal inconsistencies.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, defense, evasion, timestomping`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse analyzeMFT CSV output for timestamp analysis
2. Parse various timestamp formats to datetime
3. Analyze MFT entries for timestomping indicators
4. Generate timestomping analysis report

Tools & resources

- Detection Logic
- USN Journal Analysis

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_mft_csv` — Parse analyzeMFT CSV output for timestamp analysis.
- `parse_timestamp` — Parse various timestamp formats to datetime.
- `detect_timestomping` — Analyze MFT entries for timestomping indicators.
- `generate_report` — Generate timestomping analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-defense-evasion-via-timestomping`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-dns-based-persistence (Skill Module)

`hunting-for-dns-based-persistence\SKILL.md`

name: hunting-for-dns-based-persistence
description: DNS Persistence Hunting Agent - detects DNS hijacking, dangling CNAMEs, and unauthorized zone changes. Use when the user asks to hunt for dns based persistence.
version: 1.0.0
tags:
source: hunting-for-dns-based-persistence

Hunting for DNS Based Persistence

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: SecurityTrails API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** dns based persistence.
- Capability: DNS Persistence Hunting Agent - detects DNS hijacking, dangling CNAMEs, and unauthorized zone changes.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, persistence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query SecurityTrails API for passive DNS history
2. Enumerate subdomains via SecurityTrails API
3. Resolve DNS record using dig command
4. Detect CNAME records pointing to decommissioned services (subdomain takeover risk)
5. Check for wildcard DNS records that resolve all subdomains
6. Verify NS records haven't been modified to point to unauthorized servers
7. Analyze historical DNS changes for signs of hijacking

Tools & resources

- SecurityTrails API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_securitytrails` — Query SecurityTrails API for passive DNS history.
- `get_subdomains` — Enumerate subdomains via SecurityTrails API.
- `resolve_record` — Resolve DNS record using dig command.
- `detect_dangling_cnames` — Detect CNAME records pointing to decommissioned services (subdomain takeover risk).
- `detect_wildcard_abuse` — Check for wildcard DNS records that resolve all subdomains.
- `check_ns_delegation` — Verify NS records haven't been modified to point to unauthorized servers.
- `analyze_dns_history` — Analyze historical DNS changes for signs of hijacking.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-dns-based-persistence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-dns-tunneling-with-zeek (Skill Module)

`hunting-for-dns-tunneling-with-zeek\SKILL.md`

name: hunting-for-dns-tunneling-with-zeek
description: Agent for detecting DNS tunneling using Zeek log analysis. Use when the user asks to hunt for dns tunneling with zeek.
version: 1.0.0
tags:
source: hunting-for-dns-tunneling-with-zeek

Hunting for DNS Tunneling with ZEEK

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Zeek, Detection Heuristics, DNS Tunneling Detection Thresholds, DNS Tunneling Tools

When to use

- The user wants to **hunt for** dns tunneling with zeek.
- Capability: Agent for detecting DNS tunneling using Zeek log analysis.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, tunneling, zeek`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Calculate Shannon entropy of a string
2. Load Zeek dns.log (TSV format)
3. Compute per-domain statistics for tunneling detection
4. Apply tunneling detection heuristics

Tools & resources

- Zeek
- Detection Heuristics
- DNS Tunneling Detection Thresholds
- DNS Tunneling Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of a string.
- `load_dns_log` — Load Zeek dns.log (TSV format).
- `analyze_domain_statistics` — Compute per-domain statistics for tunneling detection.
- `detect_tunneling` — Apply tunneling detection heuristics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-dns-tunneling-with-zeek`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-domain-fronting-c2-traffic (Skill Module)

hunting-for-domain-fronting-c2-traffic\SKILL.md

name: hunting-for-domain-fronting-c2-traffic
description: Detect domain fronting C2 traffic via SNI/Host header mismatch and TLS certificate analysis. Use when the user asks to hunt for domain fronting c2 traffic.
version: 1.0.0
tags:
source: hunting-for-domain-fronting-c2-traffic

Hunting for Domain Fronting C2 Traffic

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Domain Fronting Mechanism, CDN Provider Identification, Proxy Log Detection, pyOpenSSL Certificate Inspection

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** domain fronting c2 traffic.
- Capability: Detect domain fronting C2 traffic via SNI/Host header mismatch and TLS certificate analysis.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, domain, fronting, traffic`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load proxy logs CSV with columns: timestamp, src_ip, sni, host_header, dst_ip, dst_port, method, url, status
2. Extract root domain from FQDN (e.g., sub.example.com -> example.com)
3. Check if a domain belongs to a known CDN provider
4. Detect connections where SNI and Host header point to different domains
5. Retrieve TLS certificate details for a given hostname using pyOpenSSL
6. Aggregate and rank domain fronting pairs by frequency

Tools & resources

- Domain Fronting Mechanism
- CDN Provider Identification
- Proxy Log Detection
- pyOpenSSL Certificate Inspection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_proxy_logs` — Load proxy logs CSV with columns: timestamp, src_ip, sni, host_header, dst_ip, dst_port, method, url, status.
- `extract_domain_root` — Extract root domain from FQDN (e.g., sub.example.com -> example.com).
- `identify_cdn_provider` — Check if a domain belongs to a known CDN provider.
- `detect_sni_host_mismatch` — Detect connections where SNI and Host header point to different domains.
- `get_tls_certificate_info` — Retrieve TLS certificate details for a given hostname using pyOpenSSL.
- `analyze_fronting_pairs` — Aggregate and rank domain fronting pairs by frequency.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-domain-fronting-c2-traffic`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-lateral-movement-via-wmi (Skill Module)

`hunting-for-lateral-movement-via-wmi\SKILL.md`

name: hunting-for-lateral-movement-via-wmi
description: Agent for hunting lateral movement via WMI. Detects WMI-based lateral movement by parsing Windows Event ID 4688 and Sysmon Event 1 for WmiPrvSE.exe child process patterns, suspicious command lines, and WMI event.... Use when the user asks to hunt for lateral movement via wmi.
version: 1.0.0
tags:
source: hunting-for-lateral-movement-via-wmi

Hunting for Lateral Movement via WMI

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: WMI-Activity Log Location

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** lateral movement via wmi.
- Capability: Agent for hunting lateral movement via WMI. Detects WMI-based lateral movement by parsing Windows Event ID 4688 and Sysmon Event 1 for WmiPrvSE.exe child process patterns, suspicious command lines, and WMI event subscription persistence.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, lateral, movement`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Form a threat hypothesis grounded in TTPs or intel
2. Select data sources and time range for for lateral movement via wmi
3. Execute hunt queries and pivot on anomalies
4. Triage hits into benign, suspicious, or confirmed malicious
5. Capture IOCs, lessons learned, and detection opportunities

Tools & resources

- WMI-Activity Log Location

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-lateral-movement-via-wmi`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-living-off-the-cloud-techniques (Skill Module)

`hunting-for-living-off-the-cloud-techniques\SKILL.md`

name: hunting-for-living-off-the-cloud-techniques
description: Agent for hunting living-off-the-cloud (LOTC) techniques using cloud service logs. Use when the user asks to hunt for living off the cloud techniques.
version: 1.0.0
tags:
source: hunting-for-living-off-the-cloud-techniques

Hunting for Living Off The Cloud Techniques

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Elasticsearch API Calls, Detection Data Sources

When to use

- The user wants to **hunt for** living off the cloud techniques.
- Capability: Agent for hunting living-off-the-cloud (LOTC) techniques using cloud service logs.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, living, cloud, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Form a threat hypothesis grounded in TTPs or intel
2. Select data sources and time range for for living off the cloud techniques
3. Execute hunt queries and pivot on anomalies
4. Triage hits into benign, suspicious, or confirmed malicious
5. Capture IOCs, lessons learned, and detection opportunities

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Elasticsearch API Calls
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `hunt_lotc_elastic` — Run LOTC hunting queries against Elasticsearch/Elastic SIEM.
- `analyze_dns_logs` — Analyze DNS query logs for cloud C2 domain patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-living-off-the-cloud-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-living-off-the-land-binaries (Skill Module)

`hunting-for-living-off-the-land-binaries\SKILL.md`

name: hunting-for-living-off-the-land-binaries
description: Agent for hunting Living-off-the-Land Binaries (LOLBins) execution patterns. Use when the user asks to hunt for living off the land binaries.
version: 1.0.0
tags:
source: hunting-for-living-off-the-land-binaries

Hunting for Living Off The Land Binaries

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: LOLBins Covered, LOLBAS Project Reference, Threat Intelligence References, Detection Data Sources

When to use

- The user wants to **hunt for** living off the land binaries.
- Capability: Agent for hunting Living-off-the-Land Binaries (LOLBins) execution patterns.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, living, land, binaries`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Form a threat hypothesis grounded in TTPs or intel
2. Select data sources and time range for for living off the land binaries
3. Execute hunt queries and pivot on anomalies
4. Triage hits into benign, suspicious, or confirmed malicious
5. Capture IOCs, lessons learned, and detection opportunities

Key concepts

- T1218 - Signed Binary Proxy Execution: Use of trusted binaries to proxy execution of malicious payloads
- T1218.001 - Compiled HTML File
- T1218.002 - Control Panel
- T1218.004 - InstallUtil
- T1218.009 - Regsvcs/Regasm
- T1218.013 - Mavinject
- T1197 - BITS Jobs: Abuse of Background Intelligent Transfer Service
- T1140 - Deobfuscate/Decode Files or Information: certutil decode operations

Tools & resources

- LOLBins Covered
- LOLBAS Project Reference
- Threat Intelligence References
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `hunt_lolbins_elastic` — Query Elasticsearch for LOLBin execution with suspicious arguments.
- `scan_sysmon_log` — Parse Sysmon EVTX for LOLBin process creation (Event ID 1).

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-living-off-the-land-binaries`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-lolbins-execution-in-endpoint-logs (Skill Module)

`hunting-for-lolbins-execution-in-endpoint-logs\SKILL.md`

name: hunting-for-lolbins-execution-in-endpoint-logs
description: Agent for hunting LOLBin execution patterns in endpoint logs (Sysmon, EDR). Use when the user asks to hunt for lolbins execution in endpoint logs.
version: 1.0.0
tags:
source: hunting-for-lolbins-execution-in-endpoint-logs

Hunting for Lolbins Execution in Endpoint Logs

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Suspicious Parent

When to use

- The user wants to **hunt for** lolbins execution in endpoint logs.
- Capability: Agent for hunting LOLBin execution patterns in endpoint logs (Sysmon, EDR).
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, lolbins, execution, endpoint, logs`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Block identified malicious URLs and IPs at proxy/firewall
2. Isolate endpoint if active compromise confirmed
3. Collect process memory dump for malware analysis
4. Deploy targeted detection rules for observed patterns
5. Update application control policies to restrict LOLBin abuse

Tools & resources

- Suspicious Parent

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_csv_logs` — Scan exported CSV endpoint logs for LOLBin execution.
- `scan_evtx_sysmon` — Parse Sysmon EVTX for Event ID 1 matching LOLBin patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-lolbins-execution-in-endpoint-logs`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-ntlm-relay-attacks (Skill Module)

`hunting-for-ntlm-relay-attacks\SKILL.md`

name: hunting-for-ntlm-relay-attacks
description: Detect NTLM relay attacks via Windows Event 4624 analysis, IP-hostname correlation, and SMB signing audit. Use when the user asks to hunt for ntlm relay attacks.
version: 1.0.0
tags:
source: hunting-for-ntlm-relay-attacks

Hunting for NTLM Relay Attacks

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: PowerShell Detection, SMB Signing Enforcement, Response Checklist

When to use

- The user wants to **hunt for** ntlm relay attacks.
- Capability: Detect NTLM relay attacks via Windows Event 4624 analysis, IP-hostname correlation, and SMB signing audit.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, ntlm, relay, attacks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query Windows Security event log for specific event ID using wevtutil
2. Parse Event 4624 logon events from XML output
3. Parse Event 5145 network share access events for named pipe monitoring
4. Detect when WorkstationName IP doesn't match source IpAddress
5. Detect machine accounts authenticating from unexpected IPs
6. Detect rapid multi-host authentication from a single account
7. Detect logon events with NULL SID and missing LogonGUID — relay indicator
8. Check SMB signing status on specified hosts using PowerShell

Tools & resources

- PowerShell Detection
- SMB Signing Enforcement
- Response Checklist

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_security_log` — Query Windows Security event log for specific event ID using wevtutil.
- `parse_4624_events` — Parse Event 4624 logon events from XML output.
- `parse_5145_events` — Parse Event 5145 network share access events for named pipe monitoring.
- `detect_ip_hostname_mismatch` — Detect when WorkstationName IP doesn't match source IpAddress.
- `detect_machine_account_relay` — Detect machine accounts authenticating from unexpected IPs.
- `detect_rapid_authentication` — Detect rapid multi-host authentication from a single account.
- `detect_null_sid_logons` — Detect logon events with NULL SID and missing LogonGUID — relay indicator.
- `check_smb_signing` — Check SMB signing status on specified hosts using PowerShell.
- `run_hunt` — Run full NTLM relay detection hunt.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-ntlm-relay-attacks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-persistence-mechanisms-in-windows (Skill Module)

`hunting-for-persistence-mechanisms-in-windows\SKILL.md`

name: hunting-for-persistence-mechanisms-in-windows
description: Agent for hunting Windows persistence mechanisms across registry, services, and scheduled tasks. Use when the user asks to hunt for persistence mechanisms in windows.
version: 1.0.0
tags:
source: hunting-for-persistence-mechanisms-in-windows

Hunting for Persistence Mechanisms in Windows

Role: Threat hunting
Audience: threat hunters and SOC analysts

When to use

- The user wants to **hunt for** persistence mechanisms in windows.
- Capability: Agent for hunting Windows persistence mechanisms across registry, services, and scheduled tasks.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, persistence, mechanisms, windows`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Enumerate common Windows registry persistence locations using reg query
2. Parse reg query output into structured entries
3. List scheduled tasks and flag suspicious ones
4. List Windows services and flag those running from unusual paths

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `enumerate_registry_persistence` — Enumerate common Windows registry persistence locations using reg query.
- `parse_reg_output` — Parse reg query output into structured entries.

- `enumerate_scheduled_tasks` — List scheduled tasks and flag suspicious ones.
- `enumerate_services` — List Windows services and flag those running from unusual paths.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-persistence-mechanisms-in-windows`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-persistence-via-wmi-subscriptions (Skill Module)

`hunting-for-persistence-via-wmi-subscriptions\SKILL.md`

name: hunting-for-persistence-via-wmi-subscriptions
description: Agent for hunting WMI event subscription persistence (T1546.003). Use when the user asks to hunt for persistence via wmi subscriptions.
version: 1.0.0
tags:
source: hunting-for-persistence-via-wmi-subscriptions

Hunting for Persistence via WMI Subscriptions

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: WMI Classes Enumerated, WMI Subscription Components, Consumer Types and Risk, Detection Events, Known APT Usage

When to use

- The user wants to **hunt for** persistence via wmi subscriptions.
- Capability: Agent for hunting WMI event subscription persistence (T1546.003).
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, persistence, subscriptions`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Enumerate all WMI event subscriptions on the local system
2. Parse WMIC /format:list output into list of dicts
3. Parse Sysmon EVTX for WMI events (Event IDs 19, 20, 21)
4. Use PowerShell Get-WMIObject to enumerate WMI subscriptions

Tools & resources

- WMI Classes Enumerated
- WMI Subscription Components
- Consumer Types and Risk
- Detection Events
- Known APT Usage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `enumerate_wmi_subscriptions` — Enumerate all WMI event subscriptions on the local system.
- `parse_wmic_list` — Parse WMIC /format:list output into list of dicts.
- `scan_sysmon_wmi_events` — Parse Sysmon EVTX for WMI events (Event IDs 19, 20, 21).
- `query_powershell_wmi` — Use PowerShell Get-WMIObject to enumerate WMI subscriptions.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-persistence-via-wmi-subscriptions`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-process-injection-techniques (Skill Module)

`hunting-for-process-injection-techniques\SKILL.md`

name: hunting-for-process-injection-techniques
description: Detect process injection techniques (T1055) via Sysmon Event IDs 8 and 10. Use when the user asks to hunt for process injection techniques.
version: 1.0.0
tags:
source: hunting-for-process-injection-techniques

Hunting for Process Injection Techniques

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Dangerous Access Rights Masks, Atomic Red Team Tests

When to use

- The user wants to **hunt for** process injection techniques.
- Capability: Detect process injection techniques (T1055) via Sysmon Event IDs 8 and 10.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, process, injection, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Sysmon JSON events, focusing on Event IDs 1, 8, and 10
2. Detect suspicious CreateRemoteThread (Sysmon Event ID 8)
3. Detect suspicious ProcessAccess (Sysmon Event ID 10)
4. Build a source->target injection relationship graph
5. Generate process injection hunt report

Tools & resources

- Dangerous Access Rights Masks
- Atomic Red Team Tests

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_sysmon_events` — Parse Sysmon JSON events, focusing on Event IDs 1, 8, and 10.
- `detect_remote_thread_injection` — Detect suspicious CreateRemoteThread (Sysmon Event ID 8).
- `detect_suspicious_process_access` — Detect suspicious ProcessAccess (Sysmon Event ID 10).
- `build_injection_graph` — Build a source->target injection relationship graph.
- `generate_report` — Generate process injection hunt report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-process-injection-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-registry-persistence-mechanisms (Skill Module)

`hunting-for-registry-persistence-mechanisms\SKILL.md`

name: hunting-for-registry-persistence-mechanisms
description: Agent for hunting registry-based persistence mechanisms on Windows. Use when the user asks to hunt for registry persistence mechanisms.
version: 1.0.0
tags:
source: hunting-for-registry-persistence-mechanisms

Hunting for Registry Persistence Mechanisms

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Registry Categories Scanned, Detection Data Sources

When to use

- The user wants to **hunt for** registry persistence mechanisms.
- Capability: Agent for hunting registry-based persistence mechanisms on Windows.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, registry, persistence, mechanisms`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Form a threat hypothesis grounded in TTPs or intel
2. Select data sources and time range for for registry persistence mechanisms
3. Execute hunt queries and pivot on anomalies
4. Triage hits into benign, suspicious, or confirmed malicious
5. Capture IOCs, lessons learned, and detection opportunities

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Registry Categories Scanned
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_persistence_keys` — Scan specified registry persistence key categories.
- `compare_baseline` — Compare current registry state against a known-good baseline.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-registry-persistence-mechanisms`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-registry-run-key-persistence (Skill Module)

`hunting-for-registry-run-key-persistence\SKILL.md`

name: hunting-for-registry-run-key-persistence
description: Registry Run Key Persistence Hunter - detects T1547.001 persistence via Sysmon Event 13 analysis and registry run key auditing. Use when the user asks to hunt for registry run key persistence.
version: 1.0.0
tags:
source: hunting-for-registry-run-key-persistence

Hunting for Registry Run Key Persistence

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Sysmon Configuration, Sigma Rule

When to use

- The user wants to **hunt for** registry run key persistence.
- Capability: Registry Run Key Persistence Hunter - detects T1547.001 persistence via Sysmon Event 13 analysis and registry run key auditing
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, registry, persistence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check if registry path matches Run/RunOnce keys
2. Analyze Sysmon Event ID 13 (RegistryEvent Value Set) for persistence
3. Analyze a static registry snapshot for suspicious Run key values
4. Generate a Sigma detection rule from a finding

Tools & resources

- Sysmon Configuration
- Sigma Rule

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_data`
- `is_run_key` — Check if registry path matches Run/RunOnce keys.
- `analyze_sysmon_events` — Analyze Sysmon Event ID 13 (RegistryEvent Value Set) for persistence.
- `analyze_registry_snapshot` — Analyze a static registry snapshot for suspicious Run key values.
- `generate_sigma_rule` — Generate a Sigma detection rule from a finding.
- `analyze`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-registry-run-key-persistence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-scheduled-task-persistence (Skill Module)

hunting-for-scheduled-task-persistence\SKILL.md

name: hunting-for-scheduled-task-persistence
description: Agent for hunting scheduled task persistence mechanisms (T1053.005). Use when the user asks to hunt for scheduled task persistence.
version: 1.0.0
tags:
source: hunting-for-scheduled-task-persistence

Hunting for Scheduled Task Persistence

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Risk Classification, Detection Data Sources

When to use

- The user wants to **hunt for** scheduled task persistence.
- Capability: Agent for hunting scheduled task persistence mechanisms (T1053.005).
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, scheduled, task, persistence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Enumerate all scheduled tasks and flag suspicious ones
2. Parse Security EVTX for Event ID 4698 (Scheduled Task Created)
3. Export a specific scheduled task's XML configuration for analysis

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Risk Classification
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `enumerate_tasks` — Enumerate all scheduled tasks and flag suspicious ones.
- `scan_event_log_4698` — Parse Security EVTX for Event ID 4698 (Scheduled Task Created).
- `export_task_xml` — Export a specific scheduled task's XML configuration for analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-scheduled-task-persistence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-shadow-copy-deletion (Skill Module)

`hunting-for-shadow-copy-deletion\SKILL.md`

name: hunting-for-shadow-copy-deletion
description: Agent for hunting shadow copy deletion activity indicating ransomware or anti-forensics. Use when the user asks to hunt for shadow copy deletion.
version: 1.0.0
tags:
source: hunting-for-shadow-copy-deletion

Hunting for Shadow Copy Deletion

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: python-evtx, Splunk SPL, Detection Data Sources

When to use

- The user wants to **hunt for** shadow copy deletion.
- Capability: Agent for hunting shadow copy deletion activity indicating ransomware or anti-forensics.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, shadow, copy, deletion`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse Windows EVTX file for shadow copy deletion events
2. Check a command line string against shadow copy deletion patterns
3. Hunt for shadow copy deletion in EVTX logs
4. Scan JSON-exported Sysmon logs for shadow copy deletion
5. Generate a Sigma detection rule for shadow copy deletion

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- python-evtx
- Splunk SPL
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_evtx_file` — Parse Windows EVTX file for shadow copy deletion events.
- `scan_command_line` — Check a command line string against shadow copy deletion patterns.
- `hunt_evtx` — Hunt for shadow copy deletion in EVTX logs.
- `scan_sysmon_json` — Scan JSON-exported Sysmon logs for shadow copy deletion.
- `generate_sigma_rule` — Generate a Sigma detection rule for shadow copy deletion.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-shadow-copy-deletion`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-spearphishing-indicators (Skill Module)

`hunting-for-spearphishing-indicators\SKILL.md`

name: hunting-for-spearphishing-indicators
description: Agent for hunting spearphishing indicators across email and endpoint logs. Use when the user asks to hunt for spearphishing indicators.
version: 1.0.0
tags:
source: hunting-for-spearphishing-indicators

Hunting for Spearphishing Indicators

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Email Header Analysis, Suspicious Attachment Types, Splunk SPL, KQL, SPF/DKIM/DMARC Validation, Detection Data Sources

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** spearphishing indicators.
- Capability: Agent for hunting spearphishing indicators across email and endpoint logs.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, spearphishing, indicators`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load email logs from JSON lines file
2. Assess risk of email attachment by extension
3. Find emails with dangerous attachment types
4. Detect phishing URLs in email body or links
5. Detect social engineering urgency keywords
6. Detect display name vs envelope sender mismatches

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Email Header Analysis
- Suspicious Attachment Types
- Splunk SPL
- KQL
- SPF/DKIM/DMARC Validation
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_email_logs` — Load email logs from JSON lines file.
- `check_attachment_risk` — Assess risk of email attachment by extension.
- `detect_suspicious_attachments` — Find emails with dangerous attachment types.
- `detect_phishing_urls` — Detect phishing URLs in email body or links.
- `detect_urgency_lures` — Detect social engineering urgency keywords.
- `detect_sender_spoofing` — Detect display name vs envelope sender mismatches.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-spearphishing-indicators`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-startup-folder-persistence (Skill Module)

`hunting-for-startup-folder-persistence\SKILL.md`

name: hunting-for-startup-folder-persistence
description: Agent for hunting T1547.001 startup folder persistence on Windows. Use when the user asks to hunt for startup folder persistence.
version: 1.0.0
tags:
source: hunting-for-startup-folder-persistence

Hunting for Startup Folder Persistence

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Startup Folder Paths, File Extension Risk Scores, Additional Risk Factors

When to use

- The user wants to **hunt for** startup folder persistence.
- Capability: Agent for hunting T1547.001 startup folder persistence on Windows.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, startup, folder, persistence`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Return user-specific and all-users startup folder paths
2. Compute SHA-256 hash of a file
3. Analyze a single file in a startup directory
4. Scan all startup directories and analyze contents
5. Check Registry Run keys for autostart entries
6. Monitor startup folders in real-time using watchdog
7. Run comprehensive startup persistence threat hunt

Tools & resources

- Startup Folder Paths
- File Extension Risk Scores
- Additional Risk Factors

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_startup_paths` — Return user-specific and all-users startup folder paths.
- `compute_file_hash` — Compute SHA-256 hash of a file.
- `analyze_file` — Analyze a single file in a startup directory.
- `scan_startup_folders` — Scan all startup directories and analyze contents.
- `check_registry_run_keys` — Check Registry Run keys for autostart entries.
- `monitor_startup` — Monitor startup folders in real-time using watchdog.
- `full_hunt` — Run comprehensive startup persistence threat hunt.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-startup-folder-persistence`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-supply-chain-compromise (Skill Module)

`hunting-for-supply-chain-compromise\SKILL.md`

name: hunting-for-supply-chain-compromise
description: Agent for hunting supply chain compromise indicators in software dependencies and builds. Use when the user asks to hunt for supply chain compromise.
version: 1.0.0
tags:
source: hunting-for-supply-chain-compromise

Hunting for Supply Chain Compromise

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: NPM Lock File Analysis, pip-audit, Hash Verification, Dependency Confusion Checks, Splunk SPL, Supply Chain Indicators

When to use

- The user wants to **hunt for** supply chain compromise.
- Capability: Agent for hunting supply chain compromise indicators in software dependencies and builds.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, supply, chain, compromise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan package-lock.json for suspicious dependencies
2. Scan pip requirements.txt for suspicious packages
3. Verify binary hashes against a known-good manifest
4. Scan CI/CD build logs for supply chain indicators
5. Check if internal package names exist on public registries

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- NPM Lock File Analysis
- pip-audit
- Hash Verification
- Dependency Confusion Checks
- Splunk SPL
- Supply Chain Indicators
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_npm_lockfile` — Scan package-lock.json for suspicious dependencies.
- `scan_pip_requirements` — Scan pip requirements.txt for suspicious packages.
- `verify_binary_hashes` — Verify binary hashes against a known-good manifest.
- `scan_build_logs` — Scan CI/CD build logs for supply chain indicators.
- `check_dependency_confusion` — Check if internal package names exist on public registries.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-supply-chain-compromise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-suspicious-scheduled-tasks (Skill Module)

hunting-for-suspicious-scheduled-tasks\SKILL.md

name: hunting-for-suspicious-scheduled-tasks
description: Agent for hunting suspicious scheduled tasks on Windows endpoints. Use when the user asks to hunt for suspicious scheduled tasks.
version: 1.0.0
tags:
source: hunting-for-suspicious-scheduled-tasks

Hunting for Suspicious Scheduled Tasks

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: python-evtx, Splunk SPL, PowerShell Enumeration, Suspicious Task Indicators

When to use

- The user wants to **hunt for** suspicious scheduled tasks.
- Capability: Agent for hunting suspicious scheduled tasks on Windows endpoints.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, suspicious, scheduled, tasks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse output of schtasks /query /fo CSV /v
2. Analyze scheduled tasks for suspicious properties
3. Hunt Event ID 4698 (scheduled task creation) in EVTX
4. Generate Sigma rule for suspicious scheduled task creation

Tools & resources

- python-evtx
- Splunk SPL
- PowerShell Enumeration
- Suspicious Task Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_schtasks_csv` — Parse output of schtasks /query /fo CSV /v.
- `analyze_tasks` — Analyze scheduled tasks for suspicious properties.
- `hunt_evtx_4698` — Hunt Event ID 4698 (scheduled task creation) in EVTX.
- `generate_sigma_rule` — Generate Sigma rule for suspicious scheduled task creation.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-suspicious-scheduled-tasks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-t1098-account-manipulation (Skill Module)

hunting-for-t1098-account-manipulation\SKILL.md

name: hunting-for-t1098-account-manipulation
description: Agent for hunting MITRE ATT&CK T1098 account manipulation. Detects shadow admin creation, SID history injection, privileged group membership changes, and credential modifications by parsing Windows Security Event.... Use when the user asks to hunt for t1098 account manipulation.
version: 1.0.0
tags:
source: hunting-for-t1098-account-manipulation

Hunting for T1098 Account Manipulation

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Privileged Groups to Monitor, Sensitive AD Attributes, python-evtx Parsing, MITRE T1098 Sub-Techniques

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **hunt for** t1098 account manipulation.
- Capability: Agent for hunting MITRE ATT&CK T1098 account manipulation. Detects shadow admin creation, SID history injection, privileged group membership changes, and credential modifications by parsing Windows Security Event Logs for key event IDs.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, t1098, account, manipulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Form a threat hypothesis grounded in TTPs or intel
2. Select data sources and time range for for t1098 account manipulation
3. Execute hunt queries and pivot on anomalies
4. Triage hits into benign, suspicious, or confirmed malicious
5. Capture IOCs, lessons learned, and detection opportunities

Tools & resources

- Privileged Groups to Monitor
- Sensitive AD Attributes
- python-evtx Parsing
- MITRE T1098 Sub-Techniques

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-t1098-account-manipulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-unusual-network-connections (Skill Module)

`hunting-for-unusual-network-connections\SKILL.md`

name: hunting-for-unusual-network-connections
description: Agent for hunting unusual network connections from endpoint and firewall logs. Use when the user asks to hunt for unusual network connections.
version: 1.0.0
tags:
source: hunting-for-unusual-network-connections

Hunting for Unusual Network Connections

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Splunk SPL, KQL, Zeek conn.log Analysis, Beaconing Detection, Detection Data Sources

When to use

- The user wants to **hunt for** unusual network connections.
- Capability: Agent for hunting unusual network connections from endpoint and firewall logs.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, unusual, network, connections`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Convert dotted IP to integer
2. Check if IP is in private RFC1918 range
3. Load network connection logs from JSON lines
4. Find connections to unusual destination ports
5. Find rarely contacted external destinations
6. Find unusually long-lived connections (potential C2)
7. Detect periodic connections suggestive of beaconing

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Splunk SPL
- KQL
- Zeek conn.log Analysis
- Beaconing Detection
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `ip_to_int` — Convert dotted IP to integer.
- `is_private` — Check if IP is in private RFC1918 range.
- `load_connection_logs` — Load network connection logs from JSON lines.
- `detect_non_standard_ports` — Find connections to unusual destination ports.
- `detect_rare_destinations` — Find rarely contacted external destinations.
- `detect_long_connections` — Find unusually long-lived connections (potential C2).
- `detect_high_frequency_beaconing` — Detect periodic connections suggestive of beaconing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-unusual-network-connections`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-unusual-service-installations (Skill Module)

`hunting-for-unusual-service-installations\SKILL.md`

name: hunting-for-unusual-service-installations
description: Agent for hunting suspicious Windows service installations (T1543.003) via Event ID 7045. Use when the user asks to hunt for unusual service installations.
version: 1.0.0
tags:
source: hunting-for-unusual-service-installations

Hunting for Unusual Service Installations

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Legitimate Service Path Prefixes

When to use

- The user wants to **hunt for** unusual service installations.
- Capability: Agent for hunting suspicious Windows service installations (T1543.003) via Event ID 7045.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, unusual, service, installations`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse System.evtx and extract Event ID 7045 records
2. Analyze a service binary path for suspicious indicators
3. Main hunting function: parse events and analyze each service
4. Generate summary statistics from hunting results
5. Run comprehensive service installation threat hunt

Tools & resources

- Legitimate Service Path Prefixes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_evtx_events` — Parse System.evtx and extract Event ID 7045 records.
- `analyze_service_path` — Analyze a service binary path for suspicious indicators.
- `hunt_suspicious_services` — Main hunting function: parse events and analyze each service.
- `generate_statistics` — Generate summary statistics from hunting results.
- `full_hunt` — Run comprehensive service installation threat hunt.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-unusual-service-installations`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

hunting-for-webshell-activity (Skill Module)

`hunting-for-webshell-activity\SKILL.md`

name: hunting-for-webshell-activity
description: Agent for hunting web shell activity on web servers via process tree and log analysis. Use when the user asks to hunt for webshell activity.
version: 1.0.0
tags:
source: hunting-for-webshell-activity

Hunting for Webshell Activity

Role: Threat hunting
Audience: threat hunters and SOC analysts
Primary tools: Process Tree Detection, Splunk SPL, KQL, Sigma Rule, Detection Data Sources

When to use

- The user wants to **hunt for** webshell activity.
- Capability: Agent for hunting web shell activity on web servers via process tree and log analysis.
- Goal: proactively search for adversary activity using hypotheses and telemetry.
- Trigger keywords: `hunting, webshell, activity`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load process creation logs (JSON lines)
2. Detect shell processes spawned by web server processes
3. Analyze web access logs for webshell indicators
4. Detect new script files created in web server directories
5. Detect reconnaissance commands typically run through webshells

Key concepts

- Red Canary Threat Detection Report
- SANS Threat Hunting Summit Resources

Tools & resources

- Process Tree Detection
- Splunk SPL
- KQL
- Sigma Rule
- Detection Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_process_logs` — Load process creation logs (JSON lines).
- `detect_webserver_child_shells` — Detect shell processes spawned by web server processes.
- `analyze_web_access_logs` — Analyze web access logs for webshell indicators.
- `detect_file_creation_in_webroot` — Detect new script files created in web server directories.
- `detect_post_exploitation` — Detect reconnaissance commands typically run through webshells.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `hunting-for-webshell-activity`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aes-encryption-for-data-at-rest (Skill Module)

implementing-aes-encryption-for-data-at-rest\SKILL.md

name: implementing-aes-encryption-for-data-at-rest
description: Agent for implementing AES-256-GCM encryption for data at rest. Use when the user asks to implement aes encryption for data at rest.
version: 1.0.0
tags:
source: implementing-aes-encryption-for-data-at-rest

Implementing Aes Encryption for Data At Rest

Role: Security control implementation
Audience: security architects and engineers
Primary tools: cryptography Library - AESGCM, Key Derivation - PBKDF2, AES Modes Comparison, Primary Standards, RFC Standards, Compliance Frameworks

When to use

- The user wants to **implement** aes encryption for data at rest.
- Capability: Agent for implementing AES-256-GCM encryption for data at rest.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, encryption, data, rest`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Derive AES-256 key from password using PBKDF2-HMAC-SHA256
2. Encrypt a file using AES-256-GCM with PBKDF2 key derivation
3. Decrypt an AES-256-GCM encrypted file
4. Encrypt all files in a directory tree
5. Generate a random AES-256 key
6. Verify encryption by decrypting and comparing

Key concepts

- Description: Defines the AES algorithm (Rijndael) with key sizes of 128, 192, and 256 bits
- Block size: 128 bits (16 bytes)
- Key sizes: 128, 192, or 256 bits
- Rounds: 10 (128-bit), 12 (192-bit), 14 (256-bit)
- Description: Specifies Galois/Counter Mode (GCM) for authenticated encryption
- IV length: 96 bits recommended for GCM
- Tag length: 128 bits recommended (minimum 96 bits)
- Max plaintext: 2^39 - 256 bits per invocation

Tools & resources

- cryptography Library - AESGCM
- Key Derivation - PBKDF2
- AES Modes Comparison
- Primary Standards
- RFC Standards
- Compliance Frameworks
- Python Library References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `derive_key` — Derive AES-256 key from password using PBKDF2-HMAC-SHA256.
- `encrypt_file` — Encrypt a file using AES-256-GCM with PBKDF2 key derivation.
- `decrypt_file` — Decrypt an AES-256-GCM encrypted file.
- `encrypt_directory` — Encrypt all files in a directory tree.
- `generate_random_key` — Generate a random AES-256 key.
- `verify_encryption` — Verify encryption by decrypting and comparing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aes-encryption-for-data-at-rest`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-alert-fatigue-reduction (Skill Module)

implementing-alert-fatigue-reduction\SKILL.md

name: implementing-alert-fatigue-reduction
description: Alert fatigue reduction agent for SOC operations using Splunk SDK. Use when the user asks to implement alert fatigue reduction.
version: 1.0.0
tags:
source: implementing-alert-fatigue-reduction

Implementing Alert Fatigue Reduction

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Risk, Metrics Targets, External References

When to use

- The user wants to **implement** alert fatigue reduction.
- Capability: Alert fatigue reduction agent for SOC operations using Splunk SDK.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, alert, fatigue, reduction`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to Splunk instance
2. Query alert disposition data to measure alert quality
3. Identify rules exceeding false positive or volume thresholds
4. Calculate alerts per analyst per shift
5. Generate a plan to convert threshold alerts to risk-based alerting
6. Generate tuning recommendations for noisy rules
7. Build comprehensive alert fatigue reduction report

Tools & resources

- Risk
- Metrics Targets
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_splunk` — Connect to Splunk instance.
- `get_alert_quality_metrics` — Query alert disposition data to measure alert quality.
- `identify_noisy_rules` — Identify rules exceeding false positive or volume thresholds.
- `calculate_analyst_capacity` — Calculate alerts per analyst per shift.
- `generate_rba_conversion_plan` — Generate a plan to convert threshold alerts to risk-based alerting.
- `generate_tuning_recommendations` — Generate tuning recommendations for noisy rules.
- `build_fatigue_report` — Build comprehensive alert fatigue reduction report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-alert-fatigue-reduction`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-anti-phishing-training-program (Skill Module)

implementing-anti-phishing-training-program\SKILL.md

name: implementing-anti-phishing-training-program
description: Agent for managing and analyzing anti-phishing training program metrics. Use when the user asks to implement anti phishing training program.
version: 1.0.0
tags:
source: implementing-anti-phishing-training-program

Implementing Anti Phishing Training Program

Role: Security control implementation
Audience: security architects and engineers
Primary tools: KnowBe4 API, Key Metrics, pandas Simulation Analysis, SANS Maturity Model Levels, NIST Guidelines, Regulatory Requirements

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** anti phishing training program.
- Capability: Agent for managing and analyzing anti-phishing training program metrics.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, anti, phishing, training, program`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load phishing simulation results CSV
2. Calculate phishing susceptibility metrics per department
3. Analyze phishing simulation trends over time
4. Identify users who repeatedly click phishing links
5. Calculate training module completion rates
6. Generate overall organization risk score based on phishing metrics
7. Generate training recommendations based on metrics

Key concepts

- NIST SP 800-50: Building an Information Technology Security Awareness and Training Program
- NIST SP 800-16: Information Technology Security Training Requirements
- NIST SP 800-53 Rev.5: AT-1 through AT-6 - Awareness and Training family
- PCI DSS 4.0: Requirement 12.6 - Security awareness training for all personnel
- HIPAA: 45 CFR 164.308(a)(5) - Security awareness and training
- SOX: Section 404 - Internal controls requiring security awareness
- GDPR: Article 39(1)(b) - Data protection awareness training
- CMMC 2.0: AT.L2-3.2.1/2/3 - Awareness and training practices

Tools & resources

- KnowBe4 API
- Key Metrics
- pandas Simulation Analysis

- SANS Maturity Model Levels
- NIST Guidelines
- Regulatory Requirements
- Industry Frameworks
- Training Content Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_simulation_results` — Load phishing simulation results CSV.
- `calculate_department_metrics` — Calculate phishing susceptibility metrics per department.
- `analyze_trend` — Analyze phishing simulation trends over time.
- `identify_repeat_clickers` — Identify users who repeatedly click phishing links.
- `calculate_training_completion` — Calculate training module completion rates.
- `generate_risk_score` — Generate overall organization risk score based on phishing metrics.
- `recommend_training` — Generate training recommendations based on metrics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-anti-phishing-training-program`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-anti-ransomware-group-policy (Skill Module)

implementing-anti-ransomware-group-policy\SKILL.md

name: implementing-anti-ransomware-group-policy
description: Anti-ransomware Group Policy audit and configuration agent. Audits Windows endpoints for GPO-based ransomware defenses including AppLocker rules, Controlled Folder Access, Attack Surface Reduction rules, and network.... Use when the user asks to implement anti ransomware group policy.
version: 1.0.0
tags:
source: implementing-anti-ransomware-group-policy

Implementing Anti Ransomware Group Policy

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AppLocker PowerShell Cmdlets, GPO Paths Reference, GPResult Verification

When to use

- The user wants to **implement** anti ransomware group policy.
- Capability: Anti-ransomware Group Policy audit and configuration agent. Audits Windows endpoints for GPO-based ransomware defenses including AppLocker rules, Controlled Folder Access, Attack Surface Reduction rules, and network restriction settings. Generates compliance reports against recommended baselines.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, anti, ransomware, group, policy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run a single GPO compliance check
2. Assess ASR rule compliance against recommended baseline
3. Generate a full GPO anti-ransomware compliance report
4. Generate PowerShell remediation commands for failed checks

Tools & resources

- AppLocker PowerShell Cmdlets
- GPO Paths Reference
- GPResult Verification

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_check` — Run a single GPO compliance check.
- `assess_asr_compliance` — Assess ASR rule compliance against recommended baseline.
- `generate_gpo_report` — Generate a full GPO anti-ransomware compliance report.
- `generate_remediation_script` — Generate PowerShell remediation commands for failed checks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-anti-ransomware-group-policy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-abuse-detection-with-rate-limiting (Skill Module)

implementing-api-abuse-detection-with-rate-limiting\SKILL.md

name: implementing-api-abuse-detection-with-rate-limiting
description: Agent for implementing API abuse detection with rate limiting analysis. Use when the user asks to implement api abuse detection with rate limiting.
version: 1.0.0
tags:
source: implementing-api-abuse-detection-with-rate-limiting

Implementing API Abuse Detection with Rate Limiting

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Rate Limit Response Headers, NGINX Rate Limiting, Abuse Detection Thresholds, Flask-Limiter

When to use

- The user wants to **implement** api abuse detection with rate limiting.
- Capability: Agent for implementing API abuse detection with rate limiting analysis.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, abuse, detection, rate, limiting`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load API access logs from JSON lines
2. Detect brute force attacks by counting auth failures per IP
3. Detect API scraping by high request volume per IP
4. Detect credential stuffing: many unique usernames from single IP
5. Detect attempts to bypass rate limiting
6. Generate recommended rate limit configuration based on traffic patterns

Tools & resources

- Rate Limit Response Headers
- NGINX Rate Limiting
- Abuse Detection Thresholds
- Flask-Limiter

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_access_logs` — Load API access logs from JSON lines.
- `detect_brute_force` — Detect brute force attacks by counting auth failures per IP.
- `detect_api_scraping` — Detect API scraping by high request volume per IP.
- `detect_credential_stuffing` — Detect credential stuffing: many unique usernames from single IP.
- `detect_rate_limit_bypass` — Detect attempts to bypass rate limiting.
- `generate_rate_limit_config` — Generate recommended rate limit configuration based on traffic patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-abuse-detection-with-rate-limiting`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-gateway-security-controls (Skill Module)

implementing-api-gateway-security-controls\SKILL.md

name: implementing-api-gateway-security-controls
description: Agent for auditing API gateway security controls (Kong, AWS API Gateway). Use when the user asks to implement api gateway security controls.
version: 1.0.0
tags:
source: implementing-api-gateway-security-controls

Implementing API Gateway Security Controls

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Kong Admin API, Security Controls Checklist, NGINX Gateway Security

When to use

- The user wants to **implement** api gateway security controls.
- Capability: Agent for auditing API gateway security controls (Kong, AWS API Gateway).
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, gateway, security, controls`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for api gateway security controls
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Kong Admin API
- Security Controls Checklist
- NGINX Gateway Security

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `audit_aws_api_gateway` — Audit AWS API Gateway configurations for security issues.
- `audit_kong_gateway`
- `analyze_gateway_logs` — Analyze API gateway access logs for security issues.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-gateway-security-controls`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-key-security-controls (Skill Module)

implementing-api-key-security-controls\SKILL.md

name: implementing-api-key-security-controls
description: Agent for implementing and auditing API key security controls. Use when the user asks to implement api key security controls.
version: 1.0.0
tags:
source: implementing-api-key-security-controls

Implementing API Key Security Controls

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Secure Key Generation, Key Rotation Policy, TruffleHog Scanning, GitHub Secret Scanning API

When to use

- The user wants to **implement** api key security controls.
- Capability: Agent for implementing and auditing API key security controls.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, controls`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate a secure API key with prefix and checksum
2. Hash an API key for secure storage using SHA-256
3. Scan files for leaked API key patterns
4. Audit API key age and rotation compliance
5. Analyze API key usage patterns for anomalies

Tools & resources

- Secure Key Generation
- Key Rotation Policy
- TruffleHog Scanning
- GitHub Secret Scanning API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_api_key` — Generate a secure API key with prefix and checksum.
- `hash_api_key` — Hash an API key for secure storage using SHA-256.
- `scan_for_leaked_keys` — Scan files for leaked API key patterns.
- `audit_key_rotation` — Audit API key age and rotation compliance.
- `analyze_key_usage` — Analyze API key usage patterns for anomalies.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-key-security-controls`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-rate-limiting-and-throttling (Skill Module)

implementing-api-rate-limiting-and-throttling\SKILL.md

name: implementing-api-rate-limiting-and-throttling
description: Agent for implementing and testing API rate limiting and throttling. Use when the user asks to implement api rate limiting and throttling.
version: 1.0.0
tags:
source: implementing-api-rate-limiting-and-throttling

Implementing API Rate Limiting and Throttling

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Token Bucket Algorithm, Redis Sliding Window, HTTP 429 Response Headers, Kong Rate Limiting Plugin

When to use

- The user wants to **implement** api rate limiting and throttling.
- Capability: Agent for implementing and testing API rate limiting and throttling.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, rate, limiting, throttling`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze API logs to assess rate limiting effectiveness
2. Simulate rate limiting to test configuration
3. Generate rate limit recommendations from traffic patterns

Tools & resources

- Token Bucket Algorithm
- Redis Sliding Window
- HTTP 429 Response Headers
- Kong Rate Limiting Plugin

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_rate_limit_effectiveness` — Analyze API logs to assess rate limiting effectiveness.
- `simulate_rate_limit_test` — Simulate rate limiting to test configuration.
- `generate_rate_limit_recommendations` — Generate rate limit recommendations from traffic patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-rate-limiting-and-throttling`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-schema-validation-security (Skill Module)

`implementing-api-schema-validation-security\SKILL.md`

name: implementing-api-schema-validation-security
description: Agent for auditing API schema validation security using OpenAPI specs. Use when the user asks to implement api schema validation security.
version: 1.0.0
tags:
source: implementing-api-schema-validation-security

Implementing API Schema Validation Security

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OpenAPI Security Checks

When to use

- The user wants to **implement** api schema validation security.
- Capability: Agent for auditing API schema validation security using OpenAPI specs.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, schema, validation, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load an OpenAPI specification file
2. Audit OpenAPI spec for missing schema validation
3. Check security scheme definitions in OpenAPI spec
4. Validate a request payload against a JSON schema

Tools & resources

- OpenAPI Security Checks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_openapi_spec` — Load an OpenAPI specification file.
- `audit_schema_validation` — Audit OpenAPI spec for missing schema validation.
- `check_security_definitions` — Check security scheme definitions in OpenAPI spec.
- `validate_request_payload` — Validate a request payload against a JSON schema.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-schema-validation-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-security-posture-management (Skill Module)

implementing-api-security-posture-management\SKILL.md

name: implementing-api-security-posture-management
description: Agent for API Security Posture Management - discovery, classification, and risk scoring. Use when the user asks to implement api security posture management.
version: 1.0.0
tags:
source: implementing-api-security-posture-management

Implementing API Security Posture Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: API Discovery from Traffic, API Sensitivity Classification, 42Crunch API Audit, Salt Security API

When to use

- The user wants to **implement** api security posture management.
- Capability: Agent for API Security Posture Management - discovery, classification, and risk scoring.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, posture, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover APIs from network traffic logs
2. Classify APIs by data sensitivity level
3. Calculate risk score for each API endpoint
4. Check security controls for discovered APIs

Tools & resources

- API Discovery from Traffic
- API Sensitivity Classification
- 42Crunch API Audit
- Salt Security API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_apis_from_traffic` — Discover APIs from network traffic logs.
- `classify_api_sensitivity` — Classify APIs by data sensitivity level.
- `score_api_risk` — Calculate risk score for each API endpoint.
- `check_api_security_controls` — Check security controls for discovered APIs.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-security-posture-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-security-testing-with-42crunch (Skill Module)

`implementing-api-security-testing-with-42crunch\SKILL.md`

name: implementing-api-security-testing-with-42crunch
description: Agent for API security testing using 42Crunch audit methodology. Use when the user asks to implement api security testing with 42crunch.
version: 1.0.0
tags:
source: implementing-api-security-testing-with-42crunch

Implementing API Security Testing with 42crunch

Role: Security control implementation
Audience: security architects and engineers
Primary tools: 42Crunch API Security Audit, Security Score Deductions

When to use

- The user wants to **implement** api security testing with 42crunch.
- Capability: Agent for API security testing using 42Crunch audit methodology.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, testing, 42crunch`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load OpenAPI spec
2. Perform static security audit of OpenAPI specification
3. Calculate security score (0-100) based on findings

Tools & resources

- 42Crunch API Security Audit
- Security Score Deductions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_spec` — Load OpenAPI spec.
- `audit_spec_security` — Perform static security audit of OpenAPI specification.
- `calculate_security_score` — Calculate security score (0-100) based on findings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-security-testing-with-42crunch`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-api-threat-protection-with-apigee (Skill Module)

`implementing-api-threat-protection-with-apigee\SKILL.md`

name: implementing-api-threat-protection-with-apigee
description: Agent for implementing API threat protection policies with Google Apigee. Use when the user asks to implement api threat protection with apigee.
version: 1.0.0
tags:
source: implementing-api-threat-protection-with-apigee

Implementing API Threat Protection with Apigee

Role: Security control implementation
Audience: security architects and engineers
Primary tools: JSONThreatProtection Policy, SpikeArrest Policy, RegularExpressionProtection, Apigee Management API, Recommended Policy Limits

When to use

- The user wants to **implement** api threat protection with apigee.
- Capability: Agent for implementing API threat protection policies with Google Apigee.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, threat, protection, apigee`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate Apigee JSONThreatProtection policy XML
2. Generate Apigee SpikeArrest policy XML
3. Generate RegularExpressionProtection policy for SQL/XSS
4. Analyze an Apigee proxy bundle for security policy gaps
5. Audit a threat protection policy for weak configurations

Tools & resources

- JSONThreatProtection Policy
- SpikeArrest Policy
- RegularExpressionProtection
- Apigee Management API
- Recommended Policy Limits

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_json_threat_policy` — Generate Apigee JSONThreatProtection policy XML.
- `generate_spike_arrest_policy` — Generate Apigee SpikeArrest policy XML.
- `generate_regex_protection_policy` — Generate RegularExpressionProtection policy for SQL/XSS.
- `analyze_apigee_proxy_bundle` — Analyze an Apigee proxy bundle for security policy gaps.
- `audit_threat_protection_config` — Audit a threat protection policy for weak configurations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-api-threat-protection-with-apigee`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-application-whitelisting-with-applocker (Skill Module)

`implementing-application-whitelisting-with-applocker\SKILL.md`

name: implementing-application-whitelisting-with-applocker
description: Agent for implementing and auditing AppLocker application whitelisting policies. Use when the user asks to implement application whitelisting with applocker.
version: 1.0.0
tags:
source: implementing-application-whitelisting-with-applocker

Implementing Application Whitelisting with Applocker

Role: Security control implementation
Audience: security architects and engineers
Primary tools: PowerShell AppLocker Management, Rule Collections, GPO Configuration Path, Default Rule Paths, Primary Standards, Compliance Mappings

When to use

- The user wants to **implement** application whitelisting with applocker.
- Capability: Agent for implementing and auditing AppLocker application whitelisting policies.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, application, whitelisting, applocker`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse an exported AppLocker policy XML file
2. Audit AppLocker rules for security weaknesses
3. Analyze AppLocker event logs for blocked and audit events
4. Generate a baseline AppLocker policy recommendation

Key concepts

- Scope: Comprehensive guidance on planning, implementing, and maintaining application whitelisting
- Key sections: Technology overview, planning process, policy creation, maintenance operations
- Publisher: Australian Cyber Security Centre
- Scope: Application control is Mitigation Strategy #1 in the Essential Eight
- Maturity levels: L1 (block executables in user profiles), L2 (block from all user-writable paths), L3 (Microsoft recommended bl
- Publisher: Center for Internet Security
- Relevance: CIS Controls v8 Control 2 requires software allowlisting for authorized applications
- WDAC and AppLocker Feature Availability: Comparison of capabilities between AppLocker and WDAC

Tools & resources

- PowerShell AppLocker Management
- Rule Collections
- GPO Configuration Path
- Default Rule Paths
- Primary Standards
- Compliance Mappings
- Microsoft Documentation
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_applocker_policy` — Parse an exported AppLocker policy XML file.
- `audit_applocker_rules` — Audit AppLocker rules for security weaknesses.
- `analyze_applocker_events` — Analyze AppLocker event logs for blocked and audit events.
- `generate_baseline_policy` — Generate a baseline AppLocker policy recommendation.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-application-whitelisting-with-applocker`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aqua-security-for-container-scanning (Skill Module)

implementing-aqua-security-for-container-scanning\SKILL.md

name: implementing-aqua-security-for-container-scanning
description: Container image vulnerability scanning agent using Trivy CLI via subprocess. Use when the user asks to implement aqua security for container scanning.
version: 1.0.0
tags:
source: implementing-aqua-security-for-container-scanning

Implementing Aqua Security for Container Scanning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: NIST SP 800-190

When to use

- The user wants to **implement** aqua security for container scanning.
- Capability: Container image vulnerability scanning agent using Trivy CLI via subprocess.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, aqua, security, container, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check if Trivy CLI is available
2. Scan a container image for vulnerabilities using Trivy
3. Scan a container image for misconfigurations
4. Scan a container image for embedded secrets
5. Generate SBOM in CycloneDX format for an image
6. Parse Trivy JSON output into structured vulnerability list
7. Compute severity summary from vulnerability list
8. Scan multiple container images and aggregate results

Tools & resources

- NIST SP 800-190

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_trivy_installed` — Check if Trivy CLI is available.
- `scan_image` — Scan a container image for vulnerabilities using Trivy.
- `scan_image_misconfig` — Scan a container image for misconfigurations.
- `scan_image_secrets` — Scan a container image for embedded secrets.
- `generate_sbom` — Generate SBOM in CycloneDX format for an image.
- `parse_vuln_results` — Parse Trivy JSON output into structured vulnerability list.
- `compute_summary` — Compute severity summary from vulnerability list.
- `scan_multiple_images` — Scan multiple container images and aggregate results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aqua-security-for-container-scanning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-attack-path-analysis-with-xm-cyber (Skill Module)

`implementing-attack-path-analysis-with-xm-cyber\SKILL.md`

name: implementing-attack-path-analysis-with-xm-cyber
description: Attack path analysis agent using XM Cyber REST API for exposure management. Use when the user asks to implement attack path analysis with xm cyber.
version: 1.0.0
tags:
source: implementing-attack-path-analysis-with-xm-cyber

Implementing Attack Path Analysis with Xm Cyber

Role: Security control implementation
Audience: security architects and engineers
Primary tools: XM Cyber Resources, Industry Frameworks, Related Technologies

When to use

- The user wants to **implement** attack path analysis with xm cyber.
- Capability: Attack path analysis agent using XM Cyber REST API for exposure management.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, attack, path, analysis, cyber`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze choke points to identify highest-impact remediation targets
2. Compute risk score based on attack path complexity and critical asset exposure
3. Generate comprehensive attack path analysis report

Key concepts

- Gartner CTEM: Continuous Threat Exposure Management framework (2022)
- MITRE ATT&CK: Lateral movement and privilege escalation techniques
- NIST CSF 2.0: Identify, Protect, Detect functions
- CIS Controls v8.1 Control 7: Continuous Vulnerability Management
- BloodHound/SharpHound: Active Directory attack path analysis
- PurpleKnight: AD security assessment
- CrowdStrike Falcon Exposure Management
- Tenable Identity Exposure

Tools & resources

- XM Cyber Resources
- Industry Frameworks
- Related Technologies

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_choke_points` — Analyze choke points to identify highest-impact remediation targets.
- `compute_risk_score` — Compute risk score based on attack path complexity and critical asset exposure.
- `generate_report` — Generate comprehensive attack path analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-attack-path-analysis-with-xm-cyber`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-attack-surface-management (Skill Module)

implementing-attack-surface-management\SKILL.md

name: implementing-attack-surface-management
description: Agent for implementing external attack surface management (EASM). Combines Shodan, Censys, ProjectDiscovery tools (subfinder, httpx, nuclei), and a custom exposure scoring algorithm for comprehensive ASM. DISCLAIMER:.... Use when the user asks to implement attack surface management.
version: 1.0.0
tags:
source: implementing-attack-surface-management

Implementing Attack Surface Management

Role: Security control implementation
Audience: security architects and engineers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** attack surface management.
- Capability: Agent for implementing external attack surface management (EASM). Combines Shodan, Censys, ProjectDiscovery tools (subfinder, httpx, nuclei), and a custom exposure scoring algorithm for comprehensive ASM. DISCLAIMER: This tool is intended for authorized security testing and attack surface management only. Ensure you have written authorization before scanning any targets. Unauthorized scanning of systems you do not own or have explicit permission to test is illegal and unethical.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, attack, surface, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for attack surface management
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-attack-surface-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aws-config-rules-for-compliance (Skill Module)

`implementing-aws-config-rules-for-compliance\SKILL.md`

name: implementing-aws-config-rules-for-compliance
description: AWS Config compliance monitoring agent using boto3. Use when the user asks to implement aws config rules for compliance.
version: 1.0.0
tags:
source: implementing-aws-config-rules-for-compliance

Implementing AWS Config Rules for Compliance

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, Managed Rule Source Identifiers, SSM Remediation Documents, Conformance Packs, External References

When to use

- The user wants to **implement** aws config rules for compliance.
- Capability: AWS Config compliance monitoring agent using boto3.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, config, rules, compliance`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create AWS Config client
2. Verify AWS Config recorder is running
3. Deploy AWS-managed Config rules for CIS compliance
4. Get compliance summary across all Config rules
5. List non-compliant resources for a specific rule
6. Set up auto-remediation for a Config rule
7. Run a full compliance audit and generate report

Tools & resources

- AWS
- Managed Rule Source Identifiers
- SSM Remediation Documents
- Conformance Packs
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_config_client` — Create AWS Config client.
- `check_recorder_status` — Verify AWS Config recorder is running.
- `deploy_managed_rules` — Deploy AWS-managed Config rules for CIS compliance.
- `get_compliance_summary` — Get compliance summary across all Config rules.
- `get_non_compliant_resources` — List non-compliant resources for a specific rule.
- `configure_remediation` — Set up auto-remediation for a Config rule.
- `run_compliance_audit` — Run a full compliance audit and generate report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aws-config-rules-for-compliance`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aws-iam-permission-boundaries (Skill Module)

implementing-aws-iam-permission-boundaries\SKILL.md

name: implementing-aws-iam-permission-boundaries
description: AWS IAM permission boundary management agent using boto3. Use when the user asks to implement aws iam permission boundaries.
version: 1.0.0
tags:
source: implementing-aws-iam-permission-boundaries

Implementing AWS IAM Permission Boundaries

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, boto3 IAM Methods Used

When to use

- The user wants to **implement** aws iam permission boundaries.
- Capability: AWS IAM permission boundary management agent using boto3.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, permission, boundaries`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create IAM client with optional profile
2. Create a permission boundary policy restricting services and regions
3. Attach permission boundary to an IAM role
4. Find IAM roles that lack a permission boundary
5. Audit effective permissions for a role with boundary
6. Generate permission boundary compliance report

Key concepts

- SEC02-BP02: Use temporary credentials (permission boundaries enforce this)
- SEC02-BP05: Audit and rotate credentials regularly
- SEC02-BP06: Employ user groups and attributes for fine-grained access
- SEC03-BP01: Define access requirements (boundary defines maximum)
- SEC03-BP02: Grant least privilege access
- SEC03-BP06: Manage access based on lifecycle
- SEC03-BP07: Analyze public and cross-account access
- AC-2: Account Management (boundary controls role creation)

Tools & resources

- AWS
- boto3 IAM Methods Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_iam_client` — Create IAM client with optional profile.
- `create_permission_boundary` — Create a permission boundary policy restricting services and regions.
- `attach_boundary_to_role` — Attach permission boundary to an IAM role.
- `audit_roles_without_boundary` — Find IAM roles that lack a permission boundary.
- `audit_boundary_effectiveness` — Audit effective permissions for a role with boundary.
- `generate_report` — Generate permission boundary compliance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aws-iam-permission-boundaries`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aws-macie-for-data-classification (Skill Module)

implementing-aws-macie-for-data-classification\SKILL.md

name: implementing-aws-macie-for-data-classification
description: AWS Macie data classification agent using boto3 for S3 sensitive data discovery. Use when the user asks to implement aws macie for data classification.
version: 1.0.0
tags:
source: implementing-aws-macie-for-data-classification

Implementing AWS Macie for Data Classification

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, boto3 Macie2 Methods Used, Compliance Frameworks Supported, NIST 800-53 Controls

When to use

- The user wants to **implement** aws macie for data classification.
- Capability: AWS Macie data classification agent using boto3 for S3 sensitive data discovery.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, macie, data, classification`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create Macie2 client
2. Enable Macie in the account if not already enabled
3. Get Macie's summary of S3 bucket inventory
4. Create a one-time sensitive data discovery job for specified buckets
5. Get statistics on Macie findings by severity and type
6. List recent Macie findings filtered by severity
7. Generate Macie data classification report

Key concepts

- GDPR Article 30: Records of processing activities
- CCPA: California Consumer Privacy Act data discovery
- HIPAA: Protected health information identification
- PCI DSS 4.0: Cardholder data discovery (Requirement 3)
- SOC 2: Data classification and protection controls
- SEC 8: Protect data at rest
- SEC 10: Prepare for security events
- RA-5: Vulnerability Monitoring and Scanning

Tools & resources

- AWS
- boto3 Macie2 Methods Used
- Compliance Frameworks Supported
- NIST 800-53 Controls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_macie_client` — Create Macie2 client.
- `enable_macie` — Enable Macie in the account if not already enabled.
- `list_s3_buckets_summary` — Get Macie's summary of S3 bucket inventory.
- `create_classification_job` — Create a one-time sensitive data discovery job for specified buckets.
- `get_finding_statistics` — Get statistics on Macie findings by severity and type.
- `list_findings` — List recent Macie findings filtered by severity.
- `generate_report` — Generate Macie data classification report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aws-macie-for-data-classification`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aws-nitro-enclave-security (Skill Module)

implementing-aws-nitro-enclave-security\SKILL.md

name: implementing-aws-nitro-enclave-security
description: AWS Nitro Enclave Security Agent - Validates enclave attestation, audits KMS policies, and verifies enclave isolation. Use when the user asks to implement aws nitro enclave security.
version: 1.0.0
tags:
source: implementing-aws-nitro-enclave-security

Implementing AWS Nitro Enclave Security

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, Arguments, Key Functions, Exit Codes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** aws nitro enclave security.
- Capability: AWS Nitro Enclave Security Agent - Validates enclave attestation, audits KMS policies, and verifies enclave isolation.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, nitro, enclave, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Find EC2 instances with Nitro Enclave support enabled
2. Audit a KMS key policy for Nitro Enclave attestation conditions
3. Check if an IAM role has appropriate permissions for enclave operations
4. Check enclave allocator configuration via SSM (if available)
5. Validate the structure of a base64-encoded attestation document
6. Search CloudTrail for enclave-related security events
7. Generate comprehensive Nitro Enclave security assessment report

Tools & resources

- AWS
- Arguments
- Key Functions
- Exit Codes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_nitro_instances` — Find EC2 instances with Nitro Enclave support enabled.
- `audit_kms_key_policy` — Audit a KMS key policy for Nitro Enclave attestation conditions.
- `audit_iam_role_for_enclave` — Check if an IAM role has appropriate permissions for enclave operations.
- `check_enclave_allocator_config` — Check enclave allocator configuration via SSM (if available).
- `validate_attestation_document_structure` — Validate the structure of a base64-encoded attestation document.
- `audit_cloudtrail_enclave_events` — Search CloudTrail for enclave-related security events.
- `generate_report` — Generate comprehensive Nitro Enclave security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aws-nitro-enclave-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aws-security-hub (Skill Module)

`implementing-aws-security-hub\SKILL.md`

name: implementing-aws-security-hub
description: AWS Security Hub CSPM agent using boto3 securityhub client. Use when the user asks to implement aws security hub.
version: 1.0.0
tags:
source: implementing-aws-security-hub

Implementing AWS Security Hub

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, Standard ARNs, External References

When to use

- The user wants to **implement** aws security hub.
- Capability: AWS Security Hub CSPM agent using boto3 securityhub client.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create Security Hub client
2. Enable Security Hub with default standards
3. Enable specific compliance standards
4. List all enabled security standards and their status
5. Retrieve active findings grouped by severity
6. Get compliance scores for all enabled standards
7. Create a custom Security Hub insight
8. Update findings workflow status in batch

Tools & resources

- AWS
- Standard ARNs
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_hub_client` — Create Security Hub client.
- `enable_security_hub` — Enable Security Hub with default standards.
- `enable_standards` — Enable specific compliance standards.
- `get_enabled_standards` — List all enabled security standards and their status.
- `get_findings_summary` — Retrieve active findings grouped by severity.
- `get_compliance_scores` — Get compliance scores for all enabled standards.
- `create_custom_insight` — Create a custom Security Hub insight.
- `batch_update_findings` — Update findings workflow status in batch.
- `run_security_hub_audit` — Run a full Security Hub audit and print report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aws-security-hub`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-aws-security-hub-compliance (Skill Module)

implementing-aws-security-hub-compliance\SKILL.md

name: implementing-aws-security-hub-compliance
description: AWS Security Hub compliance monitoring agent with automated remediation. Use when the user asks to implement aws security hub compliance.
version: 1.0.0
tags:
source: implementing-aws-security-hub-compliance

Implementing AWS Security Hub Compliance

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, Finding Filters, External References

When to use

- The user wants to **implement** aws security hub compliance.
- Capability: AWS Security Hub compliance monitoring agent with automated remediation.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, compliance`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create Security Hub and S3 clients
2. Get failed compliance findings, optionally filtered by standard
3. Analyze findings to identify compliance gaps by control and account
4. Block public access on an S3 bucket
5. Auto-remediate safe-to-fix findings (S3 public access)
6. Create a custom Security Hub insight for compliance tracking
7. Generate full compliance report

Tools & resources

- AWS
- Finding Filters
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_clients` — Create Security Hub and S3 clients.
- `get_compliance_findings` — Get failed compliance findings, optionally filtered by standard.
- `analyze_compliance_gaps` — Analyze findings to identify compliance gaps by control and account.
- `remediate_s3_public_access` — Block public access on an S3 bucket.
- `auto_remediate_findings` — Auto-remediate safe-to-fix findings (S3 public access).
- `create_compliance_insight` — Create a custom Security Hub insight for compliance tracking.
- `run_compliance_report` — Generate full compliance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-aws-security-hub-compliance`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-azure-ad-privileged-identity-management (Skill Module)

implementing-azure-ad-privileged-identity-management\SKILL.md

name: implementing-azure-ad-privileged-identity-management
description: Azure AD Privileged Identity Management agent using Microsoft Graph API via requests. Use when the user asks to implement azure ad privileged identity management.
version: 1.0.0
tags:
source: implementing-azure-ad-privileged-identity-management

Implementing Azure AD Privileged Identity Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Azure

When to use

- The user wants to **implement** azure ad privileged identity management.
- Capability: Azure AD Privileged Identity Management agent using Microsoft Graph API via requests.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, azure, privileged, identity, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Identify permanent (non-PIM) role assignments that should be converted to eligible
2. Calculate PIM coverage metrics
3. Generate PIM compliance report

Key concepts

- AC-2(4): Automated Audit Actions (PIM audit logs)
- AC-2(5): Inactivity Logout (time-bound activations)
- AC-6(1): Authorize Access to Security Functions
- AC-6(2): Non-Privileged Access for Non-Security Functions
- AC-6(5): Privileged Accounts (eligible vs. active)
- CC6.1: Logical and physical access controls
- CC6.2: Prior to issuing credentials, registration and authorization
- CC6.3: Authorize, modify, or remove access timely

Tools & resources

- Azure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_permanent_assignments` — Identify permanent (non-PIM) role assignments that should be converted to eligible.
- `compute_pim_coverage` — Calculate PIM coverage metrics.
- `generate_report` — Generate PIM compliance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-azure-ad-privileged-identity-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-azure-defender-for-cloud (Skill Module)

implementing-azure-defender-for-cloud\SKILL.md

name: implementing-azure-defender-for-cloud
description: Azure Defender for Cloud security posture agent using azure-mgmt-security. Use when the user asks to implement azure defender for cloud.
version: 1.0.0
tags:
source: implementing-azure-defender-for-cloud

Implementing Azure Defender for Cloud

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Azure, SecurityCenter Client Methods, Defender Plan Names, Severity Levels, External References

When to use

- The user wants to **implement** azure defender for cloud.
- Capability: Azure Defender for Cloud security posture agent using azure-mgmt-security.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, azure, defender, cloud`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create Azure Security Center client
2. List available Azure subscriptions
3. Retrieve the current secure score
4. List all unhealthy security assessments (recommendations)
5. Check which Defender plans are enabled
6. Retrieve active security alerts
7. Check regulatory compliance standard status
8. List Just-In-Time VM access policies

Tools & resources

- Azure
- SecurityCenter Client Methods
- Defender Plan Names
- Severity Levels
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_security_client` — Create Azure Security Center client.
- `list_subscriptions` — List available Azure subscriptions.
- `get_secure_score` — Retrieve the current secure score.
- `get_security_assessments` — List all unhealthy security assessments (recommendations).
- `get_pricing_tiers` — Check which Defender plans are enabled.
- `get_security_alerts` — Retrieve active security alerts.
- `get_regulatory_compliance` — Check regulatory compliance standard status.
- `get_jit_policies` — List Just-In-Time VM access policies.
- `run_defender_audit` — Run a full Defender for Cloud audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-azure-defender-for-cloud`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-beyondcorp-zero-trust-access-model (Skill Module)

`implementing-beyondcorp-zero-trust-access-model\SKILL.md`

name: implementing-beyondcorp-zero-trust-access-model
description: BeyondCorp zero trust access assessment agent using Google Cloud IAP API via requests. Use when the user asks to implement beyondcorp zero trust access model.
version: 1.0.0
tags:
source: implementing-beyondcorp-zero-trust-access-model

Implementing Beyondcorp Zero Trust Access Model

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Google Cloud APIs Used, NIST SP 800-207, Google BeyondCorp Papers, Google Cloud IAP Documentation, NIST SP 800-63-3

When to use

- The user wants to **implement** beyondcorp zero trust access model.
- Capability: BeyondCorp zero trust access assessment agent using Google Cloud IAP API via requests.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, beyondcorp, zero, trust, access, model`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Get access token from gcloud CLI
2. List IAP-protected resources in a GCP project
3. Get IAP settings for a specific resource
4. List Access Context Manager access levels
5. Audit IAM policy bindings for IAP-secured resources
6. Assess BeyondCorp zero trust posture for a project
7. Generate BeyondCorp zero trust assessment report

Key concepts

- Section 2: Zero Trust Tenets - defines the core principles BeyondCorp implements
- Section 3.1: Policy Engine (PE) and Policy Administrator (PA) - maps to IAP and Access Context Manager
- Section 3.2: Trust Algorithm - corresponds to Access Levels evaluation
- Section 4.1: Device Agent/Gateway-Based Deployment - matches BeyondCorp connector model
- Identity Pillar: MFA enforcement, continuous validation - maps to IAP re-authentication
- Device Pillar: Device health monitoring, compliance enforcement - maps to Endpoint Verification
- Network Pillar: Micro-segmentation, encrypted traffic - maps to IAP tunnel encryption
- Application Pillar: Application access authorization - maps to per-service IAP policies

Tools & resources

- Google Cloud APIs Used
- NIST SP 800-207
- Google BeyondCorp Papers
- Google Cloud IAP Documentation
- NIST SP 800-63-3

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_gcloud_token` — Get access token from gcloud CLI.
- `list_iap_resources` — List IAP-protected resources in a GCP project.
- `get_iap_settings` — Get IAP settings for a specific resource.
- `list_access_levels` — List Access Context Manager access levels.
- `audit_iap_bindings` — Audit IAM policy bindings for IAP-secured resources.
- `assess_zero_trust_posture` — Assess BeyondCorp zero trust posture for a project.
- `generate_report` — Generate BeyondCorp zero trust assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-beyondcorp-zero-trust-access-model`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-bgp-security-with-rpki (Skill Module)

implementing-bgp-security-with-rpki\SKILL.md

name: implementing-bgp-security-with-rpki
description: BGP RPKI validation agent using RIPEstat and Cloudflare RPKI APIs. Use when the user asks to implement bgp security with rpki.
version: 1.0.0
tags:
source: implementing-bgp-security-with-rpki

Implementing Bgp Security with Rpki

Role: Security control implementation
Audience: security architects and engineers
Primary tools: APIs Used

When to use

- The user wants to **implement** bgp security with rpki.
- Capability: BGP RPKI validation agent using RIPEstat and Cloudflare RPKI APIs.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, rpki`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Validate a prefix against RPKI using RIPEstat
2. Get Route Origin Authorizations for an ASN from Cloudflare RPKI
3. Get prefix routing overview from RIPEstat
4. Check RPKI adoption status for an ASN
5. Validate multiple prefixes against RPKI
6. Generate RPKI validation report for an ASN and its prefixes

Tools & resources

- APIs Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `validate_prefix_rpki` — Validate a prefix against RPKI using RIPEstat.
- `get_roas_for_asn` — Get Route Origin Authorizations for an ASN from Cloudflare RPKI.
- `get_prefix_overview` — Get prefix routing overview from RIPEstat.
- `check_rpki_adoption` — Check RPKI adoption status for an ASN.
- `validate_multiple_prefixes` — Validate multiple prefixes against RPKI.
- `generate_report` — Generate RPKI validation report for an ASN and its prefixes.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-bgp-security-with-rpki`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-browser-isolation-for-zero-trust (Skill Module)

implementing-browser-isolation-for-zero-trust\SKILL.md

name: implementing-browser-isolation-for-zero-trust
description: Agent for implementing browser isolation within a Zero Trust architecture. Deploys remote browser isolation (RBI) policies with URL categorization, content disarming and reconstruction (CDR), DLP controls, and.... Use when the user asks to implement browser isolation for zero trust.
version: 1.0.0
tags:
source: implementing-browser-isolation-for-zero-trust

Implementing Browser Isolation for Zero Trust

Role: Security control implementation
Audience: security architects and engineers
Primary tools: BrowserIsolationPolicyEngine

When to use

- The user wants to **implement** browser isolation for zero trust.
- Capability: Agent for implementing browser isolation within a Zero Trust architecture. Deploys remote browser isolation (RBI) policies with URL categorization, content disarming and reconstruction (CDR), DLP controls, and integration with Secure Web Gateway and ZTNA platforms. Based on Cloudflare Browser Isolation, Menlo Security, and Zscaler RBI approaches.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, browser, isolation, zero, trust`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for browser isolation for zero trust
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- BrowserIsolationPolicyEngine

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-browser-isolation-for-zero-trust`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-canary-tokens-for-network-intrusion (Skill Module)

implementing-canary-tokens-for-network-intrusion\SKILL.md

name: implementing-canary-tokens-for-network-intrusion
description: Agent for deploying and managing canary tokens for network intrusion detection. Supports DNS, HTTP, and AWS API key canary tokens via Canarytokens.org API and Thinkst Canary enterprise console. Provides webhook alert.... Use when the user asks to implement canary tokens for network intrusion.
version: 1.0.0
tags:
source: implementing-canary-tokens-for-network-intrusion

Implementing Canary Tokens for Network Intrusion

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Canarytokens.org Public API, Thinkst Canary Enterprise API

When to use

- The user wants to **implement** canary tokens for network intrusion.
- Capability: Agent for deploying and managing canary tokens for network intrusion detection. Supports DNS, HTTP, and AWS API key canary tokens via Canarytokens.org API and Thinkst Canary enterprise console. Provides webhook alert integration with Slack, Microsoft Teams, email, and generic HTTP endpoints.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, canary, tokens, network, intrusion`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate a unique canary token tracking identifier
2. Create a canary token via Canarytokens.org public API. Args: token_type: One of the SUPPORTED_TOKEN_TYPES keys email: Notification email address memo: Human-readable description for alert context webhook_url: Optional webhook URL for real-time alerts Returns: dict with token details from the API
3. Create a DNS canary token that alerts on any DNS resolution
4. Create an HTTP canary token that alerts on HTTP requests
5. Create an AWS API key canary token that alerts on any AWS API usage
6. Create a web image beacon canary token
7. Create a canary token via Thinkst Canary enterprise console API. Args: console_domain: Your Thinkst Canary console domain (e.g., 'yourcompany') auth_token: API authentication token kind: Token kind (dns, http, aws-id, doc-msword, etc.) memo: Description for the token flock_id: Optional flock identifier for grouping Returns: dict with token details from the Thinkst API
8. List all canary tokens from the Thinkst Canary console

Tools & resources

- Canarytokens.org Public API
- Thinkst Canary Enterprise API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_token_id` — Generate a unique canary token tracking identifier.
- `create_canarytoken` — Create a canary token via Canarytokens.org public API. Args: token_type: One of the SUPPORTED_TOKEN_TYPES keys email: Notification email address memo: Human-readable description for alert context webhook_url: Optional webhook URL for real-time alerts Returns: dict with token details from the API
- `create_dns_token` — Create a DNS canary token that alerts on any DNS resolution.
- `create_http_token` — Create an HTTP canary token that alerts on HTTP requests.
- `create_aws_key_token` — Create an AWS API key canary token that alerts on any AWS API usage.
- `create_web_image_token` — Create a web image beacon canary token.
- `thinkst_create_token` — Create a canary token via Thinkst Canary enterprise console API. Args: console_domain: Your Thinkst Canary console domain (e.g., 'yourcompany') auth_token: API authentication token kind: Token kind (dns, http, aws-id, doc-msword, etc.) memo: Description for the token flock_id: Optional flock identifier for grouping Returns: dict with token details from the Thinkst API
- `thinkst_list_tokens` — List all canary tokens from the Thinkst Canary console.
- `thinkst_get_alerts` — Retrieve triggered canary token alerts from Thinkst Canary console.
- `deploy_aws_credentials_file` — Deploy a fake AWS credentials file as a canary token. Places realistic-looking AWS credentials in the target path. When an attacker finds and uses these credentials, the canary token triggers an alert.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-canary-tokens-for-network-intrusion`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cisa-zero-trust-maturity-model (Skill Module)

implementing-cisa-zero-trust-maturity-model\SKILL.md

name: implementing-cisa-zero-trust-maturity-model
description: CISA Zero Trust Maturity Model assessment agent for organizational ZT posture evaluation. Use when the user asks to implement cisa zero trust maturity model.
version: 1.0.0
tags:
source: implementing-cisa-zero-trust-maturity-model

Implementing Cisa Zero Trust Maturity Model

Role: Security control implementation
Audience: security architects and engineers
Primary tools: CISA ZT Pillars, Primary Standards, Supporting Standards

When to use

- The user wants to **implement** cisa zero trust maturity model.
- Capability: CISA Zero Trust Maturity Model assessment agent for organizational ZT posture evaluation.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cisa, zero, trust, maturity, model`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Assess a single control's implementation status and maturity
2. Assess a single CISA ZT pillar based on control responses
3. Load assessment responses from JSON file
4. Compute overall zero trust maturity from pillar assessments
5. Generate prioritized recommendations based on assessment gaps
6. Generate CISA Zero Trust Maturity assessment report

Key concepts

- Source: Cybersecurity and Infrastructure Security Agency
- Scope: Federal agencies and organizations implementing zero trust
- Five Pillars: Identity, Devices, Networks, Applications & Workloads, Data
- Four Maturity Stages: Traditional, Initial, Advanced, Optimal
- Cross-Cutting: Visibility & Analytics, Automation & Orchestration, Governance
- Tenets: Never trust, always verify; assume breach; least privilege access
- Deployment Models: Device agent/gateway, enclave, resource portal
- Key Requirement: Policy decision point (PDP) and policy enforcement point (PEP)

Tools & resources

- CISA ZT Pillars
- Primary Standards
- Supporting Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `assess_control` — Assess a single control's implementation status and maturity.
- `assess_pillar` — Assess a single CISA ZT pillar based on control responses.
- `load_assessment_data` — Load assessment responses from JSON file.
- `compute_overall_maturity` — Compute overall zero trust maturity from pillar assessments.
- `generate_recommendations` — Generate prioritized recommendations based on assessment gaps.
- `generate_report` — Generate CISA Zero Trust Maturity assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cisa-zero-trust-maturity-model`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cloud-dlp-for-data-protection (Skill Module)

implementing-cloud-dlp-for-data-protection\SKILL.md

name: implementing-cloud-dlp-for-data-protection
description: Cloud DLP agent for sensitive data discovery using Google Cloud DLP and AWS Macie. Use when the user asks to implement cloud dlp for data protection.
version: 1.0.0
tags:
source: implementing-cloud-dlp-for-data-protection

Implementing Cloud Dlp for Data Protection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: GCP DLP Info Types, De-identification Methods, Macie Finding Types, External References

When to use

- The user wants to **implement** cloud dlp for data protection.
- Capability: Cloud DLP agent for sensitive data discovery using Google Cloud DLP and AWS Macie.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cloud, data, protection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan text content for sensitive data using Google Cloud DLP
2. De-identify sensitive data in text using masking
3. Enable Amazon Macie for S3 sensitive data discovery
4. Create a Macie classification job to scan S3 buckets
5. Retrieve Macie findings for sensitive data discoveries
6. Generate a DLP discovery report

Tools & resources

- GCP DLP Info Types
- De-identification Methods
- Macie Finding Types
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_text_with_gcp_dlp` — Scan text content for sensitive data using Google Cloud DLP.
- `deidentify_text_with_gcp` — De-identify sensitive data in text using masking.
- `enable_macie` — Enable Amazon Macie for S3 sensitive data discovery.
- `create_macie_classification_job` — Create a Macie classification job to scan S3 buckets.
- `get_macie_findings` — Retrieve Macie findings for sensitive data discoveries.
- `run_dlp_report` — Generate a DLP discovery report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cloud-dlp-for-data-protection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cloud-security-posture-management (Skill Module)

implementing-cloud-security-posture-management\SKILL.md

name: implementing-cloud-security-posture-management
description: Cloud Security Posture Management (CSPM) agent across AWS, Azure, and GCP. Use when the user asks to implement cloud security posture management.
version: 1.0.0
tags:
source: implementing-cloud-security-posture-management

Implementing Cloud Security Posture Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS Posture Check APIs, Prowler Check Categories, External References

When to use

- The user wants to **implement** cloud security posture management.
- Capability: Cloud Security Posture Management (CSPM) agent across AWS, Azure, and GCP.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cloud, security, posture, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Prowler CSPM scan against a cloud provider
2. Run basic AWS security posture checks using boto3
3. Generate a CSPM posture report from findings

Tools & resources

- AWS Posture Check APIs
- Prowler Check Categories
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_prowler_scan` — Run Prowler CSPM scan against a cloud provider.
- `check_aws_security_posture` — Run basic AWS security posture checks using boto3.
- `generate_posture_report` — Generate a CSPM posture report from findings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cloud-security-posture-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cloud-trail-log-analysis (Skill Module)

implementing-cloud-trail-log-analysis\SKILL.md

name: implementing-cloud-trail-log-analysis
description: CloudTrail log analysis agent for security monitoring and threat detection. Use when the user asks to implement cloud trail log analysis.
version: 1.0.0
tags:
source: implementing-cloud-trail-log-analysis

Implementing Cloud Trail Log Analysis

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Lookup Attributes, CloudWatch Logs Insights, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** cloud trail log analysis.
- Capability: CloudTrail log analysis agent for security monitoring and threat detection.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cloud, trail, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create CloudTrail client
2. Look up CloudTrail events with optional filtering
3. Scan CloudTrail for suspicious API calls
4. Detect failed authentication attempts
5. Detect API calls from unauthorized AWS regions
6. Analyze all activity for a specific user
7. Run comprehensive CloudTrail security analysis

Tools & resources

- Lookup Attributes
- CloudWatch Logs Insights
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_cloudtrail_client` — Create CloudTrail client.
- `lookup_events` — Look up CloudTrail events with optional filtering.
- `detect_suspicious_activity` — Scan CloudTrail for suspicious API calls.
- `detect_failed_auth` — Detect failed authentication attempts.
- `detect_unauthorized_regions` — Detect API calls from unauthorized AWS regions.
- `analyze_user_activity` — Analyze all activity for a specific user.
- `run_cloudtrail_analysis` — Run comprehensive CloudTrail security analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cloud-trail-log-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cloud-vulnerability-posture-management (Skill Module)

implementing-cloud-vulnerability-posture-management\SKILL.md

name: implementing-cloud-vulnerability-posture-management
description: Cloud Security Posture Management agent using boto3 for AWS Security Hub and Prowler. Use when the user asks to implement cloud vulnerability posture management.
version: 1.0.0
tags:
source: implementing-cloud-vulnerability-posture-management

Implementing Cloud Vulnerability Posture Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: boto3 Security Hub Methods, Cloud Security Standards, Tools

When to use

- The user wants to **implement** cloud vulnerability posture management.
- Capability: Cloud Security Posture Management agent using boto3 for AWS Security Hub and Prowler.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cloud, vulnerability, posture, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create Security Hub client
2. Get Security Hub findings grouped by severity
3. Get compliance status across enabled security standards
4. Run Prowler security scan via subprocess
5. Generate CSPM report combining Security Hub and Prowler

Key concepts

- Relevance: Prescriptive hardening guidance for cloud service configurations
- Key Controls: AC-6 (Least Privilege), CM-6 (Configuration Settings), SC-7 (Boundary Protection)
- Relevance: Cloud-specific security control framework aligned with major compliance standards

Tools & resources

- boto3 Security Hub Methods
- Cloud Security Standards
- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_securityhub_client` — Create Security Hub client.
- `get_findings_summary` — Get Security Hub findings grouped by severity.
- `get_compliance_summary` — Get compliance status across enabled security standards.
- `run_prowler_scan` — Run Prowler security scan via subprocess.
- `generate_report` — Generate CSPM report combining Security Hub and Prowler.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cloud-vulnerability-posture-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cloud-waf-rules (Skill Module)

implementing-cloud-waf-rules\SKILL.md

name: implementing-cloud-waf-rules
description: Cloud WAF rules management agent using AWS WAFv2 boto3 client. Use when the user asks to implement cloud waf rules.
version: 1.0.0
tags:
source: implementing-cloud-waf-rules

Implementing Cloud WAF Rules

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS Managed Rule Groups, Rule Statement Types, Rule Actions, External References

When to use

- The user wants to **implement** cloud waf rules.
- Capability: Cloud WAF rules management agent using AWS WAFv2 boto3 client.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cloud, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create WAFv2 client
2. Create a new Web ACL with default block action
3. Add a managed rule group to an existing Web ACL
4. Create a rate-limiting rule for DDoS/brute-force protection
5. Create a geo-blocking rule for specified country codes
6. List all Web ACLs
7. Get sampled requests for WAF rule analysis
8. Run WAF configuration audit

Tools & resources

- AWS Managed Rule Groups
- Rule Statement Types
- Rule Actions
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_waf_client` — Create WAFv2 client.
- `create_web_acl` — Create a new Web ACL with default block action.
- `add_managed_rule_group` — Add a managed rule group to an existing Web ACL.
- `create_rate_limit_rule` — Create a rate-limiting rule for DDoS/brute-force protection.
- `create_geo_block_rule` — Create a geo-blocking rule for specified country codes.
- `list_web_acls` — List all Web ACLs.
- `get_sampled_requests` — Get sampled requests for WAF rule analysis.
- `run_waf_audit` — Run WAF configuration audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cloud-waf-rules`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-cloud-workload-protection (Skill Module)

implementing-cloud-workload-protection\SKILL.md

name: implementing-cloud-workload-protection
description: Agent for cloud workload protection on AWS EC2 instances. Use when the user asks to implement cloud workload protection.
version: 1.0.0
tags:
source: implementing-cloud-workload-protection

Implementing Cloud Workload Protection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: CloudWatch CPU Monitoring

When to use

- The user wants to **implement** cloud workload protection.
- Capability: Agent for cloud workload protection on AWS EC2 instances.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, cloud, workload, protection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. List all running EC2 instances
2. Execute commands on instances via SSM Run Command
3. Detect cryptomining processes on instances
4. Detect potential reverse shell connections
5. Check integrity of critical system files
6. Detect CPU usage anomalies via CloudWatch

Tools & resources

- CloudWatch CPU Monitoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_running_instances` — List all running EC2 instances.
- `run_ssm_command` — Execute commands on instances via SSM Run Command.
- `scan_for_cryptominers` — Detect cryptomining processes on instances.
- `scan_for_reverse_shells` — Detect potential reverse shell connections.
- `check_file_integrity` — Check integrity of critical system files.
- `check_cpu_anomaly` — Detect CPU usage anomalies via CloudWatch.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-cloud-workload-protection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-code-signing-for-artifacts (Skill Module)

implementing-code-signing-for-artifacts\SKILL.md

name: implementing-code-signing-for-artifacts
description: Code signing verification agent using cryptography library for Ed25519/RSA signature operations. Use when the user asks to implement code signing for artifacts.
version: 1.0.0
tags:
source: implementing-code-signing-for-artifacts

Implementing Code Signing for Artifacts

Role: Security control implementation
Audience: security architects and engineers
Primary tools: cryptography API Used, SLSA Framework, Executive Order 14028

When to use

- The user wants to **implement** code signing for artifacts.
- Capability: Code signing verification agent using cryptography library for Ed25519/RSA signature operations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, code, signing, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Compute hash digest of a file
2. Generate Ed25519 signing keypair
3. Sign a file artifact using Ed25519
4. Verify an Ed25519 signature against a file
5. Verify container image signature using cosign CLI
6. Verify signatures for multiple artifacts
7. Generate code signing verification report

Key concepts

- Document the build process (source, builder, dependencies)
- Provenance metadata available for all artifacts
- Build provenance is signed by the build platform
- Signatures tied to authenticated builder identity
- Build process runs on a hardened, isolated platform
- Provenance is non-falsifiable by the build service
- PS.2.1: Make integrity verification information available to consumers
- Code signing provides cryptographic proof of artifact integrity

Tools & resources

- cryptography API Used
- SLSA Framework
- Executive Order 14028

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compute_file_hash` — Compute hash digest of a file.
- `generate_ed25519_keypair` — Generate Ed25519 signing keypair.
- `sign_artifact` — Sign a file artifact using Ed25519.
- `verify_signature` — Verify an Ed25519 signature against a file.
- `verify_cosign_signature` — Verify container image signature using cosign CLI.
- `batch_verify` — Verify signatures for multiple artifacts.
- `generate_report` — Generate code signing verification report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-code-signing-for-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-conditional-access-policies-azure-ad (Skill Module)

implementing-conditional-access-policies-azure-ad\SKILL.md

name: implementing-conditional-access-policies-azure-ad
description: Azure AD Conditional Access policy audit agent using Microsoft Graph API. Use when the user asks to implement conditional access policies azure ad.
version: 1.0.0
tags:
source: implementing-conditional-access-policies-azure-ad

Implementing Conditional Access Policies Azure AD

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Azure

When to use

- The user wants to **implement** conditional access policies azure ad.
- Capability: Azure AD Conditional Access policy audit agent using Microsoft Graph API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, conditional, access, policies, azure`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Audit a single conditional access policy for security gaps
2. Check for essential baseline conditional access policies
3. Generate conditional access audit report

Tools & resources

- Azure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `audit_policy` — Audit a single conditional access policy for security gaps.

- `check_baseline_policies` — Check for essential baseline conditional access policies.
- `generate_report` — Generate conditional access audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-conditional-access-policies-azure-ad`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-conduit-security-for-ot-remote-access (Skill Module)

implementing-conduit-security-for-ot-remote-access\SKILL.md

name: implementing-conduit-security-for-ot-remote-access
description: OT remote access conduit security assessment agent for ICS/SCADA environments. Use when the user asks to implement conduit security for ot remote access.
version: 1.0.0
tags:
source: implementing-conduit-security-for-ot-remote-access

Implementing Conduit Security for Ot Remote Access

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OT Protocols Scanned, IEC 62443 Controls Checked

When to use

- The user wants to **implement** conduit security for ot remote access.
- Capability: OT remote access conduit security assessment agent for ICS/SCADA environments.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, conduit, security, remote, access`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan for exposed OT protocol ports on a target
2. Assess conduit security controls against IEC 62443 requirements
3. Compute conduit risk score based on controls and exposed ports
4. Generate OT conduit security assessment report

Tools & resources

- OT Protocols Scanned
- IEC 62443 Controls Checked

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_ot_ports` — Scan for exposed OT protocol ports on a target.
- `assess_conduit_controls` — Assess conduit security controls against IEC 62443 requirements.
- `compute_conduit_risk_score` — Compute conduit risk score based on controls and exposed ports.
- `generate_report` — Generate OT conduit security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-conduit-security-for-ot-remote-access`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-container-image-minimal-base-with-distroless (Skill Module)

implementing-container-image-minimal-base-with-distroless\SKILL.md

name: implementing-container-image-minimal-base-with-distroless
description: Distroless container image analysis agent using Trivy for comparing image security posture. Use when the user asks to implement container image minimal base with distroless.
version: 1.0.0
tags:
source: implementing-container-image-minimal-base-with-distroless

Implementing Container Image Minimal Base with Distroless

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Distroless Properties Checked, NIST SP 800-190, CIS Docker Benchmark v1.6, OWASP Docker Security

When to use

- The user wants to **implement** container image minimal base with distroless.
- Capability: Distroless container image analysis agent using Trivy for comparing image security posture.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, container, image, minimal, base, distroless`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan image with Trivy and return JSON results
2. Get image size using docker inspect
3. Count total packages found in Trivy scan
4. Count vulnerabilities by severity from Trivy results
5. Compare a standard base image against its distroless equivalent
6. Check if an image exhibits distroless properties (no shell, no package manager)
7. Generate distroless adoption report

Key concepts

- Section 3.1.1: Minimize image content to reduce attack surface
- Section 4.1.1: Use minimal base images for container builds
- D2: Patch Management Strategies (fewer packages = fewer patches)
- D3: Network Segmentation and Firewalling
- D4: Secure Defaults and Hardening (no shell = hardened by default)

Tools & resources

- Distroless Properties Checked
- NIST SP 800-190
- CIS Docker Benchmark v1.6
- OWASP Docker Security

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_trivy_scan` — Scan image with Trivy and return JSON results.
- `get_image_size` — Get image size using docker inspect.
- `count_packages` — Count total packages found in Trivy scan.
- `count_vulns_by_severity` — Count vulnerabilities by severity from Trivy results.
- `compare_images` — Compare a standard base image against its distroless equivalent.
- `check_distroless_properties` — Check if an image exhibits distroless properties (no shell, no package manager).
- `generate_report` — Generate distroless adoption report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-container-image-minimal-base-with-distroless`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-container-network-policies-with-calico (Skill Module)

implementing-container-network-policies-with-calico\SKILL.md

name: implementing-container-network-policies-with-calico
description: Agent for implementing container network policies with Calico. Audits Kubernetes network policies, identifies unprotected namespaces, validates Calico policy enforcement, and generates default-deny baseline policy.... Use when the user asks to implement container network policies with calico.
version: 1.0.0
tags:
source: implementing-container-network-policies-with-calico

Implementing Container Network Policies with Calico

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Kubernetes NetworkPolicy vs Calico, Default, Python kubernetes Client

When to use

- The user wants to **implement** container network policies with calico.
- Capability: Agent for implementing container network policies with Calico. Audits Kubernetes network policies, identifies unprotected namespaces, validates Calico policy enforcement, and generates default-deny baseline policy manifests.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, container, network, policies, calico`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for container network policies with calico
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Kubernetes NetworkPolicy vs Calico
- Default
- Python kubernetes Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-container-network-policies-with-calico`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-continuous-security-validation-with-bas (Skill Module)

implementing-continuous-security-validation-with-bas\SKILL.md

name: implementing-continuous-security-validation-with-bas
description: Breach and Attack Simulation (BAS) agent for continuous security validation using MITRE ATT&CK. Use when the user asks to implement continuous security validation with bas.
version: 1.0.0
tags:
source: implementing-continuous-security-validation-with-bas

Implementing Continuous Security Validation with Bas

Role: Security control implementation
Audience: security architects and engineers
Primary tools: BAS Platforms, Industry Standards, Key Metrics

When to use

- The user wants to **implement** continuous security validation with bas.
- Capability: Breach and Attack Simulation (BAS) agent for continuous security validation using MITRE ATT&CK.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, continuous, security, validation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Simulate a single ATT&CK technique and record detection status
2. Check if SIEM generated an alert for the simulated technique
3. Compute detection coverage across tested techniques
4. Run BAS simulation campaign and generate detection gap report

Key concepts

- Gartner BAS Market Guide: Breach and Attack Simulation Tools
- NIST CSF 2.0 DE.CM: Security Continuous Monitoring
- CIS Controls v8.1 Control 18: Penetration Testing
- Picus Security: 2024 Customers' Choice for BAS Tools
- Category evolution: BAS -> Adversarial Exposure Validation (2025)

Tools & resources

- BAS Platforms
- Industry Standards
- Key Metrics

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `simulate_technique` — Simulate a single ATT&CK technique and record detection status.
- `check_siem_detection` — Check if SIEM generated an alert for the simulated technique.
- `compute_detection_coverage` — Compute detection coverage across tested techniques.
- `generate_report` — Run BAS simulation campaign and generate detection gap report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-continuous-security-validation-with-bas`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-data-loss-prevention-with-microsoft-purview (Skill Module)

implementing-data-loss-prevention-with-microsoft-purview\SKILL.md

name: implementing-data-loss-prevention-with-microsoft-purview
description: Microsoft Purview DLP Management Agent - Automates DLP policy deployment and monitoring via Graph API. Use when the user asks to implement data loss prevention with microsoft purview.
version: 1.0.0
tags:
source: implementing-data-loss-prevention-with-microsoft-purview

Implementing Data Loss Prevention with Microsoft Purview

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Arguments, Key Classes, Key Functions, Output Files, Health Finding Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** data loss prevention with microsoft purview.
- Capability: Microsoft Purview DLP Management Agent - Automates DLP policy deployment and monitoring via Graph API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, data, loss, prevention, microsoft, purview`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Retrieve DLP alerts from Microsoft Graph Security API
2. Retrieve all sensitivity labels from the tenant
3. Evaluate DLP protection scope for a specific user
4. Generate summary statistics from DLP alerts
5. Generate a report of sensitivity label configuration
6. Analyze DLP policy health based on alert patterns
7. Export DLP alerts to CSV for compliance reporting
8. Generate comprehensive DLP compliance report

Tools & resources

- Arguments
- Key Classes
- Key Functions
- Output Files
- Health Finding Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_dlp_alerts` — Retrieve DLP alerts from Microsoft Graph Security API.
- `get_sensitivity_labels` — Retrieve all sensitivity labels from the tenant.
- `evaluate_dlp_protection_scope` — Evaluate DLP protection scope for a specific user.
- `generate_alert_summary` — Generate summary statistics from DLP alerts.
- `generate_label_report` — Generate a report of sensitivity label configuration.
- `check_policy_health` — Analyze DLP policy health based on alert patterns.
- `export_alerts_csv` — Export DLP alerts to CSV for compliance reporting.
- `generate_compliance_report` — Generate comprehensive DLP compliance report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-data-loss-prevention-with-microsoft-purview`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ddos-mitigation-with-cloudflare (Skill Module)

implementing-ddos-mitigation-with-cloudflare\SKILL.md

name: implementing-ddos-mitigation-with-cloudflare
description: DDoS mitigation agent using Cloudflare API for traffic analysis and rule management. Use when the user asks to implement ddos mitigation with cloudflare.
version: 1.0.0
tags:
source: implementing-ddos-mitigation-with-cloudflare

Implementing ddos Mitigation with Cloudflare

Role: Security control implementation
Audience: security architects and engineers

When to use

- The user wants to **implement** ddos mitigation with cloudflare.
- Capability: DDoS mitigation agent using Cloudflare API for traffic analysis and rule management.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, ddos, mitigation, cloudflare`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for ddos mitigation with cloudflare
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_traffic` — Analyze traffic patterns for DDoS indicators.
- `generate_report` — Generate DDoS mitigation posture report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ddos-mitigation-with-cloudflare`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-deception-based-detection-with-canarytoken (Skill Module)

implementing-deception-based-detection-with-canarytoken\SKILL.md

name: implementing-deception-based-detection-with-canarytoken
description: Agent for deploying and monitoring Canary Tokens via the Thinkst Canary API. Use when the user asks to implement deception based detection with canarytoken.
version: 1.0.0
tags:
source: implementing-deception-based-detection-with-canarytoken

Implementing Deception Based Detection with Canarytoken

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Supported Token Types

When to use

- The user wants to **implement** deception based detection with canarytoken.
- Capability: Agent for deploying and monitoring Canary Tokens via the Thinkst Canary API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, deception, detection, canarytoken`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create multiple tokens based on a deployment plan
2. Audit deployed token coverage and generate report
3. Run comprehensive Canarytoken deployment audit

Tools & resources

- Supported Token Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `create_deployment` — Create multiple tokens based on a deployment plan.

- `audit_token_coverage` — Audit deployed token coverage and generate report.
- `full_audit` — Run comprehensive Canarytoken deployment audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-deception-based-detection-with-canarytoken`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-delinea-secret-server-for-pam (Skill Module)

implementing-delinea-secret-server-for-pam\SKILL.md

name: implementing-delinea-secret-server-for-pam
description: Delinea Secret Server PAM agent for privileged credential management. Use when the user asks to implement delinea secret server for pam.
version: 1.0.0
tags:
source: implementing-delinea-secret-server-for-pam

Implementing Delinea Secret Server for PAM

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Secret Server REST API, Common Secret Templates, Search Filters, External References

When to use

- The user wants to **implement** delinea secret server for pam.
- Capability: Delinea Secret Server PAM agent for privileged credential management.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, delinea, secret, server`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for delinea secret server for pam
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Secret Server REST API
- Common Secret Templates
- Search Filters
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_pam_audit` — Run a PAM security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-delinea-secret-server-for-pam`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-device-posture-assessment-in-zero-trust (Skill Module)

implementing-device-posture-assessment-in-zero-trust\SKILL.md

name: implementing-device-posture-assessment-in-zero-trust
description: Device posture assessment agent for zero trust endpoint compliance evaluation. Use when the user asks to implement device posture assessment in zero trust.
version: 1.0.0
tags:
source: implementing-device-posture-assessment-in-zero-trust

Implementing Device Posture Assessment in Zero Trust

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Posture Checks, NIST SP 800-207, NIST SP 800-124r2, CrowdStrike ZTA Documentation, Microsoft Intune Compliance, Jamf Pro Compliance

When to use

- The user wants to **implement** device posture assessment in zero trust.
- Capability: Device posture assessment agent for zero trust endpoint compliance evaluation.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, device, posture, assessment, zero, trust`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check OS version and patch level
2. Check if disk encryption is enabled
3. Check if host firewall is enabled
4. Check if antivirus/EDR is running
5. Check if screen lock is configured with timeout
6. Compute device posture compliance score
7. Generate device posture assessment report

Key concepts

- Section 2, Tenet 5: "The enterprise monitors and measures the integrity and security posture of all owned and associated assets"
- Section 3.3: Agent/Gateway Model includes device health as access input
- Traditional: Limited visibility into device health
- Initial: Compliance enforcement via MDM
- Advanced: Continuous monitoring with automated remediation
- Optimal: Real-time posture integrated into every access decision
- Section 3.2: Device compliance checking requirements
- Section 4.1: MDM security capabilities for posture enforcement

Tools & resources

- Posture Checks
- NIST SP 800-207
- NIST SP 800-124r2
- CrowdStrike ZTA Documentation
- Microsoft Intune Compliance
- Jamf Pro Compliance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_os_version` — Check OS version and patch level.
- `check_disk_encryption` — Check if disk encryption is enabled.
- `check_firewall_status` — Check if host firewall is enabled.
- `check_antivirus` — Check if antivirus/EDR is running.
- `check_screen_lock` — Check if screen lock is configured with timeout.
- `compute_posture_score` — Compute device posture compliance score.
- `generate_report` — Generate device posture assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-device-posture-assessment-in-zero-trust`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-devsecops-security-scanning (Skill Module)

implementing-devsecops-security-scanning\SKILL.md

name: implementing-devsecops-security-scanning
description: DevSecOps security scanning pipeline agent. Orchestrates Semgrep (SAST), Trivy (container/SCA), and Gitleaks (secrets) scans via subprocess, aggregates findings, and enforces severity gates. Use when the user asks to implement devsecops security scanning.
version: 1.0.0
tags:
source: implementing-devsecops-security-scanning

Implementing Devsecops Security Scanning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: CI/CD Pipeline Gate Logic

When to use

- The user wants to **implement** devsecops security scanning.
- Capability: DevSecOps security scanning pipeline agent. Orchestrates Semgrep (SAST), Trivy (container/SCA), and Gitleaks (secrets) scans via subprocess, aggregates findings, and enforces severity gates.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, devsecops, security, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Semgrep SAST scan and return findings
2. Run Trivy vulnerability scan on image or filesystem
3. Run Gitleaks secret detection scan
4. Enforce security gate based on severity threshold
5. Aggregate findings from all scanners

Tools & resources

- CI/CD Pipeline Gate Logic

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_semgrep` — Run Semgrep SAST scan and return findings.
- `run_trivy` — Run Trivy vulnerability scan on image or filesystem.
- `run_gitleaks` — Run Gitleaks secret detection scan.
- `enforce_gate` — Enforce security gate based on severity threshold.
- `aggregate_report` — Aggregate findings from all scanners.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-devsecops-security-scanning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-diamond-model-analysis (Skill Module)

implementing-diamond-model-analysis\SKILL.md

name: implementing-diamond-model-analysis
description: Diamond Model intrusion analysis agent for structuring threat intelligence events. Use when the user asks to implement diamond model analysis.
version: 1.0.0
tags:
source: implementing-diamond-model-analysis

Implementing Diamond Model Analysis

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Applicable Standards, Industry Frameworks

When to use

- The user wants to **implement** diamond model analysis.
- Capability: Diamond Model intrusion analysis agent for structuring threat intelligence events.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, diamond, model, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create a Diamond Model event from the four vertices
2. Load Diamond Model events from JSON file
3. Pivot analysis: find all events sharing a vertex value
4. Build an activity thread for an adversary across events
5. Cluster events by shared infrastructure to identify campaigns
6. Compute statistics across all Diamond Model vertices
7. Generate Diamond Model analysis report

Key concepts

- STIX 2.1: Structured Threat Information eXpression for CTI data representation
- TAXII 2.1: Transport protocol for sharing CTI over HTTPS
- MITRE ATT&CK: Adversary tactics, techniques, and procedures taxonomy
- Diamond Model: Intrusion analysis framework (Adversary, Capability, Infrastructure, Victim)
- Traffic Light Protocol (TLP): Information sharing classification (CLEAR, GREEN, AMBER, RED)
- Technique mapping for threat actor behavior classification
- Data sources for detection capability assessment
- Mitigation strategies linked to specific techniques

Tools & resources

- Applicable Standards
- Industry Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `create_event` — Create a Diamond Model event from the four vertices.
- `load_events` — Load Diamond Model events from JSON file.
- `pivot_on_vertex` — Pivot analysis: find all events sharing a vertex value.
- `build_activity_thread` — Build an activity thread for an adversary across events.
- `cluster_by_infrastructure` — Cluster events by shared infrastructure to identify campaigns.
- `compute_vertex_statistics` — Compute statistics across all Diamond Model vertices.
- `generate_report` — Generate Diamond Model analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-diamond-model-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-digital-signatures-with-ed25519 (Skill Module)

`implementing-digital-signatures-with-ed25519\SKILL.md`

name: implementing-digital-signatures-with-ed25519
description: Ed25519 digital signature agent using the cryptography library. Use when the user asks to implement digital signatures with ed25519.
version: 1.0.0
tags:
source: implementing-digital-signatures-with-ed25519

Implementing Digital Signatures with Ed25519

Role: Security control implementation
Audience: security architects and engineers
Primary tools: cryptography API, Primary Standards, Related

When to use

- The user wants to **implement** digital signatures with ed25519.
- Capability: Ed25519 digital signature agent using the cryptography library.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, digital, signatures, ed25519`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate Ed25519 keypair and save to PEM files
2. Load Ed25519 private key from PEM file
3. Load Ed25519 public key from PEM file
4. Sign a message with Ed25519 private key
5. Sign a file with Ed25519 and write signature to .sig file
6. Verify an Ed25519 signature on a message
7. Verify a file's Ed25519 signature
8. Verify signatures for multiple files

Key concepts

- Description: Defines Ed25519 and Ed448 signature algorithms
- Description: SSH key format for Ed25519
- Description: Includes EdDSA as approved signature algorithm
- Description: Defines Curve25519 and Curve448
- Ed25519: `cryptography.hazmat.primitives.asymmetric.ed25519`
- Ed25519: `nacl.signing`
- Description: Original Ed25519 paper and reference implementation

Tools & resources

- cryptography API
- Primary Standards
- Related

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_keypair` — Generate Ed25519 keypair and save to PEM files.
- `load_private_key` — Load Ed25519 private key from PEM file.
- `load_public_key` — Load Ed25519 public key from PEM file.
- `sign_message` — Sign a message with Ed25519 private key.
- `sign_file` — Sign a file with Ed25519 and write signature to .sig file.
- `verify_message` — Verify an Ed25519 signature on a message.
- `verify_file` — Verify a file's Ed25519 signature.
- `batch_verify` — Verify signatures for multiple files.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-digital-signatures-with-ed25519`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-disk-encryption-with-bitlocker (Skill Module)

`implementing-disk-encryption-with-bitlocker\SKILL.md`

name: implementing-disk-encryption-with-bitlocker
description: Agent for auditing and managing BitLocker disk encryption across endpoints. Use when the user asks to implement disk encryption with bitlocker.
version: 1.0.0
tags:
source: implementing-disk-encryption-with-bitlocker

Implementing Disk Encryption with Bitlocker

Role: Security control implementation
Audience: security architects and engineers
Primary tools: manage-bde CLI, PowerShell BitLocker Cmdlets, Compliance Checks, Primary Standards, Compliance Mappings, Microsoft References

When to use

- The user wants to **implement** disk encryption with bitlocker.
- Capability: Agent for auditing and managing BitLocker disk encryption across endpoints.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, disk, encryption, bitlocker`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Get BitLocker status on local machine via manage-bde
2. Parse BitLocker compliance report (CSV or JSON)
3. Audit BitLocker compliance across fleet
4. Generate Group Policy recommendations for BitLocker
5. Calculate fleet encryption compliance metrics

Key concepts

- Scope: Guidance on planning, implementing, and maintaining storage encryption solutions
- Relevance: BitLocker uses FIPS 140-2 validated cryptographic modules when configured in FIPS-compliant mode

Tools & resources

- manage-bde CLI
- PowerShell BitLocker Cmdlets
- Compliance Checks
- Primary Standards
- Compliance Mappings
- Microsoft References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_bitlocker_status` — Get BitLocker status on local machine via manage-bde.
- `parse_bitlocker_report` — Parse BitLocker compliance report (CSV or JSON).
- `audit_bitlocker_compliance` — Audit BitLocker compliance across fleet.
- `generate_gpo_recommendations` — Generate Group Policy recommendations for BitLocker.
- `calculate_compliance_metrics` — Calculate fleet encryption compliance metrics.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-disk-encryption-with-bitLocker`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-dmarc-dkim-spf-email-security (Skill Module)

implementing-dmarc-dkim-spf-email-security\SKILL.md

name: implementing-dmarc-dkim-spf-email-security
description: Agent for implementing and auditing DMARC, DKIM, and SPF email security. Use when the user asks to implement dmarc dkim spf email security.
version: 1.0.0
tags:
source: implementing-dmarc-dkim-spf-email-security

Implementing Dmarc Dkim Spf Email Security

Role: Security control implementation
Audience: security architects and engineers
Primary tools: dnspython Lookups, DMARC Policy Levels, Recommended DNS Records, RFC Standards, NIST Guidelines, Government Mandates

When to use

- The user wants to **implement** dmarc dkim spf email security.
- Capability: Agent for implementing and auditing DMARC, DKIM, and SPF email security.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, dmarc, dkim, email, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check SPF record for a domain
2. Check DMARC record for a domain
3. Check DKIM record for a domain and selector
4. Audit multiple domains for email security records
5. Generate recommended SPF, DMARC, and DKIM DNS records

Key concepts

- RFC 7208 (SPF): Sender Policy Framework - authorizing sending hosts via DNS
- RFC 6376 (DKIM): DomainKeys Identified Mail Signatures
- RFC 7489 (DMARC): Domain-based Message Authentication, Reporting and Conformance
- RFC 8301: Cryptographic Algorithm and Key Usage Update to DKIM (mandates RSA 2048-bit minimum)
- RFC 8616: Email Authentication for Internationalized Mail
- RFC 8617 (ARC): Authenticated Received Chain - preserving authentication through forwarding
- RFC 7960: Interoperability Issues between DMARC and Indirect Email Flows
- RFC 6591: Authentication Failure Reporting Using ARF

Tools & resources

- dnspython Lookups
- DMARC Policy Levels
- Recommended DNS Records
- RFC Standards
- NIST Guidelines
- Government Mandates
- Industry Best Practices
- DMARC Tag Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_spf_record` — Check SPF record for a domain.
- `check_dmarc_record` — Check DMARC record for a domain.
- `check_dkim_record` — Check DKIM record for a domain and selector.
- `audit_domains` — Audit multiple domains for email security records.
- `generate_dns_records` — Generate recommended SPF, DMARC, and DKIM DNS records.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-dmarc-dkim-spf-email-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-dragos-platform-for-ot-monitoring (Skill Module)

implementing-dragos-platform-for-ot-monitoring\SKILL.md

name: implementing-dragos-platform-for-ot-monitoring
description: Agent for implementing Dragos Platform OT network monitoring. Use when the user asks to implement dragos platform for ot monitoring.
version: 1.0.0
tags:
source: implementing-dragos-platform-for-ot-monitoring

Implementing Dragos Platform for Ot Monitoring

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Dragos Platform API, Monitored OT Protocols, Detection Categories, ICS-CERT Vulnerability Feeds

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** dragos platform for ot monitoring.
- Capability: Agent for implementing Dragos Platform OT network monitoring.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, dragos, platform, monitoring`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query the Dragos Platform API
2. Retrieve OT asset inventory from Dragos
3. Retrieve threat detections from Dragos
4. Retrieve OT vulnerabilities from Dragos
5. Analyze OT protocol traffic from exported logs
6. Generate OT monitoring configuration template

Tools & resources

- Dragos Platform API
- Monitored OT Protocols
- Detection Categories
- ICS-CERT Vulnerability Feeds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_dragos_api` — Query the Dragos Platform API.
- `get_asset_inventory` — Retrieve OT asset inventory from Dragos.
- `get_threat_detections` — Retrieve threat detections from Dragos.
- `get_vulnerabilities` — Retrieve OT vulnerabilities from Dragos.
- `analyze_ot_protocols` — Analyze OT protocol traffic from exported logs.
- `generate_monitoring_config` — Generate OT monitoring configuration template.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-dragos-platform-for-ot-monitoring`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ebpf-security-monitoring (Skill Module)

implementing-ebpf-security-monitoring\SKILL.md

name: implementing-ebpf-security-monitoring
description: Agent for deploying and managing eBPF security monitoring with Cilium Tetragon. Use when the user asks to implement ebpf security monitoring.
version: 1.0.0
tags:
source: implementing-ebpf-security-monitoring

Implementing Ebpf Security Monitoring

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Common Kprobe Hook Points, Argument Types, Selector Operators, Match Actions, Tetragon gRPC API, Log Export Configuration

When to use

- The user wants to **implement** ebpf security monitoring.
- Capability: Agent for deploying and managing eBPF security monitoring with Cilium Tetragon.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, ebpf, security, monitoring`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Verify that required tools are available on the system
2. Generate a TracingPolicy for monitoring file access via fd_install kprobe
3. Generate a TracingPolicy for monitoring TCP connections via tcp_connect kprobe
4. Generate a TracingPolicy for detecting setuid(0) and commit_creds calls
5. Generate a TracingPolicy that kills crypto miner processes via Sigkill
6. Generate a TracingPolicy for detecting reverse shell network connections
7. Generate a TracingPolicy for detecting container escape attempts
8. Generate a TracingPolicy for monitoring file writes to sensitive paths

Tools & resources

- Common Kprobe Hook Points
- Argument Types
- Selector Operators
- Match Actions
- Tetragon gRPC API
- Log Export Configuration

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_prerequisites` — Verify that required tools are available on the system.
- `generate_file_access_policy` — Generate a TracingPolicy for monitoring file access via fd_install kprobe.
- `generate_network_monitor_policy` — Generate a TracingPolicy for monitoring TCP connections via tcp_connect kprobe.
- `generate_privilege_escalation_policy` — Generate a TracingPolicy for detecting setuid(0) and commit_creds calls.
- `generate_crypto_miner_enforcement_policy` — Generate a TracingPolicy that kills crypto miner processes via Sigkill.
- `generate_reverse_shell_detection_policy` — Generate a TracingPolicy for detecting reverse shell network connections.
- `generate_container_escape_policy` — Generate a TracingPolicy for detecting container escape attempts.
- `generate_write_monitoring_policy` — Generate a TracingPolicy for monitoring file writes to sensitive paths.
- `apply_policy` — Apply a TracingPolicy to the Kubernetes cluster or write to file.
- `list_tracing_policies` — List all TracingPolicy resources in the cluster.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ebpf-security-monitoring`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-email-sandboxing-with-proofpoint (Skill Module)

`implementing-email-sandboxing-with-proofpoint\SKILL.md`

name: implementing-email-sandboxing-with-proofpoint
description: Agent for implementing and monitoring Proofpoint email sandboxing. Use when the user asks to implement email sandboxing with proofpoint.
version: 1.0.0
tags:
source: implementing-email-sandboxing-with-proofpoint

Implementing Email Sandboxing with Proofpoint

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Proofpoint TAP SIEM API, Threat Categories, URL Defense Configuration, NIST Guidelines, Sandbox File Types

When to use

- The user wants to **implement** email sandboxing with proofpoint.
- Capability: Agent for implementing and monitoring Proofpoint email sandboxing.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, email, sandboxing, proofpoint`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Query Proofpoint TAP SIEM API for threats
2. Analyze Proofpoint sandbox detonation results
3. Calculate email security metrics from logs
4. Generate Proofpoint URL Defense configuration

Key concepts

- T1566.001: Phishing: Spearphishing Attachment (primary detection)
- T1566.002: Phishing: Spearphishing Link (URL Defense)
- T1204.001/002: User Execution: Malicious Link/File
- T1059: Command and Scripting Interpreter (macro detection)
- T1027: Obfuscated Files or Information
- NIST SP 800-177: Trustworthy Email - attachment security
- NIST SP 800-83 Rev.1: Guide to Malware Incident Prevention
- NIST SP 800-53: SI-3 Malicious Code Protection, SI-8 Spam Protection

Tools & resources

- Proofpoint TAP SIEM API
- Threat Categories
- URL Defense Configuration
- NIST Guidelines
- Sandbox File Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_tap_threats` — Query Proofpoint TAP SIEM API for threats.
- `analyze_sandbox_results` — Analyze Proofpoint sandbox detonation results.
- `calculate_email_metrics` — Calculate email security metrics from logs.
- `generate_url_defense_config` — Generate Proofpoint URL Defense configuration.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-email-sandboxing-with-proofpoint`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-end-to-end-encryption-for-messaging (Skill Module)

implementing-end-to-end-encryption-for-messaging\SKILL.md

name: implementing-end-to-end-encryption-for-messaging
description: Agent for implementing end-to-end encryption (E2EE) for messaging using X25519 + AES-GCM. Use when the user asks to implement end to end encryption for messaging.
version: 1.0.0
tags:
source: implementing-end-to-end-encryption-for-messaging

Implementing End to End Encryption for Messaging

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Cryptography API Calls, Signal Protocol Specifications, Cryptographic Standards

When to use

- The user wants to **implement** end to end encryption for messaging.
- Capability: Agent for implementing end-to-end encryption (E2EE) for messaging using X25519 + AES-GCM.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, encryption, messaging`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate X25519 key pair for Diffie-Hellman key exchange
2. Derive shared secret using X25519 ECDH + HKDF-SHA256
3. Encrypt a message using AES-256-GCM with a shared key
4. Decrypt a message using AES-256-GCM
5. Simulate a complete key exchange between two parties
6. Demonstrate complete E2EE message flow

Key concepts

- Description: Core key management algorithm for E2EE messaging
- Description: Initial key exchange using Extended Triple Diffie-Hellman
- Description: Multi-device session management
- Description: X25519 Diffie-Hellman key exchange
- Description: Key derivation for chain key updates
- Description: Identity key signatures
- Description: Authenticated encryption for messages
- X25519: `cryptography.hazmat.primitives.asymmetric.x25519`

Tools & resources

- Cryptography API Calls
- Signal Protocol Specifications
- Cryptographic Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_keypair` — Generate X25519 key pair for Diffie-Hellman key exchange.
- `derive_shared_secret` — Derive shared secret using X25519 ECDH + HKDF-SHA256.
- `encrypt_message` — Encrypt a message using AES-256-GCM with a shared key.
- `decrypt_message` — Decrypt a message using AES-256-GCM.
- `simulate_key_exchange` — Simulate a complete key exchange between two parties.
- `demo_full_flow` — Demonstrate complete E2EE message flow.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-end-to-end-encryption-for-messaging`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-endpoint-detection-with-wazuh (Skill Module)

`implementing-endpoint-detection-with-wazuh\SKILL.md`

name: implementing-endpoint-detection-with-wazuh
description: Agent for implementing endpoint detection with Wazuh. Manages Wazuh agents, queries alerts, tests custom decoder/rule logic via logtest, and audits endpoint coverage using the Wazuh REST API. Use when the user asks to implement endpoint detection with wazuh.
version: 1.0.0
tags:
source: implementing-endpoint-detection-with-wazuh

Implementing Endpoint Detection with Wazuh

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Wazuh, Custom Decoder XML

When to use

- The user wants to **implement** endpoint detection with wazuh.
- Capability: Agent for implementing endpoint detection with Wazuh. Manages Wazuh agents, queries alerts, tests custom decoder/rule logic via logtest, and audits endpoint coverage using the Wazuh REST API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, endpoint, detection, wazuh`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for endpoint detection with wazuh
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Wazuh
- Custom Decoder XML

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-endpoint-detection-with-wazuh`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-endpoint-dlp-controls (Skill Module)

implementing-endpoint-dlp-controls\SKILL.md

name: implementing-endpoint-dlp-controls
description: Agent for implementing and monitoring endpoint DLP controls. Use when the user asks to implement endpoint dlp controls.
version: 1.0.0
tags:
source: implementing-endpoint-dlp-controls

Implementing Endpoint Dlp Controls

Role: Security control implementation
Audience: security architects and engineers
Primary tools: DLP Channels, Microsoft Purview DLP API, CrowdStrike Falcon DLP, File Scanning

When to use

- The user wants to **implement** endpoint dlp controls.
- Capability: Agent for implementing and monitoring endpoint DLP controls.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, endpoint, controls`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan a single file for sensitive data patterns
2. Scan a directory for files containing sensitive data
3. Monitor file transfers to USB/removable devices
4. Monitor file uploads to cloud storage services
5. Generate endpoint DLP policy recommendations

Key concepts

- NIST SP 800-53 SC-7: Boundary Protection - DLP enforces data boundaries
- PCI DSS 4.0 Req 3: Protect stored account data
- GDPR Article 32: Security of processing - preventing unauthorized data transfer
- HIPAA 164.312(e)(1): Transmission security for ePHI

Tools & resources

- DLP Channels
- Microsoft Purview DLP API
- CrowdStrike Falcon DLP
- File Scanning

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_file_for_sensitive_data` — Scan a single file for sensitive data patterns.
- `scan_directory` — Scan a directory for files containing sensitive data.
- `monitor_usb_transfers` — Monitor file transfers to USB/removable devices.
- `monitor_cloud_uploads` — Monitor file uploads to cloud storage services.
- `generate_dlp_policy` — Generate endpoint DLP policy recommendations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-endpoint-dlp-controls`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-envelope-encryption-with-aws-kms (Skill Module)

`implementing-envelope-encryption-with-aws-kms\SKILL.md`

name: implementing-envelope-encryption-with-aws-kms
description: Agent for implementing envelope encryption using AWS KMS. Use when the user asks to implement envelope encryption with aws kms.
version: 1.0.0
tags:
source: implementing-envelope-encryption-with-aws-kms

Implementing Envelope Encryption with AWS Kms

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS, boto3 KMS API Calls, Cryptographic Standards, Compliance Frameworks

When to use

- The user wants to **implement** envelope encryption with aws kms.
- Capability: Agent for implementing envelope encryption using AWS KMS.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, envelope, encryption`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate a data encryption key using AWS KMS
2. Encrypt data using envelope encryption with AWS KMS
3. Decrypt envelope-encrypted data using AWS KMS
4. Encrypt a file using envelope encryption
5. Decrypt an envelope-encrypted file
6. List available KMS keys
7. Audit a KMS key's policy for overly permissive access

Key concepts

- Relevance: Key hierarchy and key wrapping concepts
- Description: AES Key Wrap specification used by KMS internally
- Description: KMS HSMs are validated at FIPS 140-2 Level 2 (Level 3 for CloudHSM)
- Key management with separation of DEK and KEK
- KMS satisfies key management requirements
- AWS KMS is SOC 2 compliant
- Encryption controls map to CC6.1 (logical access controls)
- KMS encryption satisfies encryption requirements for ePHI

Tools & resources

- AWS
- boto3 KMS API Calls
- Cryptographic Standards
- Compliance Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_data_key` — Generate a data encryption key using AWS KMS.
- `encrypt_data` — Encrypt data using envelope encryption with AWS KMS.
- `decrypt_data` — Decrypt envelope-encrypted data using AWS KMS.
- `encrypt_file` — Encrypt a file using envelope encryption.
- `decrypt_file` — Decrypt an envelope-encrypted file.
- `list_kms_keys` — List available KMS keys.
- `audit_key_policy` — Audit a KMS key's policy for overly permissive access.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-envelope-encryption-with-aws-kms`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-epss-score-for-vulnerability-prioritization (Skill Module)

implementing-epss-score-for-vulnerability-prioritization\SKILL.md

name: implementing-epss-score-for-vulnerability-prioritization
description: Agent for implementing EPSS (Exploit Prediction Scoring System) for vulnerability prioritization. Use when the user asks to implement epss score for vulnerability prioritization.
version: 1.0.0
tags:
source: implementing-epss-score-for-vulnerability-prioritization

Implementing Epss Score for Vulnerability Prioritization

Role: Security control implementation
Audience: security architects and engineers
Primary tools: FIRST.org EPSS API, Primary Standards, Research Papers

When to use

- The user wants to **implement** epss score for vulnerability prioritization.
- Capability: Agent for implementing EPSS (Exploit Prediction Scoring System) for vulnerability prioritization.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, epss, score, vulnerability, prioritization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Fetch EPSS scores for a list of CVE IDs from the FIRST.org API
2. Download the full EPSS score CSV from FIRST.org
3. Prioritize vulnerabilities based on EPSS score and percentile
4. Enrich a vulnerability scan CSV with EPSS scores

Key concepts

- Source: Forum of Incident Response and Security Teams
- Model: Machine learning trained on real exploitation events, updated daily
- Versions: v1 (2021), v2 (2022), v3 (2023), v4 (2025)
- Relevance: EPSS complements CVSS; CVSS measures severity, EPSS measures exploitation probability
- Relevance: SSVC uses exploitation status as a key decision point; EPSS provides data-driven input
- Relevance: KEV confirms active exploitation; EPSS predicts future exploitation probability
- Title: "Improving Vulnerability Remediation Through Better Exploit Prediction"
- Authors: Jay Jacobs, Sasha Romanosky, Benjamin Edwards, Michael Roytman, Idris Adjerid

Tools & resources

- FIRST.org EPSS API
- Primary Standards
- Research Papers

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_epss_scores` — Fetch EPSS scores for a list of CVE IDs from the FIRST.org API.
- `get_epss_csv` — Download the full EPSS score CSV from FIRST.org.
- `prioritize_vulnerabilities` — Prioritize vulnerabilities based on EPSS score and percentile.
- `enrich_from_scan` — Enrich a vulnerability scan CSV with EPSS scores.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-epss-score-for-vulnerability-prioritization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-file-integrity-monitoring-with-aide (Skill Module)

implementing-file-integrity-monitoring-with-aide\SKILL.md

name: implementing-file-integrity-monitoring-with-aide
description: AIDE (Advanced Intrusion Detection Environment) configuration generator and check runner. Use when the user asks to implement file integrity monitoring with aide.
version: 1.0.0
tags:
source: implementing-file-integrity-monitoring-with-aide

Implementing File Integrity Monitoring with Aide

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Available Check Attributes, Cron Automation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** file integrity monitoring with aide.
- Capability: AIDE (Advanced Intrusion Detection Environment) configuration generator and check runner.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, file, integrity, monitoring, aide`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate aide.conf configuration file content
2. Run aide --init to create the baseline database
3. Run aide --check and parse the results
4. Parse AIDE check output into structured findings
5. Generate cron job for automated AIDE integrity checks
6. Generate AIDE deployment and check report

Tools & resources

- Available Check Attributes
- Cron Automation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_aide_config` — Generate aide.conf configuration file content.
- `initialize_baseline` — Run aide --init to create the baseline database.
- `run_integrity_check` — Run aide --check and parse the results.
- `parse_aide_output` — Parse AIDE check output into structured findings.
- `generate_cron_config` — Generate cron job for automated AIDE integrity checks.
- `generate_report` — Generate AIDE deployment and check report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-file-integrity-monitoring-with-aide`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-fuzz-testing-in-cicd-with-aflplusplus (Skill Module)

implementing-fuzz-testing-in-cicd-with-aflplusplus\SKILL.md

name: implementing-fuzz-testing-in-cicd-with-aflplusplus
description: Agent for implementing AFL++ fuzz testing in CI/CD pipelines. Use when the user asks to implement fuzz testing in cicd with aflplusplus.
version: 1.0.0
tags:
source: implementing-fuzz-testing-in-cicd-with-aflplusplus

Implementing Fuzz Testing in Cicd with Aflplusplus

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OWASP Testing Guide v4.2, Fuzzing Maturity Levels

When to use

- The user wants to **implement** fuzz testing in cicd with aflplusplus.
- Capability: Agent for implementing AFL++ fuzz testing in CI/CD pipelines.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, fuzz, testing, cicd, aflplusplus`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Compile target binary with AFL++ instrumentation
2. Prepare and minimize seed corpus using afl-cmin
3. Minimize seed corpus using afl-cmin
4. Run AFL++ fuzzer for a specified duration
5. Parse AFL++ fuzzer_stats file into a dict
6. Triage crash inputs to deduplicate and classify

Key concepts

- WSTG-INPV-07: Testing for Input Validation --- AFL++ systematically tests boundary conditions
- WSTG-ERRH-01: Error Handling --- Crash analysis reveals improper error handling

Tools & resources

- OWASP Testing Guide v4.2
- Fuzzing Maturity Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `compile_target` — Compile target binary with AFL++ instrumentation.
- `prepare_corpus` — Prepare and minimize seed corpus using afl-cmin.
- `minimize_corpus` — Minimize seed corpus using afl-cmin.
- `run_fuzzer` — Run AFL++ fuzzer for a specified duration.
- `parse_fuzzer_stats` — Parse AFL++ fuzzer_stats file into a dict.
- `triage_crashes` — Triage crash inputs to deduplicate and classify.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-fuzz-testing-in-cicd-with-aflplusplus`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-gcp-binary-authorization (Skill Module)

implementing-gcp-binary-authorization\SKILL.md

name: implementing-gcp-binary-authorization
description: Agent for auditing and managing GCP Binary Authorization policies. Use when the user asks to implement gcp binary authorization.
version: 1.0.0
tags:
source: implementing-gcp-binary-authorization

Implementing GCP Binary Authorization

Role: Security control implementation
Audience: security architects and engineers
Primary tools: GCP, Policy Structure, Break-Glass Annotation, SLSA Framework Levels, NIST 800-53, CIS GKE Benchmark

When to use

- The user wants to **implement** gcp binary authorization.
- Capability: Agent for auditing and managing GCP Binary Authorization policies.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, binary, authorization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run a gcloud command and return parsed JSON output
2. Retrieve the Binary Authorization policy for a project
3. List all attestors in the project
4. List attestations for a given attestor
5. Check if a container image has a valid attestation
6. Audit a Binary Authorization policy for security issues
7. Generate a Binary Authorization policy YAML
8. Check continuous validation status on a GKE cluster

Key concepts

- SLSA 1: Documentation of build process
- SLSA 2: Tamper resistance of build service
- SLSA 3: Extra resistance to threats
- SLSA 4: Highest levels of confidence and trust
- SA-10: Developer Configuration Management
- SA-12: Supply Chain Protection
- SI-7: Software, Firmware, and Information Integrity

Tools & resources

- GCP
- Policy Structure
- Break-Glass Annotation
- SLSA Framework Levels
- NIST 800-53
- CIS GKE Benchmark

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_gcloud` — Run a gcloud command and return parsed JSON output.
- `get_binauthz_policy` — Retrieve the Binary Authorization policy for a project.
- `list_attestors` — List all attestors in the project.
- `list_attestations` — List attestations for a given attestor.
- `verify_image_attested` — Check if a container image has a valid attestation.
- `audit_policy` — Audit a Binary Authorization policy for security issues.
- `generate_policy` — Generate a Binary Authorization policy YAML.
- `check_cv_status` — Check continuous validation status on a GKE cluster.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-gcp-binary-authorization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-gcp-organization-policy-constraints (Skill Module)

implementing-gcp-organization-policy-constraints\SKILL.md

name: implementing-gcp-organization-policy-constraints
description: Agent for auditing and managing GCP Organization Policy constraints. Use when the user asks to implement gcp organization policy constraints.
version: 1.0.0
tags:
source: implementing-gcp-organization-policy-constraints

Implementing GCP Organization Policy Constraints

Role: Security control implementation
Audience: security architects and engineers
Primary tools: GCP, Baseline Security Constraints, Terraform Resource

When to use

- The user wants to **implement** gcp organization policy constraints.
- Capability: Agent for auditing and managing GCP Organization Policy constraints.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, organization, policy, constraints`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run a gcloud command and return parsed JSON
2. List all active organization policies
3. Get details of a specific org policy constraint
4. Audit organization against security baseline constraints
5. Audit organization policies effective on a specific project
6. Check if resource location constraint is configured
7. Generate Terraform HCL for baseline org policies
8. Generate a compliance summary from audit findings

Key concepts

- AC-3: Access Enforcement
- AC-6: Least Privilege
- CM-7: Least Functionality
- SC-7: Boundary Protection
- SC-12: Cryptographic Key Establishment
- Principle of least privilege for organization policies
- Hierarchical policy inheritance model
- Dry-run testing before enforcement

Tools & resources

- GCP
- Baseline Security Constraints
- Terraform Resource

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_gcloud` — Run a gcloud command and return parsed JSON.
- `list_org_policies` — List all active organization policies.
- `describe_policy` — Get details of a specific org policy constraint.
- `audit_baseline_compliance` — Audit organization against security baseline constraints.
- `audit_project_policies` — Audit organization policies effective on a specific project.
- `check_resource_location_constraint` — Check if resource location constraint is configured.
- `generate_terraform_policies` — Generate Terraform HCL for baseline org policies.
- `generate_compliance_report` — Generate a compliance summary from audit findings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-gcp-organization-policy-constraints`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-gcp-vpc-firewall-rules (Skill Module)

implementing-gcp-vpc-firewall-rules\SKILL.md

name: implementing-gcp-vpc-firewall-rules
description: GCP VPC firewall rules audit and management. Use when the user asks to implement gcp vpc firewall rules.
version: 1.0.0
tags:
source: implementing-gcp-vpc-firewall-rules

Implementing GCP VPC Firewall Rules

Role: Security control implementation
Audience: security architects and engineers
Primary tools: GCP, gcloud CLI Equivalents, Hierarchical Firewall Policies, External References

When to use

- The user wants to **implement** gcp vpc firewall rules.
- Capability: GCP VPC firewall rules audit and management agent.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, firewall, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create GCP Compute Engine firewall client
2. List all VPC firewall rules in a project
3. Identify firewall rules that are overly permissive (0.0.0.0/0)
4. Audit default network firewall rules for security issues
5. Create a new VPC firewall rule
6. Delete a firewall rule by name
7. Check which firewall rules have logging enabled
8. Run a comprehensive firewall audit

Tools & resources

- GCP
- gcloud CLI Equivalents
- Hierarchical Firewall Policies
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_firewall_client` — Create GCP Compute Engine firewall client.
- `list_firewall_rules` — List all VPC firewall rules in a project.
- `find_overly_permissive_rules` — Identify firewall rules that are overly permissive (0.0.0.0/0).
- `audit_default_rules` — Audit default network firewall rules for security issues.
- `create_firewall_rule` — Create a new VPC firewall rule.
- `delete_firewall_rule` — Delete a firewall rule by name.
- `check_logging_status` — Check which firewall rules have logging enabled.
- `run_firewall_audit` — Run a comprehensive firewall audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-gcp-vpc-firewall-rules`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-gdpr-data-protection-controls (Skill Module)

`implementing-gdpr-data-protection-controls\SKILL.md`

name: implementing-gdpr-data-protection-controls
description: Agent for assessing and managing GDPR data protection compliance. Use when the user asks to implement gdpr data protection controls.
version: 1.0.0
tags:
source: implementing-gdpr-data-protection-controls

Implementing Gdpr Data Protection Controls

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Key GDPR Articles, Cross, Primary Legislation, Key Articles Reference, Supporting Standards and Guidance, Supervisory Authorities

When to use

- The user wants to **implement** gdpr data protection controls.
- Capability: Agent for assessing and managing GDPR data protection compliance.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, gdpr, data, protection, controls`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan a file for GDPR-relevant personal data patterns
2. Classify data into GDPR special categories
3. Assess Records of Processing Activities (ROPA) completeness per Art 30
4. Assess Data Subject Request handling compliance
5. Assess breach notification compliance (Art 33/34)
6. Generate Article 32 security measures compliance checklist

Key concepts

- Adopted: April 14, 2016
- Effective: May 25, 2018
- Scope: Applies to any organization processing personal data of EU/EEA residents
- Chapters: 11 chapters, 99 articles, 173 recitals
- Enforcement: Supervisory authorities in each EU member state
- Penalties: Up to EUR 20 million or 4% of annual global turnover (whichever is greater)
- Art. 5: Core processing principles (lawfulness, fairness, transparency, purpose limitation, data minimization, accurac
- Art. 6: Six lawful bases for processing

Tools & resources

- Key GDPR Articles
- Cross
- Primary Legislation
- Key Articles Reference
- Supporting Standards and Guidance
- Supervisory Authorities

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_for_pii` — Scan a file for GDPR-relevant personal data patterns.
- `classify_data_category` — Classify data into GDPR special categories.
- `assess_ropa_completeness` — Assess Records of Processing Activities (ROPA) completeness per Art 30.
- `assess_dsr_handling` — Assess Data Subject Request handling compliance.
- `assess_breach_notification_readiness` — Assess breach notification compliance (Art 33/34).
- `generate_art32_checklist` — Generate Article 32 security measures compliance checklist.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-gdpr-data-protection-controls`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-gdpr-data-subject-access-request (Skill Module)

implementing-gdpr-data-subject-access-request\SKILL.md

name: implementing-gdpr-data-subject-access-request
description: GDPR Data Subject Access Request (DSAR) Workflow Automation Agent. Implements end-to-end DSAR processing: intake, identity verification, PII discovery using regex and NER, data mapping to Article 15 categories,.... Use when the user asks to implement gdpr data subject access request.
version: 1.0.0
tags:
source: implementing-gdpr-data-subject-access-request

Implementing Gdpr Data Subject Access Request

Role: Security control implementation
Audience: security architects and engineers
Primary tools: PIIPatternMatcher, PIIDiscoveryEngine, DataMapper, ExemptionReviewer, DSARResponseGenerator, DSARWorkflowEngine

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** gdpr data subject access request.
- Capability: GDPR Data Subject Access Request (DSAR) Workflow Automation Agent. Implements end-to-end DSAR processing: intake, identity verification, PII discovery using regex and NER, data mapping to Article 15 categories, exemption review, response generation, deadline tracking, and audit logging. References: - GDPR Article 15: https://gdpr-info.eu/art-15-gdpr/ - ICO DSAR Guidance: https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/subject-access-requests/ - EDPB Guidelines 01/2022 on Right of Access
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, gdpr, data, subject, access, request`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for gdpr data subject access request
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- PIIPatternMatcher
- PIIDiscoveryEngine
- DataMapper
- ExemptionReviewer
- DSARResponseGenerator
- DSARWorkflowEngine
- DSARAuditLogger

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-gdpr-data-subject-access-request`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-github-advanced-security-for-code-scanning (Skill Module)

`implementing-github-advanced-security-for-code-scanning\SKILL.md`

name: implementing-github-advanced-security-for-code-scanning
description: Agent for managing GitHub Advanced Security code scanning with CodeQL. Use when the user asks to implement github advanced security for code scanning.
version: 1.0.0
tags:
source: implementing-github-advanced-security-for-code-scanning

Implementing Github Advanced Security for Code Scanning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: GitHub Code Scanning API, Secret Scanning API, Supported Languages, NIST SP 800

When to use

- The user wants to **implement** github advanced security for code scanning.
- Capability: Agent for managing GitHub Advanced Security code scanning with CodeQL.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, github, advanced, security, code, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Call GitHub API via gh CLI
2. Get code scanning alerts for a repository
3. Get secret scanning alerts for a repository
4. Get Dependabot alerts for a repository
5. Analyze code scanning alerts and produce summary
6. Analyze secret scanning alerts
7. Analyze Dependabot vulnerability alerts
8. Generate a CodeQL analysis GitHub Actions workflow

Key concepts

- PO.3: Define security requirements --- CodeQL enforces security policies through query suites
- PW.4: Reuse existing, well-secured software --- Dependabot ensures dependencies are patched
- PW.7: Review and test code for vulnerabilities --- Automated code scanning on every PR
- PW.8: Test executable code --- SARIF integration enables combining SAST with DAST results
- RV.1: Identify and confirm vulnerabilities --- Security overview tracks alerts across the organization
- SCS-1: Source code management security --- Branch protection rules, required reviewers
- SCS-2: Build pipelines --- CodeQL runs in GitHub Actions with pinned action versions
- SCS-5: Artifact management --- Dependency review prevents vulnerable packages from merging

Tools & resources

- GitHub Code Scanning API
- Secret Scanning API
- Supported Languages
- NIST SP 800

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `gh_api` — Call GitHub API via gh CLI.
- `get_code_scanning_alerts` — Get code scanning alerts for a repository.
- `get_secret_scanning_alerts` — Get secret scanning alerts for a repository.
- `get_dependabot_alerts` — Get Dependabot alerts for a repository.
- `analyze_code_scanning_alerts` — Analyze code scanning alerts and produce summary.
- `analyze_secret_alerts` — Analyze secret scanning alerts.
- `analyze_dependabot_alerts` — Analyze Dependabot vulnerability alerts.
- `generate_codeql_workflow` — Generate a CodeQL analysis GitHub Actions workflow.
- `full_security_audit` — Run full GHAS security audit for a repository.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-github-advanced-security-for-code-scanning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-google-workspace-admin-security (Skill Module)

`implementing-google-workspace-admin-security\SKILL.md`

name: implementing-google-workspace-admin-security
description: Google Workspace admin security hardening agent using Admin SDK. Use when the user asks to implement google workspace admin security.
version: 1.0.0
tags:
source: implementing-google-workspace-admin-security

Implementing Google Workspace Admin Security

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Admin SDK Directory API, OAuth Scopes Required, External References

When to use

- The user wants to **implement** google workspace admin security.
- Capability: Google Workspace admin security hardening agent using Admin SDK.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, google, workspace, admin, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Build Google Admin SDK service with domain-wide delegation
2. Build Reports API service
3. List users who have not enrolled in 2-Step Verification
4. List all admin users and their admin roles
5. Get login audit events to detect suspicious activity
6. List suspended users that may still have active sessions
7. Audit users with recovery email/phone that could be used for account takeover
8. Run comprehensive Google Workspace security audit

Tools & resources

- Admin SDK Directory API
- OAuth Scopes Required
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_admin_service` — Build Google Admin SDK service with domain-wide delegation.
- `get_reports_service` — Build Reports API service.
- `list_users_without_2fa` — List users who have not enrolled in 2-Step Verification.
- `list_admin_users` — List all admin users and their admin roles.
- `get_login_audit_events` — Get login audit events to detect suspicious activity.
- `check_suspended_users` — List suspended users that may still have active sessions.
- `check_recovery_settings` — Audit users with recovery email/phone that could be used for account takeover.
- `run_workspace_audit` — Run comprehensive Google Workspace security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-google-workspace-admin-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-google-workspace-phishing-protection (Skill Module)

implementing-google-workspace-phishing-protection\SKILL.md

name: implementing-google-workspace-phishing-protection
description: Agent for auditing Google Workspace phishing and malware protection settings. Use when the user asks to implement google workspace phishing protection.
version: 1.0.0
tags:
source: implementing-google-workspace-phishing-protection

Implementing Google Workspace Phishing Protection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Admin Console Path, Gmail Safety Controls, Recommended Actions for Spoofing, Key Protection Settings

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** google workspace phishing protection.
- Capability: Agent for auditing Google Workspace phishing and malware protection settings.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, google, workspace, phishing, protection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run a GAM (Google Apps Manager) command
2. Retrieve Gmail safety settings via Admin SDK (simulated via GAM)
3. Audit Google Workspace phishing protection configuration
4. Analyze DMARC aggregate report for alignment issues
5. Analyze phishing incident log for patterns
6. Generate recommended Google Workspace phishing protection settings

Key concepts

- Admin Console > Apps > Google Workspace > Gmail > Safety
- Admin Console > Security > Gmail Enhanced Safe Browsing
- Admin Console > Apps > Google Workspace > Gmail > Authenticate email
- T1566.001: Phishing: Spearphishing Attachment
- T1566.002: Phishing: Spearphishing Link
- T1656: Impersonation
- T1586.002: Compromise Accounts: Email Accounts

Tools & resources

- Admin Console Path
- Gmail Safety Controls
- Recommended Actions for Spoofing
- Key Protection Settings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `gam_command` — Run a GAM (Google Apps Manager) command.
- `get_gmail_safety_settings` — Retrieve Gmail safety settings via Admin SDK (simulated via GAM).
- `audit_phishing_protection` — Audit Google Workspace phishing protection configuration.
- `audit_dmarc_alignment` — Analyze DMARC aggregate report for alignment issues.
- `analyze_phishing_incidents` — Analyze phishing incident log for patterns.
- `generate_recommended_settings` — Generate recommended Google Workspace phishing protection settings.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-google-workspace-phishing-protection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-google-workspace-sso-configuration (Skill Module)

`implementing-google-workspace-sso-configuration\SKILL.md`

name: implementing-google-workspace-sso-configuration
description: Agent for auditing and configuring Google Workspace SAML SSO. Use when the user asks to implement google workspace sso configuration.
version: 1.0.0
tags:
source: implementing-google-workspace-sso-configuration

Implementing Google Workspace SSO Configuration

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Admin Console Path, SAML Configuration Fields, Common IdP Providers, SAML 2.0 Standard

When to use

- The user wants to **implement** google workspace sso configuration.
- Capability: Agent for auditing and configuring Google Workspace SAML SSO.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, google, workspace, configuration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse and validate an IdP SAML signing certificate
2. Audit Google Workspace SSO configuration
3. Generate Google Workspace SAML SSO configuration
4. Audit SSO login activity for anomalies

Key concepts

- Assertions: Authentication statements, attribute statements
- Protocols: AuthnRequest, Response, LogoutRequest
- Profiles: Web Browser SSO, Single Logout
- SAML 2.0 compliant IdP
- HTTP POST binding for Assertion Consumer Service
- Signed SAML assertions (RSA-SHA256 recommended)
- NameID format: emailAddress (user's primary email)
- X.509 PEM certificate for signature verification

Tools & resources

- Admin Console Path
- SAML Configuration Fields
- Common IdP Providers
- SAML 2.0 Standard

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_saml_certificate` — Parse and validate an IdP SAML signing certificate.
- `audit_sso_config` — Audit Google Workspace SSO configuration.
- `generate_saml_config` — Generate Google Workspace SAML SSO configuration.
- `audit_sso_login_activity` — Audit SSO login activity for anomalies.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-google-workspace-sso-configuration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-hardware-security-key-authentication (Skill Module)

implementing-hardware-security-key-authentication\SKILL.md

name: implementing-hardware-security-key-authentication
description: FIDO2/WebAuthn Hardware Security Key Authentication Server. Implements a complete WebAuthn relying party with registration ceremonies, authentication flows, YubiKey enrollment management, and passkey support using.... Use when the user asks to implement hardware security key authentication.
version: 1.0.0
tags:
source: implementing-hardware-security-key-authentication

Implementing Hardware Security Key Authentication

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Arguments, Key Functions, Security Features

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** hardware security key authentication.
- Capability: FIDO2/WebAuthn Hardware Security Key Authentication Server. Implements a complete WebAuthn relying party with registration ceremonies, authentication flows, YubiKey enrollment management, and passkey support using the python-fido2 library. For authorized deployment and security testing only.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, hardware, security, authentication`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Initialize SQLite database for credential and user storage
2. Create a new user with a random user handle
3. Retrieve user by username
4. Retrieve user by user handle (for discoverable credential flows)
5. Store a new WebAuthn credential in the database
6. Get all active credentials for a user
7. Get all active credentials across all users (for discoverable flows)
8. Revoke a credential (soft delete)

Tools & resources

- Arguments
- Key Functions
- Security Features

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `init_database` — Initialize SQLite database for credential and user storage.
- `create_user` — Create a new user with a random user handle.
- `get_user_by_username` — Retrieve user by username.
- `get_user_by_handle` — Retrieve user by user handle (for discoverable credential flows).
- `store_credential` — Store a new WebAuthn credential in the database.
- `get_user_credentials` — Get all active credentials for a user.
- `get_all_credentials` — Get all active credentials across all users (for discoverable flows).
- `revoke_credential` — Revoke a credential (soft delete).
- `update_sign_count` — Update the sign count and last-used timestamp after authentication.
- `generate_recovery_codes` — Generate one-time recovery codes for account recovery.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-hardware-security-key-authentication`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-hashicorp-vault-dynamic-secrets (Skill Module)

implementing-hashicorp-vault-dynamic-secrets\SKILL.md

name: implementing-hashicorp-vault-dynamic-secrets
description: HashiCorp Vault dynamic secrets management agent using hvac client. Use when the user asks to implement hashicorp vault dynamic secrets.
version: 1.0.0
tags:
source: implementing-hashicorp-vault-dynamic-secrets

Implementing Hashicorp Vault Dynamic Secrets

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS Secrets Engine, PKI Secrets Engine, Lease Management, System Operations, External References

When to use

- The user wants to **implement** hashicorp vault dynamic secrets.
- Capability: HashiCorp Vault dynamic secrets management agent using hvac client.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, hashicorp, vault, dynamic, secrets`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to Vault using token or AppRole authentication
2. Enable the database secrets engine
3. Configure a PostgreSQL database connection in Vault
4. Create a dynamic database role for credential generation
5. Generate dynamic database credentials for a role
6. Enable the AWS secrets engine for dynamic IAM credentials
7. Configure AWS root credentials for dynamic IAM generation
8. Create an AWS dynamic role for generating IAM credentials

Tools & resources

- AWS Secrets Engine
- PKI Secrets Engine
- Lease Management
- System Operations
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_vault` — Connect to Vault using token or AppRole authentication.
- `enable_database_secrets_engine` — Enable the database secrets engine.
- `configure_postgres_connection` — Configure a PostgreSQL database connection in Vault.
- `create_database_role` — Create a dynamic database role for credential generation.
- `generate_database_credentials` — Generate dynamic database credentials for a role.
- `enable_aws_secrets_engine` — Enable the AWS secrets engine for dynamic IAM credentials.
- `configure_aws_root` — Configure AWS root credentials for dynamic IAM generation.
- `create_aws_role` — Create an AWS dynamic role for generating IAM credentials.
- `generate_aws_credentials` — Generate dynamic AWS credentials for a role.
- `enable_pki_engine` — Enable PKI secrets engine for dynamic certificate generation.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-hashicorp-vault-dynamic-secrets`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-honeypot-for-ransomware-detection (Skill Module)

implementing-honeypot-for-ransomware-detection\SKILL.md

name: implementing-honeypot-for-ransomware-detection
description: Agent for deploying and monitoring ransomware honeypot canary files. Use when the user asks to implement honeypot for ransomware detection.
version: 1.0.0
tags:
source: implementing-honeypot-for-ransomware-detection

Implementing Honeypot for Ransomware Detection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Thinkst Canary, Canarytokens, OpenCanary, FSRM, Elastic Ransomware Protection

When to use

- The user wants to **implement** honeypot for ransomware detection.
- Capability: Agent for deploying and monitoring ransomware honeypot canary files.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, honeypot, ransomware, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create canary files in strategic locations for ransomware detection
2. Save canary file manifest for integrity monitoring
3. Check canary files against manifest to detect tampering/encryption
4. Detect rapid file modifications indicative of ransomware
5. Generate SMB honeypot share configuration
6. Analyze honeypot access logs for suspicious activity

Key concepts

- T1486: Data Encrypted for Impact (detection via canary file modification)
- T1021.002: SMB/Windows Admin Shares (detection via honeypot share access)
- T1083: File and Directory Discovery (detection via honeypot folder browsing)
- SI-4: System Monitoring (honeypots as monitoring component)
- AU-6: Audit Record Review, Analysis, and Reporting (canary file audit logging)
- Control 13.1: Centralize security event alerting (integrate honeypot alerts into SIEM)
- Control 13.8: Deploy intrusion detection/prevention systems

Tools & resources

- Thinkst Canary
- Canarytokens
- OpenCanary
- FSRM
- Elastic Ransomware Protection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `create_canary_files` — Create canary files in strategic locations for ransomware detection.
- `generate_canary_manifest` — Save canary file manifest for integrity monitoring.
- `check_canary_integrity` — Check canary files against manifest to detect tampering/encryption.
- `detect_ransomware_indicators` — Detect rapid file modifications indicative of ransomware.
- `generate_honeypot_share_config` — Generate SMB honeypot share configuration.
- `analyze_honeypot_logs` — Analyze honeypot access logs for suspicious activity.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-honeypot-for-ransomware-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-honeytokens-for-breach-detection (Skill Module)

`implementing-honeytokens-for-breach-detection\SKILL.md`

name: implementing-honeytokens-for-breach-detection
description: Agent for deploying and managing honeytokens for breach detection. Use when the user asks to implement honeytokens for breach detection.
version: 1.0.0
tags:
source: implementing-honeytokens-for-breach-detection

Implementing Honeytokens for Breach Detection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Canarytokens.org API, Token Deployment Locations, Webhook Alert Payload

When to use

- The user wants to **implement** honeytokens for breach detection.
- Capability: Agent for deploying and managing honeytokens for breach detection.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, honeytokens, breach, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create a DNS canary token via Canarytokens.org API
2. Create a web bug (image beacon) canary token
3. Create an AWS credential canary token
4. Generate a unique honeytoken identifier
5. Deploy fake AWS credentials file as a honeytoken
6. Generate SQL to insert honeytoken records into a database
7. Embed a DNS canary token in a configuration file
8. Create a honeytoken deployment plan for an environment

Tools & resources

- Canarytokens.org API
- Token Deployment Locations
- Webhook Alert Payload

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `create_dns_canarytoken` — Create a DNS canary token via Canarytokens.org API.
- `create_web_bug_token` — Create a web bug (image beacon) canary token.
- `create_aws_key_token` — Create an AWS credential canary token.
- `generate_honeytoken_id` — Generate a unique honeytoken identifier.
- `deploy_aws_credential_token` — Deploy fake AWS credentials file as a honeytoken.
- `deploy_database_honeytoken` — Generate SQL to insert honeytoken records into a database.
- `deploy_dns_token_in_config` — Embed a DNS canary token in a configuration file.
- `create_deployment_plan` — Create a honeytoken deployment plan for an environment.
- `check_token_alerts` — Parse webhook logs to check for honeytoken trigger alerts.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-honeytokens-for-breach-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ics-firewall-with-tofino (Skill Module)

implementing-ics-firewall-with-tofino\SKILL.md

name: implementing-ics-firewall-with-tofino
description: Agent for auditing and configuring Tofino ICS firewall rules. Use when the user asks to implement ics firewall with tofino.
version: 1.0.0
tags:
source: implementing-ics-firewall-with-tofino

Implementing Ics Firewall with Tofino

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OT Protocol Ports, Risky Modbus Function Codes, Tofino Xenon Configuration, Rule Audit Checks

When to use

- The user wants to **implement** ics firewall with tofino.
- Capability: Agent for auditing and configuring Tofino ICS firewall rules.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, firewall, tofino`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Audit Tofino firewall rules for security issues
2. Analyze OT network traffic log for anomalies
3. Generate Tofino firewall rules from zone configuration

Tools & resources

- OT Protocol Ports
- Risky Modbus Function Codes
- Tofino Xenon Configuration
- Rule Audit Checks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `audit_firewall_rules` — Audit Tofino firewall rules for security issues.
- `analyze_ot_traffic_log` — Analyze OT network traffic log for anomalies.
- `generate_zone_rules` — Generate Tofino firewall rules from zone configuration.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ics-firewall-with-tofino`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-identity-governance-with-sailpoint (Skill Module)

`implementing-identity-governance-with-sailpoint\SKILL.md`

name: implementing-identity-governance-with-sailpoint
description: Agent for auditing SailPoint IdentityNow/IIQ identity governance. Use when the user asks to implement identity governance with sailpoint.
version: 1.0.0
tags:
source: implementing-identity-governance-with-sailpoint

Implementing Identity Governance with Sailpoint

Role: Security control implementation
Audience: security architects and engineers
Primary tools: SailPoint, Identity Lifecycle Events

When to use

- The user wants to **implement** identity governance with sailpoint.
- Capability: Agent for auditing SailPoint IdentityNow/IIQ identity governance.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, identity, governance, sailpoint`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Call SailPoint IdentityNow API
2. List identities from SailPoint
3. List access profiles
4. List active certification campaigns
5. Audit certification campaign completion and compliance
6. Audit Separation of Duties policy violations
7. Find orphan accounts (no identity correlation)
8. Audit access review data for over-provisioned users

Tools & resources

- SailPoint
- Identity Lifecycle Events

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `sailpoint_api` — Call SailPoint IdentityNow API.
- `list_identities` — List identities from SailPoint.
- `list_access_profiles` — List access profiles.
- `list_certifications` — List active certification campaigns.
- `audit_certification_campaigns` — Audit certification campaign completion and compliance.
- `audit_sod_violations` — Audit Separation of Duties policy violations.
- `audit_orphan_accounts` — Find orphan accounts (no identity correlation).
- `audit_access_reviews` — Audit access review data for over-provisioned users.
- `generate_lifecycle_policy` — Generate identity lifecycle policy recommendations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-identity-governance-with-sailpoint`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-identity-verification-for-zero-trust (Skill Module)

implementing-identity-verification-for-zero-trust\SKILL.md

name: implementing-identity-verification-for-zero-trust
description: Agent for assessing identity verification controls in zero trust architecture. Use when the user asks to implement identity verification for zero trust.
version: 1.0.0
tags:
source: implementing-identity-verification-for-zero-trust

Implementing Identity Verification for Zero Trust

Role: Security control implementation
Audience: security architects and engineers
Primary tools: FIDO2/WebAuthn Registration, Conditional Access Signals, NIST SP 800-207, NIST SP 800-63B, FIDO Alliance Standards, Okta Identity Security

When to use

- The user wants to **implement** identity verification for zero trust.
- Capability: Agent for assessing identity verification controls in zero trust architecture.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, identity, verification, zero, trust`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Assess authentication method strength against zero trust requirements
2. Assess conditional access policies for zero trust alignment
3. Assess identity pillar maturity against CISA Zero Trust Maturity Model
4. Analyze authentication events for zero trust insights

Key concepts

- All resource authentication and authorization are dynamic and strictly enforced before access is allowed
- User identity and attributes from the IdP
- Device identity and posture from the endpoint management system
- Behavioral attributes from user and entity behavior analytics (UEBA)
- Environmental attributes (location, time, network)
- FIDO2/WebAuthn: Cryptographic authentication bound to origin domain
- PIV/CAC smart cards: Certificate-based authentication
- Not phishing-resistant: SMS OTP, voice calls, push notifications, TOTP

Tools & resources

- FIDO2/WebAuthn Registration
- Conditional Access Signals
- NIST SP 800-207
- NIST SP 800-63B
- FIDO Alliance Standards
- Okta Identity Security

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `assess_authentication_methods` — Assess authentication method strength against zero trust requirements.
- `assess_conditional_access` — Assess conditional access policies for zero trust alignment.
- `assess_identity_maturity` — Assess identity pillar maturity against CISA Zero Trust Maturity Model.
- `analyze_auth_events` — Analyze authentication events for zero trust insights.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-identity-verification-for-zero-trust`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-iec-62443-security-zones (Skill Module)

implementing-iec-62443-security-zones\SKILL.md

name: implementing-iec-62443-security-zones
description: Agent for designing and auditing IEC 62443 security zones and conduits. Use when the user asks to implement iec 62443 security zones.
version: 1.0.0
tags:
source: implementing-iec-62443-security-zones

Implementing Iec 62443 Security Zones

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Purdue Reference Model Levels, IEC 62443 Security Levels, Zone Audit Checks, Conduit Security Controls

When to use

- The user wants to **implement** iec 62443 security zones.
- Capability: Agent for designing and auditing IEC 62443 security zones and conduits.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, 62443, security, zones`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Audit IEC 62443 zone architecture for compliance
2. Generate IEC 62443 zone template based on Purdue model
3. Map risk assessment results to IEC 62443 Security Level targets

Tools & resources

- Purdue Reference Model Levels
- IEC 62443 Security Levels
- Zone Audit Checks
- Conduit Security Controls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `audit_zone_architecture` — Audit IEC 62443 zone architecture for compliance.
- `generate_zone_template` — Generate IEC 62443 zone template based on Purdue model.
- `assess_sl_requirements` — Map risk assessment results to IEC 62443 Security Level targets.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-iec-62443-security-zones`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-image-provenance-verification-with-cosign (Skill Module)

implementing-image-provenance-verification-with-cosign\SKILL.md

name: implementing-image-provenance-verification-with-cosign
description: Agent for verifying container image provenance using Sigstore Cosign. Use when the user asks to implement image provenance verification with cosign.
version: 1.0.0
tags:
source: implementing-image-provenance-verification-with-cosign

Implementing Image Provenance Verification with Cosign

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Sigstore Components, Attestation Types, NIST SP 800-190, Compliance Mappings

When to use

- The user wants to **implement** image provenance verification with cosign.
- Capability: Agent for verifying container image provenance using Sigstore Cosign.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, image, provenance, verification, cosign`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run a cosign CLI command and return output
2. Verify a container image signature with Cosign
3. Verify an in-toto attestation attached to an image
4. Sign a container image with Cosign
5. Get the signature and attestation image references
6. Audit multiple container images for valid signatures
7. Generate Kyverno ClusterPolicy for image verification
8. Generate GitHub Actions workflow for Cosign signing

Key concepts

- Level 1: Documentation of build process
- Level 2: Source version controlled + signed provenance
- Level 3: Hardened build platform, non-falsifiable provenance
- Level 4: Two-party review, hermetic reproducible builds
- Section 4.1: Image vulnerabilities - Verify image integrity before deployment
- Section 5.1: Image security - Cryptographic signing and verification
- Section 4(e): SBOM requirements for software supply chain
- Section 4(g): Software supply chain security guidelines

Tools & resources

- Sigstore Components
- Attestation Types
- NIST SP 800-190
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_cosign` — Run a cosign CLI command and return output.
- `verify_image` — Verify a container image signature with Cosign.
- `verify_attestation` — Verify an in-toto attestation attached to an image.
- `sign_image` — Sign a container image with Cosign.
- `triangulate_image` — Get the signature and attestation image references.
- `audit_registry_images` — Audit multiple container images for valid signatures.
- `generate_kyverno_policy` — Generate Kyverno ClusterPolicy for image verification.
- `generate_cosign_ci_workflow` — Generate GitHub Actions workflow for Cosign signing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-image-provenance-verification-with-cosign`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-immutable-backup-with-restic (Skill Module)

implementing-immutable-backup-with-restic\SKILL.md

name: implementing-immutable-backup-with-restic
description: Immutable backup agent using restic with S3 Object Lock for ransomware-resistant storage. Automates backup creation, integrity verification, snapshot retention, and restore testing against S3-compatible storage with.... Use when the user asks to implement immutable backup with restic.
version: 1.0.0
tags:
source: implementing-immutable-backup-with-restic

Implementing Immutable Backup with Restic

Role: Security control implementation
Audience: security architects and engineers
Primary tools: S3 Object Lock Configuration, Retention Policy Defaults

When to use

- The user wants to **implement** immutable backup with restic.
- Capability: Immutable backup agent using restic with S3 Object Lock for ransomware-resistant storage. Automates backup creation, integrity verification, snapshot retention, and restore testing against S3-compatible storage with WORM protection.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, immutable, backup, restic`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute a restic command and return parsed output
2. Initialize a new restic repository with encryption
3. Configure S3 Object Lock on backup bucket via AWS CLI
4. Create a new backup snapshot
5. Verify repository integrity by checking all data packs
6. List all snapshots in the repository
7. Apply snapshot retention policy using restic forget
8. Test restore by extracting files and verifying checksums

Tools & resources

- S3 Object Lock Configuration
- Retention Policy Defaults

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_restic` — Execute a restic command and return parsed output.
- `init_repository` — Initialize a new restic repository with encryption.
- `configure_s3_object_lock` — Configure S3 Object Lock on backup bucket via AWS CLI.
- `create_backup` — Create a new backup snapshot.
- `verify_backup_integrity` — Verify repository integrity by checking all data packs.
- `list_snapshots` — List all snapshots in the repository.
- `apply_retention_policy` — Apply snapshot retention policy using restic forget.
- `test_restore` — Test restore by extracting files and verifying checksums.
- `get_repository_stats` — Get repository statistics including size and deduplication ratio.
- `format_bytes` — Format byte count to human-readable string.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-immutable-backup-with-restic`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-infrastructure-as-code-security-scanning (Skill Module)

`implementing-infrastructure-as-code-security-scanning\SKILL.md`

name: implementing-infrastructure-as-code-security-scanning
description: Agent for scanning Infrastructure as Code templates for security misconfigurations. Use when the user asks to implement infrastructure as code security scanning.
version: 1.0.0
tags:
source: implementing-infrastructure-as-code-security-scanning

Implementing Infrastructure As Code Security Scanning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Checkov CLI, tfsec CLI, Common IaC Security Checks, CIS Cloud Benchmarks, OWASP SAMM

When to use

- The user wants to **implement** infrastructure as code security scanning.
- Capability: Agent for scanning Infrastructure as Code templates for security misconfigurations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, infrastructure, code, security, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run Checkov IaC security scanner
2. Run tfsec Terraform security scanner
3. Analyze Checkov scan results and categorize findings
4. Scan Terraform files for common security misconfigurations
5. Generate CI/CD pipeline config for IaC scanning

Key concepts

- Maps directly to Checkov CKV_AWS_* checks
- Covers IAM, logging, monitoring, networking, and storage security
- Automated scanning validates 100+ benchmark controls
- Maps to Checkov CKV_AZURE_* checks
- Covers identity, security center, storage, database, and network controls
- Maps to Checkov CKV_GCP_* checks
- Covers IAM, logging, networking, VM, storage, and database controls
- Validate infrastructure configurations against security standards before deployment

Tools & resources

- Checkov CLI
- tfsec CLI
- Common IaC Security Checks
- CIS Cloud Benchmarks
- OWASP SAMM

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_checkov` — Run Checkov IaC security scanner.
- `run_tfsec` — Run tfsec Terraform security scanner.
- `analyze_checkov_results` — Analyze Checkov scan results and categorize findings.
- `scan_terraform_files` — Scan Terraform files for common security misconfigurations.
- `generate_ci_config` — Generate CI/CD pipeline config for IaC scanning.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-infrastructure-as-code-security-scanning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-iso-27001-information-security-management (Skill Module)

implementing-iso-27001-information-security-management\SKILL.md

name: implementing-iso-27001-information-security-management
description: Agent for assessing ISO 27001:2022 ISMS compliance. Use when the user asks to implement iso 27001 information security management.
version: 1.0.0
tags:
source: implementing-iso-27001-information-security-management

Implementing Iso 27001 Information Security Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: ISO 27001, Risk Assessment Formula, Primary Standards, Certification Process, Accreditation Bodies

When to use

- The user wants to **implement** iso 27001 information security management.
- Capability: Agent for assessing ISO 27001:2022 ISMS compliance.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, 27001, information, security, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Assess Statement of Applicability completeness
2. Check required ISMS documentation against inventory
3. Assess risk register for completeness and currency
4. Generate ISO 27001 gap analysis from current state assessment

Key concepts

- Title: Information Security, Cybersecurity and Privacy Protection - Information Security Management Systems - Require
- Published: October 2022
- Scope: Specifies requirements for establishing, implementing, maintaining, and continually improving an ISMS
- Certification: Organizations can be certified against this standard by accredited certification bodies
- Annex A restructured from 14 domains to 4 themes (Organizational, People, Physical, Technological)
- Title expanded to include "Cybersecurity and Privacy Protection"
- Clause 6.3 added for planning changes to the ISMS
- Title: Information Security, Cybersecurity and Privacy Protection - Information Security Controls

Tools & resources

- ISO 27001
- Risk Assessment Formula
- Primary Standards
- Certification Process
- Accreditation Bodies

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `assess_soa_completeness` — Assess Statement of Applicability completeness.
- `assess_documentation` — Check required ISMS documentation against inventory.
- `assess_risk_register` — Assess risk register for completeness and currency.
- `generate_gap_analysis` — Generate ISO 27001 gap analysis from current state assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-iso-27001-information-security-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-just-in-time-access-provisioning (Skill Module)

`implementing-just-in-time-access-provisioning\SKILL.md`

name: implementing-just-in-time-access-provisioning
description: Agent for implementing and auditing Just-In-Time access provisioning. Use when the user asks to implement just in time access provisioning.
version: 1.0.0
tags:
source: implementing-just-in-time-access-provisioning

Implementing Just in Time Access Provisioning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Microsoft Entra PIM, CyberArk JIT, SailPoint, HashiCorp Boundary, StrongDM

When to use

- The user wants to **implement** just in time access provisioning.
- Capability: Agent for implementing and auditing Just-In-Time access provisioning.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, just, time, access, provisioning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Audit JIT access requests for compliance and anomalies
2. Identify standing privileges that should be converted to JIT
3. Calculate JIT program operational metrics
4. Generate JIT access provisioning policy

Key concepts

- NIST SP 800-207: Zero Trust Architecture - Section 3 (Logical Components)
- AC-2(2): Automated Temporary and Emergency Account Management
- AC-2(3): Disable Accounts
- AC-6: Least Privilege
- AC-6(5): Privileged Accounts
- NIST SP 1800-35: Implementing a Zero Trust Architecture
- CISA Zero Trust Maturity Model: Identity pillar - dynamic access provisioning
- DoD Zero Trust Reference Architecture: JIT/JEA requirements

Tools & resources

- Microsoft Entra PIM
- CyberArk JIT
- SailPoint
- HashiCorp Boundary
- StrongDM

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_jit_requests` — Audit JIT access requests for compliance and anomalies.
- `audit_standing_privileges` — Identify standing privileges that should be converted to JIT.
- `calculate_jit_metrics` — Calculate JIT program operational metrics.
- `generate_jit_policy` — Generate JIT access provisioning policy.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-just-in-time-access-provisioning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-jwt-signing-and-verification (Skill Module)

implementing-jwt-signing-and-verification\SKILL.md

name: implementing-jwt-signing-and-verification
description: Agent for JWT signing, verification, and security auditing. Use when the user asks to implement jwt signing and verification.
version: 1.0.0
tags:
source: implementing-jwt-signing-and-verification

Implementing JWT Signing and Verification

Role: Security control implementation
Audience: security architects and engineers
Primary tools: PyJWT Library, JWT Algorithms, Common JWT Attacks, Primary Standards, OWASP References

When to use

- The user wants to **implement** jwt signing and verification.
- Capability: Agent for JWT signing, verification, and security auditing.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, signing, verification`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Base64url encode without padding
2. Base64url decode with padding restoration
3. Create a JWT signed with HMAC-SHA256
4. Verify an HS256 JWT and return claims
5. Decode a JWT without verification (for inspection only)
6. Audit a JWT for common security vulnerabilities
7. Generate RSA key pair for RS256 JWT signing

Key concepts

- Description: JWT format and claims specification
- Description: Signature mechanism for JWTs
- Description: Key format for JWT signing keys
- Description: Cryptographic algorithms for JWS/JWE
- Description: Security best practices for JWT implementations

Tools & resources

- PyJWT Library
- JWT Algorithms
- Common JWT Attacks
- Primary Standards
- OWASP References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `b64url_encode` — Base64url encode without padding.
- `b64url_decode` — Base64url decode with padding restoration.
- `create_jwt_hs256` — Create a JWT signed with HMAC-SHA256.
- `verify_jwt_hs256` — Verify an HS256 JWT and return claims.
- `decode_jwt_unsafe` — Decode a JWT without verification (for inspection only).
- `audit_jwt_security` — Audit a JWT for common security vulnerabilities.
- `generate_rsa_keypair` — Generate RSA key pair for RS256 JWT signing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-jwt-signing-and-verification`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-kubernetes-network-policy-with-calico (Skill Module)

implementing-kubernetes-network-policy-with-calico\SKILL.md

name: implementing-kubernetes-network-policy-with-calico
description: Agent for auditing and generating Calico Kubernetes network policies. Use when the user asks to implement kubernetes network policy with calico.
version: 1.0.0
tags:
source: implementing-kubernetes-network-policy-with-calico

Implementing Kubernetes Network Policy with Calico

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Kubernetes, Calico GlobalNetworkPolicy, calicoctl CLI, Industry Standards, Calico Documentation References, Zero Trust Network Model

When to use

- The user wants to **implement** kubernetes network policy with calico.
- Capability: Agent for auditing and generating Calico Kubernetes network policies.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, kubernetes, network, policy, calico`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Get Kubernetes resources via kubectl
2. List all NetworkPolicy and Calico GlobalNetworkPolicy resources
3. Audit network policies for security gaps
4. Generate a default deny-all NetworkPolicy for a namespace
5. Generate a policy allowing DNS egress to kube-dns
6. Generate a Calico-style network policy for an application
7. Find namespaces without any network policies

Key concepts

- Section 4.4: Container Networking - Isolate container network traffic using network policies
- Section 5.3: Network Security - Implement micro-segmentation between containers
- Recommends default-deny policies with explicit allowlisting
- SC-7: Boundary Protection - Implement network segmentation controls
- AC-4: Information Flow Enforcement - Control network traffic between pods
- SC-7(5): Deny by Default / Allow by Exception
- Section 3: Network Separation and Hardening
- Use network policies to isolate workloads

Tools & resources

- Kubernetes
- Calico GlobalNetworkPolicy
- calicoctl CLI
- Industry Standards
- Calico Documentation References
- Zero Trust Network Model
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `kubectl_get` — Get Kubernetes resources via kubectl.
- `list_network_policies` — List all NetworkPolicy and Calico GlobalNetworkPolicy resources.
- `audit_network_policies` — Audit network policies for security gaps.
- `generate_default_deny` — Generate a default deny-all NetworkPolicy for a namespace.
- `generate_allow_dns_egress` — Generate a policy allowing DNS egress to kube-dns.
- `generate_app_policy` — Generate a Calico-style network policy for an application.
- `check_unprotected_namespaces` — Find namespaces without any network policies.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-kubernetes-network-policy-with-calico`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-kubernetes-pod-security-standards (Skill Module)

`implementing-kubernetes-pod-security-standards\SKILL.md`

name: implementing-kubernetes-pod-security-standards
description: Agent for auditing Kubernetes Pod Security Standards enforcement. Use when the user asks to implement kubernetes pod security standards.
version: 1.0.0
tags:
source: implementing-kubernetes-pod-security-standards

Implementing Kubernetes Pod Security Standards

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Kubernetes, Pod Security Standard Levels, PSA Modes, Baseline Violations

When to use

- The user wants to **implement** kubernetes pod security standards.
- Capability: Agent for auditing Kubernetes Pod Security Standards enforcement.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, kubernetes, security, standards`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run kubectl and return JSON
2. Audit PSA labels on all namespaces
3. Audit pods against Pod Security Standards
4. Generate PSA label patch for a namespace
5. Generate PSS compliance summary

Key concepts

- No restrictions applied
- Used for: kube-system, monitoring agents, CNI plugins, storage drivers
- Use PSA in enforce mode for production namespaces
- Set restricted profile as default for all non-system namespaces
- Require seccomp profiles on all pods
- Prevent privileged containers in all workload namespaces
- Require non-root user for all containers
- Drop all capabilities and only add NET_BIND_SERVICE if needed

Tools & resources

- Kubernetes
- Pod Security Standard Levels
- PSA Modes
- Baseline Violations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `kubectl_json` — Run kubectl and return JSON.
- `audit_namespace_labels` — Audit PSA labels on all namespaces.
- `audit_pod_security` — Audit pods against Pod Security Standards.
- `generate_namespace_labels` — Generate PSA label patch for a namespace.
- `generate_compliance_report` — Generate PSS compliance summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-kubernetes-pod-security-standards`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-llm-guardrails-for-security (Skill Module)

implementing-llm-guardrails-for-security\SKILL.md

name: implementing-llm-guardrails-for-security
description: LLM Guardrails Security Agent Implements input and output validation guardrails for LLM-powered applications. Provides multi-layered security including prompt injection blocking, PII detection and redaction, content.... Use when the user asks to implement llm guardrails for security.
version: 1.0.0
tags:
source: implementing-llm-guardrails-for-security

Implementing LLM Guardrails for Security

Role: Security control implementation
Audience: security architects and engineers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** llm guardrails for security.
- Capability: LLM Guardrails Security Agent Implements input and output validation guardrails for LLM-powered applications. Provides multi-layered security including prompt injection blocking, PII detection and redaction, content policy enforcement, topic restriction, and output validation. Supports NVIDIA NeMo Guardrails Colang integration and custom Python validators.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, guardrails, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for llm guardrails for security
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `format_result_text` — Format a validation result as human-readable text.
- `format_result_json` — Format a validation result as JSON.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-llm-guardrails-for-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-log-forwarding-with-fluentd (Skill Module)

implementing-log-forwarding-with-fluentd\SKILL.md

name: implementing-log-forwarding-with-fluentd
description: Fluentd/Fluent Bit log forwarding configuration generator and tester. Use when the user asks to implement log forwarding with fluentd.
version: 1.0.0
tags:
source: implementing-log-forwarding-with-fluentd

Implementing Log Forwarding with Fluentd

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Fluent Bit CLI, Fluentd CLI, Fluent Bit Configuration Sections, Fluentd Configuration Directives, Python fluent-logger

When to use

- The user wants to **implement** log forwarding with fluentd.
- Capability: Fluentd/Fluent Bit log forwarding configuration generator and tester.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, forwarding, fluentd`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate Fluent Bit configuration for log collection and forwarding
2. Generate Fluentd aggregator configuration
3. Validate configuration syntax for common issues
4. Send a test log event to Fluentd/Fluent Bit via forward protocol
5. Generate deployment report

Tools & resources

- Fluent Bit CLI
- Fluentd CLI
- Fluent Bit Configuration Sections
- Fluentd Configuration Directives
- Python fluent-logger

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_fluentbit_config` — Generate Fluent Bit configuration for log collection and forwarding.
- `generate_fluentd_config` — Generate Fluentd aggregator configuration.
- `validate_config` — Validate configuration syntax for common issues.
- `send_test_event` — Send a test log event to Fluentd/Fluent Bit via forward protocol.
- `generate_report` — Generate deployment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-log-forwarding-with-fluentd`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-log-integrity-with-blockchain (Skill Module)

implementing-log-integrity-with-blockchain\SKILL.md

name: implementing-log-integrity-with-blockchain
description: Log Integrity Chain Agent - Implements SHA-256 hash-chained append-only log for tamper detection. Use when the user asks to implement log integrity with blockchain.
version: 1.0.0
tags:
source: implementing-log-integrity-with-blockchain

Implementing Log Integrity with Blockchain

Role: Security control implementation
Audience: security architects and engineers
Primary tools: hashlib, Chain Entry Structure, Chain Construction Algorithm, Verification Algorithm, Checkpoint Structure, Tamper Detection Properties

When to use

- The user wants to **implement** log integrity with blockchain.
- Capability: Log Integrity Chain Agent - Implements SHA-256 hash-chained append-only log for tamper detection.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, integrity, blockchain`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a string
2. Create a single chain entry with hash linking
3. Load an existing hash chain from a JSON file
4. Save the hash chain to a JSON file
5. Read log entries from a file (one entry per line)
6. Read structured log entries from a JSON array file
7. Append new log entries to the hash chain
8. Verify the integrity of the entire hash chain

Tools & resources

- hashlib
- Chain Entry Structure
- Chain Construction Algorithm
- Verification Algorithm
- Checkpoint Structure
- Tamper Detection Properties

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hash` — Compute SHA-256 hash of a string.
- `create_chain_entry` — Create a single chain entry with hash linking.
- `load_chain` — Load an existing hash chain from a JSON file.
- `save_chain` — Save the hash chain to a JSON file.
- `ingest_log_file` — Read log entries from a file (one entry per line).
- `ingest_json_log` — Read structured log entries from a JSON array file.
- `append_entries` — Append new log entries to the hash chain.
- `verify_chain` — Verify the integrity of the entire hash chain.
- `find_tampered_range` — Identify the range of entries affected by tampering.
- `create_checkpoint` — Create an integrity checkpoint with the current chain head hash.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-log-integrity-with-blockchain`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-memory-protection-with-dep-aslr (Skill Module)

implementing-memory-protection-with-dep-aslr\SKILL.md

name: implementing-memory-protection-with-dep-aslr
description: DEP/ASLR Memory Protection Agent - audits processes and binaries for memory protection mitigations. Use when the user asks to implement memory protection with dep aslr.
version: 1.0.0
tags:
source: implementing-memory-protection-with-dep-aslr

Implementing Memory Protection with Dep Aslr

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Memory Protection Mechanisms, Linux ASLR Check, GCC Compilation Flags

When to use

- The user wants to **implement** memory protection with dep aslr.
- Capability: DEP/ASLR Memory Protection Agent - audits processes and binaries for memory protection mitigations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, memory, protection, aslr`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check Windows DEP/NX policy via bcdedit
2. Check process mitigation policies using PowerShell Get-ProcessMitigation
3. Check Linux ASLR configuration
4. Check NX bit support on Linux
5. Check ELF binary for PIE, RELRO, stack canary, and NX using readelf/checksec
6. Scan directory for binaries and check protections

Key concepts

- MITRE ATT&CK T1055: Process Injection - mitigated by DEP, CFG, ASLR
- CIS Benchmark Section 18.9: Exploit Protection configuration
- NIST 800-53 SI-16: Memory Protection

Tools & resources

- Memory Protection Mechanisms
- Linux ASLR Check
- GCC Compilation Flags

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_windows_dep_policy` — Check Windows DEP/NX policy via bcdedit.
- `check_windows_process_mitigations` — Check process mitigation policies using PowerShell Get-ProcessMitigation.
- `check_linux_aslr` — Check Linux ASLR configuration.
- `check_linux_nx_support` — Check NX bit support on Linux.
- `check_elf_pie_relro` — Check ELF binary for PIE, RELRO, stack canary, and NX using readelf/checksec.
- `scan_directory_binaries` — Scan directory for binaries and check protections.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-memory-protection-with-dep-aslr`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-microsegmentation-with-guardicore (Skill Module)

implementing-microsegmentation-with-guardicore\SKILL.md

name: implementing-microsegmentation-with-guardicore
description: Guardicore Microsegmentation Agent - audits segmentation policies and network flow visibility. Use when the user asks to implement microsegmentation with guardicore.
version: 1.0.0
tags:
source: implementing-microsegmentation-with-guardicore

Implementing Microsegmentation with Guardicore

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Akamai Guardicore API, Policy Modes, Segmentation Metrics, Traffic Analysis Fields

When to use

- The user wants to **implement** microsegmentation with guardicore.
- Capability: Guardicore Microsegmentation Agent - audits segmentation policies and network flow visibility.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, microsegmentation, guardicore`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate to Guardicore Centra and get access token
2. Retrieve all segmentation policies
3. Get recent network flow data for analysis
4. Get all asset labels/tags
5. Get deployed agent status
6. Analyze how well policies cover observed traffic
7. Identify potential lateral movement patterns in east-west traffic
8. Audit deployment agent health status

Tools & resources

- Akamai Guardicore API
- Policy Modes
- Segmentation Metrics
- Traffic Analysis Fields

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `gc_request`
- `authenticate` — Authenticate to Guardicore Centra and get access token.
- `get_segmentation_policies` — Retrieve all segmentation policies.
- `get_network_flows` — Get recent network flow data for analysis.
- `get_labels` — Get all asset labels/tags.
- `get_agents` — Get deployed agent status.
- `analyze_policy_coverage` — Analyze how well policies cover observed traffic.
- `detect_lateral_movement_risk` — Identify potential lateral movement patterns in east-west traffic.
- `audit_agent_health` — Audit deployment agent health status.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-microsegmentation-with-guardicore`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-mimecast-targeted-attack-protection (Skill Module)

implementing-mimecast-targeted-attack-protection\SKILL.md

name: implementing-mimecast-targeted-attack-protection
description: Mimecast Targeted Attack Protection Agent - monitors TAP events and URL/attachment threats. Use when the user asks to implement mimecast targeted attack protection.
version: 1.0.0
tags:
source: implementing-mimecast-targeted-attack-protection

Implementing Mimecast Targeted Attack Protection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: URL Protection Actions, Attachment Sandbox Results, Impersonation Types, Mimecast TTP Components, Compliance Alignment, Impersonation Protection Indicators

When to use

- The user wants to **implement** mimecast targeted attack protection.
- Capability: Mimecast Targeted Attack Protection Agent - monitors TAP events and URL/attachment threats.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, mimecast, targeted, attack, protection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute authenticated Mimecast API request
2. Get URL Protection logs
3. Get impersonation protection logs
4. Get attachment protection logs
5. Analyze URL click patterns and threat categories
6. Analyze impersonation attack patterns

Key concepts

- T1566.001: Phishing: Spearphishing Attachment
- T1566.002: Phishing: Spearphishing Link
- T1534: Internal Spearphishing
- T1598: Phishing for Information
- T1656: Impersonation
- T1204.001: User Execution: Malicious Link
- T1204.002: User Execution: Malicious File
- CIS Controls v8 Control 9.2: Use DNS filtering services

Tools & resources

- URL Protection Actions
- Attachment Sandbox Results
- Impersonation Types
- Mimecast TTP Components
- Compliance Alignment
- Impersonation Protection Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `mimecast_request` — Execute authenticated Mimecast API request.
- `get_url_logs` — Get URL Protection logs.
- `get_impersonation_logs` — Get impersonation protection logs.
- `get_attachment_logs` — Get attachment protection logs.
- `analyze_url_threats` — Analyze URL click patterns and threat categories.
- `analyze_impersonation` — Analyze impersonation attack patterns.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-mimecast-targeted-attack-protection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-mitre-attack-coverage-mapping (Skill Module)

implementing-mitre-attack-coverage-mapping\SKILL.md

name: implementing-mitre-attack-coverage-mapping
description: MITRE ATT&CK Coverage Mapping Agent - maps detection rules to ATT&CK techniques and identifies gaps. Use when the user asks to implement mitre attack coverage mapping.
version: 1.0.0
tags:
source: implementing-mitre-attack-coverage-mapping

Implementing MITRE ATT&CK Coverage Mapping

Role: Security control implementation
Audience: security architects and engineers
Primary tools: MITRE ATT&CK, Coverage Score, Detection Maturity Model, Related Frameworks

When to use

- The user wants to **implement** mitre attack coverage mapping.
- Capability: MITRE ATT&CK Coverage Mapping Agent - maps detection rules to ATT&CK techniques and identifies gaps.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, mitre, attack, coverage, mapping`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load detection rules with ATT&CK mappings
2. Load ATT&CK enterprise matrix (techniques per tactic)
3. Map detection rules to ATT&CK technique IDs
4. Calculate coverage percentage per tactic
5. Identify high-priority coverage gaps
6. Assess detection depth per technique (number of rules covering it)
7. Generate ATT&CK Navigator layer JSON

Key concepts

- Current version: v18.1 (December 2025)
- Domains: Enterprise, Mobile, ICS
- MITRE D3FEND (Defensive techniques)
- MITRE ATT&CK Data Sources
- NIST CSF Detection function
- SANS Detection Maturity Level model

Tools & resources

- MITRE ATT&CK
- Coverage Score
- Detection Maturity Model
- Related Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_detection_rules` — Load detection rules with ATT&CK mappings.
- `load_attack_matrix` — Load ATT&CK enterprise matrix (techniques per tactic).
- `map_rules_to_techniques` — Map detection rules to ATT&CK technique IDs.
- `calculate_coverage` — Calculate coverage percentage per tactic.
- `identify_priority_gaps` — Identify high-priority coverage gaps.
- `calculate_detection_depth` — Assess detection depth per technique (number of rules covering it).
- `generate_navigator_layer` — Generate ATT&CK Navigator layer JSON.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-mitre-attack-coverage-mapping`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-mobile-application-management (Skill Module)

implementing-mobile-application-management\SKILL.md

name: implementing-mobile-application-management
description: Mobile Application Management Agent - audits MDM policies, app inventory, and compliance posture. Use when the user asks to implement mobile application management.
version: 1.0.0
tags:
source: implementing-mobile-application-management

Implementing Mobile Application Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: MAM Policy Settings, Conditional Launch Settings, CIS Controls v8

When to use

- The user wants to **implement** mobile application management.
- Capability: Mobile Application Management Agent - audits MDM policies, app inventory, and compliance posture.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, mobile, application, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute MDM API request via curl
2. Get all managed devices
3. Get managed application inventory
4. Get device compliance policies
5. Get MAM app protection policies
6. Audit device compliance status
7. Audit managed app security configuration
8. Audit MAM protection policy configuration

Key concepts

- Section 4: Enterprise mobile security technologies (MDM, MAM, MTD)
- Section 5: App vetting and management requirements

Tools & resources

- MAM Policy Settings
- Conditional Launch Settings
- CIS Controls v8

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `mdm_api_request` — Execute MDM API request via curl.
- `get_managed_devices` — Get all managed devices.
- `get_managed_apps` — Get managed application inventory.
- `get_compliance_policies` — Get device compliance policies.
- `get_app_protection_policies` — Get MAM app protection policies.
- `audit_device_compliance` — Audit device compliance status.
- `audit_app_security` — Audit managed app security configuration.
- `audit_protection_policies` — Audit MAM protection policy configuration.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-mobile-application-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-mtls-for-zero-trust-services (Skill Module)

`implementing-mtls-for-zero-trust-services\SKILL.md`

name: implementing-mtls-for-zero-trust-services
description: Agent for implementing and auditing mutual TLS between services. Use when the user asks to implement mtls for zero trust services.
version: 1.0.0
tags:
source: implementing-mtls-for-zero-trust-services

Implementing Mtls for Zero Trust Services

Role: Security control implementation
Audience: security architects and engineers
Primary tools: cert

When to use

- The user wants to **implement** mtls for zero trust services.
- Capability: Agent for implementing and auditing mutual TLS between services.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, mtls, zero, trust, services`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate a self-signed CA certificate and key
2. Issue a service certificate signed by the CA
3. Save key and certificate to PEM files
4. Test mTLS connectivity to an endpoint
5. Audit all certificates in a directory for expiration and key size

Tools & resources

- cert

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_ca` — Generate a self-signed CA certificate and key.
- `issue_service_cert` — Issue a service certificate signed by the CA.
- `save_pem` — Save key and certificate to PEM files.
- `verify_mtls_endpoint` — Test mTLS connectivity to an endpoint.
- `audit_certificates` — Audit all certificates in a directory for expiration and key size.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-mtls-for-zero-trust-services`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-nerc-cip-compliance-controls (Skill Module)

`implementing-nerc-cip-compliance-controls\SKILL.md`

name: implementing-nerc-cip-compliance-controls
description: NERC CIP Compliance Agent - audits critical infrastructure against NERC CIP standards. Use when the user asks to implement nerc cip compliance controls.
version: 1.0.0
tags:
source: implementing-nerc-cip-compliance-controls

Implementing Nerc Cip Compliance Controls

Role: Security control implementation
Audience: security architects and engineers
Primary tools: CIP-005 ESP Requirements, CIP-007 Patching Timeline

When to use

- The user wants to **implement** nerc cip compliance controls.
- Capability: NERC CIP Compliance Agent - audits critical infrastructure against NERC CIP standards.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, nerc, compliance, controls`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check Electronic Security Perimeter controls via network scan
2. Check CIP-007 system hardening controls
3. Audit compliance status against all NERC CIP controls

Tools & resources

- CIP-005 ESP Requirements
- CIP-007 Patching Timeline

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_esp_controls` — Check Electronic Security Perimeter controls via network scan.
- `check_system_hardening` — Check CIP-007 system hardening controls.
- `audit_compliance_status` — Audit compliance status against all NERC CIP controls.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-nerc-cip-compliance-controls`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-access-control (Skill Module)

implementing-network-access-control\SKILL.md

name: implementing-network-access-control
description: Network Access Control (802.1X/NAC) monitoring agent using RADIUS and SNMP. Use when the user asks to implement network access control.
version: 1.0.0
tags:
source: implementing-network-access-control

Implementing Network Access Control

Role: Security control implementation
Audience: security architects and engineers
Primary tools: 802.1X SNMP OIDs, RADIUS Attributes, EAP Methods, PacketFence NAC API, External References

When to use

- The user wants to **implement** network access control.
- Capability: Network Access Control (802.1X/NAC) monitoring agent using RADIUS and SNMP.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, access, control`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test RADIUS authentication for a user credential pair
2. Parse FreeRADIUS log file for authentication events
3. Query switch via SNMP for 802.1X port authentication status
4. Analyze authentication events for security issues
5. Generate a NAC compliance policy checklist
6. Run NAC security audit

Tools & resources

- 802.1X SNMP OIDs
- RADIUS Attributes
- EAP Methods
- PacketFence NAC API
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_radius_auth` — Test RADIUS authentication for a user credential pair.
- `parse_radius_log` — Parse FreeRADIUS log file for authentication events.
- `check_switch_port_status` — Query switch via SNMP for 802.1X port authentication status.
- `analyze_auth_events` — Analyze authentication events for security issues.
- `generate_nac_policy_check` — Generate a NAC compliance policy checklist.
- `run_nac_audit` — Run NAC security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-access-control`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-access-control-with-cisco-ise (Skill Module)

implementing-network-access-control-with-cisco-ise\SKILL.md

name: implementing-network-access-control-with-cisco-ise
description: Cisco ISE NAC Agent - audits ISE policies, endpoint posture, and 802.1X configuration. Use when the user asks to implement network access control with cisco ise.
version: 1.0.0
tags:
source: implementing-network-access-control-with-cisco-ise

Implementing Network Access Control with Cisco Ise

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Cisco ISE ERS API, ISE Policy Components, RADIUS Attributes

When to use

- The user wants to **implement** network access control with cisco ise.
- Capability: Cisco ISE NAC Agent - audits ISE policies, endpoint posture, and 802.1X configuration.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, access, control, cisco`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute Cisco ISE ERS API request
2. Get active RADIUS sessions via MnT API
3. Audit authorization profiles for security best practices
4. Analyze active session posture and authentication methods
5. Check 802.1X deployment across network devices

Tools & resources

- Cisco ISE ERS API
- ISE Policy Components
- RADIUS Attributes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `ise_request` — Execute Cisco ISE ERS API request.
- `get_network_devices`
- `get_endpoint_groups`
- `get_authorization_policies`
- `get_active_sessions` — Get active RADIUS sessions via MnT API.
- `audit_authorization_profiles` — Audit authorization profiles for security best practices.
- `analyze_session_data` — Analyze active session posture and authentication methods.
- `check_dot1x_compliance` — Check 802.1X deployment across network devices.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-access-control-with-cisco-ise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-deception-with-honeypots (Skill Module)

implementing-network-deception-with-honeypots\SKILL.md

name: implementing-network-deception-with-honeypots
description: Honeypot Deployment Agent - deploys OpenCanary honeypots and analyzes interaction logs. Use when the user asks to implement network deception with honeypots.
version: 1.0.0
tags:
source: implementing-network-deception-with-honeypots

Implementing Network Deception with Honeypots

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OpenCanary Installation, OpenCanary CLI, Available Service Modules, Log Type Codes, Cowrie SSH Honeypot, Syslog Forwarding

When to use

- The user wants to **implement** network deception with honeypots.
- Capability: Honeypot Deployment Agent - deploys OpenCanary honeypots and analyzes interaction logs.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, deception, honeypots`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate OpenCanary configuration with specified services enabled
2. Deploy OpenCanary with generated configuration
3. Parse OpenCanary JSON log file for interaction events
4. Analyze honeypot interactions for threat intelligence
5. Check if OpenCanary daemon is running
6. Generate honeypot deployment and interaction report

Tools & resources

- OpenCanary Installation
- OpenCanary CLI
- Available Service Modules
- Log Type Codes
- Cowrie SSH Honeypot
- Syslog Forwarding

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_config` — Generate OpenCanary configuration with specified services enabled.
- `deploy_opencanary` — Deploy OpenCanary with generated configuration.
- `parse_opencanary_log` — Parse OpenCanary JSON log file for interaction events.
- `analyze_interactions` — Analyze honeypot interactions for threat intelligence.
- `check_honeypot_status` — Check if OpenCanary daemon is running.
- `generate_report` — Generate honeypot deployment and interaction report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-deception-with-honeypots`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-intrusion-prevention-with-suricata (Skill Module)

`implementing-network-intrusion-prevention-with-suricata\SKILL.md`

name: implementing-network-intrusion-prevention-with-suricata
description: Suricata IPS Agent - manages rules, analyzes alerts, and tunes Suricata intrusion prevention. Use when the user asks to implement network intrusion prevention with suricata.
version: 1.0.0
tags:
source: implementing-network-intrusion-prevention-with-suricata

Implementing Network Intrusion Prevention with Suricata

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Suricata, Performance Tuning

When to use

- The user wants to **implement** network intrusion prevention with suricata.
- Capability: Suricata IPS Agent - manages rules, analyzes alerts, and tunes Suricata intrusion prevention.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, intrusion, prevention, suricata`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse Suricata EVE JSON log for alerts
2. Analyze alert distribution by signature, severity, and source
3. Identify rules that generate excessive alerts for tuning
4. Detect coordinated attack patterns from alert correlation
5. Check Suricata service status and configuration

Tools & resources

- Suricata
- Performance Tuning

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_eve_alerts` — Parse Suricata EVE JSON log for alerts.
- `analyze_alert_distribution` — Analyze alert distribution by signature, severity, and source.
- `identify_noisy_rules` — Identify rules that generate excessive alerts for tuning.
- `detect_attack_patterns` — Detect coordinated attack patterns from alert correlation.
- `check_suricata_status` — Check Suricata service status and configuration.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-intrusion-prevention-with-suricata`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-policies-for-kubernetes (Skill Module)

implementing-network-policies-for-kubernetes\SKILL.md

name: implementing-network-policies-for-kubernetes
description: Kubernetes Network Policy Agent - audits pod-to-pod communication and network policy coverage. Use when the user asks to implement network policies for kubernetes.
version: 1.0.0
tags:
source: implementing-network-policies-for-kubernetes

Implementing Network Policies for Kubernetes

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Kubernetes

When to use

- The user wants to **implement** network policies for kubernetes.
- Capability: Kubernetes Network Policy Agent - audits pod-to-pod communication and network policy coverage.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, policies, kubernetes`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute kubectl command and return JSON output
2. Determine which pods are covered by network policies
3. Find network policies that allow all traffic
4. Check which namespaces have default-deny policies

Key concepts

- Implement network segmentation between namespaces
- Apply default-deny network policies
- Restrict pod-to-pod communication to required paths only
- Block access to cloud metadata endpoints

Tools & resources

- Kubernetes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `kubectl_json` — Execute kubectl command and return JSON output.
- `get_namespaces`
- `get_network_policies`
- `get_pods`
- `analyze_policy_coverage` — Determine which pods are covered by network policies.
- `detect_overly_permissive_policies` — Find network policies that allow all traffic.
- `analyze_namespace_isolation` — Check which namespaces have default-deny policies.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-policies-for-kubernetes`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-segmentation-for-ot (Skill Module)

implementing-network-segmentation-for-ot\SKILL.md

name: implementing-network-segmentation-for-ot
description: OT Network Segmentation Agent - audits IT/OT boundaries, firewall rules, and zone compliance. Use when the user asks to implement network segmentation for ot.
version: 1.0.0
tags:
source: implementing-network-segmentation-for-ot

Implementing Network Segmentation for Ot

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Purdue Reference Model, Zone Audit Checks, Common OT Protocols

When to use

- The user wants to **implement** network segmentation for ot.
- Capability: OT Network Segmentation Agent - audits IT/OT boundaries, firewall rules, and zone compliance.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, segmentation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover hosts in an OT zone via nmap ping scan
2. Parse firewall rules for IT/OT boundary enforcement
3. Check for exposed OT protocols on the network
4. Audit zone assignment compliance against Purdue model

Tools & resources

- Purdue Reference Model
- Zone Audit Checks
- Common OT Protocols

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_zone_hosts` — Discover hosts in an OT zone via nmap ping scan.
- `check_firewall_rules` — Parse firewall rules for IT/OT boundary enforcement.
- `check_ot_protocol_exposure` — Check for exposed OT protocols on the network.
- `audit_zone_compliance` — Audit zone assignment compliance against Purdue model.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-segmentation-for-ot`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-segmentation-with-firewall-zones (Skill Module)

implementing-network-segmentation-with-firewall-zones\SKILL.md

name: implementing-network-segmentation-with-firewall-zones
description: Firewall Zone Segmentation Agent - audits zone-based firewall rules and inter-zone traffic policies. Use when the user asks to implement network segmentation with firewall zones.
version: 1.0.0
tags:
source: implementing-network-segmentation-with-firewall-zones

Implementing Network Segmentation with Firewall Zones

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Zone Trust Levels, Palo Alto Zone-Based CLI, PCI DSS Segmentation Requirements

When to use

- The user wants to **implement** network segmentation with firewall zones.
- Capability: Firewall Zone Segmentation Agent - audits zone-based firewall rules and inter-zone traffic policies.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, segmentation, firewall, zones`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for network segmentation with firewall zones
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Zone Trust Levels
- Palo Alto Zone-Based CLI
- PCI DSS Segmentation Requirements

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_firewall_config`
- `get_iptables_zones`
- `audit_zone_rules`
- `check_default_zone_policies`
- `analyze_rule_shadowing`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-segmentation-with-firewall-zones`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-traffic-analysis-with-arkime (Skill Module)

implementing-network-traffic-analysis-with-arkime\SKILL.md

name: implementing-network-traffic-analysis-with-arkime
description: Arkime Network Traffic Analysis Agent - Queries Arkime API for session analysis and anomaly detection. Use when the user asks to implement network traffic analysis with arkime.
version: 1.0.0
tags:
source: implementing-network-traffic-analysis-with-arkime

Implementing Network Traffic Analysis with Arkime

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Arkime, Session Search, PCAP Download, Session Fields, Search Expressions, Beaconing Detection Logic

When to use

- The user wants to **implement** network traffic analysis with arkime.
- Capability: Arkime Network Traffic Analysis Agent - Queries Arkime API for session analysis and anomaly detection.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, traffic, analysis, arkime`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Make an authenticated request to Arkime API v3
2. Search Arkime sessions with an expression filter
3. Get connection graph data from Arkime
4. Get SPI view field statistics from Arkime
5. Detect C2 beaconing by analyzing connection intervals
6. Detect DNS tunneling via abnormally long DNS queries
7. Detect unusually large data transfers
8. Detect TLS certificate anomalies (self-signed, expired, unusual issuers)

Tools & resources

- Arkime
- Session Search
- PCAP Download
- Session Fields
- Search Expressions
- Beaconing Detection Logic

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `arkime_request` — Make an authenticated request to Arkime API v3.
- `search_sessions` — Search Arkime sessions with an expression filter.
- `get_connections` — Get connection graph data from Arkime.
- `get_spi_view` — Get SPI view field statistics from Arkime.
- `detect_beaconing` — Detect C2 beaconing by analyzing connection intervals.
- `detect_dns_tunneling` — Detect DNS tunneling via abnormally long DNS queries.
- `detect_large_transfers` — Detect unusually large data transfers.
- `detect_tls_anomalies` — Detect TLS certificate anomalies (self-signed, expired, unusual issuers).
- `generate_report` — Generate network traffic analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-traffic-analysis-with-arkime`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-network-traffic-baselining (Skill Module)

implementing-network-traffic-baselining\SKILL.md

name: implementing-network-traffic-baselining
description: Network traffic baselining agent using pandas for NetFlow/IPFIX statistical analysis. Use when the user asks to implement network traffic baselining.
version: 1.0.0
tags:
source: implementing-network-traffic-baselining

Implementing Network Traffic Baselining

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Pandas Analysis Functions, NetFlow Export Tools

When to use

- The user wants to **implement** network traffic baselining.
- Capability: Network traffic baselining agent using pandas for NetFlow/IPFIX statistical analysis.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, traffic, baselining`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load NetFlow/IPFIX records from CSV export
2. Compute hourly traffic volume baseline
3. Compute per-source-IP traffic baselines
4. Compute protocol distribution baseline
5. Detect anomalous hosts using z-score on bytes transferred
6. Detect outlier hosts using IQR method on bytes per flow
7. Detect hosts connecting to an unusually high number of unique ports

Tools & resources

- Pandas Analysis Functions
- NetFlow Export Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_netflow_csv` — Load NetFlow/IPFIX records from CSV export.
- `compute_hourly_baseline` — Compute hourly traffic volume baseline.
- `compute_host_baselines` — Compute per-source-IP traffic baselines.
- `compute_protocol_baseline` — Compute protocol distribution baseline.
- `detect_zscore_anomalies` — Detect anomalous hosts using z-score on bytes transferred.
- `detect_iqr_anomalies` — Detect outlier hosts using IQR method on bytes per flow.
- `detect_port_scan_pattern` — Detect hosts connecting to an unusually high number of unique ports.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-network-traffic-baselining`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-next-generation-firewall-with-palo-alto (Skill Module)

implementing-next-generation-firewall-with-palo-alto\SKILL.md

name: implementing-next-generation-firewall-with-palo-alto
description: Palo Alto NGFW Agent - audits security policies, threat prevention, and App-ID usage via XML API. Use when the user asks to implement next generation firewall with palo alto.
version: 1.0.0
tags:
source: implementing-next-generation-firewall-with-palo-alto

Implementing Next Generation Firewall with Palo Alto

Role: Security control implementation
Audience: security architects and engineers
Primary tools: PAN-OS XML API, Configuration XPaths, pan-python Library, Industry Standards, Palo Alto Specific Standards

When to use

- The user wants to **implement** next generation firewall with palo alto.
- Capability: Palo Alto NGFW Agent - audits security policies, threat prevention, and App-ID usage via XML API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, next, generation, firewall, palo, alto`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for next generation firewall with palo alto
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Key concepts

- ISO 27001: 2022** - A.13.1 (Network Security Management)

Tools & resources

- PAN-OS XML API
- Configuration XPaths
- pan-python Library
- Industry Standards
- Palo Alto Specific Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `pan_api_request`
- `get_security_rules`
- `audit_security_rules`
- `calculate_appid_coverage`
- `check_system_health`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-next-generation-firewall-with-palo-alto`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-opa-gatekeeper-for-policy-enforcement (Skill Module)

implementing-opa-gatekeeper-for-policy-enforcement\SKILL.md

name: implementing-opa-gatekeeper-for-policy-enforcement
description: OPA Gatekeeper Policy Enforcement Agent - audits constraint templates and violation status. Use when the user asks to implement opa gatekeeper for policy enforcement.
version: 1.0.0
tags:
source: implementing-opa-gatekeeper-for-policy-enforcement

Implementing Opa Gatekeeper for Policy Enforcement

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Gatekeeper CRDs, ConstraintTemplate Example, Industry Standards, Gatekeeper Policy Library, Compliance Mappings

When to use

- The user wants to **implement** opa gatekeeper for policy enforcement.
- Capability: OPA Gatekeeper Policy Enforcement Agent - audits constraint templates and violation status.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, gatekeeper, policy, enforcement`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for opa gatekeeper for policy enforcement
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Key concepts

- Section 4.1: Image vulnerabilities - Enforce image registry restrictions
- Section 4.2: Image configuration defects - Enforce security context requirements
- Section 5.2: Registry security - Restrict allowed image sources
- Section 2: Pod Security - Admission control enforcement
- Recommends admission controllers to enforce security baselines
- Req 2.2: Secure system components per configuration standards
- Req 6.3.2: Develop software securely with automated controls
- CC6.1: Logical access to system components is restricted

Tools & resources

- Gatekeeper CRDs
- ConstraintTemplate Example
- Industry Standards
- Gatekeeper Policy Library
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `kubectl_json`
- `get_constraint_templates`
- `get_constraints`
- `audit_constraint_violations`
- `analyze_policy_coverage`
- `check_audit_status`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-opa-gatekeeper-for-policy-enforcement`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ot-incident-response-playbook (Skill Module)

implementing-ot-incident-response-playbook\SKILL.md

name: implementing-ot-incident-response-playbook
description: OT Incident Response Playbook Agent - executes ICS/SCADA incident response procedures. Use when the user asks to implement ot incident response playbook.
version: 1.0.0
tags:
source: implementing-ot-incident-response-playbook

Implementing Ot Incident Response Playbook

Role: Security control implementation
Audience: security architects and engineers
Primary tools: ICS-CERT Incident Categories, Purdue Model Containment Zones, Reporting Obligations

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** ot incident response playbook.
- Capability: OT Incident Response Playbook Agent - executes ICS/SCADA incident response procedures.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, incident, response, playbook`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for ot incident response playbook
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- ICS-CERT Incident Categories
- Purdue Model Containment Zones
- Reporting Obligations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `assess_incident`
- `generate_containment_steps`
- `generate_recovery_plan`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ot-incident-response-playbook`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ot-network-traffic-analysis-with-nozomi (Skill Module)

`implementing-ot-network-traffic-analysis-with-nozomi\SKILL.md`

name: implementing-ot-network-traffic-analysis-with-nozomi
description: Nozomi Networks OT Traffic Analysis Agent - monitors ICS protocols and detects anomalies. Use when the user asks to implement ot network traffic analysis with nozomi.
version: 1.0.0
tags:
source: implementing-ot-network-traffic-analysis-with-nozomi

Implementing Ot Network Traffic Analysis with Nozomi

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Nozomi Guardian REST API, Supported OT Protocols, Alert Risk Levels, Sensor Deployment

When to use

- The user wants to **implement** ot network traffic analysis with nozomi.
- Capability: Nozomi Networks OT Traffic Analysis Agent - monitors ICS protocols and detects anomalies.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, network, traffic, analysis, nozomi`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for ot network traffic analysis with nozomi
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Nozomi Guardian REST API
- Supported OT Protocols
- Alert Risk Levels
- Sensor Deployment

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `nozomi_api`
- `get_alerts`
- `get_assets`
- `get_sessions`
- `analyze_ot_protocols`
- `detect_anomalies`
- `audit_asset_inventory`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ot-network-traffic-analysis-with-nozomi`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-pam-for-database-access (Skill Module)

`implementing-pam-for-database-access\SKILL.md`

name: implementing-pam-for-database-access
description: PAM Database Access Agent - audits privileged database sessions and access controls. Use when the user asks to implement pam for database access.
version: 1.0.0
tags:
source: implementing-pam-for-database-access

Implementing PAM for Database Access

Role: Security control implementation
Audience: security architects and engineers
Primary tools: hvac Python Client, CyberArk Privileged Cloud API, Session Proxy Configuration, NIST 800-53 PAM Controls

When to use

- The user wants to **implement** pam for database access.
- Capability: PAM Database Access Agent - audits privileged database sessions and access controls.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, database, access`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for pam for database access
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- hvac Python Client
- CyberArk Privileged Cloud API
- Session Proxy Configuration
- NIST 800-53 PAM Controls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `query_db_users`
- `audit_shared_accounts`
- `audit_session_logging`
- `check_tls`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-pam-for-database-access`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-passwordless-auth-with-microsoft-entra (Skill Module)

implementing-passwordless-auth-with-microsoft-entra\SKILL.md

name: implementing-passwordless-auth-with-microsoft-entra
description: Microsoft Entra ID passwordless authentication audit agent using MS Graph API. Use when the user asks to implement passwordless auth with microsoft entra.
version: 1.0.0
tags:
source: implementing-passwordless-auth-with-microsoft-entra

Implementing Passwordless Auth with Microsoft Entra

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Conditional Access API, Sign-In Logs API, Required Graph Permissions, External References

When to use

- The user wants to **implement** passwordless auth with microsoft entra.
- Capability: Microsoft Entra ID passwordless authentication audit agent using MS Graph API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, passwordless, auth, microsoft, entra`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate to Microsoft Graph using client credentials
2. Make authenticated GET request to Microsoft Graph
3. Get authentication methods policy to check passwordless configuration
4. Get FIDO2 security key configuration
5. Get Microsoft Authenticator configuration
6. Get Windows Hello for Business configuration
7. List authentication methods registered by a specific user
8. Identify users who have not registered any passwordless method

Tools & resources

- Conditional Access API
- Sign-In Logs API
- Required Graph Permissions
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_access_token` — Authenticate to Microsoft Graph using client credentials.
- `graph_get` — Make authenticated GET request to Microsoft Graph.
- `get_auth_methods_policy` — Get authentication methods policy to check passwordless configuration.
- `get_fido2_policy` — Get FIDO2 security key configuration.
- `get_microsoft_authenticator_policy` — Get Microsoft Authenticator configuration.
- `get_windows_hello_policy` — Get Windows Hello for Business configuration.
- `list_user_auth_methods` — List authentication methods registered by a specific user.
- `get_users_without_passwordless` — Identify users who have not registered any passwordless method.
- `get_sign_in_logs` — Get recent sign-in logs to analyze authentication methods used.
- `get_conditional_access_policies` — List conditional access policies related to authentication strength.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-passwordless-auth-with-microsoft-entra`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-passwordless-authentication-with-fido2 (Skill Module)

implementing-passwordless-authentication-with-fido2\SKILL.md

name: implementing-passwordless-authentication-with-fido2
description: FIDO2 Passwordless Auth Agent - audits FIDO2 deployment readiness and credential status. Use when the user asks to implement passwordless authentication with fido2.
version: 1.0.0
tags:
source: implementing-passwordless-authentication-with-fido2

Implementing Passwordless Authentication with Fido2

Role: Security control implementation
Audience: security architects and engineers
Primary tools: WebAuthn Registration Flow, python-fido2 Server Library, FIDO2 Authenticator Types, COSE Algorithm Identifiers, Azure AD FIDO2 Configuration, FIDO Standards

When to use

- The user wants to **implement** passwordless authentication with fido2.
- Capability: FIDO2 Passwordless Auth Agent - audits FIDO2 deployment readiness and credential status.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, passwordless, authentication, fido2`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. User receives FIDO2 security key (YubiKey, Titan Key)
2. User navigates to enrollment portal
3. System generates WebAuthn registration challenge
4. Browser prompts user to insert/tap security key
5. User verifies with PIN or biometric on key
6. Key generates unique public/private key pair
7. Public key registered with relying party
8. User tests authentication with enrolled key
9. User visits login page, enters username
10. Server sends WebAuthn authentication challenge

Key concepts

- WebAuthn Level 2: W3C Web Authentication API
- CTAP2: Client to Authenticator Protocol 2.0
- NIST SP 800-63B: AAL3 - Hardware-based phishing-resistant authenticator
- NIST SP 800-53 Rev 5: IA-2(6), IA-2(8) Replay-resistant authentication
- NIST SP 800-157: PIV Derived Credentials
- Phishing-Resistant MFA: Required for federal agencies under EO 14028
- OMB M-22-09: Federal zero trust strategy requiring phishing-resistant MFA
- Google Passkeys: Android and Chrome WebAuthn support

Tools & resources

- WebAuthn Registration Flow
- python-fido2 Server Library
- FIDO2 Authenticator Types
- COSE Algorithm Identifiers

- Azure AD FIDO2 Configuration
- FIDO Standards
- NIST Standards
- CISA Guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `graph_api`
- `get_fido2_policy`
- `get_registrations`
- `audit_fido2_policy`
- `analyze_adoption`
- `check_rp_config`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-passwordless-authentication-with-fido2`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-patch-management-for-ot-systems (Skill Module)

implementing-patch-management-for-ot-systems\SKILL.md

name: implementing-patch-management-for-ot-systems
description: OT Patch Management Agent - tracks ICS/SCADA patch status and risk assessment. Use when the user asks to implement patch management for ot systems.
version: 1.0.0
tags:
source: implementing-patch-management-for-ot-systems

Implementing Patch Management for Ot Systems

Role: Security control implementation
Audience: security architects and engineers
Primary tools: ICS-CERT Advisory API, Vendor Patch Sources, Patch Prioritization Matrix, NERC CIP-007-6 R2 Requirements, WSUS/SCCM OT Configuration

When to use

- The user wants to **implement** patch management for ot systems.
- Capability: OT Patch Management Agent - tracks ICS/SCADA patch status and risk assessment.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, patch, management, systems`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for patch management for ot systems
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- ICS-CERT Advisory API
- Vendor Patch Sources
- Patch Prioritization Matrix
- NERC CIP-007-6 R2 Requirements
- WSUS/SCCM OT Configuration

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_data`
- `assess_patch_risk`
- `check_compliance`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-patch-management-for-ot-systems`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-patch-management-workflow (Skill Module)

implementing-patch-management-workflow\SKILL.md

name: implementing-patch-management-workflow
description: Patch management workflow agent. Audits system patch compliance by checking installed package versions against known vulnerabilities, tracking patch SLA adherence, and generating remediation reports. Supports Linux.... Use when the user asks to implement patch management workflow.
version: 1.0.0
tags:
source: implementing-patch-management-workflow

Implementing Patch Management Workflow

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Tenable.io API, Qualys API, Patch Prioritization, Industry Standards, Patch Management Tools, Vendor Patch Schedules

When to use

- The user wants to **implement** patch management workflow.
- Capability: Patch management workflow agent. Audits system patch compliance by checking installed package versions against known vulnerabilities, tracking patch SLA adherence, and generating remediation reports. Supports Linux (apt/yum) and basic CVE cross-referencing via the CISA KEV catalog.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, patch, management, workflow`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check for available security updates on Debian/Ubuntu systems
2. Check for available security updates on RHEL/CentOS systems
3. Check for pending Windows updates via PowerShell
4. Cross-reference package CVEs against CISA KEV catalog
5. Assess patch compliance against SLA targets
6. Print patch management summary

Key concepts

- NIST SP 800-40 Rev 4: Guide to Enterprise Patch Management Planning
- NIST SP 800-53 SI-2: Flaw Remediation control
- CIS Controls v8 Control 7.3: Perform automated patch management
- PCI DSS v4.0 Req 6.3: Identify and address security vulnerabilities
- ISO 27001: 2022 A.8.8**: Management of technical vulnerabilities

Tools & resources

- Tenable.io API
- Qualys API
- Patch Prioritization
- Industry Standards
- Patch Management Tools
- Vendor Patch Schedules

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_apt_updates` — Check for available security updates on Debian/Ubuntu systems.
- `check_yum_updates` — Check for available security updates on RHEL/CentOS systems.
- `check_windows_updates` — Check for pending Windows updates via PowerShell.
- `check_kev_exposure` — Cross-reference package CVEs against CISA KEV catalog.
- `assess_patch_sla` — Assess patch compliance against SLA targets.
- `format_summary` — Print patch management summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-patch-management-workflow`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-pci-dss-compliance-controls (Skill Module)

implementing-pci-dss-compliance-controls\SKILL.md

name: implementing-pci-dss-compliance-controls
description: PCI DSS compliance control audit agent. Audits systems and configurations against PCI DSS v4.0 requirements including network segmentation, encryption, access controls, logging, vulnerability management, and secure.... Use when the user asks to implement pci dss compliance controls.
version: 1.0.0
tags:
source: implementing-pci-dss-compliance-controls

Implementing Pci Dss Compliance Controls

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Compliance Checks, Summary, Findings, Primary Standard, Key Requirements, Cross-References

When to use

- The user wants to **implement** pci dss compliance controls.
- Capability: PCI DSS compliance control audit agent. Audits systems and configurations against PCI DSS v4.0 requirements including network segmentation, encryption, access controls, logging, vulnerability management, and secure configuration checks.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, compliance, controls`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. PCI DSS Req 4.2.1 - Strong cryptography for transmission
2. PCI DSS Req 8.3.6 - Password complexity requirements
3. PCI DSS Req 10.2 - Audit logging configuration
4. PCI DSS Req 11.5.2 - File integrity monitoring
5. PCI DSS Req 2.2.2 - Change vendor defaults
6. PCI DSS Req 1.3 - Network segmentation check
7. Print PCI DSS audit summary

Key concepts

- Standard: PCI DSS v4.0.1
- Governing Body: PCI Security Standards Council
- Requirement 1: Install and maintain network security controls
- Requirement 2: Apply secure configurations to all system components
- Requirement 3: Protect stored account data
- Requirement 4: Protect cardholder data with strong cryptography during transmission
- Requirement 5: Protect all systems and networks from malicious software
- Requirement 6: Develop and maintain secure systems and software

Tools & resources

- Core Compliance Checks
- Summary
- Findings
- Primary Standard
- Key Requirements

- Cross-References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_tls_configuration` — PCI DSS Req 4.2.1 - Strong cryptography for transmission.
- `check_password_policy` — PCI DSS Req 8.3.6 - Password complexity requirements.
- `check_audit_logging` — PCI DSS Req 10.2 - Audit logging configuration.
- `check_file_integrity` — PCI DSS Req 11.5.2 - File integrity monitoring.
- `check_default_credentials` — PCI DSS Req 2.2.2 - Change vendor defaults.
- `check_network_segmentation` — PCI DSS Req 1.3 - Network segmentation check.
- `format_summary` — Print PCI DSS audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-pci-dss-compliance-controls`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-pod-security-admission-controller (Skill Module)

implementing-pod-security-admission-controller\SKILL.md

name: implementing-pod-security-admission-controller
description: Kubernetes Pod Security Admission audit agent. Audits Kubernetes namespaces and workloads for Pod Security Standards (PSS) compliance using kubectl. Checks namespace labels for enforce/audit/warn modes, analyzes pod.... Use when the user asks to implement pod security admission controller.
version: 1.0.0
tags:
source: implementing-pod-security-admission-controller

Implementing Pod Security Admission Controller

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Operations, kubectl Equivalents, Kubernetes Pod Security Standards, CIS Kubernetes Benchmark v1.8, NIST SP 800-190, Compliance Mappings

When to use

- The user wants to **implement** pod security admission controller.
- Capability: Kubernetes Pod Security Admission audit agent. Audits Kubernetes namespaces and workloads for Pod Security Standards (PSS) compliance using kubectl. Checks namespace labels for enforce/audit/warn modes, analyzes pod specs against Baseline and Restricted profiles, and reports violations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, admission, controller`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute a kubectl command and return parsed JSON output
2. Get all namespaces with their PSA labels
3. Audit PSA label configuration across namespaces
4. Audit pods for security spec violations against PSS profiles
5. Print audit summary

Key concepts

- Section 4.3: Container runtime security
- Section 5.4: Admission control enforcement
- Section 1: Pod Security - Use Pod Security Standards
- PCI DSS v4.0 Req 2.2: Configuration standards
- SOC 2 CC6.1: Logical access controls
- HIPAA 164.312(a)(1): Access controls

Tools & resources

- Core Operations
- kubectl Equivalents
- Kubernetes Pod Security Standards
- CIS Kubernetes Benchmark v1.8
- NIST SP 800-190
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_kubectl` — Execute a kubectl command and return parsed JSON output.
- `get_namespaces` — Get all namespaces with their PSA labels.
- `audit_namespace_psa` — Audit PSA label configuration across namespaces.
- `audit_pod_security` — Audit pods for security spec violations against PSS profiles.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-pod-security-admission-controller`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-policy-as-code-with-open-policy-agent (Skill Module)

`implementing-policy-as-code-with-open-policy-agent\SKILL.md`

name: implementing-policy-as-code-with-open-policy-agent
description: Open Policy Agent (OPA) policy-as-code agent. Evaluates security policies against infrastructure configurations using the OPA REST API or CLI. Supports evaluating Rego policies for Kubernetes admission control,.... Use when the user asks to implement policy as code with open policy agent.
version: 1.0.0
tags:
source: implementing-policy-as-code-with-open-policy-agent

Implementing Policy As Code with Open Policy Agent

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Operations, OPA CLI Reference, NIST SP 800-53

When to use

- The user wants to **implement** policy as code with open policy agent.
- Capability: Open Policy Agent (OPA) policy-as-code agent. Evaluates security policies against infrastructure configurations using the OPA REST API or CLI. Supports evaluating Rego policies for Kubernetes admission control, Terraform plans, IAM policies, and custom security rules.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, policy, code, open, policy, agent`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Locate the OPA binary on the system
2. Evaluate a Rego policy using OPA CLI
3. Evaluate a policy via OPA REST API
4. Run OPA test suite against policy directory
5. Check Rego policy syntax
6. Extract policy violations from OPA evaluation result
7. Print evaluation summary

Key concepts

- Enforce Pod Security Standards via admission control
- Restrict container capabilities using OPA policies
- Enforce network policies and resource quotas
- Validate image provenance and signatures

Tools & resources

- Core Operations
- OPA CLI Reference
- NIST SP 800-53

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `find_opa_binary` — Locate the OPA binary on the system.
- `eval_policy_cli` — Evaluate a Rego policy using OPA CLI.
- `eval_policy_api` — Evaluate a policy via OPA REST API.
- `test_policies` — Run OPA test suite against policy directory.
- `check_policy_syntax` — Check Rego policy syntax.
- `extract_violations` — Extract policy violations from OPA evaluation result.
- `format_summary` — Print evaluation summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-policy-as-code-with-open-policy-agent`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-privileged-access-management-with-cyberark (Skill Module)

implementing-privileged-access-management-with-cyberark\SKILL.md

name: implementing-privileged-access-management-with-cyberark
description: CyberArk PAM configuration audit agent. Audits CyberArk Privileged Access Management via the REST API to verify safe configurations, privileged account inventory, platform assignments, and password rotation compliance. Use when the user asks to implement privileged access management with cyberark.
version: 1.0.0
tags:
source: implementing-privileged-access-management-with-cyberark

Implementing Privileged Access Management with Cyberark

Role: Security control implementation
Audience: security architects and engineers
Primary tools: CyberArk, Core Operations, NIST Standards, Industry Standards, Compliance Frameworks

When to use

- The user wants to **implement** privileged access management with cyberark.
- Capability: CyberArk PAM configuration audit agent. Audits CyberArk Privileged Access Management via the REST API to verify safe configurations, privileged account inventory, platform assignments, and password rotation compliance.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, privileged, access, management, cyberark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return CyberArk PVWA URL
2. Authenticate and get session token
3. Make authenticated API call
4. Audit safe configurations
5. Audit privileged accounts
6. Audit platform configurations
7. End the CyberArk session
8. Print audit summary

Key concepts

- NIST SP 800-53 Rev 5: Security and Privacy Controls
- AC-2: Account Management
- AC-5: Separation of Duties
- AC-6: Least Privilege
- AC-6(7): Review of User Privileges (Privileged Accounts)
- AU-14: Session Audit
- IA-5: Authenticator Management
- NIST SP 800-171: Protecting CUI - 3.1.5 Least Privilege, 3.1.7 Privileged Functions

Tools & resources

- CyberArk
- Core Operations
- NIST Standards
- Industry Standards

- Compliance Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_cyberark_config` — Return CyberArk PVWA URL.
- `authenticate` — Authenticate and get session token.
- `api_call` — Make authenticated API call.
- `audit_safes` — Audit safe configurations.
- `audit_accounts` — Audit privileged accounts.
- `audit_platforms` — Audit platform configurations.
- `logoff` — End the CyberArk session.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-privileged-access-management-with-cyberark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-privileged-access-workstation (Skill Module)

implementing-privileged-access-workstation\SKILL.md

name: implementing-privileged-access-workstation
description: Agent for implementing and auditing Privileged Access Workstation (PAW) configurations. Use when the user asks to implement privileged access workstation.
version: 1.0.0
tags:
source: implementing-privileged-access-workstation

Implementing Privileged Access Workstation

Role: Security control implementation
Audience: security architects and engineers
Primary tools: PAW Hardening Checks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** privileged access workstation.
- Capability: Agent for implementing and auditing Privileged Access Workstation (PAW) configurations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, privileged, access, workstation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Audit local device hardening controls for PAW compliance
2. Enumerate local Administrators group membership for JIT audit
3. Audit installed software against PAW allowlist
4. Verify PAW network isolation and restrictions
5. Run comprehensive PAW compliance audit

Tools & resources

- PAW Hardening Checks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_device_hardening` — Audit local device hardening controls for PAW compliance.
- `check_local_admin_group` — Enumerate local Administrators group membership for JIT audit.
- `check_installed_software` — Audit installed software against PAW allowlist.
- `check_network_restrictions` — Verify PAW network isolation and restrictions.
- `full_paw_audit` — Run comprehensive PAW compliance audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-privileged-access-workstation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-privileged-session-monitoring (Skill Module)

implementing-privileged-session-monitoring\SKILL.md

name: implementing-privileged-session-monitoring
description: Privileged session monitoring agent. Monitors SSH auth logs, Windows RDP events, and database sessions to track privileged access, detect anomalies, and generate audit reports. Use when the user asks to implement privileged session monitoring.
version: 1.0.0
tags:
source: implementing-privileged-session-monitoring

Implementing Privileged Session Monitoring

Role: Security control implementation
Audience: security architects and engineers
Primary tools: SSH Auth Log Parsing, CyberArk PSM REST API, Session Policy Fields

When to use

- The user wants to **implement** privileged session monitoring.
- Capability: Privileged session monitoring agent. Monitors SSH auth logs, Windows RDP events, and database sessions to track privileged access, detect anomalies, and generate audit reports.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, privileged, session, monitoring`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse SSH authentication log for session events
2. Parse Windows RDP logon events via PowerShell
3. Check if a command violates session policy
4. Detect anomalous session patterns
5. Generate privileged session audit report

Tools & resources

- SSH Auth Log Parsing
- CyberArk PSM REST API
- Session Policy Fields

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_ssh_auth_log` — Parse SSH authentication log for session events.
- `parse_rdp_events_windows` — Parse Windows RDP logon events via PowerShell.
- `check_command_policy` — Check if a command violates session policy.
- `detect_session_anomalies` — Detect anomalous session patterns.
- `generate_audit_report` — Generate privileged session audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-privileged-session-monitoring`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-proofpoint-email-security-gateway (Skill Module)

implementing-proofpoint-email-security-gateway\SKILL.md

name: implementing-proofpoint-email-security-gateway
description: Proofpoint email security gateway audit agent. Audits Proofpoint TAP (Targeted Attack Protection) via the SIEM API to retrieve blocked threats, clicked URLs, delivered messages, and campaign attribution data for.... Use when the user asks to implement proofpoint email security gateway.
version: 1.0.0
tags:
source: implementing-proofpoint-email-security-gateway

Implementing Proofpoint Email Security Gateway

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Operations, Industry Standards, Proofpoint-Specific References, Compliance Alignment, Email Security Gateway Market

When to use

- The user wants to **implement** proofpoint email security gateway.
- Capability: Proofpoint email security gateway audit agent. Audits Proofpoint TAP (Targeted Attack Protection) via the SIEM API to retrieve blocked threats, clicked URLs, delivered messages, and campaign attribution data for email security monitoring.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, proofpoint, email, security, gateway`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return Proofpoint TAP API credentials
2. Make authenticated Proofpoint TAP API call
3. Get URLs that were blocked when clicked
4. Get messages that were blocked
5. Get threats that were delivered (missed by filters)
6. Get URLs that were permitted when clicked (potential misses)
7. Analyze threat data for security insights
8. Print email security summary

Key concepts

- NIST SP 800-177 Rev.1: Trustworthy Email - guidelines for email security deployment
- RFC 7208: Sender Policy Framework (SPF) for authorizing use of domains in email
- RFC 6376: DomainKeys Identified Mail (DKIM) Signatures
- RFC 7489: Domain-based Message Authentication, Reporting & Conformance (DMARC)
- CIS Controls v8 Control 9: Email and Web Browser Protections
- T1566.001: Phishing: Spearphishing Attachment
- T1566.002: Phishing: Spearphishing Link
- T1566.003: Phishing: Spearphishing via Service

Tools & resources

- Core Operations
- Industry Standards
- Proofpoint-Specific References
- Compliance Alignment

- Email Security Gateway Market

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_pp_config` — Return Proofpoint TAP API credentials.
- `pp_api` — Make authenticated Proofpoint TAP API call.
- `get_blocked_clicks` — Get URLs that were blocked when clicked.
- `get_blocked_messages` — Get messages that were blocked.
- `get_delivered_threats` — Get threats that were delivered (missed by filters).
- `get_permitted_clicks` — Get URLs that were permitted when clicked (potential misses).
- `analyze_threats` — Analyze threat data for security insights.
- `format_summary` — Print email security summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-proofpoint-email-security-gateway`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-purdue-model-network-segmentation (Skill Module)

implementing-purdue-model-network-segmentation\SKILL.md

name: implementing-purdue-model-network-segmentation
description: Purdue model OT network segmentation audit agent. Audits OT/ICS network segmentation against the Purdue Enterprise Reference Architecture by testing connectivity between network zones, verifying firewall rules, and.... Use when the user asks to implement purdue model network segmentation.
version: 1.0.0
tags:
source: implementing-purdue-model-network-segmentation

Implementing Purdue Model Network Segmentation

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Purdue Model Levels, Core Audit Functions

When to use

- The user wants to **implement** purdue model network segmentation.
- Capability: Purdue model OT network segmentation audit agent. Audits OT/ICS network segmentation against the Purdue Enterprise Reference Architecture by testing connectivity between network zones, verifying firewall rules, and mapping discovered hosts to Purdue levels.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, purdue, model, network, segmentation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test TCP connectivity between two hosts on specified ports
2. Audit network segmentation between Purdue zones
3. Check for exposed OT/ICS protocols on target hosts
4. Load zone-to-host mapping from config file
5. Print audit summary

Tools & resources

- Purdue Model Levels
- Core Audit Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_connectivity` — Test TCP connectivity between two hosts on specified ports.
- `audit_zone_separation` — Audit network segmentation between Purdue zones.
- `audit_ot_protocols` — Check for exposed OT/ICS protocols on target hosts.
- `load_zone_map` — Load zone-to-host mapping from config file.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-purdue-model-network-segmentation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ransomware-backup-strategy (Skill Module)

implementing-ransomware-backup-strategy\SKILL.md

name: implementing-ransomware-backup-strategy
description: Ransomware backup strategy audit agent. Audits backup infrastructure for ransomware resilience by checking 3-2-1 backup rule compliance, air-gapped/immutable backup presence, backup encryption status, recovery point.... Use when the user asks to implement ransomware backup strategy.
version: 1.0.0
tags:
source: implementing-ransomware-backup-strategy

Implementing Ransomware Backup Strategy

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS S3 Backup Audit, AWS Backup Service, 3-2-1 Backup Rule Audit, Industry Standards, Regulatory Requirements, Vendor Documentation

When to use

- The user wants to **implement** ransomware backup strategy.
- Capability: Ransomware backup strategy audit agent. Audits backup infrastructure for ransomware resilience by checking 3-2-1 backup rule compliance, air-gapped/immutable backup presence, backup encryption status, recovery point objectives (RPO), and backup integrity verification schedules.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, ransomware, backup, strategy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check Veeam backup status via REST API
2. Audit a Restic backup repository for integrity and freshness
3. Audit compliance with the 3-2-1 backup rule
4. Print audit summary

Key concepts

- Defines security controls for storage systems including backup infrastructure
- Covers access control, encryption, integrity verification, and audit logging for storage
- Section 5.3: Backup and recovery security controls
- Identifies backup as a critical control in the Recover function
- Recommends maintaining offline, encrypted backups with regular testing
- Emphasizes credential separation for backup administration
- Prescribes 3-2-1 backup rule as baseline, recommends extending to 3-2-1-1-0
- Mandates backup credential isolation from production domains

Tools & resources

- AWS S3 Backup Audit
- AWS Backup Service
- 3-2-1 Backup Rule Audit
- Industry Standards
- Regulatory Requirements
- Vendor Documentation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_veeam_backups` — Check Veeam backup status via REST API.
- `check_restic_repository` — Audit a Restic backup repository for integrity and freshness.
- `audit_321_rule` — Audit compliance with the 3-2-1 backup rule.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ransomware-backup-strategy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ransomware-kill-switch-detection (Skill Module)

implementing-ransomware-kill-switch-detection\SKILL.md

name: implementing-ransomware-kill-switch-detection
description: Ransomware kill switch detection and mutex vaccination agent. Detects ransomware kill switch mechanisms (mutexes, domains, registry keys) and can proactively deploy mutex vaccinations to prevent known ransomware.... Use when the user asks to implement ransomware kill switch detection.
version: 1.0.0
tags:
source: implementing-ransomware-kill-switch-detection

Implementing Ransomware Kill Switch Detection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Known Kill Switch Domains, Velociraptor Mutex Hunting, DNS Kill Switch Monitoring, Mutex Vaccination Deployment Methods

When to use

- The user wants to **implement** ransomware kill switch detection.
- Capability: Ransomware kill switch detection and mutex vaccination agent. Detects ransomware kill switch mechanisms (mutexes, domains, registry keys) and can proactively deploy mutex vaccinations to prevent known ransomware families from executing. Monitors for kill switch domain DNS queries.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, ransomware, kill, switch, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check if a named mutex exists on Windows using PowerShell
2. Create a named mutex on Windows for vaccination
3. Check if a kill switch domain resolves (indicating it is active)
4. Scan for all known kill switch mechanisms
5. Deploy mutex vaccinations to prevent ransomware execution
6. Generate a PowerShell script for persistent mutex vaccination

Tools & resources

- Known Kill Switch Domains
- Velociraptor Mutex Hunting
- DNS Kill Switch Monitoring
- Mutex Vaccination Deployment Methods

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_mutex_exists_windows` — Check if a named mutex exists on Windows using PowerShell.
- `create_mutex_windows` — Create a named mutex on Windows for vaccination.
- `check_kill_switch_domain` — Check if a kill switch domain resolves (indicating it is active).
- `scan_all_kill_switches` — Scan for all known kill switch mechanisms.
- `vaccinate_endpoint` — Deploy mutex vaccinations to prevent ransomware execution.
- `generate_vaccination_script` — Generate a PowerShell script for persistent mutex vaccination.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ransomware-kill-switch-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-rapid7-insightvm-for-scanning (Skill Module)

implementing-rapid7-insightvm-for-scanning\SKILL.md

name: implementing-rapid7-insightvm-for-scanning
description: Rapid7 InsightVM vulnerability scanning agent. Interfaces with the InsightVM (Nexpose) REST API to manage scan configurations, launch scans, retrieve vulnerability results, and generate remediation reports. Supports.... Use when the user asks to implement rapid7 insightvm for scanning.
version: 1.0.0
tags:
source: implementing-rapid7-insightvm-for-scanning

Implementing Rapid7 Insightvm for Scanning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Operations, Pagination, Official Documentation, Scan Configuration References, Industry Standards, Compliance Scan Templates

When to use

- The user wants to **implement** rapid7 insightvm for scanning.
- Capability: Rapid7 InsightVM vulnerability scanning agent. Interfaces with the InsightVM (Nexpose) REST API to manage scan configurations, launch scans, retrieve vulnerability results, and generate remediation reports. Supports site management, asset discovery, and vulnerability prioritization.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, rapid7, insightvm, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return InsightVM API connection config
2. Make authenticated API call to InsightVM
3. List all scan sites
4. Get vulnerabilities for a specific site
5. Launch a scan on a site
6. Poll scan status until completion
7. Get scan results summary
8. Print summary

Key concepts

- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment
- NIST SP 800-40 Rev 4: Guide to Enterprise Patch Management Planning
- PCI DSS v4.0 Req 11.3: External and internal vulnerability scanning
- CIS Controls v8.1 Control 7: Continuous Vulnerability Management
- ISO 27001: 2022 A.8.8**: Management of technical vulnerabilities

Tools & resources

- Core Operations
- Pagination
- Official Documentation
- Scan Configuration References
- Industry Standards
- Compliance Scan Templates

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_insightvm_config` — Return InsightVM API connection config.
- `api_call` — Make authenticated API call to InsightVM.
- `list_sites` — List all scan sites.
- `get_site_vulnerabilities` — Get vulnerabilities for a specific site.
- `launch_scan` — Launch a scan on a site.
- `poll_scan_status` — Poll scan status until completion.
- `get_scan_report` — Get scan results summary.
- `format_summary` — Print summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-rapid7-insightvm-for-scanning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-rbac-hardening-for-kubernetes (Skill Module)

implementing-rbac-hardening-for-kubernetes\SKILL.md

name: implementing-rbac-hardening-for-kubernetes
description: Kubernetes RBAC hardening audit agent. Audits Kubernetes Role-Based Access Control configuration for security weaknesses including overly permissive ClusterRoles, wildcard permissions, privilege escalation paths, and.... Use when the user asks to implement rbac hardening for kubernetes.
version: 1.0.0
tags:
source: implementing-rbac-hardening-for-kubernetes

Implementing RBAC Hardening for Kubernetes

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Kubernetes, RBAC API Methods, Core Audit Operations, kubectl Equivalents, NIST SP 800-190

When to use

- The user wants to **implement** rbac hardening for kubernetes.
- Capability: Kubernetes RBAC hardening audit agent. Audits Kubernetes Role-Based Access Control configuration for security weaknesses including overly permissive ClusterRoles, wildcard permissions, privilege escalation paths, and service account misconfigurations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, rbac, hardening, kubernetes`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute kubectl and return parsed JSON
2. Audit ClusterRoles for dangerous permissions
3. Audit ClusterRoleBindings for overly broad subject assignments
4. Audit service accounts for misconfigurations
5. Print RBAC audit summary

Key concepts

- Section 3.4: Orchestrator security -- access control hardening
- Section 4.4: Countermeasures for orchestrator vulnerabilities
- T1078.004: Valid Accounts: Cloud Accounts -- compromised service accounts
- T1098: Account Manipulation -- RBAC escalation
- T1069: Permission Groups Discovery -- enumerating RBAC bindings

Tools & resources

- Kubernetes
- RBAC API Methods
- Core Audit Operations
- kubectl Equivalents
- NIST SP 800-190

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_kubectl` — Execute kubectl and return parsed JSON.
- `audit_cluster_roles` — Audit ClusterRoles for dangerous permissions.
- `audit_cluster_role_bindings` — Audit ClusterRoleBindings for overly broad subject assignments.
- `audit_service_accounts` — Audit service accounts for misconfigurations.
- `format_summary` — Print RBAC audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-rbac-hardening-for-kubernetes`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-rsa-key-pair-management (Skill Module)

implementing-rsa-key-pair-management\SKILL.md

name: implementing-rsa-key-pair-management
description: RSA key pair lifecycle management agent. Generates, audits, rotates, and manages RSA key pairs using the cryptography library. Supports key generation with configurable sizes, PEM export with encryption, public key.... Use when the user asks to implement rsa key pair management.
version: 1.0.0
tags:
source: implementing-rsa-key-pair-management

Implementing Rsa Key Pair Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Key Generation, Signing and Verification, Encryption and Decryption, Key Audit and Rotation, Self-Signed Certificate Generation, Primary Standards

When to use

- The user wants to **implement** rsa key pair management.
- Capability: RSA key pair lifecycle management agent. Generates, audits, rotates, and manages RSA key pairs using the cryptography library. Supports key generation with configurable sizes, PEM export with encryption, public key extraction, key strength auditing, and expiration tracking.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, pair, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate an RSA key pair
2. Save key pair to files with secure permissions
3. Audit an existing RSA key file for security issues
4. Scan a directory for key files and audit each one
5. Print a human-readable summary

Key concepts

- Description: Specifies RSA digital signature generation and verification
- RSA minimum: 2048-bit keys
- Description: Defines RSA key formats, OAEP encryption, and PSS signatures
- Key operations: RSAEP, RSADP, RSASP1, RSAVP1
- Description: Private key information syntax for storage
- Description: PEM encoding format specification
- Description: Key length recommendations and lifecycle management
- RSA 2048: Acceptable through 2030

Tools & resources

- Key Generation
- Signing and Verification
- Encryption and Decryption
- Key Audit and Rotation
- Self-Signed Certificate Generation
- Primary Standards
- Python Library References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_key_pair` — Generate an RSA key pair.
- `save_key_pair` — Save key pair to files with secure permissions.
- `audit_key_file` — Audit an existing RSA key file for security issues.
- `scan_directory_for_keys` — Scan a directory for key files and audit each one.
- `format_summary` — Print a human-readable summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-rsa-key-pair-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-runtime-application-self-protection (Skill Module)

implementing-runtime-application-self-protection\SKILL.md

name: implementing-runtime-application-self-protection
description: RASP Agent - audits runtime application protection config, attack logs, and coverage. Use when the user asks to implement runtime application self protection.
version: 1.0.0
tags:
source: implementing-runtime-application-self-protection

Implementing Runtime Application Self Protection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OpenRASP Deployment, Detection Hooks, RASP vs WAF Comparison

When to use

- The user wants to **implement** runtime application self protection.
- Capability: RASP Agent - audits runtime application protection config, attack logs, and coverage.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, runtime, application, self, protection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse OpenRASP or RASP agent configuration
2. Audit RASP detection policies against OWASP Top 10 coverage
3. Analyze RASP attack detection logs
4. Verify RASP agent deployment health
5. Calculate OWASP Top 10 coverage percentage

Tools & resources

- OpenRASP Deployment
- Detection Hooks
- RASP vs WAF Comparison

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_rasp_config` — Parse OpenRASP or RASP agent configuration.
- `audit_rasp_policy` — Audit RASP detection policies against OWASP Top 10 coverage.
- `analyze_attack_logs` — Analyze RASP attack detection logs.
- `check_agent_health` — Verify RASP agent deployment health.
- `calculate_coverage` — Calculate OWASP Top 10 coverage percentage.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-runtime-application-self-protection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-runtime-security-with-tetragon (Skill Module)

implementing-runtime-security-with-tetragon\SKILL.md

name: implementing-runtime-security-with-tetragon
description: Agent for auditing Cilium Tetragon runtime security configuration. Use when the user asks to implement runtime security with tetragon.
version: 1.0.0
tags:
source: implementing-runtime-security-with-tetragon

Implementing Runtime Security with Tetragon

Role: Security control implementation
Audience: security architects and engineers
Primary tools: TracingPolicy CRD, Industry Standards, CNCF Landscape Positioning

When to use

- The user wants to **implement** runtime security with tetragon.
- Capability: Agent for auditing Cilium Tetragon runtime security configuration.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, runtime, security, tetragon`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check if Tetragon is deployed in the cluster
2. Check TracingPolicy custom resources
3. Check tetra CLI availability and events

Key concepts

- Section 4.2: Runtime monitoring and anomaly detection for containers
- Section 4.4: Container-level network monitoring requirements
- Recommends kernel-level security monitoring for container environments
- Control 5.7.1: Create administrative boundaries between resources using namespaces
- Control 5.7.3: Apply Security Context to pods and containers
- Control 5.7.4: The default namespace should not be used
- T1611: Escape to Host -- Tetragon detects namespace manipulation attempts
- T1059.004: Command and Scripting Interpreter: Unix Shell -- process execution monitoring

Tools & resources

- TracingPolicy CRD
- Industry Standards
- CNCF Landscape Positioning

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_tetragon_deployment` — Check if Tetragon is deployed in the cluster.
- `check_tracing_policies` — Check TracingPolicy custom resources.
- `check_tetragon_cli` — Check tetra CLI availability and events.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-runtime-security-with-tetragon`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-saml-sso-with-okta (Skill Module)

implementing-saml-sso-with-okta\SKILL.md

name: implementing-saml-sso-with-okta
description: Agent for implementing and auditing SAML SSO with Okta. Validates SAML configuration, checks certificate expiry, tests assertion encryption, audits attribute mappings, and verifies signature algorithms for enterprise.... Use when the user asks to implement saml sso with okta.
version: 1.0.0
tags:
source: implementing-saml-sso-with-okta

Implementing SAML SSO with Okta

Role: Security control implementation
Audience: security architects and engineers
Primary tools: SAML, Okta, NIST Standards, Security Best Practices, XML Security Standards, Compliance Frameworks

When to use

- The user wants to **implement** saml sso with okta.
- Capability: Agent for implementing and auditing SAML SSO with Okta. Validates SAML configuration, checks certificate expiry, tests assertion encryption, audits attribute mappings, and verifies signature algorithms for enterprise SSO deployments.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, saml, okta`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. User accesses protected resource on Service Provider
2. SP checks for existing session - none found
3. SP generates SAML AuthnRequest with:
4. SP redirects user to Okta SSO URL with base64-encoded AuthnRequest
5. Okta authenticates user (credentials, MFA if configured)
6. Okta generates SAML Response containing:
7. Okta POSTs SAML Response to SP ACS URL
8. SP validates SAML Response:
9. SP extracts user identity and attributes
10. SP creates local session and grants access

Key concepts

- OASIS SAML 2.0 Core: Defines assertions, protocols, bindings, and profiles
- http: //docs.oasis-open.org/security/saml/v2.0/saml-core-2.0-os.pdf
- http: //docs.oasis-open.org/security/saml/v2.0/saml-bindings-2.0-os.pdf
- SAML 2.0 Profiles: Web Browser SSO Profile, Single Logout Profile
- http: //docs.oasis-open.org/security/saml/v2.0/saml-profiles-2.0-os.pdf
- NIST SP 800-63B: Digital Identity Guidelines - Authentication and Lifecycle Management
- https: //pages.nist.gov/800-63-3/sp800-63b.html
- NIST SP 800-53 Rev 5: Security and Privacy Controls

Tools & resources

- SAML
- Okta

- NIST Standards
- Security Best Practices
- XML Security Standards
- Compliance Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-saml-sso-with-okta`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-scim-provisioning-with-okta (Skill Module)

`implementing-scim-provisioning-with-okta\SKILL.md`

name: implementing-scim-provisioning-with-okta
description: Okta SCIM provisioning audit agent. Audits Okta SCIM provisioning configuration by querying the Okta API for provisioned applications, user assignments, group memberships, and deprovisioning status. Identifies.... Use when the user asks to implement scim provisioning with okta.
version: 1.0.0
tags:
source: implementing-scim-provisioning-with-okta

Implementing Scim Provisioning with Okta

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Okta, Core Operations, Protocol Standards, Compliance Standards

When to use

- The user wants to **implement** scim provisioning with okta.
- Capability: Okta SCIM provisioning audit agent. Audits Okta SCIM provisioning configuration by querying the Okta API for provisioned applications, user assignments, group memberships, and deprovisioning status. Identifies orphaned accounts, mismatched assignments, and provisioning failures.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, scim, provisioning, okta`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return Okta org URL and API token
2. Make authenticated Okta API call with pagination
3. List applications with SCIM provisioning enabled
4. Audit user assignments for a provisioning app
5. Audit group-based provisioning assignments
6. Check for deactivated Okta users that still have active app assignments
7. Print audit summary

Key concepts

- Defines the RESTful API for managing identity resources
- Specifies HTTP methods, headers, and response formats
- Mandates JSON as the data interchange format
- Requires TLS for all communications
- Defines User, Group, and EnterpriseUser schemas
- Specifies attribute types: string, boolean, decimal, integer, dateTime, reference, complex, binary
- Defines mutability: readOnly, readWrite, immutable, writeOnly
- Specifies attribute uniqueness: none, server, global

Tools & resources

- Okta
- Core Operations
- Protocol Standards
- Compliance Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_okta_config` — Return Okta org URL and API token.
- `okta_api` — Make authenticated Okta API call with pagination.
- `list_provisioning_apps` — List applications with SCIM provisioning enabled.
- `audit_app_assignments` — Audit user assignments for a provisioning app.
- `audit_group_assignments` — Audit group-based provisioning assignments.
- `check_deprovisioning` — Check for deactivated Okta users that still have active app assignments.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-scim-provisioning-with-okta`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-secret-scanning-with-gitleaks (Skill Module)

implementing-secret-scanning-with-gitleaks\SKILL.md

name: implementing-secret-scanning-with-gitleaks
description: Gitleaks secret scanning agent. Wraps the Gitleaks CLI to scan git repositories, directories, or specific commits for hardcoded secrets, API keys, tokens, and credentials. Parses JSON output into structured findings. Use when the user asks to implement secret scanning with gitleaks.
version: 1.0.0
tags:
source: implementing-secret-scanning-with-gitleaks

Implementing Secret Scanning with Gitleaks

Role: Security control implementation
Audience: security architects and engineers
Primary tools: OWASP SAMM, PCI DSS v4.0

When to use

- The user wants to **implement** secret scanning with gitleaks.
- Capability: Gitleaks secret scanning agent. Wraps the Gitleaks CLI to scan git repositories, directories, or specific commits for hardcoded secrets, API keys, tokens, and credentials. Parses JSON output into structured findings.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, secret, scanning, gitleaks`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Locate the gitleaks binary on the system
2. Run gitleaks detect on a repository or directory
3. Run gitleaks protect for pre-commit scanning
4. Parse gitleaks JSON output into structured findings
5. Print human-readable summary of secrets found

Key concepts

- Hardcoded credentials in source code enable unauthorized access
- Gitleaks detects API keys, passwords, tokens, and private keys before they reach repositories
- CWE-798: Use of Hard-coded Credentials is a direct mapping
- PW.1.1: Identify security requirements including credential management
- Secrets should never be stored in source code; use environment variables or secrets managers
- PS.1.1: Store code securely with access controls and secret scanning
- Implement pre-commit hooks to prevent secrets from entering version control
- Code signing and secret scanning ensure software releases do not contain embedded credentials

Tools & resources

- OWASP SAMM
- PCI DSS v4.0

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `find_gitleaks_binary` — Locate the gitleaks binary on the system.
- `run_detect` — Run gitleaks detect on a repository or directory.
- `run_protect` — Run gitleaks protect for pre-commit scanning.
- `parse_findings` — Parse gitleaks JSON output into structured findings.
- `format_summary` — Print human-readable summary of secrets found.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-secret-scanning-with-gitleaks`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-secrets-management-with-vault (Skill Module)

`implementing-secrets-management-with-vault\SKILL.md`

name: implementing-secrets-management-with-vault
description: HashiCorp Vault secrets management agent. Manages secrets lifecycle using the HashiCorp Vault KV v2 secrets engine via the hvac Python library. Supports reading, writing, listing, deleting secrets, and auditing.... Use when the user asks to implement secrets management with vault.
version: 1.0.0
tags:
source: implementing-secrets-management-with-vault

Implementing Secrets Management with Vault

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core API, System Backend, Transit Secrets Engine, Error Handling

When to use

- The user wants to **implement** secrets management with vault.
- Capability: HashiCorp Vault secrets management agent. Manages secrets lifecycle using the HashiCorp Vault KV v2 secrets engine via the hvac Python library. Supports reading, writing, listing, deleting secrets, and auditing secret access patterns.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, secrets, management, vault`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create and authenticate a Vault client
2. Write or update a secret in KV v2
3. Read a secret from KV v2
4. List secret paths under a given prefix
5. Delete or destroy a secret
6. Read secret metadata including version history
7. Audit all secrets under a path, reporting metadata and age

Tools & resources

- Core API
- System Backend
- Transit Secrets Engine
- Error Handling

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_vault_client` — Create and authenticate a Vault client.
- `write_secret` — Write or update a secret in KV v2.
- `read_secret` — Read a secret from KV v2.
- `list_secrets` — List secret paths under a given prefix.
- `delete_secret` — Delete or destroy a secret.
- `read_metadata` — Read secret metadata including version history.
- `audit_secrets` — Audit all secrets under a path, reporting metadata and age.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-secrets-management-with-vault`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-secrets-scanning-in-ci-cd (Skill Module)

implementing-secrets-scanning-in-ci-cd\SKILL.md

name: implementing-secrets-scanning-in-ci-cd
description: Secrets scanning CI/CD gate using gitleaks and trufflehog. Use when the user asks to implement secrets scanning in ci cd.
version: 1.0.0
tags:
source: implementing-secrets-scanning-in-ci-cd

Implementing Secrets Scanning in Ci Cd

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Gitleaks CLI, TruffleHog CLI, GitHub Secret Scanning API

When to use

- The user wants to **implement** secrets scanning in ci cd.
- Capability: Secrets scanning CI/CD gate using gitleaks and trufflehog.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, secrets, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for secrets scanning in ci cd
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Gitleaks CLI
- TruffleHog CLI
- GitHub Secret Scanning API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_gitleaks`
- `run_trufflehog`
- `evaluate_gate`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-secrets-scanning-in-ci-cd`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-security-chaos-engineering (Skill Module)

implementing-security-chaos-engineering\SKILL.md

name: implementing-security-chaos-engineering
description: Agent for security chaos engineering experiments. Use when the user asks to implement security chaos engineering.
version: 1.0.0
tags:
source: implementing-security-chaos-engineering

Implementing Security Chaos Engineering

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Experiment Catalog

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** security chaos engineering.
- Capability: Agent for security chaos engineering experiments.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, chaos, engineering`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Experiment: Open a security group and verify detection
2. Experiment: Create IAM admin user and verify detection
3. Experiment: Stop CloudTrail and verify detection
4. List available experiments without executing them

Tools & resources

- Experiment Catalog

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `experiment_open_security_group` — Experiment: Open a security group and verify detection.
- `experiment_create_admin_user` — Experiment: Create IAM admin user and verify detection.
- `experiment_stop_cloudtrail` — Experiment: Stop CloudTrail and verify detection.
- `dry_run_experiments` — List available experiments without executing them.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-security-chaos-engineering`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-security-information-sharing-with-stix2 (Skill Module)

`implementing-security-information-sharing-with-stix2\SKILL.md`

name: implementing-security-information-sharing-with-stix2
description: STIX 2.1 threat intelligence sharing agent. Creates, validates, and exports STIX 2.1 objects including indicators, malware, campaigns, and relationships using the stix2 Python library. Use when the user asks to implement security information sharing with stix2.
version: 1.0.0
tags:
source: implementing-security-information-sharing-with-stix2

Implementing Security Information Sharing with Stix2

Role: Security control implementation
Audience: security architects and engineers
Primary tools: stix2 Python Library, STIX 2.1 Object Types, TAXII 2.1 Publishing, TLP Marking Definitions

When to use

- The user wants to **implement** security information sharing with stix2.
- Capability: STIX 2.1 threat intelligence sharing agent. Creates, validates, and exports STIX 2.1 objects including indicators, malware, campaigns, and relationships using the stix2 Python library.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, information, sharing, stix2`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create a STIX 2.1 Indicator object
2. Create a STIX 2.1 Malware object
3. Create a STIX 2.1 Campaign object
4. Create a STIX 2.1 Relationship
5. Build a STIX 2.1 Bundle with all objects and relationships
6. Publish STIX bundle to TAXII 2.1 collection
7. Validate a STIX 2.1 bundle

Tools & resources

- stix2 Python Library
- STIX 2.1 Object Types
- TAXII 2.1 Publishing
- TLP Marking Definitions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `create_indicator` — Create a STIX 2.1 Indicator object.
- `create_malware` — Create a STIX 2.1 Malware object.
- `create_campaign` — Create a STIX 2.1 Campaign object.
- `create_relationship` — Create a STIX 2.1 Relationship.
- `build_threat_report` — Build a STIX 2.1 Bundle with all objects and relationships.
- `publish_to_taxii` — Publish STIX bundle to TAXII 2.1 collection.
- `validate_bundle` — Validate a STIX 2.1 bundle.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-security-information-sharing-with-stix2`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-security-monitoring-with-datadog (Skill Module)

implementing-security-monitoring-with-datadog\SKILL.md

name: implementing-security-monitoring-with-datadog
description: Agent for implementing security monitoring with Datadog. Configures Cloud SIEM detection rules, queries security signals, manages log pipelines, and audits security monitoring coverage using the Datadog API. Use when the user asks to implement security monitoring with datadog.
version: 1.0.0
tags:
source: implementing-security-monitoring-with-datadog

Implementing Security Monitoring with Datadog

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Datadog Security Monitoring API, Detection Rule Types, Signal Severity Levels

When to use

- The user wants to **implement** security monitoring with datadog.
- Capability: Agent for implementing security monitoring with Datadog. Configures Cloud SIEM detection rules, queries security signals, manages log pipelines, and audits security monitoring coverage using the Datadog API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, security, monitoring, datadog`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for security monitoring with datadog
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Datadog Security Monitoring API
- Detection Rule Types
- Signal Severity Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-security-monitoring-with-datadog`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-semgrep-for-custom-sast-rules (Skill Module)

`implementing-semgrep-for-custom-sast-rules\SKILL.md`

name: implementing-semgrep-for-custom-sast-rules
description: Semgrep SAST scanning agent. Wraps the Semgrep CLI to perform static application security testing using built-in rulesets and custom rules. Parses JSON output to produce structured vulnerability findings with.... Use when the user asks to implement semgrep for custom sast rules.
version: 1.0.0
tags:
source: implementing-semgrep-for-custom-sast-rules

Implementing Semgrep for Custom Sast Rules

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Semgrep Registry Rule Packs, CWE Coverage, Compliance

When to use

- The user wants to **implement** semgrep for custom sast rules.
- Capability: Semgrep SAST scanning agent. Wraps the Semgrep CLI to perform static application security testing using built-in rulesets and custom rules. Parses JSON output to produce structured vulnerability findings with severity, CWE, and OWASP mappings.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, semgrep, custom, sast, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Locate the semgrep binary on the system
2. Run semgrep scan and return JSON results
3. Parse semgrep JSON output into structured findings
4. Print human-readable scan summary

Key concepts

- SA-11: Developer Security Testing
- SA-15: Development Process, Standards, and Tools
- PCI DSS v4.0 Req 6.3.2: Secure development with automated tools
- SOC 2 CC8.1: Change management with code scanning

Tools & resources

- Semgrep Registry Rule Packs
- CWE Coverage
- Compliance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `find_semgrep_binary` — Locate the semgrep binary on the system.
- `run_scan` — Run semgrep scan and return JSON results.
- `parse_findings` — Parse semgrep JSON output into structured findings.
- `format_summary` — Print human-readable scan summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-semgrep-for-custom-sast-rules`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-siem-correlation-rules-for-apt (Skill Module)

implementing-siem-correlation-rules-for-apt\SKILL.md

name: implementing-siem-correlation-rules-for-apt
description: SIEM Correlation Rules Agent - Builds and deploys multi-event APT detection rules via Splunk and Sigma. Use when the user asks to implement siem correlation rules for apt.
version: 1.0.0
tags:
source: implementing-siem-correlation-rules-for-apt

Implementing SIEM Correlation Rules for APT

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Splunk REST API

When to use

- The user wants to **implement** siem correlation rules for apt.
- Capability: SIEM Correlation Rules Agent - Builds and deploys multi-event APT detection rules via Splunk and Sigma.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, siem, correlation, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Authenticate to Splunk and return session headers
2. Deploy a correlation search to Splunk ES
3. Generate a Sigma-format YAML rule from a correlation definition
4. Audit existing Splunk saved searches for coverage gaps
5. Execute a correlation search and return matching events
6. Generate correlation rules deployment report

Tools & resources

- Splunk REST API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `authenticate_splunk` — Authenticate to Splunk and return session headers.
- `deploy_correlation_search` — Deploy a correlation search to Splunk ES.
- `generate_sigma_rule` — Generate a Sigma-format YAML rule from a correlation definition.
- `audit_existing_searches` — Audit existing Splunk saved searches for coverage gaps.
- `run_test_search` — Execute a correlation search and return matching events.
- `generate_report` — Generate correlation rules deployment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-siem-correlation-rules-for-apt`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-siem-use-case-tuning (Skill Module)

implementing-siem-use-case-tuning\SKILL.md

name: implementing-siem-use-case-tuning
description: SIEM use case tuning agent - analyzes alert data to reduce false positives and optimize detection rules. Use when the user asks to implement siem use case tuning.
version: 1.0.0
tags:
source: implementing-siem-use-case-tuning

Implementing SIEM Use Case Tuning

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Elastic Detection Rule Tuning, Alert Tuning Metrics

When to use

- The user wants to **implement** siem use case tuning.
- Capability: SIEM use case tuning agent - analyzes alert data to reduce false positives and optimize detection rules.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, siem, case, tuning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load alert/notable event export (CSV with columns: rule_name, timestamp, disposition, source, user)
2. Calculate per-rule alert volume, FP rate, and disposition breakdown
3. Identify source/user pairs that consistently trigger FPs for a given rule
4. Compute statistical threshold for a rule based on hourly alert distribution
5. Generate comprehensive tuning report with recommendations

Tools & resources

- Elastic Detection Rule Tuning
- Alert Tuning Metrics

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_alert_data` — Load alert/notable event export (CSV with columns: rule_name, timestamp, disposition, source, user).
- `calculate_rule_metrics` — Calculate per-rule alert volume, FP rate, and disposition breakdown.
- `identify_whitelist_candidates` — Identify source/user pairs that consistently trigger FPs for a given rule.
- `compute_threshold_recommendation` — Compute statistical threshold for a rule based on hourly alert distribution.
- `generate_tuning_report` — Generate comprehensive tuning report with recommendations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-siem-use-case-tuning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-siem-use-cases-for-detection (Skill Module)

implementing-siem-use-cases-for-detection\SKILL.md

name: implementing-siem-use-cases-for-detection
description: SIEM detection use case management agent with ATT&CK coverage mapping. Use when the user asks to implement siem use cases for detection.
version: 1.0.0
tags:
source: implementing-siem-use-cases-for-detection

Implementing SIEM Use Cases for Detection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Use Case Lifecycle, Detection Quality Metrics, External References

When to use

- The user wants to **implement** siem use cases for detection.
- Capability: SIEM detection use case management agent with ATT&CK coverage mapping.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, siem, cases, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate ATT&CK coverage percentage
2. Map all use case templates to ATT&CK techniques and tactics
3. Validate that required data sources are available for a use case
4. Generate a Sigma detection rule for a use case
5. Generate SIEM detection coverage report

Tools & resources

- Use Case Lifecycle
- Detection Quality Metrics
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_attack_coverage` — Calculate ATT&CK coverage percentage.
- `map_use_cases_to_attack` — Map all use case templates to ATT&CK techniques and tactics.
- `validate_use_case_data_sources` — Validate that required data sources are available for a use case.
- `generate_sigma_rule` — Generate a Sigma detection rule for a use case.
- `run_detection_coverage_report` — Generate SIEM detection coverage report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-siem-use-cases-for-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-sigstore-for-software-signing (Skill Module)

implementing-sigstore-for-software-signing\SKILL.md

name: implementing-sigstore-for-software-signing
description: Sigstore Software Signing Agent - Automates cosign keyless signing, Rekor transparency log verification, and Fulcio certificate inspection for container images and software artifacts. Use when the user asks to implement sigstore for software signing.
version: 1.0.0
tags:
source: implementing-sigstore-for-software-signing

Implementing Sigstore for Software Signing

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Arguments, Key Functions, Common OIDC Issuers

When to use

- The user wants to **implement** sigstore for software signing.
- Capability: Sigstore Software Signing Agent - Automates cosign keyless signing, Rekor transparency log verification, and Fulcio certificate inspection for container images and software artifacts.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, sigstore, software, signing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute SHA-256 hash of a file
2. Execute a cosign CLI command and return the result
3. Verify cosign CLI is available and return version info
4. Sign a file blob using cosign keyless signing with Fulcio and Rekor. This triggers an OIDC authentication flow. In CI, set SIGSTORE_ID_TOKEN environment variable to provide the identity token non-interactively
5. Verify a signed blob against expected identity and OIDC issuer
6. Sign a container image using cosign keyless signing. The image_uri should include the digest (e.g., registry/image@sha256:abc...). Signing by tag instead of digest is unreliable because tags are mutable
7. Verify a container image signature against expected identity and issuer
8. Search the Rekor transparency log for entries matching an artifact hash. Queries the Rekor REST API /api/v1/index/retrieve endpoint

Tools & resources

- Arguments
- Key Functions
- Common OIDC Issuers

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_sha256` — Compute SHA-256 hash of a file.
- `run_cosign` — Execute a cosign CLI command and return the result.
- `check_cosign_installed` — Verify cosign CLI is available and return version info.
- `sign_blob_keyless` — Sign a file blob using cosign keyless signing with Fulcio and Rekor. This triggers an OIDC authentication flow. In CI, set SIGSTORE_ID_TOKEN environment variable to provide the identity token non-interactively.
- `verify_blob_keyless` — Verify a signed blob against expected identity and OIDC issuer.
- `sign_container_keyless` — Sign a container image using cosign keyless signing. The image_uri should include the digest (e.g., registry/image@sha256:abc...). Signing by tag instead of digest is unreliable because tags are mutable.
- `verify_container_keyless` — Verify a container image signature against expected identity and issuer.
- `search_rekor_by_hash` — Search the Rekor transparency log for entries matching an artifact hash. Queries the Rekor REST API /api/v1/index/retrieve endpoint.
- `search_rekor_by_email` — Search the Rekor transparency log for entries matching an email identity.
- `get_rekor_entry` — Retrieve a specific entry from the Rekor transparency log by UUID.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-sigstore-for-software-signing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-soar-automation-with-phantom (Skill Module)

`implementing-soar-automation-with-phantom\SKILL.md`

name: implementing-soar-automation-with-phantom
description: Splunk SOAR (Phantom) automation agent for playbook management. Use when the user asks to implement soar automation with phantom.
version: 1.0.0
tags:
source: implementing-soar-automation-with-phantom

Implementing SOAR Automation with Phantom

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Splunk SOAR REST API, Common App Actions, Playbook Types, External References

When to use

- The user wants to **implement** soar automation with phantom.
- Capability: Splunk SOAR (Phantom) automation agent for playbook management.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, soar, automation, phantom`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate phishing response playbook configuration
2. Generate malware containment playbook configuration
3. Run SOAR platform audit

Tools & resources

- Splunk SOAR REST API
- Common App Actions
- Playbook Types
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `create_phishing_response_playbook_data` — Generate phishing response playbook configuration.
- `create_malware_containment_playbook_data` — Generate malware containment playbook configuration.
- `run_soar_audit` — Run SOAR platform audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-soar-automation-with-phantom`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-soar-playbook-for-phishing (Skill Module)

implementing-soar-playbook-for-phishing\SKILL.md

name: implementing-soar-playbook-for-phishing
description: Splunk SOAR phishing playbook automation via REST API. Use when the user asks to implement soar playbook for phishing.
version: 1.0.0
tags:
source: implementing-soar-playbook-for-phishing

Implementing SOAR Playbook for Phishing

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Splunk SOAR REST API, Common Phishing Playbook Actions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** soar playbook for phishing.
- Capability: Splunk SOAR phishing playbook automation via REST API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, soar, playbook, phishing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for soar playbook for phishing
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- Splunk SOAR REST API
- Common Phishing Playbook Actions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_email_file`
- `run_phishing_workflow`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-soar-playbook-for-phishing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-soar-playbook-with-palo-alto-xsoar (Skill Module)

`implementing-soar-playbook-with-palo-alto-xsoar\SKILL.md`

name: implementing-soar-playbook-with-palo-alto-xsoar
description: Cortex XSOAR playbook management agent. Interfaces with the Cortex XSOAR (Demisto) API to manage and audit security playbooks, automation scripts, incidents, and integrations. Supports listing playbooks, checking.... Use when the user asks to implement soar playbook with palo alto xsoar.
version: 1.0.0
tags:
source: implementing-soar-playbook-with-palo-alto-xsoar

Implementing SOAR Playbook with Palo Alto Xsoar

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Operations, SOAR Industry Standards, XSOAR Architecture Standards

When to use

- The user wants to **implement** soar playbook with palo alto xsoar.
- Capability: Cortex XSOAR playbook management agent. Interfaces with the Cortex XSOAR (Demisto) API to manage and audit security playbooks, automation scripts, incidents, and integrations. Supports listing playbooks, checking incident statistics, and verifying integration health.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, soar, playbook, palo, alto, xsoar`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return XSOAR server URL and API key
2. Make authenticated XSOAR API call
3. List all playbooks
4. Get incident statistics
5. List configured integrations and their health
6. Audit playbooks for common issues
7. Print XSOAR audit summary

Key concepts

- Security Orchestration and Automation (SOA)
- Security Incident Response Platforms (SIRP)
- Threat Intelligence Platforms (TIP)

Tools & resources

- Core Operations
- SOAR Industry Standards
- XSOAR Architecture Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_xsoar_config` — Return XSOAR server URL and API key.
- `xsoar_api` — Make authenticated XSOAR API call.
- `list_playbooks` — List all playbooks.
- `get_incident_stats` — Get incident statistics.
- `list_integrations` — List configured integrations and their health.
- `audit_playbook_health` — Audit playbooks for common issues.
- `format_summary` — Print XSOAR audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-soar-playbook-with-palo-alto-xsoar`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-stix-taxii-feed-integration (Skill Module)

implementing-stix-taxii-feed-integration\SKILL.md

name: implementing-stix-taxii-feed-integration
description: STIX/TAXII threat intelligence feed integration agent. Connects to TAXII 2.0/2.1 servers to discover and consume threat intelligence feeds in STIX format. Extracts indicators of compromise (IOCs), threat actors,.... Use when the user asks to implement stix taxii feed integration.
version: 1.0.0
tags:
source: implementing-stix-taxii-feed-integration

Implementing STIX Taxii Feed Integration

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Operations, Public TAXII Feeds

When to use

- The user wants to **implement** stix taxii feed integration.
- Capability: STIX/TAXII threat intelligence feed integration agent. Connects to TAXII 2.0/2.1 servers to discover and consume threat intelligence feeds in STIX format. Extracts indicators of compromise (IOCs), threat actors, malware families, and attack patterns from STIX bundles.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, stix, taxii, feed, integration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Connect to a TAXII server and return Server object
2. Discover available collections on the TAXII server
3. Fetch STIX objects from a TAXII collection
4. Extract indicators of compromise from STIX objects
5. Extract threat actor information from STIX objects
6. Extract malware family info from STIX objects
7. Extract MITRE ATT&CK patterns from STIX objects
8. Group and count STIX objects by type

Key concepts

- STIX Bundle: Top-level container for STIX objects (type: "bundle")
- STIX ID Format: `type--uuid` (e.g., `indicator--a1b2c3d4-...`)
- Versioning: Objects use `modified` timestamp for version tracking
- Confidence: 0-100 scale for reliability assessment
- Content-Type: `application/stix+json;version=2.1`
- Accept: `application/taxii+json;version=2.1`

Tools & resources

- Core Operations
- Public TAXII Feeds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `connect_taxii_server` — Connect to a TAXII server and return Server object.
- `discover_collections` — Discover available collections on the TAXII server.
- `fetch_collection_objects` — Fetch STIX objects from a TAXII collection.
- `extract_indicators` — Extract indicators of compromise from STIX objects.
- `extract_threat_actors` — Extract threat actor information from STIX objects.
- `extract_malware` — Extract malware family info from STIX objects.
- `extract_attack_patterns` — Extract MITRE ATT&CK patterns from STIX objects.
- `summarize_stix_objects` — Group and count STIX objects by type.
- `format_summary` — Print human-readable summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-stix-taxii-feed-integration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-supply-chain-security-with-in-toto (Skill Module)

implementing-supply-chain-security-with-in-toto\SKILL.md

name: implementing-supply-chain-security-with-in-toto
description: in-toto supply chain security agent. Implements software supply chain verification using the in-toto framework. Creates and verifies supply chain layouts, generates link metadata for build steps, and validates that.... Use when the user asks to implement supply chain security with in toto.
version: 1.0.0
tags:
source: implementing-supply-chain-security-with-in-toto

Implementing Supply Chain Security with in Toto

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Python API, Key Concepts, Industry Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** supply chain security with in toto.
- Capability: in-toto supply chain security agent. Implements software supply chain verification using the in-toto framework. Creates and verifies supply chain layouts, generates link metadata for build steps, and validates that all steps were performed by authorized functionaries with correct materials and products.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, supply, chain, security, toto`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Locate in-toto CLI tools
2. Generate a supply chain layout template
3. Execute a supply chain step and record link metadata
4. Verify the supply chain against the layout
5. Audit existing link metadata files
6. Print supply chain audit summary

Key concepts

- in-toto provides the attestation framework that SLSA builds upon
- SLSA provenance attestations use in-toto attestation format
- Graduated to v1.0 specification with clear level requirements
- PO.1.1: Define and document security requirements for software
- PS.1.1: Verify third-party software components
- PW.4.1: Review and analyze source code for security vulnerabilities
- Section 4: Build system integrity verification
- Section 5: Attestation-based supply chain verification

Tools & resources

- Python API
- Key Concepts

- Industry Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `find_intoto_binary` — Locate in-toto CLI tools.
- `create_layout_template` — Generate a supply chain layout template.
- `run_step` — Execute a supply chain step and record link metadata.
- `verify_layout` — Verify the supply chain against the layout.
- `audit_existing_links` — Audit existing link metadata files.
- `format_summary` — Print supply chain audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-supply-chain-security-with-in-toto`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-syslog-centralization-with-rsyslog (Skill Module)

implementing-syslog-centralization-with-rsyslog\SKILL.md

name: implementing-syslog-centralization-with-rsyslog
description: Rsyslog Centralization Agent - Generates and deploys TLS-secured rsyslog configurations. Use when the user asks to implement syslog centralization with rsyslog.
version: 1.0.0
tags:
source: implementing-syslog-centralization-with-rsyslog

Implementing Syslog Centralization with Rsyslog

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Rsyslog Server Configuration Directives, Rsyslog Client Forwarding, Paramiko SSH Deployment, OpenSSL Certificate Generation

When to use

- The user wants to **implement** syslog centralization with rsyslog.
- Capability: Rsyslog Centralization Agent - Generates and deploys TLS-secured rsyslog configurations.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, syslog, centralization, rsyslog`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate rsyslog server configuration with TLS
2. Generate rsyslog client configuration with TLS forwarding
3. Generate CA, server, and client TLS certificates using OpenSSL
4. Deploy rsyslog configuration to a remote host via SSH
5. Validate TLS connectivity to the rsyslog server
6. Generate syslog centralization deployment report

Tools & resources

- Rsyslog Server Configuration Directives
- Rsyslog Client Forwarding
- Paramiko SSH Deployment
- OpenSSL Certificate Generation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_server_config` — Generate rsyslog server configuration with TLS.
- `generate_client_config` — Generate rsyslog client configuration with TLS forwarding.
- `generate_tls_certificates` — Generate CA, server, and client TLS certificates using OpenSSL.
- `deploy_config_ssh` — Deploy rsyslog configuration to a remote host via SSH.
- `validate_tls_connection` — Validate TLS connectivity to the rsyslog server.
- `generate_report` — Generate syslog centralization deployment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-syslog-centralization-with-rsyslog`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-taxii-server-with-opentaxii (Skill Module)

`implementing-taxii-server-with-opentaxii\SKILL.md`

name: implementing-taxii-server-with-opentaxii
description: OpenTAXII server configuration and health audit agent. Audits an OpenTAXII server instance by checking service discovery, collection availability, content block statistics, and API health. Supports both TAXII 1.1 and.... Use when the user asks to implement taxii server with opentaxii.
version: 1.0.0
tags:
source: implementing-taxii-server-with-opentaxii

Implementing Taxii Server with Opentaxii

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Server Configuration, Server Administration, Client Operations, Health Check

When to use

- The user wants to **implement** taxii server with opentaxii.
- Capability: OpenTAXII server configuration and health audit agent. Audits an OpenTAXII server instance by checking service discovery, collection availability, content block statistics, and API health. Supports both TAXII 1.1 and 2.0/2.1 endpoints.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, taxii, server, opentaxii`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check TAXII 1.1 discovery service
2. Check TAXII 2.0/2.1 discovery and collections
3. Check basic server health
4. Print audit summary

Tools & resources

- Server Configuration
- Server Administration
- Client Operations
- Health Check

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_taxii1_discovery` — Check TAXII 1.1 discovery service.
- `check_taxii2_discovery` — Check TAXII 2.0/2.1 discovery and collections.
- `check_server_health` — Check basic server health.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-taxii-server-with-opentaxii`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-threat-intelligence-lifecycle-management (Skill Module)

`implementing-threat-intelligence-lifecycle-management\SKILL.md`

name: implementing-threat-intelligence-lifecycle-management
description: Threat intelligence lifecycle management agent. Manages the threat intelligence lifecycle: collection from feeds, processing/normalization of IOCs, analysis/enrichment via VirusTotal and AbuseIPDB, dissemination to.... Use when the user asks to implement threat intelligence lifecycle management.
version: 1.0.0
tags:
source: implementing-threat-intelligence-lifecycle-management

Implementing Threat Intelligence Lifecycle Management

Role: Security control implementation
Audience: security architects and engineers
Primary tools: MISP API Operations, STIX 2.1 Intelligence Objects, Intelligence Lifecycle Phases

When to use

- The user wants to **implement** threat intelligence lifecycle management.
- Capability: Threat intelligence lifecycle management agent. Manages the threat intelligence lifecycle: collection from feeds, processing/normalization of IOCs, analysis/enrichment via VirusTotal and AbuseIPDB, dissemination to SIEM/firewalls, and tracking of IOC aging and confidence scoring.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, threat, intelligence, lifecycle, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract IOCs from unstructured text
2. Load IOCs from a file (JSON, CSV, or plain text)
3. Normalize IOC into standard format
4. Enrich IOC via VirusTotal API v3
5. Calculate confidence score for an IOC (0-100)
6. Print lifecycle report

Tools & resources

- MISP API Operations
- STIX 2.1 Intelligence Objects
- Intelligence Lifecycle Phases

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_iocs` — Extract IOCs from unstructured text.
- `load_ioc_feed` — Load IOCs from a file (JSON, CSV, or plain text).
- `normalize_ioc` — Normalize IOC into standard format.
- `enrich_ioc_virustotal` — Enrich IOC via VirusTotal API v3.
- `calculate_confidence` — Calculate confidence score for an IOC (0-100).
- `format_summary` — Print lifecycle report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-threat-intelligence-lifecycle-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-threat-modeling-with-mitre-attack (Skill Module)

`implementing-threat-modeling-with-mitre-attack\SKILL.md`

name: implementing-threat-modeling-with-mitre-attack
description: Threat modeling agent using MITRE ATT&CK framework with attackcti. Use when the user asks to implement threat modeling with mitre attack.
version: 1.0.0
tags:
source: implementing-threat-modeling-with-mitre-attack

Implementing Threat Modeling with MITRE ATT&CK

Role: Security control implementation
Audience: security architects and engineers
Primary tools: MITRE ATT&CK, Threat Modeling Workflow, Priority Scoring, External References

When to use

- The user wants to **implement** threat modeling with mitre attack.
- Capability: Threat modeling agent using MITRE ATT&CK framework with attackcti.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, threat, modeling, mitre, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Get all ATT&CK techniques used by a specific threat group
2. Build a threat profile for an industry based on relevant threat actors
3. Assess detection coverage gaps against threat profile
4. Generate ATT&CK Navigator layer JSON for visualization
5. Prioritize defensive investments based on threat model
6. Run full threat modeling exercise for an industry

Tools & resources

- MITRE ATT&CK
- Threat Modeling Workflow
- Priority Scoring
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_group_techniques` — Get all ATT&CK techniques used by a specific threat group.
- `build_threat_profile` — Build a threat profile for an industry based on relevant threat actors.
- `assess_detection_coverage` — Assess detection coverage gaps against threat profile.
- `generate_navigator_layer` — Generate ATT&CK Navigator layer JSON for visualization.
- `prioritize_defenses` — Prioritize defensive investments based on threat model.
- `run_threat_model` — Run full threat modeling exercise for an industry.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-threat-modeling-with-mitre-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-ticketing-system-for-incidents (Skill Module)

implementing-ticketing-system-for-incidents\SKILL.md

name: implementing-ticketing-system-for-incidents
description: Incident ticketing system agent supporting ServiceNow and TheHive. Use when the user asks to implement ticketing system for incidents.
version: 1.0.0
tags:
source: implementing-ticketing-system-for-incidents

Implementing Ticketing System for Incidents

Role: Security control implementation
Audience: security architects and engineers
Primary tools: ServiceNow REST API, SLA Target Reference, External References

When to use

- The user wants to **implement** ticketing system for incidents.
- Capability: Incident ticketing system agent supporting ServiceNow and TheHive.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, ticketing, system, incidents`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Capture requirements, risks, and success metrics
2. Design the control architecture for ticketing system for incidents
3. Implement configuration, policies, and integrations
4. Test effectiveness and failure modes
5. Hand off operational docs and monitoring

Tools & resources

- ServiceNow REST API
- SLA Target Reference
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `calculate_sla_metrics` — Calculate SLA compliance metrics from incident data.
- `run_ticketing_audit` — Run ticketing system audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-ticketing-system-for-incidents`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-usb-device-control-policy (Skill Module)

implementing-usb-device-control-policy\SKILL.md

name: implementing-usb-device-control-policy
description: USB device control policy audit agent. Audits USB device control policies on Linux and Windows systems by checking udev rules, USBGuard configuration, Windows Group Policy settings, and connected device history..... Use when the user asks to implement usb device control policy.
version: 1.0.0
tags:
source: implementing-usb-device-control-policy

Implementing USB Device Control Policy

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Windows USB Device Audit, Linux USB Device Audit, Device Whitelist Management, Primary Standards, Supporting References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** usb device control policy.
- Capability: USB device control policy audit agent. Audits USB device control policies on Linux and Windows systems by checking udev rules, USBGuard configuration, Windows Group Policy settings, and connected device history. Reports unauthorized or unwhitelisted USB devices.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, device, control, policy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Audit USBGuard configuration and rules on Linux
2. Check for USB-related udev rules on Linux
3. List currently connected USB devices
4. Print audit summary

Key concepts

- NIST SP 800-53 MP-7: Media Use - restricting removable media types and usage
- PCI DSS 4.0 Req 3.4.2: Restrict removable electronic media for cardholder data
- CIS Control 10.4: Configure device control to prevent removable media autorun
- ISO 27001 A.8.3.1: Management of removable media
- CrowdStrike Device Control: Falcon Console documentation

Tools & resources

- Windows USB Device Audit
- Linux USB Device Audit
- Device Whitelist Management
- Primary Standards
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `audit_linux_usbguard` — Audit USBGuard configuration and rules on Linux.
- `audit_linux_udev` — Check for USB-related udev rules on Linux.
- `list_connected_usb_devices` — List currently connected USB devices.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-usb-device-control-policy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-velociraptor-for-ir-collection (Skill Module)

implementing-velociraptor-for-ir-collection\SKILL.md

name: implementing-velociraptor-for-ir-collection
description: Velociraptor incident response collection agent. Interfaces with the Velociraptor DFIR platform API to schedule artifact collections on endpoints, retrieve results, and generate IR reports. Supports common forensic.... Use when the user asks to implement velociraptor for ir collection.
version: 1.0.0
tags:
source: implementing-velociraptor-for-ir-collection

Implementing Velociraptor for IR Collection

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Velociraptor, gRPC API, Key VQL Functions, Built-in Artifact Categories, NIST SP 800-86, ForensicArtifacts Standard

When to use

- The user wants to **implement** velociraptor for ir collection.
- Capability: Velociraptor incident response collection agent. Interfaces with the Velociraptor DFIR platform API to schedule artifact collections on endpoints, retrieve results, and generate IR reports. Supports common forensic artifacts including process listings, network connections, autoruns, event logs, and file system evidence.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, velociraptor, collection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return Velociraptor API configuration
2. Make an authenticated API call to Velociraptor
3. Search for Velociraptor clients by hostname, label, or client ID
4. Schedule an artifact collection on a specific client
5. Check the status of a collection flow
6. Poll until a collection flow completes
7. Retrieve collected artifact results
8. Print collection summary

Key concepts

- Evidence collection procedures for digital investigations
- Chain of custody requirements for forensic data
- Volatile and non-volatile evidence prioritization
- Cross-platform artifact repository
- Community-contributed artifact definitions
- FOR500: Windows Forensic Analysis artifact prioritization
- FOR508: Advanced Incident Response collection methodology
- Evidence acquisition order of volatility

Tools & resources

- Velociraptor
- gRPC API
- Key VQL Functions
- Built-in Artifact Categories

- NIST SP 800-86
- ForensicArtifacts Standard
- SANS DFIR Collection Standards
- Sigma Detection Standard

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_velo_config` — Return Velociraptor API configuration.
- `velo_api_call` — Make an authenticated API call to Velociraptor.
- `search_clients` — Search for Velociraptor clients by hostname, label, or client ID.
- `collect_artifact` — Schedule an artifact collection on a specific client.
- `get_flow_status` — Check the status of a collection flow.
- `wait_for_collection` — Poll until a collection flow completes.
- `get_flow_results` — Retrieve collected artifact results.
- `format_summary` — Print collection summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-velociraptor-for-ir-collection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-vulnerability-management-with-greenbone (Skill Module)

implementing-vulnerability-management-with-greenbone\SKILL.md

name: implementing-vulnerability-management-with-greenbone
description: Greenbone/OpenVAS Vulnerability Management agent - creates scan targets, executes scans, and parses reports via python-gvm GMP protocol. Use when the user asks to implement vulnerability management with greenbone.
version: 1.0.0
tags:
source: implementing-vulnerability-management-with-greenbone

Implementing Vulnerability Management with Greenbone

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core GMP API Methods, Default Resource IDs

When to use

- The user wants to **implement** vulnerability management with greenbone.
- Capability: Greenbone/OpenVAS Vulnerability Management agent - creates scan targets, executes scans, and parses reports via python-gvm GMP protocol
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, vulnerability, management, greenbone`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to GVM daemon and authenticate via GMP
2. Create a scan target. Default port list is 'All IANA assigned TCP'
3. Create scan task and start it. Default config is 'Full and fast', default scanner is 'OpenVAS Default'
4. Parse GMP report XML into structured findings
5. Analyze pre-exported GVM report data (JSON format)

Tools & resources

- Core GMP API Methods
- Default Resource IDs

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_data`
- `connect_gvm` — Connect to GVM daemon and authenticate via GMP.
- `create_target` — Create a scan target. Default port list is 'All IANA assigned TCP'.
- `create_and_start_task` — Create scan task and start it. Default config is 'Full and fast', default scanner is 'OpenVAS Default'.
- `parse_report_xml` — Parse GMP report XML into structured findings.
- `analyze_offline_report` — Analyze pre-exported GVM report data (JSON format).
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-vulnerability-management-with-greenbone`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-vulnerability-remediation-sla (Skill Module)

implementing-vulnerability-remediation-sla\SKILL.md

name: implementing-vulnerability-remediation-sla
description: Vulnerability remediation SLA tracking agent. Tracks vulnerability remediation against defined SLA targets based on severity. Ingests vulnerability data from scanners (JSON/CSV format), calculates SLA compliance,.... Use when the user asks to implement vulnerability remediation sla.
version: 1.0.0
tags:
source: implementing-vulnerability-remediation-sla

Implementing Vulnerability Remediation Sla

Role: Security control implementation
Audience: security architects and engineers
Primary tools: SLA Tiers, Core Operations, Regulatory SLA Requirements, Industry Benchmarks

When to use

- The user wants to **implement** vulnerability remediation sla.
- Capability: Vulnerability remediation SLA tracking agent. Tracks vulnerability remediation against defined SLA targets based on severity. Ingests vulnerability data from scanners (JSON/CSV format), calculates SLA compliance, identifies overdue items, and generates remediation priority reports.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, vulnerability, remediation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load vulnerabilities from a JSON or CSV file
2. Normalize vulnerability fields from various scanner formats
3. Calculate SLA compliance for each vulnerability
4. Generate SLA compliance metrics
5. Print SLA tracking summary

Key concepts

- PCI DSS v4.0 Req 6.3.3: Address vulnerabilities by risk ranking (critical/high within 30 days)
- CISA BOD 22-01: Federal agencies must remediate KEV within specified timeframes
- NIST SP 800-40 Rev 4: Enterprise Patch Management Planning
- SOX: Timely remediation of IT control deficiencies
- HIPAA: Reasonable and appropriate security measures including patching

Tools & resources

- SLA Tiers
- Core Operations
- Regulatory SLA Requirements
- Industry Benchmarks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_vulnerabilities` — Load vulnerabilities from a JSON or CSV file.
- `normalize_vulnerability` — Normalize vulnerability fields from various scanner formats.
- `calculate_sla_status` — Calculate SLA compliance for each vulnerability.
- `generate_metrics` — Generate SLA compliance metrics.
- `format_summary` — Print SLA tracking summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-vulnerability-remediation-sla`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-vulnerability-sla-breach-alerting (Skill Module)

implementing-vulnerability-sla-breach-alerting\SKILL.md

name: implementing-vulnerability-sla-breach-alerting
description: Vulnerability SLA breach alerting agent. Monitors vulnerability remediation timelines and generates alerts when SLA breaches occur or are imminent. Supports webhook notifications (Slack, Teams, PagerDuty), email.... Use when the user asks to implement vulnerability sla breach alerting.
version: 1.0.0
tags:
source: implementing-vulnerability-sla-breach-alerting

Implementing Vulnerability Sla Breach Alerting

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Alert Channels, SLA Breach Detection, Orchestration, Primary Standards, Industry SLA Benchmarks

When to use

- The user wants to **implement** vulnerability sla breach alerting.
- Capability: Vulnerability SLA breach alerting agent. Monitors vulnerability remediation timelines and generates alerts when SLA breaches occur or are imminent. Supports webhook notifications (Slack, Teams, PagerDuty), email alerts, and escalation workflows.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, vulnerability, breach, alerting`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load vulnerability data from JSON file
2. Check for SLA breaches and upcoming deadlines
3. Send SLA breach alert to Slack via webhook
4. Send SLA breach alert to Microsoft Teams via webhook
5. Send PagerDuty incident for critical SLA breaches
6. Print alert summary

Key concepts

- Title: Guide to Enterprise Patch Management Planning
- Relevance: Defines organizational patch management lifecycle and remediation timelines
- Title: Reducing the Significant Risk of Known Exploited Vulnerabilities
- SLA Mandate: Federal agencies must remediate KEV-listed vulnerabilities within specified timeframes (typically 14 days for
- Title: Security Vulnerabilities Are Identified and Addressed
- SLA Requirement: Critical and high-severity vulnerabilities must be patched within 30 days of release; risk-ranked approach for
- Title: Detection and Monitoring of Security Events
- Relevance: Requires evidence of vulnerability management program with defined remediation timelines and tracking

Tools & resources

- Alert Channels
- SLA Breach Detection
- Orchestration
- Primary Standards
- Industry SLA Benchmarks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_vulnerabilities` — Load vulnerability data from JSON file.
- `check_sla_breaches` — Check for SLA breaches and upcoming deadlines.
- `send_slack_alert` — Send SLA breach alert to Slack via webhook.
- `send_teams_alert` — Send SLA breach alert to Microsoft Teams via webhook.
- `send_pagerduty_alert` — Send PagerDuty incident for critical SLA breaches.
- `format_summary` — Print alert summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-vulnerability-sla-breach-alerting`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-web-application-logging-with-modsecurity (Skill Module)

implementing-web-application-logging-with-modsecurity\SKILL.md

name: implementing-web-application-logging-with-modsecurity
description: ModSecurity WAF audit log analysis and rule tuning. Use when the user asks to implement web application logging with modsecurity.
version: 1.0.0
tags:
source: implementing-web-application-logging-with-modsecurity

Implementing Web Application Logging with Modsecurity

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Key ModSecurity Directives, Audit Log Sections, CRS Paranoia Levels, Configuration Example, External References

When to use

- The user wants to **implement** web application logging with modsecurity.
- Capability: ModSecurity WAF audit log analysis and rule tuning agent.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, application, logging, modsecurity`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse ModSecurity serial audit log format
2. Analyze which rules fire most frequently for tuning
3. Identify rules that may be false positives based on frequency and pattern
4. Generate ModSecurity rule exclusion configuration
5. Summarize detected attacks by category and severity
6. Execute ModSecurity audit log analysis

Tools & resources

- Key ModSecurity Directives
- Audit Log Sections
- CRS Paranoia Levels
- Configuration Example
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_audit_log` — Parse ModSecurity serial audit log format.
- `analyze_rule_frequency` — Analyze which rules fire most frequently for tuning.
- `identify_false_positive_candidates` — Identify rules that may be false positives based on frequency and pattern.
- `generate_exclusion_rules` — Generate ModSecurity rule exclusion configuration.
- `analyze_attack_summary` — Summarize detected attacks by category and severity.
- `run_audit` — Execute ModSecurity audit log analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-web-application-logging-with-modsecurity`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-knowledge-proof-for-authentication (Skill Module)

`implementing-zero-knowledge-proof-for-authentication\SKILL.md`

name: implementing-zero-knowledge-proof-for-authentication
description: Agent for implementing zero-knowledge proof authentication using Schnorr protocol. Use when the user asks to implement zero knowledge proof for authentication.
version: 1.0.0
tags:
source: implementing-zero-knowledge-proof-for-authentication

Implementing Zero Knowledge Proof for Authentication

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Schnorr Protocol Steps, Fiat, ZKP Properties, Academic References

When to use

- The user wants to **implement** zero knowledge proof for authentication.
- Capability: Agent for implementing zero-knowledge proof authentication using Schnorr protocol.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, knowledge, proof, authentication`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate a ZKP key pair (private key x, public key y = g^x mod p)
2. Generate a Schnorr ZKP proof (commitment, challenge, response)
3. Verify a Schnorr ZKP proof without learning the private key
4. Register a password using ZKP (server stores only public key)
5. Authenticate using ZKP (prove password knowledge without revealing it)
6. Demonstrate ZKP authentication protocol with multiple rounds
7. Demonstrate ZKP password authentication

Key concepts

- Paper: "Efficient Signature Generation by Smart Cards" (Claus-Peter Schnorr, 1989)
- Standard: ISO/IEC 9798-5 (Entity authentication using zero-knowledge techniques)
- Paper: "How To Prove Yourself" (Fiat, Shamir, 1986)
- Description: Converts interactive ZKP to non-interactive using hash function
- Description: Standardized Schnorr NIZKP
- Description: Zero-knowledge password authentication protocol
- Description: Elliptic curve operations for ZKPs
- Description: Hash functions, modular arithmetic support

Tools & resources

- Schnorr Protocol Steps
- Fiat
- ZKP Properties
- Academic References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_keypair` — Generate a ZKP key pair (private key x, public key y = g^x mod p).
- `schnorr_prove` — Generate a Schnorr ZKP proof (commitment, challenge, response).
- `schnorr_verify` — Verify a Schnorr ZKP proof without learning the private key.
- `zkp_password_register` — Register a password using ZKP (server stores only public key).
- `zkp_password_authenticate` — Authenticate using ZKP (prove password knowledge without revealing it).
- `run_protocol_demo` — Demonstrate ZKP authentication protocol with multiple rounds.
- `run_password_demo` — Demonstrate ZKP password authentication.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-knowledge-proof-for-authentication`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-standing-privilege-with-cyberark (Skill Module)

implementing-zero-standing-privilege-with-cyberark\SKILL.md

name: implementing-zero-standing-privilege-with-cyberark
description: Agent for auditing CyberArk Zero Standing Privilege (ZSP) configuration via REST API. Use when the user asks to implement zero standing privilege with cyberark.
version: 1.0.0
tags:
source: implementing-zero-standing-privilege-with-cyberark

Implementing Zero Standing Privilege with Cyberark

Role: Security control implementation
Audience: security architects and engineers
Primary tools: CyberArk, TEA Framework, Zero Trust Frameworks, Compliance Requirements

When to use

- The user wants to **implement** zero standing privilege with cyberark.
- Capability: Agent for auditing CyberArk Zero Standing Privilege (ZSP) configuration via REST API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, standing, privilege, cyberark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Authenticate to CyberArk PVWA and obtain session token
2. List all safes to audit access policies
3. Audit members and permissions of a specific safe
4. List platforms to verify JIT/ZSP configuration
5. Check recent privileged sessions for JIT compliance
6. Find privileged accounts with standing (non-JIT) access enabled
7. Generate ZSP compliance audit report

Key concepts

- Never trust, always verify
- Least privilege access to resources
- Microsegmentation and policy enforcement points
- Dynamic, risk-based access policies
- Identity pillar: JIT/JEA access for all identities
- Advanced maturity: Automated privilege provisioning/deprovisioning
- Optimal maturity: Continuous verification with ephemeral access
- Session duration: minimum required for task completion

Tools & resources

- CyberArk
- TEA Framework
- Zero Trust Frameworks
- Compliance Requirements

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `authenticate` — Authenticate to CyberArk PVWA and obtain session token.
- `list_safes` — List all safes to audit access policies.
- `audit_safe_members` — Audit members and permissions of a specific safe.
- `list_platforms` — List platforms to verify JIT/ZSP configuration.
- `check_jit_sessions` — Check recent privileged sessions for JIT compliance.
- `audit_accounts_standing_access` — Find privileged accounts with standing (non-JIT) access enabled.
- `generate_report` — Generate ZSP compliance audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-standing-privilege-with-cyberark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-dns-with-nextdns (Skill Module)

`implementing-zero-trust-dns-with-nextdns\SKILL.md`

name: implementing-zero-trust-dns-with-nextdns
description: Agent for configuring and auditing NextDNS zero trust DNS filtering via API. Use when the user asks to implement zero trust dns with nextdns.
version: 1.0.0
tags:
source: implementing-zero-trust-dns-with-nextdns

Implementing Zero Trust DNS with Nextdns

Role: Security control implementation
Audience: security architects and engineers
Primary tools: NextDNS REST API, DNS Security Standards, Zero Trust DNS Standards

When to use

- The user wants to **implement** zero trust dns with nextdns.
- Capability: Agent for configuring and auditing NextDNS zero trust DNS filtering via API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, nextdns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Retrieve NextDNS profile configuration
2. Audit security features enabled on a NextDNS profile
3. Retrieve recent DNS query logs for analysis
4. Retrieve DNS analytics and threat statistics
5. Check configured denylists and custom blocked domains
6. Generate NextDNS audit report

Key concepts

- Encrypts DNS queries over HTTPS on port 443
- Prevents DNS eavesdropping and manipulation
- Browser and OS-level support
- Encrypts DNS queries over TLS on port 853
- Dedicated port enables network-level policy control
- Supported by Android 9+, systemd-resolved, Unbound
- Lower latency than DoH/DoT
- Multiplexed queries without head-of-line blocking

Tools & resources

- NextDNS REST API
- DNS Security Standards
- Zero Trust DNS Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_profile` — Retrieve NextDNS profile configuration.
- `audit_security_settings` — Audit security features enabled on a NextDNS profile.
- `get_query_logs` — Retrieve recent DNS query logs for analysis.
- `get_analytics` — Retrieve DNS analytics and threat statistics.
- `check_denylist` — Check configured denylists and custom blocked domains.
- `generate_report` — Generate NextDNS audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-dns-with-nextdns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-for-saas-applications (Skill Module)

implementing-zero-trust-for-saas-applications\SKILL.md

name: implementing-zero-trust-for-saas-applications
description: Agent for auditing zero trust controls on SaaS applications via Microsoft Graph API. Use when the user asks to implement zero trust for saas applications.
version: 1.0.0
tags:
source: implementing-zero-trust-for-saas-applications

Implementing Zero Trust for Saas Applications

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Microsoft Graph API v1.0

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **implement** zero trust for saas applications.
- Capability: Agent for auditing zero trust controls on SaaS applications via Microsoft Graph API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, saas, applications`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Acquire OAuth2 token for Microsoft Graph API
2. List Entra ID conditional access policies for SaaS apps
3. List enterprise applications (service principals) for shadow IT discovery
4. Audit OAuth2 permission grants for overprivileged consents
5. Check risky sign-ins to SaaS applications
6. Generate SaaS zero trust audit report

Tools & resources

- Microsoft Graph API v1.0

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_token` — Acquire OAuth2 token for Microsoft Graph API.
- `list_conditional_access_policies` — List Entra ID conditional access policies for SaaS apps.
- `list_enterprise_apps` — List enterprise applications (service principals) for shadow IT discovery.
- `check_oauth_app_consents` — Audit OAuth2 permission grants for overprivileged consents.
- `check_sign_in_risk` — Check risky sign-ins to SaaS applications.
- `generate_report` — Generate SaaS zero trust audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-for-saas-applications`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-in-cloud (Skill Module)

implementing-zero-trust-in-cloud\SKILL.md

name: implementing-zero-trust-in-cloud
description: Zero trust cloud architecture assessment agent using AWS, Azure, and GCP SDKs. Use when the user asks to implement zero trust in cloud.
version: 1.0.0
tags:
source: implementing-zero-trust-in-cloud

Implementing Zero Trust in Cloud

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS Zero Trust Services, Azure Zero Trust Services, GCP Zero Trust Services, Maturity Levels, External References

When to use

- The user wants to **implement** zero trust in cloud.
- Capability: Zero trust cloud architecture assessment agent using AWS, Azure, and GCP SDKs.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, cloud`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Assess AWS zero trust posture
2. Generate a zero trust maturity scorecard from findings
3. Run comprehensive zero trust assessment

Tools & resources

- AWS Zero Trust Services
- Azure Zero Trust Services
- GCP Zero Trust Services
- Maturity Levels
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `assess_aws_zero_trust` — Assess AWS zero trust posture.
- `generate_zero_trust_scorecard` — Generate a zero trust maturity scorecard from findings.
- `run_zero_trust_assessment` — Run comprehensive zero trust assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-in-cloud`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-network-access (Skill Module)

implementing-zero-trust-network-access\SKILL.md

name: implementing-zero-trust-network-access
description: Zero Trust Network Access (ZTNA) Assessment Agent Evaluates ZTNA readiness across AWS, Azure, and GCP by checking IAP configs, Verified Access endpoints, conditional access policies, and micro-segmentation. Use when the user asks to implement zero trust network access.
version: 1.0.0
tags:
source: implementing-zero-trust-network-access

Implementing Zero Trust Network Access

Role: Security control implementation
Audience: security architects and engineers
Primary tools: AWS Verified Access API, GCP Identity-Aware Proxy API

When to use

- The user wants to **implement** zero trust network access.
- Capability: Zero Trust Network Access (ZTNA) Assessment Agent Evaluates ZTNA readiness across AWS, Azure, and GCP by checking IAP configs, Verified Access endpoints, conditional access policies, and micro-segmentation.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, network, access`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Enumerate AWS Verified Access instances, groups, and endpoints
2. Check for overly permissive security groups that undermine micro-segmentation
3. Check GCP Identity-Aware Proxy configuration
4. Generate a ZTNA assessment report

Tools & resources

- AWS Verified Access API
- GCP Identity-Aware Proxy API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_aws_verified_access` — Enumerate AWS Verified Access instances, groups, and endpoints.
- `check_aws_security_groups_segmentation` — Check for overly permissive security groups that undermine micro-segmentation.
- `check_gcp_iap_status` — Check GCP Identity-Aware Proxy configuration.
- `generate_ztna_report` — Generate a ZTNA assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-network-access`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-network-access-with-zscaler (Skill Module)

implementing-zero-trust-network-access-with-zscaler\SKILL.md

name: implementing-zero-trust-network-access-with-zscaler
description: Agent for auditing Zscaler Private Access (ZPA) zero trust configuration via API. Use when the user asks to implement zero trust network access with zscaler.
version: 1.0.0
tags:
source: implementing-zero-trust-network-access-with-zscaler

Implementing Zero Trust Network Access with Zscaler

Role: Security control implementation
Audience: security architects and engineers
Primary tools: ZPA Management API, NIST SP 800-207, CSA Software

When to use

- The user wants to **implement** zero trust network access with zscaler.
- Capability: Agent for auditing Zscaler Private Access (ZPA) zero trust configuration via API.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, network, access, zscaler`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Authenticate to Zscaler ZPA API
2. List ZPA application segments
3. List server groups and their connectors
4. List ZPA access policies to verify least-privilege enforcement
5. Check connector enrollment and health status
6. Generate ZPA audit report

Key concepts

- Enhanced Identity Governance: ZPA implements this model by using identity as the primary decision factor combined with device trust signals
- Micro-Segmentation: ZPA application segments function as software-defined microsegments at the application layer
- Software Defined Perimeters: ZPA directly implements the SDP model with its broker-based architecture
- Extends zero trust principles to multi-cloud environments
- ZPA App Connectors can be deployed across AWS, Azure, GCP, and on-premises
- Supports workload-to-workload zero trust with ZPA for workloads
- Client-to-Gateway: Standard ZPA deployment (user to application via Service Edge)
- Client-to-Server: ZPA Browser Access (direct browser connection through Service Edge)

Tools & resources

- ZPA Management API
- NIST SP 800-207
- CSA Software

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `authenticate` — Authenticate to Zscaler ZPA API.
- `list_app_segments` — List ZPA application segments.
- `list_server_groups` — List server groups and their connectors.
- `list_access_policies` — List ZPA access policies to verify least-privilege enforcement.
- `check_connector_health` — Check connector enrollment and health status.
- `generate_report` — Generate ZPA audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-network-access-with-zscaler`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-with-beyondcorp (Skill Module)

`implementing-zero-trust-with-beyondcorp\SKILL.md`

name: implementing-zero-trust-with-beyondcorp
description: BeyondCorp Zero Trust Agent - audits IAP configuration, access levels, and policy bindings. Use when the user asks to implement zero trust with beyondcorp.
version: 1.0.0
tags:
source: implementing-zero-trust-with-beyondcorp

Implementing Zero Trust with Beyondcorp

Role: Security control implementation
Audience: security architects and engineers
Primary tools: IAP Roles, Python SDK

When to use

- The user wants to **implement** zero trust with beyondcorp.
- Capability: BeyondCorp Zero Trust Agent - audits IAP configuration, access levels, and policy bindings.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, beyondcorp`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute gcloud command and return JSON output
2. List resources protected by Identity-Aware Proxy
3. Retrieve Access Context Manager access levels
4. Audit IAM bindings on IAP-protected resources
5. Audit access levels for security strength
6. Check Endpoint Verification deployment status

Tools & resources

- IAP Roles
- Python SDK

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `gcloud_json` — Execute gcloud command and return JSON output.
- `list_iap_protected_resources` — List resources protected by Identity-Aware Proxy.
- `get_access_levels` — Retrieve Access Context Manager access levels.
- `audit_iap_iam_bindings` — Audit IAM bindings on IAP-protected resources.
- `audit_access_level_strength` — Audit access levels for security strength.
- `check_endpoint_verification` — Check Endpoint Verification deployment status.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-with-beyondcorp`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

implementing-zero-trust-with-hashicorp-boundary (Skill Module)

`implementing-zero-trust-with-hashicorp-boundary\SKILL.md`

name: implementing-zero-trust-with-hashicorp-boundary
description: Agent for auditing HashiCorp Boundary zero trust access configuration. Use when the user asks to implement zero trust with hashicorp boundary.
version: 1.0.0
tags:
source: implementing-zero-trust-with-hashicorp-boundary

Implementing Zero Trust with Hashicorp Boundary

Role: Security control implementation
Audience: security architects and engineers
Primary tools: Core Standards, Boundary Security Model

When to use

- The user wants to **implement** zero trust with hashicorp boundary.
- Capability: Agent for auditing HashiCorp Boundary zero trust access configuration.
- Goal: design and implement durable security controls and architectures.
- Trigger keywords: `implementing, zero, trust, hashicorp, boundary`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute a boundary CLI command and return parsed JSON
2. List organization and project scopes
3. List targets (resources users can connect to) in a scope
4. List host catalogs (static or dynamic)
5. List credential stores (Vault integration check)
6. List active sessions for audit
7. Audit configured authentication methods
8. Generate Boundary audit report

Key concepts

- Boundary implements identity-aware proxy architecture (Section 3.2.2)
- Default-deny access model aligns with ZTA core principle
- Dynamic credential brokering eliminates standing privileges
- Session-level authorization satisfies continuous verification
- AC-2: Account Management (identity lifecycle via OIDC)
- AC-3: Access Enforcement (role-based grants)
- AC-6: Least Privilege (fine-grained permissions)
- AU-2: Audit Events (session recording and logging)

Tools & resources

- Core Standards
- Boundary Security Model

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_boundary_cmd` — Execute a boundary CLI command and return parsed JSON.
- `list_scopes` — List organization and project scopes.
- `list_targets` — List targets (resources users can connect to) in a scope.
- `list_host_catalogs` — List host catalogs (static or dynamic).
- `list_credential_stores` — List credential stores (Vault integration check).
- `list_sessions` — List active sessions for audit.
- `check_auth_methods` — Audit configured authentication methods.
- `generate_report` — Generate Boundary audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `implementing-zero-trust-with-hashicorp-boundary`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

integrating-dast-with-owasp-zap-in-pipeline (Skill Module)

integrating-dast-with-owasp-zap-in-pipeline\SKILL.md

name: integrating-dast-with-owasp-zap-in-pipeline
description: Agent for running OWASP ZAP DAST scans and parsing results for CI/CD integration. Use when the user asks to integrating dast with owasp zap in pipeline.
version: 1.0.0
tags:
source: integrating-dast-with-owasp-zap-in-pipeline

Integrating Dast with Owasp Zap in Pipeline

Role: Security workflow
Audience: security practitioners
Primary tools: ZAP Docker Scan Scripts, OWASP Top 10, OWASP SAMM, PCI DSS v4.0

When to use

- The user wants to **integrat** dast with owasp zap in pipeline.
- Capability: Agent for running OWASP ZAP DAST scans and parsing results for CI/CD integration.
- Goal: complete the task.
- Trigger keywords: `integrating, dast, owasp, pipeline`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run ZAP baseline scan (passive only, fast)
2. Run ZAP full scan (active + passive, slower)
3. Run ZAP API scan against an OpenAPI/Swagger spec
4. Parse ZAP JSON report and extract findings by risk level
5. Apply quality gate: fail if findings at or above threshold
6. Convert ZAP findings to SARIF format for GitHub Advanced Security

Key concepts

- Automated DAST scanning in CI/CD pipelines
- Baseline scans on every deployment, full scans weekly
- Findings tracked and triaged with defined SLAs
- Authenticated scanning with session management
- API-specific scanning with OpenAPI specifications
- Custom scan policies tuned for application-specific risks
- PW.8.1: Test executable code using dynamic analysis
- DAST validates running application behavior against security requirements

Tools & resources

- ZAP Docker Scan Scripts
- OWASP Top 10
- OWASP SAMM
- PCI DSS v4.0

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_baseline_scan` — Run ZAP baseline scan (passive only, fast).
- `run_full_scan` — Run ZAP full scan (active + passive, slower).
- `run_api_scan` — Run ZAP API scan against an OpenAPI/Swagger spec.
- `parse_zap_report` — Parse ZAP JSON report and extract findings by risk level.
- `apply_quality_gate` — Apply quality gate: fail if findings at or above threshold.
- `generate_sarif` — Convert ZAP findings to SARIF format for GitHub Advanced Security.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `integrating-dast-with-owasp-zap-in-pipeline`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

integrating-sast-into-github-actions-pipeline (Skill Module)

`integrating-sast-into-github-actions-pipeline\SKILL.md`

name: integrating-sast-into-github-actions-pipeline
description: Agent for running Semgrep SAST scans and generating SARIF for GitHub Advanced Security. Use when the user asks to integrating sast into github actions pipeline.
version: 1.0.0
tags:
source: integrating-sast-into-github-actions-pipeline

Integrating Sast Into Github Actions Pipeline

Role: Security workflow
Audience: security practitioners
Primary tools: Semgrep CLI, OWASP SAMM

When to use

- The user wants to **integrat** sast into github actions pipeline.
- Capability: Agent for running Semgrep SAST scans and generating SARIF for GitHub Advanced Security.
- Goal: complete the task.
- Trigger keywords: `integrating, sast, into, github, actions, pipeline`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run Semgrep SAST scan on a code directory
2. Parse Semgrep findings into structured format
3. Convert Semgrep findings to SARIF 2.1.0 format
4. Apply quality gate based on severity threshold
5. Generate a GitHub Actions SAST workflow YAML

Key concepts

- Perform automated SAST scanning with default rulesets on all application code
- Results are visible to development teams through IDE or CI/CD integration
- Customize SAST rules to reduce false positives below 20%
- Track and triage all findings with defined SLAs per severity
- Integrate SAST results into a centralized vulnerability management system
- Correlate SAST findings with DAST and SCA results for comprehensive coverage
- Measure and improve detection accuracy through benchmarking against known vulnerabilities
- Custom rules cover organization-specific vulnerability patterns and deprecated APIs

Tools & resources

- Semgrep CLI
- OWASP SAMM

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_semgrep_scan` — Run Semgrep SAST scan on a code directory.
- `parse_semgrep_results` — Parse Semgrep findings into structured format.
- `generate_sarif` — Convert Semgrep findings to SARIF 2.1.0 format.
- `apply_quality_gate` — Apply quality gate based on severity threshold.
- `generate_github_actions_workflow` — Generate a GitHub Actions SAST workflow YAML.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `integrating-sast-into-github-actions-pipeline`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

intercepting-mobile-traffic-with-burpsuite (Skill Module)

`intercepting-mobile-traffic-with-burpsuite\SKILL.md`

name: intercepting-mobile-traffic-with-burpsuite
description: Agent for analyzing intercepted mobile app traffic via mitmproxy for security testing. Use when the user asks to intercepting mobile traffic with burpsuite.
version: 1.0.0
tags:
source: intercepting-mobile-traffic-with-burpsuite

Intercepting Mobile Traffic with Burpsuite

Role: Security workflow
Audience: security practitioners
Primary tools: Burp Suite, mitmproxy Alternative, Certificate Pinning Bypass, CWE Mappings

When to use

- The user wants to **intercept** mobile traffic with burpsuite.
- Capability: Agent for analyzing intercepted mobile app traffic via mitmproxy for security testing.
- Goal: complete the task.
- Trigger keywords: `intercepting, mobile, traffic, burpsuite`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load and parse an HTTP Archive (HAR) file from proxy capture
2. Identify HTTP (non-HTTPS) requests from mobile app
3. Scan request/response bodies for sensitive data patterns
4. Analyze authentication headers and token handling
5. Check for certificate pinning indicators in traffic
6. Check API response security headers
7. Generate mobile traffic analysis report

Tools & resources

- Burp Suite
- mitmproxy Alternative
- Certificate Pinning Bypass
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_har_file` — Load and parse an HTTP Archive (HAR) file from proxy capture.
- `find_insecure_requests` — Identify HTTP (non-HTTPS) requests from mobile app.
- `detect_sensitive_data_leakage` — Scan request/response bodies for sensitive data patterns.
- `check_auth_headers` — Analyze authentication headers and token handling.
- `check_certificate_pinning` — Check for certificate pinning indicators in traffic.
- `check_api_security_headers` — Check API response security headers.
- `generate_report` — Generate mobile traffic analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `intercepting-mobile-traffic-with-burpsuite`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

interview-prep (Skill Module)

journalism-core\skills\interview-prep\SKILL.md

name: interview-prep

description: Prepare for journalism interviews with research checklists, question frameworks, and attribution guidelines. Use when preparing to interview sources, planning follow-up questions, or managing interview logistics. Covers consent, recording laws, and professional protocols.

tags:

Interview preparation

Interviews fail in the preparation, not the conversation. This [[skill]] covers pre-interview research, question design, logistics, and follow-up.

When to use

- Preparing to interview a source
- Developing question frameworks for recurring interview types
- Managing interview logistics and consent
- Planning follow-up after initial interviews
- Training new reporters on interview technique

Pre-interview research checklist

Background research

```
## Source background check

### Public records
- [ ] Professional licenses verified
- [ ] Court records checked (civil/criminal)
- [ ] Business registrations confirmed
- [ ] Property records (if relevant)
- [ ] Campaign finance (if political figure)
- [ ] SEC filings (if corporate)

### Professional background
- [ ] LinkedIn profile reviewed
- [ ] Current employer confirmed
- [ ] Previous employers noted
- [ ] Published work reviewed
- [ ] Conference appearances checked
- [ ] Professional associations

### Social media audit
- [ ] All platforms identified (X, Bluesky, Threads, Mastodon, Instagram, Facebook, LinkedIn, TikTok, YouTube, Substack, etc.)
- [ ] Post history reviewed
- [ ] Connections/followers analyzed
- [ ] Previous statements on topic found
- [ ] Any deleted content recovered? (See **web-archiving** skill for Wayback / Archive.today retrieval)

### Media appearances
- [ ] Previous interviews found
- [ ] Statements consistent with current position?
- [ ] Other journalists' assessments
- [ ] Any retractions or corrections involving them?
```

Context research

```
## Topic preparation

### Essential knowledge
- [ ] Key facts about the topic confirmed
- [ ] Timeline of events established
- [ ] Other stakeholders identified
- [ ] Conflicting accounts noted
- [ ] Documents/data reviewed

### What to know before you dial
- [ ] How do they fit into the story?
- [ ] What do I NEED from this interview?
- [ ] What might they be reluctant to discuss?
- [ ] What have they said publicly before?
```

Question framework

The essential questions

Every interview should be built to answer:
1. **What happened?** (Facts)
2. **Why did it happen?** (Causes)
3. **What did you do/decide/see?** (Actions)
4. **What does it mean?** (Significance)
5. **What's next?** (Implications)

Question types

Type	Purpose	Example
Open-ended	Get the full story	"Walk me through what happened that day."
Clarifying	Pin down details	"When you say 'soon after,' do you mean minutes or hours?"
Probing	Go deeper	"Why do you think that happened?"
Follow-up	Catch inconsistencies	"Earlier you said X, but now you mentioned Y. Help me understand."
Confrontational	Challenge statements	"Documents show [fact]. How do you respond?"
Closing	Ensure completeness	"Is there anything I didn't ask that you think I should know?"

Question templates by interview type

Profile interview:

```
1. Background: "Tell me about where you grew up / how you got started."
2. Turning point: "When did you realize [X] was your path?"
3. Challenge: "What was the hardest moment in [period]?"
4. Values: "What principle guides your work?"
5. Future: "What are you working on next?"
```

Investigative interview:

```
1. Establish rapport: Non-threatening background questions first
2. Timeline: "Walk me through [event] from the beginning."
3. Details: "Who else was there? What did you see/hear?"
4. Documentation: "Do you have any records of this?"
5. Corroboration: "Who else can confirm this?"
6. Response: "What did [other party] say when you raised this?"
```

Expert/explainer interview:

```
1. Credentials: "What's your expertise in this area?"
2. Plain language: "Explain [concept] as if I'm not a specialist."
3. Context: "How common/unusual is [situation]?"
4. Significance: "Why does this matter?"
5. Sources: "Where can I learn more? Who else should I talk to?"
```

Victim/sensitive interview:

```
1. Control: "Take your time. You can stop at any point."
2. Open: "Tell me what you're comfortable sharing."
3. Specific: "Can you describe [specific detail]?"
4. Impact: "How has this affected you?"
5. Agency: "What do you want people to understand?"
6. Check-in: "Are you okay to continue?"
```

Recording and consent

Recording laws by state

State recording-consent law is jurisdiction-specific and shifts. Some states distinguish telephone from in-person recording; others treat electronic communications under a separate statute; case law in several states (e.g., Massachusetts, Michigan, Connecticut) has narrowed or expanded the original statutory rule.

The single most reliable reference is the Reporters Committee for Freedom of the Press *Reporter's Recording Guide* at `rcfp.org/reporters-recording-guide`. It is maintained continuously and provides per-state breakdowns covering one-party vs. all-party consent, phone-vs-in-person distinctions, hidden-recording rules, and federal preemption.

General categories:

- **One-party consent (federal default).** You can record without telling the other person if you are a party to the conversation. You *should* tell them anyway for ethical reasons.
- **All-party (two-party) consent.** All participants must consent. As of 2025-2026 this category includes (non-exhaustive, verify against RCFP before relying): California, Florida, Illinois, Maryland, Massachusetts, Montana, Nevada, New Hampshire, Pennsylvania, Washington. Several other states (Connecticut, Michigan, Oregon, Delaware, Vermont, Hawaii) apply two-party rules in some contexts but not all — RCFP has the per-context details.
- **Cross-state calls.** When the parties are in different states, the stricter state's law generally controls. If you are in a one-party state interviewing someone in a two-party state, get consent.

Always do, regardless of jurisdiction:

- State clearly at the start: "I'm recording this interview. Is that okay?"
- Get affirmative consent on the recording itself.
- Note the consent in your notes.

This protects you legally everywhere and is the ethical baseline.

Consent

[illegible]

```
## Recording consent

Date: [date]
Interviewer: [your name]
Subject: [their name]
Medium: [phone/video/in-person]

[At start of recording:]
"This is [your name] with [publication], interviewing [their name] on [date].
I'm recording this conversation. Do I have your permission to record?"

[Their response: yes/no]

"And you understand this may be used for publication?"

[Their response: yes/no]
```

Attribution guidelines

On the record (default)

- Everything can be published with full name and title
- This is the standard expectation unless otherwise agreed

On background

- Information can be used, but source not identified by name
- Agree on description: "a senior official," "someone familiar with the matter"
- Confirm exact wording before interview ends

Deep background

- Information can guide reporting but cannot be attributed at all

- Verify independently before publishing
- Rarely appropriate—push for at least background

Off the record

- Information is for your knowledge only
- Cannot be published or used to seek confirmation elsewhere
- Agree to this BEFORE they share information, not after
- If they say something on the record then try to take it off, you can refuse

Clarifying attribution

```
Before starting:
"Just to be clear on attribution—are we on the record?"

If they request otherwise:
"I'd prefer on the record. What concerns you about that?"

If they insist:
"Okay, we'll go on background. What description can I use?"

Document it:
"So I can refer to you as [agreed description]—is that right?"
```

Interview logistics

Scheduling

[[[

During the interview

Opening

- Small talk to build rapport (brief)
- Confirm time available
- State recording and get consent
- Start with easy, open questions

Active listening

- Let them finish sentences
- Use silence—don't fill every pause
- Take notes even if recording
- Note non-verbal cues separately

Real-time verification

- Ask for specifics: dates, names, locations
- Request documentation during interview
- Ask "How do you know that?"
- Note inconsistencies for follow-up

Closing

- "Is there anything I didn't ask that I should have?"
- "Who else should I talk to?"
- "Can I follow up if I have more questions?"
- Thank them for their time

Follow-up protocols

For recording backup, transcription tooling, and quote verification mechanics, use the **interview-transcription** [[skill]]. The notes here cover only the post-interview *editorial* steps that aren't covered there.

Immediate (same day)

- [ ] Back up recording (see interview-transcription for storage and naming)
- [ ] Note observations not on recording (body language, environment, off-mic remarks)
- [ ] Send thank-you if appropriate

Within 48 hours

- [ ] [[Fact-check]] claims against available records
- [ ] Identify gaps requiring follow-up
- [ ] Add source to contacts database

Before [[[[publication]]]]

- [ ] Verify quotes are accurate
- [ ] Confirm attribution terms
- [ ] Offer to read back quotes if promised
- [ ] Give chance to respond to characterizations (if newsworthy)

Difficult situations

They want to go off the record mid-interview

"Before I agree to that, let me hear what you want to tell me, and then we can discuss how to handle it."

They refuse to answer

"I understand you can't discuss that. Can you point me to someone who can?"

They're hostile

Stay calm. Keep questions factual. "I'm just trying to understand what happened."

They're crying/emotional

Pause. "Take your time. We can stop whenever you need."

They lie

Don't accuse. Present contradicting evidence: "Documents show [X]. Can you help me understand the discrepancy?"

Good interviews require good preparation. The conversation is the easy part.

interview-transcription (Skill Module)

journalism-core\skills\interview-transcription\SKILL.md

name: interview-transcription

description: Transcription workflows, recording management, and quote extraction for journalists. Use when processing audio/video recordings, generating transcripts with timestamps, extracting quotes for fact-checking, or building source-and-recording databases. For interview question design and pre-interview preparation, see the interview-prep skill.

tags:

Interview transcription and management

Practical
[[[

```
from pathlib import Path
import subprocess
import json

def transcribe_interview(
    audio_path: str,
    output_dir: str = "./transcripts",
    diarize: bool = True,
    hf_token: str | None = None,
    min_speakers: int = 2,
    max_speakers: int = 2,
) -> dict:
    """
    Transcribe an interview using WhisperX (Whisper + pyannote diarization).
    Returns a transcript with word-level timestamps and speaker labels.

    Diarization needs a Hugging Face token with access to the pyannote
    speaker-diarization-3.1 model. Accept the model EULA at
    huggingface.co/pyannote/speaker-diarization-3.1 once, then pass the token.
    """
    Path(output_dir).mkdir(exist_ok=True)

    cmd = [
        'whisperx', audio_path,
        '--model', 'large-v3',
        '--output_format', 'json',
        '--output_dir', output_dir,
        '--language', 'en',
        '--compute_type', 'int8',     # CPU-friendly; use 'float16' on GPU
        '--min_speakers', str(min_speakers),
        '--max_speakers', str(max_speakers),
    ]

    if diarize:
        cmd.append('--diarize')
        if hf_token:
            cmd += ['--hf_token', hf_token]

    subprocess.run(cmd, check=True, capture_output=True)

    json_path = Path(output_dir) / f"{Path(audio_path).stem}.json"
    with open(json_path) as f:
        return json.load(f)

def format_for_editing(transcript: dict) -> str:
    """Convert to journalist-friendly format with timestamps."""
    lines = []
    for segment in transcript.get('segments', []):
        timestamp = format_timestamp(segment['start'])
        text = segment['text'].strip()
        lines.append(f"[{timestamp}] {text}")
    return '\n\n'.join(lines)

def format_timestamp(seconds: float) -> str:
    """Convert seconds to HH:MM:SS format."""
    h = int(seconds // 3600)
    m = int((seconds % 3600) // 60)
    s = int(seconds % 60)
    return f"{h:02d}:{m:02d}:{s:02d}"
```

Falling back to plain Whisper. If diarization is overkill or you can't get a Hugging Face token, drop the `--diarize` flag — the model still produces accurate timestamped transcription and you label speakers manually based on context. `faster-whisper` (CTranslate2 backend) is the speed-optimized variant and works the same way at the CLI. `whisper.cpp` is the C++ port for resource-constrained machines (Raspberry Pi, older laptops); it doesn't include diarization but runs the small/medium models on CPU comfortably.

Manual transcription

[[[

```
## Transcript: [Source] - [Date]

**Recording file**: [filename]
**Duration**: [XX:XX]
**Transcribed by**: [name]
**Verified against recording**: [ ] Yes / [ ] No

---

[00:00:15] **Q**: [Your question]

[00:00:45] **A**: [Source response - verbatim, including ums, pauses noted as (...)]

[00:01:30] **Q**: [Follow-up]

[00:01:42] **A**: [Response]

---

## Notes
- [Anything not captured in audio: gestures, documents shown, etc.]

## Potential quotes
- [00:01:42] "Quote that stands out" - context: [why it matters]
```

Quote extraction and verification

Pull quotes workflow

```
from dataclasses import dataclass
from typing import Optional
import re

@dataclass
class Quote:
    text: str
    timestamp: str
    speaker: str
    context: str
    verified: bool = False
    used_in: Optional[str] = None

class QuoteBank:
    """Manage quotes from interview transcripts."""

    def __init__(self):
        self.quotes = []

    def extract_quote(self, transcript: str, start_time: str,
                      end_time: str, speaker: str, context: str) -> Quote:
        """Extract and store a quote with metadata."""
        # Pull text between timestamps
        pattern = rf'\[{re.escape(start_time)}\](.+?)(?=\[\d|$)'
        match = re.search(pattern, transcript, re.DOTALL)

        if match:
            text = match.group(1).strip()
            quote = Quote(
                text=text,
                timestamp=start_time,
                speaker=speaker,
                context=context
            )
            self.quotes.append(quote)
            return quote
        return None

    def verify_quote(self, quote: Quote, audio_path: str) -> bool:
        """Mark quote as verified against original recording."""
        # In practice: listen to audio at timestamp, confirm accuracy
        quote.verified = True
        return True

    def export_for_story(self) -> str:
        """Export verified quotes ready for publication."""
        output = []
        for q in self.quotes:
            if q.verified:
                output.append(f'"{q.text}"\n— {q.speaker}\n[Timestamp: {q.timestamp}]')
        return '\n\n'.join(output)
```

Quote accuracy checklist

Before publishing any quote:

```
- [ ] Listened to original recording at timestamp
- [ ] Quote is verbatim (or clearly marked as paraphrased)
- [ ] Context preserved (not cherry-picked to change meaning)
- [ ] Speaker identified correctly
- [ ] Timestamp documented for fact-checker
- [ ] Source approved quote (if agreement made)
```

Source management database

Interview tracking schema

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional
from enum import Enum

class SourceStatus(Enum):
    ACTIVE = "active"           # Currently engaged
    DORMANT = "dormant"         # Not recently contacted
    DECLINED = "declined"       # Refused to participate
    OFF_RECORD = "off_record"   # Background only

class InterviewType(Enum):
    ON_RECORD = "on_record"
    BACKGROUND = "background"
    DEEP_BACKGROUND = "deep_background"
    OFF_RECORD = "off_record"

@dataclass
class Source:
    name: str
    organization: str
    contact_info: dict  # email, phone, signal, etc.
    beat: str
    status: SourceStatus = SourceStatus.ACTIVE
    interviews: List['Interview'] = field(default_factory=list)
    notes: str = ""

    # Relationship tracking
    first_contact: Optional[datetime] = None
    trust_level: int = 1  # 1-5 scale

@dataclass
class Interview:
    source: str
    date: datetime
    interview_type: InterviewType
    recording_path: Optional[str] = None
    transcript_path: Optional[str] = None
    story_slug: Optional[str] = None
    key_quotes: List[str] = field(default_factory=list)
    follow_up_needed: bool = False
    notes: str = ""
```

Quick source lookup

```
def find_sources_for_story(sources: List[Source], topic: str,
                           beat: str = None) -> List[Source]:
    """Find relevant sources for a new story."""
    matches = []
    for source in sources:
        # Filter by beat if specified
        if beat and source.beat != beat:
            continue
        # Only suggest active sources
        if source.status != SourceStatus.ACTIVE:
            continue
        # Check if they've spoken on similar topics
        for interview in source.interviews:
            if topic.lower() in interview.notes.lower():
                matches.append(source)
                break

    # Sort by trust level
    return sorted(matches, key=lambda s: s.trust_level, reverse=True)
```

Audio/video processing

Batch processing multiple recordings

```
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor
import json

def batch_transcribe(recordings_dir: str, output_dir: str) -> dict:
    """Process all recordings in a directory."""
    recordings = list(Path(recordings_dir).glob('*.wav')) + \
                 list(Path(recordings_dir).glob('*.mp3')) + \
                 list(Path(recordings_dir).glob('*.m4a'))

    results = {}

    with ProcessPoolExecutor(max_workers=4) as executor:
        futures = {
            executor.submit(transcribe_interview, str(rec), output_dir): rec
            for rec in recordings
        }

        for future in futures:
            rec = futures[future]
            try:
                transcript = future.result()
                results[rec.name] = {
                    'status': 'success',
                    'transcript': transcript
                }
            except Exception as e:
                results[rec.name] = {
                    'status': 'error',
                    'error': str(e)
                }

    return results
```

Video interview extraction

```
import subprocess

def extract_audio_from_video(video_path: str, output_path: str = None) -> str:
    """Extract audio track from video for transcription."""
    if output_path is None:
        output_path = video_path.rsplit('.', 1)[0] + '.wav'

    subprocess.run([
        'ffmpeg', '-i', video_path,
        '-vn',  # No video
        '-acodec', 'pcm_s16le',  # WAV format
        '-ar', '44100',  # Sample rate
        '-ac', '1',  # Mono
        output_path
    ], check=True)

    return output_path
```

Legal and ethical considerations

Consent documentation

```
## Recording consent record

**Date**:
**Source name**:
**Recording type**: [ ] Audio [ ] Video
**Interview type**: [ ] On record [ ] Background [ ] Off record

### Consent obtained:
- [ ] Verbal consent recorded at start of interview
- [ ] Written consent form signed
- [ ] Email confirmation of consent

### Jurisdiction notes:
- Interview location state/country:
- One-party or two-party consent jurisdiction:
- Any specific restrictions agreed:

### Agreed terms:
- [ ] Full attribution allowed
- [ ] Organization attribution only
- [ ] Anonymous source
- [ ] Review quotes before publication
- [ ] Embargo until [date]:
```

Recording-consent jurisdiction

For the per-state breakdown of one-party vs. all-party consent, hidden-recording rules, and federal preemption, use the **interview-prep** [[skill]] (which points to the Reporters Committee for Freedom of the Press *Reporter's Recording Guide* — the authoritative continuously-updated source).

Always get explicit consent on recording regardless of jurisdiction. Note the consent verbatim at the head of every transcript file (timestamp, speaker, response). This protects you legally everywhere and gives the fact-checker a clean starting point.

Tools and resources

Tool	Purpose	Notes
OpenAI Whisper	Local transcription, no diarization	Free, runs offline. `large-v3` is the current best model
WhisperX	Whisper + speaker diarization	`m-bain/whisperX` . Free. Word-level timestamps with speaker IDs. Needs a Hugging Face token for the pyannote model
faster-whisper	Speed-optimized Whisper	CTranslate2 backend. ~4x faster than vanilla Whisper at the same accuracy. Used internally by WhisperX
whisper.cpp	CPU-friendly Whisper port	C++ implementation. Runs the small/medium models on a Raspberry Pi
pyannote.audio	Standalone speaker diarization	Use directly when you already have transcripts from another source
MacWhisper / Buzz	GUI wrappers for Whisper	macOS / cross-platform GUIs for journalists who don't want a CLI
Otter.ai	Cloud transcription, real-time	Verify privacy posture before using with sensitive sources — Otter Pilot has historically joined meetings unannounced and indexed transcripts; check current settings
Descript	Edit audio like text	Good for pulling clips. Cloud-hosted
Rev (human + AI)	Human transcription for sensitive material	Slower, more accurate. Cloud-hosted
Trint	Journalist-focused, collaboration	Cloud-hosted. Has team features
oTranscribe	Free web-based manual transcription aid	Local-only (browser); no upload. Good for off-the-record material you can't hand to a cloud service

Related [[skills]]

- **interview-prep** — Pre-interview research, question design, consent scripts, and recording-law jurisdiction
- **source-verification** — Verify source credentials before interview
- **[[fact-check]]-workflow** — Verify quotes against the recording before [[[[publication]]]]
- **foia-requests** — Get documents to inform interview questions
- **data-journalism** — Analyze data sources mentioned in interviews
- **newsroom-style** — Convert verbatim quotes into AP-style copy for [[[[publication]]]]

[[Skill]] metadata

Field	Value
version	1.0.0
created	2025-12-26
updated	2026-05-08
author	Joe Amditis
domain	journalism, research
complexity	intermediate

investigating-insider-threat-indicators (Skill Module)

investigating-insider-threat-indicators\SKILL.md

name: investigating-insider-threat-indicators
description: Insider Threat Investigation Agent Automates insider threat indicator collection by correlating SIEM data, DLP alerts, access logs, and HR events to build investigation timelines. Use when the user asks to investigating insider threat indicators.
version: 1.0.0
tags:
source: investigating-insider-threat-indicators

Investigating Insider Threat Indicators

Role: Security workflow
Audience: security practitioners
Primary tools: Evidence Handling

When to use

- The user wants to **investigat** insider threat indicators.
- Capability: Insider Threat Investigation Agent Automates insider threat indicator collection by correlating SIEM data, DLP alerts, access logs, and HR events to build investigation timelines.
- Goal: complete the task.
- Trigger keywords: `investigating, insider, threat, indicators`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load DLP alert data from exported CSV
2. Load authentication and access logs from exported JSON
3. Analyze data exfiltration indicators for the subject
4. Detect anomalous access patterns for the subject
5. Identify pre-departure behavioral indicators
6. Create chain-of-custody evidence log with file hashes
7. Generate insider threat investigation report

Tools & resources

- Evidence Handling

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_dlp_alerts` — Load DLP alert data from exported CSV.
- `load_access_logs` — Load authentication and access logs from exported JSON.
- `analyze_data_movement` — Analyze data exfiltration indicators for the subject.
- `analyze_access_patterns` — Detect anomalous access patterns for the subject.
- `detect_pre_departure_indicators` — Identify pre-departure behavioral indicators.
- `create_evidence_log` — Create chain-of-custody evidence log with file hashes.
- `generate_report` — Generate insider threat investigation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `investigating-insider-threat-indicators`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

investigating-phishing-email-incident (Skill Module)

investigating-phishing-email-incident\SKILL.md

name: investigating-phishing-email-incident
description: Phishing Email Investigation Agent Analyzes reported phishing emails by parsing headers, checking URL/attachment reputation via VirusTotal and URLScan.io, and identifying impacted recipients. Use when the user asks to investigating phishing email incident.
version: 1.0.0
tags:
source: investigating-phishing-email-incident

Investigating Phishing Email Incident

Role: Security workflow
Audience: security practitioners
Primary tools: URLScan.io API, VirusTotal API v3, MalwareBazaar API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **investigat** phishing email incident.
- Capability: Phishing Email Investigation Agent Analyzes reported phishing emails by parsing headers, checking URL/attachment reputation via VirusTotal and URLScan.io, and identifying impacted recipients.
- Goal: complete the task.
- Trigger keywords: `investigating, phishing, email, incident`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse an .eml file and extract investigation-relevant headers
2. Extract all URLs from email body
3. Submit URL to URLScan.io for analysis
4. Check URL reputation on VirusTotal
5. Calculate MD5 and SHA-256 hashes for an attachment
6. Look up file hash on VirusTotal
7. Compile indicators of compromise from the investigation
8. Generate phishing investigation report

Tools & resources

- URLScan.io API
- VirusTotal API v3
- MalwareBazaar API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_email_headers` — Parse an .eml file and extract investigation-relevant headers.
- `extract_urls_from_email` — Extract all URLs from email body.
- `check_url_urlscan` — Submit URL to URLScan.io for analysis.
- `check_url_virustotal` — Check URL reputation on VirusTotal.
- `hash_attachment` — Calculate MD5 and SHA-256 hashes for an attachment.
- `check_hash_virustotal` — Look up file hash on VirusTotal.
- `generate_ioc_list` — Compile indicators of compromise from the investigation.
- `generate_report` — Generate phishing investigation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `investigating-phishing-email-incident`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

investigating-ransomware-attack-artifacts (Skill Module)

investigating-ransomware-attack-artifacts\SKILL.md

name: investigating-ransomware-attack-artifacts
description: Ransomware Attack Artifact Investigation Agent Collects and analyzes ransomware artifacts including ransom notes, encrypted file samples, registry modifications, and event logs to identify the variant, attack vector,.... Use when the user asks to investigating ransomware attack artifacts.
version: 1.0.0
tags:
source: investigating-ransomware-attack-artifacts

Investigating Ransomware Attack Artifacts

Role: Security workflow
Audience: security practitioners
Primary tools: VirusTotal API v3, ID Ransomware, No More Ransom Project, MalwareBazaar API

When to use

- The user wants to **investigat** ransomware attack artifacts.
- Capability: Ransomware Attack Artifact Investigation Agent Collects and analyzes ransomware artifacts including ransom notes, encrypted file samples, registry modifications, and event logs to identify the variant, attack vector, and encryption scope.
- Goal: complete the task.
- Trigger keywords: `investigating, ransomware, attack, artifacts`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Search filesystem for common ransom note filenames
2. Identify encrypted files by extension and calculate scope
3. Extract IOCs and payment details from ransom notes
4. Look up file hash on VirusTotal for ransomware identification
5. Parse exported Windows event log CSV for ransomware indicators
6. Generate ransomware investigation report

Tools & resources

- VirusTotal API v3
- ID Ransomware
- No More Ransom Project
- MalwareBazaar API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `collect_ransom_notes` — Search filesystem for common ransom note filenames.
- `identify_encrypted_files` — Identify encrypted files by extension and calculate scope.
- `analyze_ransom_note_content` — Extract IOCs and payment details from ransom notes.
- `check_hash_virustotal` — Look up file hash on VirusTotal for ransomware identification.
- `parse_windows_event_logs` — Parse exported Windows event log CSV for ransomware indicators.
- `generate_report` — Generate ransomware investigation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `investigating-ransomware-attack-artifacts`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

managing-cloud-identity-with-okta (Skill Module)

managing-cloud-identity-with-okta\SKILL.md

name: managing-cloud-identity-with-okta
description: Okta Cloud Identity Management Agent Audits Okta tenant configuration including SSO apps, MFA policies, lifecycle rules, and user provisioning status using the Okta Python SDK. Use when the user asks to managing cloud identity with okta.
version: 1.0.0
tags:
source: managing-cloud-identity-with-okta

Managing Cloud Identity with Okta

Role: Security workflow
Audience: security practitioners
Primary tools: Okta, Rate Limits

When to use

- The user wants to **mana** cloud identity with okta.
- Capability: Okta Cloud Identity Management Agent Audits Okta tenant configuration including SSO apps, MFA policies, lifecycle rules, and user provisioning status using the Okta Python SDK.
- Goal: complete the task.
- Trigger keywords: `managing, cloud, identity, okta`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Clarify the objective for cloud identity with okta
2. Gather inputs and required access
3. Execute the core workflow using skill scripts and references
4. Validate results and document findings
5. Recommend next actions

Tools & resources

- Okta
- Rate Limits

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_report` — Generate Okta identity audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `managing-cloud-identity-with-okta`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

managing-intelligence-lifecycle (Skill Module)

`managing-intelligence-lifecycle\SKILL.md`

name: managing-intelligence-lifecycle
description: Cyber Threat Intelligence Lifecycle Management Agent Manages the CTI lifecycle from requirements gathering through dissemination, tracking PIRs, collection sources, and intelligence product metrics. Use when the user asks to managing intelligence lifecycle.
version: 1.0.0
tags:
source: managing-intelligence-lifecycle

Managing Intelligence Lifecycle

Role: Security workflow
Audience: security practitioners
Primary tools: Recorded Future API, MISP REST API, OpenCTI GraphQL API

When to use

- The user wants to **mana** intelligence lifecycle.
- Capability: Cyber Threat Intelligence Lifecycle Management Agent Manages the CTI lifecycle from requirements gathering through dissemination, tracking PIRs, collection sources, and intelligence product metrics.
- Goal: complete the task.
- Trigger keywords: `managing, intelligence, lifecycle`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load Priority Intelligence Requirements (PIRs) from config
2. Evaluate intelligence collection source coverage and quality
3. Assess how well collection sources cover PIRs
4. Track intelligence products and dissemination metrics
5. Assess CTI program maturity using simplified FIRST CTI-SIG model
6. Generate CTI lifecycle management report

Tools & resources

- Recorded Future API
- MISP REST API
- OpenCTI GraphQL API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_intelligence_requirements` — Load Priority Intelligence Requirements (PIRs) from config.
- `evaluate_collection_sources` — Evaluate intelligence collection source coverage and quality.
- `assess_pir_coverage` — Assess how well collection sources cover PIRs.
- `track_intelligence_products` — Track intelligence products and dissemination metrics.
- `assess_maturity` — Assess CTI program maturity using simplified FIRST CTI-SIG model.
- `generate_report` — Generate CTI lifecycle management report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `managing-intelligence-lifecycle`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

mapping-mitre-attack-techniques (Skill Module)

mapping-mitre-attack-techniques\SKILL.md

name: mapping-mitre-attack-techniques
description: MITRE ATT&CK Technique Mapping Agent Maps detection rules and security alerts to ATT&CK techniques using the mitreattack-python library. Generates coverage heatmaps and identifies gaps. Use when the user asks to mapping mitre attack techniques.
version: 1.0.0
tags:
source: mapping-mitre-attack-techniques

Mapping MITRE ATT&CK Techniques

Role: Security workflow
Audience: security practitioners
Primary tools: MITRE ATT&CK, mitreattack-python Library, Key Libraries, Data Sources

When to use

- The user wants to **mapp** mitre attack techniques.
- Capability: MITRE ATT&CK Technique Mapping Agent Maps detection rules and security alerts to ATT&CK techniques using the mitreattack-python library. Generates coverage heatmaps and identifies gaps.
- Goal: complete the task.
- Trigger keywords: `mapping, mitre, attack, techniques`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load MITRE ATT&CK STIX data bundle
2. Retrieve all Enterprise ATT&CK techniques with metadata. Returns list[AttackPattern] (STIX objects supporting dict-like access)
3. Get techniques used by a specific threat group. Groups are IntrusionSet STIX objects; techniques retrieved via relationship query
4. Load detection rules with ATT&CK technique tags
5. Calculate ATT&CK coverage statistics by tactic
6. Generate ATT&CK Navigator JSON layer for visualization
7. Identify high-priority coverage gaps based on threat group activity
8. Generate ATT&CK mapping report

Tools & resources

- MITRE ATT&CK
- mitreattack-python Library
- Key Libraries
- Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_attack_data` — Load MITRE ATT&CK STIX data bundle.
- `get_all_techniques` — Retrieve all Enterprise ATT&CK techniques with metadata. Returns list[AttackPattern] (STIX objects supporting dict-like access).
- `get_techniques_by_group` — Get techniques used by a specific threat group. Groups are IntrusionSet STIX objects; techniques retrieved via relationship query.
- `load_detection_rules` — Load detection rules with ATT&CK technique tags.
- `calculate_coverage` — Calculate ATT&CK coverage statistics by tactic.
- `generate_navigator_layer` — Generate ATT&CK Navigator JSON layer for visualization.
- `identify_priority_gaps` — Identify high-priority coverage gaps based on threat group activity.
- `generate_report` — Generate ATT&CK mapping report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `mapping-mitre-attack-techniques`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

mobile-debugging (Skill Module)

dev-toolkit\skills\mobile-debugging\SKILL.md

name: mobile-debugging

description: Remote JavaScript console access and debugging on mobile devices. Use when debugging web pages on phones/tablets, accessing console errors without desktop DevTools, testing responsive designs on real devices, or diagnosing mobile-specific issues. Covers Eruda, vConsole, Chrome/Safari remote debugging, and cloud testing platforms.

tags:

Mobile debugging methodology

Patterns for accessing JavaScript console and debugging web pages on mobile devices without traditional desktop DevTools.

Quick-start: Inject console on any page

Eruda bookmarklet (recommended)

Add this as a bookmark on your mobile browser, then tap it on any page:

```
javascript:(function(){var
script=document.createElement('script');script.src='https://cdn.jsdelivr.net/npm/eruda';document.body.append(script);script.onload=function()
{eruda.init();}})();
```

vConsole bookmarklet

```
javascript:(function(){var
script=document.createElement('script');script.src='https://unpkg.com/vconsole@latest/dist/vconsole.min.js';document.body.append(script);script.onload=funct
ion(){new VConsole();}})();
```

In-page console tools

Eruda [[setup]]

Eruda provides a full DevTools-like experience in a floating panel. Eruda 3.x (3.4.3 current as of 2026-05) is the right baseline; it ships ES2020 syntax and assumes a modern mobile browser.

```
<!-- CDN (development only) -->
<script src="https://cdn.jsdelivr.net/npm/eruda"></script>
<script>eruda.init();</script>

<!-- Conditional loading (recommended for production) -->
<script>
(function() {
    var src = 'https://cdn.jsdelivr.net/npm/eruda';
    // Only load when ?eruda=true or localStorage flag set
    if (!/eruda=true/.test(window.location) &&
        localStorage.getItem('active-eruda') !== 'true') return;

    var script = document.createElement('script');
    script.src = src;
    script.onload = function() { eruda.init(); };
    document.body.appendChild(script);
})();
</script>
```

```
// NPM installation
// npm install eruda --save-dev

import eruda from 'eruda';

// Initialize with options
eruda.init({
    container: document.getElementById('eruda-container'),
    tool: ['console', 'elements', 'network', 'resources', 'info'],
    useShadowDom: true,
    autoScale: true
});

// Add custom buttons
eruda.add({
    name: 'Clear Storage',
    init($el) {
        $el.html('<button>Clear All Storage</button>');
        $el.find('button').on('click', () => {
            localStorage.clear();
            sessionStorage.clear();
            console.log('Storage cleared');
        });
    }
});

// Remove when done
eruda.destroy();
```

Eruda features:
- Console (logs, errors, warnings)
- Elements (DOM inspector)
- Network (XHR/fetch requests)
- Resources (localStorage, cookies, sessionStorage)
- Sources (page source code)
- Info (page/device information)
- Snippets (saved code snippets)

vConsole [[setup]]

Lighter weight alternative, official tool for WeChat debugging.

```
<!-- CDN -->
<script src="https://unpkg.com/vconsole@latest/dist/vconsole.min.js"></script>
<script>
var vConsole = new VConsole();
</script>
```

```
// NPM
// npm install vconsole

import VConsole from 'vconsole';

// Initialize with options
const vConsole = new VConsole({
    theme: 'dark',
    onReady: function() {
        console.log('vConsole is ready');
    },
    log: {
        maxLogNumber: 1000
    }
});

// Dynamic configuration
vConsole.setOption('log.maxLogNumber', 5000);

// Destroy when done
vConsole.destroy();
```

vConsole features:
- Log panel (console.log, info, warn, error)
- System panel (device info)
- Network panel (XHR, fetch)

- Element panel (DOM tree)
- Storage panel (cookies, localStorage)

Comparison: Eruda vs vConsole

Feature	Eruda	vConsole
Size	~100KB	~85KB
DOM Editing	Yes	View only
Network Details	Full	Basic
Plugin System	Yes	Yes
Dark Theme	Via plugin	Built-in
Best For	Full debugging	Quick logging

Native remote debugging

Chrome DevTools (Android)

```
# 1. Enable USB debugging on Android
#    Settings → Developer Options → USB Debugging = ON

# 2. Connect via USB to computer

# 3. Open Chrome on computer, navigate to:
#    chrome://inspect#devices

# 4. Enable "Discover USB devices"

# 5. Accept debugging prompt on Android device

# 6. Click "Inspect" next to the page you want to debug
```

Port forwarding for localhost:

```
# In chrome://inspect, click "Port forwarding"
# Add: localhost:3000 → localhost:3000
# Now Android Chrome can access your dev server at localhost:3000
```

Android 11+ wireless debugging (no USB needed):

```
# 1. On the Android device:
#    Settings → Developer Options → Wireless debugging = ON
#    Tap "Pair device with pairing code"
#    Note the IP:PORT and 6-digit code shown

# 2. On the computer (Android Platform Tools 30.0.0+):
adb pair <DEVICE_IP>:<PAIRING_PORT>
# Enter the 6-digit code when prompted

# 3. Connect to the debug port (different from pairing port):
adb connect <DEVICE_IP>:<DEBUG_PORT>

# 4. Verify and proceed to chrome://inspect#devices as usual:
adb devices
```

Wireless debugging persists across reboots once paired, but the `adb connect` step is needed each session.

Safari Web Inspector (iOS)

```
# 1. On iPhone/iPad:
#    Settings → Safari → Advanced → Web Inspector = ON

# 2. On Mac:
#    Safari → Preferences → Advanced → "Show Develop menu" = ON

# 3. Connect device via USB (or enable Wi-Fi debugging)

# 4. Open Safari on Mac:
#    Develop → [Device Name] → [Page to debug]

# Wireless debugging (after initial USB setup):
#    Develop → [Device] → Connect via Network
```

Firefox Remote Debugging (Android)

```
# 1. On Android Firefox:
#    Settings → Advanced → Remote debugging = ON

# 2. On Desktop Firefox:
#    Open about:debugging

# 3. Connect Android via USB

# 4. Enable USB devices in about:debugging

# 5. Click "Connect" next to your device
```

iOS debugging without Mac

Using ios-webkit-debug-proxy

```
# Install on Windows (via Scoop)
scoop bucket add extras
scoop install ios-webkit-debug-proxy

# Install on Linux
sudo apt-get install ios-webkit-debug-proxy

# Install on Mac
brew install ios-webkit-debug-proxy

# Run the proxy
ios_webkit_debug_proxy -f chrome-devtools://devtools/bundled/inspector.html

# Connect to http://localhost:9221 to see connected devices
```

Commercial: Inspect.dev

Inspect.dev provides iOS debugging from Windows/Linux with a familiar DevTools interface.

```
# Download from https://inspect.dev/
# 1. Install application
# 2. Connect iOS device via USB
# 3. Enable Web Inspector on iOS
# 4. Inspect.dev auto-detects pages
# 5. Click to open DevTools interface
```

Cloud testing platforms

LambdaTest (freemium)

```
# LambdaTest provides real device cloud with console access
# Free tier: 100 minutes/month

import requests

# LambdaTest REST API for automation
LAMBDATEST_API = "https://api.lambdatest.com/automation/api/v1"

# For manual testing:
# 1. Go to https://www.lambdatest.com/
# 2. Select device/browser
# 3. Enter URL
# 4. DevTools available in toolbar

# Selenium/Playwright integration for automated console capture
from playwright.sync_api import sync_playwright

def test_on_lambdatest():
    with sync_playwright() as p:
        # Connect to LambdaTest
        browser = p.chromium.connect(
            f"wss://cdp.lambdatest.com/playwright?capabilities="
            f"{{\"browserName\":\"Chrome\",\"platform\":\"android\"}}"
        )
        page = browser.new_page()

        # Capture console logs
        logs = []
        page.on('console', lambda msg: logs.append(msg.text()))

        page.goto('https://example.com')
        browser.close()

        return logs
```

BrowserStack

```
# BrowserStack: $29/month+, 10,000+ real devices
# Selenium 4 removed DesiredCapabilities — pass capabilities via Options instead.

from selenium import webdriver
from selenium.webdriver.chrome.options import Options

def get_browserstack_driver():
    """Create BrowserStack WebDriver with console logging."""

    options = Options()
    bstack_options = {
        'deviceName': 'Samsung Galaxy S21',
        'osVersion': '11.0',
        'realMobile': 'true',
        'consoleLogs': 'verbose',     # Capture console logs
        'networkLogs': 'true',
        'userName': 'YOUR_USERNAME',
        'accessKey': 'YOUR_KEY'
    }
    options.set_capability('bstack:options', bstack_options)
    options.set_capability('browserName', 'chrome')

    driver = webdriver.Remote(
        command_executor='https://hub-cloud.browserstack.com/wd/hub',
        options=options
    )

    return driver

# After test, retrieve logs from BrowserStack dashboard or API
```

Programmatic console capture

Playwright console capture

```
const { chromium, devices } = require('playwright');

async function captureConsoleLogs(url) {
    const browser = await chromium.launch();

    // Emulate mobile device. Playwright ships an updated devices map per
    // release; iPhone 15 / Pixel 8 are reasonable 2026 baselines. List with
    // `npx playwright devices` if you need an exact name.
    const context = await browser.newContext({
        ...devices['iPhone 15']
    });

    const page = await context.newPage();

    // Capture all console messages
    const logs = [];
    page.on('console', msg => {
        logs.push({
            type: msg.type(),
            text: msg.text(),
            location: msg.location(),
            timestamp: new Date().toISOString()
        });
    });

    // Capture page errors
    const errors = [];
    page.on('pageerror', error => {
        errors.push({
            message: error.message,
            stack: error.stack,
            timestamp: new Date().toISOString()
        });
    });

    // Capture failed requests
    const failedRequests = [];
    page.on('requestfailed', request => {
        failedRequests.push({
            url: request.url(),
            failure: request.failure().errorText,
            timestamp: new Date().toISOString()
        });
    });

    await page.goto(url);
    await page.waitForLoadState('networkidle');

    await browser.close();

    return { logs, errors, failedRequests };
}

// Usage
captureConsoleLogs('https://example.com')
    .then(result => console.log(JSON.stringify(result, null, 2)));
```

Puppeteer console capture

```
const puppeteer = require('puppeteer');

async function debugMobilePage(url) {
    const browser = await puppeteer.launch();
    const page = await browser.newPage();

    // Set mobile viewport
    await page.setViewport({
        width: 375,
        height: 812,
        isMobile: true,
        hasTouch: true
    });

    // Mobile user agent
    await page.setUserAgent(
        'Mozilla/5.0 (iPhone; CPU iPhone OS 14_0 like Mac OS X) ' +
        'AppleWebKit/605.1.15 (KHTML, like Gecko) Version/14.0 Mobile/15E148 Safari/604.1'
    );

    // Console capture with full details
    page.on('console', async msg => {
        const args = await Promise.all(
            msg.args().map(arg => arg.jsonValue().catch(() => arg.toString()))
        );

        console.log(`[${msg.type().toUpperCase()}]`, ...args);

        // Get source location
        const location = msg.location();
        if (location.url) {
            console.log(`  at ${location.url}:${location.lineNumber}`);
        }
    });

    // Unhandled promise rejections
    page.on('pageerror', err => {
        console.error('[PAGE ERROR]', err.message);
    });

    await page.goto(url, { waitUntil: 'networkidle0' });

    // Execute JavaScript and capture result
    const result = await page.evaluate(() => {
        // Check for common mobile issues
        return {
            viewportWidth: window.innerWidth,
            devicePixelRatio: window.devicePixelRatio,
            touchSupport: 'ontouchstart' in window,
            errors: window.__capturedErrors || []
        };
    });

    console.log('Page info:', result);

    await browser.close();
}
```

Error monitoring services

Sentry [[integration]]

```
// npm install @sentry/browser
// Sentry SDK v8+ uses functional integrations; class-based
// `new Sentry.BrowserTracing()` / `new Sentry.Replay()` were
// deprecated in v8 and removed in v9.

import * as Sentry from '@sentry/browser';

Sentry.init({
    dsn: 'YOUR_SENTRY_DSN',
    environment: 'production',

    integrations: [
        Sentry.browserTracingIntegration(),
        Sentry.replayIntegration()  // Session replay for debugging
    ],

    // Sample rates
    tracesSampleRate: 0.1,
    replaysSessionSampleRate: 0.1,
    replaysOnErrorSampleRate: 1.0,

    beforeSend(event) {
        // Filter or modify events
        return event;
    }
});

// Manual error capture
try {
    riskyOperation();
} catch (error) {
    Sentry.captureException(error);
}

// Add context (also functional in v8+)
Sentry.setUser({ id: 'user123' });
Sentry.setTag('page', 'checkout');
```

LogRocket for session replay

```
// npm install logrocket

import LogRocket from 'logrocket';

LogRocket.init('your-app/your-project');

// Identify user
LogRocket.identify('user123', {
    name: 'Test User',
    email: 'user@example.com'
});

// Console logs automatically captured
console.log('This appears in LogRocket');

// Manual logging
LogRocket.log('Custom event', { data: 'value' });

// Track errors
LogRocket.captureException(new Error('Something went wrong'));
```

Android screen mirroring with Scrcpy

```
# Install scrcpy
# Windows: scoop install scrcpy
# Mac: brew install scrcpy
# Linux: apt install scrcpy

# Basic mirroring
scrcpy

# With specific options
scrcpy --max-size 1024 --bit-rate 2M

# Wireless connection (after initial USB)
adb tcpip 5555
adb connect <device-ip>:5555
scrcpy

# Record session
scrcpy --record session.mp4

# Turn off device screen while mirroring
scrcpy --turn-screen-off
```

Mobile debugging workflow

```
                MOBILE DEBUGGING DECISION TREE


  Q: Do you have physical access to the device?

       YES: Can you connect via USB?

          Android: Use Chrome DevTools Remote
                    chrome://inspect#devices

          iOS: Have a Mac?

              YES: Use Safari Web Inspector

              NO: Use Inspect.dev or
                    ios-webkit-debug-proxy

       NO USB: Inject Eruda/vConsole via bookmarklet

  Q: Remote/production debugging?

       Add conditional Eruda loading
        (?eruda=true parameter)

       Set up Sentry/LogRocket for error monitoring

  Q: Automated testing?

       Playwright/Puppeteer with mobile emulation

       Cloud platforms (LambdaTest, BrowserStack)
```

Common mobile debugging issues

Touch events not firing

```
// Check if touch events are supported
eruda.init();
console.log('Touch support:', 'ontouchstart' in window);
console.log('Pointer events:', 'onpointerdown' in window);

// Debug touch events
document.addEventListener('touchstart', e => {
    console.log('touchstart', e.touches.length, 'touches');
}, { passive: true });

document.addEventListener('click', e => {
    console.log('click at', e.clientX, e.clientY);
});
```

Viewport issues

```
// Log viewport information
console.log('Viewport:', {
    innerWidth: window.innerWidth,
    innerHeight: window.innerHeight,
    outerWidth: window.outerWidth,
    outerHeight: window.outerHeight,
    devicePixelRatio: window.devicePixelRatio,
    orientation: screen.orientation?.type
});

// Check meta viewport
const viewport = document.querySelector('meta[name="viewport"]');
console.log('Viewport meta:', viewport?.content);
```

Performance debugging

```
// Check performance timing
const perf = performance.getEntriesByType('navigation')[0];
console.log('Page load timing:', {
    dns: perf.domainLookupEnd - perf.domainLookupStart,
    tcp: perf.connectEnd - perf.connectStart,
    request: perf.responseStart - perf.requestStart,
    response: perf.responseEnd - perf.responseStart,
    domParsing: perf.domInteractive - perf.responseEnd,
    domComplete: perf.domComplete - perf.domInteractive,
    total: perf.loadEventEnd - perf.navigationStart
});

// Check memory (Chrome only)
if (performance.memory) {
    console.log('Memory:', {
        usedJSHeapSize: (performance.memory.usedJSHeapSize / 1048576).toFixed(2) + ' MB',
        totalJSHeapSize: (performance.memory.totalJSHeapSize / 1048576).toFixed(2) + ' MB'
    });
}
```

Platform comparison

Tool	Cost	Platforms	[[Setup]] Difficulty	Best For
Eruda	Free	All browsers	Easy (bookmarklet)	Quick debugging
vConsole	Free	All browsers	Easy	WeChat apps
Chrome Remote	Free	Android only	Medium	Full DevTools
Safari Inspector	Free	iOS only	Easy (Mac required)	Full DevTools
Inspect.dev	Paid	iOS from any OS	Easy	iOS without Mac
LambdaTest	Freemium	All	Easy	Cloud testing
BrowserStack	Paid	All	Easy	Real devices
Sentry	Freemium	All	Medium	Error monitoring

monitoring-darkweb-sources (Skill Module)

monitoring-darkweb-sources\SKILL.md

name: monitoring-darkweb-sources
description: Dark Web Source Monitoring Agent Monitors dark web forums, paste sites, and ransomware leak sites for organizational asset mentions using commercial APIs and OSINT tools. Use when the user asks to monitoring darkweb sources.
version: 1.0.0
tags:
source: monitoring-darkweb-sources

Monitoring Darkweb Sources

Role: Security workflow
Audience: security practitioners
Primary tools: Dehashed API, Ransomware.live API, Rate Limits

When to use

- The user wants to **monitor** darkweb sources.
- Capability: Dark Web Source Monitoring Agent Monitors dark web forums, paste sites, and ransomware leak sites for organizational asset mentions using commercial APIs and OSINT tools.
- Goal: complete the task.
- Trigger keywords: `monitoring, darkweb, sources`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check Have I Been Pwned for domain breach exposure
2. Search paste site archives for organization mentions
3. Search Dehashed for exposed credentials matching domain
4. Check ransomware leak site intelligence feeds for organization mentions. Uses Ransomware.live API (public aggregator)
5. Generate dark web monitoring report

Tools & resources

- Dehashed API
- Ransomware.live API
- Rate Limits

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_breach_exposure` — Check Have I Been Pwned for domain breach exposure.
- `search_paste_sites` — Search paste site archives for organization mentions.
- `check_credential_exposure` — Search Dehashed for exposed credentials matching domain.
- `monitor_ransomware_leak_sites` — Check ransomware leak site intelligence feeds for organization mentions. Uses Ransomware.live API (public aggregator).
- `generate_monitoring_report` — Generate dark web monitoring report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `monitoring-darkweb-sources`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

monitoring-scada-modbus-traffic-anomalies (Skill Module)

monitoring-scada-modbus-traffic-anomalies\SKILL.md

name: monitoring-scada-modbus-traffic-anomalies
description: Modbus TCP Traffic Anomaly Detector - Monitors SCADA networks for suspicious Modbus activity. Use when the user asks to monitoring scada modbus traffic anomalies.
version: 1.0.0
tags:
source: monitoring-scada-modbus-traffic-anomalies

Monitoring Scada Modbus Traffic Anomalies

Role: Security workflow
Audience: security practitioners
Primary tools: Arguments, Key Classes, Key Functions, Alert Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **monitor** scada modbus traffic anomalies.
- Capability: Modbus TCP Traffic Anomaly Detector - Monitors SCADA networks for suspicious Modbus activity.
- Goal: complete the task.
- Trigger keywords: `monitoring, scada, modbus, traffic, anomalies`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse 7-byte Modbus Application Protocol header from raw TCP payload
2. Parse Modbus PDU to extract function code, register addresses, and values
3. Analyze a pcap file for Modbus TCP anomalies using Scapy
4. Capture and analyze Modbus TCP traffic in real-time using Scapy

Tools & resources

- Arguments
- Key Classes
- Key Functions
- Alert Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_mbap_header` — Parse 7-byte Modbus Application Protocol header from raw TCP payload.
- `parse_modbus_pdu` — Parse Modbus PDU to extract function code, register addresses, and values.
- `analyze_pcap` — Analyze a pcap file for Modbus TCP anomalies using Scapy.
- `live_capture` — Capture and analyze Modbus TCP traffic in real-time using Scapy.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `monitoring-scada-modbus-traffic-anomalies`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

newsletter-publishing (Skill Module)

journalism-core\skills\newsletter-publishing\SKILL.md

name: newsletter-publishing

description: Email newsletter workflows for journalists and researchers. Use when creating, managing, or optimizing email newsletters, building subscriber lists, designing email templates, analyzing engagement metrics, or planning newsletter content calendars. For independent journalists, academic communicators, and media organizations building direct audience relationships.

tags:

[[Newsletter]] publishing

Practical
[[[

Issue structure

```
## [Newsletter Name] - Issue #[XX]
**Date**: [Date]
**Subject line**: [Subject]
**Preview text**: [First 50-90 characters readers see]

---

### Opening hook
[2-3 sentences that make readers want to keep reading]

### Main story
[Your primary content - 300-600 words for most newsletters]

### Secondary items (if applicable)
- **Quick hit 1**: [Brief item with link]
- **Quick hit 2**: [Brief item with link]

### Recurring section
[Weekly column, data point, recommendation, etc.]

### Sign-off
[Personal note, call to action, or preview of next issue]

---

**Unsubscribe** | **Preferences** | **Forward to a friend**
```

Technical implementation

HTML email

[[[

(responsive)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{newsletter_name}}</title>
  <style>
    /* Reset styles for email clients */
    body { margin: 0; padding: 0; width: 100%; }
    table { border-collapse: collapse; }
    img { border: 0; display: block; }

    /* Responsive container */
    .container {
      max-width: 600px;
      margin: 0 auto;
      font-family: Georgia, serif;
      font-size: 18px;
      line-height: 1.6;
      color: #333;
    }

    /* Dark mode support */
    @media (prefers-color-scheme: dark) {
      .container { background-color: #1a1a1a; color: #e0e0e0; }
      a { color: #6db3f2; }
    }

    /* Mobile styles */
    @media only screen and (max-width: 480px) {
      .container { padding: 15px !important; }
      h1 { font-size: 24px !important; }
    }
  </style>
</head>
<body>
  <table role="presentation" width="100%">
    <tr>
      <td align="center" style="padding: 20px;">
        <div class="container">
          <!-- Header -->
          <table width="100%">
            <tr>
              <td style="padding-bottom: 20px; border-bottom: 2px solid #333;">
                <h1 style="margin: 0;">{{newsletter_name}}</h1>
                <p style="margin: 5px 0 0; color: #666;">{{issue_date}}</p>
              </td>
            </tr>
          </table>

          <!-- Content -->
          <table width="100%">
            <tr>
              <td style="padding: 30px 0;">
                {{content}}
              </td>
            </tr>
          </table>

          <!-- Footer -->
          <table width="100%">
            <tr>
              <td style="padding-top: 20px; border-top: 1px solid #ddd; font-size: 14px; color: #666;">
                <p>You're receiving this because you subscribed to {{newsletter_name}}.</p>
                <p>
                  <a href="{{unsubscribe_url}}">Unsubscribe</a> |
                  <a href="{{preferences_url}}">Update preferences</a>
                </p>
              </td>
            </tr>
          </table>
        </div>
      </td>
    </tr>
  </table>
```

```
</body>
</html>
```

Python [[newsletter]] sender

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Dict, Optional
from enum import Enum
import hashlib

class SubscriberStatus(Enum):
    ACTIVE = "active"
    UNSUBSCRIBED = "unsubscribed"
    BOUNCED = "bounced"
    COMPLAINED = "complained"

@dataclass
class Subscriber:
    email: str
    name: Optional[str] = None
    subscribed_at: datetime = field(default_factory=datetime.now)
    status: SubscriberStatus = SubscriberStatus.ACTIVE
    tags: List[str] = field(default_factory=list)
    custom_fields: Dict = field(default_factory=dict)

    @property
    def hash_id(self) -> str:
        """Generate unique ID for unsubscribe links."""
        return hashlib.md5(self.email.encode()).hexdigest()[:12]

@dataclass
class NewsletterIssue:
    subject: str
    preview_text: str
    html_content: str
    plain_text: str
    scheduled_at: Optional[datetime] = None
    sent_at: Optional[datetime] = None
    issue_number: int = 0

    # Metrics
    sent_count: int = 0
    delivered_count: int = 0
    opened_count: int = 0
    clicked_count: int = 0
    bounced_count: int = 0
    unsubscribed_count: int = 0

    @property
    def open_rate(self) -> float:
        if self.delivered_count == 0:
            return 0.0
        return (self.opened_count / self.delivered_count) * 100

    @property
    def click_rate(self) -> float:
        if self.delivered_count == 0:
            return 0.0
        return (self.clicked_count / self.delivered_count) * 100

class NewsletterManager:
    """Core newsletter operations."""

    def __init__(self, name: str):
        self.name = name
        self.subscribers: List[Subscriber] = []
        self.issues: List[NewsletterIssue] = []

    def add_subscriber(self, email: str, name: str = None,
                       tags: List[str] = None) -> Subscriber:
        """Add new subscriber with double opt-in pending."""
        sub = Subscriber(
            email=email.lower().strip(),
            name=name,
            tags=tags or []
        )
        self.subscribers.append(sub)
        return sub

    def segment_subscribers(self, tags: List[str] = None,
```

```
                          min_engagement: float = None) -> List[Subscriber]:
        """Get subscribers matching criteria."""
        active = [s for s in self.subscribers
                  if s.status == SubscriberStatus.ACTIVE]

        if tags:
            active = [s for s in active
                      if any(t in s.tags for t in tags)]

        return active

    def calculate_engagement_score(self, subscriber: Subscriber) -> float:
        """Score subscriber engagement 0-100."""
        # Implementation would track opens/clicks per subscriber
        return 50.0  # Placeholder
```

Subscriber management

List hygiene workflow

```
from datetime import datetime, timedelta

def clean_subscriber_list(manager: NewsletterManager,
                          inactive_threshold_days: int = 180) -> dict:
    """Identify and handle inactive subscribers."""
    cutoff = datetime.now() - timedelta(days=inactive_threshold_days)

    results = {
        'total': len(manager.subscribers),
        'active': 0,
        'inactive': [],
        'bounced': [],
        'unsubscribed': []
    }

    for sub in manager.subscribers:
        if sub.status == SubscriberStatus.BOUNCED:
            results['bounced'].append(sub.email)
        elif sub.status == SubscriberStatus.UNSUBSCRIBED:
            results['unsubscribed'].append(sub.email)
        elif sub.status == SubscriberStatus.ACTIVE:
            # Check last engagement
            engagement = manager.calculate_engagement_score(sub)
            if engagement < 10:  # Very low engagement
                results['inactive'].append(sub.email)
            else:
                results['active'] += 1

    return results

def run_reengagement_campaign(inactive_subscribers: List[str]) -> None:
    """Send win-back campaign to inactive subscribers."""
    # Send "We miss you" campaign
    # If no engagement after 2 attempts, mark for removal
    pass
```

Subscriber segmentation

```
## Recommended segments

### By engagement
- **VIPs**: Open rate > 80%, always click
- **Engaged**: Open rate 40-80%
- **Casual**: Open rate 10-40%
- **At-risk**: Haven't opened in 90 days
- **Inactive**: Haven't opened in 180 days

### By interest (tag-based)
- Topic preferences from signup
- Content they've clicked
- Surveys/polls they've answered

### By source
- Organic (website signup)
- Referral (forwarded by friend)
- Social media
- Paywall/registration wall
```

Subject line optimization

High-performing patterns

```
## Subject line formulas that work

### For news/journalism
- **Breaking format**: "Breaking: [Concise news]"
- **Numbers**: "[X] things we learned about [topic]"
- **Question**: "Why did [entity] do [thing]?"
- **Direct**: "[Topic]: What you need to know"

### For analysis/opinion
- **Take**: "The real story behind [event]"
- **Contrarian**: "Why everyone is wrong about [topic]"
- **Insider**: "What [industry] insiders know about [topic]"

### What to avoid
- ALL CAPS
- Excessive punctuation!!!
- Clickbait that doesn't deliver
- Spam trigger words (FREE, URGENT, ACT NOW)
- Misleading preview text
```

A/B testing framework

```
import random
from typing import List, Tuple

def ab_test_subject_lines(subscribers: List[Subscriber],
                          subject_a: str,
                          subject_b: str,
                          test_percentage: float = 0.2) -> dict:
    """
    Test two subject lines on subset before full send.
    """
    test_size = int(len(subscribers) * test_percentage)
    test_group = random.sample(subscribers, test_size)

    # Split test group
    half = len(test_group) // 2
    group_a = test_group[:half]
    group_b = test_group[half:]

    remaining = [s for s in subscribers if s not in test_group]

    return {
        'group_a': {
            'subject': subject_a,
            'subscribers': group_a,
            'size': len(group_a)
        },
        'group_b': {
            'subject': subject_b,
            'subscribers': group_b,
            'size': len(group_b)
        },
        'remaining': {
            'subscribers': remaining,
            'size': len(remaining),
            'note': 'Send winner to this group after test period'
        },
        'test_duration_hours': 4
    }
```

Deliverability best practices

Email [[authentication]] [[setup]]

```
## DNS records for deliverability

### SPF record
```

v=spf1 include:_spf.{{esp_sending_domain}} ~all

```

### DKIM
- Generate keys through your ESP
- Add TXT record with public key
- Verify signature is applied to outgoing mail

### DMARC
```

v=DMARC1; p=quarantine; rua=mailto:dmarc@yourdomain.com

```

### Checklist before sending
- [ ] SPF, DKIM, DMARC configured
- [ ] Sending domain warmed up
- [ ] List is clean (no hard bounces)
- [ ] Unsubscribe link works
- [ ] Physical address in footer (CAN-SPAM)
- [ ] Test email received in inbox (not spam)
```

Gmail, Yahoo, and Outlook bulk-sender requirements

Bulk senders must meet [[authentication]] and unsubscribe requirements introduced by Gmail and Yahoo in February 2024. Microsoft Outlook adopted a parallel set in May 2025. Since November 2025, Gmail rejects non-compliant mail with permanent 5xx errors rather than soft-deferring it — non-compliant newsletters now bounce

hard.

Who is covered. A "bulk sender" is one mailing more than 5,000 messages per day to Gmail addresses. The 5,000 threshold is counted at the **primary domain level across all subdomains**, not per sending subdomain. A newsroom sending 2,500/day from `transactional.example.com` and 2,500/day from `news.example.com` is over the threshold.

Required:

- **SPF and DKIM [[authentication]] on the sending domain.** Both must pass. SPF alone is no longer sufficient. DKIM keys must be at least 1024 bits.
- **DMARC policy at minimum `p=none`.** Production senders should move to `p=quarantine` or `p=reject` once aligned.
- **Domain alignment. One** of SPF or DKIM must align with the organizational domain in the visible `From:` header — not both. Relaxed alignment is acceptable.
- **One-click unsubscribe (RFC 8058).** The mail must include a `List-Unsubscribe` header with an HTTPS URL and a `List-Unsubscribe-Post: List-Unsubscribe=One-Click` header. The HTTPS endpoint must process the unsubscribe within two days without requiring login. A visible unsubscribe link must also appear in the message body.
- **Spam complaint rate below 0.3 percent**, measured in Google Postmaster Tools. Google's recommended target ceiling is 0.1 percent; sustained rates above 0.3 percent trigger rejection.
- **Valid PTR record (reverse DNS) on the sending IP.** Forward and reverse DNS must match.
- **TLS for inbound connections** (Google requirement since December 2023).

Operational implications:

Most reputable ESPs handle [[authentication]], headers, and TLS once the sending domain is verified. The two parts that remain the operator's responsibility are complaint rate and unsubscribe behavior — re-engagement campaigns and prompt list hygiene matter here. Re-engaging dormant subscribers is risky precisely because they complain at 5–10× the rate of active ones; one bad re-engagement campaign can push complaint rate over 0.3 percent and trigger rejections across the entire sending domain.

References:
- Google, *Email sender guidelines* — `support.google.com/mail/answer/81126`
- Google, *Email sender guidelines FAQ* — `support.google.com/a/answer/14229414`
- Yahoo, *Sender Best Practices* — `senders.yahooinc.com/best-practices/`

Spam score checklist

```
## Before you send

### Content checks
- [ ] No spam trigger words
- [ ] Text-to-image ratio good (mostly text)
- [ ] All links are to reputable domains
- [ ] No URL shorteners (use full links)
- [ ] Plain text version included

### Technical checks
- [ ] From address matches sending domain
- [ ] Reply-to address is monitored
- [ ] Preheader text is set
- [ ] Images have alt text
- [ ] Links are not broken
```

Analytics and optimization

Key metrics [[dashboard]]

```
from dataclasses import dataclass

@dataclass
class NewsletterAnalytics:
    """Track newsletter performance over time."""

    issue: NewsletterIssue

    def summary(self) -> dict:
        return {
            'issue_number': self.issue.issue_number,
            'sent': self.issue.sent_count,
            'delivered': self.issue.delivered_count,
            'delivery_rate': self._pct(self.issue.delivered_count,
                                       self.issue.sent_count),
            'opens': self.issue.opened_count,
            'open_rate': self.issue.open_rate,
            'clicks': self.issue.clicked_count,
            'click_rate': self.issue.click_rate,
            'click_to_open': self._pct(self.issue.clicked_count,
                                       self.issue.opened_count),
            'unsubscribes': self.issue.unsubscribed_count,
            'unsubscribe_rate': self._pct(self.issue.unsubscribed_count,
                                          self.issue.delivered_count),
        }

    def _pct(self, numerator: int, denominator: int) -> float:
        if denominator == 0:
            return 0.0
        return round((numerator / denominator) * 100, 2)

# Benchmarks (journalism newsletters)
BENCHMARKS = {
    'open_rate': {'good': 40, 'excellent': 55},
    'click_rate': {'good': 4, 'excellent': 8},
    'unsubscribe_rate': {'acceptable': 0.5, 'concerning': 1.0},
}
```

Platform comparison

Platform	Best for	Pricing model	Key feature
Substack	Writer-first, paid subs	Revenue share	Built-in payments
Buttondown	Developers, minimal	Per subscriber	Markdown native
Ghost	Publishers, memberships	Flat fee	Full CMS included
beehiiv	Growth-focused	Freemium	Referral tools
Kit (formerly ConvertKit)	Creators	Per subscriber	Automation
Mailchimp	Small orgs	Tiered	Easy templates

Legal compliance

CAN-SPAM requirements (US)

```
- [ ] Accurate "From" name and email
- [ ] Non-deceptive subject line
- [ ] Physical postal address included
- [ ] Working unsubscribe mechanism
- [ ] Unsubscribe honored within 10 days
- [ ] No purchased lists
```

GDPR requirements (EU subscribers)

```
- [ ] Explicit consent obtained (not pre-checked)
- [ ] Clear privacy policy linked
- [ ] Easy unsubscribe process
- [ ] Data export available on request
- [ ] Data deletion on request
- [ ] Record of consent stored
```

Related [[skills]]

- **web-scraping** — Automate content gathering for newsletters
- **data-journalism** — Include data visualizations in emails
- **academic-writing** — Write clear, structured content
- **newsroom-style** — AP Style enforcement on [[newsletter]] copy
- **[[fact-check]]-workflow** — Verify claims before they hit subscribers' inboxes
- **ai-writing-detox** — Strip AI patterns from drafts

[[Skill]] metadata

Field	Value
version	1.0.0
created	2025-12-26
updated	2026-05-08
author	Joe Amditis
domain	publishing, marketing
complexity	intermediate

newsroom-style (Skill Module)

`journalism-core\skills\newsroom-style\SKILL.md`

name: newsroom-style
description: Enforce AP Style and newsroom conventions for journalism writing. Use when writing news articles, editing drafts, creating headlines, or converting notes into publishable copy. Ensures professional standards for attribution, numbers, dates, and formatting.
tags:

Newsroom style guide

Write like a professional journalist. Follow AP Style. Keep it tight.

When to use

- Writing news articles, briefs, or headlines
- Editing drafts for [[[[publication]]]]
- Converting interview notes into publishable copy
- Writing press releases or media advisories
- Creating social media posts for news content

Core AP Style rules

Numbers

Rule	Example
Spell out one through nine	"three witnesses" not "3 witnesses"
Use numerals for 10 and above	"15 people attended"
Always use numerals for ages	"a 5-year-old girl"
Always use numerals for percentages	AP allows the `%` symbol with numerals (`5%`) since 2019; some publications still spell out "percent" as house style
Always use numerals for addresses	"123 Main St."
Always use numerals for money	"$5 million" not "five million dollars"
Spell out first through ninth for ordinals	"first place" but "10th anniversary"

Exception: Never start a sentence with a numeral. Rewrite or spell out.

Titles and names

Rule	Example
Capitalize formal titles before names	"Mayor Jane Smith said..."
Lowercase titles after names	"Jane Smith, the mayor, said..."
Lowercase titles standing alone	"The mayor said..."
No courtesy titles on second reference	First: "Jane Smith." Second: "Smith"
Use full name on first reference	"Jane Smith" not "Smith" or "Ms. Smith"

Exceptions:
- Use courtesy titles in obituaries
- Some publications use them for all subjects (house style)

Attribution

Do	Don't
"said"	"stated," "remarked," "noted," "expressed"
Attribution after quote	Attribution before quote
At first natural pause	At awkward break

Examples:

Bad: Mayor Smith stated, "We are committed to this project."

Good: "We are committed to this project," Mayor Smith said.

Good: "We are committed to this project," said Mayor Jane Smith, who has pushed for the development since 2022.

Dates and times

Months:
- Abbreviate: Jan., Feb., Aug., Sept., Oct., Nov., Dec.
- Don't abbreviate: March, April, May, June, July
- Only abbreviate with specific dates: "Sept. 15" but "September 2025"

Times:
- Use figures with a.m./p.m.: "9 a.m." not "9:00 a.m."
- Noon and midnight: Use the words, not "12 p.m." or "12 a.m."
- Time ranges: "9 a.m. to 5 p.m." or "9-11 a.m."

Days:
- Capitalize days of the week
- Don't abbreviate except in tables
- Use "Tuesday" not "on Tuesday" when possible

Common word choices

Use	Instead of
more than	over (for quantities)
fewer	less (for countable items)
less	fewer (for mass nouns)
that	which (for restrictive clauses)
which	that (for nonrestrictive clauses)
said	stated, remarked, noted
about	approximately
because	due to the fact that

Restrictive vs. nonrestrictive clauses

Restrictive (essential, use "that"):

> *The bill that passed yesterday includes tax cuts.*

(Which bill? The one that passed yesterday.)

Nonrestrictive (extra info, use "which" + comma):

> *The bill, which passed yesterday, includes tax cuts.*

(We already know which bill; the passage is extra info.)

Formatting rules

Paragraphs

- Keep paragraphs short: 1-3 sentences
- One idea per paragraph
- Break up long quotes into multiple paragraphs

Headlines

- Use sentence case, not title case
- No periods at the end
- Use present tense for past events: "Council approves budget"
- Use infinitive for future events: "Mayor to announce plan"
- No exclamation points in hard news

Quotation marks

- Periods and commas always go inside
- Colons and semicolons always go outside
- Question marks and exclamation points: inside if part of quote, outside if not

Abbreviations

- Spell out on first reference, then abbreviate
- Don't use periods in most abbreviations: FBI, CIA, NATO
- State abbreviations: Use postal codes (NY, CA) only with full addresses; otherwise use AP abbreviations or spell out

Ledes (opening paragraphs)

The inverted pyramid

Most important information first. Answer the key questions:
- **Who** is involved?
- **What** happened?
- **When** did it happen?
- **Where** did it happen?
- **Why** did it happen? (if known)
- **How** did it happen? (if relevant)

Lede length

- Aim for 35 words or fewer
- One sentence is ideal
- Focus on the news, not background

Too long:

> *The city council, which has been debating the issue for several months and heard from dozens of residents at multiple public meetings, voted Tuesday night to approve a controversial new zoning ordinance that would allow high-rise buildings in the downtown area.*

Better:

> *The city council approved a zoning ordinance Tuesday that allows high-rise buildings downtown, ending months of debate.*

Types of ledes

Hard news lede:

> *A fire killed three people in Northeast Philadelphia early Tuesday.*

Feature lede:

> *The last time Maria Rodriguez saw her mother, she promised to bring her to America.*

Question lede (use sparingly):

> *What happens when a city runs out of money?*

Quick reference card

Before you publish

- [ ] Names spelled correctly and verified
- [ ] Titles correct and styled properly
- [ ] Numbers follow AP Style
- [ ] Attribution uses "said"
- [ ] Dates and times formatted correctly
- [ ] Paragraphs are short
- [ ] Lede is under 35 words
- [ ] No editorializing in news copy
- [ ] Sources are credible and named

Red flags

- "Very" or "extremely" in news copy
- Exclamation points

- First-person pronouns (unless first-person piece)
- Unattributed opinions
- Passive voice hiding who did what
- Starting sentences with "There is" or "There are"

Example transformations

Before (informal notes):

> *Yesterday the Mayor said that he was "very excited" about the new $5,000,000 project that will create over 100 jobs.*

After (AP Style):

> *Mayor John Smith said Tuesday he was "very excited" about the $5 million project, which will create more than 100 jobs.*

Before:

> *The meeting started at 9:00 AM on Monday, October 14th, 2024.*

After:

> *The meeting began at 9 a.m. Monday, Oct. 14, 2024.*

Before:

> *5 protesters were arrested at the rally.*

After:

> *Five protesters were arrested at the rally.*

Before:

> *Smith stated that he believed the project would be "important."*

After:

> *Smith said he believed the project would be "important."*

House style notes

Different publications have their own style guides that override AP Style. Common variations:

- Oxford comma (AP doesn't use it; many publications do — verify with your editor)
- Courtesy titles (AP doesn't use them; NYT does)
- Web style (AP merged "web site" to "website" in 2010; some legacy guides still split it)

Always ask about house style before writing for a new [[[[publication]]]].

Based on the Associated Press Stylebook. Check the current AP Stylebook (apstylebook.com) for updates and edge cases.

one-way-door (Skill Module)

dev-toolkit\skills\one-way-door\SKILL.md

name: one-way-door

description: Use this skill when creating new files that represent architectural decisions — data models, infrastructure configs, auth boundaries, API contracts, CI/CD pipelines, or event systems. Flags irreversible decisions and forces a discussion about trade-offs before committing.

tags:

One-way door check

Some decisions are easy to reverse — you can change a UI component, rename a variable, or swap a utility function with no lasting consequences. These are **two-way doors**: walk through, and if it's wrong, walk back.

Other decisions create gravity. Once traffic, users, or other code depends on them, changing course gets expensive. A database schema migration after launch. An API contract that external consumers rely on. An auth boundary that shapes your entire permission model. These are **one-way doors**.

The most expensive mistakes in software aren't bugs. They're irreversible architectural decisions made too quickly.

What gets flagged

Data models and database schemas

Files matching: `schema.prisma`, `schema.graphql`, `*.sql`, `migration*`, `models.py`, `models.ts`, `entities.py`, `entities.ts`

Data models are the hardest decisions to reverse. Once your database has rows, every schema change requires a migration. Column renames break queries. Relationship changes cascade through your entire application.

Questions to ask:
- Have you mapped all the relationships between entities?
- Will this schema support the queries you need without N+1 problems?
- Are you normalizing appropriately for your read/write patterns?

Infrastructure and deployment configs

Files matching: `docker-compose*`, `Dockerfile`, `*.tf`, `terraform*`, `pulumi*`, `cdk*`, `cloudformation*`, `k8s*`, `kubernetes*`, `helm*`

Infrastructure choices constrain everything built on top of them. Switching from ECS to Kubernetes, or from Lambda to containers, affects deployment pipelines, monitoring, scaling, and team knowledge.

Questions to ask:
- Is this the simplest infrastructure that meets your needs?
- What's your team's operational experience with this stack?
- What does failure recovery look like?

[[Authentication]] and authorization

Files matching: `auth.ts`, `auth.js`, `auth.py`, `firestore.rules`, `storage.rules`, `*.rules`, `rbac*`, `permissions*`, `security*`

Auth boundaries are load-bearing walls. Session vs JWT, role-based vs attribute-based, single-tenant vs multi-tenant — each choice shapes your security model, user experience, and compliance posture.

Questions to ask:
- Does this cover all your user types and access patterns?
- How will you handle token refresh, session expiry, and revocation?
- Are you building for single-tenant or multi-tenant from the start?

API contracts and service interfaces

Files matching: `openapi*`, `swagger*`, `*.proto`, `*.graphql`, `api-schema*`, `routes.ts`, `routes.js`, `routes.py`

Published APIs are promises to consumers. Breaking changes require versioning, deprecation periods, and migration guides. Internal APIs between services create coupling that's hard to unwind.

Questions to ask:
- Who will consume this API? Internal services, external developers, or both?
- How will you version breaking changes?
- Are you exposing implementation details that should stay private?

[[[[Event]]]] systems and message buses

Files matching: `events.ts`, `eventbus.ts`, `eventemitter.py`, `eventhandler.py`, `pubsub*`, `queue*`, `kafka*`, `rabbit*`

[[[[Event]]]] schemas are contracts between producers and consumers. Once multiple services subscribe to an [[[[event]]]], changing its shape requires coordinated deploys. [[[[Event]]]] ordering assumptions become architectural constraints.

Questions to ask:
- Have you defined the [[[[event]]]] schema, including required vs optional fields?
- What happens when a consumer fails to process an [[[[event]]]]?
- Do you need ordering guarantees?

CI/CD pipelines

Files in: `.github/`, `.gitlab/`, `.circleci/`, or matching `Jenkinsfile`, `.travis.yml`, `cloudbuild*`

CI/CD pipelines become the backbone of your release process. Teams build muscle memory around deploy [[[[[

Option 2: PreToolUse hook (automated enforcement)

Add this to your Claude Code `settings.json`:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "/path/to/one-way-door-check.sh"
          }
        ]
      }
    ]
  }
}
```

The hook script

```
#!/bin/sh
# One-way door check hook (PreToolUse:Write)
# Flags architectural decisions that are hard to reverse.

INPUT=$(cat)
[ -z "$INPUT" ] && exit 0

# Extract the file path from tool_input
FILE_PATH=$(echo "$INPUT" | grep -oP '"file_path"\s*:\s*"[^"]*"' | head -1 | sed 's/.*"file_path"\s*:\s*"//;s/"//')
[ -z "$FILE_PATH" ] && exit 0

FILENAME=$(basename "$FILE_PATH")
FILENAME_LOWER=$(echo "$FILENAME" | tr "[:upper:]" "[:lower:]")
DIR=$(dirname "$FILE_PATH")

ONE_WAY=0
REASON=""

# Database schemas and migrations
if echo "$FILENAME_LOWER" | grep -qE "schema\.(prisma|graphql|sql)|migration|\.sql$|models?\.(py|ts|js)$|entities?\.(py|ts|js)$"; then
    ONE_WAY=1
    REASON="data model / database schema"
fi

# Infrastructure and deployment configs
if echo "$FILENAME_LOWER" | grep -qE "^(docker-compose|dockerfile|terraform|pulumi|cdk)|\.tf$|cloudformation|k8s|kubernetes|helm"; then
    ONE_WAY=1
    REASON="infrastructure / deployment config"
fi

# Authentication and authorization
if echo "$FILENAME_LOWER" | grep -qE "auth\.(ts|js|py)|firestore\.rules|storage\.rules|security|\.rules$|rbac|permissions"; then
    ONE_WAY=1
    REASON="auth / security rules"
fi

# API contracts and service interfaces
if echo "$FILENAME_LOWER" | grep -qE "openapi|swagger|\.proto$|\.graphql$|api-schema|routes\.(ts|js|py)$"; then
    ONE_WAY=1
    REASON="API contract / service interface"
fi

# Event systems and message queues
if echo "$FILENAME_LOWER" | grep -qE "event(s|bus|emitter|handler)\.(ts|js|py)$|pubsub|queue|kafka|rabbit"; then
    ONE_WAY=1
    REASON="event system / message bus"
fi

# Package manager configs (dependency choices)
if echo "$FILENAME_LOWER" | grep -qE "^(package\.json|cargo\.toml|go\.mod|requirements\.txt|pyproject\.toml|gemfile)$"; then
    ONE_WAY=1
    REASON="dependency / package config"
fi

# Firebase and cloud service configs
if echo "$FILENAME_LOWER" | grep -qE "^firebase\.json$|^\.firebaserc$|firestore\.indexes"; then
    ONE_WAY=1
    REASON="cloud service config (Firebase)"
fi

# CI/CD pipelines
if echo "$DIR" | grep -qE "\.(github|gitlab|circleci)" || echo "$FILENAME_LOWER" | grep -qE "^(jenkinsfile|\.travis\.yml|cloudbuild)"; then
    ONE_WAY=1
    REASON="CI/CD pipeline"
fi

if [ "$ONE_WAY" = "1" ]; then
    cat >&2 <<HOOK_MSG
ONE_WAY_DOOR: You tried to create $FILENAME ($REASON). This write has been blocked because it is a one-way door -- a decision that becomes hard to reverse once other code, data, or users depend on it.

REQUIRED ACTION: You MUST use the AskUserQuestion tool before retrying this write. Present the user with:
1. What this file does and why it is a one-way door
2. At least 2 alternative approaches (if any exist) with their trade-offs
3. An option to proceed as planned
```

```
Frame the question around the specific architectural decision, not just "should I create this file?" The user needs to understand what they are committing
to.

After the user responds, proceed according to their choice.
HOOK_MSG
    exit 2
fi

exit 0
```

How it works:

1. The hook intercepts every `Write` tool call (new file creation)
2. It extracts the file path and checks it against known one-way-door patterns
3. If the file matches, the hook exits with code 2 (block) and sends a message to stderr
4. Claude receives the block message and must use `AskUserQuestion` to discuss the decision with the user
5. Two-way door files pass through silently (exit 0)

Exit codes:
- `0` — Allow (two-way door, proceed normally)
- `2` — Block (one-way door, requires discussion)

The three questions

Before committing to any one-way door, ask:

1. **What am I committing to?** — What does this decision constrain? What becomes harder to change?
2. **What are the alternatives?** — Is there a simpler approach? A more reversible one?
3. **What's the migration path?** — If this turns out to be wrong, how do we change course?

If you can't answer these questions clearly, you're not ready to walk through the door.

page-monitoring (Skill Module)

research-toolkit\skills\page-monitoring\SKILL.md

name: page-monitoring

description: Web page monitoring, change detection, and availability tracking. Use when tracking content changes, detecting when pages go down, monitoring for updates, preserving content before deletion, or generating feeds for pages without RSS. Covers Visualping, ChangeTower, Distill.io, and self-hosted monitoring solutions.

tags:

Page monitoring methodology

Patterns for tracking web page changes, detecting content removal, and preserving important pages before they disappear.

Monitoring service comparison

Free-tier limits and retention windows shift annually — verify at the service's pricing page before relying on a specific number. The columns below reflect a 2026 snapshot.

Service	Free Tier	Best For	History	Alert Speed
Visualping	A few daily checks (free plan tightened in recent years)	Visual changes	Standard	Minutes
ChangeTower	Yes (verify current limits)	Compliance, archiving	Multi-year on paid plans	Minutes
Distill.io	~5 monitors with 7-day history	Element-level tracking	Limited on free tier	Seconds
Wachete	Limited	Login-protected pages	12 months	Minutes
UptimeRobot	50 monitors at 5-minute intervals (free SMS removed)	Uptime only	60 days	5-min checks
changedetection.io	Self-hosted; free	Privacy / DIY	Disk space	Configurable
urlwatch	Self-hosted; free	Cron-driven CLI	Configurable	Configurable

Quick-start: Monitor a page

Distill.io element monitoring

```
// Distill.io allows CSS/XPath selectors for precise monitoring
// Example selectors for common use cases:

// Monitor news article headlines
const newsSelector = '.article-headline, h1.title, .story-title';

// Monitor price changes
const priceSelector = '.price, .product-price, [data-price]';

// Monitor stock/availability
const availabilitySelector = '.in-stock, .availability, .stock-status';

// Monitor specific paragraph or section
const sectionSelector = '#main-content p:first-child';

// Monitor table data
const tableSelector = 'table.data-table tbody tr';
```

Python monitoring script

```
import requests
import hashlib
import json
import smtplib
from email.mime.text import MIMEText
from datetime import datetime
from pathlib import Path
from typing import Optional
from bs4 import BeautifulSoup

class PageMonitor:
    """Simple page change monitor with local storage."""

    def __init__(self, storage_dir: Path):
        self.storage_dir = storage_dir
        self.storage_dir.mkdir(parents=True, exist_ok=True)
        self.state_file = storage_dir / 'monitor_state.json'
        self.state = self._load_state()

    def _load_state(self) -> dict:
        if self.state_file.exists():
            return json.loads(self.state_file.read_text())
        return {'pages': {}}

    def _save_state(self):
        self.state_file.write_text(json.dumps(self.state, indent=2))

    def _get_page_hash(self, url: str, selector: Optional[str] = None) -> tuple[str, str]:
        """Get content hash and content for a page or element."""

        response = requests.get(url, timeout=30, headers={
            'User-Agent': 'Mozilla/5.0 (PageMonitor/1.0)'
        })
        response.raise_for_status()

        if selector:
            soup = BeautifulSoup(response.text, 'html.parser')
            element = soup.select_one(selector)
            content = element.get_text(strip=True) if element else ''
        else:
            content = response.text

        content_hash = hashlib.sha256(content.encode()).hexdigest()
        return content_hash, content

    def add_page(self, url: str, name: str, selector: Optional[str] = None):
        """Add a page to monitor."""

        content_hash, content = self._get_page_hash(url, selector)

        self.state['pages'][url] = {
            'name': name,
            'selector': selector,
            'last_hash': content_hash,
            'last_check': datetime.now().isoformat(),
            'last_content': content[:1000],  # Store preview
            'change_count': 0
        }

        self._save_state()
        print(f"Added: {name} ({url})")

    def check_page(self, url: str) -> Optional[dict]:
        """Check single page for changes."""

        if url not in self.state['pages']:
            return None

        page = self.state['pages'][url]
        selector = page.get('selector')

        try:
            new_hash, new_content = self._get_page_hash(url, selector)
        except Exception as e:
            return {
                'url': url,
```

```
                'name': page['name'],
                'status': 'error',
                'error': str(e)
            }

        changed = new_hash != page['last_hash']

        result = {
            'url': url,
            'name': page['name'],
            'status': 'changed' if changed else 'unchanged',
            'previous_content': page['last_content'],
            'new_content': new_content[:1000] if changed else None
        }

        if changed:
            page['last_hash'] = new_hash
            page['last_content'] = new_content[:1000]
            page['change_count'] += 1

            # Archive the change
            archive_file = self.storage_dir / f"{hashlib.md5(url.encode()).hexdigest()}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"
            archive_file.write_text(new_content)

        page['last_check'] = datetime.now().isoformat()
        self._save_state()

        return result

    def check_all(self) -> list[dict]:
        """Check all monitored pages."""
        results = []
        for url in self.state['pages']:
            result = self.check_page(url)
            if result:
                results.append(result)
        return results

# Usage
monitor = PageMonitor(Path('./page_monitor_data'))

# Add pages to monitor
monitor.add_page(
    'https://example.com/important-page',
    'Important Page',
    selector='.main-content'  # Optional: monitor specific element
)

# Check for changes
results = monitor.check_all()
for result in results:
    if result['status'] == 'changed':
        print(f"CHANGED: {result['name']}")
        print(f"  Previous: {result['previous_content'][:100]}...")
        print(f"  New: {result['new_content'][:100]}...")
```

Uptime monitoring

UptimeRobot API [[integration]]

```
import requests
from typing import List, Optional

class UptimeRobotClient:
    """UptimeRobot API client for monitoring page availability."""

    def __init__(self, api_key: str):
        self.api_key = api_key
        # v2 still works as of 2026 but is in maintenance mode; v3 is
        # the current REST API at https://api.uptimerobot.com/v3 with
        # a different request shape (Bearer auth, JSON bodies).
        self.base_url = "https://api.uptimerobot.com/v2"

    def _request(self, endpoint: str, params: dict = None) -> dict:
        data = {'api_key': self.api_key}
        if params:
            data.update(params)

        response = requests.post(f"{self.base_url}/{endpoint}", data=data)
        return response.json()

    def get_monitors(self) -> List[dict]:
        """Get all monitors."""
        result = self._request('getMonitors')
        return result.get('monitors', [])

    def create_monitor(self, friendly_name: str, url: str,
                       monitor_type: int = 1) -> dict:
        """Create a new monitor.

        Types: 1=HTTP(s), 2=Keyword, 3=Ping, 4=Port
        """
        return self._request('newMonitor', {
            'friendly_name': friendly_name,
            'url': url,
            'type': monitor_type
        })

    def get_monitor_uptime(self, monitor_id: int,
                           custom_uptime_ratios: str = "7-30-90") -> dict:
        """Get uptime statistics for a monitor."""
        return self._request('getMonitors', {
            'monitors': monitor_id,
            'custom_uptime_ratios': custom_uptime_ratios
        })

    def pause_monitor(self, monitor_id: int) -> dict:
        """Pause a monitor."""
        return self._request('editMonitor', {
            'id': monitor_id,
            'status': 0
        })

    def resume_monitor(self, monitor_id: int) -> dict:
        """Resume a monitor."""
        return self._request('editMonitor', {
            'id': monitor_id,
            'status': 1
        })

# Usage
client = UptimeRobotClient('your-api-key')

# Create monitors for important pages
client.create_monitor('News Homepage', 'https://example-news.com')
client.create_monitor('API Status', 'https://api.example.com/health')

# Check all monitors
for monitor in client.get_monitors():
    status = 'UP' if monitor['status'] == 2 else 'DOWN'
    print(f"{monitor['friendly_name']}: {status}")
```

RSS feed generation

Generate RSS from pages without feeds

```
import requests
from bs4 import BeautifulSoup
from feedgen.feed import FeedGenerator
from datetime import datetime
import hashlib

class RSSGenerator:
    """Generate RSS feeds from web pages."""

    def __init__(self, feed_id: str, title: str, link: str):
        self.fg = FeedGenerator()
        self.fg.id(feed_id)
        self.fg.title(title)
        self.fg.link(href=link)
        self.fg.description(f'Auto-generated feed for {title}')

    def add_from_page(self, url: str, item_selector: str,
                      title_selector: str, link_selector: str,
                      description_selector: Optional[str] = None):
        """Parse a page and add items to feed.

        Args:
            url: Page URL to parse
            item_selector: CSS selector for each item container
            title_selector: CSS selector for title (relative to item)
            link_selector: CSS selector for link (relative to item)
            description_selector: Optional CSS selector for description
        """

        response = requests.get(url, timeout=30)
        soup = BeautifulSoup(response.text, 'html.parser')

        items = soup.select(item_selector)

        for item in items[:20]:  # Limit to 20 items
            title_elem = item.select_one(title_selector)
            link_elem = item.select_one(link_selector)

            if not title_elem or not link_elem:
                continue

            title = title_elem.get_text(strip=True)
            link = link_elem.get('href', '')

            # Make absolute URL if relative
            if link.startswith('/'):
                from urllib.parse import urljoin
                link = urljoin(url, link)

            fe = self.fg.add_entry()
            fe.id(hashlib.md5(link.encode()).hexdigest())
            fe.title(title)
            fe.link(href=link)

            if description_selector:
                desc_elem = item.select_one(description_selector)
                if desc_elem:
                    fe.description(desc_elem.get_text(strip=True))

            fe.published(datetime.now())

    def generate_rss(self) -> str:
        """Generate RSS XML string."""
        return self.fg.rss_str(pretty=True).decode()

    def save_rss(self, filepath: str):
        """Save RSS feed to file."""
        self.fg.rss_file(filepath)

# Example: Generate feed for a news site without RSS
rss = RSSGenerator(
    'https://example.com/news',
    'Example News Feed',
    'https://example.com/news'
```

```
)

rss.add_from_page(
    'https://example.com/news',
    item_selector='.news-item',
    title_selector='h2 a',
    link_selector='h2 a',
    description_selector='.summary'
)

# Save the feed
rss.save_rss('example_feed.xml')
```

Using RSS-Bridge (self-hosted)

```
# RSS-Bridge generates feeds for sites without them
# Supports Twitter, Instagram, YouTube, and many others

# Docker installation
docker pull rssbridge/rss-bridge
docker run -d -p 3000:80 rssbridge/rss-bridge

# Access at http://localhost:3000
# Select a bridge, enter parameters, get RSS feed URL
```

Social media monitoring

Twitter/X archiving with Twarc

```
# Twarc requires X (Twitter) API credentials.
#
# IMPORTANT (2023+): X eliminated the free Twitter API tier and the
# free academic research access program. Current X API pricing has
# shifted multiple times since the 2023 changes — the Basic / Pro /
# Enterprise tier names and pay-per-use credit model have been
# revised; check the current pricing page at
# https://developer.x.com/en/products/x-api before estimating cost.
# For one-off archiving, the snscrape / nitter / web-scraping path
# may be more cost-effective — see the web-scraping skill.

# Installation
# pip install twarc

# Configure (interactive — provides API keys + bearer token)
# twarc2 configure

import subprocess
import json
from pathlib import Path

class TwitterArchiver:
    """Archive Twitter searches and timelines."""

    def __init__(self, output_dir: Path):
        self.output_dir = output_dir
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def search(self, query: str, max_results: int = 100) -> Path:
        """Search tweets and save to file."""

        output_file = self.output_dir / f"search_{query.replace(' ', '_')}.jsonl"

        subprocess.run([
            'twarc2', 'search',
            '--max-results', str(max_results),
            query,
            str(output_file)
        ], check=True)

        return output_file

    def get_timeline(self, username: str, max_results: int = 100) -> Path:
        """Get user timeline."""

        output_file = self.output_dir / f"timeline_{username}.jsonl"

        subprocess.run([
            'twarc2', 'timeline',
            '--max-results', str(max_results),
            username,
            str(output_file)
        ], check=True)

        return output_file

    def parse_archive(self, filepath: Path) -> list[dict]:
        """Parse archived tweets."""

        tweets = []
        with open(filepath) as f:
            for line in f:
                data = json.loads(line)
                if 'data' in data:
                    tweets.extend(data['data'])

        return tweets
```

Webhook notifications

Send alerts on changes

```
import requests
from datetime import datetime
from typing import Optional

class AlertManager:
    """Send alerts when monitored pages change."""

    def __init__(self, slack_webhook: str = None,
                 discord_webhook: str = None,
                 email_config: dict = None):
        self.slack_webhook = slack_webhook
        self.discord_webhook = discord_webhook
        self.email_config = email_config

    def send_slack(self, message: str, channel: str = None):
        """Send Slack notification."""
        if not self.slack_webhook:
            return

        payload = {'text': message}
        if channel:
            payload['channel'] = channel

        requests.post(self.slack_webhook, json=payload)

    def send_discord(self, message: str):
        """Send Discord notification."""
        if not self.discord_webhook:
            return

        requests.post(self.discord_webhook, json={'content': message})

    def send_email(self, subject: str, body: str, to: str):
        """Send email notification."""
        if not self.email_config:
            return

        import smtplib
        from email.mime.text import MIMEText

        msg = MIMEText(body)
        msg['Subject'] = subject
        msg['From'] = self.email_config['from']
        msg['To'] = to

        with smtplib.SMTP(self.email_config['smtp_host'],
                          self.email_config['smtp_port']) as server:
            server.starttls()
            server.login(self.email_config['username'],
                         self.email_config['password'])
            server.send_message(msg)

    def alert_change(self, page_name: str, url: str,
                     old_content: str, new_content: str):
        """Send change alert to all configured channels."""

        message = f"""
Page Changed: {page_name}
URL: {url}
Time: {datetime.now().isoformat()}

Previous content (preview):
{old_content[:200]}...

New content (preview):
{new_content[:200]}...
"""

        if self.slack_webhook:
            self.send_slack(message)

        if self.discord_webhook:
            self.send_discord(message)
```

Scheduled monitoring with cron

Cron [[setup]] for continuous monitoring

```
# Edit crontab
crontab -e

# Check pages every 15 minutes
*/15 * * * * /usr/bin/python3 /path/to/monitor_script.py >> /var/log/monitor.log 2>&1

# Check critical pages every 5 minutes
*/5 * * * * /usr/bin/python3 /path/to/critical_monitor.py >> /var/log/critical.log 2>&1

# Daily summary report at 8 AM
0 8 * * * /usr/bin/python3 /path/to/daily_report.py
```

Monitoring script

lll

```
#!/usr/bin/env python3
"""Page monitoring script for cron execution."""

import sys
from pathlib import Path
from datetime import datetime

# Add project to path
sys.path.insert(0, str(Path(__file__).parent))

from monitor import PageMonitor
from alerts import AlertManager

def main():
    # Initialize
    monitor = PageMonitor(Path('./data'))
    alerts = AlertManager(
        slack_webhook='https://hooks.slack.com/services/...',
        discord_webhook='https://discord.com/api/webhooks/...'
    )

    # Check all pages
    results = monitor.check_all()

    # Process results
    changes = [r for r in results if r['status'] == 'changed']
    errors = [r for r in results if r['status'] == 'error']

    # Alert on changes
    for change in changes:
        alerts.alert_change(
            change['name'],
            change['url'],
            change['previous_content'],
            change['new_content']
        )
        print(f"[{datetime.now()}] CHANGE: {change['name']}")

    # Alert on errors
    for error in errors:
        alerts.send_slack(f"Monitor error for {error['name']}: {error['error']}")
        print(f"[{datetime.now()}] ERROR: {error['name']} - {error['error']}")

    # Summary
    print(f"[{datetime.now()}] Checked {len(results)} pages, "
          f"{len(changes)} changes, {len(errors)} errors")

if __name__ == '__main__':
    main()
```

Archive on change

Automatic archiving when changes detected

```
# `MultiArchiver` is the cascade-archive helper from the sibling
# web-archiving skill (see research-toolkit/skills/web-archiving/).
# Replace this import with your own multi-service archiver, or
# port the MultiArchiver class from that skill.
from multiarchiver import MultiArchiver

class ArchivingMonitor(PageMonitor):
    """Page monitor that archives content when changes detected."""

    def __init__(self, storage_dir: Path):
        super().__init__(storage_dir)
        self.archiver = MultiArchiver()

    def check_page(self, url: str) -> dict:
        """Check page and archive if changed."""

        result = super().check_page(url)

        if result and result['status'] == 'changed':
            # Archive to multiple services
            archive_results = self.archiver.archive_url(url)

            successful_archives = [
                r.archived_url for r in archive_results
                if r.success
            ]

            result['archives'] = successful_archives

            # Log archive URLs
            print(f"Archived {url} to:")
            for archive_url in successful_archives:
                print(f"  - {archive_url}")

        return result
```

Monitoring strategy by use case

News monitoring

```
## News/Current Events Monitoring

### Pages to monitor:
- Breaking news sections
- Press release pages
- Government announcement pages
- Company newsrooms

### Monitoring frequency:
- Breaking news: Every 5 minutes
- Press releases: Every 15-30 minutes
- General news: Every hour

### Archive strategy:
- Archive immediately on detection
- Use both Wayback Machine and Archive.today
- Save local copy with timestamp
```

Research monitoring

```
## Academic/Research Monitoring

### Pages to monitor:
- Preprint servers (arXiv, SSRN)
- Journal table of contents
- Conference proceedings
- Researcher profiles

### Monitoring frequency:
- Daily for active topics
- Weekly for general monitoring

### Tools recommended:
- Google Scholar alerts (free, built-in)
- Semantic Scholar alerts
- RSS feeds where available
- Custom monitors for specific pages
```

Competitive intelligence

```
## Competitor Monitoring

### Pages to monitor:
- Pricing pages
- Product pages
- Job postings
- Press releases
- Executive bios

### Monitoring frequency:
- Pricing: Daily
- Products: Daily
- Jobs: Weekly
- Press: Daily

### Legal considerations:
- Don't violate terms of service
- Don't circumvent access controls
- Public pages only
- Don't scrape at high frequency
```

Best practices

Monitoring checklist

```
## Before monitoring a page:

- [ ] Is the page publicly accessible?
- [ ] Are you respecting robots.txt?
- [ ] Is monitoring frequency reasonable?
- [ ] Do you have a legitimate purpose?
- [ ] Are you storing data securely?
- [ ] Do you have alerts configured?
- [ ] Is archiving set up for important pages?

## Maintenance:

- [ ] Review monitors monthly
- [ ] Remove stale monitors
- [ ] Update selectors if pages change
- [ ] Check alert delivery
- [ ] Verify archives are working
```

Rate limiting

```
import time
from functools import wraps

def rate_limit(min_interval: float = 1.0):
    """Decorator to rate limit function calls."""
    last_call = [0.0]

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_call[0]
            if elapsed < min_interval:
                time.sleep(min_interval - elapsed)
            last_call[0] = time.time()
            return func(*args, **kwargs)
        return wrapper
    return decorator

# Usage
@rate_limit(min_interval=2.0)  # Max once per 2 seconds
def check_page(url: str):
    return requests.get(url)
```

pdf-design (Skill Module)

pdf-design\SKILL.md

name: pdf-design
description: Design and edit professional PDF reports and proposals with live preview
user_invocable: true
trigger_patterns:
tags:

PDF Design System

Create and edit professional PDF reports and funding proposals with live [[preview]] and iterative design.

Interactive editing mode

During a design session, use these commands:

Command	Action
`[[preview]]`	Screenshot current state
`[[preview]] page N`	Screenshot specific page
`show cover`	[[Preview]] cover page
`show budget`	[[Preview]] budget section
`regenerate`	Create new PDF
`upload`	Upload to Google Drive
`done`	Finish session

Workflow:
1. You say "[[preview]]" → I show current state
2. You describe changes → I implement them
3. Repeat until done → Generate final PDF

Quick start

```
# Copy template to start new report
cp ~/.claude/plugins/pdf-design/templates/democracy-day-proposal.html ./new-report.html

# Generate PDF (must use snap-accessible path)
mkdir -p ~/snap/chromium/common/pdf-work
cp new-report.html ~/snap/chromium/common/pdf-work/
chromium-browser --headless --disable-gpu \
  --print-to-pdf="$HOME/snap/chromium/common/pdf-work/output.pdf" \
  --no-pdf-header-footer \
  "file://$HOME/snap/chromium/common/pdf-work/new-report.html"
```

Document types

- **Funding proposals** — Grant requests with budgets
- **Program reports** — Initiative updates
- **Impact reports** — Metrics and outcomes
- **Budget summaries** — Financial breakdowns

Key principles

1. **Sentence case** — Never Title Case
2. **Left-aligned** — Never justified text
3. **Print-ready** — 8.5" × 11" letter size
4. **Brand consistent** — CCM red or program palettes

Brand guidelines

CCM standard colors

```
:root {
    --ccm-red: #CA3553;
    --ccm-black: #000000;
    --ccm-gray: #666666;
    --ccm-light: #e2e8f0;
}
```

Program-specific (Democracy Day)

```
:root {
    --civic-navy: #1a2b4a;
    --civic-blue: #2d4a7c;
    --civic-gold: #c9a227;
    --civic-red: #b31942;
}
```

Typography

```
<link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;600;700&family=Source+Sans+Pro:wght@300;400;600&display=swap" rel="stylesheet">
```

```
body {
    font-family: 'Source Sans Pro', sans-serif;
    font-size: 0.875rem;
    line-height: 1.6;
}

h1, h2, h3 {
    font-family: 'Montserrat', sans-serif;
}
```

HTML structure

Page [[setup]]

```
@page { size: letter; margin: 0; }

.page {
    width: 8.5in;
    height: 11in;
    display: grid;
    grid-template-rows: auto 1fr auto;
    overflow: hidden;
    page-break-after: always;
}
```

Cover page

```
<div class="page cover">
    <div class="cover-header">
        <div class="cover-org">Center for Cooperative Media</div>
        <h1 class="cover-title">Report title</h1>
        <p class="cover-intro">Brief description.</p>
    </div>
    <div class="cover-footer">
        <div class="cover-stats"><!-- Stats --></div>
        <div class="cover-footer-right">
            <div class="cover-date">February 2026</div>
            <div class="cover-logo"><img src="..." alt="Logo"></div>
        </div>
    </div>
</div>
```

Content page

```
<div class="page content-page">
    <div class="page-header">
        <div class="page-header-title">Document title</div>
        <div class="page-number">2</div>
    </div>
    <div class="page-body">
        <!-- Content goes here -->
    </div>
    <footer class="page-footer">
        <!-- Footer -->
    </footer>
</div>
```

Budget table

```
<table class="budget-table">
    <thead>
        <tr><th>Expense</th><th>Per year</th><th>Total</th></tr>
    </thead>
    <tbody>
        <tr>
            <td>Item<span class="item-desc">Details</span></td>
            <td>$10,000</td>
            <td>$20,000</td>
        </tr>
    </tbody>
    <tfoot>
        <tr><td>Total</td><td>$50,000</td><td>$100,000</td></tr>
    </tfoot>
</table>
```

Page footer

```
.page-body {
    padding: 0.2in 0.65in 0.3in;
    overflow: hidden;
}

.page-footer {
    padding: 0 0.65in 0.5in;
    border-top: 1px solid #e2e8f0;
    font-size: 0.8rem;
}
```

Footer clearance

Content must not touch or overlap the page footer. These rules apply to **content pages** — cover pages and special layouts may use different structures.

- Content pages must use `display: grid; grid-[[[

PDF generation

Chromium (snap-confined)

```
# Must use ~/snap/chromium/common/ path
mkdir -p ~/snap/chromium/common/pdf-work
cp template.html ~/snap/chromium/common/pdf-work/
chromium-browser --headless --disable-gpu \
  --print-to-pdf="$HOME/snap/chromium/common/pdf-work/output.pdf" \
  --no-pdf-header-footer \
  "file://$HOME/snap/chromium/common/pdf-work/template.html"
cp ~/snap/chromium/common/pdf-work/output.pdf ./
```

[[Preview]] pages

```
# PDF to PNG
pdftoppm -png -f 1 -l 1 output.pdf preview

# Page count
pdfinfo output.pdf | grep Pages
```

Legion browser [[preview]]

```
~/.claude/scripts/legion-browser.py screenshot "file:///path/to/template.html" -o preview.png
```

Google Drive upload

```
cd ~/.claude/workstation/mcp-servers/gmail && source .venv/bin/activate
python3 << 'PYEOF'
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload
from google.oauth2.credentials import Credentials
import json

with open('/home/jamditis/.claude/google/drive-token.json') as f:
    token_data = json.load(f)

creds = Credentials(
    token=token_data['access_token'],
    refresh_token=token_data.get('refresh_token'),
    token_uri='https://oauth2.googleapis.com/token',
    client_id=token_data.get('client_id'),
    client_secret=token_data.get('client_secret')
)

service = build('drive', 'v3', credentials=creds)

# Upload new file
file_metadata = {
    'name': 'Report.pdf',
    'parents': ['1lKTdwq4_5uErj-tBN112WCdJGD2YtetO']  # Shared with Joe
}
media = MediaFileUpload('/path/to/output.pdf', mimetype='application/pdf')
file = service.files().create(body=file_metadata, media_body=media, fields='id,webViewLink').execute()
print(f"Uploaded: {file.get('webViewLink')}")
PYEOF
```

Drive folders

- **Shared with Joe:** `1lKTdwq4_5uErj-tBN112WCdJGD2YtetO`
- **Claude Workspace:** `1e5dtKOiuvk0PPrFq3UyNI2UAa6RFiom3`

Reusable content blocks

These patterns were proven out in the NJ Public TV walkthrough deck (pdf-[[playground]] 1.3.0) and work just as well inside [[[[report]]]] and [[proposal]] pages. Drop them into any `.page-body` or `.cover-footer`.

Headline with red accent rule

A tight 0.95in × 0.08in red bar under the headline reads cleaner than a full-width border. Use for section headers inside content pages.

```
.section-header h2 {
    font-family: 'Montserrat', sans-serif;
    font-size: 22pt;
    font-weight: 800;
    color: var(--ccm-black);
    line-height: 1.08;
}

.section-header h2::after {
    content: '';
    display: block;
    width: 0.95in;
    height: 0.08in;
    background: var(--ccm-red);
    margin-top: 0.14in;
}
```

Stats strip (big-number row)

Row of 3–4 big numbers with a short caption and a red left rule. Great for executive-summary numbers on a cover or intro page.

```
<div class="stats-strip">
    <div class="stat"><div class="big">23,000+</div><div class="label">Students enrolled</div></div>
    <div class="stat"><div class="big">$660M</div><div class="label">Annual operating budget</div></div>
    <div class="stat"><div class="big">$2.3B</div><div class="label">Economic impact</div></div>
    <div class="stat"><div class="big">252</div><div class="label">Acre main campus</div></div>
</div>
```

```
.stats-strip {
    display: grid;
    grid-template-columns: repeat(var(--stat-cols, 4), 1fr);
    gap: 0.3in;
}
.stats-strip .stat {
    padding: 0.05in 0 0.1in 0.24in;
    border-left: 4px solid var(--ccm-red);
}
.stats-strip .big {
    font-family: 'Montserrat', sans-serif;
    font-size: 28pt;
    font-weight: 800;
    line-height: 1;
    color: var(--ccm-black);
    letter-spacing: -0.015em;
}
.stats-strip .label {
    font-size: 9pt;
    font-weight: 600;
    margin-top: 0.1in;
    color: var(--ccm-gray);
    text-transform: uppercase;
    letter-spacing: 0.05em;
}
```

Three-column content

For breaking one topic into three parallel facets with a dashed divider between columns.

```
.three-col {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    gap: 0.3in;
}
.three-col > div + div {
    padding-left: 0.3in;
    border-left: 2px dashed #d9d9d9;
}
.three-col h3 {
    font-size: 10pt;
    font-weight: 800;
    color: var(--ccm-red);
    text-transform: uppercase;
    letter-spacing: 0.04em;
    margin-bottom: 0.12in;
}
```

Four-tile pillars

Numbered cards with a red top rule — for parallel capabilities, themes, or commitments. Common on [[proposal]] executive summary pages.

```
<div class="four-col-tiles">
    <div class="tile">
        <div class="tile-num">Pillar 01</div>
        <h3>Proven facilities and expertise</h3>
        <p>Short 2–3 line description.</p>
    </div>
    <!-- 3 more tiles -->
</div>
```

```
.four-col-tiles {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 0.15in;
}
.four-col-tiles .tile {
    border-top: 4px solid var(--ccm-red);
    padding: 0.18in 0.15in 0.15in;
    background: #f7f6f5;
}
.four-col-tiles .tile-num {
    font-size: 8pt;
    font-weight: 800;
    color: var(--ccm-red);
    letter-spacing: 0.15em;
    text-transform: uppercase;
    margin-bottom: 0.08in;
}
.four-col-tiles .tile h3 {
    font-size: 12pt;
    font-weight: 800;
    color: var(--ccm-black);
    margin-bottom: 0.08in;
}
.four-col-tiles .tile p {
    font-size: 9pt;
    line-height: 1.38;
    color: var(--ccm-gray);
    margin: 0;
}
```

Partner / label grid

4-column grid of labeled tiles with a red left accent bar. Use for sponsor lists, letters of support, or advisory board rosters.

```
.partner-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 0.12in 0.18in;
}
.partner-grid .partner {
    position: relative;
    padding: 0.14in 0.15in 0.14in 0.22in;
    background: #f7f6f5;
    font-size: 9pt;
    font-weight: 600;
    line-height: 1.25;
    color: var(--ccm-black);
}
.partner-grid .partner::before {
    content: '';
    position: absolute;
    left: 0; top: 0; bottom: 0;
    width: 0.06in;
    background: var(--ccm-red);
}
```

Vertical rhythm

Every block above was tightened based on real presentation feedback. Key principles:

- Accent rule sits ~0.14in below the headline, not further
- Lede paragraph sits ~0.18in below the headline or rule
- Body content sits ~0.22in below the lede
- Between body sections, 0.25–0.3in gap is enough — don't add more
- Between grid cards, use 0.15–0.2in gaps
- The instinct to "add breathing room" almost always makes pages feel emptier rather than cleaner

If a page feels too crowded, *reduce content*, don't expand spacing.

Known issues

1. **Base64 images** — Don't read HTML with large base64 using Read tool (API error). Use sed/grep/Python.
2. **Snap confinement** — Chromium can only write to `~/snap/chromium/common/`
3. **Fonts** — Google Fonts via CDN; for offline, embed as base64

Logo locations

- CCM logo: `~/.claude/plugins/pdf-design/templates/` (embedded in [[[[

performing-access-recertification-with-saviynt (Skill Module)

performing-access-recertification-with-saviynt\SKILL.md

name: performing-access-recertification-with-saviynt
description: Agent for managing Saviynt access recertification campaigns via REST API. Use when the user asks to perform access recertification with saviynt.
version: 1.0.0
tags:
source: performing-access-recertification-with-saviynt

Performing Access Recertification with Saviynt

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Saviynt Campaign Configuration Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** access recertification with saviynt.
- Capability: Agent for managing Saviynt access recertification campaigns via REST API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, access, recertification, saviynt`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Authenticate to Saviynt EIC and get OAuth token
2. List certification campaigns
3. Get detailed campaign status including item counts
4. Get items pending review in a certification campaign
5. Certify or revoke items in a campaign
6. Find campaigns past their due date
7. Generate recertification status report

Key concepts

- Quarterly access reviews for financially significant applications
- Evidence of review decisions with justification
- Remediation of revoked access within defined SLA
- Separation of duties validation during certification
- CC6.1: Logical access controls
- CC6.2: User registration and authorization
- CC6.3: Access modification and removal
- Semi-annual certification campaigns required for trust service criteria

Tools & resources

- Saviynt Campaign Configuration Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `authenticate` — Authenticate to Saviynt EIC and get OAuth token.
- `list_campaigns` — List certification campaigns.
- `get_campaign_details` — Get detailed campaign status including item counts.
- `get_pending_items` — Get items pending review in a certification campaign.
- `certify_items` — Certify or revoke items in a campaign.
- `check_overdue_campaigns` — Find campaigns past their due date.
- `generate_report` — Generate recertification status report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-access-recertification-with-saviynt`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-access-review-and-certification (Skill Module)

performing-access-review-and-certification\SKILL.md

name: performing-access-review-and-certification
description: Agent for conducting access review and certification using identity governance APIs. Use when the user asks to perform access review and certification.
version: 1.0.0
tags:
source: performing-access-review-and-certification

Performing Access Review and Certification

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Compliance Frameworks, NIST Standards, Compliance Requirements, Industry Frameworks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** access review and certification.
- Capability: Agent for conducting access review and certification using identity governance APIs.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, access, review, certification`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load access entitlement data from CSV export
2. Find accounts with no manager or terminated status
3. Check for separation of duties violations
4. Find users with entitlement counts above threshold
5. Find entitlements not used within the stale period
6. Generate access review report

Key concepts

- AC-2: Account Management (periodic review requirement)
- AC-2(3): Disable accounts after inactivity period
- AC-5: Separation of Duties
- AC-6: Least Privilege
- AC-6(7): Review of User Privileges
- AU-6: Audit Record Review, Analysis, and Reporting
- NIST SP 800-171: 3.1.1 Authorized Access Control, 3.1.2 Transaction Control
- SOX Section 404: Quarterly access reviews for financial systems

Tools & resources

- Compliance Frameworks
- NIST Standards
- Compliance Requirements
- Industry Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_access_data` — Load access entitlement data from CSV export.
- `identify_orphaned_accounts` — Find accounts with no manager or terminated status.
- `check_sod_violations` — Check for separation of duties violations.
- `identify_excessive_access` — Find users with entitlement counts above threshold.
- `check_last_used` — Find entitlements not used within the stale period.
- `generate_report` — Generate access review report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-access-review-and-certification`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-active-directory-bloodhound-analysis (Skill Module)

performing-active-directory-bloodhound-analysis\SKILL.md

name: performing-active-directory-bloodhound-analysis
description: BloodHound Attack Path Analysis Agent - Queries Neo4j for AD attack paths to Domain Admin. Use when the user asks to perform active directory bloodhound analysis.
version: 1.0.0
tags:
source: performing-active-directory-bloodhound-analysis

Performing Active Directory BloodHound Analysis

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: BloodHound, neo4j Python Driver, NIST References, CIS Benchmarks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** active directory bloodhound analysis.
- Capability: BloodHound Attack Path Analysis Agent - Queries Neo4j for AD attack paths to Domain Admin.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, active, directory, bloodhound, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Connect to Neo4j database containing BloodHound data
2. Find all members of the Domain Admins group
3. Find shortest attack paths from owned users to Domain Admin
4. Find users with SPNs set (Kerberoastable) that have paths to high-value targets
5. Find users with Kerberos pre-auth disabled (AS-REP Roastable)
6. Find computers with unconstrained delegation enabled
7. Find users with local admin rights on a target computer
8. Find GPO-based attack paths that could lead to privilege escalation

Key concepts

- T1087.002** - Account Discovery: Domain Account
- T1069.001** - Permission Groups Discovery: Local Groups
- T1069.002** - Permission Groups Discovery: Domain Groups
- T1550.002** - Use Alternate Authentication Material: Pass the Hash
- T1550.003** - Use Alternate Authentication Material: Pass the Ticket
- T1021.002** - Remote Services: SMB/Windows Admin Shares
- T1021.001** - Remote Services: Remote Desktop Protocol
- T1021.006** - Remote Services: Windows Remote Management

Tools & resources

- BloodHound
- neo4j Python Driver

- NIST References
- CIS Benchmarks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `connect_neo4j` — Connect to Neo4j database containing BloodHound data.
- `find_domain_admins` — Find all members of the Domain Admins group.
- `find_shortest_paths_to_da` — Find shortest attack paths from owned users to Domain Admin.
- `find_kerberoastable_users` — Find users with SPNs set (Kerberoastable) that have paths to high-value targets.
- `find_asrep_roastable` — Find users with Kerberos pre-auth disabled (AS-REP Roastable).
- `find_unconstrained_delegation` — Find computers with unconstrained delegation enabled.
- `find_local_admin_paths` — Find users with local admin rights on a target computer.
- `find_gpo_attack_paths` — Find GPO-based attack paths that could lead to privilege escalation.
- `assess_ad_risk` — Calculate overall AD security risk score.
- `generate_report` — Generate BloodHound analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-active-directory-bloodhound-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-active-directory-compromise-investigation (Skill Module)

performing-active-directory-compromise-investigation\SKILL.md

name: performing-active-directory-compromise-investigation
description: Active Directory Compromise Investigation agent - analyzes Windows Security event logs for indicators of AD compromise including DCSync, Golden Ticket, Kerberoasting, and lateral movement. Use when the user asks to perform active directory compromise investigation.
version: 1.0.0
tags:
source: performing-active-directory-compromise-investigation

Performing Active Directory Compromise Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: DCSync Detection, Kerberoasting Detection, Golden Ticket Detection, Lateral Movement Detection, CISA Alert, Microsoft Security Best Practices

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** active directory compromise investigation.
- Capability: Active Directory Compromise Investigation agent - analyzes Windows Security event logs for indicators of AD compromise including DCSync, Golden Ticket, Kerberoasting, and lateral movement.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, active, directory, compromise, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse Windows event logs exported as JSON
2. Detect DCSync attacks via Event ID 4662 with replication GUIDs
3. Detect Kerberoasting via Event ID 4769 with RC4 encryption
4. Detect Golden Ticket indicators via Event ID 4768 anomalies
5. Detect lateral movement via Type 3 logons and explicit credential use
6. Detect persistence mechanisms via account creation and group changes
7. Run all detection checks and produce consolidated report

Key concepts

- Provides incident response lifecycle: Preparation, Detection & Analysis, Containment, Eradication & Recovery, Post-Incident Activity
- AD compromise investigations follow all four phases with emphasis on scoping the identity compromise
- Published guidance on AD-specific threats and detection methods
- Covers DCSync, Golden Ticket, Silver Ticket, and Kerberoasting attacks
- Recommends tiered administration, Protected Users group, and credential hygiene
- T1003.006: OS Credential Dumping - DCSync
- T1558.001: Steal or Forge Kerberos Tickets - Golden Ticket
- T1558.002: Steal or Forge Kerberos Tickets - Silver Ticket

Tools & resources

- DCSync Detection

- Kerberoasting Detection
- Golden Ticket Detection
- Lateral Movement Detection
- CISA Alert
- Microsoft Security Best Practices
- SANS FOR500

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_evtx_json` — Parse Windows event logs exported as JSON.
- `detect_dcsync` — Detect DCSync attacks via Event ID 4662 with replication GUIDs.
- `detect_kerberoasting` — Detect Kerberoasting via Event ID 4769 with RC4 encryption.
- `detect_golden_ticket` — Detect Golden Ticket indicators via Event ID 4768 anomalies.
- `detect_lateral_movement` — Detect lateral movement via Type 3 logons and explicit credential use.
- `detect_persistence` — Detect persistence mechanisms via account creation and group changes.
- `generate_report` — Run all detection checks and produce consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-active-directory-compromise-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-active-directory-forest-trust-attack (Skill Module)

performing-active-directory-forest-trust-attack\SKILL.md

name: performing-active-directory-forest-trust-attack
description: Agent for AD forest trust enumeration and security assessment using impacket. Use when the user asks to perform active directory forest trust attack.
version: 1.0.0
tags:
source: performing-active-directory-forest-trust-attack

Performing Active Directory Forest Trust Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Trust Direction Values, Trust Attribute Flags, Impacket RPC Calls

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** active directory forest trust attack.
- Capability: Agent for AD forest trust enumeration and security assessment using impacket.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, active, directory, forest, trust, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Enumerate AD trust relationships via LDAP trustedDomain objects
2. Find foreign security principals (cross-forest members) in the domain
3. Resolve a SID across forest trust using LSA LookupSids RPC call
4. Assess security risk of trust relationships
5. Run comprehensive forest trust security audit

Tools & resources

- Trust Direction Values
- Trust Attribute Flags
- Impacket RPC Calls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `enumerate_trusts_ldap` — Enumerate AD trust relationships via LDAP trustedDomain objects.
- `enumerate_foreign_principals` — Find foreign security principals (cross-forest members) in the domain.
- `lookup_sid_cross_forest` — Resolve a SID across forest trust using LSA LookupSids RPC call.
- `assess_trust_risk` — Assess security risk of trust relationships.
- `full_audit` — Run comprehensive forest trust security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-active-directory-forest-trust-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-active-directory-penetration-test (Skill Module)

performing-active-directory-penetration-test\SKILL.md

name: performing-active-directory-penetration-test
description: Active Directory Penetration Test agent - automates AD enumeration using ldap3 for LDAP queries, subprocess for impacket tools, and generates a structured pentest findings report. Use when the user asks to perform active directory penetration test.
version: 1.0.0
tags:
source: performing-active-directory-penetration-test

Performing Active Directory Penetration Test

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: ldap3 Library, Impacket Tools, Attack Techniques, Key Frameworks, AD Security Benchmarks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** active directory penetration test.
- Capability: Active Directory Penetration Test agent - automates AD enumeration using ldap3 for LDAP queries, subprocess for impacket tools, and generates a structured pentest findings report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, active, directory, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Establish authenticated LDAP connection
2. Extract domain functional level and naming context
3. Enumerate all domain users with security-relevant attributes
4. Identify accounts vulnerable to AS-REP Roasting
5. Identify accounts vulnerable to Kerberoasting
6. Audit domain password policy
7. Run impacket GetUserSPNs for Kerberoasting
8. Run full AD pentest enumeration and build report

Key concepts

- CIS Microsoft Windows Server Benchmark
- STIG (Security Technical Implementation Guide) for Windows
- Microsoft Security Compliance Toolkit

Tools & resources

- ldap3 Library
- Impacket Tools
- Attack Techniques
- Key Frameworks
- AD Security Benchmarks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `connect_ldap` — Establish authenticated LDAP connection.
- `get_domain_info` — Extract domain functional level and naming context.
- `enumerate_users` — Enumerate all domain users with security-relevant attributes.
- `find_asrep_roastable` — Identify accounts vulnerable to AS-REP Roasting.
- `find_kerberoastable` — Identify accounts vulnerable to Kerberoasting.
- `check_password_policy` — Audit domain password policy.
- `run_impacket_getspns` — Run impacket GetUserSPNs for Kerberoasting.
- `generate_report` — Run full AD pentest enumeration and build report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-active-directory-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-active-directory-vulnerability-assessment (Skill Module)

performing-active-directory-vulnerability-assessment\SKILL.md

name: performing-active-directory-vulnerability-assessment
description: Active Directory Vulnerability Assessment agent - parses PingCastle XML reports and performs LDAP checks to assess AD security posture against common vulnerability categories. Use when the user asks to perform active directory vulnerability assessment.
version: 1.0.0
tags:
source: performing-active-directory-vulnerability-assessment

Performing Active Directory Vulnerability Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Vulnerability Categories, Primary Standards, Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** active directory vulnerability assessment.
- Capability: Active Directory Vulnerability Assessment agent - parses PingCastle XML reports and performs LDAP checks to assess AD security posture against common vulnerability categories.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, active, directory, vulnerability, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse PingCastle XML health check report
2. Check domain password policy via LDAP
3. Check krbtgt account password age
4. Convert PingCastle risk rules into standardized findings
5. Run all assessments and build consolidated report

Key concepts

- T1558.003 - Kerberoasting
- T1558.004 - AS-REP Roasting
- T1557.001 - LLMNR/NBT-NS Poisoning
- T1484.001 - Group Policy Modification
- T1134.005 - SID-History Injection
- Relevance: Provides prescriptive hardening guidance for AD configuration
- Relevance: Baseline GPO settings for domain controllers and member servers
- Title: Digital Identity Guidelines - Authentication and Lifecycle Management

Tools & resources

- Vulnerability Categories
- Primary Standards
- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_pingcastle_report` — Parse PingCastle XML health check report.
- `check_password_policy_ldap` — Check domain password policy via LDAP.
- `check_krbtgt_age` — Check krbtgt account password age.
- `assess_pingcastle_risks` — Convert PingCastle risk rules into standardized findings.
- `generate_report` — Run all assessments and build consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-active-directory-vulnerability-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-adversary-in-the-middle-phishing-detection (Skill Module)

performing-adversary-in-the-middle-phishing-detection\SKILL.md

name: performing-adversary-in-the-middle-phishing-detection
description: Adversary-in-the-Middle (AiTM) Phishing Detection agent - analyzes sign-in logs and inbox rules to detect AiTM phishing campaigns that bypass MFA by proxying authentication sessions. Use when the user asks to perform adversary in the middle phishing detection.
version: 1.0.0
tags:
source: performing-adversary-in-the-middle-phishing-detection

Performing Adversary in The Middle Phishing Detection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AiTM Attack Overview, Detection Methods, Haversine Formula, Detection Rule Categories, Phishing-Resistant MFA Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** adversary in the middle phishing detection.
- Capability: Adversary-in-the-Middle (AiTM) Phishing Detection agent - analyzes sign-in logs and inbox rules to detect AiTM phishing campaigns that bypass MFA by proxying authentication sessions.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, adversary, middle, phishing, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Calculate great-circle distance between two points
2. Load Azure AD / Entra ID sign-in logs in JSON format
3. Detect impossible travel - logins from distant locations in short time
4. Detect inbox rules commonly created by AiTM attackers
5. Detect potential session token replay from new device/location
6. Run all detection checks and build consolidated report

Key concepts

- T1557: Adversary-in-the-Middle
- T1539: Steal Web Session Cookie
- T1550.004: Use Alternate Authentication Material: Web Session Cookie
- T1566.002: Phishing: Spearphishing Link
- T1114.003: Email Collection: Email Forwarding Rule
- T1098.005: Account Manipulation: Device Registration
- Over 1 million PhaaS attacks detected in Jan-Feb 2025 (Barracuda)
- Tycoon 2FA most prevalent followed by EvilProxy and Evilginx

Tools & resources

- AiTM Attack Overview
- Detection Methods

- Haversine Formula
- Detection Rule Categories
- Phishing-Resistant MFA Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `haversine_km` — Calculate great-circle distance between two points.
- `load_sign_in_logs` — Load Azure AD / Entra ID sign-in logs in JSON format.
- `detect_impossible_travel` — Detect impossible travel - logins from distant locations in short time.
- `detect_suspicious_inbox_rules` — Detect inbox rules commonly created by AiTM attackers.
- `detect_token_replay` — Detect potential session token replay from new device/location.
- `generate_report` — Run all detection checks and build consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-adversary-in-the-middle-phishing-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-agentless-vulnerability-scanning (Skill Module)

performing-agentless-vulnerability-scanning\SKILL.md

name: performing-agentless-vulnerability-scanning
description: Agentless Vulnerability Scanning agent - uses AWS Inspector2 and SSM APIs via boto3 to perform agentless scans of EC2 instances through EBS snapshot analysis without requiring installed agents. Use when the user asks to perform agentless vulnerability scanning.
version: 1.0.0
tags:
source: performing-agentless-vulnerability-scanning

Performing Agentless Vulnerability Scanning

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Vuls (Open Source), Microsoft Defender for Cloud Agentless, Tenable Agentless Discovery, Wiz Agentless VM, Datadog Agentless Scanning

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** agentless vulnerability scanning.
- Capability: Agentless Vulnerability Scanning agent - uses AWS Inspector2 and SSM APIs via boto3 to perform agentless scans of EC2 instances through EBS snapshot analysis without requiring installed agents.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, agentless, vulnerability, scanning`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create AWS Inspector2 client
2. Create EC2 client
3. Check Inspector2 enablement status
4. List EC2 instances and their scan coverage status
5. List vulnerability findings from Inspector2
6. Create EBS snapshots for agentless scanning
7. Run agentless scanning assessment and build report

Key concepts

- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment
- CIS Controls v8.1 Control 7.5: Perform Automated Vulnerability Scans of Internal Assets
- PCI DSS v4.0 Req 11.3: External and internal vulnerability scanning
- ISO 27001: 2022 A.8.8**: Management of technical vulnerabilities

Tools & resources

- Vuls (Open Source)
- Microsoft Defender for Cloud Agentless
- Tenable Agentless Discovery
- Wiz Agentless VM
- Datadog Agentless Scanning

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_inspector_client` — Create AWS Inspector2 client.
- `get_ec2_client` — Create EC2 client.
- `check_inspector_status` — Check Inspector2 enablement status.
- `list_ec2_scan_coverage` — List EC2 instances and their scan coverage status.
- `list_findings` — List vulnerability findings from Inspector2.
- `create_ebs_snapshot_scan` — Create EBS snapshots for agentless scanning.
- `generate_report` — Run agentless scanning assessment and build report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-agentless-vulnerability-scanning`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ai-driven-osint-correlation (Skill Module)

performing-ai-driven-osint-correlation\SKILL.md

name: performing-ai-driven-osint-correlation
description: Agent for performing AI-driven OSINT correlation. Collects and normalizes OSINT data from multiple sources (Sherlock, theHarvester, SpiderFoot, breach databases), performs cross-source entity resolution and.... Use when the user asks to perform ai driven osint correlation.
version: 1.0.0
tags:
source: performing-ai-driven-osint-correlation

Performing Ai Driven Osint Correlation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Supported Data Sources, Correlation Confidence Scoring

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ai driven osint correlation.
- Capability: Agent for performing AI-driven OSINT correlation. Collects and normalizes OSINT data from multiple sources (Sherlock, theHarvester, SpiderFoot, breach databases), performs cross-source entity resolution and correlation, and generates unified intelligence profiles with confidence scoring.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, driven, osint, correlation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load and normalize Sherlock username enumeration results
2. Load and normalize theHarvester results
3. Load and normalize SpiderFoot scan results
4. Load and normalize breach/HIBP results
5. Load and combine findings from all OSINT sources
6. Extract unique identifiers (usernames, emails, IPs, domains) from findings
7. Perform cross-source correlation to identify linked entities
8. Generate structured OSINT correlation report

Tools & resources

- Supported Data Sources
- Correlation Confidence Scoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_sherlock_results` — Load and normalize Sherlock username enumeration results.
- `load_harvester_results` — Load and normalize theHarvester results.
- `load_spiderfoot_results` — Load and normalize SpiderFoot scan results.
- `load_breach_results` — Load and normalize breach/HIBP results.
- `normalize_all_sources` — Load and combine findings from all OSINT sources.
- `extract_identifiers` — Extract unique identifiers (usernames, emails, IPs, domains) from findings.
- `correlate_findings` — Perform cross-source correlation to identify linked entities.
- `generate_report` — Generate structured OSINT correlation report.
- `generate_markdown_report` — Generate a Markdown intelligence profile from the report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ai-driven-osint-correlation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-alert-triage-with-elastic-siem (Skill Module)

performing-alert-triage-with-elastic-siem\SKILL.md

name: performing-alert-triage-with-elastic-siem
description: Alert Triage with Elastic SIEM agent - queries Elasticsearch SIEM signals index to retrieve, prioritize, and triage security alerts using elasticsearch-py client. Use when the user asks to perform alert triage with elastic siem.
version: 1.0.0
tags:
source: performing-alert-triage-with-elastic-siem

Performing Alert Triage with Elastic SIEM

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Elastic, Triage Prioritization, SOC Maturity Model

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** alert triage with elastic siem.
- Capability: Alert Triage with Elastic SIEM agent - queries Elasticsearch SIEM signals index to retrieve, prioritize, and triage security alerts using elasticsearch-py client.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, alert, triage, elastic, siem`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create Elasticsearch client
2. Retrieve open SIEM alerts from the signals index
3. Aggregate alerts by rule, severity, and host
4. Sort and prioritize alerts by severity and risk score
5. Group related alerts that may represent a single incident
6. Run alert triage and build consolidated report

Key concepts

- Detection and Analysis (triage is the core of this phase)
- Prioritization based on functional impact, information impact, and recoverability

Tools & resources

- Elastic
- Triage Prioritization
- SOC Maturity Model

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `create_client` — Create Elasticsearch client.
- `get_open_alerts` — Retrieve open SIEM alerts from the signals index.
- `get_alert_aggregations` — Aggregate alerts by rule, severity, and host.
- `prioritize_alerts` — Sort and prioritize alerts by severity and risk score.
- `identify_alert_clusters` — Group related alerts that may represent a single incident.
- `generate_report` — Run alert triage and build consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-alert-triage-with-elastic-siem`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-android-app-static-analysis-with-mobsf (Skill Module)

performing-android-app-static-analysis-with-mobsf\SKILL.md

name: performing-android-app-static-analysis-with-mobsf
description: MobSF Android static analysis agent. Automates APK upload, static analysis scanning, and report retrieval via the MobSF REST API. Extracts security findings including manifest analysis, code analysis, binary.... Use when the user asks to perform android app static analysis with mobsf.
version: 1.0.0
tags:
source: performing-android-app-static-analysis-with-mobsf

Performing Android App Static Analysis with Mobsf

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Core Operations

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** android app static analysis with mobsf.
- Capability: MobSF Android static analysis agent. Automates APK upload, static analysis scanning, and report retrieval via the MobSF REST API. Extracts security findings including manifest analysis, code analysis, binary analysis, and certificate checks.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, android, static, analysis, mobsf`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return MobSF server URL and API key from env or defaults
2. Upload an APK file to MobSF and return the file hash
3. Trigger static analysis scan on the uploaded APK
4. Retrieve the JSON report for a scanned APK
5. Retrieve the security scorecard for a scanned APK
6. Extract structured security findings from MobSF report JSON
7. Print a human-readable summary of the analysis

Key concepts

- Section 4.1: Static analysis as mandatory step in mobile app vetting process
- Section 4.2: Automated tools should check for known vulnerability patterns
- Section 5: Integration of vetting into enterprise mobile device management

Tools & resources

- Core Operations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_mobsf_config` — Return MobSF server URL and API key from env or defaults.
- `upload_apk` — Upload an APK file to MobSF and return the file hash.
- `start_scan` — Trigger static analysis scan on the uploaded APK.
- `get_report` — Retrieve the JSON report for a scanned APK.
- `get_scorecard` — Retrieve the security scorecard for a scanned APK.
- `extract_findings` — Extract structured security findings from MobSF report JSON.
- `format_summary` — Print a human-readable summary of the analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-android-app-static-analysis-with-mobsf`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-api-fuzzing-with-restler (Skill Module)

performing-api-fuzzing-with-restler\SKILL.md

name: performing-api-fuzzing-with-restler
description: RESTler API fuzzing orchestration and result analysis. Use when the user asks to perform api fuzzing with restler.
version: 1.0.0
tags:
source: performing-api-fuzzing-with-restler

Performing API Fuzzing with Restler

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Key CLI Flags, Security Checkers, Output Directory Structure, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** api fuzzing with restler.
- Capability: RESTler API fuzzing orchestration and result analysis agent.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, fuzzing, restler`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compile OpenAPI spec into RESTler fuzzing grammar
2. Run RESTler in test, fuzz-lean, or fuzz mode
3. Parse RESTler run summary JSON from results directory
4. Parse RESTler bug bucket files for discovered vulnerabilities
5. Generate a security-focused fuzzing dictionary for RESTler
6. Execute RESTler fuzzing audit workflow

Tools & resources

- Key CLI Flags
- Security Checkers
- Output Directory Structure
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compile_spec` — Compile OpenAPI spec into RESTler fuzzing grammar.
- `run_fuzz_mode` — Run RESTler in test, fuzz-lean, or fuzz mode.
- `parse_run_summary` — Parse RESTler run summary JSON from results directory.
- `parse_bug_buckets` — Parse RESTler bug bucket files for discovered vulnerabilities.
- `generate_custom_dictionary` — Generate a security-focused fuzzing dictionary for RESTler.
- `run_audit` — Execute RESTler fuzzing audit workflow.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-api-fuzzing-with-restler`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-api-inventory-and-discovery (Skill Module)

performing-api-inventory-and-discovery\SKILL.md

name: performing-api-inventory-and-discovery
description: API inventory and discovery agent for attack surface mapping. Use when the user asks to perform api inventory and discovery.
version: 1.0.0
tags:
source: performing-api-inventory-and-discovery

Performing API Inventory and Discovery

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Common API Discovery Paths, OpenAPI Spec Parsing, API Risk Classification, OWASP API9, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** api inventory and discovery.
- Capability: API inventory and discovery agent for attack surface mapping.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, inventory, discovery`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Probe common API paths to discover active endpoints
2. Fetch and parse OpenAPI/Swagger spec to inventory endpoints
3. Fetch JavaScript file and extract API endpoint references
4. Use DNS enumeration to find API subdomains
5. Classify discovered APIs by risk level
6. Execute API inventory and discovery audit

Tools & resources

- Common API Discovery Paths
- OpenAPI Spec Parsing
- API Risk Classification
- OWASP API9
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_api_endpoints` — Probe common API paths to discover active endpoints.
- `parse_swagger_spec` — Fetch and parse OpenAPI/Swagger spec to inventory endpoints.
- `scan_javascript_for_apis` — Fetch JavaScript file and extract API endpoint references.
- `enumerate_subdomains_for_apis` — Use DNS enumeration to find API subdomains.
- `classify_api_risk` — Classify discovered APIs by risk level.
- `run_audit` — Execute API inventory and discovery audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-api-inventory-and-discovery`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-api-rate-limiting-bypass (Skill Module)

performing-api-rate-limiting-bypass\SKILL.md

name: performing-api-rate-limiting-bypass
description: API rate limiting bypass testing. Use when the user asks to perform api rate limiting bypass.
version: 1.0.0
tags:
source: performing-api-rate-limiting-bypass

Performing API Rate Limiting Bypass

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Rate Limit Response Headers, IP Spoofing Bypass Headers, Bypass Techniques, OWASP API4, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** api rate limiting bypass.
- Capability: API rate limiting bypass testing agent.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, rate, limiting, bypass`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Send initial request and extract rate limit response headers
2. Test rate limit bypass via IP spoofing headers
3. Test if rate limiting applies across HTTP methods
4. Test rate limit bypass via URL path manipulation
5. Test rate limit bypass via parameter encoding variations
6. Execute API rate limiting bypass audit

Tools & resources

- Rate Limit Response Headers
- IP Spoofing Bypass Headers
- Bypass Techniques
- OWASP API4
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_rate_limit_headers` — Send initial request and extract rate limit response headers.
- `test_header_bypass` — Test rate limit bypass via IP spoofing headers.
- `test_method_bypass` — Test if rate limiting applies across HTTP methods.
- `test_path_bypass` — Test rate limit bypass via URL path manipulation.
- `test_encoding_bypass` — Test rate limit bypass via parameter encoding variations.
- `run_audit` — Execute API rate limiting bypass audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-api-rate-limiting-bypass`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-api-security-testing-with-postman (Skill Module)

performing-api-security-testing-with-postman\SKILL.md

name: performing-api-security-testing-with-postman
description: Postman API security testing orchestration agent using Newman CLI. Use when the user asks to perform api security testing with postman.
version: 1.0.0
tags:
source: performing-api-security-testing-with-postman

Performing API Security Testing with Postman

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Tools, Postman Test Script Functions, OWASP API Security Tests, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** api security testing with postman.
- Capability: Postman API security testing orchestration agent using Newman CLI.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, security, testing, postman`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute a Postman collection using Newman CLI
2. Parse Newman JSON results for security test outcomes
3. Generate Postman collection for BOLA/IDOR testing across two user contexts
4. Generate Postman collection for injection testing
5. Execute Postman API security testing audit

Tools & resources

- Tools
- Postman Test Script Functions
- OWASP API Security Tests
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_newman_collection` — Execute a Postman collection using Newman CLI.
- `parse_newman_results` — Parse Newman JSON results for security test outcomes.
- `generate_bola_collection` — Generate Postman collection for BOLA/IDOR testing across two user contexts.
- `generate_injection_collection` — Generate Postman collection for injection testing.
- `run_audit` — Execute Postman API security testing audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-api-security-testing-with-postman`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-arp-spoofing-attack-simulation (Skill Module)

performing-arp-spoofing-attack-simulation\SKILL.md

name: performing-arp-spoofing-attack-simulation
description: ARP Spoofing Attack Simulation Agent — AUTHORIZED TESTING ONLY Simulates ARP spoofing attacks using Scapy in controlled lab environments to test network detection capabilities and validate DAI countermeasures..... Use when the user asks to perform arp spoofing attack simulation.
version: 1.0.0
tags:
source: performing-arp-spoofing-attack-simulation

Performing Arp Spoofing Attack Simulation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: ARP Packet Fields, Safety Controls

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** arp spoofing attack simulation.
- Capability: ARP Spoofing Attack Simulation Agent — AUTHORIZED TESTING ONLY Simulates ARP spoofing attacks using Scapy in controlled lab environments to test network detection capabilities and validate DAI countermeasures. WARNING: Only use with explicit written authorization on isolated test networks.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, spoofing, attack, simulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Resolve MAC address for a given IP using ARP request
2. Scan local network segment to discover active hosts
3. Craft ARP poison packets for target and gateway
4. Send ARP poison packets and log the activity
5. Restore legitimate ARP entries to undo spoofing
6. Verify that security controls detected the ARP spoofing attempt
7. Generate ARP spoofing simulation report

Tools & resources

- ARP Packet Fields
- Safety Controls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_mac` — Resolve MAC address for a given IP using ARP request.
- `scan_network` — Scan local network segment to discover active hosts.
- `craft_arp_poison_packets` — Craft ARP poison packets for target and gateway.
- `send_arp_poison` — Send ARP poison packets and log the activity.
- `restore_arp` — Restore legitimate ARP entries to undo spoofing.
- `verify_detection` — Verify that security controls detected the ARP spoofing attempt.
- `generate_report` — Generate ARP spoofing simulation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-arp-spoofing-attack-simulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-asset-criticality-scoring-for-vulns (Skill Module)

performing-asset-criticality-scoring-for-vulns\SKILL.md

name: performing-asset-criticality-scoring-for-vulns
description: Asset criticality scoring agent for vulnerability prioritization. Use when the user asks to perform asset criticality scoring for vulns.
version: 1.0.0
tags:
source: performing-asset-criticality-scoring-for-vulns

Performing Asset Criticality Scoring for Vulns

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Criticality Scoring Factors, Data Sensitivity Levels, Criticality Tiers, Risk-Adjusted Priority Formula, External References, Industry Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** asset criticality scoring for vulns.
- Capability: Asset criticality scoring agent for vulnerability prioritization.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, asset, criticality, scoring, vulns`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load asset inventory from CSV file
2. Calculate weighted criticality score for a single asset
3. Combine CVSS score with asset criticality for risk-adjusted priority
4. Generate remediation SLA based on asset criticality tier
5. Execute asset criticality scoring audit

Key concepts

- NIST SP 800-30 Rev 1: Guide for Conducting Risk Assessments
- NIST CSF 2.0 ID.AM: Asset Management
- CIS Controls v8.1 Control 1: Inventory and Control of Enterprise Assets
- ISO 27001: 2022 A.5.9**: Inventory of information and other associated assets
- FAIR Framework: Factor Analysis of Information Risk
- NIST CMDB Best Practices
- ITIL Asset Management Framework
- CIS RAM (Risk Assessment Method)

Tools & resources

- Criticality Scoring Factors
- Data Sensitivity Levels
- Criticality Tiers
- Risk-Adjusted Priority Formula
- External References

- Industry Standards
- Scoring References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_asset_inventory` — Load asset inventory from CSV file.
- `calculate_criticality_score` — Calculate weighted criticality score for a single asset.
- `calculate_risk_adjusted_priority` — Combine CVSS score with asset criticality for risk-adjusted priority.
- `generate_sla_matrix` — Generate remediation SLA based on asset criticality tier.
- `run_audit` — Execute asset criticality scoring audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-asset-criticality-scoring-for-vulns`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-authenticated-scan-with-openvas (Skill Module)

performing-authenticated-scan-with-openvas\SKILL.md

name: performing-authenticated-scan-with-openvas
description: OpenVAS/GVM authenticated vulnerability scan orchestration. Use when the user asks to perform authenticated scan with openvas.
version: 1.0.0
tags:
source: performing-authenticated-scan-with-openvas

Performing Authenticated Scan with Openvas

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Key GMP Methods, Scan Configuration IDs, Credential Types, External References, Primary Standards, OpenVAS/GVM Technical References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** authenticated scan with openvas.
- Capability: OpenVAS/GVM authenticated vulnerability scan orchestration agent.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, authenticated, scan, openvas`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Connect to GVM daemon via Unix socket
2. List available scan configurations
3. List configured scan credentials
4. Create SSH credential for authenticated Linux scans
5. Create scan target with optional credentials
6. Create and start a scan task
7. Check scan task progress and status
8. Fetch vulnerability results from a completed scan report

Key concepts

- Title: Technical Guide to Information Security Testing and Assessment
- Relevance: Defines vulnerability scanning methodologies including credentialed vs non-credentialed approaches
- Title: Vulnerability Monitoring and Scanning
- Requirement: Organizations must scan for vulnerabilities in information systems and hosted applications with both authentic
- Title: Continuous Vulnerability Management
- Title: External and Internal Vulnerabilities Are Regularly Identified, Prioritized, and Addressed
- Requirement: Internal vulnerability scans must be performed at least quarterly and after any significant change; authentica
- Components: gvmd (manager), openvas-scanner, gsad (web UI), ospd-openvas

Tools & resources

- Key GMP Methods
- Scan Configuration IDs

- Credential Types
- External References
- Primary Standards
- OpenVAS/GVM Technical References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `connect_gvm` — Connect to GVM daemon via Unix socket.
- `list_scan_configs` — List available scan configurations.
- `list_credentials` — List configured scan credentials.
- `create_ssh_credential` — Create SSH credential for authenticated Linux scans.
- `create_target` — Create scan target with optional credentials.
- `create_and_start_task` — Create and start a scan task.
- `get_task_status` — Check scan task progress and status.
- `get_report_results` — Fetch vulnerability results from a completed scan report.
- `parse_openvas_xml_report` — Parse an exported OpenVAS XML report file.
- `run_audit` — Execute OpenVAS scan audit and analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-authenticated-scan-with-openvas`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-authenticated-vulnerability-scan (Skill Module)

performing-authenticated-vulnerability-scan\SKILL.md

name: performing-authenticated-vulnerability-scan
description: Authenticated vulnerability scan orchestration agent using Nessus API. Use when the user asks to perform authenticated vulnerability scan.
version: 1.0.0
tags:
source: performing-authenticated-vulnerability-scan

Performing Authenticated Vulnerability Scan

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Severity Levels, Key Nessus Plugin Families, External References, Industry Standards, Credential Management Standards, Scanner Documentation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** authenticated vulnerability scan.
- Capability: Authenticated vulnerability scan orchestration agent using Nessus API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, authenticated, vulnerability, scan`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze scan results and categorize findings
2. Check if authenticated scan achieved credential coverage
3. Compare authenticated vs unauthenticated scan finding counts
4. Execute authenticated vulnerability scan audit

Key concepts

- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment
- NIST SP 800-53 RA-5: Vulnerability Scanning (requires credentialed scanning for compliance)
- CIS Controls v8 Control 7.5: Perform automated vulnerability scans of internal assets on a quarterly basis using authenticated scanning
- PCI DSS v4.0 Req 11.3.1: Internal vulnerability scans must use authenticated scanning
- DISA STIG: Requires credentialed scanning for compliance validation
- NIST SP 800-63B: Digital Identity Guidelines - Authentication and Lifecycle Management
- CIS Controls v8 Control 5: Account Management
- OWASP Credential Storage Cheat Sheet: Secure credential handling best practices

Tools & resources

- Severity Levels
- Key Nessus Plugin Families
- External References
- Industry Standards
- Credential Management Standards
- Scanner Documentation

- Minimum Privileges Required

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_scan_results` — Analyze scan results and categorize findings.
- `audit_credential_coverage` — Check if authenticated scan achieved credential coverage.
- `compare_auth_vs_unauth` — Compare authenticated vs unauthenticated scan finding counts.
- `run_audit` — Execute authenticated vulnerability scan audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-authenticated-vulnerability-scan`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-automated-malware-analysis-with-cape (Skill Module)

performing-automated-malware-analysis-with-cape\SKILL.md

name: performing-automated-malware-analysis-with-cape
description: CAPE Sandbox automated malware analysis agent. Submits malware samples to a CAPE Sandbox instance via its REST API, polls for analysis completion, and retrieves behavioral reports including process trees, network.... Use when the user asks to perform automated malware analysis with cape.
version: 1.0.0
tags:
source: performing-automated-malware-analysis-with-cape

Performing Automated Malware Analysis with Cape

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Core Operations, Applicable Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** automated malware analysis with cape.
- Capability: CAPE Sandbox automated malware analysis agent. Submits malware samples to a CAPE Sandbox instance via its REST API, polls for analysis completion, and retrieves behavioral reports including process trees, network activity, dropped files, and extracted configs.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, automated, malware, analysis, cape`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return CAPE server URL and optional API token
2. Submit a file to CAPE for analysis
3. Submit a URL to CAPE for analysis
4. Poll CAPE until the analysis task completes or times out
5. Retrieve the full analysis report for a completed task
6. Extract structured findings from CAPE report
7. Print human-readable analysis summary

Key concepts

- MITRE ATT&CK Framework
- NIST SP 800-83 Guide to Malware Incident Prevention
- NIST SP 800-86 Guide to Integrating Forensic Techniques

Tools & resources

- Core Operations
- Applicable Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_cape_config` — Return CAPE server URL and optional API token.
- `submit_file` — Submit a file to CAPE for analysis.
- `submit_url` — Submit a URL to CAPE for analysis.
- `poll_task_status` — Poll CAPE until the analysis task completes or times out.
- `get_report` — Retrieve the full analysis report for a completed task.
- `extract_findings` — Extract structured findings from CAPE report.
- `format_summary` — Print human-readable analysis summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-automated-malware-analysis-with-cape`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-aws-account-enumeration-with-scout-suite (Skill Module)

performing-aws-account-enumeration-with-scout-suite\SKILL.md

name: performing-aws-account-enumeration-with-scout-suite
description: ScoutSuite AWS account enumeration and security audit agent. Wraps the ScoutSuite CLI to perform comprehensive AWS security audits, parses the generated JSON results, and produces a structured findings report.... Use when the user asks to perform aws account enumeration with scout suite.
version: 1.0.0
tags:
source: performing-aws-account-enumeration-with-scout-suite

Performing AWS Account Enumeration with Scout Suite

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS, Common Findings Categories, Industry Standards, ScoutSuite Rule Mappings, Compliance Framework Coverage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** aws account enumeration with scout suite.
- Capability: ScoutSuite AWS account enumeration and security audit agent. Wraps the ScoutSuite CLI to perform comprehensive AWS security audits, parses the generated JSON results, and produces a structured findings report covering IAM, S3, EC2, RDS, Lambda, and other AWS services.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, account, enumeration, scout, suite`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Locate the scout CLI binary
2. Execute ScoutSuite AWS scan
3. Find the most recent ScoutSuite results JSON file
4. Parse ScoutSuite results JavaScript file into Python dict
5. Extract security findings from ScoutSuite results
6. Extract AWS account metadata from results
7. Print a human-readable summary

Key concepts

- Section 1: Identity and Access Management
- SEC 1: Securely operate your workload
- SEC 2: Manage identities for people and machines
- SEC 3: Manage permissions for people and machines
- SEC 6: Protect compute resources
- SEC 8: Protect data at rest
- SEC 9: Protect data in transit
- AC-2: Account Management

Tools & resources

- AWS

- Common Findings Categories
- Industry Standards
- ScoutSuite Rule Mappings
- Compliance Framework Coverage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `find_scoutsuite_binary` — Locate the scout CLI binary.
- `run_scoutsuite` — Execute ScoutSuite AWS scan.
- `find_latest_results` — Find the most recent ScoutSuite results JSON file.
- `parse_results` — Parse ScoutSuite results JavaScript file into Python dict.
- `extract_findings` — Extract security findings from ScoutSuite results.
- `extract_account_info` — Extract AWS account metadata from results.
- `format_summary` — Print a human-readable summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-aws-account-enumeration-with-scout-suite`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-aws-privilege-escalation-assessment (Skill Module)

performing-aws-privilege-escalation-assessment\SKILL.md

name: performing-aws-privilege-escalation-assessment
description: AWS Privilege Escalation Assessment Agent — AUTHORIZED TESTING ONLY Assesses AWS IAM configurations for privilege escalation paths using boto3 and enumerates dangerous policy combinations. WARNING: Only use with.... Use when the user asks to perform aws privilege escalation assessment.
version: 1.0.0
tags:
source: performing-aws-privilege-escalation-assessment

Performing AWS Privilege Escalation Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS, Dangerous IAM Actions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** aws privilege escalation assessment.
- Capability: AWS Privilege Escalation Assessment Agent — AUTHORIZED TESTING ONLY Assesses AWS IAM configurations for privilege escalation paths using boto3 and enumerates dangerous policy combinations. WARNING: Only use with explicit written authorization on approved AWS accounts.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, privilege, escalation, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Get current AWS identity information
2. Enumerate all IAM users and their attached policies
3. Enumerate IAM roles and their trust policies
4. Identify users with dangerous permission combinations for privilege escalation
5. Identify role chaining opportunities for privilege escalation
6. Generate privilege escalation assessment report

Tools & resources

- AWS
- Dangerous IAM Actions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_caller_identity` — Get current AWS identity information.
- `enumerate_iam_users` — Enumerate all IAM users and their attached policies.
- `enumerate_iam_roles` — Enumerate IAM roles and their trust policies.
- `check_dangerous_permissions` — Identify users with dangerous permission combinations for privilege escalation.
- `check_role_chaining` — Identify role chaining opportunities for privilege escalation.
- `generate_report` — Generate privilege escalation assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-aws-privilege-escalation-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-bandwidth-throttling-attack-simulation (Skill Module)

performing-bandwidth-throttling-attack-simulation\SKILL.md

name: performing-bandwidth-throttling-attack-simulation
description: Bandwidth Throttling Attack Simulation Agent — AUTHORIZED TESTING ONLY Simulates bandwidth degradation attacks using Scapy and tc (traffic control) to test QoS controls and network monitoring detection capabilities..... Use when the user asks to perform bandwidth throttling attack simulation.
version: 1.0.0
tags:
source: performing-bandwidth-throttling-attack-simulation

Performing Bandwidth Throttling Attack Simulation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Scapy Library, Safety Controls

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** bandwidth throttling attack simulation.
- Capability: Bandwidth Throttling Attack Simulation Agent — AUTHORIZED TESTING ONLY Simulates bandwidth degradation attacks using Scapy and tc (traffic control) to test QoS controls and network monitoring detection capabilities. WARNING: Only use with explicit written authorization on isolated test networks.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, bandwidth, throttling, attack, simulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Configure tc (traffic control) to throttle bandwidth on an interface
2. Remove tc throttling rules from interface
3. Generate controlled bandwidth consumption traffic using Scapy
4. Measure baseline bandwidth using iperf3 client
5. Generate bandwidth throttling simulation report

Tools & resources

- Scapy Library
- Safety Controls

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `setup_tc_throttle` — Configure tc (traffic control) to throttle bandwidth on an interface.
- `remove_tc_throttle` — Remove tc throttling rules from interface.
- `generate_bandwidth_flood` — Generate controlled bandwidth consumption traffic using Scapy.
- `measure_baseline` — Measure baseline bandwidth using iperf3 client.
- `generate_report` — Generate bandwidth throttling simulation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-bandwidth-throttling-attack-simulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-binary-exploitation-analysis (Skill Module)

performing-binary-exploitation-analysis\SKILL.md

name: performing-binary-exploitation-analysis
description: Binary exploitation analysis agent. # For authorized security testing and CTF challenges only Analyzes ELF binaries for security mitigations, discovers ROP gadgets, and assists exploit development using pwntools and.... Use when the user asks to perform binary exploitation analysis.
version: 1.0.0
tags:
source: performing-binary-exploitation-analysis

Performing Binary Exploitation Analysis

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: checksec CLI, ROPgadget CLI, Dangerous Functions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** binary exploitation analysis.
- Capability: Binary exploitation analysis agent. # For authorized security testing and CTF challenges only Analyzes ELF binaries for security mitigations, discovers ROP gadgets, and assists exploit development using pwntools and checksec.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, binary, exploitation, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze binary security mitigations using checksec
2. Find ROP gadgets using pwntools or ROPgadget
3. Find useful functions for exploitation (system, exec, write, etc.)
4. Identify potentially vulnerable functions in the binary
5. Full binary exploitation analysis

Tools & resources

- checksec CLI
- ROPgadget CLI
- Dangerous Functions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_checksec` — Analyze binary security mitigations using checksec.
- `find_rop_gadgets` — Find ROP gadgets using pwntools or ROPgadget.
- `find_useful_functions` — Find useful functions for exploitation (system, exec, write, etc.).
- `find_vulnerable_functions` — Identify potentially vulnerable functions in the binary.
- `analyze_binary` — Full binary exploitation analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-binary-exploitation-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-blind-ssrf-exploitation (Skill Module)

performing-blind-ssrf-exploitation\SKILL.md

name: performing-blind-ssrf-exploitation
description: Blind SSRF detection agent. Tests web application endpoints for Server-Side Request Forgery (SSRF) vulnerabilities by injecting payloads that trigger out-of-band callbacks. Uses configurable payload lists targeting.... Use when the user asks to perform blind ssrf exploitation.
version: 1.0.0
tags:
source: performing-blind-ssrf-exploitation

Performing Blind SSRF Exploitation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Techniques and Payloads

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** blind ssrf exploitation.
- Capability: Blind SSRF detection agent. Tests web application endpoints for Server-Side Request Forgery (SSRF) vulnerabilities by injecting payloads that trigger out-of-band callbacks. Uses configurable payload lists targeting internal services, cloud metadata endpoints, and external callback receivers. AUTHORIZED TESTING ONLY: Only use against targets you have explicit written permission to test. Unauthorized SSRF testing is illegal.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, blind, ssrf, exploitation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test a single SSRF payload against a parameter
2. Analyze HTTP response for SSRF success indicators
3. Run SSRF tests across payload categories
4. Print scan summary

Tools & resources

- Techniques and Payloads

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_ssrf_parameter` — Test a single SSRF payload against a parameter.
- `analyze_response` — Analyze HTTP response for SSRF success indicators.
- `run_ssrf_scan` — Run SSRF tests across payload categories.
- `format_summary` — Print scan summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-blind-ssrf-exploitation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-bluetooth-security-assessment (Skill Module)

performing-bluetooth-security-assessment\SKILL.md

name: performing-bluetooth-security-assessment
description: BLE security assessment using bleak for device scanning and GATT enumeration. Use when the user asks to perform bluetooth security assessment.
version: 1.0.0
tags:
source: performing-bluetooth-security-assessment

Performing Bluetooth Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Common GATT Service UUIDs, BLE Security Modes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** bluetooth security assessment.
- Capability: BLE security assessment using bleak for device scanning and GATT enumeration.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, bluetooth, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for bluetooth security assessment
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Common GATT Service UUIDs
- BLE Security Modes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-bluetooth-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-brand-monitoring-for-impersonation (Skill Module)

performing-brand-monitoring-for-impersonation\SKILL.md

name: performing-brand-monitoring-for-impersonation
description: Brand impersonation monitoring agent. Monitors for brand impersonation by checking Certificate Transparency logs for suspicious domain registrations, performing DNS lookups for typosquatting domains, and scanning.... Use when the user asks to perform brand monitoring for impersonation.
version: 1.0.0
tags:
source: performing-brand-monitoring-for-impersonation

Performing Brand Monitoring for Impersonation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Certificate Transparency Log Monitoring

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** brand monitoring for impersonation.
- Capability: Brand impersonation monitoring agent. Monitors for brand impersonation by checking Certificate Transparency logs for suspicious domain registrations, performing DNS lookups for typosquatting domains, and scanning social media profile names. Uses crt.sh API and DNS resolution to identify potential phishing domains.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, brand, monitoring, impersonation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate common typosquatting domain variants
2. Check which typosquat domains actually resolve
3. Search crt.sh for certificates containing the brand name
4. Print monitoring summary

Tools & resources

- Certificate Transparency Log Monitoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_typosquat_variants` — Generate common typosquatting domain variants.
- `check_domain_resolution` — Check which typosquat domains actually resolve.
- `search_certificate_transparency` — Search crt.sh for certificates containing the brand name.
- `format_summary` — Print monitoring summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-brand-monitoring-for-impersonation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-clickjacking-attack-test (Skill Module)

performing-clickjacking-attack-test\SKILL.md

name: performing-clickjacking-attack-test
description: Clickjacking Attack Test Agent — AUTHORIZED TESTING ONLY Tests web applications for clickjacking (UI redressing) vulnerabilities by checking frame-busting headers and generating proof-of-concept pages. WARNING: Only.... Use when the user asks to perform clickjacking attack test.
version: 1.0.0
tags:
source: performing-clickjacking-attack-test

Performing Clickjacking Attack Test

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: HTTP Security Headers, requests Library, PoC HTML Elements, OWASP Testing Guide

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** clickjacking attack test.
- Capability: Clickjacking Attack Test Agent — AUTHORIZED TESTING ONLY Tests web applications for clickjacking (UI redressing) vulnerabilities by checking frame-busting headers and generating proof-of-concept pages. WARNING: Only use with explicit written authorization for the target application.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, clickjacking, attack, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check X-Frame-Options and CSP frame-ancestors headers
2. Check multiple endpoints for clickjacking protection
3. Generate clickjacking proof-of-concept HTML page
4. Check for JavaScript-based frame-busting code
5. Generate clickjacking test report

Tools & resources

- HTTP Security Headers
- requests Library
- PoC HTML Elements
- OWASP Testing Guide

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_frame_headers` — Check X-Frame-Options and CSP frame-ancestors headers.
- `check_multiple_endpoints` — Check multiple endpoints for clickjacking protection.
- `generate_poc_html` — Generate clickjacking proof-of-concept HTML page.
- `check_javascript_frame_busting` — Check for JavaScript-based frame-busting code.
- `generate_report` — Generate clickjacking test report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-clickjacking-attack-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-asset-inventory-with-cartography (Skill Module)

performing-cloud-asset-inventory-with-cartography\SKILL.md

name: performing-cloud-asset-inventory-with-cartography
description: Cartography cloud asset inventory agent. Wraps the Cartography tool to enumerate and inventory cloud assets across AWS accounts, then queries the resulting Neo4j graph database to identify security-relevant.... Use when the user asks to perform cloud asset inventory with cartography.
version: 1.0.0
tags:
source: performing-cloud-asset-inventory-with-cartography

Performing Cloud Asset Inventory with Cartography

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Cartography Node Types, CIS Controls v8, NIST 800-53

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud asset inventory with cartography.
- Capability: Cartography cloud asset inventory agent. Wraps the Cartography tool to enumerate and inventory cloud assets across AWS accounts, then queries the resulting Neo4j graph database to identify security-relevant relationships, exposed resources, and misconfigured assets.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, asset, inventory, cartography`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute Cartography to populate the Neo4j graph
2. Execute a Cypher query against the Neo4j graph
3. Query EC2 instances from the graph
4. Query S3 buckets from the graph
5. Find publicly accessible resources
6. Find unencrypted storage resources
7. Print inventory summary

Key concepts

- Control 1: Inventory and Control of Enterprise Assets
- Control 2: Inventory and Control of Software Assets
- Control 3: Data Protection
- Control 6: Access Control Management
- CM-8: System Component Inventory
- PM-5: System Inventory
- RA-2: Security Categorization
- CCM AIS-01: Application and Interface Security

Tools & resources

- Cartography Node Types

- CIS Controls v8
- NIST 800-53

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_cartography` — Execute Cartography to populate the Neo4j graph.
- `query_neo4j` — Execute a Cypher query against the Neo4j graph.
- `inventory_ec2_instances` — Query EC2 instances from the graph.
- `inventory_s3_buckets` — Query S3 buckets from the graph.
- `find_public_resources` — Find publicly accessible resources.
- `find_unencrypted_resources` — Find unencrypted storage resources.
- `format_summary` — Print inventory summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-asset-inventory-with-cartography`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-forensics-investigation (Skill Module)

performing-cloud-forensics-investigation\SKILL.md

name: performing-cloud-forensics-investigation
description: Cloud Forensics Investigation Agent Collects and analyzes forensic evidence from AWS cloud environments including CloudTrail logs, EC2 snapshots, and IAM activity for incident response. Use when the user asks to perform cloud forensics investigation.
version: 1.0.0
tags:
source: performing-cloud-forensics-investigation

Performing Cloud Forensics Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS IAM API, Required IAM Permissions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud forensics investigation.
- Capability: Cloud Forensics Investigation Agent Collects and analyzes forensic evidence from AWS cloud environments including CloudTrail logs, EC2 snapshots, and IAM activity for incident response.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, forensics, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Collect CloudTrail management events for the investigation window
2. Identify suspicious CloudTrail events indicating compromise
3. Create forensic snapshots of all EBS volumes attached to an instance
4. Collect IAM activity for a specific user
5. Generate cloud forensics investigation report

Tools & resources

- AWS IAM API
- Required IAM Permissions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `collect_cloudtrail_events` — Collect CloudTrail management events for the investigation window.
- `identify_suspicious_activity` — Identify suspicious CloudTrail events indicating compromise.
- `snapshot_ec2_instance` — Create forensic snapshots of all EBS volumes attached to an instance.
- `collect_iam_activity` — Collect IAM activity for a specific user.
- `generate_report` — Generate cloud forensics investigation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-forensics-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-forensics-with-aws-cloudtrail (Skill Module)

performing-cloud-forensics-with-aws-cloudtrail\SKILL.md

name: performing-cloud-forensics-with-aws-cloudtrail
description: AWS CloudTrail Forensics Agent - investigates API activity for incident response using boto3. Use when the user asks to perform cloud forensics with aws cloudtrail.
version: 1.0.0
tags:
source: performing-cloud-forensics-with-aws-cloudtrail

Performing Cloud Forensics with AWS Cloudtrail

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS, boto3 CloudTrail Client, lookup_events, Paginator Usage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud forensics with aws cloudtrail.
- Capability: AWS CloudTrail Forensics Agent - investigates API activity for incident response using boto3.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, forensics, cloudtrail`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Query CloudTrail using lookup_events with pagination
2. Flag events matching suspicious API calls
3. Identify persistence mechanisms created by attacker
4. Analyze source IP distribution for anomalies
5. Analyze user agents for tool identification
6. Build chronological attack timeline
7. Generate forensic investigation report

Tools & resources

- AWS
- boto3 CloudTrail Client
- lookup_events
- Paginator Usage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `lookup_events` — Query CloudTrail using lookup_events with pagination.
- `detect_suspicious_activity` — Flag events matching suspicious API calls.
- `detect_persistence` — Identify persistence mechanisms created by attacker.
- `analyze_source_ips` — Analyze source IP distribution for anomalies.
- `analyze_user_agents` — Analyze user agents for tool identification.
- `build_timeline` — Build chronological attack timeline.
- `generate_report` — Generate forensic investigation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-forensics-with-aws-cloudtrail`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-incident-containment-procedures (Skill Module)

performing-cloud-incident-containment-procedures\SKILL.md

name: performing-cloud-incident-containment-procedures
description: AWS cloud incident containment agent. Automates incident containment procedures in AWS environments including EC2 instance isolation, IAM credential revocation, security group lockdown, S3 bucket access restriction,.... Use when the user asks to perform cloud incident containment procedures.
version: 1.0.0
tags:
source: performing-cloud-incident-containment-procedures

Performing Cloud Incident Containment Procedures

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Containment Actions, AWS Well-Architected Framework, Microsoft Cloud Security Benchmark, GCP Security Best Practices

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud incident containment procedures.
- Capability: AWS cloud incident containment agent. Automates incident containment procedures in AWS environments including EC2 instance isolation, IAM credential revocation, security group lockdown, S3 bucket access restriction, and forensic snapshot creation using boto3.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, incident, containment, procedures`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Create a boto3 session
2. Isolate an EC2 instance by replacing security groups with a deny-all SG
3. Create EBS snapshots for forensic preservation
4. Revoke all IAM credentials for a compromised user
5. Restrict S3 bucket access during incident containment
6. Print containment summary

Key concepts

- Containment strategies for cloud environments
- Evidence preservation in ephemeral infrastructure
- Cloud Security Alliance incident response procedures
- Shared responsibility model for incident handling
- Multi-cloud containment strategies
- Incident response automation with AWS services
- CloudTrail, GuardDuty, and Security Hub integration
- Azure Defender incident response procedures

Tools & resources

- Containment Actions
- AWS Well-Architected Framework

- Microsoft Cloud Security Benchmark
- GCP Security Best Practices

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `get_session` — Create a boto3 session.
- `isolate_ec2_instance` — Isolate an EC2 instance by replacing security groups with a deny-all SG.
- `create_forensic_snapshot` — Create EBS snapshots for forensic preservation.
- `revoke_iam_credentials` — Revoke all IAM credentials for a compromised user.
- `restrict_s3_bucket` — Restrict S3 bucket access during incident containment.
- `format_summary` — Print containment summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-incident-containment-procedures`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-log-forensics-with-athena (Skill Module)

performing-cloud-log-forensics-with-athena\SKILL.md

name: performing-cloud-log-forensics-with-athena
description: Agent for performing cloud log forensics using AWS Athena. Automates Athena table creation with partition projection and runs forensic SQL queries against CloudTrail, VPC Flow Logs, S3 access logs, and ALB logs. Use when the user asks to perform cloud log forensics with athena.
version: 1.0.0
tags:
source: performing-cloud-log-forensics-with-athena

Performing Cloud Log Forensics with Athena

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud log forensics with athena.
- Capability: Agent for performing cloud log forensics using AWS Athena. Automates Athena table creation with partition projection and runs forensic SQL queries against CloudTrail, VPC Flow Logs, S3 access logs, and ALB logs.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, forensics, athena`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for cloud log forensics with athena
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-log-forensics-with-athena`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-native-forensics-with-falco (Skill Module)

performing-cloud-native-forensics-with-falco\SKILL.md

name: performing-cloud-native-forensics-with-falco
description: Agent for managing Falco rules and parsing alerts for container forensics. Use when the user asks to perform cloud native forensics with falco.
version: 1.0.0
tags:
source: performing-cloud-native-forensics-with-falco

Performing Cloud Native Forensics with Falco

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Falco, Helm Deployment

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud native forensics with falco.
- Capability: Agent for managing Falco rules and parsing alerts for container forensics.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, native, forensics, falco`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate Falco rules YAML file
2. Parse Falco JSON alert output file
3. Generate alert summary statistics
4. Correlate Falco alerts with known suspicious container images

Tools & resources

- Falco
- Helm Deployment

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_falco_rules` — Generate Falco rules YAML file.
- `parse_falco_alerts` — Parse Falco JSON alert output file.
- `summarize_alerts` — Generate alert summary statistics.
- `check_falco_health`
- `get_falco_version`
- `correlate_alerts_with_k8s` — Correlate Falco alerts with known suspicious container images.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-native-forensics-with-falco`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-native-threat-hunting-with-aws-detective (Skill Module)

performing-cloud-native-threat-hunting-with-aws-detective\SKILL.md

name: performing-cloud-native-threat-hunting-with-aws-detective
description: AWS Detective Threat Hunting Script Lists behavior graphs, retrieves investigations, and analyzes entity indicators for cloud-native threat hunting. Use when the user asks to perform cloud native threat hunting with aws detective.
version: 1.0.0
tags:
source: performing-cloud-native-threat-hunting-with-aws-detective

Performing Cloud Native Threat Hunting with AWS Detective

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud native threat hunting with aws detective.
- Capability: AWS Detective Threat Hunting Script Lists behavior graphs, retrieves investigations, and analyzes entity indicators for cloud-native threat hunting.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, native, threat, hunting, detective`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Review GuardDuty HIGH/CRITICAL findings
2. Open Detective console → Finding Groups
3. Identify clustered findings pointing to same entity
4. Select entity (IAM user/role, EC2, IP)
5. Review 24h behavior timeline
6. Identify unusual API calls, new geolocations, impossible travel
7. Check for privilege escalation patterns (CreateAccessKey, AttachPolicy)
8. Trace lateral movement via AssumeRole chains
9. Check S3 data access patterns
10. Review VPC Flow Logs for unusual outbound connections

Key concepts

- TA0001** Initial Access: T1078 (Valid Accounts), T1190 (Exploit Public-Facing Application)
- TA0003** Persistence: T1098 (Account Manipulation), T1136 (Create Account)
- TA0004** Privilege Escalation: T1078, T1484 (Domain Policy Modification)
- TA0005** Defense Evasion: T1562 (Impair Defenses), T1070 (Indicator Removal)
- TA0006** Credential Access: T1528 (Steal Application Access Token)
- TA0007** Discovery: T1580 (Cloud Infrastructure Discovery), T1526 (Cloud Service Discovery)
- TA0009** Collection: T1530 (Data from Cloud Storage)
- TA0010** Exfiltration: T1537 (Transfer Data to Cloud Account)

Tools & resources

- AWS

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-native-threat-hunting-with-aws-detective`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-penetration-testing-with-pacu (Skill Module)

performing-cloud-penetration-testing-with-pacu\SKILL.md

name: performing-cloud-penetration-testing-with-pacu
description: AWS Penetration Testing with Pacu Agent — AUTHORIZED TESTING ONLY Automates Pacu module execution for AWS security assessment including IAM enumeration, privilege escalation scanning, and credential testing. WARNING:.... Use when the user asks to perform cloud penetration testing with pacu.
version: 1.0.0
tags:
source: performing-cloud-penetration-testing-with-pacu

Performing Cloud Penetration Testing with Pacu

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Pacu IAM Modules, Pacu Enumeration Modules, boto3 Fallback Methods, Pacu Session Data

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud penetration testing with pacu.
- Capability: AWS Penetration Testing with Pacu Agent — AUTHORIZED TESTING ONLY Automates Pacu module execution for AWS security assessment including IAM enumeration, privilege escalation scanning, and credential testing. WARNING: Only use with explicit written authorization on approved AWS accounts.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, penetration, testing, pacu`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute a Pacu module via subprocess
2. Fallback IAM enumeration using boto3 when Pacu is unavailable
3. Identify privilege escalation paths from IAM enumeration data
4. Test what services the current credentials can access
5. Generate Pacu penetration testing report

Tools & resources

- Pacu IAM Modules
- Pacu Enumeration Modules
- boto3 Fallback Methods
- Pacu Session Data

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_pacu_module` — Execute a Pacu module via subprocess.
- `enumerate_iam_with_boto` — Fallback IAM enumeration using boto3 when Pacu is unavailable.
- `scan_privilege_escalation` — Identify privilege escalation paths from IAM enumeration data.
- `test_credential_access` — Test what services the current credentials can access.
- `generate_report` — Generate Pacu penetration testing report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-penetration-testing-with-pacu`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cloud-storage-forensic-acquisition (Skill Module)

performing-cloud-storage-forensic-acquisition\SKILL.md

name: performing-cloud-storage-forensic-acquisition
description: Cloud storage forensic acquisition agent. Acquires forensic copies of cloud storage objects from AWS S3, Azure Blob Storage, and GCP Cloud Storage with integrity verification using SHA-256 hashes, metadata.... Use when the user asks to perform cloud storage forensic acquisition.
version: 1.0.0
tags:
source: performing-cloud-storage-forensic-acquisition

Performing Cloud Storage Forensic Acquisition

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Magnet AXIOM Cloud, Cellebrite Cloud Analyzer, kumodd, KAPE

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cloud storage forensic acquisition.
- Capability: Cloud storage forensic acquisition agent. Acquires forensic copies of cloud storage objects from AWS S3, Azure Blob Storage, and GCP Cloud Storage with integrity verification using SHA-256 hashes, metadata preservation, and chain-of-custody logging.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cloud, storage, forensic, acquisition`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Acquire S3 objects with forensic integrity verification
2. Acquire all versions of a specific S3 object
3. Verify SHA-256 hashes of acquired files
4. Print acquisition summary

Key concepts

- NIST SP 800-86: Guide to Integrating Forensic Techniques
- ISO/IEC 27037: Digital Evidence Collection
- NIST Cloud Computing Forensic Science Challenges (NISTIR 8006)
- CSA Cloud Forensics Capability Implementation Guide
- Magnet AXIOM Cloud: Commercial multi-cloud acquisition
- Cellebrite Cloud Analyzer: SaaS evidence collection
- kumodd: Open-source proof-of-concept cloud acquisition tool
- KAPE: Endpoint-based cloud artifact collection

Tools & resources

- Magnet AXIOM Cloud
- Cellebrite Cloud Analyzer
- kumodd
- KAPE

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `acquire_s3_objects` — Acquire S3 objects with forensic integrity verification.
- `acquire_s3_versions` — Acquire all versions of a specific S3 object.
- `verify_integrity` — Verify SHA-256 hashes of acquired files.
- `format_summary` — Print acquisition summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cloud-storage-forensic-acquisition`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-container-escape-detection (Skill Module)

performing-container-escape-detection\SKILL.md

name: performing-container-escape-detection
description: Agent for detecting container escape vectors in Kubernetes. Use when the user asks to perform container escape detection.
version: 1.0.0
tags:
source: performing-container-escape-detection

Performing Container Escape Detection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: kubernetes Python Client, Container Escape Vectors, Dangerous Linux Capabilities

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** container escape detection.
- Capability: Agent for detecting container escape vectors in Kubernetes.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, container, escape, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load Kubernetes configuration
2. Find pods running privileged containers
3. Find containers with dangerous Linux capabilities
4. Find pods sharing host PID, network, or IPC namespaces
5. Find pods with dangerous hostPath volume mounts
6. Specifically detect Docker/CRI socket mounts
7. Find containers running as root

Tools & resources

- kubernetes Python Client
- Container Escape Vectors
- Dangerous Linux Capabilities

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_kube_config` — Load Kubernetes configuration.
- `check_privileged_containers` — Find pods running privileged containers.
- `check_dangerous_capabilities` — Find containers with dangerous Linux capabilities.
- `check_host_namespaces` — Find pods sharing host PID, network, or IPC namespaces.
- `check_dangerous_mounts` — Find pods with dangerous hostPath volume mounts.
- `check_docker_socket` — Specifically detect Docker/CRI socket mounts.
- `check_root_containers` — Find containers running as root.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-container-escape-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-container-image-hardening (Skill Module)

performing-container-image-hardening\SKILL.md

name: performing-container-image-hardening
description: Container image hardening audit agent. Audits Docker container images for security hardening best practices using Trivy for vulnerability scanning, Dockle for CIS Docker Benchmark compliance, and Dockerfile analysis.... Use when the user asks to perform container image hardening.
version: 1.0.0
tags:
source: performing-container-image-hardening

Performing Container Image Hardening

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Trivy Image Scanning, Docker Image Inspection, Hadolint Dockerfile Linting, Hardening Checks, CIS Docker Benchmark v1.6.0, NIST SP 800

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** container image hardening.
- Capability: Container image hardening audit agent. Audits Docker container images for security hardening best practices using Trivy for vulnerability scanning, Dockle for CIS Docker Benchmark compliance, and Dockerfile analysis for security anti-patterns.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, container, image, hardening`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Locate a binary on the system PATH
2. Scan a container image with Trivy for vulnerabilities
3. Scan a container image with Dockle for CIS benchmark compliance
4. Analyze a Dockerfile for security anti-patterns
5. Extract vulnerability findings from Trivy JSON output
6. Extract CIS benchmark findings from Dockle JSON output
7. Print combined audit summary

Key concepts

- Use minimal base images to reduce attack surface
- Remove unnecessary tools, shells, and package managers
- Scan images for vulnerabilities before deployment
- Sign images for integrity verification
- Use multi-stage builds
- Use read-only root filesystem
- Pin base images to digests
- Drop all Linux capabilities

Tools & resources

- Trivy Image Scanning

- Docker Image Inspection
- Hadolint Dockerfile Linting
- Hardening Checks
- CIS Docker Benchmark v1.6.0
- NIST SP 800

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `find_binary` — Locate a binary on the system PATH.
- `run_trivy_image_scan` — Scan a container image with Trivy for vulnerabilities.
- `run_dockle_scan` — Scan a container image with Dockle for CIS benchmark compliance.
- `analyze_dockerfile` — Analyze a Dockerfile for security anti-patterns.
- `extract_trivy_findings` — Extract vulnerability findings from Trivy JSON output.
- `extract_dockle_findings` — Extract CIS benchmark findings from Dockle JSON output.
- `format_summary` — Print combined audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-container-image-hardening`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-container-security-scanning-with-trivy (Skill Module)

performing-container-security-scanning-with-trivy\SKILL.md

name: performing-container-security-scanning-with-trivy
description: Trivy Container Security Agent - scans images for vulnerabilities, misconfigs, and secrets. Use when the user asks to perform container security scanning with trivy.
version: 1.0.0
tags:
source: performing-container-security-scanning-with-trivy

Performing Container Security Scanning with Trivy

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Severity Levels, Scanner Types, Exit Codes, JSON Output Structure

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** container security scanning with trivy.
- Capability: Trivy Container Security Agent - scans images for vulnerabilities, misconfigs, and secrets.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, container, security, scanning, trivy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run Trivy image scan and return JSON results
2. Run Trivy misconfiguration scan on Dockerfile/K8s manifests
3. Generate SBOM from container image
4. Analyze vulnerability scan results and produce summary
5. Analyze secret detection results
6. Analyze misconfiguration scan results

Tools & resources

- Severity Levels
- Scanner Types
- Exit Codes
- JSON Output Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_trivy_scan` — Run Trivy image scan and return JSON results.
- `run_trivy_misconfig` — Run Trivy misconfiguration scan on Dockerfile/K8s manifests.
- `generate_sbom` — Generate SBOM from container image.
- `analyze_vulnerabilities` — Analyze vulnerability scan results and produce summary.
- `analyze_secrets` — Analyze secret detection results.
- `analyze_misconfigs` — Analyze misconfiguration scan results.
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-container-security-scanning-with-trivy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-content-security-policy-bypass (Skill Module)

performing-content-security-policy-bypass\SKILL.md

name: performing-content-security-policy-bypass
description: Content Security Policy (CSP) analysis and bypass testing agent. Fetches and analyzes CSP headers from web applications to identify misconfigurations, overly permissive directives, and potential bypass vectors. Tests.... Use when the user asks to perform content security policy bypass.
version: 1.0.0
tags:
source: performing-content-security-policy-bypass

Performing Content Security Policy Bypass

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: CSP Directive Reference, Core Operations

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** content security policy bypass.
- Capability: Content Security Policy (CSP) analysis and bypass testing agent. Fetches and analyzes CSP headers from web applications to identify misconfigurations, overly permissive directives, and potential bypass vectors. Tests for unsafe-inline, unsafe-eval, wildcard sources, missing directives, and known CSP bypass patterns. AUTHORIZED TESTING ONLY: Only use against targets you have explicit written permission to test.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, content, security, policy, bypass`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Fetch CSP header(s) from a URL
2. Parse a CSP string into a structured dict
3. Analyze CSP directives for security weaknesses
4. Print analysis summary

Tools & resources

- CSP Directive Reference
- Core Operations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `fetch_csp` — Fetch CSP header(s) from a URL.
- `parse_csp` — Parse a CSP string into a structured dict.
- `analyze_csp` — Analyze CSP directives for security weaknesses.
- `format_summary` — Print analysis summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-content-security-policy-bypass`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-credential-access-with-lazagne (Skill Module)

performing-credential-access-with-lazagne\SKILL.md

name: performing-credential-access-with-lazagne
description: LaZagne credential access detection agent. Detects evidence of LaZagne credential harvesting tool execution on endpoints by scanning for process artifacts, file system indicators, and Windows event log entries.... Use when the user asks to perform credential access with lazagne.
version: 1.0.0
tags:
source: performing-credential-access-with-lazagne

Performing Credential Access with Lazagne

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Available Modules, Official Resources, Detection References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** credential access with lazagne.
- Capability: LaZagne credential access detection agent. Detects evidence of LaZagne credential harvesting tool execution on endpoints by scanning for process artifacts, file system indicators, and Windows event log entries associated with credential dumping. Used for defensive detection and incident response, not offensive use.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, credential, access, lazagne`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan file system for LaZagne artifacts
2. Check Windows Security event logs for LaZagne-related activity
3. Check if credential stores have been recently accessed unusually
4. Print detection summary

Key concepts

- Windows Event 5379: Credential Manager credentials were read
- DPAPI CryptUnprotectData monitoring
- Chrome Login Data file access monitoring

Tools & resources

- Available Modules
- Official Resources
- Detection References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_filesystem_indicators` — Scan file system for LaZagne artifacts.
- `check_windows_event_logs` — Check Windows Security event logs for LaZagne-related activity.
- `check_credential_store_integrity` — Check if credential stores have been recently accessed unusually.
- `format_summary` — Print detection summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-credential-access-with-lazagne`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cryptographic-audit-of-application (Skill Module)

performing-cryptographic-audit-of-application\SKILL.md

name: performing-cryptographic-audit-of-application
description: Application cryptographic audit agent. Audits application code and configurations for cryptographic weaknesses including weak algorithms, insecure key sizes, hardcoded keys, deprecated protocols, and missing.... Use when the user asks to perform cryptographic audit of application.
version: 1.0.0
tags:
source: performing-cryptographic-audit-of-application

Performing Cryptographic Audit of Application

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: TLS Configuration Audit, Certificate Analysis, NIST Guidelines, OWASP References, Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cryptographic audit of application.
- Capability: Application cryptographic audit agent. Audits application code and configurations for cryptographic weaknesses including weak algorithms, insecure key sizes, hardcoded keys, deprecated protocols, and missing certificate validation. Scans source files, config files, and TLS endpoints.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cryptographic, audit, application`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan a single file for cryptographic weaknesses
2. Recursively scan a directory for cryptographic weaknesses
3. Audit TLS configuration of a remote endpoint
4. Print audit summary

Key concepts

- Description: Defines deprecated, legacy, and acceptable algorithms
- Description: Minimum key sizes and algorithm recommendations
- Description: OWASP Top 10 #2 - Cryptographic Failures
- Description: Guidelines for secure cryptographic storage
- Description: Password hashing best practices
- Description: Python security linter with crypto checks
- Description: Static analysis with crypto-specific rules
- Description: Specialized cryptographic misuse detector

Tools & resources

- TLS Configuration Audit
- Certificate Analysis
- NIST Guidelines
- OWASP References

- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `scan_file` — Scan a single file for cryptographic weaknesses.
- `scan_directory` — Recursively scan a directory for cryptographic weaknesses.
- `audit_tls_endpoint` — Audit TLS configuration of a remote endpoint.
- `format_summary` — Print audit summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cryptographic-audit-of-application`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-csrf-attack-simulation (Skill Module)

performing-csrf-attack-simulation\SKILL.md

name: performing-csrf-attack-simulation
description: CSRF Attack Simulation Agent — AUTHORIZED TESTING ONLY Tests web applications for Cross-Site Request Forgery vulnerabilities by analyzing anti-CSRF protections and generating proof-of-concept payloads. WARNING: Only.... Use when the user asks to perform csrf attack simulation.
version: 1.0.0
tags:
source: performing-csrf-attack-simulation

Performing CSRF Attack Simulation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: requests Library, PoC Generation, OWASP Testing Guide

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** csrf attack simulation.
- Capability: CSRF Attack Simulation Agent — AUTHORIZED TESTING ONLY Tests web applications for Cross-Site Request Forgery vulnerabilities by analyzing anti-CSRF protections and generating proof-of-concept payloads. WARNING: Only use with explicit written authorization for the target application.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, csrf, attack, simulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Analyze a page for CSRF protection mechanisms
2. Identify forms that perform state-changing actions (POST, PUT, DELETE)
3. Generate CSRF proof-of-concept HTML page
4. Test if CSRF token validation can be bypassed
5. Generate CSRF testing report

Tools & resources

- requests Library
- PoC Generation
- OWASP Testing Guide

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_csrf_protections` — Analyze a page for CSRF protection mechanisms.
- `find_state_changing_forms` — Identify forms that perform state-changing actions (POST, PUT, DELETE).
- `generate_csrf_poc` — Generate CSRF proof-of-concept HTML page.
- `test_csrf_token_validation` — Test if CSRF token validation can be bypassed.
- `generate_report` — Generate CSRF testing report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-csrf-attack-simulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-cve-prioritization-with-kev-catalog (Skill Module)

performing-cve-prioritization-with-kev-catalog\SKILL.md

name: performing-cve-prioritization-with-kev-catalog
description: CISA KEV CVE prioritization agent. Downloads the CISA Known Exploited Vulnerabilities (KEV) catalog and cross-references it against a list of CVEs from vulnerability scans to prioritize remediation based on active.... Use when the user asks to perform cve prioritization with kev catalog.
version: 1.0.0
tags:
source: performing-cve-prioritization-with-kev-catalog

Performing CVE Prioritization with Kev Catalog

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Data Sources, Core Operations, Official CISA Resources, Scoring Systems

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** cve prioritization with kev catalog.
- Capability: CISA KEV CVE prioritization agent. Downloads the CISA Known Exploited Vulnerabilities (KEV) catalog and cross-references it against a list of CVEs from vulnerability scans to prioritize remediation based on active exploitation status, due dates, and vendor/product impact.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, prioritization, catalog`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Download the latest CISA KEV catalog as JSON
2. Build a lookup dict keyed by CVE ID for fast matching
3. Load CVE list from file (one CVE per line or CSV) or comma-separated string
4. Cross-reference CVEs against KEV catalog and prioritize
5. Print a human-readable prioritization report
6. Compute statistics about the KEV catalog

Key concepts

- BOD 22-01: Federal agencies must remediate KEV CVEs per due dates
- PCI DSS v4.0: Prioritize remediation of actively exploited vulns
- NIST CSF 2.0: Risk-based vulnerability prioritization
- ISO 27001: 2022 A.8.8**: Technical vulnerability management

Tools & resources

- Data Sources
- Core Operations
- Official CISA Resources
- Scoring Systems

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `download_kev_catalog` — Download the latest CISA KEV catalog as JSON.
- `build_kev_index` — Build a lookup dict keyed by CVE ID for fast matching.
- `load_cve_list` — Load CVE list from file (one CVE per line or CSV) or comma-separated string.
- `prioritize_cves` — Cross-reference CVEs against KEV catalog and prioritize.
- `format_summary` — Print a human-readable prioritization report.
- `kev_stats` — Compute statistics about the KEV catalog.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-cve-prioritization-with-kev-catalog`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-dark-web-monitoring-for-threats (Skill Module)

performing-dark-web-monitoring-for-threats\SKILL.md

name: performing-dark-web-monitoring-for-threats
description: Dark web threat monitoring agent. Monitors for organization-specific threats on the dark web by checking breach databases (Have I Been Pwned API), paste sites, and public threat intelligence feeds for leaked.... Use when the user asks to perform dark web monitoring for threats.
version: 1.0.0
tags:
source: performing-dark-web-monitoring-for-threats

Performing Dark Web Monitoring for Threats

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Credential Breach Monitoring, Brand Mention Monitoring, Domain Monitoring, Alerting, Dark Web Intelligence Classification, Dark Web Source Categories

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** dark web monitoring for threats.
- Capability: Dark web threat monitoring agent. Monitors for organization-specific threats on the dark web by checking breach databases (Have I Been Pwned API), paste sites, and public threat intelligence feeds for leaked credentials, exposed data, and mentions of organizational domains.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, dark, monitoring, threats`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check Have I Been Pwned for breaches involving a domain
2. Check if a specific email appears in known breaches
3. Check if a password appears in known breaches using k-anonymity
4. Check for organization mentions in paste sites via HIBP
5. Search public threat intelligence for domain mentions
6. Print monitoring summary

Key concepts

- TLP: RED**: Dark web source details, forum credentials, HUMINT methods
- TLP: AMBER**: Specific threat actor mentions, leaked data analysis
- TLP: GREEN**: Aggregated trends, anonymized statistics
- TLP: CLEAR**: Public dark web monitoring advisories
- T1589 - Gather Victim Identity Information (credential theft)
- T1590 - Gather Victim Network Information (reconnaissance)
- T1597 - Search Closed Sources (dark web forums)
- T1598 - Phishing for Information

Tools & resources

- Credential Breach Monitoring
- Brand Mention Monitoring

- Domain Monitoring
- Alerting
- Dark Web Intelligence Classification
- Dark Web Source Categories
- Legal and Ethical Framework

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_hibp_breaches` — Check Have I Been Pwned for breaches involving a domain.
- `check_hibp_email` — Check if a specific email appears in known breaches.
- `check_hibp_password` — Check if a password appears in known breaches using k-anonymity.
- `check_paste_dumps` — Check for organization mentions in paste sites via HIBP.
- `search_threat_intel_feeds` — Search public threat intelligence for domain mentions.
- `format_summary` — Print monitoring summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-dark-web-monitoring-for-threats`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-deception-technology-deployment (Skill Module)

performing-deception-technology-deployment\SKILL.md

name: performing-deception-technology-deployment
description: Deception Technology Deployment Agent Deploys and manages honeypots, honeytokens, and canary files to detect lateral movement and credential abuse with near-zero false positive alerts. Use when the user asks to perform deception technology deployment.
version: 1.0.0
tags:
source: performing-deception-technology-deployment

Performing Deception Technology Deployment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Thinkst Canary API, Honeytoken Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** deception technology deployment.
- Capability: Deception Technology Deployment Agent Deploys and manages honeypots, honeytokens, and canary files to detect lateral movement and credential abuse with near-zero false positive alerts.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, deception, technology, deployment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate fake credential honeytokens for deployment in AD and databases
2. Generate canary files that trigger alerts when accessed
3. Generate DNS canary tokens that alert on resolution
4. Start an HTTP honeypot server in a background thread
5. Generate deception technology deployment report

Tools & resources

- Thinkst Canary API
- Honeytoken Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_honeytoken_credentials` — Generate fake credential honeytokens for deployment in AD and databases.
- `generate_canary_files` — Generate canary files that trigger alerts when accessed.
- `generate_dns_canary_tokens` — Generate DNS canary tokens that alert on resolution.
- `start_http_honeypot` — Start an HTTP honeypot server in a background thread.
- `generate_deployment_report` — Generate deception technology deployment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-deception-technology-deployment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-directory-traversal-testing (Skill Module)

performing-directory-traversal-testing\SKILL.md

name: performing-directory-traversal-testing
description: Directory Traversal Testing Agent — AUTHORIZED TESTING ONLY Tests web applications for path traversal (LFI) vulnerabilities by injecting traversal sequences into file path parameters. WARNING: Only use with explicit.... Use when the user asks to perform directory traversal testing.
version: 1.0.0
tags:
source: performing-directory-traversal-testing

Performing Directory Traversal Testing

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Traversal Payload Encodings, Vulnerability Indicators, requests Library, Key Libraries, OWASP Testing Guide

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** directory traversal testing.
- Capability: Directory Traversal Testing Agent — AUTHORIZED TESTING ONLY Tests web applications for path traversal (LFI) vulnerabilities by injecting traversal sequences into file path parameters. WARNING: Only use with explicit written authorization for the target application.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, directory, traversal, testing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Identify URL parameters that likely handle file paths
2. Test a parameter for directory traversal vulnerability
3. Test null byte injection to bypass file extension checks
4. Test PHP wrapper protocols for LFI exploitation
5. Generate directory traversal testing report

Tools & resources

- Traversal Payload Encodings
- Vulnerability Indicators
- requests Library
- Key Libraries
- OWASP Testing Guide

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `identify_file_parameters` — Identify URL parameters that likely handle file paths.
- `test_traversal` — Test a parameter for directory traversal vulnerability.
- `test_null_byte_bypass` — Test null byte injection to bypass file extension checks.
- `test_wrapper_protocols` — Test PHP wrapper protocols for LFI exploitation.
- `generate_report` — Generate directory traversal testing report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-directory-traversal-testing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-disk-forensics-investigation (Skill Module)

performing-disk-forensics-investigation\SKILL.md

name: performing-disk-forensics-investigation
description: Disk Forensics Investigation Agent Performs disk forensics analysis including image verification, file system parsing, deleted file recovery, and timeline generation using pytsk3 and hashlib for evidence integrity. Use when the user asks to perform disk forensics investigation.
version: 1.0.0
tags:
source: performing-disk-forensics-investigation

Performing Disk Forensics Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: File Metadata Fields, NTFS MFT Structure

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** disk forensics investigation.
- Capability: Disk Forensics Investigation Agent Performs disk forensics analysis including image verification, file system parsing, deleted file recovery, and timeline generation using pytsk3 and hashlib for evidence integrity.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, disk, forensics, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate and verify hash of forensic disk image
2. Parse file system from disk image using pytsk3
3. List files in a directory within the disk image
4. Search for deleted files in the disk image
5. Parse MFT entries from an exported $MFT file for NTFS analysis
6. Build a timeline of file system activity from file metadata
7. Generate disk forensics investigation report

Tools & resources

- File Metadata Fields
- NTFS MFT Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `verify_image_integrity` — Calculate and verify hash of forensic disk image.
- `parse_filesystem_pytsk3` — Parse file system from disk image using pytsk3.
- `list_files_pytsk3` — List files in a directory within the disk image.
- `find_deleted_files` — Search for deleted files in the disk image.
- `parse_mft_entries` — Parse MFT entries from an exported $MFT file for NTFS analysis.
- `build_timeline` — Build a timeline of file system activity from file metadata.
- `generate_report` — Generate disk forensics investigation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-disk-forensics-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-dmarc-policy-enforcement-rollout (Skill Module)

performing-dmarc-policy-enforcement-rollout\SKILL.md

name: performing-dmarc-policy-enforcement-rollout
description: Agent for performing DMARC policy enforcement rollout with DNS record analysis. Use when the user asks to perform dmarc policy enforcement rollout.
version: 1.0.0
tags:
source: performing-dmarc-policy-enforcement-rollout

Performing Dmarc Policy Enforcement Rollout

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: DMARC Policy Levels, Core Protocol Standards, DMARC Policy Progression, SPF Lookup Limit

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** dmarc policy enforcement rollout.
- Capability: Agent for performing DMARC policy enforcement rollout with DNS record analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, dmarc, policy, enforcement, rollout`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Query and parse DMARC record for a domain
2. Query and analyze SPF record
3. Query DKIM public key record
4. Audit multiple domains for DMARC/SPF/DKIM readiness

Key concepts

- RFC 7489: Domain-based Message Authentication, Reporting, and Conformance (DMARC)
- RFC 7208: Sender Policy Framework (SPF) for Authorizing Use of Domains in Email
- RFC 6376: DomainKeys Identified Mail (DKIM) Signatures
- RFC 8616: Email Authentication for Internationalized Mail
- RFC 8601: Message Header Field for Indicating Message Authentication Status
- Google Bulk Sender Requirements: DMARC required for senders of 5,000+ messages/day to Gmail
- Yahoo Sender Requirements: DMARC authentication required for bulk senders
- PCI DSS 4.0 Requirement 5.3: Anti-phishing mechanisms including email authentication

Tools & resources

- DMARC Policy Levels
- Core Protocol Standards
- DMARC Policy Progression
- SPF Lookup Limit

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_dmarc` — Query and parse DMARC record for a domain.
- `parse_dmarc_tags`
- `check_spf` — Query and analyze SPF record.
- `check_dkim` — Query DKIM public key record.
- `audit_domains` — Audit multiple domains for DMARC/SPF/DKIM readiness.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-dmarc-policy-enforcement-rollout`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-dns-enumeration-and-zone-transfer (Skill Module)

performing-dns-enumeration-and-zone-transfer\SKILL.md

name: performing-dns-enumeration-and-zone-transfer
description: DNS Enumeration and Zone Transfer Agent — AUTHORIZED TESTING ONLY Performs DNS reconnaissance including record enumeration, zone transfer attempts, and subdomain discovery using dnspython. WARNING: Only use with.... Use when the user asks to perform dns enumeration and zone transfer.
version: 1.0.0
tags:
source: performing-dns-enumeration-and-zone-transfer

Performing DNS Enumeration and Zone Transfer

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: dnspython Library, DNS Record Types, Email Security Records, Common Subdomain Wordlists

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** dns enumeration and zone transfer.
- Capability: DNS Enumeration and Zone Transfer Agent — AUTHORIZED TESTING ONLY Performs DNS reconnaissance including record enumeration, zone transfer attempts, and subdomain discovery using dnspython. WARNING: Only use with explicit written authorization for the target domain.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, enumeration, zone, transfer`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Enumerate common DNS record types for a domain
2. Resolve authoritative nameservers for a domain
3. Attempt AXFR zone transfer against each nameserver
4. Check SPF, DKIM, and DMARC DNS records
5. Brute-force subdomain discovery via DNS resolution
6. Generate DNS enumeration report

Tools & resources

- dnspython Library
- DNS Record Types
- Email Security Records
- Common Subdomain Wordlists

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `enumerate_dns_records` — Enumerate common DNS record types for a domain.
- `get_nameservers` — Resolve authoritative nameservers for a domain.
- `attempt_zone_transfer` — Attempt AXFR zone transfer against each nameserver.
- `check_email_security` — Check SPF, DKIM, and DMARC DNS records.
- `brute_force_subdomains` — Brute-force subdomain discovery via DNS resolution.
- `generate_report` — Generate DNS enumeration report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-dns-enumeration-and-zone-transfer`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-dns-tunneling-detection (Skill Module)

performing-dns-tunneling-detection\SKILL.md

name: performing-dns-tunneling-detection
description: Agent for detecting DNS tunneling via entropy and statistical analysis. Use when the user asks to perform dns tunneling detection.
version: 1.0.0
tags:
source: performing-dns-tunneling-detection

Performing DNS Tunneling Detection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: DNS Tunneling Indicators, Common DNS Tunnel Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** dns tunneling detection.
- Capability: Agent for detecting DNS tunneling via entropy and statistical analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, tunneling, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of a string
2. Extract DNS queries from a PCAP file using scapy
3. Flag queries with high Shannon entropy in subdomain labels
4. Detect queries with unusually long domain names
5. Detect high volume of TXT record queries to single domains
6. Detect domains with high unique subdomain count (tunneling indicator)
7. Detect non-standard character frequency in query labels

Tools & resources

- DNS Tunneling Indicators
- Common DNS Tunnel Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shannon_entropy` — Calculate Shannon entropy of a string.
- `extract_dns_queries` — Extract DNS queries from a PCAP file using scapy.
- `analyze_entropy` — Flag queries with high Shannon entropy in subdomain labels.
- `analyze_query_lengths` — Detect queries with unusually long domain names.
- `analyze_txt_records` — Detect high volume of TXT record queries to single domains.
- `analyze_subdomain_cardinality` — Detect domains with high unique subdomain count (tunneling indicator).
- `analyze_character_distribution` — Detect non-standard character frequency in query labels.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-dns-tunneling-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-docker-bench-security-assessment (Skill Module)

performing-docker-bench-security-assessment\SKILL.md

name: performing-docker-bench-security-assessment
description: Agent for performing Docker CIS Benchmark security assessment. Use when the user asks to perform docker bench security assessment.
version: 1.0.0
tags:
source: performing-docker-bench-security-assessment

Performing Docker Bench Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Docker, Scoring

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** docker bench security assessment.
- Capability: Agent for performing Docker CIS Benchmark security assessment.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, docker, bench, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run docker-bench-security and parse results
2. Parse docker-bench-security output into structured findings
3. Check running container configurations against CIS benchmarks

Key concepts

- PASS: Check meets CIS recommendation
- FAIL: Check does not meet recommendation (remediation required)
- WARN: Check requires manual verification
- INFO: Informational, no scoring impact

Tools & resources

- Docker
- Scoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_docker_bench` — Run docker-bench-security and parse results.
- `parse_bench_output` — Parse docker-bench-security output into structured findings.
- `check_container_configs` — Check running container configurations against CIS benchmarks.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-docker-bench-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-dynamic-analysis-of-android-app (Skill Module)

performing-dynamic-analysis-of-android-app\SKILL.md

name: performing-dynamic-analysis-of-android-app
description: Agent for performing dynamic analysis of Android applications using Frida. Use when the user asks to perform dynamic analysis of android app.
version: 1.0.0
tags:
source: performing-dynamic-analysis-of-android-app

Performing Dynamic Analysis of Android App

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Frida API Calls, CWE Mappings

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** dynamic analysis of android app.
- Capability: Agent for performing dynamic analysis of Android applications using Frida.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, dynamic, analysis, android`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List installed packages on Android device/emulator
2. Capture network traffic from Android device using tcpdump
3. Check if app implements SSL/TLS certificate pinning using Frida
4. List exported activities, services, receivers, and providers
5. Check for insecure data storage patterns

Tools & resources

- Frida API Calls
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `list_packages` — List installed packages on Android device/emulator.
- `capture_network_traffic` — Capture network traffic from Android device using tcpdump.
- `check_ssl_pinning` — Check if app implements SSL/TLS certificate pinning using Frida.
- `analyze_exported_components` — List exported activities, services, receivers, and providers.
- `check_data_storage` — Check for insecure data storage patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-dynamic-analysis-of-android-app`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-dynamic-analysis-with-any-run (Skill Module)

performing-dynamic-analysis-with-any-run\SKILL.md

name: performing-dynamic-analysis-with-any-run
description: Dynamic Analysis with ANY.RUN Agent Submits malware samples to ANY.RUN sandbox via API, monitors task execution, and retrieves behavioral analysis results including process trees, network indicators, and MITRE ATT&CK.... Use when the user asks to perform dynamic analysis with any run.
version: 1.0.0
tags:
source: performing-dynamic-analysis-with-any-run

Performing Dynamic Analysis with Any Run

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: ANY.RUN API v1, Submission Parameters, Key Libraries, Rate Limits

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** dynamic analysis with any run.
- Capability: Dynamic Analysis with ANY.RUN Agent Submits malware samples to ANY.RUN sandbox via API, monitors task execution, and retrieves behavioral analysis results including process trees, network indicators, and MITRE ATT&CK mappings. Supports both the official anyrun-sdk (pip install anyrun-sdk) with SandboxConnector and the legacy REST API via requests.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, dynamic, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Submit a file to ANY.RUN for dynamic analysis
2. Submit a URL to ANY.RUN for dynamic analysis
3. Retrieve the full analysis report for a completed task
4. Poll task status until analysis completes
5. Extract IOCs from an ANY.RUN analysis report
6. Generate dynamic analysis report
7. Use official anyrun-sdk (SandboxConnector) if available

Tools & resources

- ANY.RUN API v1
- Submission Parameters
- Key Libraries
- Rate Limits

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `submit_file` — Submit a file to ANY.RUN for dynamic analysis.
- `submit_url` — Submit a URL to ANY.RUN for dynamic analysis.
- `get_task_report` — Retrieve the full analysis report for a completed task.
- `wait_for_completion` — Poll task status until analysis completes.
- `get_iocs_from_report` — Extract IOCs from an ANY.RUN analysis report.
- `generate_report` — Generate dynamic analysis report.
- `run_with_sdk` — Use official anyrun-sdk (SandboxConnector) if available.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-dynamic-analysis-with-any-run`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-endpoint-forensics-investigation (Skill Module)

performing-endpoint-forensics-investigation\SKILL.md

name: performing-endpoint-forensics-investigation
description: Agent for performing endpoint forensics investigation on Windows systems. Use when the user asks to perform endpoint forensics investigation.
version: 1.0.0
tags:
source: performing-endpoint-forensics-investigation

Performing Endpoint Forensics Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Primary Standards, Compliance Mappings, Tool References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** endpoint forensics investigation.
- Capability: Agent for performing endpoint forensics investigation on Windows systems.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, endpoint, forensics, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Collect basic system information for forensic context
2. Collect running processes with parent PIDs and command lines
3. Collect active network connections
4. Collect common persistence locations
5. Calculate MD5, SHA1, SHA256 hashes of a file for evidence integrity
6. Run full endpoint forensic triage collection

Key concepts

- Scope: Forensic process for digital evidence collection, examination, and analysis
- Scope: Identification, collection, acquisition, and preservation of digital evidence
- Scope: Best practices for evidence collection including order of volatility

Tools & resources

- Primary Standards
- Compliance Mappings
- Tool References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `collect_system_info` — Collect basic system information for forensic context.
- `collect_running_processes` — Collect running processes with parent PIDs and command lines.
- `collect_network_connections` — Collect active network connections.
- `collect_autoruns` — Collect common persistence locations.
- `hash_file` — Calculate MD5, SHA1, SHA256 hashes of a file for evidence integrity.
- `full_triage` — Run full endpoint forensic triage collection.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-endpoint-forensics-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-endpoint-vulnerability-remediation (Skill Module)

performing-endpoint-vulnerability-remediation\SKILL.md

name: performing-endpoint-vulnerability-remediation
description: Agent for performing endpoint vulnerability remediation tracking and validation. Use when the user asks to perform endpoint vulnerability remediation.
version: 1.0.0
tags:
source: performing-endpoint-vulnerability-remediation

Performing Endpoint Vulnerability Remediation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Primary Standards, Compliance Mappings, Remediation SLA Benchmarks, Supporting References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** endpoint vulnerability remediation.
- Capability: Agent for performing endpoint vulnerability remediation tracking and validation.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, endpoint, vulnerability, remediation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse a vulnerability scan CSV report and prioritize remediation
2. Check installed Windows patches and identify missing ones
3. Validate that a vulnerability has been remediated
4. Generate a remediation plan from scan results

Key concepts

- Scope: Enterprise patch management lifecycle, risk-based prioritization, and operational planning
- Scope: Federal mandate to remediate CVEs in the Known Exploited Vulnerabilities catalog within specified timelines
- Timelines: Typically 14 days for internet-facing, 28 days for internal systems
- Publisher: FIRST (Forum of Incident Response and Security Teams)
- Scope: Scoring methodology for vulnerability severity
- Scope: Machine learning model predicting probability of CVE exploitation within 30 days

Tools & resources

- Primary Standards
- Compliance Mappings
- Remediation SLA Benchmarks
- Supporting References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_scan_report` — Parse a vulnerability scan CSV report and prioritize remediation.
- `check_windows_patches` — Check installed Windows patches and identify missing ones.
- `validate_remediation` — Validate that a vulnerability has been remediated.
- `generate_remediation_report` — Generate a remediation plan from scan results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-endpoint-vulnerability-remediation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-entitlement-review-with-sailpoint-iiq (Skill Module)

performing-entitlement-review-with-sailpoint-iiq\SKILL.md

name: performing-entitlement-review-with-sailpoint-iiq
description: Agent for performing entitlement review with SailPoint IdentityIQ. Automates access certification campaigns, SOD violation detection, and entitlement review reporting via the SailPoint IdentityIQ REST API. Use when the user asks to perform entitlement review with sailpoint iiq.
version: 1.0.0
tags:
source: performing-entitlement-review-with-sailpoint-iiq

Performing Entitlement Review with Sailpoint Iiq

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: SailPoint, Key Parameters

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** entitlement review with sailpoint iiq.
- Capability: Agent for performing entitlement review with SailPoint IdentityIQ. Automates access certification campaigns, SOD violation detection, and entitlement review reporting via the SailPoint IdentityIQ REST API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, entitlement, review, sailpoint`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for entitlement review with sailpoint iiq
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- SailPoint
- Key Parameters

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-entitlement-review-with-sailpoint-iiq`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-external-network-penetration-test (Skill Module)

performing-external-network-penetration-test\SKILL.md

name: performing-external-network-penetration-test
description: Agent for performing external network penetration test reconnaissance and scanning. Use when the user asks to perform external network penetration test.
version: 1.0.0
tags:
source: performing-external-network-penetration-test

Performing External Network Penetration Test

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Default Port List, Primary Standards, Compliance Frameworks, CVSS v3.1 Scoring Reference

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** external network penetration test.
- Capability: Agent for performing external network penetration test reconnaissance and scanning.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, external, network, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Scan common TCP ports on a target host
2. Run nmap scan against target
3. Enumerate DNS records for a domain
4. Check SSL/TLS certificate details

Key concepts

- Phases: Pre-engagement, Intelligence Gathering, Threat Modeling, Vulnerability Analysis, Exploitation, Post-Exploitati
- Best for: Comprehensive network penetration testing engagements
- Title: Technical Guide to Information Security Testing and Assessment
- Covers: Review techniques, target identification, vulnerability analysis, planning, execution, post-testing
- Focus: Operational security testing across physical, wireless, telecommunications, data networks, and human channels
- Focus: Web application security testing methodology
- Complement to network-level testing

Tools & resources

- Default Port List
- Primary Standards
- Compliance Frameworks
- CVSS v3.1 Scoring Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `tcp_port_scan` — Scan common TCP ports on a target host.
- `run_nmap_scan` — Run nmap scan against target.
- `dns_enumeration` — Enumerate DNS records for a domain.
- `ssl_check` — Check SSL/TLS certificate details.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-external-network-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-false-positive-reduction-in-siem (Skill Module)

performing-false-positive-reduction-in-siem\SKILL.md

name: performing-false-positive-reduction-in-siem
description: Agent for performing false positive reduction analysis in SIEM environments. Use when the user asks to perform false positive reduction in siem.
version: 1.0.0
tags:
source: performing-false-positive-reduction-in-siem

Performing False Positive Reduction in SIEM

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Expected CSV Columns, Tuning Frameworks, Allowlist Management Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** false positive reduction in siem.
- Capability: Agent for performing false positive reduction analysis in SIEM environments.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, false, positive, reduction, siem`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze SIEM alert CSV to identify false positive patterns
2. Generate SIEM rule tuning recommendations from alert analysis
3. Simulate the impact of proposed tuning changes on alert volume

Key concepts

- Requires regular assessment and adjustment of detection capabilities
- Defines metrics-based approach to monitoring effectiveness
- Level 1: Basic alerts with high FP rate
- Level 2: Tuned alerts with correlation
- Level 3: Behavioral analytics reducing noise
- Level 4: Automated tuning with ML feedback loops
- All exclusions require documented justification
- Expiry dates mandatory (90-day maximum default)

Tools & resources

- Expected CSV Columns
- Tuning Frameworks
- Allowlist Management Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_alerts` — Analyze SIEM alert CSV to identify false positive patterns.
- `generate_tuning_recommendations` — Generate SIEM rule tuning recommendations from alert analysis.
- `simulate_tuning_impact` — Simulate the impact of proposed tuning changes on alert volume.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-false-positive-reduction-in-siem`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-file-carving-with-foremost (Skill Module)

performing-file-carving-with-foremost\SKILL.md

name: performing-file-carving-with-foremost
description: Agent for performing file carving with Foremost. Automates file carving from disk images using foremost/scalpel, validates carved files, and generates evidence catalogs with hashes. Use when the user asks to perform file carving with foremost.
version: 1.0.0
tags:
source: performing-file-carving-with-foremost

Performing File Carving with Foremost

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Foremost, Scalpel CLI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** file carving with foremost.
- Capability: Agent for performing file carving with Foremost. Automates file carving from disk images using foremost/scalpel, validates carved files, and generates evidence catalogs with hashes.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, file, carving, foremost`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for file carving with foremost
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Foremost
- Scalpel CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-file-carving-with-foremost`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-firmware-extraction-with-binwalk (Skill Module)

performing-firmware-extraction-with-binwalk\SKILL.md

name: performing-firmware-extraction-with-binwalk
description: Firmware extraction and analysis agent using binwalk for signature scanning, entropy analysis, filesystem extraction, and string-based credential discovery. Use when the user asks to perform firmware extraction with binwalk.
version: 1.0.0
tags:
source: performing-firmware-extraction-with-binwalk

Performing Firmware Extraction with Binwalk

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: binwalk, unsquashfs, jefferson, sasquatch, strings, dd

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** firmware extraction with binwalk.
- Capability: Firmware extraction and analysis agent using binwalk for signature scanning, entropy analysis, filesystem extraction, and string-based credential discovery.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, firmware, extraction, binwalk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate Shannon entropy of a byte sequence (0.0 = uniform, 8.0 = max random)
2. Generate a block-by-block entropy map of a firmware image
3. Classify an entropy value into a human-readable category
4. Identify contiguous regions of high or low entropy in a firmware image
5. Scan a firmware image for known magic byte signatures
6. Parse a U-Boot image header at the given offset
7. Extract printable ASCII strings and scan for sensitive patterns
8. Run binwalk signature scan via subprocess and return parsed output

Tools & resources

- binwalk
- unsquashfs
- jefferson
- sasquatch
- strings
- dd

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `calculate_entropy` — Calculate Shannon entropy of a byte sequence (0.0 = uniform, 8.0 = max random).
- `entropy_map` — Generate a block-by-block entropy map of a firmware image.
- `classify_entropy` — Classify an entropy value into a human-readable category.
- `detect_entropy_regions` — Identify contiguous regions of high or low entropy in a firmware image.
- `scan_signatures` — Scan a firmware image for known magic byte signatures.
- `parse_uboot_header` — Parse a U-Boot image header at the given offset.
- `scan_strings` — Extract printable ASCII strings and scan for sensitive patterns.
- `run_binwalk_scan` — Run binwalk signature scan via subprocess and return parsed output.
- `run_binwalk_extract` — Run binwalk extraction via subprocess.
- `run_binwalk_entropy` — Run binwalk entropy analysis via subprocess.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-firmware-extraction-with-binwalk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-firmware-malware-analysis (Skill Module)

performing-firmware-malware-analysis\SKILL.md

name: performing-firmware-malware-analysis
description: Agent for performing firmware malware analysis. Automates firmware extraction with binwalk, filesystem analysis, string extraction, entropy analysis, and IOC identification. Use when the user asks to perform firmware malware analysis.
version: 1.0.0
tags:
source: performing-firmware-malware-analysis

Performing Firmware Malware Analysis

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: binwalk CLI, binwalk Python API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** firmware malware analysis.
- Capability: Agent for performing firmware malware analysis. Automates firmware extraction with binwalk, filesystem analysis, string extraction, entropy analysis, and IOC identification.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, firmware, malware, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for firmware malware analysis
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- binwalk CLI
- binwalk Python API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-firmware-malware-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-fuzzing-with-aflplusplus (Skill Module)

performing-fuzzing-with-aflplusplus\SKILL.md

name: performing-fuzzing-with-aflplusplus
description: AFL++ fuzzing campaign management and crash triage. Use when the user asks to perform fuzzing with aflplusplus.
version: 1.0.0
tags:
source: performing-fuzzing-with-aflplusplus

Performing Fuzzing with Aflplusplus

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AFL++ CLI Tools, afl-fuzz Key Flags, fuzzer_stats File Fields, Crash Triage Tools, External References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** fuzzing with aflplusplus.
- Capability: AFL++ fuzzing campaign management and crash triage agent.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, fuzzing, aflplusplus`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compile target with AFL++ instrumentation
2. Minimize seed corpus using afl-cmin
3. Parse afl-fuzz fuzzer_stats file for campaign metrics
4. Enumerate and classify crash files from AFL++ output
5. Minimize a crash test case with afl-tmin
6. Run afl-whatsup to get multi-instance campaign summary
7. Execute AFL++ fuzzing campaign audit

Tools & resources

- AFL++ CLI Tools
- afl-fuzz Key Flags
- fuzzer_stats File Fields
- Crash Triage Tools
- External References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `instrument_target` — Compile target with AFL++ instrumentation.
- `minimize_corpus` — Minimize seed corpus using afl-cmin.
- `parse_fuzzer_stats` — Parse afl-fuzz fuzzer_stats file for campaign metrics.
- `triage_crashes` — Enumerate and classify crash files from AFL++ output.
- `minimize_crash` — Minimize a crash test case with afl-tmin.
- `run_whatsup` — Run afl-whatsup to get multi-instance campaign summary.
- `run_audit` — Execute AFL++ fuzzing campaign audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-fuzzing-with-aflplusplus`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-gcp-penetration-testing-with-gcpbucketbrute (Skill Module)

performing-gcp-penetration-testing-with-gcpbucketbrute\SKILL.md

name: performing-gcp-penetration-testing-with-gcpbucketbrute
description: GCP penetration testing agent: bucket enumeration, IAM audit, privilege escalation analysis. # For authorized testing only. Use when the user asks to perform gcp penetration testing with gcpbucketbrute.
version: 1.0.0
tags:
source: performing-gcp-penetration-testing-with-gcpbucketbrute

Performing GCP Penetration Testing with Gcpbucketbrute

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: GCP, gcloud IAM Enumeration, gsutil Bucket Permission Testing, Privilege Escalation Permissions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** gcp penetration testing with gcpbucketbrute.
- Capability: GCP penetration testing agent: bucket enumeration, IAM audit, privilege escalation analysis. # For authorized testing only
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, penetration, testing, gcpbucketbrute`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate bucket name permutations from a keyword
2. Check if a GCS bucket exists and test access permissions
3. Test IAM permissions on a discovered bucket using gsutil
4. Enumerate project-level IAM bindings for overly permissive roles
5. Check for user-managed service account keys
6. Check testable IAM permissions for privilege escalation vectors
7. Generate GCP penetration test findings report

Tools & resources

- GCP
- gcloud IAM Enumeration
- gsutil Bucket Permission Testing
- Privilege Escalation Permissions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_bucket_names` — Generate bucket name permutations from a keyword.
- `check_bucket_exists` — Check if a GCS bucket exists and test access permissions.
- `test_bucket_permissions` — Test IAM permissions on a discovered bucket using gsutil.
- `enumerate_iam_bindings` — Enumerate project-level IAM bindings for overly permissive roles.
- `check_service_account_keys` — Check for user-managed service account keys.
- `check_priv_esc_paths` — Check testable IAM permissions for privilege escalation vectors.
- `generate_report` — Generate GCP penetration test findings report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-gcp-penetration-testing-with-gcpbucketbrute`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-gcp-security-assessment-with-forseti (Skill Module)

performing-gcp-security-assessment-with-forseti\SKILL.md

name: performing-gcp-security-assessment-with-forseti
description: Agent for performing GCP security assessment. Audits IAM policies, firewall rules, storage permissions, and Security Command Center findings using Google Cloud client libraries. Use when the user asks to perform gcp security assessment with forseti.
version: 1.0.0
tags:
source: performing-gcp-security-assessment-with-forseti

Performing GCP Security Assessment with Forseti

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: GCP, Cloud Asset Inventory API, Cloud Storage API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** gcp security assessment with forseti.
- Capability: Agent for performing GCP security assessment. Audits IAM policies, firewall rules, storage permissions, and Security Command Center findings using Google Cloud client libraries.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, security, assessment, forseti`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for gcp security assessment with forseti
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- GCP
- Cloud Asset Inventory API
- Cloud Storage API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-gcp-security-assessment-with-forseti`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-graphql-depth-limit-attack (Skill Module)

performing-graphql-depth-limit-attack\SKILL.md

name: performing-graphql-depth-limit-attack
description: Agent for performing GraphQL depth limit attack testing. Use when the user asks to perform graphql depth limit attack.
version: 1.0.0
tags:
source: performing-graphql-depth-limit-attack

Performing Graphql Depth Limit Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Severity Classification

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** graphql depth limit attack.
- Capability: Agent for performing GraphQL depth limit attack testing.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, graphql, depth, limit, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Build a deeply nested GraphQL query string
2. Probe GraphQL endpoint for query depth enforcement
3. Test circular reference queries (e.g., user.posts.author.posts...)
4. Test if batched queries bypass depth limits
5. Test wide + deep queries for resource exhaustion potential

Tools & resources

- Severity Classification

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `build_nested_query` — Build a deeply nested GraphQL query string.
- `test_depth_limit` — Probe GraphQL endpoint for query depth enforcement.
- `test_circular_query` — Test circular reference queries (e.g., user.posts.author.posts...).
- `test_batch_query` — Test if batched queries bypass depth limits.
- `test_resource_exhaustion` — Test wide + deep queries for resource exhaustion potential.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-graphql-depth-limit-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-graphql-introspection-attack (Skill Module)

performing-graphql-introspection-attack\SKILL.md

name: performing-graphql-introspection-attack
description: Agent for performing GraphQL introspection attack and schema analysis. Use when the user asks to perform graphql introspection attack.
version: 1.0.0
tags:
source: performing-graphql-introspection-attack

Performing Graphql Introspection Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** graphql introspection attack.
- Capability: Agent for performing GraphQL introspection attack and schema analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, graphql, introspection, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for graphql introspection attack
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_introspection` — Execute GraphQL introspection query against target endpoint.
- `test_depth_limit` — Test for GraphQL query depth limits.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-graphql-introspection-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-graphql-security-assessment (Skill Module)

performing-graphql-security-assessment\SKILL.md

name: performing-graphql-security-assessment
description: Agent for performing GraphQL security assessment. Tests GraphQL endpoints for introspection leaks, authorization flaws, query depth/complexity DoS, and injection vulnerabilities. Use when the user asks to perform graphql security assessment.
version: 1.0.0
tags:
source: performing-graphql-security-assessment

Performing Graphql Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: graphql-cop CLI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** graphql security assessment.
- Capability: Agent for performing GraphQL security assessment. Tests GraphQL endpoints for introspection leaks, authorization flaws, query depth/complexity DoS, and injection vulnerabilities.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, graphql, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for graphql security assessment
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- graphql-cop CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-graphql-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-hardware-security-module-integration (Skill Module)

performing-hardware-security-module-integration\SKILL.md

name: performing-hardware-security-module-integration
description: Agent for Hardware Security Module integration via PKCS#11 interface. Use when the user asks to perform hardware security module integration.
version: 1.0.0
tags:
source: performing-hardware-security-module-integration

Performing Hardware Security Module Integration

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: FIPS 140-2 Required Mechanisms

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** hardware security module integration.
- Capability: Agent for Hardware Security Module integration via PKCS#11 interface.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, hardware, security, module, integration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load PKCS#11 shared library
2. List all available PKCS#11 slots and token info
3. List all cryptographic objects stored on the HSM token
4. Generate an RSA key pair on the HSM
5. Generate an EC P-256 key pair on the HSM
6. Sign data with an RSA private key and verify with the public key
7. List all supported mechanisms for the token's slot
8. Run comprehensive HSM compliance audit

Tools & resources

- FIPS 140-2 Required Mechanisms

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_library` — Load PKCS#11 shared library.
- `enumerate_slots` — List all available PKCS#11 slots and token info.
- `list_objects` — List all cryptographic objects stored on the HSM token.
- `generate_rsa_keypair` — Generate an RSA key pair on the HSM.
- `generate_ec_keypair` — Generate an EC P-256 key pair on the HSM.
- `sign_and_verify` — Sign data with an RSA private key and verify with the public key.
- `query_mechanisms` — List all supported mechanisms for the token's slot.
- `full_audit` — Run comprehensive HSM compliance audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-hardware-security-module-integration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-hash-cracking-with-hashcat (Skill Module)

performing-hash-cracking-with-hashcat\SKILL.md

name: performing-hash-cracking-with-hashcat
description: Agent for performing hash cracking with hashcat — hash identification, attack management, and result analysis. Use when the user asks to perform hash cracking with hashcat.
version: 1.0.0
tags:
source: performing-hash-cracking-with-hashcat

Performing Hash Cracking with Hashcat

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Hashcat Mask Charsets, Tools, Standards, Legal Framework

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** hash cracking with hashcat.
- Capability: Agent for performing hash cracking with hashcat — hash identification, attack management, and result analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, hash, cracking, hashcat`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Identify hash type and return hashcat mode
2. Identify hash types from a file of hashes
3. Execute hashcat with specified attack mode
4. Parse hashcat potfile for cracked results
5. Generate hash of a plaintext string for testing

Key concepts

- RockYou: Classic breach wordlist (14 million passwords)
- Description: Password requirements and memorized secret guidelines
- Computer Fraud and Abuse Act (CFAA) - US
- Computer Misuse Act 1990 - UK
- Always obtain written authorization before testing
- Follow penetration testing rules of engagement

Tools & resources

- Hashcat Mask Charsets
- Tools
- Standards
- Legal Framework

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `identify_hash` — Identify hash type and return hashcat mode.
- `identify_hashes_file` — Identify hash types from a file of hashes.
- `run_hashcat` — Execute hashcat with specified attack mode.
- `parse_hashcat_status` — Parse hashcat potfile for cracked results.
- `generate_hash` — Generate hash of a plaintext string for testing.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-hash-cracking-with-hashcat`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-http-parameter-pollution-attack (Skill Module)

performing-http-parameter-pollution-attack\SKILL.md

name: performing-http-parameter-pollution-attack
description: Agent for performing HTTP parameter pollution (HPP) attack testing. Use when the user asks to perform http parameter pollution attack.
version: 1.0.0
tags:
source: performing-http-parameter-pollution-attack

Performing HTTP Parameter Pollution Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Payload Categories

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** http parameter pollution attack.
- Capability: Agent for performing HTTP parameter pollution (HPP) attack testing.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, http, parameter, pollution, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test which parameter value the server uses when duplicated
2. Send HPP test payloads and analyze responses
3. Test if HPP can bypass WAF parameter filtering

Tools & resources

- Payload Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_parameter_precedence` — Test which parameter value the server uses when duplicated.
- `test_hpp_payloads` — Send HPP test payloads and analyze responses.
- `test_waf_bypass` — Test if HPP can bypass WAF parameter filtering.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-http-parameter-pollution-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ics-asset-discovery-with-claroty (Skill Module)

performing-ics-asset-discovery-with-claroty\SKILL.md

name: performing-ics-asset-discovery-with-claroty
description: Agent for performing ICS asset discovery with Claroty xDome/CTD API. Use when the user asks to perform ics asset discovery with claroty.
version: 1.0.0
tags:
source: performing-ics-asset-discovery-with-claroty

Performing Ics Asset Discovery with Claroty

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: ClarotyClient Class

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ics asset discovery with claroty.
- Capability: Agent for performing ICS asset discovery with Claroty xDome/CTD API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, asset, discovery, claroty`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run asset discovery and categorize results
2. Retrieve and prioritize OT vulnerabilities
3. Get security alert summary
4. Map OT network segments

Tools & resources

- ClarotyClient Class

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_assets` — Run asset discovery and categorize results.
- `assess_vulnerabilities` — Retrieve and prioritize OT vulnerabilities.
- `get_alerts_summary` — Get security alert summary.
- `network_topology` — Map OT network segments.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ics-asset-discovery-with-claroty`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-indicator-lifecycle-management (Skill Module)

performing-indicator-lifecycle-management\SKILL.md

name: performing-indicator-lifecycle-management
description: Agent for performing indicator of compromise (IOC) lifecycle management. Use when the user asks to perform indicator lifecycle management.
version: 1.0.0
tags:
source: performing-indicator-lifecycle-management

Performing Indicator Lifecycle Management

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Applicable Standards, Industry Frameworks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** indicator lifecycle management.
- Capability: Agent for performing indicator of compromise (IOC) lifecycle management.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, indicator, lifecycle, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Extract IOCs from a text file or report
2. Ingest IOC feed from CSV and normalize
3. Check IOC database for expired indicators based on TTL
4. Deduplicate IOCs and merge metadata from multiple sources
5. Generate full IOC lifecycle status report

Key concepts

- STIX 2.1: Structured Threat Information eXpression for CTI data representation
- TAXII 2.1: Transport protocol for sharing CTI over HTTPS
- MITRE ATT&CK: Adversary tactics, techniques, and procedures taxonomy
- Diamond Model: Intrusion analysis framework (Adversary, Capability, Infrastructure, Victim)
- Traffic Light Protocol (TLP): Information sharing classification (CLEAR, GREEN, AMBER, RED)
- Technique mapping for threat actor behavior classification
- Data sources for detection capability assessment
- Mitigation strategies linked to specific techniques

Tools & resources

- Applicable Standards
- Industry Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `extract_iocs` — Extract IOCs from a text file or report.
- `ingest_ioc_feed` — Ingest IOC feed from CSV and normalize.
- `check_expiration` — Check IOC database for expired indicators based on TTL.
- `deduplicate_iocs` — Deduplicate IOCs and merge metadata from multiple sources.
- `generate_lifecycle_report` — Generate full IOC lifecycle status report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-indicator-lifecycle-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-initial-access-with-evilginx3 (Skill Module)

performing-initial-access-with-evilginx3\SKILL.md

name: performing-initial-access-with-evilginx3
description: Agent for performing initial access simulation with Evilginx3 phishlet analysis — educational/authorized pentest use. Use when the user asks to perform initial access with evilginx3.
version: 1.0.0
tags:
source: performing-initial-access-with-evilginx3

Performing Initial Access with Evilginx3

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Phishlet Structure, Industry Standards, Official Resources, Research Papers

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** initial access with evilginx3.
- Capability: Agent for performing initial access simulation with Evilginx3 phishlet analysis — educational/authorized pentest use.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, initial, access, evilginx3`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Parse an Evilginx3 YAML phishlet file and extract configuration
2. Analyze Evilginx session capture logs
3. Check if Evilginx3 is installed and get version
4. List available phishlets in a directory
5. Generate detection signatures for a phishlet's attack patterns

Key concepts

- NIST SP 800-63B** - Digital Identity Guidelines: Authentication
- Microsoft Storm-1167 AiTM Phishing Campaign Analysis (2023)
- Deepwatch: Catching the Phish - Detecting Evilginx & AiTM
- BDO Security: MFA-Phishing as Initial Access in Red Teaming

Tools & resources

- Phishlet Structure
- Industry Standards
- Official Resources
- Research Papers

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `parse_phishlet` — Parse an Evilginx3 YAML phishlet file and extract configuration.
- `analyze_session_log` — Analyze Evilginx session capture logs.
- `check_evilginx_installation` — Check if Evilginx3 is installed and get version.
- `list_phishlets` — List available phishlets in a directory.
- `generate_detection_rules` — Generate detection signatures for a phishlet's attack patterns.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-initial-access-with-evilginx3`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-insider-threat-investigation (Skill Module)

performing-insider-threat-investigation\SKILL.md

name: performing-insider-threat-investigation
description: Agent for performing insider threat investigation. Analyzes user activity logs, detects behavioral anomalies, builds activity timelines, and generates investigation reports for insider threat cases. Use when the user asks to perform insider threat investigation.
version: 1.0.0
tags:
source: performing-insider-threat-investigation

Performing Insider Threat Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Data Sources, Exabeam UEBA API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** insider threat investigation.
- Capability: Agent for performing insider threat investigation. Analyzes user activity logs, detects behavioral anomalies, builds activity timelines, and generates investigation reports for insider threat cases.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, insider, threat, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for insider threat investigation
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Data Sources
- Exabeam UEBA API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-insider-threat-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ioc-enrichment-automation (Skill Module)

performing-ioc-enrichment-automation\SKILL.md

name: performing-ioc-enrichment-automation
description: Agent for performing IOC enrichment automation. Orchestrates multi-source IOC lookups across VirusTotal, AbuseIPDB, Shodan, and GreyNoise to provide contextual scoring and disposition. Use when the user asks to perform ioc enrichment automation.
version: 1.0.0
tags:
source: performing-ioc-enrichment-automation

Performing IOC Enrichment Automation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: VirusTotal API v3, AbuseIPDB API v2, Shodan API, GreyNoise Community API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ioc enrichment automation.
- Capability: Agent for performing IOC enrichment automation. Orchestrates multi-source IOC lookups across VirusTotal, AbuseIPDB, Shodan, and GreyNoise to provide contextual scoring and disposition.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, enrichment, automation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for ioc enrichment automation
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- VirusTotal API v3
- AbuseIPDB API v2
- Shodan API
- GreyNoise Community API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ioc-enrichment-automation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ios-app-security-assessment (Skill Module)

performing-ios-app-security-assessment\SKILL.md

name: performing-ios-app-security-assessment
description: iOS App Security Assessment Agent - Automates Frida-based iOS security testing including SSL pinning bypass, keychain extraction, IPA static analysis, and runtime method hooking. Use when the user asks to perform ios app security assessment.
version: 1.0.0
tags:
source: performing-ios-app-security-assessment

Performing IOS App Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Arguments, Key Functions, Frida Script Payloads, OWASP MASVS Coverage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ios app security assessment.
- Capability: iOS App Security Assessment Agent - Automates Frida-based iOS security testing including SSL pinning bypass, keychain extraction, IPA static analysis, and runtime method hooking.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Perform static analysis on an IPA file
2. Execute a Frida script against a target iOS application
3. Execute an Objection command against a target app
4. Dump and analyze keychain items for insecure storage practices
5. Generate comprehensive iOS security assessment report

Tools & resources

- Arguments
- Key Functions
- Frida Script Payloads
- OWASP MASVS Coverage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `analyze_ipa_static` — Perform static analysis on an IPA file.
- `run_frida_script` — Execute a Frida script against a target iOS application.
- `run_objection_command` — Execute an Objection command against a target app.
- `assess_keychain_security` — Dump and analyze keychain items for insecure storage practices.
- `generate_report` — Generate comprehensive iOS security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ios-app-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-iot-security-assessment (Skill Module)

performing-iot-security-assessment\SKILL.md

name: performing-iot-security-assessment
description: Agent for performing IoT security assessment. Automates IoT device reconnaissance, firmware extraction with binwalk, network traffic analysis, and service scanning for security testing. Use when the user asks to perform iot security assessment.
version: 1.0.0
tags:
source: performing-iot-security-assessment

Performing Iot Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Tools CLI Reference

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** iot security assessment.
- Capability: Agent for performing IoT security assessment. Automates IoT device reconnaissance, firmware extraction with binwalk, network traffic analysis, and service scanning for security testing.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for iot security assessment
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Tools CLI Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-iot-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ip-reputation-analysis-with-shodan (Skill Module)

performing-ip-reputation-analysis-with-shodan\SKILL.md

name: performing-ip-reputation-analysis-with-shodan
description: Agent for performing IP reputation analysis using the Shodan API. Use when the user asks to perform ip reputation analysis with shodan.
version: 1.0.0
tags:
source: performing-ip-reputation-analysis-with-shodan

Performing Ip Reputation Analysis with Shodan

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Risk Classification

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ip reputation analysis with shodan.
- Capability: Agent for performing IP reputation analysis using the Shodan API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, reputation, analysis, shodan`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Look up IP details from Shodan
2. Check IP reputation on AbuseIPDB
3. Analyze reputation of multiple IPs

Tools & resources

- Risk Classification

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `shodan_host_lookup` — Look up IP details from Shodan.
- `abuseipdb_check` — Check IP reputation on AbuseIPDB.
- `bulk_reputation` — Analyze reputation of multiple IPs.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ip-reputation-analysis-with-shodan`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-jwt-none-algorithm-attack (Skill Module)

performing-jwt-none-algorithm-attack\SKILL.md

name: performing-jwt-none-algorithm-attack
description: Agent for performing JWT 'none' algorithm attack testing. Use when the user asks to perform jwt none algorithm attack.
version: 1.0.0
tags:
source: performing-jwt-none-algorithm-attack

Performing JWT None Algorithm Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: JWT None Variants Tested, Severity Classification

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** jwt none algorithm attack.
- Capability: Agent for performing JWT 'none' algorithm attack testing.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, none, algorithm, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Base64url encode bytes
2. Base64url decode string
3. Decode and display JWT components without verification
4. Forge a JWT with 'none' algorithm (removes signature)
5. Test algorithm confusion (RS256 -> HS256 using public key as HMAC secret)
6. Test forged JWTs against a target endpoint

Tools & resources

- JWT None Variants Tested
- Severity Classification

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `b64url_encode` — Base64url encode bytes.
- `b64url_decode` — Base64url decode string.
- `decode_jwt` — Decode and display JWT components without verification.
- `forge_none_token` — Forge a JWT with 'none' algorithm (removes signature).
- `test_alg_confusion` — Test algorithm confusion (RS256 -> HS256 using public key as HMAC secret).
- `test_jwt_endpoint` — Test forged JWTs against a target endpoint.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-jwt-none-algorithm-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-kerberoasting-attack (Skill Module)

performing-kerberoasting-attack\SKILL.md

name: performing-kerberoasting-attack
description: Agent for performing Kerberoasting attack simulation and detection — authorized testing only. Use when the user asks to perform kerberoasting attack.
version: 1.0.0
tags:
source: performing-kerberoasting-attack

Performing Kerberoasting Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Encryption Types, NIST SP 800-63B, CIS Benchmark, Detection References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** kerberoasting attack.
- Capability: Agent for performing Kerberoasting attack simulation and detection — authorized testing only.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, kerberoasting, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Enumerate service principal name (SPN) accounts via LDAP query
2. Request TGS tickets for Kerberoasting using Impacket GetUserSPNs
3. Analyze captured Kerberoast hashes
4. Detect Kerberoasting from Windows Security Event ID 4769

Key concepts

- Recommends minimum 8-character passwords
- Service accounts should use 25+ character passwords
- Managed Service Accounts (MSA/gMSA) automatically rotate passwords
- Ensure 'Network security: Configure encryption types allowed for Kerberos' excludes RC4
- Monitor Event ID 4769 for anomalous service ticket requests
- Implement AES-only encryption for service accounts
- Use Group Managed Service Accounts where possible

Tools & resources

- Encryption Types
- NIST SP 800-63B
- CIS Benchmark
- Detection References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `enumerate_spn_accounts` — Enumerate service principal name (SPN) accounts via LDAP query.
- `request_tgs_tickets` — Request TGS tickets for Kerberoasting using Impacket GetUserSPNs.
- `analyze_kerberoast_hashes` — Analyze captured Kerberoast hashes.
- `detect_kerberoasting` — Detect Kerberoasting from Windows Security Event ID 4769.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-kerberoasting-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-kubernetes-cis-benchmark-with-kube-bench (Skill Module)

performing-kubernetes-cis-benchmark-with-kube-bench\SKILL.md

name: performing-kubernetes-cis-benchmark-with-kube-bench
description: Agent for performing Kubernetes CIS benchmark assessment with kube-bench. Use when the user asks to perform kubernetes cis benchmark with kube bench.
version: 1.0.0
tags:
source: performing-kubernetes-cis-benchmark-with-kube-bench

Performing Kubernetes CIS Benchmark with Kube Bench

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Kubernetes, Pod Security Issues Detected, Compliance Frameworks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** kubernetes cis benchmark with kube bench.
- Capability: Agent for performing Kubernetes CIS benchmark assessment with kube-bench.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, kubernetes, benchmark, kube, bench`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute kube-bench CIS benchmark scan
2. Check pod security settings via kubectl
3. Audit RBAC configuration for overly permissive roles
4. Verify network policies exist and cover pods

Key concepts

- Section 1: Kubernetes Pod Security
- Section 2: Network Separation and Hardening
- Section 3: Authentication and Authorization
- Section 4: Audit Logging and Threat Detection
- Req 2.2: Develop configuration standards for all system components
- Req 6.3.2: Develop software securely
- CC6.1: Logical access security for system components
- CC8.1: Change management controls

Tools & resources

- Kubernetes
- Pod Security Issues Detected
- Compliance Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_kube_bench` — Execute kube-bench CIS benchmark scan.
- `check_pod_security` — Check pod security settings via kubectl.
- `check_rbac_config` — Audit RBAC configuration for overly permissive roles.
- `check_network_policies` — Verify network policies exist and cover pods.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-kubernetes-cis-benchmark-with-kube-bench`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-kubernetes-etcd-security-assessment (Skill Module)

performing-kubernetes-etcd-security-assessment\SKILL.md

name: performing-kubernetes-etcd-security-assessment
description: Agent for performing Kubernetes etcd security assessment. Use when the user asks to perform kubernetes etcd security assessment.
version: 1.0.0
tags:
source: performing-kubernetes-etcd-security-assessment

Performing Kubernetes Etcd Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Kubernetes, Security Checks, NIST SP 800-190

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** kubernetes etcd security assessment.
- Capability: Agent for performing Kubernetes etcd security assessment.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, kubernetes, etcd, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Check if etcd encryption at rest is configured
2. Check if secrets are stored unencrypted in etcd
3. Verify etcd TLS certificate configuration
4. Run comprehensive etcd security assessment

Key concepts

- Section 3.4.4: Data store encryption requirements
- Section 4.4.2: Secrets management for orchestrators

Tools & resources

- Kubernetes
- Security Checks
- NIST SP 800-190

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `check_etcd_encryption` — Check if etcd encryption at rest is configured.
- `check_etcd_access`
- `dump_secrets_check` — Check if secrets are stored unencrypted in etcd.
- `check_etcd_tls_config` — Verify etcd TLS certificate configuration.
- `full_assessment` — Run comprehensive etcd security assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-kubernetes-etcd-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-kubernetes-penetration-testing (Skill Module)

performing-kubernetes-penetration-testing\SKILL.md

name: performing-kubernetes-penetration-testing
description: Agent for performing Kubernetes penetration testing — authorized testing only. Use when the user asks to perform kubernetes penetration testing.
version: 1.0.0
tags:
source: performing-kubernetes-penetration-testing

Performing Kubernetes Penetration Testing

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Kubernetes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** kubernetes penetration testing.
- Capability: Agent for performing Kubernetes penetration testing — authorized testing only.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, kubernetes, penetration, testing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Enumerate basic cluster information for reconnaissance
2. Test what the default service account can do
3. Check for exposed Kubernetes dashboards and management interfaces
4. Check for container escape vectors in running pods

Key concepts

- Scan containers and pods for vulnerabilities
- Run containers as non-root users
- Use network policies to restrict traffic
- Encrypt secrets at rest
- Audit logging for all API calls
- Scan for misconfigurations regularly

Tools & resources

- Kubernetes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `enumerate_cluster_info` — Enumerate basic cluster information for reconnaissance.
- `test_service_account_permissions` — Test what the default service account can do.
- `scan_exposed_dashboards` — Check for exposed Kubernetes dashboards and management interfaces.
- `check_pod_escape_vectors` — Check for container escape vectors in running pods.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-kubernetes-penetration-testing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-lateral-movement-detection (Skill Module)

performing-lateral-movement-detection\SKILL.md

name: performing-lateral-movement-detection
description: Agent for performing lateral movement detection. Analyzes Windows event logs and network flow data to detect Pass-the-Hash, PsExec, WMI, RDP, and SMB-based lateral movement mapped to MITRE ATT&CK TA0008 techniques. Use when the user asks to perform lateral movement detection.
version: 1.0.0
tags:
source: performing-lateral-movement-detection

Performing Lateral Movement Detection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** lateral movement detection.
- Capability: Agent for performing lateral movement detection. Analyzes Windows event logs and network flow data to detect Pass-the-Hash, PsExec, WMI, RDP, and SMB-based lateral movement mapped to MITRE ATT&CK TA0008 techniques.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, lateral, movement, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for lateral movement detection
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-lateral-movement-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-lateral-movement-with-wmiexec (Skill Module)

performing-lateral-movement-with-wmiexec\SKILL.md

name: performing-lateral-movement-with-wmiexec
description: Agent for performing lateral movement detection and simulation with WMIExec — authorized testing only. Use when the user asks to perform lateral movement with wmiexec.
version: 1.0.0
tags:
source: performing-lateral-movement-with-wmiexec

Performing Lateral Movement with Wmiexec

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Impacket wmiexec.py, CrackMapExec, SharpWMI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** lateral movement with wmiexec.
- Capability: Agent for performing lateral movement detection and simulation with WMIExec — authorized testing only.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, lateral, movement, wmiexec`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Detect WMIExec lateral movement artifacts in Windows Event Logs
2. Execute command via Impacket wmiexec (authorized pentest use)
3. Analyze PCAP for WMI lateral movement network indicators
4. Check for WMI-based persistence mechanisms on local system

Key concepts

- Windows WMI Activity Log: Microsoft-Windows-WMI-Activity/Operational
- Sysmon Event 19/20/21: WMI filter/consumer/binding creation

Tools & resources

- Impacket wmiexec.py
- CrackMapExec
- SharpWMI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `detect_wmiexec_artifacts_evtx` — Detect WMIExec lateral movement artifacts in Windows Event Logs.
- `run_wmiexec_impacket` — Execute command via Impacket wmiexec (authorized pentest use).
- `detect_wmi_network_traffic` — Analyze PCAP for WMI lateral movement network indicators.
- `check_wmi_persistence` — Check for WMI-based persistence mechanisms on local system.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-lateral-movement-with-wmiexec`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-linux-log-forensics-investigation (Skill Module)

performing-linux-log-forensics-investigation\SKILL.md

name: performing-linux-log-forensics-investigation
description: Agent for performing Linux log forensics investigation. Use when the user asks to perform linux log forensics investigation.
version: 1.0.0
tags:
source: performing-linux-log-forensics-investigation

Performing Linux Log Forensics Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: journalctl, ausearch/aureport, logwatch, lnav

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** linux log forensics investigation.
- Capability: Agent for performing Linux log forensics investigation.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, linux, forensics, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze /var/log/auth.log for suspicious authentication events
2. Analyze /var/log/syslog for anomalies
3. Analyze bash history for suspicious commands
4. Create timeline from multiple log sources

Key concepts

- NIST SP 800-92: Guide to Computer Security Log Management
- NIST SP 800-86: Guide to Integrating Forensic Techniques
- RFC 5424: The Syslog Protocol
- journalctl: Systemd journal query tool
- ausearch/aureport: Audit log analysis
- logwatch, lnav: Log analysis utilities

Tools & resources

- journalctl
- ausearch/aureport
- logwatch, lnav

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_auth_log` — Analyze /var/log/auth.log for suspicious authentication events.
- `analyze_syslog` — Analyze /var/log/syslog for anomalies.
- `analyze_command_history` — Analyze bash history for suspicious commands.
- `timeline_analysis` — Create timeline from multiple log sources.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-linux-log-forensics-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-log-analysis-for-forensic-investigation (Skill Module)

performing-log-analysis-for-forensic-investigation\SKILL.md

name: performing-log-analysis-for-forensic-investigation
description: Agent for performing log analysis for forensic investigation. Parses Windows EVTX, Linux syslog, and web access logs to build correlated forensic timelines for incident investigations. Use when the user asks to perform log analysis for forensic investigation.
version: 1.0.0
tags:
source: performing-log-analysis-for-forensic-investigation

Performing Log Analysis for Forensic Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: python-evtx Library, Syslog Parsing, CLI Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** log analysis for forensic investigation.
- Capability: Agent for performing log analysis for forensic investigation. Parses Windows EVTX, Linux syslog, and web access logs to build correlated forensic timelines for incident investigations.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, analysis, forensic, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for log analysis for forensic investigation
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- python-evtx Library
- Syslog Parsing
- CLI Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-log-analysis-for-forensic-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-log-source-onboarding-in-siem (Skill Module)

performing-log-source-onboarding-in-siem\SKILL.md

name: performing-log-source-onboarding-in-siem
description: Agent for performing log source onboarding in SIEM platforms. Use when the user asks to perform log source onboarding in siem.
version: 1.0.0
tags:
source: performing-log-source-onboarding-in-siem

Performing Log Source Onboarding in SIEM

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Log Collection Protocols

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** log source onboarding in siem.
- Capability: Agent for performing log source onboarding in SIEM platforms.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, source, onboarding, siem`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Detect log format from a sample file
2. Test syslog connectivity to SIEM collector
3. Generate SIEM parsing configuration for common log formats
4. Generate a log source onboarding checklist

Key concepts

- Establish log management infrastructure
- Define log retention requirements
- Ensure log data integrity (tamper evidence)
- Configure time synchronization across all sources
- Implement log review and analysis procedures

Tools & resources

- Log Collection Protocols

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `detect_log_format` — Detect log format from a sample file.
- `validate_syslog_connectivity` — Test syslog connectivity to SIEM collector.
- `generate_parsing_config` — Generate SIEM parsing configuration for common log formats.
- `create_onboarding_checklist` — Generate a log source onboarding checklist.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-log-source-onboarding-in-siem`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-malware-hash-enrichment-with-virustotal (Skill Module)

performing-malware-hash-enrichment-with-virustotal\SKILL.md

name: performing-malware-hash-enrichment-with-virustotal
description: Agent for performing malware hash enrichment using VirusTotal API v3. Use when the user asks to perform malware hash enrichment with virustotal.
version: 1.0.0
tags:
source: performing-malware-hash-enrichment-with-virustotal

Performing Malware Hash Enrichment with Virustotal

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: VirusTotalClient API Calls, Rate Limiting

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** malware hash enrichment with virustotal.
- Capability: Agent for performing malware hash enrichment using VirusTotal API v3.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, malware, hash, enrichment, virustotal`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for malware hash enrichment with virustotal
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- VirusTotalClient API Calls
- Rate Limiting

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `enrich_hash_list` — Enrich a list of hashes with VirusTotal reports (respects rate limit).
- `hash_file` — Calculate MD5, SHA1, SHA256 of a local file.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-malware-hash-enrichment-with-virustotal`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-malware-ioc-extraction (Skill Module)

performing-malware-ioc-extraction\SKILL.md

name: performing-malware-ioc-extraction
description: Agent for performing malware IOC extraction from files, reports, and samples. Use when the user asks to perform malware ioc extraction.
version: 1.0.0
tags:
source: performing-malware-ioc-extraction

Performing Malware IOC Extraction

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Suspicious String Keywords, IOC Types and Classification, YARA Rule Structure

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** malware ioc extraction.
- Capability: Agent for performing malware IOC extraction from files, reports, and samples.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, malware, extraction`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Extract all IOC types from raw text
2. Extract IOCs from a file (text, PDF text, or report)
3. Calculate file hashes for malware sample identification
4. Extract printable strings from binary file
5. Generate comprehensive IOC extraction report

Key concepts

- DOS Header: MZ signature (0x5A4D)
- PE Header: PE signature, machine type, timestamp
- Optional Header: Entry point, image base, subsystem
- Section Table: .text, .data, .rdata, .rsrc, .reloc
- Import Table: DLLs and functions used
- Export Table: Functions exported (DLLs)
- Resource Table: Embedded resources (icons, strings, configs)

Tools & resources

- Suspicious String Keywords
- IOC Types and Classification
- YARA Rule Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `extract_iocs_from_text` — Extract all IOC types from raw text.
- `extract_from_file` — Extract IOCs from a file (text, PDF text, or report).
- `hash_file` — Calculate file hashes for malware sample identification.
- `extract_strings` — Extract printable strings from binary file.
- `generate_ioc_report` — Generate comprehensive IOC extraction report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-malware-ioc-extraction`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-malware-persistence-investigation (Skill Module)

performing-malware-persistence-investigation\SKILL.md

name: performing-malware-persistence-investigation
description: Agent for performing malware persistence investigation. Enumerates Windows registry Run keys, services, scheduled tasks, WMI subscriptions, and Linux cron/systemd persistence mechanisms. Use when the user asks to perform malware persistence investigation.
version: 1.0.0
tags:
source: performing-malware-persistence-investigation

Performing Malware Persistence Investigation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: python-registry Library, Key Windows Persistence Locations, Linux Persistence Locations

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** malware persistence investigation.
- Capability: Agent for performing malware persistence investigation. Enumerates Windows registry Run keys, services, scheduled tasks, WMI subscriptions, and Linux cron/systemd persistence mechanisms.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, malware, persistence, investigation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for malware persistence investigation
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- python-registry Library
- Key Windows Persistence Locations
- Linux Persistence Locations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-malware-persistence-investigation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-malware-triage-with-yara (Skill Module)

performing-malware-triage-with-yara\SKILL.md

name: performing-malware-triage-with-yara
description: Agent for performing malware triage with YARA. Compiles and applies YARA rules to classify malware samples, perform batch scanning, and generate triage reports. Use when the user asks to perform malware triage with yara.
version: 1.0.0
tags:
source: performing-malware-triage-with-yara

Performing Malware Triage with YARA

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: YARA, Match Object Attributes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** malware triage with yara.
- Capability: Agent for performing malware triage with YARA. Compiles and applies YARA rules to classify malware samples, perform batch scanning, and generate triage reports.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, malware, triage, yara`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for malware triage with yara
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- YARA
- Match Object Attributes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-malware-triage-with-yara`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-memory-forensics-with-volatility3 (Skill Module)

performing-memory-forensics-with-volatility3\SKILL.md

name: performing-memory-forensics-with-volatility3
description: Agent for performing memory forensics with Volatility 3. Automates memory dump analysis including process enumeration, network connection extraction, malware detection, and credential extraction using Volatility 3.... Use when the user asks to perform memory forensics with volatility3.
version: 1.0.0
tags:
source: performing-memory-forensics-with-volatility3

Performing Memory Forensics with Volatility3

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Volatility, Common Flags

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** memory forensics with volatility3.
- Capability: Agent for performing memory forensics with Volatility 3. Automates memory dump analysis including process enumeration, network connection extraction, malware detection, and credential extraction using Volatility 3 framework via subprocess.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, memory, forensics, volatility3`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for memory forensics with volatility3
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Volatility
- Common Flags

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-memory-forensics-with-volatility3`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-memory-forensics-with-volatility3-plugins (Skill Module)

performing-memory-forensics-with-volatility3-plugins\SKILL.md

name: performing-memory-forensics-with-volatility3-plugins
description: Agent for performing memory forensics with Volatility3 plugins. Use when the user asks to perform memory forensics with volatility3 plugins.
version: 1.0.0
tags:
source: performing-memory-forensics-with-volatility3-plugins

Performing Memory Forensics with Volatility3 Plugins

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Volatility

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** memory forensics with volatility3 plugins.
- Capability: Agent for performing memory forensics with Volatility3 plugins.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, memory, forensics, volatility3, plugins`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Execute a Volatility3 plugin against a memory dump
2. Run process analysis plugins to detect suspicious processes
3. Run malfind to detect code injection
4. Extract network connections from memory
5. Run full memory triage with key plugins

Tools & resources

- Volatility

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_vol3_plugin` — Execute a Volatility3 plugin against a memory dump.
- `detect_malicious_processes` — Run process analysis plugins to detect suspicious processes.
- `detect_injected_code` — Run malfind to detect code injection.
- `analyze_network_connections` — Extract network connections from memory.
- `full_triage` — Run full memory triage with key plugins.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-memory-forensics-with-volatility3-plugins`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-mobile-app-certificate-pinning-bypass (Skill Module)

performing-mobile-app-certificate-pinning-bypass\SKILL.md

name: performing-mobile-app-certificate-pinning-bypass
description: Agent for performing mobile app certificate pinning bypass testing — authorized testing only. Use when the user asks to perform mobile app certificate pinning bypass.
version: 1.0.0
tags:
source: performing-mobile-app-certificate-pinning-bypass

Performing Mobile App Certificate Pinning Bypass

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Pinning Strength Classification, OWASP MASVS v2.0, CWE Mappings

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** mobile app certificate pinning bypass.
- Capability: Agent for performing mobile app certificate pinning bypass testing — authorized testing only.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, mobile, certificate, pinning, bypass`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Detect certificate pinning implementation in an APK
2. Launch Frida SSL pinning bypass against a running app
3. Use objection to disable SSL pinning on Android/iOS
4. Verify proxy setup for traffic interception

Tools & resources

- Pinning Strength Classification
- OWASP MASVS v2.0
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `detect_pinning_implementation` — Detect certificate pinning implementation in an APK.
- `run_frida_bypass` — Launch Frida SSL pinning bypass against a running app.
- `run_objection_bypass` — Use objection to disable SSL pinning on Android/iOS.
- `check_proxy_setup` — Verify proxy setup for traffic interception.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-mobile-app-certificate-pinning-bypass`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-mobile-device-forensics-with-cellebrite (Skill Module)

performing-mobile-device-forensics-with-cellebrite\SKILL.md

name: performing-mobile-device-forensics-with-cellebrite
description: Agent for performing mobile device forensics. Analyzes mobile device extractions by parsing SQLite databases for messages, call logs, contacts, and location data from iOS and Android file system extractions. Use when the user asks to perform mobile device forensics with cellebrite.
version: 1.0.0
tags:
source: performing-mobile-device-forensics-with-cellebrite

Performing Mobile Device Forensics with Cellebrite

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** mobile device forensics with cellebrite.
- Capability: Agent for performing mobile device forensics. Analyzes mobile device extractions by parsing SQLite databases for messages, call logs, contacts, and location data from iOS and Android file system extractions.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, mobile, device, forensics, cellebrite`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for mobile device forensics with cellebrite
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-mobile-device-forensics-with-cellebrite`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-network-forensics-with-wireshark (Skill Module)

performing-network-forensics-with-wireshark\SKILL.md

name: performing-network-forensics-with-wireshark
description: Agent for performing network forensics with Wireshark/pyshark. Analyzes PCAP files to extract conversations, DNS queries, HTTP objects, detect beaconing patterns, and identify C2 communications. Use when the user asks to perform network forensics with wireshark.
version: 1.0.0
tags:
source: performing-network-forensics-with-wireshark

Performing Network Forensics with Wireshark

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Wireshark, pyshark API, tshark CLI, Display Filters

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** network forensics with wireshark.
- Capability: Agent for performing network forensics with Wireshark/pyshark. Analyzes PCAP files to extract conversations, DNS queries, HTTP objects, detect beaconing patterns, and identify C2 communications.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, network, forensics, wireshark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for network forensics with wireshark
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Wireshark
- pyshark API
- tshark CLI
- Display Filters

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-network-forensics-with-wireshark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-network-packet-capture-analysis (Skill Module)

performing-network-packet-capture-analysis\SKILL.md

name: performing-network-packet-capture-analysis
description: Agent for performing network packet capture analysis with scapy and tshark. Use when the user asks to perform network packet capture analysis.
version: 1.0.0
tags:
source: performing-network-packet-capture-analysis

Performing Network Packet Capture Analysis

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Wireshark, tshark, tcpdump, Scapy (Python), Zeek (Bro), NetworkMiner

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** network packet capture analysis.
- Capability: Agent for performing network packet capture analysis with scapy and tshark.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, network, packet, capture, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Analyze PCAP file using scapy for protocol statistics
2. Extract HTTP requests from PCAP using tshark
3. Detect suspicious network patterns in PCAP
4. Analyze TCP/UDP conversations using tshark

Key concepts

- NIST SP 800-86: Guide to Integrating Forensic Techniques
- RFC 791 (IP), RFC 793 (TCP), RFC 768 (UDP)
- Wireshark: GUI packet analyzer
- tshark: Command-line packet analyzer
- tcpdump: Packet capture utility
- Scapy (Python): Packet manipulation library
- Zeek (Bro): Network security monitoring
- NetworkMiner: Network forensic analysis tool

Tools & resources

- Wireshark
- tshark
- tcpdump
- Scapy (Python)
- Zeek (Bro)
- NetworkMiner

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `analyze_pcap_scapy` — Analyze PCAP file using scapy for protocol statistics.
- `extract_http_requests` — Extract HTTP requests from PCAP using tshark.
- `detect_suspicious_traffic` — Detect suspicious network patterns in PCAP.
- `conversation_analysis` — Analyze TCP/UDP conversations using tshark.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-network-packet-capture-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-network-traffic-analysis-with-tshark (Skill Module)

performing-network-traffic-analysis-with-tshark\SKILL.md

name: performing-network-traffic-analysis-with-tshark
description: Network traffic analysis agent using tshark and pyshark for PCAP analysis. Use when the user asks to perform network traffic analysis with tshark.
version: 1.0.0
tags:
source: performing-network-traffic-analysis-with-tshark

Performing Network Traffic Analysis with Tshark

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: PyShark Python API, Common Display Filters

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** network traffic analysis with tshark.
- Capability: Network traffic analysis agent using tshark and pyshark for PCAP analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, network, traffic, analysis, tshark`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract protocol hierarchy statistics using tshark
2. Extract IP conversations using tshark
3. Identify top source and destination IPs by packet count
4. Extract DNS queries from the capture
5. Detect DNS tunneling via high-entropy or long subdomain queries
6. Extract HTTP request URIs from the capture
7. Detect port scanning patterns (single source hitting many ports)
8. Extract all unique external IPs from the capture

Tools & resources

- PyShark Python API
- Common Display Filters

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_protocol_stats` — Extract protocol hierarchy statistics using tshark.
- `get_conversations` — Extract IP conversations using tshark.
- `get_top_talkers` — Identify top source and destination IPs by packet count.
- `extract_dns_queries` — Extract DNS queries from the capture.
- `detect_dns_tunneling` — Detect DNS tunneling via high-entropy or long subdomain queries.
- `extract_http_urls` — Extract HTTP request URIs from the capture.
- `detect_port_scan` — Detect port scanning patterns (single source hitting many ports).
- `extract_unique_ips` — Extract all unique external IPs from the capture.
- `generate_report` — Generate network traffic analysis report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-network-traffic-analysis-with-tshark`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-network-traffic-analysis-with-zeek (Skill Module)

performing-network-traffic-analysis-with-zeek\SKILL.md

name: performing-network-traffic-analysis-with-zeek
description: Agent for performing network traffic analysis with Zeek (Bro) log files. Use when the user asks to perform network traffic analysis with zeek.
version: 1.0.0
tags:
source: performing-network-traffic-analysis-with-zeek

Performing Network Traffic Analysis with ZEEK

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Zeek

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** network traffic analysis with zeek.
- Capability: Agent for performing network traffic analysis with Zeek (Bro) log files.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, network, traffic, analysis, zeek`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse a Zeek TSV log file into structured records
2. Analyze Zeek conn.log for network connection patterns
3. Analyze Zeek dns.log for DNS query patterns and anomalies
4. Analyze Zeek http.log for web traffic patterns
5. Analyze Zeek notice.log for security alerts
6. Run Zeek on a PCAP file to generate logs

Tools & resources

- Zeek

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `parse_zeek_log` — Parse a Zeek TSV log file into structured records.
- `analyze_conn_log` — Analyze Zeek conn.log for network connection patterns.
- `analyze_dns_log` — Analyze Zeek dns.log for DNS query patterns and anomalies.
- `analyze_http_log` — Analyze Zeek http.log for web traffic patterns.
- `analyze_notice_log` — Analyze Zeek notice.log for security alerts.
- `run_zeek_on_pcap` — Run Zeek on a PCAP file to generate logs.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-network-traffic-analysis-with-zeek`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-nist-csf-maturity-assessment (Skill Module)

performing-nist-csf-maturity-assessment\SKILL.md

name: performing-nist-csf-maturity-assessment
description: Agent for performing NIST Cybersecurity Framework (CSF) maturity assessment. Use when the user asks to perform nist csf maturity assessment.
version: 1.0.0
tags:
source: performing-nist-csf-maturity-assessment

Performing Nist Csf Maturity Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Maturity Levels, Primary Standard, CSF 2.0 Core Structure, Implementation Tiers Detail, Related NIST Publications

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** nist csf maturity assessment.
- Capability: Agent for performing NIST Cybersecurity Framework (CSF) maturity assessment.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, nist, maturity, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load assessment responses from CSV and calculate maturity scores
2. Generate detailed gap analysis with prioritized recommendations
3. Create a blank NIST CSF assessment CSV template
4. Generate executive-level maturity summary

Key concepts

- Published: February 26, 2024
- Publisher: National Institute of Standards and Technology (NIST)
- Document: NIST CSWP 29
- Scope: Applicable to all organizations regardless of size, sector, or maturity
- Key Change: Added Govern function as central pillar; expanded from 5 to 6 functions
- GV.OC-01 to GV.OC-05: Organizational context understood
- GV.RM-01 to GV.RM-07: Risk management strategy established
- GV.RR-01 to GV.RR-04: Roles, responsibilities, and authorities

Tools & resources

- Maturity Levels
- Primary Standard
- CSF 2.0 Core Structure
- Implementation Tiers Detail
- Related NIST Publications

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `assess_from_csv` — Load assessment responses from CSV and calculate maturity scores.
- `generate_gap_analysis` — Generate detailed gap analysis with prioritized recommendations.
- `create_assessment_template` — Create a blank NIST CSF assessment CSV template.
- `generate_executive_summary` — Generate executive-level maturity summary.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-nist-csf-maturity-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-oauth-scope-minimization-review (Skill Module)

performing-oauth-scope-minimization-review\SKILL.md

name: performing-oauth-scope-minimization-review
description: Agent for performing OAuth scope minimization review. Audits OAuth 2.0 permission grants in Microsoft Entra ID (Azure AD) to identify over-permissioned apps, stale grants, and excessive scopes. Use when the user asks to perform oauth scope minimization review.
version: 1.0.0
tags:
source: performing-oauth-scope-minimization-review

Performing Oauth Scope Minimization Review

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Microsoft Graph API, Required Permissions, Scope Risk Classification

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** oauth scope minimization review.
- Capability: Agent for performing OAuth scope minimization review. Audits OAuth 2.0 permission grants in Microsoft Entra ID (Azure AD) to identify over-permissioned apps, stale grants, and excessive scopes.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, oauth, scope, minimization, review`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for oauth scope minimization review
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Microsoft Graph API
- Required Permissions
- Scope Risk Classification

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-oauth-scope-minimization-review`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-oil-gas-cybersecurity-assessment (Skill Module)

performing-oil-gas-cybersecurity-assessment\SKILL.md

name: performing-oil-gas-cybersecurity-assessment
description: Agent for performing oil & gas sector cybersecurity assessment based on IEC 62443 and NIST frameworks. Use when the user asks to perform oil gas cybersecurity assessment.
version: 1.0.0
tags:
source: performing-oil-gas-cybersecurity-assessment

Performing Oil Gas Cybersecurity Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: OT Asset Categories, Frameworks Referenced

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** oil gas cybersecurity assessment.
- Capability: Agent for performing oil & gas sector cybersecurity assessment based on IEC 62443 and NIST frameworks.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cybersecurity, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Assess OT/IT network segmentation based on asset inventory
2. Assess IEC 62443 compliance from questionnaire responses
3. Assess Safety Instrumented System (SIS) security posture
4. Generate comprehensive oil & gas cybersecurity assessment report

Tools & resources

- OT Asset Categories
- Frameworks Referenced

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `assess_network_segmentation` — Assess OT/IT network segmentation based on asset inventory.
- `assess_iec62443_compliance` — Assess IEC 62443 compliance from questionnaire responses.
- `assess_safety_systems` — Assess Safety Instrumented System (SIS) security posture.
- `generate_assessment_report` — Generate comprehensive oil & gas cybersecurity assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-oil-gas-cybersecurity-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-open-source-intelligence-gathering (Skill Module)

performing-open-source-intelligence-gathering\SKILL.md

name: performing-open-source-intelligence-gathering
description: Agent for performing open source intelligence (OSINT) gathering. Use when the user asks to perform open source intelligence gathering.
version: 1.0.0
tags:
source: performing-open-source-intelligence-gathering

Performing Open Source Intelligence Gathering

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Security Headers Checked, PTES

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** open source intelligence gathering.
- Capability: Agent for performing open source intelligence (OSINT) gathering.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, open, source, intelligence, gathering`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Perform WHOIS lookup for domain registration info
2. Enumerate DNS records for intelligence gathering
3. Search for email addresses associated with a domain
4. Fingerprint web technologies from HTTP response headers
5. Generate social media profile URLs for a target name

Key concepts

- Search engine queries
- Social media analysis
- Public records review
- Website analysis and spidering
- Metadata extraction from documents
- Technology stack identification
- Web application fingerprinting
- Active subdomain brute-forcing

Tools & resources

- Security Headers Checked
- PTES

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `whois_lookup` — Perform WHOIS lookup for domain registration info.
- `dns_enumeration` — Enumerate DNS records for intelligence gathering.
- `email_harvest` — Search for email addresses associated with a domain.
- `technology_fingerprint` — Fingerprint web technologies from HTTP response headers.
- `social_media_search` — Generate social media profile URLs for a target name.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-open-source-intelligence-gathering`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-osint-with-spiderfoot (Skill Module)

performing-osint-with-spiderfoot\SKILL.md

name: performing-osint-with-spiderfoot
description: OSINT automation agent using SpiderFoot REST API for target profiling and reconnaissance. Use when the user asks to perform osint with spiderfoot.
version: 1.0.0
tags:
source: performing-osint-with-spiderfoot

Performing Osint with Spiderfoot

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Scan Use Cases, Data Element Types, Agent CLI Usage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** osint with spiderfoot.
- Capability: OSINT automation agent using SpiderFoot REST API for target profiling and reconnaissance.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, osint, spiderfoot`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create a requests session for SpiderFoot API
2. List available SpiderFoot modules
3. List available scan types (use cases)
4. Start a new SpiderFoot scan via REST API
5. Check scan status
6. Poll scan status until completion or timeout
7. Retrieve all results from a completed scan
8. Categorize scan results by data element type

Tools & resources

- Scan Use Cases
- Data Element Types
- Agent CLI Usage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_sf_session` — Create a requests session for SpiderFoot API.
- `list_modules` — List available SpiderFoot modules.
- `list_scan_types` — List available scan types (use cases).
- `start_scan` — Start a new SpiderFoot scan via REST API.
- `get_scan_status` — Check scan status.
- `wait_for_scan` — Poll scan status until completion or timeout.
- `get_scan_results` — Retrieve all results from a completed scan.
- `categorize_results` — Categorize scan results by data element type.
- `generate_target_profile` — Generate structured OSINT profile from categorized results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-osint-with-spiderfoot`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ot-network-security-assessment (Skill Module)

performing-ot-network-security-assessment\SKILL.md

name: performing-ot-network-security-assessment
description: Agent for performing OT network security assessment based on Purdue model. Use when the user asks to perform ot network security assessment.
version: 1.0.0
tags:
source: performing-ot-network-security-assessment

Performing Ot Network Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: OT Protocol Ports

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ot network security assessment.
- Capability: Agent for performing OT network security assessment based on Purdue model.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, network, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Assess OT asset inventory against Purdue model zones
2. Check OT network segmentation and firewall rules
3. Scan for common OT protocols on a subnet using nmap
4. Generate comprehensive OT security assessment

Key concepts

- Defines foundational concepts including zones, conduits, security levels, and the IACS reference architecture
- Establishes the vocabulary used across all parts of the standard
- Requirements for an IACS security management system (SMS)
- Covers risk assessment, security policies, organization, staff competence, and awareness
- Defines the process for partitioning an IACS into zones and conduits
- Establishes Security Level targets (SL-T) for each zone based on risk assessment
- Zone characteristics: assets, access points, communication channels, data flows
- Conduit characteristics: connected zones, protocols, security countermeasures

Tools & resources

- OT Protocol Ports

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `assess_asset_inventory` — Assess OT asset inventory against Purdue model zones.
- `assess_network_segmentation` — Check OT network segmentation and firewall rules.
- `scan_ot_protocols` — Scan for common OT protocols on a subnet using nmap.
- `generate_assessment_report` — Generate comprehensive OT security assessment.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ot-network-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ot-vulnerability-assessment-with-claroty (Skill Module)

performing-ot-vulnerability-assessment-with-claroty\SKILL.md

name: performing-ot-vulnerability-assessment-with-claroty
description: Agent for performing OT vulnerability assessment with Claroty xDome platform. Use when the user asks to perform ot vulnerability assessment with claroty.
version: 1.0.0
tags:
source: performing-ot-vulnerability-assessment-with-claroty

Performing Ot Vulnerability Assessment with Claroty

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: ClarotyVulnClient API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ot vulnerability assessment with claroty.
- Capability: Agent for performing OT vulnerability assessment with Claroty xDome platform.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, vulnerability, assessment, claroty`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Retrieve and analyze OT vulnerabilities
2. Prioritize OT vulnerabilities for remediation based on risk
3. Get overall OT risk posture

Tools & resources

- ClarotyVulnClient API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `assess_vulnerabilities` — Retrieve and analyze OT vulnerabilities.
- `prioritize_remediation` — Prioritize OT vulnerabilities for remediation based on risk.
- `get_risk_overview` — Get overall OT risk posture.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ot-vulnerability-assessment-with-claroty`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ot-vulnerability-scanning-safely (Skill Module)

performing-ot-vulnerability-scanning-safely\SKILL.md

name: performing-ot-vulnerability-scanning-safely
description: Agent for performing OT vulnerability scanning safely — passive and rate-limited approaches. Use when the user asks to perform ot vulnerability scanning safely.
version: 1.0.0
tags:
source: performing-ot-vulnerability-scanning-safely

Performing Ot Vulnerability Scanning Safely

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: OT Protocol Safety Classification

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ot vulnerability scanning safely.
- Capability: Agent for performing OT vulnerability scanning safely — passive and rate-limited approaches.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, vulnerability, scanning, safely`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Perform passive network discovery without sending packets
2. Perform rate-limited TCP SYN scan safe for OT environments
3. Run nmap with OT-safe settings (low timing, no scripts)
4. Generate pre-scan safety checklist for OT environments

Tools & resources

- OT Protocol Safety Classification

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `passive_discovery` — Perform passive network discovery without sending packets.
- `safe_tcp_scan` — Perform rate-limited TCP SYN scan safe for OT environments.
- `nmap_safe_scan` — Run nmap with OT-safe settings (low timing, no scripts).
- `pre_scan_checklist` — Generate pre-scan safety checklist for OT environments.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ot-vulnerability-scanning-safely`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-packet-injection-attack (Skill Module)

performing-packet-injection-attack\SKILL.md

name: performing-packet-injection-attack
description: Agent for performing packet injection testing. Crafts and sends test packets using Scapy for authorized security assessments to validate IDS rules, firewall configurations, and anti-spoofing controls. Use when the user asks to perform packet injection attack.
version: 1.0.0
tags:
source: performing-packet-injection-attack

Performing Packet Injection Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Scapy Python API, Scapy Packet Layers, TCP Flag Values, hping3 CLI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** packet injection attack.
- Capability: Agent for performing packet injection testing. Crafts and sends test packets using Scapy for authorized security assessments to validate IDS rules, firewall configurations, and anti-spoofing controls.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, packet, injection, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for packet injection attack
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Scapy Python API
- Scapy Packet Layers
- TCP Flag Values
- hping3 CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-packet-injection-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-paste-site-monitoring-for-credentials (Skill Module)

performing-paste-site-monitoring-for-credentials\SKILL.md

name: performing-paste-site-monitoring-for-credentials
description: Agent for performing paste site monitoring for leaked credentials. Use when the user asks to perform paste site monitoring for credentials.
version: 1.0.0
tags:
source: performing-paste-site-monitoring-for-credentials

Performing Paste Site Monitoring for Credentials

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Rate Limiting

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** paste site monitoring for credentials.
- Capability: Agent for performing paste site monitoring for leaked credentials.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, paste, site, monitoring, credentials`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check if an email appears in Have I Been Pwned breaches
2. Check for email in paste site dumps via HIBP paste API
3. Check all emails from a domain for breach exposure
4. Scan a text file (paste dump) for credential patterns
5. Generate credential exposure report for an organization

Tools & resources

- Rate Limiting

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_haveibeenpwned` — Check if an email appears in Have I Been Pwned breaches.
- `check_paste_dumps` — Check for email in paste site dumps via HIBP paste API.
- `bulk_check_domain` — Check all emails from a domain for breach exposure.
- `scan_text_for_credentials` — Scan a text file (paste dump) for credential patterns.
- `generate_exposure_report` — Generate credential exposure report for an organization.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-paste-site-monitoring-for-credentials`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-phishing-simulation-with-gophish (Skill Module)

performing-phishing-simulation-with-gophish\SKILL.md

name: performing-phishing-simulation-with-gophish
description: Agent for performing phishing simulation campaigns with GoPhish API. Use when the user asks to perform phishing simulation with gophish.
version: 1.0.0
tags:
source: performing-phishing-simulation-with-gophish

Performing Phishing Simulation with Gophish

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Industry Standards, GoPhish Technical Reference, Phishing Simulation Best Practices

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** phishing simulation with gophish.
- Capability: Agent for performing phishing simulation campaigns with GoPhish API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, phishing, simulation, gophish`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. List all phishing simulation campaigns
2. Get detailed metrics for a phishing campaign
3. Launch a new phishing simulation campaign
4. List available templates, groups, and sending profiles
5. Generate phishing simulation report with recommendations

Key concepts

- Computer Fraud and Abuse Act (CFAA): Ensure written authorization before conducting simulations
- GDPR (EU): Data protection requirements for handling employee email addresses and click data
- CCPA (California): Employee data privacy considerations
- Company Acceptable Use Policy: Must align simulation with organizational policies
- NIST SP 800-50: Building an Information Technology Security Awareness and Training Program
- NIST SP 800-16: Information Technology Security Training Requirements
- SANS Security Awareness Maturity Model: Five levels from non-existent to metrics framework
- ISO 27001: 2022**: A.6.3 - Information security awareness, education and training

Tools & resources

- Industry Standards
- GoPhish Technical Reference
- Phishing Simulation Best Practices

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `list_campaigns` — List all phishing simulation campaigns.
- `get_campaign_metrics` — Get detailed metrics for a phishing campaign.
- `launch_campaign` — Launch a new phishing simulation campaign.
- `list_resources` — List available templates, groups, and sending profiles.
- `generate_report` — Generate phishing simulation report with recommendations.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-phishing-simulation-with-gophish`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-physical-intrusion-assessment (Skill Module)

performing-physical-intrusion-assessment\SKILL.md

name: performing-physical-intrusion-assessment
description: Agent for performing physical intrusion assessment — checklist management and finding documentation. Use when the user asks to perform physical intrusion assessment.
version: 1.0.0
tags:
source: performing-physical-intrusion-assessment

Performing Physical Intrusion Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Risk Classification, NIST References, Industry Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** physical intrusion assessment.
- Capability: Agent for performing physical intrusion assessment — checklist management and finding documentation.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, physical, intrusion, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Generate physical security assessment checklist
2. Score a completed physical security assessment
3. Generate executive physical security assessment report

Key concepts

- NIST SP 800-53 Rev. 5** - PE-1 through PE-20: Physical and Environmental Protection
- NIST SP 800-53 Rev. 5** - PE-3: Physical Access Control
- NIST SP 800-53 Rev. 5** - PE-6: Monitoring Physical Access
- ISO 27001: 2022** - A.7: Physical controls
- PCI DSS 4.0** - Requirement 9: Restrict physical access to cardholder data

Tools & resources

- Risk Classification
- NIST References
- Industry Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_checklist` — Generate physical security assessment checklist.
- `score_assessment` — Score a completed physical security assessment.
- `generate_report` — Generate executive physical security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-physical-intrusion-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-plc-firmware-security-analysis (Skill Module)

performing-plc-firmware-security-analysis\SKILL.md

name: performing-plc-firmware-security-analysis
description: Agent for performing PLC firmware security analysis. Use when the user asks to perform plc firmware security analysis.
version: 1.0.0
tags:
source: performing-plc-firmware-security-analysis

Performing Plc Firmware Security Analysis

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** plc firmware security analysis.
- Capability: Agent for performing PLC firmware security analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, firmware, security, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Extract firmware image using binwalk
2. Analyze firmware file metadata and calculate hashes
3. Scan extracted firmware for hardcoded credentials
4. Scan extracted firmware for common vulnerability patterns
5. Run full firmware security analysis pipeline

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `extract_firmware` — Extract firmware image using binwalk.
- `analyze_firmware_metadata` — Analyze firmware file metadata and calculate hashes.
- `scan_for_credentials` — Scan extracted firmware for hardcoded credentials.
- `scan_for_vulnerabilities` — Scan extracted firmware for common vulnerability patterns.
- `full_analysis` — Run full firmware security analysis pipeline.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-plc-firmware-security-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-post-quantum-cryptography-migration (Skill Module)

performing-post-quantum-cryptography-migration\SKILL.md

name: performing-post-quantum-cryptography-migration
description: Agent for performing post-quantum cryptography migration assessment. Scans TLS endpoints for quantum-vulnerable algorithms, assesses crypto-agility readiness, tests hybrid TLS (X25519MLKEM768) support, validates.... Use when the user asks to perform post quantum cryptography migration.
version: 1.0.0
tags:
source: performing-post-quantum-cryptography-migration

Performing Post Quantum Cryptography Migration

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Hybrid TLS Configuration, oqs-provider for OpenSSL 3.0+, Cryptographic Inventory Scanning, Quantum-Vulnerable Algorithm Reference

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** post quantum cryptography migration.
- Capability: Agent for performing post-quantum cryptography migration assessment. Scans TLS endpoints for quantum-vulnerable algorithms, assesses crypto-agility readiness, tests hybrid TLS (X25519MLKEM768) support, validates ML-KEM and ML-DSA algorithm functionality, and generates prioritized migration roadmaps per NIST FIPS 203/204/205 standards.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, post, quantum, cryptography, migration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan a TLS endpoint to extract cryptographic algorithm details. Connects to the target, performs TLS handshake, and extracts: - Protocol version - Cipher suite (key exchange, encryption, MAC) - Certificate details (algorithm, key size, validity) - Supported groups / curves Args: host: Target hostname or IP port: Target port (default 443) timeout: Connection timeout in seconds Returns: Dict with comprehensive TLS cryptographic inventory
2. Scan multiple TLS endpoints from a targets file. Args: targets_file: Path to file with one host[:port] per line port: Default port if not specified per target Returns: List of scan results for all targets
3. Assess organizational crypto-agility based on TLS scan results. Evaluates the ability to migrate from quantum-vulnerable algorithms to post-quantum alternatives without major infrastructure changes. Args: scan_results: List of TLS scan result dicts Returns: Crypto-agility assessment report
4. Test whether a server supports hybrid post-quantum TLS key exchange. Attempts connection using X25519MLKEM768 and other hybrid groups. Requires OpenSSL 3.5+ or oqs-provider. Args: host: Target hostname port: Target port Returns: Dict with hybrid TLS support test results
5. Test ML-KEM (CRYSTALS-Kyber / FIPS 203) key encapsulation support. Tests keygen, encapsulation, and decapsulation at all three security levels using either the mlkem Python library or OpenSSL with oqs-provider. Returns: Dict with ML-KEM validation results for each security level
6. Test ML-DSA (CRYSTALS-Dilithium / FIPS 204) digital signature support. Tests key generation, signing, and verification at all three security levels using OpenSSL with native support or oqs-provider. Returns: Dict with ML-DSA validation results
7. Generate a prioritized PQC migration roadmap. Prioritizes systems based on data sensitivity, exposure, crypto-agility, compliance requirements, and dependency chains. Args: scan_results: List of TLS scan results agility_assessment: Optional crypto-agility assessment Returns: Migration roadmap with phased recommendations
8. Check the local OpenSSL installation for PQC algorithm support. Returns: Dict with OpenSSL version, provider status, and PQC algorithm availability

Tools & resources

- Hybrid TLS Configuration
- oqs-provider for OpenSSL 3.0+
- Cryptographic Inventory Scanning

- Quantum-Vulnerable Algorithm Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_tls_endpoint` — Scan a TLS endpoint to extract cryptographic algorithm details. Connects to the target, performs TLS handshake, and extracts: - Protocol version - Cipher suite (key exchange, encryption, MAC) - Certificate details (algorithm, key size, validity) - Supported groups / curves Args: host: Target hostname or IP port: Target port (default 443) timeout: Connection timeout in seconds Returns: Dict with comprehensive TLS cryptographic inventory
- `scan_multiple_endpoints` — Scan multiple TLS endpoints from a targets file. Args: targets_file: Path to file with one host[:port] per line port: Default port if not specified per target Returns: List of scan results for all targets
- `assess_crypto_agility` — Assess organizational crypto-agility based on TLS scan results. Evaluates the ability to migrate from quantum-vulnerable algorithms to post-quantum alternatives without major infrastructure changes. Args: scan_results: List of TLS scan result dicts Returns: Crypto-agility assessment report
- `test_hybrid_tls_support` — Test whether a server supports hybrid post-quantum TLS key exchange. Attempts connection using X25519MLKEM768 and other hybrid groups. Requires OpenSSL 3.5+ or oqs-provider. Args: host: Target hostname port: Target port Returns: Dict with hybrid TLS support test results
- `test_mlkem_support` — Test ML-KEM (CRYSTALS-Kyber / FIPS 203) key encapsulation support. Tests keygen, encapsulation, and decapsulation at all three security levels using either the mlkem Python library or OpenSSL with oqs-provider. Returns: Dict with ML-KEM validation results for each security level
- `test_mldsa_support` — Test ML-DSA (CRYSTALS-Dilithium / FIPS 204) digital signature support. Tests key generation, signing, and verification at all three security levels using OpenSSL with native support or oqs-provider. Returns: Dict with ML-DSA validation results
- `generate_migration_roadmap` — Generate a prioritized PQC migration roadmap. Prioritizes systems based on data sensitivity, exposure, crypto-agility, compliance requirements, and dependency chains. Args: scan_results: List of TLS scan results agility_assessment: Optional crypto-agility assessment Returns: Migration roadmap with phased recommendations
- `check_openssl_pqc_support` — Check the local OpenSSL installation for PQC algorithm support. Returns: Dict with OpenSSL version, provider status, and PQC algorithm availability
- `generate_oqs_provider_config` — Generate an OpenSSL configuration file for oqs-provider. Returns the configuration text for enabling PQC algorithms via the Open Quantum Safe provider in OpenSSL 3.x.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-post-quantum-cryptography-migration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-power-grid-cybersecurity-assessment (Skill Module)

performing-power-grid-cybersecurity-assessment\SKILL.md

name: performing-power-grid-cybersecurity-assessment
description: Agent for performing power grid cybersecurity assessment based on NERC CIP standards. Use when the user asks to perform power grid cybersecurity assessment.
version: 1.0.0
tags:
source: performing-power-grid-cybersecurity-assessment

Performing Power Grid Cybersecurity Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** power grid cybersecurity assessment.
- Capability: Agent for performing power grid cybersecurity assessment based on NERC CIP standards.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, power, grid, cybersecurity, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate NERC CIP compliance assessment template
2. Score NERC CIP compliance from completed assessment
3. Assess Electronic Security Perimeter configuration

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_assessment_template` — Generate NERC CIP compliance assessment template.

- `assess_compliance` — Score NERC CIP compliance from completed assessment.
- `assess_esp_security` — Assess Electronic Security Perimeter configuration.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-power-grid-cybersecurity-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-privacy-impact-assessment (Skill Module)

performing-privacy-impact-assessment\SKILL.md

name: performing-privacy-impact-assessment
description: Agent for performing automated Privacy Impact Assessments (PIA/DPIA). Implements the NIST Privacy Framework PRAM methodology and ICO DPIA guidance for systematic identification and mitigation of privacy risks..... Use when the user asks to perform privacy impact assessment.
version: 1.0.0
tags:
source: performing-privacy-impact-assessment

Performing Privacy Impact Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: PrivacyImpactAssessmentEngine

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** privacy impact assessment.
- Capability: Agent for performing automated Privacy Impact Assessments (PIA/DPIA). Implements the NIST Privacy Framework PRAM methodology and ICO DPIA guidance for systematic identification and mitigation of privacy risks. Supports GDPR Article 35 and CCPA/CPRA compliance checks with risk scoring matrices.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, privacy, impact, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for privacy impact assessment
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- PrivacyImpactAssessmentEngine

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-privacy-impact-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-privilege-escalation-assessment (Skill Module)

performing-privilege-escalation-assessment\SKILL.md

name: performing-privilege-escalation-assessment
description: Agent for performing privilege escalation assessment. Enumerates potential privilege escalation vectors on Linux systems including SUID binaries, sudo misconfigurations, writable cron jobs, capabilities, and kernel.... Use when the user asks to perform privilege escalation assessment.
version: 1.0.0
tags:
source: performing-privilege-escalation-assessment

Performing Privilege Escalation Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** privilege escalation assessment.
- Capability: Agent for performing privilege escalation assessment. Enumerates potential privilege escalation vectors on Linux systems including SUID binaries, sudo misconfigurations, writable cron jobs, capabilities, and kernel version checks.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, privilege, escalation, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for privilege escalation assessment
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-privilege-escalation-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-privilege-escalation-on-linux (Skill Module)

performing-privilege-escalation-on-linux\SKILL.md

name: performing-privilege-escalation-on-linux
description: Agent for performing Linux privilege escalation enumeration — authorized testing only. Use when the user asks to perform privilege escalation on linux.
version: 1.0.0
tags:
source: performing-privilege-escalation-on-linux

Performing Privilege Escalation on Linux

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Common Kernel CVEs, CIS Benchmark - Linux

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** privilege escalation on linux.
- Capability: Agent for performing Linux privilege escalation enumeration — authorized testing only.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, privilege, escalation, linux`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Gather basic system information for privilege escalation assessment
2. Check sudo permissions for current user
3. Find SUID/SGID binaries that may allow privilege escalation
4. Find world-writable files and directories of interest
5. Enumerate cron jobs for privilege escalation opportunities
6. Find binaries with Linux capabilities set
7. Run complete privilege escalation enumeration

Key concepts

- Ensure permissions on /etc/crontab are configured (600 root:root)
- Ensure SUID/SGID files are reviewed regularly
- Ensure sudo is configured to use a pseudo-TTY
- Ensure no world-writable files exist in system paths

Tools & resources

- Common Kernel CVEs
- CIS Benchmark - Linux

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `enumerate_system_info` — Gather basic system information for privilege escalation assessment.
- `check_sudo_permissions` — Check sudo permissions for current user.
- `find_suid_binaries` — Find SUID/SGID binaries that may allow privilege escalation.
- `check_writable_files` — Find world-writable files and directories of interest.
- `check_cron_jobs` — Enumerate cron jobs for privilege escalation opportunities.
- `check_capabilities` — Find binaries with Linux capabilities set.
- `full_enumeration` — Run complete privilege escalation enumeration.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-privilege-escalation-on-linux`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-privileged-account-access-review (Skill Module)

performing-privileged-account-access-review\SKILL.md

name: performing-privileged-account-access-review
description: Privileged Account Access Review agent — audits privileged accounts for compliance with least-privilege and periodic recertification requirements. Use when the user asks to perform privileged account access review.
version: 1.0.0
tags:
source: performing-privileged-account-access-review

Performing Privileged Account Access Review

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Checks Performed, Compliance Frameworks, Regulatory Requirements, Industry Frameworks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** privileged account access review.
- Capability: Privileged Account Access Review agent — audits privileged accounts for compliance with least-privilege and periodic recertification requirements.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, privileged, account, access, review`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Load privileged account inventory from CSV
2. Flag accounts not used within max_days
3. Detect shared/generic privileged accounts
4. Flag accounts with overly broad privilege sets
5. Flag accounts overdue for recertification
6. Run all checks and produce a consolidated JSON report

Key concepts

- CC6.1: Logical and physical access controls restrict access to information assets
- CC6.2: Prior to issuing system credentials, the entity registers and authorizes new users
- CC6.3: The entity authorizes, modifies, or removes access in a timely manner
- Quarterly privileged access reviews required for audit evidence
- Access control standard requiring unique user identification
- Emergency access procedure (break-glass accounts)
- Automatic logoff and encryption/decryption
- Periodic review and modification of access rights

Tools & resources

- Checks Performed
- Compliance Frameworks
- Regulatory Requirements

- Industry Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `load_accounts` — Load privileged account inventory from CSV.
- `check_stale_accounts` — Flag accounts not used within max_days.
- `check_shared_accounts` — Detect shared/generic privileged accounts.
- `check_excessive_privileges` — Flag accounts with overly broad privilege sets.
- `check_recertification` — Flag accounts overdue for recertification.
- `generate_report` — Run all checks and produce a consolidated JSON report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-privileged-account-access-review`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-privileged-account-discovery (Skill Module)

performing-privileged-account-discovery\SKILL.md

name: performing-privileged-account-discovery
description: Privileged Account Discovery agent — enumerates privileged accounts across Active Directory using ldap3 and flags shadow admin paths. Use when the user asks to perform privileged account discovery.
version: 1.0.0
tags:
source: performing-privileged-account-discovery

Performing Privileged Account Discovery

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: ldap3 Library, Default Privileged Groups

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** privileged account discovery.
- Capability: Privileged Account Discovery agent — enumerates privileged accounts across Active Directory using ldap3 and flags shadow admin paths.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, privileged, account, discovery`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Establish LDAP connection
2. Extract domain distinguished name from RootDSE
3. Find all privileged group memberships recursively
4. Resolve nested group memberships using LDAP_MATCHING_RULE_IN_CHAIN
5. Discover service accounts via servicePrincipalName
6. Find users with adminCount=1 (shadow admins or orphaned flags)
7. Run full privileged account discovery and build JSON report

Tools & resources

- ldap3 Library
- Default Privileged Groups

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_ldap` — Establish LDAP connection.
- `get_domain_dn` — Extract domain distinguished name from RootDSE.
- `enumerate_privileged_groups` — Find all privileged group memberships recursively.
- `find_nested_memberships` — Resolve nested group memberships using LDAP_MATCHING_RULE_IN_CHAIN.
- `find_service_accounts` — Discover service accounts via servicePrincipalName.
- `find_admin_count_users` — Find users with adminCount=1 (shadow admins or orphaned flags).
- `generate_report` — Run full privileged account discovery and build JSON report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-privileged-account-discovery`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-purple-team-atomic-testing (Skill Module)

performing-purple-team-atomic-testing\SKILL.md

name: performing-purple-team-atomic-testing
description: Purple Team Atomic Testing Agent Parses Atomic Red Team YAML definitions, correlates execution logs with detection results, generates MITRE ATT&CK Navigator layers, and produces coverage gap reports. Usage: python.... Use when the user asks to perform purple team atomic testing.
version: 1.0.0
tags:
source: performing-purple-team-atomic-testing

Performing Purple Team Atomic Testing

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Arguments, Key Functions, Sigma Rule Templates

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** purple team atomic testing.
- Capability: Purple Team Atomic Testing Agent Parses Atomic Red Team YAML definitions, correlates execution logs with detection results, generates MITRE ATT&CK Navigator layers, and produces coverage gap reports. Usage: python agent.py --atomics-path /path/to/atomics --log-dir /path/to/logs python agent.py --atomics-path /path/to/atomics --detections detection_results.json python agent.py --mode navigator --output-layer coverage.json python agent.py --mode report --output-report coverage_report.json Requirements: pip install pyyaml
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, purple, team, atomic, testing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Parse all Atomic Red Team YAML files into a technique inventory
2. Load atomic test execution logs from JSON files
3. Load SIEM detection validation results
4. Generate a comprehensive coverage gap analysis report
5. Produce an ATT&CK Navigator v4.5 layer JSON
6. Suggest Sigma rule stubs for blind spot techniques
7. Print formatted coverage report to stdout
8. Generate a PowerShell script to re-test blind spot techniques

Tools & resources

- Arguments
- Key Functions
- Sigma Rule Templates

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_atomics_inventory` — Parse all Atomic Red Team YAML files into a technique inventory.
- `load_execution_logs` — Load atomic test execution logs from JSON files.
- `load_detection_results` — Load SIEM detection validation results.
- `compute_coverage_report` — Generate a comprehensive coverage gap analysis report.
- `generate_navigator_layer` — Produce an ATT&CK Navigator v4.5 layer JSON.
- `suggest_sigma_rules` — Suggest Sigma rule stubs for blind spot techniques.
- `print_coverage_report` — Print formatted coverage report to stdout.
- `generate_powershell_test_script` — Generate a PowerShell script to re-test blind spot techniques.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-purple-team-atomic-testing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-purple-team-exercise (Skill Module)

performing-purple-team-exercise\SKILL.md

name: performing-purple-team-exercise
description: Agent for performing purple team exercises. Coordinates red team technique execution with blue team detection validation, tracks ATT&CK-mapped test results, and generates detection coverage reports. Use when the user asks to perform purple team exercise.
version: 1.0.0
tags:
source: performing-purple-team-exercise

Performing Purple Team Exercise

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: MITRE Caldera API

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** purple team exercise.
- Capability: Agent for performing purple team exercises. Coordinates red team technique execution with blue team detection validation, tracks ATT&CK-mapped test results, and generates detection coverage reports.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, purple, team, exercise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for purple team exercise
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- MITRE Caldera API

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-purple-team-exercise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ransomware-response (Skill Module)

performing-ransomware-response\SKILL.md

name: performing-ransomware-response
description: Agent for performing ransomware response. Automates ransomware identification, impact assessment, backup verification, IOC extraction, and recovery tracking during ransomware incident response. Use when the user asks to perform ransomware response.
version: 1.0.0
tags:
source: performing-ransomware-response

Performing Ransomware Response

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Ransomware Identification Services, OFAC Sanctions Screening, Recovery Validation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ransomware response.
- Capability: Agent for performing ransomware response. Automates ransomware identification, impact assessment, backup verification, IOC extraction, and recovery tracking during ransomware incident response.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, ransomware, response`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for ransomware response
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Ransomware Identification Services
- OFAC Sanctions Screening
- Recovery Validation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ransomware-response`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ransomware-tabletop-exercise (Skill Module)

performing-ransomware-tabletop-exercise\SKILL.md

name: performing-ransomware-tabletop-exercise
description: Ransomware Tabletop Exercise agent — generates scenario injects, tracks participant decisions, scores response effectiveness, and produces an after-action report. Use when the user asks to perform ransomware tabletop exercise.
version: 1.0.0
tags:
source: performing-ransomware-tabletop-exercise

Performing Ransomware Tabletop Exercise

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Scenario Framework, Scoring Algorithm, Expected Actions by Phase, After, Exercise Standards, Ransomware-Specific Guidance

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ransomware tabletop exercise.
- Capability: Ransomware Tabletop Exercise agent — generates scenario injects, tracks participant decisions, scores response effectiveness, and produces an after-action report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, ransomware, tabletop, exercise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Return the scenario phases, optionally shuffled for advanced exercises
2. Score participant actions against expected actions for a phase
3. Score all phases and compute overall effectiveness
4. Generate After-Action Report
5. Run a demonstration exercise with simulated participant responses

Key concepts

- Standardized methodology for exercise design, conduct, and evaluation
- Defines exercise types: seminars, workshops, tabletops, drills, functional, full-scale
- Provides templates for exercise plans, evaluation guides, and AARs
- https: //www.fema.gov/emergency-managers/national-preparedness/exercises/hseep
- Framework for developing IT plan test and exercise programs
- Section 4.3: Tabletop exercises for incident response testing
- Covers exercise scoping, objectives, scenario development, and evaluation
- Free downloadable exercise scenarios for critical infrastructure sectors

Tools & resources

- Scenario Framework
- Scoring Algorithm
- Expected Actions by Phase

- After
- Exercise Standards
- Ransomware-Specific Guidance

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `generate_scenario` — Return the scenario phases, optionally shuffled for advanced exercises.
- `score_response` — Score participant actions against expected actions for a phase.
- `evaluate_exercise` — Score all phases and compute overall effectiveness.
- `generate_aar` — Generate After-Action Report.
- `run_demo` — Run a demonstration exercise with simulated participant responses.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ransomware-tabletop-exercise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-red-team-phishing-with-gophish (Skill Module)

performing-red-team-phishing-with-gophish\SKILL.md

name: performing-red-team-phishing-with-gophish
description: GoPhish Campaign Agent - Automates phishing simulation setup, launch, and analysis. Use when the user asks to perform red team phishing with gophish.
version: 1.0.0
tags:
source: performing-red-team-phishing-with-gophish

Performing Red Team Phishing with Gophish

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Python gophish Library, Campaign Model, Result Statuses

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** red team phishing with gophish.
- Capability: GoPhish Campaign Agent - Automates phishing simulation setup, launch, and analysis.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, team, phishing, gophish`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to GoPhish server via API
2. Create an email template in GoPhish
3. Create a landing page for credential capture
4. Create an SMTP sending profile
5. Import target users from a CSV file into a GoPhish group
6. Launch a phishing simulation campaign
7. Retrieve detailed results for a campaign
8. Calculate campaign performance metrics

Tools & resources

- Python gophish Library
- Campaign Model
- Result Statuses

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_gophish` — Connect to GoPhish server via API.
- `create_email_template` — Create an email template in GoPhish.
- `create_landing_page` — Create a landing page for credential capture.
- `create_smtp_profile` — Create an SMTP sending profile.
- `import_targets_from_csv` — Import target users from a CSV file into a GoPhish group.
- `launch_campaign` — Launch a phishing simulation campaign.
- `get_campaign_results` — Retrieve detailed results for a campaign.
- `analyze_campaign_metrics` — Calculate campaign performance metrics.
- `list_campaigns` — List all campaigns and their statuses.
- `generate_report` — Generate phishing simulation report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-red-team-phishing-with-gophish`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-red-team-with-covenant (Skill Module)

performing-red-team-with-covenant\SKILL.md

name: performing-red-team-with-covenant
description: Agent for red team operations with Covenant C2 framework. Automates Covenant C2 operations through its REST API: listener management, launcher generation, grunt monitoring, and task execution for authorized adversary.... Use when the user asks to perform red team with covenant.
version: 1.0.0
tags:
source: performing-red-team-with-covenant

Performing Red Team with Covenant

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Listener Configuration, Built-in Tasks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** red team with covenant.
- Capability: Agent for red team operations with Covenant C2 framework. Automates Covenant C2 operations through its REST API: listener management, launcher generation, grunt monitoring, and task execution for authorized adversary simulation engagements.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, team, covenant`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for red team with covenant
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Listener Configuration
- Built-in Tasks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-red-team-with-covenant`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-s7comm-protocol-security-analysis (Skill Module)

performing-s7comm-protocol-security-analysis\SKILL.md

name: performing-s7comm-protocol-security-analysis
description: S7comm Protocol Security Analysis agent — analyzes Siemens S7 protocol traffic from PCAP files using pyshark to detect unauthorized PLC access, password brute-force, and dangerous write operations. Use when the user asks to perform s7comm protocol security analysis.
version: 1.0.0
tags:
source: performing-s7comm-protocol-security-analysis

Performing S7comm Protocol Security Analysis

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: pyshark Library, S7comm Function Codes, S7comm Protocol Overview

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** s7comm protocol security analysis.
- Capability: S7comm Protocol Security Analysis agent — analyzes Siemens S7 protocol traffic from PCAP files using pyshark to detect unauthorized PLC access, password brute-force, and dangerous write operations.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, s7comm, protocol, security, analysis`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load S7comm packets from PCAP file
2. Detect potential unauthorized access to PLCs
3. Flag write, run, and stop operations on PLCs
4. Detect authentication brute-force via repeated setup_communication with errors
5. Compute traffic statistics
6. Run all analyses and produce consolidated JSON report

Tools & resources

- pyshark Library
- S7comm Function Codes
- S7comm Protocol Overview

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_pcap` — Load S7comm packets from PCAP file.
- `detect_unauthorized_access` — Detect potential unauthorized access to PLCs.
- `detect_dangerous_operations` — Flag write, run, and stop operations on PLCs.
- `detect_brute_force` — Detect authentication brute-force via repeated setup_communication with errors.
- `analyze_traffic_patterns` — Compute traffic statistics.
- `generate_report` — Run all analyses and produce consolidated JSON report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-s7comm-protocol-security-analysis`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-sca-dependency-scanning-with-snyk (Skill Module)

performing-sca-dependency-scanning-with-snyk\SKILL.md

name: performing-sca-dependency-scanning-with-snyk
description: SCA Dependency Scanning with Snyk agent — runs Snyk CLI to test project dependencies for known vulnerabilities, generates SARIF output, and enforces quality gates. Use when the user asks to perform sca dependency scanning with snyk.
version: 1.0.0
tags:
source: performing-sca-dependency-scanning-with-snyk

Performing Sca Dependency Scanning with Snyk

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: JSON Output Structure, OWASP SAMM, PCI DSS v4.0, License Compliance Framework

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** sca dependency scanning with snyk.
- Capability: SCA Dependency Scanning with Snyk agent — runs Snyk CLI to test project dependencies for known vulnerabilities, generates SARIF output, and enforces quality gates.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, dependency, scanning, snyk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Run snyk test and return parsed JSON results
2. Run snyk monitor to create a snapshot for continuous monitoring
3. Extract and normalize vulnerability findings
4. Apply quality gate based on severity counts
5. Convert findings to SARIF 2.1.0 format for GitHub integration
6. Run full scan and build consolidated report

Key concepts

- Applications using components with known vulnerabilities may be exploitable
- SCA tools like Snyk identify vulnerable versions and provide upgrade paths
- Includes both direct and transitive dependency scanning
- PW.4.1: Verify that acquired software meets security requirements
- PW.4.2: Review, analyze, and test software to identify vulnerabilities
- SCA scanning of all third-party components before integration
- Track the origin and version of all third-party software components
- Snyk monitor provides continuous tracking of dependency versions

Tools & resources

- JSON Output Structure
- OWASP SAMM

- PCI DSS v4.0
- License Compliance Framework

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `run_snyk_test` — Run snyk test and return parsed JSON results.
- `run_snyk_monitor` — Run snyk monitor to create a snapshot for continuous monitoring.
- `parse_vulnerabilities` — Extract and normalize vulnerability findings.
- `apply_quality_gate` — Apply quality gate based on severity counts.
- `generate_sarif` — Convert findings to SARIF 2.1.0 format for GitHub integration.
- `generate_report` — Run full scan and build consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-sca-dependency-scanning-with-snyk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-scada-hmi-security-assessment (Skill Module)

performing-scada-hmi-security-assessment\SKILL.md

name: performing-scada-hmi-security-assessment
description: SCADA HMI Security Assessment agent — analyzes SCADA HMI configurations for security weaknesses including default credentials, unencrypted protocols, and missing access controls. Use when the user asks to perform scada hmi security assessment.
version: 1.0.0
tags:
source: performing-scada-hmi-security-assessment

Performing Scada Hmi Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: SCADA Protocol Ports, pyshark for Protocol Analysis, HMI Configuration Checks, Common Default Credentials

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** scada hmi security assessment.
- Capability: SCADA HMI Security Assessment agent — analyzes SCADA HMI configurations for security weaknesses including default credentials, unencrypted protocols, and missing access controls.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, scada, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check for open SCADA-specific ports on target
2. Check for default credentials on HMI web interface
3. Analyze PCAP for insecure SCADA protocols
4. Analyze HMI configuration file for security weaknesses
5. Run all assessments and build consolidated report

Tools & resources

- SCADA Protocol Ports
- pyshark for Protocol Analysis
- HMI Configuration Checks
- Common Default Credentials

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_open_ports` — Check for open SCADA-specific ports on target.
- `check_default_credentials_http` — Check for default credentials on HMI web interface.
- `analyze_pcap_protocols` — Analyze PCAP for insecure SCADA protocols.
- `check_hmi_configuration` — Analyze HMI configuration file for security weaknesses.
- `generate_report` — Run all assessments and build consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-scada-hmi-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-second-order-sql-injection (Skill Module)

performing-second-order-sql-injection\SKILL.md

name: performing-second-order-sql-injection
description: Second-Order SQL Injection agent — detects stored SQL injection payloads by analyzing database content and tracing data flow from input to secondary query execution points. Use when the user asks to perform second order sql injection.
version: 1.0.0
tags:
source: performing-second-order-sql-injection

Performing Second Order SQL Injection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Attack Overview, Data Flow Tracing, Prevention

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** second order sql injection.
- Capability: Second-Order SQL Injection agent — detects stored SQL injection payloads by analyzing database content and tracing data flow from input to secondary query execution points.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, second, order, injection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan a database dump (JSON format) for stored SQL injection payloads
2. Scan source code for second-order SQL injection sinks (string concatenation with DB data)
3. Correlate stored payloads with code sinks to identify complete attack paths
4. Run analysis and build consolidated report

Tools & resources

- Attack Overview
- Data Flow Tracing
- Prevention

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_database_values` — Scan a database dump (JSON format) for stored SQL injection payloads.
- `scan_source_code` — Scan source code for second-order SQL injection sinks (string concatenation with DB data).
- `trace_data_flow` — Correlate stored payloads with code sinks to identify complete attack paths.
- `generate_report` — Run analysis and build consolidated report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-second-order-sql-injection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-security-headers-audit (Skill Module)

performing-security-headers-audit\SKILL.md

name: performing-security-headers-audit
description: Agent for performing security headers audit. Analyzes HTTP response headers for HSTS, CSP, X-Frame-Options, cookie security attributes, and information disclosure to identify missing or misconfigured browser-level.... Use when the user asks to perform security headers audit.
version: 1.0.0
tags:
source: performing-security-headers-audit

Performing Security Headers Audit

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Security Headers Checked, Cookie Security Attributes, Online Scanners

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** security headers audit.
- Capability: Agent for performing security headers audit. Analyzes HTTP response headers for HSTS, CSP, X-Frame-Options, cookie security attributes, and information disclosure to identify missing or misconfigured browser-level protections.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, security, headers, audit`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for security headers audit
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Security Headers Checked
- Cookie Security Attributes
- Online Scanners

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-security-headers-audit`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-serverless-function-security-review (Skill Module)

performing-serverless-function-security-review\SKILL.md

name: performing-serverless-function-security-review
description: Serverless function security review agent using boto3. Use when the user asks to perform serverless function security review.
version: 1.0.0
tags:
source: performing-serverless-function-security-review

Performing Serverless Function Security Review

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS API Calls Used, Environment Variables

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** serverless function security review.
- Capability: Serverless function security review agent using boto3.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, serverless, function, security, review`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for serverless function security review
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- AWS API Calls Used
- Environment Variables

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_lambda_client`
- `get_iam_client`
- `list_all_functions`
- `check_deprecated_runtime`
- `audit_execution_role`
- `check_env_secrets`
- `check_public_access`
- `run_review`
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-serverless-function-security-review`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-service-account-audit (Skill Module)

performing-service-account-audit\SKILL.md

name: performing-service-account-audit
description: Agent for auditing service accounts across AD, cloud, and databases. Discovers service accounts via LDAP queries, AWS IAM, and Azure AD, checks password age, privilege levels, and orphan status, then generates a.... Use when the user asks to perform service account audit.
version: 1.0.0
tags:
source: performing-service-account-audit

Performing Service Account Audit

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Microsoft AD, AWS IAM, Azure Entra ID, CyberArk DNA, Stealthbits (Netwrix)

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** service account audit.
- Capability: Agent for auditing service accounts across AD, cloud, and databases. Discovers service accounts via LDAP queries, AWS IAM, and Azure AD, checks password age, privilege levels, and orphan status, then generates a risk-classified compliance report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, service, account, audit`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Export AD service accounts using PowerShell/LDAP queries
2. Export cloud IAM service accounts (AWS credential report, Azure SP list, GCP SA list)
3. Query databases for application-specific service accounts
4. Consolidate into single inventory spreadsheet
5. Cross-reference with CMDB for ownership data
6. Check each account against privilege policy
7. Verify password/key rotation compliance (90-day max)
8. Check last logon/activity date
9. Validate owner assignment against HR data
10. Flag accounts meeting orphaned/stale/over-privileged criteria

Key concepts

- NIST SP 800-53 Rev 5: AC-2, AC-2(3), AC-6, IA-5, AU-6
- NIST SP 800-171: 3.1.1, 3.1.2, 3.5.1, 3.5.2
- CIS Controls v8: Control 5.3 - Disable Dormant Accounts, Control 5.4 - Restrict Administrator Privileges
- MITRE ATT&CK: T1078 (Valid Accounts), T1136 (Create Account)
- PCI DSS 4.0: 7.2.5 - Review user access, 8.6 - Application/system account management
- SOX Section 404: Service account access controls for financial systems
- Microsoft AD: Get-ADServiceAccount, Get-ADUser with SPN filter
- AWS IAM: Access Analyzer, Credential Report, IAM Access Advisor

Tools & resources

- Microsoft AD
- AWS IAM
- Azure Entra ID
- CyberArk DNA
- Stealthbits (Netwrix)

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-service-account-audit`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-service-account-credential-rotation (Skill Module)

performing-service-account-credential-rotation\SKILL.md

name: performing-service-account-credential-rotation
description: Agent for automating service account credential rotation. Rotates credentials for AWS IAM access keys, Azure service principals, and database accounts via HashiCorp Vault, with post-rotation health checks and.... Use when the user asks to perform service account credential rotation.
version: 1.0.0
tags:
source: performing-service-account-credential-rotation

Performing Service Account Credential Rotation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Compliance Requirements, Rotation Frequency Standards, Technology Standards

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** service account credential rotation.
- Capability: Agent for automating service account credential rotation. Rotates credentials for AWS IAM access keys, Azure service principals, and database accounts via HashiCorp Vault, with post-rotation health checks and rollback capability.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, service, account, credential, rotation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for service account credential rotation
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- Authenticator lifecycle management
- Credential rotation for compromised or expired secrets
- Automated mechanisms for credential management preferred
- Logical access controls restrict access to information assets
- Credentials are managed throughout their lifecycle
- Service account credentials rotated per policy
- Windows Server 2012+ domain functional level
- KDS Root Key required (one per forest)

Tools & resources

- Compliance Requirements
- Rotation Frequency Standards
- Technology Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-service-account-credential-rotation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-soap-web-service-security-testing (Skill Module)

performing-soap-web-service-security-testing\SKILL.md

name: performing-soap-web-service-security-testing
description: Agent for security testing SOAP web services. Parses WSDL definitions using zeep/lxml, tests for XXE, SQL injection, SOAPAction spoofing, and WS-Security bypass vulnerabilities. Use when the user asks to perform soap web service security testing.
version: 1.0.0
tags:
source: performing-soap-web-service-security-testing

Performing Soap Web Service Security Testing

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: SOAP Request Headers, Common Test Payloads, lxml Key Methods

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** soap web service security testing.
- Capability: Agent for security testing SOAP web services. Parses WSDL definitions using zeep/lxml, tests for XXE, SQL injection, SOAPAction spoofing, and WS-Security bypass vulnerabilities.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, soap, service, security, testing`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for soap web service security testing
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- SOAP Request Headers
- Common Test Payloads
- lxml Key Methods

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-soap-web-service-security-testing`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-soc-tabletop-exercise (Skill Module)

performing-soc-tabletop-exercise\SKILL.md

name: performing-soc-tabletop-exercise
description: SOC tabletop exercise management agent with scenario generation and scoring. Use when the user asks to perform soc tabletop exercise.
version: 1.0.0
tags:
source: performing-soc-tabletop-exercise

Performing SOC Tabletop Exercise

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Scenario Templates, Scoring Criteria

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** soc tabletop exercise.
- Capability: SOC tabletop exercise management agent with scenario generation and scoring.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, tabletop, exercise`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for soc tabletop exercise
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Scenario Templates
- Scoring Criteria

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_exercise_id`
- `create_exercise`
- `score_response`
- `calculate_overall_score`
- `generate_after_action_report`
- `print_exercise_summary`
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-soc-tabletop-exercise`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-soc2-type2-audit-preparation (Skill Module)

performing-soc2-type2-audit-preparation\SKILL.md

name: performing-soc2-type2-audit-preparation
description: Agent for SOC 2 Type II audit preparation, evidence collection, and compliance monitoring. Use when the user asks to perform soc2 type2 audit preparation.
version: 1.0.0
tags:
source: performing-soc2-type2-audit-preparation

Performing Soc2 Type2 Audit Preparation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: AWS Evidence Collection APIs, GitHub Evidence Collection APIs, Compliance Automation Platforms, Evidence Collection Frequency, Primary Standards, Trust Services Criteria Detail

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** soc2 type2 audit preparation.
- Capability: Agent for SOC 2 Type II audit preparation, evidence collection, and compliance monitoring.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, soc2, type2, audit, preparation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Perform a gap assessment against all Trust Services Criteria CC1-CC9
2. Collect AWS IAM evidence for CC6 (Access Controls)
3. Collect AWS CloudTrail evidence for CC7 (System Operations)
4. Collect AWS S3 public access evidence for CC6 (Access Controls)
5. Collect GitHub PR evidence for CC8 (Change Management)
6. Collect GitHub branch protection evidence for CC8
7. Run a suite of continuous compliance checks and return findings
8. Generate a prioritized remediation plan from gap assessment results

Key concepts

- Governing Body: American Institute of Certified Public Accountants (AICPA)
- Basis: Built on COSO 2013 Internal Control Framework
- Revision: Points of Focus updated in 2022 to address cloud, supply chain, and evolving technology risks
- Description Criteria: Updated July 2025
- Standard: AT-C Section 105, 205, and 320
- Purpose: Governs the attestation engagement performed by the CPA firm
- Effective: May 1, 2017 (replaced SSAE 16)
- CC1.1: COSO Principle 1 - Demonstrates commitment to integrity and ethical values

Tools & resources

- AWS Evidence Collection APIs

- GitHub Evidence Collection APIs
- Compliance Automation Platforms
- Evidence Collection Frequency
- Primary Standards
- Trust Services Criteria Detail
- Related Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Automation entry points

- `perform_gap_assessment` — Perform a gap assessment against all Trust Services Criteria CC1-CC9.
- `collect_aws_iam_evidence` — Collect AWS IAM evidence for CC6 (Access Controls).
- `collect_aws_cloudtrail_evidence` — Collect AWS CloudTrail evidence for CC7 (System Operations).
- `collect_aws_s3_public_access_evidence` — Collect AWS S3 public access evidence for CC6 (Access Controls).
- `collect_github_change_management_evidence` — Collect GitHub PR evidence for CC8 (Change Management).
- `collect_branch_protection_evidence` — Collect GitHub branch protection evidence for CC8.
- `run_continuous_compliance_checks` — Run a suite of continuous compliance checks and return findings.
- `generate_remediation_plan` — Generate a prioritized remediation plan from gap assessment results.
- `generate_evidence_manifest` — Generate a manifest of required evidence packages organized by criteria.
- `generate_readiness_report` — Generate a comprehensive audit readiness report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-soc2-type2-audit-preparation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-sqlite-database-forensics (Skill Module)

performing-sqlite-database-forensics\SKILL.md

name: performing-sqlite-database-forensics
description: Agent for SQLite database forensics. Parses SQLite file headers, analyzes freelist pages for deleted records, examines WAL files, decodes browser/app timestamps, and extracts evidence from common forensic databases. Use when the user asks to perform sqlite database forensics.
version: 1.0.0
tags:
source: performing-sqlite-database-forensics

Performing Sqlite Database Forensics

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: DB Browser for SQLite, sqlcipher, Belkasoft Evidence Center, Exponent SQLite Explorer

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** sqlite database forensics.
- Capability: Agent for SQLite database forensics. Parses SQLite file headers, analyzes freelist pages for deleted records, examines WAL files, decodes browser/app timestamps, and extracts evidence from common forensic databases.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, sqlite, database, forensics`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for sqlite database forensics
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- NIST SP 800-86: Guide to Integrating Forensic Techniques
- SWGDE Best Practices for Mobile Device Forensics
- DB Browser for SQLite: Open-source GUI editor
- sqlcipher: Encrypted SQLite database handling
- Belkasoft Evidence Center: Commercial SQLite forensic analysis
- Exponent SQLite Explorer: Forensic SQLite viewer with timestamp auto-detection
- FORC (Forensic Operations for Recognizing SQLite Content): Automated Android extraction
- Chrome History: %LOCALAPPDATA%\Google\Chrome\User Data\Default\History

Tools & resources

- DB Browser for SQLite
- sqlcipher
- Belkasoft Evidence Center

- Exponent SQLite Explorer

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-sqlite-database-forensics`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ssl-certificate-lifecycle-management (Skill Module)

performing-ssl-certificate-lifecycle-management\SKILL.md

name: performing-ssl-certificate-lifecycle-management
description: Agent for SSL/TLS certificate lifecycle management. Generates CSRs, parses X.509 certificates using the cryptography library, monitors expiration across infrastructure, checks OCSP revocation status, and maintains a.... Use when the user asks to perform ssl certificate lifecycle management.
version: 1.0.0
tags:
source: performing-ssl-certificate-lifecycle-management

Performing Ssl Certificate Lifecycle Management

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: cryptography Library, Python ssl Module, Certificate Types, Primary Standards, Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ssl certificate lifecycle management.
- Capability: Agent for SSL/TLS certificate lifecycle management. Generates CSRs, parses X.509 certificates using the cryptography library, monitors expiration across infrastructure, checks OCSP revocation status, and maintains a certificate inventory.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, certificate, lifecycle, management`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for ssl certificate lifecycle management
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- Description: Core X.509 certificate format and Certificate Revocation List (CRL)
- Description: Real-time certificate revocation checking
- Description: Protocol for automating certificate issuance (Let's Encrypt)
- Description: Public logging framework for TLS certificates
- Description: CSR format specification
- Description: Key management guidance for TLS certificates
- Rate limits: 50 certificates per domain per week
- Description: Web security scanning including TLS configuration

Tools & resources

- cryptography Library
- Python ssl Module
- Certificate Types

- Primary Standards
- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ssl-certificate-lifecycle-management`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ssl-stripping-attack (Skill Module)

performing-ssl-stripping-attack\SKILL.md

name: performing-ssl-stripping-attack
description: SSL stripping assessment agent using subprocess wrappers for bettercap and curl. Use when the user asks to perform ssl stripping attack.
version: 1.0.0
tags:
source: performing-ssl-stripping-attack

Performing Ssl Stripping Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: External Tools Required, Risk Levels

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ssl stripping attack.
- Capability: SSL stripping assessment agent using subprocess wrappers for bettercap and curl.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, stripping, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check HSTS header on a target URL using curl
2. Check if domain is in HSTS preload list via the hstspreload.org API
3. Follow HTTP redirects and check for HTTPS upgrade
4. Fetch page and check for HTTP resources on an HTTPS page
5. Check for key security headers that complement HSTS
6. Run full SSL stripping assessment against a list of target domains

Tools & resources

- External Tools Required
- Risk Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_hsts_header` — Check HSTS header on a target URL using curl.
- `check_hsts_preload` — Check if domain is in HSTS preload list via the hstspreload.org API.
- `check_redirect_chain` — Follow HTTP redirects and check for HTTPS upgrade.
- `check_mixed_content` — Fetch page and check for HTTP resources on an HTTPS page.
- `check_security_headers` — Check for key security headers that complement HSTS.
- `run_assessment` — Run full SSL stripping assessment against a list of target domains.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ssl-stripping-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ssl-tls-inspection-configuration (Skill Module)

performing-ssl-tls-inspection-configuration\SKILL.md

name: performing-ssl-tls-inspection-configuration
description: Agent for SSL/TLS inspection configuration validation. Verifies TLS inspection is working by comparing certificate issuers, validates CA deployment on endpoints, checks TLS version enforcement, audits decryption.... Use when the user asks to perform ssl tls inspection configuration.
version: 1.0.0
tags:
source: performing-ssl-tls-inspection-configuration

Performing Ssl TLS Inspection Configuration

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Exemption Categories

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ssl tls inspection configuration.
- Capability: Agent for SSL/TLS inspection configuration validation. Verifies TLS inspection is working by comparing certificate issuers, validates CA deployment on endpoints, checks TLS version enforcement, audits decryption exemption lists, and monitors inspection health.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, inspection, configuration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for ssl tls inspection configuration
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Exemption Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ssl-tls-inspection-configuration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ssl-tls-security-assessment (Skill Module)

performing-ssl-tls-security-assessment\SKILL.md

name: performing-ssl-tls-security-assessment
description: Agent for performing SSL/TLS security assessment using sslyze. Scans TLS server configurations to evaluate cipher suites, protocol versions, certificate chains, HSTS, and known vulnerabilities like Heartbleed and ROBOT. Use when the user asks to perform ssl tls security assessment.
version: 1.0.0
tags:
source: performing-ssl-tls-security-assessment

Performing Ssl TLS Security Assessment

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: sslyze Python API, Scan Command Attributes, Weak Cipher Suite Keywords, sslyze CLI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ssl tls security assessment.
- Capability: Agent for performing SSL/TLS security assessment using sslyze. Scans TLS server configurations to evaluate cipher suites, protocol versions, certificate chains, HSTS, and known vulnerabilities like Heartbleed and ROBOT.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, security, assessment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for ssl tls security assessment
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- sslyze Python API
- Scan Command Attributes
- Weak Cipher Suite Keywords
- sslyze CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ssl-tls-security-assessment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-ssrf-vulnerability-exploitation (Skill Module)

performing-ssrf-vulnerability-exploitation\SKILL.md

name: performing-ssrf-vulnerability-exploitation
description: SSRF Vulnerability Testing Agent - Tests for Server-Side Request Forgery via URL parameters. Use when the user asks to perform ssrf vulnerability exploitation.
version: 1.0.0
tags:
source: performing-ssrf-vulnerability-exploitation

Performing SSRF Vulnerability Exploitation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: IP Encoding Bypass Techniques, Python requests, SSRF Impact Levels

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** ssrf vulnerability exploitation.
- Capability: SSRF Vulnerability Testing Agent - Tests for Server-Side Request Forgery via URL parameters.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, ssrf, vulnerability, exploitation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Send an SSRF payload through the target URL parameter
2. Test cloud metadata SSRF payloads
3. Use SSRF to scan internal ports
4. Test SSRF filter bypass techniques
5. Test non-HTTP protocol handlers for SSRF
6. Generate SSRF assessment report

Tools & resources

- IP Encoding Bypass Techniques
- Python requests
- SSRF Impact Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_ssrf_payload` — Send an SSRF payload through the target URL parameter.
- `test_metadata_endpoints` — Test cloud metadata SSRF payloads.
- `test_internal_port_scan` — Use SSRF to scan internal ports.
- `test_bypass_techniques` — Test SSRF filter bypass techniques.
- `test_protocol_handlers` — Test non-HTTP protocol handlers for SSRF.
- `generate_report` — Generate SSRF assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-ssrf-vulnerability-exploitation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-static-malware-analysis-with-pe-studio (Skill Module)

performing-static-malware-analysis-with-pe-studio\SKILL.md

name: performing-static-malware-analysis-with-pe-studio
description: Static malware analysis agent using pefile for PE binary inspection. Use when the user asks to perform static malware analysis with pe studio.
version: 1.0.0
tags:
source: performing-static-malware-analysis-with-pe-studio

Performing Static Malware Analysis with PE Studio

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Suspicious Import Categories

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** static malware analysis with pe studio.
- Capability: Static malware analysis agent using pefile for PE binary inspection.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, static, malware, analysis, studio`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for static malware analysis with pe studio
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Suspicious Import Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes`
- `analyze_sections`
- `detect_packer`
- `analyze_imports`
- `extract_strings`
- `analyze_resources`
- `analyze_pe`
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-static-malware-analysis-with-pe-studio`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-steganography-detection (Skill Module)

performing-steganography-detection\SKILL.md

name: performing-steganography-detection
description: Steganography detection agent using Pillow, numpy, and subprocess tools. Use when the user asks to perform steganography detection.
version: 1.0.0
tags:
source: performing-steganography-detection

Performing Steganography Detection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Finding Types

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** steganography detection.
- Capability: Steganography detection agent using Pillow, numpy, and subprocess tools.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, steganography, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check for data appended after JPEG/PNG end markers
2. Perform LSB analysis on image channels
3. Extract LSB data from red channel and check for file signatures
4. Run binwalk to detect embedded files
5. Run zsteg on PNG/BMP files for LSB detection
6. Attempt steghide extraction with multiple passwords
7. Full steganalysis pipeline for a single file

Tools & resources

- Finding Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_trailing_data` — Check for data appended after JPEG/PNG end markers.
- `lsb_analysis` — Perform LSB analysis on image channels.
- `extract_lsb_data` — Extract LSB data from red channel and check for file signatures.
- `run_binwalk` — Run binwalk to detect embedded files.
- `run_zsteg` — Run zsteg on PNG/BMP files for LSB detection.
- `run_steghide_extract` — Attempt steghide extraction with multiple passwords.
- `analyze_file` — Full steganalysis pipeline for a single file.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-steganography-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-subdomain-enumeration-with-subfinder (Skill Module)

performing-subdomain-enumeration-with-subfinder\SKILL.md

name: performing-subdomain-enumeration-with-subfinder
description: Agent for subdomain enumeration using subfinder and httpx. Runs ProjectDiscovery subfinder for passive subdomain discovery, validates live hosts with httpx, resolves DNS, and generates an attack surface report. Use when the user asks to perform subdomain enumeration with subfinder.
version: 1.0.0
tags:
source: performing-subdomain-enumeration-with-subfinder

Performing Subdomain Enumeration with Subfinder

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Passive Sources, Industry Standards, Tool References, API Provider Documentation, Regulatory Considerations

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** subdomain enumeration with subfinder.
- Capability: Agent for subdomain enumeration using subfinder and httpx. Runs ProjectDiscovery subfinder for passive subdomain discovery, validates live hosts with httpx, resolves DNS, and generates an attack surface report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, subdomain, enumeration, subfinder`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Configure API keys in provider-config.yaml for maximum source coverage
2. Run subfinder with `-all` flag against target domain(s)
3. Deduplicate results and remove out-of-scope entries
4. Validate live hosts with httpx including status codes and technologies
5. Resolve DNS records with dnsx to map IP infrastructure
6. Screenshot live hosts with gowitness for visual review
7. Feed live hosts into vulnerability scanner (nuclei) for automated checks
8. Schedule subfinder runs via cron (daily or weekly)
9. Compare new results against baseline subdomain list
10. Alert on newly discovered subdomains via webhook or email

Key concepts

- OWASP Testing Guide v4.2** — OTG-INFO-004: Enumerate applications on web server through subdomain discovery
- Passive subdomain enumeration does not involve active scanning and is generally legal
- Always verify scope and authorization before proceeding to active enumeration
- Bug bounty programs define specific scope for subdomain testing
- GDPR may apply when collecting data that reveals organizational structure

Tools & resources

- Passive Sources

- Industry Standards
- Tool References
- API Provider Documentation
- Regulatory Considerations

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-subdomain-enumeration-with-subfinder`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-supply-chain-attack-simulation (Skill Module)

performing-supply-chain-attack-simulation\SKILL.md

name: performing-supply-chain-attack-simulation
description: Simulate and detect software supply chain attacks: typosquatting, dependency confusion, hash verification. Use when the user asks to perform supply chain attack simulation.
version: 1.0.0
tags:
source: performing-supply-chain-attack-simulation

Performing Supply Chain Attack Simulation

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Tool Installation, PyPI JSON API, pip-audit CLI, pip Hash Verification, pypi-scan Typosquatting Detection, Levenshtein Distance Examples

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** supply chain attack simulation.
- Capability: Simulate and detect software supply chain attacks: typosquatting, dependency confusion, hash verification.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, supply, chain, attack, simulation`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute Levenshtein edit distance between two strings
2. Check if package name is suspiciously similar to popular packages
3. Fetch package metadata from PyPI JSON API
4. Check if private package names exist on public PyPI
5. Download package and verify SHA-256 hash against PyPI published digests
6. Run pip-audit to scan installed packages for known vulnerabilities
7. Detect suspicious metadata patterns in a PyPI package

Tools & resources

- Tool Installation
- PyPI JSON API
- pip-audit CLI
- pip Hash Verification
- pypi-scan Typosquatting Detection
- Levenshtein Distance Examples
- Dependency Confusion Test Structure
- Metadata Anomaly Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_levenshtein_distance` — Compute Levenshtein edit distance between two strings.
- `check_typosquatting` — Check if package name is suspiciously similar to popular packages.
- `query_pypi_metadata` — Fetch package metadata from PyPI JSON API.
- `check_dependency_confusion` — Check if private package names exist on public PyPI.
- `verify_package_hash` — Download package and verify SHA-256 hash against PyPI published digests.
- `run_pip_audit` — Run pip-audit to scan installed packages for known vulnerabilities.
- `analyze_metadata_anomalies` — Detect suspicious metadata patterns in a PyPI package.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-supply-chain-attack-simulation`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-thick-client-application-penetration-test (Skill Module)

performing-thick-client-application-penetration-test\SKILL.md

name: performing-thick-client-application-penetration-test
description: Agent for thick client application penetration testing. Performs static analysis (strings extraction, .NET detection), dynamic analysis (process monitoring, DLL search order checks), local storage auditing, and API.... Use when the user asks to perform thick client application penetration test.
version: 1.0.0
tags:
source: performing-thick-client-application-penetration-test

Performing Thick Client Application Penetration Test

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Analysis Tools, Static Analysis Targets, DLL Hijacking Detection, Local Storage Locations, Frameworks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** thick client application penetration test.
- Capability: Agent for thick client application penetration testing. Performs static analysis (strings extraction, .NET detection), dynamic analysis (process monitoring, DLL search order checks), local storage auditing, and API traffic interception assessment.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, thick, client, application, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for thick client application penetration test
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Analysis Tools
- Static Analysis Targets
- DLL Hijacking Detection
- Local Storage Locations
- Frameworks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-thick-client-application-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-threat-emulation-with-atomic-red-team (Skill Module)

performing-threat-emulation-with-atomic-red-team\SKILL.md

name: performing-threat-emulation-with-atomic-red-team
description: Agent for threat emulation with Atomic Red Team test execution. Use when the user asks to perform threat emulation with atomic red team.
version: 1.0.0
tags:
source: performing-threat-emulation-with-atomic-red-team

Performing Threat Emulation with Atomic Red Team

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: atomic

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** threat emulation with atomic red team.
- Capability: Agent for threat emulation with Atomic Red Team test execution.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, threat, emulation, atomic, team`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load Atomic Red Team test definitions for a technique
2. List all available Atomic Red Team techniques
3. Get detailed information about tests for a technique
4. Execute an Atomic Red Team test using atomic-operator
5. Execute atomic test manually by parsing YAML and running commands
6. Run cleanup commands for an atomic test
7. Compare available atomic tests against detection rules for gap analysis

Tools & resources

- atomic

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_atomic_tests` — Load Atomic Red Team test definitions for a technique.
- `list_available_techniques` — List all available Atomic Red Team techniques.
- `get_test_details` — Get detailed information about tests for a technique.
- `execute_atomic_test` — Execute an Atomic Red Team test using atomic-operator.
- `execute_atomic_manual` — Execute atomic test manually by parsing YAML and running commands.
- `run_cleanup` — Run cleanup commands for an atomic test.
- `build_coverage_matrix` — Compare available atomic tests against detection rules for gap analysis.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-threat-emulation-with-atomic-red-team`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-threat-hunting-with-elastic-siem (Skill Module)

performing-threat-hunting-with-elastic-siem\SKILL.md

name: performing-threat-hunting-with-elastic-siem
description: Threat hunting agent for Elastic SIEM using elasticsearch-py. Use when the user asks to perform threat hunting with elastic siem.
version: 1.0.0
tags:
source: performing-threat-hunting-with-elastic-siem

Performing Threat Hunting with Elastic SIEM

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Elastic, Environment Variables

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** threat hunting with elastic siem.
- Capability: Threat hunting agent for Elastic SIEM using elasticsearch-py.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, threat, hunting, elastic, siem`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Hunt for living-off-the-land binary abuse
2. Hunt for credential dumping activity (T1003)
3. Hunt for lateral movement patterns (T1021)
4. Hunt for persistence mechanisms (T1053, T1547)
5. Deploy a detection rule to Elastic Security via API

Tools & resources

- Elastic
- Environment Variables

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_es_client`
- `hunt_lolbins` — Hunt for living-off-the-land binary abuse.
- `hunt_credential_dumping` — Hunt for credential dumping activity (T1003).
- `hunt_lateral_movement` — Hunt for lateral movement patterns (T1021).
- `hunt_persistence` — Hunt for persistence mechanisms (T1053, T1547).
- `create_detection_rule` — Deploy a detection rule to Elastic Security via API.
- `run_all_hunts`
- `print_hunt_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-threat-hunting-with-elastic-siem`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-threat-hunting-with-yara-rules (Skill Module)

performing-threat-hunting-with-yara-rules\SKILL.md

name: performing-threat-hunting-with-yara-rules
description: Threat hunting agent using YARA rules. Scans files and directories with yara-python, supports rule compilation, multi-rule scanning, and structured JSON match output. Use when the user asks to perform threat hunting with yara rules.
version: 1.0.0
tags:
source: performing-threat-hunting-with-yara-rules

Performing Threat Hunting with YARA Rules

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: YARA, Match Object Fields, Community Rule Sources

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** threat hunting with yara rules.
- Capability: Threat hunting agent using YARA rules. Scans files and directories with yara-python, supports rule compilation, multi-rule scanning, and structured JSON match output.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, threat, hunting, yara, rules`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compile YARA rules from strings or directory
2. Scan a single file with compiled YARA rules
3. Recursively scan directory with YARA rules

Tools & resources

- YARA
- Match Object Fields
- Community Rule Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compile_rules` — Compile YARA rules from strings or directory.
- `scan_file` — Scan a single file with compiled YARA rules.
- `scan_directory` — Recursively scan directory with YARA rules.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-threat-hunting-with-yara-rules`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-threat-intelligence-sharing-with-misp (Skill Module)

performing-threat-intelligence-sharing-with-misp\SKILL.md

name: performing-threat-intelligence-sharing-with-misp
description: MISP Threat Intelligence Sharing agent - creates events, manages attributes, searches IOCs, and validates sharing configuration via PyMISP. Use when the user asks to perform threat intelligence sharing with misp.
version: 1.0.0
tags:
source: performing-threat-intelligence-sharing-with-misp

Performing Threat Intelligence Sharing with MISP

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: MISP, Creating Events, Searching Intelligence, Distribution Levels

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** threat intelligence sharing with misp.
- Capability: MISP Threat Intelligence Sharing agent - creates events, manages attributes, searches IOCs, and validates sharing configuration via PyMISP
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, threat, intelligence, sharing, misp`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Initialize PyMISP connection
2. Create a MISP event with attributes and tags
3. Validate event data quality for sharing readiness
4. Validate MISP sharing and feed configuration

Tools & resources

- MISP
- Creating Events
- Searching Intelligence
- Distribution Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_data`
- `connect_misp` — Initialize PyMISP connection.
- `create_event_from_data` — Create a MISP event with attributes and tags.
- `validate_event_quality` — Validate event data quality for sharing readiness.
- `validate_sharing_config` — Validate MISP sharing and feed configuration.
- `analyze`
- `generate_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-threat-intelligence-sharing-with-misp`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-threat-landscape-assessment-for-sector (Skill Module)

performing-threat-landscape-assessment-for-sector\SKILL.md

name: performing-threat-landscape-assessment-for-sector
description: Agent for sector-specific threat landscape assessment. Uses the attackcti library to query MITRE ATT&CK for threat groups targeting a sector, analyzes common techniques, maps attack vectors, and generates a strategic.... Use when the user asks to perform threat landscape assessment for sector.
version: 1.0.0
tags:
source: performing-threat-landscape-assessment-for-sector

Performing Threat Landscape Assessment for Sector

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Group Object Fields, Sector ISACs, Sector Threat Reports

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** threat landscape assessment for sector.
- Capability: Agent for sector-specific threat landscape assessment. Uses the attackcti library to query MITRE ATT&CK for threat groups targeting a sector, analyzes common techniques, maps attack vectors, and generates a strategic threat landscape report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, threat, landscape, assessment, sector`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for threat landscape assessment for sector
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Group Object Fields
- Sector ISACs
- Sector Threat Reports

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-threat-landscape-assessment-for-sector`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-threat-modeling-with-owasp-threat-dragon (Skill Module)

performing-threat-modeling-with-owasp-threat-dragon\SKILL.md

name: performing-threat-modeling-with-owasp-threat-dragon
description: Agent for threat modeling with OWASP Threat Dragon. Programmatically creates Threat Dragon JSON threat models, applies STRIDE analysis to DFD elements, manages threat inventory, and generates summary reports for.... Use when the user asks to perform threat modeling with owasp threat dragon.
version: 1.0.0
tags:
source: performing-threat-modeling-with-owasp-threat-dragon

Performing Threat Modeling with Owasp Threat Dragon

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: DFD Element Types, STRIDE Categories, Docker Deployment, OWASP Threat Modeling Process, NIST SP 800-154, Threat Severity Rating Scale

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** threat modeling with owasp threat dragon.
- Capability: Agent for threat modeling with OWASP Threat Dragon. Programmatically creates Threat Dragon JSON threat models, applies STRIDE analysis to DFD elements, manages threat inventory, and generates summary reports for security design reviews.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, threat, modeling, owasp, threat, dragon`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for threat modeling with owasp threat dragon
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- Identify data assets and their sensitivity levels
- Map data flows through system components
- Identify threat actors and attack vectors targeting data
- Assess risk based on data exposure and impact
- Document countermeasures protecting data at rest, in transit, and in use

Tools & resources

- DFD Element Types
- STRIDE Categories
- Docker Deployment
- OWASP Threat Modeling Process
- NIST SP 800-154
- Threat Severity Rating Scale

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-threat-modeling-with-owasp-threat-dragon`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-timeline-reconstruction-with-plaso (Skill Module)

performing-timeline-reconstruction-with-plaso\SKILL.md

name: performing-timeline-reconstruction-with-plaso
description: Forensic timeline reconstruction agent using Plaso subprocess wrappers. Use when the user asks to perform timeline reconstruction with plaso.
version: 1.0.0
tags:
source: performing-timeline-reconstruction-with-plaso

Performing Timeline Reconstruction with Plaso

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: External Tools Required, Default Parsers, Output Files

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** timeline reconstruction with plaso.
- Capability: Forensic timeline reconstruction agent using Plaso subprocess wrappers.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, timeline, reconstruction, plaso`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check that log2timeline.py and psort.py are available
2. Execute log2timeline.py to generate Plaso storage file
3. Export timeline from Plaso storage using psort.py
4. Create a Plaso filter file for targeted parsing
5. Analyze exported timeline CSV for patterns and anomalies
6. Export events within a specific incident time window
7. Run the full timeline reconstruction pipeline

Tools & resources

- External Tools Required
- Default Parsers
- Output Files

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `verify_plaso_installed` — Check that log2timeline.py and psort.py are available.
- `run_log2timeline` — Execute log2timeline.py to generate Plaso storage file.
- `run_psort_export` — Export timeline from Plaso storage using psort.py.
- `create_filter_file` — Create a Plaso filter file for targeted parsing.
- `analyze_timeline_csv` — Analyze exported timeline CSV for patterns and anomalies.
- `generate_incident_window` — Export events within a specific incident time window.
- `full_pipeline` — Run the full timeline reconstruction pipeline.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-timeline-reconstruction-with-plaso`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-user-behavior-analytics (Skill Module)

performing-user-behavior-analytics\SKILL.md

name: performing-user-behavior-analytics
description: User Behavior Analytics (UEBA) agent using elasticsearch-py. Use when the user asks to perform user behavior analytics.
version: 1.0.0
tags:
source: performing-user-behavior-analytics

Performing User Behavior Analytics

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Elasticsearch Index Requirements, Risk Score Weights

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** user behavior analytics.
- Capability: User Behavior Analytics (UEBA) agent using elasticsearch-py.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, user, behavior, analytics`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Calculate distance in km between two coordinates
2. Build behavioral baselines from historical authentication data
3. Detect logins from geographically distant locations within impossible timeframes
4. Detect logins outside user's normal working hours
5. Aggregate anomalies into composite risk scores per user

Tools & resources

- Elasticsearch Index Requirements
- Risk Score Weights

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_es_client`
- `haversine` — Calculate distance in km between two coordinates.
- `build_user_baselines` — Build behavioral baselines from historical authentication data.
- `detect_impossible_travel` — Detect logins from geographically distant locations within impossible timeframes.
- `detect_off_hours_access` — Detect logins outside user's normal working hours.
- `calculate_risk_scores` — Aggregate anomalies into composite risk scores per user.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-user-behavior-analytics`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-vlan-hopping-attack (Skill Module)

performing-vlan-hopping-attack\SKILL.md

name: performing-vlan-hopping-attack
description: VLAN hopping assessment agent using scapy for DTP and double-tagging tests. Use when the user asks to perform vlan hopping attack.
version: 1.0.0
tags:
source: performing-vlan-hopping-attack

Performing Vlan Hopping Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Scapy Layers Used

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** vlan hopping attack.
- Capability: VLAN hopping assessment agent using scapy for DTP and double-tagging tests.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, vlan, hopping, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Get network interface details
2. Check current VLAN configuration on the interface
3. Listen for DTP frames to assess trunk negotiation status
4. Capture CDP/LLDP frames to discover switch information
5. Send DTP desirable frames to attempt trunk negotiation
6. Create a VLAN subinterface
7. Send double-tagged 802.1Q frames for VLAN hopping
8. Remove VLAN subinterfaces created during testing

Tools & resources

- Scapy Layers Used

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_interface_info` — Get network interface details.
- `check_vlan_config` — Check current VLAN configuration on the interface.
- `listen_for_dtp` — Listen for DTP frames to assess trunk negotiation status.
- `listen_for_cdp_lldp` — Capture CDP/LLDP frames to discover switch information.
- `send_dtp_desirable` — Send DTP desirable frames to attempt trunk negotiation.
- `create_vlan_interface` — Create a VLAN subinterface.
- `send_double_tagged` — Send double-tagged 802.1Q frames for VLAN hopping.
- `cleanup_vlan_interfaces` — Remove VLAN subinterfaces created during testing.
- `run_assessment` — Run full VLAN hopping assessment.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-vlan-hopping-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-vulnerability-scanning-with-nessus (Skill Module)

performing-vulnerability-scanning-with-nessus\SKILL.md

name: performing-vulnerability-scanning-with-nessus
description: Vulnerability scanning agent using the Nessus REST API. Use when the user asks to perform vulnerability scanning with nessus.
version: 1.0.0
tags:
source: performing-vulnerability-scanning-with-nessus

Performing Vulnerability Scanning with Nessus

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Nessus, Environment Variables

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** vulnerability scanning with nessus.
- Capability: Vulnerability scanning agent using the Nessus REST API.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, vulnerability, scanning, nessus`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for vulnerability scanning with nessus
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Nessus
- Environment Variables

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `print_scan_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-vulnerability-scanning-with-nessus`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-web-application-firewall-bypass (Skill Module)

performing-web-application-firewall-bypass\SKILL.md

name: performing-web-application-firewall-bypass
description: Agent for testing WAF bypass techniques. Sends encoded, obfuscated, and protocol-level bypass payloads against a target URL to identify WAF evasion weaknesses in XSS, SQLi, and path traversal filtering. Use when the user asks to perform web application firewall bypass.
version: 1.0.0
tags:
source: performing-web-application-firewall-bypass

Performing Web Application Firewall Bypass

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Encoding Bypass Techniques, SQLi WAF Bypass Techniques, HTTP Method Bypass, WAF Detection Indicators

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** web application firewall bypass.
- Capability: Agent for testing WAF bypass techniques. Sends encoded, obfuscated, and protocol-level bypass payloads against a target URL to identify WAF evasion weaknesses in XSS, SQLi, and path traversal filtering.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, application, firewall, bypass`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for web application firewall bypass
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Encoding Bypass Techniques
- SQLi WAF Bypass Techniques
- HTTP Method Bypass
- WAF Detection Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-web-application-firewall-bypass`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-web-application-penetration-test (Skill Module)

performing-web-application-penetration-test\SKILL.md

name: performing-web-application-penetration-test
description: Web application penetration test agent using requests and subprocess. Use when the user asks to perform web application penetration test.
version: 1.0.0
tags:
source: performing-web-application-penetration-test

Performing Web Application Penetration Test

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: OWASP Test Coverage

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** web application penetration test.
- Capability: Web application penetration test agent using requests and subprocess.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, application, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Identify technology stack from response headers and cookies
2. Check for presence of security headers
3. Test for dangerous HTTP methods
4. Test CORS configuration for misconfigurations
5. Run directory enumeration using ffuf if available
6. Test for basic SQL injection indicators
7. Test for basic reflected XSS
8. Run web application security assessment

Tools & resources

- OWASP Test Coverage

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `fingerprint_technology` — Identify technology stack from response headers and cookies.
- `check_security_headers` — Check for presence of security headers.
- `test_http_methods` — Test for dangerous HTTP methods.
- `test_cors_config` — Test CORS configuration for misconfigurations.
- `run_directory_bruteforce` — Run directory enumeration using ffuf if available.
- `test_sql_injection_basic` — Test for basic SQL injection indicators.
- `test_xss_basic` — Test for basic reflected XSS.
- `run_assessment` — Run web application security assessment.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-web-application-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-web-application-scanning-with-nikto (Skill Module)

performing-web-application-scanning-with-nikto\SKILL.md

name: performing-web-application-scanning-with-nikto
description: Agent for web application scanning with Nikto. Runs Nikto via subprocess for web server vulnerability scanning, parses XML/JSON output, classifies findings by OSVDB/CVE, and generates a structured security assessment.... Use when the user asks to perform web application scanning with nikto.
version: 1.0.0
tags:
source: performing-web-application-scanning-with-nikto

Performing Web Application Scanning with Nikto

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Tuning Categories, XML Output Structure

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** web application scanning with nikto.
- Capability: Agent for web application scanning with Nikto. Runs Nikto via subprocess for web server vulnerability scanning, parses XML/JSON output, classifies findings by OSVDB/CVE, and generates a structured security assessment report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, application, scanning, nikto`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. **Nikto**: Server configuration and known vulnerability checks
2. **OWASP ZAP**: Application logic and dynamic analysis
3. **testssl.sh**: Comprehensive SSL/TLS assessment
4. **Nuclei**: Template-based CVE validation
5. **Manual Testing**: Validate and verify all findings
6. **Consolidated Report**: Merge results from all tools

Tools & resources

- Tuning Categories
- XML Output Structure

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-web-application-scanning-with-nikto`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-web-application-vulnerability-triage (Skill Module)

performing-web-application-vulnerability-triage\SKILL.md

name: performing-web-application-vulnerability-triage
description: Agent for web application vulnerability triage. Ingests scan results from multiple scanners (Nikto, ZAP, Burp), deduplicates findings, prioritizes by CVSS and exploitability, assigns SLA deadlines, and generates a.... Use when the user asks to perform web application vulnerability triage.
version: 1.0.0
tags:
source: performing-web-application-vulnerability-triage

Performing Web Application Vulnerability Triage

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: SLA Remediation Timelines, Priority Scoring Formula, Primary Standards, Scanner References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** web application vulnerability triage.
- Capability: Agent for web application vulnerability triage. Ingests scan results from multiple scanners (Nikto, ZAP, Burp), deduplicates findings, prioritizes by CVSS and exploitability, assigns SLA deadlines, and generates a triage report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, application, vulnerability, triage`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Import DAST scan results (ZAP XML/JSON, Burp XML)
2. Auto-classify findings by OWASP Top 10 category via CWE mapping
3. Filter out known false positive patterns (missing headers on non-sensitive pages, etc.)
4. Flag confirmed exploitation findings as true positives
5. Queue remaining findings for manual validation
6. Security analyst validates with manual testing in Burp/ZAP
7. Assign OWASP risk rating to validated findings
8. Push validated findings to DefectDojo/Jira
9. Import SAST scan results (Semgrep JSON, SonarQube)
10. Filter out findings in test files, example code, and dead code

Key concepts

- Purpose: Structured approach to evaluating likelihood and impact of web vulnerabilities
- Categories: A01 through A10 covering the most critical web application security risks
- Relevance: Manual validation techniques for scanner findings
- Relevance: Maps findings to common weakness enumeration for categorization
- Relevance: Industry standard vulnerability scoring complementing OWASP risk rating

Tools & resources

- SLA Remediation Timelines
- Priority Scoring Formula

- Primary Standards
- Scanner References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-web-application-vulnerability-triage`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-web-cache-deception-attack (Skill Module)

performing-web-cache-deception-attack\SKILL.md

name: performing-web-cache-deception-attack
description: Agent for testing web cache deception vulnerabilities. Appends static file extensions to authenticated URLs to test whether CDN/proxy caches serve personalized content to other users. Use when the user asks to perform web cache deception attack.
version: 1.0.0
tags:
source: performing-web-cache-deception-attack

Performing Web Cache Deception Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Attack Technique, Static Extensions to Test, Cache Detection Headers, Path Delimiter Confusion, Mitigation

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** web cache deception attack.
- Capability: Agent for testing web cache deception vulnerabilities. Appends static file extensions to authenticated URLs to test whether CDN/proxy caches serve personalized content to other users.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cache, deception, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for web cache deception attack
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Attack Technique
- Static Extensions to Test
- Cache Detection Headers
- Path Delimiter Confusion
- Mitigation

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-web-cache-deception-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-web-cache-poisoning-attack (Skill Module)

performing-web-cache-poisoning-attack\SKILL.md

name: performing-web-cache-poisoning-attack
description: Web cache poisoning assessment agent using requests and subprocess. Use when the user asks to perform web cache poisoning attack.
version: 1.0.0
tags:
source: performing-web-cache-poisoning-attack

Performing Web Cache Poisoning Attack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Unkeyed Headers Tested, Risk Levels

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** web cache poisoning attack.
- Capability: Web cache poisoning assessment agent using requests and subprocess.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, cache, poisoning, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate a unique cache buster parameter
2. Identify caching infrastructure from response headers
3. Determine if responses are being cached by comparing repeated requests
4. Test for unkeyed headers that are reflected in cached responses
5. Test cache key normalization issues
6. Test for web cache deception vulnerabilities
7. Full web cache poisoning assessment

Tools & resources

- Unkeyed Headers Tested
- Risk Levels

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `generate_cache_buster` — Generate a unique cache buster parameter.
- `identify_cache_layer` — Identify caching infrastructure from response headers.
- `test_cache_hit_miss` — Determine if responses are being cached by comparing repeated requests.
- `test_unkeyed_headers` — Test for unkeyed headers that are reflected in cached responses.
- `test_cache_key_normalization` — Test cache key normalization issues.
- `test_cache_deception` — Test for web cache deception vulnerabilities.
- `run_assessment` — Full web cache poisoning assessment.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-web-cache-poisoning-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-wifi-password-cracking-with-aircrack (Skill Module)

performing-wifi-password-cracking-with-aircrack\SKILL.md

name: performing-wifi-password-cracking-with-aircrack
description: WiFi password cracking assessment agent using aircrack-ng subprocess wrappers. Use when the user asks to perform wifi password cracking with aircrack.
version: 1.0.0
tags:
source: performing-wifi-password-cracking-with-aircrack

Performing Wifi Password Cracking with Aircrack

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: External Tools Required, Hashcat Modes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** wifi password cracking with aircrack.
- Capability: WiFi password cracking assessment agent using aircrack-ng subprocess wrappers.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, wifi, password, cracking, aircrack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Verify required tools are installed
2. List wireless interfaces
3. Enable monitor mode on wireless interface
4. Scan for nearby wireless networks
5. Capture WPA handshake from target network
6. Attempt PMKID capture using hcxdumptool
7. Crack WPA handshake using aircrack-ng with wordlist
8. Crack WPA hash using hashcat with GPU acceleration

Tools & resources

- External Tools Required
- Hashcat Modes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `check_tools` — Verify required tools are installed.
- `list_interfaces` — List wireless interfaces.
- `enable_monitor_mode` — Enable monitor mode on wireless interface.
- `scan_networks` — Scan for nearby wireless networks.
- `capture_handshake` — Capture WPA handshake from target network.
- `try_pmkid_capture` — Attempt PMKID capture using hcxdumptool.
- `crack_with_aircrack` — Crack WPA handshake using aircrack-ng with wordlist.
- `crack_with_hashcat` — Crack WPA hash using hashcat with GPU acceleration.
- `disable_monitor_mode` — Disable monitor mode and restore managed mode.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-wifi-password-cracking-with-aircrack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-windows-artifact-analysis-with-eric-zimmerman-tools (Skill Module)

performing-windows-artifact-analysis-with-eric-zimmerman-tools\SKILL.md

name: performing-windows-artifact-analysis-with-eric-zimmerman-tools
description: Agent for Windows artifact analysis with Eric Zimmerman tools. Runs EZ tools (MFTECmd, PECmd, LECmd, JLECmd, ShellBags Explorer CLI) via subprocess, parses CSV output, and builds a forensic timeline from Windows.... Use when the user asks to perform windows artifact analysis with eric zimmerman tools.
version: 1.0.0
tags:
source: performing-windows-artifact-analysis-with-eric-zimmerman-tools

Performing Windows Artifact Analysis with Eric Zimmerman Tools

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: EZ Tools Suite, Common CLI Flags, Key Artifacts and Locations, Industry Standards, Tool References, Official Resources

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** windows artifact analysis with eric zimmerman tools.
- Capability: Agent for Windows artifact analysis with Eric Zimmerman tools. Runs EZ tools (MFTECmd, PECmd, LECmd, JLECmd, ShellBags Explorer CLI) via subprocess, parses CSV output, and builds a forensic timeline from Windows filesystem and registry artifacts.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, windows, artifact, analysis, eric, zimmerman`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for windows artifact analysis with eric zimmerman tools
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- Framework for collecting, examining, and analyzing digital evidence
- Defines procedures for forensic acquisition and chain of custody
- EZ Tools align with NIST evidence handling and analysis guidelines
- International standard for identification, collection, acquisition, and preservation
- KAPE collection follows ISO 27037 acquisition methodology
- Timeline Explorer output supports ISO-compliant reporting
- Scientific Working Group on Digital Evidence guidelines
- Defines validation requirements for forensic tools

Tools & resources

- EZ Tools Suite
- Common CLI Flags
- Key Artifacts and Locations

- Industry Standards
- Tool References
- Official Resources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-windows-artifact-analysis-with-eric-zimmerman-tools`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-wireless-network-penetration-test (Skill Module)

performing-wireless-network-penetration-test\SKILL.md

name: performing-wireless-network-penetration-test
description: Agent for wireless network penetration testing. Runs aircrack-ng suite tools via subprocess for WiFi reconnaissance, WPA handshake capture, deauthentication testing, and generates a wireless security assessment report. Use when the user asks to perform wireless network penetration test.
version: 1.0.0
tags:
source: performing-wireless-network-penetration-test

Performing Wireless Network Penetration Test

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Aircrack-ng Suite, airodump-ng CSV Fields, Encryption Risk Levels, Standards, Wireless Encryption Comparison, Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** wireless network penetration test.
- Capability: Agent for wireless network penetration testing. Runs aircrack-ng suite tools via subprocess for WiFi reconnaissance, WPA handshake capture, deauthentication testing, and generates a wireless security assessment report.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, wireless, network, penetration, test`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for wireless network penetration test
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- IEEE 802.11: Wireless LAN standard
- WPA3 (WiFi Protected Access 3): Latest security protocol using SAE
- NIST SP 800-153: Guidelines for Securing Wireless Local Area Networks
- PCI DSS v4.0 Req 11.2: Wireless access point detection

Tools & resources

- Aircrack-ng Suite
- airodump-ng CSV Fields
- Encryption Risk Levels
- Standards
- Wireless Encryption Comparison
- Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-wireless-network-penetration-test`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-wireless-security-assessment-with-kismet (Skill Module)

performing-wireless-security-assessment-with-kismet\SKILL.md

name: performing-wireless-security-assessment-with-kismet
description: Agent for wireless security assessment with Kismet. Interfaces with Kismet's REST API for device enumeration, rogue AP detection, channel analysis, and wireless threat monitoring during security assessments. Use when the user asks to perform wireless security assessment with kismet.
version: 1.0.0
tags:
source: performing-wireless-security-assessment-with-kismet

Performing Wireless Security Assessment with Kismet

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: Device Fields, Kismet CLI

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** wireless security assessment with kismet.
- Capability: Agent for wireless security assessment with Kismet. Interfaces with Kismet's REST API for device enumeration, rogue AP detection, channel analysis, and wireless threat monitoring during security assessments.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, wireless, security, assessment, kismet`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for wireless security assessment with kismet
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Tools & resources

- Device Fields
- Kismet CLI

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-wireless-security-assessment-with-kismet`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

performing-yara-rule-development-for-detection (Skill Module)

performing-yara-rule-development-for-detection\SKILL.md

name: performing-yara-rule-development-for-detection
description: Agent for YARA rule development and testing. Creates YARA rules from malware samples by extracting unique strings and byte patterns, validates rules for performance, tests against sample sets, and generates detection.... Use when the user asks to perform yara rule development for detection.
version: 1.0.0
tags:
source: performing-yara-rule-development-for-detection

Performing YARA Rule Development for Detection

Role: Hands-on assessment
Audience: pentesters, blue teamers, and DFIR practitioners
Primary tools: YARA, Match Object Attributes, Condition Operators, Rule Naming Convention, Rule Quality Metrics, String Types Reference

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **perform** yara rule development for detection.
- Capability: Agent for YARA rule development and testing. Creates YARA rules from malware samples by extracting unique strings and byte patterns, validates rules for performance, tests against sample sets, and generates detection coverage reports.
- Goal: execute a focused security technique or assessment workflow.
- Trigger keywords: `performing, yara, rule, development, detection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Confirm scope, authorization, and safety constraints
2. Prepare tooling and environment for yara rule development for detection
3. Execute the technique methodically and record evidence
4. Validate outcomes and eliminate false paths
5. Report results with clear remediation guidance

Key concepts

- T1027 - Obfuscated Files: Rules detect packed/encoded malware
- T1036 - Masquerading: Rules identify file mimicry
- T1059 - Command Interpreter: Rules detect malicious scripts

Tools & resources

- YARA
- Match Object Attributes
- Condition Operators
- Rule Naming Convention
- Rule Quality Metrics
- String Types Reference

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `performing-yara-rule-development-for-detection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

prioritizing-vulnerabilities-with-cvss-scoring (Skill Module)

`prioritizing-vulnerabilities-with-cvss-scoring\SKILL.md`

name: prioritizing-vulnerabilities-with-cvss-scoring
description: Agent for prioritizing vulnerabilities with CVSS scoring. Calculates CVSS v3.1 base scores from metric vectors, enriches with EPSS exploit probability and KEV catalog data, and generates a risk-prioritized.... Use when the user asks to prioritizing vulnerabilities with cvss scoring.
version: 1.0.0
tags:
source: prioritizing-vulnerabilities-with-cvss-scoring

Prioritizing Vulnerabilities with CVSS Scoring

Role: Security workflow
Audience: security practitioners
Primary tools: CVSS v3.1 Base Metrics, Severity Ratings, CISA KEV Catalog, Priority Scoring Formula, Official CVSS Documentation, Complementary Scoring Systems

When to use

- The user wants to **prioritiz** vulnerabilities with cvss scoring.
- Capability: Agent for prioritizing vulnerabilities with CVSS scoring. Calculates CVSS v3.1 base scores from metric vectors, enriches with EPSS exploit probability and KEV catalog data, and generates a risk-prioritized remediation report.
- Goal: complete the task.
- Trigger keywords: `prioritizing, vulnerabilities, cvss, scoring`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Clarify the objective for vulnerabilities with cvss scoring
2. Gather inputs and required access
3. Execute the core workflow using skill scripts and references
4. Validate results and document findings
5. Recommend next actions

Key concepts

- NIST SP 800-40 Rev 4: Guide to Enterprise Patch Management Planning
- NIST NVD: National Vulnerability Database uses CVSS for all CVEs
- PCI DSS v4.0: Requires CVSS scoring for vulnerability prioritization
- ISO 27001: 2022 A.8.8**: Technical vulnerability management

Tools & resources

- CVSS v3.1 Base Metrics
- Severity Ratings
- CISA KEV Catalog
- Priority Scoring Formula
- Official CVSS Documentation
- Complementary Scoring Systems
- Industry Standards
- Severity Thresholds

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `prioritizing-vulnerabilities-with-cvss-scoring`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

processing-karpathy-raw (Skill Module)

processing-karpathy-raw\SKILL.md

name: processing-karpathy-raw
description: Use when processing raw capture notes, conversation exports, or dumps into structured wiki notes according to the Karpathy three-stage knowledge pipeline.
tags:

Processing Karpathy Raw

Overview

This [[skill]] details the process for transitioning captured materials from the RAW layer to structured knowledge in the WIKI and SCHEMA layers, maintaining strict formatting
[[[

[[[

processing-stix-taxii-feeds (Skill Module)

processing-stix-taxii-feeds\SKILL.md

name: processing-stix-taxii-feeds
description: STIX/TAXII threat intelligence feed processor using taxii2-client and stix2. Use when the user asks to processing stix taxii feeds.
version: 1.0.0
tags:
source: processing-stix-taxii-feeds

Processing STIX Taxii Feeds

Role: Security workflow
Audience: security practitioners
Primary tools: STIX Object Types Handled, Environment Variables

When to use

- The user wants to **process** stix taxii feeds.
- Capability: STIX/TAXII threat intelligence feed processor using taxii2-client and stix2.
- Goal: complete the task.
- Trigger keywords: `processing, stix, taxii, feeds`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Discover TAXII server API roots and collections
2. Fetch STIX objects from a TAXII collection with pagination
3. Parse and validate a STIX 2.1 bundle, categorizing objects by type
4. Extract actionable IOCs from parsed STIX indicators
5. Map relationships between STIX objects

Tools & resources

- STIX Object Types Handled
- Environment Variables

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_server` — Discover TAXII server API roots and collections.
- `fetch_collection` — Fetch STIX objects from a TAXII collection with pagination.
- `parse_stix_bundle` — Parse and validate a STIX 2.1 bundle, categorizing objects by type.
- `extract_iocs` — Extract actionable IOCs from parsed STIX indicators.
- `build_relationship_graph` — Map relationships between STIX objects.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `processing-stix-taxii-feeds`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

profiling-threat-actor-groups (Skill Module)

profiling-threat-actor-groups\SKILL.md

name: profiling-threat-actor-groups
description: Threat actor profiling agent using MITRE ATT&CK STIX data and STIX2 library. Use when the user asks to profiling threat actor groups.
version: 1.0.0
tags:
source: profiling-threat-actor-groups

Profiling Threat Actor Groups

Role: Security workflow
Audience: security practitioners
Primary tools: Data Source, STIX Object Types Queried, Example Output Fields

When to use

- The user wants to **profil** threat actor groups.
- Capability: Threat actor profiling agent using MITRE ATT&CK STIX data and STIX2 library.
- Goal: complete the task.
- Trigger keywords: `profiling, threat, actor, groups`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load MITRE ATT&CK STIX data from cache or download
2. List all threat actor groups in ATT&CK
3. Build a comprehensive profile for a specific threat actor group
4. Map a group's techniques organized by ATT&CK tactic
5. Compare techniques and tools across multiple threat actor groups

Tools & resources

- Data Source
- STIX Object Types Queried
- Example Output Fields

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_attack_data` — Load MITRE ATT&CK STIX data from cache or download.
- `list_threat_groups` — List all threat actor groups in ATT&CK.
- `get_group_profile` — Build a comprehensive profile for a specific threat actor group.
- `get_group_techniques_by_tactic` — Map a group's techniques organized by ATT&CK tactic.
- `compare_groups` — Compare techniques and tools across multiple threat actor groups.
- `print_profile`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `profiling-threat-actor-groups`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

project-memory (Skill Module)

`project-templates-toolkit\skills\project-memory\SKILL.md`

name: project-memory
description: Generate CLAUDE.md project memory files that transfer institutional knowledge, not obvious information. Use when setting up new journalism projects, onboarding collaborators, or documenting project-specific quirks. Includes templates for editorial tools, event websites, publications, research projects, content pipelines, and digital archives.
tags:

Project memory generator

Create CLAUDE.md files that transfer institutional knowledge, not obvious information. Think like a senior journalist onboarding a competent colleague — you don't explain how journalism works, you explain YOUR project's quirks.

CLAUDE.md is advisory, not enforced

Anthropic is explicit on this point: CLAUDE.md content is delivered as a user message after the system prompt. Claude reads it and tries to follow it, but there's no guarantee of strict compliance — especially with vague or conflicting instructions. Source: https://code.claude.com/docs/en/memory

This affects how you write a CLAUDE.md and what you put elsewhere:

Mechanism	Use for
CLAUDE.md	Standing facts, conventions, "always do X" rules
[[Skills]]	Multi-step procedures, on-demand [[[[

Size guidance: target under 200 lines

Anthropic's explicit guidance as of 2026: target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence. Bloated CLAUDE.md files cause Claude to ignore the actual rules. Source: https://code.claude.com/docs/en/best-practices

Going over 200 lines is a signal to use one of these instead:

- `.claude/rules/` **with** `paths:` **frontmatter** — file-pattern-scoped rules that load only when Claude works with matching files. Replaces `@import` chains as the size-management mechanism. (Note: `@imports` no longer help with context size — Anthropic explicitly notes that imports load fully at launch.)
- **[[Skills]]** — for multi-step procedures that don't need to be in every session.
- **Hooks** — for deterministic enforcement.

Where to put CLAUDE.md files

CLAUDE.md location precedence (more specific wins). Source: https://code.claude.com/docs/en/memory

Scope	Location	Use case
Managed policy	Linux/WSL: `/etc/claude-code/CLAUDE.md` macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md` Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`	Org-wide [[[

Each of the 6 journalism templates ships a sub-30-line "starter" variant alongside the fuller version. Adopt incrementally.

Auto memory is separate from CLAUDE.md

As of Claude Code v2.1.59+, there's a second memory mechanism alongside CLAUDE.md: **auto memory**, where Claude writes notes to itself in `~/.claude/projects/<project>/memory/` based on your corrections. The first 200 lines of that directory's `MEMORY.md` are loaded into every session. Source: https://code.claude.com/docs/en/memory

Practical implication for templates: don't ask the user to manually write down "things Claude learns over time" — that's auto memory's job now. CLAUDE.md is for facts you write up front; auto memory is for things Claude notices.

Voice guidelines

- Direct and terse
- Like notes you'd leave for yourself
- No marketing language
- No "Welcome to..." introductions
- No "This project is..." padding

Example: good vs bad

Bad (too verbose, obvious):

```
# CLAUDE.md

## Overview
Welcome to our newsroom's story tracking system! This is a
web application built with React and Node.js that helps
editors and reporters collaborate on stories.

## Getting started
First, make sure you have Node.js installed. Then:
npm install
npm start
```

Good (institutional knowledge only):

```
# CLAUDE.md

## Overview
Story tracker for metro desk. React + Supabase.

## Gotchas
- Story slugs must be unique across ALL desks, not just metro
- "Hold" status doesn't stop the autopublish cron — use "Kill"
- Reporter dropdown caches for 1 hour; new hires won't appear

## Commands
npm run sync-ap  # Pull latest from AP, runs automatically at :15

## Credentials
Supabase key in 1Password "Metro Desk" vault, not .env
```

Journalism-specific templates

Templates are in the `templates/` directory:

[[[[

[[[

project-retrospective (Skill Module)

`project-templates-toolkit\skills\project-retrospective\SKILL.md`

name: project-retrospective

description: Generate LESSONS.md retrospective files that capture institutional knowledge, especially failures. Use when closing out journalism projects, investigations, events, or publications. Includes templates for research projects, event post-mortems, editorial tools, and publications.

tags:

Project retrospective writer

Create LESSONS.md files that capture institutional knowledge, especially failures. Think like a journalist writing about your own project — be specific, be honest, name the actual mistakes.

Frame the retrospective: blame-aware, not pure-blameless

The 2024–2026 consensus in incident-analysis writing (PagerDuty, J. Paul Reed, Lorin Hochstein) is that pure blamelessness is neurobiologically unrealistic — humans default to blame. Healthier framing: acknowledge the blame bias exists and counter it deliberately. Source: https://postmortems.pagerduty.com/culture/blameless/

Cognitive biases to counter explicitly:
- **Fundamental attribution error** — blaming individuals for failures the system permitted
- **Confirmation bias** — looking for evidence that confirms a pre-formed narrative
- **Hindsight bias** — judging past decisions by knowledge that wasn't available at the time
- **Negativity bias** — over-weighting what went wrong vs. what worked

The voice rule isn't "no blame" — it's "name the system that permitted the human action, not just the action."

Drop the single-root-cause framing

Allspaw's canonical critique (still the dominant view in 2024–2026) calls Five Whys / single-root-cause analysis "seductively satisfying and compellingly simple — but false." It locks analysts into a linear causal chain that terminates in individual blame. Source: https://www.kitchensoap.com/2014/11/14/the-infinite-hows-or-the-dangers-of-the-five-whys/

Use "how" narratives instead, gathering on four [[[[prompts]]]]:
- **Cues** — what signals were available, who saw them when?
- **Interpretation** — how did people make sense of those signals?
- **Goals** — what were people trying to accomplish?
- **Taking action** — what did they do, and what did they expect to happen?

The retrospective's job is to surface [[contributing]] factors, not declare a root cause.

When to run a retrospective

Pre-define triggers before incidents happen, not after. Google SRE specifies criteria: user-visible downtime past a threshold, any data loss, on-call rollbacks, monitoring failures requiring manual discovery, stakeholder request. Source: https://sre.google/sre-book/postmortem-culture/

Translated to journalism:

Trigger	Examples
Live-[[[[event]]]] failure	Live-blog downtime during election night, breaking-news embed breakage, paywall regression mid-investigation
Editorial process failure	Missed correction window, source-management breach, [[fact-check]] process bypass
Tool failure affecting subscribers	CMS migration regression, paywall logic affecting >X% of readers
Stakeholder request	Editor / publisher / source explicitly asking "what happened?"

Pre-defined triggers prevent retrospective theater on routine work and ensure they happen on real failures.

Timing: aim for under one week

Google's good-vs-bad postmortem comparison cites a "four months later" example as a quality failure because contributors' memories had decayed. Source: https://sre.google/workbook/postmortem-culture/

Recommended cadence:

Project type	Target window
Live-[[[[event]]]] retro (election night, breaking news)	Same-day debrief, written within 48h
Tool / CMS incident	Within one week
Investigation or [[[[publication]]]] launch	Within one month of project close
Annual [[review]] of ongoing initiative	Once per year, scoped to discrete decision points

The empirical floor is "fresh in contributors' minds." Past one week and the narrative gets reconstructed rather than remembered.

For [[[[event]]]]-type retros, use AAR structure

The US Army's After Action [[Review]] (FM 7-0 Appendix K) was designed for facilitated post-[[[[event]]]] debriefs and fits live-news realities better than a project-close [[[[[

[[[

structure

```
# LESSONS.md

## Project
- **Name:** [Project name]
- **Dates:** [Start - End]
- **Status:** [Completed / Abandoned / Ongoing]
- **Author:** [Your name]
- **Retrospective written:** [Date — should be within timing window above]

## Summary
[One paragraph: what it did, what impact it had, why it matters]

## What worked

### Technical wins
- [Specific decision and WHY it worked]
- [Tool/pattern that saved time]

### Process wins
- [Methodology that helped]
- [Communication pattern that worked]

## What didn't work

For each item below, separate the system layer from the people layer.
"What did the system permit or require?" comes before "What did the
people inside that system do?"

### Critical failures
- **System:** [What the architecture / process / tooling permitted]
- **People:** [What humans did inside that constraint]
- **Cost:** [Quantified impact: hours, subscribers affected, money]

### Technical debt
- [Shortcut that hurt later — and why it was the rational choice at the time]
- [Complexity that wasn't needed]

### External factors
- [Things outside your control that impacted the project]

## The real problem

What we thought: [Initial assumption]
What was actually needed: [Reality]
The gap cost us: [Time / effort / money wasted]

## Action items

Each item must have an owner, a deadline, a tracker URL, and a priority.
"Postmortems without subsequent action are indistinguishable from no postmortem."
— Google SRE

| # | Action | Owner | Deadline | Tracker | Priority |
|---|--------|-------|----------|---------|----------|
| 1 | [Specific change] | [Name] | [Date] | [Issue URL] | P0 / P1 / P2 |
| 2 | [Specific change] | [Name] | [Date] | [Issue URL] | P0 / P1 / P2 |

Failure modes to avoid:
- Ambiguous wording ("make automation better")
- Equal priority across all items (defeats the purpose of priority)
- No tracker bug or issue URL (action items vanish)
- Fixes targeting human behavior instead of system redesign ("be more careful")

Add a 30/60/90-day check-in note: who reviews progress on these, and when.

## Reusable artifacts

| Component | Why it's valuable |
|-----------|-------------------|
| [Name] | [Specific reuse potential] |
| [Name] | [Why someone else should use this] |

## Questions for next time
- [Unanswered questions worth investigating]
- [Things you'd research before starting]
```

Voice guidelines

- Honest, specific, slightly self-deprecating
- Like explaining to a friend why the project took twice as long
- Name what the system permitted, then what the people did inside it
- Specific mistakes, not vague "challenges"
- Active voice; avoid "mistakes were made" passives that hide who decided what

Sourcing-decision retrospectives (journalism-native)

The canonical SRE / AAR / Allspaw frameworks don't cover the editorial-judgment retrospective an investigation needs. The `[[[[research-project]]]].md` [[[[[

> - *The schema validation step was disabled in CI for the data-import script (system) and the freelance reporter who imported new entries didn't know it had been disabled (people). Cost: 3 hours debugging a typo that caused silent failures across 12 published stories.*
> - *We built a custom date picker when the browser native input would have worked. Tracker: ENG-1247. Owner: K. Park. 2 days wasted.*

Journalism-specific templates

Templates are in the `templates/` directory:

[[[[research-project]]]].md
[[[[event]]]].md
[[[[publication]]]].md
[[[[editorial-tool]]]].md

selection

```
What kind of project?
 Investigation/analysis → research-project.md
 Conference/workshop/election night → event.md (use AAR)
 Newsletter/podcast → publication.md
 Newsroom tool → editorial-tool.md
```

Last currency sweep

2026-05-09. Sources verified: kitchensoap.com (Allspaw "Infinite Hows"), postmortems.pagerduty.com, sre.google/sre-book and /workbook, first.army.mil FM 7-0 Appendix K, adaptivecapacitylabs.com.

The best retrospectives are written by people who got burned and want to save others from the same fate.

python-pipeline (Skill Module)

dev-toolkit\skills\python-pipeline\SKILL.md

name: python-pipeline

description: Python data processing pipelines with modular architecture. Use when building content processing workflows, implementing dispatcher patterns, integrating Google Sheets/Drive APIs, or creating batch processing systems. Covers patterns from rosen-scraper, image-analyzer, and social-scraper projects.

tags:

Python data pipeline development

Patterns for building production-quality data processing pipelines with Python.

Targeted at Python 3.11+ for `asyncio.TaskGroup` and exception groups; Python 3.12+ for the lighter `type X = ...` syntax. Pin a 3.13+ runtime if you want the JIT or experimental free-threading; the patterns here don't depend on either.

Choosing a DataFrame engine: pandas vs polars vs DuckDB

For a long time pandas was the default for any tabular work in Python. As of 2026 the default has shifted: **polars** is the right pick for multi-GB pipelines on a single machine, **DuckDB** is the right pick when SQL or larger-than-RAM scans are involved, and **pandas** stays useful for small data and the ML/notebook ecosystem (scikit-learn, statsmodels, plotnine all speak it natively).

Tool	When
pandas	< ~1 GB data, ML interop, single-threaded familiarity
polars	1 GB - tens of GB on one box, performance-critical pipelines
DuckDB	SQL [[[[

```
import polars as pl
import duckdb

# Polars: read a directory of CSVs, filter, group
df = (
    pl.scan_csv('data/articles_*.csv')
      .filter(pl.col('published_at') >= '2026-01-01')
      .group_by('source')
      .agg(pl.len().alias('count'), pl.col('word_count').mean())
      .collect()
)

# DuckDB: same shape with SQL, no intermediate copy
con = duckdb.connect()
df = con.execute("""
    SELECT source, COUNT(*) AS count, AVG(word_count) AS avg_wc
    FROM 'data/articles_*.csv'
    WHERE published_at >= '2026-01-01'
    GROUP BY source
""").pl()  # returns a Polars DataFrame; use .df() for pandas

# Hand off to pandas only at the boundary that needs it (e.g. scikit-learn)
import pandas as pd
pdf = df.to_pandas()
```

If your pipeline already uses pandas everywhere, don't pre-emptively rewrite. Migrate the bottleneck stages first — typically the CSV-load + filter step.

Architecture patterns

Modular processor architecture

```
src/
 workflow.py              # Main orchestrator
 dispatcher.py            # Content-type router
 processors/
    __init__.py
    base.py             # Abstract base class
    article_processor.py
    video_processor.py
    audio_processor.py
 services/
    sheets_service.py   # Google Sheets integration
    drive_service.py    # Google Drive integration
    ai_service.py       # Gemini API wrapper
 utils/
    logger.py
    rate_limiter.py
 config.py               # Environment configuration
```

Dispatcher pattern

```
from typing import Protocol
from urllib.parse import urlparse

class Processor(Protocol):
    def can_process(self, url: str) -> bool: ...
    def process(self, url: str, metadata: dict) -> dict: ...

class Dispatcher:
    def __init__(self):
        self.processors: list[Processor] = [
            ArticleProcessor(),
            VideoProcessor(),
            AudioProcessor(),
            SocialProcessor(),
        ]

    def dispatch(self, url: str, metadata: dict) -> dict:
        for processor in self.processors:
            if processor.can_process(url):
                return processor.process(url, metadata)
        raise ValueError(f"No processor found for URL: {url}")

# Pattern-based routing
class ArticleProcessor:
    DOMAINS = ['nytimes.com', 'washingtonpost.com', 'medium.com']

    def can_process(self, url: str) -> bool:
        domain = urlparse(url).netloc.replace('www.', '')
        return any(d in domain for d in self.DOMAINS)
```

CSV-based pipeline workflow

```
import csv
from pathlib import Path
from dataclasses import dataclass, asdict
from typing import Iterator

@dataclass
class Record:
    id: str
    url: str
    title: str | None = None
    content: str | None = None
    status: str = 'pending'

def read_input(path: Path) -> Iterator[Record]:
    with open(path, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield Record(**{k: v for k, v in row.items() if k in Record.__annotations__})

def write_output(records: list[Record], path: Path):
    with open(path, 'w', encoding='utf-8', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=list(Record.__annotations__.keys()))
        writer.writeheader()
        writer.writerows(asdict(r) for r in records)

def process_batch(input_path: Path, output_path: Path):
    dispatcher = Dispatcher()
    results = []

    for record in read_input(input_path):
        try:
            processed = dispatcher.dispatch(record.url, asdict(record))
            record.status = 'completed'
            record.title = processed.get('title')
            record.content = processed.get('content')
        except Exception as e:
            record.status = f'failed: {e}'
        results.append(record)

    write_output(results, output_path)
```

Google Sheets [[integration]]

```
import gspread
from google.oauth2.service_account import Credentials

SCOPES = [
    'https://www.googleapis.com/auth/spreadsheets',
    'https://www.googleapis.com/auth/drive'
]

class SheetsService:
    def __init__(self, credentials_path: str):
        creds = Credentials.from_service_account_file(credentials_path, scopes=SCOPES)
        self.client = gspread.authorize(creds)

    def get_worksheet(self, spreadsheet_id: str, sheet_name: str):
        spreadsheet = self.client.open_by_key(spreadsheet_id)
        return spreadsheet.worksheet(sheet_name)

    def read_all(self, worksheet) -> list[dict]:
        return worksheet.get_all_records()

    def append_row(self, worksheet, row: list):
        worksheet.append_row(row, value_input_option='USER_ENTERED')

    def batch_update(self, worksheet, updates: list[dict]):
        """Update multiple cells efficiently."""
        # Format: [{'range': 'A1', 'values': [[value]]}]
        worksheet.batch_update(updates, value_input_option='USER_ENTERED')

    def find_row_by_id(self, worksheet, id_value: str, id_column: int = 1) -> int | None:
        """Find row number by ID value."""
        try:
            cell = worksheet.find(id_value, in_column=id_column)
            return cell.row
        except gspread.CellNotFound:
            return None
```

Rate limiting

```
import time
from functools import wraps
from ratelimit import limits, sleep_and_retry

# Simple rate limiter
@sleep_and_retry
@limits(calls=10, period=60)  # 10 calls per minute
def rate_limited_api_call(url: str):
    return requests.get(url)

# Custom rate limiter with backoff
class RateLimiter:
    def __init__(self, calls_per_minute: int = 10):
        self.delay = 60 / calls_per_minute
        self.last_call = 0

    def wait(self):
        elapsed = time.time() - self.last_call
        if elapsed < self.delay:
            time.sleep(self.delay - elapsed)
        self.last_call = time.time()

# Usage
limiter = RateLimiter(calls_per_minute=10)

def fetch_with_rate_limit(url: str):
    limiter.wait()
    return requests.get(url)
```

Concurrent fetching with asyncio.TaskGroup (3.11+)

For I/O-bound stages (HTTP fetches, API calls), `asyncio.TaskGroup` plus `httpx.AsyncClient` runs many requests in parallel without the boilerplate of `asyncio.gather`. TaskGroup's structured-concurrency model means an exception in one task cancels the rest and surfaces as an `ExceptionGroup` — easier to reason about than `gather(return_exceptions=True)`.

```
import asyncio
import httpx

async def fetch_one(client: httpx.AsyncClient, url: str) -> tuple[str, str | Exception]:
    try:
        response = await client.get(url, timeout=30)
        response.raise_for_status()
        return (url, response.text)
    except Exception as e:
        return (url, e)

async def fetch_many(urls: list[str], concurrency: int = 10) -> dict[str, str | Exception]:
    results: dict[str, str | Exception] = {}
    sem = asyncio.Semaphore(concurrency)

    async def _bounded(client: httpx.AsyncClient, url: str):
        async with sem:
            url, body = await fetch_one(client, url)
            results[url] = body

    async with httpx.AsyncClient(http2=True, timeout=30) as client:
        async with asyncio.TaskGroup() as tg:
            for url in urls:
                tg.create_task(_bounded(client, url))

    return results

# Usage
urls = ['https://example.com/a', 'https://example.com/b', ...]
data = asyncio.run(fetch_many(urls, concurrency=20))
```

Pair with `aiolimiter` if you need a true requests-per-second cap (semaphore alone bounds concurrency, not rate). For exponential-backoff retries, wrap `fetch_one` with `tenacity.AsyncRetrying`.

Progress tracking with resume capability

```
import json
from pathlib import Path

class ProgressTracker:
    def __init__(self, progress_file: Path):
        self.progress_file = progress_file
        self.state = self._load()

    def _load(self) -> dict:
        if self.progress_file.exists():
            return json.loads(self.progress_file.read_text())
        return {'processed_ids': [], 'last_row': 0, 'errors': []}

    def save(self):
        self.progress_file.write_text(json.dumps(self.state, indent=2))

    def mark_processed(self, record_id: str):
        self.state['processed_ids'].append(record_id)
        self.save()

    def is_processed(self, record_id: str) -> bool:
        return record_id in self.state['processed_ids']

    def log_error(self, record_id: str, error: str):
        self.state['errors'].append({'id': record_id, 'error': error})
        self.save()

# Usage in workflow
tracker = ProgressTracker(Path('progress.json'))

for record in records:
    if tracker.is_processed(record.id):
        continue  # Skip already processed

    try:
        process(record)
        tracker.mark_processed(record.id)
    except Exception as e:
        tracker.log_error(record.id, str(e))
```

Gemini AI [[integration]]

The `google-generativeai` package was deprecated August 31, 2025 and the unified `google-genai` SDK replaced it. New code should target `google-genai`:

```
pip install google-genai
```

```
import os
import json
from google import genai
from google.genai import types

# Client carries config (API key, project, location). Reuse across calls.
client = genai.Client(api_key=os.environ['GEMINI_API_KEY'])

# Pick a current model. Names drift; check ai.google.dev/gemini-api/docs/models
# for the active list. gemini-2.5-flash is a reasonable cost-efficient default.
DEFAULT_MODEL = 'gemini-2.5-flash'

class AIService:
    def __init__(self, model: str = DEFAULT_MODEL):
        self.model = model

    def categorize(self, text: str, taxonomy: dict) -> dict:
        prompt = f"""Analyze this content and categorize it.

Content:
{text[:10000]}

Taxonomy:
{json.dumps(taxonomy, indent=2)}

Respond with JSON containing:
- category: one of the taxonomy categories
- tags: list of relevant tags
- summary: 2-3 sentence summary
"""
        response = client.models.generate_content(
            model=self.model,
            contents=prompt,
            config=types.GenerateContentConfig(response_mime_type='application/json'),
        )
        return json.loads(response.text)

    def extract_entities(self, text: str) -> list[dict]:
        prompt = f"""Extract named entities from this text.

Text:
{text[:10000]}

For each entity, provide:
- name: entity name
- type: Person, Organization, Location, Event, Work, or Concept
- prominence: 1-10 score based on importance in text

Respond with JSON array of entities.
"""
        response = client.models.generate_content(
            model=self.model,
            contents=prompt,
            config=types.GenerateContentConfig(response_mime_type='application/json'),
        )
        return json.loads(response.text)

# Batch processing with token-usage tracking (cost varies by model and time;
# look up live pricing rather than hardcoding a per-1k figure).
class BatchAIProcessor:
    def __init__(self, ai_service: AIService):
        self.ai = ai_service
        self.input_tokens = 0
        self.output_tokens = 0

    def process_batch(
        self, items: list[str], prompt_template: str
    ) -> list[dict]:
        """Render each item into prompt_template via .format(item=...).
        prompt_template must instruct the model to return JSON, since this
        method enforces response_mime_type='application/json'.
        """
        results = []
        for item in items:
            response = client.models.generate_content(
                model=self.ai.model,
                contents=prompt_template.format(item=item),
                config=types.GenerateContentConfig(
```

```
                    response_mime_type='application/json'
                ),
            )
            usage = response.usage_metadata
            self.input_tokens += usage.prompt_token_count or 0
            self.output_tokens += usage.candidates_token_count or 0
            results.append(json.loads(response.text))
        return results
```

`response.usage_metadata` carries the actual token counts, which is more accurate than length heuristics. Without `response_mime_type='application/json'`, Gemini returns prose (often wrapped in markdown fences) and `json.loads` fails — every JSON-returning call needs both the config flag and a JSON-shaped prompt. For multimodal calls, pass content as a list (text + parts), not a single string.

Image classification with Gemini Vision

```
from google import genai
from google.genai import types
from PIL import Image
from pathlib import Path

client = genai.Client(api_key=os.environ['GEMINI_API_KEY'])

def classify_image(image_path: Path, categories: list[str]) -> dict:
    image = Image.open(image_path)

    prompt = f"""Analyze this image and classify it.

Available categories: {', '.join(categories)}

Respond with JSON:
{{
  "category": "category name",
  "description": "brief description",
  "suggested_filename": "descriptive-filename-with-dashes",
  "tags": ["tag1", "tag2", "tag3"]
}}
"""
    response = client.models.generate_content(
        model='gemini-2.5-flash',
        contents=[prompt, image],
        config=types.GenerateContentConfig(response_mime_type='application/json'),
    )
    return json.loads(response.text)

# pathlib.Path.glob does NOT support brace expansion (`*.{jpg,png,webp}`);
# iterate the extensions explicitly.
IMAGE_EXTS = ('.jpg', '.jpeg', '.png', '.webp')

def organize_images(source_dir: Path, output_dir: Path):
    categories = ['Nature', 'People', 'Architecture', 'Art', 'Technology', 'Other']

    image_paths = (
        p for p in source_dir.iterdir()
        if p.is_file() and p.suffix.lower() in IMAGE_EXTS
    )

    for image_path in image_paths:
        try:
            result = classify_image(image_path, categories)
            category_dir = output_dir / result['category']
            category_dir.mkdir(parents=True, exist_ok=True)

            new_name = f"{result['suggested_filename']}{image_path.suffix.lower()}"
            image_path.rename(category_dir / new_name)
        except Exception as e:
            failures = output_dir / 'failures'
            failures.mkdir(parents=True, exist_ok=True)
            image_path.rename(failures / image_path.name)
```

Environment configuration

```
from pathlib import Path
from dotenv import load_dotenv
import os

load_dotenv()

class Config:
    # API Keys
    GEMINI_API_KEY = os.environ['GEMINI_API_KEY']
    GOOGLE_SHEET_ID = os.environ['GOOGLE_SHEET_ID']

    # Paths
    PROJECT_ROOT = Path(__file__).parent.parent
    DATA_DIR = PROJECT_ROOT / 'data'
    OUTPUT_DIR = PROJECT_ROOT / 'output'
    CREDENTIALS_PATH = PROJECT_ROOT / 'google_credentials.json'

    # Rate limits
    API_CALLS_PER_MINUTE = 10
    BATCH_SIZE = 50

    @classmethod
    def ensure_dirs(cls):
        cls.DATA_DIR.mkdir(exist_ok=True)
        cls.OUTPUT_DIR.mkdir(exist_ok=True)
```

Logging [[setup]]

```
import logging
from pathlib import Path
from datetime import datetime

def setup_logging(log_dir: Path, name: str = 'pipeline') -> logging.Logger:
    log_dir.mkdir(exist_ok=True)

    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    # Console handler (INFO+)
    console = logging.StreamHandler()
    console.setLevel(logging.INFO)
    console.setFormatter(logging.Formatter('%(levelname)s: %(message)s'))

    # File handler (DEBUG+)
    log_file = log_dir / f"{name}_{datetime.now():%Y%m%d_%H%M%S}.log"
    file_handler = logging.FileHandler(log_file)
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(logging.Formatter(
        '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
    ))

    logger.addHandler(console)
    logger.addHandler(file_handler)

    return logger
```

Common pitfalls

Google Sheets cell limits:

```
MAX_CELL_LENGTH = 50000

def truncate_for_sheets(text: str) -> str:
    if len(text) > MAX_CELL_LENGTH:
        return text[:MAX_CELL_LENGTH - 20] + '... [truncated]'
    return text
```

CSV encoding issues:

```
# Always specify encoding
with open(path, 'r', encoding='utf-8-sig') as f:  # BOM handling
    reader = csv.reader(f)
```

API quota management:

```
# Cache API responses
from functools import lru_cache

@lru_cache(maxsize=1000)
def cached_api_call(url: str) -> dict:
    return api_client.fetch(url)
```

receiving-code-review (Skill Module)

superjawn\skills\receiving-code-review\SKILL.md

name: receiving-code-review

description: Use when receiving code review feedback, before implementing suggestions, especially if feedback seems unclear or technically questionable - requires technical rigor and verification, not performative agreement or blind implementation

tags:

Code [[Review]] Reception

Overview

Code [[review]] requires technical evaluation, not emotional performance.

Core principle: Verify before implementing. Ask before assuming. Technical correctness over social comfort.

The Response Pattern

```
WHEN receiving code review feedback:

1. READ: Complete feedback without reacting
2. UNDERSTAND: Restate requirement in own words (or ask)
3. VERIFY: Check against codebase reality
4. EVALUATE: Technically sound for THIS codebase?
5. RESPOND: Technical acknowledgment or reasoned pushback
6. IMPLEMENT: One item at a time, test each
```

Forbidden Responses

NEVER:
- "You're absolutely right!" (explicit CLAUDE.md violation)
- "Great point!" / "Excellent feedback!" (performative)
- "Let me implement that now" (before verification)

INSTEAD:
- Restate the technical requirement
- Ask clarifying questions
- Push back with technical reasoning if wrong
- Just start working (actions > words)

Handling Unclear Feedback

```
IF any item is unclear:
  STOP - do not implement anything yet
  ASK for clarification on unclear items

WHY: Items may be related. Partial understanding = wrong implementation.
```

Example:

```
your human partner: "Fix 1-6"
You understand 1,2,3,6. Unclear on 4,5.

 WRONG: Implement 1,2,3,6 now, ask about 4,5 later
 RIGHT: "I understand items 1,2,3,6. Need clarification on 4 and 5 before proceeding."
```

Source-Specific Handling

From your human partner

- **Trusted** - implement after understanding
- **Still ask** if scope unclear
- **No performative agreement**
- **Skip to action** or technical acknowledgment

From External Reviewers

```
BEFORE implementing:
  1. Check: Technically correct for THIS codebase?
  2. Check: Breaks existing functionality?
  3. Check: Reason for current implementation?
  4. Check: Works on all platforms/versions?
  5. Check: Does reviewer understand full context?

IF suggestion seems wrong:
  Push back with technical reasoning

IF can't easily verify:
  Say so: "I can't verify this without [X]. Should I [investigate/ask/proceed]?"

IF conflicts with your human partner's prior decisions:
  Stop and discuss with your human partner first
```

your human partner's rule: "External feedback - be skeptical, but check carefully"

YAGNI Check for "Professional" Features

```
IF reviewer suggests "implementing properly":
  grep codebase for actual usage

  IF unused: "This endpoint isn't called. Remove it (YAGNI)?"
  IF used: Then implement properly
```

your human partner's rule: "You and reviewer both [[[[report]]]] to me. If we don't need this feature, don't add it."

Implementation Order

```
FOR multi-item feedback:
  1. Clarify anything unclear FIRST
  2. Then implement in this order:
     - Blocking issues (breaks, security)
     - Simple fixes (typos, imports)
     - Complex fixes (refactoring, logic)
  3. Test each fix individually
  4. Verify no regressions
```

When To Push Back

Push back when:
- Suggestion breaks existing functionality
- Reviewer lacks full context
- Violates YAGNI (unused feature)
- Technically incorrect for this stack
- Legacy/compatibility reasons exist
- Conflicts with your human partner's architectural decisions

How to push back:
- Use technical reasoning, not defensiveness
- Ask specific questions
- Reference working tests/code
- Involve your human partner if architectural

Signal if uncomfortable pushing back out loud: "Strange things are afoot at the Circle K"

Acknowledging Correct Feedback

When feedback IS correct:

```
"Fixed. [Brief description of what changed]"
"Good catch - [specific issue]. Fixed in [location]."
[Just fix it and show in the code]

"You're absolutely right!"
"Great point!"
"Thanks for catching that!"
"Thanks for [anything]"
ANY gratitude expression
```

Why no thanks: Actions speak. Just fix it. The code itself shows you heard the feedback.

If you catch yourself about to write "Thanks": DELETE IT. State the fix instead.

Gracefully Correcting Your Pushback

If you pushed back and were wrong:

```
"You were right - I checked [X] and it does [Y]. Implementing now."
"Verified this and you're correct. My initial understanding was wrong because [reason]. Fixing."

Long apology
Defending why you pushed back
Over-explaining
```

State the correction factually and move on.

Common Mistakes

Mistake	Fix
Performative agreement	State requirement or just act
Blind implementation	Verify against codebase first
Batch without testing	One at a time, test each
Assuming reviewer is right	Check if breaks things
Avoiding pushback	Technical correctness > comfort
Partial implementation	Clarify all items first
Can't verify, proceed anyway	State limitation, ask for direction

Real Examples

Performative Agreement (Bad):

```
Reviewer: "Remove legacy code"
 "You're absolutely right! Let me remove that..."
```

Technical Verification (Good):

```
Reviewer: "Remove legacy code"
 "Checking... build target is 10.15+, this API needs 13+. Need legacy for backward compat. Current impl has wrong bundle ID - fix it or drop pre-13
support?"
```

YAGNI (Good):

```
Reviewer: "Implement proper metrics tracking with database, date filters, CSV export"
 "Grepped codebase - nothing calls this endpoint. Remove it (YAGNI)? Or is there usage I'm missing?"
```

Unclear Item (Good):

```
your human partner: "Fix items 1-6"
You understand 1,2,3,6. Unclear on 4,5.
 "Understand 1,2,3,6. Need clarification on 4 and 5 before implementing."
```

GitHub Thread Replies

When replying to inline [[review]] comments on GitHub, reply in the comment thread (`gh api repos/{owner}/{repo}/pulls/{pr}/comments/{id}/replies`), not as a top-level PR comment.

The Bottom Line

External feedback = suggestions to evaluate, not orders to follow.

Verify. Question. Then implement.

No performative agreement. Technical rigor always.

recovering-deleted-files-with-photorec (Skill Module)

`recovering-deleted-files-with-photorec\SKILL.md`

name: recovering-deleted-files-with-photorec
description: Deleted file recovery agent using PhotoRec subprocess wrapper. Use when the user asks to recovering deleted files with photorec.
version: 1.0.0
tags:
source: recovering-deleted-files-with-photorec

Recovering Deleted Files with Photorec

Role: Security workflow
Audience: security practitioners
Primary tools: External Tools Required, File Categories

When to use

- The user wants to **recover** deleted files with photorec.
- Capability: Deleted file recovery agent using PhotoRec subprocess wrapper.
- Goal: complete the task.
- Trigger keywords: `recovering, deleted, files, photorec`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Check that PhotoRec is installed and available
2. Get forensic image information
3. Run PhotoRec for file recovery using command-line interface
4. Catalog all recovered files by type, size, and hash
5. Generate SHA-256 hashes for recovered files for evidence integrity
6. Sort recovered files into categorized directories
7. Run complete file recovery pipeline

Tools & resources

- External Tools Required
- File Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `verify_photorec` — Check that PhotoRec is installed and available.
- `get_image_info` — Get forensic image information.
- `run_photorec` — Run PhotoRec for file recovery using command-line interface.
- `catalog_recovered_files` — Catalog all recovered files by type, size, and hash.
- `hash_recovered_files` — Generate SHA-256 hashes for recovered files for evidence integrity.
- `sort_recovered_files` — Sort recovered files into categorized directories.
- `full_recovery_pipeline` — Run complete file recovery pipeline.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `recovering-deleted-files-with-photorec`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

recovering-from-ransomware-attack (Skill Module)

recovering-from-ransomware-attack\SKILL.md

name: recovering-from-ransomware-attack
description: Agent for ransomware attack recovery coordination. Manages recovery workflow: backup verification, system rebuild prioritization, credential reset tracking, and post-recovery validation checklists with timeline.... Use when the user asks to recovering from ransomware attack.
version: 1.0.0
tags:
source: recovering-from-ransomware-attack

Recovering from Ransomware Attack

Role: Security workflow
Audience: security practitioners
Primary tools: Recovery Priority Order, KRBTGT Reset Procedure, 3-2-1-1-0 Backup Strategy, Frameworks, AD Recovery

When to use

- The user wants to **recover** from ransomware attack.
- Capability: Agent for ransomware attack recovery coordination. Manages recovery workflow: backup verification, system rebuild prioritization, credential reset tracking, and post-recovery validation checklists with timeline documentation.
- Goal: complete the task.
- Trigger keywords: `recovering, ransomware, attack`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Clarify the objective for from ransomware attack
2. Gather inputs and required access
3. Execute the core workflow using skill scripts and references
4. Validate results and document findings
5. Recommend next actions

Key concepts

- NIST SP 800-61r3: Computer Security Incident Handling Guide (Recover phase)
- NIST IR 8374: Ransomware Risk Management - Recovery section
- CISA #StopRansomware Guide: Recovery checklist
- CIS Controls v8: Control 11 (Data Recovery)
- NIST CSF 2.0: Recover function (RC.RP, RC.CO)
- Microsoft: AD Forest Recovery Guide - https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/ad-forest-re
- T1053: Scheduled Task/Job (persistence to check)
- T1543: Create or Modify System Process (persistence to check)

Tools & resources

- Recovery Priority Order
- KRBTGT Reset Procedure
- 3-2-1-1-0 Backup Strategy
- Frameworks
- AD Recovery

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `recovering-from-ransomware-attack`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

remediating-s3-bucket-misconfiguration (Skill Module)

remediating-s3-bucket-misconfiguration\SKILL.md

name: remediating-s3-bucket-misconfiguration
description: S3 bucket misconfiguration remediation agent using boto3. Use when the user asks to remediating s3 bucket misconfiguration.
version: 1.0.0
tags:
source: remediating-s3-bucket-misconfiguration

Remediating S3 Bucket Misconfiguration

Role: Security workflow
Audience: security practitioners
Primary tools: Audit Functions, Remediation Functions, AWS API Calls, Environment Variables

When to use

- The user wants to **remediat** s3 bucket misconfiguration.
- Capability: S3 bucket misconfiguration remediation agent using boto3.
- Goal: complete the task.
- Trigger keywords: `remediating, bucket, misconfiguration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. List all S3 buckets in the account
2. Check if S3 Block Public Access is enabled
3. Check bucket policy for public access grants
4. Check bucket ACL for public grants
5. Check if default encryption is enabled
6. Check if versioning is enabled
7. Check if access logging is enabled
8. Enable S3 Block Public Access on a bucket

Tools & resources

- Audit Functions
- Remediation Functions
- AWS API Calls
- Environment Variables

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_s3_client`
- `list_all_buckets` — List all S3 buckets in the account.
- `check_public_access_block` — Check if S3 Block Public Access is enabled.
- `check_bucket_policy` — Check bucket policy for public access grants.
- `check_bucket_acl` — Check bucket ACL for public grants.
- `check_encryption` — Check if default encryption is enabled.
- `check_versioning` — Check if versioning is enabled.
- `check_logging` — Check if access logging is enabled.
- `enable_public_access_block` — Enable S3 Block Public Access on a bucket.
- `enable_encryption` — Enable default encryption on a bucket.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `remediating-s3-bucket-misconfiguration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

requesting-code-review (Skill Module)

superjawn\skills\requesting-code-review\SKILL.md

name: requesting-code-review
description: Use when completing tasks, implementing major features, or before merging to verify work meets requirements
tags:

Requesting Code [[Review]]

Dispatch pr-[[review]]-toolkit:[[code-reviewer]] subagent to catch issues before they cascade. The reviewer gets precisely crafted context for evaluation — never your session's history. This keeps the reviewer focused on the work product, not your thought process, and preserves your own context for continued work.

Core principle: [[Review]] early, [[review]] often.

When to Request [[Review]]

Mandatory:
- After each task in subagent-driven development
- After completing major feature
- Before merge to main

Optional but valuable:
- When stuck (fresh perspective)
- Before refactoring (baseline check)
- After fixing complex bug

How to Request

1. Get git SHAs:

```
BASE_SHA=$(git rev-parse HEAD~1)  # or origin/main
HEAD_SHA=$(git rev-parse HEAD)
```

2. Dispatch [[code-reviewer]] subagent:

Use Task tool with pr-[[review]]-toolkit:[[code-reviewer]] type, fill [[[[

Example

```
[Just completed Task 2: Add verification function]

You: Let me request code review before proceeding.

BASE_SHA=$(git log --oneline | grep "Task 1" | head -1 | awk '{print $1}')
HEAD_SHA=$(git rev-parse HEAD)

[Dispatch pr-review-toolkit:code-reviewer subagent]
  WHAT_WAS_IMPLEMENTED: Verification and repair functions for conversation index
  PLAN_OR_REQUIREMENTS: Task 2 from docs/superpowers/plans/deployment-plan.md
  BASE_SHA: a7981ec
  HEAD_SHA: 3df7661
  DESCRIPTION: Added verifyIndex() and repairIndex() with 4 issue types

[Subagent returns]:
  Strengths: Clean architecture, real tests
  Issues:
    Important: Missing progress indicators
    Minor: Magic number (100) for reporting interval
  Assessment: Ready to proceed

You: [Fix progress indicators]
[Continue to Task 3]
```

[[Integration]] with

[[[

reverse-engineering-android-malware-with-jadx (Skill Module)

`reverse-engineering-android-malware-with-jadx\SKILL.md`

name: reverse-engineering-android-malware-with-jadx
description: Android malware reverse engineering agent using jadx and androguard subprocess wrappers. Use when the user asks to reverse engineer android malware with jadx.
version: 1.0.0
tags:
source: reverse-engineering-android-malware-with-jadx

Reverse Engineering Android Malware with Jadx

Role: Reverse engineering
Audience: malware analysts and reverse engineers
Primary tools: External Tools Required, Dangerous Permissions Checked

When to use

- The user wants to **reverse engineer** android malware with jadx.
- Capability: Android malware reverse engineering agent using jadx and androguard subprocess wrappers.
- Goal: disassemble and reason about binaries, malware, or protocols.
- Trigger keywords: `reverse, engineering, android, malware, jadx`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute hashes for APK identification
2. Extract and parse AndroidManifest.xml using apktool
3. Classify permissions by risk level
4. Decompile APK to Java source using JADX
5. Search decompiled source for suspicious patterns
6. Full APK analysis pipeline

Tools & resources

- External Tools Required
- Dangerous Permissions Checked

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_apk_hashes` — Compute hashes for APK identification.
- `extract_manifest` — Extract and parse AndroidManifest.xml using apktool.
- `analyze_permissions` — Classify permissions by risk level.
- `decompile_with_jadx` — Decompile APK to Java source using JADX.
- `search_source_code` — Search decompiled source for suspicious patterns.
- `analyze_apk` — Full APK analysis pipeline.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `reverse-engineering-android-malware-with-jadx`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

reverse-engineering-dotnet-malware-with-dnspy (Skill Module)

`reverse-engineering-dotnet-malware-with-dnspy\SKILL.md`

name: reverse-engineering-dotnet-malware-with-dnspy
description: .NET malware reverse engineering agent using subprocess wrappers for dnSpy/de4dot. Use when the user asks to reverse engineer dotnet malware with dnspy.
version: 1.0.0
tags:
source: reverse-engineering-dotnet-malware-with-dnspy

Reverse Engineering Dotnet Malware with Dnspy

Role: Reverse engineering
Audience: malware analysts and reverse engineers
Primary tools: Obfuscators Detected, Suspicious String Keywords

When to use

- The user wants to **reverse engineer** dotnet malware with dnspy.
- Capability: .NET malware reverse engineering agent using subprocess wrappers for dnSpy/de4dot.
- Goal: disassemble and reason about binaries, malware, or protocols.
- Trigger keywords: `reverse, engineering, dotnet, malware, dnspy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute hashes for sample identification
2. Check if file is a .NET assembly by looking for CLI header
3. Detect .NET obfuscator using Detect It Easy
4. Run de4dot to deobfuscate .NET assembly
5. Extract strings and classify for IOCs
6. Extract .NET metadata using monodis or ilspy CLI if available
7. Full .NET malware analysis pipeline

Tools & resources

- Obfuscators Detected
- Suspicious String Keywords

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Compute hashes for sample identification.
- `detect_dotnet_assembly` — Check if file is a .NET assembly by looking for CLI header.
- `detect_obfuscator` — Detect .NET obfuscator using Detect It Easy.
- `deobfuscate_with_de4dot` — Run de4dot to deobfuscate .NET assembly.
- `extract_strings` — Extract strings and classify for IOCs.
- `analyze_dotnet_metadata` — Extract .NET metadata using monodis or ilspy CLI if available.
- `analyze_dotnet_malware` — Full .NET malware analysis pipeline.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `reverse-engineering-dotnet-malware-with-dnspy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

reverse-engineering-ios-app-with-frida (Skill Module)

`reverse-engineering-ios-app-with-frida\SKILL.md`

name: reverse-engineering-ios-app-with-frida
description: Agent for iOS app reverse engineering with Frida. Uses frida-tools to attach to iOS processes, hook Objective-C methods, bypass SSL pinning, dump keychain entries, and trace API calls for security assessment. Use when the user asks to reverse engineer ios app with frida.
version: 1.0.0
tags:
source: reverse-engineering-ios-app-with-frida

Reverse Engineering IOS App with Frida

Role: Reverse engineering
Audience: malware analysts and reverse engineers
Primary tools: Frida CLI Tools, Frida JavaScript API, Common iOS Security Hooks, CWE Mappings

When to use

- The user wants to **reverse engineer** ios app with frida.
- Capability: Agent for iOS app reverse engineering with Frida. Uses frida-tools to attach to iOS processes, hook Objective-C methods, bypass SSL pinning, dump keychain entries, and trace API calls for security assessment.
- Goal: disassemble and reason about binaries, malware, or protocols.
- Trigger keywords: `reverse, engineering, frida`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Triage the sample safely (hash, packer, environment)
2. Static analysis pass for ios app with frida
3. Dynamic / behavioral analysis as needed
4. Recover capabilities, IOCs, and C2 patterns
5. Document family attribution and detection ideas

Tools & resources

- Frida CLI Tools
- Frida JavaScript API
- Common iOS Security Hooks
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `reverse-engineering-ios-app-with-frida`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

reverse-engineering-malware-with-ghidra (Skill Module)

`reverse-engineering-malware-with-ghidra\SKILL.md`

name: reverse-engineering-malware-with-ghidra
description: Malware reverse engineering agent using Ghidra headless analyzer and r2pipe. Use when the user asks to reverse engineer malware with ghidra.
version: 1.0.0
tags:
source: reverse-engineering-malware-with-ghidra

Reverse Engineering Malware with Ghidra

Role: Reverse engineering
Audience: malware analysts and reverse engineers
Primary tools: Ghidra, External Tools, Suspicious Import Categories

When to use

- The user wants to **reverse engineer** malware with ghidra.
- Capability: Malware reverse engineering agent using Ghidra headless analyzer and r2pipe.
- Goal: disassemble and reason about binaries, malware, or protocols.
- Trigger keywords: `reverse, engineering, malware, ghidra`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute file hashes for identification
2. Run Ghidra in headless mode for automated analysis
3. Export function list using Ghidra headless with a script
4. Analyze binary with radare2 via r2pipe for quick triage
5. Search binary for known cryptographic constants
6. Full malware analysis pipeline

Tools & resources

- Ghidra
- External Tools
- Suspicious Import Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_hashes` — Compute file hashes for identification.
- `run_ghidra_headless` — Run Ghidra in headless mode for automated analysis.
- `export_functions_ghidra` — Export function list using Ghidra headless with a script.
- `analyze_with_radare2` — Analyze binary with radare2 via r2pipe for quick triage.
- `extract_crypto_constants` — Search binary for known cryptographic constants.
- `analyze_malware` — Full malware analysis pipeline.
- `print_report`

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `reverse-engineering-malware-with-ghidra`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

reverse-engineering-ransomware-encryption-routine (Skill Module)

`reverse-engineering-ransomware-encryption-routine\SKILL.md`

name: reverse-engineering-ransomware-encryption-routine
description: Agent for reverse engineering ransomware encryption routines. Identifies encryption algorithms, extracts key material from memory dumps or binary analysis, detects IV/nonce patterns, and documents the cryptographic.... Use when the user asks to reverse engineer ransomware encryption routine.
version: 1.0.0
tags:
source: reverse-engineering-ransomware-encryption-routine

Reverse Engineering Ransomware Encryption Routine

Role: Reverse engineering
Audience: malware analysts and reverse engineers
Primary tools: Cryptographic Algorithm Constants, Encryption Analysis Techniques, Common Ransomware Crypto

When to use

- The user wants to **reverse engineer** ransomware encryption routine.
- Capability: Agent for reverse engineering ransomware encryption routines. Identifies encryption algorithms, extracts key material from memory dumps or binary analysis, detects IV/nonce patterns, and documents the cryptographic implementation for decryptor development.
- Goal: disassemble and reason about binaries, malware, or protocols.
- Trigger keywords: `reverse, engineering, ransomware, encryption, routine`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Triage the sample safely (hash, packer, environment)
2. Static analysis pass for ransomware encryption routine
3. Dynamic / behavioral analysis as needed
4. Recover capabilities, IOCs, and C2 patterns
5. Document family attribution and detection ideas

Key concepts

- T1486: Data Encrypted for Impact
- T1490: Inhibit System Recovery
- T1083: File and Directory Discovery
- T1082: System Information Discovery

Tools & resources

- Cryptographic Algorithm Constants
- Encryption Analysis Techniques
- Common Ransomware Crypto

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.

4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `reverse-engineering-ransomware-encryption-routine`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

reverse-engineering-rust-malware (Skill Module)

`reverse-engineering-rust-malware\SKILL.md`

name: reverse-engineering-rust-malware
description: Agent for reverse engineering Rust-compiled malware. Identifies Rust binaries, extracts crate dependencies, locates crypto/network/persistence patterns, and maps suspicious capabilities for malware analysis reporting. Use when the user asks to reverse engineer rust malware.
version: 1.0.0
tags:
source: reverse-engineering-rust-malware

Reverse Engineering Rust Malware

Role: Reverse engineering
Audience: malware analysts and reverse engineers
Primary tools: Rust Binary Indicators, Suspicious Crate Capabilities, Applicable Standards

When to use

- The user wants to **reverse engineer** rust malware.
- Capability: Agent for reverse engineering Rust-compiled malware. Identifies Rust binaries, extracts crate dependencies, locates crypto/network/persistence patterns, and maps suspicious capabilities for malware analysis reporting.
- Goal: disassemble and reason about binaries, malware, or protocols.
- Trigger keywords: `reverse, engineering, rust, malware`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Triage the sample safely (hash, packer, environment)
2. Static analysis pass for rust malware
3. Dynamic / behavioral analysis as needed
4. Recover capabilities, IOCs, and C2 patterns
5. Document family attribution and detection ideas

Key concepts

- MITRE ATT&CK Framework
- NIST SP 800-83 Guide to Malware Incident Prevention
- NIST SP 800-86 Guide to Integrating Forensic Techniques

Tools & resources

- Rust Binary Indicators
- Suspicious Crate Capabilities
- Applicable Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `reverse-engineering-rust-malware`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

scanning-container-images-with-grype (Skill Module)

scanning-container-images-with-grype\SKILL.md

name: scanning-container-images-with-grype
description: Agent for scanning container images with Anchore Grype. Runs Grype CLI against container images, parses JSON results, applies severity thresholds, and generates vulnerability reports with remediation guidance. Use when the user asks to scan container images with grype.
version: 1.0.0
tags:
source: scanning-container-images-with-grype

Scanning Container Images with Grype

Role: Security scanning
Audience: security engineers and vulnerability management teams
Primary tools: Input Sources, Severity Levels, Industry Standards, CVSS Scoring Reference, Compliance Mappings

When to use

- The user wants to **scan** container images with grype.
- Capability: Agent for scanning container images with Anchore Grype. Runs Grype CLI against container images, parses JSON results, applies severity thresholds, and generates vulnerability reports with remediation guidance.
- Goal: discover exposures, misconfigurations, and vulnerable assets.
- Trigger keywords: `scanning, container, images, grype`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Review critical/high findings from scan output
2. Check if fix versions are available (`fix.versions` in JSON output)
3. Update base image to latest patched version
4. Update application dependencies
5. Rebuild and rescan to verify remediation
6. Add accepted risks to `.grype.yaml` ignore list with documented justification

Key concepts

- Section 4.1: Image vulnerabilities - Recommends scanning images for known vulnerabilities before deployment
- Section 4.2: Image configuration defects - Covers misconfigurations in container images
- Recommends integrating vulnerability scanning into CI/CD pipelines
- Rule 4.1: Ensure a user for the container has been created
- Rule 4.6: Add HEALTHCHECK instruction to the container image
- Rule 4.9: Ensure that COPY is used instead of ADD
- Rule 4.10: Ensure secrets are not stored in Dockerfiles
- RA-5: Vulnerability Monitoring and Scanning

Tools & resources

- Input Sources
- Severity Levels
- Industry Standards
- CVSS Scoring Reference
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `scanning-container-images-with-grype`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

scanning-containers-with-trivy-in-cicd (Skill Module)

`scanning-containers-with-trivy-in-cicd\SKILL.md`

name: scanning-containers-with-trivy-in-cicd
description: Agent for scanning containers with Trivy in CI/CD pipelines. Runs Trivy vulnerability and misconfiguration scans against container images and Dockerfiles, enforces severity-based quality gates, and generates.... Use when the user asks to scan containers with trivy in cicd.
version: 1.0.0
tags:
source: scanning-containers-with-trivy-in-cicd

Scanning Containers with Trivy in Cicd

Role: Security scanning
Audience: security engineers and vulnerability management teams
Primary tools: Quality Gate Severities, NIST SP 800-190, CIS Docker Benchmark v1.6.0, SLSA Framework Alignment

When to use

- The user wants to **scan** containers with trivy in cicd.
- Capability: Agent for scanning containers with Trivy in CI/CD pipelines. Runs Trivy vulnerability and misconfiguration scans against container images and Dockerfiles, enforces severity-based quality gates, and generates CI/CD-compatible reports.
- Goal: discover exposures, misconfigurations, and vulnerable assets.
- Trigger keywords: `scanning, containers, trivy, cicd`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define asset scope and scan policy
2. Run scans for containers with trivy in cicd
3. Triage and de-duplicate findings
4. Prioritize by risk and asset criticality
5. Track remediation and retest

Key concepts

- Scan all container images for known vulnerabilities before deployment
- Establish organizational policies for maximum acceptable vulnerability severity
- Monitor images in registries for newly discovered vulnerabilities
- Maintain an up-to-date vulnerability database for scanning tools
- Verify images follow CIS Docker Benchmark configuration guidelines
- Ensure images run as non-root users unless operationally required
- Remove unnecessary packages, shells, and utilities from production images
- Scan images in the CI/CD pipeline before pushing to registries

Tools & resources

- Quality Gate Severities
- NIST SP 800-190
- CIS Docker Benchmark v1.6.0
- SLSA Framework Alignment

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `scanning-containers-with-trivy-in-cicd`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

scanning-docker-images-with-trivy (Skill Module)

scanning-docker-images-with-trivy\SKILL.md

name: scanning-docker-images-with-trivy
description: Agent for scanning Docker images with Trivy. Performs comprehensive vulnerability scanning of Docker images including OS packages, language dependencies, misconfigurations, secrets, and license compliance using Trivy CLI. Use when the user asks to scan docker images with trivy.
version: 1.0.0
tags:
source: scanning-docker-images-with-trivy

Scanning Docker Images with Trivy

Role: Security scanning
Audience: security engineers and vulnerability management teams
Primary tools: Docker, Trivy Scanner Types, NIST SP 800-190, Executive Order 14028, Trivy Vulnerability Scoring

When to use

- The user wants to **scan** docker images with trivy.
- Capability: Agent for scanning Docker images with Trivy. Performs comprehensive vulnerability scanning of Docker images including OS packages, language dependencies, misconfigurations, secrets, and license compliance using Trivy CLI.
- Goal: discover exposures, misconfigurations, and vulnerable assets.
- Trigger keywords: `scanning, docker, images, trivy`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define asset scope and scan policy
2. Run scans for docker images with trivy
3. Triage and de-duplicate findings
4. Prioritize by risk and asset criticality
5. Track remediation and retest

Key concepts

- Image Vulnerability Management: Organizations should maintain a pipeline for scanning and remediating container image vulnerabilities
- Image Provenance: Use content trust and signing to verify image source and integrity
- SBOM Generation: Produce Software Bill of Materials for all container images
- PW.4.1: Verify third-party software components have no known vulnerabilities
- PW.4.4: Verify software components are obtained from trusted sources
- RV.1.1: Gather information from vulnerability notifications
- RV.1.2: Review, analyze, and/or test code to identify vulnerabilities
- Software producers must provide SBOMs for federal software

Tools & resources

- Docker
- Trivy Scanner Types
- NIST SP 800-190
- Executive Order 14028
- Trivy Vulnerability Scoring

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `scanning-docker-images-with-trivy`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

scanning-infrastructure-with-nessus (Skill Module)

scanning-infrastructure-with-nessus\SKILL.md

name: scanning-infrastructure-with-nessus
description: Agent for scanning infrastructure with Tenable Nessus. Interacts with the Nessus REST API to create scan policies, launch scans, monitor progress, retrieve results, and generate vulnerability reports with.... Use when the user asks to scan infrastructure with nessus.
version: 1.0.0
tags:
source: scanning-infrastructure-with-nessus

Scanning Infrastructure with Nessus

Role: Security scanning
Audience: security engineers and vulnerability management teams
Primary tools: Nessus, Scan Types, Severity Levels, Industry Standards, Tenable Documentation, Compliance Audit Files

When to use

- The user wants to **scan** infrastructure with nessus.
- Capability: Agent for scanning infrastructure with Tenable Nessus. Interacts with the Nessus REST API to create scan policies, launch scans, monitor progress, retrieve results, and generate vulnerability reports with severity-based prioritization.
- Goal: discover exposures, misconfigurations, and vulnerable assets.
- Trigger keywords: `scanning, infrastructure, nessus`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. **Weekly**: Scan DMZ and internet-facing assets
2. **Bi-weekly**: Scan internal production servers
3. **Monthly**: Full infrastructure scan including workstations
4. **Quarterly**: Comprehensive scan with compliance audits
5. **Ad-hoc**: Post-patch verification scans
6. **Central Management**: Use Tenable.sc to manage multiple scanners
7. **Zone Assignment**: Assign scanners to specific network zones
8. **Scan Windowing**: Stagger scans to prevent network saturation
9. **Result Aggregation**: Consolidate results in central repository
10. **Deduplication**: Merge findings from overlapping scan ranges

Key concepts

- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment
- NIST SP 800-53 RA-5: Vulnerability Monitoring and Scanning control family
- PCI DSS v4.0 Requirement 11.3: Internal and external vulnerability scanning
- CIS Controls v8 Control 7: Continuous Vulnerability Management
- ISO 27001: 2022 A.8.8**: Management of technical vulnerabilities

Tools & resources

- Nessus
- Scan Types
- Severity Levels
- Industry Standards
- Tenable Documentation

- Compliance Audit Files
- Scan Configuration Standards

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `scanning-infrastructure-with-nessus`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

scanning-kubernetes-manifests-with-kubesec (Skill Module)

`scanning-kubernetes-manifests-with-kubesec\SKILL.md`

name: scanning-kubernetes-manifests-with-kubesec
description: Agent for scanning Kubernetes manifests with Kubesec. Runs Kubesec security risk analysis on K8s manifests, evaluates security scores, identifies critical misconfigurations, and enforces security baselines in CI/CD.... Use when the user asks to scan kubernetes manifests with kubesec.
version: 1.0.0
tags:
source: scanning-kubernetes-manifests-with-kubesec

Scanning Kubernetes Manifests with Kubesec

Role: Security scanning
Audience: security engineers and vulnerability management teams
Primary tools: Kubernetes, Kubesec HTTP API, Industry Standards, Tool Ecosystem

When to use

- The user wants to **scan** kubernetes manifests with kubesec.
- Capability: Agent for scanning Kubernetes manifests with Kubesec. Runs Kubesec security risk analysis on K8s manifests, evaluates security scores, identifies critical misconfigurations, and enforces security baselines in CI/CD pipelines.
- Goal: discover exposures, misconfigurations, and vulnerable assets.
- Trigger keywords: `scanning, kubernetes, manifests, kubesec`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Developer writes Kubernetes manifest locally
2. Pre-commit hook runs `kubesec scan` on changed YAML files
3. If score < 0 (critical issues), commit is blocked with remediation guidance
4. Developer fixes issues and retries commit
5. Pull request created with manifest changes
6. CI job runs kubesec scan on all manifests in PR
7. Results posted as PR comment with score breakdown
8. Gate: PR blocked if any manifest scores below threshold
9. Merge allowed only after all manifests pass minimum score
10. Developer applies manifest via kubectl or GitOps

Key concepts

- Section 5.2: Pod Security Standards -- Kubesec validates privileged mode, host namespaces
- Section 5.7: General Policies -- Service account configuration, resource limits
- Maps directly to kubesec scoring checks for container security contexts
- Section 3.1: Image vulnerabilities and configuration defects
- Section 3.4: Orchestrator security -- manifest validation before deployment
- Section 4.1: Countermeasures for image vulnerabilities
- Privileged: No restrictions (kubesec score = lowest)
- Baseline: Prevents known privilege escalation (kubesec validates hostPID, hostNetwork, privileged)

Tools & resources

- Kubernetes
- Kubesec HTTP API

- Industry Standards
- Tool Ecosystem

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `scanning-kubernetes-manifests-with-kubesec`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

scanning-network-with-nmap-advanced (Skill Module)

scanning-network-with-nmap-advanced\SKILL.md

name: scanning-network-with-nmap-advanced
description: Automated network scanning agent using python-nmap for authorized assessments. Use when the user asks to scan network with nmap advanced.
version: 1.0.0
tags:
source: scanning-network-with-nmap-advanced

Scanning Network with Nmap Advanced

Role: Security scanning
Audience: security engineers and vulnerability management teams
Primary tools: Nmap

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **scan** network with nmap advanced.
- Capability: Automated network scanning agent using python-nmap for authorized assessments.
- Goal: discover exposures, misconfigurations, and vulnerable assets.
- Trigger keywords: `scanning, network, nmap, advanced`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Run host discovery using multiple probe techniques
2. Run SYN scan on discovered hosts for specified ports
3. Run aggressive service version detection on open ports
4. Run NSE vulnerability scripts against open ports
5. Generate JSON and CSV reports from scan results

Tools & resources

- Nmap

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `discover_hosts` — Run host discovery using multiple probe techniques.
- `scan_ports` — Run SYN scan on discovered hosts for specified ports.
- `service_version_scan` — Run aggressive service version detection on open ports.
- `vuln_scan` — Run NSE vulnerability scripts against open ports.
- `generate_report` — Generate JSON and CSV reports from scan results.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `scanning-network-with-nmap-advanced`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

secure-auth (Skill Module)

`security-toolkit\skills\secure-auth\SKILL.md`

name: secure-auth

description: Secure authentication implementation patterns. Use when implementing user login, registration, password reset, session management, JWT authentication, OAuth, MFA, or passkeys. Provides production-ready patterns aligned with NIST SP 800-63B-4, OWASP 2026 cheat sheets, OAuth 2.1, and WebAuthn L3, with breach-driven lessons.

tags:

Secure [[authentication]]

Step 0: Research the current security landscape (do this first)

> *Security knowledge ages on a 6-12 month half-life. The recipes below were last verified on 2026-05-08; they may be stale by the time you read this. Before applying any pattern in this [[skill]], fan out research scoped to the [[authentication]] primitive being implemented (passwords, sessions, JWT, OAuth, MFA, passkeys) so the recipes are interpreted against current authoritative sources, not against this file's snapshot.*

Default-on, with a documented skip

Run the 4-angle research below by default. Skip ONLY when ALL of these hold:

- (a) You ran this same [[skill]] on this same primitive within the last 4 hours of the current session,
- (b) That prior research surfaced no urgent advisories for the [[authentication]] primitive being implemented (passwords, sessions, JWT, OAuth, MFA, passkeys),
- (c) You log a one-line `Research skipped because <reason>` note in your response.

"I think I know" / "moving fast" / "user wants this done quickly" / "already familiar" are NOT valid skip reasons. The whole point of this preamble is that future-you should not trust this [[skill]] body's defaults until current state is checked.

Fan out 4 subagents in parallel

Each subagent returns at most 300 words of bullets with citations. Dispatch all 4 in a single message so they run concurrently.

Angle 1 — Authoritative
[[[

When you cannot run subagents

If subagents are not available in your runtime, the same shape applies in-line: do 4 sequential targeted searches (web search for [[[[

```
# Python — argon2-cffi
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

ph = PasswordHasher(
    memory_cost=19456,  # KiB
    time_cost=2,
    parallelism=1,
)

hashed = ph.hash(password)

try:
    ph.verify(hashed, password)
except VerifyMismatchError:
    # invalid password
    pass
```

RFC 9106 (https://datatracker.ietf.org/doc/rfc9106/) defines a more aggressive "FIRST RECOMMENDED" profile (t=1, p=4, m=2 GiB) intended for server environments with that memory available; OWASP's 5 profiles are the practical floor.

Alternate: bcrypt

Still acceptable. OWASP says cost 10 minimum, "as large as performance allows." The current code uses cost 12, which is fine — just know the floor moved.

```
// Node — bcrypt (current 6.0.0)
const bcrypt = require('bcrypt');

// bcrypt has a 72-byte input limit. Pre-hash with SHA-256
// when accepting longer passphrases, OR reject inputs over 72 bytes.
const crypto = require('crypto');

function safeBcryptInput(password) {
  const bytes = Buffer.byteLength(password, 'utf8');
  if (bytes > 72) {
    // Pre-hash to a fixed 32-byte digest, base64-encoded (44 ASCII bytes)
    return crypto.createHash('sha256').update(password).digest('base64');
  }
  return password;
}

const hashed = await bcrypt.hash(safeBcryptInput(password), 12);
const ok = await bcrypt.compare(safeBcryptInput(password), hashed);
```

Alternate: scrypt

Acceptable. OWASP minimum is N=2^17, r=8, p=1.

PBKDF2: only when FIPS-140 required

PBKDF2 is the algorithm to use when FIPS-140 compliance is a hard requirement. Otherwise prefer argon2id. OWASP minimum: 600,000 iterations of PBKDF2-HMAC-SHA256, or 210,000 of PBKDF2-HMAC-SHA512.

Never

- The OS shell-execution primitive composing a password into a command line — pass via stdin or argv.
- Plain-text storage. No exceptions, no "just for now," no "we'll fix it before launch."
- Logging the plain-text password in any code path, including error handlers.
- A custom hashing scheme. Roll-your-own is the most common breach precondition.

Password policy (NIST SP 800-63B-4)

NIST SP 800-63B-4 went FINAL 2025-07-31 (https://csrc.nist.gov/pubs/sp/800/63/b/4/final). The old 800-63B was withdrawn 2025-08-01. The values below are normative. Don't deviate.

Length

- Single-factor passwords: **15-character minimum**.
- Multi-factor passwords (one factor among several): 8-character minimum.
- Maximum: at least **64 characters** (verifier MUST allow up to 64; it MAY allow longer).

Composition

- **No composition rules.** NIST 800-63B-4 §5.1.1 explicitly: "Verifiers and CSPs SHALL NOT impose other composition rules" (no "must contain uppercase / digit / symbol").
- **Allow Unicode.** Each Unicode code point counts as one character.
- **Allow paste.** Password managers depend on it.

Rotation

- **No periodic rotation.** Don't force "change every 90 days." Rotate only on evidence of compromise.

Blocklist (mandatory in 2026)

The verifier MUST check candidate passwords against a list of known-compromised values. Use the HaveIBeenPwned k-anonymity API (https://haveibeenpwned.com/API/v3#PwnedPasswords) — you submit the first 5 chars of a SHA-1 hash, get back the suffixes that match, never send the password itself.

```
// Check candidate password against HIBP
const crypto = require('crypto');

async function isPwned(password) {
  const sha1 = crypto.createHash('sha1').update(password).digest('hex').toUpperCase();
  const prefix = sha1.slice(0, 5);
  const suffix = sha1.slice(5);

  const res = await fetch(`https://api.pwnedpasswords.com/range/${prefix}`, {
    headers: { 'Add-Padding': 'true' }
  });
  if (!res.ok) {
    // Fail closed on availability blip? Up to your threat model.
    // Default: don't block registration if HIBP is down; log and proceed.
    return false;
  }
  const body = await res.text();
  return body.split('\n').some(line => line.startsWith(suffix));
}
```

Phishing resistance

NIST 800-63B-4 REQUIRES phishing resistance at AAL3. Passwords alone never reach AAL3. AAL2 with phishing resistance is what most consumer apps should target now — that means WebAuthn (passkey) or PIV/CAC, not TOTP and not SMS.

Session-based [[authentication]]

Complete Express.js implementation

```
const express = require('express');
const session = require('express-session');
const RedisStore = require('connect-redis').default;
const { createClient } = require('redis');
const argon2 = require('argon2');
const crypto = require('crypto');

const app = express();

// Redis client for session storage
const redisClient = createClient({ url: process.env.REDIS_URL });
redisClient.connect();

// Session configuration
app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET, // At least 32 random bytes
  name: 'sessionId', // Don't use default 'connect.sid'
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: process.env.NODE_ENV === 'production', // HTTPS only in prod
    httpOnly: true,  // Not accessible via JavaScript
    sameSite: 'lax', // CSRF protection
    maxAge: 24 * 60 * 60 * 1000 // 24 hours
  }
}));

// Rate limiting for auth endpoints
const loginAttempts = new Map();

function checkRateLimit(ip) {
  const attempts = loginAttempts.get(ip) || { count: 0, resetAt: Date.now() + 900000 };

  if (Date.now() > attempts.resetAt) {
    attempts.count = 0;
    attempts.resetAt = Date.now() + 900000; // 15 minute window
  }

  if (attempts.count >= 5) {
    return false;
  }

  attempts.count++;
  loginAttempts.set(ip, attempts);
  return true;
}

// Argon2id parameters — one of the 5 OWASP-equivalent profiles
const ARGON2_OPTS = {
  type: argon2.argon2id,
  memoryCost: 19456, // KiB
  timeCost: 2,
  parallelism: 1
};

// Precompute a real argon2id hash for the user-not-found timing-attack defense.
// Must be a valid hash string at the same parameters used for storage so that
// argon2.verify does the full work — a malformed string would short-circuit at
// parse time and reintroduce the timing channel.
let DUMMY_VERIFY_HASH = null;
(async () => {
  DUMMY_VERIFY_HASH = await argon2.hash('argon2-timing-defense-init', ARGON2_OPTS);
})();

// Registration
app.post('/auth/register', async (req, res) => {
  const { email, password } = req.body;

  // Validate input
  if (!email || !password) {
    return res.status(400).json({ error: 'Email and password required' });
  }
```

```
  // NIST 800-63B-4: 15-char minimum for single-factor passwords
  if (password.length < 15) {
    return res.status(400).json({ error: 'Password must be at least 15 characters' });
  }

  // Reject up-front pwned passwords (HIBP k-anonymity)
  if (await isPwned(password)) {
    return res.status(400).json({ error: 'This password has appeared in a known breach. Choose a different one.' });
  }

  // Check if user exists
  const existingUser = await db.query(
    'SELECT id FROM users WHERE email = $1',
    [email.toLowerCase()]
  );

  if (existingUser.rows.length > 0) {
    // Don't reveal if email exists - use same message/timing
    return res.status(400).json({ error: 'Registration failed' });
  }

  // Hash password
  const hashedPassword = await argon2.hash(password, ARGON2_OPTS);

  // Create user
  const result = await db.query(
    'INSERT INTO users (email, password_hash) VALUES ($1, $2) RETURNING id',
    [email.toLowerCase(), hashedPassword]
  );

  // Create session
  req.session.userId = result.rows[0].id;
  req.session.createdAt = Date.now();

  res.json({ success: true });
});

// Login
app.post('/auth/login', async (req, res) => {
  const { email, password } = req.body;
  const clientIp = req.ip;

  // Rate limiting
  if (!checkRateLimit(clientIp)) {
    return res.status(429).json({ error: 'Too many attempts. Try again later.' });
  }

  // Validate input
  if (!email || !password) {
    return res.status(400).json({ error: 'Email and password required' });
  }

  // Find user
  const result = await db.query(
    'SELECT id, password_hash FROM users WHERE email = $1',
    [email.toLowerCase()]
  );

  if (result.rows.length === 0) {
    // Timing attack prevention: full-cost verify against a real precomputed hash.
    // A malformed hash string would make argon2.verify fail at parse time —
    // that's faster than the valid-user path and leaks "user not found" via
    // timing. DUMMY_VERIFY_HASH is computed once at module init below so
    // this path does the same work as the real verify.
    if (DUMMY_VERIFY_HASH) {
      await argon2.verify(DUMMY_VERIFY_HASH, password).catch(() => false);
    }
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const user = result.rows[0];

  // Verify password
  const isValid = await argon2.verify(user.password_hash, password).catch(() => false);

  if (!isValid) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
```

```
  // Regenerate session to prevent fixation.
  // Also regenerate on privilege CHANGE (e.g. role escalation, MFA upgrade), not just login.
  req.session.regenerate((err) => {
    if (err) {
      return res.status(500).json({ error: 'Session error' });
    }

    req.session.userId = user.id;
    req.session.createdAt = Date.now();

    // Clear rate limit on successful login
    loginAttempts.delete(clientIp);

    res.json({ success: true });
  });
});

// Logout
app.post('/auth/logout', (req, res) => {
  req.session.destroy((err) => {
    if (err) {
      return res.status(500).json({ error: 'Logout failed' });
    }
    res.clearCookie('sessionId');
    res.json({ success: true });
  });
});

// Auth middleware
function requireAuth(req, res, next) {
  if (!req.session.userId) {
    return res.status(401).json({ error: 'Authentication required' });
  }

  // Optional: Check session age
  const maxAge = 24 * 60 * 60 * 1000; // 24 hours
  if (Date.now() - req.session.createdAt > maxAge) {
    req.session.destroy();
    return res.status(401).json({ error: 'Session expired' });
  }

  next();
}

// Protected route
app.get('/api/profile', requireAuth, async (req, res) => {
  const user = await db.query(
    'SELECT id, email, created_at FROM users WHERE id = $1',
    [req.session.userId]
  );
  res.json(user.rows[0]);
});
```

JWT [[authentication]]

Full token validation: the Storm-0558 lesson

Microsoft's Storm-0558 incident (CSRB [[review]], https://www.cisa.gov/resources-tools/resources/CSRB-[[Review]]-Summer-2023-MEO-Intrusion) traced to OWA accepting a consumer-key-signed token for enterprise mailboxes — the token-validation library skipped the issuer / audience / scope / signing-key-tenancy checks. Validate every claim every time:

- `iss` (issuer) — MUST match your expected issuer string
- `aud` (audience) — MUST include your service identifier
- `exp` , `nbf` (expiration / not-before) — MUST be enforced; reject expired or future-dated tokens
- `scope` — MUST contain the scope required for the endpoint
- Signing key — MUST belong to the issuer's tenancy, not "any key the JWKS endpoint hands out"

Complete implementation with refresh tokens

```
const jwt = require('jsonwebtoken');
const crypto = require('crypto');

// Token configuration
const ACCESS_TOKEN_SECRET = process.env.ACCESS_TOKEN_SECRET;
const REFRESH_TOKEN_SECRET = process.env.REFRESH_TOKEN_SECRET;
const ACCESS_TOKEN_EXPIRY = '15m';
const REFRESH_TOKEN_EXPIRY = '7d';
const ISSUER = 'https://auth.yourapp.com';
const AUDIENCE = 'https://api.yourapp.com';

// Store refresh tokens (use Redis in production).
// Each entry tracks the family chain so reuse can revoke siblings.
const refreshTokens = new Map();

function generateAccessToken(userId, scopes = []) {
  return jwt.sign(
    { userId, scope: scopes.join(' '), type: 'access' },
    ACCESS_TOKEN_SECRET,
    {
      expiresIn: ACCESS_TOKEN_EXPIRY,
      issuer: ISSUER,
      audience: AUDIENCE
    }
  );
}

function generateRefreshToken(userId, familyId = null) {
  const tokenId = crypto.randomBytes(32).toString('hex');
  const family = familyId || crypto.randomBytes(16).toString('hex');
  const token = jwt.sign(
    { userId, tokenId, familyId: family, type: 'refresh' },
    REFRESH_TOKEN_SECRET,
    {
      expiresIn: REFRESH_TOKEN_EXPIRY,
      issuer: ISSUER,
      audience: AUDIENCE
    }
  );

  refreshTokens.set(tokenId, {
    userId,
    familyId: family,
    createdAt: Date.now(),
    used: false,
    revoked: false
  });

  return { token, familyId: family };
}

// Revoke an entire refresh-token family (used on reuse detection).
function revokeFamily(familyId) {
  for (const [id, entry] of refreshTokens) {
    if (entry.familyId === familyId) {
      entry.revoked = true;
    }
  }
}

// Login - returns both tokens
app.post('/auth/login', async (req, res) => {
  const { email, password } = req.body;

  // ... validation and password check ...

  const accessToken = generateAccessToken(user.id, ['read', 'write']);
  const { token: refreshToken } = generateRefreshToken(user.id);

  // Set refresh token as httpOnly cookie
  res.cookie('refreshToken', refreshToken, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days
  });
```

```
  // Return access token in response body
  res.json({ accessToken });
});

// Refresh endpoint with rotation + reuse detection
app.post('/auth/refresh', (req, res) => {
  const refreshToken = req.cookies.refreshToken;

  if (!refreshToken) {
    return res.status(401).json({ error: 'Refresh token required' });
  }

  let decoded;
  try {
    decoded = jwt.verify(refreshToken, REFRESH_TOKEN_SECRET, {
      issuer: ISSUER,
      audience: AUDIENCE
    });
  } catch (err) {
    return res.status(401).json({ error: 'Invalid refresh token' });
  }

  const stored = refreshTokens.get(decoded.tokenId);
  if (!stored || stored.revoked) {
    return res.status(401).json({ error: 'Token revoked' });
  }

  // REUSE DETECTION: if a previously-rotated refresh token is presented,
  // the family is compromised. Revoke every sibling immediately.
  if (stored.used) {
    revokeFamily(decoded.familyId);
    return res.status(401).json({ error: 'Token reuse detected; family revoked' });
  }

  stored.used = true;

  // Rotate: issue a new access + refresh in the same family.
  const newAccess = generateAccessToken(decoded.userId, ['read', 'write']);
  const { token: newRefresh } = generateRefreshToken(decoded.userId, decoded.familyId);

  res.cookie('refreshToken', newRefresh, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 7 * 24 * 60 * 60 * 1000
  });

  res.json({ accessToken: newAccess });
});

// Logout - revoke entire refresh-token family
app.post('/auth/logout', (req, res) => {
  const refreshToken = req.cookies.refreshToken;

  if (refreshToken) {
    try {
      const decoded = jwt.verify(refreshToken, REFRESH_TOKEN_SECRET, {
        issuer: ISSUER,
        audience: AUDIENCE
      });
      revokeFamily(decoded.familyId);
    } catch (err) {
      // Token invalid, no action needed
    }
  }

  res.clearCookie('refreshToken');
  res.json({ success: true });
});

// Auth middleware for protected routes.
//
// Scope of THIS sample: single-issuer private-token deployments using HS256
// (shared secret). It enforces iss / aud / exp / scope / pinned algorithm —
// the claim-level half of the Storm-0558 lesson.
//
// What this sample does NOT cover and you MUST add for multi-issuer / OIDC
// (Azure AD / Auth0 / Cognito / etc.): the signing-key tenancy half. There:
```

```
// 1. Use a public-key algorithm (RS256, ES256, EdDSA) — not HS256.
// 2. Resolve the issuer's JWKS at the issuer's well-known URL.
// 3. Pin which JWKS each trusted issuer is allowed to use, and reject any
//    token whose iss does not match the JWKS that signed it. Storm-0558's
//    proximate failure was OWA accepting a consumer-tenant key for an
//    enterprise-tenant token because the verifier did NOT enforce this
//    issuer-to-key-set binding.
// 4. Select the verifying key by the JWT's `kid` — but only from within
//    the pinned key set for that issuer.
function requireAuth(requiredScope) {
  return function (req, res, next) {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Access token required' });
    }

    const token = authHeader.substring(7);

    let decoded;
    try {
      decoded = jwt.verify(token, ACCESS_TOKEN_SECRET, {
        issuer: ISSUER,
        audience: AUDIENCE,
        algorithms: ['HS256'] // pin algorithm; reject 'none' and unexpected algs
      });
    } catch (err) {
      if (err.name === 'TokenExpiredError') {
        return res.status(401).json({ error: 'Token expired', code: 'TOKEN_EXPIRED' });
      }
      return res.status(401).json({ error: 'Invalid token' });
    }

    if (decoded.type !== 'access') {
      return res.status(401).json({ error: 'Invalid token type' });
    }

    if (requiredScope) {
      const scopes = (decoded.scope || '').split(' ');
      if (!scopes.includes(requiredScope)) {
        return res.status(403).json({ error: 'Insufficient scope' });
      }
    }

    req.userId = decoded.userId;
    req.scopes = (decoded.scope || '').split(' ');
    next();
  };
}

// Usage: app.get('/api/admin', requireAuth('admin'), handler)
```

Bound tokens for high-value APIs

Bearer tokens are stealable. For payments, healthcare, government, or any context where token theft is catastrophic, use sender-constrained tokens:

- **DPoP (RFC 9449, https://datatracker.ietf.org/doc/rfc9449/)** — client signs each request with a private key; the access token is bound to the public-key thumbprint (`cnf.jkt` claim). Token alone is useless without the key.
- **mTLS (RFC 8705, https://datatracker.ietf.org/doc/rfc8705/)** — token bound to the client certificate used in the TLS handshake (`cnf.x5t#S256` claim).

Both make stolen tokens worthless to the attacker.

Frontend token handling

```
// auth.js - Frontend token management

class AuthManager {
  constructor() {
    this.accessToken = null;
  }

  async login(email, password) {
    const response = await fetch('/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include', // Important for cookies
      body: JSON.stringify({ email, password })
    });

    if (!response.ok) {
      throw new Error('Login failed');
    }

    const { accessToken } = await response.json();
    this.accessToken = accessToken;

    return true;
  }

  async refreshToken() {
    const response = await fetch('/auth/refresh', {
      method: 'POST',
      credentials: 'include'
    });

    if (!response.ok) {
      this.accessToken = null;
      throw new Error('Session expired');
    }

    const { accessToken } = await response.json();
    this.accessToken = accessToken;

    return accessToken;
  }

  async fetchWithAuth(url, options = {}) {
    if (!this.accessToken) {
      throw new Error('Not authenticated');
    }

    const response = await fetch(url, {
      ...options,
      headers: {
        ...options.headers,
        'Authorization': `Bearer ${this.accessToken}`
      }
    });

    // If token expired, try to refresh and retry
    if (response.status === 401) {
      const body = await response.json();

      if (body.code === 'TOKEN_EXPIRED') {
        await this.refreshToken();

        // Retry original request
        return fetch(url, {
          ...options,
          headers: {
            ...options.headers,
            'Authorization': `Bearer ${this.accessToken}`
          }
        });
      }
    }

    return response;
  }
```

```
  async logout() {
    await fetch('/auth/logout', {
      method: 'POST',
      credentials: 'include'
    });

    this.accessToken = null;
  }
}

export const auth = new AuthManager();
```

Don't store access tokens in `localStorage` — they're readable by any XSS payload. In-memory (as above) plus an httpOnly refresh cookie is the standard shape. For SPAs that need to survive a page refresh, use the BroadcastChannel API to share the in-memory token across tabs and accept that a hard refresh forces a `/auth/refresh` call.

Passkeys / WebAuthn

WebAuthn L3 is at W3C Candidate Recommendation Snapshot as of 2026-01-13 (https://www.w3.org/TR/webauthn-3/). Comments accepted through 2026-02-10. CTAP 2.3 became a FIDO Alliance Proposed Standard 2026-02-26.

Recommended [[libraries]]:

- **Node:** `@simplewebauthn/server` (current 13.3.0). Companion browser package: `@simplewebauthn/browser`.
- **Python:** `webauthn` on PyPI (current 2.7.1). Note: the GitHub repo is `duo-labs/py_webauthn`, but the install command is `pip install webauthn` — the PyPI package name is `webauthn`, NOT `py_webauthn`.

Registration ceremony

```
// Node — @simplewebauthn/server
const {
  generateRegistrationOptions,
  verifyRegistrationResponse,
} = require('@simplewebauthn/server');

const RP_ID = 'yourapp.com';                  // your effective domain
const RP_NAME = 'YourApp';
const ORIGIN = 'https://yourapp.com';

// 1. Server: build options for the browser
app.post('/auth/passkey/register/options', requireAuth(), async (req, res) => {
  const user = await db.query('SELECT id, email, name FROM users WHERE id = $1', [req.userId]);
  const existing = await db.query(
    'SELECT credential_id, transports FROM passkeys WHERE user_id = $1',
    [req.userId]
  );

  const options = await generateRegistrationOptions({
    rpName: RP_NAME,
    rpID: RP_ID,
    userID: Buffer.from(String(user.rows[0].id)),
    userName: user.rows[0].email,
    userDisplayName: user.rows[0].name,
    attestationType: 'none',
    excludeCredentials: existing.rows.map(c => ({
      id: c.credential_id,
      transports: c.transports || undefined,
    })),
    authenticatorSelection: {
      residentKey: 'preferred',           // discoverable credentials enable conditional UI
      userVerification: 'preferred',
    },
  });

  // Store challenge for the verify step (Redis with short TTL)
  await redisClient.setEx(
    `webauthn:reg:${req.userId}`,
    300,
    options.challenge
  );

  res.json(options);
});

// 2. Server: verify what the browser returned
app.post('/auth/passkey/register/verify', requireAuth(), async (req, res) => {
  const expectedChallenge = await redisClient.get(`webauthn:reg:${req.userId}`);

  let verification;
  try {
    verification = await verifyRegistrationResponse({
      response: req.body,
      expectedChallenge,
      expectedOrigin: ORIGIN,
      expectedRPID: RP_ID,
      requireUserVerification: false,
    });
  } catch (err) {
    return res.status(400).json({ error: err.message });
  }

  if (!verification.verified || !verification.registrationInfo) {
    return res.status(400).json({ error: 'Registration not verified' });
  }

  const { credential, credentialBackedUp, credentialDeviceType } =
    verification.registrationInfo;

  await db.query(
    `INSERT INTO passkeys
     (user_id, credential_id, public_key, counter, transports, device_type, backed_up)
     VALUES ($1, $2, $3, $4, $5, $6, $7)`,
    [
      req.userId,
      credential.id,
```

```
      credential.publicKey,
      credential.counter,
      req.body.response?.transports || null,
      credentialDeviceType,
      credentialBackedUp,
    ]
  );

  await redisClient.del(`webauthn:reg:${req.userId}`);
  res.json({ verified: true });
});
```

```
<!-- Browser — @simplewebauthn/browser -->
<script type="module">
  import { startRegistration } from 'https://unpkg.com/@simplewebauthn/browser/dist/bundle/index.js';

  async function enrollPasskey() {
    const optsRes = await fetch('/auth/passkey/register/options', { method: 'POST', credentials: 'include' });
    const options = await optsRes.json();

    const attResp = await startRegistration({ optionsJSON: options });

    const verRes = await fetch('/auth/passkey/register/verify', {
      method: 'POST',
      credentials: 'include',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(attResp),
    });
    return verRes.json();
  }

  document.getElementById('enroll').addEventListener('click', enrollPasskey);
</script>
```

[[Authentication]] ceremony (with conditional UI)

Conditional UI lets the browser surface passkeys directly inside the username autofill dropdown — the user picks a passkey from the same UI they'd use to autofill an email.

```
// Node — server side
const {
  generateAuthenticationOptions,
  verifyAuthenticationResponse,
} = require('@simplewebauthn/server');

app.post('/auth/passkey/login/options', async (req, res) => {
  const options = await generateAuthenticationOptions({
    rpID: RP_ID,
    userVerification: 'preferred',
    // allowCredentials omitted → discoverable credentials → conditional UI works
  });

  // Tie the challenge to a temporary session marker
  const challengeId = crypto.randomBytes(16).toString('hex');
  await redisClient.setEx(`webauthn:auth:${challengeId}`, 300, options.challenge);
  res.json({ options, challengeId });
});

app.post('/auth/passkey/login/verify', async (req, res) => {
  const { challengeId, response } = req.body;
  const expectedChallenge = await redisClient.get(`webauthn:auth:${challengeId}`);

  if (!expectedChallenge) {
    return res.status(400).json({ error: 'Challenge expired' });
  }

  const credentialId = response.id;
  const stored = await db.query(
    `SELECT user_id, public_key, counter, transports FROM passkeys WHERE credential_id = $1`,
    [credentialId]
  );
  if (stored.rows.length === 0) {
    return res.status(401).json({ error: 'Unknown credential' });
  }

  let verification;
  try {
    verification = await verifyAuthenticationResponse({
      response,
      expectedChallenge,
      expectedOrigin: ORIGIN,
      expectedRPID: RP_ID,
      credential: {
        id: credentialId,
        publicKey: stored.rows[0].public_key,
        counter: stored.rows[0].counter,
        transports: stored.rows[0].transports || undefined,
      },
      requireUserVerification: false,
    });
  } catch (err) {
    return res.status(401).json({ error: err.message });
  }

  if (!verification.verified) {
    return res.status(401).json({ error: 'Authentication not verified' });
  }

  // Persist new counter
  await db.query(
    `UPDATE passkeys SET counter = $1, last_used_at = NOW() WHERE credential_id = $2`,
    [verification.authenticationInfo.newCounter, credentialId]
  );

  // Issue session/tokens as usual
  req.session.regenerate(() => {
    req.session.userId = stored.rows[0].user_id;
    res.json({ verified: true });
  });
});
```

```
<!-- Browser — discoverable credential + conditional UI -->
<input type="email" name="email" autocomplete="username webauthn">
<script type="module">
  import { startAuthentication, browserSupportsWebAuthnAutofill }
    from 'https://unpkg.com/@simplewebauthn/browser/dist/bundle/index.js';

  if (await browserSupportsWebAuthnAutofill()) {
    const optsRes = await fetch('/auth/passkey/login/options', { method: 'POST' });
    const { options, challengeId } = await optsRes.json();

    const authResp = await startAuthentication({
      optionsJSON: options,
      useBrowserAutofill: true, // conditional UI
    });

    await fetch('/auth/passkey/login/verify', {
      method: 'POST',
      credentials: 'include',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ challengeId, response: authResp }),
    });
    location.href = '/dashboard';
  }
</script>
```

WebAuthn L3 signal methods

L3 adds `PublicKeyCredential` static methods so the relying party can keep syncable passkeys aligned with server state without forcing a re-enrollment:

- `signalUnknownCredential({ rpId, credentialId })` — call after a verify attempt against a credential the server no longer knows about. The authenticator hides it from the user's account-picker UI.
- `signalAllAcceptedCredentials({ rpId, userId, allAcceptedCredentialIds })` — call after the server's credential list changes (revoke, enroll). The authenticator prunes anything not on the list.
- `signalCurrentUserDetails({ rpId, userId, name, displayName })` — call after profile changes. The authenticator updates labels in the picker UI.

These are advisory and best-effort; treat them as housekeeping, not security boundaries.

When to use `webauthn` (Python)

```
# pip install webauthn   # PyPI package name is `webauthn`, NOT py_webauthn
from webauthn import (
    generate_registration_options,
    verify_registration_response,
    generate_authentication_options,
    verify_authentication_response,
)
from webauthn.helpers.structs import (
    AuthenticatorSelectionCriteria,
    ResidentKeyRequirement,
    UserVerificationRequirement,
)

RP_ID = "yourapp.com"
RP_NAME = "YourApp"
ORIGIN = "https://yourapp.com"

options = generate_registration_options(
    rp_id=RP_ID,
    rp_name=RP_NAME,
    user_id=str(user_id).encode(),
    user_name=user_email,
    authenticator_selection=AuthenticatorSelectionCriteria(
        resident_key=ResidentKeyRequirement.PREFERRED,
        user_verification=UserVerificationRequirement.PREFERRED,
    ),
)
```

Password reset flow

Secure implementation

```
const crypto = require('crypto');

// Request password reset
app.post('/auth/forgot-password', async (req, res) => {
  const { email } = req.body;

  // Always return success to prevent email enumeration
  res.json({ message: 'If an account exists, a reset link has been sent.' });

  // Find user (async, after response)
  const result = await db.query(
    'SELECT id FROM users WHERE email = $1',
    [email.toLowerCase()]
  );

  if (result.rows.length === 0) {
    return; // User doesn't exist, but don't reveal that
  }

  const user = result.rows[0];

  // Generate secure token
  const token = crypto.randomBytes(32).toString('hex');
  const tokenHash = crypto.createHash('sha256').update(token).digest('hex');
  const expiresAt = new Date(Date.now() + 3600000); // 1 hour

  // Store hashed token (not plain token)
  await db.query(
    'INSERT INTO password_resets (user_id, token_hash, expires_at) VALUES ($1, $2, $3)',
    [user.id, tokenHash, expiresAt]
  );

  // Send email with plain token
  const resetUrl = `https://yourapp.com/reset-password?token=${token}`;
  await sendEmail(email, 'Password Reset', `Reset your password: ${resetUrl}`);
});

// Reset password
app.post('/auth/reset-password', async (req, res) => {
  const { token, newPassword } = req.body;

  if (!token || !newPassword) {
    return res.status(400).json({ error: 'Token and new password required' });
  }

  // NIST 800-63B-4: 15-char minimum
  if (newPassword.length < 15) {
    return res.status(400).json({ error: 'Password must be at least 15 characters' });
  }

  if (await isPwned(newPassword)) {
    return res.status(400).json({ error: 'This password has appeared in a known breach. Choose a different one.' });
  }

  // Hash the provided token to compare with stored hash
  const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

  // Find valid reset token
  const result = await db.query(
    `SELECT user_id FROM password_resets
     WHERE token_hash = $1 AND expires_at > NOW() AND used = false`,
    [tokenHash]
  );

  if (result.rows.length === 0) {
    return res.status(400).json({ error: 'Invalid or expired token' });
  }

  const userId = result.rows[0].user_id;

  // Hash new password with argon2id
  const hashedPassword = await argon2.hash(newPassword, ARGON2_OPTS);

  // Update password and invalidate token
```

```
  await db.query('UPDATE users SET password_hash = $1 WHERE id = $2', [hashedPassword, userId]);
  await db.query('UPDATE password_resets SET used = true WHERE token_hash = $1', [tokenHash]);

  // Invalidate all existing sessions and refresh-token families for this user
  await db.query('DELETE FROM sessions WHERE user_id = $1', [userId]);
  // (also revoke all refresh-token families for the user in your token store)

  res.json({ success: true });
});
```

OAuth 2.1 (with Google example)

OAuth 2.1 is currently `draft-ietf-oauth-v2-1-15` dated 2026-03-02 (https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-15). Not yet an RFC, but providers (Google, Microsoft, Okta, Auth0) already implement these constraints today, so design to the 2.1 baseline.

What 2.1 requires (and removes)

- **PKCE mandatory for ALL clients, including confidential clients.** S256 only — `plain` is deprecated.
- **Exact-string redirect-URI matching.** No prefix or wildcard matching.
- **No implicit grant (`response_type=token`).** Use authorization code with PKCE.
- **No Resource Owner Password Credentials (ROPC).** First-party login flows go through the authorization endpoint.
- **Bearer tokens never in URL query strings.** Always Authorization header.

Server-side code-with-PKCE flow (Google)

```
const { OAuth2Client } = require('google-auth-library');
const crypto = require('crypto');

const oauth2Client = new OAuth2Client(
  process.env.GOOGLE_CLIENT_ID,
  process.env.GOOGLE_CLIENT_SECRET,
  process.env.GOOGLE_REDIRECT_URI
);

function pkcePair() {
  const verifier = crypto.randomBytes(32).toString('base64url');
  const challenge = crypto.createHash('sha256').update(verifier).digest('base64url');
  return { verifier, challenge };
}

// Step 1: Redirect to Google
app.get('/auth/google', (req, res) => {
  const state = crypto.randomBytes(32).toString('hex');
  const { verifier, challenge } = pkcePair();

  req.session.oauthState = state;
  req.session.oauthVerifier = verifier;

  const authUrl = oauth2Client.generateAuthUrl({
    access_type: 'offline',
    scope: ['email', 'profile'],
    state,
    prompt: 'consent',
    code_challenge: challenge,
    code_challenge_method: 'S256',
  });

  res.redirect(authUrl);
});

// Step 2: Handle callback
app.get('/auth/google/callback', async (req, res) => {
  const { code, state } = req.query;

  // Verify state to prevent CSRF
  if (!state || state !== req.session.oauthState) {
    return res.status(400).send('Invalid state parameter');
  }

  const verifier = req.session.oauthVerifier;
  delete req.session.oauthState;
  delete req.session.oauthVerifier;

  if (!verifier) {
    return res.status(400).send('Missing PKCE verifier');
  }

  try {
    // Exchange code for tokens — include the PKCE verifier
    const { tokens } = await oauth2Client.getToken({
      code,
      codeVerifier: verifier,
    });
    oauth2Client.setCredentials(tokens);

    // Validate the id_token: Google verifies signature; we MUST check audience
    const ticket = await oauth2Client.verifyIdToken({
      idToken: tokens.id_token,
      audience: process.env.GOOGLE_CLIENT_ID,
    });

    const payload = ticket.getPayload();
    if (payload.iss !== 'https://accounts.google.com' && payload.iss !== 'accounts.google.com') {
      return res.status(400).send('Unexpected issuer');
    }

    const { sub: googleId, email, name, picture } = payload;

    // Find or create user
    let user = await db.query('SELECT id FROM users WHERE google_id = $1', [googleId]);
```

```
    if (user.rows.length === 0) {
      user = await db.query(
        `INSERT INTO users (google_id, email, name, avatar_url)
         VALUES ($1, $2, $3, $4) RETURNING id`,
        [googleId, email, name, picture]
      );
    }

    // Create session
    req.session.regenerate((err) => {
      if (err) {
        return res.status(500).send('Session error');
      }
      req.session.userId = user.rows[0].id;
      res.redirect('/dashboard');
    });

  } catch (error) {
    console.error('OAuth error:', error);
    res.status(400).send('Authentication failed');
  }
});
```

The same pattern applies to any OAuth 2.1 / OIDC provider — substitute issuer, client, scopes, and the userinfo lookup. Confirm exact-string redirect URIs in the provider console match what your server sends.

MFA (passkey-first)

CISA "Implementing Phishing-Resistant MFA" (2022-10-31, still current 2026-05; https://www.cisa.gov/sites/default/files/publications/fact-sheet-implementing-phishing-resistant-mfa-508c.pdf) ranks factor strength:

- **Phishing-resistant tier:** FIDO2 / WebAuthn (passkeys), PKI smartcards (PIV/CAC). Required at NIST AAL3.
- **AAL2-acceptable, not phishing-resistant:** App-based push with number matching, TOTP via authenticator app.
- **Restricted, last resort:** SMS / voice OTP. NIST 800-63B-4 §5 explicitly restricts these.

In 2026: **don't add SMS as a new factor.** Migrate users off it where you can.

Passkey-as-second-factor

If passwords are still your first factor, a registered passkey is the strongest second factor available. The registration and [[authentication]] ceremonies in the Passkeys / WebAuthn section above work unchanged — gate `/auth/login` on a successful passkey verify after password verify.

TOTP (still valid for AAL2)

```
const speakeasy = require('speakeasy');
const QRCode = require('qrcode');

// Enable MFA for user
app.post('/auth/mfa/enable', requireAuth(), async (req, res) => {
  // Generate secret
  const secret = speakeasy.generateSecret({
    name: `YourApp:${req.user.email}`,
    issuer: 'YourApp'
  });

  // Store secret (encrypted) temporarily until verified
  await db.query(
    'UPDATE users SET mfa_secret_temp = $1 WHERE id = $2',
    [encrypt(secret.base32), req.userId]
  );

  // Generate QR code
  const qrCode = await QRCode.toDataURL(secret.otpauth_url);

  res.json({
    secret: secret.base32, // Show this as backup
    qrCode: qrCode
  });
});

// Verify and activate MFA
app.post('/auth/mfa/verify', requireAuth(), async (req, res) => {
  const { code } = req.body;

  const result = await db.query(
    'SELECT mfa_secret_temp FROM users WHERE id = $1',
    [req.userId]
  );

  const secret = decrypt(result.rows[0].mfa_secret_temp);

  const verified = speakeasy.totp.verify({
    secret: secret,
    encoding: 'base32',
    token: code,
    window: 1 // Allow 1 step tolerance
  });

  if (!verified) {
    return res.status(400).json({ error: 'Invalid code' });
  }

  // Move secret from temp to permanent
  await db.query(
    'UPDATE users SET mfa_secret = mfa_secret_temp, mfa_secret_temp = NULL, mfa_enabled = true WHERE id = $1',
    [req.userId]
  );

  res.json({ success: true });
});

// Login with MFA
app.post('/auth/login', async (req, res) => {
  const { email, password, mfaCode } = req.body;

  // ... verify email/password first ...

  if (user.mfa_enabled) {
    if (!mfaCode) {
      return res.status(401).json({
        error: 'MFA code required',
        requiresMfa: true
      });
    }

    const verified = speakeasy.totp.verify({
      secret: decrypt(user.mfa_secret),
      encoding: 'base32',
      token: mfaCode,
      window: 1
```

```
    });

    if (!verified) {
      return res.status(401).json({ error: 'Invalid MFA code' });
    }
  }

  // ... create session/token ...
});
```

SMS: don't add it as a new factor in 2026

If you already have SMS in production, plan its retirement. Snowflake (see breach lessons) and the broader SIM-swap landscape make SMS a liability, not a safeguard. NIST 800-63B-4 §5 restricts it; CISA recommends moving off it.

Common breach lessons

Vignettes anchoring patterns to actual incidents. Each is a 2-3 sentence summary plus the lesson encoded in the recipes above.

Change Healthcare ransomware (Feb 2024)

Attackers used compromised credentials on a Citrix remote-access portal that lacked MFA. Disruption ran for weeks; UnitedHealth Group disclosed an approximately $22M ransom and exposure of records for roughly 1 in 3 US patients. Per the UnitedHealth Group RCA at https://www.unitedhealthgroup.com/newsroom/2024/2024-04-22-uhg-[[update]]-on-change-healthcare-cyberattack.html.

Lesson: MFA on every remote-access portal — VPN, Citrix, RDP gateway, jump host — not just user-facing apps. The "internal" portal is the one attackers target precisely because it's less guarded.

Snowflake / UNC5537 (Apr-Jun 2024)

Approximately 165 customer tenants were breached because Snowflake's MFA was opt-in per tenant; attackers used infostealer-harvested credentials against accounts with no second factor. Mandiant's writeup is at https://cloud.google.com/blog/topics/threat-intelligence/unc5537-snowflake-data-theft-extortion. Snowflake enforced default-on MFA from October 2024 and has been phasing in mandatory blocking of password-only sign-in across 2025-2026; see Snowflake's MFA enforcement documentation (https://docs.snowflake.com/) for the current rollout schedule.

Lesson: MFA must default to ON, not opt-in. If your customers can disable it, attackers will find the ones who did.

Storm-0558 (CSRB [[review]], 2024-04)

A Microsoft consumer signing key was compromised; attackers forged Azure AD tokens and accessed enterprise OWA mailboxes. The proximate failure was that OWA's token-validation library accepted a consumer-key-signed token for enterprise mailboxes — it skipped scope, issuer, and signing-key tenancy validation. CSRB [[[[report]]]]: https://www.cisa.gov/resources-tools/resources/CSRB-[[Review]]-Summer-2023-MEO-Intrusion. (Microsoft's MSRC postmortem URL is decommissioned.)

Lesson: Validate every claim every time — issuer, audience, scope, expiration, pinned algorithm. AND for multi-issuer / OIDC deployments, validate signing-key tenancy: pin which JWKS each trusted issuer is allowed to use, and never select a verifying key from a different issuer's set, even if the `kid` matches. The JWT recipe above bakes in `iss` / `aud` / `scope` / pinned algorithm — that's the claim-level half of the lesson and covers single-issuer private-token deployments. The signing-key tenancy half (JWKS resolution and issuer-to-key-set binding) is not in the sample; see the comment block above the `requireAuth` function for what to add when accepting tokens from multiple tenants.

Okta HAR breach (Oct 2023)

Customers uploaded HAR (HTTP Archive) debug files to Okta support; the files contained live session tokens. Five customer sessions were hijacked. Per Okta's writeup at https://sec.okta.com/articles/harfiles/.

Lesson: Sanitize session tokens and other bearer credentials at log boundaries. HAR uploads, error reports, debug dumps — strip Authorization headers, set-cookie, and known token-shaped values before they leave the user's browser.

23andMe credential stuffing (Oct 2023, settled 2024)

Attackers credential-stuffed approximately 14,000 accounts using passwords reused from other breaches, then traversed the DNA Relatives social graph to expose data on roughly 6.9 million users.

Lesson: Per-account read quotas on relationship and graph endpoints, not just per-IP rate limits. A single compromised account should not be able to enumerate the social graph faster than a human user reasonably would. Pwned-password screening (HIBP) at registration and password-reset blocks the inbound vector entirely.

Security considerations checklist

NIST AAL terminology in parentheses where relevant.

Password storage

- [ ] Default is argon2id with one of OWASP's 5 equivalent profiles (m=47104/t=1, m=19456/t=2, m=12288/t=3, m=9216/t=4, m=7168/t=5; all p=1)
- [ ] If using bcrypt, cost 10 minimum; pre-hash inputs over 72 bytes with SHA-256 (or reject)
- [ ] PBKDF2 only when FIPS-140 compliance is required

- [ ] Never storing plain-text passwords
- [ ] Never logging passwords (including in error handlers)

Password policy (NIST SP 800-63B-4)

- [ ] 15-character minimum for single-factor; 8-char minimum for multi-factor
- [ ] 64-character minimum maximum (verifier accepts at least 64)
- [ ] No composition rules ("SHALL NOT impose")
- [ ] Unicode and paste allowed
- [ ] No periodic rotation
- [ ] Verifier checks against breach blocklist (HIBP k-anonymity or equivalent)

Session management

- [ ] Sessions stored server-side (not just in cookies)
- [ ] Session IDs are cryptographically random
- [ ] Sessions regenerated on login AND on privilege change
- [ ] Sessions invalidated on logout AND on password reset
- [ ] Sessions have a maximum lifetime
- [ ] Cookies set `httpOnly`, `secure` in prod, `sameSite=lax` or stricter

JWT security

- [ ] Short access-token lifetime (15 min or less)
- [ ] Refresh tokens stored as httpOnly cookies
- [ ] Refresh-token rotation with reuse detection (rotated token used twice → revoke entire family)
- [ ] Token revocation mechanism exists
- [ ] Secrets are at least 256 bits
- [ ] Full validation on every request: issuer, audience, expiration, scope, algorithm pinned (no `alg=none`), signing-key tenancy (Storm-0558)
- [ ] Bound tokens (DPoP per RFC 9449 or mTLS per RFC 8705) for high-value APIs

Passkeys / WebAuthn

- [ ] Registration ceremony validates challenge, origin, and RP ID
- [ ] [[Authentication]] ceremony validates challenge, origin, RP ID, and signature counter
- [ ] Discoverable credentials enabled (`residentKey: 'preferred'`) for conditional UI
- [ ] WebAuthn L3 signal methods called when server credential state changes
- [ ] CTAP 2.3 considered when picking authenticator policy

MFA (CISA tier ranks)

- [ ] Phishing-resistant tier (FIDO2/WebAuthn, PIV/CAC) for AAL3 contexts
- [ ] App-based push with number matching or TOTP for AAL2
- [ ] No new SMS factors in 2026; migrate existing users off SMS
- [ ] MFA defaults to ON, not opt-in (Snowflake lesson)
- [ ] MFA enforced on every remote-access portal (Change Healthcare lesson)

OAuth 2.1

- [ ] PKCE with S256 on all clients, including confidential
- [ ] Exact-string redirect-URI matching
- [ ] No implicit or ROPC grants
- [ ] State parameter validated on callback
- [ ] `id_token` audience and issuer validated

Rate limiting

- [ ] Login attempts limited per IP and per account
- [ ] Account lockout after N failures
- [ ] Password reset requests limited
- [ ] MFA verification attempts limited
- [ ] Per-account quotas on relationship / graph endpoints (23andMe lesson)

CSRF protection

- [ ] SameSite cookie attribute set

- [ ] CSRF tokens for state-changing operations
- [ ] OAuth state parameter verified

Information disclosure

- [ ] Same error messages for valid / invalid users
- [ ] Timing attacks mitigated (dummy hash compare on missing user)
- [ ] No user enumeration via registration / reset
- [ ] Bearer tokens stripped from log boundaries (Okta HAR lesson)

securing-api-gateway-with-aws-waf (Skill Module)

securing-api-gateway-with-aws-waf\SKILL.md

name: securing-api-gateway-with-aws-waf
description: Agent for managing AWS WAF Web ACLs protecting API Gateway endpoints. Use when the user asks to secure api gateway with aws waf.
version: 1.0.0
tags:
source: securing-api-gateway-with-aws-waf

Securing API Gateway with AWS WAF

Role: Hardening
Audience: security engineers and system owners
Primary tools: AWS, boto3 WAFv2 Client

When to use

- The user wants to **secure** api gateway with aws waf.
- Capability: Agent for managing AWS WAF Web ACLs protecting API Gateway endpoints.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, gateway`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Create WAFv2 client for regional resources
2. List all Web ACLs in the account
3. Get detailed Web ACL configuration including all rules
4. Check which resources are associated with a Web ACL
5. Get sampled requests for a specific rule to analyze blocks
6. Get IPs currently rate-limited by a rate-based rule
7. Get WAF CloudWatch metrics for blocked and allowed requests
8. Run a security audit on a Web ACL configuration

Tools & resources

- AWS
- boto3 WAFv2 Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_waf_client` — Create WAFv2 client for regional resources.
- `list_web_acls` — List all Web ACLs in the account.
- `get_web_acl_details` — Get detailed Web ACL configuration including all rules.
- `check_waf_association` — Check which resources are associated with a Web ACL.
- `get_sampled_requests` — Get sampled requests for a specific rule to analyze blocks.
- `get_rate_based_keys` — Get IPs currently rate-limited by a rate-based rule.
- `get_waf_metrics` — Get WAF CloudWatch metrics for blocked and allowed requests.
- `audit_web_acl` — Run a security audit on a Web ACL configuration.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-api-gateway-with-aws-waf`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-aws-iam-permissions (Skill Module)

`securing-aws-iam-permissions\SKILL.md`

name: securing-aws-iam-permissions
description: Agent for auditing and hardening AWS IAM permissions using least-privilege principles. Use when the user asks to secure aws iam permissions.
version: 1.0.0
tags:
source: securing-aws-iam-permissions

Securing AWS IAM Permissions

Role: Hardening
Audience: security engineers and system owners
Primary tools: AWS, boto3 IAM Client, boto3 Access Analyzer Client

When to use

- The user wants to **secure** aws iam permissions.
- Capability: Agent for auditing and hardening AWS IAM permissions using least-privilege principles.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, permissions`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate and parse the IAM credential report
2. Find access keys older than the specified threshold
3. Identify roles with AdministratorAccess or wildcard policies
4. Check which IAM users have MFA enabled
5. Check IAM Access Analyzer findings for external access
6. Generate a consolidated IAM audit report

Tools & resources

- AWS
- boto3 IAM Client
- boto3 Access Analyzer Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `get_credential_report` — Generate and parse the IAM credential report.
- `find_stale_access_keys` — Find access keys older than the specified threshold.
- `find_overprivileged_roles` — Identify roles with AdministratorAccess or wildcard policies.
- `check_mfa_status` — Check which IAM users have MFA enabled.
- `check_access_analyzer` — Check IAM Access Analyzer findings for external access.
- `generate_audit_report` — Generate a consolidated IAM audit report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-aws-iam-permissions`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-aws-lambda-execution-roles (Skill Module)

`securing-aws-lambda-execution-roles\SKILL.md`

name: securing-aws-lambda-execution-roles
description: Agent for auditing and securing AWS Lambda execution roles. Use when the user asks to secure aws lambda execution roles.
version: 1.0.0
tags:
source: securing-aws-lambda-execution-roles

Securing AWS Lambda Execution Roles

Role: Hardening
Audience: security engineers and system owners
Primary tools: AWS, boto3 Lambda Client, boto3 Access Analyzer Client

When to use

- The user wants to **secure** aws lambda execution roles.
- Capability: Agent for auditing and securing AWS Lambda execution roles.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, lambda, execution, roles`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. List all Lambda functions and their execution roles
2. Analyze attached and inline policies for a Lambda execution role
3. Check if a Lambda execution role has a permission boundary
4. Validate the trust policy for confused deputy prevention
5. Validate a policy document using IAM Access Analyzer
6. Run a complete audit of all Lambda execution roles

Tools & resources

- AWS
- boto3 Lambda Client
- boto3 Access Analyzer Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `list_lambda_roles` — List all Lambda functions and their execution roles.
- `audit_role_permissions` — Analyze attached and inline policies for a Lambda execution role.
- `check_permission_boundary` — Check if a Lambda execution role has a permission boundary.
- `check_trust_policy` — Validate the trust policy for confused deputy prevention.
- `validate_with_access_analyzer` — Validate a policy document using IAM Access Analyzer.
- `full_audit` — Run a complete audit of all Lambda execution roles.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-aws-lambda-execution-roles`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-azure-with-microsoft-defender (Skill Module)

`securing-azure-with-microsoft-defender\SKILL.md`

name: securing-azure-with-microsoft-defender
description: Agent for monitoring and managing Microsoft Defender for Cloud security posture. Use when the user asks to secure azure with microsoft defender.
version: 1.0.0
tags:
source: securing-azure-with-microsoft-defender

Securing Azure with Microsoft Defender

Role: Hardening
Audience: security engineers and system owners
Primary tools: Azure, Defender Plan Names, JIT VM Access

When to use

- The user wants to **secure** azure with microsoft defender.
- Capability: Agent for monitoring and managing Microsoft Defender for Cloud security posture.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, azure, microsoft, defender`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Execute an Azure CLI command and return parsed JSON output
2. List all Defender for Cloud pricing plans and their status
3. Retrieve the current Secure Score across subscriptions
4. List security assessments and their health status
5. Retrieve recent security alerts from Defender for Cloud
6. Verify security contact configuration for alert notifications
7. Check auto-provisioning settings for security agents
8. Generate a comprehensive security posture report

Tools & resources

- Azure
- Defender Plan Names
- JIT VM Access

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `run_az_command` — Execute an Azure CLI command and return parsed JSON output.
- `get_defender_plans` — List all Defender for Cloud pricing plans and their status.
- `get_secure_score` — Retrieve the current Secure Score across subscriptions.
- `get_security_assessments` — List security assessments and their health status.
- `get_security_alerts` — Retrieve recent security alerts from Defender for Cloud.
- `check_security_contacts` — Verify security contact configuration for alert notifications.
- `check_auto_provisioning` — Check auto-provisioning settings for security agents.
- `generate_posture_report` — Generate a comprehensive security posture report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-azure-with-microsoft-defender`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-container-registry-images (Skill Module)

`securing-container-registry-images\SKILL.md`

name: securing-container-registry-images
description: Agent for auditing container registry image security: scanning, signing, and SBOM. Use when the user asks to secure container registry images.
version: 1.0.0
tags:
source: securing-container-registry-images

Securing Container Registry Images

Role: Hardening
Audience: security engineers and system owners
Primary tools: Trivy CLI, Cosign CLI, boto3 ECR Client

When to use

- The user wants to **secure** container registry images.
- Capability: Agent for auditing container registry image security: scanning, signing, and SBOM.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, container, registry, images`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Scan a container image with Trivy for vulnerabilities
2. Generate SBOM using Syft for a container image
3. Verify image signature using Cosign
4. Audit an ECR repository for security configuration
5. Get vulnerability scan findings for an ECR image
6. Run complete container image security audit

Tools & resources

- Trivy CLI
- Cosign CLI
- boto3 ECR Client

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_image_trivy` — Scan a container image with Trivy for vulnerabilities.
- `generate_sbom` — Generate SBOM using Syft for a container image.
- `verify_image_signature` — Verify image signature using Cosign.
- `audit_ecr_repository` — Audit an ECR repository for security configuration.
- `get_ecr_scan_findings` — Get vulnerability scan findings for an ECR image.
- `full_audit` — Run complete container image security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-container-registry-images`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-container-registry-with-harbor (Skill Module)

`securing-container-registry-with-harbor\SKILL.md`

name: securing-container-registry-with-harbor
description: Agent for securing container registry with Harbor. Audits Harbor registry security configuration including RBAC, vulnerability scanning policies, content trust, immutable tags, and OIDC authentication via Harbor REST.... Use when the user asks to secure container registry with harbor.
version: 1.0.0
tags:
source: securing-container-registry-with-harbor

Securing Container Registry with Harbor

Role: Hardening
Audience: security engineers and system owners
Primary tools: Harbor REST API v2.0, Harbor Roles, Security Metadata Fields, NIST SP 800-190, CIS Docker Benchmark, Harbor Security Features

When to use

- The user wants to **secure** container registry with harbor.
- Capability: Agent for securing container registry with Harbor. Audits Harbor registry security configuration including RBAC, vulnerability scanning policies, content trust, immutable tags, and OIDC authentication via Harbor REST API v2.0.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, container, registry, harbor`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Assess current exposure and priorities
2. Apply hardening controls for container registry with harbor
3. Verify controls do not break critical workflows
4. Document residual risk and monitoring

Key concepts

- Use private registries with TLS
- Scan all images for vulnerabilities before deployment
- Sign images and verify signatures
- Implement RBAC on registry access

Tools & resources

- Harbor REST API v2.0
- Harbor Roles
- Security Metadata Fields
- NIST SP 800-190
- CIS Docker Benchmark
- Harbor Security Features

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)

- Standards, data models, event IDs: `references/standards.md` (if present)
- CLI/API flags and query snippets: `references/api-reference.md`

3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-container-registry-with-harbor`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-github-actions-workflows (Skill Module)

securing-github-actions-workflows\SKILL.md

name: securing-github-actions-workflows
description: Agent for securing GitHub Actions workflows. Audits GitHub Actions workflow files for security issues including unpinned actions, excessive permissions, script injection risks, dangerous triggers, and missing secret.... Use when the user asks to secure github actions workflows.
version: 1.0.0
tags:
source: securing-github-actions-workflows

Securing Github Actions Workflows

Role: Hardening
Audience: security engineers and system owners
Primary tools: Security Checks, Dangerous Expression Contexts, Permission Scopes, SLSA Framework

When to use

- The user wants to **secure** github actions workflows.
- Capability: Agent for securing GitHub Actions workflows. Audits GitHub Actions workflow files for security issues including unpinned actions, excessive permissions, script injection risks, dangerous triggers, and missing secret protections.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, github, actions, workflows`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Pin all actions to SHA digests
2. Set restrictive default permissions
3. Sanitize all user-controlled inputs
4. Never use pull_request_target with PR checkout
5. Enable environment protection for production
6. Configure CODEOWNERS for workflow files
7. Enable Dependabot for github-actions
8. Audit third-party actions quarterly
9. Use OIDC instead of long-lived cloud credentials
10. Add harden-runner for network monitoring

Key concepts

- Workflows are code and must be reviewed and protected
- Pin action dependencies to SHA digests
- Minimize GITHUB_TOKEN permissions
- BD-1: Define security requirements for build processes
- BD-2: Automate security validation of build configurations
- BD-3: Pin all external dependencies to immutable references
- Level 2: Hosted build service (GitHub Actions qualifies)
- Level 3: Hardened build platform with isolation guarantees

Tools & resources

- Security Checks
- Dangerous Expression Contexts

- Permission Scopes
- SLSA Framework

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-github-actions-workflows`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-helm-chart-deployments (Skill Module)

securing-helm-chart-deployments\SKILL.md

name: securing-helm-chart-deployments
description: Agent for securing Helm chart deployments. Validates chart provenance, renders and scans templates for security misconfigurations, checks security contexts, and enforces Helm deployment security baselines. Use when the user asks to secure helm chart deployments.
version: 1.0.0
tags:
source: securing-helm-chart-deployments

Securing Helm Chart Deployments

Role: Hardening
Audience: security engineers and system owners
Primary tools: Security Context Fields, Security Checks, NIST SP 800-190, CIS Kubernetes Benchmark v1.8, Helm Security Resources, Compliance Mappings

When to use

- The user wants to **secure** helm chart deployments.
- Capability: Agent for securing Helm chart deployments. Validates chart provenance, renders and scans templates for security misconfigurations, checks security contexts, and enforces Helm deployment security baselines.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, helm, chart, deployments`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Set secure defaults in values.yaml (non-root, read-only fs, resource limits)
2. Add network policy templates
3. Use external secrets references
4. Lint with `helm lint --strict`
5. Render templates: `helm template test ./chart -f values.yaml > rendered.yaml`
6. Lint: `helm lint ./chart --strict`
7. Scan: `kube-linter lint rendered.yaml`
8. Scan: `checkov -f rendered.yaml --framework kubernetes`
9. Sign chart: `helm package ./chart --sign`
10. Verify chart signature: `helm verify chart.tgz`

Key concepts

- Section 4.1: Image vulnerabilities and configuration defects
- Section 5.2: Registry security and chart provenance
- Section 5.4: Secure deployment configuration
- Level 1: Documented build process (Helm chart CI)
- Level 2: Source version controlled, signed provenance
- Level 3: Hardened build platform, signed artifacts
- Level 4: Two-party review, hermetic builds
- Req 6.3.1: Security vulnerabilities identified and managed

Tools & resources

- Security Context Fields
- Security Checks

- NIST SP 800-190
- CIS Kubernetes Benchmark v1.8
- Helm Security Resources
- Compliance Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-helm-chart-deployments`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-historian-server-in-ot-environment (Skill Module)

`securing-historian-server-in-ot-environment\SKILL.md`

name: securing-historian-server-in-ot-environment
description: Agent for securing historian servers in OT environments. Audits network exposure, authentication configuration, data integrity protections, and DMZ replication architecture of process historian servers (OSIsoft PI,.... Use when the user asks to secure historian server in ot environment.
version: 1.0.0
tags:
source: securing-historian-server-in-ot-environment

Securing Historian Server in Ot Environment

Role: Hardening
Audience: security engineers and system owners
Primary tools: Common Historian Ports

When to use

- The user wants to **secure** historian server in ot environment.
- Capability: Agent for securing historian servers in OT environments. Audits network exposure, authentication configuration, data integrity protections, and DMZ replication architecture of process historian servers (OSIsoft PI, AVEVA, Honeywell PHD).
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, historian, server, environment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Assess current exposure and priorities
2. Apply hardening controls for historian server in ot environment
3. Verify controls do not break critical workflows
4. Document residual risk and monitoring

Tools & resources

- Common Historian Ports

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-historian-server-in-ot-environment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-kubernetes-on-cloud (Skill Module)

`securing-kubernetes-on-cloud\SKILL.md`

name: securing-kubernetes-on-cloud
description: Agent for auditing Kubernetes cluster security posture on managed cloud platforms. Use when the user asks to secure kubernetes on cloud.
version: 1.0.0
tags:
source: securing-kubernetes-on-cloud

Securing Kubernetes on Cloud

Role: Hardening
Audience: security engineers and system owners
Primary tools: Kubernetes

When to use

- The user wants to **secure** kubernetes on cloud.
- Capability: Agent for auditing Kubernetes cluster security posture on managed cloud platforms.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, kubernetes, cloud`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Load kubeconfig for cluster access
2. Audit namespaces for Pod Security Standards enforcement labels
3. Audit ClusterRoleBindings for overly permissive access
4. Find pods with auto-mounted service account tokens
5. Find pods running with privileged security context
6. Check if namespaces have network policies applied
7. Audit images to ensure they come from approved registries
8. Run a complete Kubernetes security audit

Tools & resources

- Kubernetes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `load_kube_config` — Load kubeconfig for cluster access.
- `check_pod_security_standards` — Audit namespaces for Pod Security Standards enforcement labels.
- `check_rbac_clusterroles` — Audit ClusterRoleBindings for overly permissive access.
- `check_service_account_tokens` — Find pods with auto-mounted service account tokens.
- `check_privileged_pods` — Find pods running with privileged security context.
- `check_network_policies` — Check if namespaces have network policies applied.
- `check_image_registries` — Audit images to ensure they come from approved registries.
- `full_audit` — Run a complete Kubernetes security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-kubernetes-on-cloud`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-remote-access-to-ot-environment (Skill Module)

securing-remote-access-to-ot-environment\SKILL.md

name: securing-remote-access-to-ot-environment
description: Agent for securing remote access to OT environments. Manages remote access sessions with MFA verification, session recording, role-based access policies, vendor co-attendance requirements, and CIP-005 compliance auditing. Use when the user asks to secure remote access to ot environment.
version: 1.0.0
tags:
source: securing-remote-access-to-ot-environment

Securing Remote Access to Ot Environment

Role: Hardening
Audience: security engineers and system owners
Primary tools: Session States, User Roles and Policies, CIP-005-7 R2 Requirements, PAM Solutions

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **secure** remote access to ot environment.
- Capability: Agent for securing remote access to OT environments. Manages remote access sessions with MFA verification, session recording, role-based access policies, vendor co-attendance requirements, and CIP-005 compliance auditing.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, remote, access, environment`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Assess current exposure and priorities
2. Apply hardening controls for remote access to ot environment
3. Verify controls do not break critical workflows
4. Document residual risk and monitoring

Tools & resources

- Session States
- User Roles and Policies
- CIP-005-7 R2 Requirements
- PAM Solutions

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-remote-access-to-ot-environment`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

securing-serverless-functions (Skill Module)

`securing-serverless-functions\SKILL.md`

name: securing-serverless-functions
description: Agent for auditing serverless function security across AWS Lambda. Use when the user asks to secure serverless functions.
version: 1.0.0
tags:
source: securing-serverless-functions

Securing Serverless Functions

Role: Hardening
Audience: security engineers and system owners
Primary tools: boto3 Lambda Client, GuardDuty Lambda Protection

When to use

- The user wants to **secure** serverless functions.
- Capability: Agent for auditing serverless function security across AWS Lambda.
- Goal: reduce attack surface and enforce strong security posture.
- Trigger keywords: `securing, serverless, functions`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. List all Lambda functions with security-relevant configuration
2. Check for Lambda function URLs with insecure authentication
3. Scan Lambda environment variables for potential hardcoded secrets
4. Identify Lambda functions sharing the same execution role
5. Check if functions have reserved concurrency set to prevent resource exhaustion
6. Run comprehensive serverless security audit

Tools & resources

- boto3 Lambda Client
- GuardDuty Lambda Protection

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `list_functions` — List all Lambda functions with security-relevant configuration.
- `check_function_urls` — Check for Lambda function URLs with insecure authentication.
- `check_env_variables` — Scan Lambda environment variables for potential hardcoded secrets.
- `check_shared_roles` — Identify Lambda functions sharing the same execution role.
- `check_reserved_concurrency` — Check if functions have reserved concurrency set to prevent resource exhaustion.
- `full_audit` — Run comprehensive serverless security audit.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `securing-serverless-functions`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

security-checklist (Skill Module)

`security-toolkit\skills\security-checklist\SKILL.md`

name: security-checklist

description: Pre-deployment security audit for web applications, organized by OWASP Top 10:2025 categories. Use when reviewing code before shipping, auditing an existing application, or when users mention "security review," "ready to deploy," "going to production," or express concern about vulnerabilities. Covers access control, supply chain, cryptography, injection, auth, integrity, logging, and exception handling.

tags:

Security checklist

Pre-deployment security audit organized around the OWASP Top 10:2025 categories (released late 2025, succeeding the 2021 edition). This is the baseline that prevents obvious disasters — not a substitute for a real penetration test or threat model. For verification depth beyond this checklist, see OWASP ASVS 5.0 (https://owasp.org/www-project-application-security-verification-standard/). For API-specific scope, see OWASP API Security Top 10:2023 (https://owasp.org/API-Security/editions/2023/en/0x00-header/).

Step 0: Research the current security landscape (do this first)

> *Security knowledge ages on a 6-12 month half-life. The recipes below were last verified on 2026-05-08; they may be stale by the time you read this. Before applying any pattern in this [[skill]], fan out research scoped to the OWASP Top 10:2025 categories being audited so the recipes are interpreted against current authoritative sources, not against this file's snapshot.*

Default-on, with a documented skip

Run the 4-angle research below by default. Skip ONLY when ALL of these hold:

- (a) You ran this same [[skill]] on this same primitive within the last 4 hours of the current session,
- (b) That prior research surfaced no urgent advisories for the OWASP Top 10:2025 categories being audited,
- (c) You log a one-line `Research skipped because <reason>` note in your response.

"I think I know" / "moving fast" / "user wants this done quickly" / "already familiar" are NOT valid skip reasons. The whole point of this preamble is that future-you should not trust this [[skill]] body's defaults until current state is checked.

Fan out 4 subagents in parallel

Each subagent returns ≤300 words of bullets with citations. Dispatch all 4 in a single message so they run concurrently.

Angle 1 — Authoritative

Have NIST / OWASP / IETF (RFCs and Internet-Drafts) / W3C / CISA published anything new about the OWASP Top 10:2025 categories being audited in the last 6-12 months? Look for: spec finalizations, deprecations, replacement specs, RFC publications, draft revisions, NIST SP updates, OWASP project version bumps. Cite by document number + [[[[publication]]]] date.

Angle 2 — Active exploitation. What's actively being exploited that targets the OWASP Top 10:2025 categories being audited? Pull from: CISA Known Exploited Vulnerabilities (KEV) catalog (filter to last 6-12 months), recent CVE / GHSA entries with high CVSS or in-the-wild exploitation, breach postmortems and incident reports (CSRB, vendor RCAs, security-vendor research). Surface CWE patterns dominating recent KEV adds. Cite by CVE number + advisory URL.

Angle 3 — Tooling and library state. Are the [[libraries]] this [[skill]] recommends still current? What are the latest major versions in the relevant package registry (npm / PyPI / RubyGems / crates.io)? Have any been deprecated, replaced, or merged into another project? Have any flipped a secure default? Look up current versions in: registry.npmjs.org, pypi.org, rubygems.org, crates.io, pkg.go.dev. Cite by package + version + release date.

Angle 4 — Practitioner discourse. What are practitioners and security teams talking about in the last 6 months? Pull from: OWASP Cheat Sheet Series (last-modified date matters), GitHub Security Lab posts, vendor security blogs (Cloudflare, Fastly, Snyk, Datadog, Wiz, GitGuardian), conference talks (Black Hat, DEF CON, OWASP Global AppSec, USENIX Security), SANS ISC, Krebs, recent OWASP project re-releases. Surface the patterns being adopted and the anti-patterns being called out. Cite by post URL + author + date.

Synthesize before applying recipes

After the 4 returns land, write a 1-paragraph "current state for the OWASP Top 10:2025 categories being audited, as of " that names:

- The current normative ceiling (what specs say SHOULD be the default in 2026).
- 1-2 active threats specific to the OWASP Top 10:2025 categories being audited from the last 6-12 months.
- Any tooling drift (deprecated lib, new default in a framework, package merged or replaced).
- Any practitioner consensus shift visible in recent cheat sheet / blog updates.

If the synthesis flags drift in this [[skill]] body's recipes (e.g., a spec finalized after 2026-05-08, a library now deprecated, a default flipped), call that out explicitly in your response and override the [[skill]] body where they conflict. The synthesis wins. The [[skill]] body is scaffolding, not scripture.

When you cannot run subagents

If subagents are not available in your runtime, the same shape applies in-line: do 4 sequential targeted searches (web search for [illegible] KEV catalog lookup, package registry version checks, recent cheat-sheet diff). Land the same 1-paragraph synthesis. Cost goes up; the protection does not change.

How to use this checklist

Walk all 10 categories before any production deployment. For each category: read the framing paragraph, run through the must-do items, and check the code-pattern references where they apply. After the walk, file findings as one issue per category with gaps. The flat 25-item Yes/No gate at the end is the pre-deploy summary, not the audit itself.

If you can't check an item, don't ship — fix it first.

A01:2025 — Broken Access Control

Authorization failures are the most-exploited class on the web. The 2025 edition folds SSRF (Server-Side Request Forgery) into A01 because the underlying failure is the same: the server acts on a request it should have rejected. Active exemplars in 2024-2025 include broken object-level authorization in API endpoints (still the dominant API risk per OWASP API Top 10:2023 API1) and SSRF used as a pivot to cloud metadata endpoints.

- [ ] Authorization checked at every endpoint, not just at the gateway or middleware layer.
- [ ] Default-deny at the controller level (explicit allow per route, not implicit allow).
- [ ] Object-level authorization on every resource fetch (IDOR — don't expose IDs without checking the principal owns them).
- [ ] Function-level authorization on admin and privileged endpoints (check role, not just route).
- [ ] SSRF defenses: outbound allowlist for any URL fetched server-side; no `localhost`, `127.0.0.0/8`, link-local (`169.254.0.0/16`), or cloud metadata IP (`169.254.169.254`) fetches; DNS rebinding protection on resolvers.
- [ ] Database row-level security enabled where the platform supports it (see code-pattern below).
- [ ] Application uses a dedicated database user with minimum required permissions, not root/admin.

Code pattern: PostgreSQL / Supabase row-level security

```
-- Enable RLS on table
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;

-- Users can only read their own documents
CREATE POLICY "Users can read own documents" ON documents
  FOR SELECT USING (auth.uid() = user_id);

-- Users can only insert documents as themselves
CREATE POLICY "Users can insert own documents" ON documents
  FOR INSERT WITH CHECK (auth.uid() = user_id);

-- Users can only update their own documents
CREATE POLICY "Users can update own documents" ON documents
  FOR UPDATE USING (auth.uid() = user_id);

-- Users can only delete their own documents
CREATE POLICY "Users can delete own documents" ON documents
  FOR DELETE USING (auth.uid() = user_id);
```

A02:2025 — Security Misconfiguration

Misconfiguration moved up the rankings (was A05 in 2021) because default-insecure framework settings keep shipping to production. The 2024 Snowflake / UNC5537 campaign is the canonical lesson: MFA was opt-in per tenant by default, and the campaign harvested credentials at scale before Snowflake flipped the default in 2024.

- [ ] Hardened images / minimal containers (distroless or slim base; no shells unless required).
- [ ] Default credentials removed; default ports closed at the firewall.
- [ ] CORS configured for specific origins (`Access-Control-Allow-Origin: *` flagged in production).
- [ ] Production-mode flag set on web frameworks: Flask `debug=False`, Django `DEBUG=False`, Express `NODE_ENV=production`, Rails `RAILS_ENV=production`.
- [ ] Stack traces never returned to clients.
- [ ] Cloud bucket / blob public-read disabled by default; access via signed URLs or authenticated requests.
- [ ] Admin interfaces not publicly accessible (VPN, allowlist, or auth proxy).
- [ ] Unnecessary services and ports closed; firewall set to deny by default.
- [ ] No secrets in source code or git history; secrets read from environment variables or a secrets manager.
- [ ] Different secrets for development / staging / production.
- [ ] API keys scoped to minimum required permissions; rotatable without code deployment.

Code pattern: environment variables [[setup]]

```
# .env (development only, never commit)
# Replace each <placeholder> with a real value generated locally.
# Generate JWT_SECRET with `openssl rand -hex 32` (256 bits).
DATABASE_URL=<your-database-connection-string>
JWT_SECRET=<32-byte-hex-secret>
API_KEY=<api-key-from-your-provider>
```

```
# .gitignore (mandatory)
.env
.env.local
.env.*.local
*.pem
*.key
credentials.json
secrets/
```

```
// Reading environment variables (Node.js with dotenv)
require('dotenv').config();
const dbUrl = process.env.DATABASE_URL;

// Fail fast if missing
if (!process.env.JWT_SECRET) {
  throw new Error('JWT_SECRET environment variable is required');
}
```

Code pattern: CORS (Express.js)

```
const cors = require('cors');

// SAFE: specific origins
app.use(cors({
  origin: ['https://myapp.com', 'https://www.myapp.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));
```

The unsafe pattern — `cors()` with no options, which sends `Access-Control-Allow-Origin: *` — is fine for dev but never for production with credentialed requests.

A03:2025 — Software Supply Chain Failures

New category in 2025 that absorbs the old 2021 A06 "Vulnerable and Outdated Components." Broader than just patching: covers SBOM, build-system integrity, package provenance, and dependency-source trust. The xz-utils CVE-2024-3094 backdoor (March 2024) is the canonical "social-engineering of an open-source maintainer" lesson; Polyfill.io (June 2024) is the canonical "trusted CDN turned hostile" lesson.

- [ ] SBOM generated at build time (CycloneDX 1.7 = ECMA-424 2nd Edition, ratified Dec 2025; or SPDX 2.3+).
- [ ] Dependency scanning runs in CI (Trivy, Grype, Snyk, or GHSA scanner) and fails the build on critical findings.
- [ ] Versions pinned (lockfiles committed; no floating tags like `latest` in container images or `^x.y.z` for security-critical libs).
- [ ] Provenance signing in place: npm provenance (GA Sep 2023), PyPI Trusted Publishing, Sigstore cosign for containers.
- [ ] SLSA v1.2 source/build track adoption tracked (released 2025-11-24, https://slsa.dev/).
- [ ] Patches applied on a schedule. Renovate or Dependabot enabled with auto-merge for patch versions.
- [ ] End-of-life runtimes flagged and replaced (Node 18 EOL April 2025; Python 3.8 EOL Oct 2024).
- [ ] Subresource Integrity on every external `<script>` and `<link rel="stylesheet">` (Polyfill.io lesson — see A08 for the SRI checklist item).

Compliance pointer: OMB M-26-05 (issued 2026-01-23) rescinded the federal-wide SBOM self-attestation mandate from M-22-18 + M-23-16. The EU Cyber Resilience Act (Reg 2024/2847) reporting obligations apply from 2026-09-11 and full obligations from 2027-12-11 — don't treat the US rescission as global rescission.

A04:2025 — Cryptographic Failures

Use vetted high-level [[libraries]]. Don't roll crypto. Don't compose AES-CBC + HMAC by hand — use libsodium, AWS Encryption SDK, or Tink. The 2026 normative ceiling is TLS 1.3 by default, TLS 1.2 minimum, TLS 1.0/1.1 disabled (RFC 8996 / BCP 195).

- [ ] HTTPS only; HTTP redirects to HTTPS.
- [ ] TLS 1.3 by default; TLS 1.2 minimum; TLS 1.0/1.1 disabled.
- [ ] HSTS preload header set: `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` (2 years).
- [ ] Secure cookie flags set (`Secure` , `HttpOnly` , `SameSite=Lax` or `Strict`).
- [ ] Database connections use SSL/TLS.

- [ ] Backups encrypted at rest and tested for restoration.
- [ ] Password storage uses argon2id with OWASP cheat-sheet parameters (m=47104/t=1, m=19456/t=2, m=12288/t=3, m=9216/t=4, or m=7168/t=5; all p=1). The full policy lives in the `secure-auth` [[skill]] in this bundle — link from here.
- [ ] No MD5 / SHA-1 / unsalted hashes anywhere in the auth or integrity path.
- [ ] No hand-rolled crypto — use libsodium, AWS Encryption SDK, Google Tink, or equivalent.

Code pattern: security headers (Express.js with Helmet)

```
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:", "https:"],
      frameAncestors: ["'none'"]
    },
  },
  hsts: {
    maxAge: 63072000,
    includeSubDomains: true,
    preload: true
  }
}));
```

```
// Manual headers — modern set
res.setHeader('X-Content-Type-Options', 'nosniff');
res.setHeader('Strict-Transport-Security', 'max-age=63072000; includeSubDomains; preload');
res.setHeader('Content-Security-Policy', "default-src 'self'; frame-ancestors 'none'");
// Note: X-Frame-Options is replaced by CSP frame-ancestors. The legacy XSS-protection header is deprecated — modern browsers ignore it.
```

A05:2025 — Injection

CWE-78 OS command injection dominates CISA KEV in 2024-2025 (14 entries in 2024, 18 in 2025) — eclipsing classic SQL injection by volume. CWE-22 path traversal also climbed (9 in 2024, 13 in 2025). MOVEit CVE-2023-34362 remains the canonical SQLi exemplar.

- [ ] All user input validated server-side. Client validation is UX, not security.
- [ ] SQL queries use parameterized statements (CWE-89). Never string concatenation.
- [ ] OS commands invoked with argv-list form, shell disabled. Allowlist-validate any path or filename used in shell-out (CWE-78).
- [ ] LDAP / NoSQL / [illegible] injection — same parameterization principle.
- [ ] HTML output escaped (XSS): framework auto-escape + CSP3 nonce + Trusted Types where supported.
- [ ] Header injection: validate any user-supplied value used in response headers (no CR/LF).
- [ ] File uploads validate type, size, and store outside webroot.
- [ ] URLs and redirects validated against an allowlist.
- [ ] JSON/XML parsers have entity expansion limits.

Code pattern: parameterized SQL (Node.js)

The unsafe pattern — interpolating user input into a SQL [illegible] string and calling `db.query` on the result — is CWE-89. Always parameterize.

```
// SAFE: parameterized query, user input bound as $1
db.query('SELECT * FROM users WHERE id = $1', [req.params.id]);
```

Code pattern: parameterized SQL (Python)

```
# SAFE: parameterized query, user input bound as %s placeholder
cursor.execute("SELECT * FROM users WHERE id = %s", [user_id])
```

The unsafe pattern — building the SQL string with an f-string that interpolates user input — is CWE-89.

Code pattern: command injection (Node.js)

The Node child_process module exposes a shell-execution primitive (the one that runs a command string through `/bin/sh`) and an argv-list primitive (the spawn family with `shell: false`). For any input that touches user data, use the argv-list primitive.

```
const { spawn } = require('child_process');

// SAFE: argv-list invocation, shell disabled
// userInput is treated as a single argument, never re-parsed by a shell
spawn('convert', [userInput, 'output.png'], { shell: false });
```

The unsafe pattern — passing a [[[[

- [ ] 15-character single-factor password minimum (NIST SP 800-63B-4). No composition rules. No periodic rotation.
- [ ] Breach blocklist check on registration / change (Have I Been Pwned API or equivalent).
- [ ] Passwords hashed with argon2id (or bcrypt cost ≥10 / scrypt as acceptable alternates). Never MD5, SHA-1, or plain text.
- [ ] MFA available for all accounts; phishing-resistant MFA (WebAuthn / passkey) REQUIRED at AAL3.
- [ ] Session tokens cryptographically random (`crypto.randomBytes` or equivalent).
- [ ] Sessions expire (24 hours for normal apps, shorter for sensitive data).
- [ ] Logout invalidates the session server-side.
- [ ] Password reset tokens are single-use and expire within 1 hour.
- [ ] Failed login attempts rate-limited (e.g., 5 attempts then cooldown).
- [ ] Session regeneration on privilege change (login, role escalation).
- [ ] Refresh-token rotation with reuse detection.
- [ ] No credentials in code, logs, or error messages.

Anchor lessons: Change Healthcare Feb 2024 (no MFA on the Citrix remote-access portal — https://www.unitedhealthgroup.com/newsroom/2024/2024-04-22-uhg-[[update]]-on-change-healthcare-cyberattack.html), Snowflake / UNC5537 2024 (default-on MFA was opt-in per tenant), Storm-0558 2023 (full token validation; CSRB [[review]] at https://www.cisa.gov/resources-tools/resources/CSRB-[[Review]]-Summer-2023-MEO-Intrusion), 23andMe 2023 (graph-traversal blast radius across linked accounts).

A08:2025 — Software or Data Integrity Failures

Note: "Software or Data" — the OR is load-bearing. Covers CI/CD pipeline integrity, signed [[update]] channels, and deserialization safety. CWE-502 (deserialization) dominates KEV in 2024-2025 (11 in 2024, 14 in 2025).

- [ ] CI/CD pipeline integrity: signed builds, branch protections, required reviews, no plaintext secrets in pipeline config.
- [ ] [[Update]] channels validated: signature checks on auto-updaters; Sigstore cosign for container images.
- [ ] Deserialization gates: never deserialize untrusted input with native binary deserializers or YAML's unrestricted-load primitives. JSON-only across trust boundaries.
- [ ] Subresource Integrity on every external `<script>` and `<link rel="stylesheet">` (Polyfill.io lesson — https://sansec.io/research/polyfill-supply-chain-attack).
- [ ] No third-party scripts referenced from CDNs without SRI hashes pinned.

Code pattern: deserialization (Python)

Python's native binary-deserialization primitive executes arbitrary code on untrusted input — `loads` runs object constructors, including any `__reduce__` payload an attacker has crafted. JSON does not. Across any trust boundary, use JSON. Same hazard for YAML: the bare loader runs Python code; use `safe_load`.

```
import json
import yaml

# DO NOT USE: pickle.loads runs arbitrary code on untrusted input —
# any object's __reduce__ method executes at parse time. CWE-502 sink.
#   import pickle
#   obj = pickle.loads(request.body)

# DO NOT USE: yaml.load with no Loader argument is equivalent to yaml.Loader and runs code:
#   config = yaml.load(request.body)

# SAFE: JSON parsing returns plain data (dict / list / str / number / bool / None)
data = json.loads(request.body)

# SAFE: when YAML is genuinely required, use safe_load
config = yaml.safe_load(request.body)
```

CWE-502 (deserialization of untrusted data) dominated the CISA KEV catalog in 2024-2025 (11 in 2024, 14 in 2025). The same warning applies in Java (`ObjectInputStream.readObject`), .NET (`BinaryFormatter` , `NetDataContractSerializer` , `LosFormatter` , `SoapFormatter`), and any Node package that "unserializes" arbitrary objects. JSON is the only safe choice for cross-trust-boundary input.

Code pattern: SRI

```
<!-- SAFE: SRI hash pinned -->
<script src="https://cdn.example.com/lib.js"
        integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"
        crossorigin="anonymous"></script>
```

The unsafe pattern — referencing an external script with no `integrity` attribute — means any change at the CDN runs in your origin's context. Polyfill.io 2024 was exactly this.

A09:2025 — Security Logging and Alerting Failures

Note: "Alerting", not "Monitoring" — the 2025 edition reframes around active response, not just collection. The Okta HAR incident (2023) is the anchor lesson: customer-uploaded debug artifacts contained live session tokens because sanitization at log boundaries was inadequate.

- [ ] Log auth events (login, logout, failed attempts), authz failures, admin actions, payment events.
- [ ] Logs do NOT include passwords, tokens, JWTs, API keys, PII, session IDs, credit cards, or full request bodies with user data.
- [ ] Centralized logging with retention policy (90 days minimum for security logs, longer for compliance).
- [ ] Logs stored securely (encrypted at rest, access-controlled).
- [ ] Alerts configured for impossible-travel, mass-failed-auth, privilege-escalation patterns.
- [ ] Error messages don't leak system information to clients (see A10).

Anchor lesson: Okta HAR file incident — https://sec.okta.com/articles/harfiles/

Code pattern: what NOT to log

```
// SAFE: structured logging with field allowlist
console.log('Login attempt:', { email, success: false, reason: 'invalid_password' });
console.log('Request:', { endpoint: req.path, method: req.method, userId: req.user?.id });
```

The unsafe pattern — logging the full request body or a credentials object — leaks every secret a user supplies. Never log password fields, token fields, full request bodies, or session identifiers.

A10:2025 — Mishandling of Exceptional Conditions

New category in 2025. CWE catalog assigned CWE-1445 to this category, covering CWE-209/234/274/476/636 among 24 CWEs. The CrowdStrike Channel File 291 incident (July 2024) is the anchor lesson: production deployment of an unvalidated config file caused 8.5M Windows hosts to BSOD.

- [ ] Fail-closed by default for security-relevant code paths (don't catch-all-and-continue on auth/authz/integrity failures).
- [ ] Error responses don't leak structure, stack traces, or internal IDs.
- [ ] Config validated before deploying to production. Staged rollouts, canary percentages, validation gates between staging and prod.
- [ ] Catch `Exception` blocks don't swallow security-relevant failures silently.
- [ ] Test exceptional paths (timeout, OOM, partial-write, network partition) — most security regressions hide here.
- [ ] Timeouts set on all external calls, with explicit handling for the timeout case.

Anchor lesson: CrowdStrike Falcon Channel File 291 RCA — https://www.crowdstrike.com/en-us/blog/falcon-content-[[update]]-preliminary-post-incident-[[[[report]]]]/

Pre-deployment summary — 25-item Yes/No gate

Run this at the end of the audit. If any answer is N, don't ship.

1. TLS 1.3 enforced (TLS 1.2 minimum, 1.0/1.1 disabled)? Y/N
2. HSTS preload header set with `max-age=63072000; includeSubDomains; preload`? Y/N
3. argon2id used for password hashing (or bcrypt cost ≥10 / scrypt)? Y/N
4. 15-character password minimum, no composition rules, no rotation? Y/N
5. Phishing-resistant MFA available for all accounts? Y/N
6. Session tokens cryptographically random and invalidated on logout? Y/N
7. SQL queries parameterized everywhere? Y/N
8. OS commands use argv-list form, shell disabled? Y/N
9. HTML output auto-escaped + CSP with `frame-ancestors`? Y/N
10. CORS scoped to specific origins (no `*` in prod)? Y/N
11. Default-deny authorization at controller level? Y/N
12. Object-level authorization on every resource fetch (no IDOR)? Y/N
13. SSRF outbound allowlist + metadata-IP blocklist? Y/N
14. Database row-level security enabled where supported? Y/N
15. No secrets in code or git history; all read from env / secrets manager? Y/N
16. Production-mode flag set (`debug=False`, `NODE_ENV=production`, etc.)? Y/N
17. Stack traces never returned to clients? Y/N
18. SBOM generated at build time and scanned in CI? Y/N
19. All third-party scripts have SRI hashes pinned? Y/N
20. Dependencies pinned via lockfile; no floating container tags? Y/N
21. End-of-life runtimes flagged and replaced? Y/N
22. Auth events, authz failures, admin actions logged (without secrets)? Y/N
23. Alerts configured for mass-failed-auth and privilege-escalation? Y/N
24. Security-relevant code paths fail closed? Y/N
25. Config validated and rolled out via staged deployment? Y/N

Incident response quick-fixes

"I stored passwords in plain text"

1. Add password hashing immediately (argon2id; see `secure-auth` [[skill]]).
2. Force password reset for all users.
3. Invalidate all existing sessions.
4. Check breach indicators (was the database ever exposed or backed up insecurely?).

"My API key is in the git history"

1. Rotate the key immediately (generate new one).
2. Revoke the old key.
3. Use `git filter-repo` (preferred over `git filter-branch`) or BFG Repo-Cleaner to remove from history.
4. Force push and coordinate with the team.

"I don't know what data I'm logging"

1. Search codebase for `console.log`, `logger.`, `print(`.
2. [[Review]] what's being logged.
3. Implement structured logging with a field allowlist.
4. Set up log rotation and retention.

"My database is publicly accessible"

1. Change database credentials immediately.
2. Configure firewall rules (allow only application-server IPs).
3. Enable SSL/TLS for connections.
4. [[Review]] access logs for unauthorized queries.

Compliance quick reference

You likely need to care about:

- **GDPR** (EU users): data deletion rights, consent, breach notification.
- **CCPA** (California users): similar to GDPR.
- **PCI DSS** (credit cards): don't store card numbers; use a payment processor.
- **HIPAA** (health data): encryption, access controls, audit logs.
- **SOC 2** (enterprise sales): security controls documentation.
- **SEC Form 8-K Item 1.05** (US-listed companies): material cybersecurity incidents disclosed within 4 business days (effective 2023-12-18).
- **EU Cyber Resilience Act** (Reg 2024/2847): products with digital elements; reporting obligations from 2026-09-11, full obligations from 2027-12-11.
- **OMB M-26-05** (US federal suppliers): rescinded the federal-wide SBOM self-attestation mandate (issued 2026-01-23). Agency-specific requirements may still apply.

Minimum for any user data:

1. Privacy policy explaining data collection.
2. Way for users to request data deletion.
3. Encryption in transit (HTTPS) and at rest.
4. Access logs and audit trail.
5. Incident response plan (even if basic).

Resources

lllllll

- MITRE Top 25 2024: https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html
- CWE-78 (OS Command Injection): https://cwe.mitre.org/data/definitions/78.html
- CWE-502 (Deserialization of Untrusted Data): https://cwe.mitre.org/data/definitions/502.html
- Have I Been Pwned: https://haveibeenpwned.com/

Supply chain

- Sigstore: https://www.sigstore.dev/
- SLSA v1.2: https://slsa.dev/
- CycloneDX 1.7 / ECMA-424: https://ecma-international.org/publications-and-[[[

social-media-intelligence (Skill Module)

journalism-core\skills\social-media-intelligence\SKILL.md

name: social-media-intelligence

description: Social media monitoring, narrative tracking, and open-source intelligence for journalists. Use when tracking viral content spread, analyzing coordinated campaigns, monitoring breaking news on social platforms, investigating accounts for authenticity, or detecting misinformation patterns. Essential for reporters covering online narratives and digital investigations.

tags:

Social media intelligence

Systematic approaches for monitoring, analyzing, and investigating social media for journalism.

When to activate

- Tracking how a story spreads across platforms
- Investigating potential coordinated inauthentic behavior
- Monitoring breaking news across social platforms
- Analyzing account networks and relationships
- Detecting bot activity or manipulation campaigns
- Building evidence trails for digital investigations
- Archiving social content before deletion

Real-time monitoring

Multi-platform tracker

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional, Dict
from enum import Enum
import hashlib

class Platform(Enum):
    TWITTER = "twitter"  # X since 2023; "twitter" retained for legacy data
    FACEBOOK = "facebook"
    INSTAGRAM = "instagram"
    TIKTOK = "tiktok"
    YOUTUBE = "youtube"
    REDDIT = "reddit"
    THREADS = "threads"
    BLUESKY = "bluesky"
    MASTODON = "mastodon"
    TELEGRAM = "telegram"

@dataclass
class SocialPost:
    platform: Platform
    post_id: str
    author: str
    content: str
    timestamp: datetime
    url: str
    engagement: Dict[str, int] = field(default_factory=dict)
    media_urls: List[str] = field(default_factory=list)
    archived_urls: List[str] = field(default_factory=list)
    content_hash: str = ""

    def __post_init__(self):
        # Hash content for duplicate detection
        self.content_hash = hashlib.md5(
            f"{self.platform.value}:{self.content}".encode()
        ).hexdigest()

@dataclass
class MonitoringQuery:
    keywords: List[str]
    platforms: List[Platform]
    accounts: List[str] = field(default_factory=list)
    hashtags: List[str] = field(default_factory=list)
    exclude_terms: List[str] = field(default_factory=list)
    start_date: Optional[datetime] = None

    def to_search_string(self, platform: Platform) -> str:
        """Generate platform-specific search query."""
        parts = []

        # Keywords
        if self.keywords:
            parts.append(' OR '.join(f'"{k}"' for k in self.keywords))

        # Hashtags
        if self.hashtags:
            parts.append(' OR '.join(f'#{h}' for h in self.hashtags))

        # Exclusions
        if self.exclude_terms:
            parts.append(' '.join(f'-{t}' for t in self.exclude_terms))

        return ' '.join(parts)
```

Breaking news monitor

```
from collections import defaultdict
from datetime import datetime, timedelta

class BreakingNewsDetector:
    """Detect sudden spikes in keyword mentions."""

    def __init__(self, baseline_window_hours: int = 24):
        self.baseline_window = timedelta(hours=baseline_window_hours)
        self.mention_history = defaultdict(list)

    def add_mention(self, keyword: str, timestamp: datetime):
        """Record a mention of a keyword."""
        self.mention_history[keyword].append(timestamp)
        # Prune old data
        cutoff = datetime.now() - self.baseline_window * 2
        self.mention_history[keyword] = [
            t for t in self.mention_history[keyword] if t > cutoff
        ]

    def is_spiking(self, keyword: str, threshold_multiplier: float = 3.0) -> bool:
        """Check if keyword is spiking above baseline."""
        now = datetime.now()
        recent = sum(1 for t in self.mention_history[keyword]
                     if t > now - timedelta(hours=1))

        baseline_hourly = len([
            t for t in self.mention_history[keyword]
            if t > now - self.baseline_window
        ]) / self.baseline_window.total_seconds() * 3600

        if baseline_hourly == 0:
            return recent > 10  # Arbitrary threshold for new topics

        return recent > baseline_hourly * threshold_multiplier

    def get_trending(self, top_n: int = 10) -> List[tuple]:
        """Get keywords sorted by spike intensity."""
        spikes = []
        for keyword in self.mention_history:
            if self.is_spiking(keyword):
                recent = sum(1 for t in self.mention_history[keyword]
                             if t > datetime.now() - timedelta(hours=1))
                spikes.append((keyword, recent))

        return sorted(spikes, key=lambda x: x[1], reverse=True)[:top_n]
```

Account analysis

Authenticity indicators

```
from dataclasses import dataclass
from datetime import datetime
from typing import List, Optional

@dataclass
class AccountAnalysis:
    username: str
    platform: Platform
    created_date: Optional[datetime] = None
    follower_count: int = 0
    following_count: int = 0
    post_count: int = 0

    # Authenticity signals
    profile_photo_is_stock: Optional[bool] = None
    bio_contains_keywords: List[str] = field(default_factory=list)
    posts_primarily_reshares: Optional[bool] = None
    posting_pattern_irregular: Optional[bool] = None
    engagement_ratio_suspicious: Optional[bool] = None

    def calculate_red_flags(self) -> dict:
        """Score account authenticity."""
        flags = {}

        # Account age
        if self.created_date:
            age_days = (datetime.now() - self.created_date).days
            if age_days < 30:
                flags['new_account'] = f"Created {age_days} days ago"

        # Follower ratio
        if self.following_count > 0:
            ratio = self.follower_count / self.following_count
            if ratio < 0.1:
                flags['low_follower_ratio'] = f"Ratio: {ratio:.2f}"

        # Posting frequency
        if self.created_date and self.post_count > 0:
            age_days = max(1, (datetime.now() - self.created_date).days)
            posts_per_day = self.post_count / age_days
            if posts_per_day > 50:
                flags['excessive_posting'] = f"{posts_per_day:.0f} posts/day"

        # Stock photo check
        if self.profile_photo_is_stock:
            flags['stock_profile_photo'] = "Profile appears to be stock image"

        return flags

    def authenticity_score(self) -> int:
        """0-100 score, higher = more likely authentic."""
        score = 100
        flags = self.calculate_red_flags()

        penalty_per_flag = 20
        score -= len(flags) * penalty_per_flag

        return max(0, score)
```

Network mapping

```
from collections import defaultdict
from typing import Set, Dict

class AccountNetwork:
    """Map relationships between accounts."""

    def __init__(self):
        self.interactions = defaultdict(lambda: defaultdict(int))
        self.accounts = {}

    def add_interaction(self, from_account: str, to_account: str,
                        interaction_type: str = "mention"):
        """Record an interaction between accounts."""
        self.interactions[from_account][to_account] += 1

    def find_clusters(self, min_interactions: int = 3) -> List[Set[str]]:
        """Find groups of accounts that frequently interact."""
        # Build adjacency with minimum threshold
        adjacency = defaultdict(set)
        for from_acc, targets in self.interactions.items():
            for to_acc, count in targets.items():
                if count >= min_interactions:
                    adjacency[from_acc].add(to_acc)
                    adjacency[to_acc].add(from_acc)

        # Find connected components
        visited = set()
        clusters = []

        for account in adjacency:
            if account in visited:
                continue

            cluster = set()
            stack = [account]

            while stack:
                current = stack.pop()
                if current in visited:
                    continue
                visited.add(current)
                cluster.add(current)
                stack.extend(adjacency[current] - visited)

            if len(cluster) > 1:
                clusters.append(cluster)

        return sorted(clusters, key=len, reverse=True)

    def coordination_score(self, accounts: Set[str]) -> float:
        """Score how coordinated a group of accounts appears."""
        if len(accounts) < 2:
            return 0.0

        total_possible = len(accounts) * (len(accounts) - 1)
        actual_connections = 0

        for acc in accounts:
            for other in accounts:
                if acc != other and self.interactions[acc][other] > 0:
                    actual_connections += 1

        return actual_connections / total_possible if total_possible > 0 else 0
```

Narrative tracking

Claim propagation tracker

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Dict, Optional

@dataclass
class Claim:
    text: str
    first_seen: datetime
    first_seen_url: str
    variations: List[str] = field(default_factory=list)
    appearances: List[Dict] = field(default_factory=list)

    def add_appearance(self, url: str, platform: Platform,
                       timestamp: datetime, author: str):
        """Track where this claim has appeared."""
        self.appearances.append({
            'url': url,
            'platform': platform.value,
            'timestamp': timestamp,
            'author': author
        })

    def spread_timeline(self) -> List[Dict]:
        """Get chronological spread of the claim."""
        return sorted(self.appearances, key=lambda x: x['timestamp'])

    def platforms_reached(self) -> Dict[str, int]:
        """Count appearances by platform."""
        counts = defaultdict(int)
        for app in self.appearances:
            counts[app['platform']] += 1
        return dict(counts)

    def velocity(self, window_hours: int = 24) -> float:
        """Calculate spread rate in appearances per hour."""
        if not self.appearances:
            return 0.0

        recent = [
            a for a in self.appearances
            if a['timestamp'] > datetime.now() - timedelta(hours=window_hours)
        ]
        return len(recent) / window_hours
```

Hashtag analysis

```
from collections import Counter
from datetime import datetime, timedelta

class HashtagAnalyzer:
    """Analyze hashtag usage patterns."""

    def __init__(self):
        self.hashtag_posts = defaultdict(list)

    def add_post(self, hashtags: List[str], post: SocialPost):
        """Record a post's hashtags."""
        for tag in hashtags:
            self.hashtag_posts[tag.lower()].append(post)

    def co_occurrence(self, hashtag: str, top_n: int = 10) -> List[tuple]:
        """Find hashtags that commonly appear with this one."""
        co_tags = Counter()

        for post in self.hashtag_posts.get(hashtag.lower(), []):
            # Extract hashtags from post content
            tags = [
                word.lower() for word in post.content.split()
                if word.startswith('#')
            ]
            for tag in tags:
                if tag != f'#{hashtag.lower()}':
                    co_tags[tag] += 1

        return co_tags.most_common(top_n)

    def posting_pattern(self, hashtag: str) -> Dict:
        """Analyze when posts with this hashtag appear."""
        posts = self.hashtag_posts.get(hashtag.lower(), [])

        hour_counts = Counter(p.timestamp.hour for p in posts)
        day_counts = Counter(p.timestamp.strftime('%A') for p in posts)

        return {
            'by_hour': dict(hour_counts),
            'by_day': dict(day_counts),
            'total_posts': len(posts),
            'unique_authors': len(set(p.author for p in posts))
        }
```

Evidence preservation

Archive before it disappears

For full archiving [[[[

```
import re
import requests
from datetime import datetime
from typing import Optional
from urllib.parse import quote, urljoin

class SocialArchiver:
    """Archive social content before deletion."""

    def __init__(self):
        self.archived = {}

    def archive_to_wayback(self, url: str) -> Optional[str]:
        """Submit URL to Internet Archive.

        Anonymous saves are rate-limited at roughly 15/minute and silently drop
        some paywalled or heavily JS-rendered pages. For high-volume archiving,
        register an Internet Archive S3 key and add an Authorization header.
        """
        try:
            save_url = f"https://web.archive.org/save/{quote(url, safe='')}"
            response = requests.get(save_url, timeout=30)
            if response.status_code == 200:
                archived_url = response.url
                self.archived[url] = {
                    'wayback': archived_url,
                    'archived_at': datetime.now().isoformat(),
                }
                return archived_url
        except Exception as e:
            print(f"Wayback archive failed: {e}")
        return None

    def archive_to_archive_today(self, url: str) -> Optional[str]:
        """Submit URL to archive.today.

        Operational notes (2026): the FBI subpoenaed archive.today's registrar
        in October 2025 to identify the operator; Wikipedia voted in February
        2026 to stop accepting it as a citation source after the site shipped
        DDoS-attack code in January 2026. The service is still useful for
        capturing content the Wayback Machine can't render, but treat it as
        secondary to the Internet Archive and document any reliance on it.
        It also rate-limits aggressively and serves CAPTCHAs to scrapers.
        """
        try:
            response = requests.post(
                'https://archive.today/submit/',
                data={'url': url, 'anyway': '1'},
                timeout=60,
                allow_redirects=False,
                headers={'User-Agent': 'Mozilla/5.0 (verification archive bot)'},
            )
            # archive.today returns the snapshot URL in one of two shapes:
            #   - 30x with Location: https://archive.today/<snapshot_id>
            #   - 200 with Refresh: 0;url=https://archive.today/<snapshot_id>
            # Following redirects silently can land on /wip/ pages or hide the
            # canonical snapshot URL, so handle both headers explicitly.
            if response.status_code in (301, 302, 303, 307, 308):
                location = response.headers.get('Location')
                if location:
                    # Location MAY be relative per RFC 7231; resolve against request URL.
                    return urljoin(response.url, location)
            if response.status_code == 200:
                refresh = response.headers.get('Refresh', '')
                # Refresh keyword is case-insensitive per HTML spec; values may
                # contain ;-separated params. Match the url= directive itself.
                m = re.search(r'\burl\s*=\s*(.+)', refresh, re.IGNORECASE)
                if m:
                    target = m.group(1).strip().strip('\'"')
                    return urljoin(response.url, target)
        except Exception as e:
            print(f"archive.today failed: {e}")
        return None

    def full_archive(self, url: str) -> dict:
        """Archive to multiple services for redundancy."""
        results = {
            'original_url': url,
```

```
            'archived_at': datetime.now().isoformat(),
            'archives': {},
        }

        wayback = self.archive_to_wayback(url)
        if wayback:
            results['archives']['wayback'] = wayback

        archive_today = self.archive_to_archive_today(url)
        if archive_today:
            results['archives']['archive_today'] = archive_today

        return results
```

Coordination detection

Behavioral signals checklist

```
## Coordinated inauthentic behavior indicators

### Timing patterns
- [ ] Multiple accounts posting same content within minutes
- [ ] Synchronized posting times across accounts
- [ ] Burst activity followed by dormancy
- [ ] Posts appear faster than human typing speed

### Content patterns
- [ ] Identical or near-identical text across accounts
- [ ] Same images/media shared by multiple accounts
- [ ] Identical typos or formatting errors
- [ ] Copy-paste artifacts visible

### Account patterns
- [ ] Accounts created around same time
- [ ] Similar naming conventions (name + numbers)
- [ ] Generic or stock profile photos
- [ ] Minimal personal content, mostly shares
- [ ] Follow the same accounts
- [ ] Engage with each other disproportionately

### Network patterns
- [ ] Form dense clusters in network analysis
- [ ] Amplify same external sources
- [ ] Target same accounts or hashtags
- [ ] Cross-platform coordination visible
```

Automated coordination scoring

```
def coordination_likelihood(posts: List[SocialPost]) -> dict:
    """Score how likely posts represent coordinated activity."""

    if len(posts) < 2:
        return {'score': 0, 'signals': []}

    signals = []
    score = 0

    # Check for identical content
    contents = [p.content for p in posts]
    unique_contents = set(contents)
    if len(unique_contents) < len(contents) * 0.5:
        signals.append("High content duplication")
        score += 30

    # Check timing clusters
    timestamps = sorted(p.timestamp for p in posts)
    rapid_posts = 0
    for i in range(1, len(timestamps)):
        if (timestamps[i] - timestamps[i-1]).seconds < 60:
            rapid_posts += 1

    if rapid_posts > len(posts) * 0.3:
        signals.append("Suspicious timing clusters")
        score += 25

    # Check unique authors
    authors = set(p.author for p in posts)
    if len(authors) > 5 and len(contents) / len(authors) > 2:
        signals.append("Few authors, many similar posts")
        score += 20

    return {
        'score': min(100, score),
        'signals': signals,
        'posts_analyzed': len(posts),
        'unique_authors': len(authors)
    }
```

Platform-specific tools

[[Status]] as of 2026. Platform APIs change rapidly — verify pricing and access before designing a project around any one path.

Platform	Research access	Notes
X (Twitter)	Pay-per-use developer API (developer.x.com); X Pro Search (consumer-facing, behind X Premium+); Brandwatch / Sprinklr (paid third-party)	Free academic/research tier ended early 2023. The 2024 Basic and Pro subscription tiers were replaced in Feb 2026 with a pay-per-use model — billed by API call, no monthly subscription. Verify current per-call rates and any rate-limit caps in the developer portal before scoping a project. Post-2023 ToS explicitly prohibits scraping.
Facebook / Instagram	Meta Content Library + Library API (research access); Junkipedia (free, journalist-friendly); NewsWhip (paid)	CrowdTangle was shut down on Aug 14, 2024 — it does not exist in any form. Meta Content Library replaced it. As of Dec 8, 2025, applications go through Meta's portal directly (previously routed via University of Michigan ICPSR). Eligibility favors academic and nonprofit researchers; most working journalists qualify only through institutional affiliation. SOMAR and other secure enclaves remain typical execution environments.
TikTok	Research API (qualifying academic and nonprofit researchers; DSA-vetted researchers in the EU); Exolyt, Pentos (paid)	Apply at developers.tiktok.com. Eligible organizations are typically academic institutions and nonprofit research entities; EU-based researchers have stronger access via DSA Article 40. Playlist Info and Commercial Content endpoints expanded in 2026.
YouTube	YouTube Data API v3	10,000 units per day default (search [[costs]] 100 units = ~100 searches/day); higher quota by application, multi-week [[review]]. No journalist-specific tier.
Reddit	Reddit API (free for non-commercial research); Arctic Shift (Pushshift successor, free dumps via Academic Torrents)	Pushshift restricted to verified moderators since 2023 — it is no longer a journalist-research path. Arctic Shift is the active successor.
Bluesky	Jetstream (filtered JSON over WebSocket, no auth required) or raw firehose	Public-by-default. Jetstream is the journalist-friendly entrypoint at ~850 MB/day filtered; raw firehose is 4-8 GB/hour and requires you to build archives yourself.
Threads (Meta)	Threads API (publishing/embedding); Meta Content Library (research)	Public profile discovery threshold lowered to 100 followers in March 2026. Bulk historical research routes through Meta Content Library, with the same academic-only restriction.
Mastodon / Fediverse	Public-timeline API (per-instance); cross-instance search at search.noc.social or fediverse.info	Many instances now set `DISALLOW_UNAUTHENTICATED_API_ACCESS`; admin-controlled. Cross-instance search is fragmented.
Telegram	Bot API + MTProto + public previews at `t.me/s/<channel>`; Bellingcat tools (Telegago, Telepathy, TelegramDB)	Public-channel scraping is legal in most jurisdictions; private groups are off-limits.

EU DSA Article 40 access

The EU Digital Services Act gives EU-based researchers stronger access rights than US researchers on TikTok and the Meta platforms. US journalists may need EU institutional partners (a university, a vetted research nonprofit) to qualify for the Research API tiers on those platforms. This is a real path, not a workaround.

OSINT tooling

The custom Python heuristics above are starting points for monitoring and pattern surfacing. For production OSINT work, the canonical external tools are:

Account authenticity and coordination

- **Botometer X** (osome.iu.edu) — academic standard for Twitter bot likelihood; archival mode only since June 2023, so it cannot score accounts created or active after May 31, 2023.
- **Hoaxy / Hoaxy2** (osome.iu.edu) — operational; Hoaxy2 added Mastodon search, Bluesky real-time monitoring, and a Facebook News Bridge. Bot scores were removed when the Twitter API died.
- **OSoMe Coordiscope** — coordinated-network visualizer, free.
- **CooRTweet** (R package) — successor to CooRnet for coordinated inauthentic behavior analysis. CooRnet was discontinued alongside CrowdTangle in August 2024.
- **Bot Sentinel** (botsentinel.com) — was offline through 2025 with relaunch announced for 2026; verify operational [[status]] before citing in reporting.

Reverse image and forensic verification

- **TinEye** — canonical exact-match and first-appearance reverse-image search.
- **Yandex Images** — strong for face and region matching, especially Russia and Eastern Europe.
- **Google Lens** — successor to the deprecated legacy Google reverse-image search.
- **Forensically** (29a.ch) and **FotoForensics** — image manipulation analysis (error level analysis, clone detection, metadata).
- **InVID-WeVerify plugin** — operational under the EU vera.ai project; new beta synthetic-image and voice-clone detectors added in 2025. Some Twitter-era features broke after the API change.

Network analysis

- **Gephi** — free, open-source; the ICIJ Panama Papers tool of record.
- **Maltego CE** — Community Edition free with registration; paid tiers expanded in 2025.
- **NodeXL** — Excel add-on, basic free / Pro paid; legacy but functional.

Methodology authorities (current, 2026)

- **Bellingcat Online Investigation Toolkit** (bellingcat.gitbook.io/toolkit) — community-maintained "Toolkit 2.0," updated continuously.
- **Information Futures Lab** at Brown SPH — successor to First Draft News (which closed June 2022); Claire Wardle co-directs.

- **WITNESS Deepfakes Rapid Response Force** (gen-ai.witness.org) — pairs reporters with media-forensics experts on deadline.
- **CJR Tow Center** — non-technical 2025 deepfake-detection guide for journalists.
- Note: **Stanford Internet Observatory** was effectively dismantled in June 2024; the election-research mantle moved to the **Stanford Social Media Lab** (Hancock) and the **NYU Stern Center for Business and Human Rights**.

Ethical guidelines

- Archive public content only.
- Don't create fake accounts for monitoring.
- Respect platform terms of service. X's post-2023 ToS explicitly prohibits scraping even of public posts; rely on the API or licensed data brokers. Don't reuse research-access tokens for non-research purposes — Meta and TikTok terminate access for ToS drift.
- Protect sources who share social content.
- Verify before publishing claims about coordination — coordination scoring is a hypothesis, not a finding.
- Consider context before amplifying harmful content.

Related [[skills]]

- **source-verification** — Verify accounts and claims found on social
- **web-archiving** — Full archiving [[[[

source-verification (Skill Module)

journalism-core\skills\source-verification\SKILL.md

name: source-verification

description: Journalism source verification and fact-checking workflows. Use when verifying claims, checking source credibility, investigating social media accounts, reverse image searching, detecting AI-generated content, or building verification trails. For reporters, fact-checkers, and researchers working with unverified information.

tags:

Source verification methodology

Systematic approaches for verifying sources, claims, and digital content in journalism and research.

Verification framework

The SIFT method

S - Stop: Don't immediately share or use unverified information
I - Investigate the source: Who is behind the information?
F - Find better coverage: What do other reliable sources say?
T - Trace claims: Find the original source of the claim

Source credibility checklist

```
## Source evaluation template

### Basic identification
- [ ] Full name/organization identified
- [ ] Contact information verifiable
- [ ] Professional credentials checkable
- [ ] Online presence consistent across platforms

### Expertise assessment
- [ ] Relevant expertise for the claim being made
- [ ] Track record in this subject area
- [ ] Recognized by peers in the field
- [ ] No history of spreading misinformation

### Motivation analysis
- [ ] Potential conflicts of interest identified
- [ ] Financial stake in the outcome?
- [ ] Political or ideological motivation?
- [ ] Personal grievance involved?

### Corroboration
- [ ] Can claims be independently verified?
- [ ] Do other credible sources confirm?
- [ ] Is documentary evidence available?
- [ ] Are there contradicting sources?
```

Digital verification techniques

Social media account analysis

For deeper open-source intelligence and platform-specific account-analysis techniques, use the **social-media-intelligence** [[skill]]. The notes here cover the verification-context subset — specifically what triggers a "verify before quoting" decision.

```
## Account verification checklist

### Account age and history
- Creation date (older accounts more credible)
- Posting frequency and patterns
- Gaps in activity (dormant then suddenly active?)
- Language consistency over time

### Network analysis
- Follower/following ratio
- Quality of followers (real accounts vs. bots)
- Interaction patterns (who engages with them?)
- Mutual connections with verified accounts

### Content patterns
- Original content vs. reshares only
- Topics discussed consistently
- Geographic indicators in posts
- Time zone of posting activity

### Red flags
- Recently created account making bold claims
- Sudden pivot in topics or tone
- Coordinated behavior with other accounts
- Stock photo profile picture
- Generic bio with no specifics
```

Reverse image search workflow

```
## Image verification process

### Step 1: Reverse image search
Tools to use:
- Google Images (images.google.com)
- TinEye (tineye.com)
- Yandex Images (yandex.com/images) - best for faces
- Bing Visual Search

### Step 2: Check metadata (EXIF)
- Original capture date/time
- Camera/device information
- GPS coordinates (if available)
- Software used to edit

Tools:
- Jeffrey's EXIF Viewer (exif.regex.info)
- FotoForensics (fotoforensics.com)
- InVID verification plugin

### Step 3: Analyze image content
- Weather conditions (match reported date?)
- Shadows (consistent with time of day?)
- Signage/text (correct language for location?)
- Architecture (matches claimed location?)
- Clothing (seasonal appropriateness?)

### Step 4: Find original source
- Earliest appearance online
- Original photographer/source
- Context of first publication
- Has it been used in other contexts?
```

Video verification

```
## Video verification checklist

### Technical analysis
- [ ] Resolution consistent throughout
- [ ] Audio sync matches video
- [ ] No visible editing artifacts
- [ ] Lighting consistent across frames
- [ ] Shadows behave naturally

### Content analysis
- [ ] Location identifiable and verifiable
- [ ] Time indicators (sun position, shadows)
- [ ] Weather matches historical records
- [ ] Background details consistent
- [ ] People's clothing appropriate for context

### Metadata check
- [ ] Upload date vs. claimed event date
- [ ] Original source identified
- [ ] Chain of custody traceable
- [ ] Multiple angles available?

### Tools
- InVID/WeVerify browser extension
- YouTube DataViewer (citizenevidence.amnestyusa.org)
- Frame-by-frame analysis tools
```

Detecting AI-generated and synthetic media

By 2026, naked-eye detection of high-end synthetic media is no longer reliable. The Columbia Journalism [[Review]]'s 2025 guide is blunt: detection tools "have largely not kept up with diffusion models." Treat any single tool's verdict as one input, not a ruling.

The verification stack now has two layers — **provenance** (was this content cryptographically signed when created?) and **detection** (does it look or sound generated?). Provenance is the stronger signal when present, but its absence doesn't mean fake.

Layer 1 — Content Credentials (C2PA) provenance check

The Coalition for Content Provenance and Authenticity (C2PA) standard ships cryptographic manifests inside image, audio, and video files describing their origin and edit history. Specification 2.2 was released in April–May 2025; the C2PA Conformance Program and Trust List launched mid-2025 and the legacy ingredient trust list was frozen January 1, 2026.

Production adoption (verified May 2026):

- **Image generators.** OpenAI DALL-E 3 (since 2023) and Sora 2 video write Credentials by default; Sora 2 also includes a visible moving watermark. Adobe Photoshop, Lightroom, and Firefly write Credentials across Creative Cloud. Microsoft Bing Image Creator, Designer, Copilot, and Azure OpenAI write Credentials. Google Gemini and Nano Banana Pro images carry C2PA plus SynthID.
- **Cameras (capture-side signing).** Leica M11-P (October 2023, first to ship), SL3-S; Sony Alpha 1 II, Alpha 9 III, PXW-Z300; Canon EOS R1 and R5 Mark II via firmware (July 2025); Google Pixel 10 (in-camera, integrated with Google Photos).
- **Cameras with known issues.** Nikon Z6 III's C2PA service was suspended in 2025 after a signing-key vulnerability and revoked certificates; not restored as of early 2026. Treat Nikon Z6 III credential claims with caution.
- **Newsrooms.** BBC, NYT, AP, and Reuters are CAI/C2PA members; production-pipeline [[integration]] is uneven across the industry.

Verification tool: drop any file at `contentcredentials.org/verify` to read its manifest, capture device, edit history, and any AI-tool involvement. Adobe's Content Authenticity Inspector and the Digimarc C2PA browser extension provide the same in plugin form.

Hard binding vs soft binding. Hard binding embeds a SHA-256 hash of the content in the signed manifest — any pixel change invalidates it (strong integrity, brittle to re-encoding). Soft binding stores a perceptual fingerprint or invisible watermark in a manifest repository — survives screenshots and transcoding but offers weaker integrity guarantees. Soft binding lets you *recover* a manifest after metadata stripping.

Known limitations.

- Screenshots strip hard-binding manifests entirely.
- Most social platforms strip metadata on upload. TikTok and Meta have started preserving Credentials on some surfaces; coverage is partial.
- Absence of Credentials does **not** mean fake. Most camera and phone images in circulation today are unsigned.
- Signing-key compromise is a real attack vector (Nikon 2025). A "valid signature" can be undermined by upstream breaches.

Layer 2 — Automated detection tools

Tool	[[Status]] (May 2026)	Pricing	Use
Hive AI (`thehive.ai`)	Operational	Demo + paid API	Image, video, audio. Strong for high volume
Reality Defender (`realitydefender.com`)	Operational	Free tier: 50 audio/image scans/month	Image, video, audio, text in one API
AI or Not (`aiornot.com`)	Operational	Free tier + paid	Fast image triage. First-pass, not authoritative
Sensity AI (`sensity.ai`)	Operational	Enterprise-priced, forensic-grade	Government/legal use; not journalist-budget-friendly
DeepFake-o-Meter (U. Buffalo)	Operational	Free, academic	Listed in CJR's recommended journalist set
Adobe Content Authenticity Inspector	Operational	Free	C2PA manifest reading only — no detection
TrueMedia.org	**Shut down January 14, 2025**	n/a	Tech open-sourced on GitHub; do not link out to the dead service
Microsoft Video Authenticator	No longer publicly offered	n/a	Skip
Intel FakeCatcher	Active research, not publicly available	n/a	Research/enterprise tier only
Optic	Unverified live [[status]] — last known still operating in 2025 CJR guide	Free	Use as one input among others; don't rely on as authoritative
Deepware Scanner	Domain active, live functionality unverified	Free web	Confirm responding before relying

Single-tool verdicts are not enough. Run at least two detectors and treat disagreement as a signal to escalate to deeper analysis or source contact.

Layer 3 — Detection by eye and ear (2026 calibration)

Older artifact tells — extra fingers, weird ears, asymmetric pupils — are largely gone in current diffusion and Sora-2-class video output. What still leaks in May 2026:

- **Boundary regions.** Hairlines, ear edges, tooth boundaries, glasses-to-skin transitions — sub-pixel inconsistency on careful inspection.
- **Lighting and shadow physics.** Highlights that don't match scene light direction; cast shadows missing or contradictory.
- **Eye reflection mismatches.** Left and right catchlights inconsistent with the scene.
- **Audio-video desync.** Phoneme-to-lip alignment drifts over multi-second clips.
- **Skin texture.** Waxy or over-smooth in places; noise pattern uniform across the frame instead of varying with surface.
- **Voice clones.** Breath placement, plosive consonants, and room tone are the remaining giveaways. Fortune (December 2025) reports voice cloning has crossed the indistinguishable threshold for casual listeners — assume voice-only verification fails.

Detection-by-eye is **unreliable on its own**. Use it for triage and to decide whether to escalate, never as the final ruling.

The verification workflow for suspect media

1. **Check Content Credentials first.** Drop the file at `contentcredentials.org/verify`. A valid manifest from a known signer is a strong positive provenance signal. Absence proves nothing.
2. **Reverse image search.** Google Lens, TinEye, Yandex (still strongest for faces). Find earliest known appearance.
3. **Run two automated detectors.** Hive + Reality Defender for image; AI or Not for fast triage. Disagreement between detectors means escalate.
4. **Frame-by-frame and audio analysis.** For video, check boundary artifacts and lip sync. For audio, examine spectrogram, breath patterns, and room-tone uniformity.
5. **Reach the source.** Direct contact remains the highest-confidence step. C2PA tells you who *signed*; it doesn't tell you who *witnessed*.

Document verification

PDF and document analysis

```
## Document verification steps

### Metadata examination
- Creation date and modification history
- Author information
- Software used to create
- Embedded fonts and images

### Visual inspection
- Consistent formatting throughout
- Font matching (no spliced text)
- Alignment of text and images
- Quality consistent across pages
- Signatures appear authentic

### Content verification
- Dates internally consistent
- Names spelled correctly throughout
- Reference numbers valid
- Contact information verifiable
- Letterhead matches known examples

### Provenance
- How was document obtained?
- Chain of custody documented?
- Original vs. copy?
- Can source provide additional context?
```

Building a verification trail

Documentation

[[[

```
import requests
from urllib.parse import quote

def archive_url(url: str, perma_cc_api_key: str | None = None) -> dict:
    """Archive a URL to Wayback Machine and Archive.today.

    Returns a dict with the archived URL (or error) for each service.
    Pass perma_cc_api_key to also archive to Perma.cc (requires an account).
    """
    results = {}

    # Internet Archive Wayback Machine
    try:
        response = requests.get(
            f'https://web.archive.org/save/{quote(url, safe="")}',
            timeout=60,
            allow_redirects=True,
        )
        if response.status_code == 200:
            results['wayback'] = response.url
        else:
            results['wayback_error'] = f'HTTP {response.status_code}'
    except requests.RequestException as e:
        results['wayback_error'] = str(e)

    # Archive.today — POST to /submit/, the archived URL appears in the
    # Refresh header (or the Location header on a 302).
    try:
        response = requests.post(
            'https://archive.ph/submit/',
            data={'url': url},
            timeout=120,
            allow_redirects=False,
            headers={'User-Agent': 'Mozilla/5.0 (verification archive bot)'},
        )
        archived = response.headers.get('Refresh', '').split('url=')[-1] \
            or response.headers.get('Location', '')
        if archived:
            results['archive_today'] = archived
        else:
            results['archive_today_error'] = f'no archived URL returned (HTTP {response.status_code})'
    except requests.RequestException as e:
        results['archive_today_error'] = str(e)

    # Perma.cc (optional, requires API key from a Perma account)
    if perma_cc_api_key:
        try:
            response = requests.post(
                'https://api.perma.cc/v1/archives/',
                json={'url': url},
                headers={'Authorization': f'ApiKey {perma_cc_api_key}'},
                timeout=60,
            )
            if response.status_code == 201:
                results['perma_cc'] = f"https://perma.cc/{response.json()['guid']}"
            else:
                results['perma_cc_error'] = f'HTTP {response.status_code}'
        except requests.RequestException as e:
            results['perma_cc_error'] = str(e)

    return results
```

Run on every primary-source URL the moment you decide it might appear in the story. Pages disappear, get edited, or go behind paywalls. An archive captured at the moment of reporting is the difference between a defensible quote and one that evaporates before [[[[publication]]]].

Screenshot documentation

```
## Screenshot best practices

1. **Full page capture**: Use browser extensions for full-page screenshots
2. **Include URL bar**: Shows the source URL
3. **Include timestamp**: System clock visible or add manually
4. **Save metadata**: Note when and how captured
5. **Multiple formats**: Save as PNG (lossless) and PDF
6. **Secure storage**: Hash files and store securely

Recommended tools:
- Hunchly (hunch.ly) - automatic capture and logging
- Screenpresso - full page with annotations
- Browser print-to-PDF - includes URL and date
```

Interview verification

Pre-interview source check

```
## Source background check

### Public records
- [ ] Professional licenses verified
- [ ] Court records checked
- [ ] Business registrations confirmed
- [ ] Property records (if relevant)
- [ ] Campaign finance records (if political)

### Professional background
- [ ] LinkedIn profile reviewed
- [ ] Employer confirmed
- [ ] Previous employers contacted
- [ ] Published work reviewed
- [ ] Conference appearances verified

### Social media audit
- [ ] All platforms identified
- [ ] Post history reviewed
- [ ] Connections/followers analyzed
- [ ] Previous statements on topic
- [ ] Any deleted content found?

### Media appearances
- [ ] Previous interviews found
- [ ] Consistency with current claims
- [ ] Other journalists' assessments
- [ ] Any retractions or corrections?
```

During interview verification

```
## Real-time verification techniques

### Document requests
- Ask for documentation during interview
- Verify documents aren't altered
- Request originals, not copies when possible
- Note document condition and provenance

### Specific detail probing
- Ask for specific dates, names, locations
- Request corroborating witnesses
- Ask "How do you know that?"
- Follow up on vague answers

### Consistency checks
- Note initial version of story
- Return to key points later
- Compare details across tellings
- Flag inconsistencies for follow-up

### Recording best practices
- Get consent (check local laws)
- Use reliable recording equipment
- Backup recording in real-time
- Note non-verbal cues separately
```

Verification resources

Tools

Tool	Purpose	URL
InVID / WeVerify	Video verification browser plugin	`weverify.eu`
TinEye	Reverse image search	`tineye.com`
Yandex Images	Reverse image search (best for faces)	`yandex.com/images`
Wayback Machine	Web archives	`web.archive.org`
Archive.today	Web archives (covers sites that block Wayback)	`archive.ph`
Meta Content Library	Meta public-content research (replaces CrowdTangle, sunset Aug 2024)	`transparency.meta.com/researchtools/meta-content-library`
Hoaxy	Claim spread visualization	`hoaxy.osome.iu.edu`
OpenCorporates	Company records	`opencorporates.com`
OCCRP Aleph	Document and entity search	`aleph.occrp.org`
FotoForensics	Image error-level analysis	`fotoforensics.com`

About CrowdTangle. Meta sunset CrowdTangle on August 14, 2024. The replacement, the Meta Content Library, requires institutional affiliation (qualifying academic, journalist with a verified .edu/.org/.gov institution) and offers slower, less-feature-rich access. [[Skill]] content elsewhere on the web that still treats CrowdTangle as available is out of date.

About Media Bias/Fact Check. This site is one observer's editorial classification, not a peer-reviewed authority. Use it as one input among several when assessing an unfamiliar outlet, not as a definitive ruling.

Training resources

- Bellingcat guides — `bellingcat.com/resources`
- Google News Initiative — `newsinitiative.withgoogle.com`
- Verification Handbook — `verificationhandbook.com`
- SPJ ethics resources — `spj.org/ethics`
- First Draft News — `firstdraftnews.org` (note: First Draft was wound down in 2022; the site remains a useful archive but is not actively maintained)

Related [[skills]]

- **[[fact-check]]-workflow** — Structured claim verification and rating
  [[[[

- **interview-prep** — Verifying source backgrounds before interviews
- **interview-transcription** — Verifying quotes against the original recording
- **web-archiving** — Full archiving [[[[

story-pitch (Skill Module)

journalism-core\skills\story-pitch\SKILL.md

name: story-pitch
description: Craft effective story pitches for different publication types and formats. Use when pitching to editors, preparing query letters, or developing story angles. Includes templates for daily news, features, investigations, op-eds, and freelance queries.
tags:

Story pitch guide

Good stories die in bad pitches. This [[skill]] covers pitch structure, angle development, and outlet-specific formatting.

When to use

- Pitching stories to editors (internal or external)
- Developing freelance query letters
- Refining story angles before reporting
- Competing for assignment in a newsroom
- [[Proposal]] writing for grants or fellowships

The pitch fundamentals

What every pitch needs

1. **The hook**: Why should anyone care? Why now?
2. **The story**: What will you show/reveal/explain?
3. **The evidence**: What do you have? What can you get?
4. **The access**: Who will you talk to? What will you see?
5. **The ask**: What do you need (time, budget, resources)?

The "so what" test

Before pitching, answer:
- **Why this story?** (significance)
- **Why now?** (timeliness/news hook)
- **Why you?** (access, expertise, angle)
- **Why this outlet?** (audience fit)

If you can't answer all four, you're not ready to pitch.

Pitch templates by type

Daily news pitch

For quick-turn stories to assignment editors.

```
**Slug:** [2-3 word identifier]
**Pitch:** [One sentence: What's the news?]
**Why now:** [What makes this timely?]
**Sources:** [Who you'll talk to]
**Timeline:** [When you can file]
**Format:** [Length, multimedia]

Example:
**Slug:** Council-Budget-Vote
**Pitch:** City council votes tonight on controversial police budget increase.
**Why now:** Final vote after months of debate, protests expected.
**Sources:** Council president, protest organizers, police union rep.
**Timeline:** File by 10pm for morning.
**Format:** 600 words + photos from hearing.
```

Feature pitch

For longer-form stories with more development time.

```
## [Working title]

**The story in one sentence:**
[What is this story about?]

**The hook:**
[Why should readers care? What's the emotional/intellectual entry point?]

**What's new:**
[What do you know/have that hasn't been told?]

**The shape:**
[How will you tell it? Narrative structure, key scenes, characters]

**Sources and access:**
[Who will you interview? What will you witness? Documents?]

**Visuals:**
[Photo opportunities, graphics, multimedia]

**Timeline and resources:**
[How long to report? Write? What do you need?]

**Why me:**
[Your connection to story, relevant expertise, access]

Example:

## The last ferry captain

**The story in one sentence:**
After 47 years, the last original captain of the Staten Island Ferry is retiring—and taking decades of institutional knowledge with him.

**The hook:**
8 million New Yorkers depend on this ferry. Most have never met the people who run it. Captain Mike Carbone has seen mayors come and go, survived 9/11 from the water, and trained every current captain. His retirement is the end of an era.

**What's new:**
No one has profiled the ferry crews since the 2003 accident. Carbone has agreed to a ride-along on his final week. He's kept a journal since 1977.

**The shape:**
Narrative profile. Open with his final departure. Flash back through career highlights. Include voices from crew, family, passengers he's befriended across decades.

**Sources and access:**
- Carbone (confirmed, ride-along approved)
- NYC DOT ferry division chief
- Other captains he's trained
- Regular commuters
- His family

**Visuals:**
Photographer on ride-along. Harbor shots at dawn. Carbone's home with memorabilia. Archive photos from his collection.

**Timeline and resources:**
2 weeks reporting, 1 week writing. Need photographer availability for 3 early mornings.

**Why me:**
I grew up taking this ferry and have always wanted to tell this story. I've already built rapport with Carbone through initial conversations.
```

Investigation pitch

For accountability journalism requiring significant resources.

```
## [Working title]

**The thesis:**
[What we can prove / what we suspect]

**Why it matters:**
[Impact: Who is harmed? How much money? How many people?]

**What we have:**
[Documents, sources, data already obtained]

**What we need:**
[Additional reporting required]

**Key sources:**
[Who will talk? On/off record? Whistleblowers?]

**Documentary evidence:**
[FOIA requests needed? Court records? Financial filings?]

**Legal considerations:**
[Defamation risk? Source protection needs?]

**Competitive risk:**
[Is anyone else on this? Why now?]

**Timeline:**
[Realistic assessment of time to report, write, edit, legal review]

**Resources needed:**
[FOIA costs, travel, data analysis, graphics, multimedia]

**Publication plan:**
[Single story? Series? Rolling publication?]

Example:

## State contracts going to insiders

**The thesis:**
More than $50 million in state construction contracts across five years went to companies with personal ties to procurement officials, circumventing competitive bidding.

**Why it matters:**
Taxpayers are overpaying for public projects. At least three bridges have needed early repairs after work by connected contractors.

**What we have:**
- State contract database (scraped, analyzed)
- Corporate filings showing ownership connections
- Two former procurement employees willing to talk (one on record)
- Email chain showing official steering business to friend

**What we need:**
- Additional FOIA for internal communications
- More contractors willing to describe losing bids unfairly
- Engineering review of the bridge repair patterns
- Response from officials named

**Key sources:**
- Whistleblower A (on record, documented complaint)
- Whistleblower B (background only, fears retaliation)
- Contractor who lost bids (on record)
- Engineering expert for bridge assessment
- Officials (will request comment before publication)

**Documentary evidence:**
- Contract awards (public, obtained)
- Corporate filings (public, obtained)
- Internal emails (FOIA pending)
- Inspection reports (FOIA pending)

**Legal considerations:**
All claims will be documented. We'll seek comment from everyone named. Standard defamation review needed.

**Competitive risk:**
Local TV did a segment on one bridge last year but didn't connect the pattern. This is a deeper investigation.

**Timeline:**
```

```
8-10 weeks total. 6 weeks additional reporting, 2 weeks writing/editing, 2 weeks legal/response period.

**Resources needed:**
$2,000 FOIA costs, data analysis support, one reporting trip to state capital.

**Publication plan:**
Main story (3,000 words) + database for readers to search contracts + methodology box.
```

Op-ed pitch

For opinion/commentary pieces.

```
Subject: Op-ed submission: [Headline]

Dear [Editor name],

[One paragraph: What's your argument? Why now?]

[One paragraph: Why are you qualified to make this argument?]

[One paragraph: Why is this right for your publication's audience?]

The piece is [word count] words and is exclusive to [publication].

[Your name]
[Credentials/bio]
[Contact info]

---

Example:

Subject: Op-ed submission: School board's DEI cuts will hurt all students

Dear Opinion Editor,

When the Westfield school board votes Tuesday to eliminate diversity coordinators, they won't just hurt minority students—they'll deprive every student of the skills employers demand. I've spent 20 years in HR at Fortune 500 companies, and I can tell you: businesses want graduates who can work across difference. This decision makes our kids less competitive.

As the former Chief People Officer at [Company] and a Westfield parent, I've seen both sides of this issue. I'm not an activist; I'm a business leader who knows what the job market requires.

Your editorial page has covered school budget debates extensively. Your readers—many of them parents in our district—need to hear from someone who can translate business needs into educational priorities.

The piece is 750 words and is exclusive to [Publication].

Jane Smith
Former CPO, [Company]
Westfield School District parent
jane@email.com / 555-0123
```

Freelance query letter

For pitching to publications you don't work for.

```
Subject: Pitch: [Compelling headline]

Dear [Editor name],

[Opening hook—the most compelling detail or question from your story]

[What the story is, who it's about, what you'll show]

[Why now—the news hook or timely angle]

[What you have—sources confirmed, access secured, documents obtained]

[Why this publication—how it fits their coverage, audience]

[Why you—relevant clips, expertise, access]

I estimate [word count] words and can deliver in [timeframe].

Thank you for your consideration.

[Your name]
[Brief bio]
[Links to 2-3 relevant clips]
[Contact info]

---

Example:

Subject: Pitch: The nurse who tracks fake diplomas

Dear [Editor],

Maria Chen has caught 47 nurses with fake degrees in three years. None of them through background checks—through a spreadsheet she maintains in her free time.

Chen, a nursing supervisor at a rural Oklahoma hospital, started her database after discovering a colleague had purchased credentials from a diploma mill. State nursing boards, she found, rarely verify foreign credentials thoroughly. Her volunteer work has exposed a patient safety gap that regulators have ignored.

Her story comes as Oklahoma debates licensure reform this session—a bill she's testified for and the hospital lobby has opposed. The vote is in six weeks.

I've interviewed Chen twice, reviewed her database and documentation, and spoken with three of the fraudulent nurses she identified (two on background). The state nursing board has agreed to comment.

This fits [Publication]'s healthcare accountability coverage. Your recent piece on physician oversight touched on similar regulatory gaps.

I've covered healthcare regulation for [outlets] and have sources at several state nursing boards. My recent piece on travel nurse agencies for [outlet] is attached.

I estimate 2,500 words and can deliver in four weeks.

Thank you for your consideration.

[Name]
Freelance health reporter
[Clip links]
[Contact]
```

Developing the angle

Finding the hook

Ask yourself:
- What's the **conflict**? (Who's fighting whom?)
- What's the **tension**? (What's at stake?)
- What's the **surprise**? (What will readers not expect?)
- What's the **emotion**? (What will readers feel?)
- What's the **question**? (What don't we know?)

The "dinner table" test

Can you explain this story in one sentence to someone at dinner?

Too vague: "It's about problems with the foster care system."

Too inside baseball: "It's about how HHS regulations from 2018 created incentive misalignment in Title IV-E funding."

Just right: "Kids in foster care are being moved to different homes so often that some have attended 10 schools in one year—and no one's tracking the damage."

Angle versus topic

Topic	Angle
Homelessness	The family that's been on the housing waitlist for 3 years
Climate change	The town that's already moving because of rising seas
Healthcare [[costs]]	The insulin price that tripled while the patent was extended
Education	The teacher who's spent $10k of her own money on supplies

Topic is what you're covering. **Angle** is how you're covering it.

Common pitch mistakes

The information dump

Wrong: "There's a lot going on with the school board. They've had budget issues, enrollment decline, a superintendent search, and facilities problems. I want to cover all of it."

Better: "The superintendent search is down to two finalists with opposite visions for the district. I'll profile both and explain what's at stake."

The missing news hook

Wrong: "Homelessness is a growing problem in our city."

Better: "The city's eviction filings hit a five-year high last month. Here's what's driving it."

The vague access

Wrong: "I'll talk to homeless people and officials."

Better: "I've been following one family through the shelter system for two months. The housing director has agreed to an interview."

The no-stakes pitch

Wrong: "The museum is having an exhibit on local history."

Better: "The museum's new exhibit includes artifacts a local family claims were stolen during redevelopment. The family wants them back."

After the pitch

If accepted

- Clarify deadline, length, edit process
- Confirm kill fee (for freelance)
- Get editor's preferred contact method
- Ask about style guide, multimedia needs

If rejected

- Ask for feedback (briefly, politely)
- Ask if there's a different angle that would work
- Keep relationship warm for future pitches

If silence

- Follow up once after 1-2 weeks
- If still nothing, you can pitch elsewhere (unless exclusive)
- Don't take it personally—editors are overwhelmed

The pitch sells the story. The story sells itself after that.

subagent-driven-development (Skill Module)

superjawn\skills\subagent-driven-development\SKILL.md

name: subagent-driven-development
description: Use when executing implementation plans with independent tasks in the current session
tags:

Subagent-Driven Development

Execute plan by dispatching fresh subagent per task, with two-stage [[review]] after each: spec compliance [[review]] first, then code quality [[review]].

Why subagents: You delegate tasks to specialized [[agents]] with isolated context. By precisely crafting their instructions and context, you ensure they stay focused and succeed at their task. They should never inherit your session's context or history — you construct exactly what they need. This also preserves your own context for coordination work.

Core principle: Fresh subagent per task + two-stage [[review]] (spec then quality) = high quality, fast iteration

When to Use

```
digraph when_to_use {
    "Have implementation plan?" [shape=diamond];
    "Tasks mostly independent?" [shape=diamond];
    "Stay in this session?" [shape=diamond];
    "subagent-driven-development" [shape=box];
    "executing-plans" [shape=box];
    "Manual execution or brainstorm first" [shape=box];

    "Have implementation plan?" -> "Tasks mostly independent?" [label="yes"];
    "Have implementation plan?" -> "Manual execution or brainstorm first" [label="no"];
    "Tasks mostly independent?" -> "Stay in this session?" [label="yes"];
    "Tasks mostly independent?" -> "Manual execution or brainstorm first" [label="no - tightly coupled"];
    "Stay in this session?" -> "subagent-driven-development" [label="yes"];
    "Stay in this session?" -> "executing-plans" [label="no - parallel session"];
}
```

vs. Executing Plans (parallel session):
- Same session (no context switch)
- Fresh subagent per task (no context pollution)
- Two-stage [[review]] after each task: spec compliance first, then code quality
- Faster iteration (no human-in-loop between tasks)

The Process

```
digraph process {
    rankdir=TB;

    subgraph cluster_per_task {
        label="Per Task";
        "Dispatch implementer subagent (./implementer-prompt.md)" [shape=box];
        "Implementer subagent asks questions?" [shape=diamond];
        "Answer questions, provide context" [shape=box];
        "Implementer subagent implements, tests, commits, self-reviews" [shape=box];
        "Dispatch spec reviewer subagent (./spec-reviewer-prompt.md)" [shape=box];
        "Spec reviewer subagent confirms code matches spec?" [shape=diamond];
        "Implementer subagent fixes spec gaps" [shape=box];
        "Dispatch code quality reviewer subagent (./code-quality-reviewer-prompt.md)" [shape=box];
        "Code quality reviewer subagent approves?" [shape=diamond];
        "Implementer subagent fixes quality issues" [shape=box];
        "Mark task complete in TodoWrite" [shape=box];
    }

    "Read plan, extract all tasks with full text, note context, create TodoWrite" [shape=box];
    "More tasks remain?" [shape=diamond];
    "Dispatch final code reviewer subagent for entire implementation" [shape=box];
    "Use superjawn:finishing-a-development-branch" [shape=box style=filled fillcolor=lightgreen];

    "Read plan, extract all tasks with full text, note context, create TodoWrite" -> "Dispatch implementer subagent (./implementer-prompt.md)";
    "Dispatch implementer subagent (./implementer-prompt.md)" -> "Implementer subagent asks questions?";
    "Implementer subagent asks questions?" -> "Answer questions, provide context" [label="yes"];
    "Answer questions, provide context" -> "Dispatch implementer subagent (./implementer-prompt.md)";
    "Implementer subagent asks questions?" -> "Implementer subagent implements, tests, commits, self-reviews" [label="no"];
    "Implementer subagent implements, tests, commits, self-reviews" -> "Dispatch spec reviewer subagent (./spec-reviewer-prompt.md)";
    "Dispatch spec reviewer subagent (./spec-reviewer-prompt.md)" -> "Spec reviewer subagent confirms code matches spec?";
    "Spec reviewer subagent confirms code matches spec?" -> "Implementer subagent fixes spec gaps" [label="no"];
    "Implementer subagent fixes spec gaps" -> "Dispatch spec reviewer subagent (./spec-reviewer-prompt.md)" [label="re-review"];
    "Spec reviewer subagent confirms code matches spec?" -> "Dispatch code quality reviewer subagent (./code-quality-reviewer-prompt.md)" [label="yes"];
    "Dispatch code quality reviewer subagent (./code-quality-reviewer-prompt.md)" -> "Code quality reviewer subagent approves?";
    "Code quality reviewer subagent approves?" -> "Implementer subagent fixes quality issues" [label="no"];
    "Implementer subagent fixes quality issues" -> "Dispatch code quality reviewer subagent (./code-quality-reviewer-prompt.md)" [label="re-review"];
    "Code quality reviewer subagent approves?" -> "Mark task complete in TodoWrite" [label="yes"];
    "Mark task complete in TodoWrite" -> "More tasks remain?";
    "More tasks remain?" -> "Dispatch implementer subagent (./implementer-prompt.md)" [label="yes"];
    "More tasks remain?" -> "Dispatch final code reviewer subagent for entire implementation" [label="no"];
    "Dispatch final code reviewer subagent for entire implementation" -> "Use superjawn:finishing-a-development-branch";
}
```

Model Selection

Use the least powerful model that can handle each role to conserve cost and increase speed.

Mechanical implementation tasks (isolated functions, clear specs, 1-2 files): use a fast, cheap model. Most implementation tasks are mechanical when the plan is well-specified.

[[Integration]] and judgment tasks (multi-file coordination, pattern matching, debugging): use a standard model.

Architecture, design, and [[review]] tasks: use the most capable available model.

Task complexity signals:
- Touches 1-2 files with a complete spec → cheap model
- Touches multiple files with [[integration]] concerns → standard model
- Requires design judgment or broad codebase understanding → most capable model

Handling Implementer [[Status]]

Implementer subagents [[[[report]]]] one of four statuses. Handle each appropriately:

DONE: Proceed to spec compliance [[review]].

DONE_WITH_CONCERNS: The implementer completed the work but flagged doubts. Read the concerns before proceeding. If the concerns are about correctness or scope, address them before [[review]]. If they're observations (e.g., "this file is getting large"), note them and proceed to [[review]].

NEEDS_CONTEXT: The implementer needs information that wasn't provided. Provide the missing context and re-dispatch.

BLOCKED: The implementer cannot complete the task. Assess the blocker:
1. If it's a context problem, provide more context and re-dispatch with the same model
2. If the task requires more reasoning, re-dispatch with a more capable model

3. If the task is too large, break it into smaller pieces
4. If the plan itself is wrong, escalate to the human

Never ignore an escalation or force the same model to retry without changes. If the implementer said it's stuck, something needs to change.

Prompt Templates

- `./[[implementer-prompt]].md` - Dispatch implementer subagent
- `./[[spec-reviewer-prompt]].md` - Dispatch spec compliance reviewer subagent
- `./[[code-quality-reviewer-prompt]].md` - Dispatch code quality reviewer subagent

Example Workflow

```
You: I'm using Subagent-Driven Development to execute this plan.

[Read plan file once: docs/superpowers/plans/feature-plan.md]
[Extract all 5 tasks with full text and context]
[Create TodoWrite with all tasks]

Task 1: Hook installation script

[Get Task 1 text and context (already extracted)]
[Dispatch implementation subagent with full task text + context]

Implementer: "Before I begin - should the hook be installed at user or system level?"

You: "User level (~/.config/superpowers/hooks/)"

Implementer: "Got it. Implementing now..."
[Later] Implementer:
  - Implemented install-hook command
  - Added tests, 5/5 passing
  - Self-review: Found I missed --force flag, added it
  - Committed

[Dispatch spec compliance reviewer]
Spec reviewer:  Spec compliant - all requirements met, nothing extra

[Get git SHAs, dispatch code quality reviewer]
Code reviewer: Strengths: Good test coverage, clean. Issues: None. Approved.

[Mark Task 1 complete]

Task 2: Recovery modes

[Get Task 2 text and context (already extracted)]
[Dispatch implementation subagent with full task text + context]

Implementer: [No questions, proceeds]
Implementer:
  - Added verify/repair modes
  - 8/8 tests passing
  - Self-review: All good
  - Committed

[Dispatch spec compliance reviewer]
Spec reviewer:  Issues:
  - Missing: Progress reporting (spec says "report every 100 items")
  - Extra: Added --json flag (not requested)

[Implementer fixes issues]
Implementer: Removed --json flag, added progress reporting

[Spec reviewer reviews again]
Spec reviewer:  Spec compliant now

[Dispatch code quality reviewer]
Code reviewer: Strengths: Solid. Issues (Important): Magic number (100)

[Implementer fixes]
Implementer: Extracted PROGRESS_INTERVAL constant

[Code reviewer reviews again]
Code reviewer:  Approved

[Mark Task 2 complete]

...

[After all tasks]
[Dispatch final code-reviewer]
Final reviewer: All requirements met, ready to merge

Done!
```

Advantages

vs. Manual execution:
- Subagents follow TDD naturally
- Fresh context per task (no confusion)
- Parallel-safe (subagents don't interfere)
- Subagent can ask questions (before AND during work)

vs. Executing Plans:
- Same session (no [[handoff]])
- Continuous progress (no waiting)
- [[Review]] checkpoints automatic

Efficiency gains:
- No file reading overhead (controller provides full text)
- Controller curates exactly what context is needed
- Subagent gets complete information upfront
- Questions surfaced before work begins (not after)

Quality gates:
- Self-[[review]] catches issues before [[handoff]]
- Two-stage [[review]]: spec compliance, then code quality
- [[Review]] loops ensure fixes actually work
- Spec compliance prevents over/under-building
- Code quality ensures implementation is well-built

Cost:
- More subagent invocations (implementer + 2 reviewers per task)
- Controller does more prep work (extracting all tasks upfront)
- [[Review]] loops add iterations
- But catches issues early (cheaper than debugging later)

Red Flags

Never:
- Start implementation on main/master branch without explicit user consent
- Skip reviews (spec compliance OR code quality)
- Proceed with unfixed issues
- Dispatch multiple implementation subagents in parallel (conflicts)
- Make subagent read plan file (provide full text instead)
- Skip scene-setting context (subagent needs to understand where task fits)
- Ignore subagent questions (answer before letting them proceed)
- Accept "close enough" on spec compliance (spec reviewer found issues = not done)
- Skip [[review]] loops (reviewer found issues = implementer fixes = [[review]] again)
- Let implementer self-[[review]] replace actual [[review]] (both are needed)
- **Start code quality [[review]] before spec compliance is ** (wrong order)
- Move to next task while either [[review]] has open issues

If subagent asks questions:
- Answer clearly and completely
- Provide additional context if needed
- Don't rush them into implementation

If reviewer finds issues:
- Implementer (same subagent) fixes them
- Reviewer reviews again
- Repeat until approved
- Don't skip the re-[[review]]

If subagent fails task:
- Dispatch fix subagent with specific instructions
- Don't try to fix manually (context pollution)

[[Integration]]

Required workflow [[skills]]:
- **superjawn:using-git-worktrees** - REQUIRED: Set up isolated workspace before starting
- **superjawn:writing-plans** - Creates the plan this [[skill]] executes
- **superjawn:requesting-code-[[review]]** - Code [[review]]
[[[

for reviewer subagents
- **superjawn:finishing-a-development-branch** - Complete development after all tasks

Subagents should use:
- **superjawn:test-driven-development** - Subagents follow TDD for each task

Alternative workflow:
- **superjawn:executing-plans** - Use for parallel session instead of same-session execution

supply-chain-hardening (Skill Module)

security-toolkit\skills\supply-chain-hardening\SKILL.md

name: supply-chain-hardening

description: Configure install-time cooldowns for npm/bun (minimum release age) and run a sandboxed pre-install scan when the cooldown has to be bypassed. Use when the user asks about supply-chain attacks, npm/bun security, "minimum release age", a "cooldown" for installs, hardening against Shai-Hulud-class worms, or how to safely install a package that was just published. Also use after any recent supply-chain incident in the npm ecosystem.

tags:

Supply-chain hardening

Defends a journalism toolchain against the dominant npm/bun supply-chain attack pattern: a maintainer account or CI pipeline is compromised, a malicious version ships, and machines install it before anyone notices. Recent example: the **Mini Shai-Hulud TanStack attack (2026-05-11)** compromised 84 versions across 42 `@tanstack/*` packages and exfiltrated AWS / GCP / Vault / GitHub / SSH credentials via a postinstall script.

The defense is **layered** and intentionally simple:

1. **Install-time cooldown** — only install package versions older than N days (default 7). This is the primary defense. By the time the cooldown expires, the security community has almost always flagged a compromised version and the registry has yanked it.
2. **Sandboxed pre-install scan** — when the cooldown has to be bypassed (CVE patch, fresh dep, urgent install), run the candidate tarball through a static-analysis scan that looks for the diagnostic signatures of supply-chain malware. The scan runs inside `bwrap` / `firejail` / `unshare` so a malicious package can't escape the inspection.
3. **`--ignore-scripts` at install** — postinstall is the #1 attack vector. Skip lifecycle scripts on every cooldown-bypass install.

These three together would have blocked the Mini Shai-Hulud TanStack attack on a stock laptop with no human in the loop.

Configure the cooldown

Verified config keys (npm v11+ and bun 1.3+):

Manager	File	Key	Units	Exclusion key
npm	`~/.npmrc` (or project `.npmrc`)	`min-release-age`	days	none yet — proposed in npm/cli#8994
bun	`~/.bunfig.toml` (or project `bunfig.toml`)	`[install] minimumReleaseAge`	seconds	`[install] minimumReleaseAgeExcludes = []` (exact names, no globs)

Minimal config:

```
# ~/.npmrc
min-release-age=7
```

```
# ~/.bunfig.toml
[install]
minimumReleaseAge = 604800  # 7 days
minimumReleaseAgeExcludes = []
```

Requires npm 11+. Older npm silently ignores unknown keys, so the config looks correct but does nothing. Check with `npm --version` and `npm config get min-release-age` (should echo `7`, not `null`).

Per-command bypass

When the cooldown blocks an install you actually want:

```
npm install <pkg>@<version> --min-release-age=0 --ignore-scripts
bun add     <pkg>@<version> --minimum-release-age=0 --ignore-scripts
```

The `bun add --minimum-release-age=0` CLI flag works in 1.3+ even though the docs don't list it — it follows bun's `bunfig key → kebab-case flag` convention.

Always pair the bypass with `--ignore-scripts`. Postinstall is the most common payload-execution path in supply-chain malware (Mini Shai-Hulud, [[[[event]]]]-stream, ua-parser-js, coa, all used it). Native modules that legitimately need postinstall can have the script run manually after a human-readable [[review]]:

```
(cd node_modules/<pkg> && cat package.json | jq .scripts) # eyeball it
(cd node_modules/<pkg> && npm run postinstall)            # run if it checks out
```

When to scan before bypassing

The scan is for the dangerous moment: you've decided to bypass the cooldown and need a sanity check. The [[skill]] ships a reference script (`scripts/[[hotpatch]].example.sh`) implementing the heuristics. Adapt it to your machine — Bash assumes `bwrap` (Linux); macOS users substitute `sandbox-exec` or skip the sandbox layer with the trade-off documented.

Static checks the scan should perform (each backed by a real attack):

Check	Diagnostic of	Severity
`optionalDependencies` / `dependencies` containing `github:` or `git+` URLs	Mini Shai-Hulud (delivered payload via `github:tanstack/router#<sha>` ref)	RED
Large JS file at package root not referenced by `main` / `module` / `exports` / `bin` / `files`	Planted payload pattern (`router_init.js` in Mini Shai-Hulud)	RED
Unpacked size >3x the prior stable version	Bulk payload smuggling	RED
`fileCount` delta of 1–4 paired with >2x size jump	Single planted file	RED
`preinstall` / `install` / `postinstall` / `prepare` scripts present	Lifecycle-script attack vector ([[[[event]]]]-stream, ua-parser-js, etc.)	YELLOW
JS files referencing `.ssh/` , `.aws/` , `.npmrc` , `GITHUB_TOKEN` , `AWS_SECRET` , kube config	Credential exfiltration	YELLOW
Version flagged `deprecated` in npm registry with "security"/"compromised"/"malicious" wording	Maintainer/registry yank	RED
OSV.dev returns known vulnerabilities for `<pkg>@<version>`	Disclosed CVE	RED (severity-dependent)

Why prerelease versions are skipped from the size-delta baseline: dev/beta/rc versions have wildly different sizes than stable releases and produce false positives.

What the cooldown does *not* catch

Be honest about the limits with whoever you're configuring this for:

- **Old packages with new malicious versions still in the cooldown window are blocked**, but if the bad version *also* passes the cooldown (rare but possible — a compromise that goes >7 days undetected), the cooldown alone won't help. The scan catches most of those.
- **Transitive deps**. A clean `<pkg>` you install can pull in a compromised transitive. Defenses: scan against the *resolved* tree (`npm audit` , `osv-scanner`), and keep the cooldown active globally so transitive resolution also waits.
- `npm ci` against an existing lockfile. The cooldown applies during *resolution*, not installation of already-pinned versions. If your lockfile pins a compromised version, `npm ci` will install it. Mitigation: scan lockfiles in CI with `osv-scanner --lockfile=package-lock.json` .
- **Pre-existing compromised packages in** `node_modules` . Hardening protects future installs, not past ones. Audit existing deps separately (`npm audit` , `osv-scanner` , manual [[review]] of recently-published deps in your tree).

Quick-start workflow for a new machine

1. Verify npm >= 11: `npm --version` . If older, upgrade (`sudo npm i -g npm@latest` or tarball-swap if self-upgrade races).
2. Write `~/.npmrc` and `~/.bunfig.toml` with the config above.
3. Verify: `npm config get min-release-age` returns `7` . `cat ~/.bunfig.toml` shows the `[install]` block.
4. Copy `scripts/[[hotpatch]].example.sh` to `~/.claude/[[hotpatch]].sh` (or wherever fits). Make executable. Run `./[[hotpatch]].sh --self-test` against the synthetic Mini Shai-Hulud fixture (also shipped) to confirm the heuristics fire.
5. Document the bypass workflow somewhere your team will find it. The whole [[skill]] assumes the bypass is *rare* and *reviewed*, not the default.

Threat model: what this defends and what it doesn't

Defends against	Doesn't defend against
Maintainer account compromise (npm token theft)	Targeted attack tailored to wait through the cooldown
CI/CD pipeline hijack (Mini Shai-Hulud, valid OIDC tokens, SLSA-attested malice)	Compromise of a transitive dep already pinned in a lockfile
Typosquatting (lookalike package names) — when paired with `npm pkg fix` and lockfile [[review]]	Malicious code in your own dev dependencies that you authored
Postinstall payload execution (cooldown + `--ignore-scripts` = belt and suspenders)	Runtime supply-chain attacks (e.g., dynamic loading of bad code from a CDN)
Drive-by `npm install` of a brand-new transitive	Compromise of the registry itself (very rare; out of scope)

Further reading

- Original Mini Shai-Hulud StepSecurity analysis (TanStack attack, 2026-05-11): https://www.stepsecurity.io/blog/mini-shai-hulud-is-back-a-self-spreading-supply-chain-attack-hits-the-npm-ecosystem
- npm `min-release-age` config: https://docs.npmjs.com/cli/v11/using-npm/config

- bun `minimumReleaseAge` config: https://bun.com/docs/runtime/bunfig
- OSV.dev API (free, unauth): https://osv.dev/docs/#tag/api
- Earlier Shai-Hulud worm context: https://github.com/advisories?query=shai-hulud

systematic-debugging (Skill Module)

`superjawn\skills\systematic-debugging\SKILL.md`

name: systematic-debugging
description: Use when encountering any bug, test failure, or unexpected behavior, before proposing fixes
tags:

Systematic Debugging

Overview

Random fixes waste time and create new bugs. Quick patches mask underlying issues.

Core principle: ALWAYS find root cause before attempting fixes. Symptom fixes are failure.

Violating the letter of this process is violating the spirit of debugging.

The Iron Law

```
NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST
```

If you haven't completed Phase 1, you cannot propose fixes.

When to Use

Use for ANY technical issue:
- Test failures
- Bugs in production
- Unexpected behavior
- Performance problems
- Build failures
- [[Integration]] issues

Use this ESPECIALLY when:
- Under time pressure (emergencies make guessing tempting)
- "Just one quick fix" seems obvious
- You've already tried multiple fixes
- Previous fix didn't work
- You don't fully understand the issue

Don't skip when:
- Issue seems simple (simple bugs have root causes too)
- You're in a hurry (rushing guarantees rework)
- Manager wants it fixed NOW (systematic is faster than thrashing)

The Four Phases

Move through the phases in order. Phase 1 is mandatory; if it produces a clear root cause and a confident fix, you may go directly to Phase 4 (Implementation). Otherwise complete Phase 2 (Pattern Analysis) and Phase 3 (Hypotheses) in order before Phase 4. Between Phase 1 and Phase 2 — when you've decided pattern analysis is needed — run the research phase (default-on, see below) so pattern analysis has external context to build on.

Phase 1: Root Cause Investigation

BEFORE attempting ANY fix:

1. **Read Error Messages Carefully**
   - Don't skip past errors or warnings
   - They often contain the exact solution
   - Read stack traces completely
   - Note line numbers, file paths, error codes

2. **Reproduce Consistently**
   - Can you trigger it reliably?
   - What are the exact steps?

- Does it happen every time?
- If not reproducible → gather more data, don't guess

3. **Check Recent Changes**
   - What changed that could cause this?
   - Git diff, recent commits
   - New dependencies, config changes
   - Environmental differences

4. **Gather Evidence in Multi-Component Systems**

WHEN system has multiple components (CI → build → signing, API → service → database):

BEFORE proposing fixes, add diagnostic instrumentation:
```
For EACH component boundary:
- Log what data enters component
- Log what data exits component
- Verify environment/config propagation
- Check state at each layer

Run once to gather evidence showing WHERE it breaks
THEN analyze evidence to identify failing component
THEN investigate that specific component
```

Example (multi-layer system):
```bash
# Layer 1: Workflow
echo "=== Secrets available in workflow: ==="
echo "IDENTITY: ${IDENTITY:+SET}${IDENTITY:-UNSET}"

# Layer 2: Build script
echo "=== Env vars in build script: ==="
env | grep IDENTITY || echo "IDENTITY not in environment"

# Layer 3: Signing script
echo "=== Keychain state: ==="
security list-keychains
security find-identity -v

# Layer 4: Actual signing
codesign --sign "$IDENTITY" --verbose=4 "$APP"
```

This reveals: Which layer fails (secrets → workflow , workflow → build)

5. **Trace Data Flow**

WHEN error is deep in call stack:

See `[[root-cause-tracing]].md` in this directory for the complete backward tracing technique.

Quick version:
- Where does bad value originate?
- What called this with bad value?
- Keep tracing up until you find the source
- Fix at source, not at symptom

Research phase

After Phase 1 (Root Cause Investigation) and before Phase 2 (Pattern Analysis), gather outside context. Phase 1 produced internal evidence (error messages, repro steps, recent diffs); the research phase adds external information that pattern analysis can build on.

Default-on. Skip only with explicit, justified statement per the skip protocol below.

This phase only fires when entering Phase 2. If Phase 1 yielded the root cause directly and you're going Phase 1 → Phase 4 (Implementation), you don't reach the research phase. The skip protocol governs the case where you've decided pattern analysis is needed but want to skip the research that would inform it.

1. Default research kinds (all four)

Kind	Purpose	Tool
Web	Search the literal error string and the framework/library's open GitHub issues for prior reports	WebSearch + WebFetch (via subagent)
Codebase prior-bugs	`git log --grep` for related historical fixes; spots regressions and prior work in the same area	Bash + Grep (via subagent)
Authoritative	Fetch current live docs/spec for the API or library involved; catches "am I using this wrong" cases	WebFetch (via subagent)
User-context	Check `MEMORY.md` for related debugging history	Read (inline, no subagent)

2. Dispatch

Three subagents in parallel (web via `general-purpose`, codebase prior-bugs via `Explore`, authoritative via `general-purpose`) plus the inline memory check running concurrently in the main thread. The subagent [[[[prompts]]]] each carry the bug context from Phase 1 (error message, repro, current diff) so they don't have to rediscover it.

3. Findings location

Findings land in `.[[superpowers]]/debug-log-<slug>.md` where `<slug>` is `YYYY-MM-DD-<short-bug-description>`. Examples:

```
.superpowers/debug-log-2026-05-05-test-failure-auth-handler.md
.superpowers/debug-log-2026-05-12-build-fails-on-arm64.md
.superpowers/debug-log-2026-06-03-flaky-redis-pubsub.md
```

The [[skill]] creates the file on first invocation if it doesn't exist; subsequent invocations on the same bug append. Each entry includes:

- Date/time of the research run
- Which research kinds fired (or were skipped, with the locked skip-justification line)
- Findings: 3-5 bullets per kind that fired, including load-bearing links/refs
- "Considered but ruled out" notes so future-you knows what was checked

Add `.[[superpowers]]/` to your project's `.gitignore` so debug logs don't get accidentally committed. The upstream `[[superpowers]]` plugin uses this convention; superjawn follows the same pattern.

4. Skip protocol

If skipping, write one line to `.[[superpowers]]/debug-log-<slug>.md`: `Skipped research because <reason>. <Verifiable pointer if applicable>.`

Valid reasons:
- Trivial scope (typo, comment edit, single-line config)
- Fresh prior research — same topic in current session OR within last 7 days with verifiable spec/plan pointer. **If the pointer doesn't resolve, the skip is invalid.** (Beyond 7 days, repeat the research even if you remember the prior findings — the landscape drifts.)
- User explicit — **must quote the phrase** that authorized the skip.
- Repeat of identical task — **must include a pointer** to the prior successful run.

Invalid reasons: "I think I know", "seems straightforward", "moving fast", "user wants this done quickly", "already familiar with this codebase". If those are tempting, do the research.

Phase 2: Pattern Analysis

Find the pattern before fixing:

1. **Find Working Examples**
   - Locate similar working code in same codebase
   - What works that's similar to what's broken?
2. **Compare Against References**
   - If implementing pattern, read reference implementation COMPLETELY
   - Don't skim - read every line
   - Understand the pattern fully before applying
3. **Identify Differences**
   - What's different between working and broken?
   - List every difference, however small
   - Don't assume "that can't matter"
4. **Understand Dependencies**
   - What other components does this need?
   - What settings, config, environment?
   - What assumptions does it make?

Phase 3: Hypothesis and Testing

Scientific method:

1. **Form Single Hypothesis**
   - State clearly: "I think X is the root cause because Y"
   - Write it down
   - Be specific, not vague

2. **Test Minimally**
   - Make the SMALLEST possible change to test hypothesis
   - One variable at a time
   - Don't fix multiple things at once

3. **Verify Before Continuing**
   - Did it work? Yes → Phase 4
   - Didn't work? Form NEW hypothesis
   - DON'T add more fixes on top

4. **When You Don't Know**
   - Say "I don't understand X"
   - Don't pretend to know
   - Ask for help
   - Research more

Phase 4: Implementation

Fix the root cause, not the symptom:

1. **Create Failing Test Case**
   - Simplest possible reproduction
   - Automated test if possible
   - One-off test script if no framework
   - MUST have before fixing
   - Use the `superjawn:test-driven-development` [[skill]] for writing proper failing tests

2. **Implement Single Fix**
   - Address the root cause identified
   - ONE change at a time
   - No "while I'm here" improvements
   - No bundled refactoring

3. **Verify Fix**
   - Test passes now?
   - No other tests broken?
   - Issue actually resolved?

4. **If Fix Doesn't Work**
   - STOP
   - Count: How many fixes have you tried?
   - If < 3: Return to Phase 1, re-analyze with new information
   - **If ≥ 3: STOP and question the architecture (step 5 below)**
   - DON'T attempt Fix #4 without architectural discussion

5. **If 3+ Fixes Failed: Question Architecture**

Pattern indicating architectural problem:
- Each fix reveals new shared state/coupling/problem in different place
- Fixes require "massive refactoring" to implement
- Each fix creates new symptoms elsewhere

STOP and question fundamentals:
- Is this pattern fundamentally sound?
- Are we "sticking with it through sheer inertia"?
- Should we refactor architecture vs. continue fixing symptoms?

Discuss with your human partner before attempting more fixes

This is NOT a failed hypothesis - this is a wrong architecture.

Red Flags - STOP and Follow Process

If you catch yourself thinking:
- "Quick fix for now, investigate later"
- "Just try changing X and see if it works"
- "Add multiple changes, run tests"
- "Skip the test, I'll manually verify"
- "It's probably X, let me fix that"
- "I don't fully understand but this might work"
- "Pattern says X but I'll adapt it differently"
- "Here are the main problems: [lists fixes without investigation]"
- Proposing solutions before tracing data flow
- **"One more fix attempt" (when already tried 2+)**
- **Each fix reveals new problem in different place**

ALL of these mean: STOP. Return to Phase 1.

If 3+ fixes failed: Question the architecture (see Phase 4.5)

your human partner's Signals You're Doing It Wrong

Watch for these redirections:
- "Is that not happening?" - You assumed without verifying
- "Will it show us...?" - You should have added evidence gathering
- "Stop guessing" - You're proposing fixes without understanding
- "Ultrathink this" - Question fundamentals, not just symptoms
- "We're stuck?" (frustrated) - Your approach isn't working

When you see these: STOP. Return to Phase 1.

Common Rationalizations

Excuse	Reality
"Issue is simple, don't need process"	Simple issues have root causes too. Process is fast for simple bugs.
"Emergency, no time for process"	Systematic debugging is FASTER than guess-and-check thrashing.
"Just try this first, then investigate"	First fix sets the pattern. Do it right from the start.
"I'll write test after confirming fix works"	Untested fixes don't stick. Test first proves it.
"Multiple fixes at once saves time"	Can't isolate what worked. Causes new bugs.
"Reference too long, I'll adapt the pattern"	Partial understanding guarantees bugs. Read it completely.
"I see the problem, let me fix it"	Seeing symptoms ≠ understanding root cause.
"One more fix attempt" (after 2+ failures)	3+ failures = architectural problem. Question pattern, don't fix again.

Quick Reference

Phase	Key Activities	Success Criteria
1. Root Cause	Read errors, reproduce, check changes, gather evidence	Understand WHAT and WHY
2. Pattern	Find working examples, compare	Identify differences
3. Hypothesis	Form theory, test minimally	Confirmed or new hypothesis
4. Implementation	Create test, fix, verify	Bug resolved, tests pass

When Process Reveals "No Root Cause"

If systematic investigation reveals issue is truly environmental, timing-dependent, or external:

1. You've completed the process
2. Document what you investigated
3. Implement appropriate handling (retry, timeout, error message)
4. Add monitoring/logging for future investigation

But: 95% of "no root cause" cases are incomplete investigation.

Supporting Techniques

These techniques are part of systematic debugging and available in this directory:

- `[[root-cause-tracing]].md` - Trace bugs backward through call stack to find original trigger
- `[[defense-in-depth]].md` - Add validation at multiple layers after finding root cause
- `[[condition-based-waiting]].md` - Replace arbitrary timeouts with condition polling

Related [[skills]]:
- **superjawn:test-driven-development** - For creating failing test case (Phase 4, Step 1)
- **superjawn:verification-before-completion** - Verify fix worked before claiming success

Real-World Impact

From debugging sessions:
- Systematic approach: 15-30 minutes to fix
- Random fixes approach: 2-3 hours of thrashing
- First-time fix rate: 95% vs 40%
- New bugs introduced: Near zero vs common

template-selector (Skill Module)

`project-templates-toolkit\skills\template-selector\SKILL.md`

name: template-selector

description: Choose the correct CLAUDE.md or LESSONS.md template for journalism projects. Use when starting a new project, setting up documentation, or unsure which template category fits best. Provides decision trees and selection guidance for 6 journalism-focused template types.

tags:

[[[[

- Helps with a specific editorial workflow step
- Requires accuracy/trust for [[[[publication]]]] decisions

[[[[[[event]]]]-website]]

Use when: Building sites for conferences, summits, workshops, or events with specific dates

Not for: Ongoing content, organization websites without events, [[[[event]]]] management software (that's [[[[editorial-tool]]]])

Signs it's an [[[[event]]]] website:
- Has a specific [[[[event]]]] date (or date range)
- Registration/attendance is involved
- Content includes speakers, schedule, venue info

[[[[publication]]]]

Use when: Building newsletters, podcasts, blogs, magazines, or ongoing content series with regular publishing schedule

Not for: One-off articles, static content sites, CMSes that power publications (that's [[[[editorial-tool]]]])

Signs it's a [[[[publication]]]]:
- Regular publishing cadence (daily, weekly, monthly)
- Growing subscriber/reader base
- Consistent voice and format

[[[[research-project]]]]

Use when: Building investigative journalism projects, data journalism, analysis with defined scope and end date

Not for: Ongoing research operations, general data analysis, tools that support research (that's [[[[editorial-tool]]]])

Signs it's a research project:
- Answers a specific question or hypothesis
- Has a defined end point ([[[[publication]]]] date)
- Involves data collection, analysis, and findings

[[content-pipeline]]

Use when: Building CMS [[[[[[[

The litmus test

If you're still unsure, ask:

1. **Is this a tool that helps journalists do X, or is this X itself?**
   - Tool → [[[[editorial-tool]]]]
   - The thing itself → other categories
2. **Does it have a specific end date?**
   - Yes, [[[[event]]]] dates → [[[[[[event]]]]-website]]
   - Yes, [[[[publication]]]] date → [[[[research-project]]]]
   - No, ongoing → [[[[publication]]]], [[content-pipeline]], or [[digital-archive]]
3. **Is preservation the primary goal?**
   - Yes → [[digital-archive]]
   - No → other categories

Using templates

For project memory (CLAUDE.md)

1. Identify project category using decision tree above
2. Copy appropriate [[[[[[

test-driven-development (Skill Module)

superjawn\skills\test-driven-development\SKILL.md

name: test-driven-development
description: Use when implementing any feature or bugfix, before writing implementation code
tags:

Test-Driven Development (TDD)

Overview

Write the test first. Watch it fail. Write minimal code to pass.

Core principle: If you didn't watch the test fail, you don't know if it tests the right thing.

Violating the letter of the rules is violating the spirit of the rules.

When to Use

Always:
- New features
- Bug fixes
- Refactoring
- Behavior changes

Exceptions (ask your human partner):
- Throwaway prototypes
- Generated code
- Configuration files

Thinking "skip TDD just this once"? Stop. That's rationalization.

The Iron Law

```
NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST
```

Write code before the test? Delete it. Start over.

No exceptions:
- Don't keep it as "reference"
- Don't "adapt" it while writing tests
- Don't look at it
- Delete means delete

Implement fresh from tests. Period.

Red-Green-Refactor

```
digraph tdd_cycle {
    rankdir=LR;
    red [label="RED\nWrite failing test", shape=box, style=filled, fillcolor="#ffcccc"];
    verify_red [label="Verify fails\ncorrectly", shape=diamond];
    green [label="GREEN\nMinimal code", shape=box, style=filled, fillcolor="#ccffcc"];
    verify_green [label="Verify passes\nAll green", shape=diamond];
    refactor [label="REFACTOR\nClean up", shape=box, style=filled, fillcolor="#ccccff"];
    next [label="Next", shape=ellipse];

    red -> verify_red;
    verify_red -> green [label="yes"];
    verify_red -> red [label="wrong\nfailure"];
    green -> verify_green;
    verify_green -> refactor [label="yes"];
    verify_green -> green [label="no"];
    refactor -> verify_green [label="stay\ngreen"];
    verify_green -> next;
    next -> red;
}
```

RED - Write Failing Test

Write one minimal test showing what should happen.

```
test('retries failed operations 3 times', async () => {
  let attempts = 0;
  const operation = () => {
    attempts++;
    if (attempts < 3) throw new Error('fail');
    return 'success';
  };

  const result = await retryOperation(operation);

  expect(result).toBe('success');
  expect(attempts).toBe(3);
});
```

Clear name, tests real behavior, one thing

```
test('retry works', async () => {
  const mock = jest.fn()
    .mockRejectedValueOnce(new Error())
    .mockRejectedValueOnce(new Error())
    .mockResolvedValueOnce('success');
  await retryOperation(mock);
  expect(mock).toHaveBeenCalledTimes(3);
});
```

Vague name, tests mock not code

Requirements:
- One behavior
- Clear name
- Real code (no mocks unless unavoidable)

Verify RED - Watch It Fail

MANDATORY. Never skip.

```
npm test path/to/test.test.ts
```

Confirm:
- Test fails (not errors)
- Failure message is expected
- Fails because feature missing (not typos)

Test passes? You're testing existing behavior. Fix test.

Test errors? Fix error, re-run until it fails correctly.

GREEN - Minimal Code

Write simplest code to pass the test.

```
async function retryOperation<T>(fn: () => Promise<T>): Promise<T> {
  for (let i = 0; i < 3; i++) {
    try {
      return await fn();
    } catch (e) {
      if (i === 2) throw e;
    }
  }
  throw new Error('unreachable');
}
```

Just enough to pass

```
async function retryOperation<T>(
  fn: () => Promise<T>,
  options?: {
    maxRetries?: number;
    backoff?: 'linear' | 'exponential';
    onRetry?: (attempt: number) => void;
  }
): Promise<T> {
  // YAGNI
}
```

Over-engineered

Don't add features, refactor other code, or "improve" beyond the test.

Verify GREEN - Watch It Pass

MANDATORY.

```
npm test path/to/test.test.ts
```

Confirm:
- Test passes
- Other tests still pass
- Output pristine (no errors, warnings)

Test fails? Fix code, not test.

Other tests fail? Fix now.

REFACTOR - Clean Up

After green only:
- Remove duplication
- Improve names
- Extract helpers

Keep tests green. Don't add behavior.

Repeat

Next failing test for next feature.

Good Tests

Quality	Good	Bad
Minimal	One thing. "and" in name? Split it.	`test('validates email and domain and whitespace')`
Clear	Name describes behavior	`test('test1')`
Shows intent	Demonstrates desired API	Obscures what code should do

Why Order Matters

"I'll write tests after to verify it works"

Tests written after code pass immediately. Passing immediately proves nothing:
- Might test wrong thing
- Might test implementation, not behavior
- Might miss edge cases you forgot
- You never saw it catch the bug

Test-first forces you to see the test fail, proving it actually tests something.

"I already manually tested all the edge cases"

Manual testing is ad-hoc. You think you tested everything but:
- No record of what you tested
- Can't re-run when code changes
- Easy to forget cases under pressure
- "It worked when I tried it" ≠ comprehensive

Automated tests are systematic. They run the same way every time.

"Deleting X hours of work is wasteful"

Sunk cost fallacy. The time is already gone. Your choice now:
- Delete and rewrite with TDD (X more hours, high confidence)
- Keep it and add tests after (30 min, low confidence, likely bugs)

The "waste" is keeping code you can't trust. Working code without real tests is technical debt.

"TDD is dogmatic, being pragmatic means adapting"

TDD IS pragmatic:
- Finds bugs before commit (faster than debugging after)
- Prevents regressions (tests catch breaks immediately)
- Documents behavior (tests show how to use code)
- Enables refactoring (change freely, tests catch breaks)

"Pragmatic" shortcuts = debugging in production = slower.

"Tests after achieve the same goals - it's spirit not ritual"

No. Tests-after answer "What does this do?" Tests-first answer "What should this do?"

Tests-after are biased by your implementation. You test what you built, not what's required. You verify remembered edge cases, not discovered ones.

Tests-first force edge case discovery before implementing. Tests-after verify you remembered everything (you didn't).

30 minutes of tests after ≠ TDD. You get coverage, lose proof tests work.

Common Rationalizations

Excuse	Reality
"Too simple to test"	Simple code breaks. Test takes 30 seconds.
"I'll test after"	Tests passing immediately prove nothing.
"Tests after achieve same goals"	Tests-after = "what does this do?" Tests-first = "what should this do?"
"Already manually tested"	Ad-hoc ≠ systematic. No record, can't re-run.
"Deleting X hours is wasteful"	Sunk cost fallacy. Keeping unverified code is technical debt.
"Keep as reference, write tests first"	You'll adapt it. That's testing after. Delete means delete.
"Need to explore first"	Fine. Throw away exploration, start with TDD.
"Test hard = design unclear"	Listen to test. Hard to test = hard to use.
"TDD will slow me down"	TDD faster than debugging. Pragmatic = test-first.
"Manual test faster"	Manual doesn't prove edge cases. You'll re-test every change.
"Existing code has no tests"	You're improving it. Add tests for existing code.

Red Flags - STOP and Start Over

- Code before test
- Test after implementation
- Test passes immediately
- Can't explain why test failed

- Tests added "later"
- Rationalizing "just this once"
- "I already manually tested it"
- "Tests after achieve the same purpose"
- "It's about spirit not ritual"
- "Keep as reference" or "adapt existing code"
- "Already spent X hours, deleting is wasteful"
- "TDD is dogmatic, I'm being pragmatic"
- "This is different because..."

All of these mean: Delete code. Start over with TDD.

Example: Bug Fix

Bug: Empty email accepted

RED

```
test('rejects empty email', async () => {
  const result = await submitForm({ email: '' });
  expect(result.error).toBe('Email required');
});
```

Verify RED

```
$ npm test
FAIL: expected 'Email required', got undefined
```

GREEN

```
function submitForm(data: FormData) {
  if (!data.email?.trim()) {
    return { error: 'Email required' };
  }
  // ...
}
```

Verify GREEN

```
$ npm test
PASS
```

REFACTOR
Extract validation for multiple fields if needed.

Verification Checklist

Before marking work complete:

- [ ] Every new function/method has a test
- [ ] Watched each test fail before implementing
- [ ] Each test failed for expected reason (feature missing, not typo)
- [ ] Wrote minimal code to pass each test
- [ ] All tests pass
- [ ] Output pristine (no errors, warnings)
- [ ] Tests use real code (mocks only if unavoidable)
- [ ] Edge cases and errors covered

Can't check all boxes? You skipped TDD. Start over.

When Stuck

Problem	Solution
Don't know how to test	Write wished-for API. Write assertion first. Ask your human partner.
Test too complicated	Design too complicated. Simplify interface.
Must mock everything	Code too coupled. Use dependency injection.
Test [[setup]] huge	Extract helpers. Still complex? Simplify design.

Debugging [[Integration]]

Bug found? Write failing test reproducing it. Follow TDD cycle. Test proves fix and prevents regression.

Never fix bugs without a test.

Testing Anti-Patterns

When adding mocks or test utilities, read @[[testing-anti-patterns]].md to avoid common pitfalls:
- Testing mock behavior instead of real behavior
- Adding test-only methods to production classes
- Mocking without understanding dependencies

Final Rule

```
Production code → test exists and failed first
Otherwise → not TDD
```

No exceptions without your human partner's permission.

test-first-bugs (Skill Module)

dev-toolkit\skills\test-first-bugs\SKILL.md

name: test-first-bugs

description: This skill should be used when the user reports a bug, describes unexpected behavior, says something is "broken", "not working", "failing", mentions an "error", "issue", or "problem" in code, or asks to "fix" something. Enforces test-driven bug fixing workflow.

tags:

Test-first bug fixing

Enforce a disciplined bug-fixing workflow that prevents regression and parallelizes fix attempts.

Core workflow

When a bug is reported, follow these steps in order:

Phase 1: Reproduce and document

1. **Understand the bug** — Gather details about expected vs actual behavior
2. **Identify the test location** — Determine where tests live in the project (check for `tests/`, `__tests__/`, `spec/`, `*.test.*`, `*.spec.*` patterns)
3. **Write a failing test** — Create a test that demonstrates the bug

Phase 2: Fix with subagents

4. **Launch fix subagents** — Use the Task tool with `subagent_type=general-purpose` to attempt fixes
5. **Run the test** — Verify the fix by running the specific test
6. **Iterate if needed** — If test still fails, launch additional subagents with new approaches

Phase 3: Verify and complete

7. **Run full test suite** — Ensure no regressions were introduced
8. **[[[[Report]]]] success** — Confirm the bug is fixed with passing test as proof

Writing the failing test

Test naming convention

Name the test to describe the bug:

```
# Python (pytest)
def test_user_login_fails_when_email_has_uppercase():
    ...

# Python (unittest)
def test_should_handle_empty_input_without_crashing(self):
    ...
```

```
// JavaScript (Jest/Vitest)
it('should not crash when input array is empty', () => { ... });
test('handles special characters in username', () => { ... });
```

```
// TypeScript
describe('UserService', () => {
  it('returns null when user not found instead of throwing', () => { ... });
});
```

Test structure

Every bug reproduction test follows this pattern:

```
def test_bug_description():
    # 1. ARRANGE - Set up the conditions that trigger the bug
    input_data = create_problematic_input()

    # 2. ACT - Perform the action that causes the bug
    result = function_under_test(input_data)

    # 3. ASSERT - Verify the expected (correct) behavior
    assert result == expected_value  # This should FAIL initially
```

Finding the right test file

Check the project structure for existing test patterns:

```
# Find test files
find . -name "*.test.*" -o -name "*.spec.*" -o -name "test_*.py" | head -20

# Find test directories
ls -la tests/ __tests__/ spec/ test/ 2>/dev/null

# Check package.json for test command
grep -A5 '"test"' package.json
```

Launching fix subagents

Use the Task tool to parallelize fix attempts:

```
Task tool parameters:
- subagent_type: "general-purpose"
- description: "Fix [bug description]"
- prompt: Include:
  1. The bug description
  2. The failing test location and contents
  3. Suspected cause (if known)
  4. Constraint: "Run the test to verify your fix works"
```

Parallel fix strategies

Launch multiple subagents with different approaches:

1. **Direct fix agent** — Focus on the immediate code causing the bug
2. **Root cause agent** — Investigate deeper architectural issues
3. **Edge case agent** — Look for similar bugs in related code

When projects lack tests

If the project has no test infrastructure:

1. **Set up minimal test framework** first
2. **Create the test file** in a sensible location
3. **Document the test [[setup]]** for future use

Quick test [[setup]] commands

```
# Python
pip install pytest
mkdir -p tests && touch tests/__init__.py

# JavaScript/TypeScript
npm install --save-dev jest
# or
npm install --save-dev vitest

# Go
# Tests are built-in, create *_test.go files
```

Verifying the fix

After subagent reports completion:

```
# Run the specific test
pytest tests/test_module.py::test_bug_description -v
npm test -- --grep "bug description"
go test -run TestBugDescription -v

# Run full suite to check for regressions
pytest
npm test
go test ./...
```

Example workflow

User reports: "The login function crashes when email has spaces"

Phase 1 — Write failing test:

```
# tests/test_auth.py
def test_login_handles_email_with_spaces():
    """Bug: Login crashes when email contains spaces"""
    auth = AuthService()

    # This should return an error, not crash
    result = auth.login("user @example.com", "password")

    assert result.success == False
    assert "invalid email" in result.error.lower()
```

Run test to confirm it fails:

```
pytest tests/test_auth.py::test_login_handles_email_with_spaces -v
# Expected: FAILED (demonstrates the bug)
```

Phase 2 — Launch subagent:

```
Task tool:
- subagent_type: "general-purpose"
- description: "Fix email space crash"
- prompt: "Fix the login crash when email contains spaces.

  Bug: AuthService.login() crashes instead of returning error when email has spaces.

  Failing test: tests/test_auth.py::test_login_handles_email_with_spaces

  After fixing, run: pytest tests/test_auth.py::test_login_handles_email_with_spaces -v

  The test must pass to confirm the fix."
```

Phase 3 — Verify:

```
# Specific test passes
pytest tests/test_auth.py::test_login_handles_email_with_spaces -v
# PASSED

# No regressions
pytest tests/test_auth.py -v
# All tests pass
```

[[Integration]] with hooks

The `[[bug-[[[[report]]]]-detector]]` hook in this plugin automatically:
1. Detects when a user reports a bug
2. Reminds Claude to follow the test-first workflow
3. Blocks Edit/Write tools until a test file has been created or modified

Additional resources

Reference files

- **`references/[[test-frameworks]].md`** — Framework-specific test patterns
- **`references/[[common-bugs]].md`** — Common bug patterns and test strategies

Example files

- `examples/python-bug-test.py` — Python pytest example
- `examples/js-bug-test.js` — JavaScript Jest example

Scripts

- `scripts/find-tests.sh` — Locate test infrastructure in a project

testing-android-intents-for-vulnerabilities (Skill Module)

`testing-android-intents-for-vulnerabilities\SKILL.md`

name: testing-android-intents-for-vulnerabilities
description: Agent for testing Android intents for vulnerabilities. Uses ADB and Drozer to enumerate exported components, test intent injection, content provider SQL injection, broadcast receiver abuse, and pending intent.... Use when the user asks to test android intents for vulnerabilities.
version: 1.0.0
tags:
source: testing-android-intents-for-vulnerabilities

Testing Android Intents for Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Drozer Modules, Component Types, OWASP MASVS v2.0, CWE Mappings

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** android intents for vulnerabilities.
- Capability: Agent for testing Android intents for vulnerabilities. Uses ADB and Drozer to enumerate exported components, test intent injection, content provider SQL injection, broadcast receiver abuse, and pending intent hijacking vulnerabilities.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, android, intents, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for android intents for vulnerabilities
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- Drozer Modules
- Component Types
- OWASP MASVS v2.0
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-android-intents-for-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-api-authentication-weaknesses (Skill Module)

`testing-api-authentication-weaknesses\SKILL.md`

name: testing-api-authentication-weaknesses
description: Agent for testing API authentication weaknesses. Tests JWT implementation flaws, unauthenticated endpoint access, token lifecycle issues, password policy enforcement, and credential brute-force resistance aligned.... Use when the user asks to test api authentication weaknesses.
version: 1.0.0
tags:
source: testing-api-authentication-weaknesses

Testing API Authentication Weaknesses

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: JWT Security Checks, OWASP API2, Common JWT HMAC Secrets, JWT Attack Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** api authentication weaknesses.
- Capability: Agent for testing API authentication weaknesses. Tests JWT implementation flaws, unauthenticated endpoint access, token lifecycle issues, password policy enforcement, and credential brute-force resistance aligned with OWASP API2:2023.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, authentication, weaknesses`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for api authentication weaknesses
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- JWT Security Checks
- OWASP API2
- Common JWT HMAC Secrets
- JWT Attack Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-api-authentication-weaknesses`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-api-for-broken-object-level-authorization (Skill Module)

`testing-api-for-broken-object-level-authorization\SKILL.md`

name: testing-api-for-broken-object-level-authorization
description: Agent for testing APIs for Broken Object Level Authorization (BOLA). Tests REST and GraphQL APIs for IDOR/BOLA vulnerabilities by systematically swapping object IDs between authenticated users to detect missing.... Use when the user asks to test api for broken object level authorization.
version: 1.0.0
tags:
source: testing-api-for-broken-object-level-authorization

Testing API for Broken Object Level Authorization

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: BOLA Test Types, Object ID Types, OWASP API1, Automated Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** api for broken object level authorization.
- Capability: Agent for testing APIs for Broken Object Level Authorization (BOLA). Tests REST and GraphQL APIs for IDOR/BOLA vulnerabilities by systematically swapping object IDs between authenticated users to detect missing per-object authorization checks. OWASP API1:2023.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, broken, object, level, authorization`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for api for broken object level authorization
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- BOLA Test Types
- Object ID Types
- OWASP API1
- Automated Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-api-for-broken-object-level-authorization`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-api-for-mass-assignment-vulnerability (Skill Module)

`testing-api-for-mass-assignment-vulnerability\SKILL.md`

name: testing-api-for-mass-assignment-vulnerability
description: Agent for testing APIs for mass assignment vulnerabilities. Tests API endpoints for auto-binding vulnerabilities where clients can modify privileged fields (role, balance, permissions) by including extra parameters.... Use when the user asks to test api for mass assignment vulnerability.
version: 1.0.0
tags:
source: testing-api-for-mass-assignment-vulnerability

Testing API for Mass Assignment Vulnerability

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Privilege Field Categories, Framework-Specific Payloads, OWASP API3, Test Tools

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** api for mass assignment vulnerability.
- Capability: Agent for testing APIs for mass assignment vulnerabilities. Tests API endpoints for auto-binding vulnerabilities where clients can modify privileged fields (role, balance, permissions) by including extra parameters in request bodies. OWASP API3:2023.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, mass, assignment, vulnerability`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for api for mass assignment vulnerability
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- Privilege Field Categories
- Framework-Specific Payloads
- OWASP API3
- Test Tools

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-api-for-mass-assignment-vulnerability`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-api-security-with-owasp-top-10 (Skill Module)

`testing-api-security-with-owasp-top-10\SKILL.md`

name: testing-api-security-with-owasp-top-10
description: Agent for automated API security testing against OWASP API Security Top 10. Use when the user asks to test api security with owasp top 10.
version: 1.0.0
tags:
source: testing-api-security-with-owasp-top-10

Testing API Security with Owasp Top 10

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: requests Library, Security Header Checks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** api security with owasp top 10.
- Capability: Agent for automated API security testing against OWASP API Security Top 10.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, security, owasp`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test for Broken Object Level Authorization (API1)
2. Test for Broken Authentication (API2) - rate limiting on login
3. Test for Broken Object Property Level Authorization (API3)
4. Test for mass assignment vulnerabilities
5. Test for Security Misconfiguration (API8)
6. Test CORS configuration on API endpoints
7. Test for Improper Inventory Management (API9)
8. Generate OWASP API Security assessment report

Tools & resources

- requests Library
- Security Header Checks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_bola` — Test for Broken Object Level Authorization (API1).
- `test_broken_auth` — Test for Broken Authentication (API2) - rate limiting on login.
- `test_data_exposure` — Test for Broken Object Property Level Authorization (API3).
- `test_mass_assignment` — Test for mass assignment vulnerabilities.
- `test_security_headers` — Test for Security Misconfiguration (API8).
- `test_cors` — Test CORS configuration on API endpoints.
- `test_api_versions` — Test for Improper Inventory Management (API9).
- `generate_report` — Generate OWASP API Security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-api-security-with-owasp-top-10`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-cors-misconfiguration (Skill Module)

`testing-cors-misconfiguration\SKILL.md`

name: testing-cors-misconfiguration
description: Agent for testing CORS misconfiguration vulnerabilities during authorized assessments. Use when the user asks to test cors misconfiguration.
version: 1.0.0
tags:
source: testing-cors-misconfiguration

Testing Cors Misconfiguration

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: requests Library, CORS Response Headers, Testing Checklist

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** cors misconfiguration.
- Capability: Agent for testing CORS misconfiguration vulnerabilities during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, cors, misconfiguration`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Generate domain-specific bypass origins for testing
2. Test if server reflects arbitrary Origin headers
3. Test OPTIONS preflight request handling
4. Test for wildcard CORS with credentials (invalid but sometimes misconfigured)
5. Test if null Origin is accepted (exploitable via sandboxed iframes)
6. Test if internal/development origins are trusted
7. Scan multiple endpoints for CORS issues
8. Generate CORS misconfiguration assessment report

Tools & resources

- requests Library
- CORS Response Headers
- Testing Checklist

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `build_dynamic_origins` — Generate domain-specific bypass origins for testing.
- `test_origin_reflection` — Test if server reflects arbitrary Origin headers.
- `test_preflight` — Test OPTIONS preflight request handling.
- `test_wildcard_with_credentials` — Test for wildcard CORS with credentials (invalid but sometimes misconfigured).
- `test_null_origin` — Test if null Origin is accepted (exploitable via sandboxed iframes).
- `test_internal_origins` — Test if internal/development origins are trusted.
- `scan_endpoints` — Scan multiple endpoints for CORS issues.
- `generate_report` — Generate CORS misconfiguration assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-cors-misconfiguration`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-broken-access-control (Skill Module)

testing-for-broken-access-control\SKILL.md

name: testing-for-broken-access-control
description: Agent for testing broken access control vulnerabilities during authorized assessments. Use when the user asks to test for broken access control.
version: 1.0.0
tags:
source: testing-for-broken-access-control

Testing for Broken Access Control

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: requests Library, Test Categories, OWASP References

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for broken access control.
- Capability: Agent for testing broken access control vulnerabilities during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, broken, access, control`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test if a regular user can access admin endpoints
2. Test if a user can access another user's resources
3. Test HTTP method override headers for access control bypass
4. Test if protected endpoints are accessible without authentication
5. Test if role/privilege fields can be modified via profile update
6. Test cross-tenant data access
7. Generate access control assessment report

Tools & resources

- requests Library
- Test Categories
- OWASP References

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_vertical_escalation` — Test if a regular user can access admin endpoints.
- `test_horizontal_escalation` — Test if a user can access another user's resources.
- `test_method_override` — Test HTTP method override headers for access control bypass.
- `test_unauthenticated_access` — Test if protected endpoints are accessible without authentication.
- `test_mass_assignment` — Test if role/privilege fields can be modified via profile update.
- `test_tenant_isolation` — Test cross-tenant data access.
- `generate_report` — Generate access control assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-broken-access-control`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-business-logic-vulnerabilities (Skill Module)

`testing-for-business-logic-vulnerabilities\SKILL.md`

name: testing-for-business-logic-vulnerabilities
description: Agent for testing business logic vulnerabilities during authorized assessments. Use when the user asks to test for business logic vulnerabilities.
version: 1.0.0
tags:
source: testing-for-business-logic-vulnerabilities

Testing for Business Logic Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: requests Library, Business Logic Test Categories, OWASP Category

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for business logic vulnerabilities.
- Capability: Agent for testing business logic vulnerabilities during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, business, logic, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test price and quantity manipulation in purchase flows
2. Test if the checkout total can be overridden in the request
3. Test if a coupon can be applied multiple times
4. Test if workflow steps can be skipped
5. Test for race conditions by sending concurrent requests
6. Test if self-referral is possible
7. Generate business logic vulnerability report

Tools & resources

- requests Library
- Business Logic Test Categories
- OWASP Category

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `test_price_manipulation` — Test price and quantity manipulation in purchase flows.
- `test_checkout_total_override` — Test if the checkout total can be overridden in the request.
- `test_coupon_reuse` — Test if a coupon can be applied multiple times.
- `test_workflow_bypass` — Test if workflow steps can be skipped.
- `test_race_condition` — Test for race conditions by sending concurrent requests.
- `test_self_referral` — Test if self-referral is possible.
- `generate_report` — Generate business logic vulnerability report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-business-logic-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-email-header-injection (Skill Module)

testing-for-email-header-injection\SKILL.md

name: testing-for-email-header-injection
description: Agent for testing email header injection vulnerabilities. Tests web application email functionality for SMTP header injection via CRLF sequences, allowing injection of CC/BCC headers, From spoofing, MIME.... Use when the user asks to test for email header injection.
version: 1.0.0
tags:
source: testing-for-email-header-injection

Testing for Email Header Injection

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: CRLF Encoding Variants, Injectable Headers, Common Injection Points, Attack Scenarios

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for email header injection.
- Capability: Agent for testing email header injection vulnerabilities. Tests web application email functionality for SMTP header injection via CRLF sequences, allowing injection of CC/BCC headers, From spoofing, MIME manipulation, and spam relay abuse.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, email, header, injection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for for email header injection
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- CRLF Encoding Variants
- Injectable Headers
- Common Injection Points
- Attack Scenarios

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-email-header-injection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-host-header-injection (Skill Module)

testing-for-host-header-injection\SKILL.md

name: testing-for-host-header-injection
description: Agent for testing HTTP Host header injection vulnerabilities. Tests web applications for password reset poisoning, web cache poisoning, SSRF, and virtual host routing manipulation via Host header and alternative host.... Use when the user asks to test for host header injection.
version: 1.0.0
tags:
source: testing-for-host-header-injection

Testing for Host Header Injection

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Alternative Host Headers, Attack Scenarios, Test Techniques

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for host header injection.
- Capability: Agent for testing HTTP Host header injection vulnerabilities. Tests web applications for password reset poisoning, web cache poisoning, SSRF, and virtual host routing manipulation via Host header and alternative host header manipulation.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, host, header, injection`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for for host header injection
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- Alternative Host Headers
- Attack Scenarios
- Test Techniques

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-host-header-injection`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-json-web-token-vulnerabilities (Skill Module)

`testing-for-json-web-token-vulnerabilities\SKILL.md`

name: testing-for-json-web-token-vulnerabilities
description: Agent for testing JSON Web Token vulnerabilities. Tests JWT implementations for algorithm confusion, none algorithm bypass, weak HMAC secrets, kid injection, missing claims, and token forgery to detect authentication.... Use when the user asks to test for json web token vulnerabilities.
version: 1.0.0
tags:
source: testing-for-json-web-token-vulnerabilities

Testing for Json Web Token Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: JWT Attack Types, JWT Structure, JWT Testing Tools, Standard Claims

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for json web token vulnerabilities.
- Capability: Agent for testing JSON Web Token vulnerabilities. Tests JWT implementations for algorithm confusion, none algorithm bypass, weak HMAC secrets, kid injection, missing claims, and token forgery to detect authentication bypass risks.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, json, token, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for for json web token vulnerabilities
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- JWT Attack Types
- JWT Structure
- JWT Testing Tools
- Standard Claims

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-json-web-token-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-open-redirect-vulnerabilities (Skill Module)

`testing-for-open-redirect-vulnerabilities\SKILL.md`

name: testing-for-open-redirect-vulnerabilities
description: Agent for testing open redirect vulnerabilities. Tests URL redirection parameters for open redirect flaws using bypass techniques including URL encoding, protocol-relative URLs, domain confusion, and parameter pollution. Use when the user asks to test for open redirect vulnerabilities.
version: 1.0.0
tags:
source: testing-for-open-redirect-vulnerabilities

Testing for Open Redirect Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Common Redirect Parameters, Bypass Techniques, HTTP Redirect Codes

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for open redirect vulnerabilities.
- Capability: Agent for testing open redirect vulnerabilities. Tests URL redirection parameters for open redirect flaws using bypass techniques including URL encoding, protocol-relative URLs, domain confusion, and parameter pollution.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, open, redirect, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for for open redirect vulnerabilities
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- Common Redirect Parameters
- Bypass Techniques
- HTTP Redirect Codes

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-open-redirect-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-sensitive-data-exposure (Skill Module)

testing-for-sensitive-data-exposure\SKILL.md

name: testing-for-sensitive-data-exposure
description: Agent for testing sensitive data exposure vulnerabilities during authorized assessments. Use when the user asks to test for sensitive data exposure.
version: 1.0.0
tags:
source: testing-for-sensitive-data-exposure

Testing for Sensitive Data Exposure

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: requests Library, Exposed File Checks, Sensitive API Response Fields, Cache-Control for Sensitive Pages

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for sensitive data exposure.
- Capability: Agent for testing sensitive data exposure vulnerabilities during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, sensitive, data, exposure`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Download and scan JavaScript files for hardcoded secrets
2. Check for exposed configuration files
3. Check API responses for excessive sensitive data
4. Check Cache-Control and security headers on sensitive pages
5. Basic TLS configuration check
6. Test if error responses leak sensitive information
7. Generate sensitive data exposure report

Tools & resources

- requests Library
- Exposed File Checks
- Sensitive API Response Fields
- Cache-Control for Sensitive Pages

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `scan_javascript_files` — Download and scan JavaScript files for hardcoded secrets.
- `check_config_files` — Check for exposed configuration files.
- `check_api_data_exposure` — Check API responses for excessive sensitive data.
- `check_security_headers` — Check Cache-Control and security headers on sensitive pages.
- `check_tls_config` — Basic TLS configuration check.
- `check_error_verbosity` — Test if error responses leak sensitive information.
- `generate_report` — Generate sensitive data exposure report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-sensitive-data-exposure`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-xml-injection-vulnerabilities (Skill Module)

`testing-for-xml-injection-vulnerabilities\SKILL.md`

name: testing-for-xml-injection-vulnerabilities
description: Agent for testing XML injection vulnerabilities. Tests web applications for XXE (XML External Entity), XPath injection, and XML entity expansion attacks that enable file disclosure, SSRF, and denial of service. Use when the user asks to test for xml injection vulnerabilities.
version: 1.0.0
tags:
source: testing-for-xml-injection-vulnerabilities

Testing for Xml Injection Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: XXE Payload Types, XPath Injection Payloads, Detection Indicators

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for xml injection vulnerabilities.
- Capability: Agent for testing XML injection vulnerabilities. Tests web applications for XXE (XML External Entity), XPath injection, and XML entity expansion attacks that enable file disclosure, SSRF, and denial of service.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, injection, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for for xml injection vulnerabilities
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- XXE Payload Types
- XPath Injection Payloads
- Detection Indicators

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-xml-injection-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-xss-vulnerabilities (Skill Module)

testing-for-xss-vulnerabilities\SKILL.md

name: testing-for-xss-vulnerabilities
description: Agent for testing Cross-Site Scripting (XSS) vulnerabilities during authorized assessments. Use when the user asks to test for xss vulnerabilities.
version: 1.0.0
tags:
source: testing-for-xss-vulnerabilities

Testing for XSS Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: XSS Payload Categories, XSS Types, CSP Analysis, Cookie Security Flags

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for xss vulnerabilities.
- Capability: Agent for testing Cross-Site Scripting (XSS) vulnerabilities during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Determine the rendering context of reflected input
2. Test URL parameters for reflected XSS
3. Test stored XSS via form submission
4. Analyze Content Security Policy header for XSS protection
5. Generate XSS assessment report

Tools & resources

- XSS Payload Categories
- XSS Types
- CSP Analysis
- Cookie Security Flags

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_reflection_context` — Determine the rendering context of reflected input.
- `test_reflected_xss` — Test URL parameters for reflected XSS.
- `test_stored_xss` — Test stored XSS via form submission.
- `check_csp_header` — Analyze Content Security Policy header for XSS protection.
- `generate_report` — Generate XSS assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-xss-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-xss-vulnerabilities-with-burpsuite (Skill Module)

testing-for-xss-vulnerabilities-with-burpsuite\SKILL.md

name: testing-for-xss-vulnerabilities-with-burpsuite
description: Agent for XSS testing workflows complementing Burp Suite during authorized assessments. Use when the user asks to test for xss vulnerabilities with burpsuite.
version: 1.0.0
tags:
source: testing-for-xss-vulnerabilities-with-burpsuite

Testing for XSS Vulnerabilities with Burpsuite

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Burp Suite, Burp Extensions for XSS, CSP Bypass Techniques

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for xss vulnerabilities with burpsuite.
- Capability: Agent for XSS testing workflows complementing Burp Suite during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, vulnerabilities, burpsuite`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Crawl pages and find parameters that reflect user input
2. Test which special characters are reflected unencoded
3. Fuzz a parameter with XSS payloads and check for reflection
4. Test stored XSS via common input endpoints
5. Analyze CSP header for XSS bypass opportunities
6. Generate XSS assessment report

Tools & resources

- Burp Suite
- Burp Extensions for XSS
- CSP Bypass Techniques

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `find_reflection_points` — Crawl pages and find parameters that reflect user input.
- `test_character_encoding` — Test which special characters are reflected unencoded.
- `fuzz_xss_payloads` — Fuzz a parameter with XSS payloads and check for reflection.
- `test_stored_xss_endpoints` — Test stored XSS via common input endpoints.
- `analyze_csp` — Analyze CSP header for XSS bypass opportunities.
- `generate_report` — Generate XSS assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-xss-vulnerabilities-with-burpsuite`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-for-xxe-injection-vulnerabilities (Skill Module)

testing-for-xxe-injection-vulnerabilities\SKILL.md

name: testing-for-xxe-injection-vulnerabilities
description: Agent for testing XXE injection vulnerabilities during authorized assessments. Use when the user asks to test for xxe injection vulnerabilities.
version: 1.0.0
tags:
source: testing-for-xxe-injection-vulnerabilities

Testing for Xxe Injection Vulnerabilities

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: XXE Payload Types, File Paths for Testing

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** for xxe injection vulnerabilities.
- Capability: Agent for testing XXE injection vulnerabilities during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, injection, vulnerabilities`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Test if endpoints accept XML content type
2. Test if a JSON endpoint also accepts XML
3. Test an endpoint with XXE payloads
4. Test SVG file upload for XXE
5. Demonstrate safe XML parsing with defusedxml
6. Generate XXE assessment report

Tools & resources

- XXE Payload Types
- File Paths for Testing

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `detect_xml_endpoints` — Test if endpoints accept XML content type.
- `test_content_type_switch` — Test if a JSON endpoint also accepts XML.
- `test_xxe_payloads` — Test an endpoint with XXE payloads.
- `test_svg_upload` — Test SVG file upload for XXE.
- `verify_safe_parsing` — Demonstrate safe XML parsing with defusedxml.
- `generate_report` — Generate XXE assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-for-xxe-injection-vulnerabilities`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-jwt-token-security (Skill Module)

`testing-jwt-token-security\SKILL.md`

name: testing-jwt-token-security
description: Agent for testing JWT token security during authorized assessments. Use when the user asks to test jwt token security.
version: 1.0.0
tags:
source: testing-jwt-token-security

Testing JWT Token Security

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: PyJWT Library, JWT Attack Types, JWT Structure, Standard Claims

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** jwt token security.
- Capability: Agent for testing JWT token security during authorized assessments.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, token, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Decode and display JWT header and payload without verification
2. Test algorithm none attack - forge token without signature
3. Brute force HMAC secret using a wordlist
4. Create a forged JWT with custom claims
5. Test if expired tokens are still accepted
6. Test if token is still valid after logout
7. Check for JWKS and OpenID configuration endpoints
8. Generate JWT security assessment report

Tools & resources

- PyJWT Library
- JWT Attack Types
- JWT Structure
- Standard Claims

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `decode_jwt` — Decode and display JWT header and payload without verification.
- `test_alg_none` — Test algorithm none attack - forge token without signature.
- `test_hmac_brute_force` — Brute force HMAC secret using a wordlist.
- `forge_token` — Create a forged JWT with custom claims.
- `test_expired_token` — Test if expired tokens are still accepted.
- `test_token_after_logout` — Test if token is still valid after logout.
- `check_jwks_endpoint` — Check for JWKS and OpenID configuration endpoints.
- `generate_report` — Generate JWT security assessment report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-jwt-token-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-mobile-api-authentication (Skill Module)

`testing-mobile-api-authentication\SKILL.md`

name: testing-mobile-api-authentication
description: Agent for testing mobile API authentication security. Tests mobile app backend APIs for broken authentication, insecure token management, session fixation, privilege escalation, and IDOR vulnerabilities using.... Use when the user asks to test mobile api authentication.
version: 1.0.0
tags:
source: testing-mobile-api-authentication

Testing Mobile API Authentication

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Mobile-Specific JWT Claims, Test Categories, OWASP MASVS v2.0, CWE Mappings

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** mobile api authentication.
- Capability: Agent for testing mobile API authentication security. Tests mobile app backend APIs for broken authentication, insecure token management, session fixation, privilege escalation, and IDOR vulnerabilities using intercepted traffic analysis.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, mobile, authentication`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for mobile api authentication
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- Mobile-Specific JWT Claims
- Test Categories
- OWASP MASVS v2.0
- CWE Mappings

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-mobile-api-authentication`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-oauth2-implementation-flaws (Skill Module)

testing-oauth2-implementation-flaws\SKILL.md

name: testing-oauth2-implementation-flaws
description: Agent for testing OAuth 2.0 implementation flaws. Tests OAuth authorization code flow, redirect URI validation, state/PKCE enforcement, token leakage, scope escalation, and OIDC ID token validation weaknesses. Use when the user asks to test oauth2 implementation flaws.
version: 1.0.0
tags:
source: testing-oauth2-implementation-flaws

Testing Oauth2 Implementation Flaws

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: AWS, OAuth 2.0 Grant Types, Redirect URI Bypass Techniques

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** oauth2 implementation flaws.
- Capability: Agent for testing OAuth 2.0 implementation flaws. Tests OAuth authorization code flow, redirect URI validation, state/PKCE enforcement, token leakage, scope escalation, and OIDC ID token validation weaknesses.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, oauth2, implementation, flaws`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for oauth2 implementation flaws
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- AWS
- OAuth 2.0 Grant Types
- Redirect URI Bypass Techniques

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-oauth2-implementation-flaws`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-ransomware-recovery-procedures (Skill Module)

`testing-ransomware-recovery-procedures\SKILL.md`

name: testing-ransomware-recovery-procedures
description: Agent for testing and validating ransomware recovery procedures. Measures RTO/RPO against targets, validates backup restore integrity, tracks recovery sequencing, and generates compliance reports. Use when the user asks to test ransomware recovery procedures.
version: 1.0.0
tags:
source: testing-ransomware-recovery-procedures

Testing Ransomware Recovery Procedures

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: Recovery Phases Tracked, RTO/RPO Calculation, Tier Definitions, Validation Checks

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** ransomware recovery procedures.
- Capability: Agent for testing and validating ransomware recovery procedures. Measures RTO/RPO against targets, validates backup restore integrity, tracks recovery sequencing, and generates compliance reports.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, ransomware, recovery, procedures`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute hashes for all files in a directory for integrity verification
2. Compare two hash manifests to detect missing, added, or changed files
3. Check if a service is running and responsive
4. Verify database is accessible after restore
5. Execute a recovery drill based on a configuration dict
6. Generate a recovery test report

Tools & resources

- Recovery Phases Tracked
- RTO/RPO Calculation
- Tier Definitions
- Validation Checks

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.

5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_file_hashes` — Compute hashes for all files in a directory for integrity verification.
- `compare_manifests` — Compare two hash manifests to detect missing, added, or changed files.
- `check_service_health` — Check if a service is running and responsive.
- `check_database_connectivity` — Verify database is accessible after restore.
- `run_recovery_drill` — Execute a recovery drill based on a configuration dict.
- `generate_report` — Generate a recovery test report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-ransomware-recovery-procedures`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

testing-websocket-api-security (Skill Module)

testing-websocket-api-security\SKILL.md

name: testing-websocket-api-security
description: Agent for testing WebSocket API security. Tests WebSocket endpoints for missing authentication, Cross-Site WebSocket Hijacking (CSWSH), injection attacks, message flooding, and authorization bypass vulnerabilities. Use when the user asks to test websocket api security.
version: 1.0.0
tags:
source: testing-websocket-api-security

Testing Websocket API Security

Role: Security testing
Audience: QA security, pentesters, and AppSec engineers
Primary tools: WebSocket Handshake Headers, Injection Payload Categories

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **test** websocket api security.
- Capability: Agent for testing WebSocket API security. Tests WebSocket endpoints for missing authentication, Cross-Site WebSocket Hijacking (CSWSH), injection attacks, message flooding, and authorization bypass vulnerabilities.
- Goal: validate defenses, applications, or controls under controlled conditions.
- Trigger keywords: `testing, websocket, security`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Define test objectives, success criteria, and out-of-scope items
2. Prepare harnesses and targets for websocket api security
3. Execute tests and capture raw results
4. Analyze failures, edge cases, and coverage
5. Report issues with severity and fix recommendations

Tools & resources

- WebSocket Handshake Headers
- Injection Payload Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `testing-websocket-api-security`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

tracking-threat-actor-infrastructure (Skill Module)

`tracking-threat-actor-infrastructure\SKILL.md`

name: tracking-threat-actor-infrastructure
description: Agent for tracking threat actor infrastructure. Uses passive DNS, certificate transparency, Shodan, WHOIS, and network fingerprinting to discover, pivot across, and map adversary-controlled infrastructure. Use when the user asks to tracking threat actor infrastructure.
version: 1.0.0
tags:
source: tracking-threat-actor-infrastructure

Tracking Threat Actor Infrastructure

Role: Security workflow
Audience: security practitioners
Primary tools: Pivoting Techniques, Network Fingerprinting, STIX 2.1 Infrastructure Object, Passive DNS Record Types

When to use

- The user wants to **track** threat actor infrastructure.
- Capability: Agent for tracking threat actor infrastructure. Uses passive DNS, certificate transparency, Shodan, WHOIS, and network fingerprinting to discover, pivot across, and map adversary-controlled infrastructure.
- Goal: complete the task.
- Trigger keywords: `tracking, threat, actor, infrastructure`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Clarify the objective for threat actor infrastructure
2. Gather inputs and required access
3. Execute the core workflow using skill scripts and references
4. Validate results and document findings
5. Recommend next actions

Key concepts

- Adversary: Threat actor or group
- Capability: Tools, techniques, and malware
- Infrastructure: C2 servers, domains, hosting
- Victim: Targeted organization or individual
- command-and-control, botnet, exfiltration, hosting-malware
- hosting-target-lists, phishing, staging, undefined
- A/AAAA: Domain to IP mapping
- CNAME: Domain alias

Tools & resources

- Pivoting Techniques
- Network Fingerprinting
- STIX 2.1 Infrastructure Object
- Passive DNS Record Types

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.

2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `tracking-threat-actor-infrastructure`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

triaging-security-alerts-in-splunk (Skill Module)

`triaging-security-alerts-in-splunk\SKILL.md`

name: triaging-security-alerts-in-splunk
description: Agent for triaging security alerts in Splunk Enterprise Security. Use when the user asks to triaging security alerts in splunk.
version: 1.0.0
tags:
source: triaging-security-alerts-in-splunk

Triaging Security Alerts in Splunk

Role: Security workflow
Audience: security practitioners
Primary tools: Splunk, Disposition Categories

When to use

- The user wants to **tria** security alerts in splunk.
- Capability: Agent for triaging security alerts in Splunk Enterprise Security.
- Goal: complete the task.
- Trigger keywords: `triaging, security, alerts, splunk`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Connect to Splunk Enterprise instance
2. Query notable events from Splunk ES Incident Review
3. Investigate brute force activity from a source IP
4. Correlate alerts across multiple data sources for a given IP
5. Check an indicator against Splunk ES threat intelligence
6. Enrich an alert with asset and identity context
7. Get triage performance metrics
8. Generate a structured triage report

Tools & resources

- Splunk
- Disposition Categories

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `connect_splunk` — Connect to Splunk Enterprise instance.
- `get_notable_events` — Query notable events from Splunk ES Incident Review.
- `investigate_brute_force` — Investigate brute force activity from a source IP.
- `correlate_across_sources` — Correlate alerts across multiple data sources for a given IP.
- `check_threat_intel` — Check an indicator against Splunk ES threat intelligence.
- `enrich_with_asset_identity` — Enrich an alert with asset and identity context.
- `get_triage_metrics` — Get triage performance metrics.
- `generate_triage_report` — Generate a structured triage report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `triaging-security-alerts-in-splunk`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

triaging-security-incident (Skill Module)

triaging-security-incident\SKILL.md

name: triaging-security-incident
description: Agent for triaging security incidents using NIST SP 800-61 and SANS PICERL frameworks. Use when the user asks to triaging security incident.
version: 1.0.0
tags:
source: triaging-security-incident

Triaging Security Incident

Role: Security workflow
Audience: security practitioners
Primary tools: Severity Matrix, SANS PICERL Framework

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **tria** security incident.
- Capability: Agent for triaging security incidents using NIST SP 800-61 and SANS PICERL frameworks.
- Goal: complete the task.
- Trigger keywords: `triaging, security, incident`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Classify incident type based on alert indicators
2. Calculate incident severity based on threat and asset context
3. Check an indicator against VirusTotal
4. Map incident to MITRE ATT&CK techniques
5. Generate a structured incident triage report

Tools & resources

- Severity Matrix
- SANS PICERL Framework

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `classify_incident` — Classify incident type based on alert indicators.
- `assess_severity` — Calculate incident severity based on threat and asset context.
- `check_virustotal` — Check an indicator against VirusTotal.
- `build_mitre_mapping` — Map incident to MITRE ATT&CK techniques.
- `generate_triage_record` — Generate a structured incident triage report.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `triaging-security-incident`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

triaging-security-incident-with-ir-playbook (Skill Module)

triaging-security-incident-with-ir-playbook\SKILL.md

name: triaging-security-incident-with-ir-playbook
description: Agent for triaging security incidents with IR playbooks. Classifies alerts by incident type, assigns severity using a structured matrix, selects the appropriate IR playbook, and generates triage decisions with.... Use when the user asks to triaging security incident with ir playbook.
version: 1.0.0
tags:
source: triaging-security-incident-with-ir-playbook

Triaging Security Incident with IR Playbook

Role: Security workflow
Audience: security practitioners
Primary tools: Incident Classification Types, Severity Matrix, Escalation Paths, SANS PICERL, FIRST CSIRT Services Framework, US

Authorization & safety

Use this skill **only** with explicit written authorization on systems you own or are contracted to assess (labs, CTFs, approved engagements). Unauthorized access or exploitation is illegal. Prefer non-destructive validation, minimize blast radius, and stop if scope is unclear.

When to use

- The user wants to **tria** security incident with ir playbook.
- Capability: Agent for triaging security incidents with IR playbooks. Classifies alerts by incident type, assigns severity using a structured matrix, selects the appropriate IR playbook, and generates triage decisions with escalation recommendations.
- Goal: complete the task.
- Trigger keywords: `triaging, security, incident, playbook`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).
4. Completed or draft artifacts from `assets/` templates when applicable.

High-level procedure

1. Submit IPs to VirusTotal, AbuseIPDB, Shodan
2. Submit file hashes to VirusTotal, MalwareBazaar, Hybrid Analysis
3. Submit domains to URLScan.io, VirusTotal, PassiveTotal
4. Check against internal IOC database and watchlists
5. Look up asset in CMDB for criticality and owner
6. Check user in HR system for role and access level
7. Verify network zone and data classification
8. Cross-reference with recent threat intelligence reports
9. Search SIEM for related alerts in past 72 hours
10. Check if same IOCs appeared in other incidents

Key concepts

- Detect (DE): Alert analysis and triage
- DE.AE-02: Potentially adverse events are analyzed to better understand associated activities
- DE.AE-03: Information is correlated from multiple sources
- DE.AE-04: The estimated impact and scope of adverse events is understood
- Respond (RS): Incident classification and escalation
- RS.AN-03: Analysis performed to establish awareness of incident scope
- RS.CO-02: Incidents reported consistent with established criteria
- Phase 2 focuses on detecting and validating security events

Tools & resources

- Incident Classification Types
- Severity Matrix
- Escalation Paths
- SANS PICERL
- FIRST CSIRT Services Framework
- US

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis
`assets/`	Templates, checklists, and report scaffolds

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `triaging-security-incident-with-ir-playbook`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

triaging-vulnerabilities-with-ssvc-framework (Skill Module)

`triaging-vulnerabilities-with-ssvc-framework\SKILL.md`

name: triaging-vulnerabilities-with-ssvc-framework
description: Agent for triaging vulnerabilities with the SSVC framework. Implements CISA's Stakeholder-Specific Vulnerability Categorization decision tree to produce actionable priorities: Track, Track*, Attend, or Act based on.... Use when the user asks to triaging vulnerabilities with ssvc framework.
version: 1.0.0
tags:
source: triaging-vulnerabilities-with-ssvc-framework

Triaging Vulnerabilities with Ssvc Framework

Role: Security workflow
Audience: security practitioners
Primary tools: SSVC Decision Outcomes, SSVC Decision Points, Enrichment APIs, Decision Tree Key Paths, Primary Standards, Regulatory and Compliance Context

When to use

- The user wants to **tria** vulnerabilities with ssvc framework.
- Capability: Agent for triaging vulnerabilities with the SSVC framework. Implements CISA's Stakeholder-Specific Vulnerability Categorization decision tree to produce actionable priorities: Track, Track*, Attend, or Act based on exploitation status, technical impact, automatability, and mission prevalence.
- Goal: complete the task.
- Trigger keywords: `triaging, vulnerabilities, ssvc, framework`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. New vulnerability scan results imported from Nessus, Qualys, OpenVAS, or other scanner
2. **Ingest Scan Results**
3. **Enrich with External Intelligence**
4. **Evaluate SSVC Decision Points**
5. **Apply Decision Tree**
6. **Generate Prioritized Report**
7. Daily scheduled job (cron or CI/CD pipeline)
8. **Refresh CISA KEV Catalog**
9. **Check Previously Tracked CVEs Against Updated KEV**
10. **Refresh EPSS Scores**

Key concepts

- Source: Cybersecurity and Infrastructure Security Agency (CISA)
- Version: SSVC v2.0 (2022 revision by CISA with SEI)
- Purpose: Provides a decision-tree methodology for vulnerability prioritization based on five decision points specific t
- Source: Carnegie Mellon University Software Engineering Institute
- Publication: "Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization" (2019)
- Authors: Jonathan Spring, Eric Hatleback, Allen Householder, Art Manion, Deana Shick
- Source: Forum of Incident Response and Security Teams (FIRST)
- Relevance: SSVC complements CVSS by adding contextual decision points beyond base score severity

Tools & resources

- SSVC Decision Outcomes
- SSVC Decision Points
- Enrichment APIs

- Decision Tree Key Paths
- Primary Standards
- Regulatory and Compliance Context
- Data Sources

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `triaging-vulnerabilities-with-ssvc-framework`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

using-git-worktrees (Skill Module)

superjawn\skills\using-git-worktrees\SKILL.md

name: using-git-worktrees

description: Use when starting feature work that needs isolation from current workspace or before executing implementation plans - creates isolated git worktrees with smart directory selection and safety verification

tags:

Using Git Worktrees

Overview

Git worktrees create isolated workspaces sharing the same repository, allowing work on multiple branches simultaneously without switching.

Core principle: Systematic directory selection + safety verification = reliable isolation.

Announce at start: "I'm using the using-git-worktrees [[skill]] to set up an isolated workspace."

Directory Selection Process

Follow this priority order:

1. Check Existing Directories

```
# Check in priority order
ls -d .worktrees 2>/dev/null     # Preferred (hidden)
ls -d worktrees 2>/dev/null      # Alternative
```

If found: Use that directory. If both exist, `.worktrees` wins.

2. Check CLAUDE.md

```
grep -i "worktree.*director" CLAUDE.md 2>/dev/null
```

If preference specified: Use it without asking.

3. Ask User

If no directory exists and no CLAUDE.md preference:

```
No worktree directory found. Where should I create worktrees?

1. .worktrees/ (project-local, hidden)
2. ~/.config/superpowers/worktrees/<project-name>/ (global location)

Which would you prefer?
```

Safety Verification

For Project-Local Directories (.worktrees or worktrees)

MUST verify directory is ignored before creating worktree:

```
# Check if directory is ignored (respects local, global, and system gitignore)
git check-ignore -q .worktrees 2>/dev/null || git check-ignore -q worktrees 2>/dev/null
```

If NOT ignored:

Per Jesse's rule "Fix broken things immediately":
1. Add appropriate line to .gitignore
2. Commit the change
3. Proceed with worktree creation

Why critical: Prevents accidentally committing worktree contents to repository.

For Global Directory (~/.config/[[superpowers]]/worktrees)

No .gitignore verification needed - outside project entirely.

Creation Steps

1. Detect Project Name

```
project=$(basename "$(git rev-parse --show-toplevel)")
```

2. Create Worktree

```
# Determine full path
case $LOCATION in
  .worktrees|worktrees)
    path="$LOCATION/$BRANCH_NAME"
    ;;
  ~/.config/superpowers/worktrees/*)
    path="~/.config/superpowers/worktrees/$project/$BRANCH_NAME"
    ;;
esac

# Create worktree with new branch
git worktree add "$path" -b "$BRANCH_NAME"
cd "$path"
```

3. Run Project [[Setup]]

Auto-detect and run appropriate [[setup]]:

```
# Node.js
if [ -f package.json ]; then npm install; fi

# Rust
if [ -f Cargo.toml ]; then cargo build; fi

# Python
if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
if [ -f pyproject.toml ]; then poetry install; fi

# Go
if [ -f go.mod ]; then go mod download; fi
```

4. Verify Clean Baseline

Run tests to ensure worktree starts clean:

```
# Examples - use project-appropriate command
npm test
cargo test
pytest
go test ./...
```

If tests fail: [[[[Report]]]] failures, ask whether to proceed or investigate.

If tests pass: [[[[Report]]]] ready.

5. [[[[Report]]]] Location

```
Worktree ready at <full-path>
Tests passing (<N> tests, 0 failures)
Ready to implement <feature-name>
```

Quick Reference

Situation	Action
`.worktrees/` exists	Use it (verify ignored)
`worktrees/` exists	Use it (verify ignored)
Both exist	Use `.worktrees/`
Neither exists	Check CLAUDE.md → Ask user
Directory not ignored	Add to .gitignore + commit
Tests fail during baseline	[[[[Report]]]] failures + ask
No package.json/Cargo.toml	Skip dependency install

Common Mistakes

Skipping ignore verification

- **Problem:** Worktree contents get tracked, pollute git [[status]]
- **Fix:** Always use `git check-ignore` before creating project-local worktree

Assuming directory location

- **Problem:** Creates inconsistency, violates project conventions
- **Fix:** Follow priority: existing > CLAUDE.md > ask

Proceeding with failing tests

- **Problem:** Can't distinguish new bugs from pre-existing issues
- **Fix:** [[[[Report]]]] failures, get explicit permission to proceed

Hardcoding [[setup]] commands

- **Problem:** Breaks on projects using different tools
- **Fix:** Auto-detect from project files (package.json, etc.)

Example Workflow

```
You: I'm using the using-git-worktrees skill to set up an isolated workspace.

[Check .worktrees/ - exists]
[Verify ignored - git check-ignore confirms .worktrees/ is ignored]
[Create worktree: git worktree add .worktrees/auth -b feature/auth]
[Run npm install]
[Run npm test - 47 passing]

Worktree ready at /Users/jesse/myproject/.worktrees/auth
Tests passing (47 tests, 0 failures)
Ready to implement auth feature
```

Red Flags

Never:
- Create worktree without verifying it's ignored (project-local)
- Skip baseline test verification
- Proceed with failing tests without asking
- Assume directory location when ambiguous
- Skip CLAUDE.md check

Always:
- Follow directory priority: existing > CLAUDE.md > ask
- Verify directory is ignored for project-local
- Auto-detect and run project [[setup]]
- Verify clean test baseline

[[Integration]]

Called by:
- **brainstorming** (Phase 4) - REQUIRED when design is approved and implementation follows

- **subagent-driven-development** - REQUIRED before executing any tasks
- **executing-plans** - REQUIRED before executing any tasks
- Any [[skill]] needing isolated workspace

Pairs with:
- **finishing-a-development-branch** - REQUIRED for cleanup after work complete

using-superjawn (Skill Module)

`superjawn\skills\using-superjawn\SKILL.md`

name: using-superjawn
description: Use when starting any conversation - establishes how to find and use skills, requiring Skill tool invocation before ANY response including clarifying questions
tags:

If you were dispatched as a subagent to execute a specific task, skip this [[skill]].

If you think there is even a 1% chance a [[skill]] might apply to what you are doing, you ABSOLUTELY MUST invoke the [[skill]].

IF A [[SKILL]] APPLIES TO YOUR TASK, YOU DO NOT HAVE A CHOICE. YOU MUST USE IT.

This is not negotiable. This is not optional. You cannot rationalize your way out of this.

Instruction Priority

superjawn [[skills]] override default system prompt behavior, but **user instructions always take precedence**:

1. **User's explicit instructions** (CLAUDE.md, GEMINI.md, [[AGENTS]].md, direct requests) — highest priority
2. **superjawn [[skills]]** — override default system behavior where they conflict
3. **Default system prompt** — lowest priority

If CLAUDE.md, GEMINI.md, or [[AGENTS]].md says "don't use TDD" and a [[skill]] says "always use TDD," follow the user's instructions. The user is in control.

How to Access [[Skills]]

In Claude Code: Use the `[[Skill]]` tool. When you invoke a [[skill]], its content is loaded and presented to you—follow it directly. Never use the Read tool on [[skill]] files.

In Copilot CLI: Use the `[[skill]]` tool. [[Skills]] are auto-discovered from installed plugins. The `[[skill]]` tool works the same as Claude Code's `[[Skill]]` tool.

In Gemini CLI: [[Skills]] activate via the `activate_skill` tool. Gemini loads [[skill]] metadata at session start and activates the full content on demand.

In other environments: Check your platform's documentation for how [[skills]] are loaded.

Platform Adaptation

[[Skills]] use Claude Code tool names. Non-CC platforms: see `references/[[copilot-tools]].md` (Copilot CLI), `references/[[codex-tools]].md` (Codex) for tool equivalents. Gemini CLI users get the tool mapping loaded automatically via GEMINI.md.

Using [[Skills]]

The Rule

Invoke relevant or requested [[skills]] BEFORE any response or action. Even a 1% chance a [[skill]] might apply means that you should invoke the [[skill]] to check. If an invoked [[skill]] turns out to be wrong for the situation, you don't need to use it.

```
digraph skill_flow {
    "User message received" [shape=doublecircle];
    "About to EnterPlanMode?" [shape=doublecircle];
    "Already brainstormed?" [shape=diamond];
    "Invoke brainstorming skill" [shape=box];
    "Might any skill apply?" [shape=diamond];
    "Invoke Skill tool" [shape=box];
    "Announce: 'Using [skill] to [purpose]'" [shape=box];
    "Has checklist?" [shape=diamond];
    "Create TodoWrite todo per item" [shape=box];
    "Follow skill exactly" [shape=box];
    "Respond (including clarifications)" [shape=doublecircle];

    "About to EnterPlanMode?" -> "Already brainstormed?";
    "Already brainstormed?" -> "Invoke brainstorming skill" [label="no"];
    "Already brainstormed?" -> "Might any skill apply?" [label="yes"];
    "Invoke brainstorming skill" -> "Might any skill apply?";

    "User message received" -> "Might any skill apply?";
    "Might any skill apply?" -> "Invoke Skill tool" [label="yes, even 1%"];
    "Might any skill apply?" -> "Respond (including clarifications)" [label="definitely not"];
    "Invoke Skill tool" -> "Announce: 'Using [skill] to [purpose]'";
    "Announce: 'Using [skill] to [purpose]'" -> "Has checklist?";
    "Has checklist?" -> "Create TodoWrite todo per item" [label="yes"];
    "Has checklist?" -> "Follow skill exactly" [label="no"];
    "Create TodoWrite todo per item" -> "Follow skill exactly";
}
```

Red Flags

These thoughts mean STOP—you're rationalizing:

Thought	Reality
"This is just a simple question"	Questions are tasks. Check for [[skills]].
"I need more context first"	[[Skill]] check comes BEFORE clarifying questions.
"Let me explore the codebase first"	[[Skills]] tell you HOW to explore. Check first.
"I can check git/files quickly"	Files lack conversation context. Check for [[skills]].
"Let me gather information first"	[[Skills]] tell you HOW to gather information.
"This doesn't need a formal [[skill]]"	If a [[skill]] exists, use it.
"I remember this [[skill]]"	[[Skills]] [[evolve]]. Read current version.
"This doesn't count as a task"	Action = task. Check for [[skills]].
"The [[skill]] is overkill"	Simple things become complex. Use it.
"I'll just do this one thing first"	Check BEFORE doing anything.
"This feels productive"	Undisciplined action wastes time. [[Skills]] prevent this.
"I know what that means"	Knowing the concept ≠ using the [[skill]]. Invoke it.

[[Skill]] Priority

When multiple [[skills]] could apply, use this order:

1. **Process [[skills]] first** (brainstorming, debugging) - these determine HOW to approach the task
2. **Implementation [[skills]] second** (frontend-design, mcp-builder) - these guide execution

"Let's build X" → brainstorming first, then implementation [[skills]].
"Fix this bug" → debugging first, then domain-specific [[skills]].

[[Skill]] Types

Rigid (TDD, debugging): Follow exactly. Don't adapt away discipline.

Flexible (patterns): Adapt principles to context.

The [[skill]] itself tells you which.

User Instructions

Instructions say WHAT, not HOW. "Add X" or "Fix Y" doesn't mean skip
[[[

validating-backup-integrity-for-recovery (Skill Module)

validating-backup-integrity-for-recovery\SKILL.md

name: validating-backup-integrity-for-recovery
description: Agent for validating backup integrity for disaster recovery. Computes cryptographic hashes, compares manifests, detects corruption, scans for ransomware artifacts, measures file entropy, and validates backup.... Use when the user asks to validating backup integrity for recovery.
version: 1.0.0
tags:
source: validating-backup-integrity-for-recovery

Validating Backup Integrity for Recovery

Role: Security workflow
Audience: security practitioners
Primary tools: Hash Algorithms Supported, Entropy Scan Output, Entropy Reference Values, Ransomware Scan Output, Known Ransomware Extensions Detected

When to use

- The user wants to **validat** backup integrity for recovery.
- Capability: Agent for validating backup integrity for disaster recovery. Computes cryptographic hashes, compares manifests, detects corruption, scans for ransomware artifacts, measures file entropy, and validates backup recoverability.
- Goal: complete the task.
- Trigger keywords: `validating, backup, integrity, recovery`

Outcomes

A successful run should leave the user with:
1. A clear answer to the objective (findings, status, or deliverable).
2. Evidence pointers (commands, logs, artifacts, hashes, or query results).
3. Prioritized next actions (contain, remediate, hunt further, or document).

High-level procedure

1. Compute cryptographic hash of a single file
2. Generate hash manifest for all files in a directory
3. Compare two manifest files to detect integrity issues
4. Calculate Shannon entropy of a file (0-8 bits per byte)
5. Scan directory for files with suspiciously high entropy (possible encryption)
6. Scan restored backup for ransomware indicators
7. Run full backup validation suite

Tools & resources

- Hash Algorithms Supported
- Entropy Scan Output
- Entropy Reference Values
- Ransomware Scan Output
- Known Ransomware Extensions Detected

How the agent should work

1. **Orient** — Restate the user's goal, scope, and constraints in one short paragraph.
2. **Load depth as needed** — Do **not** dump raw API tables into the chat. Read only the reference sections required for the current step.
   - Workflows and playbooks: `references/workflows.md` (if present)
   - Standards, data models, event IDs: `references/standards.md` (if present)
   - CLI/API flags and query snippets: `references/api-reference.md`
3. **Prefer automation** — Use `scripts/agent.py` (and `scripts/process.py` if present) for repeatable collection/analysis when the environment allows.
4. **Explain at the right altitude** — Lead with decisions and findings; put flags, full queries, and hex dumps in collapsible detail or appendices.
5. **Close the loop** — Summarize confidence, residual risk, and concrete next steps.

Package layout

Path	Purpose
`SKILL.md`	This high-level playbook (start here)
`references/`	Deep procedure, standards, and CLI/API detail
`scripts/`	Automation helpers for collection/analysis

Automation entry points

- `compute_file_hash` — Compute cryptographic hash of a single file.
- `generate_manifest` — Generate hash manifest for all files in a directory.
- `compare_manifests` — Compare two manifest files to detect integrity issues.
- `calculate_entropy` — Calculate Shannon entropy of a file (0-8 bits per byte).
- `entropy_scan` — Scan directory for files with suspiciously high entropy (possible encryption).
- `scan_ransomware_artifacts` — Scan restored backup for ransomware indicators.
- `validate_backup` — Run full backup validation suite.

Quality bar

- Prefer **evidence-backed** claims (artifact path, event ID, hash, query).
- Separate **fact**, **inference**, and **recommendation**.
- Call out false-positive pitfalls for this technique.
- When unsure, say so and propose the cheapest validation step.

High-level skill guide for `validating-backup-integrity-for-recovery`. Low-level flags and schemas live under `references/` — load them on demand rather than treating them as the skill definition.

verification-before-completion (Skill Module)

`superjawn\skills\verification-before-completion\SKILL.md`

name: verification-before-completion

description: Use when about to claim work is complete, fixed, or passing, before committing or creating PRs - requires running verification commands and confirming output before making any success claims; evidence before assertions always

tags:

Verification Before Completion

Overview

Claiming work is complete without verification is dishonesty, not efficiency.

Core principle: Evidence before claims, always.

Violating the letter of this rule is violating the spirit of this rule.

The Iron Law

```
NO COMPLETION CLAIMS WITHOUT FRESH VERIFICATION EVIDENCE
```

If you haven't run the verification command in this message, you cannot claim it passes.

The Gate Function

```
BEFORE claiming any status or expressing satisfaction:

1. IDENTIFY: What command proves this claim?
2. RUN: Execute the FULL command (fresh, complete)
3. READ: Full output, check exit code, count failures
4. VERIFY: Does output confirm the claim?
   - If NO: State actual status with evidence
   - If YES: State claim WITH evidence
5. ONLY THEN: Make the claim

Skip any step = lying, not verifying
```

Common Failures

Claim	Requires	Not Sufficient
Tests pass	Test command output: 0 failures	Previous run, "should pass"
Linter clean	Linter output: 0 errors	Partial check, extrapolation
Build succeeds	Build command: exit 0	Linter passing, logs look good
Bug fixed	Test original symptom: passes	Code changed, assumed fixed
Regression test works	Red-green cycle verified	Test passes once
Agent completed	VCS diff shows changes	Agent reports "success"
Requirements met	Line-by-line checklist	Tests passing

Red Flags - STOP

- Using "should", "probably", "seems to"
- Expressing satisfaction before verification ("Great!", "Perfect!", "Done!", etc.)
- About to commit/push/PR without verification
- Trusting agent success reports
- Relying on partial verification
- Thinking "just this once"
- Tired and wanting work over
- **ANY wording implying success without having run verification**

Rationalization Prevention

Excuse	Reality
"Should work now"	RUN the verification
"I'm confident"	Confidence ≠ evidence
"Just this once"	No exceptions
"Linter passed"	Linter ≠ compiler
"Agent said success"	Verify independently
"I'm tired"	Exhaustion ≠ excuse
"Partial check is enough"	Partial proves nothing
"Different words so rule doesn't apply"	Spirit over letter

Key Patterns

Tests:

```
[Run test command] [See: 34/34 pass] "All tests pass"
"Should pass now" / "Looks correct"
```

Regression tests (TDD Red-Green):

```
Write → Run (pass) → Revert fix → Run (MUST FAIL) → Restore → Run (pass)
"I've written a regression test" (without red-green verification)
```

Build:

```
[Run build] [See: exit 0] "Build passes"
"Linter passed" (linter doesn't check compilation)
```

Requirements:

```
Re-read plan → Create checklist → Verify each → Report gaps or completion
"Tests pass, phase complete"
```

Agent delegation:

```
Agent reports success → Check VCS diff → Verify changes → Report actual state
Trust agent report
```

Why This Matters

From 24 failure memories:
- your human partner said "I don't believe you" - trust broken
- Undefined functions shipped - would crash
- Missing requirements shipped - incomplete features
- Time wasted on false completion → redirect → rework
- Violates: "Honesty is a core value. If you lie, you'll be replaced."

When To Apply

ALWAYS before:
- ANY variation of success/completion claims
- ANY expression of satisfaction
- ANY positive statement about work state
- Committing, PR creation, task completion
- Moving to next task
- Delegating to [[agents]]

Rule applies to:
- Exact phrases
- Paraphrases and synonyms
- Implications of success
- ANY communication suggesting completion/correctness

The Bottom Line

No shortcuts for verification.

Run the command. Read the output. THEN claim the result.

This is non-negotiable.

vibe-coding (Skill Module)

dev-toolkit\skills\vibe-coding\SKILL.md

name: vibe-coding

description: Methodology for effective AI-assisted software development. Use when helping users build software with AI coding assistants, debugging AI-generated code, planning features for AI implementation, managing version control in AI workflows, or when users mention "vibe coding," Claude Code, Cursor, GitHub Copilot, Aider, Continue, Cline, Codex, Windsurf, or similar AI coding tools. Provides strategies for planning, testing, debugging, and iterating on code written with LLM assistance.

tags:

Vibe coding methodology

Practical strategies for building software effectively with AI coding assistants.

> ***Tool landscape moves fast.*** *This [[skill]] was last swept 2026-05-08. The methodology (planning, version control, testing, bug-fixing) is stable; the specific tool names, instruction-file conventions, and pricing details drift quarterly. Treat the named tools as representative, not exhaustive.*

Planning process

Start by working with the AI to write a detailed implementation plan in a markdown file.

Scope management: [[Review]] and refine the plan—delete unnecessary items, mark complex features as "won't do," and keep a separate section for ideas to implement later. This prevents scope creep and maintains focus.

Incremental implementation: Work section by section rather than building everything at once. Have the AI mark sections complete after successful implementation, and commit each working section to git before moving to the next.

Track progress visibly: Use todo lists, markdown checklists, or inline [[status]] markers so both you and the AI can see what's done and what remains. This prevents re-implementing completed work and keeps sessions focused.

Version control strategies

Git is your safety net — don't rely solely on the AI tool's revert functionality.

Branch per attempt: Begin each new feature on a fresh feature branch (`git switch -c feature/xyz`) and commit small chunks as the AI makes progress. The branch boundary is your "if this goes off the rails, throw it away" boundary — you discard the branch, not your working tree.

When the AI goes down a bad path: Prefer reversible commands. `git restore .` discards uncommitted changes; `git stash` parks them; `git switch -` jumps back to your previous branch. Reach for `git reset --hard HEAD` only when you've confirmed there's nothing in the working tree worth keeping — destructive commands skip the reflog niceties and can swallow uncommitted experiments. (If the agent has been creating new files, a separate `git clean -fd` is also part of "really, throw it all away" — same caveats.)

Clean re-implementation: When you finally find a working solution after several attempts, branch from main, implement it fresh, and discard the throwaway branch. Multiple failed AI attempts leave layers of dead code that compound future confusion — a clean re-implementation of a known-good solution is faster and more maintainable than untangling the spaghetti.

Testing framework

Prioritize end-to-end [[integration]] tests over unit tests. Focus on simulating user behavior—testing features by simulating someone clicking through the site or app.

Regression prevention: LLMs often make unnecessary changes to unrelated logic. Tests catch these regressions before they compound.

Tests as guardrails: Consider starting with test cases to provide clear boundaries for what the AI should and shouldn't change. Ensure tests pass before moving to the next feature.

Effective bug fixing

Error messages: Simply copy-pasting error messages is often enough context for the AI to identify and fix issues.

Analyze before coding: Ask the AI to consider multiple possible causes before jumping to implementation. This prevents chasing the wrong problem.

Reset after failures: Start with a clean slate after each unsuccessful fix attempt rather than layering fixes on top of broken code.

Strategic logging: Add logging statements to better understand what's happening when bugs are opaque.

Switch models: Try different AI models when one gets stuck on a problem.

AI tool landscape (as of 2026-05)

The current tools cluster into four shapes. Pick by where you work, not by hype.

Shape	Examples	When
CLI [[agents]]	Claude Code, Aider, Codex CLI, Gemini CLI, GitHub Copilot CLI, opencode, Goose	Repo-wide changes, multi-file refactors, automation, headless / cron use
Standalone IDEs	Cursor, Windsurf, Zed, Kiro	Day-to-day editing with chat + autocomplete tightly integrated
IDE extensions	GitHub Copilot, Continue, Cline, Roo Code, Amazon Q	Stay in your existing editor (VS Code, JetBrains, Neovim)
Cloud [[agents]]	Devin, OpenHands, Jules, GitHub Copilot Coding Agent	Async / background work via PR, no local terminal needed

A common stack many developers converge on: **Cursor or Copilot for daily editing + Claude Code (or Codex CLI) for repo-wide / agentic tasks**. They're complementary — fast inline edits in the IDE, longer agentic loops at the terminal.

AI tool optimization

Instruction files: Write project-specific context for your AI assistants. Conventions have splintered, but several tools converge on `[[AGENTS]].md` as a shared format. Current naming as of 2026-05:

Tool	File(s)	Notes
Claude Code	`CLAUDE.md` (per-directory, nested)	Loaded automatically; see docs.anthropic.com/en/docs/claude-code/memory
Cursor	`.cursor/rules/*.mdc` (modern) — Markdown + YAML frontmatter (`description` , `globs` , `alwaysApply`)	Legacy `.cursorrules` single-file still works but Cursor recommends migrating
Windsurf	`.windsurfrules` or `.windsurf/rules/*.md`	Same dual pattern as Cursor
GitHub Copilot	`.github/copilot-instructions.md`	Single repo-level file, ~4k char practical cap
Cline	`.clinerules`	Single file
Aider	`.aider.conf.yml` (config) + chat history files	Git-native; reads `CONVENTIONS.md` if you point it there
Continue	`.continue/config.json`	JSON config; per-repo
Codex CLI / Gemini CLI / Aider / Continue	`[[AGENTS]].md` (vendor-neutral fallback)	Becoming the cross-tool common denominator

When working across multiple tools, keep the canonical guidance in `[[AGENTS]].md` and reference it from tool-specific files (`CLAUDE.md` : "Also read [[AGENTS]].md."). That avoids drift between siblings.

Local documentation: Download API documentation to your project folder. AI tools work more accurately against local docs than against recalled training data — especially for [[libraries]] that release breaking changes faster than training cutoffs (e.g., Sentry SDK, Google GenAI SDK, Selenium).

Run multiple tools: There's no penalty for running Cursor for inline edits while a Claude Code or Codex CLI session works in another terminal on a separate task. Different shapes for different work.

Compare outputs: For high-stakes decisions, generate solutions from two different model families (e.g., Claude + GPT-5) and pick the better one. They make different mistakes.

Complex feature development

Standalone prototypes: Build complex features in a clean codebase first, then integrate once working. This isolates problems and makes debugging easier.

Reference implementations: Point the AI to working examples to follow. Existing code patterns provide concrete guidance.

Clear boundaries: Maintain consistent external APIs while allowing internal changes. Service-based architectures with clear boundaries work better than monorepos for AI-assisted development.

Tech stack considerations

Established frameworks: Ruby on Rails and similar mature frameworks work well due to 20+ years of consistent conventions in training data.

Training data matters: Newer languages like Rust or Elixir may have less training data, leading to more errors or outdated patterns.

Modularity: Small, modular files are easier for both humans and AIs to work with. Avoid files with thousands of lines—they exceed context windows and create confusion.

Beyond coding

AI assistants help with more than writing code:

- **DevOps**: Configuring servers, DNS, and hosting
- **Design**: Generating favicons and other design elements
- **Documentation**: Drafting docs and marketing materials
- **Education**: Explaining implementations line by line

- **Visual input**: Share screenshots for UI bugs or design inspiration. Most modern assistants (Claude Code via paste, Cursor, Copilot Chat) accept image input directly.
- **Voice input**: Whisper-based transcription tools (Whispr Flow, Superwhisper, MacWhisper, Aqua) reach 130-180 wpm with current OpenAI / Whisper.cpp models. Useful for long-form prompting and rubber-ducking.

Continuous improvement

Regular refactoring: Once tests are in place, refactor frequently. Ask the AI to identify refactoring candidates.

Stay current: Try every new model release. Different models excel at different tasks—experiment to find which works best for your use case.

visual-explainer (Skill Module)

visual-explainer\SKILL.md

name: visual-explainer

description: Generate self-contained HTML pages that visually explain systems, data stories, investigations, editorial workflows, and code changes. Use when the user asks for a diagram, architecture overview, diff review, plan review, project recap, source map, comparison table, timeline, or any visual explanation of technical or editorial concepts. Also use proactively when about to render a complex ASCII table (4+ rows or 3+ columns) — present it as a styled HTML page instead. Adapted from nicobailon/visual-explainer with journalism, newsroom, and academic design sensibilities.

license: MIT

compatibility: Requires a browser to view generated HTML files. Optional surf-cli for AI image generation.

metadata:

author: nicobailon (upstream); journalism adaptations by jamditis

version: 0.7.1

upstream: https://github.com/nicobailon/visual-explainer

tags:

Visual explainer

Generate self-contained HTML files for technical diagrams, editorial visualizations, and data tables. Always open the result in the browser. Never fall back to ASCII art when this [[skill]] is loaded.

Proactive table rendering. When you're about to present tabular data as an ASCII box-drawing table in the terminal (comparisons, audits, feature matrices, [[status]] reports, source verification grids — any structured rows/columns), generate an HTML page instead. The threshold: if the table has 4+ rows or 3+ columns, it belongs in the browser. Don't wait for the user to ask — render it as HTML automatically and tell them the file path. You can still include a brief text summary in the chat, but the table itself should be the HTML page.

Available commands

Detailed prompt templates in `./commands/`. In Claude Code, namespaced (`/visual-explainer:[[diff-[[review]]]]`). In Pi, these are slash commands (`/[[diff-[[review]]]]`). In Codex, use `/[[[[prompts]]]]:[[diff-[[review]]]]` (if installed to `~/.codex/[[[[prompts]]]]/`) or invoke `$visual-explainer` and describe the workflow.

Command	What it does
`[[generate-web-diagram]]`	Generate an HTML diagram for any topic
`[[generate-visual-plan]]`	Generate a visual implementation plan for a feature
`[[generate-[[slides]]]]`	Generate a magazine-quality slide deck
`[[diff-[[review]]]]`	Visual diff [[review]] with architecture comparison and code [[review]]
`[[plan-[[review]]]]`	Compare a plan against the codebase with risk assessment
`[[project-recap]]`	Mental model snapshot for context-switching back to a project
`[[fact-check]]`	Verify accuracy of a document against actual code
`[[share-page]]`	Deploy an HTML page to Vercel and get a live URL

Workflow

1. Think (5 seconds, not 5 minutes)

Before writing HTML, commit to a direction. Don't default to "dark theme with blue accents" every time.

Visual is always default. Even essays, blog posts, and articles get visual treatment — extract structure into cards, diagrams, grids, tables.

Prose patterns (lead paragraphs, pull quotes, callout boxes) are **accent elements** within visual pages, not a separate mode. Use them to highlight key points or provide breathing room, but the page structure remains visual.

For prose accents, see "Prose Page Elements" in `./references/[[[[css-patterns]]]].md`. For everything else, use the standard freeform approach with aesthetic directions below.

Who is looking? A developer understanding a system? A reporter revisiting an investigation? An editor reviewing a story? A PM seeing the big picture? An academic auditing methodology? This shapes information density and visual complexity.

What type of content? Architecture, flowchart, sequence, data flow, schema/ER, state machine, mind map, class diagram, C4 architecture, data table, timeline, [[dashboard]], source network, investigation map, editorial workflow, story structure, or prose-first page. Each has distinct layout needs and rendering approaches (see

Diagram types below).

What aesthetic? Pick one and commit. The constrained aesthetics (Blueprint, Editorial, Paper/ink) are safer — they have specific requirements that prevent generic output. The flexible ones (IDE-inspired) require more discipline.

Constrained aesthetics (prefer these):
- Blueprint (technical drawing feel, subtle grid background, deep slate/blue palette, monospace labels, precise borders)
- Editorial broadsheet (serif headlines like Playfair Display or Crimson Pro, generous whitespace, muted earth tones or deep navy + gold)
- Paper and ink (warm cream `#faf7f5` background, terracotta/sage accents, informal feel — good for newsroom features)
- Magazine feature (large display serif, off-axis composition, rich pull quotes, photo-led)
- Academic / research paper (Libre Baskerville or Source Serif, restrained palette, footnotes-style detail, generous margins)
- Newsroom board (cork/pushpin texture, handwritten-style notes, pinned-card layouts — for editorial planning, story maps)
- Investigation wall (red string aesthetic, connected entities, graph-paper background, monochrome with red accents — for relationship maps)
- Monochrome terminal (green/amber on near-black, monospace everything, CRT glow optional)

Flexible aesthetics (use with caution):
- IDE-inspired (borrow a real, named color scheme: Dracula, Nord, Catppuccin Mocha/Latte, Solarized Dark/Light, Gruvbox, One Dark, Rosé Pine) — commit to the actual palette, don't approximate
- Data-dense / wire service (small type, tight spacing, maximum information, muted colors)
- Hand-drawn sketch (Mermaid `look: 'handDrawn'`, casual annotations — for whiteboard/early-stage thinking)

Explicitly forbidden:
- Neon [[dashboard]] (cyan + magenta + purple on dark) — always produces AI slop
- Gradient mesh (pink/purple/cyan blobs) — too generic
- Any combination of Inter font + violet/indigo accents + gradient text

Vary the choice each time. If the last diagram was dark and technical, make the next one light and editorial. The swap test: if you replaced your styling with a generic dark theme and nobody would notice the difference, you haven't designed anything.

2. Structure

Read the reference material before generating. Don't memorize it — read it each time to absorb the patterns.
- For text-heavy architecture overviews (card content matters more than topology): read `./templates/architecture.html`
- For flowcharts, sequence diagrams, ER, state machines, mind maps, class diagrams, C4: read `./templates/mermaid-flowchart.html`
- For data tables, comparisons, audits, feature matrices, source verification grids: read `./templates/data-table.html`
- For slide deck presentations (when `--[[slides]]` flag is present or `/[[generate-[[slides]]]]` is invoked): read `./templates/slide-deck.html` and `./references/[[slide-patterns]].md`
- For prose-heavy publishable pages (READMEs, articles, blog posts, essays): read the "Prose Page Elements" section in `./references/[[[[css-patterns]]]].md` and "Typography by Content Voice" in `./references/[[libraries]].md`

For CSS/layout patterns and SVG connectors, read `./references/[[[[css-patterns]]]].md`.

For pages with 4+ sections (reviews, recaps, dashboards), also read `./references/[[responsive-nav]].md` for section navigation with sticky sidebar TOC on desktop and horizontal scrollable bar on mobile.

Choosing a rendering approach:

Content type	Approach	Why
Architecture (text-heavy)	CSS Grid cards + flow arrows	Rich card content (descriptions, code, tool lists) needs CSS control
Architecture (topology-focused)	**Mermaid**	Visible connections between components need automatic edge routing
Flowchart / pipeline	**Mermaid**	Automatic node positioning and edge routing
Sequence diagram	**Mermaid**	Lifelines, messages, and activation boxes need automatic layout
Data flow	**Mermaid** with edge labels	Connections and data descriptions need automatic edge routing
ER / schema diagram	**Mermaid**	Relationship lines between many entities need auto-routing
State machine	**Mermaid**	State transitions with labeled edges need automatic layout
Mind map	**Mermaid**	Hierarchical branching needs automatic positioning
Class diagram	**Mermaid**	Inheritance, composition, aggregation lines with automatic routing
C4 architecture	**Mermaid**	Use `graph TD` + `subgraph` for C4 (not native `C4Context` — it ignores themes)
Data table / comparison / audit	HTML `<table>`	Semantic markup, accessibility, copy-paste behavior
Timeline / chronology	CSS (central line + cards)	Simple linear layout doesn't need a layout engine
[[Dashboard]] / metrics	CSS Grid + Chart.js	Card grid with embedded charts
Source network	Mermaid or CSS Grid cards	Map relationships between sources in an investigation or story
Editorial workflow	Mermaid flowchart	Story lifecycle from pitch through publish/distribute
Investigation map	CSS Grid cards + Mermaid	Connect entities, documents, and events in investigative work
Story structure	CSS Grid	Visualize narrative arc, sections, sourcing distribution before [[[[publication]]]]

Mermaid theming. Always use `theme: 'base'` with custom `themeVariables` so colors match your page palette. Use `layout: 'elk'` for complex graphs (requires the `@mermaid-js/layout-elk` package — see `./references/[[libraries]].md` for the CDN import). Override Mermaid's SVG classes with CSS for pixel-perfect control. See `./references/[[libraries]].md` for full theming guide.

Mermaid containers. Always center Mermaid diagrams with `display: flex; justify-content: center;`. Add zoom controls (+/−/reset/expand) to every `.mermaid-wrap` container. Include the click-to-expand JavaScript so clicking the diagram (or the button) opens it full-size in a new tab.

Never use bare `<pre class="mermaid">`. It renders but has no zoom/pan controls — diagrams become tiny and unusable. Always use the journalism `mermaid-flowchart.html` pattern: a `.mermaid-wrap` container with `.zoom-controls` (+/−/reset/expand buttons) wrapping a `.mermaid` element. The container's CSS handles overflow and pan affordances; the bundled JS wires zoom buttons, ctrl+scroll wheel zoom, click-and-drag panning, and click-to-expand into a new tab. Copy the [[[[

When to use: Hero banners that establish the page's visual tone. Conceptual illustrations for abstract systems that Mermaid can't express (physical infrastructure, user journeys, mental models). Educational diagrams that benefit from artistic or photorealistic rendering. Decorative accents that reinforce the aesthetic.

When to skip: Anything Mermaid or CSS handles well. Generic decoration that doesn't convey meaning. Data-heavy pages where images would distract. Always degrade gracefully — if surf isn't available, skip images without erroring. The page should stand on its own with CSS and typography alone.

Prompt craft: Match the image to the page's palette and aesthetic direction. Specify the style (3D render, technical illustration, watercolor, isometric, flat vector, etc.) and mention dominant colors from your CSS variables. Use `--aspect-ratio 16:9` for hero banners, `--aspect-ratio 1:1` for inline illustrations. Keep [[[[prompts]]]] specific — "isometric illustration of a message queue with cyan nodes on dark navy background" beats "a diagram of a queue."

3. Style

Apply these principles to every diagram:

Typography is the diagram. Pick a distinctive font pairing from the list in `./references/[[libraries]].md`. Every page should use a different pairing from recent generations. The journalism font pairings (Playfair Display + Source Serif Pro for editorial broadsheet, Libre Baskerville + Fira Code for academic, Bitter + Inconsolata for newsroom) are listed alongside the upstream technical pairings.

Forbidden as `--font-body`: Inter, Roboto, Arial, Helvetica, system-ui alone. These are AI slop signals.

Good upstream pairings (use these):
- DM Sans + Fira Code (technical, precise)
- Instrument Serif + JetBrains Mono (editorial, refined)
- IBM Plex Sans + IBM Plex Mono (reliable, readable)
- Bricolage Grotesque + Fragment Mono (bold, characterful)
- Plus Jakarta Sans + Azeret Mono (rounded, approachable)

Good journalism pairings (use these for editorial / academic / newsroom contexts):
- Playfair Display + Source Serif Pro (classic newspaper)
- DM Serif Display + DM Sans (contemporary magazine)
- Libre Baskerville + Fira Code (research paper)
- Bitter + Inconsolata (working newsroom document)
- Instrument Serif + JetBrains Mono (wire service / data)

Load via `<link>` in `<head>`. Include a system font fallback in the `font-family` stack for offline resilience.

Color tells a story. Use CSS custom properties for the full palette. Define at minimum: `--bg`, `--surface`, `--border`, `--text`, `--text-dim`, and 3–5 accent colors. Each accent should have a full and a dim variant (for backgrounds). Name variables semantically when possible (`--pipeline-step` not `--blue-3`). Support both themes.

Forbidden accent colors: `#8b5cf6` `#7c3aed` `#a78bfa` (indigo/violet), `#d946ef` (fuchsia), the cyan-magenta-pink combination. These are Tailwind defaults that signal zero design intent.

Good accent palettes (use these):
- Terracotta + sage (`#c2410c`, `#65a30d`) — warm, earthy (newsroom, paper-and-ink)
- Teal + slate (`#0891b2`, `#0369a1`) — technical, precise (architecture, blueprint)
- Rose + cranberry (`#be123c`, `#881337`) — editorial, refined (verification audits)
- Amber + emerald (`#d97706`, `#059669`) — data-focused (dashboards)
- Deep blue + gold (`#1e3a5f`, `#d4a73a`) — premium, sophisticated (academic, executive)

Put your primary aesthetic in `:root` and the alternate in the media query:

```
/* Light-first (editorial, paper/ink, blueprint, academic): */
:root { /* light values */ }
@media (prefers-color-scheme: dark) { :root { /* dark values */ } }

/* Dark-first (terminal, IDE-inspired): */
:root { /* dark values */ }
@media (prefers-color-scheme: light) { :root { /* light values */ } }
```

Surfaces whisper, they don't shout. Build depth through subtle lightness shifts (2–4% between levels), not dramatic color changes. Borders should be low-opacity rgba (`rgba(255,255,255,0.08)` in dark mode, `rgba(0,0,0,0.08)` in light) — visible when you look, invisible when you don't.

Backgrounds create atmosphere. Don't use flat solid colors for the page background. Subtle gradients, faint grid patterns via CSS, or gentle radial glows behind focal areas. The background should feel like a space, not a void.

Visual weight signals importance. Not every section deserves equal visual treatment. Executive summaries and key metrics should dominate the viewport on load (larger type, more padding, subtle accent-tinted background zone). Reference sections (file maps, dependency lists, decision logs, source lists) should be compact and stay out of the way. Use `<details>/<summary>` for sections that are useful but not primary — the collapsible pattern is in `./references/[[[[css-patterns]]]].md`.

Surface depth creates hierarchy. Vary card depth to signal what matters. Hero sections get elevated shadows and accent-tinted backgrounds (`ve-card--hero` pattern). Body content stays flat (default `.ve-card`). Code blocks and secondary content feel recessed (`ve-card--recessed`). See the depth tiers in `./references/[[[[css-patterns]]]].md`. Don't make everything elevated — when everything pops, nothing does.

Animation earns its place. Staggered fade-ins on page load are almost always worth it — they guide the eye through the diagram's hierarchy. Mix animation types by role: `fadeUp` for cards, `fadeScale` for KPIs and badges, `drawIn` for SVG connectors, `countUp` for hero numbers. Hover transitions on interactive-feeling elements make the diagram feel alive. Always respect `prefers-reduced-motion`. CSS transitions and keyframes handle most cases. For orchestrated multi-element sequences, anime.js via CDN is available (see `./references/[[libraries]].md`).

Forbidden animations:
- Animated glowing box-shadows (`@keyframes glow { box-shadow: 0 0 20px... }`) — this is AI slop
- Pulsing/breathing effects on static content
- Continuous animations that run after page load (except for progress indicators)

Keep animations purposeful: entrance reveals, hover feedback, and user-initiated interactions. Nothing should glow or pulse on its own.

Accessibility. All generated pages must meet WCAG 2.1 AA minimum. This means:
- Color contrast ratios: 4.5:1 for text, 3:1 for large text and UI components
- [[Status]] indicators use shape/text alongside color (never color alone)
- Tables have proper `<thead>`, `<th scope>`, and `<caption>` elements
- Charts include data table alternatives
- Focus indicators visible for keyboard navigation
- Language declared in `<html lang="en">`

4. Deliver

Output location. Write to `~/.agent/diagrams/`. Use a descriptive filename based on content: `[[authentication]]-flow.html`, `source-network.html`, `budget-analysis.html`, `editorial-pipeline.html`. The directory persists across sessions.

Open in browser:
- macOS: `open ~/.agent/diagrams/filename.html`
- Linux: `xdg-open ~/.agent/diagrams/filename.html`

Tell the user the file path so they can re-open or share it.

Diagram types

Architecture / system diagrams

Three approaches depending on complexity:

Simple topology (under 10 elements): Use Mermaid. A `graph TD` with custom `themeVariables` produces readable diagrams with automatic edge routing.

Text-heavy overviews (under 15 elements): CSS Grid with explicit row/column placement. Sections as rounded cards with colored borders and monospace labels. Vertical flow arrows between sections. The reference [[[[

Schema / ER diagrams

Use Mermaid. Relationship lines between entities need automatic routing. Use Mermaid's `erDiagram` syntax with entity attributes. Style via `themeVariables` and CSS overrides on `.er.entityBox` and `.er.relationshipLine`.

State machines / decision trees

Use Mermaid. Use `stateDiagram-v2` for states with labeled transitions. Supports nested states, forks, joins, and notes. Decision trees can use `graph TD` with diamond decision nodes.

`stateDiagram-v2` label caveat: Transition labels have a strict parser — colons, parentheses, `<br/>`, HTML entities, and most special characters cause silent parse failures ("Syntax error in text"). If your labels need any of these (e.g., `cancel()`, `curate: true`, multi-line labels), use `flowchart TD` instead with rounded nodes and quoted edge labels (`|"label text"|`). Flowcharts handle all special characters and support `<br/>` for line breaks. Reserve `stateDiagram-v2` for simple single-word or plain-text labels.

Mind maps / hierarchical breakdowns

Use Mermaid. Use `mindmap` syntax for hierarchical branching from a root node. Mermaid handles the radial layout automatically. Style with `themeVariables` to control node colors at each depth level.

Class diagrams

Use Mermaid. Use `classDiagram` syntax for domain modeling, OOP design, and entity relationships with typed properties and methods. Supports relationships: association (`-->`), composition (`*--`), aggregation (`o--`), and inheritance (`<|--`). Add multiplicity labels (e.g., `"1" --> "*"`) and abstract/interface markers (`<<interface>>`, `<<abstract>>`). For simple entity boxes without OOP semantics (no methods, no inheritance), prefer `erDiagram` instead — it produces cleaner output for pure data modeling.

C4 architecture diagrams

Use Mermaid flowchart syntax — NOT native C4. Use `graph TD` with `subgraph` blocks for C4 boundaries. Native `C4Context` hardcodes sharp corners, its own font, blue icons, and inline SVG colors that ignore `themeVariables` — it always clashes with custom palettes.

Flowchart-as-C4 pattern: Persons → rounded nodes `(("Name"))`, systems → rectangles `["Name"]`, databases → cylinders `[("Name")]`, boundaries → `subgraph` blocks, relationships → labeled arrows `-->|"protocol"|`. Use `classDef` + `:::className` to visually differentiate external systems (e.g., dashed borders). This inherits `themeVariables`, `fontFamily`, and CSS overrides like every other Mermaid diagram.

Data tables / comparisons / audits

Use a real `<table>` element — not CSS Grid pretending to be a table. Tables get accessibility, copy-paste behavior, and column alignment for free. The reference template at `./templates/data-table.html` demonstrates all patterns below in the context of a source verification audit.

Use proactively. Any time you'd render an ASCII box-drawing table in the terminal, generate an HTML table instead. This includes: requirement audits (request vs plan), feature comparisons, status reports, configuration matrices, test result summaries, dependency lists, permission tables, API endpoint inventories, **source verification grids, FOIA request logs, document inventories, claim-by-claim audits** — any structured rows and columns.

Layout patterns:
- Sticky `<thead>` so headers stay visible when scrolling long tables
- Alternating row backgrounds via `tr:nth-child(even)` (subtle, 2–3% lightness shift)
- First column optionally sticky for wide tables with horizontal scroll
- Responsive wrapper with `overflow-x: auto` for tables wider than the viewport
- Column width hints via `<colgroup>` or `th` widths — let text-heavy columns breathe
- Row hover highlight for scanability

Status indicators (use styled `<span>` elements, never emoji):
- Match/pass/yes/verified: colored dot or checkmark with green background
- Gap/fail/no/unverified: colored dot or cross with red background
- Partial/warning: amber indicator
- Neutral/info: dim text or muted badge

Cell content:
- Wrap long text naturally — don't truncate or force single-line
- Use `<code>` for technical references within cells
- Secondary detail text in `<small>` with dimmed color
- Keep numeric columns right-aligned with `tabular-nums`

Timeline / chronology / roadmap

Vertical or horizontal timeline with a central line (CSS pseudo-element). Phase markers as circles on the line. Content cards branching left/right (alternating) or all to one side. Date labels on the line. Color progression from past (muted) to future (vivid). Good for investigation timelines, event chronologies, project histories.

Dashboard / metrics overview

Card grid layout. Hero numbers large and prominent. Sparklines via inline SVG `<polyline>`. Progress bars via CSS `linear-gradient` on a div. For real charts (bar, line, pie), use **Chart.js via CDN** (see `./references/libraries.md`). KPI cards with trend indicators (up/down arrows, percentage deltas). Good for newsroom analytics, grant reporting dashboards, audience metrics.

Source network

Map relationships between sources in an investigation or story. Can use Mermaid for connection-heavy maps or CSS Grid cards for detail-heavy source profiles. Include: source name, role, credibility indicators, what they provided, cross-references to other sources. Color-code by source category (official records, named human, anonymous, document, expert).

Investigation map

Connect entities (people, organizations, documents, events) with visual links. CSS Grid cards for entity profiles, Mermaid for relationship diagrams. Color-code by entity type. Show strength of connections. Good for investigative reporting, following the money, mapping power structures. The "investigation wall" aesthetic (red string, monochrome with red accents) suits this well.

Editorial workflow

Mermaid flowchart showing story lifecycle: pitch → assign → research → draft → edit → legal review → copy edit → layout → publish → distribute. Color-code stages. Show decision gates (kill/revise/approve). Include role labels on transitions. The reference template at `./templates/mermaid-flowchart.html` demonstrates this pattern.

Story structure

CSS Grid visualization of narrative arc. Show sections, word counts, source distribution, multimedia placement. Good for planning longform features or reviewing story architecture before publication.

Implementation plans

For visualizing implementation plans, extension designs, or feature specifications. The goal is **understanding the approach**, not reading the full source code.

Don't dump full files. Displaying entire source files inline overwhelms the page and defeats the purpose of a visual explanation. Instead:
- Show **file structure with descriptions** — list functions/exports with one-line explanations
- Show **key snippets only** — the 5–10 lines that illustrate the core logic
- Use **collapsible sections** for full code if truly needed

Code blocks require explicit formatting. Without `white-space: pre-wrap`, code runs together into an unreadable wall. See the "Code blocks" section in `./references/css-patterns.md` for the correct pattern.

Structure for implementation plans:
1. Overview/purpose (what problem does this solve?)
2. Flow diagram (Mermaid or CSS cards)
3. File structure with descriptions (not full code)
4. Key implementation details (snippets)
5. API/interface summary
6. Usage examples

Documentation (READMEs, library docs, API references)

When visualizing documentation, extract structure into visual elements:

Content	Visual treatment
Features	Card grid (2–3 columns)
Install/setup steps	Numbered cards or vertical flow
API endpoints/commands	Table with sticky header
Config options	Table
Architecture	Mermaid diagram or CSS card layout
Comparisons	Side-by-side panels or table
Warnings/notes	Callout boxes

Don't just format the prose — transform it. A feature list becomes a card grid. Install steps become a numbered flow. An API reference becomes a table.

Prose accent elements

Use these sparingly within visual pages to highlight key points or provide breathing room. See "Prose page elements" in `./references/css-patterns.md` for CSS patterns.

- **Lead paragraph** — larger intro text to set context before diving into cards/grids
- **Pull quote** — highlight a key insight; one per page maximum
- **Callout box** — warnings, tips, important notes
- **Section divider** — visual break between major sections

When to use: A visual page explaining an essay might use a lead paragraph for the thesis, then cards for key arguments. A README visualization might use callout boxes for warnings but otherwise stay card/table-focused.

Slide deck mode

An alternative output format for presenting content as a magazine-quality slide presentation instead of a scrollable page. **Opt-in only** — generate slides when the user invokes `/generate-slides`, passes `--slides` to an existing prompt (e.g., `/diff-review --slides`), or explicitly asks for a slide deck. Never auto-select slide format.

Before generating slides, read `./references/slide-patterns.md` (engine CSS, slide types, transitions, nav chrome, presets) and `./templates/slide-deck.html` (reference template showing all 10 types). Also read `./references/css-patterns.md` for shared patterns and `./references/libraries.md` for Mermaid/Chart.js theming.

Slides are not pages reformatted. They're a different medium. Each slide is exactly one viewport tall (100dvh) with no scrolling. Typography is 2–3× larger. Compositions are bolder. The agent composes a narrative arc (impact → context → deep dive → resolution) rather than mechanically paginating the source.

Content completeness. Changing the medium does not mean dropping content. Follow the "Planning a Deck from a Source Document" process in `slide-patterns.md` before writing any HTML: inventory the source, map every item to slides, verify coverage. Every section, decision, data point, specification, and collapsible detail from the source must appear in the deck. If a plan has 7 sections, the deck covers all 7. If there are 6 decisions, present all 6 — not the 2 that fit on one slide. Collapsible details in the source become their own slides. Add more slides rather than cutting content. A 22-slide deck that covers everything beats a 13-slide deck that looks polished but is missing 40% of the source.

Slide types (10): Title, Section Divider, Content, Split, Diagram, Dashboard, Table, Code, Quote, Full-Bleed. Each has a defined layout in `slide-patterns.md`. Content that exceeds a slide's density limit splits across multiple slides — never scrolls within a slide.

Visual richness: Check `which surf` at the start. If surf-cli is available, generate 2–4 images (title slide background, full-bleed background, optional content illustrations) before writing HTML — see the Proactive Imagery section in `slide-patterns.md` for the workflow. Also use SVG decorative accents, per-slide background gradients, inline sparklines, and small Mermaid diagrams. Visual-first, text-second.

Compositional variety: Consecutive slides must vary spatial approach — centered, left-heavy, right-heavy, split, edge-aligned, full-bleed. Three centered slides in a row means push one off-axis.

Curated presets: Four slide-specific presets as starting points (Midnight Editorial, Warm Signal, Terminal Mono, Swiss Clean) plus the existing aesthetic directions adapted for slides. Pick one and commit. See `slide-patterns.md` for preset CSS values.

`--slides` flag on existing prompts: When a user passes `--slides` to `/diff-review`, `/plan-review`, `/project-recap`, or other prompts, the agent gathers data using the prompt's normal data-gathering instructions, then presents the content as a slide deck instead of a scrollable page. The slide version tells the same story with different structure and pacing — but the same breadth of coverage. Don't use the slide format as an excuse to summarize or skip sections that the scrollable version would have included.

File structure

Every diagram is a single self-contained `.html` file. No external assets except CDN links (fonts, optional libraries). Structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Descriptive Title</title>
  <link href="https://fonts.googleapis.com/css2?family=...&display=swap" rel="stylesheet">
  <style>
    /* CSS custom properties, theme, layout, components — all inline */
  </style>
</head>
<body>
  <!-- Semantic HTML: sections, headings, lists, tables, inline SVG -->
  <!-- No script needed for static CSS-only diagrams -->
  <!-- Optional: <script> for Mermaid, Chart.js, or anime.js when used -->
</body>
</html>
```

Sharing pages

Share visual explainer pages instantly via Vercel when a Pi-compatible `vercel-deploy` skill is available. No account or authentication required.

Usage with the installed skill path:

```
bash ~/.pi/agent/skills/visual-explainer/scripts/share.sh <html-file>
```

If the skill lives somewhere else, use that install path instead, such as `~/.codex/skills/visual-explainer/scripts/share.sh`, `~/.config/opencode/skill/visual-explainer/scripts/share.sh`, or `./plugins/visual-explainer/scripts/share.sh` from a repository checkout.

How it works:
1. Runs the `share.sh` script from the installed `visual-explainer` skill directory
2. Copies HTML file to temp directory as `index.html`
3. Deploys via the Pi-compatible `vercel-deploy` skill
4. URL is live immediately — works in any browser

Requirements:
- vercel-deploy skill in a standard Pi-compatible skill location (in Pi: `pi install npm:vercel-deploy`)

Notes:
- Deployments are public — anyone with the URL can view
- Preview deployments have configurable retention (default: 30 days)
- Claim URL lets you transfer the deployment to your Vercel account
- Other harnesses can generate and open HTML normally; `/share-page` depends on the Pi-compatible `vercel-deploy` script being available

See `./commands/share-page.md` for the `/share-page` command template.

Quality checks

Before delivering, verify:
- **The squint test**: Blur your eyes. Can you still perceive hierarchy? Are sections visually distinct?
- **The swap test**: Would replacing your fonts and colors with a generic dark theme make this indistinguishable from a template? If yes, push the aesthetic further.
- **Both themes**: Toggle your OS between light and dark mode. Both should look intentional, not broken.
- **Information completeness**: Does the diagram actually convey what the user asked for? Pretty but incomplete is a failure.
- **No overflow**: Resize the browser to different widths. No content should clip or escape its container. Every grid and flex child needs `min-width: 0`. Side-by-side panels need `overflow-wrap: break-word`. Never use `display: flex` on `<li>` for marker characters — it creates anonymous flex items that can't shrink, causing lines with many inline `<code>` badges to overflow. Use absolute positioning for markers instead. See the Overflow Protection section in `./references/css-patterns.md`.
- **Mermaid zoom controls**: Every `.mermaid-wrap` container must have zoom controls (+/−/reset/expand buttons), Ctrl/Cmd+scroll zoom, click-and-drag panning, and click-to-expand (clicking without dragging opens the diagram full-size in a new tab). The expand button () provides the same functionality. See `./references/css-patterns.md` for the full pattern including the `openMermaidInNewTab()` function.
- **Accessibility**: Color contrast passes. Status indicators don't rely on color alone. Tables have proper semantic markup.
- **File opens cleanly**: No console errors, no broken font loads, no layout shifts.

Anti-patterns (AI slop)

These patterns are explicitly forbidden. They signal "AI-generated template" and undermine the skill's purpose of producing distinctive, high-quality diagrams. Review every generated page against this list.

Typography

Forbidden fonts as primary `--font-body`:
- Inter — the single most overused AI default
- Roboto, Arial, Helvetica — generic system fallbacks promoted to primary
- system-ui, sans-serif alone — no character, no intent

Required: Pick from the font pairings in `./references/libraries.md`. Every generation should use a different pairing from the last.

Color palette

Forbidden accent colors:
- Indigo-500/violet-500 (`#8b5cf6`, `#7c3aed`, `#a78bfa`) — Tailwind's default purple range
- The cyan + magenta + pink neon gradient combination (`#06b6d4` → `#d946ef` → `#f472b6`)
- Any palette that could be described as "Tailwind defaults with purple/pink/cyan accents"

Forbidden color effects:
- Gradient text on headings (`background: linear-gradient(...); background-clip: text;`) — this screams AI-generated
- Animated glowing box-shadows on cards (`box-shadow: 0 0 20px var(--glow); animation: glow 2s...`)
- Multiple overlapping radial glows in accent colors creating a "neon haze"

Required: Build palettes from the reference templates (terracotta/sage, teal/cyan, rose/cranberry, slate/blue) or derive from real IDE themes (Dracula, Nord, Solarized, Gruvbox, Catppuccin). Accents should feel intentional, not default.

Section headers

Forbidden:
- Emoji icons in section headers (no decorative emoji at all in section UI)
- Section headers that all use the same icon-in-rounded-box pattern

Required: Use styled monospace labels with colored dot indicators (see `.section-label` in templates), numbered badges (`section__num` pattern), or asymmetric section dividers. If an icon is genuinely needed, use an inline SVG that matches the palette — not emoji.

Layout & hierarchy

Forbidden:
- Perfectly centered everything with uniform padding
- All cards styled identically with the same border-radius, shadow, and spacing
- Every section getting equal visual treatment — no hero/primary vs. secondary distinction
- Symmetric layouts where left and right halves mirror each other

Required: Vary visual weight. Hero sections should dominate (larger type, more padding, accent-tinted background). Reference sections should feel compact. Use the depth tiers (hero → elevated → default → recessed). Asymmetric layouts create interest.

Template patterns

Forbidden:
- Three-dot window chrome (red/yellow/green dots) on code blocks — this is a cliché
- KPI cards where every metric has identical gradient text treatment
- "Neon Dashboard" as an aesthetic choice — it always produces generic results
- Gradient meshes with pink/purple/cyan blobs in the background

Required: Code blocks use a simple header with filename or language label. KPI cards vary by importance — hero numbers for the primary metric, subdued treatment for supporting metrics. Pick aesthetics with natural constraints: Blueprint (must feel technical/precise), Editorial (must have generous whitespace and serif typography), Paper/ink (must feel warm and informal), Newsroom board (must read like cork-and-pushpin).

The slop test

Before delivering, apply this test: **Would a developer or editor looking at this page immediately think "AI generated this"?** The telltale signs:

1. Inter or Roboto font with purple/violet gradient accents
2. Every heading has `background-clip: text` gradient
3. Emoji icons leading every section
4. Glowing cards with animated shadows
5. Cyan-magenta-pink color scheme on dark background
6. Perfectly uniform card grid with no visual hierarchy
7. Three-dot code block chrome

If two or more of these are present, the page is slop. Regenerate with a different aesthetic direction — Editorial, Blueprint, Paper/ink, Newsroom board, Academic, or a specific IDE theme. These constrained aesthetics are harder to mess up because they have specific visual requirements that prevent defaulting to generic patterns.

Related skills

- **journalism-core/data-journalism** — data analysis and storytelling
- **journalism-core/source-verification** — SIFT method, verification trails
- **journalism-core/editorial-workflow** — assignment tracking, deadline management
- **journalism-core/fact-check-workflow** — claim verification
- **dev-toolkit/accessibility-compliance** — WCAG compliance, accessible charts
- **dev-toolkit/zero-build-frontend** — CDN patterns, React via esm.sh, Leaflet maps
- **pdf-design** — print-ready documents with brand system

Adapted from [nicobailon/visual-explainer](#) (MIT) with design principles from [Anthropic's frontend-design skill](#). Tailored for journalism, media, and academic

web-archiving (Skill Module)

research-toolkit\skills\web-archiving\SKILL.md

name: web-archiving

description: Web page archiving and retrieval from cached/deleted sources. Use when accessing unavailable pages, preserving web content, creating legal evidence archives, or building redundant archival workflows. Covers Wayback Machine, Archive.today, ArchiveBox, and evidence preservation tools.

tags:

Web archiving methodology

Patterns for accessing inaccessible web pages and preserving web content for journalism, research, and legal purposes.

Archive service hierarchy

Try services in this order for maximum coverage:

```
                ARCHIVE RETRIEVAL CASCADE


1. Wayback Machine (archive.org)
    900B+ pages, historical depth, API access
                    ↓ not found
2. Archive.today (archive.is/archive.ph)
    On-demand snapshots, paywall bypass
    Caveat (2026): FBI subpoenaed registrar in Oct 2025;
      Wikipedia deprecated as citation source in Feb 2026 —
      prefer Wayback / Perma.cc for legal or citation use
                    ↓ not found
3. Memento Time Travel (aggregator)
    Searches multiple archives simultaneously

Retired (do not use): Google Cache (`cache:` operator) was
shut down in Sept 2024; Bing Cache dropdown was removed in
the same year. Both formerly fed this cascade.
```

Wayback Machine API

Check if URL is archived

```
import requests
from typing import Optional
from datetime import datetime
from urllib.parse import quote, unquote

def check_wayback_availability(url: str) -> Optional[dict]:
    """Check if URL exists in Wayback Machine."""
    api_url = "https://archive.org/wayback/available"

    try:
        response = requests.get(api_url, params={'url': url}, timeout=10)
        data = response.json()

        if data.get('archived_snapshots', {}).get('closest'):
            snapshot = data['archived_snapshots']['closest']
            return {
                'available': snapshot.get('available', False),
                'url': snapshot.get('url'),
                'timestamp': snapshot.get('timestamp'),
                'status': snapshot.get('status')
            }
        return None
    except Exception as e:
        return None

def get_wayback_url(url: str, timestamp: str = None) -> str:
    """Generate Wayback Machine URL for a page.

    Returns the canonical raw form (`.../web/<timestamp>/<url>`) per
    Wayback's replay-URL convention. If you intend to navigate to the
    returned link in a browser AND the target URL has `#` fragments,
    encode at the call site with urllib.parse.quote so the browser
    doesn't strip the fragment before request dispatch.

    Args:
        url: Original URL to retrieve
        timestamp: Optional YYYYMMDDHHMMSS format, or None for latest
    """
    if timestamp:
        return f"https://web.archive.org/web/{timestamp}/{url}"
    return f"https://web.archive.org/web/{url}"
```

Save page to Wayback Machine

```
def save_to_wayback(url: str, s3_keys: Optional[tuple[str, str]] = None) -> Optional[str]:
    """Request Wayback Machine to archive a URL via Save Page Now.

    Returns the archived URL if successful.

    Anonymous requests are rate-limited at roughly 15/minute. Pass
    `s3_keys=(access_key, secret)` from an Internet Archive account
    to raise the cap (anonymous → ~50/min with auth) and avoid silent
    drops on paywalled / heavily JS-rendered pages.
    """
    # quote(unquote(url), ...) normalizes any existing %xx escapes
    # first so they don't get double-encoded into %25xx.
    save_url = f"https://web.archive.org/save/{quote(unquote(url), safe='')}"

    headers = {'User-Agent': 'Mozilla/5.0 (research-archiver)'}
    if s3_keys:
        headers['Authorization'] = f'LOW {s3_keys[0]}:{s3_keys[1]}'

    try:
        response = requests.get(save_url, headers=headers, timeout=60)

        if response.status_code == 200:
            # SPN delivers the canonical archive URL via the final URL
            # after redirect-following (or the `Link` header on async
            # captures). `response.url` is the reliable common case.
            return response.url
        return None
    except Exception:
        return None
```

CDX API for historical snapshots

```
def get_all_snapshots(url: str, limit: int = 100) -> list[dict]:
    """Get all archived snapshots of a URL using CDX API.

    Returns list of snapshots with timestamps and status codes.
    """
    cdx_url = "https://web.archive.org/cdx/search/cdx"
    params = {
        'url': url,
        'output': 'json',
        'limit': limit,
        'fl': 'timestamp,original,statuscode,digest,length'
    }

    try:
        response = requests.get(cdx_url, params=params, timeout=30)
        data = response.json()

        if len(data) < 2:  # First row is headers
            return []

        headers = data[0]
        snapshots = []

        for row in data[1:]:
            snapshot = dict(zip(headers, row))
            snapshot['wayback_url'] = (
                f"https://web.archive.org/web/{snapshot['timestamp']}/{snapshot['original']}"
            )
            snapshots.append(snapshot)

        return snapshots
    except Exception:
        return []
```

Archive.today [[integration]]

Save to Archive.today

```
import re
import requests
from urllib.parse import quote, unquote, urljoin

def save_to_archive_today(url: str) -> Optional[str]:
    """Submit URL to Archive.today for archiving.

    Note: Archive.today has rate limiting and CAPTCHA requirements.
    This function works for basic archiving but may require
    manual intervention for high-volume use.

    Operational notes (2026): the FBI subpoenaed archive.today's
    registrar in October 2025; Wikipedia stopped accepting it as a
    citation source in February 2026 after the site shipped
    DDoS-attack code in January 2026. Still useful for capturing
    content the Wayback Machine can't render — but treat as
    secondary to Wayback / Perma.cc for legal or citation use.
    """
    submit_url = "https://archive.today/submit/"

    data = {
        'url': url,
        'anyway': '1'  # Archive even if recent snapshot exists
    }

    try:
        response = requests.post(
            submit_url,
            data=data,
            timeout=60,
            allow_redirects=False,
            headers={'User-Agent': 'Mozilla/5.0 (research-archiver)'},
        )
        # archive.today returns the snapshot URL in one of two shapes:
        #   - 30x with Location: https://archive.today/<snapshot_id>
        #     (Location MAY be relative per RFC 7231)
        #   - 200 with Refresh: 0;url=https://archive.today/<snapshot_id>
        #     (Refresh keyword is case-insensitive per HTML spec)
        # Following redirects silently can land on /wip/ pages or hide
        # the canonical snapshot URL, so handle both headers explicitly.
        if response.status_code in (301, 302, 303, 307, 308):
            location = response.headers.get('Location')
            if location:
                return urljoin(response.url, location)
        if response.status_code == 200:
            refresh = response.headers.get('Refresh', '')
            m = re.search(r'\burl\s*=\s*(.+)', refresh, re.IGNORECASE)
            if m:
                target = m.group(1).strip().strip('\'"')
                return urljoin(response.url, target)
        return None
    except Exception:
        return None

def search_archive_today(url: str) -> Optional[str]:
    """Search for existing Archive.today snapshot.

    Uses the /newest/<url> lookup which 302s to the most recent
    snapshot (or to a CAPTCHA page if rate-limited).
    """
    search_url = f"https://archive.ph/newest/{quote(unquote(url), safe='')}"

    try:
        response = requests.get(
            search_url,
            timeout=30,
            allow_redirects=False,
            headers={'User-Agent': 'Mozilla/5.0 (research-archiver)'},
        )
        if response.status_code in (301, 302, 303, 307, 308):
            location = response.headers.get('Location')
            if location:
                resolved = urljoin(response.url, location)
                # Only return if we landed on an archive page, not CAPTCHA
```

```
                if 'archive.' in resolved and '/newest/' not in resolved:
                    return resolved
        return None
    except Exception:
        return None
```

Multi-archive redundancy

Archive cascade for maximum preservation

```
from dataclasses import dataclass
from typing import Optional, List
from concurrent.futures import ThreadPoolExecutor, as_completed

@dataclass
class ArchiveResult:
    service: str
    url: str
    archived_url: Optional[str]
    success: bool
    error: Optional[str] = None

class MultiArchiver:
    """Archive URLs to multiple services for redundancy."""

    def __init__(self):
        self.services = [
            ('wayback', self._save_wayback),
            ('archive_today', self._save_archive_today),
            ('perma_cc', self._save_perma),  # Requires API key
        ]

    def archive_url(self, url: str, parallel: bool = True) -> List[ArchiveResult]:
        """Archive URL to all services.

        Args:
            url: URL to archive
            parallel: If True, archive to all services simultaneously
        """
        results = []

        if parallel:
            with ThreadPoolExecutor(max_workers=3) as executor:
                futures = {
                    executor.submit(save_func, url): name
                    for name, save_func in self.services
                }

                for future in as_completed(futures):
                    service = futures[future]
                    try:
                        archived_url = future.result()
                        results.append(ArchiveResult(
                            service=service,
                            url=url,
                            archived_url=archived_url,
                            success=archived_url is not None
                        ))
                    except Exception as e:
                        results.append(ArchiveResult(
                            service=service,
                            url=url,
                            archived_url=None,
                            success=False,
                            error=str(e)
                        ))
        else:
            for name, save_func in self.services:
                try:
                    archived_url = save_func(url)
                    results.append(ArchiveResult(
                        service=name,
                        url=url,
                        archived_url=archived_url,
                        success=archived_url is not None
                    ))
                except Exception as e:
                    results.append(ArchiveResult(
                        service=name,
                        url=url,
                        archived_url=None,
                        success=False,
                        error=str(e)
                    ))
```

```
        return results

    def _save_wayback(self, url: str) -> Optional[str]:
        return save_to_wayback(url)

    def _save_archive_today(self, url: str) -> Optional[str]:
        return save_to_archive_today(url)

    def _save_perma(self, url: str) -> Optional[str]:
        # Requires Perma.cc API key
        # Implementation depends on having API credentials
        return None
```

Self-hosted archiving with ArchiveBox

ArchiveBox [[setup]]

```
# Recommended: Docker Compose (v0.8.x ships with Chromium, yt-dlp,
# wget, single-file, and other capture tools preinstalled).
mkdir ~/web-archives && cd ~/web-archives
curl -O 'https://docker-compose.archivebox.io' && mv docker-compose.archivebox.io docker-compose.yml
docker compose run archivebox init --setup
docker compose up -d                    # start the web UI on :8000

# Pip-based install still works but you'll need to install Chromium /
# yt-dlp / wget / single-file separately for full capture coverage:
# pip install archivebox && archivebox init

# Add URLs to archive (from inside the archive directory)
docker compose run archivebox add "https://example.com/article"

# Add multiple URLs from file
docker compose run archivebox add --depth=0 < urls.txt

# Schedule regular archiving
docker compose run archivebox schedule --every=day --depth=1 "https://example.com/feed.rss"
```

ArchiveBox Python [[integration]]

```
import subprocess
from pathlib import Path
from typing import List, Optional

class ArchiveBoxManager:
    """Manage local ArchiveBox instance."""

    def __init__(self, archive_dir: Path):
        self.archive_dir = archive_dir
        self._ensure_initialized()

    def _ensure_initialized(self):
        """Initialize ArchiveBox if needed."""
        if not (self.archive_dir / 'index.sqlite3').exists():
            subprocess.run(
                ['archivebox', 'init'],
                cwd=self.archive_dir,
                check=True
            )

    def add_url(self, url: str, depth: int = 0) -> bool:
        """Archive a single URL.

        Args:
            url: URL to archive
            depth: 0 for single page, 1 to follow links one level deep
        """
        result = subprocess.run(
            ['archivebox', 'add', f'--depth={depth}', url],
            cwd=self.archive_dir,
            capture_output=True,
            text=True
        )
        return result.returncode == 0

    def add_urls_from_file(self, filepath: Path) -> bool:
        """Archive URLs from a text file (one per line)."""
        with open(filepath) as f:
            result = subprocess.run(
                ['archivebox', 'add', '--depth=0'],
                cwd=self.archive_dir,
                stdin=f,
                capture_output=True
            )
        return result.returncode == 0

    def search(self, query: str) -> List[dict]:
        """Search archived content."""
        result = subprocess.run(
            ['archivebox', 'list', '--filter-type=search', query],
            cwd=self.archive_dir,
            capture_output=True,
            text=True
        )
        # Parse output...
        return []
```

Legal evidence preservation

Chain of custody documentation

```
import hashlib
import sys
import json
import requests
from datetime import datetime, timezone
from dataclasses import dataclass, asdict
from typing import List

@dataclass
class EvidenceRecord:
    """Legally defensible evidence record."""

    # Content identification
    original_url: str
    archived_urls: List[str]  # Multiple archive copies
    content_hash_sha256: str

    # Timestamps
    capture_time_utc: str
    first_observed: str

    # Metadata
    page_title: str
    captured_by: str
    capture_method: str
    tool_versions: dict

    # Chain of custody
    custody_log: List[dict]  # Who accessed when

    def add_custody_entry(self, accessor: str, action: str, notes: str = ""):
        """Log access to evidence."""
        self.custody_log.append({
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'accessor': accessor,
            'action': action,
            'notes': notes
        })

    def to_json(self) -> str:
        return json.dumps(asdict(self), indent=2)

    @classmethod
    def from_capture(cls, url: str, content: bytes, captured_by: str):
        """Create evidence record from captured content."""
        now = datetime.now(timezone.utc).isoformat()
        py = sys.version_info
        return cls(
            original_url=url,
            archived_urls=[],
            content_hash_sha256=hashlib.sha256(content).hexdigest(),
            capture_time_utc=now,
            first_observed=now,
            page_title="",
            captured_by=captured_by,
            capture_method="automated_capture",
            tool_versions={
                # Replace 'archiver' with your tool's actual __version__
                'archiver': '1.0.0',
                'python': f'{py.major}.{py.minor}.{py.micro}',
                'requests': requests.__version__,
            },
            custody_log=[]
        )

def capture_as_evidence(url: str, captured_by: str) -> EvidenceRecord:
    """Capture URL with full evidence chain documentation."""

    # Capture content
    response = requests.get(url)
    content = response.content

    # Create evidence record
    record = EvidenceRecord.from_capture(url, content, captured_by)
```

```
    record.page_title = extract_title(content)

    # Archive to multiple services
    archiver = MultiArchiver()
    results = archiver.archive_url(url)

    for result in results:
        if result.success:
            record.archived_urls.append(result.archived_url)

    # Log initial capture
    record.add_custody_entry(
        captured_by,
        'initial_capture',
        f'Captured from {url}, archived to {len(record.archived_urls)} services'
    )

    return record
```

Perma.cc for legal citations

```
import requests
from typing import Optional

class PermaCC:
    """Perma.cc API client for legal-grade archiving.

    Requires API key from perma.cc (free for limited use).
    Used by US courts and legal professionals.
    """

    def __init__(self, api_key: str):
        self.api_key = api_key
        self.base_url = "https://api.perma.cc/v1"
        self.headers = {
            'Authorization': f'ApiKey {api_key}',
            'Content-Type': 'application/json'
        }

    def create_archive(self, url: str, folder_id: int = None) -> Optional[dict]:
        """Create a new Perma.cc archive.

        Returns dict with guid, creation_timestamp, and captures.
        """
        data = {'url': url}
        if folder_id:
            data['folder'] = folder_id

        try:
            response = requests.post(
                f"{self.base_url}/archives/",
                json=data,
                headers=self.headers,
                timeout=60
            )

            if response.status_code == 201:
                result = response.json()
                return {
                    'guid': result['guid'],
                    'url': f"https://perma.cc/{result['guid']}",
                    'creation_timestamp': result['creation_timestamp'],
                    'title': result.get('title', '')
                }
            return None
        except Exception:
            return None

    def get_archive(self, guid: str) -> Optional[dict]:
        """Retrieve archive metadata by GUID."""
        try:
            response = requests.get(
                f"{self.base_url}/archives/{guid}/",
                headers=self.headers,
                timeout=30
            )
            return response.json() if response.status_code == 200 else None
        except Exception:
            return None
```

Browser extensions and bookmarklets

Quick archive bookmarklet

```
// Save to Wayback Machine - add as bookmark
javascript:(function(){
    window.open('https://web.archive.org/save/' + encodeURIComponent(location.href), '_blank');
})();

// Save to Archive.today
javascript:(function(){
    window.open('https://archive.today/?run=1&url=' + encodeURIComponent(location.href), '_blank');
})();

// Check all archives (Memento)
javascript:(function(){
    window.open('https://timetravel.mementoweb.org/list/0/' + encodeURIComponent(location.href), '_blank');
})();
```

Resurrect dead pages bookmarklet

```
// Try multiple archives for dead pages
// Note: Google Cache (webcache.googleusercontent.com) was retired in
// Sept 2024 and is omitted here.
javascript:(function(){
    // Encode location.href so any '#' / '?' inside it travels as part
    // of the path argument; raw concatenation lets the browser strip
    // fragments and re-attach query strings to the outer URL.
    var encoded = encodeURIComponent(location.href);
    var archives = [
        'https://web.archive.org/web/*/' + encoded,
        'https://archive.ph/newest/' + encoded,
        'https://timetravel.mementoweb.org/list/0/' + encoded
    ];
    archives.forEach(function(a){ window.open(a, '_blank'); });
})();
```

Archive service comparison

Service	Best For	API	Deletions	Max Size	Notes
Wayback Machine	Historical research	Yes (free)	On request	Unlimited	Anonymous SPN ~15/min; auth via S3 keys raises cap
Archive.today	Paywall bypass, quick saves	Informal	Never	50MB	FBI subpoena Oct 2025; Wikipedia deprecated as citation source Feb 2026 — avoid for legal/citation use
Perma.cc	Legal citations	Yes (free tier)	By creator	Standard pages	Used by US courts; `Authorization: ApiKey <key>`
ArchiveBox	Self-hosted, privacy	Local	Never	Disk space	v0.8 ships Docker Compose with Chromium / yt-dlp / wget
Browsertrix Cloud	Interactive / JS-heavy capture	Yes	By creator	Plan-based	Webrecorder.net successor to Conifer; outputs WARC
Conifer	Interactive content	Yes	By creator	5GB free	Older Webrecorder service; Browsertrix Cloud is the active path

Error handling and fallbacks

```
import requests
from enum import Enum
from typing import Optional
from urllib.parse import quote, unquote

class ArchiveError(Enum):
    NOT_FOUND = "No archive found"
    RATE_LIMITED = "Rate limited by service"
    BLOCKED = "URL blocked from archiving"
    TIMEOUT = "Request timed out"
    SERVICE_DOWN = "Archive service unavailable"

def get_archived_page(url: str) -> tuple[Optional[str], Optional[ArchiveError]]:
    """Try all archive services with proper error handling."""

    # 1. Try Wayback Machine first
    try:
        result = check_wayback_availability(url)
        if result and result.get('available'):
            return result['url'], None
    except requests.Timeout:
        pass  # Try next service
    except Exception:
        pass

    # 2. Try Archive.today
    try:
        result = search_archive_today(url)
        if result:
            return result, None
    except Exception:
        pass

    # 3. Try Memento aggregator
    try:
        memento_url = f"https://timetravel.mementoweb.org/api/json/0/{quote(unquote(url), safe='')}"
        response = requests.get(memento_url, timeout=30)
        data = response.json()

        if data.get('mementos', {}).get('closest'):
            return data['mementos']['closest']['uri'][0], None
    except Exception:
        pass

    return None, ArchiveError.NOT_FOUND
```

Best practices

When to archive

- **Before publishing**: Archive all sources cited in your work
- **Breaking news**: Archive immediately, content may change or disappear
- **Legal matters**: Create timestamped evidence with multiple archives
- **Research**: Archive primary sources for reproducibility
- **Social media**: Archive posts before they can be deleted

Archive redundancy

Always archive to at least two services:

```
def ensure_archived(url: str) -> bool:
    """Ensure URL is archived in at least 2 services."""
    archiver = MultiArchiver()
    results = archiver.archive_url(url)

    successful = [r for r in results if r.success]
    return len(successful) >= 2
```

Rate limiting and ethics

- Respect `robots.txt` for bulk archiving
- Add delays between requests (1-3 seconds minimum)

- Don't archive personal/private pages without consent
- Use API keys when available for better rate limits
- Cache results to avoid redundant requests

web-scraping (Skill Module)

dev-toolkit\skills\web-scraping\SKILL.md

name: web-scraping

description: Web scraping with anti-bot bypass, content extraction, undocumented APIs and poison pill detection. Use when extracting content from websites, handling paywalls, implementing scraping cascades or processing social media. Covers requests, trafilatura, Playwright with stealth mode, yt-dlp and instaloader patterns.

tags:

Web scraping methodology

Patterns for reliable, ethical web scraping with fallback strategies and anti-bot handling.

Scraping cascade architecture

Implement multiple extraction strategies with automatic fallback:

```
from abc import ABC, abstractmethod
from typing import Optional
import requests
from bs4 import BeautifulSoup
import trafilatura

#for .py files
from playwright.sync_api import sync_playwright
from playwright_stealth import stealth_sync

#for .ipynb files
import asyncio
from playwright.async_api import async_playwright

class ScrapingResult:
    def __init__(self, content: str, title: str, method: str):
        self.content = content
        self.title = title
        self.method = method  # Track which method succeeded

class Scraper(ABC):
    @abstractmethod
    def fetch(self, url: str) -> Optional[ScrapingResult]: ...

class TrafilaturaCscraper(Scraper):
    """Fast, lightweight extraction for standard articles."""

    def fetch(self, url: str) -> Optional[ScrapingResult]:
        try:
            downloaded = trafilatura.fetch_url(url)
            if not downloaded:
                return None

            content = trafilatura.extract(
                downloaded,
                include_comments=False,
                include_tables=True,
                favor_recall=True
            )

            if not content or len(content) < 100:
                return None

            # Extract title separately
            soup = BeautifulSoup(downloaded, 'html.parser')
            title = soup.find('title')
            title_text = title.get_text() if title else ''

            return ScrapingResult(content, title_text, 'trafilatura')
        except Exception:
            return None

class RequestsScraper(Scraper):
    """HTTP requests with rotating user agents."""

    USER_AGENTS = [
        'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
        'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36',
        'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36',
    ]

    def fetch(self, url: str) -> Optional[ScrapingResult]:
        import random

        headers = {
            'User-Agent': random.choice(self.USER_AGENTS),
            'Accept': 'text/html,application/xhtml+xml',
            'Accept-Language': 'en-US,en;q=0.9',
        }

        try:
            response = requests.get(url, headers=headers, timeout=30)
            response.raise_for_status()

            soup = BeautifulSoup(response.text, 'html.parser')

            # Remove script/style elements
            for element in soup(['script', 'style', 'nav', 'footer', 'aside']):
```

```
                element.decompose()

            # Find main content
            main = soup.find('main') or soup.find('article') or soup.find('body')
            content = main.get_text(separator='\n', strip=True) if main else ''

            title = soup.find('title')
            title_text = title.get_text() if title else ''

            if len(content) < 100:
                return None

            return ScrapingResult(content, title_text, 'requests')
        except Exception:
            return None

class PlaywrightScraper(Scraper):
    """Heavy JavaScript rendering with stealth mode for anti-bot bypass."""

    def fetch(self, url: str) -> Optional[ScrapingResult]:
        try:
            with sync_playwright() as p:
                browser = p.chromium.launch(headless=True)
                context = browser.new_context(
                    viewport={'width': 1920, 'height': 1080},
                    user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
                )
                page = context.new_page()

                # Apply stealth to avoid detection
                stealth_sync(page)

                page.goto(url, wait_until='networkidle', timeout=60000)

                # Wait for content to load
                page.wait_for_timeout(2000)

                # Extract content
                content = page.evaluate('''() => {
                    const article = document.querySelector('article, main, .content, #content');
                    return article ? article.innerText : document.body.innerText;
                }''')

                title = page.title()

                browser.close()

                if len(content) < 100:
                    return None

                return ScrapingResult(content, title, 'playwright')
        except Exception:
            return None

class PlaywrightScraperAsync:
    """Async Playwright scraper for Jupyter notebooks (.ipynb files).

    Jupyter notebooks run their own event loop, so sync Playwright won't work.
    Use this async version with `await` in notebook cells.
    """

    async def fetch(self, url: str) -> Optional[ScrapingResult]:
        try:
            async with async_playwright() as p:
                browser = await p.chromium.launch(headless=True)
                context = await browser.new_context(
                    viewport={'width': 1920, 'height': 1080},
                    user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
                )
                page = await context.new_page()

                # Note: playwright-stealth async version
                # from playwright_stealth import stealth_async
                # await stealth_async(page)

                await page.goto(url, wait_until='networkidle', timeout=60000)

                # Wait for content to load
                await page.wait_for_timeout(2000)
```

```
                # Extract content
                content = await page.evaluate('''() => {
                    const article = document.querySelector('article, main, .content, #content');
                    return article ? article.innerText : document.body.innerText;
                }''')

                title = await page.title()

                await browser.close()

                if len(content) < 100:
                    return None

                return ScrapingResult(content, title, 'playwright_async')
        except Exception:
            return None

# Usage in Jupyter notebook cells:
# scraper = PlaywrightScraperAsync()
# result = await scraper.fetch('https://example.com')

class ScrapingCascade:
    """Try multiple scrapers in order until one succeeds."""

    def __init__(self):
        self.scrapers = [
            TrafilaturaCscraper(),
            RequestsScraper(),
            PlaywrightScraper(),
        ]

    def fetch(self, url: str) -> Optional[ScrapingResult]:
        for scraper in self.scrapers:
            result = scraper.fetch(url)
            if result:
                return result
        return None
```

Anti-bot landscape (as of 2026-05)

The cascade above (`requests` → `trafilatura` → Playwright + `playwright-stealth`) handles plain HTML and lightly-protected JS sites. Modern anti-bot stacks (Cloudflare Bot Management / Turnstile, DataDome, Akamai Bot Manager, PerimeterX) layer multiple detection signals: TLS / HTTP-2 fingerprints, browser fingerprints, JS-execution proofs, residential-IP reputation, session behavior. No single tool defeats all of them.

`playwright-stealth` (2.0+, current) patches obvious detection vectors — `navigator.webdriver`, `chrome.runtime`, plugin enumeration, language settings, WebGL fingerprints. Treat it as the floor, not the ceiling. If a target fingerprints TLS or runs Turnstile, stealth alone won't pass.

Tool	Layer it addresses	Notes
`curl_cffi`	TLS / HTTP-2 fingerprint	Drop-in replacement for `requests` that mimics Chrome/Safari/Edge JA3+ALPN. Can't run JS — pair with a parsed-HTML extractor when JS isn't required.
`playwright-stealth` 2.x	JS-runtime fingerprint	The starting line for Playwright/Chromium. Updates lag the bot stacks; expect to combine with rotation.
Camoufox	JS + browser fingerprint at C++ level	Firefox-based stealth browser. Spoofs fingerprint values low enough that JS-side checks can't see through them. Use when Chromium-based stealth is detected.
SeleniumBase UC Mode	Turnstile + browser fingerprint	The closest thing to a one-shot Turnstile solver in 2026, but heavier than playwright-stealth.
Residential proxy pool	IP reputation	Datacenter IPs (DigitalOcean, AWS) get challenged on first request. Residential pools cost more but bypass the cheapest layer of defense.

Use the lightest tool that works. Targets without aggressive defense don't need Camoufox or proxy pools — `curl_cffi` plus a sleep is usually enough. Reserve heavier tools for sites that explicitly serve a Turnstile challenge or DataDome interstitial.

Undocumented APIs

Finding undocumented APIs

Use browser developer tools to discover APIs:

1. **Open developer tools** (right-click → Inspect, or F12)
2. **Go to the Network tab** to monitor all requests
3. **Filter by Fetch/XHR** to show only API calls

4. **Trigger the action** you want to capture (search, scroll, click)
5. **Analyze the response** — usually JSON with key-value pairs
6. **Copy as cURL** (right-click the request)
7. **Convert to code** using curlconverter.com

Stripping down API requests

When you copy a cURL from dev tools, it includes many parameters. Strip it down by:

1. **Remove unnecessary cookies** — test without them first
2. **Keep [[authentication]] tokens** if required
3. **Identify the input parameters** you can modify (like `prefix` for search terms)
4. **Test parameter values** — some expire, so periodically verify

Example: Reverse-engineering an autocomplete API

```
import requests
import time

def search_suggestions(keyword: str) -> dict:
    """
    Get autocompleted search suggestions from an undocumented API.
    Stripped down from browser dev tools capture.
    """
    headers = {
        'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:100.0) Gecko/20100101 Firefox/100.0',
        'Accept': 'application/json, text/javascript, */*; q=0.01',
        'Accept-Language': 'en-US,en;q=0.5',
    }

    params = {
        'prefix': keyword,
        'suggestion-type': ['WIDGET', 'KEYWORD'],
        'alias': 'aps',
        'plain-mid': '1',
    }

    response = requests.get(
        'https://completion.amazon.com/api/2017/suggestions',
        params=params,
        headers=headers
    )
    return response.json()

# Collect suggestions for multiple keywords
keywords = ['a', 'b', 'cookie', 'sock']
data = []

for keyword in keywords:
    suggestions = search_suggestions(keyword)
    suggestions['search_word'] = keyword  # track seed keyword
    time.sleep(1)  # rate limit yourself
    data.extend(suggestions.get('suggestions', []))
```

Source: Leon Yin, "Finding Undocumented APIs," Inspect Element, 2023

Poison pill detection

Detect paywalls, anti-bot pages, and other failures:

```
from dataclasses import dataclass
from enum import Enum
import re

class PoisonPillType(Enum):
    PAYWALL = 'paywall'
    CAPTCHA = 'captcha'
    RATE_LIMIT = 'rate_limit'
    CLOUDFLARE = 'cloudflare'
    LOGIN_REQUIRED = 'login_required'
    NOT_FOUND = 'not_found'
    NONE = 'none'

@dataclass
class PoisonPillResult:
    detected: bool
    type: PoisonPillType
    confidence: float
    details: str

class PoisonPillDetector:
    PATTERNS = {
        PoisonPillType.PAYWALL: [
            r'subscribe to continue',
            r'subscription required',
            r'become a member',
            r'sign up to read',
            r'you\'ve reached your limit',
            r'article limit reached',
        ],
        PoisonPillType.CAPTCHA: [
            r'verify you are human',
            r'captcha',
            r'robot verification',
            r'prove you\'re not a robot',
        ],
        PoisonPillType.RATE_LIMIT: [
            r'too many requests',
            r'rate limit exceeded',
            r'slow down',
            r'429',
        ],
        PoisonPillType.CLOUDFLARE: [
            r'checking your browser',
            r'cloudflare',
            r'ddos protection',
            r'please wait while we verify',
        ],
        PoisonPillType.LOGIN_REQUIRED: [
            r'sign in to continue',
            r'log in required',
            r'create an account',
        ],
    }

    PAYWALL_DOMAINS = {
        'nytimes.com': PoisonPillType.PAYWALL,
        'wsj.com': PoisonPillType.PAYWALL,
        'washingtonpost.com': PoisonPillType.PAYWALL,
        'ft.com': PoisonPillType.PAYWALL,
        'bloomberg.com': PoisonPillType.PAYWALL,
    }

    def detect(self, url: str, content: str, status_code: int = 200) -> PoisonPillResult:
        # Check status code
        if status_code == 429:
            return PoisonPillResult(True, PoisonPillType.RATE_LIMIT, 1.0, 'HTTP 429')
        if status_code == 403:
            return PoisonPillResult(True, PoisonPillType.CLOUDFLARE, 0.8, 'HTTP 403')
        if status_code == 404:
            return PoisonPillResult(True, PoisonPillType.NOT_FOUND, 1.0, 'HTTP 404')

        # Check known paywall domains
        from urllib.parse import urlparse
        domain = urlparse(url).netloc.replace('www.', '')
        for paywall_domain, pill_type in self.PAYWALL_DOMAINS.items():
            if paywall_domain in domain:
                # Check if content is suspiciously short (paywall truncation)
```

```
                if len(content) < 500:
                    return PoisonPillResult(True, pill_type, 0.9, f'Short content from {domain}')

        # Pattern matching
        content_lower = content.lower()
        for pill_type, patterns in self.PATTERNS.items():
            for pattern in patterns:
                if re.search(pattern, content_lower):
                    return PoisonPillResult(True, pill_type, 0.7, f'Pattern match: {pattern}')

        return PoisonPillResult(False, PoisonPillType.NONE, 0.0, '')
```

Social media scraping

YouTube with yt-dlp

```
import yt_dlp
from pathlib import Path

def download_video_metadata(url: str) -> dict:
    """Extract metadata without downloading video."""
    ydl_opts = {
        'skip_download': True,
        'quiet': True,
        'no_warnings': True,
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(url, download=False)
        return {
            'title': info.get('title'),
            'description': info.get('description'),
            'duration': info.get('duration'),
            'upload_date': info.get('upload_date'),
            'view_count': info.get('view_count'),
            'channel': info.get('channel'),
            'thumbnail': info.get('thumbnail'),
        }

def download_video(url: str, output_dir: Path, audio_only: bool = False) -> Path:
    """Download video or audio."""
    output_template = str(output_dir / '%(title)s.%(ext)s')

    ydl_opts = {
        'outtmpl': output_template,
        'quiet': True,
    }

    if audio_only:
        ydl_opts['format'] = 'bestaudio/best'
        ydl_opts['postprocessors'] = [{
            'key': 'FFmpegExtractAudio',
            'preferredcodec': 'mp3',
        }]

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(url, download=True)
        filename = ydl.prepare_filename(info)
        if audio_only:
            filename = filename.rsplit('.', 1)[0] + '.mp3'
        return Path(filename)

def get_transcript(url: str) -> list[dict]:
    """Extract auto-generated or manual subtitles."""
    ydl_opts = {
        'skip_download': True,
        'writesubtitles': True,
        'writeautomaticsub': True,
        'subtitleslangs': ['en'],
        'quiet': True,
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(url, download=False)

        # Check for subtitles
        subtitles = info.get('subtitles', {})
        auto_captions = info.get('automatic_captions', {})

        # Prefer manual subtitles over auto-generated
        subs = subtitles.get('en') or auto_captions.get('en')
        if not subs:
            return []

        # Get the vtt or json format
        for sub in subs:
            if sub['ext'] in ['vtt', 'json3']:
                # Download and parse subtitle file
                # ... implementation depends on format
                pass
```

```
        return []
```

Instagram with instaloader

```
import instaloader
from pathlib import Path

class InstagramScraper:
    def __init__(self, username: str = None, session_file: str = None):
        self.loader = instaloader.Instaloader(
            download_videos=True,
            download_video_thumbnails=False,
            download_geotags=False,
            download_comments=False,
            save_metadata=True,
            compress_json=False,
        )

        if session_file and Path(session_file).exists():
            self.loader.load_session_from_file(username, session_file)

    def get_profile_posts(self, username: str, limit: int = 50) -> list[dict]:
        """Get recent posts from a profile."""
        profile = instaloader.Profile.from_username(self.loader.context, username)
        posts = []

        for i, post in enumerate(profile.get_posts()):
            if i >= limit:
                break

            posts.append({
                'shortcode': post.shortcode,
                'url': f'https://instagram.com/p/{post.shortcode}/',
                'caption': post.caption,
                'timestamp': post.date_utc.isoformat(),
                'likes': post.likes,
                'comments': post.comments,
                'is_video': post.is_video,
                'video_url': post.video_url if post.is_video else None,
            })

        return posts

    def download_post(self, shortcode: str, output_dir: Path):
        """Download a single post's media."""
        post = instaloader.Post.from_shortcode(self.loader.context, shortcode)
        self.loader.download_post(post, target=str(output_dir))
```

TikTok with yt-dlp

```
def scrape_tiktok_profile(username: str, output_dir: Path, limit: int = 50) -> list[dict]:
    """Scrape TikTok profile videos."""
    profile_url = f'https://tiktok.com/@{username}'

    ydl_opts = {
        'quiet': True,
        'extract_flat': True,  # Don't download, just get info
        'playlistend': limit,
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(profile_url, download=False)
        videos = []

        for entry in info.get('entries', []):
            videos.append({
                'id': entry.get('id'),
                'title': entry.get('title'),
                'url': entry.get('url'),
                'timestamp': entry.get('timestamp'),
                'view_count': entry.get('view_count'),
            })

        return videos

def download_tiktok_video(url: str, output_dir: Path) -> Path:
    """Download a single TikTok video."""
    ydl_opts = {
        'outtmpl': str(output_dir / '%(id)s.%(ext)s'),
        'quiet': True,
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        info = ydl.extract_info(url, download=True)
        return Path(ydl.prepare_filename(info))
```

Request patterns

Rotating user [[agents]] and headers

```
import random
from fake_useragent import UserAgent

class RequestManager:
    def __init__(self):
        self.ua = UserAgent()
        self.session = requests.Session()

    def get_headers(self) -> dict:
        return {
            'User-Agent': self.ua.random,
            'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8',
            'Accept-Language': 'en-US,en;q=0.5',
            'Accept-Encoding': 'gzip, deflate, br',
            'DNT': '1',
            'Connection': 'keep-alive',
            'Upgrade-Insecure-Requests': '1',
        }

    def fetch(self, url: str, retry_count: int = 3) -> requests.Response:
        for attempt in range(retry_count):
            try:
                response = self.session.get(
                    url,
                    headers=self.get_headers(),
                    timeout=30
                )
                response.raise_for_status()
                return response
            except requests.RequestException as e:
                if attempt == retry_count - 1:
                    raise
                time.sleep(2 ** attempt)  # Exponential backoff
```

Respectful scraping with delays

```
import time
import random
from urllib.parse import urlparse

class PoliteRequester:
    def __init__(self, min_delay: float = 1.0, max_delay: float = 3.0):
        self.min_delay = min_delay
        self.max_delay = max_delay
        self.last_request_per_domain = {}

    def wait_for_domain(self, url: str):
        domain = urlparse(url).netloc
        last_request = self.last_request_per_domain.get(domain, 0)

        elapsed = time.time() - last_request
        delay = random.uniform(self.min_delay, self.max_delay)

        if elapsed < delay:
            time.sleep(delay - elapsed)

        self.last_request_per_domain[domain] = time.time()
```

Ethics, robots.txt, and the legal landscape

Scraping is technically simple, ethically nuanced, and legally a moving target. The current state in the US (2026):

Computer Fraud and Abuse Act (CFAA). *Van Buren v. United States* (2021) and *hiQ Labs v. LinkedIn* (2022) narrowed the CFAA so that scraping public, non-credentialed pages does NOT constitute "unauthorized access." Logging in (or using credentials), bypassing technical access controls, or scraping after an explicit cease-and-desist letter remains legally fraught. State equivalents (e.g., California's CDAFA) sometimes go further than federal law.

Terms of service. Many sites' ToS forbid scraping. ToS is a contract, not a criminal statute — breach exposes you to civil claims (breach of contract, tortious interference, trespass to chattels in some jurisdictions), not jail. The risk profile differs sharply from CFAA.

robots.txt is a polite request, not a legal mandate. Ignoring it doesn't make you criminally liable, but courts have cited it as evidence of intent. For journalism in the public interest, that intent can be defensible; for commercial use, it's harder.

EU GDPR / UK DPA. If your scraping pulls personal data of EU/UK residents, GDPR/DPA apply regardless of where you run the scraper. Public availability does NOT exempt personal data from these regimes — `Lloyd v. Google` (UK Supreme Court 2021) and CJEU's `Schrems II` lineage make scraping personal data without a lawful basis a real liability.

Practical baseline:
- Always read `robots.txt`. Honor crawl delays. Honor `Disallow:`.
- Respect rate limits; add jitter; back off on `429`.
- Don't scrape behind [[authentication]] unless you have explicit permission.
- Don't scrape personal data (names, emails, photos) without a lawful basis.
- Identify yourself with a descriptive User-Agent and a contact URL when crawling at volume.
- Cache aggressively to avoid redundant requests.
- Stop if you receive a cease-and-desist or explicit blocking signal — escalating past one is the move that turns a civil dispute into a CFAA case.

Notes on specific platforms. Instagram's `instaloader` and TikTok scraping via `yt-dlp` work today but break frequently — Meta and TikTok roll out anti-bot updates monthly. Account bans on the credentials you used are common. For journalism, the official APIs (Meta Content Library, TikTok Research API) are slower but more durable.

web-ui-best-practices (Skill Module)

dev-toolkit\skills\web-ui-best-practices\SKILL.md

name: web-ui-best-practices

description: Signs of taste in web UI. Use when building or reviewing any user-facing web interface — dashboards, SaaS apps, marketing sites, internal tools. Covers interaction speed, navigation depth, visual restraint, copy quality, and the small details that separate polished products from rough ones.

tags:

Web UI best practices

Principles for building web interfaces that feel fast, intentional, and respectful of the user's time. Every rule here is a smell test — violating one is fine if you have a reason, violating several means the UI needs work.

Speed

Every interaction completes in under 100ms. If it can't, fake it.

- Optimistic UI updates — show the result before the server confirms
- Debounce inputs, but never debounce perceived response
- Prefetch likely next routes on hover or viewport entry
- Use `will-change` and `transform` for animations, never `top` / `left`
- Measure with `performance.now()`, not gut feel

```
// Optimistic delete — remove from UI immediately, reconcile later
async function handleDelete(id) {
  setItems(prev => prev.filter(i => i.id !== id));
  try {
    await api.delete(`/items/${id}`);
  } catch {
    setItems(prev => [...prev, originalItem]);
    toast("Couldn't delete. Restored.");
  }
}
```

Skeleton loading states

Never show a spinner when you know the shape of what's coming. Render a skeleton that matches the layout, then swap in real content.

```
.skeleton {
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
  border-radius: 4px;
}

@keyframes shimmer {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}
```

Modern CSS toolkit

Four capabilities matured between 2023 and 2026 that change how you build component-level responsive layouts and SPA-like transitions without JavaScript. Reach for them before adding a framework.

Container queries

Container queries let a component respond to **its container's** size, not the viewport's. The same card can render in a 300px sidebar and a 900px main column without media-query coordination at the page level.

```
.card-list {
  container-type: inline-size;
  container-name: cards;
}

@container cards (min-width: 480px) {
  .card { display: grid; grid-template-columns: 120px 1fr; }
}
```

Stable in all major browsers since 2023. Replaces most "the same component in two places needs to look different" hacks.

`:has()` parent selector

`:has()` lets a parent style itself based on its descendants — the long-requested "parent selector." Useful for marking a form field as in-error, a card as having an attached image, or a row as containing a focused input — all without JS.

```
/* Highlight a form group when its input has focus */
.form-group:has(input:focus) {
  outline: 2px solid var(--color-primary);
}

/* Add bottom margin to articles that contain a figure */
article:has(figure) {
  margin-bottom: 2rem;
}
```

Stable in Chrome, Safari, and Firefox since late 2023. Cuts a real category of JS-driven class toggling.

View transitions

The View Transitions API animates between two DOM states (route changes, modal open/close, list-item swaps) without a framework. The browser snapshots the old state, swaps in the new state, then crossfades or [[slides]] between them.

```
// Same-document transition (Chrome 111+, Safari TP, Firefox behind a flag)
function navigate(newView) {
  if (!document.startViewTransition) {
    renderView(newView);
    return;
  }
  document.startViewTransition(() => renderView(newView));
}
```

```
/* Smooth crossfade by default; override per element */
::view-transition-old(*) { animation-duration: 200ms; }
::view-transition-new(*) { animation-duration: 200ms; }
```

Cross-document view transitions (between full page navigations) shipped to Chrome 126 in 2024 and let MPAs feel like SPAs. Pair with `prefers-reduced-motion` so users with motion sensitivity get an instant swap, not an animation.

Scroll-driven animations

`animation-timeline: scroll()` and `animation-timeline: view()` drive CSS animations from scroll position instead of wall-clock time. The classic use case is a progress indicator at the top of an article that fills as you scroll.

```
@keyframes fill { from { transform: scaleX(0); } to { transform: scaleX(1); } }

.read-progress {
  position: fixed; top: 0; left: 0; right: 0; height: 3px;
  background: var(--color-primary);
  transform-origin: left;
  animation: fill linear;
  animation-timeline: scroll(root);
}
```

Stable in Chromium-based browsers (Chrome 115+, Edge); not yet in Safari or Firefox as of 2026-05. Use as progressive enhancement; provide a JS fallback or accept a less-flashy baseline elsewhere.

No product tours

If you need a tour to explain your UI, the UI is wrong. Instead:

- Empty states that teach by doing ("Create your first project")

- Progressive disclosure — show features when they become relevant
- Inline hints that disappear after first use
- Defaults that work without configuration

URLs

Slugs are short, readable, and human-guessable. No UUIDs, no query param soup.

```
Good:  /projects/weather-app
       /settings/billing
       /docs/api/auth

Bad:   /projects/550e8400-e29b-41d4-a716-446655440000
       /app?view=settings&tab=billing&subsection=plan
       /dashboard#!/module/documents/list?filter=active
```

- Use slugs derived from user-provided names
- Keep nesting to 3 segments max
- Make URLs copyable and shareable — they are the product's memory

Persistent resumable state

Users leave and come back. Respect that.

- Save draft form state to `localStorage` or the server
- Restore scroll position on back navigation
- Preserve filter/sort selections across sessions
- URL encodes the current view state — sharing a URL reproduces the view

```
// Persist form state across sessions
function usePersistentForm(key, defaults) {
  const [state, setState] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : defaults;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(state));
  }, [key, state]);

  return [state, setState];
}
```

Color restraint

Not more than 3 colors. One primary, one accent, one for danger/destructive. Everything else is shades of gray.

```
:root {
  --color-primary: #2563eb;
  --color-accent: #f59e0b;
  --color-danger: #ef4444;

  --gray-50: #fafafa;
  --gray-100: #f4f4f5;
  --gray-200: #e4e4e7;
  --gray-400: #a1a1aa;
  --gray-600: #52525b;
  --gray-900: #18181b;
}
```

- Use opacity and lightness to create hierarchy, not new hues
- Dark mode is the same 3 colors with inverted grays
- If you reach for a 4th color, you're compensating for weak layout

No visible scrollbars

Hide them unless the user is actively scrolling. Content feels infinite, not trapped.

```
/* Hide scrollbar across browsers */
.scroll-container {
  overflow-y: auto;
  scrollbar-width: none;          /* Firefox */
  -ms-overflow-style: none;       /* IE/Edge */
}
.scroll-container::-webkit-scrollbar {
  display: none;                  /* Chrome/Safari */
}
```

Use scroll shadows to hint at overflow without chrome:

```
.scroll-shadow {
  background:
    linear-gradient(white 30%, transparent),
    linear-gradient(transparent, white 70%) 0 100%,
    radial-gradient(farthest-side at 50% 0, rgba(0,0,0,.15), transparent),
    radial-gradient(farthest-side at 50% 100%, rgba(0,0,0,.15), transparent) 0 100%;
  background-repeat: no-repeat;
  background-size: 100% 40px, 100% 40px, 100% 12px, 100% 12px;
  background-attachment: local, local, scroll, scroll;
}
```

Navigation depth

All navigation is 3 steps or fewer from anywhere. If the user needs more than 3 clicks to reach a destination, flatten the hierarchy.

- Breadcrumbs for depth, not for navigation
- Global nav always visible, never hidden behind a hamburger on desktop
- Use `Cmd+K` / `Ctrl+K` as the escape hatch for power users

Command palette (Cmd+K)

Every app with more than one page needs a command palette.

```
// Minimal Cmd+K listener
useEffect(() => {
  function handleKeyDown(e) {
    if ((e.metaKey || e.ctrlKey) && e.key === "k") {
      e.preventDefault();
      setCommandPaletteOpen(true);
    }
  }
  document.addEventListener("keydown", handleKeyDown);
  return () => document.removeEventListener("keydown", handleKeyDown);
}, []);
```

Keep the palette simple:
- Fuzzy search over page names, recent actions, settings
- Show keyboard shortcuts inline
- Most recent items first
- No categories until you have 20+ commands

Clipboard

Copy and paste should work everywhere the user expects it.

- One-click copy on codes, URLs, API keys, IDs
- Paste from clipboard into file uploads, image fields
- Show brief confirmation on copy ("Copied!") that auto-dismisses

```
async function copyToClipboard(text, label = "Copied") {
  await navigator.clipboard.writeText(text);
  toast(label, { duration: 1500 });
}
```

Hit targets

Larger hit targets for buttons and inputs. WCAG 2.2 (success criterion 2.5.8) sets the floor at **24×24 CSS pixels**; Apple's Human Interface Guidelines and most native iOS/Android conventions recommend **44×44 points** as the comfortable target. Use 44px as the working minimum for primary actions; 24px as the absolute legal floor for secondary controls (e.g., dense table-row icons).

```
button, .btn, [role="button"] {
  min-height: 44px;
  min-width: 44px;
  padding: 10px 20px;
}

input, select, textarea {
  min-height: 44px;
  padding: 10px 12px;
  font-size: 16px;  /* Prevents iOS Safari zoom on focus */
}
```

- Adjacent clickable elements need at least 8px gap
- Icon-only buttons get larger padding than labeled buttons
- Don't rely on hover states for critical affordances — they don't exist on touch

Honest cancellation

One-click cancel. No guilt trips, no dark patterns, no "Are you sure you want to miss out?"

- Cancel button is always visible alongside confirm
- Account deletion works on the first try
- Unsubscribe is one click, not a preference center
- Downgrade flows don't require contacting support

Tooltips

Very minimal. Tooltips are a confession that the UI doesn't speak for itself.

- Only on icon-only buttons (to provide the label)
- Never on text that's already readable
- Show on hover after 300ms delay, not instantly
- Dismiss on scroll
- Never use tooltips for essential information

Copy

Active voice. Max 7 words per sentence. Talk like a person, not a legal document.

```
Good:  "Project created"
       "Saved 2 minutes ago"
       "Delete this file?"

Bad:   "Your project has been successfully created!"
       "Changes were last saved approximately 2 minutes ago"
       "Are you sure you want to permanently delete this file? This action cannot be undone."
```

- Buttons are verbs: "Save", "Delete", "Send" — not "Submit", "OK", "Confirm"
- Error messages say what happened and what to do next
- Never blame the user ("Invalid input" → "Enter a valid email")
- Use sentence case everywhere, never Title Case in UI copy

Optical alignment

Optical alignment over geometric alignment. The eye doesn't see pixels, it sees weight.

- Play icons shift 2-3px right inside circles to look centered
- Text with leading capital letters aligns optically left of its bounding box
- Icons next to text need 1-2px vertical offset depending on the glyph
- Padding around text is visually balanced, not mathematically equal — bottom padding is often 1-2px more than top

```
/* Geometric center ≠ optical center */
.play-button svg {
  transform: translateX(2px);
}

/* Visually balanced card padding */
.card {
  padding: 20px 24px 22px 24px;
}
```

Left-to-right reading flow

Optimized for L-to-R reading and the F-pattern scan.

- Most important content in the top-left quadrant
- Primary actions on the right (where the eye ends a line)
- Labels above inputs, not beside them
- Tables: most-scanned column is leftmost
- Don't center-align body text — left-align everything except single-line headings

Reassurance about loss

Users fear losing work. Prevent it and prove it.

- Auto-save with visible "Saved" indicator and timestamp
- Undo after destructive actions (soft delete, not hard delete)
- "You have unsaved changes" on navigation away
- Version history for anything longer than a tweet
- Confirmation only for irreversible actions, not routine ones

```
// Warn on unsaved changes
useEffect(() => {
  function handleBeforeUnload(e) {
    if (hasUnsavedChanges) {
      e.preventDefault();
      e.returnValue = "";
    }
  }
  window.addEventListener("beforeunload", handleBeforeUnload);
  return () => window.removeEventListener("beforeunload", handleBeforeUnload);
}, [hasUnsavedChanges]);
```

Copyable brand assets

Ship a `/brand` or `/press` page with a downloadable SVG logo and brand kit. Don't make people screenshot your logo.

- SVG logo with transparent background
- Color codes (hex, RGB, HSL)
- Font names and weights
- Usage guidelines (minimum size, clear space, don'ts)
- One-click download as ZIP

Checklist

Use this when reviewing any web UI:

- [ ] Every interaction under 100ms (or optimistically faked)
- [ ] No product tour or onboarding modal
- [ ] URLs are short, readable, no UUIDs
- [ ] State persists across sessions and page reloads
- [ ] 3 colors max (plus grays)
- [ ] No visible scrollbars at rest
- [ ] Any destination reachable in 3 steps or fewer
- [ ] SVG logo and brand kit downloadable
- [ ] Skeleton loaders, not spinners
- [ ] Clipboard copy works on codes, keys, URLs
- [ ] Touch targets 44px minimum

- [ ] Cancel is honest and one-click
- [ ] Cmd+K command palette exists
- [ ] Tooltips only on icon-only buttons
- [ ] Copy is active voice, 7 words max
- [ ] Optical alignment, not geometric
- [ ] Content follows L-to-R F-pattern
- [ ] Auto-save with visible [[status]] and undo

writing-plans (Skill Module)

`superjawn\skills\writing-plans\SKILL.md`

name: writing-plans
description: Use when you have a spec or requirements for a multi-step task, before touching code
tags:

Writing Plans

Overview

Write comprehensive implementation plans assuming the engineer has zero context for our codebase and questionable taste. Document everything they need to know: which files to touch for each task, code, testing, docs they might need to check, how to test it. Give them the whole plan as bite-sized tasks. DRY. YAGNI. TDD. Frequent commits.

Assume they are a skilled developer, but know almost nothing about our toolset or problem domain. Assume they don't know good test design very well.

Announce at start: "I'm using the superjawn:writing-plans [[skill]] to create the implementation plan."

Context: This should be run in a dedicated worktree (set up via superjawn:using-git-worktrees).

Save plans to: `docs/[[superpowers]]/plans/YYYY-MM-DD-<feature-name>.md`
- (User preferences for plan location override this default)

Scope Check

If the spec covers multiple independent subsystems, it should have been broken into sub-project specs during brainstorming. If it wasn't, suggest breaking this into separate plans — one per subsystem. Each plan should produce working, testable software on its own.

[[File Structure]]

Before defining tasks, map out which files will be created or modified and what each one is responsible for. This is where decomposition decisions get locked in.

- Design units with clear boundaries and well-defined interfaces. Each file should have one clear responsibility.
- You reason best about code you can hold in context at once, and your edits are more reliable when files are focused. Prefer smaller, focused files over large ones that do too much.
- Files that change together should live together. Split by responsibility, not by technical layer.
- In existing codebases, follow established patterns. If the codebase uses large files, don't unilaterally restructure - but if a file you're modifying has grown unwieldy, including a split in the plan is reasonable.

This structure informs the task decomposition. Each task should produce self-contained changes that make sense independently.

Bite-Sized Task Granularity

Each step is one action (2-5 minutes):
- "Write the failing test" - step
- "Run it to make sure it fails" - step
- "Implement the minimal code to make the test pass" - step
- "Run the tests and make sure they pass" - step
- "Commit" - step

Plan Document Header

Every plan MUST start with this header:

````
# [Feature Name] Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superjawn:subagent-driven-development (recommended) or superjawn:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

**Goal:** [One sentence describing what this builds]

**Architecture:** [2-3 sentences about approach]

**Tech Stack:** [Key technologies/libraries]

---
````

Task Structure

````
### Task N: [Component Name]

**Files:**
- Create: `exact/path/to/file.py`
- Modify: `exact/path/to/existing.py:123-145`
- Test: `tests/exact/path/to/test.py`

- [ ] **Step 1: Write the failing test**

```python
def test_specific_behavior():
    result = function(input)
    assert result == expected
```

- [ ] **Step 2: Run test to verify it fails**

Run: `pytest tests/path/test.py::test_name -v`
Expected: FAIL with "function not defined"

- [ ] **Step 3: Write minimal implementation**

```python
def function(input):
    return expected
```

- [ ] **Step 4: Run test to verify it passes**

Run: `pytest tests/path/test.py::test_name -v`
Expected: PASS

- [ ] **Step 5: Commit**

```bash
git add tests/path/test.py src/path/file.py
git commit -m "feat: add specific feature"
```
````

No Placeholders

Every step must contain the actual content an engineer needs. These are **plan failures** — never write them:
- "TBD", "TODO", "implement later", "fill in details"
- "Add appropriate error handling" / "add validation" / "handle edge cases"
- "Write tests for the above" (without actual test code)
- "Similar to Task N" (repeat the code — the engineer may be reading tasks out of order)
- Steps that describe what to do without showing how (code blocks required for code steps)
- References to types, functions, or methods not defined in any task

Remember

- Exact file paths always
- Complete code in every step — if a step changes code, show the code
- Exact commands with expected output
- DRY, YAGNI, TDD, frequent commits

Self-[[Review]]

After writing the complete plan, look at the spec with fresh eyes and check the plan against it. This is a checklist you run yourself — not a subagent dispatch.

1. Spec coverage: Skim each section/requirement in the spec. Can you point to a task that implements it? List any gaps.

2. Placeholder scan: Search your plan for red flags — any of the patterns from the "No Placeholders" section above. Fix them.

3. Type consistency: Do the types, method signatures, and property names you used in later tasks match what you defined in earlier tasks? A function called `clearLayers()` in Task 3 but `clearFullLayers()` in Task 7 is a bug.

If you find issues, fix them inline. No need to re-[[review]] — just fix and move on. If you find a spec requirement with no task, add the task.

Execution [[Handoff]]

After saving the plan, offer execution choice:

"Plan complete and saved to `docs/[[superpowers]]/plans/<filename>.md`. Two execution options:

1. Subagent-Driven (recommended) - I dispatch a fresh subagent per task, [[review]] between tasks, fast iteration

2. Inline Execution - Execute tasks in this session using superjawn:executing-plans, batch execution with checkpoints

Which approach?"

If Subagent-Driven chosen:
- **REQUIRED SUB-[[SKILL]]:** Use superjawn:subagent-driven-development
- Fresh subagent per task + two-stage [[review]]

If Inline Execution chosen:
- **REQUIRED SUB-[[SKILL]]:** Use superjawn:executing-plans
- Batch execution with checkpoints for [[review]]

writing-skills (Skill Module)

superjawn\skills\writing-skills\SKILL.md

name: writing-skills
description: Use when creating new skills, editing existing skills, or verifying skills work before deployment
tags:

Writing [[Skills]]

Overview

Writing [[skills]] IS Test-Driven Development applied to process documentation.

Personal [[skills]] live in agent-specific directories (`~/.claude/[[skills]]` for Claude Code, `~/.[[agents]]/[[skills]]/` for Codex)

You write test cases (pressure scenarios with subagents), watch them fail (baseline behavior), write the [[skill]] (documentation), watch tests pass ([[agents]] comply), and refactor (close loopholes).

Core principle: If you didn't watch an agent fail without the [[skill]], you don't know if the [[skill]] teaches the right thing.

REQUIRED BACKGROUND: You MUST understand superjawn:test-driven-development before using this [[skill]]. That [[skill]] defines the fundamental RED-GREEN-REFACTOR cycle. This [[skill]] adapts TDD to documentation.

Official guidance: For Anthropic's official [[skill]] authoring best practices, see [[anthropic-best-practices]].md. This document provides additional patterns and guidelines that complement the TDD-focused approach in this [[skill]].

What is a [[Skill]]?

A **[[skill]]** is a reference guide for proven techniques, patterns, or tools. [[Skills]] help future Claude instances find and apply effective approaches.

[[Skills]] are: Reusable techniques, patterns, tools, reference guides

[[Skills]] are NOT: Narratives about how you solved a problem once

TDD Mapping for [[Skills]]

TDD Concept	[[Skill]] Creation
Test case	Pressure scenario with subagent
Production code	[[Skill]] document ([[SKILL]].md)
Test fails (RED)	Agent violates rule without [[skill]] (baseline)
Test passes (GREEN)	Agent complies with [[skill]] present
Refactor	Close loopholes while maintaining compliance
Write test first	Run baseline scenario BEFORE writing [[skill]]
Watch it fail	Document exact rationalizations agent uses
Minimal code	Write [[skill]] addressing those specific violations
Watch it pass	Verify agent now complies
Refactor cycle	Find new rationalizations → plug → re-verify

The entire [[skill]] creation process follows RED-GREEN-REFACTOR.

When to Create a [[Skill]]

Create when:
- Technique wasn't intuitively obvious to you
- You'd reference this again across projects
- Pattern applies broadly (not project-specific)
- Others would benefit

Don't create for:
- One-off solutions
- Standard practices well-documented elsewhere

- Project-specific conventions (put in CLAUDE.md)
- Mechanical constraints (if it's enforceable with regex/validation, automate it—save documentation for judgment calls)

Research phase

Before writing tests for a new [[skill]], gather outside context. This is **default-on**: skip only with explicit, justified statement.

The aim is to ground the [[skill]] design in real prior art and current best practice, not just your own recall of patterns.

1. Pick research kinds

From the menu — patterns + best practices, prior art, authoritative guidance, user-context.

For writing-[[skills]], the **defaults are: web ([[skill]] authoring patterns + recent discourse) and codebase (prior art — does this overlap with an existing [[skill]] in this repo or a sibling plugin?)**. Add others when warranted — authoritative when the [[skill]] encodes a specific external standard (W3C, RFC, vendor docs), or user-context when prior decisions in memory shape the right shape for the [[skill]].

2. Dispatch

Subagent by default:
- `Explore` for codebase / prior-art questions ("does this repo or any installed plugin already have a [[skill]] for X?", "what naming convention do existing [[skills]] use here?")
- `general-purpose` for web / authoring patterns ("what shape do effective Claude Code [[skills]] take?", "common pitfalls for [[skills]] in this domain?")
- Run multiple in parallel when the kinds are independent

Inline only for light-touch research (single grep across `~/.claude/[[skills]]/`, memory check).

3. Record findings

Findings land at `.[[superpowers]]/[[skill]]-design-<[[skill]]-slug>.md` where `<[[skill]]-slug>` is the kebab-case name of the [[skill]] you are designing. Write 3–5 tight bullets — load-bearing links/refs, prior-art notes, anything considered-but-ruled-out so future-you knows it was checked. The directory `.[[superpowers]]/` is git-ignored by upstream convention.

4. Skip protocol

If skipping, write one line to `.[[superpowers]]/[[skill]]-design-<[[skill]]-slug>.md`: `Skipped research because <reason>. <Verifiable pointer if applicable>.`

Valid reasons:
- Trivial scope (typo, comment edit, single-line config)
- Fresh prior research — same topic in current session OR within last 7 days with verifiable spec/plan pointer. **If the pointer doesn't resolve, the skip is invalid.** (Beyond 7 days, repeat the research even if you remember the prior findings — the landscape drifts.)
- User explicit — **must quote the phrase** that authorized the skip.
- Repeat of identical task — **must include a pointer** to the prior successful run.

Invalid reasons: "I think I know", "seems straightforward", "moving fast", "user wants this done quickly", "already familiar with this codebase". If those are tempting, do the research.

[[Skill]] Types

Technique

Concrete method with steps to follow ([[condition-based-waiting]], [[root-cause-tracing]])

Pattern

Way of thinking about problems (flatten-with-flags, test-invariants)

Reference

API docs, syntax guides, tool documentation (office docs)

Directory Structure

```
skills/
  skill-name/
    SKILL.md              # Main reference (required)
    supporting-file.*     # Only if needed
```

Flat namespace - all [[skills]] in one searchable namespace

Separate files for:
1. **Heavy reference** (100+ lines) - API docs, comprehensive syntax
2. **Reusable tools** - Scripts, utilities, templates

Keep inline:
- Principles and concepts
- Code patterns (< 50 lines)
- Everything else

[[SKILL]].md Structure

Frontmatter (YAML):
- Two required fields: `name` and `description` (see agentskills.io/specification for all supported fields)
- Max 1024 characters total
- `name`: Use letters, numbers, and hyphens only (no parentheses, special chars)
- `description`: Third-person, describes ONLY when to use (NOT what it does)
- Start with "Use when..." to focus on triggering conditions
- Include specific symptoms, situations, and contexts
- **NEVER summarize the [[skill]]'s process or workflow** (see CSO section for why)
- Keep under 500 characters if possible

```
---
name: Skill-Name-With-Hyphens
description: Use when [specific triggering conditions and symptoms]
---

# Skill Name

## Overview
What is this? Core principle in 1-2 sentences.

## When to Use
[Small inline flowchart IF decision non-obvious]

Bullet list with SYMPTOMS and use cases
When NOT to use

## Core Pattern (for techniques/patterns)
Before/after code comparison

## Quick Reference
Table or bullets for scanning common operations

## Implementation
Inline code for simple patterns
Link to file for heavy reference or reusable tools

## Common Mistakes
What goes wrong + fixes

## Real-World Impact (optional)
Concrete results
```

Claude Search Optimization (CSO)

Critical for discovery: Future Claude needs to FIND your [[skill]]

1. Rich Description Field

Purpose: Claude reads description to decide which [[skills]] to load for a given task. Make it answer: "Should I read this [[skill]] right now?"

Format: Start with "Use when..." to focus on triggering conditions

CRITICAL: Description = When to Use, NOT What the [[Skill]] Does

The description should ONLY describe triggering conditions. Do NOT summarize the [[skill]]'s process or workflow in the description.

Why this matters: Testing revealed that when a description summarizes the [[skill]]'s workflow, Claude may follow the description instead of reading the full [[skill]] content. A description saying "code [[review]] between tasks" caused Claude to do ONE [[review]], even though the [[skill]]'s flowchart clearly showed TWO reviews (spec compliance then code quality).

When the description was changed to just "Use when executing implementation plans with independent tasks" (no workflow summary), Claude correctly read the flowchart and followed the two-stage [[review]] process.

The trap: Descriptions that summarize workflow create a shortcut Claude will take. The [[skill]] body becomes documentation Claude skips.

```
#  BAD: Summarizes workflow - Claude may follow this instead of reading skill
description: Use when executing plans - dispatches subagent per task with code review between tasks

#  BAD: Too much process detail
description: Use for TDD - write test first, watch it fail, write minimal code, refactor

#  GOOD: Just triggering conditions, no workflow summary
description: Use when executing implementation plans with independent tasks in the current session

#  GOOD: Triggering conditions only
description: Use when implementing any feature or bugfix, before writing implementation code
```

Content:
- Use concrete triggers, symptoms, and situations that signal this [[skill]] applies
- Describe the *problem* (race conditions, inconsistent behavior) not *language-specific symptoms* (setTimeout, sleep)
- Keep triggers technology-agnostic unless the [[skill]] itself is technology-specific
- If [[skill]] is technology-specific, make that explicit in the trigger
- Write in third person (injected into system prompt)
- **NEVER summarize the [[skill]]'s process or workflow**

```
#  BAD: Too abstract, vague, doesn't include when to use
description: For async testing

#  BAD: First person
description: I can help you with async tests when they're flaky

#  BAD: Mentions technology but skill isn't specific to it
description: Use when tests use setTimeout/sleep and are flaky

#  GOOD: Starts with "Use when", describes problem, no workflow
description: Use when tests have race conditions, timing dependencies, or pass/fail inconsistently

#  GOOD: Technology-specific skill with explicit trigger
description: Use when using React Router and handling authentication redirects
```

2. Keyword Coverage

Use words Claude would search for:
- Error messages: "Hook timed out", "ENOTEMPTY", "race condition"
- Symptoms: "flaky", "hanging", "zombie", "pollution"
- Synonyms: "timeout/hang/freeze", "cleanup/teardown/afterEach"
- Tools: Actual commands, library names, file types

3. Descriptive Naming

Use active voice, verb-first:
- `creating-[[skills]]` not `[[skill]]-creation`
- `[[condition-based-waiting]]` not `async-test-helpers`

4. Token Efficiency (Critical)

Problem: getting-started and frequently-referenced [[skills]] load into EVERY conversation. Every token counts.

Target word counts:
- getting-started [[[[

```
#  BAD: Repeat workflow details
When searching, dispatch subagent with template...
[20 lines of repeated instructions]

#  GOOD: Reference other skill
Always use subagents (50-100x context savings). REQUIRED: Use [other-skill-name] for workflow.
```

Compress examples:

```
#  BAD: Verbose example (42 words)
your human partner: "How did we handle authentication errors in React Router before?"
You: I'll search past conversations for React Router authentication patterns.
[Dispatch subagent with search query: "React Router authentication error handling 401"]

#  GOOD: Minimal example (20 words)
Partner: "How did we handle auth errors in React Router?"
You: Searching...
[Dispatch subagent → synthesis]
```

Eliminate redundancy:
- Don't repeat what's in cross-referenced [[skills]]
- Don't explain what's obvious from command
- Don't include multiple examples of same pattern

Verification:

```
wc -w skills/path/SKILL.md
# getting-started workflows: aim for <150 each
# Other frequently-loaded: aim for <200 total
```

Name by what you DO or core insight:
- `[[condition-based-waiting]]` > `async-test-helpers`
- `using-[[skills]]` not `[[skill]]-usage`
- `flatten-with-flags` > `data-structure-refactoring`
- `[[root-cause-tracing]]` > `debugging-techniques`

Gerunds (-ing) work well for processes:
- `creating-[[skills]]` , `testing-[[skills]]` , `debugging-with-logs`
- Active, describes the action you're taking

4. Cross-Referencing Other [[Skills]]

When writing documentation that references other [[skills]]:

Use [[skill]] name only, with explicit requirement markers:
- Good: `**REQUIRED SUB-[[SKILL]]:** Use superjawn:test-driven-development`
- Good: `**REQUIRED BACKGROUND:** You MUST understand superjawn:systematic-debugging`
- Bad: `See [[skills]]/testing/test-driven-development` (unclear if required)
- Bad: `@[[skills]]/testing/test-driven-development/[[SKILL]].md` (force-loads, burns context)

Why no @ links: `@` syntax force-loads files immediately, consuming 200k+ context before you need them.

Flowchart Usage

```
digraph when_flowchart {
    "Need to show information?" [shape=diamond];
    "Decision where I might go wrong?" [shape=diamond];
    "Use markdown" [shape=box];
    "Small inline flowchart" [shape=box];

    "Need to show information?" -> "Decision where I might go wrong?" [label="yes"];
    "Decision where I might go wrong?" -> "Small inline flowchart" [label="yes"];
    "Decision where I might go wrong?" -> "Use markdown" [label="no"];
}
```

Use flowcharts ONLY for:
- Non-obvious decision points
- Process loops where you might stop too early
- "When to use A vs B" decisions

Never use flowcharts for:
- Reference material → Tables, lists

- Code examples → Markdown blocks
- Linear instructions → Numbered lists
- Labels without semantic meaning (step1, helper2)

See @graphviz-conventions.dot for graphviz style rules.

Visualizing for your human partner: Use `render-graphs.js` in this directory to render a [[skill]]'s flowcharts to SVG:

```
./render-graphs.js ../some-skill          # Each diagram separately
./render-graphs.js ../some-skill --combine # All diagrams in one SVG
```

Code Examples

One excellent example beats many mediocre ones

Choose most relevant language:
- Testing techniques → TypeScript/JavaScript
- System debugging → Shell/Python
- Data processing → Python

Good example:
- Complete and runnable
- Well-commented explaining WHY
- From real scenario
- Shows pattern clearly
- Ready to adapt (not generic

[[[

Write [[skill]] before testing? Delete it. Start over.
Edit [[skill]] without testing? Same violation.

No exceptions:
- Not for "simple additions"
- Not for "just adding a section"
- Not for "documentation updates"
- Don't keep untested changes as "reference"
- Don't "adapt" while running tests
- Delete means delete

REQUIRED BACKGROUND: The superjawn:test-driven-development [[skill]] explains why this matters. Same principles apply to documentation.

Testing All [[Skill]] Types

Different [[skill]] types need different test approaches:

Discipline-Enforcing [[Skills]] (rules/requirements)

Examples: TDD, verification-before-completion, designing-before-coding

Test with:
- Academic questions: Do they understand the rules?
- Pressure scenarios: Do they comply under stress?
- Multiple pressures combined: time + sunk cost + exhaustion
- Identify rationalizations and add explicit counters

Success criteria: Agent follows rule under maximum pressure

Technique [[Skills]] (how-to guides)

Examples: [[condition-based-waiting]], [[root-cause-tracing]], defensive-programming

Test with:
- Application scenarios: Can they apply the technique correctly?
- Variation scenarios: Do they handle edge cases?
- Missing information tests: Do instructions have gaps?

Success criteria: Agent successfully applies technique to new scenario

Pattern [[Skills]] (mental models)

Examples: reducing-complexity, information-hiding concepts

Test with:
- Recognition scenarios: Do they recognize when pattern applies?
- Application scenarios: Can they use the mental model?
- Counter-examples: Do they know when NOT to apply?

Success criteria: Agent correctly identifies when/how to apply pattern

Reference [[Skills]] (documentation/APIs)

Examples: API documentation, command references, library guides

Test with:
- Retrieval scenarios: Can they find the right information?
- Application scenarios: Can they use what they found correctly?
- Gap testing: Are common use cases covered?

Success criteria: Agent finds and correctly applies reference information

Common Rationalizations for Skipping Testing

Excuse	Reality
"[[Skill]] is obviously clear"	Clear to you ≠ clear to other [[agents]]. Test it.
"It's just a reference"	References can have gaps, unclear sections. Test retrieval.
"Testing is overkill"	Untested [[skills]] have issues. Always. 15 min testing saves hours.
"I'll test if problems emerge"	Problems = [[agents]] can't use [[skill]]. Test BEFORE deploying.
"Too tedious to test"	Testing is less tedious than debugging bad [[skill]] in production.
"I'm confident it's good"	Overconfidence guarantees issues. Test anyway.
"Academic [[review]] is enough"	Reading ≠ using. Test application scenarios.
"No time to test"	Deploying untested [[skill]] wastes more time fixing it later.

All of these mean: Test before deploying. No exceptions.

Bulletproofing [[Skills]] Against Rationalization

[[Skills]] that enforce discipline (like TDD) need to resist rationalization. [[Agents]] are smart and will find loopholes when under pressure.

Psychology note: Understanding WHY persuasion techniques work helps you apply them systematically. See [[persuasion-principles]].md for research foundation (Cialdini, 2021; Meincke et al., 2025) on authority, commitment, scarcity, social proof, and unity principles.

Close Every Loophole Explicitly

Don't just state the rule - forbid specific workarounds:

```
Write code before test? Delete it.
```

```
Write code before test? Delete it. Start over.

**No exceptions:**
- Don't keep it as "reference"
- Don't "adapt" it while writing tests
- Don't look at it
- Delete means delete
```

Address "Spirit vs Letter" Arguments

Add foundational principle early:

```
**Violating the letter of the rules is violating the spirit of the rules.**
```

This cuts off entire class of "I'm following the spirit" rationalizations.

Build Rationalization Table

Capture rationalizations from baseline testing (see Testing section below). Every excuse [[agents]] make goes in the table:

```
| Excuse | Reality |
|--------|---------|
| "Too simple to test" | Simple code breaks. Test takes 30 seconds. |
| "I'll test after" | Tests passing immediately prove nothing. |
| "Tests after achieve same goals" | Tests-after = "what does this do?" Tests-first = "what should this do?" |
```

Create Red Flags List

Make it easy for [[agents]] to self-check when rationalizing:

```
## Red Flags - STOP and Start Over

- Code before test
- "I already manually tested it"
- "Tests after achieve the same purpose"
- "It's about spirit not ritual"
- "This is different because..."

**All of these mean: Delete code. Start over with TDD.**
```

[[Update]] CSO for Violation Symptoms

Add to description: symptoms of when you're ABOUT to violate the rule:

```
description: use when implementing any feature or bugfix, before writing implementation code
```

RED-GREEN-REFACTOR for [[Skills]]

Follow the TDD cycle:

RED: Write Failing Test (Baseline)

Run pressure scenario with subagent WITHOUT the [[skill]]. Document exact behavior:
- What choices did they make?
- What rationalizations did they use (verbatim)?
- Which pressures triggered violations?

This is "watch the test fail" - you must see what [[agents]] naturally do before writing the [[skill]].

GREEN: Write Minimal [[Skill]]

Write [[skill]] that addresses those specific rationalizations. Don't add extra content for hypothetical cases.

Run same scenarios WITH [[skill]]. Agent should now comply.

REFACTOR: Close Loopholes

Agent found new rationalization? Add explicit counter. Re-test until bulletproof.

Testing methodology: See @[[testing-[[skills]]-with-subagents]].md for the complete testing methodology:
- How to write pressure scenarios
- Pressure types (time, sunk cost, authority, exhaustion)
- Plugging holes systematically
- Meta-testing techniques

Anti-Patterns

Narrative Example

"In session 2025-10-03, we found empty projectDir caused..."
Why bad: Too specific, not reusable

Multi-Language Dilution

example-js.js, example-py.py, example-go.go
Why bad: Mediocre quality, maintenance burden

Code in Flowcharts

```
step1 [label="import fs"];
step2 [label="read file"];
```

Why bad: Can't copy-paste, hard to read

Generic Labels

helper1, helper2, step3, pattern4
Why bad: Labels should have semantic meaning

STOP: Before Moving to Next [[Skill]]

After writing ANY [[skill]], you MUST STOP and complete the deployment process.

Do NOT:
- Create multiple [[skills]] in batch without testing each
- Move to next [[skill]] before current one is verified
- Skip testing because "batching is more efficient"

The deployment checklist below is MANDATORY for EACH [[skill]].

Deploying untested [[skills]] = deploying untested code. It's a violation of quality
[[[

[[Skill]] Creation Checklist (TDD Adapted)

IMPORTANT: Use TodoWrite to create todos for EACH checklist item below.

RED Phase - Write Failing Test:
- [ ] Create pressure scenarios (3+ combined pressures for discipline skills)
- [ ] Run scenarios WITHOUT skill - document baseline behavior verbatim
- [ ] Identify patterns in rationalizations/failures

GREEN Phase - Write Minimal Skill:
- [ ] Name uses only letters, numbers, hyphens (no parentheses/special chars)
- [ ] YAML frontmatter with required `name` and `description` fields (max 1024 chars; see spec)
- [ ] Description starts with "Use when..." and includes specific triggers/symptoms
- [ ] Description written in third person
- [ ] Keywords throughout for search (errors, symptoms, tools)
- [ ] Clear overview with core principle
- [ ] Address specific baseline failures identified in RED
- [ ] Code inline OR link to separate file
- [ ] One excellent example (not multi-language)
- [ ] Run scenarios WITH [[skill]] - verify [[agents]] now comply

REFACTOR Phase - Close Loopholes:
- [ ] Identify NEW rationalizations from testing
- [ ] Add explicit counters (if discipline [[skill]])
- [ ] Build rationalization table from all test iterations
- [ ] Create red flags list
- [ ] Re-test until bulletproof

Quality Checks:
- [ ] Small flowchart only if decision non-obvious
- [ ] Quick reference table
- [ ] Common mistakes section
- [ ] No narrative storytelling
- [ ] Supporting files only for tools or heavy reference

Deployment:
- [ ] Commit [[skill]] to git and push to your fork (if configured)
- [ ] Consider [[contributing]] back via PR (if broadly useful)

Discovery Workflow

How future Claude finds your [[skill]]:

1. **Encounters problem** ("tests are flaky")
2. **Finds [[SKILL]]** (description matches)
3. **Scans overview** (is this relevant?)
4. **Reads patterns** (quick reference table)
5. **Loads example** (only when implementing)

Optimize for this flow - put searchable terms early and often.

The Bottom Line

Creating [[skills]] IS TDD for process documentation.

Same Iron Law: No [[skill]] without failing test first.
Same cycle: RED (baseline) → GREEN (write [[skill]]) → REFACTOR (close loopholes).
Same benefits: Better quality, fewer surprises, bulletproof results.

If you follow TDD for code, follow it for [[skills]]. It's the same discipline applied to documentation.

zero-build-frontend (Skill Module)

dev-toolkit\skills\zero-build-frontend\SKILL.md

name: zero-build-frontend

description: Zero-build frontend development with CDN-loaded React, Tailwind CSS, and vanilla JavaScript. Use when building static web apps without bundlers, creating Leaflet maps, integrating Google Sheets as database, or developing browser extensions. Covers patterns from rosen-frontend, NJCIC map, and PocketLink projects.

tags:

Zero-build frontend development

Patterns for building production-quality web applications without build tools, bundlers, or complex toolchains.

Picking a stack

Three current zero-build approaches, each with different trade-offs:

Stack	When	Bundle size impact
React via esm.sh + htm	Component-heavy SPAs, existing React mental model, Tailwind styling	~50 KB gzipped (React + ReactDOM + htm)
htmx 2.x + server-rendered HTML	CRUD apps, traditional MPA flow, want server-side state of truth	~14 KB gzipped (htmx alone)
Alpine.js 3.x + plain HTML	Light interactivity sprinkled into mostly-static pages, no full SPA	~15 KB gzipped (Alpine alone)

You can mix htmx and Alpine.js in the same page — htmx handles server interactions, Alpine handles client-side UI state. Many production sites converge on this combo.

ESM import maps

Import maps let you write `import x from 'react'` in a `<script type="module">` without a bundler — the browser resolves the bare specifier against the map. Stable in all major browsers since 2023.

```
<script type="importmap">
{
  "imports": {
    "react": "https://esm.sh/react@19.0.0",
    "react-dom/client": "https://esm.sh/react-dom@19.0.0/client",
    "lodash-es": "https://esm.sh/lodash-es@4.17.21",
    "@my-app/": "/src/"
  },
  "scopes": {
    "https://esm.sh/": {
      "scheduler": "https://esm.sh/scheduler@0.23.0"
    }
  }
}
</script>
```

The `scopes` block lets a sub-tree of imports resolve differently. Useful when one CDN package needs a specific transitive dependency. The trailing `/` form (`"@my-app/": "/src/"`) lets you import any file under that prefix.

Pin versions in production. `esm.sh/react` (without a version) and `esm.sh/react@latest` resolve at request time and can shift under you. Use exact pinned versions or SHA-locked URLs.

htmx 2.x — server-rendered interactivity

htmx 2.0 (released June 2024) lets you add AJAX, WebSockets, and SSE to plain HTML through `hx-*` attributes. The server sends HTML fragments; the client swaps them in. No JS framework required.

```
<script src="https://unpkg.com/htmx.org@2.0.4"></script>

<!-- Click button → POST to server → swap response into #result -->
<button hx-post="/api/clicked" hx-target="#result" hx-swap="innerHTML">
  Click me
</button>
<div id="result"></div>

<!-- Search-as-you-type with debounce -->
<input
  type="search"
  name="q"
  hx-get="/api/search"
  hx-trigger="input changed delay:300ms"
  hx-target="#results"
/>
<div id="results"></div>

<!-- Infinite scroll -->
<div hx-get="/api/items?page=2"
     hx-trigger="revealed"
     hx-swap="afterend">
  ...
</div>
```

htmx 2.x dropped IE support and tightened the API; if you're on htmx 1.x and don't need to migrate, 1.x still receives security patches. New code should target 2.x.

Alpine.js 3.x — client-side reactivity in HTML

Alpine.js (current 3.14+) is a minimal alternative to Vue/React for sprinkles of interactivity. State and behavior live as `x-*` attributes in the markup.

```
<script defer src="https://unpkg.com/alpinejs@3.14.1/dist/cdn.min.js"></script>

<!-- Toggle visibility -->
<div x-data="{ open: false }">
  <button @click="open = !open">Toggle</button>
  <div x-show="open" x-transition>Content here</div>
</div>

<!-- Two-way binding + computed -->
<div x-data="{ first: '', last: '' }">
  <input x-model="first" placeholder="First">
  <input x-model="last" placeholder="Last">
  <p x-text="`Hello, ${first} ${last}`"></p>
</div>

<!-- Fetch on mount -->
<div x-data="{ items: [] }"
     x-init="items = await (await fetch('/api/items')).json()">
  <template x-for="item in items" :key="item.id">
    <li x-text="item.title"></li>
  </template>
</div>
```

Alpine pairs naturally with htmx: htmx swaps a server-rendered fragment in, Alpine handles whatever client-side state that fragment needs (open/close, optimistic toggles, form validation).

React via CDN (esm.sh)

Basic [[setup]]

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Zero-Build React App</title>

  <!-- Tailwind CSS via CDN.
       cdn.tailwindcss.com is the Play CDN; Tailwind explicitly recommends
       it for prototyping only — it ships an in-browser JIT compiler that
       runs at every page load. For production, use the standalone CLI
       binary or the Vite/PostCSS plugin. Tailwind 4 (released Jan 2025)
       is the current major; the Play CDN serves v3 by default. -->
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      theme: {
        extend: {
          fontFamily: {
            display: ['Special Elite', 'monospace'],
            body: ['Roboto Mono', 'monospace'],
          },
          colors: {
            brand: {
              primary: '#2dc8d2',
              secondary: '#f34213',
              dark: '#183642',
            }
          }
        }
      }
    }
  </script>

  <!-- Google Fonts -->
  <link href="https://fonts.googleapis.com/css2?family=Special+Elite&family=Roboto+Mono:wght@400;500;700&display=swap" rel="stylesheet">

  <!-- Custom styles -->
  <link rel="stylesheet" href="index.css">
</head>
<body>
  <div id="root"></div>

  <!-- ES Module imports.
       React 19 (Dec 2024) is the current major; 18.x still works fine
       for sites that pin to it. Pin a specific version in production —
       don't ship `react@latest`, since esm.sh resolves at request time. -->
  <script type="importmap">
  {
    "imports": {
      "react": "https://esm.sh/react@19.0.0",
      "react-dom/client": "https://esm.sh/react-dom@19.0.0/client",
      "htm": "https://esm.sh/htm@3.1.1"
    }
  }
  </script>

  <script type="module" src="index.js"></script>
</body>
</html>
```

React with htm (no JSX, no build)

```
// index.js
import React, { useState, useEffect, useRef } from 'react';
import { createRoot } from 'react-dom/client';
import htm from 'htm';

// Bind htm to React.createElement
const html = htm.bind(React.createElement);

// Components use html`` instead of JSX
function App() {
  const [records, setRecords] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');

  useEffect(() => {
    loadData();
  }, []);

  async function loadData() {
    try {
      const response = await fetch('data/archive-data.json');
      const data = await response.json();
      setRecords(data.records);
    } catch (error) {
      console.error('Failed to load data:', error);
    } finally {
      setLoading(false);
    }
  }

  const filtered = records.filter(r =>
    r.title.toLowerCase().includes(search.toLowerCase())
  );

  if (loading) {
    return html`<div class="flex items-center justify-center h-screen">
      <div class="animate-spin w-8 h-8 border-4 border-brand-primary border-t-transparent rounded-full"></div>
    </div>`;
  }

  return html`
    <div class="min-h-screen bg-gray-900 text-white">
      <header class="p-4 border-b border-gray-700">
        <h1 class="font-display text-2xl">Archive Explorer</h1>
        <input
          type="text"
          placeholder="Search records..."
          value=${search}
          onInput=${(e) => setSearch(e.target.value)}
          class="mt-2 w-full p-2 bg-gray-800 rounded border border-gray-600 focus:border-brand-primary outline-none"
        />
      </header>

      <main class="p-4">
        <${RecordList} records=${filtered} />
      </main>
    </div>
  `;
}

function RecordList({ records }) {
  return html`
    <div class="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
      ${records.map(record => html`
        <${RecordCard} key=${record.id} record=${record} />
      `)}
    </div>
  `;
}

function RecordCard({ record }) {
  return html`
    <article class="p-4 bg-gray-800 rounded-lg border border-gray-700 hover:border-brand-primary transition-colors">
      <h2 class="font-display text-lg mb-2">${record.title}</h2>
      <p class="text-sm text-gray-400 mb-2">${record.publication_date}</p>
      <p class="text-sm line-clamp-3">${record.summary}</p>
```

```
      <div class="mt-2 flex flex-wrap gap-1">
        ${record.tags?.map(tag => html`
          <span key=${tag} class="px-2 py-1 text-xs bg-gray-700 rounded">${tag}</span>
        `)}
      </div>
    </article>
  `;
}

// Mount app
const root = createRoot(document.getElementById('root'));
root.render(html`<${App} />`);
```

Data caching with localStorage

```
// services/cacheService.js

const CACHE_TTL = 60 * 60 * 1000; // 1 hour

export function getCached(key) {
  const cached = localStorage.getItem(key);
  if (!cached) return null;

  try {
    const { data, timestamp } = JSON.parse(cached);
    if (Date.now() - timestamp > CACHE_TTL) {
      localStorage.removeItem(key);
      return null;
    }
    return data;
  } catch {
    localStorage.removeItem(key);
    return null;
  }
}

export function setCache(key, data) {
  localStorage.setItem(key, JSON.stringify({
    data,
    timestamp: Date.now()
  }));
}

export async function fetchWithCache(url, cacheKey) {
  // Check cache first
  const cached = getCached(cacheKey);
  if (cached) return cached;

  // Fetch fresh data
  const response = await fetch(url);
  const data = await response.json();

  // Cache for next time
  setCache(cacheKey, data);

  return data;
}

// Usage
const records = await fetchWithCache('data/archive-data.json', 'archive-records');
```

Leaflet.js maps

Basic map [[setup]]

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
  <link rel="stylesheet" href="https://unpkg.com/leaflet.markercluster@1.4.1/dist/MarkerCluster.css" />
  <link rel="stylesheet" href="https://unpkg.com/leaflet.markercluster@1.4.1/dist/MarkerCluster.Default.css" />
  <style>
    #map { height: 85vh; width: 100%; }
  </style>
</head>
<body>
  <div id="map"></div>

  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
  <script src="https://unpkg.com/leaflet.markercluster@1.4.1/dist/leaflet.markercluster.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

Map application with clustering

```
// js/app.js

class MapApp {
  constructor() {
    this.map = null;
    this.markers = null;
    this.data = [];
    this.filters = {
      year: null,
      county: null,
      status: null
    };
  }

  async init() {
    this.setupMap();
    await this.loadData();
    this.renderMarkers();
    this.setupFilters();
  }

  setupMap() {
    // Initialize map centered on NJ
    this.map = L.map('map', {
      center: [40.0583, -74.4057],
      zoom: 8,
      scrollWheelZoom: false,  // Disable mouse wheel zoom
      zoomControl: false       // We'll add custom controls
    });

    // Add tile layer (CARTO Voyager)
    L.tileLayer('https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png', {
      attribution: '&copy; OpenStreetMap, &copy; CARTO',
      maxZoom: 19
    }).addTo(this.map);

    // Add custom zoom control (top-right)
    L.control.zoom({ position: 'topright' }).addTo(this.map);

    // Initialize marker cluster group
    this.markers = L.markerClusterGroup({
      spiderfyOnMaxZoom: true,
      showCoverageOnHover: false,
      maxClusterRadius: 50,
      spiderLegPolylineOptions: { weight: 1.5, color: '#2dc8d2' }
    });

    this.map.addLayer(this.markers);
  }

  async loadData() {
    const response = await fetch('data/grantees.json');
    this.data = await response.json();
  }

  renderMarkers() {
    this.markers.clearLayers();

    const filtered = this.data.filter(item => {
      if (this.filters.year && item.year !== this.filters.year) return false;
      if (this.filters.county && item.county !== this.filters.county) return false;
      if (this.filters.status && item.status !== this.filters.status) return false;
      return true;
    });

    filtered.forEach(item => {
      if (!item.lat || !item.lng) return;

      const marker = L.marker([item.lat, item.lng], {
        icon: this.createIcon(item.status)
      });

      marker.bindPopup(this.createPopup(item));
      this.markers.addLayer(marker);
    });
```

```
    // Update count display
    document.getElementById('count').textContent = filtered.length;
  }

  createIcon(status) {
    const colors = {
      'Active': '#2dc8d2',
      'Completed': '#666666',
      'Pending': '#f34213'
    };

    return L.divIcon({
      html: `<div style="background: ${colors[status] || '#2dc8d2'}; width: 12px; height: 12px; border-radius: 50%; border: 2px solid white;"></div>`,
      className: 'custom-marker',
      iconSize: [16, 16],
      iconAnchor: [8, 8]
    });
  }

  createPopup(item) {
    return `
      <div class="popup-content">
        <h3 class="font-bold text-lg">${item.name}</h3>
        <p class="text-sm text-gray-600">${item.county} County</p>
        <p class="text-sm mt-2">${item.description || ''}</p>
        <div class="mt-2">
          <span class="px-2 py-1 text-xs rounded bg-gray-200">${item.status}</span>
          <span class="px-2 py-1 text-xs rounded bg-gray-200">${item.year}</span>
        </div>
        ${item.website ? `<a href="${item.website}" target="_blank" class="block mt-2 text-brand-primary">Visit Website →</a>` : ''}
      </div>
    `;
  }

  setupFilters() {
    // Year filter
    const years = [...new Set(this.data.map(d => d.year))].sort();
    const yearSelect = document.getElementById('year-filter');
    years.forEach(year => {
      const option = document.createElement('option');
      option.value = year;
      option.textContent = year;
      yearSelect.appendChild(option);
    });

    yearSelect.addEventListener('change', (e) => {
      this.filters.year = e.target.value || null;
      this.renderMarkers();
    });

    // Similar for county, status filters...
  }
}

// Initialize on load
document.addEventListener('DOMContentLoaded', () => {
  const app = new MapApp();
  app.init();
});
```

Google Sheets as database

Fetching published CSV

```
// Google Sheets published as CSV
const SHEET_URL = 'https://docs.google.com/spreadsheets/d/e/SPREADSHEET_ID/pub?gid=0&single=true&output=csv';

async function loadFromSheets() {
  const response = await fetch(SHEET_URL);
  const csv = await response.text();

  // Parse with PapaParse (CDN)
  const { data, errors } = Papa.parse(csv, {
    header: true,
    skipEmptyLines: true,
    transformHeader: (h) => h.trim().toLowerCase().replace(/\s+/g, '_')
  });

  if (errors.length > 0) {
    console.warn('CSV parsing errors:', errors);
  }

  return data;
}
```

Real-time state with localStorage

```
class DataManager {
  constructor(sheetUrl, cacheKey) {
    this.sheetUrl = sheetUrl;
    this.cacheKey = cacheKey;
    this.data = [];
    this.localState = this.loadLocalState();
  }

  loadLocalState() {
    const stored = localStorage.getItem(`${this.cacheKey}-state`);
    return stored ? JSON.parse(stored) : {};
  }

  saveLocalState() {
    localStorage.setItem(`${this.cacheKey}-state`, JSON.stringify(this.localState));
  }

  async refresh() {
    const response = await fetch(this.sheetUrl);
    const csv = await response.text();
    this.data = Papa.parse(csv, { header: true, skipEmptyLines: true }).data;

    // Merge with local state
    this.data.forEach(row => {
      const localData = this.localState[row.id];
      if (localData) {
        Object.assign(row, localData);
      }
    });

    return this.data;
  }

  updateLocal(id, updates) {
    this.localState[id] = { ...this.localState[id], ...updates };
    this.saveLocalState();

    // Update in-memory data too
    const item = this.data.find(d => d.id === id);
    if (item) Object.assign(item, updates);
  }
}

// Usage
const manager = new DataManager(SHEET_URL, 'volunteer-data');
await manager.refresh();

// Mark task as complete (stored locally)
manager.updateLocal('task-123', { completed: true, completed_at: new Date().toISOString() });
```

Browser extension (Manifest V3)

manifest.json

```
{
  "manifest_version": 3,
  "name": "PocketLink",
  "version": "1.0.0",
  "description": "Create shortlinks from right-click context menu",

  "permissions": [
    "contextMenus",
    "storage",
    "activeTab",
    "scripting",
    "notifications",
    "offscreen"
  ],

  "background": {
    "service_worker": "background.js",
    "type": "module"
  },

  "action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icons/icon16.png",
      "48": "icons/icon48.png",
      "128": "icons/icon128.png"
    }
  },

  "options_page": "options.html",

  "icons": {
    "16": "icons/icon16.png",
    "48": "icons/icon48.png",
    "128": "icons/icon128.png"
  }
}
```

Service worker (background.js)

```
// background.js - Service Worker

// Create context menu on install
chrome.runtime.onInstalled.addListener(() => {
  chrome.contextMenus.create({
    id: 'create-shortlink',
    title: 'Create Shortlink',
    contexts: ['page', 'link']
  });
});

// Handle context menu click
chrome.contextMenus.onClicked.addListener(async (info, tab) => {
  if (info.menuItemId !== 'create-shortlink') return;

  const url = info.linkUrl || info.pageUrl;

  try {
    const shortUrl = await createShortlink(url);
    await copyToClipboard(shortUrl);
    showNotification('Shortlink Created', shortUrl);
  } catch (error) {
    showNotification('Error', error.message);
  }
});

async function createShortlink(longUrl) {
  const { apiToken } = await chrome.storage.sync.get('apiToken');
  if (!apiToken) throw new Error('API token not configured');

  const response = await fetch('https://api-ssl.bitly.com/v4/shorten', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${apiToken}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ long_url: longUrl })
  });

  if (!response.ok) throw new Error('API request failed');

  const data = await response.json();
  return data.link;
}

// Clipboard methods (three fallback strategies)

// Method 1: Offscreen API (preferred)
async function copyToClipboard(text) {
  try {
    await copyViaOffscreen(text);
  } catch {
    try {
      await copyViaContentScript(text);
    } catch {
      await copyViaPopup(text);
    }
  }
}

async function copyViaOffscreen(text) {
  await chrome.offscreen.createDocument({
    url: 'offscreen.html',
    reasons: ['CLIPBOARD'],
    justification: 'Copy shortlink to clipboard'
  });

  await chrome.runtime.sendMessage({ type: 'copy', text });
  await chrome.offscreen.closeDocument();
}

async function copyViaContentScript(text) {
  const [tab] = await chrome.tabs.query({ active: true, currentWindow: true });
  await chrome.scripting.executeScript({
    target: { tabId: tab.id },
    func: (text) => navigator.clipboard.writeText(text),
```

```
    args: [text]
  });
}

function showNotification(title, message) {
  chrome.notifications.create({
    type: 'basic',
    iconUrl: 'icons/icon48.png',
    title,
    message
  });
}
```

Options page

```
<!-- options.html -->
<!DOCTYPE html>
<html>
<head>
  <style>
    /* Inline CSS for extension compliance (no remote code) */
    body {
      font-family: system-ui, sans-serif;
      padding: 20px;
      max-width: 400px;
      margin: 0 auto;
    }
    h1 { font-size: 1.5rem; margin-bottom: 1rem; }
    label { display: block; margin-bottom: 0.5rem; font-weight: 500; }
    input {
      width: 100%;
      padding: 8px;
      border: 1px solid #ccc;
      border-radius: 4px;
      font-size: 14px;
    }
    button {
      margin-top: 1rem;
      padding: 10px 20px;
      background: #2dc8d2;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }
    button:hover { background: #25a8b0; }
    .status { margin-top: 1rem; padding: 10px; border-radius: 4px; }
    .success { background: #d4edda; color: #155724; }
    .error { background: #f8d7da; color: #721c24; }
  </style>
</head>
<body>
  <h1>PocketLink Settings</h1>

  <label for="apiToken">Bit.ly API Token</label>
  <input type="password" id="apiToken" placeholder="Enter your API token">

  <button id="save">Save Settings</button>

  <div id="status" class="status" style="display: none;"></div>

  <script src="options.js"></script>
</body>
</html>
```

```
// options.js
document.addEventListener('DOMContentLoaded', async () => {
  const tokenInput = document.getElementById('apiToken');
  const saveButton = document.getElementById('save');
  const status = document.getElementById('status');

  // Load saved token
  const { apiToken } = await chrome.storage.sync.get('apiToken');
  if (apiToken) tokenInput.value = apiToken;

  saveButton.addEventListener('click', async () => {
    const token = tokenInput.value.trim();

    if (!token) {
      showStatus('Please enter an API token', 'error');
      return;
    }

    // Validate token by making test request
    try {
      const response = await fetch('https://api-ssl.bitly.com/v4/user', {
        headers: { 'Authorization': `Bearer ${token}` }
      });

      if (!response.ok) throw new Error('Invalid token');

      await chrome.storage.sync.set({ apiToken: token });
      showStatus('Settings saved successfully!', 'success');
    } catch {
      showStatus('Invalid API token', 'error');
    }
  });

  function showStatus(message, type) {
    status.textContent = message;
    status.className = `status ${type}`;
    status.style.display = 'block';
    setTimeout(() => { status.style.display = 'none'; }, 3000);
  }
});
```

Cache busting for deployments

```
<!-- Manual versioning for static files -->
<link rel="stylesheet" href="styles.css?v=1.3.0">
<script src="app.js?v=1.3.0"></script>

<!-- Or use build timestamp -->
<script>
  const version = Date.now();
  document.write(`<link rel="stylesheet" href="styles.css?v=${version}">`);
</script>
```

Deployment patterns

Static hosting (FTP/SFTP)

```
# Directory structure for WordPress wp-content deployment
wp-content/
 archive-explorer/
     index.html
     index.js
     index.css
     components/
        Sidebar.js
        RecordList.js
        RecordCard.js
     data/
         archive-data.json
```

Path management for subdirectory deployment

```
// constants.js

// Auto-detect base path from current URL
const getBasePath = () => {
  const path = window.location.pathname;
  const lastSlash = path.lastIndexOf('/');
  return path.substring(0, lastSlash + 1);
};

export const BASE_PATH = getBasePath();
export const DATA_URL = `${BASE_PATH}data/archive-data.json`;

// Usage
const response = await fetch(DATA_URL);
```

Performance tips

- **Lazy load large JSON**: Parse incrementally or paginate
- **Use CSS containment**: `contain: layout style` on repeated elements
- **Debounce search input**: Wait 300ms after typing stops
- **Virtualize long lists**: Only render visible items
- **Preconnect to CDNs**: `<link rel="preconnect" href="https://esm.sh">`